整书阅读版

《C 语言》

从源文件到能维护的程序 · 零基础 · 真编译真运行

单元 20 个 · 生成于构建期

全书目录

从源文件到可执行文件：你的第一个 C 程序
编译器的脾气：诊断、警告与构建命令
类型与取值范围：整型、字符与浮点
整数的边界：提升、溢出、位运算与枚举
表达式与求值：优先级、转换与顺序
分支与循环：把流程写对
函数：声明、定义、调用栈与 static 局部变量
数组与字符串：连续的一块内存
指针与内存：地址、类型与生命周期
指针的用法与约定：接口、多级指针与函数指针
结构体：把数据打包，以及看不见的 padding
动态内存：malloc 与生命周期
自引用结构与链表：用指针串起数据
文件与 I/O：把结果留下来
标准库工具箱：字符串、解析、排序与数学
预处理与宏：符号在编译之前怎么变形
多文件工程：头文件、编译单元与链接性
调试与排错：gdb、退出码与排错套路
未定义行为与优化边界：编译器有多大自由度
收尾：从会写 C 到能维护一个 C 程序

第 1 章

起步：从源文件到能跑的程序

本章 2 个单元 · 页眉从这里之后的页面开始出现

1. 从源文件到可执行文件：你的第一个 C 程序 零基础 · 第 1 单元 · 建议 40 分钟
2. 编译器的脾气：诊断、警告与构建命令 零基础 · 第 2 单元 · 建议 70 分钟

从源文件到可执行文件：你的第一个 C 程序

学习目标 · 看完这一单元，你应该能

- 说清从 `hello.c` 到 `hello.exe` 中间**真的**发生了哪四步，每一步产出了什么文件；
- 亲手把中间产物留下来（`gcc --save-temps`），并用 `ls` 看见它们；
- 解释 `printf` 凭什么能被你调用：它住在哪、谁把它的名字告诉编译器、谁把它的实现体接进来；
- 看见一个编译错误时，读懂 `文件:行:列: error: …` 这行字在说什么，并且知道往哪里改；
- 说出字符串 `"hi"` 在内存里占几个字节，以及为什么不是两个。

1.1 先看工程长什么样

学 C 最容易走偏的地方，是一上来就背语法。我们先看**文件和目录**：一段程序从哪个文件开始，编译器要读几个文件，最后产出什么。

大白话 · 把这一节的目的说成人话

我写的文件叫**源文件**，编译器读它，产出一个**可执行文件**。中间它会顺手写几个临时文件（预处理结果、汇编、目标文件），平常它们写完就删，所以你看不到。这一节我们要做的，就是让它们别删，看一眼它们长什么样——因为**你以后 80% 的报错，都发生在这几个文件之间的接缝上。**

目录树 **hello 工程：这一节里它长成了什么样**

第 1 步 一开始只有草稿，一个源文件

```
hello/
└── hello.c
```

第 2 步 加上清单，编译与输出才能被自动对账

```
hello/
├── hello.c
└── hello.json ← 本课新增
```

第 3 步 这一节结束时，书里的例子目录长这样

```
examples/ch01/
├── hello.c ← 本课新增
├── hello.json ← 本课新增
├── trace.c ← 本课新增
├── noinclude.c ← 本课新增
└── string_memory.c ← 本课新增
```

第 1 帧是上手第一分钟的样子：一个目录、一个 `.c` 文件。第 2 帧多出来的 `hello.json` 不是 C 的东西，它是**这本书的流水线**要的东西：它声明「这段程序怎么编译、跑完应该输出什么」，这样正文里的输出就不可能是手抄的（后面 1.3 会用到它）。第 3 帧是这一节结束时 `examples/ch01/` 里的全部文件：多出来的四个 `.c` 各有用途，我们在 1.4、1.5、1.6 一个一个拆。

工程惯例 · 工程惯例：目录怎么起名

`hello/` 这种「一个程序一个目录」的摆法不是标准规定的，是三十年攒下来的主流做法：

- 每个可独立编译运行的小程序放**自己的目录**，目录名用英文小写，不用空格与中文；
- 源文件用 `.c`（C）与 `.cpp`（C++），头文件用 `.h`——这是**惯例**，标准里没写「必须叫 .c」；
- 一个目录里只放这个程序的相关文件，编译产出的 `*.exe`、`*.o` **不要**和源文件混在一起（本书统一产出到 `dist/work/` 下）。

第 3 条是本书的硬规定：从第二个项目开始，**不允许出现单文件工程。**原因是单文件工程一旦超过 300 行，你就会开始分不清「哪些代码在互相依赖」；而分文件、写头文件这件事，越早练越省事。第一个程序（就是下面这个）允许单文件，因为它的全部意义是让你看见整条链。

1.2 你的第一个 C 程序

下面这段程序来自真实文件 `examples/ch01/hello.c`，正文里的每一行、连同行号，都是这个文件里的真实内容——本书正文里的代码块一律这么来，**不手抄**：

c examples/ch01/hello.c 第 1–10 行

```c
#include <stdio.h>

int main(void) {
    int total = 0;                  /* 累加器：三行加起来是 6 */ 一个变量，四个字节
    for (int i = 1; i <= 3; i++) {
        total += i;                 /* 中文注释也能正常高亮 */
    }
    printf("三行相加 = %d\n", total); 把结果打到屏幕上
    return 0;
}
```

4 一个变量，四个字节
8 把结果打到屏幕上

跑出来是这样：

终端输出 终端输出 hello-out

```
三行相加 = 6
```

十行代码，你至少要认出五样东西：

1. 第 1 行 `#include <stdio.h>`：在**编译之前**把另一个文件的内容搬进来（1.3 会看到搬进来之后的样子）。
2. 第 3 行 `int main(void) {`：程序的入口。操作系统把控制权交给它，它 `return 0;` 表示「一切正常」。
3. 第 4 行 `int total = 0;`：一个变量，四个字节，初值是 0。
4. 第 8 行 `printf(...)`：把文字打到屏幕上。注意它**不是**语言的关键字，它是运行库里的一个函数——这件事是 1.4 的主角。
5. 第 9 行 `return 0;`：把 0 交回给操作系统。终端里 `echo $?` 看到的 `0` 就是这个数。

{} 语法规则 · **一个 C 程序的最小骨架**

骨架只有三件事（对着前面那个代码块的第 1、3、9 行看）：**带进来**（`#include`）、**写一个 main**、**还一个整数**（`return 0;`）。其余所有代码，都在 `main` 的花括号里面。分号 `;` 是**语句的结尾**，不是可有可无的装饰——少了它，编译器从出错的那一行的下一行开始报错（1.5 有真例子）。

⚠ 陷阱 · **中文注释与中文字符串：编码这件事现在不用管，但要记住它在**

你在源文件里写的中文注释、`printf` 里写的中文，能不能正确显示，取决于**源文件的字节**、**编译器用什么编码去读它**、**终端用什么编码去显示它**三件事。本机实测（Windows 10 x64 (MSYS2 UCRT64)，gcc 16.1.0）这三件事是一致的：源文件是 UTF-8，编译出来的程序写出的也是 UTF-8，终端按 UTF-8 显示，所以上面那行 `三行相加 = 6` 是正常的。

你会被坑到的场合是：把同一个 `.c` 文件复制到另一台用 GBK 默认编码的机器上，或者把程序输出重定向进一个按 GBK 解释的文件里——那时你会看到一串像 `涓婚棶鍑犲彿` 的乱码。**看到乱码先别怀疑代码**：代码没有错，错的是「谁把字节当成什么编码读」。记住这个排查顺序：先看源文件编码，再看编译器有没有加编码参数，最后看终端/文件的编码。

1.3 一条命令背后：预处理 → 编译 → 汇编 → 链接

你敲的那条命令只有一行，但它让编译器做了四件事。我们先让中间产物别删掉——`--save-temps` 就是干这个的：

终端会话 让中间产物别删掉：--save-temps　　ch01-stages

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g --save-temps -c hello.c -o hello.o
$ ls -l hello.c hello.i hello.s hello.o
-rw-r--r-- 1 Administrator 197121   290 Oct 17 20:05 hello.c
-rw-r--r-- 1 Administrator 197121 78372 Oct 17 20:05 hello.i
-rw-r--r-- 1 Administrator 197121  2659 Oct 17 20:05 hello.o
-rw-r--r-- 1 Administrator 197121  5253 Oct 17 20:05 hello.s
$ wc -l hello.c hello.i hello.s
   10 hello.c
 1525 hello.i
  319 hello.s
 1854 total
```

这条会话是从真实终端里抓下来的：**10 行源文件**，展开成 **1525 行预处理结果**，再压成 **319 行汇编**。这三个数字就是本节最想说的事——你以为你在写 10 行，编译器实际要处理 1525 行。

图 / 搜索过程　**四个阶段：每一段都在把「上一种东西」翻译成下一种**

第 1 步　起点

hello.c ：你敲的源文件，10 行

第 2 步　预处理

hello.i ：展开 #include 与宏，1525 行 ← 本步新增

hello.c → hello.i　gcc -E

第 3 步　编译

hello.s ：这台机器的汇编，319 行 ← 本步新增

hello.i → hello.s　gcc -S

第 4 步　汇编

hello.o ：机器码 + 符号表，还没接库 ← 本步新增

hello.s → hello.o　as

第 5 步　链接

hello.exe ：把库接上，可以运行 ← 本步新增

hello.o → hello.exe　ld（gcc 替你调）

第 1 帧是你手上的东西。第 2 帧是**预处理**：`#include <stdio.h>` 被就地展开成整个头文件的内容，`hello.c` 因此变成 1525 行。第 3 帧是**编译**：这 1525 行被翻成这台机器认识的汇编——注意是「这台机器」，换一个 CPU 架构，同一份 C 代码翻出来的汇编完全不同。第 4 帧是**汇编**：汇编被翻成机器码，装进目标文件 `hello.o`，但里面缺 `printf` 的实现体，它不知道要打印的到底是哪来的函数。第 5 帧是**链接**：链接器把 `hello.o` 与运行库接起来，`printf` 的实现体被接进去，产出一个能运行的可执行文件。

预处理之后，`printf` 还是 `printf` 吗

预处理结果有 1525 行，但真正属于你的只有最后 7 行：

终端会话 预处理结果 hello.i 的最后 7 行　　ch01-pre

```
$ tail -n 7 hello.i
    int total = 0;
    for (int i = 1; i <= 3; i++) {
        total += i;
    }
    printf("三行相加 = %d\n", total);
    return 0;
}
```

你写的代码**原封不动**在文件末尾，前面 1500 多行全是头文件带进来的声明。这里有个反直觉的点：`#include` 不是「去库里拿东西」，而是**把那个文件的文字搬过来**。为了让编译器知道 `printf` 这个名字长什么样，你得把它的声明搬进来——这就是 1.4 的主题。

预处理结果里的行号已经被打乱了（你的第 3 行变成了第 1517 行），所以预处理结果里夹着行号标记：

c **预处理结果（.i）的末尾：你的代码在这里** examples/ch01/hello.i 第 1517–1525 行

```c
# 3 "hello.c"
int main(void) {
    int total = 0;
    for (int i = 1; i <= 3; i++) {
        total += i;
    }
    printf("三行相加 = %d\n", total);
    return 0;
}
```

行首那些 `# 3 "hello.c"` 就是**行号标记**。编译器靠它们，才能在报错时告诉你「问题在 `hello.c` 的第 8 行」，而不是「在 `hello.i` 的第 1524 行」。你以后看到的每条诊断都带着原始文件与行号，功劳就在这里。

汇编里，字符串与调用长这样。先看数据部分——第 8 行那串 `\344\270\211` 就是你写的「三」：

asm **汇编（.s）：字符串被拆成字节，main 变成一个标号** examples/ch01/hello.s 第 6–13 行

```asm
    .section .rdata,"dr"
.LC0:
    .ascii "\344\270\211\350\241\214\347\233\270\345\212\240 = %d\12\0"
    .text
    .globl  main
    .def    main;   .scl    2;  .type   32; .endef
    .seh_proc   main
main:
```

asm **汇编（.s）：printf 在这里只是一个待填的名字** examples/ch01/hello.s 第 48–52 行

```asm
    .loc 1 8 5
    movl    -4(%rbp), %eax
    leaq    .LC0(%rip), %rcx
    movl    %eax, %edx
    call    printf
```

`call printf` 这一行里，`printf` 只是一个**名字**，地址还没定下来。链接器的工作就是在这个名字旁边填上真正的地址；填不上，你看到的报错是 `undefined reference to 'printf'`——那是链接期的错误，不是编译期的。1.5 会把这两类错误分清。

链接与运行

终端会话 链接 + 运行 + 退出码 ch01-link-run

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g hello.c -o hello.exe
$ ./hello.exe
三行相加 = 6
$ echo $?
0
```

这就是完整的证据链：`hello.c` 在里面，`hello.exe` 出来了，跑出来的正是你写的那行字，退出码是 **0**。本节出现的命令、字节数、行数、退出码，都不是我写下来的，而是这台机器上真跑出来的；本书的规矩是——**凡写作「实测」的数字，必须来自真跑，并且由流水线逐字对账。**

实验记录 · **本机实测：这一节的工具链与命令**

- 编译器：gcc 16.1.0，本书统一用 `-std=c17`（也就是 C17），目标三元组 x86_64-w64-mingw32；
- 编译参数固定为 `-Wall -Wextra -Wpedantic -O0 -g`：警告全开、**不做优化**（`-O0`）、带调试信息（`-g`）；
- `--save-temps` 让中间产物留在当前目录：`hello.i`（预处理）、`hello.s`（汇编）、`hello.o`（目标文件）；
- 源文件 10 行 290 字节，预处理结果 **1525 行 78372 字节**——头文件带进来的量是你自己代码的 270 倍；
- `./hello.exe` 的 stdout 是 `三行相加 = 6`，退出码 **0**。

上面的数字里，行数与字节数由流水线在构建时抓取，路径写在 `examples/ch01/session-savetemps.txt`；对不上就说明源文件被改过。

标准原文 · **标准原文：`#include` 到底做了什么**

> C 语言标准（ISO/IEC 9899，C17）6.10.2 节「Source file inclusion」对 `#include <...>` 的规定是：
>
> > A preprocessing directive of the form `# include <h-char-sequence> new-line` searches a sequence of implementation-defined places for a named header, and causes the entire contents of that header to be replaced by the contents of the file.
>
> 大意：`#include` **把那个头文件的全部内容就地替换进来**。注意标准刻意没说「去哪个目录找」——它写的是 `implementation-defined places`（由实现决定）。所以 `stdio.h` 在哪、里面写了什么，是**工具链**的事，不是语言的事；换一个编译器，`#include <stdio.h>` 搬进来的可能是完全不同的一份文字。
>
> 出处：ISO/IEC 9899:2018（C17）§6.10.2 第 3 段。

编译器视角 · **编译器视角：它看到的不是「一个文件」，而是一条行号链**

`gcc` 在你按下回车后做的事，可以概括成「带着一张行号地图，逐段改写文本」：

1. 预处理输出 `.i`，每段前面夹着 `# 行号 "文件名"` 标记（上面那 9 行的开头就是 `# 3 "hello.c"`）；
2. 编译把 `.i` 翻成 `.s`，并写进 `.loc 1 8 5` 这样的调试指令——「接下来这段机器码对应源文件第 8 行第 5 列」；
3. 汇编把 `.s` 变成 `.o`，符号表里留下「我引用了一个叫 `printf` 的外部名字」；链接再把地址填上。

所以看到报错先问一句：这是**哪一步**在说话？两类的修法完全不同。

调试实录 · **调试实录：确认「程序真的从 main 开始」**

新手最常见的怀疑是「我的 `main` 到底有没有被调用」。这个问题不需要猜，用调试器看一眼就行。下面这段会话是真的（`trace.c` 也在 `examples/ch01/` 里）：

下面两块是会话原文，按两次命中拆开；读法也只有三步：`break add` 是「在 add 的第一行停住」，`bt`（backtrace）是「把调用链打出来」，`continue` 是「继续跑、等下一次停」。

终端会话 第一次命中：里面那个 add(2, 3) ch01-gdb-trace-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g trace.c -o trace.exe
$ gdb --batch -ex "break add" -ex run -ex bt -ex continue -ex bt ./trace.exe
Breakpoint 1 at 0x1400014ba: file trace.c, line 4.
[New Thread 63756.0x109e0]
Thread 1 hit Breakpoint 1, add (a=2, b=3) at trace.c:4
4           return a + b;
#0  add (a=2, b=3) at trace.c:4
#1  0x00007ff76f0714e0 in main () at trace.c:8
```

终端会话 （续）第二次命中：外面那个 add(1, 5) ch01-gdb-trace-2

```
Thread 1 hit Breakpoint 1, add (a=1, b=5) at trace.c:4
4           return a + b;
#0  add (a=1, b=5) at trace.c:4
#1  0x00007ff76f0714ec in main () at trace.c:8
```

两块合起来读，两次命中说明：**里面那次** `add(2, 3)` **先算**（`a=2, b=3`），返回之后**外面那次** `add(1, 5)` **再算**。`#1 … in main () at trace.c:8` 就是「谁在调用我」——调用链的下一层是 `main`。

`#1` 后面那串 `0x00007ff76f0714e0` 是**返回地址**，每次运行、每次改代码都会变，正常输出里不用管它；要紧的是 `#0`、`#1` 后面的行号（`trace.c:4`、`trace.c:8`），那些是稳定的。 那次调试跑的程序 `trace.c` 自己也打印一行，输出可以直接对上：

终端输出 终端输出 trace-out

```
total = 6
```

调用树 **调用树：把 gdb 打出来的两份 bt 画成一棵树**

第 1 步 **main 在第 8 行准备算 total**

`main()` ：`trace.c:8`，准备算 `total`

第 2 步 **先把括号里那个 add 算出来**

`main()` ：等我先算里面的
`add(2, 3)` ：`trace.c:4`，第一次命中 ← 本步新增
`main()` → `add(2, 3)` 内层调用先发生

第 3 步 **内层返回 5，外层接着算**

`add(2, 3)` ：已经返回 5
`add(1, 5)` ：第二次命中，`a=1 b=5` ← 本步新增
`add(2, 3)` → `main()` 返回值 5 交给调用者
`main()` → `add(1, 5)` 再算外层

第 1 帧对应「刚进 `main`」，第 2 帧对应第一次 `Thread 1 hit Breakpoint 1, add (a=2, b=3)`，第 3 帧对应第二次命中 `add (a=1, b=5)` —— `b` 变成 5，说明内层的 2+3 已经算完了。**调用树讲的是「谁在等谁」**：`main` 一直没走，它在等 `add` 返回。这一条以后会变成你理解递归的全部基础。

1.4 `printf` 凭什么能用：谁能看见谁

`printf` 不是你写的函数，也不在 `hello.c` 里。它凭什么能被调用？把「谁看得见谁」画出来，这件事就清楚了：

谁能看见谁 **谁能看见谁：从 main.c 一路到运行库**

第 1 步 **只写 main.c，编译器不认识 printf**

`main.c（编译单元）：只有 main 一份代码` `printf：你在用的名字`
`main.c` → `printf` 没有声明，编译器不知道它长什么样

第 2 步 **#include <stdio.h> 之后**

`main.c（编译单元）` **`stdio.h（头文件）：里面写着 printf 的声明`** ← 本步新增
`main.c` → `stdio.h` #include <stdio.h>
`stdio.h` → `printf` 声明：名字、参数、返回值

第 3 步 **链接时把实现体接上**

`运行库（UCRT）：printf 的实现体住在这里` ← 本步新增
`printf` → 运行库（UCRT） 链接器把两边对上
运行库（UCRT） → `hello.exe` 实现体被接进可执行文件

第 1 帧是新手最容易踩的坑：编译器**只认它看见过的名字**。你在 `main.c` 里写 `printf("...")`，如果没有任何地方告诉它 `printf` 长什么样，它会当场报错（1.5 有真实的报错文本）。第 2 帧是 `#include <stdio.h>` 干的事：把 `stdio.h` 的文字搬进来，于是编译器看到了 `int printf(const char * restrict format, ...);` 这行**声明**。第 3 帧是链接期：声明只是一张「名字与参数表」，真正的实现体住在运行库里，链接器负责把两边对上。

用调试器能直接看到第 3 帧：在 `printf` 上打断点，程序停下时 `bt` 会告诉你它住在哪儿、谁调用了它。

终端会话 gdb：printf 住在这里，调用它的是 main　　ch01-gdb-printf

```
$ gdb --batch -ex "set print frame-arguments none" -ex "break printf" -ex run -ex bt ./hello.exe
Breakpoint 1 at 0x1400024f0: file D:/W/B/src/mingw-w64/mingw-w64-crt/stdio/ucrt_printf.c, line 12.
[New Thread 38364.0xd300]
Thread 1 hit Breakpoint 1, printf (_Format=...) at D:/W/B/src/mingw-w64/mingw-w64-crt/stdio/ucrt_printf.c:12
warning: 12     D:/W/B/src/mingw-w64/mingw-w64-crt/stdio/ucrt_printf.c: No such file or directory
#0  printf (_Format=...) at D:/W/B/src/mingw-w64/mingw-w64-crt/stdio/ucrt_printf.c:12
#1  0x00007ff685d714ee in main () at hello.c:8
```

三行关键信息：

- `Breakpoint 1 at 0x1400024f0: file D:/W/B/src/mingw-w64/mingw-w64-crt/stdio/ucrt_printf.c, line 12.` —— `printf` 的实现体在运行库的源码文件 `ucrt_printf.c` 里（那是编译运行库时留下的调试信息，你机器上不一定有那个文件，所以会看到 `No such file or directory` 的提示）；
- `#0 printf (...) at …/ucrt_printf.c:12` —— 现在停在运行库的代码里；
- `#1 … in main () at hello.c:8` —— 调用者是**你的** `main`，第 8 行，正是 `printf` 那一行。

标准原文 · **标准原文：`printf` 是什么**

同一个标准，7.21.6.3 节「The `printf` function」：

> The `printf` function is equivalent to `fprintf` with the argument `stdout` interposed before the arguments to `printf`.

大意：`printf(...)` 等价于 `fprintf(stdout, ...)`——它只是「往标准输出写」的一个便利包装。这一节同时给出它的声明，**声明**决定了编译器允不允许你调用它、允许你传什么参数；**实现**在运行库里，标准只规定行为，不规定代码。

出处：ISO/IEC 9899:2018（C17）§7.21.6.3。

⚠ **陷阱** · **「声明」与「实现」分家，你会被这样坑到**

你把程序从这台机器挪到另一台机器，`hello.c` 一个字没改，报错却变成了：

- `undefined reference to 'printf'` —— 链接器说「这个名字找不到实现体」；不是你的语法错了，是运行库没接上。
- 反过来，`%d` 写成 `%s` 编译器**多半不拦你**（参数不是常量时看不出来），程序会在运行期打出荒唐的数字或直接崩掉。

你怎么会被坑到：看到 `undefined reference`，新手的反应是回 `hello.c` 里检查拼写、改来改去——而问题根本不在你的文件里。记住分工：**编译期管声明，链接期管实现，运行期管格式与参数。**

1.5 编译器说不行的时候

现在把两个真实的编译错误摆出来。第一个：忘了写 `#include <stdio.h>`。源文件只有 6 行：

c　**故意不写 `#include <stdio.h>`** examples/ch01/noinclude.c　　第 1–6 行

```c
/* 故意不写 #include <stdio.h>：
   看看编译器会不会自己猜到 printf 是什么。 */
int main(void) {
    printf("没有声明也能打印? \n");
    return 0;
}
```

编译它。下面这块是真实的 stderr 原文——本书的诊断块一律照抄，连对齐用的空格都不动，所以你看到的插入符位置，就是编译器看到的位置；你按行号去源码里找，能找到同一列。

编译诊断 gcc 的真实诊断（原文照抄） ch01-noinclude-diag

```
noinclude.c: In function 'main':
noinclude.c:4:5: error: implicit declaration of function 'printf' [-Wimplicit-function-declaration]
    4 |     printf("没有声明也能打印? \n");
      |     ^~~~~~
noinclude.c:1:1: note: include '<stdio.h>' or provide a declaration of 'printf'
  +++ |+#include <stdio.h>
    1 | /* 故意不写 #include <stdio.h>:
noinclude.c:4:5: warning: incompatible implicit declaration of built-in function 'printf' [-Wbuiltin-
declaration-mismatch]
    4 |     printf("没有声明也能打印? \n");
      |     ^~~~~~
noinclude.c:4:5: note: include '<stdio.h>' or provide a declaration of 'printf'
```

第一个诊断一次给了三样东西。读它要按 `文件:行:列: 级别: 说明` 拆：

- `noinclude.c:4:5: error: implicit declaration of function 'printf' [-Wimplicit-function-declaration]` —— 第 4 行第 5 列（调用 `printf` 的位置）出错；`implicit declaration` 是「你调用了一个我没见过声明的函数」；方括号里是**控制这条诊断的开关名**，可以用 `-W…` 打开、用 `-Wno-…` 关掉。本书固定打开 `-Wall -Wextra -Wpedantic`，就是要把这类信号全都放出来；
- 紧跟着它把**出错的那一行源码**抄给你，并在下一行用 `^~~~~~` 标出位置——行号、源码、插入符一起看，就不会数错行；
- `noinclude.c:1:1: note: include '<stdio.h>' or provide a declaration of 'printf'` 是 `note:`，它甚至把**该补的那一行**都写好了（`+++ |+#include <stdio.h>`）。gcc 经常这样：报错归报错，顺手把正解放在你面前。

最后那条 `warning: incompatible implicit declaration of built-in function 'printf'` 是同一个问题的另一面：编译器认得 `printf` 这个内建名字，先按默认规则猜了个声明，再提醒你「这个猜测和真正的 `printf` 不兼容」。同一个根因，`error` 与 `warning` 各说了一遍——这也是为什么**看到第一条错误就够了**，后面几条经常是它的连锁反应。

第二个错误：漏了一个分号。源文件是这样——先自己找一找，再往下看编译器怎么说。

c **第 4 行末尾少了一个分号** examples/ch01/semicolon.c 第 1–7 行

```c
#include <stdio.h>

int main(void) {
    int total = 0
    printf("total = %d\n", total);
    return 0;
}
```

编译它。这次编译器会给你两个信号：一个 `error`、一个 `warning`，两个都值得看：先看错误本身，再看那个警告——它往往是错误的连带产物，改掉错误之后多半会自己消失。

编译诊断 gcc 的真实诊断（原文照抄） ch01-semicolon-diag

```
semicolon.c: In function 'main':
semicolon.c:5:5: error: expected ',' or ';' before 'printf'
    5 |     printf("total = %d\n", total);
      |     ^~~~~~
semicolon.c:4:9: warning: unused variable 'total' [-Wunused-variable]
    4 |     int total = 0
      |         ^~~~~
```

注意报的位置：`error` 在第 **5** 行，但你要改的是第 **4** 行的末尾。这是 C 报错最典型的一种「错位」：编译器只有读到第 5 行的 `printf` 时，才能确定第 4 行的 `int total = 0` 后面少了东西。**不要盯着报错的行号改，先往前看一行，看语句有没有收尾。**

底下那条 `warning: unused variable 'total'` 也值得看一眼：编译器发现 `total` 定义了却没用上——因为第 5 行的 `printf` 没能成为一条合法语句，这个变量就悬空了。**警告有时是错误的下游产物**，改掉错误之后记得再看一遍：警告还在不在。

⚠ 陷阱 · 分号与括号：C 里最贵的两个字符

分号 `;` 结束一条**语句**，而 `if`、`for`、`while`、函数定义的花括号 `{}` 不是语句、后面**不加**分号。这两条你会踩的坑：

- 在 `int main(void) { … }` 后面加了分号（`};`）：很多编译器会**默默接受**，于是你无意中写出了一个「空定义」，紧接着的报错会跑到很远的地方，看起来完全不相干；
- 在 `for (...) ;` 后面误加分号：循环体变成了那个空语句，循环白跑，程序不报错但结果全错——**这一类错误编译器抓不到，只能靠输出对不上来发现。**

这也是为什么本书要求每个程序都要有「真实输出」：输出是这类静默错误唯一的照妖镜。

编译器视角 · 编译器视角：诊断的「级别」是它给你的分级信号

同一条问题，编译器可能用三种口吻说：

- `error:` —— **我拒绝继续翻译。**比如 `implicit declaration of function`、`expected ';' before …`；
- `warning:` —— **我照你说的翻译了，但我认为你有地方不对。**比如「变量定义了没用」（`-Wunused-variable`）、「格式串与参数类型不匹配」（`-Wformat`）；
- `note:` —— **给上面那条诊断的补充信息**，最常见的是一句 `note: include <stdio.h> or provide a declaration of 'printf'`。

本书的编译参数里**不做优化**（`-O0`）是刻意的：优化会把代码改得面目全非，调试器看到的行号会和你的源码错位（`-O2` 下 `bt` 里的行号经常跳到循环外）。等你要看「优化到底改了哪里」时，再单独用 `-O2` 做对照实验——那时你会用到 `-fanalyzer` 这类静态分析参数，它们在**编译期**就能报出越界、空指针这类问题（第 4 单元开始）。

1.6 字符串在内存里长什么样

上面那些 `.i`、`.s`、`.o`、`.exe` 都是**文件**。程序跑起来之后，数据住在**内存**里。`"hi"` 这种字符串在内存里是什么样？这段程序是真的：

c examples/ch01/string_memory.c 第 1–10 行

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char word[] = "hi";                 /* 数组里有几个字节？ */
    printf("sizeof(\"hi\") = %zu\n", sizeof("hi"));
    printf("strlen(\"hi\") = %zu\n", strlen("hi"));
    printf("word[0]=%c word[1]=%c word[2]=%d\n", word[0], word[1], word[2]);
    return 0;
}
```

5 数组里有几个字节？
6 sizeof 看的是「占几个字节」
7 strlen 数的是「到 \0 为止有几个字符」

终端输出 终端输出 strings-out

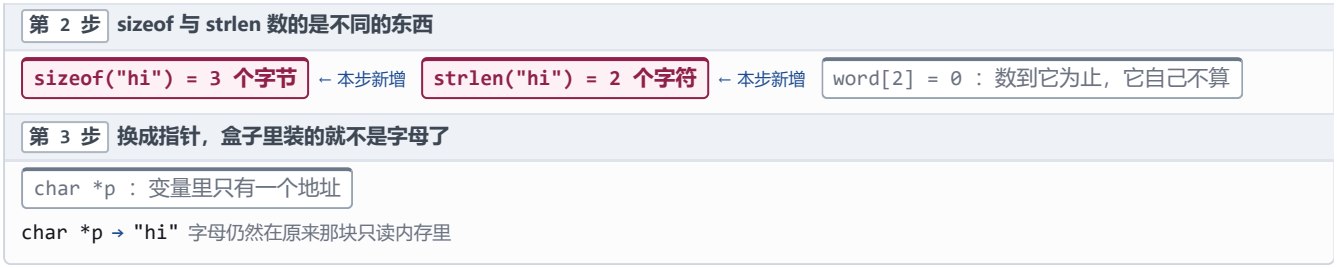
```
sizeof("hi") = 3
strlen("hi") = 2
word[0]=h word[1]=i word[2]=0
```

三个数字要连起来读：`sizeof("hi")` 是 **3**，`strlen("hi")` 是 **2**，`word[2]` 是 **0**。

内存形态 **hi**

第 1 步 字面量 "hi" 不是一个字符，是三个字节

`word[0] = 'h'` `word[1] = 'i'` `word[2] = 0 （字符串结束标记 '\0'）`



第 1 帧是本节最要紧的一张图：`"hi"` 占**三个**字节——`h`、`i`、还有一个值为 0 的字节。C 用这个 0 来标记「字符串到这里结束」，写出来是 `'\0'`（注意是反斜杠加零，不是数字 0 那个字符 `'0'`）。第 2 帧解释了为什么两个函数给出不同的数：`sizeof` 报的是**这块内存有几个字节**，`strlen` 报的是**数到 `\0` 之前有几个字符**。第 3 帧是你以后会反复见到的对比：`char word[]` 是「一整块数组」，`char *p` 是「一个装着地址的变量」。

大白话 · **用大白话说这三帧**

把内存想成一排信箱，每个信箱装一个字节。

- `"hi"` 不是「两个字母」，而是「两个字母 + 一个写着 0 的信箱」——那个信箱专门告诉函数「到这里就没了」；
- `sizeof` 数「这排一共几个信箱」所以是 3；`strlen` 数「读到写 0 的信箱为止有几个」所以是 2；
- `char word[]` 是**自己复制了一排三个信箱**，`char *p` 是**手里只有一张写着「那排在哪」的纸条。**

所以改 `word[0]` 影响不到别处，而改 `*p`（指向只读字面量时）可能当场崩掉：**它们不是同一块内存。**

实验记录 · **本机实测：三个数字都是跑出来的**

- `sizeof("hi")` = **3**、`strlen("hi")` = **2**、`word[2]` = **0**（printf 用 `%d` 打出来的就是这个 0）；
- 这三个数来自 `examples/ch01/string_memory.c` 的真实运行，输出由流水线逐字对账（命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g string_memory.c -o string_memory.exe`）；
- `%zu` 是打印 `sizeof` 结果的正确格式（它的类型是 `size_t`）；用 `%d` 打印在某些平台会打出负数。

别把这些数字当定律背：`sizeof("hi")` 处处是 3，但 `sizeof(int)` 在本机是 4、别的机器可能是 2。

1.7 这一节新增了什么，以及接下来

回到 1.1 的那棵目录树：现在你可以对着**第 3 帧**逐个点名了。

- `examples/ch01/hello.c` —— 第一个程序，10 行，`printf` 打出一行字；
- `examples/ch01/hello.json` —— 清单：声明「怎么编译、跑完的退出码是多少、正文里哪个输出块负责对账」；
- `examples/ch01/hello.i`、`hello.s` —— 预处理与汇编的**真实产物**，本节就是拿它们讲四阶段的；
- `examples/ch01/noinclude.c`、`semicolon.c` —— 两个故意写错的程序，用来展示真实的编译错误；
- `examples/ch01/string_memory.c` —— 用三个数字讲清 `sizeof`、`strlen` 与 `\0`；
- `examples/ch01/trace.c` —— 两层嵌套调用，用来演示 gdb 的 `bt` 与调用树。

从下一单元开始，我们把这份「单文件草稿」变成一个真正的工程：源文件进 `src/`，头文件进 `include/`，还会加一个把编译命令固定下来的脚本。**记住本书的硬规定：从第二个项目起，不允许出现单文件工程。**

工程惯例 · **工程惯例：为什么第二个项目起必须分文件**

标准只要求「一个程序有一个 `main`」，它对你分几个文件**没有任何规定**。分文件是主流做法，理由有三条：

- **编译速度**：改一个 `.c` 只需要重新编译它自己，链接一下就好；单文件工程每次都是全量重编；
- **接缝可见**：`#include` 把文件之间的依赖写在明面上。头文件里放**声明**，`.c` 里放**定义**，谁依赖谁一目了然；
- **能被别人接手**：一个 3000 行的 `main.c` 是没人愿意接的。

代价是你要多学一件事：头文件里写什么、不写什么（写重了会撞符号，写漏了会 `implicit declaration`）。这件事在第 3 单元讲透，现在只要记住方向：**声明放头文件，定义放源文件，编译单元各管各的。**

小结 · 这一节的要点

- 从 `hello.c` 到 `hello.exe` 有**四步**：预处理（`.i`）→ 编译（`.s`）→ 汇编（`.o`）→ 链接（`.exe`）；`--save-temps` 能把前三个留下来看；
- `#include` 就是「把那个文件的全部内容就地替换进来」，所以 `.i` 比你写的大几十倍；报错里的行号靠预处理阶段写下的行号标记才能对回你的源文件；
- `printf` 靠**声明**（从 `stdio.h` 搬进来）通过编译，靠**实现体**（在运行库里）通过链接；调试器能证明这一点；
- 编译错误读法：`文件:行:列: 级别: 说明`，`error` 中断、`warning` 继续、`note` 是补充；报错的行号经常要比实际错误**晚一行**；
- 字符串 `"hi"` 占 **3** 个字节，多出来的那个是 `'\0'`；`sizeof` 数字节，`strlen` 数到 `\0` 为止；
- 本节的每个数字、每段诊断、每次运行，都来自真实文件与真实运行，由流水线逐字对账——正文里的版本号由构建期从工具链事实文件注入，不手写。

自测题 · 自测题（先自己答，再对答案）

1. `hello.i` 有 1525 行，而 `hello.c` 只有 10 行。多出来的 1515 行是什么？它们是从哪一步来的？
2. 如果把 `#include <stdio.h>` 那一行删掉，程序在**哪一步**失败？报错里的关键词是什么？
3. `sizeof("hi")` 和 `strlen("hi")` 分别是多少？为什么不一样？
4. gdb 里 `bt` 打出来的 `#0` 与 `#1` 分别代表什么？为什么说「`#1 … in main ()` 就是调用链的下一层」？
5. 正文里的代码块与输出块，为什么不允许手抄？如果输出块与真实 stdout 差一个字符，哪条命令会失败、失败码是什么？

注 · 自测题答案

1. 多出来的是 `stdio.h`（以及它自己包含的头文件）的全部内容——`#include` 就地展开的结果，发生在**预处理**这一步。
2. 失败在**编译**这一步（源码没变化，不会走到链接）。关键词是 `implicit declaration of function 'printf'`；如果编译器把它降级成警告，就会走到链接期再报 `undefined reference`。
3. 分别是 **3** 和 **2**。`sizeof` 数的是这块内存有几个字节（含结尾的 `'\0'`），`strlen` 数的是到 `'\0'` 为止有几个字符（不含 `'\0'`）。
4. `#0` 是当前停住的函数（最内层），`#1` 是它的调用者。`#1 … in main () at trace.c:8` 说明这个函数是被 `main` 第 8 行调用进来的，所以调用链的下一层（外面一层）就是 `main`。
5. 因为手抄的东西没人对账，读者无法判断它是真跑出来的还是编的。差一个字符时，`node build/run.js --all` 会以失败码 `stale_output`（输出块）或 `stale_diagnostic`（编译诊断块）报错，并给出第一处不同的行号。

编译器的脾气：诊断、警告与构建命令

学习目标 · 看完这一单元，你应该能

- 把一条诊断按「文件、位置、级别、说明」四段拆开读，并说清方括号里那个开关名是干什么的；
- 说清 `-Wall`、`-Wextra`、`-Wpedantic` 各打开了什么、**不含**什么，知道 `-Wconversion`、`-Wshadow`、`-Wfloat-equal`、`-Wformat=2` 要自己加；
- 认出六类高频警告，并说出每一类在真实代码里会变成什么事故；
- 分清编译期、链接期、运行期三件事各由谁负责，失败时该去看哪一份输出；
- 用 `gcc -c` 与「一条命令」两种方式构建，并说出 `-O0` 与 `-g` 为什么是本书的默认；
- 把退出码当判据用（`gcc … && ./app`），并说出 `-Werror`、`-fanalyzer` 各自给你什么、代价是什么。

2.1 工程长什么样：这一单元往 examples/ch01/ 里加了什么

上一单元我们在 `hello.c` 上看见了四个阶段，也看见了两条故意写错的诊断。这一单元要把「编译器说的话」当成一门手艺来学：它说什么、按什么规矩说、哪些话只在特定开关下才说、它不说话的时候你该怎么办。

新增的东西全是**实验文件**：八个小程序（多数各自只考一件事），每个配一份清单（清单负责声明「怎么编译、跑完退出码是多少、正文里哪个块负责对账」），再加一份把几十条真实命令抓下来的会话文件。为什么不拿现成的 `hello.c` 来学？因为它太干净——诊断这门手艺得在**脏**代码上学。

目录树 **examples/ch01/：这一单元里它长成了什么样**

第 1 步 上一单元结束时，这一层里已经有这些

```
examples/ch01/
├── hello.c / trace.c / noinclude.c
├── semicolon.c / string_memory.c
├── gallery_long.c / analyzer_demo.c
└── hello.i / hello.s、清单、四份会话
```

第 2 步 这一单元新增八个小程序、八份清单、一份会话

```
examples/ch01/
├── （上一帧的家底都还在）
├── warn_zoo.c / extra_warnings.c ← 本课新增
├── format_mismatch.c / format_security.c ← 本课新增
├── werror_blocks.c / diag_chain.c ← 本课新增
├── undef_ref.c / stages_c.c ← 本课新增
├── 每个都配一份同名 .json 清单 ← 本课新增
└── session-diagnostics.txt ← 本课新增
```

第 3 步 这一章还是单文件项目

```
examples/ch01/
├── （所有 .c 与清单都平铺在这一层，没有子目录）
├── src/ ：本章不建
└── include/ ：本章不建
```

第 1 帧是上一单元留下的家底；第 2 帧是这一单元加进来的——八个小程序、八份清单，加一份抓下来的会话（前四个偏「诊断」，后四个偏「构建」，正好对应本单元的两半）；第 3 帧说的是**没加什么**：这一章仍然是单文件项目，源文件与清单平铺在 `examples/ch01/` 这一层，不建 `src/` 与 `include/`。这不是偷懒：单文件阶段要先把「一条命令怎么编、诊断怎么读」学会，再谈目录怎么分；从下一个项目（`examples/ch02/`）起，本书不再允许单文件工程（1.1 的惯例块里说过它的代价）。

下面这一整单元的所有命令，都是在这台机器上真跑出来的，命令与输出原样抓在 `examples/ch01/session-diagnostics.txt` 里；正文里的每一段会话都从那个文件按行号注入，一个字都没有手打。

标准原文 · 标准原文：诊断是「至少一条」，不是「说清楚」

> C 标准对诊断只规定了底线：
>
> > A conforming implementation shall produce at least one diagnostic message (identified in an implementation-defined manner) if a preprocessing translation unit or translation unit contains a violation of any syntax rule or constraint ...
>
> 大意：只要代码**违反了一条语法规则或约束**，编译器就至少得吐出一条诊断——**说什么、叫什么名字、怎么排版，全是实现自己的事。**标准甚至没说它必须是 `error`，也没规定「一条报错只许报一处」。
>
> 所以下面这些规矩都不是标准定的，是 gcc 自己定的：诊断长成 `文件:行:列: 级别: 说明`、方括号里挂一个 `-W…` 开关名、`error` 就停下、`warning` 就接着编。换一个编译器，同一份代码可能给你完全不同的文字。
>
> 出处：ISO/IEC 9899:2018（C17）§5.1.1.3 第 1 段。

2.2 一条诊断的四段读法

先把一条最短的诊断摆出来。这是我们马上要拆的那份 `warn_zoo.c` 的编译结果，命令在会话里（`grep` 只是把警告行挑出来，不改变内容）：

终端会话 挑出一条警告来读 ch02-diag-four-parts

```
$ grep 'warning:' /tmp/w.txt
warn_zoo.c:9:9: warning: unused variable 'scratch' [-Wunused-variable]
```

这一行里塞了四段信息，按顺序是：

1. `warn_zoo.c`：哪个文件。如果编译命令里写的是相对路径，这里就是相对路径；这就是为什么本书要求「在源文件所在的目录里编译」——报错里的文件名短、能直接对回你的文件。
2. `9:9`：**行:列**，都是从 1 数起。行是你在编辑器里按 `Ctrl+G` 能跳过去的地方；列是那一行的第几个字符。上面这条说的是「第 9 行的第 9 列」——去 `warn_zoo.c` 第 9 行看，那一列正好是 `scratch` 的第一个字母。
3. `warning:`：**级别**。三种口吻（1.5 见过前两种）：`error` 是「我拒绝继续翻译」，`warning` 是「我照做了，但我认为你有地方不对」，`note` 是「给上面那条的补充」。级别写在说明之前，是判断「编不编得下去」的唯一依据。
4. `unused variable 'scratch'`：说明。它是人话，但**它不是稳定的接口**：换一个编译器版本，同一句可能被改写、被拆成两句、或者消失。

说明的末尾还挂着一对方括号，**这一对方括号是全书最值得你留意的一处细节**：

> **{} 语法规则 · 方括号里是「控制这条诊断的开关名」**
>
> `[-Wunused-variable]` 的意思是：这条警告由 `-Wunused-variable` 打开，也能用 `-Wno-unused-variable` 关掉。末尾带着 `=` 的（比如 `[-Wformat=]`）表示它属于一族开关，`-Wformat=2` 这种写法就是在给这一族指定档位。

有了这个名字，你能做三件事：查手册（`gcc` 的手册里搜 `-Wunused-variable`，能读到它到底在什么条件下触发）；单独关掉它（`-Wno-unused-variable`）；以及在团队里精确地讨论「这条要不要开」。**没有这对方括号，你只能对着一句英文猜。**

书里有一条纪律要说在前面：**本书的清单里不允许出现 `-Wno-*`**（构建脚本会直接拒掉，报 `flag_not_allowed`）。理由是：关掉一条警告不等于问题解决了，只是把眼睛闭上了；如果一条警告真的在某个场景下没意义，正确的做法是改写代码让它不再触发，或者用局部标注说明这一处是有意的——这两种做法都会在正文里留下痕迹，`-Wno-` 不会。

还有一处容易看漏：诊断下面常常跟着**出错那一行的源码**，再下一行用 `^~~~~~` 把出错的位置指出来。行号、源码、插入符三样一起看，你就不会数错行、不会改错列。`note:` 那一行更贴心，gcc 经常把「该补的那行代码」都替你写好了（1.5 的 `noinclude.c` 就是这样）。

2.3 三道开关：-Wall、-Wextra、-Wpedantic 各管什么

新手的第一个误解是「`-Wall` 就是打开全部警告」。**不是。** `-Wall` 是一张手挑的名单，挑的是「几乎总是真问题」的那一类。

> **⚠ 陷阱 · 「-Wall」里的 W 是 warnings，不是 war on all**
>
> **你会看到什么**：加了 `-Wall`，编译器还是对你代码里的隐式转换、变量遮蔽一言不发；于是你以为「编译器没意见 = 代码没问题」。
>
> **你会怎么误判**：把「编译通过」当成「代码是对的」。
>
> **真相是**：`-Wall` 只打开一份固定名单；`-Wextra` 再打开一份；还有一批开关（`-Wconversion`、`-Wshadow`、`-Wfloat-equal`、`-Wformat=2` …）**要你自己加，两个都不含。**而本书的基础参数里也**没有**它们——所以「在本书的参数下编译干净」不等于「这段代码没有隐患」。

空口说「多」没用，我们让 gcc 自己报数。它会带一张自己的开关表，可以用 `-Q --help=warnings` 打出来；我们数一下每个档位下**处于开启状态**的开关有多少条：

终端会话 让 gcc 自己数：几个档位各开着多少条警告开关 ch02-switch-count

```
$ gcc -Q --help=warnings 2> /dev/null | grep -c '\[enabled\]'
113
$ gcc -Q --help=warnings -Wall 2> /dev/null | grep -c '\[enabled\]'
171
$ gcc -Q --help=warnings -Wall -Wextra 2> /dev/null | grep -c '\[enabled\]'
191
$ gcc -Q --help=warnings -Wall -Wextra -Wconversion -Wshadow -Wfloat-equal -Wformat=2 2> /dev/null | grep -c
'\[enabled\]'
201
```

四个数字连起来读：**什么都不加，就有 113 条开关是开着的**（默认档不是「什么都不报」）；`-Wall` 把它抬到 171；`-Wextra` 到 191；再加三个加严开关才到 201。要留神的是，这张表只在「开关是怎么被设的」这一层说话，它**不**告诉你某条开关在你的代码上会不会报——那是代码的事（下面马上有例子）。所以查表只能当参考，**最终判据永远是真跑一遍。**

那么这些开关落到具体代码上是几条诊断？拿这一单元的 `warn_zoo.c` 做同一件事的四步加严，每一步都真跑：

图 / 搜索过程 **四步加严：同一份 warn_zoo.c 上被点亮的诊断条数**

第 1 步 只加 -Wall

命令行 ： gcc -std=c17 -Wall -O0 -g warn_zoo.c　诊断条数 = 3

unused-variable / parentheses / format ： 这一档能看见的三类

第 2 步 补上 -Wextra 与 -Wpedantic

命令行 ： gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g warn_zoo.c　诊断条数 = 5

新冒出来两类 ： unused-parameter / sign-compare

第 3 步 再加三个加严开关

命令行 ： … -Wconversion -Wshadow -Wformat=2 warn_zoo.c　诊断条数 = 7

新冒出来两类 ： shadow / float-conversion

第 4 步 最后追加 -Werror

命令行 ： … 加严开关 -Werror warn_zoo.c　编译结果 = 失败 ： 同一份代码，退出码从 0 变成 1

四个数字（3 → 5 → 7 → 编不过）都来自这一份文件，命令与输出在会话里：

终端会话 同一份 warn_zoo.c：3 条 → 5 条 → 7 条 ch02-ladder

```
$ gcc -std=c17 -Wall -O0 -g -fdiagnostics-color=never warn_zoo.c -o /tmp/x.exe 2> /tmp/w.txt
$ grep -c 'warning:' /tmp/w.txt
3
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never warn_zoo.c -o /tmp/x.exe 2>
/tmp/w.txt
$ grep -c 'warning:' /tmp/w.txt
5
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Wconversion -Wshadow -Wformat=2
warn_zoo.c -o /tmp/x.exe 2> /tmp/w.txt
$ grep -c 'warning:' /tmp/w.txt
7
```

按第 1 帧到第 4 帧的顺序读：第 2 帧的两条增量（`unused-parameter`、`sign-compare`）来自 `-Wextra`；第 3 帧的两条（`shadow`、`float-conversion`）来自我们自己加的加严开关；第 4 帧的 `-Werror` 不新增诊断，它只是让编译器**把警告当成错误**——所以同一份代码从「编过、运行、退出码 0」变成「编译失败、退出码 1」。

现在把「干净」这件事摆到台面上。`extra_warnings.c` 这份文件在基础参数下一声不吭，加上三个加严开关才现原形：

c **extra_warnings.c：基础参数下 0 条警告** examples/ch01/extra_warnings.c 第 1–22 行

```
1  /* extra_warnings.c：基础参数下一声不吭，加上加严开关才现原形。
2     这份文件存在的意义就是那句话：「-O0 + -Wall 干净」不等于没问题。 */
```

```c
#include <stdio.h>

int main(void) {
    unsigned char small = 250;
    int wide = 300;
    double d = 2.5;
    int x = 1;

    small = wide;                   /* 隐式转换：int → unsigned char，值会变 */
    x = d;                          /* 隐式转换：double → int，小数被丢掉 */
    {
        int x = 2;                  /* 变量遮蔽：内层 x 挡住外层 x */
        printf("内层 x = %d\n", x);
    }
    if (d == 2.5) {                 /* 浮点相等比较：-Wfloat-equal */
        printf("d 正好等于 2.5\n");
    }
    printf("small=%d x=%d d=%.1f\n", small, x, d);
    return 0;
}
```

终端会话 同一份 extra_warnings.c：0 条 → 4 条 ch02-strict-diff

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never extra_warnings.c -o /tmp/x.exe 2>
/tmp/base.txt
$ echo $?
0
$ grep -c 'warning:' /tmp/base.txt
0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Wconversion -Wshadow -Wfloat-equal
extra_warnings.c -o /tmp/x.exe 2> /tmp/strict.txt
$ echo $?
0
$ grep -c 'warning:' /tmp/strict.txt
4
$ wc -l < /tmp/strict.txt
16
```

上面那段会话要连起来读：**第一次编译退出码 0、stderr 里 0 条警告**（`grep -c` 的输出就是 0）；第二次多了三个开关，**退出码仍然是 0**，但 stderr 里有 **4 条 warning、16 行**。这 4 条不会拦住你，却每一条都对应一个能真出事的行为：

终端输出 终端输出 extra-warnings-out

```
内层 x = 2
d 正好等于 2.5
small=44 x=2 d=2.5
```

程序自己跑了，输出看着挺正常——`small` 从 250 变成 44（300 截成 8 位），`x` 从 1 变成 2（`2.5` 的小数被丢掉），`d` 真的等于 2.5 所以那行也打印了。**编译器的四条警告指的就是这几处**，而程序一个错都没报。这就是那句要记住的话：**警告在说「你写的代码在语义上滑了一下」，运行结果往往照样「正常」**，正常到你看不出来。

工程惯例 · 工程惯例：加严开关怎么加、加在哪

`-Wconversion`、`-Wshadow`、`-Wfloat-equal`、`-Wformat=2` 这一类，属于「打开会更啰嗦、但能挡住真问题」的开关。惯例做法是：

- **开发时全开**：本地和持续集成里把它们一起加上，让问题在提交前就冒出来；
- **一次只加一个**：老代码上一次性全开，你会得到几百条警告，然后就会习惯性忽略——那等于没开；一次加一个、修完再加下一个，才修得动；
- **别用 `-Wno-*` 换安静**：关掉一条警告不等于问题解决了，只是把眼睛闭上了；真觉得某条警告没意义，正确做法是改代码让它不再触发，或者就地标注说明这一处是有意的——两种做法都会在正文里留下痕迹，`-Wno-` 不会（本书的清单也直接拒收这类参数）。

代价是真实的：`-Wconversion` 在大量做整数运算、位操作的老代码上会非常吵，而大部分结果其实是安全的。**这是惯例，不是标准**——标准里没有「警告」的第二层，只有 2.1 里那句「至少给一条诊断」。

2.4 六类高频警告：你会撞上的是这六种

`warn_zoo.c` 是一份「故意的脏代码」：六类高频警告一次收齐，而且每一处的错都写在注释里。加严档下它一共 7 条 warning（`unused-parameter` 和 `unused-variable` 算同一类）：

终端会话 warn_zoo.c 的七条警告（加严档，原文照抄） ch02-zoo-list

```
$ grep 'warning:' /tmp/w.txt
warn_zoo.c:9:9: warning: unused variable 'scratch' [-Wunused-variable]
warn_zoo.c:7:28: warning: unused parameter 'unused_flag' [-Wunused-parameter]
warn_zoo.c:24:11: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned
int' [-Wsign-compare]
warn_zoo.c:29:9: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
warn_zoo.c:35:13: warning: declaration of 'x' shadows a previous local [-Wshadow]
warn_zoo.c:40:32: warning: format '%d' expects argument of type 'int', but argument 2 has type 'double' [-
Wformat=]
warn_zoo.c:43:9: warning: conversion from 'double' to 'int' may change value [-Wfloat-conversion]
```

七条行号都在真源文件里，你可以逐条对回去。先看前两条的源码——它们在同一段里：

c examples/ch01/warn_zoo.c 第 6–15 行

```c
/* ① 参数没用上：-Wunused-parameter */
int sum_all(int limit, int unused_flag) {
    int total = 0;
    int scratch = 99;               /* ① 变量没用上：-Wunused-variable */
    for (int i = 0; i < limit; i++) {
        total += i;
    }
    return total;
}
```

7 参数声明了没用
9 变量定义了没用

① 未用变量与未用参数（`-Wunused-variable` / `-Wunused-parameter`）。它是六类里最像「唠叨」的一条，也是最不该忽略的一条。第 7 行的 `unused_flag` 是「函数签名留着、实现里没用」——常见于「为了和别处保持一致先占个位」；第 9 行的 `scratch` 更值得警惕：写它的人本来想拿它做点什么，后来忘了——**一个定义了却没用的变量，往往是「本该写的那几行还没写」**。1.5 里漏分号那个例子就是它的变体：因为语句没收尾，`total` 悬空了，于是多出一条「变量没用」。

再看中段的两处，它们的共同点是「代码能跑，但意思不是你以为的那个意思」：

c examples/ch01/warn_zoo.c 第 17–31 行

```c
int main(void) {
    unsigned int len = 3;
    int n = -1;
    double ratio = 3.14;
    int x = 10;

    /* ② 有符号与无符号比较：-Wsign-compare */
    if (n < len) {
        printf("n < len 成立\n");
    }

    /* ③ 赋值与比较混淆：-Wparentheses */
    if (x = 5) {
        printf("x 现在是 %d\n", x);
    }
```

24 拿 int 和 unsigned 比大小
29 这是赋值，不是判断相等

② 有符号与无符号比较（`-Wsign-compare`）。第 24 行的 `n` 是 `-1`，`len` 是 `3u`。你可能觉得 `-1 < 3` 当然成立——但比较之前，C 会把 `n` 转成 `unsigned int`，`-1` 于是变成一个非常大的正数，`n < len` **不成立**。编译器的说法

是 `comparison of integer expressions of different signedness`：两种符号性的整数放在一起比，出事的概率高到它决定提醒你。注意这一条要 `-Wextra` 才有：2.3 那张图的第 1 帧里就没有它。

③ 赋值与比较混淆（`-Wparentheses`）。第 29 行写的是 `if (x = 5)`，一个等号。它是**合法**的：先把 5 赋给 `x`，再判断 5 是不是非零——因为 5 非零，所以条件永远成立，里面的 `printf` 每次都执行。写这段的人本来想写 `x == 5`（判断 `x` 是不是 5）。编译器不拦你，因为它不能确定你是不是故意这么写的，它只说 `suggest parentheses around assignment used as truth value`：如果你想用赋值的结果当条件，那就加一层括号把意图写清楚。

下一处更隐蔽——它在同一段代码里又声明了一个同名的变量：

c examples/ch01/warn_zoo.c 第 33–37 行

```
33      /* ④ 变量遮蔽：-Wshadow，要自己加这个开关 */
34      {
35          int x = 1; 内层 x 挡住了外层
36          printf("内层 x = %d\n", x);
37      }
```

```
35 内层 x 挡住了外层
```

④ 变量遮蔽（`-Wshadow`）。第 35 行在内层花括号里又声明了一个 `x`。从这一行到那个花括号结束，内层的 `x` 把外层的 `x`（第 21 行）遮住了：你在这个块里写 `x = 7`，改的是内层那个，出了块就没了。这个坑最阴的地方是**它不报错、行为也「合理」**——只是你改的不是你以为的那个变量。它要自己加 `-Wshadow`，`-Wall` 、`-Wextra` 都没有。

最后两条在这段里：

c examples/ch01/warn_zoo.c 第 39–44 行

```
39      /* ⑤ 格式串与参数类型不匹配：-Wformat */
40      printf("ratio 当成整数打：%d\n", ratio); 用 %d 打 double
41
42      /* ⑥ 隐式转换：-Wconversion，要自己加这个开关 */
43      x = ratio; double 赋给 int
44      printf("x = %d\n", x);
```

```
40 用 %d 打 double
43 double 赋给 int
```

⑤ 格式串与参数类型不匹配（`-Wformat`）。这一条**不需要加严开关**，`-Wall` 就管——因为 `printf` 的格式串是编译期能看懂的：第 40 行给了它 `%d`，而参数是个 `double`。这类错误的后果值得单独用一节看（2.4.1）。

⑥ 隐式转换（`-Wconversion` 及其子开关 `-Wfloat-conversion`）。第 43 行把一个 `double` 赋给 `int`，编译器按规则把小数部分丢掉（不是四舍五入）。`-Wconversion` 要自己加；这一条也被列进了加严名单的原因很简单：C 里到处都是隐式转换，而其中一小部分会悄悄改变值。

这七条警告拦住了吗？没有。程序照样编过、照样跑，输出是这样的：

终端输出 终端输出 warn-zoo-out

```
x 现在是 5
内层 x = 1
ratio 当成整数打：1374389535
x = 3
sum_all(4, 1) = 6
```

请对着上面那三段源码读这五行：

- 第一行 `x 现在是 5` 是**第 ③ 类**的现场——`if (x = 5)` 真的进了分支；
- 第二行 `内层 x = 1` 是**第 ④ 类**的现场——打印的是内层那个 `x`；
- 第三行 `ratio 当成整数打：1374389535` 是**第 ⑤ 类**的现场，也是本书正文里第一个「看着像乱码、其实有来历」的数字，下面单独说。

2.4.1 一个 double 用 %d 打出来，为什么会是这个数

先把这行单独请出来看（`warn_zoo.c` 与 `format_mismatch.c` 里各写了一处，数字一样）：

c examples/ch01/format_mismatch.c 第 1–11 行

```c
/* format_mismatch.c：格式串与参数类型不匹配。
   编译器警告会告诉你哪里错，程序照样跑——跑出来的数字看着像乱码。 */
#include <stdio.h>

int main(void) {
    double price = 3.14;

    printf("用 %%f 打 price: %f\n", price); 正确：double 配 %f
    printf("用 %%d 打 price: %d\n", price); 故意错：double 配 %d
    return 0;
}
```

8 正确：double 配 %f
9 故意错：double 配 %d

终端输出 终端输出 format-mismatch-out

```
用 %f 打 price: 3.140000
用 %d 打 price: 1374389535
```

`1374389535` 不是随机数，它**每一次运行都是这个值**（这两行输出由流水线逐字对账，对不上就报 `stale_output`）。它是什么？把 `3.14` 按双精度浮点放进内存，八个字节是 `1f 85 eb 51 b8 1e 09 40`；取低四位字节 `51 eb 85 1f`，按小端读出来就是十进制的 **1374389535**。换句话说，`%d` 读到的不是「那个数」，而是**那个 double 机器码的低 32 位。**

标准原文 · 标准原文：格式串对不上，行为是未定义的

C 标准在 `fprintf` 那一节里写得很干脆：

> If any argument is not the correct type for the corresponding conversion specification, the behavior is undefined.

大意：**只要某个参数的类型和它对应的转换说明对不上，行为就是未定义的。**注意它没有说「会打印垃圾」——未定义的意思是**标准对此不作任何承诺**：换一台机器、换一个编译器，打印什么都有可能。

出处：ISO/IEC 9899:2018（C17）§7.21.6.1 第 9 段。

那为什么本机每次都打到同一个值？因为**这台机器的调用约定**替我们定死了这件事：在 Windows x86-64 上，可变参数函数里的浮点参数会被**同时抄一份**到对应的整数寄存器里；`%d` 按整数去找，正好找到了那份副本的低 32 位。这个解释只对**这个平台**成立——它不是语言保证，是实现的巧合。

⚠ 陷阱 · 「这次打出来是 1374389535，那我改代码时它还会是这个数吧」

你会看到什么：改一改上面的代码，数字跟着变；不改代码，数字稳定复现。

你会怎么误判：把它当成一个「有定义的行为」，甚至拿它去写测试、去推断程序逻辑。

真相是：标准说这是未定义行为。今天你看到 1374389535，是因为寄存器里恰好躺着那个 double 的副本；把 `-O2` 打开、把参数顺序换一下、把 `printf` 换成 `fprintf`，它就可能变成另一个数甚至直接崩掉。**见到这类「稳定复现的怪值」，要做的不是记下它，而是回到格式串去改它。**

（`warn_zoo.c` 那份文件里还有第 ⑥ 类的现场：`x = ratio;` 之后 `x` 是 3，小数没有四舍五入——这一条留给你自己去看。）

2.5 一串诊断先改第一条

到这里你大概已经发现：一次编译往往不止给你一条诊断。那就得有个读法。`diag_chain.c` 只错了一个字符——第 6 行末尾少了一个分号：

c examples/ch01/diag_chain.c 第 3–11 行

```c
#include <stdio.h>
```

```c
int main(void) {
    int total = 0 这一行末尾少了一个分号
    int count = 3;
    int average = (total + count) / 2;
    printf("average = %d\n", average);
    return 0;
}
```

6 这一行末尾少了一个分号

一个字符的错，编译器给了三条诊断：

编译诊断 gcc 的真实诊断（原文照抄） diag-chain-diag

```
diag_chain.c: In function 'main':
diag_chain.c:7:5: error: expected ',' or ';' before 'int'
    7 |     int count = 3;
      |     ^~~
diag_chain.c:8:28: error: 'count' undeclared (first use in this function)
    8 |     int average = (total + count) / 2;
      |                            ^~~~~
diag_chain.c:8:28: note: each undeclared identifier is reported only once for each function it appears in
```

按 2.2 的四段读法拆开：

- **第 1 条** `error` 报在 `diag_chain.c:7:5`，说的是 `expected ',' or ';' before 'int'`——「在第 7 行这个 `int` 前面，我期待一个逗号或分号」。注意：**要改的是第 6 行的末尾。**为什么报在第 7 行？因为编译器只有读到第 7 行的 `int`，才能确定第 6 行那条语句还没收尾。这个「报错行号比真正的错误晚一行」的现象，1.5 的 `semicolon.c` 里见过一次，这里是它的第二次出现。
- **第 2 条** `error` 报在 `diag_chain.c:8:28`，`'count' undeclared (first use in this function)`——「`count` 没有声明」。可是第 7 行明明写了 `int count = 3;`。原因是第 7 行自己也没被解析成一条完整语句，于是 `count` 这个名字**没有被承认**，第 8 行用它的地方就报「没声明」。
- **第 3 条** `note` 是给第 2 条的补充：`each undeclared identifier is reported only once for each function it appears in`——「同一个没声明的名字，在一个函数里我只报一次」。这是编译器在告诉你**别指望后面的重复错误**，也是它劝你「先修上面那条」。

三条诊断里只有第一条是真错误，第二、三条都是它的**下游**。所以有一条读法要背下来：**先只改第一条，重编，再看剩下的还在不在。** 一次全改会让你分不清哪条诊断是哪个改动带来的；而很多时候，改完第一条，后面几条自己就消失了。这条规则在 1.5 里已经用过一次（漏分号时那条「变量没用」的警告），这一节是它更清楚的版本。

⚠ **陷阱** · **「一屏二十条错误，我从第一条一口气改到底」**

你会看到什么：几十条 `error` 刷满屏幕，第一条和第二条看起来毫无关系。

你会怎么误判：以为代码有几十个毛病，一边改一边怀疑自己的水平。

真相是：绝大多数情况下，**真错误只有一两条**，剩下的全是它们的下游——一个漏掉的分号可以让编译器整段看不懂，于是「每一样东西都不认识」。做法永远是：改第一条 → 重新编译 → 看条数掉到几条 → 再改新的第一条。上面那个例子里，补上第 6 行末尾的分号，三条诊断一起消失。

2.6 编译期、链接期、运行期：三份输出归谁管

1.3 里我们把「四步」讲了一遍（预处理、编译、汇编、链接）。诊断层面只需要记住一条更粗的分界线：**编译期一件事、链接期一件事、运行期又一件事**，而且它们的失败长得完全不一样。

先看一个只有声明、没有实现体的函数。`undef_ref.c` 故意调用了一个「住在别处」的函数，而那个别处——我们**没有把它列进命令**：

c examples/ch01/undef_ref.c 第 4–11 行

```c
#include <stdio.h>

extern int helper_add(int a, int b);   /* 声明在，实现体不在 */ 声明在，实现体不在

int main(void) {
    printf("sum = %d\n", helper_add(2, 3));
    return 0;
}
```

6 声明在，实现体不在

先只编译、不链接（`-c` 的含义 2.7 讲）：

终端会话 -c 只做编译：退出码 0，还产出了目标文件 ch02-undef-compile-ok

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -c undef_ref.c -o /tmp/undef_ref.o
$ echo $?
0
$ ls -l /tmp/undef_ref.o
-rw-r--r-- 1 Administrator 197121 2632 Oct 19 16:50 /tmp/undef_ref.o
```

编译期一次错都没报，退出码 0。 为什么？因为编译期只对**一个源文件**负责：它看见了 `extern int helper_add(int a, int b);` 这行声明，于是知道「这个名字存在，参数是两个 int，返回值是 int」，调用它合法。至于这个名字后面到底有没有人实现——编译期不管，那是链接期的事。

现在把链接也做了，也就是不给 `-c`、直接出可执行文件。这一次编译器交给你三行：

编译诊断 gcc 的真实诊断（原文照抄） undef-ref-diag

```
C:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe:
C:\Users\Administrator\Documents\Study\dev-notes\dist\work\ch01-undef-ref\ch01-undef-ref-undef_ref.o: in
function `main':
C:/Users/Administrator/Documents/Study/dev-notes/books/c/examples/ch01/undef_ref.c:9:(.text+0x18): undefined
reference to `helper_add'
collect2.exe: error: ld returned 1 exit status
```

三行各说一件事：

1. **第一行**：`ld.exe` 是链接器，它说问题出在某个目标文件里。留意那个 `.o` 的名字 `ch01-undef-ref-undef_ref.o` ——**它不是随机临时名**，这是目标文件被钉在产物目录里的结果（下面那个惯例块解释为什么）。
2. **第二行**：真正有用的一句。`undef_ref.c:9:` 指出源文件第 9 行，`undefined reference to 'helper_add'` 指出缺的名字是 `helper_add`。**`undefined reference to` 是链接期错误的招牌。**
3. **第三行**：`collect2.exe` 是 gcc 用来调链接器的那层壳，它只转述「链接器失败了，退出码 1」。

跟编译期的诊断比一比：链接期**没有列号、没有插入符、也没有把源码那行抄给你**——因为它手里只有名字，没有源码（也就是下面那张表的分工）。但它给了编译期给不了的东西：**名字**。所以排查有一条快路：**先看 `undefined reference to` 后面那个名字，再回到命令看哪个源文件没被列进去。**

工程惯例 · **工程惯例：为了让诊断可复现，把临时文件名钉住**

默认情况下，gcc 把编译出的目标文件放进临时目录，名字是随机的（`ccKC1gS4.o` 这种）。链接失败时 ld 会把这个名字写进第一行——于是同一条错误，每次跑出来的**字节都不一样**。本书正文的诊断要逐字对账，所以这个例子的清单里加了 `-save-temps`：它让中间产物落到**由产物名推出来的固定名字**上（这里是 `ch01-undef-ref-undef_ref.o`），诊断才稳定。

代价有两个：产物目录里会多出 `.i` / `.s` / `.o` 几个文件；诊断里会出现一长串绝对路径（上面第一行那个长度就是）。这是「可复现」换来的，**不是标准的规矩**——标准压根不管诊断怎么排版。

> ⚠ **陷阱 · 同一个链接错误，可能是三行，也可能只剩一行**
>
> **你会看到什么**：在某些终端里，同一份代码、同一条命令，只得到一行 `collect2.exe: error: ld returned 5 exit status` ——没有 `undefined reference`、没有文件名，连个列号都没有。
>
> **你会怎么误判**：以为「本机的链接错误就这么短、这么没用」，于是放弃从诊断里找线索，直接开始瞎改代码。
>
> **真相是**：那**不是链接器想说的话，是它的话被环境吞了。**我在同一台机器上实测过两种环境：默认 PATH 下只剩一行、`ld returned 5`；把 PATH 清成只剩 MSYS2 的几个目录（`/c/msys64/ucrt64/bin:/c/msys64/usr/bin:/c/Windows/System32`）再跑，就变成上面那三行、`ld returned 1`。同一个 gcc、同一个 `ld.exe` ——连单独跑一句 `ld --version`，默认 PATH 下也会丢掉第一行（4 行 vs 5 行）。差别只在环境变量；具体是哪个 DLL 在捣乱我没有继续追，但**现象是可复现的。**
>
> **正文的三行诊断怎么保证每次都一样？** 这个例子的清单里有一个 `env` 字段，把编译时的 PATH 钉成了上面那组干净值。环境变量和 `-save-temps` 一样，都是实验条件的一部分——不钉住，诊断就会随终端变样。
>
> **怎么办**：看到只有 collect2 那一行时，换一个干净终端再编一次，别把「诊断短」当成「没有线索」。

那次对照的原文在这里，三件事实一眼就能对上（先记下默认 PATH 的样子，再把 PATH 清掉重跑）：

终端会话 同一个链接错误：一行 vs 三行，同一个 ld — ch02-link-shape

```
$ echo $PATH | cut -d: -f1-3
/c/Users/Administrator/bin:/mingw64/bin:/usr/local/bin
$ /c/msys64/ucrt64/x86_64-w64-mingw32/bin/ld.exe --version | wc -l
4
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never undef_ref.c -o /tmp/shape1.exe 2>&1
| tail -1
collect2.exe: error: ld returned 5 exit status
$ export PATH=/c/msys64/ucrt64/bin:/c/msys64/usr/bin:/c/Windows/System32
$ /c/msys64/ucrt64/x86_64-w64-mingw32/bin/ld.exe --version | wc -l
5
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never undef_ref.c -o /tmp/shape2.exe 2>&1
| tail -1
collect2.exe: error: ld returned 1 exit status
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never undef_ref.c -o /tmp/shape3.exe 2>&1
| grep -c "undefined reference"
1
```

补上缺的那个源文件，一条命令编两个文件就通过了：

终端会话 补上缺的那个源文件：一条命令编两个 .c — ch02-two-files

```
$ printf 'int helper_add(int a, int b) { return a + b; }\n' > /tmp/helper_add.c
$ cat /tmp/helper_add.c
int helper_add(int a, int b) { return a + b; }
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never undef_ref.c /tmp/helper_add.c -o
/tmp/undef_ok.exe
$ echo $?
0
$ /tmp/undef_ok.exe
sum = 5
```

（那个 `helper_add.c` 是拿来补位的——它不属于这本书的任何例子，所以只放在临时目录里演一遍；**命令的形状才是重点**：`gcc … 源文件1.c 源文件2.c -o 程序`。）

把三个期摆在一起，你就能回答「报错了该看哪份输出」：

期	谁在干活	它看得见什么	失败时长什么样
编译期	编译器，一次一个源文件	这份文件 + 它 `#include` 进来的声明	`文件:行:列: error: …`，有插入符、有源文件那行
链接期	链接器，看所有 `.o` 加库	只有**名字**，没有源码	`undefined reference to 'xxx'` + `collect2.exe: error: ld returned 1 exit status`；没有列号与插入符

期	谁在干活	它看得见什么	失败时长什么样
运行期	程序自己	真数据	打印出离谱的值、或者进程突然结束、或者退出码不是 0

运行期的样子，这一单元已经出现过一次：`format_security.c` 把用户给的字符串当格式串用，**同一个输入跑两次，给出两组不同的数字**——

终端会话 运行期：同一个命令跑两次，结果不一样 ch02-run-phase

```
$ /tmp/fs.exe "%d %d %d"
6953296 6969872 6931280
$ /tmp/fs.exe "%d %d %d"
7805648 7821840 7783248
```

这段会话就是「运行期」这一课的核心：**编译器没为这个缺陷说一句话**（它唯一的提醒是「参数没用上」，跟这个漏洞无关），程序退出码 0，而输出的数字每次都不一样。谁该为它负责？标准说这是未定义行为（§7.21.6.1 第 9 段，2.4.1 引过）：**格式串配上不合适的参数，行为没有任何保证。**运行期的错误不需要编译器批准，它只看你喂进去什么。

`format_security.c` 的源码值得看一眼——它就长在你的命令行输入和 `printf` 之间：

c examples/ch01/format_security.c 第 6–15 行

```c
int main(int argc, char *argv[]) {
    if (argv[1] == NULL) {
        printf("用法: format_security.exe <要打印的文字>\n");
        return 1;
    }

    printf(argv[1]);                /* 危险：格式串来自命令行 */
    printf("\n");
    return 0;
}
```

6 argc 没用上，加严档会报
12 用户给的字符串直接当格式串

终端输出 终端输出 format-security-out

```
hello from argv[1]
```

按清单里的约定，这个例子跑的时候带了一个参数（`hello from argv[1]`），输出就是它。而上面的危险输入是我在终端里手工喂给它的——**同一份程序，正常输入看着人畜无害，一个 `%` 就把它变成了未定义行为。**这就是「格式串漏洞」这个说法的来历：你以为是数据的东西，被当成了指令。

2.7 `-c` 与一条命令：本机没有 make

`stages_c.c` 是一个再普通不过的小程序：

c **stages_c.c：用来对照两种编法** examples/ch01/stages_c.c 第 1–11 行

```c
/* stages_c.c：用来对照「-c 两步走」与「一条命令」两种编法。 */
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main(void) {
    printf("add(2, 3) = %d\n", add(2, 3));
    return 0;
}
```

终端输出 终端输出 stages-c-out

```
add(2, 3) = 5
```

它有两种编法，产物不一样：

终端会话 `-c 只出 .o，链接以后才出 .exe` `ch02-c-vs-link`

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -c stages_c.c -o /tmp/stages_c.o
$ ls -l /tmp/stages_c.o
-rw-r--r-- 1 Administrator 197121 2813 Oct 19 16:50 /tmp/stages_c.o
$ gcc /tmp/stages_c.o -o /tmp/stages_c.exe
$ ls -l /tmp/stages_c.exe
-rwxr-xr-x 1 Administrator 197121 135094 Oct 19 16:50 /tmp/stages_c.exe
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never stages_c.c -o /tmp/stages_c-one.exe
$ ls -l /tmp/stages_c-one.exe
-rwxr-xr-x 1 Administrator 197121 135094 Oct 19 16:50 /tmp/stages_c-one.exe
```

三个数字要连起来读：目标文件 `stages_c.o` 是 **2813 字节**，链接之后的可执行文件是 **135094 字节**——**是它的约 48 倍**（135094 ÷ 2813 = 48.02），多出来的全是从运行库里接进来的东西（`printf` 的实现体就在里面，1.4 讲过这件事）。而最后一条命令不带 `-c`、直接出可执行文件，产物大小是 **135094 字节**，和「先 `-c` 再链接」的结果**一模一样**。所以：

- `-c` = **只编译不链接**，产出 `.o`；`-o` 决定产物叫什么、放哪（不写 `-o`，默认在当前目录产出 `a.exe`，很容易找不到）；
- 不带 `-c` = 编译 + 汇编 + **链接**一条龙，产出可执行文件；
- 两条路殊途同归，`-c` 只是把中间那一步的产物**留下来了**。什么时候要留？改了一个文件只重编它自己、或者想把别人给的 `.o` 和你的 `.o` 拼在一起的时候——就是下面这种：

终端会话 `两个 .o 拼成一个程序` `ch02-two-objects`

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -c /tmp/helper_add.c -o
/tmp/helper_add.o
$ gcc /tmp/undef_ref.o /tmp/helper_add.o -o /tmp/undef_ok2.exe
$ /tmp/undef_ok2.exe
sum = 5
```

一条命令可以列多个源文件，也可以列多个目标文件。 这就是本机（没有 make、没有 cmake）编多文件工程的唯一办法：把源文件全列在命令里。

工程惯例 · **工程惯例：为什么本机给「一条命令」而不是构建脚本**

主流工程会用 `make` / `cmake` / `ninja` 把「哪些文件该重编」这件事算出来，你只敲一句 `make`。这套书**不用它们**，原因有两个：本机没装（真的没有，不是不想用）；而且在第 1 章就上构建工具，你会先学会敲 `make`、后知道 `make` 底下在干什么——顺序反了。

代价是真实的：项目一大，命令会变得很长，而且**改一个头文件要重编所有包含它的文件**这件事得你自己记得。第 17 单元会补上这一课：多文件工程怎么摆、头文件里写什么、以及为什么「一条命令列全部源文件」很快就撑不住。

大白话 · **用大白话说这三条命令**

- `-c`：把 `.c` 翻成机器码，**装进一个盒子（`.o`）就不管了**。盒子上贴着「我引用了这些名字，但它们的地址还不知道」；
- 不带 `-c`：翻完机器码之后，**再找一个叫链接器的搬运工**，把所有盒子里的「不知道的地址」一个个填上，填不上就报错——2.6 里那一行 `collect2.exe` 就是搬运工在喊；
- `-o`：给产物起名。你不写，它自己起名 `a.exe`，然后你就得满硬盘找它。

一句话：**`-c` 管「翻译一个文件」，链接管「把文件凑成一个程序」。** 2.4 里 `printf` 能从 `stdio.h` 的声明走到运行库的实现体，走的就是链接这一步。

2.8 `-O0` 与 `-g`：为什么本书默认不做优化

本书的参数里有两个看起来「无用」的开关：`-O0`（不做优化）和 `-g`（带上调试信息）。它们不是可选项，是这门课能不能学下去的前提。我们用调试器看同一份 `stages_c.c` 的两种编译结果——`-O0 -g` 和 `-O2 -g`：

终端会话 `-O0`：断点停住了，参数看得见，调用链完整 `ch02-gdb-o0`

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never stages_c.c -o /tmp/stages-O0.exe
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -O2 stages_c.c -o /tmp/stages-O2.exe
$ gdb --batch -ex "break add" -ex run -ex "info args" -ex bt /tmp/stages-O0.exe 2>&1
Breakpoint 1 at 0x1400014ba: file stages_c.c, line 5.
[New Thread 88152.0x141e0]
Thread 1 hit Breakpoint 1, add (a=2, b=3) at stages_c.c:5
5           return a + b;
a = 2
b = 3
#0  add (a=2, b=3) at stages_c.c:5
#1  0x00007ff63e3114e0 in main () at stages_c.c:9
```

终端会话 `-O2`：断点根本没停，栈是空的 `ch02-gdb-o2`

```
$ gdb --batch -ex "break add" -ex run -ex "info args" -ex bt /tmp/stages-O2.exe 2>&1
Breakpoint 1 at 0x1400014b0: file stages_c.c, line 5.
[New Thread 67672.0x1542c]
add(2, 3) = 5
[Thread 67672.0x1542c exited with code 0]
[Inferior 1 (process 67672) exited normally]
No frame selected.
No stack.
```

（第一块开头那两条 `gcc` 命令是连着敲的：先出 `-O0` 那份、再出 `-O2` 那份；两块 gdb 输出分别对应它们。）

两块对着读，差别一目了然：

- `-O0 -g`：gdb 停住了，`a = 2`、`b = 3` 都看得见，`bt` 打出 `#0 add … line 5` 和 `#1 … in main () at stages_c.c:9`——**行号和你源文件里的行号一一对应**，调用链完整。
- `-O2 -g`：程序自己把 `add(2, 3) = 5` 打印完就正常退出了，`break add` **一次都没停**，`bt` 说 `No frame selected. / No stack.`。原因是 `-O2` 把 `add` 内联进了 `main`——函数还在，但**不再是一个可以停下来的地方**了。

「内联」这件事不需要猜，看汇编就知道。同一份 `stages_c.c` 用 `-S` 只生成汇编，再数一数有没有 `call add`：

终端会话 汇编里数一数：`-O0` 有一次 `call add`，`-O2` 一次都没有 `ch02-inline-proof`

```
$ gcc -std=c17 -O0 -S stages_c.c -o /tmp/s0.s
$ grep -c 'call.*add' /tmp/s0.s
1
$ gcc -std=c17 -O2 -S stages_c.c -o /tmp/s2.s
$ grep -c 'call.*add' /tmp/s2.s
0
```

`-O0` 的汇编里有 **1** 次 `call add`，`-O2` 的是 **0** 次：`add` 的函数体还在（所以 gdb 还能在它上面下断点），但 `main` 已经不调用它了——加法的两条指令被直接搬进了 `main`。断点停不住，不是 gdb 坏了，是**程序里已经没有那次调用了**。

这就是 `-O0` 与 `-g` 的价值，说清楚它们的边界：

- `-g` 让编译器把「哪条机器码对应源文件哪一行、哪个变量住在哪」写进目标文件。没有它，gdb 只能给你一串地址。
- `-O0` 让编译器**照着你的写法生成代码**：不内联、不把变量塞进寄存器、不删掉「显然没用」的语句。所以断点停得住、`info args` 有值、行号能对上。
- 代价是真实的：`-O0` 跑得慢、汇编长、有些优化相关的分析不做。本书接受这个代价，因为**这一阶段你要看的是「程序在按你写的样子走」**。

工程惯例 · 工程惯例：什么时候用什么档

标准里没有 `-O` 这个概念，它纯粹是编译器开关。惯例是：**发布**用 `-O2`、不带 `-g`（快、体积小）；**要读诊断、要单步调试**时用 `-O0 -g`。`-O2 -g` 也能同时存在，只是行号和变量不一定对得上（上面第二块就是活证据）。所以本书把 `-O0 -g` 定成默认，把 `-O2` 留作对照实验。

2.8.1 有些警告只在开优化时才出现

`-O0` 有一个不显眼的副作用：**一部分警告根本不会触发**，因为它们要靠优化阶段的数据流分析才看得见。所以「`-O0` + `-Wall` 编得干净」这句话**不能**推广成「这段代码没问题」——2.3 讲的是「加严开关」那一半（开关不开，诊断不出现），这一节讲「优化等级」那一半（优化不开，分析看不见）。

先看一个让人泄气的现象：gcc 那张开关表**看不出**这件事。同一份表，`-O0` 和 `-O2` 数出来一样多：

终端会话 开关表在 -O0 与 -O2 下报一样的数 ch02-opt-table

```
$ gcc -Q --help=warnings -O0 2> /dev/null | grep -c '\[enabled\]'
113
$ gcc -Q --help=warnings -O2 2> /dev/null | grep -c '\[enabled\]'
113
```

两个 113 说明：**优化等级不体现在这张表里**，它改变的是「编译器分析得多深」，不是「哪些开关被打开」。所以这类差异只能靠真跑。跑的结果很直接——同一份 `analyzer_demo.c`（一个真实的栈越界，样板间里出现过），`-O0` 和 `-O2` 给的**不是同一条**诊断：

终端会话 同一份越界代码：-O0 报一条，-O2 报另一条 ch02-opt-diff

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never analyzer_demo.c -o /tmp/ad0.exe 2>&1
| grep "warning:" | sed "s/.*warning: //"
'__builtin_memcpy' writing 11 bytes into a region of size 8 overflows the destination [-Wstringop-overflow=]
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -fdiagnostics-color=never analyzer_demo.c -o /tmp/ad2.exe 2>&1
| grep "warning:" | sed "s/.*warning: //"
'__builtin_memcpy' forming offset [8, 10] is out of the bounds [0, 8] of object 'buf' with type 'char[8]' [-
Warray-bounds=]
```

（`sed` 那一小段只是把行首的 `文件:行:列: warning:` 前缀剪掉，好让两条并排比——输出的正文一个字没改。）

两条警告说的是同一个越界，但开关名不同：`-O0` 下是 `-Wstringop-overflow=`，`-O2` 下是 `-Warray-bounds=`。**`-O2` 那一条在 `-O0` 的报告里根本不存在**——它需要优化阶段把 `strcpy` 展开、把数组大小推出来，才谈得上「越界」。

再给一个更纯的例子：**一条在 `-O0` 下彻底不出现、只在 `-O2` 下出现的警告**。它太小了，不值得占一个 example（就五行），所以在临时目录里现写现编：

终端会话 探针：五行代码，现写现编 ch02-opt-probe

```
$ cd /tmp
$ cat > opt_only.c <<'EOF'
int pick(int n) {
    int r;
    if (n > 0) { r = n; }
    return r;
}
EOF
```

终端会话 探针：-O0 下安静，-O2 下多出一条 ch02-opt-probe-run

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -c opt_only.c -o /tmp/oo.o
$ echo $?
0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -fdiagnostics-color=never -c opt_only.c -o /tmp/oo.o 2>&1
opt_only.c: In function 'pick':
opt_only.c:4:12: warning: 'r' may be used uninitialized [-Wmaybe-uninitialized]
    4 |     return r;
      |            ^
opt_only.c:2:9: note: 'r' was declared here
    2 |     int r;
      |         ^
$ echo $?
0
```

`-O0`：一行没说，退出码 **0**。`-O2`：多出一条 `'r' may be used uninitialized [-Wmaybe-uninitialized]`，而**退出码仍然是 0**（警告不拦你）。这条警告说的是「`r` 可能没被赋值就被用了」——在这个探针里它其实是**保守的误报**（`r`

只在 `n > 0` 的路径上被用，那条路径上确实赋了值），但它精确地证明了一件事：**换一个优化等级，编译器能看见的东西就变了。**

所以本书的参数纪律里，`-O0` 与「加严开关」是**两把不同的钥匙**：`-O0` 保证你能调试、行号对得上；加严开关保证更多问题被说出来。两者换来的是「干净」这个词的可靠性。**记住这个顺序：看到「编译干净」，先问「用了什么参数编的」，再决定要不要放心。**

2.9 `-Werror`：让警告拦住你，以及它的代价

2.3 的第 4 帧已经看到，加上 `-Werror` 之后同一份 `warn_zoo.c` 的退出码从 0 变成 1：

终端会话 警告升格成错误：退出码 1 — ch02-werror-exit

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Wconversion -Wshadow -Wformat=2 -
Werror warn_zoo.c -o /tmp/x.exe 2> /tmp/w.txt
$ echo $?
1
```

`-Werror` 做的事只有一件：**把所有警告当错误处理。**诊断的说明里会多出两个字（`warning:` 变成 `error:`），编译在第一条警告处停住，退出码变 1。为了让「同一份代码、两种结果」看得最清楚，我们把这份代码缩到只有一个警告——`werror_blocks.c`：

c examples/ch01/werror_blocks.c — 第 1–11 行

```c
/* werror_blocks.c：这份代码只有一个「变量定义了没用」的警告，
   不加 -Werror 时编译退出码 0、程序照跑；
   加了 -Werror，同一个警告就变成 error，编译直接失败。 */
#include <stdio.h>

int main(void) {
    int unused_total = 0;          // 定义了没用，就这一个警告

    printf("程序本身没问题：%s\n", "这两行就是全部输出");   // 程序本身没有别的问题
    return 0;
}
```

7 定义了没用，就这一个警告
9 程序本身没有别的问题

先不加 `-Werror`：

终端会话 不加 -Werror：退出码 0，警告照说，程序照跑 — ch02-no-werror

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never werror_blocks.c -o /tmp/werror.exe
2> /tmp/wb.txt
$ echo $?
0
$ cat /tmp/wb.txt
werror_blocks.c: In function 'main':
werror_blocks.c:7:9: warning: unused variable 'unused_total' [-Wunused-variable]
    7 |     int unused_total = 0;
      |         ^~~~~~~~~~~~
$ /tmp/werror.exe && echo "两件事都成了"
程序本身没问题：这两行就是全部输出
两件事都成了
```

最后那条命令值得单独看一眼：`gcc … && ./werror.exe && echo "两件事都成了"`——`&&` **按前一条命令的退出码决定要不要继续。**编译退出码 0 才轮到运行，运行退出码 0 才轮到 `echo`，所以最后那行 `两件事都成了` 是「三件事都成了」的证明。这就是「退出码是判据」的日常用法，也是脚本里最基本的写法。

现在把 `-Werror` 加上，同一份文件、同一个警告：

编译诊断 gcc 的真实诊断（原文照抄） werror-blocks-diag

```
werror_blocks.c: In function 'main':
werror_blocks.c:7:9: error: unused variable 'unused_total' [-Werror=unused-variable]
    7 |     int unused_total = 0;
      |         ^~~~~~~~~~~~
cc1.exe: all warnings being treated as errors
```

两个变化：`warning:` 变成了 `error:`，开关名也变成了 `[-Werror=unused-variable]`——方括号里那个 `-Werror=` 前缀就是「这条原本是警告，我按错误处理了」的记号。最后一行 `cc1.exe: all warnings being treated as errors` 是编译器把话说白了。

工程惯例 · **工程惯例：`-Werror` 什么时候开、什么时候别开**

它是一把双刃剑，所以主流做法是**分场合**：

- **该开**：持续集成（CI）、发布前的构建、新写的代码、以及任何「这份代码必须干净」的场合。它的价值在于**不让警告攒下来**——警告一旦攒到几百条，就没人看了；
- **不该默认开**：本地开发。你会为了通过编译去写一些没意义的代码（比如把没用上的参数加个 `(void)` 骗过编译器），而不是去想「这个参数是不是本来就该删掉」；
- **它的真实代价**：`-Werror` 把「警告」这个**可以商量的信号**变成了**硬门槛**，而这个信号是编译器版本相关的。换一个编译器版本、加一个新开关，**原本能编过的代码会突然编不过**——而且失败的原因不是你改了代码，是工具变严了。所以用 `-Werror` 的工程通常会把「编译器版本」也钉住（或至少知道自己在哪个版本上过）。

顺带说一句本书的做法：`-Werror` 只用在**故意编不过**的教学例子上（这份清单写的就是 `mode=compile_fail`），所以正文里它的任务是演示「警告升格」，不是推荐你默认打开。这是惯例、不是标准——标准里同样没有 `-Werror` 这个东西。

2.10 `-fanalyzer` 与「本机没有的那些检测手段」

上面所有警告都是在**翻译你的代码时顺手说一句**。编译器还有另一种模式：**专门去找某类缺陷**——它叫静态分析。gcc 把这一块挂在 `-fanalyzer` 上：

终端会话 `-fanalyzer`：一类专门的编译器，有检查器也有盲区 ch02-fanalyzer

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -fanalyzer -c analyzer_demo.c -o
/tmp/analyzer_demo.o 2> /tmp/an.txt
$ echo $?
0
$ grep -c 'warning:' /tmp/an.txt
2
$ grep 'Wanalyzer' /tmp/an.txt
analyzer_demo.c:6:5: warning: stack-based buffer overflow [CWE-121] [-Wanalyzer-out-of-bounds]
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -fanalyzer -c format_security.c -o
/tmp/fs.o 2> /tmp/an2.txt
$ echo $?
0
$ grep -c 'Wanalyzer' /tmp/an2.txt
0
$ wc -l < /tmp/an2.txt
4
```

这段会话里有四件事要读出来：

1. **它给的东西和普通警告不一样**：诊断里带 `[CWE-121]`（一个公开的缺陷分类编号）和 `[-Wanalyzer-out-of-bounds]` 这个专属标签；完整输出里还会配一张缓冲区示意图（这里用 `grep` 只挑出了带 `Wanalyzer` 的那一行）。
2. **编译退出码仍然是 0**——`-fanalyzer` 报的是「我认为这里有缺陷」，不是「我拒绝翻译」。所以它不能替代测试，**更不能当成运行结果**。
3. **它有盲区**：同一个 `-fanalyzer` 去看 `format_security.c`，一条分析器诊断都没有（`grep -c` 数出来 0）；那 4 行 stderr 是 `-Wextra` 给的「参数没用上」，跟缺陷无关。它**没看见**那个格式串漏洞。

4. 所以它的定位是：**锦上添花的第三只眼**，覆盖范围是一批特定缺陷（堆/栈越界、空指针之类），不是「全面体检」。

> **编译器视角** · **编译器视角：为什么「静态分析」不等于「运行结果」**
>
> `-fanalyzer` 干的事是**在你的代码上跑一遍模拟**：假设程序会怎么走、内存会怎么变，然后看有没有一条路径上出界了。它的两个特点是：
>
> - **可以报到你没执行过的路径**——这很有用，因为**测试覆盖不到的分支正是 bug 最容易藏的地方**；
> - **会误报**（false positive）——它的模型不可能和真实机器一样，宁可疑心重一点也不放过。
>
> 对照着看：`-fanalyzer` 是**编译期**的话（它可能报一个永远走不到的路径，也可能漏掉一个真缺陷）；2.4.1 里的 `1374389535` 是**运行期**的话（它只在真跑的时候才存在）。**这两类证据不能互相替代**，本书会一直把它们分开写。

那么「一台机器上到底有哪些检测手段」这件事，得如实交代。本机的实测结论是：**没有 ASan / UBSan。**

终端会话 ASan / UBSan：链接失败，本机没有 ch02-no-asan

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -fsanitize=address,undefined
stages_c.c -o /tmp/san.exe 2>&1
collect2.exe: error: ld returned 5 exit status
$ echo $?
1
```

`-fsanitize=address,undefined` 是 GCC 提供的运行期检测器（ASan 抓内存越界，UBSan 抓未定义行为）。在这台机器上它**编译能过、链接失败**——那一行 `collect2.exe` 说明链接器找不到它要的那两个运行库；按本机工具链事实文件的记录，这个 MSYS2 安装里没有 libasan / libubsan，软件仓库里也没有对应的包（这一条我没有再去仓库里核实，但**链接失败本身**是上面刚跑出来的）。标准里没有这两个工具的位置，它们是**工具链提供的**，有就有、没有就没有。

那本机剩下什么？把可用的手段和它们的边界列全（下面这张表里的结论来自本机的工具链事实文件，其中 `-fanalyzer` 与 ASan 这两条我在这一单元亲手跑过，输出就在上面）：

手段	什么时候说话	打开方式	抓住时你会看到	边界（它管不到什么）
`-fanalyzer`	编译期	`-fanalyzer`	`… [-Wanalyzer-out-of-bounds]`，编译退出码仍 0	只覆盖一批缺陷，会误报；不是运行结果
`-fstack-protector-all`	运行期	`-fstack-protector-all`	stderr 打出 `*** stack smashing detected ***: terminated`，退出码 **3221226505**	只保护**有局部数组的函数**里的栈溢出；堆越界抓不到
`-ftrapv`	运行期	`-ftrapv`	进程被终止，退出码 **3221226505**；缓冲区里没来得及打印的内容会丢	只管**有符号整数溢出**；无符号回绕、浮点都不算
`-D_GLIBCXX_ASSERTIONS`	运行期	`-D_GLIBCXX_ASSERTIONS`	打印 `Assertion '__n < this->size()' failed.`，退出码 **3**	只保护 **C++ 标准库容器/迭代器/`string`**；裸指针不保护（C 代码用不上它）
优化对照	对比两个构建	同一程序分别用 `-O0` 和 `-O2` 编译运行	行为不同即算抓到（可能表现为输出不同，也可能表现为一个变成死循环）	只能证明「这里存在优化敏感的未定义行为」，**不能断定是哪种**

这张表里没有「运行期精确的越界位置」这一项——没有 ASan 就没有它。本书写到内存错误时，证据只能是上表里的那几类：编译器的分析（`-fanalyzer`）、运行期真崩时的退出码与 stderr、或者 gdb 的栈回溯。**「应该会崩」这种话本书不写。**

工程惯例 · 工程惯例：为什么这些开关要写进清单的 `detect` 字段

每个例子的清单里有一项 `detect`，声明「靠哪个检测手段、证据是什么」。理由和 2.2 那对方括号一样：**把「谁抓到的」写下来，别人才能复核。**规矩三条：只能用本机真能用的手段；必须声明证据（编译期诊断文本、运行期非零退出码）；证据还得**归属于这个手段**——脚本会去掉检测参数再算一遍，同样的证据还在就判不通过。「程序自己打印一行 `stack smashing detected` 假装被抓住」这种例子在这里过不了。

最后说一个不起眼但贯穿全书的参数：`-fdiagnostics-color=never`。它的唯一作用是**让 gcc 不要把 ANSI 颜色码写进诊断文本。**这不是教学内容，是本书的流水线要求：正文里的每一段诊断都要和真实输出逐字一致，颜色码会随终端环境变化、在纸上也没意义，关掉它诊断文本才稳定可对账。你自己在终端里看时**不用**加它——彩色诊断更好读。

2.11 这一单元新增了什么，以及接下来

回到 2.1 的第 2 帧，现在可以逐个点名了：

- `warn_zoo.c` —— 六类高频警告一次收齐，每一处错都标了原因（2.3、2.4）；
- `extra_warnings.c` —— 基础参数下 0 条警告、加严开关下 4 条（2.3），用来证明「干净」这个词取决于参数；
- `format_mismatch.c` —— `printf("%d", 3.14)` 的警告，以及跑出来的 `1374389535`（2.4.1）；
- `format_security.c` —— `printf(argv[1])` 的格式串漏洞；基础参数下只报一条未用参数，加 `-Wformat=2` 才多出 `-Wformat-security`（2.4、2.6）；
- `werror_blocks.c` —— 同一份代码：不加 `-Werror` 退出码 0，加了退出码 1（2.9）；
- `diag_chain.c` —— 一个漏分号引出三条诊断，训练「先改第一条」（2.5）；
- `undef_ref.c` —— 编译能过、链接失败，诊断里给出 `undefined reference to 'helper_add'`（2.6）；
- `stages_c.c` —— `-c` 两步走与一条命令的对照，以及 `-O0` / `-O2` 的调试差异（2.7、2.8）；
- `session-diagnostics.txt` —— 上面所有命令与输出的抓取，正文里的每一段会话都从它注入；
- 另外这一单元还借用了已有的 `analyzer_demo.c`（越界那一段）和一份五行探针，用来讲「警告与优化等级的关系」（2.8.1）。

这一章仍然是单文件项目（第 3 帧）：`.c` 与清单平铺在 `examples/ch01/`，没有 `src/` 和 `include/`。为什么可以这样？因为这一章的任务是「**把一条命令和它的输出读懂**」——诊断、警告、退出码、产物，全都发生在一个源文件上。等这些看熟了，下一章开始换题目：**数据本身。**下一单元我们从 `int` 到底能装多大的数讲起，你会第一次遇到「`0.1 + 0.2` 不等于 `0.3`」这种看起来像编译器坏了的事——那时候你会很需要这一单元学的东西：**先问「用了什么参数」，再看「真实的输出是什么」。**

小结 · 这一单元的要点

- 一条诊断按 `文件:行:列: 级别: 说明` 四段读，**方括号里的开关名**告诉你这条由谁控制（`-Wno-…` 能关，但本书不许关）；级别只有三种口吻：`error` 停、`warning` 继续、`note` 补充；
- 标准对诊断只要求「至少一条」（§5.1.1.3），文字的格式、名字、分级全是实现的事；
- `-Wall` **不是全部**：基础档在 `warn_zoo.c` 上报 5 条，加严开关才到 7 条；同一份 `extra_warnings.c` 是 0 条 → 4 条；
- 六类高频警告各有真实的代价：未用变量与参数常是「该写的还没写」，格式串不匹配是未定义行为（§7.21.6.1），`-1 < 3u` 会判成假，变量遮蔽让你改到另一个变量，隐式转换会悄悄改值，`if (x = 5)` 永远成立。

小结 · （续）这一单元的要点

- 三个期各管一段：`undef_ref.c` 编译退出码 0、链接失败时给出 `undefined reference to 'helper_add'`；`1374389535` 是运行期的事；
- `-c` 只编译（`.o`，2813 字节），链接后才是程序（135094 字节）；一条命令可以列多个源文件/目标文件；
- `-O0` 与 `-g` 是本书默认：断点停得住、行号对得上（`-O2` 下 `add` 被内联，`bt` 直接说 `No stack.`）；有些警告只在优化或加严开关下出现；
- `-Werror` 把警告变成硬门槛（退出码 0 → 1），代价是编译器升级可能让你的代码突然编不过——放 CI，别放本地默认；
- 本机没有 ASan/UBSan（`-fsanitize=…` 链接失败）；可用的检测手段各管一摊，边界必须写清楚。

自测题 · 自测题（先自己答，再对答案）

1. 下面这条诊断由哪个开关控制？在本书的基础参数下它会报出来吗？只加 `-Wall` 呢？ `warn_zoo.c:24:11: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]`
2. `printf("用 %%d 打 price: %d\n", price);`（`price` 是 `double`）在本机跑出来是 `1374389535`，而且每次运行都一样。这个数字是怎么来的？C 标准对「格式串与参数类型不匹配」是怎么规定的？为什么不能因为它稳定复现就把它当成程序的行为？
3. `diag_chain.c` 只漏了一个分号，为什么会有三条诊断？你打算从哪一条开始改、改完第一件事做什么？
4. `undef_ref.c` 用 `-c` 编译退出码是 0，不加 `-c` 就失败。这两次分别在哪个阶段失败/成功？失败那段 stderr 里哪一行真正指出了「缺什么」？如果你看到的只有一行 `collect2.exe: error: ld returned 5 exit status`，那说明什么？
5. `-Werror` 的代价是什么？什么场合值得开？ `gcc … && ./app && echo ok` 这串命令里，`&&` 依据什么决定要不要往下走？

注 · 自测题答案

1. 由 `-Wsign-compare` 控制。**基础参数下会报**（因为基础参数里有 `-Wextra`），**只加 `-Wall` 时不报**——实测同一份 `warn_zoo.c`：`-Wall` 下 3 条 warning，加上 `-Wextra` 变 5 条，多出来的正是这条和 `-Wunused-parameter`。可以用 `-Wno-sign-compare` 关掉，但本书的清单不允许减警告参数（构建脚本会拒）。
2. `3.14` 作为双精度浮点，八个字节是 `1f 85 eb 51 b8 1e 09 40`，低四位字节是 `51 eb 85 1f`，按小端读成十进制就是 **1374389535**——`%d` 读到的是**这个 double 机器码的低 32 位**，因为本机（Windows x86-64）的可变参数调用约定会把浮点参数同时抄一份到整数寄存器里。标准的规定是：**参数类型与转换说明对不上就是未定义行为**（§7.21.6.1 第 9 段），也就是「不作任何承诺」。稳定复现只是因为这套寄存器约定在这台机器、这个编译选项下没变——换 `-O2`、换参数顺序、换平台就不再有任何保证。标准说的是「未定义」，不是「会打印 1374389535」，所以不能拿它当程序的行为来依赖。
3. 三条诊断里只有第一条是真错误：报在第 **7** 行第 5 列（`expected ',' or ';' before 'int'`），而**要改的是第 6 行的末尾**——编译器要读到第 7 行才能确定第 6 行那条语句没收尾。第二条 `'count' undeclared` 是下游：第 7 行自己没被解析成完整语句，`count` 这个名字就没被承认，第 8 行用它的地方自然报「没声明」；第三条 `note` 是补充（同一个没声明的名字只报一次）。做法：**只改第一条 → 重新编译 → 看剩下的还在不在**（这个例子里三条会一起消失）。
4. `-c` 只做**编译**（把源文件翻成目标文件），退出码 **0**：编译期只对**一个源文件**负责，它看见 `extern int helper_add(int a, int b);` 这行声明，就认为「这个名字存在、调用合法」。不加 `-c` 要把目标文件**链接**成程序，链接期才去落实「这个名字到底有没有实现体」，于是失败。三行里**第二行**才是关键：`undefined reference to 'helper_add'`，它直接点出缺的名字（第一行说问题在哪个 `.o` 里，第三行只是 collect2 的总结「退出码 1」）。如果只看到一行 `collect2.exe: error: ld returned 5 exit status`，说明链接器的正文被环境吞掉了（我实测：默认 PATH 下如此，换一个干净的 MSYS2 PATH 就恢复成三行），这时换终端重跑一次，或者直接回命令里找：**哪个源文件没被列进去**（补上它之后 `gcc undef_ref.c helper_add.c -o app` 退出码 0，程序打印 `sum = 5`）。
5. 代价：`-Werror` 把「警告」这个可以商量的信号变成硬门槛，而警告是**编译器版本相关**的——升级编译器、多开一个开关，**原本能编过的代码会突然编不过**，失败原因还不是你改了代码。值得开的场合：持续集成、发布前构建、新写的代码（核心目的是不让警告攒起来）。别默认开在本地开发。`&&` 看的是**前一条命令的退出码**：退出码 0 才执行下一条，非 0 就停下——所以 `gcc … && ./app && echo ok` 要能走到 `echo ok`，前提是编译和运行都成功。

第 2 章

数据与表达式：类型、取值与运算

本章 3 个单元 · 页眉从这里之后的页面开始出现

1. 类型与取值范围：整型、字符与浮点 零基础 · 第 3 单元 · 建议 70 分钟
2. 整数的边界：提升、溢出、位运算与枚举 零基础 · 第 2 章第 2 单元 · 建议 60 分钟
3. 表达式与求值：优先级、转换与顺序 零基础 · 第 5 单元 · 建议 90 分钟

类型与取值范围：整型、字符与浮点

◎ 学习目标 · 看完这一单元，你应该能

- 报出本机每种基本类型占几个字节，并说清「这个数字是这台机器的事实，不是 C 语言的规定」；
- 用 `<limits.h>` 与 `<float.h>` 里的宏问出每种类型的边界，并说出 `'0'`、`0`、`'\0'` 三个零各自是什么；
- 解释 `0.1 + 0.2` 为什么不等于 `0.3`，并在需要判断「够不够近」时改用 eps；
- 认出无符号数的两个坑（`i >= 0` 的倒序循环永不结束、`1u - 2` 得到 4294967295），并给每种值挑对格式串（`%d` / `%u` / `%f` / `%zu` / `%lld` / `PRIu32`）；
- 说出隐式转换最常藏身的四个地方，以及 `sizeof` 为什么（除变长数组外）不求值、`const` 为什么不是常量表达式。

3.1 先看工程长什么样

第 1 单元那个项目是**草稿**：一个目录、几个 `.c`，编译命令里只有一个源文件。从这一节起，本书的硬规定③生效——**从第二个项目（`examples/ch02/`）开始，不允许出现单文件工程。**所以这一节的第一件事不是讲类型，而是看目录。

目录树 **工程一：examples/ 下多出第二个项目**

第 1 步 本节开始时，只有第 1 章

```
examples/
└── ch01/ : 第 1 章（本节不动）
```

第 2 步 建起第 2 章的项目，先摆四个例子

```
examples/
├── ch01/ : 第 1 章
└── ch02/ : 第 2 章的项目 ← 本课新增
    ├── type_sizes/ ← 本课新增
    ├── char_identity/ ← 本课新增
    ├── float_trap/ ← 本课新增
    └── unsigned_sink/ ← 本课新增
```

第 3 步 再摆四个，八个到齐

```
examples/
└── ch02/
    ├── 前四个例子 : 同上
    ├── format_zoo/ ← 本课新增
    ├── conversion_lab/ ← 本课新增
    ├── literal_zoo/ ← 本课新增
    └── bad_init/ ← 本课新增
```

目录树 **工程二：一个例子目录里面，以及本节留下的会话**

第 1 步 type_sizes 里面：src/ 与 include/ 第一次出现

```
examples/
└── ch02/
    └── type_sizes/
        ├── type_sizes.json ← 本课新增
        ├── src/ ← 本课新增
        └── include/ ← 本课新增
```

第 2 步 src/ 放定义，include/ 放声明

```
examples/
└── ch02/
    └── type_sizes/
        ├── src/
        │   ├── main.c ← 本课新增
        │   └── report.c ← 本课新增
        └── include/
            └── report.h ← 本课新增
```

第 3 步 四段抓下来的警告会话

```
examples/
└── ch02/
    ├── conversion_lab/session-conversion.txt ← 本课新增
    ├── float_trap/session-float-equal.txt ← 本课新增
    ├── unsigned_sink/session-warnings.txt ← 本课新增
    └── literal_zoo/session-vla.txt ← 本课新增
```

第 4 步 外加一个配错格式串的探针

```
examples/
└── ch02/
    ├── format_zoo/wrong_formats.c ← 本课新增
    └── format_zoo/session-wrong-formats.txt ← 本课新增
```

上一张图的第 1 帧是这一节故事的起点，第 2、3 帧是本节搭起来的 `examples/ch02/`：**八个**例子目录（分两帧摆完），每个只负责一件事——量字节、认字符、抓浮点、数回绕、对格式串、看转换、玩字面量、故意编不过。下面那张图的第 1、2 帧是每个例子目录**里面**的样子，也是全书第一次出现 `src/` 与 `include/`：`main.c` 只写「做实验、报结果」，打印这件事单独放进 `report.c`，而 `report.c` 里那几个函数长什么样，写在 `report.h` 里——**声明放头文件，定义放源文件。**它的第 3、4 帧是本节顺手抓下来的五段会话与一个探针，3.5 节到 3.9 节会逐个点到它们。

为什么要分家？标准对这件事**没有任何规定**（它只要求一个程序有一个 `main`）。分家是工程惯例，理由在第 1 单元 1.7 节讲过：编译速度、接缝可见、能被接手。这里只补代价——**改一次头文件，所有包含它的编译单元都要重编**；头文件里多写一行不该写的（比如函数定义），链接时会撞符号。

清单也跟着变了。第 1 章那个清单长这样：

json **第 1 章的清单：一个源文件就够** examples/ch01/hello.json 第 1–9 行

```json
{
  "id": "ch01-hello",
  "lang": "c",
  "sources": ["hello.c"],
  "mode": "run",
  "unit": "chapters/ch01/01-from-source-to-program.md",
  "output": "hello-out",
  "expect": { "exit": 0 }
}
```

这一节的清单多了两样东西：`sources` 里两个源文件、`flags` 里一个 `-Iinclude`：

json **这一节的清单：两个源文件 + 一个 -Iinclude** examples/ch02/type_sizes/type_sizes.json 第 1–10 行

```json
{
  "id": "ch02-type-sizes",
  "lang": "c",
  "sources": ["src/main.c", "src/report.c"],
  "flags": ["-Iinclude"],
  "mode": "run",
  "unit": "chapters/ch02/01-types-and-ranges.md",
  "output": "type-sizes-out",
  "expect": { "exit": 0 }
}
```

`-Iinclude` 是给编译器加一条**找头文件的搜索路径**：`main.c` 里写 `#include "report.h"`，编译器先在 `main.c` 自己所在的 `src/` 里找，找不到再去 `-I` 给出的 `include/` 里找。本机没有 make/cmake，所以一个项目就是一条命令编

完（工作目录是清单所在的那个例子目录）：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/report.c -o app`。

工程惯例 · 工程惯例：头文件里放什么、不放什么

这是**惯例**，不是标准（标准里压根没有「头文件」这个概念，只有「被 `#include` 就地展开的文件」）：

- 放**声明**（函数原型、类型定义、宏）——多个 `.c` 都需要知道「这个名字长什么样」；
- **不**放函数定义——`#include` 是文本复制，同一个定义被复制进两个 `.c`，链接期就是 `multiple definition`；
- 头文件自己包一层 `#ifndef XXX_H / #define XXX_H / #endif`，防止同一个文件被包含两次。

代价是你要多维护一层「谁声明、谁定义」的对应关系；好处是接缝摆在明面上：`report.h` 里那几行，就是这个模块对外承诺的全部内容。

3.2 几个字节：`sizeof` 与 `<limits.h>`

第一个例子 `type_sizes` 只做一件事：把每种类型占几个字节、每种类型的上下界，统统打出来。先看它的头文件——这是全书第一次把声明单独放一个文件：

c examples/ch02/type_sizes/include/report.h 第 1–15 行

```c
#ifndef REPORT_H
#define REPORT_H

/* 打一节标题，单独占一行 */
void head(const char *title);                                   /* 声明，没有函数体 */

/* 在同一行里打一项「名字 值」，不换行；名字前面统一空两格 */
void item_i(const char *name, long long value);                 /* 参数类型写清楚 */
void item_u(const char *name, unsigned long long value);        /* unsigned 的那份另写一个 */
void item_g(const char *name, double value);

/* 结束当前这一行 */
void nl(void);

#endif
```

5 声明，没有函数体
8 参数类型写清楚
9 unsigned 的那份另写一个

这五个声明（`head`、`item_i`、`item_u`、`item_g`、`nl`）里没有一个函数体：它们只是告诉编译器「有这么个函数、参数是什么类型」。真正的函数体在 `report.c` 里，一共 19 行：

c examples/ch02/type_sizes/src/report.c 第 5–23 行

```c
void head(const char *title) {
    printf("== %s ==\n", title);
}

void item_i(const char *name, long long value) {                /* 有符号的那份用 %lld */
    printf("  %s %lld", name, value);
}

void item_u(const char *name, unsigned long long value) {       /* 无符号的那份用 %llu */
    printf("  %s %llu", name, value);
}

void item_g(const char *name, double value) {
    printf("  %s %.17g", name, value);
}

void nl(void) {
    printf("\n");
```

```
23  }
```

9 有符号的那份用 %lld
13 无符号的那份用 %llu

`main.c` 里就只剩「问值、报值」了。它做的事，是把 `sizeof` 与 `<limits.h>` 的宏一项一项喂给 `report`：

c examples/ch02/type_sizes/src/main.c 第 6–23 行

```c
#include "report.h"
int main(void) {
    head("sizeof: 每种类型占几个字节");
    item_u("_Bool", (unsigned long long)sizeof(_Bool));
    item_u("char", (unsigned long long)sizeof(char));
    item_u("short", (unsigned long long)sizeof(short));
    item_u("int", (unsigned long long)sizeof(int));
    item_u("long", (unsigned long long)sizeof(long));
    nl();
    item_u("long long", (unsigned long long)sizeof(long long));
    item_u("float", (unsigned long long)sizeof(float));
    item_u("double", (unsigned long long)sizeof(double));
    item_u("long double", (unsigned long long)sizeof(long double));
    nl();
    item_u("void *", (unsigned long long)sizeof(void *));
    item_u("size_t", (unsigned long long)sizeof(size_t));
    nl();
```

6 双引号：先找本目录再找 -I
13 sizeof(int) 本机是 4
16 long long 是另一种类型

它跑出来的东西，比那 17 行代码值钱得多（这 16 行是这一个例子全部的 stdout）：

终端输出 终端输出 type-sizes-out

```
== sizeof: 每种类型占几个字节 ==
  _Bool 1  char 1  short 2  int 4  long 4
  long long 8  float 4  double 8  long double 16
  void * 8  size_t 8
== <limits.h>: 每个整数类型的边界 ==
  CHAR_BIT 8  CHAR_MIN -128  CHAR_MAX 127  MB_LEN_MAX 5
  SCHAR_MIN -128  SCHAR_MAX 127  UCHAR_MAX 255
  SHRT_MIN -32768  SHRT_MAX 32767  USHRT_MAX 65535
  INT_MIN -2147483648  INT_MAX 2147483647  UINT_MAX 4294967295
  LONG_MIN -2147483648  LONG_MAX 2147483647  ULONG_MAX 4294967295
  LLONG_MIN -9223372036854775808  LLONG_MAX 9223372036854775807
  ULLONG_MAX 18446744073709551615
== <float.h>: 浮点的有效位数与边界 ==
  FLT_DIG 6  DBL_DIG 15  LDBL_DIG 18
  FLT_MIN 1.1754943508222875e-38  FLT_MAX 3.4028234663852886e+38
  DBL_MIN 2.2250738585072014e-308  DBL_MAX 1.7976931348623157e+308
```

先只看第一段表格，这些数字要背下来（背的是本机的，不是永远的）：

- `char` **1** 字节、`short` **2**、`int` **4**、`long` **4**、`long long` **8**；
- `float` **4**、`double` **8**、`long double` **16**（注意：16 是它的存储开销，不是它的精度）；
- `void *` **8**：本机是 64 位系统，一个地址 8 个字节；
- `_Bool` **1**：它是最小的整数类型，只能存 0 与 1。`unsigned char/short/int/long/long long` 不用逐个打——标准要求每个有符号整数类型都有一个**同样宽**的无符号版本（6.2.5 节），所以它们的字节数与上面那一半相同。

`int` 是 4 个字节、`long` **也是 4 个字节**，这一条是本书特意要你记住的：**在别的机器上，`long` 常常是 8。**所以同一份代码里，`int` 顶不顶得住、`long` 够不够长，都是要看平台的。

内存形态 **int / double / char 各占几个盒子（本机实测）**

第 1 步 int 是 4 个盒子

`int i ：本机 4 字节 = 32 位` `i 的第 0 个字节 ：最靠前的那个` `i 的第 3 个字节 ：最后一个`

`i 的第 0 个字节 → i 的第 3 个字节` 四个连成一排才是一个 int

第 2 步 double 是 8 个盒子

`double d ：本机 8 字节 = 64 位` `d 的第 0 个字节 ：第一个` `d 的第 7 个字节 ：最后一个`

`d 的第 0 个字节 → d 的第 7 个字节` 八个连成一排才是一个 double

第 3 步 char 只有 1 个盒子

`char c ：本机 1 字节 = 8 位` `一个字节能装 256 种值 ：CHAR_BIT 就是 8`

第 4 步 盒子数定了，能装的值就有上下界

`int i ：-2147483648 .. 2147483647` `unsigned u ：0 .. 4294967295`

`int i → unsigned u` 一样宽，能表示的值不是一套

第 1、2、3 帧说的是一件很容易被忽略的事：**类型决定你一次搬动几个盒子。** `int` 一次 4 个、`double` 一次 8 个、`char` 一次 1 个。第 4 帧是下一段的引子：盒子的**个数**定了，能表示的值就有上下界，而 `int` 与 `unsigned` 用的是同样多的盒子、能表示的值却不是一套。

边界在哪：问 `<limits.h>`，别背

`INT_MAX` 是多少、`CHAR_BIT` 是几，都不用猜——`<limits.h>` 里每个都有名字。`<limits.h>` 与 `<float.h>` 就是干这个的：它们不提供函数，只提供一堆**宏**，每个宏是一个常量表达式，展开出来就是这台机器的那个数字。

上面那个输出块的 `== <limits.h>` 那一段就是这些边界。三行要连着念（我把它们抄出来，你对着块里的原文看）：

- `CHAR_BIT 8 CHAR_MIN -128 CHAR_MAX 127 MB_LEN_MAX 5`：一个字节 8 位；plain char 的范围是 −128..127，所以本机的 `char` 是**有符号**的——这条 3.4 节还要用；`MB_LEN_MAX` 是「多字节字符最多几个字节」，本机是 5；
- `INT_MIN -2147483648 INT_MAX 2147483647 UINT_MAX 4294967295`：`int` 能表示 $-2^{31}..2^{31}-1$；同样四个字节的 `unsigned` 换成 $0..2^{32}-1$；
- `LONG_MIN -2147483648 LONG_MAX 2147483647 ULONG_MAX 4294967295`：这一行是「`long` 与 `int` 一样宽」的另一个说法——`LONG_MAX` 和 `INT_MAX` 是同一个数（**换台机器通常就不是了**）。

标准原文 · 标准原文：`<limits.h>` 给的是「至少这么大」

> Their implementation-defined values shall be equal or greater in magnitude (absolute value) to those shown, with the same sign.

大意：标准只规定**下界**——比如它要求 `INT_MAX` 至少是 +32767、`INT_MIN` 至多是 −32767，至于本机是 2147483647 还是 9223372036854775807，由实现决定（`implementation-defined`）。所以「`int` 有多大」这个问题，标准给的答案是**范围**，本机给你的才是**数字**。

同一节里 `CHAR_BIT` 的定义是「number of bits for smallest object that is not a bit-field (byte)」，列出的是 **8**；本机实测也是 8，但它属于「实现定义的 8」，不是「永远 8」。

这一节的名字是 5.2.4.2.1 节「Sizes of integer types `<limits.h>`」。

出处：ISO/IEC 9899:2018（C17）§5.2.4.2.1。

实验记录 · 本机实测：这一节的数字全是跑出来的

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32，Windows 10 x64 (MSYS2 UCRT64)；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/report.c -o app`，工作目录是 `examples/ch02/type_sizes/`；
- 结果：编译退出码 **0**，程序退出码 **0**，stdout 共 **16 行**（就是上面那个输出块的全部内容）；
- 关键读数：`sizeof(long)` = **4**、`LONG_MAX` = **2147483647**、`CHAR_BIT` = **8**、`sizeof(void *)` = **8**、`sizeof(long double)` = **16**；
- `long double` 占 16 个字节不代表它有 128 位精度：本机它的有效十进制位数是 `LDBL_DIG` = **18**，比 `double` 的 15 位多不了多少。

这些数字由流水线逐字对账（清单 `examples/ch02/type_sizes/type_sizes.json`，`mode=run`，没有额外的检测参数）：正文里那个 `output` 块与程序真实 stdout 差一个字符都会报 `stale_output`。

实验记录 · （续）本单元用的是哪几种「检测」

本单元**没有用任何检测模式**（`detect`）：八个清单里七个是 `mode=run`、一个是 `mode=compile_fail`。正文里另外那几条诊断，全部来自**显式打开的警告开关**（`-Wfloat-equal`、`-Wconversion -Wsign-compare`、`-Wvla`，以及基础参数就带着的 `-Wtype-limits`）——它们都是真编译的 stderr，会话原文存在各自例子的 `session-*.txt` 里。

陷阱 · `sizeof` 是「字节数」，不是「能装多大的数」

这两个问题看起来像一回事，其实差了八个数量级：`sizeof(int)` 是 **4**，`INT_MAX` 是 **2147483647**。前者的单位是字节，后者是一个数本身。

你怎么会被坑到：写循环算容量时，把 `sizeof(int)` 当成「能装的最大整数」（以为 4 就是上限），于是给 `int` 加个 5 就以为溢出了；或者反过来，以为 `INT_MAX` 是「4 个字节」，写出 `char buf[INT_MAX]` 这种申请 2GB 的声明。

3.3 `<float.h>`：浮点能信几位

`<float.h>` 回答的是另一个问题：**浮点数的精度**。还是刚才那个输出块，最后四行（`== <float.h>：浮点的有效位数与边界 ==` 那一段）：

- `FLT_DIG 6 DBL_DIG 15 LDBL_DIG 18`
- `FLT_MIN 1.1754943508222875e-38 FLT_MAX 3.4028234663852886e+38`
- `DBL_MIN 2.2250738585072014e-308 DBL_MAX 1.7976931348623157e+308`

三个 `_DIG` 宏值得一个一个字读：

- `FLT_DIG` = **6**：`float` 在十进制下能信 **6** 位数字；
- `DBL_DIG` = **15**：`double` 能信 **15** 位；
- `LDBL_DIG` = **18**：本机 `long double` 能信 **18** 位（虽然它占了 16 个字节）。

`FLT_MIN` / `FLT_MAX` 那一对是**取值范围**（不是精度）：本机 `float` 能表示到 `3.4028234663852886e+38`，往下到 `1.1754943508222875e-38`（再小就是非规格化数了）。`%.17g` 打印出来这么长，是因为这个打印精度足以暴露双精度里每一位——3.5 节会看到，正是因为只有 15 位可信，第 16、17 位才那么难看。

标准原文 · 标准原文：`DBL_DIG` 说的是什么

> `<float.h>` 的 `DBL_DIG` 定义是：
>
> > number of decimal digits, q, such that any floating-point number with q decimal digits can be rounded into a floating-point number with p radix b digits and back again without change to the q decimal digits
>
> 大意：`DBL_DIG` 是「十进制下，来回转换不变形的位数」——把一个十进制数读进来再打印回去，前 q 位是可信的，第 q+1 位开始就可能变。标准给出的**下界**是 `DBL_DIG` 至少 **10**，本机给的是 15。
>
> 这一节的名字是 5.2.4.2.2 节「Characteristics of floating types `<float.h>`」。
>
> 出处：ISO/IEC 9899:2018（C17）§5.2.4.2.2。

标准原文 · 标准原文：浮点长什么样，标准只给一个模型

> The characteristics of floating types are defined in terms of a model that describes a representation of floating-point numbers and values that provide information about an implementation's floating-point arithmetic.
>
> An implementation that defines `__STDC_IEC_559__` shall implement floating point types and arithmetic conforming to IEC 60559 as specified in Annex F.
>
> 大意：标准只规定一个**模型**（符号位 `s`、基数 `b`、指数 `e`、尾数），**不要求**浮点一定是 IEEE 754；只有实现自己声明了 `__STDC_IEC_559__`，才必须按 IEC 60559（就是大家说的 IEEE 754）来。
>
> 这一节的名字是 5.2.4.2.2 节「Characteristics of floating types `<float.h>`」。
>
> 出处：ISO/IEC 9899:2018（C17）§5.2.4.2.2 第 1 段。

这段「模型 vs 具体实现」的区分，3.5 节会变成一行真实输出：`0.1 + 0.2` 不等于 `0.3`，原因不在 C 语言这一层，在**用二进制存十进制小数**这件事上。

3.4 char 的三重身份：`'0'`、`0`、`'\0'`

第二个例子 `char_identity` 要拆穿 C 里最容易混的三个「零」。它跑出来是这样：

c examples/ch02/char_identity/src/main.c 第 9–12 行

```
9       printf("== 三个看起来都像「零」的东西 ==\n");
10      printf("'0'   十进制 %3d  十六进制 0x%02X  %s\n", '0', (unsigned)'0', kind_of('0')); 字符零，值是 48
11      printf("0     十进制 %3d  十六进制 0x%02X  %s\n", 0, 0u, kind_of(0)); 数字零，值是 0
12      printf("'\\0'  十进制 %3d  十六进制 0x%02X  %s\n", '\0', 0u, kind_of('\0')); 看不见的那个零，值也是 0
```

```
10 字符零，值是 48
11 数字零，值是 0
12 看不见的那个零，值也是 0
```

终端输出 终端输出 char-identity-out

```
== 三个看起来都像「零」的东西 ==
'0'   十进制  48  十六进制 0x30  可打印
0     十进制   0  十六进制 0x00  不可打印（控制字符）
'\0'  十进制   0  十六进制 0x00  不可打印（控制字符）
== 字符字面量在 C 里的类型是 int ==
sizeof('0') = 4 字节   sizeof((char)'0') = 1 字节
== 字符串 "0" 与 "\0" 各自几个字节 ==
sizeof("0") = 2 字节   sizeof("\0") = 2 字节
== c 里装的是字符 '0'，两种比法结果不同 ==
c = '0'; c == '0' 是 真，c == 0 是 假，c == '\0' 是 假
== 同一个 200，装进三种 char ==
plain char 在本机是 有符号（等同 signed char）
(char)200          = -56
(signed char)200   = -56
(unsigned char)200 = 200
CHAR_BIT = 8，一个字节能装 256 种值
```

三个数的答案，一句话记牢：

- `'0'` **是 48**（十六进制 `0x30`）。它是**字符零**，键盘上那个 `0` 键打出来的东西。所有数字字符都是连着的：`'0'` 是 48，`'1'` 是 49，`'9'` 是 57——所以「字符转数字」才写 `c - '0'`。
- `0` **是 0**。它就是整数零。
- `'\0'` **也是 0**。它是**转义写法**，写的是「那个值为 0 的字节」，也就是字符串的结束标记。

`'0'` 与 `0` 差 48，`'\0'` 与 `0` 一模一样——这就是为什么第 1 单元里 `word[2]` 打出来是 `0`。

同一个例子还回答了另外三个问题，都在输出里：

- **字符字面量的类型是** `int`**，不是** `char`：输出里 `sizeof('0') = 4`，而 `sizeof((char)'0') = 1`。这不是笔误，是标准明文规定的（见下面的标准块）。所以 `'0'` 和 `sizeof` 一撞，结果会是 4。
- `"0"` **与** `"\0"` **都是 2 个字节**：一个是「字符 `0` + 结束标记」，另一个是「值为 0 的字节 + 结束标记」。它俩长得完全不同，`sizeof` 却一样大——看 `sizeof` 是分不出来的，得看内容。
- `c == '0'` **与** `c == 0` **是两件事**：输出里 `c` 里装的是字符 `'0'`，所以 `c == '0'` 是**真**，`c == 0` 是**假**。把这两行连起来读：一个变量里「有个 0 的样子」和「值是 0」完全是两码事。

标准原文 · 标准原文：字符字面量是 int

> An integer character constant has type `int`.
>
> 大意：**单引号包起来的字符常量，类型是** `int` ——注意不是 `char`。（顺带一提，在 C++ 里它是 `char`；这是 C 与 C++ 少数会咬人的差异之一。）
>
> 这一节的名字是 6.4.4.4 节「Character constants」。
>
> 出处：ISO/IEC 9899:2018（C17）§6.4.4.4 第 10 段。

⚠ 陷阱 · `char c = getchar(); if (c == EOF)` 这个坑

`getchar()` 返回的是 `int`（因为要能表示 EOF，EOF 是 −1）。如果你把结果先存进一个 `char`（本机 `char` 有符号，装得下 −1，看起来还对），再和 `EOF` 比，在**另一台** `char` **无符号的机器上**就会永远不相等。

你怎么会被坑到：本机这么写能跑对，你会以为它是对的，然后把这份代码带到别的平台。想知道本机的 `plain char` 到底是有符号还是无符号，别背书，问 `CHAR_MIN`：输出里那行 `(char)200 = -56` 就是答案的一半——`(char)200` 变成了负数，说明 `char` 装不下 200，它是有符号的（本机 `CHAR_MIN = −128`）。

上面 `char_identity` 那个输出块里，`== 同一个 200，装进三种 char ==` 那一段是最值钱的三行：`(char)200 = -56`、`(signed char)200 = -56`、`(unsigned char)200 = 200`。

同一个 200，装进三种 char 得到**三行、两种结果**——本机的 plain `char` 就是 `signed char`，所以它俩一样；只有 `unsigned char` 走另一条路。`char` 装不下 200，于是它变成了 −56——注意这**不是**溢出，这是**转换**，标准允许它这样

（见 3.8 节的标准块）：`char` 是有符号的、只有 256 个格子，200 落在外面，结果由实现决定。本机 gcc 的选择是「把 200 按 8 位补码读回来」。

标准原文 · **标准原文：`char` 到底有没有符号**

> The three types `char`, `signed char`, and `unsigned char` are collectively called the character types. The implementation shall define `char` to have the same range, representation, and behavior as either `signed char` or `unsigned char`.

大意：`char`、`signed char`、`unsigned char` 是三种**不同的类型**；`char`（书里常叫 plain char）具体走哪一路，**由实现决定**——要么跟 `signed char` 一样，要么跟 `unsigned char` 一样，没有第三种可能。

这一节的名字是 6.2.5 节「Types」。

出处：ISO/IEC 9899:2018（C17）§6.2.5 第 15 段。

工程惯例 · **工程惯例：什么时候用哪一种 char**

三种 char 各有岗位，这是**惯例**（标准只说它们是三种类型）：

- `char`：只当**文本**用——存字母、做字符串，从不拿它做算术；
- `signed char` / `unsigned char`：当**小整数**用。特别地，「一个字节的原始数据」（二进制、位运算、`memcpy` 的目标）一律用 `unsigned char`，因为它没有符号位、不会在提升时变成负数（第 4 单元会讲提升）。
- 判「这个字节是不是可打印字符」这类事，先把它转成 `unsigned char` 再传给 `<ctype.h>` 的函数：那些函数的**形参是 `int`**（本机 `ctype.h` 里写的是 `int __cdecl isprint(int _c);`），但要求传进来的值要么是 `EOF`、要么落在 `unsigned char` 的范围内——直接传一个负的 `char` 会越界查表。

代价是你要在「这是文本还是数据」上做一个明确的决定——而这个决定恰好能挡掉后面一大类 bug。

3.5 浮点：`0.1 + 0.2` 为什么不等于 `0.3`

现在到了这一节的主菜。`float_trap` 这个例子只有两件事要做：算一遍 `0.1 + 0.2`，再用两种办法判断它等不等于 `0.3`。

c examples/ch02/float_trap/src/main.c 第 10–15 行

```c
        printf("== 0.1 + 0.2 到底等于多少 ==\n");
        printf("0.1 + 0.2 = %.20f\n", sum); // 20 位小数看个够
        printf("0.3       = %.20f\n", 0.3);
        printf("差多少    = %.20f\n", sum - 0.3);
        printf("=== 比较：sum == 0.3 是 %s\n", exactly_equal(sum, 0.3) ? "真" : "假"); // 逐位比较
        printf("eps 比较：near(sum, 0.3, 1e-9) 是 %s\n", nearly_equal(sum, 0.3, 1e-9) ? "真" : "假"); // eps 比较
```

11 20 位小数看个够
14 逐位比较
15 eps 比较

终端输出 终端输出 float-trap-out

```
== 0.1 + 0.2 到底等于多少 ==
0.1 + 0.2 = 0.30000000000000004441
0.3       = 0.29999999999999998890
差多少    = 0.00000000000000005551
=== 比较：sum == 0.3 是 假
eps 比较：near(sum, 0.3, 1e-9) 是 真
== 同一个 0.1，float 与 double 存的不是一回事 ==
(double)0.1f = 0.10000000149011611938
(double)0.1f == 0.1 是 假
== <float.h> 说了一件更根本的事 ==
FLT_DIG = 6, DBL_DIG = 15：十进制下能信几位数字
(float)INT_MAX = 2147483648：float 装不下它的全部有效位
```

把输出的前三个数抄下来对着看：

- `0.1 + 0.2` = `0.30000000000000004441`
- `0.3` = `0.29999999999999998890`
- 差 = `0.00000000000000005551`

三个数都不是「0.3」，也都不是乱码——它们是**真实存在的双精度数**。写 `0.1` 的时候，编译器要把它塞进 8 个字节（第 2 帧那 8 个盒子），而 `0.1` 在二进制里是**无限循环小数**（就像十进制里 `1/3 = 0.333…` 写不完）。所以存进去的从一开始就是「离 0.1 最近的那个双精度数」，不是 0.1 本身。两个近似值相加，误差累到一起，结果自然不是 0.3。

所以「用 `==` 比较两个浮点数」是错的——不是编译器有 bug，是你问的问题本身没有意义：`sum == 0.3` 问的是「这 64 位全一样吗」，答案是「不」，于是输出里那行是 **假**。

正确的问法是「够不够近」：

c examples/ch02/float_trap/src/approx.c 第 3–13 行

```c
int exactly_equal(double a, double b) {
    return a == b;                    /* 这一行就是 -Wfloat-equal 要报的那一行 */
}

int nearly_equal(double a, double b, double eps) {
    double diff = a - b;
    if (diff < 0) {
        diff = -diff;
    }
    return diff <= eps;
}
```

4 这一行就是那个错的写法
12 靠近到 eps 以内就算相等

第 12 行是 eps 比较的全部内容：`|a - b| <= eps` **就算相等**。eps 取多大是工程决定，不是数学定理：本书例子里取 `1e-9`（十亿分之一），因为这一节的数都是「量级为 1」的数。经验规则是 eps 取「你要的幸福度」——比较 1 米长的东西，`1e-9` 米就是 1 纳米；比较 10 亿，那 `1e-9` 就太严了。

输出里 `eps 比较：near(sum, 0.3, 1e-9) 是 真` ——这就是「够不够近」的答案。

同一个输出里还有一组更吓人的数：`(double)0.1f = 0.100000001490116119 38`。`0.1f` 是 `float`（4 个字节），转成 `double` 之后，你能清楚看到那 8 个字节的真实内容：**它不是 0.1**。所以 `(double)0.1f == 0.1` 也是**假**。

编译器其实会提醒你——只要你开口

`a == b` 这一行，gcc 默认**一声不响**（本书的基础参数 `-Wall -Wextra -Wpedantic` 里没有这一条），你得显式打开 `-Wfloat-equal`：

终端会话 -Wfloat-equal：不打开不报，打开就告诉你 ch02-float-equal

```
$ CFLAGS="-std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude"
$ gcc $CFLAGS src/*.c -o app.exe 2>&1 | grep -c warning:
0
$ gcc $CFLAGS -Wfloat-equal src/*.c -o app.exe
src/approx.c: In function 'exactly_equal':
src/approx.c:4:14: warning: comparing floating-point with '==' or '!=' is unsafe [-Wfloat-equal]
    4 |     return a == b;                    /* 这一行就是 -Wfloat-equal 要报的那一行 */
      |              ^~
$ echo $?
0
```

读这段会话要分三步：第 1 行是把参数放进 `$CFLAGS`，第 2、3 行是「**不加**这个开关时的警告条数」——**0**；第 4 行起的命令加了 `-Wfloat-equal`，第 6 行就是它给的原话：`comparing floating-point with '==' or '!=' is unsafe`，后面那行 `return a == b;` 就是被点名的代码；最后两行是**编译退出码 0**——它只是警告，不拦你。

> ⚠ **陷阱 · 浮点比较这个坑，真正的坑法是「它有时又是对的」**
>
> `0.5 + 0.25 == 0.75` 是真的，`1.0 / 8.0 == 0.125` 也是真的——**只要是 2 的幂组合出来的数，二进制能精确表示**，`==` 就正常工作。
>
> **你怎么会被坑到**：你拿几个「友好」的数试了试，`==` 都是对的，于是放心地把它写进代码；等到某一天数据变成 0.1 那一类，比较开始莫名其妙地失败，你回头怀疑的是「是不是我算错了」。记住这条分界线：**能写成 `/ 2^n` 的十进制小数是精确的，其他的都不是。**想验证，把 `%.20f` 加上去打印——40 毫秒就能看出真相。

`FLT_DIG` / `DBL_DIG` 在这里派上用场：它们的含义就是上一条陷阱的量化版本——`double` 只有 **15** 位十进制数字可信，`%.17g` 那种打印（3.7 节的 `format_zoo` 会用到）已经能看到「不可信」的那些位了。所以本书写代码的规矩是：**要判断浮点相等，写一个 `nearly_equal`；`==` 只准用在「这个值是从哪抄过来的，我一清二楚」的场合。**

输出块最后一行 `(float)INT_MAX = 2147483648` 是同一件事的另一种量法：`INT_MAX` 的 31 位有效数字，`float` 只有 24 位尾数，装不下，于是它被舍到最近的「24 位能表示的数」上。整数能精确表示而浮点不能，这条界线在这里看得最清楚。

3.6 无符号的两个坑：倒序循环与回绕

`unsigned` 不是「只有正数的 int」这么简单，它有自己的算术规则。第四个例子 `unsigned_sink` 把两个最常踩的坑放在一起：

c examples/ch02/unsigned_sink/src/countdown.c 第 6-20 行

```c
void countdown_bad(unsigned n, int show) {
    for (unsigned i = n; i >= 0; i--) {      /* 这个条件永远为真 */ unsigned 永远大于等于 0
        if (show <= 0) {
            return;                          /* 闸门有超时，这里提前收手 */
        }
        printf("  i = %u\n", i);
        show--;
    }
}

void countdown_good(int n) {
    for (int i = n; i >= 0; i--) { 换成 int 就停得下来
        printf("  i = %d\n", i);
    }
}
```

7 unsigned 永远大于等于 0
17 换成 int 就停得下来

终端输出 终端输出 unsigned-sink-out

```
== 倒着数：unsigned 版本（只打 6 行，之后它会一直转下去）==
  i = 3
  i = 2
  i = 1
  i = 0
  i = 4294967295
  i = 4294967294
== 倒着数：signed 版本 ==
  i = 3
  i = 2
  i = 1
  i = 0
== 0 减 1 那一圈 ==
  u - 1 = 4294967295
  (int)(u - 1) = -1
  UINT_MAX = 4294967295，一圈有 4294967296 个值
```

先读上面那个输出块的第一段。`countdown_bad` 打出来的是 `i = 3`、`i = 2`、`i = 1`、`i = 0`、`i = 4294967295`、`i = 4294967294`——**它在 0 那里没有停。**走到 0 之后再 `i--`，`i` 变成了 4294967295（`UINT_MAX`），而不是 −1。这就是第一个坑：`i >= 0` 对 `unsigned` 来说**永远为真**，循环条件恒成立，真实的程序在这里会一直转下去（本书的闸

门有 15 秒超时，所以 `countdown_bad` 多带了一个 `show` 参数：打满 6 行就 `return`，让你看见它**不在 0 处停**）。同一个输出块的第二段是 `int` 版本，它正常停在 0。

第二个坑是「减出来一个巨大的数」：

- `u - 1` = **4294967295**（`u` 是 `unsigned` 0）
- `(int)(u - 1)` = **−1**

第一行是真·无符号算术：**无符号的运算结果按 2^{32} 取模**，−1 落在圈上就是 4294967295。第二行是「把它当 int 读出来」的结果，那是**实现定义**的（下一节讲）。

图 / 搜索过程 **unsigned 的四字节一圈：0 减 1 落到哪**

第 1 步 一圈上的三个点

`0：最小值` `1：0 加 1 到这里` `4294967295：一圈的最后一个值（UINT_MAX）`

`0` → `1` 加 1

第 2 步 0 减 1 是往回退一格，越过 0 就绕到另一头

`0：现在要算 u - 1` `4294967295：落点在这里`

`0` → `4294967295` 减 1 回绕

第 3 步 所以倒着数的循环停不下来

`0：i 走到这里，看起来该结束了` `4294967295：再减一绕到这里` `4294967294：接着是它`

`i 大于等于 0：这三个点上都是真`

`0` → `4294967295` i--

`4294967295` → `4294967294` 再减一

第 1 帧是这一圈的全貌：`unsigned` 的四个字节能表示 2^{32} 个值，0 和 4294967295 是**相邻**的。第 2 帧是 `u - 1` 的落点——不是 −1，是 4294967295。第 3 帧解释了为什么 `i >= 0` 在这圈上处处为真：这一圈上**没有负数**，所以「大于等于 0」是对所有值都成立的废话。

标准原文 · **标准原文：无符号运算不会溢出，只会绕圈**

> A computation involving unsigned operands can never overflow, because a result that cannot be represented by the resulting unsigned integer type is reduced modulo the number that is one greater than the largest value that can be represented by the resulting type.

大意：无符号运算**不会溢出**（也不会是未定义行为）——超出范围的结果按「最大值加一」取模，也就是**在圈上绕**。四个字节的 `unsigned` 最大值加一是 2^{32}，所以 0 − 1 得到 $2^{32} - 1$ = 4294967295。这是标准**规定**的行为，不是本机的怪癖：换台机器、换个编译器，这个数还是 4294967295（只要 `unsigned` 是 32 位）。

出处：ISO/IEC 9899:2018（C17）§6.2.5 第 9 段。

编译器视角 · **编译器视角：`i >= 0` 恒真这件事，它知道**

不用等程序跑起来，编译器在编译期就看出来了。同样是那一条编译命令，只是把里面的文件换成 `unsigned_sink`：

终端会话 -Wtype-limits：恒真的比较，它直接点出来 ch02-unsigned-warn

```
$ CFLAGS="-std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude"
$ gcc $CFLAGS src/*.c -o app.exe
src/countdown.c: In function 'countdown_bad':
src/countdown.c:7:28: warning: comparison of unsigned expression in '>= 0' is always true [-Wtype-limits]
    7 |     for (unsigned i = n; i >= 0; i--) {      /* 这个条件永远为真 */
      |                            ^~
```

编译器点的是 `src/countdown.c` 第 7 行的 `i >= 0`，原话是 `comparison of unsigned expression in '>= 0' is always true [-Wtype-limits]`——`-Wextra` 就打开着这条。注意它只给**警告**：这一行不是错的，是**能编译能跑、跑出来不对**的那种错；所以本书的规矩是「警告不当噪音，要一条条看」（第 1 单元 1.5 节讲过 `error` / `warning` / `note` 三级）。

> ⚠ **陷阱** · `for (unsigned i = n - 1; i >= 0; i--)` 这一行会咬人
>
> 它不是「多转一圈」，它是**永远不结束**：如果循环体里有 `printf` 或写文件，你会得到一个几十 GB 的输出文件；如果循环体里有数组下标，你会读到 `a[4294967295]`（越界读，按标准是未定义行为——这两句是**推断**，本书没有为它们单独做例子）。
>
> **你怎么会被坑到**：编译器给的 `-Wtype-limits` 警告会被你当成「风格建议」忽略掉——毕竟程序确实编译通过了。写倒序循环时，正确的写法有两条：**用 `int`**（`for (int i = (int)n - 1; i >= 0; i--)`），或者**用 `i > 0` 加一次单独处理 0**。本书推荐第一条：下标本来就是 `int` 的活。

3.7 格式串与类型必须匹配

`printf` 的格式串与实参之间，**运行期没有任何检查**：`%d` 说「从这个位置上取走一个 `int`」，`printf` 就真的去取一个 `int`。你递上去的要是别的东西，取出来的就是别的东西。`format_zoo` 把同一个值换成各种格式串打了一遍：

c examples/ch02/format_zoo/src/views.c 第 24–41 行

```c
void show_double(const char *label, double value) {
    printf("  double %s: %%f %f   %%e %e   %%g %g   %%.17g %.17g\n", 同一个值，四种浮点格式
           label, value, value, value, value);
}

void show_correct_formats(void) {
    size_t z = sizeof(int);
    long long ll = 1234567890123LL;
    uint32_t u32 = 4294967295u;
    int32_t i32 = INT32_MAX;
    int64_t i64 = INT64_MAX;

    printf("  size_t    -> %%zu    %zu\n", z);
    printf("  long long -> %%lld   %lld\n", ll);
    printf("  uint32_t  -> PRIu32  %" PRIu32 "   PRIx32  %" PRIx32 "\n", u32, u32); PRIu32 是个宏，展开成格式串
    printf("  int32_t   -> PRId32  %" PRId32 " (INT32_MAX) \n", i32);
    printf("  int64_t   -> PRId64  %" PRId64 " (INT64_MAX) \n", i64);
}
```

25 同一个值，四种浮点格式

38 PRIu32 是个宏，展开成格式串

终端输出 终端输出 format-zoo-out

```
== 同一个 int，换着格式串看 ==
  int 42: %d 42   %x 2a   %X 2A   %o 52   %c *
  int -1: %d -1   %x ffffffff   %X FFFFFFFF   %o 37777777777   %c（这个值打印出来是看不见的字节，这里不打）
== 同一个 unsigned，换着格式串看 ==
  unsigned 42: %u 42   %x 2a   %o 52
== 同一个 double，换着格式串看 ==
  double 1.0/3: %f 0.333333   %e 3.333333e-01   %g 0.333333   %.17g 0.33333333333333331
  double 0.1: %f 0.100000   %e 1.000000e-01   %g 0.1   %.17g 0.10000000000000001
== 每种整数类型配哪个格式串 ==
  size_t    -> %zu    4
  long long -> %lld   1234567890123
  uint32_t  -> PRIu32  4294967295   PRIx32  ffffffff
  int32_t   -> PRId32  2147483647 (INT32_MAX)
  int64_t   -> PRId64  9223372036854775807 (INT64_MAX)
```

一行一行对着看，规律很硬：

- **整数按有无符号分家**：`int` 用 `%d`，`unsigned` 用 `%u`。同一个 42，`%d` 与 `%u` 打出来一样；同一个 −1，`%d` 打出来是 `-1`，而 `%x` 打出来是 `ffffffff`——那正是 4294967295 的十六进制写法；
- **同一个整数可以换进制看**：`%d` 十进制、`%x` 小写十六进制、`%X` 大写、`%o` 八进制。输出里 `int -1` 的十六进制是 `ffffffff`、八进制是 `37777777777`——这就是 3.6 节那一圈在**十六进制**下的样子。注意 `show_int` 里对 `%x/%X/%o` 传的是 `(unsigned)value`：**这三个格式串要的是 `unsigned`**，把 `int` 直接递上去就是下面标准块说的那种未定义行为（本机这样写也能打出同样的数，但那是 ABI 的运气，不是保证）；

- **浮点有四种常用长相**：`%f`（定点，6 位小数）、`%e`（科学计数法）、`%g`（自动挑前两种里短的那个）、`%.17g`（把双精度里每一位都摊开）。同一个 `1.0/3`：`%f` 是 `0.333333`，`%e` 是 `3.333333e-01`，`%.17g` 是 `0.33333333333333331`——最后这个尾巴就是 3.5 节说的「不可信的那些位」；
- **`size_t` 用 `%zu`、`long long` 用 `%lld`、定宽类型用 `PRI` 宏**：输出的最后五行就是这五条。`size_t` 是 `sizeof` 的结果类型，也是 `strlen` 的返回类型——这两个地方的返回值用它准没错。

{} 语法规则 · 格式串的配对规则

一个格式串里的每个 `%…` 都对应后面一个实参，**类型必须对得上**：整数的有符号与无符号要分家（`%d` / `%u`），长度要匹配（`%zu` 给 `size_t`、`%lld` 给 `long long`），浮点用 `%f` / `%e` / `%g` 三兄弟。配错了标准不要求编译器报错（见下面的标准块），所以这条规矩**只能靠你自己守**。

一份「谁配谁」的对照表：

值的类型	格式串	说明
`int`	`%d`（或 `%i`）	十进制，有符号
`unsigned`	`%u`	十进制，无符号
`unsigned`（`int` 要先转过去）	`%x` / `%X` / `%o`	十六进制小写 / 大写 / 八进制
`double`	`%f` / `%e` / `%g`	定点 / 科学 / 自动
`size_t`	`%zu`	`sizeof` 与 `strlen` 的结果
`long long`	`%lld`	八字节整数
`uint32_t` 这类定宽类型	`PRIu32` / `PRId64` / `PRIx32`	宏，定义在 `<inttypes.h>`

最后一行要单独说一句：`uint32_t` 在有的平台是 `unsigned int`、在别的平台可能是 `unsigned long`。所以你**不能写死** `%u` 或 `%lu`，得写 `PRIu32`——它是个**宏**，展开出来正好是这台机器该用的那个格式串。

`<stdint.h>` 给的正是这种「宽度写在名字里」的类型：`int8_t` / `int16_t` / `int32_t` / `int64_t` 与它们的 `uint` 版本，外加 `INT32_MAX`、`INT64_MAX` 这类边界宏。输出最后两行就是实测：`int32_t` 的上界是 **2147483647**，`int64_t` 是 **9223372036854775807**，各自用 `PRId32` / `PRId64` 打出来。它们的**含义**是「恰好这么多位」，而 `int` 只保证「至少 16 位」——这就是为什么文件格式、协议、二进制数据要指定宽类型：**跨机器时你要的是那个位数，不是那台机器顺手给的那个宽度。**

标准原文 · 标准原文：`PRIu32` 这类宏是什么

> Each of the following object-like macros expands to a character string literal containing a conversion specifier, possibly modified by a length modifier, suitable for use within the format argument of a formatted input/output function when converting the corresponding integer type.

大意：`PRI` 打头的这些宏展开出来**是一段格式串**（是字符串字面量），专门给对应的整数类型用。所以写法是 `printf("%" PRIu32 "\n", v)`——三段文字在预处理期拼成一个格式串。

这一节的名字是 7.8.1 节「Macros for format specifiers」。

出处：ISO/IEC 9899:2018（C17）§7.8.1 第 1 段。

标准原文 · 标准原文：格式串配错了会怎样

> If any argument is not the correct type for the corresponding conversion specification, the behavior is undefined.

大意：**实参类型与转换说明不匹配，就是未定义行为。**注意它说的是 `undefined behavior`，不是「报错」也不是「打出来难看」——编译器可以什么都不说，程序可以照跑，打出来的东西可以是任何样子。

出处：ISO/IEC 9899:2018（C17）§7.21.6.1 第 9 段。

「可以是任何样子」这句不是吓唬人。`format_zoo` 这个目录里还留了一个探针 `wrong_formats.c`，故意配错两处：

探针：两处故意配错的格式串 examples/ch02/format_zoo/wrong_formats.c 第 5–10 行

```c
int main(void) {
    printf("%f\n", 1);              /* %f 配 int：它会按 double 去读 */
    printf("%d\n", sizeof(int));    /* %d 配 size_t：低 32 位刚好是 4 */
    fflush(stdout);
    return 0;
}
```

它不在任何清单里（没有对应的 `.json`），编它、跑它的结果是这样：

终端会话 配错格式串：gcc 在编译期就报了一条 ch02-wrong-formats-diag

```
$ CFLAGS="-std=c17 -Wall -Wextra -Wpedantic -O0 -g"
$ gcc $CFLAGS wrong_formats.c -o wrong_formats.exe
wrong_formats.c: In function 'main':
wrong_formats.c:6:14: warning: format '%f' expects argument of type 'double', but argument 2 has type 'int'
[-Wformat=]
    6 |     printf("%f\n", 1);              /* %f 配 int：它会按 double 去读 */
      |             ~^     ~
      |              |     |
      |              |     int
      |              double
      |             %d
```

第二条是同一次编译里的下一个警告：

终端会话 （续）第二条：%d 配 size_t ch02-wrong-formats-diag2

```
wrong_formats.c:7:14: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long long
unsigned int' [-Wformat=]
    7 |     printf("%d\n", sizeof(int));    /* %d 配 size_t：低 32 位刚好是 4 */
      |             ~^     ~~~~~~~~~~~
      |              |     |
      |              int   long long unsigned int
      |             %lld
```

终端会话 （续）跑起来：一个荒唐，一个「看着还挺对」 ch02-wrong-formats-run

```
$ ./wrong_formats.exe
0.000000
4
$ echo $?
0
```

最后一块里那两行输出，是这一节最值得盯着看的证据：

- `0.000000` —— `%f` 配了一个 `int`，`printf` 按双精度去读，读出来的东西跟 1 没半点关系。**荒唐得一眼可见**；
- `4` —— `%d` 配的是 `sizeof(int)`（类型是 `size_t`，本机 8 字节），`printf` 只取走低 32 位，而低 32 位正好是 4。**看着完全正常。**

这就是格式串不匹配最讨厌的地方：**你没法靠「跑一遍看看」发现它。**上面两条警告（`-Wformat=` 是 `-Wall` 打开的）是你**唯一能拿到手的信号**；但它只在格式串是**字符串字面量**的时候才报得出来——格式串由运行期拼出来时，这条信号也不存在。标准也不要求编译器做这个检查。

3.8 隐式转换的四处在哪

C 里的类型转换有两种：一种是你亲手写的（`(char)200`），另一种是**不用你写、编译器自己做的**，叫隐式转换。它最常藏身的四个地方，全在 `conversion_lab` 这一个例子里。先看头文件——前四条声明，每条注释就是一处（后两条是另外两种相关的转换，3.8 节后半段会用到）：

c examples/ch02/conversion_lab/include/narrow.h 第 4–20 行

```c
/* 1. 赋值：int 直接赋给 char（值超出范围时结果由实现决定） */ 第 1 处：赋值
char narrow_char_assign(int value);

/* 2. 返回值：return 也是一次赋值 */ 第 2 处：返回值
char narrow_char_return(int value);

/* 3. 函数参数：把 int 传给要 char 的函数，再把收到的那个 char 还回去 */ 第 3 处：实参
char narrow_pass_char(int value);

/* 4. 算术表达式：double 与 int 混用，结果被截断成 int */ 第 4 处：算术表达式
int narrow_int(double value);

/* int 赋给 unsigned：负数按一圈回绕 */
unsigned narrow_unsigned(int value);

/* 有符号与无符号直接比较：int 先被转成 unsigned */
int compare_int_unsigned(int a, unsigned b);
```

4 第 1 处：赋值
7 第 2 处：返回值
10 第 3 处：实参
13 第 4 处：算术表达式

对应的函数体。前三处在一个文件里排着，全是「`int` 进 `char` 出」：

c examples/ch02/conversion_lab/src/narrow.c 第 3–18 行

```c
char narrow_char_assign(int value) {
    char c = value;                 /* 隐式转换第 1 处：赋值 */ 赋值这里窄化
    return c;
}

char narrow_char_return(int value) {
    return value;                   /* 隐式转换第 2 处：返回值 */ return 也是一次赋值
}

static char pass_through(char c) {
    return c;
}

char narrow_pass_char(int value) {
    return pass_through(value);     /* 隐式转换第 3 处：实参传给形参 */ 实参传给形参
}
```

4 赋值这里窄化
9 return 也是一次赋值
17 实参传给形参

下面这个文件里还有三处，性质跟前三处都不同：第 4 处是 `double` → `int`（**丢掉小数**），另外两处是 `int` → `unsigned`（**把负数变成巨大的正数**）与有符号/无符号比较（**比较前先转换**）：

c examples/ch02/conversion_lab/src/narrow.c 第 20–30 行

```c
int narrow_int(double value) {
    return value;                   /* 隐式转换第 4 处：表达式里的 double 转 int */ 向零截断
}

unsigned narrow_unsigned(int value) {
    return value;                   /* int 装进 unsigned */ int 装进 unsigned
}
```

```
27
28  int compare_int_unsigned(int a, unsigned b) {
29      return a < b;                   /* 比较之前，a 先被转成 unsigned */ 比较之前先转 unsigned
30  }
```

21 向零截断
25 int 装进 unsigned
29 比较之前先转 unsigned

这些转换跑出来的结果：

终端输出 终端输出 conversion-lab-out

```
== int 塞进 char（本机 char 的范围 -128..127）==
  int   65 -> char   65
  int  127 -> char  127
  int  128 -> char -128
  int  200 -> char  -56
  int  300 -> char   44
  int   -1 -> char   -1
  返回值那条路 (char)200 -> -56；实参那条路也是 -56
== double 塞进 int：小数被丢掉，向零截断 ==
         3.900 -> 3
        -3.900 -> -3
         0.500 -> 0
    123456.789 -> 123456
== int 塞进 unsigned，以及 int 与 unsigned 相遇 ==
  -1 -> 4294967295
  -1 < 1u 是 假
```

读输出按三段来：

- `int` → `char`**（第一段）**：本机 `char` 的范围是 −128..127。65、127 原样过去；**128 变成 −128、200 变成 −56、300 变成 44。**这三个数是**本机 gcc 的选择**：它把值截掉高位、留下低八位、再按有符号读回来——200 的低八位是 `11001000`，当有符号读就是 −56；300 的低八位是 `00101100`，读出来是 44。**这不是 UB，但也不是标准规定的「截断规则」**：标准把这种情况整个交给实现（见下面的标准块），换一个编译器完全可能给出别的值。
- `double` → `int`**（第二段）**：`3.9 → 3`、`−3.9 → −3`——**向零截断**，不是四舍五入，也不是向下取整（−3.9 的两个「整数邻居」是 −3 和 −4，截断给 −3）。
- `int` → `unsigned` **与混合比较（第三段）**：`−1 → 4294967295` 就是 3.6 节那一圈；`−1 < 1u` 判出来是**假**——因为 `int` 与 `unsigned` 相遇时，`int` 先被转成 `unsigned`，−1 变成 4294967295，当然不小于 1。

标准原文 · 标准原文：`int` 装进 `char`，装不下时怎么办

> Otherwise, the new type is signed and the value cannot be represented in it; either the result is implementation-defined or an implementation-defined signal is raised.

大意：往**有符号**类型里塞一个装不下的值，结果是**实现定义**的（或者由实现决定要不要抛个信号）——标准允许 gcc 给出 −56，也允许别的实现给出 0、给出 127，甚至报错。所以「200 装进 `char` 会变成 −56」这句话，正确的说法是「**本机**是这样」；想写可移植的代码，就别依赖这个值。

出处：ISO/IEC 9899:2018（C17）§6.3.1.3 第 3 段。

标准原文 · 标准原文：`double` 装进 `int` 是另一回事

> When a finite value of real floating type is converted to an integer type other than `_Bool`, the fractional part is discarded (i.e., the value is truncated toward zero). If the value of the integral part cannot be represented by the integer type, the behavior is undefined.

大意：浮点转整数**规定**丢掉小数部分（向零截断）——这一半是确定的；但如果连整数部分都装不下（比如把 `1e10` 塞进 `int`），那就是**未定义行为**，不是「绕一圈」。

出处：ISO/IEC 9899:2018（C17）§6.3.1.4 第 1 段。

上面这两条合起来，正好回答本节反复出现的那个问题：**值不对的时候，先分清是哪一种。**

- **又窄又装不下**：`int` → `char` 越界是**实现定义**（6.3.1.3 节，本机 −56、别的编译器可以是别的值，不可移植）；`double` → `int` 丢小数那半是**标准规定**的向零截断，但整数部分装不下就升级成 **UB**（6.3.1.4 节）；
- **溢出**（下一单元）：`INT_MAX + 1`、`1e10` 塞进 `int`——这类是**未定义行为**，编译器有自由，下一单元（`chapters/ch02/02-integer-promotion-and-overflow.md`）整节讲它。

所以「值不对」要分三层看：**标准规定的**（无符号回绕、向零截断）、**实现定义的**（有符号类型装不下、`char` 有没有符号）、**未定义的**（有符号溢出、浮点转整数越界）。越往下越不能靠猜。

编译器不报，但加一个开关就报

上面那几处隐式转换，本书的基础参数 `-Wall -Wextra -Wpedantic` **只抓到一处**（第 29 行那个有符号与无符号的比较）。要抓全，得加两个开关：

终端会话 `-Wconversion -Wsign-compare`：从 1 条警告变成 6 条　　ch02-conversion-warn

```
$ CFLAGS="-std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude"
$ gcc $CFLAGS src/*.c -o app.exe 2>&1 | grep -c warning:
1
$ gcc $CFLAGS -Wconversion -Wsign-compare src/*.c -o app.exe 2>&1 | grep -c warning:
6
$ gcc $CFLAGS -Wconversion -Wsign-compare src/*.c -o app.exe 2>&1 | grep warning:
src/narrow.c:4:14: warning: conversion from 'int' to 'char' may change value [-Wconversion]
src/narrow.c:9:12: warning: conversion from 'int' to 'char' may change value [-Wconversion]
src/narrow.c:17:25: warning: conversion from 'int' to 'char' may change value [-Wconversion]
src/narrow.c:21:12: warning: conversion from 'double' to 'int' may change value [-Wfloat-conversion]
src/narrow.c:25:12: warning: conversion to 'unsigned int' from 'int' may change the sign of the result [-
Wsign-conversion]
src/narrow.c:29:14: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned
int' [-Wsign-compare]
```

读这段会话（命令里带了 `| grep`，是为了省掉每条警告的插入符图、只看结论行；原文一个字符没改）：

- 第 1 行把编译参数放进 `$CFLAGS`，后面几条命令都用它（本机 bash 的变量展开）；
- 第 2、3 行：**基础参数**下，这个例子只有 **1** 条警告；
- 第 4、5 行：加上 `-Wconversion -Wsign-compare` 之后变成 **6** 条——**新增 5 条**；
- 第 6 行起是这 6 条的原话。新增的 5 条分别是：三处「`conversion from 'int' to 'char' may change value`」（第 4、9、17 行，也就是赋值、返回值、实参）、一处「`conversion from 'double' to 'int' may change value`」（第 21 行，它带的是 `-Wfloat-conversion` 标签）、一处「`conversion to 'unsigned int' from 'int' may change the sign of the result [-Wsign-conversion]`」（第 25 行）。

所以 `-Wconversion` 报的是「**这次转换可能改变值**」，`-Wsign-conversion`（它包含在 `-Wconversion` 里）报的是「**符号可能变**」，`-Wsign-compare` 报的是「**两个不同符号的数在比较**」。三条合起来，才能把「值悄悄变了」这类问题在编译期拦住——代价是这类警告在真实工程里会很多，本书的做法是：写新代码时开着它们。

编译器视角 · 编译器视角：`-Wconversion` 只看类型，不看值

它的边界很明确：**运行期的值它一概不知道**，只看两边的类型；但**编译期就能算出范围的常量表达式，它算得出来，所以不报。** `char c = 300 % 256;` 就是后者——结果是 44，落在 `char` 的范围里，编译器看得出来。

反过来，`char c = f();`（`f` 返回 `int`，但你心里清楚它永远返回 ≤ 127）它**一定**会报——机器不知道你的保证。这就是「警告要人看」的原因：它不是判决，是提醒。

一个真的编不过的例子

隐式转换里有一条是**硬错误**：整数不会自己变成指针。第八个例子 `bad_init` 就干这一件事：

c　**想给指针塞一个整数：这一行编译器直接拒绝**　examples/ch02/bad_init/src/bad_init.c　　第 1–6 行

```c
#include "bad_init.h"

char *bad_pointer(void) {
    char *p = 300;                  /* 这一行就编不过：整数不会自己变成指针 */
    return p;
```

```
    }
```

它的真诊断（原文照抄，连插入符的位置都不动）：

编译诊断 gcc 的真实诊断（原文照抄） bad-init-diag

```
src/bad_init.c: In function 'bad_pointer':
src/bad_init.c:4:15: error: initialization of 'char *' from 'int' makes pointer from integer without a cast
[-Wint-conversion]
    4 |     char *p = 300;                  /* 这一行就编不过：整数不会自己变成指针 */
      |               ^~~
```

注意这条诊断给的三样东西：**文件名与行列**（`src/bad_init.c:4:15`）、**它认为你在干什么**（`initialization of 'char *' from 'int'`）、**它看到的缺失**（`makes pointer from integer without a cast`：你少写了一个强转）。注意那句不是「修法」：写成 `char *p = (char *)300;` 确实能让它闭嘴（本机实测编译通过、退出码 0），但那条指针仍然是一个值为 300 的非法地址——**那只是把诊断藏起来，不是修好**。真正的修法只有一个：别把整数当指针；确实要在整数和指针之间转，用 `<stdint.h>` 的 `uintptr_t` 并说清场景。这条错误是本书**故意**制造的：清单里写的是 `"mode": "compile_fail"`，它必须真的编不过，编过了反而要报错（`expected_compile_failure`）。

顺带记住一件事：**这条诊断在本机这套 gcc（16.1.0）里是 `error`，不是 `warning`**。有些老书和旧编译器里它只是警告，程序能编出来、跑起来拿到一个非法地址——那是更糟的处境。

3.9 字面量、`sizeof` 与常量

第七个例子 `literal_zoo` 只干一件事：把「同一个数的不同写法」摆出来看。先说**进制**——C 认三种写法：

c examples/ch02/literal_zoo/src/main.c 第 8–14 行

```
8       printf("== 同一个 31 的三种进制写法 ==\n");
9       printf("  31 = %d, 0x1F = %d, 037 = %d, 三个都是同一个值\n", 31, 0x1F, 037); 037 是八进制
10
11      printf("== 八进制看着像十进制：多一个 0 就不是那个数了 ==\n");
12      int x = 010; 010 等于 8
13      printf("  int x = 010; 之后 x = %d（不是 10）\n", x);
```

9 037 是八进制
12 010 等于 8

紧接着是**后缀**那一组（同一个文件接着往下 7 行）：

c **（续）后缀决定类型** examples/ch02/literal_zoo/src/main.c 第 15–21 行

```
15      printf("== 后缀决定类型：1 是 int，1LL 才是 long long ==\n");
16      literal_row("1", 1, (unsigned long long)sizeof(1));
17      literal_row("1u", 1u, (unsigned long long)sizeof(1u));
18      literal_row("1L", 1L, (unsigned long long)sizeof(1L));
19      literal_row("1LL", 1LL, (unsigned long long)sizeof(1LL));
20      double_row("1.0f", 1.0f, (unsigned long long)sizeof(1.0f));
21      double_row("1.0", 1.0, (unsigned long long)sizeof(1.0));
```

终端输出 终端输出 literal-zoo-out

```
== 同一个 31 的三种进制写法 ==
  31 = 31, 0x1F = 31, 037 = 31, 三个都是同一个值
== 八进制看着像十进制：多一个 0 就不是那个数了 ==
  int x = 010; 之后 x = 8 (不是 10)
== 后缀决定类型：1 是 int, 1LL 才是 long long ==
  1         值 1        sizeof 4 字节
  1u        值 1        sizeof 4 字节
  1L        值 1        sizeof 4 字节
  1LL       值 1        sizeof 8 字节
  1.0f      值 1        sizeof 4 字节
  1.0       值 1        sizeof 8 字节
== 1u - 2：无符号的那一圈 ==
  1u - 2 = 4294967295 (不是 -1)
== sizeof 不求值：括号里那件事不会发生 ==
  sizeof(i++) = 4, 执行完这一行 i 还是 0
== 数组长度：三种写法 ==
  sizeof(a) 20   sizeof(b) 24   sizeof(c) 28
```

输出的前两段说明了两件事：`31 = 31, 0x1F = 31, 037 = 31` 是同一个值；`int x = 010;` 之后 `x` 是 **8**。三条规则：

- **`0x` 开头是十六进制**（`0x1F` = 31）；
- **`0` 开头是八进制**——所以 `010` 是 **8**，不是 10。这是 C 里最经典的安静陷阱之一：`int mode = 010;` 看起来是「十号模式」，实际是「八号」；
- 其余是十进制。

再往后是**后缀**，它决定这个字面量的**类型**（不只是值）：`1` 是 `int`（4 字节）、`1u` 是 `unsigned`（4 字节）、`1L` 是 `long`（本机也是 **4** 字节，因为它跟 `int` 一样宽）、`1LL` 是 `long long`（**8** 字节）、`1.0f` 是 `float`（**4** 字节）、`1.0` 是 `double`（**8** 字节）。**后缀不是可有可无的装饰**：`1u` 与 `1` 在算术里会走出不同的结果，这就是下一个代码块里 `1u - 2 = 4294967295` 的原因——`1u` 是无符号的，减出来按 3.6 节那一圈走；写成 `1 - 2` 就老老实实是 −1。

第二块讲 `sizeof` 的一个反直觉性质：

c examples/ch02/literal_zoo/src/main.c 第 23–28 行

```
23      printf("== 1u - 2：无符号的那一圈 ==\n");
24      printf("  1u - 2 = %u (不是 -1) \n", 1u - 2);  无符号的那一圈
25
26      printf("== sizeof 不求值：括号里那件事不会发生 ==\n");
27      size_t n = sizeof(i++);  sizeof 只问类型，括号里不执行
28      printf("  sizeof(i++) = %zu, 执行完这一行 i 还是 %d\n", n, i);
```

24 无符号的那一圈
27 sizeof 只问类型，括号里不执行

输出里那行是 `sizeof(i++) = 4, 执行完这一行 i 还是 0`。**`i++` 写了，但没发生**——因为 `sizeof` 只需要知道「这个表达式的类型是什么」，不需要算出它的值。这是 `sizeof` 与函数调用最大的区别：它是**编译期**的东西。唯一的例外是变长数组——`sizeof(VLA)` 的操作数会被求值、结果也不是常量，3.9 节会看到本机的那个 VLA。

标准原文 · **标准原文：`sizeof` 求什么、返回什么**

> The `sizeof` operator yields the size (in bytes) of its operand, which may be an expression or the parenthesized name of a type. The size is determined from the type of the operand. The result is an integer. If the type of the operand is a variable length array type, the operand is evaluated; otherwise, the operand is not evaluated and the result is an integer constant.
>
> The value of the result of both operators is implementation-defined, and its type (an unsigned integer type) is `size_t`, defined in `<stddef.h>` (and other headers).

大意：两句话，一句管「求不求值」，一句管「结果是什么类型」。

- **除了变长数组那种情况，操作数不被求值**——所以 `sizeof(i++)` 不会让 `i` 变；
- 结果的**值由实现定义**（这就是「本机 `sizeof(int)` 是 4，别的机器可能是 2」的标准出处），**类型是 `size_t`**（一个无符号整数类型，`%zu` 就是给它用的）。

这一节的名字是 6.5.3.4 节「The `sizeof` and `_Alignof` operators」。

出处：ISO/IEC 9899:2018（C17）§6.5.3.4 第 2、5 段。

⚠ 陷阱 · **「字节数是实现事实」这句话，会以两种方式咬你（一）**

第一种：**跨平台时数字变**。本机 `sizeof(long)` 是 4，Linux 64 位上是 8。你用 `long` 当「64 位整数」写代码，换台机器就溢出或者截断。要固定宽度，用 `<stdint.h>` 的 `int32_t` / `int64_t`。

⚠ 陷阱 · **（续）它会咬你的第二种方式**

第二种：**同一台机器上也变——`sizeof` 给的是「几个字节」，不是「能装多大的数」**。`sizeof(int) == sizeof(float)`（都是 4），但 `int` 装不下 `3.5`，`float` 也装不准 `INT_MAX`：`float_trap` 的输出里有一行 `(float)INT_MAX = 2147483648`（float 的尾数只有 24 位，装不下 31 位里的每一位）。别把两个 4 当成「可以互相替代」。

你怎么会被坑到：写文件读写时用 `fwrite(&n, sizeof(int), 1, f)`，看起来「平台无关」，实际上文件的字节序与宽度两台机器可以不同——那种场景该用定宽类型加显式的字节序转换。

定长数组的长度必须是常量表达式——`const` 不够

C 里有个规则很容易被误解：**定长数组的长度必须是「整数常量表达式」**（长度不是常量的那种叫变长数组，是另一回事）。三种写法里，只有前两种算。`literal_zoo` 里有第三个源文件专门量这件事：

`c` examples/ch02/literal_zoo/src/array_len.c 第 5–14 行

```c
const int LEN_BY_CONST = 7;

void array_len_demo(void) {
    int a[LEN_BY_DEFINE];          /* 宏：预处理期就变成 5，是常量表达式 */ 宏，预处理期就变成 5
    int b[LEN_BY_ENUM];            /* 枚举常量：也是常量表达式 */ 枚举常量也是常量表达式
    int c[LEN_BY_CONST];           /* 只读变量：它不是常量表达式，这里是变长数组 */ 这个是变长数组

    printf("  sizeof(a) %zu   sizeof(b) %zu   sizeof(c) %zu\n",
           sizeof(a), sizeof(b), sizeof(c));
}
```

8 宏，预处理期就变成 5
9 枚举常量也是常量表达式
10 这个是变长数组

（`LEN_BY_DEFINE` 是 `describe.h` 里的宏 `5`，`LEN_BY_ENUM` 是枚举常量 `6`，`LEN_BY_CONST` 是 `const int` 且等于 7。）这三行的命运，输出块的最后一行写得很清楚：`sizeof(a) 20 sizeof(b) 24 sizeof(c) 28`——都是「元素数 × 4」。这里有个细节值得停一下：`a`、`b` 的 `sizeof` 是编译期算出来的常量，而 `c` 那个是**运行期**算出来的——因为它的大小要到运行期才知道。这就是 3.2 节那句「`sizeof` 不求值」的例外。注意：**`-Wall -Wextra -Wpedantic` 下第三行一条警告都没有**，程序照跑。但只要多给一个 `-Wvla`，它立刻露馅：

终端会话 `-Wvla：第三行是变长数组，不是常量长度` `ch02-vla`

```
$ CFLAGS="-std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude"
$ gcc $CFLAGS -Wvla src/*.c -o app.exe
src/array_len.c: In function 'array_len_demo':
src/array_len.c:10:5: warning: ISO C90 forbids variable length array 'c' [-Wvla]
   10 |     int c[LEN_BY_CONST];            /* 只读变量：它不是常量表达式，这里是变长数组 */
      |     ^~~
$ echo $?
0
```

会话里那行诊断的原文是 `warning: ISO C90 forbids variable length array 'c' [-Wvla]`，指向 `src/array_len.c` 第 10 行；最后是**编译退出码 0**——又是「能编能跑、但意思和你想的不一样」。

`const int` 是「只读变量」，不是「常量」——这是 C 与 C++ 的一个重要差异：在 C++ 里上面第三行是合法的常量数组，在 C 里它是变长数组。原因在标准的一句话里：

标准原文 · **标准原文：什么叫「整数常量表达式」**

> An integer constant expression shall have integer type and shall only have operands that are integer constants, enumeration constants, character constants, `sizeof` expressions whose results are integer constants, `_Alignof` expressions, and floating constants that are the immediate operands of casts.

大意：能进「整数常量表达式」这个俱乐部的，只有**整数常量、枚举常量、字符常量、`sizeof` 的结果、`_Alignof` 的结果**这几样。**变量不在名单里**——哪怕它被 `const` 修饰、哪怕它明明等于 7。`const` 管的是「不准通过这个名字改它」，那是运行期的性质。

注意这条只约束**定长数组**：长度不是整数常量表达式时，那个数组就是变长数组（VLA），标准对它有另一套规定（6.7.6.2 节），`sizeof` 也改在运行期求值。

这一节的名字是 6.6 节「Constant expressions」。

出处：ISO/IEC 9899:2018（C17）§6.6 第 6 段。

变长数组本书不教，这是一个明确的取舍：它从 C99 起进入标准，但**从 C11 起变成「实现可选」**：实现可以定义 `__STDC_NO_VLA__` 表示不支持（MSVC 就不支持，那份代码在它那里编不过）。而且在栈上按运行期的大小开数组，很容易在输入变大时把栈撑爆（`int a[n]`，`n` 来自用户输入——这一条是**推断**，本书没有为栈溢出单独做例子）。要动态大小，用 `malloc`（第 12 单元）——**显式的堆分配能报错，隐式的栈增长只能崩。**

工程惯例 · **工程惯例：`#define` / `enum` / `const` 怎么选**

标准对这三样**没有任何偏好**，怎么选是惯例问题：

- `enum`：一组相关的整数常量，且它们**要一起出现在类型系统里**（状态码、模式）。优点：是常量表达式，能当数组长度、能进 `case`，调试器里还能显示名字；
- `#define`：需要「还没到语法层就得是常量」的场合（`#if` 里判断、条件编译），或者要给字符串拼格式串（`PRIu32` 就是）。缺点：它是文本替换，没有类型，出错时报错信息很难看；
- `const`：真正的「只读对象」——指针目标、函数参数、`const char *` 这类接口约定。**别拿它当常量表达式用**（原因见上）。

工程惯例 · **（续）这个选择的代价**

代价是你要在动手前想一下「这个东西是常量还是只读对象」——这个问题想清楚了，第 16 单元讲宏的时候会轻松很多。

工程惯例 · **工程惯例：整数类型怎么选**

一个能省掉大量 bug 的经验顺序（**惯例**，标准不管这些）：

1. **默认用 `int`**。算术、下标、循环计数，`int` 够用就用 `int`——它是这台机器最自然的宽度；
2. **需要「不比 `int` 小」时用 `long long`**，本机的 `long` 与 `int` 一样宽，别指望 `long` 更宽；
3. **需要确定的宽度就用 `<stdint.h>` 的定宽类型**（`int32_t` / `uint64_t`）：文件格式、协议、二进制数据、哈希值——这些地方「恰好 32 位」是需求的一部分。打印它们用 `PRIx32` / `PRId64` 这些宏；
4. **只在确实需要回绕语义时用 `unsigned`**（哈希、位运算、掩码）。「负数不可能出现」**不是**用 `unsigned` 的理由——它换来的是 3.6 节那两个坑；
5. **不碰 `short` 与 `char` 做算术**：它们会先被提升成 `int`，中间那一堆转换是下一单元的主题。

代价：定宽类型在接口上会逼你写 `#include <stdint.h>` 与 `PRI` 宏，代码看着比 `int` 啰嗦——但换来的可移植性是真实的。

3.10 这一节新增了什么，以及接下来

回到 3.1 那棵目录树，现在你可以对着**第 2、3、4 帧**逐个点名了。

第 2 帧的八个例子目录，各自负责一件事：

- `type_sizes/` —— `sizeof` 与 `<limits.h>` / `<float.h>` 的全部读数（3.2、3.3 节）；
- `char_identity/` —— `'0'` / `0` / `'\0'` 与三种 `char`（3.4 节）；
- `float_trap/` —— `0.1 + 0.2`、`==` 与 eps（3.5 节）；
- `unsigned_sink/` —— 恒真的倒序循环与 0 减 1 的回绕（3.6 节）；
- `format_zoo/` —— 同一组值的各种格式串，外加配错的探针（3.7 节）；
- `conversion_lab/` —— 隐式转换的四处藏身处（3.8 节）；
- `literal_zoo/` —— 进制、后缀、`sizeof(i++)`（3.9 节）；
- `bad_init/` —— 唯一一个**编不过**的例子，`char *p = 300;`（3.8 节）。

下面那张图的第 2 帧那套 `src/` 与 `include/` 的结构，第一次把「声明放头文件、定义放源文件」摆到了明面上：`report.h`、`charid.h`、`approx.h`、`countdown.h`、`narrow.h`、`describe.h`、`views.h`、`bad_init.h` 八个头文件里放的都是**对外的约定**——函数声明（`describe.h` 还多一组宏、一个枚举与一个 `extern` 声明）；对应的那些 `.c` 是**只有定义**（`literal_zoo` 还多一个 `array_len.c`，专管数组长度那一件事）；每个目录的 `main.c` 只做实验与调用。一个项目一条命令编完：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/report.c -o app`。

下面那张图的第 3、4 帧的五段会话与那个探针，是这一节所有「编译器说的话」的来源：四段真诊断里有三段靠**额外打开**的开关（`-Wfloat-equal`、`-Wconversion -Wsign-compare`、`-Wvla`），另一段（`-Wtype-limits`）是基础参数里的 `-Wextra` 本来就带着的；最后是一个配错格式串的探针。它们都由流水线逐字对账。

下一单元是同一章的第二个单元 `chapters/ch02/02-integer-promotion-and-overflow.md`：这一节反复出现的两个悬念——**整数提升**（`char` 参与算术时先变成 `int`，`-Wconversion` 为什么报「从 `int` 到 `char`」而不是反过来）与**溢出**（`INT_MAX + 1` 到底是 UB 还是有定义）——在那里一次讲完。本机 `long` 与 `int` 一样宽这件事，到那时会变成一个具体的问题：想装下两个 `int` 的乘积，你得用 `long long`。

小结 · 这一节的要点

- **字节数是实现事实**：本机 `_Bool` 1、`char` 1、`short` 2、`int` 4、`long` **4**、`long long` 8、`float` 4、`double` 8、`long double` 16、`void *` 8；标准只规定下界（5.2.4.2.1 节），`sizeof` 的结果值本身「由实现定义」（6.5.3.4 节）——只有 `char` 的 1 个字节是标准保证的。
- **三个零不是一回事**：`'0'` 是 48、`0` 是 0、`'\0'` 也是 0；字符字面量是 `int`（`sizeof('0')` 为 4，6.4.4.4 节）；plain `char` 是否有符号由实现决定（本机有符号）。
- **值不对有三种来源**：浮点不精确（`0.1 + 0.2` 是 `0.30000000000000004441`，用 eps 而不是 `==`）、无符号回绕（`0u - 1` 是 4294967295，6.2.5 节）、实现定义的窄化（`(char)200` 本机是 −56，6.3.1.3 节——**不是 UB，但也不是标准规定的确定值**）。
- **格式串配错是 UB**（7.21.6.1 节）且可能「看着还挺对」；隐式转换藏在赋值、返回值、实参、算术表达式四处；`sizeof` 不求值、结果是 `size_t`，定长数组的长度要整数常量表达式（`const int` 不算，变长数组是例外）。

自测题 · 自测题（先自己答，再对答案）

1. 本机 `sizeof(int)`、`sizeof(long)`、`sizeof(double)` 各是几？这些数字里，哪一个才是标准保证不动的？
2. `'0'`、`0`、`'\0'` 的值分别是多少？`sizeof('0')` 是几？为什么不是 1？
3. 为什么 `0.1 + 0.2 == 0.3` 是假？想判断「够不够近」，代码该怎么写、eps 取多少合适？
4. `for (unsigned i = n - 1; i >= 0; i--)` 有什么问题？给出两种能正常停下的改法。
5. 下面两行，哪一行会被 `-Wconversion` 报出来，哪一行不会？为什么？
   - `char c = 300 % 256;`
   - `char d = f();`（`int f(void);`）

注 · 自测题答案

1. **4 / 4 / 8。**这三个都是**实现事实**：标准只规定「至少」多大（`INT_MAX` 至少 +32767），本机 `long` 与 `int` 一样宽、Linux 64 位上 `long` 是 8 个字节，三者都没有标准保证的固定值。标准唯一钉死的是 **`sizeof(char)` 恒为 1**——「一个字节」的定义就是「一个 `char` 的大小」，所以 `sizeof(char)` 在任何实现上都是 1，同时 `CHAR_BIT` 是那个字节有几位（本机 8，标准只保证至少 8）。
2. **48 / 0 / 0。**`sizeof('0')` 是 **4**，因为**字符字面量的类型是 `int`**（6.4.4.4 节），不是 `char`；想要 1 个字节，得写 `sizeof((char)'0')`——本机实测这一行是 1。
3. 因为 `0.1`、`0.2`、`0.3` 在二进制里都存不准（都是无限循环小数），存进去的是「最近的那个双精度数」，相加之后再和 0.3 的近似值比，64 位当然对不上（本机实测 `0.1 + 0.2` = `0.30000000000000004441`，与 `0.3` 差 `0.00000000000000005551`）。正确写法是 `nearly_equal(a, b, eps)`：先算 `|a − b|`，再和 eps 比。eps 的大小取决于你的数据量级——本机例子里比较的都是量级为 1 的数，取 `1e-9`；这不是定理，是工程决定。

注 · （续）答案 4、5

4. 它**永远不结束**：`unsigned` 减到 0 之后再减会绕到 4294967295（6.2.5 节规定按 2^{32} 取模），而 `i >= 0` 对 `unsigned` 恒为真。两种改法：① 把循环变量换成 `int`（`for (int i = (int)n - 1; i >= 0; i--)`）；② 条件改成 `i > 0`，循环体里单独处理 0 那一项。
5. 第二行会被报：`warning: conversion from 'int' to 'char' may change value [-Wconversion]`。第一行不会——`300 % 256` 是常量表达式，编译器在编译期就算出结果是 44，落在 `char` 的范围内，它**算得出来**所以不报。`-Wconversion` 看不见的是**运行期的值**：`f()` 返回什么它不知道，「你心里知道它永远 ≤ 127」这种保证它也不认。

整数的边界：提升、溢出、位运算与枚举

◎ 学习目标 · 看完这一单元，你应该能

- 说清 `~(unsigned char)0` 为什么是 -1、`-1 > 0u` 为什么为真，以及编译器各给什么警告；
- 说出有符号溢出与无符号回绕在标准里的两种待遇，并用真跑出来的退出码描述未定义行为；
- 用 `-ftrapv` / `-fwrapv` 钉住溢出行为，说清 `INT_MIN` 上那三个坑为什么都算未定义行为；
- 分清 `&` / `&&`、`|` / `||`、`~` / `!`，会用 `1u << n` 做掩码，写出不触发未定义行为的溢出检查；
- 用枚举做状态与数组下标，说清枚举常量到底是什么类型、位域为什么不可移植。

2.1 先看工程长什么样

上一单元把 `examples/ch02/` 变成了**一个例子一个小工程**：每个例子一个目录，`src/` 放源文件、`include/` 放头文件。这一单元不新增目录类型——还是 `src/ + include/`——只是往里加例子，并且第一次加**抓下来的会话文件**与**汇编产物**。

目录树 **examples/ch02/：这一单元里它长成了什么样**

第 1 步 上一单元留下的样子（每个例子一个小工程）

```
examples/ch02/
└── type_sizes/
    ├── include/report.h
    ├── src/main.c
    ├── src/report.c
    └── type_sizes.json
```

第 2 步 这一单元新增 12 个例子目录 + 3 个会话文件

```
examples/ch02/
├── type_sizes/ ：上一单元
├── promotion_lab/ mixed_sign/ overflow_u0/ overflow_check/ ← 本课新增
├── int_min_trap/ twos_complement/ enum_demo/ shift_guards/ ← 本课新增
├── bitops/ bitfield_pack/ overflow_literal/ overflow_loop/ ← 本课新增
└── session-opt-diff.txt session-analyzer.txt session-warnings.txt ← 本课新增
```

第 3 步 拆开 overflow_loop/：清单有三个，还有汇编产物

```
examples/ch02/overflow_loop/
├── include/overflow_loop.h ：只有声明
├── src/main.c ：只负责调用
├── src/overflow_loop.c ：循环在这里
├── overflow_loop.json + 两个检测清单
└── overflow_loop-O0.s +overflow_loop-O2.s ：汇编对照 ← 本课新增
```

第 1 帧是上一单元结束时的形态：一个例子一个目录，目录里是头文件、两个源文件、一个清单。第 2 帧是这一单元新增的内容：**十二个**例子目录（`promotion_lab`、`mixed_sign`、`overflow_u0`、`overflow_check`、`int_min_trap`、`twos_complement`、`enum_demo`、`shift_guards`、`bitops`、`bitfield_pack`、`overflow_literal`、`overflow_loop`），外加三个会话文件；为了不把帧撑得太高，每行的 `+` 只标在行首，那一行整行都是新的。第 3 帧拆开最小的那个例子：`overflow_loop.h` 里只有**声明**，`overflow_loop.c` 里是**定义**，`main.c` 只负责调用；同一个目录里可以放**多个清单**，还能放**汇编产物**（`.s` 不是编译输入，是拿来讲道理的证物）。

工程惯例 · **工程惯例：为什么第二个项目起必须分文件**

标准只要求一个程序有一个 `main`，分文件是本书的硬规定、也是主流做法：**声明放头文件、定义放源文件**，`main.c` 用 `#include "overflow_u0.h"` 把声明拿进来。好处是**接缝可见**——谁用哪个函数、参数长什么样，读头文件就够。

代价有两条：**头文件一改，所有包含它的编译单元都得重编**（本书一条 gcc 命令编完，就是整条重跑）；声明与定义对不上是**编译错误**（本机实测 `error: conflicting types for 'f'`），而「声明了却没有定义」才会拖到链接期（本机 ld 退出码 1）。

闸门把这条写成了判据：第 2 章起，清单的 `sources` 里 `.c` 少于两个、或者没有 `.h` 被 `#include "…"` 进来，就报 `single_file_project_forbidden`。

2.2 比 int 窄的整数，先变成 int

第一个反直觉的东西：把一个 `unsigned char` 取反，结果**不是**一个无符号数，而是一个负的 `int`。

c examples/ch02/promotion_lab/src/promotion_lab.c 第 5–17 行

```c
void promotion_lab_report(void) {
    unsigned char uc = 0;
    uint8_t u8 = 255;
    short sh = -1;

    printf("~(unsigned char)0        = %d\n", ~uc); 无符号数取反，居然是负数
    printf("sizeof(~(unsigned char)0) = %zu\n", sizeof(~uc));
    printf("(uint8_t)255 + 1         = %d\n", u8 + 1); 255 加 1 没有回绕成 0
    printf("sizeof((uint8_t)255 + 1) = %zu\n", sizeof(u8 + 1));
    printf("short(-1) == -1          = %d\n", sh == -1);
    printf("sizeof(char)=%zu sizeof(short)=%zu sizeof(int)=%zu\n",
           sizeof(char), sizeof(short), sizeof(int));
}
```

10 无符号数取反，居然是负数
12 255 加 1 没有回绕成 0

终端输出 终端输出 promotion-lab-out

```
~(unsigned char)0        = -1
sizeof(~(unsigned char)0) = 4
(uint8_t)255 + 1         = 256
sizeof((uint8_t)255 + 1) = 4
short(-1) == -1          = 1
sizeof(char)=1 sizeof(short)=2 sizeof(int)=4
```

四个数字要连起来读。第 10 行 `~uc` 先把 `uc`（一个 8 位的 0）**提成 `int` 的 0**，再按 `int` 的 32 位取反，得到 `11111111 11111111 11111111 11111111`，也就是 -1；所以第 11 行 `sizeof(~uc)` 报的是 **4**——如果结果和 `uc` 一样宽，这里该是 1。第 12 行更能说明问题：`uint8_t` 最大值是 255，加 1 若按 8 位算就该回绕成 0，实测却是 **256**，因为这次加法在 `int` 里做。第 14 行的 `sh == -1` 是同一道理的另一面：`short` 的 -1 提成 `int` 的 -1，再比较。

大白话 · **用大白话说「整数提升」**

把整数想成一排大小不同的盒子：`char`、`short` 是小盒子，`int` 是大盒子。

- C 在做算术之前，会先把小盒子里的值**原样搬进 `int` 这个大盒子**，再开始算；
- 搬的过程不丢值（小盒子能装的，大盒子都装得下），但**装进去之后就是 `int` 了**——原来那个「无符号」的身份不再参与运算；
- 所以 `~uc` 是在 32 位里取反的，得到 -1；`u8 + 1` 是在 32 位里加的，得到 256。

严谨的说法：这叫**整数提升**（integer promotions），标准规定比 `int` 窄的类型先转成 `int`（装不下才转 `unsigned int`）。它是**自动的、隐式的**，你什么都没写，它就在那里。

标准原文 · **标准原文：整数提升**

6.3.1.1 节「Boolean, characters, and integers」：

> If an `int` can represent all values of the original type (as restricted by the width, for a bit-field), the value is converted to an `int`; otherwise, it is converted to an `unsigned int`. These are called the integer promotions.

大意：只要 `int` 装得下原类型的全部取值（位域按它的位宽看），就转成 `int`，否则转成 `unsigned int`。**注意括号里那半句**：位域也要做提升，2.14 节会用到这个伏笔。

出处：ISO/IEC 9899:2018（C17）§6.3.1.1 第 2 段。

标准原文 · **标准原文（续）：寻常算术转换**

`int` 与 `unsigned int` 相遇时的规矩，在 6.3.1.8 节「Usual arithmetic conversions」：

> … if the operand that has unsigned integer type has rank greater or equal to the rank of the type of the other operand, then the operand with signed integer type is converted to the type of the operand with unsigned integer type.

大意：无符号那个操作数的**等级**不低于有符号的那个时，有符号的被转成无符号——**负号到这里就没了**。这就是下一节 `-1 > 0u` 的全部原因。

出处：ISO/IEC 9899:2018（C17）§6.3.1.8 第 1 段。

2.3 `-1 > 0u` 为什么是真的

上一条规矩最出名的后果，是这段程序里的前四行：

c examples/ch02/mixed_sign/src/mixed_sign.c 第 4–12 行

```c
void mixed_sign_compare(void) {
    int a = -1;
    unsigned int b = 0u;

    printf("-1 > 0u             = %d\n", a > b); -1 变成 4294967295 才去比
    printf("(unsigned)-1        = %u\n", (unsigned)a);
    printf("-1 == (unsigned)-1 = %d\n", a == (unsigned)a);
    printf("sizeof(a + b)       = %zu\n", sizeof(a + b));
}
```

8 -1 变成 4294967295 才去比

后四行是另一件事——同一个数值用不同格式串打印：

c （续）格式串与类型 examples/ch02/mixed_sign/src/mixed_sign.c 第 14–20 行

```c
void mixed_sign_format(void) {
    unsigned int big = 4294967295u;

    printf("同一个值，%%d 打出来是 %d\n", big);
    printf("同一个值，%%u 打出来是 %u\n", big);
    printf("(int)4294967295u    = %d\n", (int)big);
}
```

终端输出 终端输出 mixed-sign-out

```
-1 > 0u             = 1
(unsigned)-1        = 4294967295
-1 == (unsigned)-1 = 1
sizeof(a + b)       = 4
同一个值，%d 打出来是 -1
同一个值，%u 打出来是 4294967295
(int)4294967295u    = -1
```

第 8 行按 6.3.1.8 节先转再比：`a` 从 -1 转成 `4294967295u`，于是 `4294967295 > 0` 成立，打出来是 **1**。第 10 行的 `a == (unsigned)a` 同样是 **1**——**它永远是真的**，因为右边那个强转就是你亲手写出来的同一个转换。编译器会给你一条 `-Wsign-compare` 警告（见下面第一段会话），但它说的是「两个操作数符号不同」，读起来像噪音，很多人扫一眼就过去了。

格式这一处更糟：同一个值 `4294967295u`，`%d` 打出来是 `-1`，`%u` 打出来才是 `4294967295`。

终端会话 默认参数下就会报的 `-Wsign-compare` ch02-warn-signcompare

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -c mixed_sign/src/mixed_sign.c -Imixed_sign/include -o
/dev/null
mixed_sign/src/mixed_sign.c: In function 'mixed_sign_compare':
mixed_sign/src/mixed_sign.c:8:43: warning: comparison of integer expressions of different signedness: 'int'
and 'unsigned int' [-Wsign-compare]
    8 |     printf("-1 > 0u            = %d\n", a > b);
      |                                          ^
mixed_sign/src/mixed_sign.c:10:43: warning: comparison of integer expressions of different signedness: 'int'
and 'unsigned int' [-Wsign-compare]
   10 |     printf("-1 == (unsigned)-1 = %d\n", a == (unsigned)a);
      |                                          ^~
```

格式串那一处，本书固定的 `-Wall -Wextra -Wpedantic` **抓不到**——`%d` 与 `unsigned int` 在 gcc 眼里是「同宽的有符号 / 无符号」。要显式加上 `-Wformat-signedness` 再编一次，它才说话：

终端会话 加 `-Wformat-signedness` 之后才出现的诊断 ch02-warn-format

```
mixed_sign/src/mixed_sign.c: In function 'mixed_sign_format':
mixed_sign/src/mixed_sign.c:17:37: warning: format '%d' expects argument of type 'int', but argument 2 has
type 'unsigned int' [-Wformat=]
   17 |     printf("同一个值，%%d 打出来是 %d\n", big);
      |                                    ~^     ~~~
      |                                     |     |
      |                                     int   unsigned int
      |                                    %d
```

⚠ 陷阱 · 混合比较与格式串：警告有，但它救不了你的语义

你会看到什么：`-1 > 0u` 打印出 `1`；`printf("%d", 4294967295u)` 打印出 `-1`。比较那一处有 `-Wsign-compare`，格式那一处默认**一句话都没有**。

你会怎么误判：看到 `comparison of integer expressions of different signedness` 以为「编译器在唠叨风格问题」，加个 `(unsigned)` 强转就把它压掉了——可你压掉的正是这条警告想让你想一想的那个问题。

真相：`-Wsign-compare` 只告诉你「两边符号不同」，不告诉你「这次转换会不会改变结果」；`-Wformat` 只管宽度，符号要 `-Wformat-signedness` 才管。两处都得靠你**读懂**，不是靠编译器闭嘴。防法只有两条：有符号与无符号混在一个表达式里时，先问「负值会变成什么」；打印定宽类型时格式串去查表（`%zu`、`%" PRIu32 "`），别凭手感写 `%d`。

2.4 有符号溢出是未定义行为，无符号回绕是模运算

这两件事长得几乎一样，标准给它们的待遇却完全相反。先看标准怎么规定：

标准原文 · 标准原文：有符号溢出

> 6.5 节「Expressions」第 5 段：
>
> > If an exceptional condition occurs during the evaluation of an expression (that is, if the result is not mathematically defined or not in the range of representable values for its type), the behavior is undefined.
>
> 大意：求值过程中出现了异常情况（数学上没有定义，或者结果超出这个类型的可表示范围），行为未定义。
>
> 出处：ISO/IEC 9899:2018（C17）§6.5 第 5 段。

标准原文 · **标准原文（续）：什么叫「未定义行为」**

3.4.3 节第 1 段：

> behavior, upon use of a nonportable or erroneous program construct or of erroneous data, for which this International Standard imposes no requirements

大意：对于不可移植或错误的程序构造、或者错误的数据，标准**不提任何要求**——注意是「不提要求」，不是「规定它会崩」。这句话是本节所有「本机实测」的注脚：下面每一条退出码，都只是**这台机器这一次**的选择。

出处：ISO/IEC 9899:2018（C17）§3.4.3 第 1 段。

标准原文 · **标准原文（续）：无符号永远不会溢出**

6.2.5 节「Types」第 9 段：

> A computation involving unsigned operands can never overflow, because a result that cannot be represented by the resulting unsigned integer type is reduced modulo the number that is one greater than the largest value that can be represented by the resulting type.

大意：无符号参与的计算**永远不会溢出**——装不下的结果按「模 2 的 N 次方」折回来（`unsigned int` 就是模 2^{32}）。这是**有定义**的行为，任何编译器都必须这么算。

出处：ISO/IEC 9899:2018（C17）§6.2.5 第 9 段。

现在看真跑出来的现象：

c examples/ch02/overflow_u0/src/overflow_u0.c 第 5–11 行

```c
int overflow_u0_plus_one(int n) {
    return n + 1;              /* n == INT_MAX 时，这一行就是有符号溢出 */ 这一行就是有符号溢出
}

unsigned int overflow_u0_round_down(unsigned int n) {
    return n - 1;              /* n == 0u 时，按模 2^32 回绕，有定义 */ 无符号减法按模 2 的 32 次方
}
```

6 这一行就是有符号溢出
10 无符号减法按模 2 的 32 次方

c **（续）把值当参数送进去，编译器就折不了** examples/ch02/overflow_u0/src/overflow_u0.c 第 13–20 行

```c
void overflow_u0_report(void) {
    int imax = INT_MAX;
    unsigned int zero = 0u;

    printf("INT_MAX      = %d\n", imax);
    printf("INT_MAX + 1  = %d\n", overflow_u0_plus_one(imax));
    printf("n + 1 < n 能查出溢出吗 = %d\n", overflow_u0_check(imax));
    printf("0u - 1       = %u\n", overflow_u0_round_down(zero));
```

c **（续）剩下的两行统计** examples/ch02/overflow_u0/src/overflow_u0.c 第 21–24 行

```c
    printf("UINT_MAX     = %u\n", UINT_MAX);
    printf("sizeof(int)=%zu sizeof(unsigned)=%zu\n", sizeof(int), sizeof(unsigned int));
    return;
}
```

终端输出 终端输出 overflow-u0-out

```
INT_MAX      = 2147483647
INT_MAX + 1  = -2147483648
n + 1 < n 能查出溢出吗 = 0
0u - 1       = 4294967295
UINT_MAX     = 4294967295
sizeof(int)=4 sizeof(unsigned)=4
```

第 18 行传进去的是 `INT_MAX`，函数里做的是 `n + 1`：这是**运行期**的溢出（不是常量表达式），所以编译时**一点警告都没有**——这段程序编译退出码 0、stderr 一个字都没有。它打出来 `-2147483648`，也就是 `INT_MIN`。这件事**不是标准保证的**：标准说它是未定义行为，本机这次恰好给了「回绕」这个结果，换个编译器、换个优化等级可能完全不同（2.5 节就去看那个不同）。第 20 行的 `0u - 1` 则相反：**标准保证**它按模 2^{32} 折回来，任何机器上都是 `4294967295`。

第 19 行是本节埋的一个钉子：`n + 1 < n` 这种「溢出检查」，计算结果打出来是 **0**——即使传进去的正是 `INT_MAX`。它不是这次算错了，是**这个检查从原理上就不可能成立**：编译器按 6.5 第 5 段认定 `n + 1` 不会溢出，于是 `n + 1 < n` 永远为假。2.5 节会把它的汇编拿出来看。

实验记录 · **本机实测：这一节的四个数字**

- 工具链：gcc 16.1.0，`-std=c17`（C17），目标三元组 x86_64-w64-mingw32，固定参数 `-Wall -Wextra -Wpedantic -O0 -g`；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/overflow_u0.c -o app`（工作目录 `examples/ch02/overflow_u0/`）；
- 结果：编译退出码 **0**、stderr 为空；`INT_MAX + 1` 打出 **-2147483648**，`0u - 1` 打出 **4294967295**，`n + 1 < n` 打出 **0**，退出码 0；
- `sizeof(int)` = **4**、`sizeof(unsigned int)` = **4**，所以两者的模都是 2^{32}。

上面的输出由流水线逐字对账，对不上就报 `stale_output`；这一节的清单 id 是 `ch02-overflow-u0`。

工程惯例 · **工程惯例：无符号回绕当成「别人给你的确定性」**

标准对无符号的规定是**确定性**的（模 2^N），所以有两类写法在工程里被广泛接受：

- **散列、校验和、CRC、随机数发生器**：就是要它回绕，靠的是「模 2^{32}」这条保证；
- **位掩码与协议字段**：`0xFFFFFFFFu` 这类值是「位」而不是「数」，回绕是它的语义。

而**依赖有符号溢出**在工程里一律算 bug，哪怕某台机器上「看起来正常」。代价说清楚：写无符号算术时你得自己盯着「减法会不会把小的减成大的」——那类 bug 编译器不会提醒你。

为什么标准要把有符号溢出的行为「留空」

这不是标准偷懒，是一笔交换。把有符号溢出的行为空着，编译器才敢做下面这些推理：

- `i + 1 > i` **恒成立**——于是循环要跑多少次可以提前算出来，数组边界检查能消除掉，归纳变量的乘法能换成加法。这类推理（归纳变量分析）是循环优化的主力；
- `a + b` **不会回绕**——于是「这两条分支加起来覆盖了所有情况」这句话可以放心拿去化简。

如果反过来把回绕规定成「必须发生」（也就是 `-fwrapv` 那种语义），上面两条推理立刻作废，编译器就得为每个有符号加法准备「万一回绕了怎么办」——这正是 2.6 节说 `-fwrapv` 「要放弃一部分优化」的原因。

标准的选择是把正确性的责任交给写代码的人、把优化空间留给编译器。所以工程上的主流态度是「不制造溢出」，而不是「依赖溢出时会怎么样」。

2.5 未定义行为之下，编译器有多大自由

同一个函数，只改优化等级。程序是这一段：

c examples/ch02/overflow_loop/src/overflow_loop.c 第 5–12 行

```c
void overflow_loop_run(void) {
    int i;

    for (i = INT_MAX - 5; i > 0; i += 2) {
        /* 空循环体：循环要停下来，只能靠 i 溢出成负数 */
    }
    printf("循环结束时 i = %d\n", i);
}
```

8 循环要停下来，只能靠 i 溢出

终端输出 终端输出 overflow-loop-out

```
循环结束时 i = -2147483648
```

这个循环写得很老实：`i` 从 `INT_MAX - 5` 开始，每次加 2，加到溢出为止。标准说这次溢出是未定义行为，于是**编译器的想法和你不一样。**先看 `-O0`：

终端会话 -O0：循环真的跑完 ch02-session-o0

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/overflow_loop.c -o loop-O0.exe
$ ./loop-O0.exe
循环结束时 i = -2147483648
$ echo $?
0
```

再换成 `-O2`（会话里那句 bash 提示带进程号，抓的时候丢掉了，只留退出码）：

终端会话 （续）-O2：进程被非法指令干掉 ch02-session-o2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/overflow_loop.c -o loop-O2.exe
src/overflow_loop.c: In function 'overflow_loop_run':
src/overflow_loop.c:8:36: warning: iteration 2 invokes undefined behavior [-Waggressive-loop-optimizations]
    8 |     for (i = INT_MAX - 5; i > 0; i += 2) {
      |                                  ~~^~~~
src/overflow_loop.c:8:29: note: within this loop
    8 |     for (i = INT_MAX - 5; i > 0; i += 2) {
      |                           ~~^~~
$ ( ./loop-O2.exe ) 2>/dev/null
$ echo $?
132
```

会话里三件事值得停下来看：

- `-O2` 编译时 gcc 直接告诉你 `iteration 2 invokes undefined behavior`——它在编译期就算清楚了第几次迭代会踩到溢出；
- **那个 `2` 是从 0 数起的**：`i` 依次取 `INT_MAX-5`、`INT_MAX-3`、`INT_MAX-1`，然后溢出——溢出发生在第三次迭代，gcc 叫它 `iteration 2`。这类编号差一位的细节，读诊断时要留意；
- `-O0` 那次退出码 **0**、打印了 `i = -2147483648`；`-O2` 那次退出码 **132**，也就是 bash 说的 `128 + 4`（信号 4 = SIGILL，非法指令）。

为什么 -O2 会产生一条非法指令？ 把汇编拿出来看就明白了。下面是 `gcc -O0 -S` 与 `gcc -O2 -S` 对**同一个文件**生成的两段真实汇编：

asm **-O0：它老老实实生成一个循环** examples/ch02/overflow_loop/overflow_loop-O0.s 第 28–38 行

```
28      .loc 1 8 12
29      movl    $2147483642, -4(%rbp)
30      .loc 1 8 5
31      jmp .L2
32  .L3:
33      .loc 1 8 36 discriminator 1
34      addl    $2, -4(%rbp)
35  .L2:
36      .loc 1 8 29 discriminator 2
37      cmpl    $0, -4(%rbp)
38      jg  .L3
```

asm **（续）-O2：整个函数体只剩下一条 ud2** examples/ch02/overflow_loop/overflow_loop-O2.s 第 10–21 行

```
10  overflow_loop_run:
11  .LFB101:
12      .file 1 "src/overflow_loop.c"
13      .loc 1 5 30 view -0
14      .cfi_startproc
15      .seh_endprologue
16      .loc 1 6 5 view .LVU1
17      .loc 1 8 5 view .LVU2
18      .loc 1 8 29 discriminator 2 view .LVU3
```

```
19      .loc 1 10 5 view .LVU4
20      .loc 1 8 36 discriminator 1 view .LVU5
21      ud2
```

第一块是正常机器的样子：`movl $2147483642` 给 `i` 赋初值，`.L3` 里 `addl $2` 加 2，`cmpl $0` 比 0，`jg .L3` 缩回去继续。第二块里，函数**从头到尾只剩一条 `ud2`**——`ud2` 是 x86 上专门制造「非法指令」的指令。gcc 的推理分两步：先按 `i` 不溢出算出这个循环**必然在第三次迭代**碰到溢出——而溢出是未定义行为，所以「溢出之后」的代码**不需要有结果**；于是它把整个函数体替换成一条 `ud2`。

下面这张图把四个档位放在一起：

图 / 搜索过程 **同一份溢出代码：四个编译档下的四种行为**

第 1 步 -O0 —— 循环真的跑完，退出 0

overflow_loop.c ：一份源码，循环靠溢出停下 → **cc -O0 ：老老实实编译** ← 本步新增

运行 ：打印 i = -2147483648，退出 0 ← 本步新增

第 2 步 -O2 —— 整个函数变成一条非法指令

cc -O2 ：认定溢出不会发生 ← 本步新增 → **ud2 ：函数体只剩这一条** ← 本步新增 → **运行 ：SIGILL，退出码 3221225501** ← 本步新增

第 3 步 -ftrapv —— 每次加法都检查，溢出就终止

cc -O0 -ftrapv ：加法带上检查 ← 本步新增 → **终止 ：退出码 3221226505，stdout 空** ← 本步新增

第 4 步 -O2 -fwrapv —— 明确告诉编译器「就当回绕」

cc -O2 -fwrapv ：不再当未定义行为 ← 本步新增 → **运行 ：打印 i = -2147483648，退出 0** ← 本步新增

第 1、2 帧就是刚才两次运行。第 2 帧那个 `3221225501` 不是 bash 给的 132，而是**闸门按自己的口径抓到的数**：`0xC000001D`，Windows 的「非法指令」异常码。两句话指同一件事，只是 bash 报「128 + 信号号」，闸门报异常码。第 3、4 帧是 2.6 节要真跑的东西。

> **编译器视角 · 编译器视角：`-Waggressive-loop-optimizations` 是它递给你的线索**
>
> 那句 `warning: iteration 2 invokes undefined behavior [-Waggressive-loop-optimizations]` 只在 `-O2` 下出现，`-O0` 下**一个字都没有**。警告名字本身就是提示：编译器在循环优化时发现「这里会踩到未定义行为」，顺手告诉你一句。
>
> **它的用法不是「照着改」，而是当作气味**：看到它，就回去问「我是不是在依赖某个会溢出的运算？」反过来不成立——`examples/ch02/overflow_u0` 的编译 stderr 是空的，而它的第 18 行在运行期真的溢出了。**没警告 ≠ 没 UB。**

编译器的三种自由

「未定义行为」经常被读成「结果随机」或者「会崩」，其实它的含义更狠。标准说完「不提任何要求」之后，实际发生的事可以分成三档，下面每一档都有本机的真凭据：

1. **假设它不发生。** `overflow_u0_report` 里那次 `INT_MAX + 1`，在 `-O2` 下 gcc 直接把结果算成常量塞进调用参数：`movl $-2147483648, %edx`。它没有「算一遍看看」，而是**认定**这个加法就是 `INT_MAX + 1` 的数学结果。

asm **-O2: INT_MAX + 1 被折成常量，直接传给 printf** examples/ch02/overflow_u0/overflow_u0-O2.s 第 81–85 行

```
81      .loc 1 18 5 view .LVU14
82      .loc 1 18 5 is_stmt 0 discriminator 1 view .LVU15
83      movl    $-2147483648, %edx
84      leaq    .LC1(%rip), %rcx
85      call    printf
```

2. **删掉分支。** 2.4 节那个 `n + 1 < n` 的「溢出检查」，`-O2` 下编译出来是这样：

asm **(续) -O2: 整个 if 分支没有了，只剩 return 0** examples/ch02/overflow_u0/overflow_u0-O2.s 第 121–132 行

```
121 overflow_u0_check:
122 .LVL9:
123 .LFB104:
124     .loc 1 26 30 is_stmt 1 view -0
```

```
125     .cfi_startproc
126     .loc 1 26 30 is_stmt 0 view .LVU25
127     .seh_endprologue
128     .loc 1 27 5 is_stmt 1 view .LVU26
129     .loc 1 28 5 view .LVU27
130     .loc 1 29 1 is_stmt 0 view .LVU28
131     xorl    %eax, %eax
132     ret
```

`if (n + 1 < n) return 1;` 整行没了，只剩 `xorl %eax, %eax`（把 0 放进返回值）。你写的检查代码不在程序里了——不是因为编译器坏，而是因为**你已经说了「这里不会溢出」**。

3. **任意行为，包括「看起来很正常」**。这是最容易骗人的一档：同一个 `overflow_u0` 在 `-O0` 下打印 `-2147483648`、退出码 0，看上去完全正常；它之所以正常，只是**这一次**这条编译路径恰好用「回绕」实现了加法。同一份源码换成 `-O2`，它**还是**打印 `-2147483648`（我把两档的 stdout 抓下来用 `cmp` 比过，逐字相同）——「看起来正常」并不意味着你的程序是对的。真正翻脸的是 `overflow_loop.c` 那种「结果依赖溢出」的代码：`-O2` 直接给你 `ud2`。

三档里最危险的是第 3 档：前两档至少能看出代码被改了，第 3 档会给你一个**能通过测试的结果**。「它在我的机器上是对的」这句话，在未定义行为面前不算证据。这也是本书每讲一处未定义行为就要贴一次真实退出码的原因——**现象比断言可靠**。

这一节的清单用的是检测模式 `opt-diff`：闸门按 `build/toolchain.json` 里声明的两档（`-O0` 与 `-O2`）各编译一次、各跑一次，然后比较。清单里写的是 `"expect": { "differs": "exit" }`——只认「退出码不同」这一种差异。实测两档退出码分别是 **0** 与 **3221225501**，`differs` 成立，所以这条清单算「检测到」。这也是 `opt-diff` 的边界：它能证明「这里存在与优化相关的未定义行为」，**不能**告诉你那是溢出、还是别的什么 UB。

2.6 `-ftrapv` 与 `-fwrapv`：两种把它钉住的办法

未定义行为不是只能干瞪眼。gcc 给了两个开关，让你**自己选**「溢出时怎么办」，代价各不相同。先看 `-ftrapv`：它让编译器在每次有符号加减乘后面插一段检查，溢出就当场终止进程。它拿的是 2.4 节 `overflow_u0` 的源码——那里做的 `INT_MAX + 1` 正是会被抓住的那一步；闸门的检测模式 `trapv` 会自动追加 `-ftrapv`，再真跑一遍：

实验记录 · 本机实测：`-ftrapv` 把溢出变成「终止」

- 工具链：gcc 16.1.0，`-std=c17`（C17），检测模式 `trapv`（清单 `ch02-overflow-u0-trapv`，`"detect": "trapv"`）；
- 编译里追加了 `-ftrapv`，编译退出码 0（这个开关不改语法，只改生成的代码）；
- 运行结果：进程被终止，**退出码 3221226505**（`0xC0000409`）；**stdout 是空的**——`printf` 一个都没落地，因为 stdout 进管道时是全缓冲的，进程没来得及 flush（常态，不是 bug）；
- **阴性对照**：同一份源码去掉 `-ftrapv` 再编译运行，退出码 **0**、stdout 打印 `INT_MAX + 1 = -2147483648`。证据在对照里不出现，所以「是谁抓到的」是清楚的。

学会看这三条：**退出码 + stdout 是否为空 + 阴性对照**。少了最后一条，你分不清「是检测器抓到的」还是「程序本来就这样」。

`-fwrapv` 是另一个方向：它明确告诉编译器「有符号溢出就按补码回绕算，别当未定义行为」。把 `-O2` 与 `-fwrapv` 一起加到 2.5 那个循环的源码上（清单 `ch02-overflow-fwrapv`，`"flags": ["-Iinclude", "-O2", "-fwrapv"]`）：

终端输出 终端输出 overflow-loop-fwrapv-out

```
循环结束时 i = -2147483648
```

对照 2.5 的第 2 帧：同样是 `-O2`、只是**不加** `-fwrapv` 的那一次编译成了 `ud2`，退出码 3221225501。同一份源码、同一档优化，差别只在「有没有告诉编译器溢出该怎么办」。

工程惯例 · 工程惯例：这两个开关的取舍

两个开关都**不是标准的一部分**，是 gcc 的实现选项。`-fwrapv` 把回绕变成**有定义的行为**，代价是**放弃一部分优化**（编译器不能再用「这里不可能溢出」去化简）；`-ftrapv` 让溢出**当场终止**，代价是每个有符号运算都多一条检查，而且它只抓有符号整数——无符号回绕、浮点都不算。

两种都不是「修好了」：真正的修法是**不写会溢出的运算**（2.13 节给三种写法）。顺带一句：本书**没有** `-fsanitize=undefined` 可用——本机 `-fsanitize=address,undefined` 是链接直接失败（记录在 `build/TOOLCHAIN.md` §4.1），所以 `-ftrapv` 是替代品，不是等价品。

2.7 补码：`-1` 的位模式，与 `INT_MIN` 这个死角

上面那些「回绕成 -2147483648」的说法，背后是同一件事：这台机器上的有符号整数用**补码**表示。

c examples/ch02/twos_complement/src/twos_complement.c 第 7–15 行

```c
7  void twos_complement_print_bits(int value) {
8      uint32_t bits = (uint32_t)value;      /* 位模式原样搬进无符号，好逐位看 */ 位模式原样搬进无符号
9
10     for (int i = 31; i >= 0; i--) {
11         putchar(((bits >> i) & 1u) ? '1' : '0'); 从最高位开始逐位打
12         if (i % 8 == 0 && i > 0) putchar(' ');
13     }
14     printf("   %11d   0x%08" PRIx32 "\n", value, bits); 十六进制用 PRIx32
15 }
```

8 位模式原样搬进无符号
11 从最高位开始逐位打
14 十六进制用 PRIx32

c （续）要看的五个值 examples/ch02/twos_complement/src/twos_complement.c 第 17–23 行

```c
17 void twos_complement_report(void) {
18     twos_complement_print_bits(1);
19     twos_complement_print_bits(0);
20     twos_complement_print_bits(-1);
21     twos_complement_print_bits(INT_MAX);
22     twos_complement_print_bits(INT_MIN);
23 }
```

终端输出 终端输出 twos-complement-out

```
00000000 00000000 00000000 00000001            1   0x00000001
00000000 00000000 00000000 00000000            0   0x00000000
11111111 11111111 11111111 11111111           -1   0xffffffff
01111111 11111111 11111111 11111111   2147483647   0x7fffffff
10000000 00000000 00000000 00000000  -2147483648   0x80000000
```

内存形态 四个字节，三种读法

第 1 步 -1 的位模式是四个 FF

byte[0] = FF ：最低地址 | byte[1] = FF | byte[2] = FF | byte[3] = FF ：最高位是 1，所以读成 -1

第 2 步 INT_MAX 加一，最高位翻成 1

byte[0] = 00 ← 本步新增 | byte[1] = 00 ← 本步新增 | byte[2] = 00 ← 本步新增

byte[3] = 80 ：只剩最高位，读成 -2147483648 ← 本步新增

第 3 步 unsigned 的 0 减一，回到四个 FF

byte[0] = FF ← 本步新增 | byte[1] = FF ← 本步新增 | byte[2] = FF ← 本步新增

byte[3] = FF ：同一串位，读成 4294967295 ← 本步新增

第 1 帧对上输出的第 3 行：`-1` 的 32 位**全是 1**。第 2 帧对上第 5 行：`INT_MIN` 是 `10000000 …`——最高位是 1、其余全 0，它没有对应的正数。第 3 帧是同一串位换个读法：四个 FF 按 `unsigned int` 读就是 `4294967295`。**位是位，「读成几」是你（或者类型）选的。**

大白话 · 用大白话说补码

想一个只有 4 位的玩具钟，刻度是 `0000` 到 `1111`（0 到 15）。补码的规矩是：

- **最高位当负号用**：最高位是 0，值就是剩下的位读出来的数；最高位是 1，值等于「剩下的位读出来的数 − 2^3」（4 位是 −8，32 位就是 -2^{31}）；
- 于是 `1111` 读成 −1，`1000` 读成 −8（32 位就是 −2147483648）；
- 加法的机器指令**不用管正负**：`1111` 加 `0001` 得到 `0000`（进位被丢掉），读出来是 0——−1 + 1 = 0，对上了。

所以「取反加一 = 取负」在这个表示里自然成立，代价是**正负数的个数不对称**：32 位里有一个值（`10000000 …`）**没有正数伙伴**，它取负还是它自己。这就是 `INT_MIN` 所有麻烦的源头。

标准原文 · 标准原文：标准允许三种表示

6.2.6.2 节「Integer types」第 2 段：

> If the sign bit is one, the value shall be modified in one of the following ways: — the corresponding value with sign bit 0 is negated (sign and magnitude); — the sign bit has the value −(2^M) (two's complement); — the sign bit has the value −(2^M − 1) (ones' complement).

大意：符号位为 1 时，值必须是这三种之一——**符号与绝对值**、**补码**（`1000` 是 −8）、**反码**（`1000` 是 −7）。本机走的是**补码**：`-1` 是全 1、`INT_MIN` 是 `1000…`。

出处：ISO/IEC 9899:2018（C17）§6.2.6.2 第 2 段。

标准原文 · 标准原文（续）：所以「位模式」只是本机事实

这一节所有位模式结论都是**本机事实**（gcc 16.1.0、目标 x86_64-w64-mingw32），不是语言定律。2023 年那一版 C 标准把补码定为唯一的表示，但你要读的代码库里还有大量按本书这一版标准（C17）写的代码。

要用位运算处理有符号数时，先把它转成无符号再做——`(uint32_t)value` 就是这个意思。转过去只改「怎么读」，不改位。

2.8 `INT_MIN` 上的三个坑

`INT_MIN` 是那个「没有正数伙伴」的值，于是三件再自然不过的事全都出问题：取负、取绝对值、除以 −1。**三件都是未定义行为**——这一点 2.4 节的 6.5 第 5 段已经定死了：结果不在可表示范围里，就是 UB。

c examples/ch02/int_min_trap/src/int_min_trap.c 第 6–12 行

```c
int int_min_negate(int n) {
    return -n;                  /* n == INT_MIN 时溢出 */ -INT_MIN 还是它自己
}

int int_min_abs_own(int n) {
    return n < 0 ? -n : n;      /* 自己写的「取绝对值」，在 INT_MIN 上照样失效 */ 自己写的绝对值照样失效
}
```

7 -INT_MIN 还是它自己
11 自己写的绝对值照样失效

c （续）除与取模：换成 volatile 让编译器别化简 examples/ch02/int_min_trap/src/int_min_trap.c 第 14–22 行

```c
int int_min_div_minus_one(int n) {
    volatile int d = -1;        /* volatile: 别让编译器把它化简成 -n */
    return n / d;
}
```

```c

int int_min_mod_minus_one(int n) {
    volatile int d = -1;
    return n % d;
}
```

c **（续）除零那一版：除数从参数来** examples/ch02/int_min_trap/src/int_min_trap.c 第 24–27 行

```c
int int_div_by(int a, int b) {
    volatile int d = b;        /* volatile：不让编译器把 b == 0 这件事提前看出来 */
    return a / d;
}
```

终端输出 终端输出 int-min-trap-out

```
INT_MIN          = -2147483648
-INT_MIN         = -2147483648
abs(INT_MIN)     = -2147483648
自己写的 abs     = -2147483648
换成 long long   = 2147483648
(unsigned)INT_MIN = 2147483648
-7 /  2 = -3    7 / -2 = -3
-7 %  2 = -1    7 % -2 = 1
-7 / -2 = 3    -7 % -2 = -1
```

输出前六行是「取负 / 取绝对值」这条线：`-INT_MIN`、`abs(INT_MIN)`、自己写的 `n < 0 ? -n : n`，三个打出来**都是 -2147483648**；`换成 long long` 那一行才是它的绝对值 **2147483648**（那个函数在文件末尾，三行）。

这些数是未定义行为在本机的一次表现，不是结果保证。 `abs` 那一族（`abs` / `labs` / `llabs`）在结果无法表示时同样是未定义行为（7.22.6.1 节）；`abs` 的内部实现就是「负数取负」，所以它在 `INT_MIN` 上与你自己写的那行一样无能为力。要算绝对值，先把类型变宽（`long long`）或者改用 `unsigned`，别指望 `abs` 帮你兜住。

后三行是**除法与取模的符号规矩**：结果一律**向零截断**（`-7 / 2` 是 `-3` 而不是 `-4`），余数的符号**跟着被除数走**（`-7 % 2` 是 `-1`，`7 % -2` 是 `1`）。这条规矩从 C99 起就是标准写死的，不是你机器的脾气。

`INT_MIN / -1` 与 `INT_MIN % -1` 则在标准里明确是**未定义行为**：商 `2147483648` 放不进 `int`。程序里那条路径长这样（清单给它传一个命令行参数就走到那里）：

c examples/ch02/int_min_trap/src/main.c 第 9–14 行

```c
    if (argc > 1 && strcmp(argv[1], "div") == 0) {
        printf("准备算 INT_MIN / -1，这一行先刷出去\n");   先刷出去，不然崩溃时字会丢
        fflush(stdout);                          /* 不刷的话，崩溃时缓冲区里的字会丢 */   flush 是这里的必要动作
        printf("INT_MIN / -1 = %d\n", int_min_div_minus_one(v));
        return 0;
    }
```

10 先刷出去，不然崩溃时字会丢
11 flush 是这里的必要动作

终端输出 终端输出 int-min-div-out

```
准备算 INT_MIN / -1，这一行先刷出去
```

实验记录 · **本机实测：三个陷阱路径的退出码**

同一份源码（`examples/ch02/int_min_trap/`）、同一套编译参数，靠命令行参数挑路径：

- `./app div`（算 `INT_MIN / -1`）：退出码 **3221225621**（`0xC0000095`，Windows 的「整数溢出」异常码），stdout 只有 `fflush` 出去的那一行；
- `./app mod`（算 `INT_MIN % -1`）：退出码 **3221225621**，stdout 只有它自己那行（文字与 `div` 那条不同）；
- 不带参数（只算取负 / 绝对值那一组）：退出码 **0**，正常打印。

实验记录 · 本机实测（续）：除零是另一个码

- `./app divzero`（算 `5 / 0`）：退出码 **3221225620**（`0xC0000094`，除零异常），stdout 只有那一行；
- 三条清单是 `ch02-int-min-div` / `ch02-int-min-mod` / `ch02-int-div-zero`，模式都是 `runtime_error`——「必须死，退出码必须等于 `expect.exit`」；
- `0xC0000095` 与 `0xC0000094` **不是**「标准规定的崩溃方式」：标准只说这里未定义；这两个码是 x86 的除法指令碰到这两种输入时操作系统替它报出来的，换个架构可能是别的方式。

2.9 除法与取模的符号规矩

标准原文 · 标准原文：`/` 与 `%`

6.5.5 节第 5 段：

> The result of the `/` operator is the quotient from the division of the first operand by the second; the result of the `%` operator is the remainder. In both operations, if the value of the second operand is zero, the behavior is undefined.

大意：`/` 给商、`%` 给余数；**除数为 0 是未定义行为。**

出处：ISO/IEC 9899:2018（C17）§6.5.5 第 5 段。

标准原文 · 标准原文（续）：余数必须让等式成立

6.5.5 节第 6 段：

> When integers are divided, the result of the `/` operator is the algebraic quotient with any fractional part discarded. ... If the quotient `a/b` is representable, the expression `(a/b)*b + a%b` shall equal `a`; otherwise, the behavior of both `a/b` and `a%b` is undefined.

大意：整数除法把小数部分**丢掉**（不是往小里取整）；只要商能表示，就必须满足 `(a/b)*b + a%b == a`——这条式子决定了余数的符号。

出处：ISO/IEC 9899:2018（C17）§6.5.5 第 6 段。

「余数永远让 `(a/b)*b + a%b == a` 成立」是最好用的记忆锚：`-7 / 2 = -3`，那么 `%` 必须给出 `-1`，因为 `-3 * 2 + (-1) = -7`。

回到 2.8 那个输出块的最后三行，逐个核对：`-7 / 2 = -3`、`7 / -2 = -3`（两个都是**向零**截断），`-7 % 2 = -1`、`7 % -2 = 1`（余数跟着被除数），`-7 / -2 = 3`、`-7 % -2 = -1`。**注意 `%` 的结果可以是负数**——如果你用 `x % 2 == 1` 判断奇偶，`-7 % 2` 是 `-1`，判断就失败。修法有两种：用位运算 `x & 1`（对补码机器成立），或者 `((x % 2) + 2) % 2` 把结果折回非负区间。

陷阱 · 除零不是「会崩」，是「标准什么都没说」

你会看到什么：`5 / 0` 在本机的退出码是 3221225620，stdout 只有崩溃前 flush 出去的那一行。看起来像「除零会崩」，很自然。

你会怎么误判：于是你觉得「反正会崩，我一眼就能看出来」。真正坑你的是**编译期能算出来的那一种**：`int x = 1 / 0;` 会让 gcc 直接给警告（`[-Wdiv-by-zero]`），但 `a / b` 里 `b` 是变量时它一个字都不说。

真相：除零是**未定义行为**，和溢出同一档——「崩掉」只是本机这次的表现。编译器有权假设除数为 0 的代码不存在，从而把「`b == 0` 的分支」删掉。所以判断「除数会不会是 0」必须在**你自己写的代码里**做（而且要在除法之前做），不能指望机器报错。

2.10 移位：三条红线

移位看起来是位运算里最简单的，但它的未定义行为密度最高。先看一个「先检查、再移位」的小函数：

c examples/ch02/shift_guards/include/shift_guards.h 第 6–10 行

```c
/* 先检查、再移位：断言通过才允许这一行执行。 */
static inline unsigned int shift_left_guarded(unsigned int value, int n) {
    assert(n >= 0 && n < 32); 先断言，再移位
    return value << n; 过了断言才轮到这一行
}
```

8 先断言，再移位
9 过了断言才轮到这一行

两种越界写法，以及三种「边界附近」的实测：

c examples/ch02/shift_guards/src/shift_guards.c 第 4–12 行

```c
int shift_guards_by_var(int n) {
    int one = 1;
    return one << n;            /* n 从参数来：越界就是 UB */ 移位量来自参数，越界就是 UB
}

int shift_guards_analyzer_demo(void) {
    int n = 40;                 /* 40 比 int 的宽度大得多 */ 40 比 int 的宽度还大
    return shift_guards_by_var(n);
}
```

6 移位量来自参数，越界就是 UB
10 40 比 int 的宽度还大

c **（续）同一件事，换成负的移位量** examples/ch02/shift_guards/src/shift_guards.c 第 14–17 行

```c
int shift_guards_negative_demo(void) {
    int n = -1;                 /* 负的移位量：同样越界 */
    return shift_guards_by_var(n);
}
```

c **（续）合法值长什么样** examples/ch02/shift_guards/src/shift_guards.c 第 19–27 行

```c
void shift_guards_report(void) {
    unsigned int v = 1u;

    printf("1u << 31                   = %u\n", v << 31);
    printf("shift_left_guarded(1u, 31) = %d\n", (int)shift_left_guarded(v, 31));
    printf("负数右移 -8 >> 1           = %d\n", -8 >> 1);
    printf("负数左移 -8 << 1           = %d\n", -8 << 1);
    printf("越界那两种 (1 << 40、1 << -1) 不在本程序里执行，见正文\n");
}
```

终端输出 终端输出 shift-guards-out

```
1u << 31                   = 2147483648
shift_left_guarded(1u, 31) = -2147483648
负数右移 -8 >> 1           = -4
负数左移 -8 << 1           = -16
越界那两种 (1 << 40、1 << -1) 不在本程序里执行，见正文
```

终端会话 -fanalyzer 抓到了负的移位量 ch02-warn-analyzer

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fanalyzer -Iinclude src/main.c src/shift_guards.c -o sg.exe
src/shift_guards.c: In function 'shift_guards_report':
src/shift_guards.c:25:52: warning: left shift of negative value [-Wshift-negative-value]
   25 |     printf("负数左移 -8 << 1           = %d\n", -8 << 1);
      |                                                    ^~
src/shift_guards.c: In function 'shift_guards_by_var':
src/shift_guards.c:6:16: warning: shift by negative count ('-1') [-Wanalyzer-shift-count-negative]
    6 |     return one << n;            /* n 从参数来：越界就是 UB */
      |            ~~~~^~~~
```

终端会话 （续）同一个函数，40 ≥ 32 也被抓到 ch02-warn-analyzer2

```
src/shift_guards.c:6:16: warning: shift by count ('40') >= precision of type ('32') [-Wanalyzer-shift-count-overflow]
    6 |     return one << n;              /* n 从参数来：越界就是 UB */
      |            ~~~~^~~~
```

三条红线，对着输出与会话一条条看：

1. **移位量 ≥ 类型宽度，或者是个负数 → 未定义行为。** `shift_guards_analyzer_demo` 的 `n = 40` 走的是前半条，`shift_guards_negative_demo` 的 `n = -1` 走的是后半条；`-fanalyzer` 两种都在**编译期**报了（两段会话），编译退出码仍是 0。这条清单用的是检测模式 `fanalyzer`（清单 `ch02-shift-analyzer`，`"detect": "fanalyzer"`，两条 needle 都写在 `expect.stderrContains` 里）；阴性对照（不加 `-fanalyzer`）里这两句诊断一个字都不出现，所以证据归得到分析器头上。
2. **左移一个负数 → 未定义行为。** 输出第 4 行 `-8 << 1` 打出来是 `-16`，看起来「对」，但标准不允许你依赖它——会话第 3 行那条 `left shift of negative value` 就是 gcc 在提醒你。
3. **右移一个负数 → 实现定义（不是未定义）。** 输出第 3 行 `-8 >> 1` 在本机是 `-4`（算术右移，符号位补 1）；标准明确允许实现选择「补 0」还是「补 1」。**「实现定义」比「未定义」安全**：它不会乱来，但也不保证到处一样。

合法的那两种也在输出里：`1u << 31` 打出 `2147483648`（无符号左移，标准保证是「乘 2 的 31 次方、按模折回」），带断言的小函数在参数合法时正常工作。

顺带把边界划清：在这台机器上，`-fanalyzer` 与本节有关的检查器只有 `shift-count-negative`、`shift-count-overflow` 与 `tainted-divisor` 三项（对 `gcc -Q --help=warnings` 的实测输出做过滤的结果）；它**没有整数溢出检查器，也没有通用的除零检查器**（`tainted-divisor` 只管「被污染的值当除数」那一种）。所以 2.4 与 2.8 节那些溢出、除零，本书只能靠 `opt-diff`、`-ftrapv` 和真实退出码给证据——硬把它们写成 `detect=fanalyzer` 就是伪造证据，闸门的阴性对照会把它打回来（`detect_not_attributable`）。

标准原文 · 标准原文：移位量与宽度

6.5.7 节「Bitwise shift operators」第 3 段：

> The integer promotions are performed on each of the operands. … If the value of the right operand is negative or is greater than or equal to the width of the promoted left operand, the behavior is undefined.

大意：两个操作数都先做**整数提升**（2.2 节那条规矩在这里也生效）；右操作数为负、或者不小于提升后左操作数的**宽度**，行为未定义。

出处：ISO/IEC 9899:2018（C17）§6.5.7 第 3 段。

标准原文 · 标准原文（续）：左移的边界

6.5.7 节第 4 段：

> If E1 has a signed type and nonnegative value, and E1 × 2^E2 is representable in the result type, then that is the resulting value; otherwise, the behavior is undefined.

大意：左移时若左操作数有符号且非负、乘完还放得下，结果就是那个值；**否则未定义**——负的左操作数、或者结果放不下，都落进「否则」。

出处：ISO/IEC 9899:2018（C17）§6.5.7 第 4 段。

标准原文 · 标准原文（续）：负数右移是实现定义

6.5.7 节第 5 段：

> If E1 has a signed type and a negative value, the resulting value is implementation-defined.

大意：有符号且为负时，右移的结果由**实现**决定（补 0 还是补 1，标准不管）。

出处：ISO/IEC 9899:2018（C17）§6.5.7 第 5 段。

> ⚠ **陷阱 · 断言放在头文件里，是「接口的守卫」**
>
> `shift_left_guarded` 把 `assert` 写在**函数体第一行**，这不是装饰：
>
> - 它守的是**调用方**——任何人在任何地方调它，参数越界就当场止住，不会悄悄算出错误的结果；
> - 代价是**运行期开销**（一次比较 + 一次分支），可以用 `-DNDEBUG` 整体关掉（`assert` 在定义了这个宏之后什么都不做）——但那时守卫就没了，所以关它之前先确认测试跑过了；
> - 本机的 `assert` 失败会**终止进程**并打印 `Assertion ... failed.`（这与 2.6 节 `-ftrapv` 的终止是两回事：一个是你自己写的检查，一个是编译器插的检查）。
>
> 这类「边界值在函数入口判掉」的写法，第 3 章讲接口设计时会再遇到。

2.11 位运算与逻辑运算：一个字符的差别

`&` 与 `&&`、`|` 与 `||`、`~` 与 `!` 是六件事，不是三件。先看三个掩码函数和打印二进制：

c examples/ch02/bitops/src/bitops.c 第 6–16 行

```c
uint32_t bit_set(uint32_t flags, unsigned n)   { return flags | (1u << n); } 置位：或上 1u 左移 n
uint32_t bit_clear(uint32_t flags, unsigned n) { return flags & ~(1u << n); } 清位：与上「取反的掩码」
int bit_test(uint32_t flags, unsigned n)       { return (flags >> n) & 1u; } 查位：右移 n 再取最低位

void bitops_print_binary(uint32_t value) {
    for (int i = 31; i >= 0; i--) putchar(((value >> i) & 1u) ? '1' : '0');
}

int bitops_precedence_bug(int a, int b) {
    return a & b == 0;          /* 这里的 == 先算 —— gcc 会用 -Wparentheses 提醒 */ == 先算，这里是个陷阱
}
```

6 置位：或上 1u 左移 n
7 清位：与上「取反的掩码」
8 查位：右移 n 再取最低位
15 == 先算，这里是个陷阱

c **（续）置位、查位、清位** examples/ch02/bitops/src/bitops.c 第 23–33 行

```c
    flags = bit_set(flags, 0);
    flags = bit_set(flags, 3);
    flags = bit_set(flags, 31);
    printf("置位第 0、3、31 位后 = 0x%08" PRIx32 " = ", flags);
    bitops_print_binary(flags);
    printf("\n");

    printf("查第 3 位 = %d, 查第 1 位 = %d\n", bit_test(flags, 3), bit_test(flags, 1));
    flags = bit_clear(flags, 3);
    printf("清掉第 3 位后 = 0x%08" PRIx32 ", 再查第 3 位 = %d\n",
           flags, bit_test(flags, 3));
```

c **（续）标志位、以及 & 与 && 的对照** examples/ch02/bitops/src/bitops.c 第 35–45 行

```c
    printf("FLAG_READ=%u FLAG_WRITE=%u FLAG_EXEC=%u FLAG_HIDDEN=%u\n",
           (unsigned)FLAG_READ, (unsigned)FLAG_WRITE, (unsigned)FLAG_EXEC, (unsigned)FLAG_HIDDEN);
    printf("(FLAG_READ | FLAG_EXEC) = 0x%" PRIx32 "\n", (uint32_t)(FLAG_READ | FLAG_EXEC));

    printf("a & b = %d, a && b = %d\n", a & b, a && b);
    printf("a | b = %d, a || b = %d\n", a | b, a || b);
    printf("!a = %d, ~a = %d, ~0u = %u\n", !a, ~a, ~0u);
    printf("(a & b) == 0 -> %d\n", (a & b) == 0);
    printf("bug(4, 0) = %d, 正确是 %d\n", bitops_precedence_bug(4, 0), (4 & 0) == 0);
    printf("bug(5, 0) = %d, 正确是 %d\n", bitops_precedence_bug(5, 0), (5 & 0) == 0);
}
```

终端输出 终端输出 bitops-out

```
置位第 0、3、31 位后 = 0x80000009 = 10000000000000000000000000001001
查第 3 位 = 1，查第 1 位 = 0
清掉第 3 位后 = 0x80000001，再查第 3 位 = 0
FLAG_READ=1 FLAG_WRITE=2 FLAG_EXEC=4 FLAG_HIDDEN=8
(FLAG_READ | FLAG_EXEC) = 0x5
a & b = 0，a && b = 0
a | b = 4，a || b = 1
!a = 0，~a = -5，~0u = 4294967295
(a & b) == 0 -> 1
bug(4, 0) = 0，正确是 1
bug(5, 0) = 1，正确是 1
```

掩码三件套值得背下来，因为它们是位运算的全部基础：

- **置位**：`flags | (1u << n)` —— `1u << n` 是只有第 n 位是 1 的掩码，按位或把它塞进去；
- **清位**：`flags & ~(1u << n)` ——先把掩码取反（除了第 n 位全是 1），再按位与，第 n 位被抹掉；
- **查位**：`(flags >> n) & 1u` ——挪到最低位再取那一位，结果是 0 或 1。

`1u` 里那个 `u` 不是装饰：`1u << 31` 合法（无符号左移），而 `1 << 31` 在 32 位机器上**会溢出** `int`（2.10 节第一条红线）。写掩码时一律用 `1u`，这是本书的纪律。

再看 `&` / `&&` 的差别（输出最后四行）：`a = 4`、`b = 0` 时，`a & b` 是 **0**（按位与，两个位都为 1 才是 1），`a && b` 也是 **0**，但含义完全不同—— `&&` 是「两个条件都成立吗」，结果只有 0 或 1，而且**有短路求值**（左边为假时右边根本不求值）。`a | b` = 4、`a || b` = 1 同理：按位或保留所有 1 位，逻辑或只回答真假。`!a` = 0（逻辑非）、`~a` = -5（按位取反）、`~0u` = 4294967295，也是同一对。

为什么掩码要用定宽类型写

上面三个函数的参数是 `uint32_t` 而不是 `unsigned int`，这不是啰嗦：

- `uint32_t` 来自 `<stdint.h>`，**32 位就是 32 位**，换平台也一样；`unsigned int` 在别的平台上可能是 16 位或 64 位，那时「第 31 位」这句话本身就不成立了；
- 打印它要用 `<inttypes.h>` 里的宏：`"0x%08" PRIx32`（小写十六进制）或 `PRIX32`（大写）。**别写** `%x` ——类型一换平台，格式串就可能对不上，而这类错误在运行期才现形；
- 反过来，`uint8_t` 这种窄类型**参与运算时先提升为** `int`（2.2 节那条规矩在这里同样生效）：`uint8_t a = 200, b = 200;` 时 `a + b` 是 `int` 的 **400**，不会按 8 位回绕成 144。想真的按 8 位回绕，得显式写 `(uint8_t)(a + b)`。

一句话：**宽度写在类型里，格式写在宏里。**这两条合起来，位运算代码才能跨平台。

> ⚠ **陷阱** · `if (a & b == 0)` —— **优先级把意思改掉了**
>
> **你会看到什么**：位运算的优先级**低于**比较。`a & b == 0` 被解析成 `a & (b == 0)`：先算 `b == 0` 得到一个 0 或 1，再和 `a` 按位与。输出的最后两行就是证据：`bug(4, 0)` 给出 **0**，正确答案是 **1**；而 `bug(5, 0)` 给出 **1**，碰巧对了。
>
> **你会怎么误判**：这个 bug 最坑的地方不是永远错，而是**有些输入碰巧对。**`a = 5, b = 0` 时它看起来完全正常，于是你以为掩码写对了，只在某次上线后才发现判断反了。
>
> **真相**：错的是**优先级**，不是掩码。gcc 的 `-Wparentheses` 会提醒你（下面会话第 3 行），读懂它：`suggest parentheses around comparison in operand of '&'`。同类陷阱还有：`~` 的优先级**高**于 `*`、`+`，但 `<<` **低**于 `+` —— `1u << n + 1` 是 `1u << (n + 1)`。**位运算表达式里，该加括号就加括号。**

终端会话 gcc 对 a & b == 0 的真实诊断 ch02-warn-paren

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -c bitops/src/bitops.c -Ibitops/include -o /dev/null
bitops/src/bitops.c: In function 'bitops_precedence_bug':
bitops/src/bitops.c:15:18: warning: suggest parentheses around comparison in operand of '&' [-Wparentheses]
   15 |     return a & b == 0;          /* 这里的 == 先算 — gcc 会用 -Wparentheses 提醒 */
      |                  ~~^~~~
```

2.12 枚举：有名字的整数

枚举（`enum`）是「给一组整数起名字」的工具。先看**类型**这一面：

c examples/ch02/enum_demo/include/enum_demo.h 第 4–11 行

```c
/* 枚举常量是编译期常量：CONN_STATE_COUNT 可以直接当数组长度。 */
enum conn_state {                 // 枚举类型与它的取值
    CONN_IDLE = 0,
    CONN_OPEN,
    CONN_BUSY,
    CONN_CLOSED,
    CONN_STATE_COUNT              // 哨兵：正好是状态个数
};
```

5 枚举类型与它的取值
10 哨兵：正好是状态个数

c examples/ch02/enum_demo/src/enum_demo.c 第 4–6 行

```c
static const char *const short_name[CONN_STATE_COUNT] = {   // 枚举常量直接当数组长度
    "IDLE", "OPEN", "BUSY", "CLOSED"
};
```

4 枚举常量直接当数组长度

c **（续）switch 覆盖每个取值** examples/ch02/enum_demo/src/enum_demo.c 第 8–16 行

```c
const char *conn_state_by_switch(enum conn_state s) {
    switch (s) {
    case CONN_IDLE:   return "空闲，可以连接";
    case CONN_OPEN:   return "已连接，可以收发";
    case CONN_BUSY:   return "忙，等一会儿";
    case CONN_CLOSED: return "已关闭，要重连";
    default:          return "不认识的值";
    }
}
```

c **（续）把枚举当整数用：循环、下标** examples/ch02/enum_demo/src/enum_demo.c 第 18–27 行

```c
void enum_demo_report(void) {
    printf("CONN_IDLE=%d CONN_OPEN=%d CONN_BUSY=%d CONN_CLOSED=%d\n",
           CONN_IDLE, CONN_OPEN, CONN_BUSY, CONN_CLOSED);
    printf("CONN_STATE_COUNT = %d（拿它当数组长度）\n", CONN_STATE_COUNT);
    printf("sizeof(enum conn_state) = %zu\n", sizeof(enum conn_state));

    for (enum conn_state s = CONN_IDLE; s < CONN_STATE_COUNT; s++) {
        printf("值 %d：下标名 %-7s switch 说：%s\n",
               (int)s, short_name[s], conn_state_by_switch(s));
    }
```

c **（续）与 int 的转换，以及它到底是什么类型** examples/ch02/enum_demo/src/enum_demo.c 第 29–35 行

```c
    enum conn_state st = 2;
    printf("把 int 2 塞进枚举变量：%d，它的名字是 %s\n", (int)st, short_name[st]);
    printf("枚举直接做下标：short_name[CONN_CLOSED] = %s\n", short_name[CONN_CLOSED]);
    printf("与 int 兼容 = %d，与 unsigned int 兼容 = %d\n",
           __builtin_types_compatible_p(enum conn_state, int),
           __builtin_types_compatible_p(enum conn_state, unsigned int));
}
```

终端输出 终端输出 enum-demo-out

```
CONN_IDLE=0 CONN_OPEN=1 CONN_BUSY=2 CONN_CLOSED=3
CONN_STATE_COUNT = 4 (拿它当数组长度)
sizeof(enum conn_state) = 4
值 0: 下标名 IDLE    switch 说: 空闲, 可以连接
值 1: 下标名 OPEN    switch 说: 已连接, 可以收发
值 2: 下标名 BUSY    switch 说: 忙, 等一会儿
值 3: 下标名 CLOSED  switch 说: 已关闭, 要重连
把 int 2 塞进枚举变量: 2, 它的名字是 BUSY
枚举直接做下标: short_name[CONN_CLOSED] = CLOSED
与 int 兼容 = 0, 与 unsigned int 兼容 = 1
```

输出印证了四件事：

1. **枚举常量是编译期常量**：`CONN_STATE_COUNT` 正好是 4，可以当**数组长度**（第 4 行 `short_name[CONN_STATE_COUNT]`）——`const` 变量做不到，数组长度必须是真正的常量表达式；
2. **枚举常量默认从 0 开始逐个加一**：`CONN_IDLE=0`、`CONN_OPEN=1`、`CONN_BUSY=2`、`CONN_CLOSED=3`；也可以自己指定，但**别依赖具体数字**，代码里一律用名字；
3. **枚举变量的本质是「某个整数类型」**：`sizeof(enum conn_state) = 4`，`enum conn_state st = 2;` 能编译能跑（标准允许把 `int` 赋给枚举，**不做检查**），`short_name[st]` 直接当数组下标。**但它不一定就是 `int`**：输出最后一行是本机实测，`与 int 兼容 = 0, 与 unsigned int 兼容 = 1`——这台机器给它挑的是 `unsigned int`（因为所有取值都非负）。加上一个负的枚举常量（比如 `= -1`）它才会变成 `int`。这也是为什么第 3 点要写成「某个整数类型」而不是「`int`」；
4. **`switch` 是枚举的天配**：每个 `case` 写一个名字，`default` 兜底；`case` 里直接 `return`，所以不需要 `break`。

标准原文 · 标准原文：枚举常量是 `int`

6.7.2.2 节「Enumeration specifiers」第 3 段：

> The identifiers in an enumerator list are declared as constants that have type `int` and may appear wherever such are permitted.

大意：枚举列表里的每个名字，首先是**类型为 `int` 的常量**，凡是 `int` 常量能出现的地方它都能出现。所以 `CONN_BUSY + 1` 能算，按 `int` 的规矩算。

出处：ISO/IEC 9899:2018（C17）§6.7.2.2 第 3 段。

标准原文 · 标准原文（续）：枚举类型由实现挑

6.7.2.2 节第 4 段：

> Each enumerated type shall be compatible with `char`, a signed integer type, or an unsigned integer type. The choice of type is implementation-defined...

大意：枚举类型要与 `char`、某一个有符号整数类型、或某一个无符号整数类型**兼容**——**选哪个由实现决定**（本机选了 `unsigned int`，正如上面输出最后一行）。所以「枚举是有名字的整数」这句话对，但把 `enum conn_state` 断言成 `int` 就过度了。

工程惯例 · 工程惯例：什么时候用 `enum`，什么时候用一组 `#define`

标准只规定「枚举是一种整数类型」，怎么用是工程问题。主流的取舍：

- **用 `enum`**：取值是**一组有限的、有名字的状态**（连接状态、日志级别、错误分类）。好处是类型名 + 一组名字一起出现在头文件里，`switch` 能一次列全，调试器里还能看到名字；
- **用 `#define` / 常量**：值是**位掩码**（`FLAG_READ = 1u << 0` 那种，可以按位或起来）。`enum` 的各个取值是**互斥**的整数，而位掩码需要「组合」，两者语义不一样——本书的 `bitops` 例子用的就是 `#define`；
- **代价**：C 的 `enum` **不做类型检查**（`enum conn_state st = 99;` 编译器不拦），而且底层类型由实现挑（这台是 `unsigned int`）。所以「枚举变量只能取合法值」要靠你在**接口**上守，跨模块传枚举时也别假设它一定是 `int`。

2.13 不触发未定义行为的溢出检查

前面讲了这么多「不要写会溢出的运算」，那**怎么检查**？三种写法，都能编译能跑：

c examples/ch02/overflow_check/src/overflow_check.c 第 5–10 行

```c
int check_by_compare(int a, int b, int *out) {
    if (b > 0 && a > INT_MAX - b) return 0; 先比后加，避开溢出
    if (b < 0 && a < INT_MIN - b) return 0; 负数那一边也要比
    *out = a + b;
    return 1;
}
```

6 先比后加，避开溢出
7 负数那一边也要比

c （续）更宽的类型，以及编译器内建 examples/ch02/overflow_check/src/overflow_check.c 第 12–21 行

```c
int check_by_wider(int a, int b, int *out) {
    long long sum = (long long)a + (long long)b;   /* 本机 long 只有 32 位，要更宽得用 long long */
    if (sum > INT_MAX || sum < INT_MIN) return 0;
    *out = (int)sum;
    return 1;
}

int check_by_builtin(int a, int b, int *out) {
    return __builtin_add_overflow(a, b, out) ? 0 : 1;
}
```

终端输出 终端输出 overflow-check-out

```
 2147483647 +           1 : compare=溢出       wider=溢出       builtin=溢出
 2147483647 +           0 : compare=2147483647  wider=2147483647  builtin=2147483647
-2147483648 +          -1 : compare=溢出       wider=溢出       builtin=溢出
         200 +         300 : compare=500          wider=500          builtin=500
```

三种写法思路不同，结果一致（输出四行的三列都对得上）：

- **运算前比较**：在加法**之前**用减法判断「这一加会不会越界」，分正负两条。**这两步减法本身不会溢出**——`b > 0` 时 `INT_MAX - b` 落在 `[INT_MAX - INT_MAX, INT_MAX - 1]`，`b < 0` 时 `INT_MIN - b` 落在 `[INT_MIN + 1, INT_MIN + INT_MAX]`，两个区间都在 `int` 范围内，所以这个检查是自洽的；
- **更宽的类型**：把两个 `int` 提成 `long long` 再相加，看结果还在不在 `int` 范围里。**本机 `long` 和 `int` 都是 32 位**（`LONG_MAX` 也是 2147483647），所以不能用 `long` 偷懒——必须 `long long`。这一条最不容易写错，代价是多一次转换；
- **编译器内建**：`__builtin_add_overflow(a, b, &out)` 溢出时返回**真**，没溢出就把结果写进 `out`、返回假。它是 gcc/clang 的扩展（不是标准 C），优点是**一条语句、不会写错**；代价是它不跨编译器（换 MSVC 就得另写）。

先看**编译期**能算出来的那一种：常量表达式溢出，编译器**会**报错。下面这个例子的清单写的是 `"mode": "compile_fail"`，并且加了 `-Werror`——闸门要求它真的编不过，编过了反而算失败：

c examples/ch02/overflow_literal/src/overflow_literal.c 第 1–8 行

```c
#include "overflow_literal.h"

/* 这一行的加法在编译期就能算完，算出来的值放不进 int：常量表达式溢出。
   清单给这个例子加了 -Werror，所以它必须编不过。 */
int overflow_literal_value(void) {
    int x = 2147483647 + 1; 常量表达式溢出，编译期就能算出来
    return x;
}
```

6 常量表达式溢出，编译期就能算出来

编译诊断 gcc 的真实诊断（原文照抄） overflow-literal-diag

```
src/overflow_literal.c: In function 'overflow_literal_value':
src/overflow_literal.c:6:24: error: integer overflow in expression of type 'int' results in '-2147483648' [-
Werror=overflow]
    6 |     int x = 2147483647 + 1;
      |                        ^
cc1.exe: all warnings being treated as errors
```

再看**运行期**那一种：2.4 节的 `overflow_u0` 做的是同一件事（`2147483647 + 1`），但它把加数当**参数**传进去，编译期算不出来，于是编译退出码 **0**、stderr **一个字都没有**。同一个「加一」，一个被编译器拦下、一个悄悄过去。

实验记录 · **本机实测：同一句 `+ 1`，两种命运**

- **常量表达式那一处**（`ch02-overflow-literal`）：`-Werror` 把 `[-Woverflow]` 警告升级成错误，编译器退出码 **1**，诊断原文就是上面那个块；
- **运行期那一处**（`ch02-overflow-u0`）：编译退出码 **0**、stderr **一个字都没有**，程序照样打出 `-2147483648`、退出码 0；
- 差别只有一个：**编译器当时知不知道操作数的值**。所以别指望它替你发现溢出——要么在设计上避免它（用 `long long` 或 `unsigned` 承载中间结果），要么在代码里显式检查（上面三种写法选一种），要么在测试环境里用 `-ftrapv` 把它抖出来（2.6 节）。三种可以同时用。

2.14 位域：能省几个字节，代价是「不可移植」

位域（bit-field）让你在结构体里**按位**分配成员：

c examples/ch02/bitfield_pack/include/bitfield_pack.h 第 4–9 行

```c
/* 位域：结构体里按「位」分配的成员。占几个字节由实现决定，不是标准定的。 */
struct pack_bits {
    unsigned int a : 1; 三个 1 位的域
    unsigned int b : 1;
    unsigned int c : 1;
};
```

6 三个 1 位的域

c **（续）换分配单位，字节数就变了** examples/ch02/bitfield_pack/include/bitfield_pack.h 第 11–19 行

```c
struct pack_nibbles {
    unsigned int lo : 4;
    unsigned int hi : 4;
};

struct pack_bytes {
    unsigned char a : 1;
    unsigned char b : 3;
};
```

终端输出 终端输出 bitfield-pack-out

```
sizeof(struct pack_bits)    = 4 (三个 1 位的域)
sizeof(struct pack_nibbles) = 4 (两个 4 位的域)
sizeof(struct pack_bytes)   = 1 (unsigned char 当分配单位)
bits = { a=1, b=0, c=1 }
nib  = { lo=0xA, hi=0x5 }
给 hi 赋 0x1F 之后 nib.hi = 0xF
bytes = { a=0, b=5 }, 按整数提升后打印
sizeof(bits.a + 0) = 4 (位域参与运算前先提升为 int)
```

三个 `sizeof` 排在一起看，位域的脾气就清楚了：`struct pack_bits`（三个 1 位）是 **4** 字节，`struct pack_nibbles`（两个 4 位）也是 **4** 字节，而 `struct pack_bytes`（用 `unsigned char` 当分配单位）是 **1** 字节。**「三个 1 位的成员」并没有让它变成 1 个字节**——前两个结构体的分配单位是 `unsigned int`，编译器按 4 字节为单位打包。想让位域真的省空间，得换**分配单位**（类型），不是减位宽。

取值部分：`bits = { a=1, b=0, c=1 }` 赋值正常；`nib = { lo=0xA, hi=0x5 }`；而给 4 位的 `hi` 赋 `0x1F`（31）之后拿到的是 **0xF**（15）——**超出的高位被丢掉**，gcc 会给一条真实警告：

终端会话 给 4 位的位域塞 31：gcc 的真实警告 `ch02-warn-bitfield`

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -c bitfield_pack/src/bitfield_pack.c -Ibitfield_pack/include -o /dev/null
bitfield_pack/src/bitfield_pack.c: In function 'bitfield_pack_report':
bitfield_pack/src/bitfield_pack.c:16:14: warning: unsigned conversion from 'int' to 'unsigned char:4' changes value from '31' to '15' [-Woverflow]
   16 |     nib.hi = 0x1F;                /* 31 放不进 4 位，会被截断 */
      |              ^~~~
```

标准原文 · **标准原文：位域的布局由实现决定**

> 6.7.2.1 节「Structure and union specifiers」：
>
> > A bit-field is interpreted as having a signed or unsigned integer type consisting of the specified number of bits.
>
> 大意：位域是一个**指定位数**的整数（有符号或无符号由声明里的类型决定）。但标准**没有**规定「位域之间怎么排、怎么打包、分配单位是几个字节」——那些是**实现定义**的。
>
> 出处：ISO/IEC 9899:2018（C17）§6.7.2.1。

标准原文 · **标准原文（续）：由此带来的三件事**

- 同一份 `struct pack_bits`，在这台机器上是 4 字节，换一个编译器可能是 1 字节或 8 字节；
- 位域成员**不能取地址**（`&p.a` 非法，因为一个字节里塞了好几个成员），也不能直接 `sizeof`；
- 位域参与运算前同样做**整数提升**（6.3.1.1 节那句「as restricted by the width, for a bit-field」）——所以 `sizeof(bits.a + 0)` 是 **4**。

> 还有一条：结构体的**对齐与填充**，标准同样只给宽松约束——第 5 章讲内存布局时会再回到这件事。

⚠ **陷阱** · **位域一用，可移植性就没了**

你会看到什么：`sizeof(struct pack_nibbles)` 是 4。你可能顺手把它当成「两个半字节 = 1 字节」来写协议解析。

你会怎么误判：以为位域的字节数是「位数加起来除以 8」。真正决定字节数的是**分配单位**，而且打包方向（先放的成员占低位还是高位）也是实现定义的。

真相：要跟外部数据打交道（读文件、解网络包、对硬件寄存器），**不要用位域**——用 `uint8_t` 数组加移位与掩码（2.11 节那三件套）自己拼，位序由你写死，到处都一样。位域适合「内部用、不涉及字节序、只图代码好读」的标志集合。知道它不可移植，然后**决定**用不用——而不是不知道。

2.15 这一节新增了什么，以及接下来

回到 2.1 的目录树，对着**第 2、3 帧**逐个点名：

- `promotion_lab/` ——整数提升：`~(unsigned char)0` 是 -1、`uint8_t` 加 1 得 256；
- `mixed_sign/` —— `-1 > 0u` 为真、`%d` 打 `unsigned` 得 -1；
- `overflow_u0/` ——有符号溢出打 `INT_MIN`、无符号回绕打 `4294967295`，外加 `n + 1 < n` 这个永远查不出溢出的「检查」；两个清单：跑一遍、`-ftrapv` 检测一遍；
- `overflow_loop/` ——靠溢出停下的循环，三个清单（`-O0` 跑、`opt-diff` 对比、`-O2 -fwrapv` 回绕），两份汇编 `overflow_loop-O0.s` / `overflow_loop-O2.s`；
- `overflow_check/` ——三种不触发 UB 的溢出检查；`overflow_literal/` ——常量表达式溢出，必须编不过；
- `int_min_trap/` ——取负、`abs`、除以 −1、取模 −1、除零，四个清单（一个正常跑、三个 `runtime_error`）；
- `twos_complement/` —— `-1` / `INT_MAX` / `INT_MIN` 的 32 位位模式；
- `enum_demo/` ——枚举做状态、`switch`、数组下标，并实测它在本机与谁兼容；

- `shift_guards/` ——移位三条红线、带断言的守卫函数，两个清单（跑一遍、`-fanalyzer` 抓两处越界）；
- `bitops/` ——掩码三件套、`PRIx32`、`&` 与 `&&` 的对照；`bitfield_pack/` ——位域的 `sizeof` 与截断；
- `session-opt-diff.txt` / `session-analyzer.txt` / `session-warnings.txt` ——三段抓下来的终端会话，正文里用 `term` 块逐字注入。

下一单元（第 2 章第 3 单元）接着讲**表达式与求值**：优先级只决定「怎么分组」，求值顺序、序列点与 `i = i++ + 1` 才是下一批坑。本单元已经把「未定义行为」的三种自由演示过一遍，下一单元会再见到它——这次是在一个表达式里。

2.16 小结

小结 · **这一节的要点**

- **提升与转换**：比 `int` 窄的整数（含位域）先提升为 `int` ——所以 `~(unsigned char)0` 是 **-1**、`(uint8_t)255 + 1` 是 **256**；`int` 与 `unsigned` 相遇时 `int` 变成 `unsigned` ——所以 `-1 > 0u` 为真（6.3.1.1 / 6.3.1.8）；
- **溢出有两种命运**：有符号溢出是**未定义行为**（6.5 第 5 段），无符号回绕是**有定义的模 2^N**（6.2.5 第 9 段）。未定义行为的真意是「编译器可以假设它不发生」：同一份循环 `-O0` 打印 `-2147483648`、`-O2` 变 `ud2`（3221225501）；`-ftrapv` 当场终止（3221226505、stdout 空），`-fwrapv` 让它回绕；
- **`INT_MIN` 是死角，三个坑都是 UB**：`-INT_MIN`、`abs(INT_MIN)`（7.22.6.1）、`INT_MIN / -1` 与 `% -1`（实测 3221225621），除零是 3221225620——都是现象，不是保证。掩码用 `1u << n`、`a & b == 0` 要加括号；枚举类型由实现挑（本机 `unsigned int`）；位域碰外部数据别用。

2.17 自测题

自测题 · **自测题（先自己答，再对答案）**

1. `~(unsigned char)0` 是多少？`sizeof` 是多少？为什么两件事同时成立？
2. `int a = -1; unsigned int b = 0;` 时 `a > b` 是真还是假？比较与格式两处各有什么警告？
3. 2.5 的循环在 `-O2` 下为什么变成 `ud2`？bash 的 `132` 与闸门的 `3221225501` 是什么关系？
4. `-INT_MIN`、`abs(INT_MIN)`、`INT_MIN / -1` 各属于哪种行为？`abs` 为什么救不了？
5. `-ftrapv` 与 `-fwrapv` 各做什么、代价是什么？怎么证明是它在起作用？

注 · **自测题答案**

1. 结果是 **-1**，`sizeof` 是 **4**。因为 `unsigned char` 参与运算前先被**整数提升**成 `int`，取反在 32 位的 `int` 上做（8 位全 0 取反在 8 位里才是 255）。
2. `a > b` 为**真**（1）：`a` 被转成 `unsigned int`，-1 变成 `4294967295`。编译器给 `-Wsign-compare`（`comparison of integer expressions of different signedness`）。`printf("%d", 4294967295u)` **默认不报**——要加 `-Wformat-signedness` 才出现 `format '%d' expects argument of type 'int', but argument 2 has type 'unsigned int'`；打出来的 `-1` 是错的，没人告诉你。
3. 循环只能靠有符号溢出停下，而这是**未定义行为**：gcc 于是假设 `i` 不会溢出、`i > 0` 恒成立，那条路「不需要有结果」，于是把整个函数体换成 `ud2`（非法指令）。bash 的 `132` 是 `128 + 4`（信号 4 = SIGILL），闸门的 `3221225501` 是同一事件在 Windows 上的异常码 `0xC000001D` ——**同一个现象，两种口径。**

注 · **自测题答案（续）**

4. 三个**都是未定义行为**：`-INT_MIN` 与 `INT_MIN / -1`（还有 `% -1`）是有符号溢出（6.5 第 5 段）；`abs(INT_MIN)` 是运行库在结果不可表示时的 UB（7.22.6.1），它的实现就是「负数取负」，所以和自己写的那行一样失效。本机三者都得到 `-2147483648`，那是现象；要真正算绝对值，先换 `long long`（实测得 2147483648）。
5. `-ftrapv`：有符号溢出时**终止进程**（实测 3221226505，stdout 因为没 flush 是空的），代价是每个有符号运算多一条检查、变慢，且只抓有符号整数；`-fwrapv`：把溢出定义成**回绕**（实测 `-O2 -fwrapv` 退出 0、打印 `i = -2147483648`），代价是**放弃一部分优化**。证明靠**阴性对照**：同一份源码去掉开关再跑，清单声明的证据（非零退出码、stderr 那段文字）必须在对照里**不出现**；闸门会自动复算这一步。

表达式与求值：优先级、转换与顺序

◎ 学习目标 · 看完这一单元，你应该能

- 背出「后缀 > 一元 > 乘加移位 > 关系 > 相等 > 位 > 逻辑 > 条件 > 赋值 > 逗号」这条次序，并解释 `(a < b) < c` 为什么不是「a 小于 b 小于 c」；
- 说出 `&&` 与 `||` 到底保证了什么（从左往右、右边可能根本不算），并说明 `&` 与 `|` 为什么没有这条保证；
- 把「顺序没规定所以会出事」的代码改成有确定顺序的两句话，并说清「未指定」与「未定义」的修法为什么不一样；
- 用 `-Wsequence-point` 与 `-Wparentheses` 两条警告，抓住同一句话里的两次修改、以及被当成条件的赋值；
- 把「我假设本机 `int` 是 4 字节」写成 `_Static_assert`，并在它失败时读懂那句诊断。

5.1 先看工程长什么样：这一节要建的九个例子

学表达式最容易走偏的方式，是背一张优先级表然后以为万事大吉。这一节照老规矩来：先看**文件**。因为「谁先算」这件事，最后都会落成两种可观察的东西——**输出里那几行的先后**，以及**编译器真的按什么次序算了**（那要去汇编里看）。

这是第 2 章的第三个单元，也就是全书的第 5 单元。本章前两个单元（类型与取值范围、整数的边界）的例子目录已经在 `examples/ch02/` 下躺着了，本节**不碰它们**，只往这个项目里加自己的九个例子目录。

目录树 **这一节的例子工程（一）：本节新增的九个目录**

第 1 步 本节开始时，本章前两个单元的例子已经躺在 ch02 里

```
examples/
├── ch01/ : 第 1 章的项目
└── ch02/
    └── 前两个单元的例子目录 : 类型与取值范围、整数的边界
```

第 2 步 本节新增第一批五个例子

```
examples/ch02/
├── precedence_zoo/ : 优先级与结合性 ← 本课新增
├── short_circuit/ : && 与 & 的差别 ← 本课新增
├── eval_order/ : f() + g() 谁先跑 ← 本课新增
├── seq_point/ : 一句话里改两次 ← 本课新增
└── assign_vs_compare/ : if (x = 0) ← 本课新增
```

第 3 步 再加四个例子，九个到齐

```
examples/ch02/
├── 五个例子目录 : 同上
├── ternary_comma/ : ?: 与逗号 ← 本课新增
├── cast_lab/ : 四种显式强转 ← 本课新增
├── static_assert_ok/ : 把假设写进代码 ← 本课新增
└── static_assert_fail/ : 写错了会被拦下 ← 本课新增
```

第二个图元换个角度看同一批文件：一个**例子目录**里到底摆了什么，以及本节在目录里额外留下了哪些「证据文件」：

目录树 **这一节的例子工程（二）：每个例子目录里长什么样，本节另外留下什么**

第 1 步 每个例子目录里的形状（以 precedence_zoo 为例）

```
examples/ch02/precedence_zoo/
├── include/precedence_zoo.h : 只放声明
├── src/main.c : 组装与打印
├── src/precedence_zoo.c : 放定义
└── precedence_zoo.json : 编译与对账的参数
```

第 2 步 本节另外抓下来的五段会话（都在各自的例子目录里）

```
examples/ch02/ ：九个例子目录都在这里
├── precedence_zoo/session-precedence.txt ：& 与 ! 的警告
├── short_circuit/session-short-circuit.txt ：& 解引用空指针
├── eval_order/session-eval-order.txt ：两档输出一样
├── seq_point/session-seq-point.txt ：-Wsequence-point 的警告
└── assign_vs_compare/session-assign.txt ：赋值当条件的警告
```

第 3 步 还有两份汇编，是 5.5 节的对照材料

```
examples/ch02/seq_point/
├── seq_point-O0.s ：-O0 的汇编，两次写都在
└── seq_point-O2.s ：-O2 的汇编，两次写被折叠
```

第 1 帧是本节开始时 `examples/` 的样子：`examples/ch02/` 里已经有本章前两个单元的例子。第 2、3 帧是本节新建的九个例子目录（分两批列完），每个都是「源文件放 `src/`、声明放 `include/`、编译与对账的参数写在 `.json` 清单里」的最小工程——从第二个项目起，本书不允许出现单文件工程；第二个图元的第 1 帧把这个形状画了出来。第二个图元的第 2 帧是本节另外抓下来的**五段终端会话**（警告与崩溃的现场，上面已经按 5.2 到 5.6 的顺序用过），第 3 帧是**两份汇编**（`-O0` 与 `-O2` 各一份，5.5 节拿它们对照）。

九个目录各自只负责**一件事**，后面每一节点名一个：

- `precedence_zoo`、`short_circuit`、`eval_order`、`seq_point` —— 「谁先算」的四个层次：运算符之间、逻辑运算的两边、两次函数调用之间、同一句话里的两次修改（5.2 到 5.5 节）；
- `assign_vs_compare`、`ternary_comma`、`cast_lab`、`static_assert_ok`、`static_assert_fail` —— 赋值表达式的值、两个不常用的运算符、显式强转、以及把平台假设写进代码（5.6 到 5.9 节）。

工程惯例 · **工程惯例：为什么还是一个例子一个目录**

标准对这件事只有一个要求：一个程序有且只有一个 `main`。分几个文件、怎么分，全是惯例。本书从第二个项目起要求每个例子都是 `src/` + `include/` 的最小工程，理由有三条：

- **每条结论都要能被单独验证**：这个例子只演示「表达式里的先后」这一件事，出问题时你能立刻知道是哪里错了；
- **声明与定义分家，接缝写在明面上**：`include/xxx.h` 只放声明，`src/xxx.c` 放定义，`src/main.c` 只负责组装与打印；
- **能被别人接手**：九个小文件比一个三千行的 `main.c` 好读得多。

代价是要多学一件事：头文件里写什么、不写什么。写重了会撞符号，写漏了会 `implicit declaration`（第 1 单元 1.5 节拆过那份诊断）；头文件、编译单元与链接性这套东西，在第 17 单元讲透。现在只要记住方向：**声明进头文件，定义进源文件。**

5.2 优先级：谁先算，标准写得像一张表

优先级这件事有一个残酷的事实：**它是查表，不是推理。**C 的运算符有四十多个，先算谁后算谁全写在标准的一张表里，而且是**十几级**、不是粗分的几大块。下面是与你最相关的那些级（从高到低）：

级别	运算符	结合性	你会写的样子
后缀	`()` `[]` `->` `.` 后缀 `++` `--`	左	`f(x)`、`a[i]`、`p->x`、`i++`
一元	`!` `~` `+` `-` `*` `&` 前缀 `++` `--` `(类型)` `sizeof`	右	`*p`、`&x`、`-i`、`(int)v`
乘除	`*` `/` `%`	左	`a * b / c`
加减	`+` `-`	左	`a + b - c`
移位	`<<` `>>`	左	`x << 2`
关系	`<` `<=` `>` `>=`	左	`a < b`
相等	`==` `!=`	左	`a == b`

再往下是低优先级的一半——**陷阱几乎都长在这一半里**（因为它们会「等」前面的先算完）。注意 `&`、`^`、`|` 是**三级**，`&&`、`||` 也是**两级**：

级别	运算符	结合性	你会写的样子
按位与	`&`	左	`x & mask`
按位异或	`^`	左	`a ^ b`
按位或	`|`	左	`a | b`
逻辑与	`&&`	左	`p && p->x`
逻辑或	`||`	左	`a || b`
条件	`?:`	右	`c ? a : b`
赋值	`=` `+=` `-=` …	右	`a = b = c`
逗号	`,`	左	`(a = 1, a + 1)`

{} 语法规则 · 这张表最常咬人的三处

1. `==` **比** `&` **紧**：所以 `x & 1 == 0` 是 `x & (1 == 0)`，不是 `(x & 1) == 0`。这一条每年都要坑掉一批人；
2. `<<` **比** `+` **松，** `&` **比** `^` **比** `|` **紧，** `&&` **比** `||` **紧**：所以 `x << 1 + 2` 是 `x << (1 + 2)`，`a | b & c` 是 `a | (b & c)`，`a || b && c` 是 `a || (b && c)`。这三条本机都实测过（`x=2` 时 `x << 1 + 2` 得 **16**，`4 | 1 & 1` 得 **5**，`1 || 0 && 0` 得 **1**），而且 gcc 对三条各给一条 `-Wparentheses`——它不是沉默的；
3. **赋值右结合、逗号左结合**：`a = b = c` 读作 `a = (b = c)`；而 `(a, b, c)` 从左边开始一个个算，整个表达式的值取最右边那个。

粗记一句「后缀 > 一元 > 乘加移位 > 关系 > 相等 > 位 > 逻辑 > 条件 > 赋值 > 逗号」可以，但要知道它只是**分大块**的口诀：同一块里的那几行仍然不是一级——上面第 2 条那三个陷阱，来源正是「把一块当成一级」。

这张表还能顺手解掉两个常见写法。`!a == b` 读作 `(!a) == b`——一元 `!` 比 `==` 紧，所以它问的是「`!a` 与 `b` 相不相等」：`a=1 b=0` 时为真（`!1` 是 0，`0 == 0`），`a=1 b=1` 时为假；它**不是**「`a` 不等于 `b`」（那要写 `a != b`）。本机实测：写 `!a == b` 时 gcc 给一条 `-Wlogical-not-parentheses`，原文是 `logical not is only applied to the left hand side of comparison`。`a = b == c` 读作 `a = (b == c)`——`==` 比 `=` 紧，这一句是「把比较结果（0 或 1）写进 `a`」，本身合法，只是扫一眼很容易看反；本机 `b=2 c=2` 时 `a` 得 1。

不背表也能活，但要看得出**哪里需要加括号**。下面这个程序把会出事的地方摆在一起，分三块看。第一块是关系链与后缀 `++`：

c examples/ch02/precedence_zoo/src/main.c 第 1–16 行

```c
#include <stdio.h>
#include "precedence_zoo.h"

int main(void) {
    int a = 1, b = 2, c = 3;
    printf("a < b < c (a=1 b=2 c=3) = %d\n", cmp_chain(a, b, c));  碰巧「对」的那一组

    int x = 3, y = 2, z = 1;
    printf("x < y < z (x=3 y=2 z=1) = %d  <- 3 < 2 明明是假\n", cmp_chain(x, y, z));  这一组才是陷阱

    int arr[4] = {10, 20, 30, 40};
    int *p = arr;
    int v1 = *p++;  后缀 ++ 先算，但取的是旧值
    printf("*p++  : 表达式值 = %d, 之后 *p = %d (p 前进了一格) \n", v1, *p);  表达式值 10，之后 *p 已经是 20
    int v2 = value_then_bump(p);
    printf("(*p)++: 表达式值 = %d, 之后 *p = %d (p 没动) \n", v2, *p);
```

```
6 碰巧「对」的那一组
9 这一组才是陷阱
13 后缀 ++ 先算，但取的是旧值
14 表达式值 10，之后 *p 已经是 20
```

第二块是掩码、`!` 与移位——**表的组内层级**就是在这里咬人的：

```c file=examples/ch02/precedence_zoo/src/main.c lines=18-25 mark=19,20,22,23,24,25 title=（续）掩码、`!` 与移位 int n = 6; printf("x & 1 == 0（x=6）= %d\n", mask_eq_zero(n)); printf("(x & 1) == 0（x=6）= %d\n", mask_then_compare(n));

    printf("!a == b (a=1 b=0) = %d <- 不是「a 不等于 b」\n", not_equal(1, 0));
    printf("x << 1 + 2 (x=2) = %d <- 先算 1 + 2 再移位\n", shift_add(2));
    printf("4 | 1 & 1 = %d (先 & 后 |) \n", 4 | 1 & 1);
    printf("1 || 0 && 0 = %d (先 && 后 ||) \n", 1 || 0 && 0);
```

第三块是 `a = b == c`：赋值、相等、后缀三级凑在一句里。

```c file=examples/ch02/precedence_zoo/src/main.c lines=27-31 mark=28,29 title=（续）a = b == c
    int p2 = 2, q2 = 2, a2 = 0;
    a2 = p2 == q2;
    printf("a = b == c (b=2 c=2) 之后 a = %d\n", a2);
    return 0;
}
```

六个函数住在模块里。注意第 12 行与第 20 行：它们**故意**不加括号，因为要演示的正是「编译器怎么理解这句话」：

c examples/ch02/precedence_zoo/src/precedence_zoo.c 第 3–9 行

```c
int cmp_chain(int a, int b, int c) {
    return (a < b) < c;             /* 括号是编译器本来就那么理解的写法 */ 括号只是把本来就有的次序写出来
}

int value_then_bump(int *p) {
    return (*p)++;                  /* 先取值，再把那个值加一 */ 先取值，再把那个值加一
}
```

4 括号只是把本来就有的次序写出来
8 先取值，再把那个值加一

后两个是「同一个问题的两种问法」：

c **（续）掩码那两处：一字之差，意思全反** examples/ch02/precedence_zoo/src/precedence_zoo.c 第 11–17 行

```c
int mask_eq_zero(int x) {
    return x & 1 == 0;              /* 实际算的是 x & (1 == 0) */
}

int mask_then_compare(int x) {
    return (x & 1) == 0;            /* 这才是「x 是偶数吗」 */
}
```

最后两个是「组内层级」的现场——`!` 与 `==`、`+` 与 `<<` 各差一级：

```c file=examples/ch02/precedence_zoo/src/precedence_zoo.c lines=19-25 mark=20,24 title=（续） `!a == b` 与 `x << 1 + 2` int not_equal(int a, int b) { return !a == b; /* 读作 (!a) == b，不是「a 不等于 b」 */ }

int shift_add(int x) { return x << 1 + 2; /* 先算 1 + 2，再左移三位 */ }
```

真跑出来是这样（这块输出由流水线逐字对账，不是我抄的）：

```output id=precedence-zoo-out
a < b < c (a=1 b=2 c=3) = 1
x < y < z (x=3 y=2 z=1) = 1  <- 3 < 2 明明是假
*p++  ：表达式值 = 10，之后 *p = 20 (p 前进了一格)
(*p)++：表达式值 = 20，之后 *p = 21 (p 没动)
x & 1 == 0 (x=6) = 0
(x & 1) == 0 (x=6) = 1
!a == b (a=1 b=0) = 1  <- 不是「a 不等于 b」
x << 1 + 2 (x=2) = 16  <- 先算 1 + 2 再移位
4 | 1 & 1 = 5 (先 & 后 |)
1 || 0 && 0 = 1 (先 && 后 ||)
a = b == c (b=2 c=2) 之后 a = 1
```

十一行输出，按三块读：

- **`a < b < c` 是 1，`x < y < z` 也是 1。**第二行才是重点：你心里读的是「3 小于 2 小于 1」，编译器读的是 `(3 < 2) < 1`——`3 < 2` 得 0（假），`0 < 1` 得 1（真），所以整个表达式为真。**关系运算符是左结合的**，`a < b < c` 永远等价于 `(a < b) < c`，它想表达的意思只可能靠 `a < b && b < c`（5.3 节的主角）写出来；
- **`*p++` 与 `(*p)++` 的差别就在那两个括号。**第 13 行写的是 `*p++`：后缀 `++` 的优先级高于一元 `*`，所以它读作 `*(p++)`——先取出 `p` 当前指向的值（10），再让 `p` 前进一格，之后 `*p` 变成 20；而 `(*p)++` 先把 `p` 解引用，再让**那个整数**加一：表达式交出来的还是 20，`*p` 变成 21，`p` 自己一步没动；
- **`x & 1 == 0` 给出 0，`(x & 1) == 0` 给出 1**（`x = 6`）。前者算的是 `6 & (1 == 0)`，也就是 `6 & 0`，结果是 0；后者才是「6 是偶数吗」，答案是 1；
- **组内层级那四行**：`!a == b` 是 1（`!1` 得 0，`0 == 0` 为真），`x << 1 + 2` 是 **16**（先算 `1+2`，再左移三位），`4 | 1 & 1` 是 **5**（先 `&`），`1 || 0 && 0` 是 **1**（先 `&&`）。后三行如果按「一块一级、从左往右」去读，会得到 6、1、0——错得毫无规律可循；这也正是「靠猜」的代价；
- **`a = b == c` 之后 `a` 是 1**：`p2 == q2` 先算（真，值为 1），再写进 `a2`。这里没有陷阱，只是提醒你赋值和相等挨着写时，读的人要多看一眼。

编译器对这些写法**不是沉默的**——下面三块是编译与运行的原文：五条警告（四条 `-Wparentheses`、一条 `-Wlogical-not-parentheses` 加它的 note），然后才是程序跑起来的样子。

终端会话 编译期的四条警告：每一次都在提醒「你可能读错了」 ch02-precedence-warn

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c
src/precedence_zoo.c -o pz.exe
src/main.c: In function 'main':
src/main.c:24:47: warning: suggest parentheses around arithmetic in operand of '|' [-Wparentheses]
   24 |     printf("4 | 1 & 1 = %d (先 & 后 |) \n", 4 | 1 & 1);
      |                                               ^
src/main.c:25:51: warning: suggest parentheses around '&&' within '||' [-Wparentheses]
   25 |     printf("1 || 0 && 0 = %d (先 && 后 ||) \n", 1 || 0 && 0);
      |                                                   ^~
src/precedence_zoo.c: In function 'mask_eq_zero':
src/precedence_zoo.c:12:14: warning: suggest parentheses around comparison in operand of '&' [-Wparentheses]
   12 |     return x & 1 == 0;              /* 实际算的是 x & (1 == 0) */
      |              ^
src/precedence_zoo.c: In function 'not_equal':
src/precedence_zoo.c:20:15: warning: logical not is only applied to the left hand side of comparison [-
Wlogical-not-parentheses]
   20 |     return !a == b;                 /* 读作 (!a) == b，不是「a 不等于 b」 */
      |               ^~
```

终端会话 （续）note 与第五条：gcc 连括号该加在哪都画出来了 ch02-precedence-warn-2

```
src/precedence_zoo.c:20:12: note: add parentheses around left hand side expression to silence this warning
   20 |     return !a == b;                 /* 读作 (!a) == b，不是「a 不等于 b」 */
      |            ^~
      |            ( )
src/precedence_zoo.c: In function 'shift_add':
src/precedence_zoo.c:24:14: warning: suggest parentheses around '+' inside '<<' [-Wparentheses]
   24 |     return x << 1 + 2;              /* 先算 1 + 2，再左移三位 */
      |              ^~
```

终端会话 （续）跑起来：十一行输出，退出码 0 ch02-precedence-run

```
$ ./pz.exe
a < b < c (a=1 b=2 c=3) = 1
x < y < z (x=3 y=2 z=1) = 1  <- 3 < 2 明明是假
*p++  : 表达式值 = 10，之后 *p = 20 (p 前进了一格)
(*p)++: 表达式值 = 20，之后 *p = 21 (p 没动)
x & 1 == 0 (x=6) = 0
(x & 1) == 0 (x=6) = 1
!a == b (a=1 b=0) = 1  <- 不是「a 不等于 b」
x << 1 + 2 (x=2) = 16  <- 先算 1 + 2 再移位
4 | 1 & 1 = 5 (先 & 后 |)
1 || 0 && 0 = 1 (先 && 后 ||)
a = b == c (b=2 c=2) 之后 a = 1
退出码 0
$ rm -f pz.exe
```

⚠ **陷阱** · **三个优先级的坑，各自长什么样**

你怎么会被坑到：这三处代码都**能编译、能跑、不崩溃**，只是算出来的东西不是你以为的那个。

- `a < b < c`：如果 `a`、`b`、`c` 是「a 最小、c 最大」这种巧合，它给出的答案碰巧是对的（第一组数据就是），于是你以为写法没问题。换一组数据立刻翻车（第二组）。**真相**：关系运算符左结合，先算的那一对会变成一个 0 或 1，再拿去跟下一个数比；
- `x & 1 == 0`：这类写法在「判断奇偶」的代码里到处都是。`1 == 0` 恒为 0，`x & 0` 恒为 0，于是整个条件**永远为假**——包括你想走进的那个分支。**真相**：`==` 比 `&` 紧；
- `*p++` 与 `(*p)++`：两个写法都对、都常用，区别只在「动的是指针还是它指的值」。库函数里遍历数组几乎都是 `while (*p++)` 那种形状，而累加器代码里常见 `(*total)++`。**真相**：后缀 `++` 与 `->`、`[]`、`()` 同属「后缀」这一级，比一元的 `*` 高。

⚠ **陷阱** · **（续）把一块当成一级，是另一半坑**

你会怎么被坑到：口诀「乘加移位 > 关系 > …」好背，于是你把它当成「同一块里从左往右算」。按这个读法，`x << 1 + 2`（`x=2`）是 `(2 << 1) + 2 = 6`、`4 | 1 & 1` 是 `(4 | 1) & 1 = 1`、`1 || 0 && 0` 是 `(1 || 0) && 0 = 0`——本机实测三个真值是 **16、5、1**。

真相：`+` `-` 比 `<<` `>>` 紧，`&` 比 `^` 比 `|` 紧，`&&` 比 `||` 紧，各差一级。**我会怎么误判**：我以前把 `a & 0xFF == 0` 读成「低八位是不是全 0」，直到 `-Wparentheses` 把它指出来——它算的是 `a & (0xFF == 0)`，恒为 0。上面第一块会话里那四条 `-Wparentheses`，说的都是这一类。

工程惯例 · **工程惯例：拿不准就加括号，长表达式就拆行**

标准只规定先算谁，它**从不管你写得看不看得懂**。主流做法有两条：

- **拿不准就加括号**：`(a < b) < c` 与 `a < b < c` 对编译器完全一样，但对读到这行的人不一样。加括号不花钱（现代编译器不会为它多生成一条指令），省的是后面接手那个人的时间；
- **一行里最多一个副作用**：`*p++` 这种「取值 + 挪指针」是允许的、也是惯用的写法，但同一行里出现两个以上带副作用的子表达式（比如 `*p++ = *q++` 之外的组合），就该拆成两行。

代价说清楚：多加的括号会让表达式看起来啰嗦，拆行会让函数变长几行。换来的东西很实在——**编译器不在乎你怎么写，接手的人在乎。**

5.3 短路求值：`&&` 与 `||` 有保证，位运算没有

上一节说的是「谁先算」的表；这一节说的是**一条比表更硬的保证**：`&&` 与 `||` 规定了从左往右算，而且**可能根本不算右边**。这条保证有真名字，叫**短路求值**（short-circuit evaluation）。

先看它长什么样。三个「会记数」的函数放在模块里，每被调用一次就把计数器加一：

c **每个函数都返回一个真假值，并且各记一次数** examples/ch02/short_circuit/src/short_circuit.c 第 6–14 行

```c
int no_a(void) {
    a_calls++;
    return 0;
}

int yes_a(void) {
    a_calls++;
    return 1;
}
```

`yes_b` 与两个「查计数」的函数在同一个文件的后半段：

c **（续）b 的计数与两个查询函数** examples/ch02/short_circuit/src/short_circuit.c 第 16–27 行

```c
int yes_b(void) {
    b_calls++;
    return 1;
}

int calls_a(void) { return a_calls; }
int calls_b(void) { return b_calls; }

void reset_calls(void) {
    a_calls = 0;
    b_calls = 0;
}
```

`main` 把 `&&` / `||` 与 `&` / `|` 各跑一遍，每次都先清计数：

c examples/ch02/short_circuit/src/main.c 第 4–13 行

```c
int main(void) {
    int r;

    reset_calls();
    r = no_a() && yes_b();
    printf("no_a() && yes_b(): 结果 %d, a 被调 %d 次, b 被调 %d 次\n", r, calls_a(), calls_b());

    reset_calls();
    r = no_a() & yes_b();
    printf("no_a() &  yes_b(): 结果 %d, a 被调 %d 次, b 被调 %d 次\n", r, calls_a(), calls_b());
```

8 逻辑与
9 打完计数再打印
12 位与
13 两边都算了

c （续）或运算与判空 examples/ch02/short_circuit/src/main.c 第 15-26 行

```c
    reset_calls();
    r = yes_a() || yes_b();
    printf("yes_a() || yes_b(): 结果 %d, a 被调 %d 次, b 被调 %d 次\n", r, calls_a(), calls_b());

    reset_calls();
    r = yes_a() | yes_b();
    printf("yes_a() |  yes_b(): 结果 %d, a 被调 %d 次, b 被调 %d 次\n", r, calls_a(), calls_b());

    int *p = NULL;
    printf("判空写在左边: p != NULL && *p > 0 = %d (安全) \n", (p != NULL && *p > 0));
    return 0;
}
```

终端输出 终端输出 short-circuit-out

```
no_a() && yes_b(): 结果 0, a 被调 1 次, b 被调 0 次
no_a() &  yes_b(): 结果 0, a 被调 1 次, b 被调 1 次
yes_a() || yes_b(): 结果 1, a 被调 1 次, b 被调 0 次
yes_a() |  yes_b(): 结果 1, a 被调 1 次, b 被调 1 次
判空写在左边: p != NULL && *p > 0 = 0 (安全)
```

四行计数就是这一节的中心证据，逐行对：

- `no_a() && yes_b()`：`b` **被调 0 次。**左边已经是假，`&&` 直接给出 0，**右边根本没算**——`yes_b` 一次都没跑；
- `no_a() & yes_b()`：`b` **被调 1 次。**位运算 `&` 要先把两边都算出来才能做按位与，所以右边照样跑了。**结果是一样的 0**，差的是「右边算没算」；
- 后两行是同一件事的镜像：`yes_a() || yes_b()` 里左边已经为真，右边不跑；`yes_a() | yes_b()` 里两边都跑；
- 最后一行是这个保证最实用的地方：`p` 是空指针，`p != NULL && *p > 0` 返回 0 而**程序没崩**——因为 `*p > 0` 那半句被短路跳过了。

标准原文 · 标准原文：`&&` 保证的那件事

C 语言标准（ISO/IEC 9899，C17）6.5.13 节「Logical AND operators」：

> Unlike the bitwise binary & operator, the && operator guarantees left-to-right evaluation; if the second operand is evaluated, there is a sequence point between the evaluations of the first and second operands. If the first operand compares equal to 0, the second operand is not evaluated.

大意：**与位运算 `&` 不同，`&&` 保证从左往右求值**；如果第二个操作数真的被求值了，那么「求值第一个」与「求值第二个」之间存在一个**序列点**；如果第一个操作数等于 0，第二个操作数**不被求值。**

标准原文 · 标准原文（续）：`||` 是同一套措辞，方向相反

6.5.14 节「Logical OR operators」：

> Unlike the bitwise | operator, the || operator guarantees left-to-right evaluation; if the second operand is evaluated, there is a sequence point between the evaluations of the first and second operands. If the first operand compares unequal to 0, the second operand is not evaluated.

大意：`||` 同样保证从左往右；第一个操作数不等于 0 时，第二个操作数不被求值。

出处：ISO/IEC 9899:2018（C17）§6.5.13、§6.5.14。

「序列点」这个词第一次出现，先当它是一句人话读：**序列点是代码里一个「前后有先后顺序」的位置。**`&&` 在第一个操作数算完之后、第二个操作数开算之前，正好有一个。这句话是下面这条惯例的**标准依据。**

工程惯例 · **工程惯例：`p != NULL && p->x > 0` 是接口的一部分**

「判空写在左边」不是为了省一次比较，它是一条**接口惯例**：把「这个指针可能是空的」和「它有效时我该怎么用它」写在同一个条件里。

代价有两面，都得说清：

- **左边的义务**：既然是你在判空，你就得保证左边那半句**不会解引用**那个指针。写成 `p->x > 0 && p != NULL` 就全反了——右边还没判，左边已经把空指针解引用了（下面那个例子就是它的现场）；
- **它不是万能的**：如果 `p` 是别人交给你的，而契约上说「不会是空」，那每次判空就是白花一次分支；反过来，契约上说「可能是空」而你没判，后果就是进程没了。所以这条惯例背后真正的规矩是 **「谁知道自己该怎么处理，谁判」**。

顺带一句：`&&` 的短路让「先判空、再用」这种写法**不需要嵌套 `if`**，这是它比 `&` 实用得多的原因。

上面那条惯例写反了会怎样？把两边换个顺序，再换成位运算，就是下一个例子的现场：

c examples/ch02/short_circuit/src/main_crash.c 第 4–13 行

```c
int main(void) {
    int *p = NULL;

    printf("马上要算 (p != NULL) & (*p > 0)，两边都会算\n");
    fflush(stdout);                          /* 想让这句话活下来，先显式刷缓冲 */ 先把话刷出去
                                             
    int bad = (p != NULL) & (*p > 0);        /* 右边要解引用一个空指针 */ 右边要解引用空指针
    printf("这一行不会执行：%d\n", bad);
    return 0;
}
```

8 先把话刷出去
10 右边要解引用空指针

终端会话 & 的两个操作数都要算：进程在解引用空指针那一刻被终止 ch02-short-circuit-crash

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main_crash.c
src/short_circuit.c -o crash.exe
$ ./crash.exe
马上要算 (p != NULL) & (*p > 0)，两边都会算
退出码 139
$ rm -f crash.exe
```

屏幕上的**那行字是 `fflush(stdout)` 挣来的**：`printf` 的结果平时先攒在缓冲区里，进程被终止时缓冲区来不及刷，你就什么都看不到（第 9 单元讲内存时见过的同一件事）。清单里声明的真实退出码是 **3221225477**（`0xC0000005`，访问冲突），会话里 bash 报的 139 是「128 + 信号 11」的另一种口径，同一个终止。

终端输出 终端输出 short-circuit-crash-out

```
马上要算 (p != NULL) & (*p > 0)，两边都会算
```

大白话 · **用大白话把这一节说一遍**

`&&` 像这样办事：**先听左边说完，不对劲就当场闭嘴。**左边是假，它知道无论右边说什么，整个结论都是假，于是右边那半句连念都不念。

`&` 不是这样：它是「把两边的话都听完，再按位对齐了算」。所以右边那句「去读一个空指针」照样被执行了。

一句话记住：**`&&` / `||` 是「看情况决定要不要算」，`&` / `|` 是「两个都要算，然后按位算」。**前者带来的是「可以安全地先判空」，后者带来的是「你可能读到不该读的东西」。

5.4 求值顺序：标准没规定的那一半

短路求值里，顺序是**标准保证**的。但只要出了 `&&` / `||` / `?:` / 逗号这几处（以及函数调用那个「实参算完、调用之前」的序列点），顺序就换了一个身份：**未指定（unspecified）**。这一节用一个最简单的写法把它逼出来——两个各打

印一行的函数相加：

c **两个函数各打印一行、各返回一个数** examples/ch02/eval_order/src/eval_order.c 第 4–12 行

```c
int f(void) {
    printf("  我是 f()，现在被调用\n");
    return 100;
}

int g(void) {
    printf("  我是 g()，现在被调用\n");
    return 7;
}
```

第三个函数把同一件事写成两句，顺序就是我们自己的了：

c **（续）拆成两句：顺序由你定** examples/ch02/eval_order/src/eval_order.c 第 14–18 行

```c
int sum_in_two_steps(void) {
    int first = f();
    int second = g();
    return first + second;
}
```

c **一句话相加，与拆成两句相加** examples/ch02/eval_order/src/main.c 第 4–12 行

```c
int main(void) {
    printf("算 f() + g()：\n");
    int total = f() + g();
    printf("结果 = %d\n", total);

    printf("拆成两句再算：\n");
    printf("结果 = %d\n", sum_in_two_steps());
    return 0;
}
```

终端输出 终端输出 eval-order-out

```
算 f() + g()：
  我是 f()，现在被调用
  我是 g()，现在被调用
结果 = 107
拆成两句再算：
  我是 f()，现在被调用
  我是 g()，现在被调用
结果 = 107
```

终端会话 第一次：基础参数（-O0）跑出来的顺序 ch02-eval-order-1

```
$ ./eo_o0.exe
算 f() + g()：
  我是 f()，现在被调用
  我是 g()，现在被调用
结果 = 107
拆成两句再算：
  我是 f()，现在被调用
  我是 g()，现在被调用
结果 = 107
退出码 0
```

终端会话 （续）换 -O2 再编一次，四行打印的顺序与结果一字不差 ch02-eval-order-2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -fdiagnostics-color=never -Iinclude src/main.c
src/eval_order.c -o eo_o2.exe
$ ./eo_o2.exe
算 f() + g()：
  我是 f()，现在被调用
  我是 g()，现在被调用
结果 = 107
拆成两句再算：
  我是 f()，现在被调用
  我是 g()，现在被调用
结果 = 107
退出码 0
```

把上面两块摆在一起读：

- **两档的输出一模一样，两次运行的输出也一模一样**：都是先 `f()` 后 `g()`，结果都是 `107`。这不是碰巧——同一份源码、同一个编译器、同一台机器，这次编译出来的代码就是先调 `f`；
- **但这不是保证。**标准对 `f() + g()` 的顺序说的是「未指定」：编译器有权先调 `g` 再调 `f`，有权在两次编译之间换一种顺序，有权在你加了 `-O2` 之后换一种顺序。本机这次稳定，只是**这一次的实现选择**；
- **想让它稳，就把它写死**：`sum_in_two_steps` 里那两句 `int first = f(); int second = g();` 之间有一个序列点（语句的结束），顺序是你写的，不是编译器挑的。两段输出里的 `结果 = 107` 相同，但只有第二段的顺序是**你保证的。**

调用树 **f() + g() 的两条调用顺序：标准两条都允许**

第 1 步 **main 准备算 f() + g()**

```
main() ：eval_order/src/main.c:6，准备算 f() + g()
```

第 2 步 **本机这次是 f 先**

```
main() ：等两个结果
f() ：eval_order/src/eval_order.c:5，先说一声再返回 100 ← 本步新增
main() → f() 本机这次的顺序
f() ⇒ main() 返回 100
```

第 3 步 **g 接着被调**

```
main() ：手上已有 100
g() ：eval_order/src/eval_order.c:10，返回 7 ← 本步新增
main() ：100 + 7 = 107
main() → g() 本机这次的顺序
g() ⇒ main() 返回 7
```

第 4 步 **标准允许的另一条顺序（本机没有走这一条）**

```
main() ：同一个表达式，等两个结果
g() ：标准不保证这个顺序
f() ：也允许 f 后
main() → g() 允许：g 先
main() → f() 结果仍是 107
```

第 2、3 帧是本机实测的那条顺序（`f` 先、`g` 后，对着上面两块会话）；第 4 帧是标准同样允许、本机这次没有走的另一条。**两帧都合法**——因为标准在这里只说了「未指定」，没说「必须这样」。这也是为什么本书不把「先 f 后 g」写成结论：它今天是本机的读数，明天换个编译器就可能不是。

标准原文 · **标准原文：这里标准说的是「未指定」**

同一个标准 6.5 节「Expressions」第 3 段：

> Except as specified later (for the function-call (), &&, ||, ?:, and comma operators), the order of evaluation of subexpressions and the order in which side effects take place are both unspecified.

大意：**除了后面几处专门规定的（函数调用 `()`、`&&`、`||`、`?:`、逗号运算符），子表达式的求值顺序、以及副作用发生的顺序，都是未指定的。**

那五处「专门规定」要拆开看：`&&` / `||` / `?:` / 逗号是**先后**上的规定（谁先算）；而函数调用 `()` 只规定了「**实参全部算完、真正调用之前**有一个序列点」——它**没有**规定实参之间谁先谁后。标准 6.5.2.2 节第 10 段明说：函数指示符、各个实参、以及实参内部的子表达式，求值顺序都是未指定的，只在调用之前有一个序列点。本单元 `printf("%d %d\n", i++, i++)` 打出 `11 10`，就是右边那个实参先算的现场。

标准原文 · **标准原文（续）：「未指定」这个词的确切含义**

「未指定」不是一句随口的话，标准 3.4.4 节「unspecified behavior」给了它定义：

> unspecified behavior: behavior, for a well-formed program construct and correct data, that depends on the implementation. The implementation is not required to document which behavior occurs.

大意：**未指定行为**指的是——程序本身是合法的、数据也是对的，但结果取决于实现；而且实现**没有义务告诉你**它这次选了哪一种。

出处：ISO/IEC 9899:2018（C17）§6.5 第 3 段、§3.4.4。

「未指定」这个身份带来两个推论，都很实用：

1. **它不是错误。** `f() + g()` 是一句完全合法的代码，标准只是不承诺顺序。当你需要顺序时（比如 `f` 必须先跑完），**你自己的责任是把它写成两句话**，而不是指望编译器讲良心；
2. **它和「未定义」不是一回事。**下一节的主角就是「未定义」——那时修法完全不同。

5.5 同一句话里改两次：这次轮到「未定义」

上一节的 `f() + g()` 只是「顺序没规定」，程序本身是干净的。这一节要看的写法连**程序都不再干净**：同一句话里，同一个对象被改了两次。

c examples/ch02/seq_point/src/seq_point.c 第 4–11 行

```c
int bad_increment(int i) {
    i = i++ + 1; 一句里对 i 又读又写
    return i;
}

void print_two_postfix(int i) {
    printf("  一次调用里两个 i++: %d %d\n", i++, i++); 两个参数里各有一个 i++
}
```

5 一句里对 i 又读又写
10 两个参数里各有一个 i++

第三个函数放在一起对照：这一句**有**唯一确定的读法。

c **(续) a = b = c：右结合，读法唯一** examples/ch02/seq_point/src/seq_point.c 第 13–17 行

```c
int chain_assign(int c) {
    int a, b;
    a = b = c;
    return a * 100 + b;
}
```

`main` 只负责把三个函数各调一次：

c examples/ch02/seq_point/src/main.c 第 4–9 行

```c
int main(void) {
    printf("int i = 5; 算完 i = i++ + 1; 之后 i = %d\n", bad_increment(5));
    print_two_postfix(10);
    printf("a = b = 7 之后，a 与 b = %d\n", chain_assign(7));
    return 0;
}
```

编译它。**这次编译器不再沉默**——两条警告，一字不落地贴在这里（`-Wsequence-point` 是本书固定参数里 `-Wall` 带来的一条）：

终端会话 -Wsequence-point：一句话里同一个对象被改了两次 ch02-seq-point

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/seq_point.c
-o seq_point.exe
src/seq_point.c: In function 'bad_increment':
src/seq_point.c:5:7: warning: operation on 'i' may be undefined [-Wsequence-point]
    5 |     i = i++ + 1;
      |     ~~^~~~~~~~~
src/seq_point.c: In function 'print_two_postfix':
src/seq_point.c:10:51: warning: operation on 'i' may be undefined [-Wsequence-point]
   10 |     printf("  一次调用里两个 i++: %d %d\n", i++, i++);
      |                                                   ~^~
```

终端会话 （续）编译通过了，跑起来是这个样子 ch02-seq-point-2

```
$ ./seq_point.exe
int i = 5; 算完 i = i++ + 1; 之后 i = 6
  一次调用里两个 i++: 11 10
a = b = 7 之后，a 与 b = 707
退出码 0
$ rm -f seq_point.exe
```

这份现场有四个要点，一个都不能漏：

1. **警告说的是** `operation on 'i' may be undefined` ——关键词是 `undefined`（未定义），不是「顺序不确定」；方括号里的 `-Wsequence-point` 就是管这条的开关名；
2. **两次命中的位置不同**：一处是第 5 行的 `i = i++ + 1`（`~~^~~~~~~~~` 标出整个表达式），一处是第 10 行 `printf` 参数里的第二个 `i++`（`~^~` 只标那个后缀）。第二处值得多看一眼：问题**不在** `printf` **里**，而在「一次调用的两个参数之间没有先后顺序」这件事上；
3. **它照样编译通过、照样跑出结果**：退出码 0，i 是 6，两个 `i++` 给出 `11 10`。这就是 UB 最危险的地方——**它这次表现得像一个正常程序**；
4. **本机在 -O0 与 -O2 下跑出来一模一样**（我两种档位各跑了三次，输出都是上面这三行）。这**不是**「所以它没事」的意思，见下面那块编译器视角。

终端输出 终端输出 seq-point-out

```
int i = 5; 算完 i = i++ + 1; 之后 i = 6
  一次调用里两个 i++: 11 10
a = b = 7 之后，a 与 b = 707
```

结果「看起来还挺合理」：`i = 5` 时 `i++` 交出 5，`5 + 1 = 6`，写回 `i`，得到 6。问题不在这个数，而在**这句话有没有语义**：

图 / 搜索过程 **a = b = c 与 i = i++ + 1：一个读法唯一，一个没有标准语义**

第 1 步 **a = b = c —— 赋值右结合，只有一种读法**

a = (b = c)：整个赋值表达式　b = c：先把 c 写进 b，这一步的值就是 c 的值　a = …：再把上一步的值写进 a

b = c → a = … 值往左传，顺序唯一

第 2 步 i = i++ + 1 —— 一次读、两次写，之间没有先后

i++ ：值取旧值 5，副作用是把 i 写成 6　　i + 1 ：加法用的就是 i++ 交出的 5，得 6

赋值 ：把 6 写进 i，这是对 i 的第二次写

i++ → 赋值 两次写都落在 i 上，顺序没规定

第 3 步 于是标准给的结论不是「7 或 6」，而是「没有语义」

3.4.3 ：未定义行为，本标准不施加任何要求　　本机实测 ：-O0 与 -O2 都得到 6（两次写都是 6）

危险之处 ：答案看起来完全正确，你查不出来

第 1 帧是句法层面的次序：`a = b = c` 因为赋值**右结合**，读法唯一（`a = (b = c)`，先写 `b`，再把那个值写进 `a`）。第 2 帧把 `i = i++ + 1` 拆成三件事：**一次读**（`i++` 交出旧值 5）、**两次写**（`i++` 的副作用把 6 写回 `i`；赋值再把 6 写进 `i`）——这两次写之间没有先后顺序。第 3 帧是结论：标准对这种情况说的是「未定义」（3.4.3 节），**不是「两种排法里选一个」**。

这里容易讲错，我把话说死：`i++` 的值按 6.5.2.4 节**就是自增前的旧值**，所以加法那一步永远用 5、永远得 6，两次写也都是 6——本机 `-O0` 与 `-O2` 都得到 6，换几种写法也还是 6。危险恰恰在这里：**答案是「对的」，程序却是未定义行为。**标准没有给这句话任何语义，编译器今天给出 6 只是它这一种选择；换个编译器、换个上下文，它有权做任何事。所以修法不是「挑一种排法」，而是**把这句话拆掉重写。**

标准原文 · 标准原文：「未定义」是另一个词

同一个标准 3.4.3 节「undefined behavior」给「未定义行为」的定义：

> undefined behavior: behavior, upon use of a nonportable or erroneous program construct or of erroneous data, for which this document imposes no requirements.

大意：**未定义行为**指的是——你用了不可移植的写法、错误的写法、或者错误的数据，对这种情形，本标准**不施加任何要求。**对照一下上一节引的 3.4.4 节：**未指定**说的是「程序合法、数据正确，只是结果取决于实现」，**未定义**说的是「这个写法本身就有问题」。一个是「不承诺」，一个是「不负责」。

出处：ISO/IEC 9899:2018（C17）§3.4.3、§3.4.4。

这两个词的区别，落到修法上是两条完全不同的路：

- **未指定**（`f() + g()`）：这句代码没错，只是顺序没承诺。修法是把顺序**写出来**——拆成两句，或者用 `&&` / `?:` / 逗号把它们连起来；
- **未定义**（`i = i++ + 1`）：这句话**本身**就是错的。修法是**把它删掉重写**——写成两句：`int old = i; i = old + 1;` 之类，别在「标准会选哪一边」上做文章。

「本机稳定」这件事必须解释清楚，否则你会以为那条警告是多余的。证据在两份汇编里——同一份 `seq_point.c`，左边是基础参数（`-O0`），右边是 `-O2`：

asm　examples/ch02/seq_point/seq_point-O0.s　　第 22–29 行

```
22      movl    %ecx, 16(%rbp)
23      .loc 1 5 10
24      movl    16(%rbp), %eax
25      leal    1(%rax), %edx   把 i+1 先算出来
26      movl    %edx, 16(%rbp)  写回 i（这次写的是 i++ 的效果）
27      .loc 1 5 7
28      addl    $1, %eax        再把 5+1 算出来
29      movl    %eax, 16(%rbp)  又写回 i（这次是赋值）
```

25 把 i+1 先算出来
26 写回 i（这次写的是 i++ 的效果）
28 再把 5+1 算出来
29 又写回 i（这次是赋值）

asm　**（续）-O2：那两句写被折叠成一句**　examples/ch02/seq_point/seq_point-O2.s　　第 18–24 行

```
18      .loc 1 5 5 is_stmt 1 view .LVU2
```

```
19  .LVL1:
20      .loc 1 6 5 view .LVU3
21      .loc 1 5 7 is_stmt 0 view .LVU4
22      leal    1(%rcx), %eax
23  .LVL2:
24      .loc 1 7 1 view .LVU5
```

`-O0` 那份把这句话老实译成了「一次读、两次写」：`movl 16(%rbp), %eax` 读出 5（整个函数只有这一次读 `i`）；`leal 1(%rax), %edx` 加 `movl %edx, 16(%rbp)` 把 6 写回 `i`——那是 `i++` 的副作用；`addl $1, %eax` 加 `movl %eax, 16(%rbp)` 再把 6 写回 `i`——那是赋值。**两次写，写的都是 6。**而 `-O2` 那一份只剩一句 `leal 1(%rcx), %eax`：优化器直接把这句算成「i + 1」，两次写被折叠成一次。

同一份源码、同一个编译器，两个优化档位给出的是两种不同的机器码——这就回答了「为什么本机稳定却仍然危险」：

编译器视角 · 编译器视角：优化器凭什么可以这样折叠

`-O2` 的推理链每一步都合法：

1. 按标准，`i = i++ + 1` 是**未定义行为**。未定义行为的意思是：**编译器有权假设它永远不会出现在程序里**；
2. 既然它不会出现，编译器就不必保留「先写一次、再写一次」这个现象——它可以按自己方便的方式算出「`i + 1`」；
3. 于是两遍写变成了一遍 `lea`，`i` 的结果碰巧还是 6。

注意它这次「碰巧对」：`-O0` 与 `-O2` 都给 6，只是因为这台机器、这个编译器、这两个档位这次的算法恰好一致。同一段代码换成另一个编译器、另一个版本、或者把周围代码改几个字，答案就可能是 7、可能是别的数——甚至「按 `-O0` 编出来是 6、按 `-O2` 编出来是 7」这种事也完全在许可范围内。

再看一眼第三个函数 `chain_assign` 在 `-O2` 下的样子，一句话就能说明「折叠」有多彻底（它的源码是 `a = b = c; return a * 100 + b;`）：

asm examples/ch02/seq_point/seq_point-O2.s 第 74–79 行

```
74  .LVL8:
75      .loc 1 16 5 view .LVU18
76      .loc 1 16 20 is_stmt 0 view .LVU19
77      imull   $101, %ecx, %eax
78      .loc 1 17 1 view .LVU20
79      ret
```

`-O0` 下它是两次写、四次读（`c`、`b`、`a`、`b` 各读一次），`-O2` 把它折成了一条 `imull $101`：`100c + c` 就是 `101 × c`。**优化器看得见整条数据流**，所以它写出的代码与你写下的语句几乎一一对应不了。这也是本书从第 1 单元起就固定用 `-O0 -g` 做教学实验的原因：只有不做优化，行号才对得上，你才能看见「这句话被算成了什么」。

实验记录 · 本机实测：这一节的六组读数

- 工具链：gcc 16.1.0，本书统一 `-std=c17`（也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令就是清单里那一条：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/seq_point.c -o …`；
- 那两份汇编是这样来的：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -S src/seq_point.c -o seq_point-O0.s`，把 `-O0` 换成 `-O2` 再跑一次，就是 `seq_point-O2.s`；
- `-Wsequence-point` 两次命中（上面两块会话的原文），**编译退出码仍是 0**；
- 运行：`i = i++ + 1` 得 **6**，两个 `i++` 打出 **11 10**，`a = b = c` 得 **707**，进程退出码 **0**；`-O0` 与 `-O2` 各跑三次，输出逐字相同；
- 清单口径：这一条是 `mode=run`、`expect.exit` 为 0——**不是 detect**。本机两档没有可观察差异，opt-diff 无从谈起；`-Wsequence-point` 只是编译期警告，不能当 opt-diff 的证据。

> ⚠ **陷阱 · 「未指定」与「未定义」混为一谈，你会修错地方**
>
> **你会怎么被坑到**：`i = i++ + 1` 编出来只给一条警告，程序照跑、结果还「对」（6），于是你有两个可能的误判——
>
> - 误判一：**「有警告而已，跑起来没事」**。真相是这本机这次的选择，不是承诺。同一句话在 `-O2` 下被折叠成一条 `lea`（上面那块汇编就是证据），而这**还只是这一台机器的这一次**；
> - 误判二：**「它跟 `f() + g()` 一样，是顺序没规定，那我加个括号就好了」**。加括号解决不了：`(i) = (i++) + (1)` 仍然是在读写同一个对象而没有顺序；写成 `i = i + 1;` 才是把两件事拆成两句话。
>
> **怎么分清**：把两个词的定义拿来对照（上面 standard 块里给了出处）——**未指定**是「程序合法，只是结果不承诺」，修法是**把顺序写出来**；**未定义**是「这个写法本身就是错的」，修法是**把这句话拆掉重写**。

`-Wsequence-point` 能帮你抓一部分，但别把它当保险：它抓的是「编译器在这次编译时看出来的形状」——本机实测，本节这两处（同一句里的 `i = i++ + 1`、一次调用的两个实参里各有一个 `i++`）它都报了（上面两块会话的原文），可它是**启发式**的：形状稍变（更长的表达式、跨语句的间接修改），它就可能沉默。**没警告不等于没 UB。它也不是「优化差异」的证据**：它报的是编译期的警告，`-O0` 与 `-O2` 都会报同一句；本书衡量「优化差异」用的是 opt-diff（同一份源码两档各跑一次，看**行为**是否不同，第 4 单元用过）。

5.6 赋值表达式也有值：`if (x = 0)` 为什么会编译过

到上一节为止，「一句话里有先后」这件事的三种身份都见过了：`&&` 那类是**标准保证**的，`f() + g()` 那类是**未指定**的，`i = i++ + 1` 那类是**未定义**的。从这一节起换主题：**表达式的值**。

第一个问题很小也很常见：`if (x = 0)` 为什么能编译过？答案一句话——**赋值本身是一个表达式，它有值**。标准 6.5.16 节把这件事写得很清楚：

c examples/ch02/assign_vs_compare/src/assign_vs_compare.c 第 3–10 行

```c
int assign_zero(int *x) {
    int r = (*x = 0);        /* 括号里是一个赋值表达式，它有一个值 */ 括号里是一个赋值表达式
    return r;
}

int assign_one(int *x) {
    return (*x = 1);         /* 交出的是刚写进去的那个值 */ 交出的就是刚写进去的值
}
```

4 括号里是一个赋值表达式
9 交出的就是刚写进去的值

`main` 把「谁走进哪个分支」和「赋值交出来的值」都打了出来：

c examples/ch02/assign_vs_compare/src/main.c 第 4–15 行

```c
int main(void) {
    int x = 1;

    if (x = 0) {  这是一个赋值，不是一个比较
        printf("if (x = 0) 走进真分支，x = %d\n", x);
    } else {
        printf("if (x = 0) 走进假分支，x 已经被改成 %d\n", x);
    }

    if (x = 1) {  x 先被改成 1，再当成条件用
        printf("if (x = 1) 走进真分支，x = %d\n", x);
    }
```

7 这是一个赋值，不是一个比较
13 x 先被改成 1，再当成条件用

c **（续）把赋值交出来的值单独取出来看** examples/ch02/assign_vs_compare/src/main.c 第 17–22 行

```c
    int r1 = assign_zero(&x);
    printf("assign_zero(&x) 交出来的是 %d，之后 x = %d\n", r1, x);
    int r2 = assign_one(&x);
    printf("assign_one(&x)  交出来的是 %d，之后 x = %d\n", r2, x);
    return 0;
}
```

跑它，同时把编译期的警告一起抓下来：

终端会话 -Wparentheses：赋值被当成条件用了 ch02-assign

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c
src/assign_vs_compare.c -o avc.exe
src/main.c: In function 'main':
src/main.c:7:9: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    7 |     if (x = 0) {
      |         ^
src/main.c:13:9: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
   13 |     if (x = 1) {
      |         ^
```

终端会话 （续）跑起来：走了哪条分支、x 变成了几 ch02-assign-run

```
$ ./avc.exe
if (x = 0) 走进假分支，x 已经被改成 0
if (x = 1) 走进真分支，x = 1
assign_zero(&x) 交出来的是 0，之后 x = 0
assign_one(&x)  交出来的是 1，之后 x = 1
退出码 0
$ rm -f avc.exe
```

终端输出 终端输出 assign-vs-compare-out

```
if (x = 0) 走进假分支，x 已经被改成 0
if (x = 1) 走进真分支，x = 1
assign_zero(&x) 交出来的是 0，之后 x = 0
assign_one(&x)  交出来的是 1，之后 x = 1
```

输出与警告对着看，就三件事：

1. **`if (x = 0)` 走的是假分支，而且 `x` 被改成了 0。**这不是「比较失败」，是**先把 0 写进 `x`**，再把赋值表达式的值（0）当条件——0 是假，所以走 `else`；
2. **`if (x = 1)` 走的是真分支。**同样：先把 1 写进 `x`，值 1 是真。**这类写法最危险的地方是它有时「看起来对」**——如果你本来想写 `if (x == 1)` 考察「x 是不是 1」，那么当 `x` 恰好等于 1 时 `if (x = 1)` 照样走真分支，你什么都发现不了；
3. **编译器给了 `-Wparentheses`**：`warning: suggest parentheses around assignment used as truth value`——「把赋值当真值用的时候，请加括号」。`-Wparentheses` 也是 `-Wall` 带来的一条（上面那块会话是原文）。

标准原文 · 标准原文：赋值表达式的值

同一个标准 6.5.16 节「Assignment operators」：

> An assignment expression has the value of the left operand after the assignment but is not an lvalue.

大意：**赋值表达式有一个值，它就是赋值之后左操作数的值**；但它不是 lvalue（不能出现在赋值号左边，`(a = b) = c` 是非法的）。同一节还规定简单赋值 `=` 的行为：把右操作数的值**转换**成赋值表达式的类型，然后存进左操作数指定的对象里（这个「转换」是 5.8 节强转的自动化版本）。

出处：ISO/IEC 9899:2018（C17）§6.5.16。

工程惯例 · **工程惯例：`if (x = 0)` 该怎么处理**

标准允许赋值表达式当条件用，惯例的做法有三种，按场景挑：

- **就是想比较**：写 `==`。真被坑过的人常用「尤达写法」`if (0 == x)`——把常量放左边，一旦手滑写成 `if (0 = x)`，编译器直接给一条硬错误：本机实测 gcc 报 `error: lvalue required as left operand of assignment`（编译退出码 1）。注意它与「给 const 变量赋值」**不是同一条诊断**：`const int c = 0; if (c = x)` 报的是 `error: assignment of read-only variable 'c'`。代价是可读性变差（人读起来是倒的），所以本书不推广，但你可以知道有这招；
- **就是想把赋值当条件用**：**加一层括号，把意图说出来**——`if ((c = getchar()) != EOF)` 是经典写法，括号既让 `-Wparentheses` 闭嘴，也告诉读代码的人「我是故意这样写的」；
- **在 `if` 条件里做赋值、又在别处用这个变量**：拆成两句。`if (x = 0)` 这种写法最大的问题不是编译不过，而是**读代码的人会以为那是比较。**

这一块是**惯例，不是标准**：标准允许赋值表达式当条件（6.5.16 节），它不管你写得更好读还是更难读。几种做法的代价不同——尤达写法牺牲可读性换「手滑就编不过」；加括号牺牲一点空间换「警告消失、意图明确」；拆成两句牺牲行数换「谁都不会看错」。`a = b = c` 也属于这一族：它的读法由结合性定死了（右结合），但一眼看过去仍然容易数错，写开成两句同样不算亏。

顺手把 `a = b = c` 的读法钉住：赋值运算符**右结合**，它读作 `a = (b = c)`——先把 `c` 的值写进 `b`，`b = c` 这个表达式的值就是写入后的 `b`，再把它写进 `a`。本机实测 `chain_assign(7)` 给出 `707`（5.5 节那块输出里的第三行），也就是 `a` 与 `b` 都成了 7。

5.7 `?:` 与逗号：两个不常用但必须认识的运算符

`?:` 是 C 里**唯一**的三元运算符（要三个操作数）。它的形状是 `条件 ? 表达式甲 : 表达式乙`：条件为真算甲，为假算乙，而且**整个表达式的类型由甲和乙一起决定**——这一条最容易出错。

c examples/ch02/ternary_comma/src/ternary_comma.c 第 4–14 行

```c
double pick_double(int flag) {
    return flag ? 1 : 2.5; 两个分支类型不同
}

int comma_value(int *log) {
    return (log[0] = 1, log[1] = 2, log[2] = 3); 逗号从左往右，值取最右
}

const char *grade_of(int score) {
    return score >= 90 ? "A" : score >= 60 ? "及格" : "不及格"; 嵌套的 ?: 从右往左读
}
```

5 两个分支类型不同
9 逗号从左往右，值取最右
13 嵌套的 ?: 从右往左读

c **用 sizeof 把「这个表达式是什么类型」问出来** examples/ch02/ternary_comma/src/main.c 第 4–15 行

```c
int main(void) {
    printf("flag ? 1 : 2.5 = %.1f (整个表达式是 double) \n", pick_double(1));
    printf("sizeof(0 ? 1 : 2)   = %zu\n", sizeof(0 ? 1 : 2));
    printf("sizeof(0 ? 'a' : 1) = %zu (字符常量在 C 里是 int) \n", sizeof(0 ? 'a' : 1));
    printf("sizeof(0 ? 1 : 2.5) = %zu\n", sizeof(0 ? 1 : 2.5));

    int log[3] = {0, 0, 0};
    int v = comma_value(log);          /* 先算完再打印，别把两件事塞进一行 */
    printf("逗号运算符的值 = %d, log = {%d, %d, %d}\n", v, log[0], log[1], log[2]);
    printf("三个分数: %s / %s / %s\n", grade_of(95), grade_of(75), grade_of(40));
    return 0;
}
```

终端输出 终端输出 ternary-comma-out

```
flag ? 1 : 2.5 = 1.0 (整个表达式是 double)
sizeof(0 ? 1 : 2)   = 4
sizeof(0 ? 'a' : 1) = 4 (字符常量在 C 里是 int)
sizeof(0 ? 1 : 2.5) = 8
逗号运算符的值 = 3, log = {1, 2, 3}
三个分数：A / 及格 / 不及格
```

四件事：

1. `flag ? 1 : 2.5` **的结果是** `double`：一个分支是 `int`、另一个是 `double`，标准会把它们「往一起调」，整个表达式取 `double`，所以 `pick_double(1)` 返回的是 `1.0` 而不是整数 1。`sizeof(0 ? 1 : 2)` 给出 4（两个分支都是 `int`），`sizeof(0 ? 1 : 2.5)` 给出 8（调成 `double`）；
2. `sizeof(0 ? 'a' : 1)` **也是 4**：在 C 里**字符常量本身是** `int`（不是 `char`），所以两个分支都是 `int`，结果是 4 而不是 1。这一条是第 3 单元「`char` 的三重身份」的延续；
3. **逗号运算符的值是最右边那个**：`comma_value` 里三个赋值从左往右依次做，整个表达式的值取最后一个 `log[2] = 3` 的值，也就是 3，同时 `log` 变成 `{1, 2, 3}`；
4. **嵌套的** `?:` **从右往左读**：`score >= 90 ? "A" : score >= 60 ? "及格" : "不及格"` 等价于 `... : (score >= 60 ? "及格" : "不及格")`。三个分数打出来是 `A / 及格 / 不及格`。

标准原文 · **标准原文：`?:` 先算条件，中间的序列点**

同一个标准 6.5.15 节「Conditional operator」：

> The first operand is evaluated; there is a sequence point between its evaluation and the evaluation of the second or third operand (whichever is evaluated).

大意：**先算条件**；条件算完之后，与被选中的那个分支之间有一个**序列点**。所以 `?:` 与 `&&` / `||` 一样，顺序是标准保证的（这也是上一节说「除了这五处之外都未指定」里的一处）。

标准原文 · **标准原文（续）：两个分支的类型怎么合**

同一个 6.5.15 节的第二句话，解释了我们为什么能用 `sizeof` 问出类型：

> If both the second and third operands have arithmetic type, the result type that would be determined by the usual arithmetic conversions, were they applied to those two operands, is the type of the result.

大意：**两个分支都是算术类型时，整个表达式的类型由「寻常算术转换」在它们之间决定**——`int` 与 `double` 相遇取 `double`，`int` 与 `long` 相遇取 `long`。这就是上面 `sizeof` 那两行的依据。

逗号运算符的规定在 6.5.17 节：它左边按 void 表达式求值、左右之间有序列点、整个表达式取右边的类型与值。

出处：ISO/IEC 9899:2018（C17）§6.5.15、§6.5.17。

工程惯例 · **工程惯例：`?:` 该用在哪儿**

先说明：这一块是**惯例，不是标准**——标准只说 `?:` 能怎么用，不管你读起来累不累。`?:` 的惯例用法是**「按条件挑一个值」，而且**两边的类型要一致**：

- 好用法：`const char *msg = ok ? "成功" : "失败";`、`printf("%d 个%s\n", n, n == 1 ? "苹果" : "苹果们");`；
- 该避免的用法一：**嵌套超过两层**。上面那个三档打分已经到可接受的边界了，再多一层就该换成 `if / else if / else` —— `if` 有缩进，读的人不用在脑子里配括号；
- 该避免的用法二：**两边的类型不一样**（`flag ? 1 : 2.5`）。它能编过，但你在读这行时得先在心里做一遍类型转换。

代价：`?:` 把「分支」压进一个表达式，省的是行数，付的是**可读性**——每多一层嵌套，读的人就要在脑子里多配一组冒号。真需要三档以上，`if / else if` 更好读。

工程惯例 · **工程惯例：参数里的逗号不是逗号运算符**

先说明：这是**惯例，不是标准**。逗号运算符**除了** `for (i = 0, j = n - 1; i < j; i++, j--)` **这种明确的场景，别用**。它极易和函数参数、初始化列表里的逗号混淆——`f(a, b)` 里那个逗号是**参数分隔符**，不是逗号运算符，所以 `f((a, b))` 才是「先把两个都算了、只把 `b` 传进去」。

代价：逗号运算符让一句里塞进几个动作，顺序读者得自己数；写开成几条语句没有任何额外开销，编译器生成的代码一样。想在一个表达式里做两件事，写两句话更清楚。

5.8 显式强转：四种用途，四种理由

「强转」这个词听着就像坏事，但 `(类型)值` 这种写法在工程里有**四种正当用途**。前两种是**转换值**（原来是什么数、现在变成什么数），后两种是**告诉编译器一件它不知道的事**（我要丢掉这个返回值、我要换一种方式读这块内存）。

c **前两种：转换值、截断浮点** examples/ch02/cast_lab/src/cast_lab.c 第 4–10 行

```c
double half_of(int n) {
    return (double)n / 2;               /* 不转的话，7 / 2 先算成 3 */
}

int truncate_toward_zero(double v) {
    return (int)v;                      /* 朝零截断：3.99 得 3，-3.99 得 -3 */
}
```

后两种不是「换个数值」，而是「换一种看法」与「表个态」：

c **（续）指针转换，与（void）吞返回值** examples/ch02/cast_lab/src/cast_lab.c 第 12–18 行

```c
int first_byte_as_int(const int *p) {
    return (int)*(const unsigned char *)p;    /* 换个类型，只看第一个字节 */
}

void write_line_once(void) {
    (void)printf("  这一行的返回值被 (void) 吞掉了\n");
}
```

c examples/ch02/cast_lab/src/main.c 第 4–16 行

```c
int main(void) {
    int n = 7;
    printf("7 / 2 = %d; half_of(7) = %.1f\n", n / 2, half_of(7));  不转与转的差别

    printf("(int)3.99 = %d, (int)-3.99 = %d\n", truncate_toward_zero(3.99),
truncate_toward_zero(-3.99));  两个方向都朝零

    int v = 0x41424344;
    printf("一个 int 的第一个字节 = 0x%02X\n", first_byte_as_int(&v));  只看第一个字节

    write_line_once();
    (void)printf("main 里也能这么写\n");  (void) 也可以写在别处
    return 0;
}
```

6 不转与转的差别
8 两个方向都朝零
11 只看第一个字节
14 (void) 也可以写在别处

终端输出 终端输出 cast-lab-out

```
7 / 2 = 3; half_of(7) = 3.5
(int)3.99 = 3, (int)-3.99 = -3
一个 int 的第一个字节 = 0x44
  这一行的返回值被 (void) 吞掉了
main 里也能这么写
```

四种用途逐个对上：

1. **转换值**（第 5 行）：`7 / 2` 是整数除法，得 3；`(double)7 / 2` 先把 `7` 变成 `7.0`，于是整句变成浮点除法，得 3.5。这里的关键是**转换发生在除法之前**——如果写成 `(double)(7 / 2)`，括号里的除法先算成 3，再转成 3.0，那是另一个意思；
2. **截断浮点**（第 9 行）：`(int)3.99` 得 3，`(int)-3.99` 得 **-3** 而不是 -4——标准规定这是**朝零截断**（丢掉小数部分），不是四舍五入、也不是向下取整。第 4 单元讲除法与取模时，两个整数相除也是朝零截断，同一条规则。
3. **指针转换**（第 13 行）：`(const unsigned char *)p` 把「指向 `int` 的指针」变成「指向无符号字符的指针」，于是这次解引用只读**一个字节**。本机是小端，`0x41424344` 的最低字节 `0x44` 住在最低地址，所以打出来是 `0x44`；
4. **`(void)` 吞返回值**（第 17 行、`main` 第 14 行）：`printf` 有返回值（写了几个字符），返回 `int`。有些编译器与静态检查工具会提醒「你忽略了返回值」，写 `(void)printf(...)` 就是明说「我知道它返回值，我故意不要」。**它是到 `void` 的一次显式转换**（标准 6.5.4 节里的 cast），只是转换结果没有值可用——它不改变任何东西，作用只是「把这个表达式标成故意丢弃的」。

标准原文 · **标准原文：浮点转整数是朝零截断**

同一个标准 6.3.1.4 节「Real floating and integer」：

> When a finite value of real floating type is converted to an integer type other than _Bool, the fractional part is discarded (i.e., the value is truncated toward zero).

大意：**把有限的浮点值转换成一个不是 `_Bool` 的整数类型时，小数部分被丢掉**——也就是朝零截断。所以 `(int)3.99` 是 3、`(int)-3.99` 是 -3（朝零，不是朝下）。

至于 `(类型)值` 这个写法本身，标准 6.5.4 节「Cast operators」规定了它的语法与含义：**在一个表达式前面加上带括号的类型名，就把这个表达式的值转换到那个类型**；`(void)expr` 也在这一节里——那是转换到 `void`，结果没有值可用。

「装不下」的后果要分三种说（转换规则在 6.3 节，不在 6.5.4）：无符号整数之间装不下 → 按 2 的 N 次方取模；有符号整数装不下 → 实现定义（或者由实现定义的信号）；浮点转整数而整数部分装不下 → **未定义行为**。指针转换是另一套规则：转出来的指针没按目标类型对齐就是 UB（第 9 单元见过那个现场）。

出处：ISO/IEC 9899:2018（C17）§6.3.1.3、§6.3.1.4、§6.5.4。

工程惯例 · **工程惯例：强转是一把「关掉提醒」的钥匙**

显式强转写下去，编译器就不再问「这里从 `double` 变 `int` 会丢小数，你确定吗？」。所以惯例是：

- **先问「能不能不改类型」**：`7 / 2` 想要小数，最干净的做法是把变量本身声明成 `double`，或者写成 `7.0 / 2`，而不是在表达式里塞一个强转。强转是**局部**的，改类型是**全局**的，后者更能让下一个人看懂；
- **`(void)` 是唯一鼓励多加的**：它没有任何副作用，只是把「故意丢弃」写出来。标准库里的 `(void)` 到处都是。

工程惯例 · **工程惯例（续）：指针转换要谨慎，代价要算清**

- **指针转换要谨慎**：`(char *)` / `(unsigned char *)` 用来「按字节看内存」是常规做法（这也是标准允许的少数几种别名用法之一），但把 `(int *)` 硬套到一个没对齐的地址上是 UB（第 9 单元的 `ptr_types` 就是那个现场）。**跨类型看字节，`memcpy` 才是标准保证的写法。**
- **代价**：每加一个强转，你就少了一条编译器的提示。判断标准是「我能不能用一句话解释这个转换为什么安全」——解释不了，就别加括号把它关掉。

5.9 把假设写进代码：`_Static_assert`

上面每一节都在用「本机 `int` 是 4 字节」「本机一个字节是 8 位」这类事实。这些事实来自工具链，不是语言。写一段依赖它们的代码时，**最好的做法是把假设写进代码**，让它不成立时编译就过不去：

c examples/ch02/static_assert_ok/src/static_assert_ok.c 第 1–7 行

```c
#include <limits.h>
#include "static_assert_ok.h"
```

```c
/* 三条假设写在文件作用域：编译期就检查，不成立就编不过 */
_Static_assert(CHAR_BIT == 8, "本机一个字节不是 8 位");
_Static_assert(sizeof(int) == 4, "本机 int 不是 4 字节");
_Static_assert(sizeof(char) == 1, "本机 char 不是 1 字节");
```

5 一个字节必须是 8 位
6 这是本单元反复用到的那条假设
7 char 必须占一个字节

模块剩下的两个函数负责把这两个数「问出来」：

(续) 把 sizeof 与 CHAR_BIT 交出来 examples/ch02/static_assert_ok/src/static_assert_ok.c 第 9–15 行

```c
int int_bytes(void) {
    return (int)sizeof(int);
}

int char_bits(void) {
    return CHAR_BIT;
}
```

examples/ch02/static_assert_ok/src/main.c 第 4–8 行

```c
int main(void) {
    printf("sizeof(int) = %d 字节，CHAR_BIT = %d 位\n", int_bytes(), char_bits());
    printf("三条 _Static_assert 都成立了，这个程序才编得出来\n");
    return 0;
}
```

终端输出 终端输出 static-assert-ok-out

```
sizeof(int) = 4 字节，CHAR_BIT = 8 位
三条 _Static_assert 都成立了，这个程序才编得出来
```

`_Static_assert(常量表达式，"消息")` 有两个操作数：一个**编译期就能算出真假**的表达式，一句给人看的消息。三个条件都成立，所以这个程序照常编译、照常运行。现在把中间那条改成故意错的——这一条单列一个例子目录，它**注定编不过**：

examples/ch02/static_assert_fail/src/static_assert_fail.c 第 1–8 行

```c
#include "static_assert_fail.h"

/* 本机 int 是 4 字节，这一条故意写成 8：编译期就该被拦下 */
_Static_assert(sizeof(int) == 8, "int is not 8 bytes on this machine");

int never_reached(void) {
    return 0;
}
```

4 这一条故意写成 8，本机 int 是 4 字节

编它。下面这块是 gcc 真实吐出来的 stderr（逐字，连对齐的空格都没动）：

编译诊断 编译诊断 static-assert-fail-diag

```
src/static_assert_fail.c:4:1: error: static assertion failed: "int is not 8 bytes on this machine"
    4 | _Static_assert(sizeof(int) == 8, "int is not 8 bytes on this machine");
      | ^~~~~~~~~~~~~~
```

读它只要三样：**级别是** `error`（不是警告，编译就此停下）、**位置是** `src/static_assert_fail.c:4:1`（我们写断言的那一行）、**消息就是字符串字面量里那串字。**这就是 `_Static_assert` 的全部价值：它把「我以为」变成「编译器替我检查过」。

标准原文 · **标准原文：断言失败时必须说什么**

同一个标准 6.7.10 节「Static assertions」规定：

> If the value of the constant expression compares unequal to 0, the declaration has no effect. Otherwise, the constraint is violated and the implementation shall produce a diagnostic message that includes the text of the string literal.

大意：**常量表达式的值不等于 0 时，这条声明什么也不做**；等于 0 时就是违反约束，**实现必须给出一条诊断，并且这条诊断要包含那个字符串字面量的内容**（标准这句话正是上面那块诊断里能看到我们写的那串字的原因）。

出处：ISO/IEC 9899:2018（C17）§6.7.10。

工程惯例 · **工程惯例：哪些假设值得写成 `_Static_assert`**

先说明：**该写哪些假设是惯例，标准只规定了 `_Static_assert` 怎么用。** `_Static_assert` 是 C11 加进来的，本书统一用的 C17 当然有。惯例是把**「如果它不成立，后面所有推理都会崩」**的那些假设写下来：

- **平台假设**：`sizeof(int) == 4`、`CHAR_BIT == 8`、`sizeof(void *) == 8`（本机 8 字节）；
- **结构体布局假设**：`sizeof(struct header) == 16` 这类，第 11 单元讲 padding 时会反复用到；
- **协议常量**：某个字段必须恰好 4 字节、某个表必须恰好 256 项。

工程惯例 · **工程惯例（续）：它能查什么、查不了什么**

- **别把它写成运行时检查**：`_Static_assert` 在**编译期**发作，代价是零（生成的代码里什么都没有）；而 `assert(x)` 在运行期发作，代价是每个进程都带着这些判断。分工很清楚：**能编译期查的，绝不留到运行期**；
- **它只能查常量表达式**：`sizeof`、宏、`enum` 常量可以，变量不行。这是它的**代价**——运行时才能确定的假设它管不了，那部分只能靠 `assert` 或测试。另外，断言里的中文消息在诊断里会以转义形式出现（本机实测：gcc 把非 ASCII 字符按字节转义成一串 `\377…`），所以这个例子里我把消息写成了英文——**消息是给人看的，英文在任何终端上都读得出来。**

5.10 这一节新增了什么，以及三条要写成习惯的规矩

回到 5.1 的那两棵树：现在可以对着第一个图元的**第 2、3 帧**逐个点名了。本节新增的九个例子目录，每个都带 `src/`、`include/` 与一份 `.json` 清单（清单里写着「怎么编译、跑完输出什么、退出码是多少」，所以本节每一块输出、每一段诊断、每一个退出码都不是我写的，是流水线真跑出来再与我写下的核对过的）：

- `precedence_zoo` —— 优先级三处现场（`a < b < c`、`*p++` 与 `(*p)++`、`x & 1 == 0`），外加一条 `-Wparentheses`；
- `short_circuit` —— 四组计数证明「`&&` 右边可能不算」，外加一个 `&` 解引用空指针的崩溃现场（`mode` 是 `runtime_error`，退出码 3221225477）；
- `eval_order` —— 两档输出对照，证明「本机稳定、标准不保证」；
- `seq_point` —— 两条 `-Wsequence-point` 警告、两份汇编（`-O0` / `-O2`）与一条折叠证据；
- `assign_vs_compare` —— `if (x = 0)` 的行为与 `-Wparentheses`；
- `ternary_comma` —— `?:` 的类型调整、嵌套 `?:`、逗号运算符；
- `cast_lab` —— 四种强转用途各一例；
- `static_assert_ok` / `static_assert_fail` —— 把假设写进代码，以及它失败时的诊断原文。

第二个图元的第 2 帧那五样东西是**证据的另一种形式**：抓下来的**会话**（警告与崩溃的原文）；第 3 帧那两份**汇编**证明「优化真的把那两次写折叠成了一条」。会话适合「每次都不一样的输出」（崩溃退出码、带交互的 gdb），汇编适合「让你看见编译器把哪几句改成了一句」——两者都不能被 `output` 块逐字对账，所以它们以文件的形式留在例子目录里，正文按 `file=` 注入。

工程惯例 · 工程惯例：三条该写成习惯的规矩

这一节的坑，归根到底都是「你脑子里想的」与「编译器看的」不是一回事。把它收成三条可执行的规矩：

1. **看到一行里有两个副作用，先拆。** `*p++` 这种只有一个副作用的写法没问题；同一句话里出现两个改同一对象的操作（`i = i++ + 1`、`f(i++, i++)`）就拆成两句。`-Wsequence-point` 会帮你抓一部分（本机实测它抓得到本节这两处），但**它看不见的写法照样是 UB**，所以规矩是「拆」，不是「等警告」；
2. **需要顺序的地方，用有保证的构造把它写出来。** `&&`、`||`、`?:`、逗号运算符这四处规定了**谁先算**；函数调用与每条语句的结尾有序列点（但**实参之间没有**先后，见 5.4 节）。想要「先 `f` 后 `g`」，就写成两句；想要「先判空再用」，就写 `p != NULL && *p > 0`；
3. **把平台假设写成 `_Static_assert`，别写在注释里。**注释不会编不过，断言会。

三条都不是标准逼你做的，是**给接手的人省时间**——代价只是多打几个字符、多写一行断言。

最后一条不是惯例而是本书的纪律，但值得在这里说一次：**编译器不在乎你怎么写，接手的人在乎。**本节每一个「坑」在汇编里都只是几条 `mov`，而读代码的人要在脑子里模拟一遍求值顺序。加一对括号、拆一行，成本是几个字符。

小结 · 这一节的要点

- **优先级是查表，不是推理**：`(a < b) < c`、`x & (1 == 0)`、`*(p++)` 都能编译能跑，只是算的不是你以为的东西；`*p++` 动指针，`(*p)++` 动它指的值；
- **`&&` / `||` 的短路是标准保证的**（6.5.13、6.5.14），`&` / `|` 没有——所以 `p != NULL && *p > 0` 安全（实测返回 0），`(p != NULL) & (*p > 0)` 踩死（退出码 **3221225477**）；
- **未指定与未定义是两件事**：`f() + g()` 是未指定（本机两档都先调 `f`、结果 107，但标准不承诺），修法是**把顺序写出来**；`i = i++ + 1` 是未定义（`-Wsequence-point` 报 `operation on 'i' may be undefined`，`-O0` 两次写、`-O2` 折叠成一条 `lea`），修法是**把这句删掉重写**；
- **赋值的值、`?:` 的类型、强转与断言**：`if (x = 0)` 合法且会改 `x`（`-Wparentheses` 提示），`a = b = c` 右结合；两个分支都是算术类型时，`?:` 的类型由寻常算术转换决定；浮点转整数朝零截断；`(void)` 表示故意丢弃返回值；`_Static_assert` 把平台假设变成编译期检查。

自测题 · 自测题（先自己答，再对答案）

1. `int a = 3, b = 2, c = 1;` 时，`a < b < c` 的结果是多少？为什么不是「3 不小于 2 所以为假」？同一张表里，`x & 1 == 0` 实际算的是什么？
2. `*p++` 与 `(*p)++` 各改了什么？`p` 会动吗？表达式交出来的值一样吗？
3. 用计数说清 `no_a() && yes_b()` 与 `no_a() & yes_b()` 的差别。为什么 `p != NULL && *p > 0` 在 `p` 是空指针时不会崩，而把 `&&` 换成 `&` 就会？
4. `f() + g()` 与 `i = i++ + 1` 分别属于「未指定」还是「未定义」？两者的修法有什么不同？
5. `(int)-3.99` 是多少，为什么不是 -4？`sizeof(0 ? 1 : 2.5)` 是多少，依据是哪一条规则？`_Static_assert(sizeof(int) == 8, "…")` 在本机编出来是什么级别、什么内容？

注 · 自测题答案

1. 结果是 **1（真）**。关系运算符是**左结合**的，所以 `a < b < c` 读作 `(a < b) < c`：`3 < 2` 得 0，再算 `0 < 1` 得 1。想表达「3 小于 2 且 2 小于 1」，必须写 `a < b && b < c`（它算出来是 0，这才是诚实的结果）。`x & 1 == 0` 实际算的是 `x & (1 == 0)`，也就是 `x & 0`，**恒为 0**；要判断偶数得写 `(x & 1) == 0`。
2. `*p++` 读作 `*(p++)`：**先取出 `p` 当前指向的值，再让 `p` 前进一格**，表达式交出来的是**旧值**（本机实测 10），`p` 动了。`(*p)++`：**先把 `p` 解引用，再让那个值加一**，表达式交出来的也是**旧值**（实测 20），但 `p` 一步没动、`*p` 变成了 21。两者的表达式值都是「加一之前的那个值」，区别在「被改的是指针还是它指的对象」。
3. `no_a()` 返回 0（假）：`&&` 直接给 0，**右边一次都没算**（本机实测 `b` 被调 0 次）；`&` 要把两边都算出来才能按位与，所以右边照样跑了（`b` 被调 1 次），结果同样是 0。`p != NULL && *p > 0` 安全，是因为 `p != NULL` 为假时 `&&` **根本不求值右边的** `*p`（标准 6.5.13 保证）；换成 `&` 后两个操作数都要算，右边那次解引用落在空指针上，本机进程以退出码 **3221225477** 被终止。
4. `f() + g()` 是**未指定**：程序合法，只是子表达式的求值顺序没被规定（标准 6.5 节第 3 段）。`i = i++ + 1` 是**未定义**：同一句话里对同一个对象既读又写、还写了两次，标准对这种写法不施加任何要求（3.4.3 节）。修法不同：前者

> **把顺序写出来**（拆成两句，或用 `&&` / `?:` / 逗号这类有序列点的构造）；后者**把这句删掉重写**（比如 `int old = i; i = old + 1;`）——它不能靠加括号解决。

5. `(int)-3.99` 是 **-3**。标准 6.3.1.4 规定浮点转整数是**朝零截断**（丢掉小数部分），不是向下取整，所以负数往 0 的方向靠、不是更小。`sizeof(0 ? 1 : 2.5)` 是 **8**（本机 `double` 的大小）：6.5.15 规定两个分支都是算术类型时，整个 `?:` 表达式的类型由**寻常算术转换**在它们之间决定——`int` 与 `double` 相遇取 `double`。最后那条 `_Static_assert` 在本机给出的是 **`error`（编译失败，退出码 1）**，内容是 `static assertion failed: "int is not 8 bytes on this machine"`，位置指向写断言的那一行；标准 6.7.10 要求这条诊断**必须包含字符串字面量的内容**。

第 3 章

控制流与函数

本章 2 个单元 · 页眉从这里之后的页面开始出现

1. 分支与循环：把流程写对 零基础 · 第 6 单元 · 建议 60 分钟
2. 函数：声明、定义、调用栈与 static 局部变量 零基础 · 第 7 单元 · 建议 90 分钟

分支与循环：把流程写对

◎ 学习目标 · 看完这一单元，你应该能

- 说出一段 `if … else …` 里的 `else` 到底属于哪个 `if`，并用花括号把这件事写死；
- 看着一段漏了 `break` 的 `switch`，说出程序会走到哪一行，并解释为什么 `-Wall` 不吭声、`-Wextra` 才吭声；
- 说清 `for`、`while`、`do-while` 三者的语义差，并用「每一轮结束后什么为真」检查一个循环；
- 认出差一错误的三种样子，说出循环退出时循环变量等于几；
- 用 gdb 的条件断点与 `until` 把一个循环拆开看，并说出累加大数为什么必须用 `long long`。

6.1 先看工程长什么样

前两个单元的坑都在**表达式**里：一个数是什么类型、一次运算按哪种规则算。从这一单元起，坑换了地方——它跑到**流程**里：程序下一句去执行哪一行。流程错了编译器几乎不吭声，程序照跑，只是结果不对；这一节要练的，就是把「哪一行」和「哪一轮」都钉死。

照老规矩，先看文件。这一节要建起第三个项目 `examples/ch03/`，它是全书的第 6 单元（第 3 章的第一个单元），下面有七个例子目录，每个都是一个 `src/ + include/ + 清单` 的最小工程。

目录树 **examples/：从两个项目到三个项目**

第 1 步 本节开始时，examples/ 下已经有两个项目

```
examples/
├── ch01/ : 第 1 章的项目（本节不动它）
└── ch02/ : 第 2 章的项目（本节不动它）
```

第 2 步 本节建起第三个项目，先摆上四个例子

```
examples/ : ch01、ch02 照旧
└── ch03/ ← 本课新增
    ├── branch_basics/ : 悬空 else 与花括号 ← 本课新增
    ├── switch_fallthrough/ : 漏 break 与故意贯穿 ← 本课新增
    ├── loop_offbyone/ : <= 多算一个元素 ← 本课新增
    └── stats_loop/ : 求和、最大、平均、边界 ← 本课新增
```

第 3 步 剩下三个例子到齐，一共七个

```
examples/
└── ch03/
    ├── branch_basics/
    ├── switch_fallthrough/
    ├── loop_offbyone/
    ├── stats_loop/
    ├── while_eof/ : int 与 char 接 getchar ← 本课新增
    ├── goto_multi_break/ : 两层跳出与统一收尾 ← 本课新增
    └── nested_continue/ : continue 只跳内层 ← 本课新增
```

目录树还有两帧，单独画在这里（同一个目录，换一个观察角度）：

目录树 **（续）目录树：输入数据文件与证据材料**

第 1 步 第一次出现输入数据文件（.txt，不是 .json）

```
examples/
└── ch03/
    └── stats_loop/
        ├── data-empty.txt : 0 字节的空输入 ← 本课新增
        ├── data-one.txt : 只有一行 ← 本课新增
        └── data-numbers.txt : 六行数 ← 本课新增
```

第 2 步 本节另外留下的证据材料

```
examples/
└── ch03/
    ├── session-*.txt : 九段抓下来的会话 ← 本课新增
    ├── case_const_probe.c : case 写变量的探针 ← 本课新增
    ├── switch_float_probe.c : switch 写浮点的探针 ← 本课新增
    ├── float_loop_probe.c : 三种停不下来的循环 ← 本课新增
    └── stats_loop/ : 两份汇编（-O0 与 -O2）都在这里
```

第一棵树的第 1 帧是这一节故事的起点：`examples/` 下已经有第 1 章和第 2 章那两个项目。它的第 2、3 帧是本节搭起来的第三个项目：**七个**例子目录，每个都是「源文件进 `src/`、声明进 `include/`、编译与对账的参数写在 `.json` 清单里」的最小工程（第 1 单元讲过为什么分文件、代价是什么；从第二个项目起本书不允许单文件工程）。第二棵树的第 1 帧是本节第一次出现的东西：**输入数据文件**——`stats_loop` 要读的三个 `.txt`，其中一个是 **0 字节**的空文件，专门用来考「一个元素都没有」这个边界；它的第 2 帧是本节另外留下的两类材料：九段**抓下来的会话**（编译警告、gdb、汇编对照、timeout 掐掉的死循环）与三个**探针**（它们没有清单，因为它们的证据是会话本身，不是逐字对账的输出块）。

七个例子各管一件事，后面每一节都会点名其中一个：`branch_basics` 管配对与花括号（6.2），`switch_fallthrough` 管贯穿（6.3），`stats_loop` 管循环与类型（6.4、6.6），`loop_offbyone` 管差一错误（6.5），`goto_multi_break` 与 `nested_continue` 管跳出与跳过的范围（6.7），`while_eof` 管读输入的循环（6.8）。全书的收口清单在 6.9，这里先有个印象就够。

工程惯例 · **工程惯例：为什么一个例子一个最小工程**

标准对「分几个文件」只有一个要求：一个程序有且只有一个 `main`。其余全是惯例。本书从第二个项目起要求每个例子都摆成 `src/ + include/`，理由和上一单元一样，但在这里更值钱：

- **一个例子只演示一件事**：`loop_offbyone` 里只有差一错误，`nested_continue` 里只有 `continue` 的范围。出问题时你能立刻知道该看哪里；
- **声明与定义分家**：`include/xxx.h` 只放声明，`src/xxx.c` 放定义，`src/main.c` 只负责把输入喂进去、把结果打出来；
- **清单把编译参数固定下来**：这些例子的编译命令一律是 `gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/<例名>.c -o app` ——`-Iinclude` 让编译器找到头文件，其余参数与第 1、2 单元完全一样（普通 `-I` 参数可以进清单，`-w` / `-Wno-*` 这类**减警告**的参数不允许）。

代价是每次都要多写两个文件。这笔账在第 17 单元（多文件工程）会彻底划算，现在先照着摆。

6.2 if 与 else：配对、花括号、真值规则

先给结论：**在一段 `if` 里，`else` 属于哪一个 `if`，由语法决定，跟你怎么缩进完全没有关系。**下面这段程序把同一件事写了两遍，一遍不加花括号、一遍加花括号：

c examples/ch03/branch_basics/src/branch_basics.c 第 3–21 行

```c
const char *classify_dangling(int a, int b) {
    if (a > 0)                    // 这个 if 后面没有花括号
        if (b > 0)
            return "两个都为正";
    else                          // 这个 else 属于紧邻的那个 if
        return "a 非正";
    return "a 正、b 非正";
}

const char *classify_braced(int a, int b) {
    if (a > 0) {
        if (b > 0) {
            return "两个都为正";
        }
    } else {                      // 同样的判断，加上花括号
        return "a 非正";
    }
```

```
20        return "a 正、b 非正";
21    }
```

4 这个 if 后面没有花括号
7 这个 else 属于紧邻的那个 if
17 同样的判断，加上花括号

别被缩进骗了：第 4 行的 `if` 没有花括号，它的「身体」只有紧跟的那**一条**语句（第 5 到 6 行那个 `if`）；第 7 行的 `else` 就属于**里面那个** `if`。所以当 `a` 不是正数时，外层 `if` 直接跳过，那个 `else` 也跟着一起被跳过——程序从第 9 行返回。真跑出来的结果是这样：

终端输出 终端输出 branch-basics-out

```
a= 1 b= 1  悬空版：两个都为正；花括号版：两个都为正
a= 1 b=-1  悬空版：a 非正；花括号版：a 正、b 非正
a=-1 b= 5  悬空版：a 正、b 非正；花括号版：a 非正
n=-3  没花括号：  10；有花括号：   5
n= 3  没花括号：  16；有花括号：  16
真值：0 假、非 0 真 -> if (0) 假，if (-1) 真，if (0.1) 真，if ('0') 真，if (0.0) 假
if (x = 0.1) 进了真分支：x 现在是 0.1（这是赋值，不是比较）
sign_name(-5) = 负数
sign_name( 0) = 零
sign_name( 7) = 正数
```

第二行最值得盯着看：输入 `a=1、b=-1`，**悬空版**说「a 非正」——`a` 明明是 1，这是错的。这就是**悬空 `else`**（dangling else）。标准对这件事的规定只有一句话：

标准原文 · 标准原文：`else` 归谁，真值怎么算

C 语言标准（ISO/IEC 9899，C17）6.8.4.1 节「The if statement」的语义部分：

> In both forms, the first substatement is executed if the expression compares unequal to 0. In the else form, the second substatement is executed if the expression compares equal to 0.
>
> An else is associated with the lexically nearest preceding if that is allowed by the syntax.

大意：两种形式下，只要表达式的值**不等于 0** 就执行第一个子语句；`else` 形式里，表达式**等于 0** 时执行第二个子语句。而 `else` 只跟**按语法来说最近的那个还没有 `else` 的 `if`** 配对——标准说的是「语法上最近」，不是「看起来最近」。

出处：ISO/IEC 9899:2018（C17）§6.8.4.1。

这段话同时回答了本节两个问题。第一，`else` 归谁：**最近的、还没配对的那个 `if`**，缩进不参与判断。第二，真值规则：**0 假、非 0 真**，跟类型没关系——上面输出里那行真值表就是这条规则的实物：

- `if (0)` 假、`if (0.0)` 假——值为 0 就是假；
- `if (-1)` 真——**负数也是非 0**，所以是真；
- `if (0.1)` 真——浮点只要不是 0 就是真；
- `if ('0')` 真——这一条最容易看错：`'0'` 是**字符常量**，它的值是这个字符在执行字符集里的编码（本机是 **48**），不是数字 0。

⚠ 陷阱 · `if (x = 0.1)`：一次赋值被你当成了比较

看输出最后一行，再回头看源码：`if (x = 0.1)` 里只有一个等号，它是**赋值**：先把 0.1 存进 `x`，再拿 `x` 的新值（0.1）当条件——非 0，所以进了真分支。而写这段代码的人多半想写的是 `if (x == 0.1)`。

你会怎么被坑到：这段代码**能编译**，只是多一条警告（下面的编译会话里就贴着这条警告的原文）。新手看到警告一划而过，程序里那个「永远进真分支」的分支就一直留着，直到某天发现「这个判断根本没起作用」。

怎么防：一是习惯——条件里只要出现单个 `=` 就停下来数一遍；二是让编译器替你数——本书固定打开的 `-Wall` 里包含 `-Wparentheses`，它会说 `suggest parentheses around assignment used as truth value`（建议给「当条件用的赋值」加括号）。它的意思不是「加个括号就对了」，而是「这里看着像写错了」：真的想一边赋值一边判断，才写 `if ((x = 0.1))`。

缩进骗人的地方不止 `else` 的归属。下面这个函数想做的事是「`n` 是正数就把 `r` 加上 `n` 再翻倍」，作者少写了花括号：

c examples/ch03/branch_basics/src/branch_basics.c 第 23–38 行

```c
int double_without_braces(int n) {
    int r = 5;
    if (n > 0)          没有花括号，if 只管一条语句
        r += n;
        r *= 2;         /* 这一行缩进得像在 if 里，但它不在 */  这一行不在 if 里，永远执行
    return r;
}

int double_with_braces(int n) {
    int r = 5;
    if (n > 0) {        加上花括号才是本意
        r += n;
        r *= 2;
    }
    return r;
}
```

25 没有花括号，if 只管一条语句
27 这一行不在 if 里，永远执行
33 加上花括号才是本意

对照输出里那两行：`n = -3` 时没花括号的版本给出 **10**（`5` 也翻倍了），有花括号的给出 **5**；而 `n = 3` 时两者都是 16——**这个 bug 偏偏在条件为真的时候看不出来**，只有在 `n <= 0` 时才露馅。这就是「缩进骗人」最典型的形态：你读到的控制流和编译器读到的不是一回事。

编译器这次也说话了。下面两块是同一段代码的编译会话原文（一次编译，警告按出现顺序排在同一个文件里），第一块是别人替我们数出来的那处赋值，第二块是悬空 `else` 与缩进骗人：

终端会话 一条编译命令，三条警告（第 1 块：赋值当条件） c06-branch-warn-1

```
$ cd examples/ch03/branch_basics
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/branch_basics.c -o app.exe
src/main.c: In function 'main':
src/main.c:23:9: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
   23 |     if (x = 0.1) {
      |         ^
```

终端会话 （续）第 2 块：悬空 else 与缩进骗人 c06-branch-warn-2

```
src/branch_basics.c: In function 'classify_dangling':
src/branch_basics.c:4:8: warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
    4 |     if (a > 0)
      |        ^
src/branch_basics.c: In function 'double_without_braces':
src/branch_basics.c:25:5: warning: this 'if' clause does not guard... [-Wmisleading-indentation]
   25 |     if (n > 0)
      |     ^~
src/branch_basics.c:27:9: note: ...this statement, but the latter is misleadingly indented as if it were
guarded by the 'if'
   27 |         r *= 2;         /* 这一行缩进得像在 if 里，但它不在 */
      |         ^
$ echo $?
0
```

三条警告的开关名连起来读，正好是本节的三件事：`-Wparentheses`（赋值当条件）、`-Wdangling-else`（悬空 `else`）、`-Wmisleading-indentation`（缩进骗人）。注意最后那个 `$ echo $?` 打出来的 **0**：编译**成功**了，程序照样跑——**这三条都只是警告**。警告的全部价值在于「它知道你想干什么，而你知道结果不对」，所以要把它当信号看。

> **大白话** · **花括号的代价是零，收益是「我读到的就是编译器读到的」**
>
> `if` 后面的花括号不是语法要求（一条语句时可以不写），它是一笔**零成本的保险**：多打两个字符，换来的是「控制流的边界在纸上看得见」。真的只有一条语句、而且短到不会看错时，不写也没关系；**只要有第二个语句、只要有人可能往里加语句，就必须写**——因为后加的那一句永远不会被认为在 `if` 里，而读代码的人恰恰会这么以为。
>
> 回到严谨说法：`if (表达式) 语句` 里的「语句」是**一条**语句；`{ … }` 是**复合语句**，在语法上算作**一条**语句——所以花括号不是「多了一层」，而是「把好几条打包成一条」。

- **不止两条分支时：`else if` 是一条链，不是一个关键字**

C 里**没有 `else if` 这个关键字**。你写的 `else if` 实际是「`else` 后面又跟了一条 `if` 语句」——`if` 语句本身算一条语句，所以 `else` 接它是合法的。于是整条链在语法上仍然是一层套一层，配对规则还是那句「就近」：

c examples/ch03/branch_basics/src/branch_basics.c 第 40–48 行

```c
const char *sign_name(int n) {
    if (n > 0) {
        return "正数"; 外层 if 先判断
    } else if (n < 0) {
        return "负数"; 这里的 else 属于外层的 if
    } else {
        return "零";
    }
}
```

42 外层 if 先判断
44 这里的 else 属于外层的 if

驱动它的是一小段循环（`main.c` 的最后几行）：

c examples/ch03/branch_basics/src/main.c 第 27–31 行

```c
    const int signs[3] = {-5, 0, 7};
    for (int i = 0; i < 3; i++) { 三个样本：负、零、正
        printf("sign_name(%2d) = %s\n", signs[i], sign_name(signs[i]));
    }
    return 0;
```

28 三个样本：负、零、正

输出块的**最后三行**就是答案：`sign_name(-5) = 负数`、`sign_name( 0) = 零`、`sign_name( 7) = 正数`。链有两件事要记住：

- **第一个为真的分支赢，后面的不再判断。** `n = 7` 只走第一个分支，后面的 `else if` / `else` 根本不会求值（这和第 5 单元讲的 `&&` 短路是同一个思路）；
- **最后一个 `else` 也是「就近」配的。**链里再加一个 `if`，它自己的 `else` 会先被配掉——所以链越长，越要给每条分支写花括号。

6.3 switch：整型表达式、常量、贯穿

`switch` 的坑比 `if` 更安静：它能编译、能跑、还经常「看起来对」。这一节的例子是一位「分数线」函数和一位「代码转名字」函数，先看后者的前半段：

c examples/ch03/switch_fallthrough/src/switch_fallthrough.c 第 3–21 行

```c
const char *kind_missing_break(int code) {
    const char *name = "未知";
    switch (code) {
    case 1:
        name = "一";        /* 这里少了一个 break */ 这里少了一个 break
    case 2:
        name = "二";
        break;
```

```
11      case 3:
12          name = "三";
13          /* fall through */ 把「故意贯穿」写成了注释
14      case 4:
15          name = "三或四";
16          break;
17      default:
18          break;
19      }
20      return name;
21  }
```

7 这里少了一个 break
13 把「故意贯穿」写成了注释

`switch` 的规则一句话：**拿 `code` 跟每个 `case` 的值比，相等就从那一行开始往下执行，一直执行到 `break` 或者 `switch` 的末尾。**所以第 7 行少了 `break`，执行完 `name = "一"` 之后**不会停**，它会继续把第 8 行的 `case 2` 那条语句也执行掉——这叫**贯穿**（fallthrough）。真跑起来是这样的：

终端输出 终端输出 switch-fallthrough-out

```
kind_missing_break(1) = 二    <- 本想拿到「一」
kind_missing_break(3) = 三或四    <- 这一处是故意的
kind_missing_break(9) = 未知    <- default 收尾
kind_on_purpose(4)    = 三或四（空 case 叠在一起，没有贯穿）
kind_on_purpose(7)    = 其它
grade_points('A') = 4
grade_points('B') = 3
grade_points('C') = 2
grade_points('D') = 1
grade_points('E') = -1
grade_points('F') = 0
grade_points('Z') = -1    <- default 兜底
kind_default_first(1) = 一
kind_default_first(9) = 未知    <- default 在最前面也能兜住
```

第一行就是漏 `break` 的代价：输入 1，拿到的是「二」。第三行是故意贯穿（第 13 行那句 `/* fall through */` 是本例故意留的标记）：输入 3 会一路执行到 `case 4`，拿到「三或四」。**贯穿本身不是错，忘写 `break` 才是**——区别只在「你是不是故意的」。

编译器抓得住「忘写」这一类。下面是同一个文件的两条编译命令：第一条只有 `-Wall`，第二条加上 `-Wextra`：

终端会话 -Wall 不吭声，加 -Wextra 才报出贯穿 c06-fallthrough

```
$ cd examples/ch03/switch_fallthrough
$ gcc -std=c17 -Wall -Wpedantic -O0 -g -Iinclude -c src/switch_fallthrough.c -o sf.o
$ echo $?
0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -c src/switch_fallthrough.c -o sf.o
src/switch_fallthrough.c: In function 'kind_missing_break':
src/switch_fallthrough.c:7:14: warning: this statement may fall through [-Wimplicit-fallthrough=]
    7 |         name = "一";        /* 这里少了一个 break */
      |         ~~~~~^~~~~
src/switch_fallthrough.c:8:5: note: here
    8 |     case 2:
      |     ^~~~
$ echo $?
0
```

三件事要看清楚：① 第一条命令一个字都不说（`-Wimplicit-fallthrough` 属于 `-Wextra`，不属于 `-Wall`）；② 第二条命令给出了位置（第 7 行）与去向（`note: here` 指着第 8 行），这两行一起读就是「这一句会掉到下面去」；③ **两条命令的退出码都是 0**——警告不是错误，程序照样生成。

下面这份诊断是**加 `-Werror` 之后**的同一个编译（清单 `switch_fallthrough_werror.json` 就是这么配的）。`-Werror` 的意思是「把警告当错误」，所以那句警告变成了 `error`，编译真的失败：

编译诊断 加上 -Werror 之后：警告变成编译失败（原文照抄） switch-fallthrough-diag

```
src/switch_fallthrough.c: In function 'kind_missing_break':
src/switch_fallthrough.c:7:14: error: this statement may fall through [-Werror=implicit-fallthrough=]
    7 |         name = "一";        /* 这里少了一个 break */
      |         ~~~~~^~~~~
src/switch_fallthrough.c:8:5: note: here
    8 |     case 2:
      |     ^~~~
cc1.exe: all warnings being treated as errors
```

对比两块诊断：开关名从 `[-Wimplicit-fallthrough=]` 变成了 `[-Werror=implicit-fallthrough=]`，末尾多了一句 `cc1.exe: all warnings being treated as errors`（`cc1` 是 gcc 真正做编译的那一步）。**同一份源码、同一个警告**，加不加 `-Werror` 只影响「它算不算失败」——这就是「编译器多半不吭声」这句话的准确含义：它**知道**，只是默认不拦你。

{} 语法规则 · **`switch` 的形式与四条规矩**

形式是 `switch (整型表达式) { case 常量: 语句 … default: 语句 … }`。四条规矩：

1. 括号里必须是**整型**表达式（`int`、`char`、`enum`… 都行，**浮点与字符串不行**）；
2. 每个 `case` 后面只能跟**整数常量表达式**——写变量、写函数调用都不行；
3. `case` 的值必须互不相同（同一个 `switch` 里不许两个 `case` 撞值）；
4. `default` 最多一个，**位置由你定**：写在最后、写在中间、写在第一个都合法（写法不同只影响「没有匹配时从哪里开始往下执行」）。

规矩 2 值得单独抓一次现场。下面这个探针只有一行错——`case` 后面写了一个变量：

c 探针：case 后面写变量（没有清单） examples/ch03/case_const_probe.c 第 5–15 行

```c
int main(void) {
    int n = 2;
    switch (n) {
    case n:
        puts("case 必须是常量");
        break;
    default:
        break;
    }
    return 0;
}
```

终端会话 case 后面写变量的真实诊断 c06-case-const

```
$ cd examples/ch03
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g case_const_probe.c -o probe.exe || echo "退出码 $?"
case_const_probe.c: In function 'main':
case_const_probe.c:8:5: error: case label does not reduce to an integer constant
    8 |     case n:
      |     ^~~~
退出码 1
```

这句话里 `reduce to an integer constant` 的意思就是「折算不成一个整数常量」。标准把这两条写成了硬要求：

标准原文 · 标准原文：`switch` 要什么、`case` 要什么

> C 语言标准（ISO/IEC 9899，C17）6.8.4.2 节「The switch statement」的约束部分：
>
> > The controlling expression of a switch statement shall have integer type.
> >
> > The expression of each case label shall be an integer constant expression and no two of the case constant expressions in the same switch statement shall have the same value after conversion.
>
> 大意：`switch` 括号里的控制表达式**必须是整型**；每个 `case` 的表达式必须是**整数常量表达式**，而且同一个 `switch` 里任何两个 `case` 折算之后的值**不许相同**。
>
> 出处：ISO/IEC 9899:2018（C17）§6.8.4.2 第 1、2 段。

最后看 `default` 的位置与「分数线」函数的下半段。`grade_points` 只列了 A、B、C、D、F，`E` 没有列——输出里 `grade_points('E') = -1`，说明它落到了 `default`：

c examples/ch03/switch_fallthrough/src/switch_fallthrough.c 第 33–48 行

```c
int grade_points(char g) {
    switch (g) {
    case 'A': case 后面是字符常量，也就是整数
        return 4;
    case 'B':
        return 3;
    case 'C':
        return 2;
    case 'D':
        return 1;
    case 'F':
        return 0;
    default: 没列 E：它会落到这里
        return -1;          /* default 写在最后，位置由你定 */ default 写最后，位置由你定
    }
}
```

35 case 后面是字符常量，也就是整数
45 没列 E：它会落到这里
46 default 写最后，位置由你定

`default` 的位置真的可以动。同一个文件里还有一个把 `default` 写在**最前面**的版本：

c examples/ch03/switch_fallthrough/src/switch_fallthrough.c 第 50–59 行

```c
const char *kind_default_first(int code) {
    switch (code) {
    default:                /* 放在最前面：没有匹配时从这里开始 */ default 在最前面
        return "未知"; 没有匹配时从这里开始
    case 1:
        return "一";
    case 2:
        return "二";
    }
}
```

52 default 在最前面
53 没有匹配时从这里开始

输出块的最后两行是 `kind_default_first(1) = 一` 与 `kind_default_first(9) = 未知`——`1` 照样匹配 `case 1`（`default` 放哪儿不影响匹配），`9` 没有匹配就从 `default` 开始。**位置是自由的，但这份自由有前提**：每一支都自己收尾（`return` / `break`）。一旦 `default` 不自己收尾，它就会掉进下面的 `case`——那正是本节开头那个漏 `break` 的坑，只是这次坑开在 `default` 上。

工程惯例 · **工程惯例：故意的贯穿必须留下痕迹**

这一条是惯例，标准里没写。上面那句 `/* fall through */` 不是给我看的，是给编译器与下一个读代码的人看的：gcc 在 `-Wimplicit-fallthrough=3` 这一档下认得几种固定写法（英文的 `fall through` / `fallthrough` / `FALLTHRU` 等），写了它就闭嘴；**写中文注释它不认**——这也是为什么本例故意的贯穿写英文、忘写 `break` 的那处写中文。

工程惯例 · **（续）工程惯例：`default` 要么有、要么写注释，位置按「最后」写**

这一条同样是惯例。`default` 的作用不是「必须有」，而是「没被我列到的值怎么办」这件事要在代码里看得见：查表型的 `switch`（分数线、状态名）建议一律写上 `default`，而且末尾要能回答「走到这里意味着输入不合法」还是「走到这里意味着我漏了一类」。

为什么不推荐把它放最前面：单个 `default` 配 5 个 `case` 时，放最后，读者一眼就知道「其余都走这里」；放最前面，读者要读到最后才发现「哦，没有匹配是回到第一行开始的」——两种写法都对，但一种更好读。代价也说清楚：`grade_points` 的 `default` 返回 -1，`'E'` 因此得到一个**看得见的结果**；要是删掉 `default`，`'E'` 会走出 `switch`、走到函数末尾那个 `}`——那是一个没有 `return` 的终点，调用者一旦用到这个返回值，按 6.9.1 节就是**未定义行为**（不是「返回一个大概的垃圾值」）。

最后补规矩 1 的现场：控制表达式必须是**整型**。用浮点试：

c **探针：switch 的控制表达式写成浮点（没有清单）** examples/ch03/switch_float_probe.c 第 5–15 行

```c
int main(void) {
    double x = 1.5;
    switch (x) {
    case 1:
        puts("不会编译通过");
        break;
    default:
        break;
    }
    return 0;
}
```

终端会话 浮点做控制表达式的真实诊断 c06-switch-float

```
$ cd examples/ch03
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g switch_float_probe.c -o sfp.exe || echo "退出码 $?"
switch_float_probe.c: In function 'main':
switch_float_probe.c:7:13: error: switch quantity not an integer
    7 |     switch (x) {
      |             ^
退出码 1
```

`switch quantity not an integer`——翻译过来就是「`switch` 的这个量不是整数」。注意它是 `error` 不是 `warning`：**编译直接失败，退出码 1**，没有「跑起来看看」的机会。字符串同理（`switch (name)` 会报 `switch quantity not an integer` 或指针相关的错），原因一样：`case` 比的是整数，字符串没有「等于」这回事（它比的是首地址）。

拿它和本节开头那个 `case n` 的探针对照：一个考 `switch (…)` 这一侧（规矩 1），一个考 `case` 那一侧（规矩 2）。两条规矩各有一句诊断，**都拦在编译期**——这一处比 `if` 那一节幸运得多。

6.4 三种循环的语义差与循环不变式

三种循环长得像，语法上差得可不少。先看形式：

{} 语法规则 · 三种循环：`for` / `while` / `do-while`

- `while（条件）语句`：**每一轮开始前**判断条件，为假就一轮都不跑；
- `for（初始化；条件；每轮之后）语句`：三段各管一段——第一段只在进入循环前跑**一次**，第二段在**每一轮开始前**判断，第三段在**每一轮结束之后**跑；
- `do 语句 while（条件）;`：**先执行一轮，再看条件**——所以循环体**至少执行一次**，这一条是它和前两者的全部区别。

顺序记牢这一条就不会混：`while` 与 `for` 都是「**先看后做**」，`do-while` 是「**先做后看**」。另外 `break` 能用在三者里，`continue` 也可以——但 `for` 的 `continue` **会照跑第三段**（`i++` 不会被跳过），这是它和 `while` 里 `continue` 最容易搞混的地方。

拿一个最短的循环把上面这些逐条落地。这是 `stats_loop` 里那个求和函数：

c examples/ch03/stats_loop/src/stats_loop.c 第 8–15 行

```c
long long sum_upto(int n) {
    long long total = 0;
    int i;
    for (i = 1; i <= n; i++) {   // 进入每一轮前判断
        total += i;              // 每一轮结束前执行这一句
    }
    return total;
}
```

11 进入每一轮前判断
12 每一轮结束前执行这一句

它是 `1 + 2 + … + n`。把 `n = 4` 的四轮摊开，每一步的变量值是这样的：

图 / 搜索过程 **sum_upto(4)：四轮循环，每一轮结束后什么为真**

第 1 步 进入循环之前

`i 还没有值：初始化还没跑` `total = 0：第 0 轮`

第 2 步 第 1 轮结束，i 变成 2

`i = 2：刚才那一轮用的是 i = 1` ← 本步新增 **`total = 1：1`** ← 本步新增

第 3 步 第 2 轮结束，i 变成 3

`i = 3` ← 本步新增 **`total = 3：1 + 2`** ← 本步新增

第 4 步 第 3 轮结束，i 变成 4

`i = 4` ← 本步新增 **`total = 6：1 + 2 + 3`** ← 本步新增

第 5 步 第 4 轮结束，i 变成 5

`i = 5：4 也已经加完了` ← 本步新增 **`total = 10：1 + 2 + 3 + 4`** ← 本步新增

第 6 步 条件 i <= 4 为假，循环退出

`i = 5：退出时 i 是 5，不是 4` `total = 10：返回值`

第 2 到 5 帧都是「这一轮跑完、`i++` 也做完了」的时刻，每一帧里都有一句话是真的：**「`total` 等于 1 加到 `i-1` 的和」**（第 2 帧 `i=2`、`total=1`；第 5 帧 `i=5`、`total=10`）。这句话就是**循环不变式**（loop invariant）：

大白话 · 循环不变式：每一轮结束之后，什么话一定是真的

不用记术语，它就是一个提问方式：**「每一轮跑完之后，我能拍胸脯说哪句话是真的？」**

拿 `sum_upto` 举例。你盯着第 11 到 13 行问自己：**每一轮开始、还没执行 `total += i` 的那一刻，`total` 是不是 1 到 `i-1` 的和？** 第 1 轮开始时 `i = 1`、`total = 0`，成立。假设某一轮开始时它成立，这一轮 `total += i` 之后 `total` 就成了 1 到 `i` 的和；接着第三段 `i++` 把 `i` 变成 `i+1`，于是「`total` 是 1 到（新 `i`）减 1 的和」这句话在**下一轮开始时仍然成立**——这就是归纳那一步。再问两个边界：**开始之前**它真的吗（`total = 0`、`i = 1`，真的）；**循环退出时**它有用吗（退出时 `i` 是 `n+1`，代入就是「`total` 是 1 到 `n` 的和」，正是返回值）。三句话都对，这个循环就对了。

大白话 · （续）把「第几轮」和 `i` 的值分开

上面图里第 2 帧是「第 1 轮跑完、`i++` 也做完了」的时刻，所以 `i` 已经是 2、`total` 是 1。「第 `k` 轮结束时 `total` 是 1 到 `k` 的和」与「`i = k+1` 时 `total` 是 1 到 `i-1` 的和」说的是同一件事，只是写法不同——把两种说法混着用，公式就会差一位（差一错误的另一种写法）。

它的好处是**不靠跑**：一个循环要跑几百万轮你也得先相信它是对的。差值 1、少加一项、多加一项，几乎都会让「每一轮开始时的那句话」写不出来——写不出来的地方，就是 bug 住的地方。6.5 的差一错误马上会用到它。

上面这六个帧里的数字不是编的：`sum_upto(4)` 在真跑时返回 **10**，`sum_upto(10)` 返回 **55**，两条都在 6.6 那节的输出块里。循环退出时 `i` 是几，也有一个实测的现场——6.8 的 gdb 会话里，循环跑完之后 `p i` 打出的是 **11**（`n = 10`）：**退出时循环变量比最后一个有效值大 1**，这句话请记到形成条件反射。

`do-while` 的「至少执行一次」不是小事，先把它真跑出来看。`loop_offbyone` 里有一对函数，做的是同一件「跑 n 轮」的事，只差 `while` 与 `do-while`：

c examples/ch03/loop_offbyone/src/loop_offbyone.c 第 59–75 行

```c
int rounds_while(int n) {
    int rounds = 0;
    while (n > 0) {              /* 每一轮开始前判断：n 是 0 就一轮都不跑 */ 先看后做：n 为 0 就一轮不跑
        rounds++;
        n--;
    }
    return rounds;
}

int rounds_do(int n) {
    int rounds = 0;
    do {                         /* 先执行一轮，再看条件 */ 先做后看：至少跑一轮
        rounds++;
        n--;
    } while (n > 0);
    return rounds;
}
```

61 先看后做：n 为 0 就一轮不跑
70 先做后看：至少跑一轮

结果在 6.5 的输出块里：`n=0: while 跑 0 轮, do-while 跑 1 轮`，`n=3: while 跑 3 轮, do-while 跑 3 轮`——**`n = 0` 是唯一能把两者分开的输入**，其余输入它们跑得一样多。

这条差别在「读输入」的循环里会咬人：如果上来就用 `do { … } while (fscanf(…) == 1)` 读数据，**空文件也会被读一次**——`fscanf` 一个数都没读到，`value` 还是它出生时的样子；如果它是个没给初值的自动变量，这一次读取本身就是**未定义行为**（6.3.2.1 节）。所以读输入的循环几乎总写成 `while (读到了) { … }`，让「空」这件事在第一次判断时就落到退出那一支。标准对这三种循环的求值时机写得非常明确：

> 标准原文 · 标准原文：`do-while` 在每轮之后才判断
>
> C 语言标准（ISO/IEC 9899，C17）6.8.5.2 节「The do statement」：
>
> > The evaluation of the controlling expression takes place after each execution of the loop body.
>
> 大意：`do-while` 的控制表达式在**每次执行完循环体之后**求值——先做后看，所以循环体至少执行一次。
>
> 出处：ISO/IEC 9899:2018（C17）§6.8.5.2。

对照来看，6.8.5.1 节「The while statement」与 6.8.5.3 节「The for statement」里的判断都发生在**每一轮之前**（`while` 是「进入循环体之前」，`for` 是「每次执行循环体之前」求值第二段）。后者的同一节还给了一条容易忽略的规定：

> 标准原文 · 标准原文：`for` 省掉第二段，等于永真
>
> C 语言标准（ISO/IEC 9899，C17）6.8.5.3 节「The for statement」：
>
> > An omitted expression-2 is replaced by a nonzero constant.
>
> 大意：三段里中间那一段**省略不写**时，等于填了一个「恒不为 0」的常量——所以 `for (;;)` 是**故意写出来的死循环**，不是笔误，它只能靠 `break` / `return` / `goto`（跳去同一个函数里的标签）出来，`exit` 则是干脆结束整个进程。
>
> 出处：ISO/IEC 9899:2018（C17）§6.8.5.3。

这就是 6.8 要讲的「不终止」的**第三种**成因（条件被写成恒真）的根源：条件由你写，写错了（或者省掉了）循环就永远不进「退出」这一步。

6.5 差一错误：`<` 与 `<=`、从 0 还是 1 开始

差一错误（off-by-one）是循环里最常见的错，也是本书唯一要求你「先手算、再跑」的那一类。看这个例子：数据数组有 8 个数，要求「前 `n` 个的和」，三种写法放在一起：

c examples/ch03/loop_offbyone/src/loop_offbyone.c 第 3–17 行

```c
int sum_first_ok(const int *data, int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {          从 0 开始配 <：正好 n 个
        total += data[i];
    }
    return total;
}

int sum_first_le(const int *data, int n) {
    int total = 0;
    for (int i = 0; i <= n; i++) {      /* 多跑了一轮 */      从 0 开始配 <=：多算一个
        total += data[i];
    }
    return total;
}
```

5 从 0 开始配 <：正好 n 个
13 从 0 开始配 <=：多算一个

第三种写法把起点也挪了：

c （续）起点与终点一起错的那种 examples/ch03/loop_offbyone/src/loop_offbyone.c 第 19–25 行

```c
int sum_first_from1(const int *data, int n) {
    int total = 0;
    for (int i = 1; i <= n; i++) {      /* 起点错、终点也错 */
        total += data[i];
    }
    return total;
}
```

三种写法的结果，连同**手算的期望值**一起打了出来——期望值不是程序算的，是我写在源码里的一个数组：

c examples/ch03/loop_offbyone/src/main.c 第 5–22 行

```c
    /* 数组开成 8 个，而下面只用前 4 个：
       这样「多算一个」只是算错，不会踩到越界（越界那件事在第 4 章讲）。 */
    const int data[8] = {10, 20, 30, 40, 50, 60, 70, 80};
    const int expect[4] = {0, 10, 30, 60};   /* 手算：前 0/1/2/3 个的和 */ 期望值是我手算后写死的

    printf("data = {10, 20, 30, 40, 50, 60, 70, 80}\n");
    for (int n = 0; n <= 3; n++) { n 从 0 起的第一行
        printf("n=%d 手算期望=%2d  i<n 得 %2d  i<=n 得 %2d  i=1;i<=n 得 %2d\n",
               n, expect[n], sum_first_ok(data, n), sum_first_le(data, n),
               sum_first_from1(data, n));
    }

    printf("轮数：i=0;i<10 跑 %d 轮，i=0;i<=10 跑 %d 轮\n", rounds_lt(10), rounds_le(10));
    printf("1..10 的和：i=1;i<=n 得 %lld，i=1;i<n 得 %lld\n",
           sum_1_to_n_le(10), sum_1_to_n_lt(10));
    printf("n=0：while 跑 %d 轮，do-while 跑 %d 轮\n", rounds_while(0), rounds_do(0));
    printf("n=3：while 跑 %d 轮，do-while 跑 %d 轮\n", rounds_while(3), rounds_do(3));
    printf("整组 8 个（n 等于容量）：和 = %d\n", sum_first_ok(data, 8));
```

8 期望值是我手算后写死的

11 n 从 0 起的第一行

终端输出 终端输出 loop-offbyone-out

```
data = {10, 20, 30, 40, 50, 60, 70, 80}
n=0 手算期望= 0  i<n 得  0  i<=n 得 10  i=1;i<=n 得  0
n=1 手算期望=10  i<n 得 10  i<=n 得 30  i=1;i<=n 得 20
n=2 手算期望=30  i<n 得 30  i<=n 得 60  i=1;i<=n 得 50
n=3 手算期望=60  i<n 得 60  i<=n 得 100  i=1;i<=n 得 90
轮数：i=0;i<10 跑 10 轮，i=0;i<=10 跑 11 轮
1..10 的和：i=1;i<=n 得 55，i=1;i<n 得 45
n=0：while 跑 0 轮，do-while 跑 1 轮
n=3：while 跑 3 轮，do-while 跑 3 轮
整组 8 个（n 等于容量）：和 = 360
```

把四列对着读，三种错法各有一条铁证：

- `i <= n` 这一列**每一行都比期望值多一个元素**（n=0 时最刺眼：期望 0，它给出 10——把第一个元素加进来了）；而且**多到的轮数永远是 1**，所以它比期望多的是 `data[n]` 这个值。
- `i = 1; i <= n` 这一列是「两头错」：它跳过了 `data[0]`，又补上了 `data[n]`。n=1 时期望 10，得到 20（跳过 10、加上了 20）；n=0 时期望 0，它得到 0——**但这种「碰巧对」最危险**，因为它会在别的 n 上错。
- `1..10` 那一行（输出块里倒数第四行）是「同样的 `<=` 换个起点就是对的」：`1..10` 的和用 `i = 1; i <= n` 得 **55**（对：1+…+10），用 `i = 1; i < n` 得 **45**（少加了 10）。`<=` **本身不是错**——错的是「起点、终点、轮数」这三件事没有对上。
- 输出块最后三行（两行 `do-while` 轮数对照 + 一行整组求和）分别是 6.4 与本节「最大规模」那一段的证据，不是差一错误的部分。

工程惯例 · **工程惯例：空与一个，是最先该试的两个输入**

这三件套是循环的例行体检，**是工程惯例、不是标准**（标准没规定你必须试这三个边界，它只是不会替你试）。任何循环在交出去之前都该问一遍：

- **空**：`n = 0` 会怎样？`sum_first_ok` 给出 0（对），`sum_first_le` 给出 10（错）——空输入最能暴露差一错误，因为循环本该一轮都不跑；
- **一个**：`n = 1` 会怎样？这时「跳过第 0 个」与「多算一个」两种错会撞在一起，是检验起点错没错的最好样本。

工程惯例 · （续）工程惯例：最大规模，以及先手算期望值

- **最大规模**：这里的最大规模是数组容量（`data` 有 8 个）。本例前三列故意**只用到前 4 个**，所以「多算一个」只是算错；输出块最后一行是真跑到最大规模的那一次：`整组 8 个（n 等于容量）：和 = 360`——它用的是正确的 `i < n`，`n` 正好等于容量也不越界（10+20+…+80 = 360）。反过来，如果这里写 `i <= n`，它就会去读 `data[8]`——**那已经是越界**了（第 4 章讲），所以真代码里 `i < 数组长度` 这个写法要形成肌肉记忆。

先手算期望值是这套体检里最被跳过、又最值钱的一步：期望值写下来，你才知道程序「应该」输出什么；不写，你就只能盯着程序自己的输出点头。本例的 `expect[]` 就是我手算的 0 / 10 / 30 / 60，写死在源码里跟程序对比。

⚠ 陷阱 · 「从 0 开始」和「从 1 开始」是两种不同的生物

差一错误之所以难，是因为**同一个 `<=` 在两个形状里一错一对**：

- **从 0 开始 + `< n`**：跑 `n` 轮，用到的下标 0…n-1——这是「数组前 n 个」的标准形状；
- **从 1 开始 + `<= n`**：跑 `n` 轮，用到 1…n——这是「1 到 n 的和」的标准形状。

你怎么会被坑到：这两个形状长得像、轮数也一样，于是你把「从 1 开始」的终点抄到「从 0 开始」的循环里，程序不报错，只是多算一个（或多读一个）。判据只有一条：**先定「我要哪些下标」，再决定起点与比较符**，而不是反过来先想起点。写完再补一句「循环退出时 `i` 是几」——从 0 开始配 `< n`，退出时 `i == n`；配 `<= n`，退出时 `i == n + 1`。这两句一写，错没错立刻看得见（6.8 的 gdb 会话里 `p i` 打出 11 就是这个道理）。

6.6 输入驱动的循环：数据文件、空输入与类型选择

真实程序的循环很少「跑固定次数」，多半是「读到没有为止」。这一节的例子 `stats_loop` 要读三个数据文件，把每个文件里的数求和、求最大、求平均；三个文件是**故意挑的**：一个是空文件、一个只有一行、一个六行。先看读文件的那个循环：

c examples/ch03/stats_loop/src/stats_loop.c 第 25–38 行

```c
int read_numbers(const char *path, int *out, int cap) {
    FILE *fp = fopen(path, "r");  打不开时给回空指针
    if (fp == NULL) {  所以这里必须判一下
        return -1;              /* 打不开：调用者要自己判断 */
    }
    int n = 0;
    int value;
    while (n < cap && fscanf(fp, "%d", &value) == 1) {   /* 返回值 = 读到了几个 */  返回值是成功读到的个数
        out[n] = value;
        n++;
    }
    fclose(fp);
    return n;
}
```

26 打不开时给回空指针
27 所以这里必须判一下
32 返回值是成功读到的个数

这个循环有三个「输入驱动循环」的通用零件，逐个说：

1. **`fscanf` 的返回值是「成功读到了几个」**。用 `%d` 去读一个整数，成功就是 1，读到文件末尾就是 `EOF`（本机是 -1），遇到不是数字的东西就是 0。所以写 `== 1` 的意思正是「这一次真读到了一个整数，继续」；一旦读不到（末尾、或者数据坏了），循环退出——**不需要提前知道有几个数。**
2. **`n < cap` 是防越界的上界。** `out` 是调用者给的缓冲区，容量 `cap` 由调用者保证；不写上界，数据文件比缓冲区长就会写出去（那是第 4 章的事）。
3. **打不开要能和「打开但空的文件」区分开。**空文件返回 0，打不开返回 -1——这是**约定**，标准没规定，是我在头文件里写下的契约（`stats_loop.h` 里那句注释）。

同一个规则在键盘输入上是同一个函数：`scanf("%d", &x) == 1` 是「从键盘读到了一个整数」。`scanf` 与 `fscanf` 只差「从哪儿读」，**返回值规则一模一样。**

三个数据文件是本单元第一次出现的**输入数据文件**：它们是纯文本（`.txt`），一行一个数，**不放进** `.json`（`examples/` 下的 `.json` 一律被流水线当清单校验，要塞数据请用 `.txt` / `.csv`）。它们有多小、里面是什么，一条命令就看得见：

终端会话 三个数据文件：0 字节、一行、六行 c06-stats-data

```
$ cd examples/ch03/stats_loop
$ wc -c data-empty.txt data-one.txt data-numbers.txt
 0 data-empty.txt
 3 data-one.txt
16 data-numbers.txt
19 total
$ cat data-numbers.txt
3
-7
12
0
42
-5
```

`wc -c` 报的是**字节数**：`data-empty.txt` 是 **0 字节**（建它的命令是 `: > data-empty.txt`，一个字节都不写）；`data-one.txt` 3 字节（`42` 加一个换行）；`data-numbers.txt` 16 字节。驱动它们的 `main` 有二十几行，前半段是读三个文件，后半段是类型对照：

c examples/ch03/stats_loop/src/main.c 第 13–26 行

```c
int main(void) {
    const char *files[3] = {"data-empty.txt", "data-one.txt", "data-numbers.txt"}; 三个文件名写在一起
    int data[64];

    for (int f = 0; f < 3; f++) { 一个一个读
        int n = read_numbers(files[f], data, 64);
        if (n < 0) {
            printf("%s: 打不开\n", files[f]);
            continue; 读不到就跳过这一个
        }
        printf("%s: 读到 %d 个数\n", files[f], n);
        stats s = stats_of(data, n);
        print_stats(&s);
    }
```

14 三个文件名写在一起
17 一个一个读
21 读不到就跳过这一个

（续）后半段只有四行 printf，是本节最后那五个数字的来源：

c **（续）四个打印：两个和、两个 sizeof** examples/ch03/stats_loop/src/main.c 第 28–35 行

```c
    printf("sum_upto(10) = %lld\n", sum_upto(10));
    printf("sum_upto(4)  = %lld\n", sum_upto(4));
    printf("用 int 累加 1..299999: %d\n", sum_int_upto(299999));
    printf("用 long long 累加 1..299999: %lld\n", sum_upto(299999));
    printf("sizeof(int)=%zu sizeof(long)=%zu sizeof(long long)=%zu\n",
           sizeof(int), sizeof(long), sizeof(long long));
    return 0;
}
```

跑出来的结果：

终端输出 终端输出 stats-loop-out

```
data-empty.txt: 读到 0 个数
  count=0 sum=0 max=0 min=0 平均: 一个元素都没有，平均值没有意义
data-one.txt: 读到 1 个数
  count=1 sum=42 max=42 min=42 平均=42.000
data-numbers.txt: 读到 6 个数
  count=6 sum=45 max=42 min=-7 平均=7.500
sum_upto(10) = 55
sum_upto(4)  = 10
用 int 累加 1..299999: 2050177040
用 long long 累加 1..299999: 44999850000
sizeof(int)=4 sizeof(long)=4 sizeof(long long)=8
```

前六行正是 6.5 那套「边界三件套」的实物，而且这次是**输入**在变，不是循环变量在变：

- **空**：`data-empty.txt` 读到 **0** 个数。这时 `stats_of(data, 0)` 走的是「一轮都不跑」的那条路：`sum = 0`、`max = min = 0`（这两个 0 不是答案，是占位）。**平均值**在这里是**没有意义**的——除数是 0。程序的做法是先判 `count == 0` 再决定打印什么，而不是让它去做 0 除法：`printf("平均=%.3f\n", (double)s->sum / s->count)` 那一行只在 `count != 0` 时执行。这条判断就是「空输入」这个边界在代码里的样子。
- **一个**：`data-one.txt` 读到 1 个数 42，和、最大、最小都是 42，平均 `42.000`。一个元素的样本专治两种错：「最大值的初值取 0」在全是负数时会给出 0（错的），「`for (i = 1; i < n; i++)`」在这里一轮都不跑（和是 0，错的）。
- **几个**：六个数 3、-7、12、0、42、-5 的眼睛验算：`3-7+12+0+42-5 = 45`（对上了），最大 42、最小 -7，平均 `45/6 = 7.5`。**平均是打印时算的**（`(double)s->sum / s->count`），所以先转成 `double` 再除，不然 45/6 会整除成 7。

后面五行是本节要重点讲的一件事：**求和值用什么类型。**同一个和 `1 + 2 + … + 299999`，用 `int` 累加得到 **2050177040**，用 `long long` 累加得到 **44999850000**。正确的答案是后者（`299999 × 300000 / 2 = 44999850000`）。前者的现象是**回绕**：

标准原文 · 标准原文：有符号溢出没有「结果是几」这回事

> C 语言标准（ISO/IEC 9899，C17）6.5 节「Expressions」第 5 段：
>
> > If an exceptional condition occurs during the evaluation of an expression (that is, if the result is not mathematically defined or not in the range of representable values for its type), the behavior is undefined.
>
> 大意：求值过程中出现异常情况（数学上没有定义，或者结果超出这个类型的可表示范围），**行为未定义。**`int` 累加到超过 `INT_MAX` 就属于这一条：标准没有说「它会回绕成某个数」，它说的是「标准对此不作要求」。
>
> 出处：ISO/IEC 9899:2018（C17）§6.5 第 5 段。

实验记录 · 本机实测：两个和、三个 sizeof 与一次 -O2 对照

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/stats_loop.c -o app`（清单 `ch03-stats-loop` 的参数，本节全部数字由它跑出）；
- 结果：`int` 版 **2050177040**、`long long` 版 **44999850000**（真值），退出码 0；
- `sizeof(int) = 4`、`sizeof(long) = 4`、`sizeof(long long) = 8`——**本机的 `long` 只有 32 位**（这是 Windows 的 LLP64 模型；Linux 上 `long` 是 8 字节）。所以「用 `long` 就够了」这句话在本机不成立，累加大数一律写 `long long`；
- 那 2050177040 是怎么来的：它是 `44999850000` 对 2^{32} 取模后的余数，也就是「32 位寄存器里留下的东西」。**它不是标准承诺的结果**，同一份代码换个编译器、换个优化档都可能不一样。
- 用 `-O2` 重编同一份源码再跑：这三行数字与 `-O0` 完全相同（`-O2` 的输出在本节的会话块里）。这是**这一次**的实测，不是保证——编译器对这处未定义行为有完全的自由。

「同样的数字」这一条，看一眼汇编就知道不是偶然。下面两条命令生成两份汇编（`-S` 只要汇编，不生成目标文件），再比一下行数：

终端会话 `-O0` 与 `-O2` 的两份汇编，以及 `-O2` 跑出来的数字 c06-opt-diff

```
$ cd examples/ch03/stats_loop
$ gcc -std=c17 -O0 -S -Iinclude src/stats_loop.c -o stats_loop-O0.s
$ gcc -std=c17 -O2 -S -Iinclude src/stats_loop.c -o stats_loop-O2.s
$ wc -l stats_loop-O0.s stats_loop-O2.s
 215 stats_loop-O0.s
 204 stats_loop-O2.s
 419 total
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/stats_loop.c -o app-O2.exe
$ ./app-O2.exe | tail -n 4
sum_upto(4)  = 10
用 int 累加 1..299999: 2050177040
用 long long 累加 1..299999: 44999850000
sizeof(int)=4 sizeof(long)=4 sizeof(long long)=8
```

两份文件都在 `examples/ch03/stats_loop/` 下，可以自己打开看。同一个 `sum_upto`，`-O0` 下是「老老实实一句一句」，`-O2` 下已经不是同一个形状了：

asm examples/ch03/stats_loop/stats_loop-O0.s 第 18–30 行

```
18  .L3:
19      movl    -12(%rbp), %eax   把 i 从栈里取出来
20      cltq
21      addq    %rax, -8(%rbp)    加到 total 上
22      addl    $1, -12(%rbp)
23  .L2:
24      movl    -12(%rbp), %eax
25      cmpl    16(%rbp), %eax
26      jle .L3   条件成立就跳回循环体
27      movq    -8(%rbp), %rax
28      addq    $16, %rsp
29      popq    %rbp
30      ret
```

19 把 i 从栈里取出来
21 加到 total 上
26 条件成立就跳回循环体

asm **（续）`-O2` 的同一个循环：一轮处理两个数** examples/ch03/stats_loop/stats_loop-O2.s 第 20–31 行

```
20      je  .L1
21      .p2align 4
22      .p2align 4
23      .p2align 3
24  .L3:
25      leaq    1(%rdx,%rax,2), %rdx
26      addq    $2, %rax
27      cmpq    %rcx, %rax
28      jne .L3
29  .L1:
30      movq    %rdx, %rax
31      ret
```

编译器视角 · **编译器视角：优化会重写你的循环**

`-O0` 那一份里，循环体是 4 条指令（第 19–22 行）、每轮把 `i` 加 1、拿 `i <= n` 判断、跳回去——它和源码几乎一一对应，这也是本书固定用 `-O0` 的原因：调试器看到的行号与源码对得上。

`-O2` 那一份里，`.L3` 那四条指令（第 25–28 行：`leaq`、`addq`、`cmpq`、`jne`）一轮处理**两个** `i`：`leaq 1(%rdx,%rax,2), %rdx` 一次把 `i + (i+1)` 加进 `rdx`，`addq $2, %rax` 一次前进两格。循环被**展开**（unroll）了，跳转次数减半；上面还多了三条 `p2align`（对齐填充，让循环体落在对齐的地址上）。汇编总行数从 **215** 变成 **204**。

结论有两面：**优化不会改变「这个循环算什么」**（这一版算的就是同一个和，两端数字相同），但它**会改变「这一行对应哪几条指令」**。所以两件事要分开：`-O0` 用来对上源码与调试器，`-O2` 用来看性能与「优化到底动了什么」。真要读优化后的汇编，别指望逐行对上源码——要找的是「哪个循环消失了」。

工程惯例 · 工程惯例：先估规模，再写嵌套循环

这一条是经验，不是标准。一层循环 n 轮是 n；两层嵌套是 n^2；三层是 n^3。n = 1000 时，n^2 是一百万轮（还跑得完），n^3 是十亿轮（要等很久）。**看到三层循环就该停下来问一句「数据量真有这么大吗」**；同样的算法常常能少一层（比如用「一边扫一边记账」代替「两两比较」）。

工程惯例 · （续）工程惯例：累加值的类型按最大可能值选

这条也是经验。单个元素是 `int`，不代表和也是 `int`：10 万个 `int` 相加就可能超出 32 位。**先估上界，再选类型**——本机 `long` 只有 32 位（`sizeof(long) = 4`），所以这段代码写的是 `long long`（8 字节），打印它要用 `%lld`。

代价说清楚：`long long` 占 **8 个字节**（`int` 是 4 个）。速度上我不给你结论——本单元没有做计时实验；能确定的是它在 x86_64 上与 32 位加减是同一类指令，差别远小于「选错类型」带来的错误。在「和」这种一定会长大的中间结果上用它是划算的；在几十个元素的计数这种明确跑不出去的地方，用 `int` 完全没问题。

6.7 `break`、`continue`、`goto`：只影响最内层

`break` 与 `continue` 的作用范围只有一条规则：**最内层**。多层嵌套里想「一次跳出两层」，标准给的工具是 `goto`。这个例子把同一件事写了三遍——在 3×4 的矩阵里找一个数：

c examples/ch03/goto_multi_break/src/goto_multi_break.c 第 3–21 行

```c
int find_with_flag(const int *m, int rows, int cols, int target, hit *out) {
    int found = 0;
    for (int r = 0; r < rows && !found; r++) {          /* 每层循环都得看着标志位 */ 两层循环的每一层都得看着标志位
        for (int c = 0; c < cols && !found; c++) {
            if (m[r * cols + c] == target) {
                out->row = r;
                out->col = c;
                out->value = target;
                found = 1;
            }
        }
    }
    if (!found) {                                       /* 收尾要写第二遍 */ 收尾要写第二遍
        out->row = -1;
        out->col = -1;
        out->value = 0;
    }
    return found;
}
```

5 两层循环的每一层都得看着标志位
15 收尾要写第二遍

c examples/ch03/goto_multi_break/src/goto_multi_break.c 第 23–43 行

```c
int find_with_goto(const int *m, int rows, int cols, int target, hit *out) {
    int found = 0;
    for (int r = 0; r < rows; r++) {                    两层循环，条件干净
        for (int c = 0; c < cols; c++) {
            if (m[r * cols + c] == target) {
                out->row = r;
                out->col = c;
                out->value = target;
                found = 1;
                goto done;                              /* 一次跳出两层 */ 找到就一次跳出两层
            }
        }
    }
done:                                                   /* 收尾只写一处 */ 收尾只写一处
    if (!found) {
        out->row = -1;
        out->col = -1;
```

```
        out->value = 0;
    }
    return found;
}
```

25 两层循环，条件干净
32 找到就一次跳出两层
36 收尾只写一处

第三种写法（`find_with_helper`，源码第 45 行起）把「找一行」抽成一个小函数 `scan_row`：找到就 `return 1`，外层循环拿到 1 之后直接 `return 1`——**一层循环就够**，因为它不再需要「跳出」，`return` 本身就是跳出去。三种写法的结果必须一样，跑出来确实一样：

终端输出 终端输出 goto-multi-break-out

```
找 7:
find_with_flag   找到 7: 第 1 行第 2 列
find_with_goto   找到 7: 第 1 行第 2 列
find_with_helper 找到 7: 第 1 行第 2 列
找 13:
find_with_flag   没找到
find_with_goto   没找到
find_with_helper 没找到
```

（7 在矩阵里：第 1 行是 `5, 6, 7, 8`，所以是「第 1 行第 2 列」——行列都从 0 数。13 不在矩阵里，三种写法都返回「没找到」；「没找到」时 `find_*` 会把 `out` 的 `row` / `col` 填成 -1、`value` 填成 0，但 `report` 的失败分支只打印「没找到」——想看这三个数得自己加一行 `printf`，或者在调试器里 `p *h`。）

三种写法读下来，取舍很清楚：

- **标志位版**：不引入新语法，但「条件里带标志位」这件事要每层写一遍（第 5 行那个 `&& !found`），而且**收尾代码写了第二遍**（找到时在循环里填 `out`、没找到时在循环外填）。层数一多，条件就越写越长。
- **`goto` 版**：循环条件保持干净（就是「遍历」），跳出口只有一个，收尾也只有一个。代价是**控制流的出口多了一个**——读代码的人必须记住 `done:` 这个标签。
- **小函数版**：多数情况下这才是最好的版本：`scan_row` 一个函数一件事，`return` 天然就是一层的出口，没有标志位、没有标签。**能靠函数解决就不要用 `goto`。**

同一件事在单层循环里更简单：`continue` 是「跳过这一轮、接着下一轮」，`break` 是「结束这一层循环」。它们在嵌套里各是什么范围，看这个例子：

c examples/ch03/nested_continue/src/nested_continue.c 第 19–30 行

```
int count_inner_continue(int n) {
    int c = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (!keep(i, j)) {          keep 返回 0 的对
                continue;           /* 只跳过这一轮内层 */  只跳过这一对，内层继续
            }
            c++;
        }
    }
    return c;
}
```

23 keep 返回 0 的对
24 只跳过这一对，内层继续

c examples/ch03/nested_continue/src/nested_continue.c 第 32–46 行

```
int count_outer_continue(int n) {
    int c = 0;
    for (int i = 0; i < n; i++) {
        if (i == 2) {          这里在外层
            continue;               /* 跳过整个 i == 2 这一行 */  跳过一次，整行都没了
        }
```

```
            for (int j = 0; j < n; j++) {
                if (!keep(i, j)) {
                    continue;
                }
                c++;
            }
        }
        return c;
    }
```

35 这里在外层
36 跳过一次，整行都没了

c examples/ch03/nested_continue/src/nested_continue.c 第 48–59 行

```
    int count_inner_break(int n) {
        int c = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (!keep(i, j)) {          同样的位置，换成 break
                    break;                  /* 结束整层内层循环，不是「跳过这一轮」 */  它结束的是整个内层
                }
                c++;
            }
        }
        return c;
    }
```

52 同样的位置，换成 break
53 它结束的是整个内层

三份代码只差关键字（`continue` / `break` / 位置），输出差得很远：

终端输出 终端输出 nested-continue-out

```
n = 4
内层 continue: 得到 12 对, keep 被调了 16 次 (n*n)
外层 continue: 得到  9 对, keep 被调了 12 次 (少了一整行)
内层 break   : 得到  6 对, keep 被调了 10 次 (每行提前收工)
内层 continue 图 (x = 被跳过的那一对) :
    i=0  x...
    i=1  .x..
    i=2  ..x.
    i=3  ...x
```

三个数字要连起来读（n = 4，一共 4×4 = 16 对）：

- **内层 `continue`**：只跳过 `i == j` 那 4 对，得到 **12** 对，`keep` 被调了 **16** 次——**每一对都调了一次**；
- **外层 `continue`**：跳过整个 `i == 2` 那一行，得到 12 − 3 = **9** 对，`keep` 只被调了 **12** 次（少了一整行，一行 4 次）；
- **内层 `break`**：不是「跳过这一对」，而是「这一行剩下的都不看了」，每行在 `j == i` 处提前收工，一共 **6** 对，`keep` 被调了 **10** 次。

`keep` 的调用次数是顺手拿到的证据：函数 `keep` 里有一个自己文件里的计数器（`static int calls`），每调一次加一。它证明了一件事——**循环体里调用的函数，每一轮都会真的被调一次**。调用树长这样：

调用树 **循环体里那一下调用：每一轮一次，一次不落**

第 1 步 **main 开始数 4x4 的对**

```
main() : main.c:8, 调 count_inner_continue(4)
main() → count_inner_continue(4) 两层循环在里面
```

第 2 步 **第 1 对：i=0, j=0**

```
main() : 在等结果
count_inner_continue(4) : i=0, j=0
count_inner_continue(4) → keep(0, 0) 循环体里那一下调用
keep(0, 0) ⇒ count_inner_continue(4) 返回 0
```

第 3 步 **下一对：i=0, j=1**

```
main() : 还在等
count_inner_continue(4) : i=0, j=1
count_inner_continue(4) → keep(0, 1) 又一枚调用
keep(0, 1) ⇒ count_inner_continue(4) 返回 1
```

第 4 步 **循环结束，函数返回**

```
main() : 拿到 12
keep_call_count() = 16 : n*n 次，每轮一次 ← 本步新增
count_inner_continue(4) ⇒ main() 返回 12
```

第 2、3 帧是同一次循环里的相邻两轮：每一轮都**真的进了一次函数**（`keep(0, 0)`、`keep(0, 1)`），返回值决定这一对算不算数。第 4 帧那个 `16` 就是 `n × n`。这件事有工程含义：**循环体里的函数每一次都会执行一遍**——如果那个函数里干了「打印」「分配内存」「读文件」这类事，它的代价要乘上循环轮数；如果只是个小判断，编译器多半会把它内联（`-O2` 下尤其常见，这也是优化后断点不好打的原因之一）。

标准原文 · **标准原文：`goto` 跳不出函数**

> C 语言标准（ISO/IEC 9899，C17）6.8.6.1 节「The goto statement」的约束部分：
>
> > The identifier in a goto statement shall name a label located somewhere in the enclosing function.
>
> 大意：`goto` 的目标标签必须**在同一个函数里**。所以 `goto` 跳不出函数，也就不可能像有些语言那样「跨函数跳转」。
>
> 出处：ISO/IEC 9899:2018（C17）§6.8.6.1。

另外两个关键字的作用范围，标准在语义部分钉死在**最内层**：

标准原文 · **标准原文：`continue` 与 `break` 只管最内层**

> C 语言标准（ISO/IEC 9899，C17）6.8.6.2 节「The continue statement」与 6.8.6.3 节「The break statement」：
>
> > ... the smallest enclosing iteration statement ...
> >
> > ... the smallest enclosing switch or iteration statement ...
>
> 大意：`continue` 跳到**最小的那层包围它的迭代语句**的「继续位置」（`for` 就是第三段 `i++`），`break` 结束**最小的那层包围它的 `switch` 或迭代语句**。两层循环里，这两个关键字都只看**紧挨着它的那一层**。
>
> 出处：ISO/IEC 9899:2018（C17）§6.8.6.2、§6.8.6.3。

工程惯例 · **工程惯例：`goto` 什么时候用**

标准允许 `goto` 跳到函数里任何地方，但工程上三十年攒下来两条约定（**是惯例，不是标准**）：

- **只往前跳，不往回跳。**往回跳等于手工造循环，可读性还不如写个 `while`；往前跳的典型场景就是我们刚看的「多层跳出」。
- **跳到的地方要么是统一收尾，要么是错误出口。**C 代码里 `goto` 最常见的用法是「一段代码要申请好几个资源，任何一步失败就 `goto fail;`，在那儿按申请的**逆序**释放」。这样申请与释放成对出现、只有一个出口。第 12 单元（动态内存）会反复用到这个形状。

代价：每多一个标签，读代码的人就多一处「可能从别处跳到这里」要记住。所以约定是**一个函数里的 `goto` 标签越少越好**（一个、最多两个），而且标签名要能读出用途（`done`、`fail`、`cleanup` 比 `L1` 强得多）。

6.8 停不下来的三种成因，以及怎么把循环拆开看

一个循环停不下来，几乎总是三种成因之一：**忘了让条件变化、条件永远不会「精确相等」、条件被写成了恒真。**三种都能真跑出来，我们一个一个掐着表看。

先看「读输入」这个场景里最容易被写错的一处：`getchar` 的返回值用什么接。下面这个函数把标准输入逐字节打印出来：

c examples/ch03/while_eof/src/while_eof.c 第 4–17 行

```c
long dump_stdin_bytes(void) {
    long n = 0;
#ifdef USE_CHAR
    char c;                 /* 想用 char 接：EOF 是 -1，char 不一定装得下 */ char 装不下 228 这样的值
#else
    int c;                  /* 标准写法：getchar 的返回值必须是 int */ 标准写法是 int
#endif
    while ((c = getchar()) != EOF) { 读到 EOF 就停
        printf("%d ", c);
        n++;
    }
    printf("\n");
    return n;
}
```

7 char 装不下 228 这样的值
9 标准写法是 int
11 读到 EOF 就停

两份清单（`ch03-while-eof-int` 与 `ch03-while-eof-char`）编的是同一个文件，只是后者多了一个 `-DUSE_CHAR`；两者喂进去的标准输入是同样四个字符 `hi 三` 加一个换行（`三` 在 UTF-8 里是三个字节 `E4 B8 89`）。输出差在数字上：

终端输出 终端输出 while-eof-int-out

```
用 int 接 getchar():
104 105 32 228 184 137 10
字节数 = 7
```

终端输出 终端输出 while-eof-char-out

```
用 char 接 getchar():
104 105 32 -28 -72 -119 10
字节数 = 7
```

`h`、`i`、空格、换行在两边都一样（104、105、32、10）；差的是 `三` 那三个字节：`int` 版打出 **228 184 137**，`char` 版打出 **-28 -72 -119。字节数都是 7**，但 `char` 版报出来的值已经不对了。原因是 `char` 在本机是**有符号**的（范围 -128…127），228 装不下，被折成了 228 − 256 = -28。这一次它还能停在 EOF 上（因为本机的 `EOF` 是 -1，有符号 `char` 装得下 -1）；换一个「`char` 无符号」的平台，`EOF` 的 -1 会变成 255，那个 `!= EOF` 就**永远不会成立**——这正是「条件永远不会精确相等」的一种。本机可以用 `-funsigned-char` 把 `char` 变成无符号，把这一幕真做出来：

终端会话 把 char 变成无符号：循环再也停不下来 c06-eof-uchar

```
$ cd examples/ch03/while_eof
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DUSE_CHAR -funsigned-char src/main.c
src/while_eof.c -o eof_uchar.exe
src/while_eof.c: In function 'dump_stdin_bytes':
src/while_eof.c:11:28: warning: comparison is always true due to limited range of data type [-Wtype-limits]
   11 |     while ((c = getchar()) != EOF) {
      |                            ^~
$ printf 'hi\n' | timeout 2 ./eof_uchar.exe > /tmp/eof-uchar-out.txt ; echo "退出码 $?"
退出码 124
$ wc -c /tmp/eof-uchar-out.txt ; head -c 60 /tmp/eof-uchar-out.txt ; echo
15103122 /tmp/eof-uchar-out.txt
用 char 接 getchar():
104 105 10 255 255 255 255 255 25
```

这一段有三样东西值得逐条读：① 编译器**先说话了**——`-Wextra` 给出 `comparison is always true due to limited range of data type [-Wtype-limits]`，翻译过来是「这个比较永远为真，因为 `char` 的范围装不下比较的另一头」。这条警告之所以在「`char` 有符号」的本机没出现，是因为那时它**可能**为真（-1 装得下）。同一句代码，两种 `char`，一条警告只出现在其中一种上——**警告的有无本身也是信息。**② `timeout 2` 给了这个程序 2 秒就被杀掉，**退出码 124**（`timeout` 的约定：到点杀掉返回 124）——这是「循环不终止」在命令行里的样子。③ 2 秒里它往文件里写了 **15103122 字节**（约 15 MB）全是 `255`（对应 -1 转成无符号的那个字节），`head -c 60` 只截了开头几个。**这个字节数不是常数**：它取决于这 2 秒里机器写了多快，我两次抓会话分别读到 15103122 与 14737090；稳定的只有「退出码 124」与「开头那几个字节」。**一个写错的循环条件能在 2 秒里写出十几 MB**，这也是为什么这类错误要先在小的数据上试。

正确写在头文件里，是一句声明加一句宏：

> **编译器视角 · 编译器视角：`EOF` 是什么、`getchar` 返回什么**
>
> 本机工具链（gcc 16.1.0，UCRT64）的 `stdio.h` 里，这两样是明明白白写着的：
>
> - `C:\msys64\ucrt64\include\stdio.h:30`：`#define EOF (-1)` —— `EOF` 只是一个宏，本机展开就是 **-1**（标准只要求它是一个负的 `int` 常量，具体值由实现定）；
> - `C:\msys64\ucrt64\include\stdio.h:601`：`int __cdecl getchar(void);` —— 它的返回类型是 **`int`**，不是 `char`。
>
> 所以 `int c; while ((c = getchar()) != EOF)` 里的 `int` 不是习惯写法，是**跟着声明写**：`getchar` 要能表达「256 个字符值 + 一个 EOF」，只有 `int` 装得下。用 `char` 接，就是在赋值那一刻做了一次**装不下的窄化转换**——标准把「装不下时怎么办」交给实现去定，本机的实现给出的就是那三个负数（228 折成 -28）。

第二种成因：**条件永远不会精确相等。**浮点数是这一类的重灾区——`0.1` 在二进制里根本没有精确表示，`d += 0.1` 加十次不会正好等于 `1.0`。探针 `float_loop_probe.c` 把三种情形做成了三个模式：

c examples/ch03/float_loop_probe.c 第 12–27 行

```c
12  static void missing_increment(void) {
13      int i = 0;
14      long total = 0;
15      while (i < 5) {
16          total += i;                 /* 这里少了一句 i++; 循环体里 i 一直不变 */ 循环体里 i 一直不变
17      }
18      printf("total = %ld\n", total);
19  }
20
21  static void float_ne(void) {
22      int rounds = 0;
23      for (double d = 0.0; d != 1.0; d += 0.1) { 0.1 加上去永远差一点
24          rounds++;
25      }
26      printf("rounds = %d\n", rounds);
27  }
```

16 循环体里 i 一直不变
23 0.1 加上去永远差一点

c **（续）同一个循环，条件换成 `d < 1.0`** examples/ch03/float_loop_probe.c 第 29–36 行

```c
29  static void float_lt(void) {
30      int rounds = 0;
31      double d = 0.0;
32      for (; d < 1.0; d += 0.1) {
33          rounds++;
34      }
35      printf("rounds = %d, 最后 d = %.17g\n", rounds, d);
36  }
```

```c file=examples/ch03/float_loop_probe.c lines=38-44 mark=40,41 title=（续）条件永真：`for (;;)` 省掉了第二段
static void forever(void) { long spins = 0; for (;;) { /* 第二段被省掉：等于恒不为 0 / spins++; / 循环体内没有出口，只能
```

靠外面杀掉 / } printf("spins = %ld\n", spins); / 到不了这里 */ }

四个模式的实测（每个给 2 秒）：

```term id=c06-loop-hangs file=examples/ch03/session-loop-hangs.txt lines=1-11 title=三个停不下来的、一个「停得不对」的循环
$ cd examples/ch03
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g float_loop_probe.c -o flp.exe
$ echo 0 | timeout 2 ./flp.exe ; echo "退出码 $?"
退出码 124
$ echo 1 | timeout 2 ./flp.exe ; echo "退出码 $?"
退出码 124
$ echo 2 | timeout 2 ./flp.exe ; echo "退出码 $?"
rounds = 11, 最后 d = 1.0999999999999999
退出码 0
$ echo 3 | timeout 2 ./flp.exe ; echo "退出码 $?"
退出码 124
```

- **模式 0（忘自增）**：`while (i < 5)` 的循环体里没有 `i++`，`i` 永远是 0，条件永远为真——**退出码 124**，2 秒后被 `timeout` 杀掉。这是三种成因里最常见的一种，**编译器一个字都不说**（它没理由认为你想让 `i` 变），只能靠「程序不结束」来发现。
- **模式 1（浮点做终止条件）**：`d != 1.0` 同样**退出码 124**。`0.1` 加十次得到的是 `0.9999999999999999`（不等于 1.0），再加一次是 `1.0999999999999999`——**跳过了 1.0**，那个 `!=` 永远不会为假。
- **模式 2（同一循环换成 `d < 1.0`）**：能停，但 `rounds = 11`！加了 11 次（`0.0` 到 `1.0999999999999999`），不是我们直觉里的 10 次。**浮点的差一错误**就长这样：终止条件写成 `<` 之后，轮数由「累加的误差」决定，而不是由数学决定。
- **模式 3（条件永真）**：`for (;;)` 省掉第二段，循环体里也没有出口——**退出码 124**，同样 2 秒后被掐掉。它和模式 0 的区别在「条件」这一侧：模式 0 的条件本来会变假，只是循环体忘了让它变；模式 3 的条件**根本没有**（被一个恒不为 0 的常量顶替了）。

第三种成因「条件永真」就是 6.4 里标准说的那句：`for (;;)` 里被省掉的第二段等于一个恒不为 0 的常量，循环只能靠 `break` / `return` / `goto` 出来。模式 3 只是**故意**这么写的结果；真正危险的是**不小心**写成恒真（比如把 `while (i > 0)` 敲成 `while (i >= 0)`）。写 `for (;;)` 时请自己数一遍有几条出口。

循环卡住的时候，怎么把它拆开看？调试器的条件断点是最顺手的工具。下面这段会话跑的是 6.4 那个 `sum_upto`（`n = 10`），断点打在**循环体**（`src/stats_loop.c` 第 12 行）上，但加了一个条件 `i == 5`：

终端会话 第 1 步：条件断点，只停在第 5 轮 c06-gdb-cond-1

```
$ cd examples/ch03/stats_loop
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/stats_loop.c -o stats_loop.exe
$ gdb --batch -ex "break stats_loop.c:12 if i==5" -ex run -ex "p i" -ex "p total" ./stats_loop.exe
Breakpoint 1 at 0x14000170c: file src/stats_loop.c, line 12.
[New Thread 50892.0x8c18]
Thread 1 hit Breakpoint 1, sum_upto (n=10) at src/stats_loop.c:12
12              total += i;
$1 = 5
$2 = 10
```

终端会话 （续）第 2 步：不带条件，再用 until 一次跑到循环外 c06-gdb-cond-2

```
$ gdb --batch -ex "break stats_loop.c:12" -ex run -ex "p i" -ex "p total" -ex "delete 1" -ex "until 14" -ex
"p i" -ex "p total" ./stats_loop.exe
Breakpoint 1 at 0x14000170c: file src/stats_loop.c, line 12.
[New Thread 69220.0x15f04]
Thread 1 hit Breakpoint 1, sum_upto (n=10) at src/stats_loop.c:12
12              total += i;
$1 = 1
$2 = 0
sum_upto (n=10) at src/stats_loop.c:14
14          return total;
$3 = 11
$4 = 55
```

调试实录 · 调试实录：把循环拆成「停在哪一轮」与「跑到哪一行」

下面两块是会话原文，按两次 gdb 调用拆开。读法也只有三步：

- `break 文件:行号 if 条件` 是**条件断点**：只有条件成立才停。第 1 块里 `if i==5`，所以它只在第 5 轮停下来——`p i` 打出 **5**、`p total` 打出 **10**，正好对上 6.4 那张图的帧 5（第 4 轮结束时 `total = 10`，而此刻还没执行这一轮的 `total += i`）。**这就是循环不变式在调试器里的样子**：你不必相信代码，可以让机器把「每一轮结束后的那句话」打给你看。
- 第 2 块先不带条件：断点立刻在第 1 轮命中，`p i` 是 **1**、`p total` 是 **0**（还没加过任何东西）。然后 `delete 1` 删掉断点、`until 14` 的意思是「一直跑，直到第 14 行再停」——中间不再被打断。
- 停在 `return total;` 时再打两个变量：`p i` 打出 **11**、`p total` 打出 **55**。**11 这个数就是本节最值钱的一句话**：`n = 10`，循环退出时 `i` 是 11 — `i++` 在最后一轮之后又跑了一次，条件才不成立。

结论写在最后：**条件断点把「哪一轮」变成可指定的一件事**（`if i==5`），`until 行号` 把「剩下的都跑完」变成一条命令；查差一错误时先在那句 `p i` 上要答案，比自己拿纸推快得多。

6.9 收口：这一单元新增了什么，下一单元看什么

回到 6.1 那两棵目录树，现在可以逐个点名了：

- `examples/ch03/branch_basics/` —— 悬空 `else`、花括号、`else if` 链、真值表、赋值当条件（6.2）；
- `examples/ch03/switch_fallthrough/` —— 漏 `break` 的贯穿、故意的贯穿、`default` 的位置、分数线表（6.3）；
- `examples/ch03/loop_offbyone/` —— 差一错误的三种写法、`while` 与 `do-while` 的轮数对照、整组容量那一次求和（6.4、6.5）；
- `examples/ch03/stats_loop/` —— 求和 / 最大 / 平均、空输入边界、类型对照，还带着两份汇编（6.4、6.6）；
- `examples/ch03/while_eof/` —— `int c` 与 `char c` 两版 `getchar`（6.8）；
- `examples/ch03/goto_multi_break/`、`examples/ch03/nested_continue/` —— 多层跳出与 `continue` / `break` 的范围（6.7）；
- `data-empty.txt`（0 字节）、`data-one.txt`（3 字节）、`data-numbers.txt`（16 字节）—— 输入数据文件，本单元第一次出现；
- `case_const_probe.c`、`switch_float_probe.c`、`float_loop_probe.c` —— 三个**没有清单**的探针，证据是会话里的真实诊断与退出码；
- 九段 `session-*.txt` —— 编译警告、gdb、汇编对照、timeout 掐掉的循环；正文里的 `term` 块都从它们注入，与文件逐字对账；
- `stats_loop/stats_loop-O0.s` 与 `stats_loop/stats_loop-O2.s` —— 6.6 那两份汇编对照的原件。

下一单元（第 7 单元）接着讲**函数**：这一节里 `sum_upto`、`keep`、`scan_row` 已经被调来调去，下一节要回答的是「调用的时候到底发生了什么」——实参怎么传进去、返回值怎么带回来、栈帧里存了什么、`static` 局部变量为什么记得住上一次。这一节用到的 gdb 手法（条件断点、`until`、`p`）会原样再用一遍，只是那时看的是调用栈。

小结 · 小结：分支的配对、真值与贯穿

- `else` **归语法上最近的那个** `if`**，缩进不算数**；一条语句的花括号可以省，但「看着像在里面」的语句永远是 bug 的温床（`-Wdangling-else` / `-Wmisleading-indentation` 会替你数）。
- **真值就是「非 0 为真」**：`if (0.0)` 是假、`if (-1)` 是真、`if ('0')` 是真（`'0'` 的值是它在本机执行字符集里的编码 48）；`if (x = 0.1)` 是赋值当条件，`-Wparentheses` 会给警告。
- `switch` **的控制表达式必须是整型**（浮点、字符串直接编译失败），`case` **后面必须是整数常量表达式**（写变量直接报错）；`default` 的位置由你定，但最好别省。
- 漏 `break` 会贯穿；`-Wimplicit-fallthrough` 属于 `-Wextra`——**只加** `-Wall` **它不吭声，只加** `-Werror` **它仍然不出现**（没有警告可升级）；必须 `-Wextra` 加 `-Werror` 才把退出码从 0 变成 1。

小结 · **（续）小结：循环的边界、跳出与类型**

- **`while` / `for` 先看后做，`do-while` 先做后看（至少执行一次）**；用「每一轮开始时什么为真」当判据检查循环，再用「退出时循环变量是几」检查差一错误（`n = 10` 时退出值为 11）。
- **跳出只有最内层**：`continue` 跳过一轮、`break` 结束一层；多层跳出用 `goto`（标签只能在自己函数里）或标志位，能靠小函数 + `return` 解决就不用 `goto`。
- **停不下来的三种成因**：忘自增（编译器不吭声）、浮点做精确相等（`d != 1.0` 永远为真）、条件恒真（`for (;;)` 与写错的不等号）；读输入要用 `int c` 接 `getchar()` 并与 `EOF`（本机是 -1）比。
- **类型按最大可能值选**：本机 `sizeof(long) == 4`，累加大数写 `long long`（`%lld`）；`int` 溢出是未定义行为，本机看到的是回绕 2050177040，而真值是 44999850000。

自测题 · **自测题（先自己答，再对答案）**

1. 下面三条语句**依次**写在同一段代码里（第二条与第三条都缩进，像是有花括号，其实没有）：第一条是 `if (a > 0)`；第二条是 `if (b > 0) return "两个都为正";`；第三条是 `else return "a 非正";`。问：`a = 1、b = -1` 时它返回什么？`else` 属于哪个 `if`？
2. 一段 `switch` 里 `case 1:` 的末尾忘了写 `break`，后面紧跟 `case 2:`。用 `-Wall` 编一次、再用 `-Wall -Wextra` 编一次，两次的**退出码**分别是多少？哪一次会告诉你这件事？
3. `for (int i = 0; i <= n; i++)` 想累加数组的前 `n` 个元素，错在哪？循环退出时 `i` 等于几？
4. `while (n < cap && fscanf(fp, "%d", &value) == 1)` 里那个 `== 1` 是什么意思？如果文件是空的（0 字节），循环跑几轮、`n` 最后是几？
5. 用 gdb 在一个循环体上打断点，想让它只在 `i == 5` 那一轮停下来，命令怎么写？停下来之后想看 `total`，命令是什么？

注 · **自测题答案**

1. 返回 **「a 非正」**——这是错的答案，正是悬空 `else` 的现象。`else` 属于**按语法最近**的那个 `if`（也就是里面的 `if (b > 0)`）：当 `b > 0` 为假时执行 `else`，于是一个 `a = 1` 的调用被报成了「a 非正」。加上花括号把外层 `if` 包起来，`else` 才会归外层。
2. 两次的退出码都是 **0**：`-Wimplicit-fallthrough` 属于 `-Wextra`，所以 `-Wall` 那次一个字都不说；`-Wall -Wextra` 那次会给出 `warning: this statement may fall through [-Wimplicit-fallthrough=]`，并附一句 `note: here` 指向下面那个 `case`，但**警告不影响退出码**。要让它变成失败，必须**先有 `-Wextra`（警告才会出现）、再加 `-Werror`（警告才会升级成错误）**——只加 `-Wall -Werror` 什么都不会发生，编译照样成功；本单元清单 `ch03-switch-fallthrough-werror` 写的是 `-Iinclude -Werror`，而流水线的基础参数里本来就有 `-Wall -Wextra`，所以那份清单的退出码是 1。
3. 错在**多算一个**：`i` 从 0 开始、条件是 `<= n`，所以它会用到下标 `0…n`，一共 `n+1` 轮；要累加前 `n` 个应该用 `i < n`（用到 `0…n-1`）。循环退出时 `i` 等于 **`n + 1`**（本单元 gdb 会话里 `n = 10`、`p i` 打出 11 就是这个数）。想让 `<= n` 正确，起点得改成 1 并且语义变成「1 到 n 的和」，那是另一个形状。
4. `fscanf(...) == 1` 的意思是「这一次**成功读到了一个整数**（`%d` 只转换一个项）」——返回值是成功转换的项数，读到文件末尾返回 `EOF`、遇到不是数字的输入返回 0，两种都不等于 1，所以循环退出。空文件（0 字节）时**一轮都不跑**，`n` 最后是 **0**；`main` 拿到 0 之后走的是「空输入」那条路：和是 0、平均值没有意义（除数为 0，必须先判 `count == 0`）。
5. 条件断点写成 `break src/stats_loop.c:12 if i==5`（`gdb` 认文件名后缀，写成 `break stats_loop.c:12 if i==5` 也行）。停下来之后打 `p i` 可以确认自己真的停在第 5 轮，打 `p total` 看此刻的累加值。想让它一直跑到循环外面，可以先 `delete` 掉断点再 `until 14`（第 14 行是 `return total;`），那时 `p i` 会给出 11、`p total` 给出 55。

函数：声明、定义、调用栈与 static 局部变量

◎ 学习目标 · 看完这一单元，你应该能

- 说出一个函数的**声明**与**定义**各住在哪个文件，并解释定义文件为什么也要包含自己的头文件；
- 用「实参是拷贝」这一句话解释 `swap_bad(x, y)` 为什么白干，并写出用指针的正确版本；
- 区分 `f(void)` 与 `f()`，说出多传一个实参时这两种声明各有什么后果、参数求值顺序为什么不能依赖；
- 用 gdb 的 `bt`、`info frame`、`finish` 把一层栈帧的**参数、局部变量、返回地址**逐个指认出来，并说出递归的基准情形与它在本机的代价（那个退出码）；
- 分清**作用域**与**生命周期**，解释 `static` 局部变量为什么能跨调用「记得住」、住在内存的哪一块上，以及 `assert` 在 `-DNDEBUG` 下为什么会整个消失。

7.1 先看工程长什么样

上一单元你把流程写对了；这一单元开始，代码要**分家**。分家不是整理癖：从这一节起，一个函数可以**声明在 A 文件、定义在 B 文件、调用在 C 文件**，三者之间靠编译器与链接器牵线。牵错了会在不同阶段报错——**报错在哪个阶段，决定了你该去翻哪个文件。**

先说清起点。上一单元（`chapters/ch03/01-branches-and-loops.md`，分支与循环）已经把第 3 章的项目建起来了：`examples/ch03/` 下有七个例子，每个都是「`src/` + `include/` + 清单」的最小工程，另外还留下八段抓下来的会话与两个探针。第 1 帧概括地画了它交棒时的样子（只点两样，其余同形）。第 2 到 5 帧是本节搭起来的**八个**新例子目录（两两一帧），第 6 帧把最基础的那个拆开，第 7 帧是本节另外留下的五段会话。

目录树 **examples/ch03/：这一单元里它长成了什么样**

第 1 步 本节开始时，第 3 章的项目已经有上一单元的七个例子

```
examples/
├── ch01/ ：第 1 章的项目（本节不动它）
├── ch02/ ：第 2 章的项目（本节不动它）
└── ch03/ ：第 3 章的项目
    ├── branch_basics/ ：上一单元的例子之一，各带 src/ + include/ + 清单
    └── session-*.txt ：上一单元抓的八段会话（不逐一列出）
```

第 2 步 先摆上两个：声明与定义、值传递

```
examples/ch03/
├── 上一单元的七个例子 ：同形
├── func_basics/ ：声明在 .h、定义在 .c ← 本课新增
└── pass_by_value/ ：两个 swap，一个白干 ← 本课新增
```

第 3 步 再加两个：栈帧与 static

```
examples/ch03/
├── 前面的例子 ：同形
├── stack_frames/ ：三层嵌套与栈帧 ← 本课新增
└── static_local/ ：static 局部 vs 自动局部 ← 本课新增
```

第 4 步 再加两个：递归的两面

```
examples/ch03/
├── 前面的例子 ：同形
├── factorial_both/ ：递归版与迭代版 ← 本课新增
└── deep_recursion/ ：几十万层之后爆栈 ← 本课新增
```

第 5 步 最后两个，八个到齐

```
examples/ch03/
├── 前面的例子 ：同形
├── assert_guard/ ：前置条件与断言 ← 本课新增
└── old_style_decl/ ：f() 与 f(void) ← 本课新增
```

第 6 步 拆开 func_basics：第一次出现「模块头文件」

```
examples/ch03/func_basics/
├── include/
│   └── func_basics.h ： 只有声明与注释 ← 本课新增
├── src/
│   ├── main.c / func_basics.c ： 调用与定义 ← 本课新增
│   ├── decl_mismatch.c ： 定义写错的那份 ← 本课新增
│   └── arg_order.c / main_arg_order.c ： 求值顺序 ← 本课新增
└── 三个清单 ： 各管一种对账 ← 本课新增
```

第 7 步 本节另外留下五段抓下来的会话

```
examples/ch03/
├── session-gdb-frames.txt ： bt / info frame / finish ← 本课新增
├── session-gdb-static.txt ： static 与自动局部的地址 ← 本课新增
├── session-warnings.txt ： 两条真实诊断 ← 本课新增
├── session-deep.txt ： 爆栈退出码与 -O2 对照 ← 本课新增
└── session-assert.txt ： 断言失败的现场 ← 本课新增
```

第 1 帧是这一节故事的起点。第 2 到 5 帧是本节搭起来的 `examples/ch03/` 里的**八个**例子目录（两两一帧，每一帧只变一处），每个都是「源文件放 `src/`、声明放 `include/`、编译与对账的参数写在 `.json` 清单里」的最小工程。第 6 帧把最基础的那个 `func_basics/` 拆开，它是这一节最要紧的一张图：**第一次出现模块头文件**——`include/func_basics.h` 里只有**声明与注释**，`src/` 下的五个源文件都包含它一次（7.2 只点名其中两个）。同一个目录里可以放**多个清单**：`func_basics_declmismatch.json` 与 `func_basics.json` 用的是同一个头文件，一个演示「定义与声明不一致会怎样」，一个演示正常调用；`func_basics_argorder.json` 用同一个头文件、另一对源文件，演示实参求值顺序。第 7 帧是本节另外留下的五段终端会话（都是抓下来的原文），后面每一节用到哪段，就在哪一节点名。

八个例子目录，每个只负责讲清一件事：

- `func_basics`、`pass_by_value` —— 声明与定义的分工、值传递（7.2、7.3）；
- `old_style_decl` —— `f()` 与 `f(void)` 的区别（7.4）；
- `stack_frames` —— 三层嵌套调用与栈帧（7.6）；
- `factorial_both`、`deep_recursion` —— 递归的写法与代价（7.7、7.8）；
- `static_local` —— `static` 局部变量与生命周期（7.9）；
- `assert_guard` —— 前置条件与断言（7.10）。

工程惯例 · **工程惯例：为什么声明放头文件、定义放源文件**

标准对「一个程序分几个文件」没有任何规定，它只要求一个程序有且只有一个 `main`。**分家是惯例**，理由有三条：

- **改一个文件不用重编全部**：`func_basics.c` 改一行，只需要重编它自己，然后重新链接一次；
- **接缝写在明面上**：`include/func_basics.h` 里谁都能看见「这个模块对外提供什么」，`src/func_basics.c` 里是「怎么做到的」。谁依赖谁，一眼能看穿；
- **能被别人接手**：三千行的 `main.c` 没人愿意读第二次。

至于「改一个文件不用重编全部」这条好处，现在**还看不出来**：本书一条 `gcc` 命令会把列出的 `.c` 全部重编一遍。要等第 7 章讲了分步编译（先 `-c` 出 `.o`、再链接），改一个文件才只需要重编它自己。

代价是多学一件事：**头文件里写什么、不写什么。**写重了（比如把函数体贴进头文件）会撞符号，写漏了会报「隐式声明」。本书从第二个项目起要求每个例子都是 `src/ + include/` 的最小工程，练的就是这件事。

7.2 声明与定义：一个函数的两半

你写 `printf("...")` 的时候，编译器凭什么允许？因为它**见过 `printf` 的声明**——`#include <stdio.h>` 把整整一份声明搬进了你的源文件（第 1 单元 1.4 节讲过）。函数这东西从第一天起就分成两半：**声明**（只说「长什么样」）与**定义**（给出「怎么算」）。这一节把一个函数的这两半分别放进两个文件，看它们怎么对上。

先看头文件——注意它里面**一行计算都没有**：

c examples/ch03/func_basics/include/func_basics.h 第 1–21 行

```c
#ifndef FUNC_BASICS_H
#define FUNC_BASICS_H

/* 头文件里只放「声明」（declaration，也叫原型 prototype）： 头文件只放声明
   它告诉编译器这个名字长什么样 —— 几个参数、什么类型、返回什么。
   函数体不在这里，在这里贴函数体会让每个包含它的 .c 都多出一份定义。 */

/* 摄氏度换华氏度：c * 9 / 5 + 32 */
double celsius_to_fahrenheit(double c); 一行声明：类型 + 名字 + 分号

/* 从 1 加到 n；n < 1 时返回 0 */
long long sum_to(long long n);

/* 返回类型是 void：这个函数不交回任何值，调用它只能当一条语句写 */
void banner(const char *text); 返回类型是 void

/* 故意写错的函数：n >= 0 这条路走到函数末尾也没写 return（见 func_basics.c） */
int missing_return(int n); 这一条 7.5 节会亲眼看到它的下场

/* 演示实参求值顺序：两个各打印一行的函数，谁先跑标准不管（见 arg_order.c） */
void show_arg_order(void); 7.4 讲求值顺序时用它
```

4 头文件只放声明
9 一行声明：类型 + 名字 + 分号
15 返回类型是 void
18 这一条 7.5 节会亲眼看到它的下场
21 7.4 讲求值顺序时用它

（文件到这儿还没完：最后一行是 `#endif`。）

定义在 `src/func_basics.c` 里。注意它的第 3 行——**定义文件也要包含自己的头文件**，这不是客套，本节末尾你会看到它拦下了什么：

c examples/ch03/func_basics/src/func_basics.c 第 1–10 行

```c
#include <stdio.h>

#include "func_basics.h" 定义文件包含自己的头文件

/* 定义文件也包含自己的头文件：让编译器当场核对「声明」与「定义」是否一致。
   声明写成 double、定义写成 float，这里就会当场报错。 */

double celsius_to_fahrenheit(double c) { 花括号里才是真正的计算
    return c * 9.0 / 5.0 + 32.0;
}
```

3 定义文件包含自己的头文件
8 花括号里才是真正的计算

调用在 `src/main.c` 里。它只看得见头文件，看不见定义——这一句话就是整个机制的钥匙：

c examples/ch03/func_basics/src/main.c 第 1–13 行

```c
#include <stdio.h>

#include "func_basics.h" main 只包含头文件

int main(void) {
    banner("声明与定义分工");
    printf("100 摄氏度 = %.1f 华氏度\n", celsius_to_fahrenheit(100.0)); 用 double 调用
    printf("1 到 100 的和 = %lld\n", sum_to(100)); 5050 就是这一行算出来的
    printf("sum_to(0) = %lld\n", sum_to(0));
    printf("missing_return(-7) = %d\n", missing_return(-7));
    printf("missing_return(7) = %d   (这条路没写 return) \n", missing_return(7));
    /* 下面没有 return 0; —— main 走到末尾等价于 return 0，退出码仍然是 0 */
```

```
13  }
```

3 main 只包含头文件
7 用 double 调用
8 5050 就是这一行算出来的

跑出来是这样（这份输出由流水线逐字对账，最后两行是 7.5 节的主角）：

终端输出 终端输出 ch03-func-basics-out

```
== 声明与定义分工 ==
100 摄氏度 = 212.0 华氏度
1 到 100 的和 = 5050
sum_to(0) = 0
missing_return(-7) = 7
missing_return(7) = 23   (这条路没写 return)
```

五个数字连起来读：`100 摄氏度 = 212.0` 是定义算出来的；`5050` 是 1 到 100 的和（这个数你能口算：`100 × 101 / 2`）；`sum_to(0)` 走的是函数里那个 `if (n < 1) return 0;` 的短路；`missing_return(-7)` 走的是有 `return` 的那条路，交回 7；最后一行 `23` 是**没有 `return` 的那条路**交回来的东西——它一点都不像 `7`，也不像任何你能推出来的数，7.5 节解释它。

{} 语法规则 · 函数的两半长什么样

- **声明**（原型，prototype）：`返回类型 名字(参数类型 参数名, …);` —— 以分号结尾，**没有花括号**。它唯一的作用是让编译器认识这个名字：能接几个参数、各自什么类型、交回什么类型。
- **定义**：`返回类型 名字(参数类型 参数名, …) { 函数体 }` —— 花括号里是真正的计算。
- 一个函数可以有**很多份声明**（每个用到它的 `.c` 都能看见一份），但**定义只能有一份**：两份定义会在链接期撞车（本机的报错只有一行 `collect2.exe: error: ld returned 5 exit status`，见本节末尾那个陷阱）。

参数名在声明里可以省（`double celsius_to_fahrenheit(double);` 也合法），但本书一律写上——它是给人看的文档。

编译器视角 · 编译器视角：定义文件包含自己的头文件，是在让编译器当场对账

`src/func_basics.c` 第 3 行那句 `#include "func_basics.h"` 的作用，是让**同一个编译单元里同时出现声明与定义**，编译器会拿两半逐项核对：返回类型、参数个数、每个参数的类型。故意写反一次给你看：把参数写成 `float`，其余不动——下面的诊断就是编译器当场翻脸的原话。

下面是 `gcc` 的真实诊断原文（来自 `examples/ch03/func_basics/src/decl_mismatch.c`，清单 `func_basics_declmismatch.json` 声明它必须编不过）：

编译诊断 gcc 的真实诊断（原文照抄） ch03-decl-mismatch-diag

```
src/decl_mismatch.c:5:8: error: conflicting types for 'celsius_to_fahrenheit'; have 'double(float)'
    5 | double celsius_to_fahrenheit(float c) {
      |        ^~~~~~~~~~~~~~~~~~~~~
In file included from src/decl_mismatch.c:1:
include/func_basics.h:9:8: note: previous declaration of 'celsius_to_fahrenheit' with type 'double(double)'
    9 | double celsius_to_fahrenheit(double c);
      |        ^~~~~~~~~~~~~~~~~~~~~
```

按 `文件:行:列: 级别: 说明` 拆开读：`error: conflicting types for 'celsius_to_fahrenheit'; have 'double(float)'` 说的是「你这份定义的类型是 `double(float)`」；紧跟着的 `note:` 把**上一次声明**的位置（`include/func_basics.h:9`）和它的类型（`double(double)`）都摆出来。两半对不上，编译期就拦下了——这与第 1 单元里 `implicit declaration` 那条是**同一件事的两面**：那边是「一份都没有」，这边是「两份不一样」。

> ⚠ **陷阱** · **头文件里贴函数体：本机的报错只有一行，而且不告诉你为什么**
>
> 你会看到什么：把函数体整个贴进 `func_basics.h`，让 `src/main.c` 与 `src/func_basics.c` **都**包含它，再编译。终端上只有一行（下面是本机实测，命令与参数与本书一致）：
>
> `collect2.exe: error: ld returned 5 exit status`，退出码 1。
>
> **没有** `multiple definition of 'celsius_to_fahrenheit'`，也没有文件名与行号。
>
> 你会怎么误判：这一行里既没有函数名也没有文件，第一反应是「命令写错了」或者「`-I` 路径不对」，于是来回改命令行——改多少遍都是同一行。教科书上那句 `multiple definition of …` 你在本机**看不到**，会以为自己遇到的不是重名问题。
>
> 真相是两层：① 重名确实发生了——`#include` 就地展开，等于在两个编译单元里各写了一遍函数体，链接器看到两个同名符号；② `ld` 把真正的理由（`multiple definition of …`）写进了临时文件，只给 `collect2` 留了一个退出码，`collect2` 再压成那一行。第 7 章 17.4 节「本机的链接错误只有一行」把这个机制拆开讲过，包括怎么确认「缺定义」与「多定义」在本机是**同一个字节序列**。头文件里放声明、`.c` 里放定义，就是为了不让这件事发生。

7.3 值传递：函数拿到的永远是拷贝

新手第一个「明明改了却没生效」的现场，几乎都是这个。先看两个长得几乎一样的交换函数——一个白干，一个真换：

c examples/ch03/pass_by_value/src/pass_by_value.c 第 3–18 行

```c
void swap_bad(int a, int b) {
    int tmp = a;
    a = b;  a = b 改的是形参
    b = tmp;  交换的是两份副本
    /* 到这一行为止，被换掉的都只是形参这份副本；它们随本函数的栈帧一起消失 */  函数一返回，这两份副本就没了
}

void swap_ok(int *a, int *b) {
    int tmp = *a;     /* tmp 是「a 指向的那个整数」的副本 */  tmp 是「*a 的值」的副本
    *a = *b;          /* 顺着 a 里的地址，改调用者那块内存 */  顺着 a 里的地址写，改的是调用者的内存
    *b = tmp;         /* 同理：改的是调用者的内存，不是副本 */
}

void bump_bad(int n) {
    n = n + 1;        /* n 是副本，加完就扔 */
}
```

5 a = b 改的是形参
6 交换的是两份副本
7 函数一返回，这两份副本就没了
11 tmp 是「*a 的值」的副本
12 顺着 a 里的地址写，改的是调用者的内存

调用它们的是 `src/main.c`。注意两处调用写法的差别：一处递进去的是 `x`、`y`，另一处递进去的是 `&x`、`&y`：

> {} **语法规则** · **先认三个符号：`&`、`*` 与指针声明**
>
> 这一节躲不开三个符号。它们属于指针，第 4 章（`chapters/ch04/`）会讲透；这里只认到「够用」：
>
> - `&x`：**取地址**，读作「x 住在哪」。它的类型是 `int *`（指向 `int` 的指针）；
> - `int *a`：声明一个**指针变量** `a`，盒子里装的是一个地址；
> - `*a`：**解引用**，读作「`a` 里那个地址上的那个整数」。写在赋值号左边就是「改那块内存」。
>
> 于是 `swap_ok(&x, &y)` 是「把 x、y 的地址递进去」，`*a = *b` 是「顺着地址，把对方的值写过来」。

c examples/ch03/pass_by_value/src/main.c 第 5–21 行

```c
int main(void) {
    int x = 1;
```

```c
    int y = 2;

    printf("调用前：x = %d，y = %d\n", x, y);
    swap_bad(x, y);  传的是 x、y 的值
    printf("swap_bad(x, y) 之后：x = %d，y = %d\n", x, y);
    swap_ok(&x, &y);  传的是两个地址
    printf("swap_ok(&x, &y) 之后：x = %d，y = %d\n", x, y);

    int n = 10;  一个普普通通的 int
    bump_bad(n);  函数里那个 n 加了 1
    printf("bump_bad(n) 之后：n = %d（函数里那个 n 加了 1，外面这个没动）\n", n);

    printf("sizeof(int) = %zu 字节，sizeof(int *) = %zu 字节\n", sizeof(int), sizeof(int *));
    return 0;
}
```

10 传的是 x、y 的值
12 传的是两个地址
15 一个普普通通的 int
16 函数里那个 n 加了 1

终端输出 终端输出 ch03-pass-by-value-out

```
调用前：x = 1，y = 2
swap_bad(x, y) 之后：x = 1，y = 2
swap_ok(&x, &y) 之后：x = 2，y = 1
bump_bad(n) 之后：n = 10（函数里那个 n 加了 1，外面这个没动）
sizeof(int) = 4 字节，sizeof(int *) = 8 字节
```

五行里藏着全部答案。第 2 行说明 `swap_bad` **真的跑了**，只是没改到外面；第 3 行说明 `swap_ok` 改到了。最后一行那两个数字（本机实测 **4** 与 **8**）是这节的钥匙：一个 `int` 是 4 个字节，一个指针是 8 个字节——但**指针也是值**，`swap_ok(&x, &y)` 递进去的仍然是「那份 8 字节内容的拷贝」，只不过拷贝上写的是一个真地址。

大白话 · 用大白话说「值传递」

把函数想成一台复印机：你递进去的每样东西，它都先复印一份再动手，你在原件上画的花它一概看不见。

- `swap_bad(x, y)` 递进去的是**两个数字**（1 和 2）。函数在两张复印件上换来换去，原件 `x`、`y` 从头到尾没动过；
- `bump_bad(n)` 同理：函数里的 `n` 是复印件，加一之后复印件被扔进碎纸机；
- `swap_ok(&x, &y)` 递进去的也还是拷贝，但拷贝上写的是**「x 住在哪」这张纸条**。纸条是复印件，**纸条上的地址是真的**——顺着它写，改到的就是原件。

所以严格的说法只有一句：**C 里所有参数都是值传递；想让函数改到调用者的东西，你只能把它的地址递进去。** C 里没有「引用参数」这回事（那是 C++ 的 `int &`），想改就传指针；不想改、也不想被拷一份大对象，也传指针（加 `const`）。

⚠ 陷阱 · 「我明明在函数里改了，外面没变」

你会看到什么：`swap_bad(x, y)` 跑完，`x`、`y` 一点没变，程序不报错、不警告，退出码还是 0。

你会怎么误判：先怀疑「函数没被调用」，于是在 `swap_bad` 里加一句 `printf` —— 它明明打印了；再怀疑「是不是要写 `swap(int &a, int &b)`」——那是 C++ 的写法，C 编译器会直接报语法错误；最后有人开始给形参加 `*`、给实参加 `&`，靠碰运气试出来。

真相是：形参 `a`、`b` 是**两个全新的变量**，住在被调函数自己的栈帧里（7.6 节会把它们的位置画出来）。函数一返回，那层栈帧连同这两个变量一起消失。**「传进来的是什么」和「改的是谁」是两件事**：传 `int` 改的只是副本，传 `int *` 才能改到别人。

7.4 参数列表：`f(void)` 与 `f()` 不是一回事

同一个函数，声明里写成 `(void)` 还是写成 `()`，看起来只差几个字符，实际是**两种完全不同的承诺**。这一节的两个函数各代表一种：

c examples/ch03/old_style_decl/include/old_style_decl.h 第 4-9 行

```c
/* (void)：参数列表写明了「没有参数」。想多传一个实参，编译期就会被拦下。 */
int modern(void); 写明了「没有参数」

/* ()：空参数列表 —— 它说的不是「没有参数」，而是「参数情况没说」。
   编译器只记下「返回 int」，别的什么都不知道。 */
int legacy(); 空参数列表 = 参数情况没说
```

5 写明了「没有参数」
9 空参数列表 = 参数情况没说

c examples/ch03/old_style_decl/src/old_style_decl.c 第 3-9 行

```c
int modern(void) {
    return 7;
}

int legacy(int n) {          /* 定义的时候是写清楚的：要一个 int */ 定义时写清楚了：要一个 int
    return n * 2;
}
```

7 定义时写清楚了：要一个 int

调用方在 `src/main.c`：第 7 行是正常调用，第 8 行**故意多传了一个实参**——`legacy` 声明里没有参数信息，编译器只能放行：

c examples/ch03/old_style_decl/src/main.c 第 5-10 行

```c
int main(void) {
    printf("modern()        = %d\n", modern());
    printf("legacy(21)      = %d\n", legacy(21)); 一个参数，正常
    printf("legacy(21, 999) = %d   (多出来的那个 999 没人管) \n", legacy(21, 999)); 多了一个 999，编译器不吭声
    return 0;
}
```

7 一个参数，正常
8 多了一个 999，编译器不吭声

终端输出 终端输出 ch03-old-style-decl-out

```
modern()        = 7
legacy(21)      = 42
legacy(21, 999) = 42   (多出来的那个 999 没人管)
```

第 3 行是真跑出来的结果：`999` **被无声地吃掉了**，程序照常退出 0。注意这不是「编译器变聪明了」，而是它**从一开始就不知道** `legacy` **要几个参数**。同一个错误换成 `modern` 就完全不同——下面是真实的 stderr 原文（来自 `src/wrong_args.c`，清单 `old_style_decl_wrongargs.json` 声明它必须编不过）：

编译诊断 gcc 的真实诊断（原文照抄） ch03-old-style-decl-diag

```
src/wrong_args.c: In function 'main':
src/wrong_args.c:5:12: error: too many arguments to function 'modern'; expected 0, have 1
    5 |     return modern(1);
      |            ^~~~~~ ~
In file included from src/wrong_args.c:1:
include/old_style_decl.h:5:5: note: declared here
    5 | int modern(void);
      |     ^~~~~~
```

`error: too many arguments to function 'modern'; expected 0, have 1` —— 「多传了一个实参」，并且 note 明确指回声明所在的那一行。**同一个「多传一个实参」的动作，因为声明的写法不同，一个当场报错、一个静默通过**，这就是这两个字符的全部意义。

`()` 这种空参数列表是 C 早期（还没有原型时）的遗留，把整个程序扫一遍还能看到它，所以本书的写法纪律是：

工程惯例 · 工程惯例：一律写 `(void)`，不写 `()`

这是惯例，不是标准要求（标准两种都接受）。三条理由：

- `(void)` **让编译器替你抓错**：参数个数不对当场报错，而不是等你运行起来看到一个荒唐的数；
- `-Wall -Wextra` **不管这件事**：本机实测，加 `-Wstrict-prototypes` 才会报（下面那块会话的原文）；这个开关不在本书的基础参数里，所以没人拦你；
- 头文件是给别人看的：`int legacy();` 会被读成「没有参数」——**读的人替你猜错，比编译器不报错更贵。**

新增的代码一律 `(void)`；看到老代码里的 `()`，动手改之前先看清它是不是在用「参数情况没说」的那半边（有些老代码靠它写「参数由调用者与被调者自己约定」的函数——那**不是**可变参数；可变参数是在参数列表末尾写 `...`，第 6 章讲 `printf` 时才展开）。

那条警告长什么样，抓下来是这样：

终端会话 `-Wstrict-prototypes 才会报的一条警告（原文照抄）` `ch03-proto-warn`

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -Wstrict-prototypes -O0 -g -fdiagnostics-color=never -Iinclude -c src/old_style_decl.c -o old_style_decl.o
In file included from src/old_style_decl.c:1:
include/old_style_decl.h:9:1: warning: function declaration isn't a prototype [-Wstrict-prototypes]
    9 | int legacy();
      | ^~~
compile-exit=0
```

注意两件事：这条警告**只在加了** `-Wstrict-prototypes` **时出现**（本书的固定参数里没有它），而且它说的是「这不是一个原型」而不是「你写错了」——`int legacy();` 在标准里是合法声明，只是**信息不足。**至于「多传实参」本身，标准的处理是「行为未定义」——**别把它当可依赖的贴心行为**，本机打印 42 只是这台机器这次的选择。

标准原文 · 标准原文：`(void)` 与 `()` 是两种不同的声明

C 语言标准 6.7.6.3 节「Function declarators (including prototypes)」第 14 段里有两句专门说这两种写法：

> An empty list in a function declarator that is not part of a definition of that function specifies that no information about the number or types of the parameters is supplied. ... The special case of an unnamed parameter of type void as the only item in the list specifies that the function has no parameters.

大意：**空参数列表说的是「参数个数与类型都没告诉你」；而唯一的无名** `void` **参数说的是「这个函数没有参数」。**两句话把 7.4 那一对函数解释得干干净净：`modern(void)` 是一条完整的信息，`legacy()` 是一条「我不知道」的占位。

出处：ISO/IEC 9899:2018（C17）§6.7.6.3 第 14 段。

实参的求值顺序：你写的顺序不是执行的顺序

`add(left(), right())` 里，`left` 先跑还是 `right` 先跑？探针只有十来行：

`c` `examples/ch03/func_basics/src/arg_order.c` 第 5–24 行

```c
/* 两个函数各打印一行，各自返回一个整数 */
static int left(void) {
    printf("left ");
    return 1;
}

static int right(void) {
    printf("right ");
    return 2;
}
```

```c
static int add(int a, int b) {
    return a + b;
}

void show_arg_order(void) {
    printf("add(left(), right()) 的打印顺序: ");
    int sum = add(left(), right());  写的是 left 在前、right 在后
    printf("→ 结果 %d\n", sum);
}
```

22 写的是 left 在前、right 在后

终端输出 终端输出 ch03-arg-order-out

```
add(left(), right()) 的打印顺序: right left → 结果 3
```

写的是 `left` 在前，打出来的却是 `right` 在前。 这不是笔误：本机实测 `-O0` 与 `-O2` 两档都是 `right left`（x86-64 上编译器从右往左准备实参）。标准对这件事的态度是「未指定」：**顺序由实现决定，实现不必告诉你它选了哪个。**所以别在实参里放「谁先谁后会影响结果」的写法——`f(i++, i)` 这种更糟，它在标准里连「未指定」都算不上，是未定义行为。

标准原文 · **标准原文：未指定行为是什么**

> C 语言标准 3.4.4 节给「未指定行为」下的定义是：
>
> > unspecified behavior: behavior, for a well-formed program construct and correct data, that depends on the implementation. The implementation is not required to document which behavior occurs.
>
> 大意：**未指定行为**是「取决于实现」的行为，实现**不必**告诉你它选了哪个。它和「未定义行为」不是一回事：未指定是「几种都合法，实现选一个」，未定义是「标准什么都没说」。
>
> 出处：ISO/IEC 9899:2018（C17）§3.4.4。

大白话 · **用大白话说「未定义」与「未指定」**

这两个词在标准里都是有确切定义的术语，别混：

- **未定义行为**：标准**什么都没说**——程序做什么都行，包括你看到的那个 42、另一个数、或者当场崩掉；
- **未指定行为**：标准给了**几种都合法**的结果，具体选哪个由实现决定，但**必须选一个**。函数实参的求值顺序就是这一类。

你唯一能做的：**别写依赖顺序的代码。**要「先算 A 再算 B」，就写成两条语句——语句之间是有先后关系的，实参之间没有。

7.5 返回值：`void`、漏写 `return`、`main` 的隐式 `return 0`

`main.c` 的最后两行输出里，藏着一个「不该出现的数」。回到 `func_basics.c` 的定义部分——`banner` 与 `missing_return`：

c examples/ch03/func_basics/src/func_basics.c 第 23–33 行

```c
void banner(const char *text) {  返回类型 void：不交回值
    printf("== %s ==\n", text);  只能当语句用，不能写 x = banner(…)
}

int missing_return(int n) {
    if (n < 0) {  n < 0 的那条路
        return 0 - n;  交回 n 的相反数
    }
    /* n >= 0 时直接走到函数末尾：这里没有 return。
       编译器会给一条 -Wreturn-type 警告（作者故意留着）。 */
```

```
33  }
```

23 返回类型 void：不交回值
24 只能当语句用，不能写 x = banner(...)
28 n < 0 的那条路
29 交回 n 的相反数

`missing_return(7)` 走的是最后那条路：**函数体从头走到尾，没有 `return`**，可调用方还在等一个 `int`。编译器对这件事有意见，而且说得很直接：

终端会话 漏写 return 时编译器的原话（原文照抄） ch03-return-warn

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/func_basics.c -o func_basics.o
src/func_basics.c: In function 'missing_return':
src/func_basics.c:33:1: warning: control reaches end of non-void function [-Wreturn-type]
   33 | }
      | ^
compile-exit=0
```

`warning: control reaches end of non-void function` 直译是「控制流走到了非 void 函数的末尾」；方括号里的 `-Wreturn-type` 是管这条诊断的开关（`-Wall` 就带着它）。**注意级别是 `warning` 不是 `error`：编译照样成功，退出码 0**——所以程序能跑，跑到你面前给你一个 23。那个 23 不是「23 这个特殊值」，它是这段代码在这次编译、这台机器上**碰巧**留在返回值寄存器里的东西。本机实测 `-O0`、`-O1`、`-O2`、`-O3` 四档**都**打印 23——四档一样不代表它「有定义」，只代表这台机器的这四个二进制在这一点上碰巧相同；换编译器、换机器、改一行动不相关的代码，它就可能变成别的数。

标准把这件事写得很干脆，6.9.1 节「Function definitions」第 12 段：

标准原文 · 标准原文：漏写 return，而调用方用了那个值

> If the } that terminates a function is reached, and the value of the function call is used by the caller, the behavior is undefined.

大意：**函数末尾那个 `}` 被走到了、而调用方又用了这次调用的返回值——行为未定义。**换句话说：`printf("...%d", missing_return(7))` 是未定义行为（调用方用了那个值），而如果你写 `missing_return(7);` 什么都不接，标准就没什么可说的。

出处：ISO/IEC 9899:2018（C17）§6.9.1 第 12 段。

本机实测的现象就是那行 `missing_return(7) = 23`：**同一个二进制反复跑，它稳定地打印 23**——「稳定」不等于「有定义」，它只是「这次编译产出的这份机器码恰好这样」。

`main` 是个例外，标准专门给了它一条宽恕，5.1.2.2.3 节「Program termination」第 1 段：

标准原文 · 标准原文：`main` 走到末尾等价于 `return 0`

> If the return type of the main function is a type compatible with int, a return from the initial call to the main function is equivalent to calling the exit function with the value returned by the main function as its argument; reaching the } that terminates the main function returns a value of 0.

大意：**`main` 走到底不给 `return`，等价于 `return 0`**。所以上面 `src/main.c` 那个「故意没写 `return 0;`」的程序，流水线对账的退出码仍然是 **0**（清单里 `"expect": { "exit": 0 }` 这一条不是装饰，退出码对不上整条清单就失败）。**这条宽恕只给 `main`**——别的函数都不享受，包括 `missing_return`。

出处：ISO/IEC 9899:2018（C17）§5.1.2.2.3 第 1 段。

实验记录 · 本机实测：这一节的几个读数

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；命令与本书固定参数一致：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/func_basics.c -o app.exe`；
- `missing_return(7)` 打印 **23**（`-Wextra` 全开、不做优化），编译退出码 **0**、程序退出码 **0**——注意：**整条程序没有一行 `return 0;` 出现在 `main` 里**，退出码 0 靠的正是上面 5.1.2.2.3 那条宽恕；
- 这份输出被写进 `output` 块并由流水线逐字对账：程序改一个字符、换个档位，块就对不上（那正是我们要的——数字必须能被复现，也要能被抓住）。

7.6 栈帧：一次调用在内存里留下什么

7.3 那句「形参住在被调函数自己的栈帧里」到现在还是个说法。这一节把它变成**能看见的东西**：三层嵌套调用，每层打印自己的局部变量的地址。

c examples/ch03/stack_frames/src/stack_frames.c 第 5–22 行

```c
/* static 函数：这个名字只在 stack_frames.c 里可见，别的 .c 里写 inner 也指不到它 */ static 函数：只有本文件看得见
static int inner(int depth) { inner 的帧
    int local_inner = depth;              /* 这一层自己的局部变量 */ 这一层自己的局部变量
    printf("inner    depth=%d  &local_inner  = %p\n", depth, (void *)&local_inner);
    return local_inner * 10;
}

static int middle(int depth) {
    int local_middle = depth; middle 也有自己的一份
    printf("middle   depth=%d  &local_middle = %p\n", depth, (void *)&local_middle);
    return inner(depth + 1) + local_middle;
}

int outer(int depth) {
    int local_outer = depth; outer 的也一样
    printf("outer    depth=%d  &local_outer  = %p\n", depth, (void *)&local_outer);
    return middle(depth + 1) + local_outer;
}
```

5 static 函数：只有本文件看得见
6 inner 的帧
7 这一层自己的局部变量
13 middle 也有自己的一份
19 outer 的也一样

c examples/ch03/stack_frames/src/main.c 第 5–11 行

```c
int main(void) {
    int local_main = 0;                   /* main 这一层的局部变量 */ main 自己也有局部变量
    printf("main     depth=0  &local_main   = %p\n", (void *)&local_main); 把它的地址打出来
    int total = outer(1); 从这里开始往下叫三层
    printf("三层返回后加起来 = %d\n", total);
    return 0;
}
```

6 main 自己也有局部变量
7 把它的地址打出来
8 从这里开始往下叫三层

终端输出 终端输出 ch03-stack-frames-out

```
main     depth=0  &local_main   = 00000000005FFEA8
outer    depth=1  &local_outer  = 00000000005FFE6C
middle   depth=2  &local_middle = 00000000005FFE2C
inner    depth=3  &local_inner  = 00000000005FFDEC
三层返回后加起来 = 33
```

四个地址连起来看：`main` 的 `0x5FFEA8` → `outer` 的 `0x5FFE6C` → `middle` 的 `0x5FFE2C` → `inner` 的 `0x5FFDEC`，**一路变小**。但先别把这几个差值当成「帧的大小」：**局部变量的地址不是帧的基址**。gdb 的 `info frame` 给出四层的帧基址是 `0x5ffec0`、`0x5ffe80`、`0x5ffe40`、`0x5ffe00`（下面第 2、3 块会话里有原文），相邻两帧**正好差 `0x40`**；而一个局部变量在帧里只占开头几个字节（`inner` 的 `local_inner` 在 `0x5ffdec`，是帧基址减 `0x14`），所以两个局部变量的地址差可以是 `0x3C`。**帧的大小看帧基址的差，别拿局部变量的地址差去顶替**——我自己在这里看错过一次。下面按帧基址画：

第 1 帧是 `main` 一进去的样子；第 2、3、4 帧是层层往下：每调一层，栈上就多出一帧（**每帧 `0x40` 字节**），里面装着**形参、这一层的局部变量、以及一条返回地址**。第 4 帧是「栈最深」的时刻——四个函数的局部变量**同时**活在内存里。返回的过程就是倒着走：`inner` 先返回，它那一帧消失；再从 `middle` 的返回地址回到 `outer` 第 21 行的下一句。

> **调试实录** · **调试实录：用 gdb 把一帧拆开**
>
> 说法要靠工具钉死。下面三段会话是真的（`examples/ch03/session-gdb-frames.txt`）：在 `inner` 上打断点，先 `bt` 看调用链，再 `info frame` 看当前这一帧，`p` 打两个值，`finish` 让函数跑完，最后重新跑一次、用 `p $rbp` 把四层的帧指针逐个打出来。
>
> 读法三步：① `bt` 里 `#0` 是最里面、编号越大越往外，每条后面的 `文件:行` 是它停的位置；② `info frame` 的每一行都对应这一帧里的一件东西（帧基址、参数、局部变量、返回地址）；③ `p $rbp` 给出帧指针，相邻两层的差就是帧的大小。
>
> 结论先放在这里：**一帧 `0x40` 字节，帧基址 = `rbp + 0x10`，局部变量在帧基址往下几个字节的地方**—第 3 块会话的四个 `rbp` 与第 2 块的 `frame at` 对得上。

终端会话 第 1 块：调用链（bt） ch03-gdb-bt

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/stack_frames.c -o stack_frames.exe
$ gdb --batch -ex "break inner" -ex run -ex bt -ex "info frame" -ex "p depth" -ex "p &local_inner" -ex finish
-ex run -ex "p $rbp" -ex "frame 1" -ex "p $rbp" -ex "frame 2" -ex "p $rbp" -ex "frame 3" -ex "p $rbp"
./stack_frames.exe
Breakpoint 1 at 0x14000150b: file src/stack_frames.c, line 7.
[New Thread 77708.0x1b498]
Thread 1 hit Breakpoint 1, inner (depth=3) at src/stack_frames.c:7
7           int local_inner = depth;              /* 这一层自己的局部变量 */
#0  inner (depth=3) at src/stack_frames.c:7
#1  0x00007ff6f09a1571 in middle (depth=2) at src/stack_frames.c:15
#2  0x00007ff6f09a15b2 in outer (depth=1) at src/stack_frames.c:21
#3  0x00007ff6f09a14e1 in main () at src/main.c:8
```

接着 `info frame` 把**当前这一帧**的内容逐项列出来，`p` 打两个值，`finish` 让函数跑完——它会告诉你这次调用的返回值：

终端会话 第 2 块：info frame / p / finish 的原文 ch03-gdb-frame

```
Stack level 0, frame at 0x5ffe00:
 rip = 0x7ff6f09a150b in inner (src/stack_frames.c:7); saved rip = 0x7ff6f09a1571
 called by frame at 0x5ffe40
 source language c.
 Arglist at 0x5ffdf0, args: depth=3
 Locals at 0x5ffdf0, Previous frame's sp is 0x5ffe00
 Saved registers:
  rbp at 0x5ffdf0, rip at 0x5ffdf8, xmm15 at 0x5ffdf8
$1 = 3
$2 = (int *) 0x5ffdec
0x00007ff6f09a1571 in middle (depth=2) at src/stack_frames.c:15
15          return inner(depth + 1) + local_middle;
Value returned is $3 = 30
```

这几行逐个点名（对着上面那张图的第 4 帧看）：

- `Stack level 0, frame at 0x5ffe00` —— 当前帧（`#0`，也就是 `inner` 的帧）的**帧基址**是 `0x5ffe00`。注意它不是局部变量的地址：`&local_inner` 是 `0x5ffdec`，比基址低 `0x14`；
- `Arglist at 0x5ffdf0, args: depth=3` —— **参数**从 `0x5ffdf0` 开始放，值就是 `3`（`p depth` 打出的 `$1 = 3` 是同一个数）；
- `Locals at 0x5ffdf0` —— **局部变量**的位置（`p &local_inner` 打出 `$2 = (int *) 0x5ffdec`，与程序自己打印的地址一致）；
- `saved rip = 0x7ff6f09a1571` —— **返回地址**：`inner` 返回时要跳回的那条指令。把它和 `bt` 里的 `#1 0x00007ff6f09a1571 in middle … at src/stack_frames.c:15` 对照：两个数是**同一个值**（`info frame` 不补前导零，`bt` 补到 16 位），而 `bt` 那一行还告诉你它落在 `middle` 的第 15 行；
- `called by frame at 0x5ffe40` —— 调用者（`middle`）的帧基址；
- `Value returned is $3 = 30` 是 `finish` 报的：`inner(3)` 返回 `3 * 10 = 30`，而 `middle` 第 15 行的表达式正是 `inner(depth + 1) + local_middle`。

最后一件事：帧与帧到底隔多远。重新跑一次，用 `p $rbp` 逐层打帧指针（`frame N` 是「切到第 N 层」，不改变正在跑的程序）：

终端会话 第 3 块：四层的帧指针（p $rbp） ch03-gdb-rbp

```
[New Thread 145712.0x24c3c]
Thread 1 hit Breakpoint 1, inner (depth=3) at src/stack_frames.c:7
7           int local_inner = depth;              /* 这一层自己的局部变量 */
$4 = (void *) 0x5ffdf0
#1  0x00007ff6f09a1571 in middle (depth=2) at src/stack_frames.c:15
15          return inner(depth + 1) + local_middle;
$5 = (void *) 0x5ffe30
#2  0x00007ff6f09a15b2 in outer (depth=1) at src/stack_frames.c:21
21          return middle(depth + 1) + local_outer;
$6 = (void *) 0x5ffe70
#3  0x00007ff6f09a14e1 in main () at src/main.c:8
8           int total = outer(1);
$7 = (void *) 0x5ffeb0
```

四个 `rbp` 是 `0x5ffdf0`、`0x5ffe30`、`0x5ffe70`、`0x5ffeb0`——**两两相差 `0x40`**，所以四层的帧基址（`rbp + 0x10`）是 `0x5ffe00`、`0x5ffe40`、`0x5ffe80`、`0x5ffec0`。这和上面第 2 块里 `frame at 0x5ffe00` / `called by frame at 0x5ffe40` 完全对得上。

注意 `#1`、`#2` 后面那串 `0x00007ff6f09axxxx` 每次编译、每次运行都会变，别去记它；**要记的是 `src/stack_frames.c:15`、`src/main.c:8` 这样的行号**，它们是稳定的，也是你真正要读的东西。

实验记录 · 本机实测：栈往哪边长、一帧多少钱

- 工具链：gcc 16.1.0（`-std=c17`）、`-O0 -g`，目标三元组 x86_64-w64-mingw32；命令与块里贴的一致（先编译，再用 `gdb --batch` 跑）；
- 帧基址：`main` **`0x5ffec0`**、`outer` **`0x5ffe80`**、`middle` **`0x5ffe40`**、`inner` **`0x5ffe00`**——相邻两帧差 **`0x40` = 64 字节**；
- 同一时刻四个局部变量的地址：`0x5FFEA8`、`0x5FFE6C`、`0x5FFE2C`、`0x5FFDEC`——它们**不是**帧基址，各自在帧里占固定偏移（`inner` 是基址减 `0x14`，`main` 是减 `0x18`），所以 `main` 与 `outer` 的两个变量差 `0x3C` 而不是 `0x40`；
- `finish` 报的返回值 **`30`**、程序自己打印的合计 **`33`**（`3*10 + 2 + 1 = 33`，两条路都算得出来）；
- 「地址一路变小」是**本机**的现象，不是标准规定的：标准只管生命周期，不管栈往哪边长。换个架构（比如栈向上长的处理器）你可能会看到相反的顺序——所以判断「谁是调用者」要靠 `bt` 的顺序，别靠地址大小。

7.7 递归：基准情形与递归情形

函数可以调用别人，也可以调用**自己**。递归看着玄，其实是「把问题缩小一点，再交给同一个函数」——它只有两个必须写对的位置：

c examples/ch03/factorial_both/src/factorial_both.c 第 13–24 行

```c
/* 带打印的递归：每一层进入、返回各打一行 */
static long long rec_traced(long long n, int indent) {
    print_at(indent, "进入 factorial_rec", n);
    if (n <= 1) {                          /* 基准情形：不再往下叫自己 */ 基准情形：不再往下叫自己
        print_at(indent, "基准情形，返回", 1);
        return 1;
    }
    long long below = rec_traced(n - 1, indent + 1);    /* 递归情形 */ 递归情形：把 n-1 交给同一个函数
    long long result = n * below;
    print_at(indent, "返回", result); 子调用返回之后才轮到这里
    return result;
}
```

16 基准情形：不再往下叫自己
20 递归情形：把 n-1 交给同一个函数
22 子调用返回之后才轮到这里

c examples/ch03/factorial_both/src/main.c 第 5–12 行

```c
int main(void) {
    printf("算 4!，把每层的进入与返回都打出来：\n");
    printf("4! = %lld\n", factorial_rec(4));  递归从这里进去，每层都会打印

    printf("\n--- 两种写法算同一个数 ---\n");
    compare(20);
    return 0;
}
```

7 递归从这里进去，每层都会打印

终端输出 终端输出 ch03-factorial-both-out

```
算 4!，把每层的进入与返回都打出来：
进入 factorial_rec(4)
  进入 factorial_rec(3)
    进入 factorial_rec(2)
      进入 factorial_rec(1)
      基准情形，返回(1)
    返回(2)
  返回(6)
返回(24)
4! = 24
--- 两种写法算同一个数 ---
20! 递归算 = 2432902008176640000
20! 迭代算 = 2432902008176640000
```

缩进是程序打出来的（`print_at` 按层数补两个空格），所以这一屏就是调用树的**真实形状**：先一路往下四层，到 `factorial_rec(1)` 撞上基准情形，然后一层一层「返回(2)、(6)、(24)」往外交。最后两行是另一种校验：同一个 `20!`，递归和迭代都得 `2432902008176640000`（这个数超过了 32 位，`long long` 刚好装得下——第 2 章讲整数边界时你见过类似的表）。

调用树 **递归展开：每层有自己的 n，谁也替不了谁**

第 1 步 **main 算出 4! 的第一步**

```
main() ：等 factorial_rec(4)
factorial_rec(4) ：本层的 n = 4，below 还没算出来
```

第 2 步 **递归情形把 n 减一再叫自己**

```
main() ：等
factorial_rec(4) ：n = 4，在等 3 * below
└─ factorial_rec(3) ：n = 3，在等 2 * below
   └─ factorial_rec(2) ：n = 2，在等 1 * below
factorial_rec(4) → factorial_rec(3) rec_traced(n - 1)
```

第 3 步 **撞上基准情形，开始往上返回**

```
main() ：等
factorial_rec(4) ：n = 4
└─ factorial_rec(3) ：n = 3
   └─ factorial_rec(2) ：n = 2
      └─ factorial_rec(1) ：n = 1，基准情形，交回 1
factorial_rec(2) ⇒ factorial_rec(1) 返回 1
```

第 4 步 **每层的乘法这才轮到自己做**

```
main()
factorial_rec(4) ：返回 24
└─ factorial_rec(3) ：返回 6
   └─ factorial_rec(2) ：返回 2
factorial_rec(2) ⇒ factorial_rec(3) 返回 2
factorial_rec(3) ⇒ factorial_rec(4) 返回 6
factorial_rec(4) ⇒ main() 返回 24
```

第 2 帧是这一节最该记住的一张：**三层 `factorial_rec`（4、3、2）同时活着，每层一份自己的 `n` 与 `below`**——这正是 7.6 讲的栈帧，只不过调用者也是被调者；第 3 帧再把基准情形那一层（`n = 1`）加进来，四层就齐了。第 3 帧也是基准情形起作用的地方：它把「一路往下」变成「开始往上」。第 4 帧说明为什么返回要一层层来：每层的乘法都得等里面的结果。

{} 语法规则 · 写递归的两半

- **基准情形（base case）**：一个能让函数**直接给出答案、不再调自己**的条件（这里是 `n <= 1`）。少写它，函数会一路往下永不回头；
- **递归情形（recursive case）**：把问题**变小一点**再交给自己（这里是 `n - 1`）。变小这件事必须是**可终止**的：每次都在向基准情形靠近。

两句检查口诀：①「**有没有一条路不调自己？**」②「**每一层的调用都在往那条路走吗？**」两问都答「是」，递归才算写对。

工程惯例 · 工程惯例：递归还是迭代，怎么选

这是**工程惯例**，标准对「该用哪种」没有任何规定；它也不是对错题，是**取舍**：

- **选递归**：问题本身就是递归定义的（树的遍历、分治、括号匹配、后续要学的链表与回溯）。这时递归版本往往比迭代版本**短得多、也更容易看出对不对**；
- **选迭代**：问题只是「重复做一件事」（累加、查找、计数）。这时循环一目了然，还不占额外的栈（7.8 会看到代价）；
- **判断标准**：先写清楚的那个版本通常是对的；只有当**深度不可控**（输入能大到几十万层）或者**每层开销真的成了瓶颈**时，才为了性能改写成迭代。

本书的立场：**先用递归把逻辑写对，再按需要换迭代。**反过来的顺序——为了「省一点栈」把递归手工改成循环——是新手最常见的自伤方式（第 7.8 节末尾会看到编译器自己就会做这件事）。

7.8 递归的代价：栈是有限的

递归的账单在**栈**上：每进一层，就要多留一帧。帧多大由**这个函数自己**决定：7.6 那四个函数各留 `0x40`（64 字节）；本节这个 `sum_down` 只留 **48 字节**（本机 `-O0` 的汇编是 `pushq %rbp` + `subq $32, %rsp`，再加一条返回地址：8 + 32 + 8 = 48）。栈有多大？不是无限的，也不是标准规定的——它由**操作系统和链接器**定，本机约 **2 MB**。`DEPTH = 200000` 时，光这些帧就要 `200000 × 48 B ≈ 9.2 MB`，是限额的四倍多——所以下面它必崩，不是「碰巧」。

c examples/ch03/deep_recursion/src/deep_recursion.c 第 3–8 行

```c
long long sum_down(long long n) {
    if (n == 0) {
        return 0;
    }
    return n + sum_down(n - 1);      /* 子调用返回之后还要做加法：本层不能提前退出 */ 子调用返回之后还要做加法，所以这一帧不能提前扔
}
```

7 子调用返回之后还要做加法，所以这一帧不能提前扔

c examples/ch03/deep_recursion/src/main_deep.c 第 5–13 行

```c
#define DEPTH 200000 二十万层

int main(void) {
    printf("开始累加 1..%d（每层都要等子调用，栈帧一层都扔不掉）\n", DEPTH);
    fflush(stdout);                  /* 崩溃时缓冲区不会自己刷，先手动刷出来 */ 崩溃时缓冲区不会自己刷，先手动刷出来
    long long total = sum_down(DEPTH);
    printf("累加结果 = %lld\n", total);
    return 0;
```

```
13  }
```

5 二十万层
9 崩溃时缓冲区不会自己刷，先手动刷出来

终端输出 终端输出 ch03-deep-recursion-out

```
开始累加 1..200000（每层都要等子调用，栈帧一层都扔不掉）
```

这行字的用处是**证明崩溃前它真的跑过**：调用 `sum_down` 之前先 `fflush(stdout)` 把这句话刷出去，否则进程被终止时缓冲区里的内容会一起消失（这是本书的一条实测纪律，第 4 章讲过同类现象）。整份现场是这样：

终端会话 二十万层递归的真实现场（-O0 与 -O2 各跑一次） ch03-deep-run

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main_deep.c src/deep_recursion.c -o deep_O0.exe
$ ./deep_O0.exe
开始累加 1..200000（每层都要等子调用，栈帧一层都扔不掉）
$ echo $?
127
$ powershell -NoProfile -Command "(Start-Process -FilePath ./deep_O0.exe -Wait -PassThru).ExitCode"
-1073741571
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main_deep.c src/deep_recursion.c -o deep_O2.exe
$ ./deep_O2.exe
开始累加 1..200000（每层都要等子调用，栈帧一层都扔不掉）
累加结果 = 20000100000
$ echo $?
0
```

三件事要连起来读：

- `-O0` **那一版爆了**：程序打印了第一行就没了下文，没有 `累加结果`，也没有任何错误信息（栈溢出不是异常，没人接得住）；
- **退出码有两种口径**：MSYS 的 bash 报 **127**，而进程真实的 Windows 退出码是 **-1073741571**（去符号读作 `0xC00000FD`，也就是 `STATUS_STACK_OVERFLOW` 栈溢出）。本书的清单写**后者**：`"expect": { "exit": 3221225725 }`（`-1073741571` 的无符号写法）。127 不是「命令找不到」吗？在这台机器的 bash 里，`0xC00000FD`（栈溢出）与 `0xC0000409`（`abort`）都被报成 127、一个字都不提示，而普通的访问冲突 `0xC0000005`（空指针解引用）会报 **139** 并打印 `Segmentation fault`（三种我都实测过）——所以别拿 bash 的数字当分类依据，**Windows 退出码才是可对账的那一个**；
- `-O2` **那一版算完了**：`20000100000`，退出码 **0**。同一份源码、同一个 `DEPTH`，只是优化档位不同，结果就不同——7.7 末尾那句「编译器自己就会做这件事」说的就是它。

第 1 帧说明爆栈不是「算错了」，而是**位置不够**：每层都老老实实留了一帧在等（非尾递归没法提前扔）。第 2 帧是那一刻的结局。

编译器视角 · 编译器视角：同一份源码，-O0 爆栈、-O2 没事

上面那份会话里两个档位的差别不是巧合，它的判据写在清单里：`examples/ch03/deep_recursion/deep_recursion_optdiff.json` 用的键是 `"mode": "detect"`、`"detect": "opt-diff"`、`"expect": { "differs": "exit" }`——这个检测模式的玩法是：同一份源码分别在 `-O0` 与 `-O2` 下**各编译一次、各跑一次**，看行为是否不同。两个档位不是本书随手写的，它们来自 `build/toolchain.json` 的 `detection.opt-diff.compare_flags`（改配置就改行为，闸门里没有硬编码的兜底）。

它抓到的差异是**退出码**：`-O0` 下 `3221225725`（栈溢出），`-O2` 下 `0`。原因不是「编译器变魔术」，而是 `-O2` 把这种递归**改写成了循环**。这不是猜的，`gcc -S` 的汇编能直接数出来：

- 命令 `gcc -std=c17 -Iinclude -O0 -S src/deep_recursion.c`，输出里 `call …sum_down` / `call … count_down_tail` 一共 **2** 处（每层真要调一次）；
- 同样的命令换成 `-O2`，这两处 `call` 变成 **0** 处：`sum_down` 的整个函数体只剩 `addq %rcx, %rax` / `subq $1, %rcx` / `jne` 组成的一个循环，连栈帧都没有了。

同一份源码，机器层面已经不是递归了。

编译器视角 · 编译器视角（续）：`-O2` 没崩，是运气不是承诺

这件事有两条推论，一条好的、一条坏的：

- 好的一面：`n + f(n-1)` 这种「先递归、回来再算」的形状，优化器也可能认出来（本机 `-O2` 两种递归都变成了一个循环：尾递归那版 `count_down_tail(300000)` 在 `-O0` 爆栈（`3221225725`）、`-O2` 打印 `45000150000` 并退出 0）；
- 坏的一面：**它只是「可能」**。标准从头到尾没承诺过「优化器必须消掉递归」，改一行代码、换一个编译器、加一个 `volatile`，同一份程序就可能又爆。**别把「-O2 下没崩」当成「我的递归是安全的」**。

上面正文里那个「尾递归那版」的源码就是下面这六行；它的调用方是同目录的 `src/main_tail.c`（`DEPTH` 换成 300000，主函数只调 `count_down_tail(DEPTH, 0)`），想复现的话照着重跑一遍即可：

c examples/ch03/deep_recursion/src/deep_recursion.c 第 10–15 行

```c
long long count_down_tail(long long n, long long acc) {
    if (n == 0) {
        return acc;
    }
    return count_down_tail(n - 1, acc + n);   /* 尾调用：本层把结果直接交出去 */
}
```

14 尾调用：本层把结果直接交出去

实验记录 · 本机实测：这台机器的栈到底能装多少层

- 工具链：gcc 16.1.0（`-std=c17 -O0 -g`），目标三元组 x86_64-w64-mingw32；探针是同目录的 `src/depth_probe.c`（与 `sum_down` 同样的非尾递归，深度从命令行读入），命令 `gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g src/depth_probe.c -o dp.exe`，再逐个深度跑；
- 实测：**43000 层通过**（退出码 0）、**44000 层崩**（bash 报 127，Windows 退出码 3221225725）；
- 与算式对得上：`43000 × 48 B ≈ 2.06 MB`、`44000 × 48 B ≈ 2.11 MB`——**本机栈的限额就在 2 MB 附近**；每层 48 字节这个数是从汇编里数出来的（`pushq %rbp` + `subq $32, %rsp` + 一条返回地址）；
- 这个数字**不是标准的一部分**：换一台机器、换一个链接器参数，阈值就变。要判断「我这台机器能撑多少层」，只能像这样自己量。

> ⚠ **陷阱** · **「递归写错了不会报错，只会崩」**
>
> 你会看到什么：程序打印了一两行，然后**一声不响地没了**——没有异常、没有堆栈、没有 `Segmentation fault` 的字样（那不是 Windows 的口吻）。
>
> 你会怎么误判：先怀疑 `printf`、再怀疑终端，最后怀疑「是不是内存不够」，于是去加内存——加多少都没用，因为**卡住的不是内存总量，是栈的限额。**
>
> 真相是：一是**基准情形写错或写不到**（`n == 0` 写成 `n = 0`、递归时忘了减一），二是**深度本来就太大**（数据规模 × 每层帧大小 > 栈的限额）。两者的现象一样，区分办法是同一个数字：把 `DEPTH` 改成 `1000` 再跑一次。跑得通说明是深度问题，跑不通说明基准情形的问题。**栈溢出时连一个好看的报错都不会给你**——这是它比段错误更难查的地方。

7.9 `static` 局部变量：作用域与生命周期是两件事

先看两个几乎一模一样的计数器。它们的差别只有一个词：

c examples/ch03/static_local/src/static_local.c 第 5–15 行

```c
int next_ticket(void) {
    static int issued = 0;          /* 静态存储期：初始化只在程序开始时做一次 */ static 局部变量：初始化只做一次
    issued = issued + 1;
    return issued;
}

int auto_ticket(void) {
    int issued = 0;                 /* 自动存储期：每次进入都是一块全新的内存 */ 自动局部变量：每次进入都重新变成 0
    issued = issued + 1;
    return issued;
}
```

6 static 局部变量：初始化只做一次
12 自动局部变量：每次进入都重新变成 0

c examples/ch03/static_local/src/main.c 第 5–20 行

```c
int main(void) {
    printf("next_ticket 连着三次：");  连叫三次 next_ticket
    for (int i = 0; i < 3; i++) {
        printf(" %d", next_ticket());
    }
    printf("\n");

    printf("auto_ticket 连着三次：");  连叫三次 auto_ticket
    for (int i = 0; i < 3; i++) {
        printf(" %d", auto_ticket());
    }
    printf("\n");

    probe_identity();
    return 0;
}
```

6 连叫三次 next_ticket
12 连叫三次 auto_ticket

终端输出 终端输出 ch03-static-local-out

```
next_ticket 连着三次：  1 2 3
auto_ticket 连着三次：  1 1 1
第 1 次进入：static 局部 issued = 1，自动局部 fresh = 0
  记下这两个变量的地址，等下一层进来再比
第 2 次进入：static 局部 issued = 2，自动局部 fresh = 1
  两次的 static 局部是同一个对象吗：是
  两次的自动局部是同一个对象吗：不是
```

第 1、2 行是这件事最干净的样子：同一个函数被调三次，`next_ticket` 给出 **1 2 3**（它**记住了**上一次），`auto_ticket` 给出 **1 1 1**（每次重新开始）。第 3 到 7 行是地址层面的证据：`probe_identity` 让 `probe_step` **递归一层**（这样两次进入的两个栈帧同时活着，比较地址不会踩到悬垂指针），于是可以合法地问一句「还是同一个对象吗」——`static` 那个 **是**，自动的那个 **不是。**

再用 gdb 把「它们住在哪」钉死：

终端会话 gdb：同一个函数里，两个局部变量的地址隔着老远 ch03-gdb-static

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/static_local.c -o static_local.exe
$ gdb --batch -ex "break probe_step" -ex run -ex "p &issued" -ex "p &fresh" -ex continue -ex "p &issued" -ex
"p &fresh" ./static_local.exe
Breakpoint 1 at 0x1400015a2: file src/static_local.c, line 22.
[New Thread 84228.0x18dbc]
Thread 1 hit Breakpoint 1, probe_step (round=0) at src/static_local.c:22
22          int fresh = round;
$1 = (int *) 0x7ff774697044 <issued>
$2 = (int *) 0x5ffe3c
Thread 1 hit Breakpoint 1, probe_step (round=1) at src/static_local.c:22
22          int fresh = round;
$3 = (int *) 0x7ff774697044 <issued>
$4 = (int *) 0x5ffdfc
```

读这四行就够：第一次进入时 `&issued = 0x7ff774697044`、`&fresh = 0x5ffe3c`；第二次进入时 `&issued` **一个字都没变**（还是 `0x7ff774697044`），而 `&fresh` 换成了 `0x5ffdfc`。**同一个函数里的两个局部变量，一个在 `0x7ff7…`（程序映像的数据区），一个在 `0x5ff…`（栈）**——它们根本不在一类内存里。

内存形态 **static 局部变量不住在栈上：对照一帧**

第 1 步 调用进行中，两个局部变量各住一处

- probe_step 的帧 ：栈上（帧基址这里不重要，看局部变量）
- fresh ：自动存储期，0x5ffe3c，跟着这一帧走
- issued ：静态存储区，0x7ff774697044
- main 的帧 ：栈上，在等 probe_step 返回

probe_step 的帧 → issued 同一次调用里的两个局部变量，家不在一起

第 2 步 递归进来的第二层，fresh 换了地方

- probe_step 的第二层 ：栈上，另一帧
- fresh ：0x5ffdfc，又是新的一块，值从 round 重新初始化
- issued ：还是 0x7ff774697044，值已经变成 2
- main 的帧 ：仍在等

第 3 步 函数返回之后，帧没了，issued 还在

- probe_step 的帧 ：已随返回消失
- fresh ：连同那一帧一起没了，下次进入按 round 重新初始化
- issued ：还在原地，值仍是 2
- main 的帧 ：还在跑（下一次进入时 issued 会先自增成 3）

第 1、2 帧是 gdb 那两行 `p` 的画法：`fresh` 每层一个地址，`issued` 从头到尾一个地址。第 3 帧才是 `static` 真正的含义：**函数返回了、那一帧没了，它还在。**这就是为什么同一个函数第二次进入时读到的是上一次留下的值——`probe_step` 的 `issued` 第二次进入是 2；而 `next_ticket` 连调三次给出 1、2、3，**第四次**调用会给出 4，不会回到 1。注意这是**两个不同的 `static` 变量**（各自在自己的函数里），只是碰巧同名。

标准把这件事叫**存储期**（storage duration）。

标准原文 · **标准原文：`static` 对象的寿命与初始化**

C 语言标准 6.2.4 节「Storage durations of objects」第 2 段：

> An object whose identifier is declared without the storage-class specifier _Thread_local, and either with external or internal linkage or with the storage-class specifier static, has static storage duration. Its lifetime is the entire execution of the program and its stored value is initialized only once, prior to program startup.

大意：**带 `static` 声明的对象有静态存储期——它的寿命是整个程序的运行期间，而且它存的值只在程序开始之前初始化一次。**「初始化只做一次」这句话常被读成「第一次调用时初始化」——**不是**：标准写的是 `prior to program startup`（程序开始之前）。所以 `static int issued = 0;` 里那个 `= 0` 并不是一条「每次进入都执行」的语句，它在 `main` 跑起来之前就由加载器安排好了。

出处：ISO/IEC 9899:2018（C17）§6.2.4 第 2 段。

大白话 · **用大白话说：作用域管「谁能叫它」，生命周期管「它活多久」**

这是两个**互不相干**的问题，新手总把它们揉成一个：

- **作用域（scope）**：名字在**代码的哪一段**能被写出来。`issued` 只在 `next_ticket` 里有效——出了这个函数，你写 `issued` 编译器会说不认识它；
- **生命周期（lifetime）**：这块内存**从什么时候存在到什么时候。** `static int issued` 从程序开始活到程序结束，就算没人叫得动它的名字，它也一直活着，值也一直在。

所以别问「这个变量在不在作用域里」和「它还活着吗」当成一个问题——它们是两个问题。

大白话 · **（续）三方对照：三把尺子量三种变量**

把两个问题分开问，三种变量的差别立刻清楚了：

变量	名字在哪能写	内存活多久	初始化
自动局部变量 `int x;`	函数内	那次调用期间	每次进入重新初始化
`static` 局部变量 `static int x;`	函数内	整个程序	只做一次
全局变量 `int x;`（文件外）	整个文件（加 `extern` 全程序）	整个程序	只做一次

最反直觉的一格是第二行：**名字没变广，寿命变长了。**这不是矛盾，是两个问题用了两把不同的尺子。

工程惯例 · **工程惯例：`static` 局部变量很顺手，但请谨慎**

这是**工程惯例**，标准对「该不该用」没有任何规定。它是「函数记住上一件事」的最省事写法（调用计数、缓存、惰性初始化都靠它）。代价有三条，都是真代价：

- **不好测**：它带着上一次调用的状态。第一个测试把计数器加到 3，第二个测试一开始读到的就是 4——除非你专门写一个「重置」函数（那又要把这个变量挪到文件作用域去）；
- **不是线程安全的**：两个线程同时进这个函数，`issued = issued + 1` 这种读-改-写会互相踩（本书不写多线程教程；数据竞争只是未定义行为的一种，第 8 章会把它和 UB 放在一起提一句）；
- **藏起来的耦合**：调用者看不见它、参数里也没有它，但它的值会影响返回值。读代码的人得钻进函数体才发现「这个函数有记性」。

判断标准：状态只属于「这个函数自己的历史」时用它（比如一个只打印一次的日志开关）；状态一旦需要被外部读取、重置或检查，就应该把它挪到文件作用域，用一组函数（`get` / `reset`）明确暴露出来。

还有一件小事值得知道：`static` 加在**函数**前面（7.6 的 `stack_frames.c` 里，`inner` 与 `middle` 就是这么写的）意思是完全不同的另一件事——**这个名字只有本文件看得见**：

c examples/ch03/stack_frames/src/stack_frames.c 第 5–6 行

```c
/* static 函数：这个名字只在 stack_frames.c 里可见，别的 .c 里写 inner 也指不到它 */ static 函数：只在 stack_frames.c 里可见
```

```
6  static int inner(int depth) {
```

5 static 函数：只在 stack_frames.c 里可见

上面块里那两行（第 5 行的注释与第 6 行开头的 `static`）就是全部写法。定义体本身没什么特别，特别的是**别的源文件即使写出 `inner` 这个名字也指不到它**——这属于「链接性」（linkage）的话题：不加 `static` 的函数是**外部链接**（全程序共用这个名字），加了就是**内部链接**（只在本编译单元里）。注意这里的 `static` 与 7.9 上半节那个「活得久」**一点关系都没有**：同一个关键字，加在**局部变量**上是改存储期，加在**函数**或文件作用域的变量上是改链接性。第 7 章（`chapters/ch07/`）会把链接性讲透，现在记住两个方向就够：

- 只在本文件用的小工具函数（`inner`、`middle`、`print_at`、`rec_quiet` 都属于这类）加 `static`，**顺便避免和别人重名**；
- 对外提供的函数不加 `static`，声明写进头文件——它就是模块的门面。

命名上再补一句（这是惯例，不是标准）：**内部函数不必加前缀**，它们不可能被外部撞见；**要装进库、给别人链接的对外函数才加模块前缀。**本书的例子为了读起来短，对外函数都没加（`sum_to`、`outer`、`factorial_rec`、`checked_isqrt` 都是裸名）——它们各自活在自己的可执行文件里，撞不到谁；你以后写一个要被别人链接的模块，再按 `fb_sum_to` 这种形式加前缀。

7.10 前置条件与断言：`assert` 该守在哪里

7.2 那些函数都默认**调用方会按约定使用**：`sum_to(n)` 假定 `n` 不是天文数字，`celsius_to_fahrenheit(c)` 假定 `c` 是个正常温度。这种「调用方必须保证的事」叫**前置条件（precondition）**。本节用 `assert_guard` 的例子把它摆到台面上：`checked_isqrt(n)` 要求 `n >= 0`。写清楚前置条件、并且让它**在出错时当场停下**，是把你从「三天后一个荒唐的结果」里救出来的办法：

c examples/ch03/assert_guard/src/assert_guard.c 第 5–20 行

```
5  int checked_isqrt(int n) {
6      assert(n >= 0);              /* 前置条件：n 必须非负 */   前置条件写成一行断言
7      if (n < 0) {
8          return -1;               /* 兜底分支：断言被 -DNDEBUG 去掉后，靠它收场 */   兜底分支：断言被关掉时靠它收场
9      }
10     int root = 0;
11     while ((root + 1) * (root + 1) <= n) {
12         root = root + 1;
13     }
14     return root;
15 }
16
17 int checked_div(int a, int b) {
18     assert(b != 0);              /* 前置条件：除数不能是 0 */   第二个前置条件：除数不能是 0
19     return a / b;
20 }
```

6 前置条件写成一行断言
8 兜底分支：断言被关掉时靠它收场
18 第二个前置条件：除数不能是 0

这个文件里有两个函数、两条前置条件，写法一样：`checked_isqrt` 要求 `n >= 0`，`checked_div` 要求 `b != 0`（`main` 打印的那行 `checked_div(7, 2) = 3` 就是后者）。区别在兜底：`checked_isqrt` 有兜底（返回 `-1`），`checked_div` 没有——除以 0 在 C 里没有「合理的返回值」可言（它是未定义行为），**前置条件一旦被违反，调用方拿到的任何值都是垃圾**，所以这里宁可让断言把它当场拦下，发布版里则要靠调用方自己守住 `b != 0`。

c examples/ch03/assert_guard/src/main.c 第 5–17 行

```
5  int main(void) {
6  #ifdef NDEBUG   #ifdef NDEBUG 在编译期选一句话
7      printf("断言状态：NDEBUG 已定义，assert 已经被预处理掉\n");   NDEBUG 版打印这一行
8  #else
9      printf("断言状态：开（NDEBUG 未定义）\n");
10 #endif
```

```
    printf("checked_isqrt(144) = %d\n", checked_isqrt(144));
    printf("checked_div(7, 2)  = %d\n", checked_div(7, 2));
    printf("下面故意违反前置条件：checked_isqrt(-4)\n");   先声明「下面要故意违反」
    fflush(stdout);                          /* 断言失败会 abort，先把已打印的内容刷出去 */   先把已打印的内容刷出去
    printf("checked_isqrt(-4) = %d   (断言被关掉时才会走到这一行) \n", checked_isqrt(-4));   故意违反前置条件
    return 0;
}
```

```
 6 #ifdef NDEBUG 在编译期选一句话
 7 NDEBUG 版打印这一行
13 先声明「下面要故意违反」
14 先把已打印的内容刷出去
15 故意违反前置条件
```

同一份源码，两次编译的差别只在 `-DNDEBUG` 这个参数上。先看不加参数的那次——**它是真的当场死了**：

终端输出 终端输出 ch03-assert-guard-out

```
断言状态：开（NDEBUG 未定义）
checked_isqrt(144) = 12
checked_div(7, 2)  = 3
下面故意违反前置条件：checked_isqrt(-4)
```

清单 `assert_guard.json` 写的是 `"mode": "runtime_error"`、`"expect": { "exit": 3221226505 }`——这个模式的意思是「**这个程序本来就该死，而且必须死在这个退出码上**」。上面四行能留下来，靠的是第 14 行的 `fflush(stdout)`。完整的现场（含 stderr 与两次运行）：

终端会话 断言打开：程序当场停下 ch03-assert-fail

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/assert_guard.c -o assert_guard.exe
$ ./assert_guard.exe
断言状态：开（NDEBUG 未定义）
checked_isqrt(144) = 12
checked_div(7, 2)  = 3
下面故意违反前置条件：checked_isqrt(-4)
Assertion failed: n >= 0, file src/assert_guard.c, line 6
$ echo $?
127
```

终端会话 （续）加 -DNDEBUG：断言消失，靠兜底分支收场 ch03-assert-ndebug

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -DNDEBUG -Iinclude src/main.c src/assert_guard.c -o
assert_guard_ndebug.exe
$ ./assert_guard_ndebug.exe
断言状态：NDEBUG 已定义，assert 已经被预处理掉
checked_isqrt(144) = 12
checked_div(7, 2)  = 3
下面故意违反前置条件：checked_isqrt(-4)
checked_isqrt(-4) = -1   (断言被关掉时才会走到这一行)
$ echo $?
0
```

三件事对起来了：

- `Assertion failed: n >= 0, file src/assert_guard.c, line 6` —— 断言失败时程序打印的是**条件原文、哪个文件、哪一行**。这行字是 assert 全部的价值：它把「你不该这么调」变成一条带坐标的现场记录；
- **退出码**：bash 里 `echo $?` 报 **127**（MSYS bash 把这一类异常状态码都报成 127，7.8 末尾解释过它和 139 的区别）；进程真实的 Windows 退出码是 **3221226505**（`0xC0000409`），清单写的是后者，流水线对账的也是后者；
- **`-DNDEBUG` 之后程序完好地跑完并退出 0**，打印了 `checked_isqrt(-4) = -1`——注意这个 `-1` 不是 assert 给的，而是**兜底分支**给的（`assert_guard.c` 第 8 行）。这句 `if (n < 0) return -1;` 有时会被当成「和 assert 重复」，它其实在守另一件事：**断言会被关掉，兜底不会。**

终端输出 终端输出 ch03-assert-guard-ndebug-out

```
断言状态：NDEBUG 已定义，assert 已经被预处理掉
checked_isqrt(144) = 12
checked_div(7, 2)  = 3
下面故意违反前置条件：checked_isqrt(-4)
checked_isqrt(-4) = -1   （断言被关掉时才会走到这一行）
```

标准原文 · **标准原文：`NDEBUG` 一定义，`assert` 就消失**

> 标准对这件事的原话很短，7.2.1 节「Program diagnostics」：
>
> > If NDEBUG is defined as a macro name at the point in <assert.h> where it is included, then assert is defined simply as #define assert(ignore) ((void)0)
>
> 大意：**只要在包含 `<assert.h>` 的那一刻 `NDEBUG` 已经被定义，`assert` 就被定义成 `#define assert(ignore) ((void)0)` ——一个什么都不做的空语句。**所以 `-DNDEBUG` 不是「关掉某个开关」，而是**让那句 `assert(n >= 0);` 从预处理结果里彻底消失**：它一次都不会被执行，条件也不会被求值。
>
> 出处：ISO/IEC 9899:2018（C17）§7.2.1。

陷阱 · **`-DNDEBUG` 的代价：断言里的副作用一起消失，边界也没了**

你会看到什么：开发时一切正常，打包发出去的程序（用 `-DNDEBUG` 编译的）行为变了——不是「断言不再报错」这么温和，而是**某条路上的检查整个不见了。**

你会怎么误判：把 `assert` 当成「运行期的参数校验」——「用户传个负数进来，assert 会拦住它」。可发布版里它压根不存在。

真相有两条，第二条更贵：

- `assert` **是给程序员抓 bug 的，不是给用户兜错的。**给用户的非法输入要用真正的判断分支加错误码（`checked_isqrt` 里的 `if (n < 0) return -1;` 才是那一层）；
- **永远不要在 `assert(...)` 里写有副作用的东西**：`assert(p = malloc(n));` 或者 `assert(++count > 0);` —— 在 `-DNDEBUG` 下这两行会**整句消失**，`malloc` 不调用、`count` 不自增。这类 bug 的可怕之处在于：**打开断言的调试版一切正常，关掉断言的发布版行为完全不同。**

7.11 这一节新增了什么，以及「一个函数一件事」

回到 7.1 那棵树：现在你能对着**第 6 帧**逐个点名了——`include/func_basics.h`（只有声明与注释，本目录五个源文件都包含它）、`src/main.c`（只负责调用）、`src/func_basics.c`（定义在这里）、`src/decl_mismatch.c`（定义写错的那份）、`src/arg_order.c` 与 `src/main_arg_order.c`（求值顺序那一对），加上三个各管一种对账的清单。八个例子目录各盯一件事，第 7 帧那五段会话分别是：`session-gdb-frames.txt`（`bt` / `info frame` / `finish`）、`session-gdb-static.txt`（static 与自动局部的地址）、`session-warnings.txt`（漏写 `return` 与 `()` 两条诊断）、`session-deep.txt`（爆栈退出码与 `-O2` 对照）、`session-assert.txt`（断言失败现场）。

最后两件事，都属于「怎么写一个函数」的工程判断。

第一件：多返回值的两条路。 C 的 `return` 一次只能交回一个值，所以「算出一个位置 + 一个状态」这种需求要自己打包。工程上常走的路有两条，第 4 章与第 5 章各会展开一条：

- **打包成结构体交回**（第 5 章 `chapters/ch05/01-structs-and-alignment.md` 讲）：`struct result { long long value; int ok; };` 然后 `return r;` —— 调用方拿到的是一整块，读起来清爽；代价是**结构体按值返回会有一次拷贝**（结构体大了不便宜）；
- **用输出参数**（第 4 章 `chapters/ch04/03-pointer-idioms.md` 讲，7.3 那个 `swap_ok(int *a, int *b)` 已经是它的雏形）：`int parse(const char *s, long long *out)` —— 函数返回 `int` 当状态码，真正的结果写进 `*out`。代价是**调用处必须传对指针**，而且 `if (parse(s, &v) == 0)` 这种「返回值既当结果又当状态」的写法容易读错。

本书的惯例：**状态用返回值、结果用输出参数**时，返回值统一约定 `0` 表示成功（本书的流水线里那一堆 `exit` 码就是这个约定），并且**在函数名或注释里写清楚这个约定**——因为编译器不认识它。

第二件：一个函数一件事。 判断标准不是行数，是这两个信号：

- **参数越来越多**（超过四五个，尤其是好几个 `bool` 开关）：多半是它同时干了好几件事，或者把「配置」和「数据」混在了一起。开关型参数尤其危险：`f(data, true)` 在调用处读起来根本不知道那个 `true` 是什么。拆成两个函数，或者把开关收进一个结构体；
- **分支越来越多**（一层层 `if` 里套 `switch`）：多半是它要处理好几种「情形」。把每种情形拆成独立的函数，主函数退化成一句 `switch` ——**读代码的人只要看那一句就知道一共有几种情形。**

这条纪律在第 1 章的目录树里就埋下了：为什么从第二个项目起不许单文件工程？因为**文件是拆函数之后，代码放下的下一个层级**——函数管「一件事」，文件管「一组相关的事」。

下一单元从这里接着走：`chapters/ch04/01-arrays-and-strings.md`（数组与字符串）。7.3 那个「把地址递进去」的动作，到了数组那里会变成日常——数组传进函数时会退化成指向首元素的指针，`sizeof` 在函数里突然变小；那正是本节的 `&` 与 `*` 的下一站，也是 `f(int a[10])` 与 `f(int *a)` 为什么是同一个声明的原因。

小结 · 这一节的要点

- **函数有两半**：声明（原型）进头文件、定义进 `.c`；定义文件也包含自己的头文件，编译器才会当场核对两半一致（不一致就是 `conflicting types`）。
- **C 只有值传递**：形参拿到的永远是拷贝，改形参改不到调用者，要改就传地址；`f(void)` 写明「没有参数」、`f()` 只是「没说」，多传一个实参一个当场报错、一个静默通过；实参求值顺序是「未指定」，别依赖。
- **一次调用 = 一层栈帧**（参数、局部变量、返回地址，本机每帧 `0x40`），gdb 的 `bt` / `info frame` / `finish` 能把它们逐个指认出来；递归 = 基准情形 + 递归情形，代价是栈（本机约 2 MB、`sum_down` 每层 48 字节），二十万层的结局是退出码 **3221225725**—— `-O2` 可能把它改成循环，但那是运气不是承诺。
- **作用域与生命周期是两件事**：`static` 局部变量名字仍只在函数内，寿命却是整个程序、只初始化一次、住在数据区；`assert` 是给程序员的前置条件，`-DNDEBUG` 会让它整个消失——所以兜底分支与「断言里不放副作用」都不能省。

自测题 · 自测题（先自己答，再对答案）

1. `func_basics.h` 里只有声明，没有一行计算。那 `src/func_basics.c` 为什么还要 `#include "func_basics.h"`？如果不包含，会发生什么？
2. `swap_bad(x, y)` 与 `swap_ok(&x, &y)` 都能编译、都能跑完，为什么只有后者换成功了？请用「值传递」这个词解释一次。
3. `int legacy();` 与 `int modern(void);` 的区别是什么？本机实测多传一个实参时，两个函数各发生了什么？
4. 本机二十万层递归的退出码是多少？为什么同一个程序加 `-O2` 就不崩了？——「`-O2` 不崩」能不能当成「这个递归安全」的证据？
5. `next_ticket()` 连调三次给出 1、2、3，`auto_ticket()` 给出 1、1、1。请分别说出这两个变量**作用域**与**生命周期**，并解释为什么 `-DNDEBUG` 会让 `assert(++count > 0);` 里的 `count` 根本不自增。

注 · 自测题答案

1. 为了让**声明与定义出现在同一个编译单元里**，编译器会逐项核对两半是否一致（参数类型、返回类型）。不包含的话就没有可比的对象：定义照着错的方式写它自己也不知道，错误往往推迟到调用方那里（甚至推迟到运行期给你一个荒唐的结果）。证据：把定义写成 `double celsius_to_fahrenheit(float c)`，编译期报 `error: conflicting types …`，并用 `note:` 指回 `include/func_basics.h:9`。
2. 因为**C 只有值传递**：`swap_bad(x, y)` 把 `1` 和 `2` 这两份**值**拷进形参，函数换的是两份副本，函数返回后副本随栈帧消失，`x`、`y` 从头到尾没动；`swap_ok(&x, &y)` 拷进去的虽然是拷贝，但**拷贝上写的是地址**，顺着地址写就能改到调用者那块内存。所以打印出来是「1/2 不变」和「2/1 变了」。
3. `(void)` 写明「这个函数没有参数」，`()` 只说「参数情况没说」（标准 6.7.6.3 节把两者区别对待）。本机实测：`modern(1)` 编译期报 `error: too many arguments to function 'modern'; expected 0, have 1`；`legacy(21, 999)` 编译通过、运行时打印 `42`（多出来的 `999` 被无声吃掉）——后者是未定义行为，本机这次的选择是「忽略多余实参」。另外 `-Wall -Wextra` 不报 `()`，要 `-Wstrict-prototypes` 才报。
4. 退出码 **3221225725**（`0xC00000FD`，栈溢出；MSYS bash 里 `echo $?` 显示 127，PowerShell 读 Start-Process 的 ExitCode 显示 -1073741571，三种写法是同一个数）。加 `-O2` 不崩是因为优化器把这种递归**改写成了循环**，栈上不再

> 一层层堆帧——它只是「可能」这么优化，标准没有任何承诺，改一行代码、换一个编译器就可能变回去。所以不能当安全证据；要安全就写成迭代，或者用 `DEPTH` 这类参数把深度控制在栈限额之内。
>
> 5. `next_ticket` 里的 `issued`：**作用域**是函数内（外面写不出这个名字），**生命周期**是整个程序（初始化只做一次，函数返回后它还在数据区）；`auto_ticket` 里的 `issued` 两者都限于那次调用（每次进入都是一块新内存）。地址层面的证据来自同一节的 `probe_step`——那是**另一对同名变量**（在另一个函数里），正好拿来对照：它的 `static issued` 两次进入都是 `0x7ff774697044`，而自动的 `fresh` 两次分别是 `0x5ffe3c` 与 `0x5ffdfc`。`-DNDEBUG` 让 `assert(...)` 被替换成 `((void)0)`（标准 §7.2.1 的原话），整句宏体消失，于是括号里的 `++count` 一次都不执行——**断言不能被用来做任何有副作用的事。**

第 4 章

数组、字符串与指针

本章 3 个单元 · 页眉从这里之后的页面开始出现

1. 数组与字符串：连续的一块内存 零基础 · 第 8 单元 · 建议 90 分钟
2. 指针与内存：地址、类型与生命周期 零基础 · 第 9 单元（试点） · 建议 90 分钟
3. 指针的用法与约定：接口、多级指针与函数指针 零基础 · 第 10 单元 · 建议 90 分钟

数组与字符串：连续的一块内存

◎ 学习目标 · 看完这一单元，你应该能

- 说出 `int a[5]` 这行声明在内存里要来几个盒子、每个盒子几个字节，并用 `sizeof a / sizeof a[0]` 数出元素个数；
- 解释「数组交给函数之后为什么突然变小了」（把 `sizeof` 打在参数上，看到它是 8 而不是 20），并说出 `a[i]` 与 `*(a + i)` 为什么是同一件事；
- 分清 `char s[] = "hi"` 与 `char *p = "hi"` 是**两块不同的内存**，并指出哪一块能写、哪一块写了会崩；
- 用 `fgets(line, sizeof line, stdin)` 读一行，并说清它比 `scanf("%s")` 多管了哪一件事；
- 遇到一个越界程序，说出它会落到「改到邻居 / 崩溃 / stack smashing」三种后果里的哪一种，并各拿到一份真跑出来的证据。

4.1 先看工程长什么样

学数组最容易走偏的方式，是把它当成一条语法背下来（「下标写在方括号里」）。这一节照老规矩：先看**文件**。数组的每一个坑，最后都落在同一件事上——**一排盒子挨着放，而唯一知道这排有几个盒子的人，是声明它的那一行。**这句话你能在文件与命令里看见。

这一节要给第 4 章建出第四个例子目录 `examples/ch04/`。按大纲，`examples/` 下面的目录是**一章一个项目**：第 1 章的项目是 `examples/ch01/`，这一章的项目就是 `examples/ch04/`。第 1 单元里那棵单文件草稿，到这里已经长成了十二个最小工程。

目录树 **第 4 章的例子工程：这一节给它建出第四个目录**

第 1 步 这一节开始时，examples/ 下是前面三章的项目

```
examples/
├── ch01/ : 第 1 章的项目（本节不动它）
├── ch02/ : 第 2 章的项目
└── ch03/ : 第 3 章的项目
```

第 2 步 先建 ch04 与第一个例子，看清「声明与定义分家」

```
examples/
├── ch01/
├── ch02/
├── ch03/
└── ch04/ ← 本课新增
    └── array_basics/ ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.c : 组装与打印 ← 本课新增
        │   └── array_util.c : 定义：函数体 ← 本课新增
        ├── include/ ← 本课新增
        │   └── array_util.h : 声明：签名 ← 本课新增
        └── array_basics.json : 清单 ← 本课新增
```

第 3 步 再摆上其余五个同类例子

```
examples/
└── ch04/
    ├── array_basics/ : 同上 (src/ + include/ + 清单)
    ├── array_decay/ : 函数里的 sizeof 为什么是 8 ← 本课新增
    ├── array_addr/ : &a[i] 与 a[i] == *(a + i) ← 本课新增
    ├── array_assign/ : 数组不能整体赋值 ← 本课新增
    ├── matrix_rowmajor/ : 二维数组按行排 ← 本课新增
    └── strlen_vs_sizeof/ : 两个函数数的是不同的东西 ← 本课新增
```

第 4 步 再加上另外六个例子目录，十二个到齐

```
examples/
└── ch04/
    ├── 前六个例子目录 : 同上
    ├── char_array_vs_ptr/ : 可写与不可写两块内存 ← 本课新增
    ├── string_literal_ro/ : 写它就是崩 ← 本课新增
    ├── oob_neighbor/ : 越界打到邻居 ← 本课新增
    ├── oob_fanalyzer/ : 分析器看得见 ← 本课新增
    ├── strcpy_overflow/ : 栈保护抓得住 ← 本课新增
    └── fgets_bound/ : 读一行输入 ← 本课新增
```

第 5 步 本节另外留下的探针与会话（不进清单，理由见 4.9）

```
examples/
└── ch04/
    ├── 十二个例子目录 : 各带清单 + src/ + include/
    ├── warn_probe.c : 采编译器警告的探针 ← 本课新增
    ├── const_probe.c : 二维数组 const 的最小复现 ← 本课新增
    └── session-*.txt : 五份抓下来的诊断原文 ← 本课新增
```

第 1 帧是这一节故事的起点：`examples/` 下躺着前面三章的项目。第 2 帧先把**一个例子拆开看**：`array_basics/` 里是 `src/` 与 `include/` 两个目录——**声明（函数签名）放 `include/array_util.h`，定义（函数体）放 `src/array_util.c`**，`main.c` 只负责组装与打印，编译参数写在清单里。第 3、4 帧是其余十一个例子，分两帧列完；第 5 帧是本节另外留下的东西：两个不进清单的**探针**（`warn_probe.c`、`const_probe.c`）和五份抓下来的**会话** `session-*.txt`（4.4、4.6、4.7、4.9 节要拿它们读编译器给的诊断原文）。

为什么值得单独画一帧？因为「数组怎么交给函数」正是这一单元最容易卡住的地方，而头文件里那两行**签名**把这件事实话实说了：`print_array(const char *label, const int *a, size_t n)` 一眼就是「一个只读的数组 + 一个长度」。从第二个项目起本书**不允许**单文件工程，这一节多出来的正是这条规矩在数组上的样子。

工程惯例 · **工程惯例：数组参数为什么必须带长度**

标准只规定了一件事：函数的形参不能真的是数组（6.7.6.3 节把 `int a[]` 与 `int *a` 规定成同一个东西）。**「数组参数必须再传一个长度」是工程惯例，不是标准要求**——语言没有任何机制替你记住「这排有几个」。

理由一句话：函数里 `sizeof a` 算的是指针大小（4.4 节有真跑的数字），长度只有调用者知道，所以让调用者说；长度和指针绑在一起递过去，接口才自解释。代价是每次调用多写一个参数，而且**写错了没人替你发现**：`print_array(a, 4)` 把 5 个元素说成 4 个，只会少打印一个；说成 6 个就踩到越界（4.3 节）。

4.2 数组是一排同类型的盒子

先说定义：`int a[5];` 向系统要**五个 `int` 大小的盒子，而且必须挨着放**——这就是数组与指针最本质的差别。下面这段程序把四种初始化写法摆在一起：

c examples/ch04/array_basics/src/main.c 第 5–20 行

```c
int main(void) {
    int a[5] = {1, 2, 3};              /* 只写三个，后面两个自动补 0 */ 只写三个，后面两个自动补 0
    int b[5] = {0};                    /* 整体清零的省事写法 */ 整体清零的省事写法
    int c[5] = {[2] = 7, [4] = 9};     /* 指定初始化：只点名第 2、第 4 个 */ 指定初始化：只点名第 2、第 4 个
    int d[] = {4, 5, 6};               /* 不写长度，编译器替你数 */ 不写长度，编译器替你数

    print_array("a = {1, 2, 3}      ", a, sizeof a / sizeof a[0]);
    print_array("b = {0}            ", b, sizeof b / sizeof b[0]);
    print_array("c = {[2]=7, [4]=9} ", c, sizeof c / sizeof c[0]);
    print_array("d[] = {4, 5, 6}    ", d, sizeof d / sizeof d[0]);

    printf("sizeof a = %zu 字节, sizeof a[0] = %zu 字节, 元素个数 = %zu\n", sizeof a 一个数组 = 整块多少字节
           sizeof a, sizeof a[0], sizeof a / sizeof a[0]);
    printf("a 的和 = %d\n", sum_array(a, sizeof a / sizeof a[0]));
    return 0;
```

```
20  }
```

6 只写三个，后面两个自动补 0
7 整体清零的省事写法
8 指定初始化：只点名第 2、第 4 个
9 不写长度，编译器替你数
16 sizeof 一个数组 = 整块多少字节

那四个 `print_array` 与最后的 `sum_array` 不在这个文件里——它们是模块 `array_util` 的两个函数，声明在头文件、定义在源文件。先看头文件，注意里面**只有名字、参数、返回值，没有函数体**：

c **声明：放在 include/ 下的接口** examples/ch04/array_basics/include/array_util.h 第 1–12 行

```c
#ifndef ARRAY_UTIL_H
#define ARRAY_UTIL_H

#include <stddef.h>

/* 打印一排 int：a 是「第一个在哪」，n 是「这排有几个」——两个都要给 */
void print_array(const char *label, const int *a, size_t n);

/* 求和：只读这一排，不改它 */
int sum_array(const int *a, size_t n);

#endif
```

c examples/ch04/array_basics/src/array_util.c 第 5–19 行

```c
void print_array(const char *label, const int *a, size_t n) {  形参是「const int *a」加长度
    printf("%s (%zu 个) :", label, n);
    for (size_t i = 0; i < n; i++) {
        printf(" %d", a[i]);
    }
    printf("\n");
}

int sum_array(const int *a, size_t n) {  求和也照规矩来
    int total = 0;
    for (size_t i = 0; i < n; i++) {
        total += a[i];
    }
    return total;
}
```

5 形参是「const int *a」加长度
13 求和也照规矩来

留意两个形参的写法：`const int *a` 里的 `const` 是「我不会改你给我的那些盒子」，`size_t n` 是「这排有几个」。这两个东西缺一不可，理由 4.4 节讲。跑出来的结果：

终端输出 终端输出 array-basics-out

```
a = {1, 2, 3}       (5 个) : 1 2 3 0 0
b = {0}             (5 个) : 0 0 0 0 0
c = {[2]=7, [4]=9}  (5 个) : 0 0 7 0 9
d[] = {4, 5, 6}     (3 个) : 4 5 6
sizeof a = 20 字节, sizeof a[0] = 4 字节, 元素个数 = 5
a 的和 = 6
```

六行输出要连着读：

- `a = {1, 2, 3}` 打出来是 `1 2 3 0 0`——**没写的那两个被自动补成了 0**，不是垃圾值；
- `b = {0}` 打出来全是 0——第一个元素是 0，其余照补 0 的规矩来；
- `c = {[2]=7, [4]=9}` 打出来 `0 0 7 0 9`——只点名了两个位置，其余照旧补 0；
- `d[] = {4, 5, 6}` 打出来是三个——`[]` 里不写数字时，长度由右边花括号里的个数决定；
- 最后两行是本节的算术：`sizeof a` 是 **20** 字节、`sizeof a[0]` 是 **4** 字节，相除得到 **5** 个元素。

第 1 帧是「一排盒子」这个说法本身：五个盒子、每个 4 字节、编号从 **0** 到 **4**。第 2 帧是本单元最重要的一个算式——`sizeof a` 量的是**整块**，`sizeof a[0]` 量的是**一个格子**，两者相除就是元素个数。第 3 帧提前把字符串摆出来：`char w[3] = "hi"` 也是三个盒子（`w` 是本节另起的一个名字，别跟上面的 `b` 混了），但多出来的那个装的是 0（写出来是 `'\0'`），它的唯一职责是告诉函数「字符串到这里结束」。4.7 节整节都在讲这一帧。

{} 语法规则 · 数组的声明与初始化

两种声明：`int a[5];`（要五个 `int` 的盒子，**不初始化**）与 `int a[] = {1, 2, 3};`（长度由初始化列表数出来，只能用在有初始化的时候）。注意 `int a[5];` 这五个盒子里是**不确定的值**，不是 0——要清零必须写 `= {0}`；「声明时不初始化」与「列表里没写的元素补 0」是两件事，别混。

初始化列表的三种写法：`{1, 2, 3}` 后面自动补 0；`{0}` 是把整排清零的惯用写法；`{[2] = 7}` 是**指定初始化**（下标写在方括号里），只想给某几个位置赋值时用它。

用 `sizeof a / sizeof a[0]` 数元素个数——这个算式只在**数组还在你手上**的时候有效（4.4 节讲它什么时候失效）。本书的规矩是：三种写法你都要能读、能写，但**日常只推荐两种**——要么明确写出全部初值，要么 `{0}` 清零；`[2]=7` 读起来省事，代价是别人要数下标才知道你改了哪几个位置。

标准原文 · 标准原文：没写的那些元素变成了什么

C 语言标准（ISO/IEC 9899:2018，也就是 C17）6.7.9 节「Initialization」第 21 段：

> If there are fewer initializers in a brace-enclosed list than there are elements or members of an aggregate, or fewer characters in a string literal used to initialize an array of known size than there are elements in the array, the remainder of the aggregate shall be initialized implicitly the same as objects that have static storage duration.

大意：**花括号里的初值比元素少时，剩下的部分按「静态存储期对象」的规矩来初始化**——对整数来说就是 0。第 19 段还规定了另一件事：初始化按**列表顺序**发生，后面写的初值会覆盖前面为同一个位置写的初值；没有显式初始化的部分，同样按静态对象处理。这两段合起来解释了 `{1, 2, 3}` 后面那两个 0、`{0}` 的整排零、以及 `{[2]=7, [4]=9}` 只在两个位置上有值。

注意标准说的是「一样」而不是「等于零」：静态对象的规矩里指针是空指针、浮点是 0.0。所以**初值缺失时不是垃圾值。**

出处：ISO/IEC 9899:2018（C17）§6.7.9 第 19、21 段。

4.3 下标从 0 起，而且没有人替你检查

一排盒子挨着放，怎么找到第 n 个？靠「第一个的地址 + n × 每个的宽度」。所以第一个盒子的编号必须是 0——它不是「第 1 个」的序号，它是**离起点几个格子的偏移量**。下面这份程序把偏移量直接打出来：

c examples/ch04/array_addr/src/main.c 第 5–19 行

```c
int main(void) {
```

```
    int a[4] = {10, 20, 30, 40};

    printf("sizeof a     = %zu (整块数组：4 个 int) \n", sizeof a); 整块 16 字节
    printf("sizeof *&a   = %zu (&a 的类型是「数组的指针」，解引用回来还是数组) \n", sizeof *&a); &a 是「数组的指针」
    printf("sizeof &a[0] = %zu (一个指针) \n", sizeof &a[0]); &a[0] 只是一个指针
    printf("&a = %p, &a[0] = %p：同一个地址，两种类型\n", (void *)&a, (void *)&a[0]); 同一个地址，两种类型

    printf("每个元素的地址：\n");
    show_addresses(a, sizeof a / sizeof a[0]);

    printf("三种写法取同一个元素：\n");
    show_aliases(a, sizeof a / sizeof a[0]);
    return 0;
}
```

8 整块 16 字节
9 &a 是「数组的指针」
10 &a[0] 只是一个指针
11 同一个地址，两种类型

其中两个函数（住在 `array_addr.c` 里）一个量地址、一个换写法取同一个元素：

c examples/ch04/array_addr/src/array_addr.c 第 5–18 行

```
void show_addresses(const int *a, size_t n) {
    for (size_t i = 0; i + 1 < n; i++) {
        printf("  &a[%zu] = %p   &a[%zu] = %p   相差 %td 字节\n", 相邻两个元素的地址
               i, (void *)&a[i], i + 1, (void *)&a[i + 1], 相减就知道隔了多远
               (char *)&a[i + 1] - (char *)&a[i]); 按字节算，所以差 4
    }
}

void show_aliases(const int *a, size_t n) {
    for (size_t i = 0; i < n; i++) {
        printf("  a[%zu] = %2d    *(a + %zu) = %2d    %zu[a] = %2d\n", 三种写法：a[i]、*(a+i)、i[a]
               i, a[i], i, *(a + i), i, i[a]);
    }
}
```

7 相邻两个元素的地址
8 相减就知道隔了多远
9 按字节算，所以差 4
15 三种写法：a[i]、*(a+i)、i[a]

终端输出 终端输出 array-addr-out

```
sizeof a     = 16 (整块数组：4 个 int)
sizeof *&a   = 16 (&a 的类型是「数组的指针」，解引用回来还是数组)
sizeof &a[0] = 8 (一个指针)
&a = 00000000005FFEA0, &a[0] = 00000000005FFEA0：同一个地址，两种类型
每个元素的地址：
  &a[0] = 00000000005FFEA0   &a[1] = 00000000005FFEA4   相差 4 字节
  &a[1] = 00000000005FFEA4   &a[2] = 00000000005FFEA8   相差 4 字节
  &a[2] = 00000000005FFEA8   &a[3] = 00000000005FFEAC   相差 4 字节
三种写法取同一个元素：
  a[0] = 10    *(a + 0) = 10    0[a] = 10
  a[1] = 20    *(a + 1) = 20    1[a] = 20
  a[2] = 30    *(a + 2) = 30    2[a] = 30
  a[3] = 40    *(a + 3) = 40    3[a] = 40
```

输出分三段看。

第一段（前四行）是三个 `sizeof` 的对照。 `sizeof a` = 16（4 个 `int`），`sizeof *&a` 也是 16，而 `sizeof &a[0]` 是 **8**。第三个就是「指针多大」；第二个说明 `&a` 的类型是「**指向整块数组的指针**」而不是「指向第一个 `int` 的指针」——对它解引用（`*&a`）拿回来的是整块数组，量出来还是 16。同一台机器上 `sizeof(int)` 是 4、指针是 8，这

两个数**不是定律**：换个平台完全可能变。所以**能用 `sizeof a / sizeof a[0]` 的地方就不要手写元素个数**，这是本书的硬纪律。

第二段是连续性的直接证据。 `&a[0]`、`&a[1]`、`&a[2]`、`&a[3]` 四个地址，每个比前一个大 **4** 字节，正好是一个 `int` 的宽度。数组之所以能被这样遍历，就是因为「下一个盒子的地址 = 这个盒子的地址 + 元素宽度」这条铁律。

第三段是三种写法等价。 `a[i]`、`*(a + i)`、`i[a]` 打出同一个数。最后那个写法不是笔误：标准把下标运算**定义**成「先加再解引用」（下一块有原文），加法可交换。

标准原文 · 标准原文：为什么 a[i] 等于 *(a + i)

标准没有单独定义「数组下标」这个语法，它做的是**替换**。6.5.2.1 节「Array subscripting」第 2 段：

> The definition of the subscript operator `[]` is that `E1[E2]` is identical to `(*((E1)+(E2)))`. … E1[E2] designates the E2-th element of E1 (counting from zero).

大意：`E1[E2]` **就是** `*((E1)+(E2))` **的另一种写法**，而且标准特意补了一句「从 0 开始数」。这句话一次解释了三件事：为什么 `a[2]` 与 `*(a + 2)` 同值；为什么 `2[a]` 合法；为什么 `a[i]` 里的 `i` 可以是负的（语法上它就是一个整数，编译器不会拦——**但走出去就是 UB**，4.3 节下面就有 `a[-1]` 的真实现象）。

出处：ISO/IEC 9899:2018（C17）§6.5.2.1 第 2 段。

⚠ 陷阱 · 越界：三种后果，都不报错

你会看到什么：程序照样编译、照样运行。编译器**不会因为「下标越界」这四个字报错**——它顶多在「长度能在编译期算出来」的写法上给一条警告（例如定长 `strcpy`，4.9 节有原文），而那种警告也拦不住程序跑起来。你会怎么误判：以为「没报错就是没错」，甚至以为「越界读到的那个值就是我要的」。真相是：**下标越界在 C 里是未定义行为（UB）——语言不管，编译器只在少数形状上提醒，运行时也不一定有反应。**它会落到三档后果里的一档，取决于越界多远、是读还是写：

1. **改到邻居**：只超出一个变量的地盘，悄悄改掉旁边变量的值，程序继续跑，结果莫名其妙——本节 `oob_neighbor` 有真证据，4.9 节把它画成图；
2. **当场崩溃**：越界跑到进程没有权限的内存，退出码 3221225477（0xC0000005，访问冲突）——往字符串字面量里写（4.8 节）就是这一档；
3. **stack smashing**：编译器在栈上放了「校验值」，函数返回时发现它被踩坏了，于是把进程杀掉并打印 `*** stack smashing detected ***`——这一档要**主动开保护**（`-fstack-protector-all`）才看得见，4.9 节有清单。

三档里最危险的是**第一档**：它什么都不说。

下面这份程序就让第一档当场现形——它先越界读，再越界写：

c examples/ch04/oob_neighbor/src/oob_neighbor.c 第 6–25 行

```c
void read_past_end(void) {
    int before = 0x11223344; 邻居甲，值好认
    int a[4] = {1, 2, 3, 4}; 正经的四个盒子
    int after = 0x55667788; 邻居乙，值也好认

    printf("  三个局部变量的位置：before=%p  a=%p  after=%p\n",
           (void *)&before, (void *)a, (void *)&after);
    for (int i = -2; i <= 6; i++) { 下标从 -2 读到 6
        printf("  a[%2d] = 0x%08X\n", i, a[i]);   /* 下标不在 0…3 里 */ 数组外的那几个盒子
    }
}

void write_past_end(void) {
    int guard = 12345; 写在数组前面的邻居
    char buf[8] = "abc"; 八字节的盒子

    printf("  写之前：buf=\"%s\"  guard=%d\n", buf, guard);
    strcpy(buf, "0123456789AB");       /* 13 个字节进了 8 个字节的盒子 */ 13 个字节硬塞进去
    printf("  写之后：buf=\"%s\"  guard=%d\n", buf, guard);
```

```
25  }
```

```
 7 邻居甲，值好认
 8 正经的四个盒子
 9 邻居乙，值也好认
13 下标从 -2 读到 6
14 数组外的那几个盒子
19 写在数组前面的邻居
20 八字节的盒子
23 13 个字节硬塞进去
```

终端输出 终端输出 oob-neighbor-out

```
越界读：
  三个局部变量的位置：before=00000000005FFE78  a=00000000005FFE60  after=00000000005FFE5C
  a[-2] = 0x005FFE01
  a[-1] = 0x55667788
  a[ 0] = 0x00000001
  a[ 1] = 0x00000002
  a[ 2] = 0x00000003
  a[ 3] = 0x00000004
  a[ 4] = 0x00000063
  a[ 5] = 0x00000000
  a[ 6] = 0x11223344
越界写：
  写之前：buf="abc"  guard=12345
  写之后：buf="0123456789AB"  guard=1111570744
越界追加：
  写之前：buf="abc"  guard=12345
  写之后：buf="abc0123456789"  guard=943142453
三件坏事都做完了，程序还活着 — 这才是最难查的那种错
```

这份输出值得一行一行对（第三段「越界追加」是 `strcat`，4.9 节单独讲）：

- `before`、`a`、`after` 三个局部变量的地址是**降序**的：`after` 在最低处、`before` 在最高处——这台机器上（gcc 16.1.0、`-O0`、目标三元组 x86_64-w64-mingw32）局部变量按声明顺序**从高地址往低地址**排，所以 `a` 的下面（低地址）是 `after`，上面（高地址）是 `before`；
- `a[-1]` 打出 `0x55667788` ——**那正是 `after` 的值。** `a[-1]` 这个盒子不存在，它算出来的地址刚好落在 `after` 里；
- `a[6]` 打出 `0x11223344` ——那是 `before` 的值；
- `a[4]`、`a[5]` 打出 `0x00000063`、`0x00000000` ——这两个盒子里是**栈上没清干净的旧东西**，和你的数组毫无关系。它们也是整份输出里**最不稳**的两个数：同一份源码换个编译输出路径再跑，`a[4]` 就从 `0x63` 变成过 `0x2D` 与 `0x34`（本机实测）——所以连这一行都不要当成规律；
- 越界写那一段：`buf` 只有 8 个字节，`"0123456789AB"` 是 12 个字符加一个结束标记，共 13 个字节。程序没有崩，`buf` 打出来完整，而邻居 `guard` 从 **12345** 变成了 **1111570744**。把这个数换成十六进制是 `0x42413938`，四个字节正好是 `'8'`、`'9'`、`'A'`、`'B'` —— `buf` 装不下的那四个字符，被写进了 `guard` 的地盘。

实验记录 · 本机实测：这一节的数字都是跑出来的

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32，系统 Windows 10 x64 (MSYS2 UCRT64)；编译参数固定为 `-Wall -Wextra -Wpedantic -O0 -g`（**没有优化**，`-O0` 是本节能看见「邻居」的前提）；
- 两个命令，工作目录分别是例子自己的目录：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/array_addr.c -o array_addr.exe`（`sizeof` 与地址递增来自这一份）；`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/oob_neighbor.c -o oob_neighbor.exe`（「打到邻居」的三个现象来自这一份）；
- `sizeof a` = **16**、`sizeof &a[0]` = **8**、相邻两个元素相隔 **4** 字节（`-O0` 实测，见上面输出第二段）；
- 越界读到的 `a[-1]`、`a[6]` 与越界写之后的 `guard`：具体数值见输出块，它们全是**这台机器、这份可执行文件、`-O0` 下的现象，不是语言保证的规律**；
- 同一份源码换一个编译器、换优化档（`-O2`）、换一个编译输出路径，邻居的排布与「读到什么」就可能整个换掉，也可能直接崩（4.3 与 4.9 的输出里，同一个 `a[4]` 在两次不同构建中就打出了不同的值）。**所以绝对不要依赖越界读到的值**：它连「会不会读到邻居」都不保证。

标准原文 · 标准原文：越界这件事，标准只说了四个字

标准里没有「下标越界」这一节——6.5.2.1 节把 `a[i]` 定义成 `*(a + i)` 之后，越界就变成了**指针算术越界**，归 6.5.6 节「Additive operators」第 8 段管：

> If both the pointer operand and the result point to elements of the same array object, or one past the last element of the array object, the evaluation shall not produce an overflow; otherwise, the behavior is undefined.

大意：指针加上一个整数之后，**结果必须还落在这个数组对象里（或者正好落在最后一个元素的后面一格）**，否则行为未定义。注意标准留的那一点余地：「最后一个元素的**后面一格**」允许被**算出来**（这样你才能写 `for (p = a; p != a + n; p++)`），但那一格**不许解引用**——同段最后一句就是这么写的。

所以「越界是 UB」不是吓人的话，而是标准里一条明文规定：编译器有权假设你不会越界，并据此优化代码——`-O2` 下越界程序行为突变，就是这个假设在起作用。

出处：ISO/IEC 9899:2018（C17）§6.5.6 第 8 段（另见 §6.5.2.1 第 2 段对下标的定义）。

4.4 数组名会退化成一个指针

上一节里 `print_array(a, …)` 把 `a` 交给了一个形参为 `const int *a` 的函数，两边为什么能对上？因为**数组名在大多数场合会自动变成「指向第一个元素的指针」**，这件事在 C 里叫**退化**。退化不是「数组变成了指针」——那排盒子还在原地，变的只是**表达式 `a` 的值**：它从「整块数组」变成了「第一个盒子的地址」。下面这份程序把退化的后果摆出来：

c examples/ch04/array_decay/src/decay.c 第 5–12 行

```c
void show_sizes(const int *a) {
    printf("  形参写成 const int *a : sizeof a = %zu  ← 指针的大小\n", sizeof a);
    printf("  sizeof *a = %zu (一个 int) \n", sizeof *a);
}

void show_sizes_paren(int a[5]) {
    printf("  形参写成 int a[5]    : sizeof a = %zu  ← 还是指针\n", sizeof a);
}
```

5 形参是指针
10 形参写成数组，其实也是指针

c examples/ch04/array_decay/src/main.c 第 5–20 行

```c
int main(void) {
    int a[5] = {1, 2, 3, 4, 5};
```

```
7
8       printf("在调用者这边：\n");
9       printf("  sizeof a      = %zu （整块数组）\n", sizeof a);  调用者这边量的是整块
10      printf("  sizeof &a[0] = %zu （一个指针）\n", sizeof &a[0]);  一个指针
11      printf("  元素个数 = sizeof a / sizeof a[0] = %zu\n", sizeof a / sizeof a[0]);  元素个数只能在调用者这
        边算
12
13      printf("把 a 交给函数之后：\n");
14      show_sizes(a);  交给收指针的函数
15      show_sizes_paren(a);  写成数组的形参也一样
16
17      printf("函数想自己数元素个数是做不到的，长度必须一起传：sum = %d\n",
18             sum_with_len(a, sizeof a / sizeof a[0]));
19      return 0;
20  }
```

9 调用者这边量的是整块
10 一个指针
11 元素个数只能在调用者这边算
14 交给收指针的函数
15 写成数组的形参也一样

终端输出 终端输出 array-decay-out

```
在调用者这边：
  sizeof a      = 20 （整块数组）
  sizeof &a[0] = 8 （一个指针）
  元素个数 = sizeof a / sizeof a[0] = 5
把 a 交给函数之后：
  形参写成 const int *a ：sizeof a = 8  ← 指针的大小
  sizeof *a = 4 （一个 int）
  形参写成 int a[5]     ：sizeof a = 8  ← 还是指针
函数想自己数元素个数是做不到的，长度必须一起传：sum = 15
```

九行输出，把「同样的写法、两个不同的数」摆在一起：

- **在调用者这边**：`sizeof a` = **20**（五个盒子都在），`sizeof &a[0]` = **8**（一个指针），相除得到 5 个元素；
- **在函数里面**：`sizeof a` = **8**。同一个名字、同一个 `sizeof` 写法，值从 20 变成 8——因为函数收到的**不是那排盒子**，而是一个装着首地址的**形参**（一个 8 字节的指针变量）；
- `sizeof *a` = **4**（一个 `int`）——这个没变，它说的是「指针指向的东西有多大」，跟数组长度无关；
- 第二行「形参写成 `int a[5]`」也是 **8**。这一条最容易被误读：`int a[5]` 里的那个 `5` **不代表**函数收到了 5 个元素，它对 `sizeof` 一点用没有。

编译器对这件事还有一条专门的警告。把 `decay.c` 单独过一遍语法与类型：

终端会话 -Wsizeof-array-argument：编译器一眼看穿「参数不是数组」 ch04-decay-warn

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -fsyntax-only src/decay.c
src/decay.c: In function 'show_sizes_paren':
src/decay.c:11:76: warning: 'sizeof' on array function parameter 'a' will return size of 'int *' [-Wsizeof-
array-argument]
   11 |     printf("  形参写成 int a[5]    : sizeof a = %zu  ← 还是指针\n", sizeof a);
      |                                                                            ^
src/decay.c:10:27: note: declared here
   10 | void show_sizes_paren(int a[5]) {
      |                       ~~~~^~~
```

`'sizeof' on array function parameter 'a' will return size of 'int *'` ——**「对数组形参 `a` 取 `sizeof`，得到的是 `int *` 的大小」**。这条警告叫 `-Wsizeof-array-argument`，只在「形参写成数组」时触发；`show_sizes(const int *a)` 那一份反而没有警告，因为它从头到尾没冒充数组。所以：**形参写成 `int a[5]` 不但救不回长度，还会让编译器专门提醒你一次。**这也正是 4.1 节那条惯例（长度必须一起传）最硬的证据。

标准原文 · **标准原文：数组名在什么时候不变**

6.3.2.1 节「Lvalues, arrays, and function designators」第 3 段：

> Except when it is the operand of the `sizeof` operator, the `_Alignof` operator, or the unary `&` operator, or is a string literal used to initialize an array, an expression that has type ''array of type'' is converted to an expression with type ''pointer to type'' that points to the initial element of the array object and is not an lvalue.

大意：**除了一组列出来的场合，数组类型的表达式都会自动转成「指向首元素的指针」**。原文的 except 子句一共列了四项，前三项是运算符：

- `sizeof` **的操作数**——所以 `sizeof a` 在调用者那边量到的是整块 20 字节（这就是「例外」的实际价值）；
- `_Alignof` **的操作数**——按标准它只接类型名（`_Alignof(int[4])` 是 **4**，本机实测），所以日常代码里几乎碰不到这一项；GCC 额外允许 `_Alignof(表达式)`，但 `-Wpedantic` 会提示这不合标准；
- **一元** `&` **的操作数**——所以 `&a` 的类型是「指向整块数组的指针」，而不是「指向首元素的指针」（4.3 节那个 `sizeof *&a` = 16 就是它）。

第四项是「用来初始化数组的字符串字面量」：它说的是 `char s[] = "hi";` 里的字面量**不**先转成指针（否则数组没法用字面量初始化），跟前三项不是一回事。本书说「三处例外」时，指的是**前三个运算符**——那才是你写代码时会遇到的。

标准还补了一句：转成指针之后的那个表达式**不是一个左值**——它不是那排盒子本身，只是「首地址」这个值。

出处：ISO/IEC 9899:2018（C17）§6.3.2.1 第 3 段。

标准原文 · **标准原文：为什么 int a[5] 与 int *a 是同一个东西**

同一件事在**参数**上有更直接的规定，6.7.6.3 节「Function declarators (including prototypes)」第 7 段：

> A declaration of a parameter as ''array of type'' shall be adjusted to ''qualified pointer to type'', where the type qualifiers (if any) are those specified within the `[` and `]` of the array type derivation.

大意：**参数里写「数组类型」，一律被改写为「（带限定符的）指针类型」**。所以 `void f(int a[5])` 与 `void f(int *a)` 是同一个函数签名；参数里的方括号和数字只是一种**写法**，不携带长度信息。

把这两段连起来读就得到一条工程结论：**函数不可能通过形参知道数组有多长。**这是语言层面的规定，不是实现的选择。

出处：ISO/IEC 9899:2018（C17）§6.7.6.3 第 7 段。

工程惯例 · **工程惯例：只读的数组参数写 const T ***

标准里 `const` 是**类型限定符**（6.7.3 节），规定「不许通过这个表达式去修改那个对象」。**把 `const int *` 写成数组参数的默认形状，是工程惯例**——语言不会强迫你写。

理由三条：① **意图写在签名上**：`const int *a` 一眼是「只读输入」，`int *a` 是「我要改你给我的东西」；② **调用者能传的东西变多**：只读接口能收 `const` 数组与字符串字面量，写接口收不了；③ **编译器替你守约**：实现里手滑写 `a[0] = 1`，编译立刻报错。代价是**挡住了「本来想改」的接口**：真要改（比如 `copy_array`），就得老实写 `int *dst`——那也正好是它该有的样子。所以 `const` 不是「更安全所以都加上」，它是**接口语义的一部分**：本书的写法是**只读 `const T *`、要改 `T *`，长度紧跟在数组参数后面。**

4.5 数组不能整体赋值，也不能按值传参

既然数组就是「一排盒子」，能不能整排搬过去？比如这样：

c examples/ch04/array_assign/src/main.c 第 5–13 行

```c
int main(void) {
    int a[3] = {1, 2, 3};
```

```c
    int b[3];

    b = a;                                  /* 这一行编译不过 */ 这一行编译不过
    copy_array(b, a, sizeof a / sizeof a[0]); /* 这一行才是正解 */ 这一行才是正解
    printf("%d %d %d\n", b[0], b[1], b[2]);
    return 0;
}
```

9 这一行编译不过
10 这一行才是正解

编译它。编译器报了两条错误：第一条在第 9 行，第二条针对文件末尾那个「想返回数组」的函数——先看第一条，第二条下面单独说。

编译诊断 编译诊断 array-assign-diag

```
src/main.c: In function 'main':
src/main.c:9:7: error: assignment to expression with array type
    9 |     b = a;                                  /* 这一行编译不过 */
      |       ^
src/main.c: At top level:
src/main.c:15:4: error: expected identifier or '(' before '[' token
   15 | int[3] make(void) {                         /* 想返回数组：连声明都写不出来 */
      |    ^
```

`src/main.c:9:7: error: assignment to expression with array type` ——**「赋值的目标是数组类型」**。注意它是**语法级**错误：不是「越界」、不是「类型不匹配」，而是「这种写法根本不允许存在」。所以数组的这三条限制，一条都不是「小心一点」能绕过的：

- **不能整体赋值**：`b = a;` 直接编不过。原因看下面那块标准——赋值运算符要求左边是「可修改左值」，而**数组类型从来不是可修改左值**；
- **不能按值传参**：`void f(int a[3])` 看起来像「按值传」，实际上**形参已经被改写成了指针**（4.4 节的标准原文），函数里改 `a[0]` 改到的就是调用者那排盒子。所以 C 里根本没有「按值传数组」这回事，只有「传首地址 + 传长度」；
- **不能返回数组**：`int[3] f(void)` 这种函数不存在——标准规定函数返回类型只能是 `void` 或**除数组类型以外**的完整对象类型。真写出来是什么样？就是文件末尾这四行：

c **（续）试一下「返回数组」：连声明都写不出来** examples/ch04/array_assign/src/main.c 第 15–18 行

```c
int[3] make(void) {                         /* 想返回数组：连声明都写不出来 */
    int a[3] = {1, 2, 3};
    return a;
}
```

对应的第二条错误是 `src/main.c:15:4: error: expected identifier or '(' before '[' token` ——**解析到 `int` 后面那个 `[` 就停了**。所以「不能返回数组」落到编译器嘴里是一句**语法错误**，比「类型不匹配」还早：它压根走不到 `return` 那一步。三条限制里，这一条最彻底：另外两条是「不让你这么写」，这一条是「连写法都不存在」。

那要「整排搬过去」怎么办？**赋值运算符不行，但按字节拷可以**：`memcpy(b, a, sizeof b)`，或者一个元素一个元素地拷。这正是 `copy_array` 干的事，它住在模块里，形参是 `int *dst` 与 `const int *src`（要改的写 `int *`、只读的写 `const int *`，跟上一条惯例对上）：

c examples/ch04/array_assign/src/array_assign.c 第 1–7 行

```c
#include "array_assign.h"

void copy_array(int *dst, const int *src, size_t n) {
    for (size_t i = 0; i < n; i++) {
        dst[i] = src[i]; 目标可写、源只读、长度由调用者给
    }
}
```

5 目标可写、源只读、长度由调用者给

标准原文 · 标准原文：为什么 b = a 是非法的

6.5.16 节「Assignment operators」第 2 段只有一句约束：

> An assignment operator shall have a modifiable lvalue as its left operand.

大意：**赋值号左边必须是「可修改左值」**。那什么不是可修改左值？6.3.2.1 节第 1 段的定义里点了第一个：

> A modifiable lvalue is an lvalue that does not have array type, does not have an incomplete type, does not have a `const`-qualified type, ...

大意：**「数组类型」从来不是可修改左值。**所以 `b = a;` 不是「运行时被拒绝」，而是**压根不是合法的 C 语句**——这就是它报 `error` 而不是警告的原因。

至于「不能返回数组」，出处是 6.9.1 节「Function definitions」第 3 段：

> The return type of a function shall be `void` or a complete object type other than array type.

大意：**函数返回类型不许是数组类型。**

出处：ISO/IEC 9899:2018（C17）§6.5.16 第 2 段、§6.3.2.1 第 1 段、§6.9.1 第 3 段。

4.6 二维数组是按行排的

`int m[3][4]` 读作「三个元素的数组，每个元素是「4 个 `int` 的数组」」。它依然遵守上面全部规矩：**一整块连续内存**，只是被切成了「三个 16 字节的行」。下面这份程序把行的边界量出来：

c examples/ch04/matrix_rowmajor/src/main.c 第 5–21 行

```c
int main(void) {
    int m[ROWS][COLS] = {                                   // 三行四列
        {1,  2,  3,  4},
        {5,  6,  7,  8},
        {9, 10, 11, 12},
    };

    printf("sizeof m = %zu, sizeof m[0] = %zu, sizeof m[0][0] = %zu\n",   // 三个 sizeof 对照
           sizeof m, sizeof m[0], sizeof m[0][0]);
    printf("行数 = %zu, 列数 = %zu\n",                          // 行数、列数都能算出来
           sizeof m / sizeof m[0], sizeof m[0] / sizeof m[0][0]);

    printf("每一行从哪开始：\n");
    show_row_starts(m);

    printf("按内存顺序读出来：\n");
    print_flat(m);
```

6 三行四列
12 三个 sizeof 对照
14 行数、列数都能算出来

两个打印函数住在 `matrix.c` 里：

c examples/ch04/matrix_rowmajor/src/matrix.c 第 5–22 行

```c
void show_row_starts(int m[ROWS][COLS]) {
    for (size_t i = 0; i < ROWS; i++) {
        printf("  &m[%zu][0] = %p", i, (void *)&m[i][0]);   // 每行第一个元素的地址
        if (i > 0) {
            printf(" (比上一行高 %td 字节) ", (char *)&m[i][0] - (char *)&m[i - 1][0]);   // 相邻两行的距离
        }
        printf("\n");
    }
}

void print_flat(int m[ROWS][COLS]) {
```

```
    for (size_t i = 0; i < ROWS; i++) {
        for (size_t j = 0; j < COLS; j++) {
            printf(" %2d", m[i][j]);   双层循环按行输出
        }
    }
    printf("\n");
}
```

7 每行第一个元素的地址
9 相邻两行的距离
18 双层循环按行输出

注意这两个形参**没有** `const`——4.4 节刚说过「只读就写 `const T *`」，这里为什么例外？因为把 `int m[3][4]` 传给 `const int (*)[4]`，在本书采用的标准版本里**不是允许的转换**：GCC 当扩展接受，但 `-Wpedantic` 会明确反对（下面这个探针就是最小复现）：

终端会话 -Wpedantic：二维数组的只读参数在本书的标准版本里不合法 ch04-const2d

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -fsyntax-only const_probe.c
const_probe.c: In function 'main':
const_probe.c:9:10: warning: invalid use of pointers to arrays with different qualifiers in ISO C before C23
[-Wpedantic]
    9 |     show(m);
      |          ^
```

也就是说，「二维数组的只读参数」在本书的标准版本里没有干净写法：要么不加 `const`（本书的选择，头文件注释里写了理由），要么把二维数组摊成一维指针 + 行宽再传。更新的标准修掉了这条限制；本书按 C17 写，所以照旧。

终端输出 终端输出 matrix-out

```
sizeof m = 48, sizeof m[0] = 16, sizeof m[0][0] = 4
行数 = 3，列数 = 4
每一行从哪开始：
  &m[0][0] = 00000000005FFE80
  &m[1][0] = 00000000005FFE90 (比上一行高 16 字节)
  &m[2][0] = 00000000005FFEA0 (比上一行高 16 字节)
按内存顺序读出来：
  1  2  3  4  5  6  7  8  9 10 11 12
sizeof rows = 24 (3 个指针) ，sizeof m = 48 (12 个 int)
rows 里三行各在哪：00000000005FFE70 / 00000000005FFE60 / 00000000005FFE50
```

前六行就是「行主序」的全部内容：

- `sizeof m` = **48**（12 个 `int`），`sizeof m[0]` = **16**（一行 4 个 `int`），`sizeof m[0][0]` = 4；两个相除分别得到行数 3 与列数 4——**行数列数也是从 `sizeof` 算出来的**，不是手写的；
- `&m[0][0]`、`&m[1][0]`、`&m[2][0]` 相差 **16** 字节，正好是一行的宽度。也就是说 `m[1][0]` 紧跟在 `m[0][3]` 后面——**内存里没有「第二行」这种东西，只有一长排 12 个 `int`，行是人脑切出来的**；
- 「按内存顺序读出来」打的是 `1 2 3 … 12`：双层循环按 `i` 行 `j` 列走，正好就是内存顺序。

那「指针数组」呢？看这段（还在同一个 `main` 里）：

c examples/ch04/matrix_rowmajor/src/main.c 第 23–32 行

```
    int r0[COLS] = {1, 2, 3, 4};
    int r1[COLS] = {5, 6, 7, 8};
    int r2[COLS] = {9, 10, 11, 12};
    int *rows[ROWS] = {r0, r1, r2};   /* 这是「指针数组」：三个指针，不是三行数据 */   三个指针，不是三行数据
    printf("sizeof rows = %zu (3 个指针) , sizeof m = %zu (12 个 int) \n",
           sizeof rows, sizeof m);
    printf("rows 里三行各在哪：%p / %p / %p\n",   三个独立数组的地址
           (void *)rows[0], (void *)rows[1], (void *)rows[2]);
    return 0;
}
```

26 三个指针，不是三行数据
29 三个独立数组的地址

最后两行输出是 `sizeof rows = 24`（3 × 8 字节的指针，而 `m` 是 48 字节的数据）。三个地址这次**恰好也相差 16 字节**——别被它骗了：那只是因为三个局部数组恰好被排在了一起，**地址挨着证明不了「它们是同一个数组」**。真正说得清区别的是 `sizeof`：`rows` 是 24 字节（三个指针），`m` 是 48 字节（十二个 `int`）。所以两者长得像、读法也像（都能写 `x[i][j]`），但**能对 `m` 做的 `sizeof`、能对 `m` 做的整块传参，`rows` 都没有。**

> **标准原文 · 标准原文：数组必须是「连续分配」的**
>
> 上面「内存里只有一长排」这句话，标准里有一句直接的规定。6.2.5 节「Types」第 20 段：
>
> > An array type describes a contiguously allocated nonempty set of objects with a particular member object type, called the element type.
>
> 大意：**数组类型描述的是一组「连续分配」的对象**，每个对象的类型相同（叫元素类型）。「连续分配」这四个字就是下面全部结论的来源：`&a[i]` 逐个递增、`a[i]` 与 `*(a + i)` 等价、二维数组排成一行、`sizeof m` 等于「行数 × 行宽」。
>
> 标准里没有出现过「row-major（行主序）」这个词——**它是「连续分配」加「按行切分这种读法」推出来的结果，也是所有主流实现的做法**，不是一条单独的规定。
>
> 出处：ISO/IEC 9899:2018（C17）§6.2.5 第 20 段。

4.7 C 字符串 = 字符数组 + '\0'

字符串是这一章里最容易被「看起来一样」骗到的类型。先把定义钉住：

> **标准原文 · 标准原文：什么叫「字符串」**
>
> 7.1.1 节「Definitions of terms」第 1 段：
>
> > A string is a contiguous sequence of characters terminated by and including the first null character. ... The length of a string is the number of bytes preceding the null character.
>
> 大意：**字符串是一串连续的字符，以第一个「空字符」（值为 0，写出来是 `'\0'`）结束**；字符串的长度指**空字符之前**有多少字节（空字符自己不算）。
>
> 这两句话解释了本节的每一个现象：`char` 数组只有加上 `'\0'` 才叫字符串；`strlen` 数到 `'\0'` 为止；没有 `'\0'` 的字符数组，交给任何字符串函数都是越界。
>
> 出处：ISO/IEC 9899:2018（C17）§7.1.1 第 1 段。

下面这份程序把「数字节」与「数到 `'\0'`」两个数并排打出来，再故意用错一次：

c examples/ch04/strlen_vs_sizeof/src/main.c 第 6–23 行

```c
int main(void) {
    char msg[] = "hello, world";
    char *p = msg;

    printf("sizeof msg = %zu (数组：12 个字符 + 1 个 '\\0') \n", sizeof msg); 整块 13 字节
    printf("strlen(msg) = %zu (数到 '\\0' 为止有几个字符) \n", strlen(msg)); 数到结束标记为止
    printf("sizeof p = %zu (指针：8 个字节，和字符串多长没关系) \n", sizeof p); 指针永远是 8

    char dst[32];
    memset(dst, '.', sizeof dst);
    copy_bad(dst, msg); 用错：只拷 8 个字节
    printf("用错之后 dst 开头 20 个字节：%.20s\n", dst);

    memset(dst, '.', sizeof dst);
    copy_n(dst, msg, strlen(msg) + 1); 用对：长度加 1
    printf("用对之后 dst：%s\n", dst);
    return 0;
```

```
23  }
```

```
10 整块 13 字节
11 数到结束标记为止
12 指针永远是 8
16 用错：只拷 8 个字节
20 用对：长度加 1
```

用错的那个函数只有一行，住在 `string_size.c` 里：

c examples/ch04/strlen_vs_sizeof/src/string_size.c 第 9–11 行

```
9   void copy_bad(char *dst, const char *src) {
10      memcpy(dst, src, sizeof src);   /* 坑：sizeof src 是 8，不是字符串长度 */
11  }
```

sizeof 一个指针 = 8，不是字符串长度

```
10 sizeof 一个指针 = 8，不是字符串长度
```

`-Wall` 里有一条警告专门盯着这个写法（把 `string_size.c` 单独过一遍语法与类型就能看到）：

终端会话 -Wsizeof-pointer-memaccess：编译器连「你想干什么」都猜到了 ch04-sizeof-warn

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -fsyntax-only
src/string_size.c
src/string_size.c: In function 'copy_bad':
src/string_size.c:10:29: warning: argument to 'sizeof' in 'memcpy' call is the same expression as the source;
did you mean to provide an explicit length? [-Wsizeof-pointer-memaccess]
   10 |     memcpy(dst, src, sizeof src);   /* 坑：sizeof src 是 8，不是字符串长度 */
      |                             ^~~
```

`argument to 'sizeof' in 'memcpy' call is the same expression as the source; did you mean to provide an explicit length?` ——「`memcpy` 里的 `sizeof` 跟源是同一个表达式，你是不是想给一个明确的长度？」这条叫 `-Wsizeof-pointer-memaccess`，`-Wall` 就开着：**在指针上用 `sizeof` 求「有多长」，编译器知道你在瞎猜。**

终端输出 终端输出 strlen-vs-sizeof-out

```
sizeof msg = 13 (数组：12 个字符 + 1 个 '\0')
strlen(msg) = 12 (数到 '\0' 为止有几个字符)
sizeof p = 8 (指针：8 个字节，和字符串多长没关系)
用错之后 dst 开头 20 个字节：hello, w............
用对之后 dst：hello, world
```

五个数、两个用途：

- `sizeof msg` = **13**、`strlen(msg)` = **12**。差的那 1 就是 `'\0'` ——**要整块拷贝（连结束标记一起）用 `sizeof`；要问「有几个字符」用 `strlen`**；
- `sizeof p` = **8**。这里最容易出事：`p` 明明指着 `"hello, world"`，可 `sizeof p` 与字符串长度**毫无关系**。所以函数里的 `sizeof src` 永远不是「源有多长」，它只是「指针多大」；
- 后两行是用错的后果：`copy_bad` 只拷了 8 个字节，`dst` 里只有 `hello, w`，后面全是原来铺的 `.` ——**少了 4 个字符（`orld`），而且没有结束标记**（这里用 `%.20s` 限住了长度才没读出去；直接 `printf("%s", dst)` 会一路读到 32 字节之外找那个不存在的 `'\0'`）。

`strlen` 与 `sizeof` 的分工就是一句话：**`sizeof` 量「这块内存多大」，`strlen` 数「到 `'\0'` 之前有几个字节」。**两者只有一处重合：当一个字符数组恰好装着一个字符串、不多不少时，`sizeof` 比 `strlen` 大 1。**`strlen` 必须真的走到 `'\0'`**：没有结束标记的字符数组，它会一直往后数，数到某个恰好为 0 的字节为止（越界读）。

char s[] = "hi" 与 char *p = "hi"

这两行在所有书上长得几乎一样，但它们要来的内存完全不同：

c examples/ch04/char_array_vs_ptr/src/main.c 第 5–23 行

```c
int main(void) {
    char s[] = "hi";       /* 自己的三个字节，可写 */
    char *p = "hi";        /* 一个指针，指向别处那三个只读字节 */
    char *q = "hi";        /* 同样的字面量再写一遍 */

    printf("s 的地址        = %p（数组自己那块内存，在栈上）\n", (void *)s);
    printf("sizeof s = %zu（三个盒子都是你的），sizeof p = %zu（p 只是一个地址）\n",
           sizeof s, sizeof p);
    printf("p == s ?        = %s（两块完全不同的内存）\n", p == s ? "是" : "不是");
    printf("p == q ?        = %s\n", p == q ? "是（两份同样的字面量被合并了）" : "不是");
    printf("p 指着的内容    = %s（字母住在别处那块只读内存里）\n", p);

    s[0] = 'H';            /* 合法：这是你自己的数组 */
    printf("改了 s[0] 之后：s = %s，p 指着的东西还是 %s\n", s, p);

    upper_first(s);
    printf("交给函数改过之后：s = %s\n", s);
    return 0;
}
```

6 自己的三个字节，可写
7 一个指针，指向别处
8 同一份字面量再写一遍
10 数组自己的地址
13 两块内存不是一回事
14 被合并了
17 改的是自己的盒子

终端输出 终端输出 char-array-vs-ptr-out

```
s 的地址        = 00000000005FFE9D（数组自己那块内存，在栈上）
sizeof s = 3（三个盒子都是你的），sizeof p = 8（p 只是一个地址）
p == s ?        = 不是（两块完全不同的内存）
p == q ?        = 是（两份同样的字面量被合并了）
p 指着的内容    = hi（字母住在别处那块只读内存里）
改了 s[0] 之后：s = Hi，p 指着的东西还是 hi
交给函数改过之后：s = Hi
```

内存形态 两种 "hi"：一块是自己的，一块是别人的

第 1 步 char s[] = "hi" 自己复制三个盒子

s[0] = 'h' | s[1] = 'i' | s[2] = 0 ：结束标记 | sizeof s = 3 ：三个盒子都是你的 ← 本步新增

第 2 步 char *p = "hi" 只有一个地址

p ：一个 8 字节的变量，里头是地址

p → 字面量 "hi" 字母住在别处的只读内存里

第 3 步 两份同样的字面量可能只有一块

q ：另一个 8 字节的变量 | p == q 为真 ：合并了，但这不是标准保证的

q → 字面量 "hi" 和 p 指到了同一处

第 4 步 改谁不改谁

s[0] = 'H' ：可以，改的是你自己的盒子 ← 本步新增 | p[0] = 'H' ：不许，那块内存是只读的 ← 本步新增

第 1、2 帧就是输出前三行说的那件事：`s` 自己占了三个字节（`sizeof s` = 3，地址就在栈上），而 `p` 只是一个 8 字节的变量（`sizeof p` = 8）——`p == s` 打出**不是**，说明它俩根本不是同一块内存。第 3 帧是这份程序给出的一个彩蛋：「`p == q ?`」那行打的是**是（两份同样的字面量被合并了）**——本机把两处写着一模一样的字面量合并成了同一块内存，于是 `p == q` 为真。**合并是允许的，不是保证的**：标准对重复字面量怎么处理没有规定，换个编译器就可能给你两块。所以：**永远不要用 `==` 去比字符串的内容**（要比起码得用 `strcmp`，那是第 6 章「标准库工具箱」的事）。第

4 帧是 4.8 节的主角：`s[0] = 'H'` 合法（输出里 `s = Hi`，而 `p` 指着的那块纹丝不动——这直接证明了它们是两块内存），`p[0] = 'H'` 当场崩。

这份程序**故意不打印字面量的地址**，这不是偷懒：那个地址不保证稳定——我在两次流水线运行里就拿到了两个不同的值（`00007FF7F7524000` 与 `00007FF6A8724000`），换个构建、换台机器更没准。所以能稳定写进正文的证据是 `sizeof` 与 `p == s` 这类**关系**，而不是一个具体的绝对地址——你以后写测试、写日志时也要记住这一点：**地址值不是程序的行为。**

输出最后两行还说明了另一件事：`s` 交给 `upper_first(s)` 之后被改成了 `Hi`。因为形参是 `char *`（退化），函数改的就是**调用者那排盒子**——这正是 4.5 节说的「C 没有按值传数组」的正面用法：想改别人的数组，收一个指针就行。

4.8 字符串字面量不许写

上一节的第 4 帧留了一句话没说完：`p[0] = 'H'` 为什么不行？因为**字符串字面量住在只读内存里。**这件事编译期完全看不出来——代码照编、程序照跑，直到那一行执行。下面这份程序就写这么一行：

c examples/ch04/string_literal_ro/src/literal_ro.c 第 5–11 行

```c
void write_to_literal(void) {
    char *p = "hi";          /* p 指向一块只读的字符串字面量 */ 指针指向只读的字面量
    printf("p 指着 \"%s\", 准备写 p[0]…\n", p);
    fflush(stdout);          /* 崩之前先把这行刷出去，否则管道里看不到 */ 崩之前先刷出去
    p[0] = 'H';              /* 写它 = 未定义行为，本机在这里真的崩 */ 写它 = 未定义行为
    printf("居然没崩：%s\n", p);
}
```

6 指针指向只读的字面量
8 崩之前先刷出去
9 写它 = 未定义行为

它是按 `mode=runtime_error` 跑起来的——这个模式要求程序**真的死掉**，而且退出码要跟清单里写的一致：

终端输出 终端输出 literal-ro-out

```
p 指着 "hi", 准备写 p[0]…
```

实验记录 · 本机实测：写一个字面量真的会崩

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/literal_ro.c -o string_literal_ro.exe`，**编译退出码 0，一条警告都没有**；
- 运行结果：退出码 **3221225477**（0xC0000005，访问冲突），stderr 是**空的**——没有任何提示，进程直接没了；
- 上面那个输出块只有一行，而且是靠第 8 行的 `fflush(stdout)` 才留下的。**经管道抓输出时 stdout 是全缓冲的**：不刷，进程被终止时缓冲区里的东西来不及写出去，你会看到一个「什么都没打印就死了」的程序——这一点在第 1 单元讲 `printf` 时就埋过伏笔，这里是它第一次咬人。
- 同样的代码在**别的机器上未必崩**：字面量放在哪、那页内存是不是只读，标准没规定。所以「不崩」不代表写对了。

内存形态 **指针能指向的三块地方**

第 1 步 **指向字符串字面量**

p ：一个变量，装着地址 `p[0] = 'H' ：退出码 3221225477` ← 本步新增

p → 字面量 "hi" 只读，写它就崩

第 2 步 **指向一个数组**

s ：你自己那三个盒子 `p[0] = 'H' ：合法，s 变成 "Hi"` ← 本步新增

p → s 写 p[0] 改的是自己的盒子

第 3 步 指向堆上要来的内存

p[0] = 'H' ：合法，用完记得还 ← 本步新增

p → malloc 要来的内存 可读可写，归你管

三帧讲的是同一件事：**「能不能写」由指针指着的那块内存决定，不由指针的类型决定。**第 1 帧的 `p` 是 `char *`，第 2 帧的 `p` 也是 `char *`——类型一模一样，一个写了崩、一个写了没事。第 3 帧是后面第 5 章的主角（`malloc` 要来的内存可读可写，代价是你要自己负责还）。所以看到 `char *` 时，光看类型判断不了能不能写，得看**它指向谁**——这也是为什么本书要求「只读的接口写 `const char *`」：让签名替你回答这个问题。

⚠ 陷阱 · 这个坑怎么把你绊倒

你会看到什么：`char *p = "hi"; p[0] = 'H';` 编译**一句警告都没有**（本机实测，`-Wall -Wextra -Wpedantic` 全开也不报）。你会怎么误判：因为编译过了，你会以为「能编译就是能写」，于是把它当成普通数组用；等到某个函数里改一个「传进来的字符串」，程序在别人电脑上崩了，你还找不到是哪一行。真相在标准 6.4.5 节「String literals」第 7 段，只有两句话：

> It is unspecified whether these arrays are distinct provided their elements have the appropriate values. If the program attempts to modify such an array, the behavior is undefined.

大意：① **两处内容相同的字面量是不是同一块内存，标准没规定**（这就是上一节那个 `p == q` 为真的来历——本机合并了，但你不能指望）；② **程序一旦试图修改字面量那个数组，行为未定义。**所以崩溃不是「编译器有 bug」，而是你把 ② 撞上了。

修法只有一条：**想改，就先拷进自己的数组**——`char s[] = "hi";` 要来的三个字节是你自己的，随便写；`char *p = "hi";` 只是拿到了别人的地址。

顺带一个更隐蔽的版本：`char *p = "hi";` 在 C 里合法，在 C++ 里**直接编不过**——很多老代码这么写纯属历史遗留。本书的建议是**不打算改就写** `const char *p = "hi";`：一旦手滑去写它，编译器当场拦下。

4.9 strcpy 不看目标多大

越界的头号来源不是手写下标，而是**字符串函数。**原因很直白：`strcpy`、`strcat` 这类函数的原型里只有「源」和「目标」两个指针，**没有一个地方能写「目标有多大」**——`strcat` 更隐蔽：它还要先信任「目标里原来那个字符串是完整的」，两处都得对，才不出事。下面这份程序就是 `strcpy` 出事的现场：

c examples/ch04/strcpy_overflow/src/strcpy_overflow.c 第 6–10 行

```c
void copy_into_small(const char *src) {
    char buf[8];              /* 盒子里只装得下 7 个字符 + '\0' */ 装得下 7 个字符 + 结束标记
    strcpy(buf, src);         /* strcpy 不看目标多大：源多长就写多长 */ 源多长就写多长
    printf("buf = %s\n", buf);
}
```

7 装得下 7 个字符 + 结束标记
8 源多长就写多长

调用它的是一行传了 20 个字符的 `main`：

c examples/ch04/strcpy_overflow/src/main.c 第 1–6 行

```c
#include "strcpy_overflow.h"

int main(void) {
    copy_into_small("ABCDEFGHIJKLMNOPQRST");   /* 20 个字符 */
    return 0; 20 个字符进 8 个字节的盒子
}
```

5 20 个字符进 8 个字节的盒子

标准对 `strcpy` 只说了两句（7.24.2.3 节「The `strcpy` function」第 2 段）：**把 `s2` 指向的字符串（含结尾的空字符）拷到 `s1` 指向的数组里**；两块内存重叠时行为未定义。翻遍这一段，你找不到「目标装不下会怎样」——因为那属

于调用者的责任。`<string.h>` 的开头倒是有一句总纲（7.24.1 节第 1 段，原文是 `If an array is accessed beyond the end of an object, the behavior is undefined.`）——**数组一旦被访问到对象末尾之外，行为未定义。**

跑一下会发生什么？这份清单用的检测模式是 `stack-protector`：

> **实验记录 · 本机实测：用 -fstack-protector-all 抓住它**
>
> - 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
> - 检测模式：清单写 `"detect": "stack-protector"`，闸门按 `toolchain.json` 给它的参数 `-fstack-protector-all` 编译，然后运行；
> - 命令（取清单目录为工作目录）：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fstack-protector-all src/main.c src/strcpy_overflow.c -o strcpy_overflow.exe`；
> - 真实结果：退出码 **3221226505**（0xC0000409），stderr 上有一行 `*** stack smashing detected ***: terminated`；stdout 是空的（崩溃前 printf 的内容没来得及刷出去）；
> - **阴性对照**（同一份源码，去掉 `-fstack-protector-all` 再编再跑）：退出码变成 **3221225477**（0xC0000005，访问冲突），stderr **一个字都没有**；
> - 两个退出码不一样，说明那句 `*** stack smashing detected ***` 确实是这个开关抓出来的，不是程序自己崩出来的巧合——这条「归属」由闸门自动复算，伪造不了；
> - 本书的工具链**没有 ASan / UBSan**（`-fsanitize=address` 在本机链接失败），所以这一节的运行期证据只能是退出码与 stderr——这也解释了为什么「越界读到邻居」这种细粒度现象，只能靠 4.3 节那种打印地址与值的办法去看。

机制：`-fstack-protector-all` 让编译器在**每个函数的栈帧里**塞一个「校验值」（canary），函数返回前检查；被动了就打印那行 `*** stack smashing detected ***` 并终止进程。所以它只抓**写坏栈上数组、还碰了校验值**这一类；堆越界、只踩到邻居没碰校验值，都抓不到。

那份「只踩到邻居」的样子在 4.3 节已经看过了：`guard` 从 12345 变成 1111570744。用盒子画出来就是这样：

第 1 帧是写之前的样子：`buf` 八个盒子（`buf[4]` 到 `buf[7]` 是空的），邻居 `guard` 紧跟在第八个盒子后面、里头是 12345。第 2 帧是 `strcpy` 干的事——它从 `buf[0]` 开始一格一格写，写到第 13 个字节才停，中间**没有任何一步问过「这个盒子还是我的吗」**。第 3 帧是结论：邻居被改了，而**函数自己不知道**；只有开了栈保护、且碰巧踩到校验值，才会在返回时被抓住（那就是上面 3221226505 与 `stack smashing` 的来历）。

strcat：从现有的结尾接着写

`strcpy` 是从头写，`strcat` 是从**现有的结尾**接着写：它先找到目标里那个 `'\0'`，再从那里往后写。所以 `strcat` 要的空间是「原有长度 + 追加长度 + 1」——两个长度哪个估错都出事。同一个 `oob_neighbor` 里的第三个函数就是它：

c examples/ch04/oob_neighbor/src/oob_neighbor.c 第 27–34 行

```c
void append_past_end(void) {
    int guard = 12345;
    char buf[8] = "abc";              /* 里面已经有 3 个字符 */ 里面已经有 3 个字符

    printf("  写之前: buf=\"%s\"  guard=%d\n", buf, guard);
    strcat(buf, "0123456789");        /* 从现有的结尾接着写 10 个字符 */ strcat 从结尾接着写 10 个字符
    printf("  写之后: buf=\"%s\"  guard=%d\n", buf, guard);
}
```

29 里面已经有 3 个字符

32 strcat 从结尾接着写 10 个字符

它跑出来的「越界追加」那一段在同一个输出块里：`buf` 里本来有 `abc`（3 个字符），`strcat` 又写了 10 个字符加一个结束标记，一共 **14** 个字节进 8 个盒子；`guard` 从 12345 变成了 **943142453**（即 `0x38373635`，四个字节正好是 `'5'`、`'6'`、`'7'`、`'8'`）。**失败方式和 `strcpy` 一模一样**：不报错、不崩，邻居悄悄变了——所以「字符串函数是越界头号来源」这句话，两个函数一人一半。

编译器其实看得见一部分——但它给的只是警告

把这份程序的写法换成「常量源 + 定长目标」，`-Wall` 就会说话。本节开头那个探针 `warn_probe.c` 就是专门采这句话的：

终端会话 `-O0: -Wall 直接指出目标装不下` ch04-warn-o0

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -c warn_probe.c -o warn_probe_o0.o
warn_probe.c: In function 'main':
warn_probe.c:10:5: warning: '__builtin_memcpy' writing 13 bytes into a region of size 8 overflows the
destination [-Wstringop-overflow=]
   10 |     strcpy(buf, "0123456789AB");     /* 定长形状: 13 个字节进 8 个字节的盒子 */
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~
warn_probe.c:8:10: note: destination object 'buf' of size 8
    8 |     char buf[8];
      |          ^~~
```

同一个文件换 `-O2`，还能多看到一种诊断：

终端会话 `（续）-O2: 越界读也被 -Warray-bounds 抓出来` ch04-warn-o2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -fdiagnostics-color=never -c warn_probe.c -o warn_probe_o2.o
warn_probe.c: In function 'main':
warn_probe.c:13:5: warning: array subscript 7 is above array bounds of 'int[4]' [-Warray-bounds=]
   13 |     printf("a[7] = %d\n", a[7]);     /* 越界读: 常量下标 */
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~
warn_probe.c:7:9: note: while referencing 'a'
    7 |     int a[4] = {1, 2, 3, 4};
      |         ^
warn_probe.c:10:5: warning: '__builtin_memcpy' forming offset [8, 12] is out of the bounds [0, 8] of object
'buf' with type 'char[8]' [-Warray-bounds=]
   10 |     strcpy(buf, "0123456789AB");     /* 定长形状: 13 个字节进 8 个字节的盒子 */
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~
warn_probe.c:8:10: note: 'buf' declared here
    8 |     char buf[8];
      |          ^~~
```

这两块会话要读出三件事：

- `-Wstringop-overflow`（`-O0` 那份）**不需要你开任何开关**：本机实测，连一个 `-W` 参数都不写（`gcc -std=c17 -O0 -g -c warn_probe.c`）它也照样报；`gcc -Q --help=warnings` 显示它默认就在档位 2。所以它不是「`-Wall` 带来的」，而是 **GCC 默认就开着**的警告。它认的是「编译期就能算出长度的写」——`strcpy(buf, "0123456789AB")` 的源是常量、目标是定长数组，编译器一算就知道要写 13 个字节进 8 个字节的盒子；
- `-Warray-bounds`（`-O2` 那份）**需要开优化才报**：`gcc -Q --help=warnings` 显示它默认是档位 0（关），`-Wall` 把它开到 1，但同一份代码在 `-O0` 下依然一句话都不说（`-O0` 那份会话里只有 `-Wstringop-overflow`）。这是本书把示例统一编在 `-O0` 的直接后果——**`-O0` 下很多问题编译器根本看不出来，这也是为什么「编过了」不能当作「写对了」**；

- 这两条**都不能当检测模式**：`-Wstringop-overflow` 本身是真的，但它**默认就开着**，压根归不到某个开关头上——实测把默认档与 `=1` 到 `=4` 一共五档全试一遍（甚至不写任何 `-W`），同一份代码吐出来的 stderr **逐字相同**（这条实测记在工具链事实文件 `build/toolchain.json` 的 `detection.wstringop-overflow.reason` 里）。所以清单里写 `"detect": "wstringop-overflow"` 会被闸门判 `detect_unavailable`——**正文按「编译器给的警告」引用它，才是诚实的说法。**

那「编译器能看见的越界」还有没有更强的证据？有，下一小节。

分析器：编译期就把越界指出来

`-Wall` 上面那两条警告靠的是「长度能算出来」。想让它连**你的意图**一起看，就用 `-fanalyzer`：这个开关会让编译器跑一遍**路径分析**，把「这个盒子里只有 4 个 int，你写了下标 7」这种推理链整个打出来。下面这份程序两种形状各写一个：

c examples/ch04/oob_fanalyzer/src/oob_fanalyzer.c 第 6–16 行

```c
void write_fixed_subscript(void) {
    int a[4] = {1, 2, 3, 4};
    a[7] = 1;                           /* 常量下标 7：越界了 */ 常量下标 7：分析器一眼看出来
    printf("a[0] = %d\n", a[0]);
}

void copy_fixed(void) {
    char buf[8];
    strcpy(buf, "0123456789ABCDEF");    /* 17 个字节进了 8 个字节的盒子 */ 常量源 + 定长目标
    printf("buf = %s\n", buf);
}
```

8 常量下标 7：分析器一眼看出来
14 常量源 + 定长目标

这份清单用的检测模式是 `fanalyzer`（编译期检测，闸门按 `toolchain.json` 给它加 `-fanalyzer`，并检查 stderr 里有没有它声明的证据）：

终端会话 -fanalyzer：常量下标越界，它连「超出几个字节」都算给你 ch04-analyzer-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -fanalyzer -c
src/oob_fanalyzer.c -o oob_fanalyzer.o
src/oob_fanalyzer.c: In function 'write_fixed_subscript':
src/oob_fanalyzer.c:8:10: warning: stack-based buffer overflow [CWE-121] [-Wanalyzer-out-of-bounds]
    8 |     a[7] = 1;                           /* 常量下标 7：越界了 */
      |     ~~~~~^~~
  'write_fixed_subscript': events 1-2
    7 |     int a[4] = {1, 2, 3, 4};
      |         ^
      |         |
      |         (1) capacity: 16 bytes
    8 |     a[7] = 1;                           /* 常量下标 7：越界了 */
      |     ~~~~~~~~
      |          |
      |          (2) ⚠  out-of-bounds write from byte 28 till byte 31 but 'a' ends at byte 16
src/oob_fanalyzer.c:8:10: note: write of 4 bytes to beyond the end of 'a'
    8 |     a[7] = 1;                           /* 常量下标 7：越界了 */
      |     ~~~~~^~~
src/oob_fanalyzer.c:8:10: note: valid subscripts for 'a' are '[0]' to '[3]'
```

终端会话 （续）定长 strcpy：第二种能触发的形状 ch04-analyzer-2

```
src/oob_fanalyzer.c: In function 'copy_fixed':
src/oob_fanalyzer.c:14:5: warning: stack-based buffer overflow [CWE-121] [-Wanalyzer-out-of-bounds]
   14 |     strcpy(buf, "0123456789ABCDEF");   /* 17 个字节进了 8 个字节的盒子 */
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  'copy_fixed': events 1-2
   13 |     char buf[8];
      |          ^~~
      |          |
      |          (1) capacity: 8 bytes
   14 |     strcpy(buf, "0123456789ABCDEF");   /* 17 个字节进了 8 个字节的盒子 */
      |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
      |     |
      |     (2) ⚠  out-of-bounds write from byte 8 till byte 16 but 'buf' ends at byte 8
src/oob_fanalyzer.c:14:5: note: write of 9 bytes to beyond the end of 'buf'
   14 |     strcpy(buf, "0123456789ABCDEF");   /* 17 个字节进了 8 个字节的盒子 */
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
src/oob_fanalyzer.c:14:5: note: valid subscripts for 'buf' are '[0]' to '[7]'
```

这两块会话里，最该记住的是五样东西：

- `warning: stack-based buffer overflow [CWE-121] [-Wanalyzer-out-of-bounds]`——诊断名是 `-Wanalyzer-out-of-bounds`，`CWE-121` 是业界给「栈上缓冲区溢出」编的号（`CWE` 是一份公开的缺陷编号表）；
- **`CWE-121` 与 `CWE-122` 的区别就是内存在哪**：本单元这两份样本的数组都在**栈**上，所以是 121（stack-based）；我把同一段代码换成 `int *a = malloc(16); a[5] = 99;` 再跑一遍，同一条开关说的是 `heap-based buffer overflow [CWE-122]`。**看到 CWE 号先问「这块内存在哪」**，不要背号；
- **形状决定它报不报**：把同样的越界改成变量下标（`for (int i = 0; i < 8; i++) sum += a[i];`）再试，本机 `-fanalyzer` **一声不吭**——它要的是「编译期就能钉死」的证据，`a[7] = 1` 与常量长度的 `strcpy` 才满足这个条件。所以「分析器没报」不等于「没有越界」，只等于「它这里没法确定」；
- `(1) capacity: 16 bytes` 与 `(2) out-of-bounds write from byte 28 till byte 31 but 'a' ends at byte 16`——分析器把「这块内存多大」和「你写到了第几到第几个字节」都算出来了；
- **编译退出码仍是 0**。分析器只负责报告，不阻断编译。所以「编过了」和「没有问题的程序」是两件事——这也解释了为什么 `detect=fanalyzer` 的清单声明的是「stderr 里必须出现哪些字样」，而不是退出码。

工程惯例 · **工程惯例：-fanalyzer 该不该常开**

`-fanalyzer` 不在本书的基础参数里，因为它慢、有误报，做的是「路径推理」。**这是惯例选择，不是标准规定**：标准对「编译器该给什么警告」一个字没写。

用法：**平时在 `-O0` 抓真实运行现象；要查内存类问题时单独开一次 `-fanalyzer`**——它在运行之前就把「装不下」指出来。但它只看得见**代码里能推出来的**越界：4.9 节那份 `copy_into_small(const char *src)` 源是形参、长度运行期才知道，`-fanalyzer` 一声不吭，只有跑起来踩到校验值才会 `*** stack smashing detected ***`。**两条证据互补。**

工程惯例 · **工程惯例：strncpy 不是「安全版 strcpy」**

先把界限说清：**这是惯例讨论，不是标准**——标准只定义每个函数做什么，不规定你该用哪一个。很多人看到 `strcpy` 出事就换成 `strncpy(dst, src, sizeof dst)`，方向对，**但有坑**：源比目标长时它**不补结束标记**，目标里的字符串「没有结尾」，后面交给 `strlen`、`printf("%s")` 就是越界读。完整答案是「先算长度、再拷、最后自己补 `'\0'`」，更重要的习惯是：**从外部拿进来的字符串，长度必须先被限住**（4.10 节的 `fgets` 就在做这件事）。具体现象与修法在第 15 单元，这里先记结论：**换函数不等于换掉责任。**

4.10 读一行输入：fgets 与 scanf 的差别

前面九节的越界都是「我们自己写坏的」。最后这一节讲一个**每写一个交互程序就会遇到**的入口：从标准输入读一行。下面这份程序用两种读法各读一次，第二种故意读进一个 8 字节的数组：

c examples/ch04/fgets_bound/src/main.c 第 5–18 行

```c
int main(void) {
    char line[16]; 一行最多装 15 个字符 + 结束标记
    int guard = 12345;
    char word[8];

    printf("== fgets(line, 16, stdin) ==\n");
    if (fgets(line, (int)sizeof line, stdin) == NULL) { 把「盒子的容量」直接交给 fgets
        return 1;
    }
    report_line("第 1 次 ", line, sizeof line);
    if (fgets(line, (int)sizeof line, stdin) == NULL) {
        return 1;
    }
    report_line("第 2 次 ", line, sizeof line); 同一行没读完的部分还在输入流里
```

6 一行最多装 15 个字符 + 结束标记
11 把「盒子的容量」直接交给 fgets
18 同一行没读完的部分还在输入流里

c examples/ch04/fgets_bound/src/main.c 第 20–29 行

```c
    printf("== scanf(\"%%s\", word): word 只有 8 个字节 ==\n"); 第二段：读进 8 字节的数组
    printf("word = %p, guard = %p, 读之前 guard = %d\n",
           (void *)word, (void *)&guard, guard);
    if (scanf("%s", word) != 1) { 没写宽度，读到空白为止
        return 1;
    }
    print_raw("word 的前 8 个字节", word, sizeof word);
    printf("读之后 guard = %d\n", guard); 邻居又被打了
    return 0;
}
```

20 第二段：读进 8 字节的数组
23 没写宽度，读到空白为止
27 邻居又被打了

「这一行读到了没有 / 有没有被截断」的判断住在 `read_line.c` 里，关键就是**看回来的最后那个字符是不是换行**：

c examples/ch04/fgets_bound/src/read_line.c 第 6–13 行

```c
void report_line(const char *label, const char *line, size_t capacity) { strlen 数到结束标记
    size_t n = strlen(line); 先把长度打出来
    printf("%s读到 %zu 个字符: ", label, n);
    fputs(line, stdout);
    if (n > 0 && line[n - 1] != '\n') { 最后不是换行，说明盒子满了
        printf("  ← 盒子满了，这一行没读完（最多装 %zu 个字符）\n", capacity - 1);
    }
}
```

6 strlen 数到结束标记
7 先把长度打出来
10 最后不是换行，说明盒子满了

清单里带了 `stdin`，喂进去两行：第一行 20 个字符、第二行 14 个字符。跑出来是：

终端输出 终端输出 fgets-bound-out

```
== fgets(line, 16, stdin) ==
第 1 次 读到 15 个字符: 0123456789ABCDE  ← 盒子满了，这一行没读完（最多装 15 个字符）
第 2 次 读到 6 个字符: FGHIJ
== scanf("%s", word): word 只有 8 个字节 ==
word = 00000000005FFE94, guard = 00000000005FFE9C, 读之前 guard = 12345
word 的前 8 个字节: ABCDEFGH
读之后 guard = 1280002633
```

输出要分两段读。

第一段是 `fgets` 的截断行为。 `line` 是 16 个字节，`fgets(line, 16, stdin)` 最多读 **15** 个字符再补 `'\0'`。输入第一行是 `0123456789ABCDEFGHIJ`（20 个字符），所以第 1 次只拿到前 15 个（`0123456789ABCDE`），而且**末尾不是换行**——`report_line` 就是靠这一条判断「盒子满了」。第 2 次 `fgets` 接着把**没读完的那 5 个字符**（`FGHIJ` 与换行）读出来，末尾是换行，所以不再报截断。这是 `fgets` 最重要的一条行为：**它只读一行的一部分，剩下的留在输入流里，等你下一次读**——所以「一行没读完」这件事你必须自己处理（提示用户、或者把剩下的丢掉）。

这条判据有个边界：一行**正好** 15 个字符时，`fgets` 同样读不到换行，会被误判成「盒子满了」（本机实测：15 个字符与 20 个字符的行打出同一句提示）。要严格区分，还得再看下一个字符是不是换行；本例输入 20 个字符，确实是截断。

第二段是 `scanf("%s")` 的危险。 `word` 只有 8 个字节，`scanf("%s", word)` 没有宽度限制，读到空白为止——输入里那个 14 个字符的词，被**整个**塞进了 8 个字节的盒子。结果和 4.3 节一模一样：`word` 的前 8 个字节是 `ABCDEFGH`，而邻居 `guard` 从 12345 变成了 **1280002633**（`0x4C4B4A49`，四个字节正好是 `'I'`、`'J'`、`'K'`、`'L'`）。程序没崩，值变了，**没有任何提示。**

标准原文 · 标准原文：fgets 与 scanf("%s") 各自承诺了什么

先看 `fgets`，7.21.7.2 节「The `fgets` function」第 2 段：

> The `fgets` function reads at most one less than the number of characters specified by `n` from the stream pointed to by `stream` into the array pointed to by `s`. No additional characters are read after a new-line character (which is retained) or after end-of-file. A null character is written immediately after the last character read into the array.

大意：**最多读 `n-1` 个字符**；遇到换行（换行**会被保留**在结果里）或文件结束就停；读完之后**一定在末尾补一个空字符。**「`n-1`」和「一定补 `'\0'`」这两条，就是 `fgets` 能当安全入口的全部理由——你把 `sizeof s` 递进去，它自己就不会写坏。

再看 `scanf` 的 `%s`，7.21.6.2 节「The `fscanf` function」第 12 段里对 `s` 的规定：

> `s` Matches a sequence of non-white-space characters. ... If no `l` length modifier is present, the corresponding argument shall be a pointer to the initial element of a character array large enough to accept the sequence and a terminating null character, which will be added automatically.

大意：`%s` 匹配一串**非空白字符**，而且**标准把「数组足够大」写成了对调用者的要求**——它自己不检查、也不接收「数组多大」这个信息。上面 `word` 只有 8 个字节而输入词有 14 个字符，责任在调用者这一侧，标准把话说明白了。

出处：ISO/IEC 9899:2018（C17）§7.21.7.2 第 2 段、§7.21.6.2 第 12 段。

工程惯例 · 工程惯例：输入一律用 fgets，再自己解析

标准提供两条路：`fgets`（自己给上限）与 `scanf` 系列（`%s` 不给上限，但能写 `%15s` 这种宽度）。**本书的惯例是走 `fgets`**，理由三条：

- **上限只写一遍**：`fgets(line, sizeof line, stdin)` 把「盒子多大」与「读几个」绑在一起；`scanf("%15s", word)` 里的 `15` 是手写数字，改了数组忘了改它就会漏。
- **给你整行**：连换行一起拿到，剩下的自己分（`sscanf`、`strtol` 都行）；`scanf("%s")` 只给一个词，容易和下一次读错位。
- **截断通常是可见的**：`fgets` 截断时末尾多半没有换行（正好 `n-1` 个字符的边界见 4.10 正文）；`scanf("%s")` 越界时什么都不说。

代价是换行会一起带回来，**经常要自己把末尾的 `'\n'` 去掉。**换来的是：**输入的边界是你自己划的，不是运气划的。**

4.11 这一节新增了什么，以及小结

回到 4.1 那棵树：第 2 帧是「一个例子拆开看」，第 3、4 帧是十二个例子目录，第 5 帧是探针与会话。现在你可以逐个点名了：

- `array_basics`、`array_addr`、`array_decay`、`array_assign` —— 数组本身：初始化与数元素、地址连续性、退化成指针、不能整体赋值；
- `matrix_rowmajor` —— 二维数组的行主序，顺手把「指针数组不是二维数组」摆在一起对照；
- `strlen_vs_sizeof`、`char_array_vs_ptr`、`string_literal_ro` —— 字符串：两个函数的区别、两块内存、不可写的字面量；
- `oob_neighbor`、`oob_fanalyzer`、`strcpy_overflow` —— 越界的三档后果与两种证据（先踩坏邻居、分析器看得见、栈保护抓得住）；
- `fgets_bound` —— 输入这一侧的边界由谁负责；`warn_probe.c` 与 `session-warnings.txt` 是不进清单的探针与会话，专门用来读编译器给的警告原文。

这十二个目录都带 `src/`、`include/` 与清单，而且这一章第一次把**数组的工具函数写成了真正的模块**（`array_util.h` + `array_util.c`）。下一单元《指针与内存》接着往前：同一个指针，为什么 `int *` 与 `char *` 读出来的东西不一样、`malloc` 的内存住在哪、`free` 之后那些指针成了什么。

小结 · 这一节的要点

- **数组是一排连续、等宽的盒子**，编号从 0 起；`sizeof a` 量整块、`sizeof a[0]` 量一格，相除就是元素个数；没写的初值按静态对象处理（整数是 0），**不是垃圾值**；`a[i]` 就是 `*(a + i)`；**越界是 UB**，后果分三档——改到邻居（本机实测 `guard` 从 12345 变成 1111570744）、崩溃（退出码 3221225477）、stack smashing（开 `-fstack-protector-all` 后 3221226505 + `*** stack smashing detected ***`）。
- **数组名会退化成指针**（例外是 `sizeof` / `_Alignof` / `&`，外加一条「初始化数组的字面量」），参数里的 `int a[5]` 也一样——所以函数里 `sizeof a` 是 **8**，**长度只能由调用者一起传**；一维只读参数写 `const T *`（二维数组在 C17 里加不了，见 4.6）。**数组不能整体赋值、不能按值传参、不能返回**（`b = a;` 与 `int[3] f(void)` 各有一条真错误），整排搬只能逐元素或 `memcpy`。
- **字符串 = 字符数组 + `'\0'`**：`sizeof` 数字节、`strlen` 数到 `'\0'` 为止；`char s[] = "hi"` 是你自己的三个字节（可写），`char *p = "hi"` 只是一个指向只读字面量的指针（写它退出码 3221225477，且字面量**可能被合并**）。
- `strcpy` / `strcat` **不接收目标长度**，是越界的头号来源；`-Wstringop-overflow` **默认就开着**（`-O0` 也报），`-Warray-bounds` 要 `-O2`，`-fanalyzer` 能把常量下标与定长 `strcpy` 两种形状**在编译期**指出来（`CWE-121`，栈上）——但这些都是「编译器看到的」，运行期证据仍是退出码与 stderr。读输入用 `fgets(s, sizeof s, stdin)`（最多 `n-1` 个字符、**一定补** `'\0'`），`scanf("%s")` **要求你自己保证数组足够大**：本机实测 14 个字符的词把一个 8 字节数组的邻居写成了 1280002633。

自测题 · 自测题（先自己答，再对答案）

1. `int a[5] = {1, 2, 3};` 之后 `a[3]` 与 `a[4]` 各是多少？说出标准里规定这条行为的那句话大意，并说出 `{0}` 为什么能整排清零。
2. 同一个 `sizeof a` 写法，在 `main` 里得 20、在函数里得 8。分别说出这两处 `a` 的**类型**，以及为什么参数里写 `int a[5]` 也救不回来。
3. `char s[] = "hi"` 与 `char *p = "hi"`：哪个的 `sizeof` 是 3、哪个是 8？哪一块内存能写？如果 `p == q` 为真（`q` 是另一份同样的字面量），说明了什么、又**不能**说明什么？
4. 同事写了一句 `char buf[8]; strcpy(buf, 从输入拿到的字符串);`——源是运行期才拿到的，长度编译器算不出来，所以 `-O0` 下编译一句警告都没有。你告诉他「这句可能出事」，他会问「怎么证明」。请给出**两个**你现在就能跑的验证手段，并说出各自能证明什么、抓不到什么。
5. 读一行输入时，为什么本书推荐 `fgets(line, sizeof line, stdin)` 而不是 `scanf("%s", line)`？`fgets` 会给你带来哪一件需要自己处理的麻烦？

ⓘ 注 · **自测题答案**

1. 都是 **0**。标准 6.7.9 节第 21 段的大意是：**花括号里的初值比元素少时，剩下的部分按静态存储期对象的规矩初始化**（整数即 0）。`{0}` 之所以能整排清零，是因为它只显式写了第一个元素为 0，其余部分走的正是同一条「按静态对象初始化」的规矩。
2. `main` 里 `a` 的类型是 `int[5]`（数组），`sizeof` 量到整块 20 字节；函数里 `a` 的类型是 `int *`（指针），标准 6.7.6.3 节第 7 段规定「参数里写数组类型会被改写成指针类型」，所以 `sizeof` 量到 8 字节。写 `int a[5]` 也一样：方括号里的数字不携带长度信息，改写之后连痕迹都没了——这正是「数组参数必须配套传长度」的原因。
3. `s` 的 `sizeof` 是 **3**（它自己有三个字节），`p` 的 `sizeof` 是 **8**（它只是一个装地址的变量）。能写的是 `s` 那块（`s[0] = 'H'` 合法，改完 `s` 变成 `Hi` 而字面量没动）；`p` 指向的字面量**不可写**，本机写它退出码 3221225477。`p == q` 为真说明本机把两份内容相同的字面量**合并**成了一块（标准 6.4.5 节第 7 段说这件事「unspecified」，即没规定），它**不能**说明「字符串可以用 `==` 比较内容」——那要用 `strcmp`。
4. 两个手段：① **把源换成常量再编一次**——`strcpy(buf, "0123456789ABCDEF")` 会让 `-Wstringop-overflow` 立刻指出「写 17 个字节进 8 个字节的盒子」（本机默认就开着，不用加任何开关）。它只能证明「编译器在长度可算时看得见」，抓不到运行期才确定长度的写法，所以**不能当成「运行期没问题」的证据**。② **开栈保护再跑一次**——把清单写成 `detect=stack-protector`（等价于编译时加 `-fstack-protector-all`），本机实测 stderr 会出现 `*** stack smashing detected ***`、退出码 **3221226505**；去掉这个开关（阴性对照）退出码变成 **3221225477**、stderr 为空，证明这句诊断确实是这个开关抓出来的。它只能抓**栈上数组**被写坏、且踩到校验值的情况，堆越界与只踩到邻居它都抓不到。再加上 ②' **用分析器看编译期**——把样本改成常量下标（`a[7] = 1;`）或定长 `strcpy` 形状，加 `-fanalyzer` 编译，本机实测报 `stack-based buffer overflow [CWE-121] [-Wanalyzer-out-of-bounds]`，并给出「capacity: 16 bytes」与「写到第 28 到第 31 个字节」的推理链；它的局限是**源是运行期才确定的形状就报不出来**（那份 `copy_into_small(const char *src)` 就一声不吭），而且编译退出码仍是 0。合起来：**一个证明「编译期看得出来」，一个证明「运行期真的坏了」，但它们都不能替代「别让目标装不下」这件事本身。**
5. 因为 `scanf("%s")` **没有上限**：标准 7.21.6.2 节要求调用者保证数组「足够大」，它自己不检查，写到内存哪儿它也不管（本机实测 14 个字符的词把 8 字节数组的邻居 `guard` 从 12345 改成了 1280002633，程序还不崩）；而 `fgets` 最多读 `n-1` 个字符并**一定补** `'\0'`（7.21.7.2 节），`sizeof line` 一递，边界就是它自己划的。`fgets` 带来的麻烦是：**它把换行一起带回来**（截断时末尾没有换行，正好可以用来判断「这一行没读完」），拿到之后常常要自己把末尾的 `'\n'` 去掉。

指针与内存：地址、类型与生命周期

◎ **学习目标** · **看完这一单元，你应该能**

- 说出指针变量里装的是什么，并解释 `&` 与 `*` 为什么是一对逆运算；
- 用 `(int *)` 与 `(char *)` 读同一块内存，说出「一次读几个字节」到底由谁决定（含未对齐访问与 `memcpy` 的分工）；
- 算出 `p + 1` 前进几个字节、`q - p` 得到几个元素，解释 `a[i]` 与 `*(a + i)` 为什么等价，并指出哪些指针运算本身已经是 UB；
- 说出栈上的对象与 `malloc` 出来的对象各自活多久、判空该写在谁那一层，以及 `free` 之后那些指针的处境（悬垂指针、double free 各自的现象与退出码）；
- 用 `-fanalyzer`、`-fstack-protector-all`、`-ftrapv` 把「悄悄出错的程序」变成看得见的证据、说出各自抓不到什么，并在 gdb 里看清一个指针的地址、值与它指向的那几个字节。

9.1 先看工程长什么样

学指针最容易走偏的方式，是把它当成一条语法规则背下来（「星号就是把地址里的值取出来」）。这一节我们照老规矩来：先看**文件**。指针的每一个坑，最后都落在两件事上——**哪块内存在哪**、**谁活着**——而这两件事都能被文件与命令钉死。

这一节是第 4 章的第二单元，也是全书的第 9 单元。按大纲，它前面还有一个单元（`chapters/ch04/01-arrays-and-strings.md`，数组与字符串），那一个也还没有落盘：本书把这一节选作**试点单元**先写。所以下面第 1 帧画的是**这一节故事的起点**——`examples/` 下已经躺着第 1 章那个项目，而第 4 章的项目（`examples/ch04/`）还不存在，本节要把它搭起来。

目录树 **这一节的例子工程：从没有 ch04，到十个例子目录加三类附件**

第 1 步 **本节开始时，examples/ 下只有第 1 章那个项目**

```
examples/
└── ch01/ ：第 1 章的项目（本节不动它）
```

第 2 步 **本节建起第 4 章的项目，先摆上前五个例子**

```
examples/
├── ch01/ ：第 1 章的项目
└── ch04/ ← 本课新增
    ├── ptr_basics/ ：& 与 * ← 本课新增
    ├── ptr_types/ ：一次读几个字节 ← 本课新增
    ├── null_deref/ ：空指针解引用 ← 本课新增
    ├── ptr_arith/ ：p + 1 与 p - q ← 本课新增
    └── dangling_stack/ ：返回局部数组地址 ← 本课新增
```

第 3 步 **再加上另外五个例子目录，十个到齐**

```
examples/
└── ch04/
    ├── 前五个例子目录 ： 同上
    ├── heap_after_free/ ：free 之后再写 ← 本课新增
    ├── heap_oob_fanalyzer/ ：堆越界（分析器） ← 本课新增
    ├── stack_smash/ ：栈越界（两种量） ← 本课新增
    ├── ptr_offset_trapv/ ：用 int 算偏移 ← 本课新增
    └── const_ptr/ ：三种 const 写法 ← 本课新增
```

第 4 步 **本节另外留下的会话与探针**

```
examples/
└── ch04/
    ├── 十个例子目录 ： 各带清单 + src/ + include/
    ├── session-gdb-pointers.txt ：gdb 看指针的会话 ← 本课新增
    ├── session-regions-pointers.txt ：三个区的地址 ← 本课新增
    ├── session-double-free-pointers.txt ：double free ← 本课新增
    ├── mem_regions.c ：三个区的探针 ← 本课新增
    └── double_free.c ：double free 的探针 ← 本课新增
```

第 5 步 **还有两份汇编，是 9.9 节的对照材料**

```
examples/
└── ch04/
    └── heap_after_free/
        ├── main-O0.s ：-O0 汇编，那句写还在 ← 本课新增
        └── main-O2.s ：-O2 汇编，那句写被删 ← 本课新增
```

第 1 帧是这一节故事的起点：`examples/` 下只有第 1 章那个项目。第 2、3 帧是本节搭起来的 `examples/ch04/`——**十个**例子目录（分两帧列完），每个都是「源文件放 `src/`、声明放 `include/`、编译与对账的参数写在 `.json` 清单里」的最小工程（第 1 单元 1.7 节讲过分文件的理由，从第二个项目起本书不允许单文件工程）。第 4 帧是本节另外留下的两类东西：三段抓下来的**终端会话**与两个**单文件探针**（没有清单，理由见 9.11）；第 5 帧是两份**汇编**（`-O0` 与 `-O2` 各一份，9.9 节拿它们对照）。

先记住这十个目录各自的**一件事**，后面每一节都会点名其中一个：

- `ptr_basics`、`ptr_types`、`null_deref`、`ptr_arith`、`dangling_stack` —— 指针本身、类型、空指针、指针运算、生命期这五件事（9.2 到 9.7 节挨个讲）；
- `heap_after_free`、`heap_oob_fanalyzer`、`stack_smash`、`ptr_offset_trapv`、`const_ptr` —— 堆上的悬垂与 double free、越界写的三种证据、`const` 的三种写法（9.8 到 9.10 节）。

工程惯例 · **工程惯例：为什么「一个例子一个目录、声明放头文件」**

标准对这件事只有一个要求：一个程序有且只有一个 `main`。**分几个文件、怎么分，全是惯例。**本书从第二个项目起要求每个例子都是 `src/` + `include/` 的最小工程，理由有三条：

- **每条结论都要能被单独验证**：这个例子只演示「类型决定读几个字节」这一件事，出问题时你能立刻知道是哪里错了；
- **声明与定义分家，接缝写在明面上**：`include/xxx.h` 里只放**声明**（函数名、参数、返回值），`src/xxx.c` 里放**定义**（函数体），`src/main.c` 只负责组装与打印；
- **能被别人接手**：十个小文件比一个三千行的 `main.c` 好读得多。

代价是要多学一件事：头文件里写什么、不写什么。写重了会撞符号，写漏了会 `implicit declaration`——这些在第 7 章讲透，现在只要记住方向：**声明进头文件，定义进源文件。**

9.2 指针是一个变量，里头装的是地址

指针不是什么神秘的东西：**它就是一个变量**，只不过里面装的是「另一块内存在哪」这个信息。这句话听起来像废话，但它能立刻解释掉三个新手疑问：为什么指针也有地址（`&p` 是合法的）、为什么指针也有大小（本机 8 个字节）、为什么给指针赋值不会顺便把指向的东西复制一份。

看真程序。这是 `examples/ch04/ptr_basics/src/main.c` 的前半段：

c examples/ch04/ptr_basics/src/main.c 第 5–16 行

```c
int main(void) {
    int x = 42;                                 /* 一个四字节的整数 */
    int *p = &x;                                /* p 里装的是 x 的地址 */  /* 一个八字节的指针 */

    printf("x 的值              = %d\n", x);
    printf("&x (x 的地址)       = %p\n", (void *)&x);
    printf("p (同一个值)        = %p\n", (void *)p);  /* p 里装的是地址，与 &x 同一个值 */
    printf("*p (顺着地址取回来) = %d\n", *p);
    printf("&x == p             = %s\n", (&x == p) ? "真" : "假");
    printf("*&x (取地址再取回来) = %d\n", *&x);
    printf("&*p == p            = %s\n", (&*p == p) ? "真" : "假");  /* 取地址再取回来，绕一圈 */
    printf("sizeof x = %zu, sizeof p = %zu\n", sizeof x, sizeof p);
```

6 一个四字节的整数
7 一个八字节的指针
11 p 里装的是地址，与 &x 同一个值
15 取地址再取回来，绕一圈

后半段演示三件更容易被忽略的事：顺着指针写会改到原来那个变量、函数里改指针指向的对象能影响调用者、而**改指针自己不能**：

c examples/ch04/ptr_basics/src/main.c 第 18–29 行

```c
    *p = 7;                                     /* 顺着地址改：x 也变了 */  /* 顺着地址改，x 也变 */
    printf("执行 *p = 7 之后，x = %d\n", x);

    bump(&x);  /* 把 &x 交给函数 */
    printf("调用 bump(&x) 之后，x = %d\n", x);

    int y = 1;
    int *q = &y;
    swap_pointers_bad(p, q);  /* 想交换两个指针 */
    printf("swap_pointers_bad 之后：*p = %d，*q = %d\n", *p, *q);
    swap_values(p, q);  /* 交换的是值 */
    printf("swap_values 之后：*p = %d，*q = %d\n", *p, *q);
```

18 顺着地址改，x 也变
21 把 &x 交给函数
26 想交换两个指针
28 交换的是值

这三个函数住在 `src/ptr_basics.c` 里——注意它们的样子，差别只有一行：

c examples/ch04/ptr_basics/src/ptr_basics.c 第 3–17 行

```c
void bump(int *p) {
    *p = *p + 1; 改的是 p 指向的那个整数
}

void swap_pointers_bad(int *a, int *b) {
    int *tmp = a; 把形参拷一份给 tmp
    a = b; a 只是这份副本
    b = tmp;
}

void swap_values(int *a, int *b) {
    int tmp = *a; 这次动的是「指向的对象」
    *a = *b;
    *b = tmp;
}
```

4 改的是 p 指向的那个整数
8 把形参拷一份给 tmp
9 a 只是这份副本
14 这次动的是「指向的对象」

跑出来是这样（这块输出由流水线逐字对账，不是我抄的）：

终端输出 终端输出 ptr-basics-out

```
x 的值              = 42
&x（x 的地址）      = 00000000005FFE9C
p（同一个值）       = 00000000005FFE9C
*p（顺着地址取回来）= 42
&x == p             = 真
*&x（取地址再取回来）= 42
&*p == p            = 真
sizeof x = 4, sizeof p = 8
执行 *p = 7 之后，x = 7
调用 bump(&x) 之后，x = 8
swap_pointers_bad 之后：*p = 8, *q = 1
swap_values 之后：*p = 1, *q = 8
```

把这份输出按三行一组读：

- **`&x` 与 `p` 打出来是同一个数**，而且 `&x == p` 是「真」。这不是巧合：`p = &x` 之后，`p` 里的值就是 `x` 的地址。`*&x` 与 `&*p` 打出来也都对得上——`&` 与 `*` 是**一对逆运算**：`&` 从对象拿到地址，`*` 从地址拿回对象，绕一圈回到原地。
- **`sizeof x = 4`、`sizeof p = 8`**：`x` 是 `int`（本机 4 字节），`p` 是**指针**，本机 8 字节。指针的大小与它指的东西无关——`int *`、`char *`、`double *` 在本机都是 8 字节，因为「一个地址」需要多少位，是这台机器的事。
- **`swap_pointers_bad` 没换成，`swap_values` 换成了。**原因就在 `src/ptr_basics.c` 第 9、10 行：`a = b; b = tmp;` 改的是**形参这份副本**——C 的函数参数永远是**值传递**，指针也不例外（`a` 是一份拷贝，拷贝的是一张地址纸条）。想改调用者的东西，必须**顺着指针去改它指向的对象**（`*a = *b`），这就是 `swap_values` 干的事。

`00000000005FFE9C` 这串数字是**本机**给这个程序的地址：换一台机器、甚至同一个程序换个编译方式，它都会变（有的系统上每次运行都不同）。所以要读的是「`&x` 与 `p` 是不是同一个数」，不是那串数字本身——本书后面凡是出现地址的地方，都是同一个口径。

把这份输出画成图，就是「一个盒子里装着另一个盒子的门牌号」：

内存形态 **x 与 p 两个盒子**

第 1 步 **int x = 42; —— 开一个盒子，写上 42**

x ： 盒子里的值是 42

第 1 帧只有一个盒子（`x`）。第 2 帧多了一个盒子（`p`），它自己也有地址（`&p`）——这一点很多人第一次见到会愣一下：**指针也是变量，所以它自己也要占内存、也有门牌号。**第 3 帧是 `*p = 7;` 的效果：改的是箭头**指着的那个盒子**，指针自己纹丝不动。

调试实录 · 调试实录：gdb 里的一个指针，三行命令全看穿

在 `p` 上打断点只能看到「停在哪一行」，要看**指针本身**长什么样，得用三种命令：`p p`（打印指针变量里装的值，也就是那张地址纸条）、`p *p`（顺着地址取回那个对象）、`x/4xb p`（不走类型，直接把 `p` 指向的 **4 个字节**按十六进制打出来），再加上 `p &x`（`x` 自己的地址）做对照。

下面这块是 `examples/ch04/session-gdb-pointers.txt` 的前半段（会话文件里的原文）。读的时候注意三处：`$2` 与 `$3` 是同一个值；`$1` 与 `$4` 都是 42；最后那行 `0x2a 0x00 0x00 0x00` 就是 42 在内存里的四个字节。

终端会话 gdb：p &x、p p、p *p、x/4xb p 各看到什么 ch04-gdb-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/ptr_basics.c -o app.exe
$ gdb --batch -ex "break src/main.c:18" -ex run -ex "p x" -ex "p &x" -ex "p p" -ex "p *p" -ex "x/4xb p" -ex
"p sizeof(int)" ./app.exe
Breakpoint 1 at 0x140001581: file src/main.c, line 18.
[New Thread 93696.0xb7d8]
Thread 1 hit Breakpoint 1, main () at src/main.c:18
18          *p = 7;                                     /* 顺着地址改：x 也变了 */
$1 = 42
$2 = (int *) 0x5ffe9c
$3 = (int *) 0x5ffe9c
$4 = 42
0x5ffe9c:       0x2a    0x00    0x00    0x00
$5 = 4
$ rm -f app.exe
```

三行命令对上三件事：`$1 = 42` 是 `x` 的**值**；`$2 = (int *) 0x5ffe9c` 与 `$3 = (int *) 0x5ffe9c` 说明 `&x` 与 `p` 是**同一个地址**；`$4 = 42` 说明顺着 `p` 走回去拿到的正是 `x`。最后一行 `x/4xb p` 打出 `0x2a 0x00 0x00 0x00`——`0x2a` 就是 42，后面三个 `0x00` 是那个 `int` 剩下三个字节，**低位字节在最前面**（小端，与 9.3 节 `0x41424344` 那件事是同一个规则）。gdb 的 `x` 命令是绕过类型的：`*p` 按 `int` 读 4 个字节，`x/4xb` 直接给你那 4 个字节。

大白话 · 用大白话说「指针不是地址」

把内存想成一排信箱，每个信箱有编号。

- `int x = 42;` 是「开一个信箱，写上 42」；
- `int *p = &x;` 是「**另外**开一个信箱（它自己也有编号），里面写的是 x 那个信箱的编号」；
- `*p = 7;` 是「按 `p` 里写的编号找过去，把那个信箱里的内容擦掉写成 7」——所以 `x` 变了，而 `p` 自己一点没变；
- `p = &y;` 是「把 `p` 里那张纸条换成 `y` 的编号」——`x` 与 `y` 的内容一个都不会变。

所以「指针」这个词指的其实是**两个东西**：一张写着编号的纸条（指针变量本身），和它指向的那个信箱（对象）。新手最容易把这两件事合成一件，然后就会问出「为什么我把指针改了，原来的数没变」。

标准原文 · **标准原文：`&` 与 `*` 是怎么定义的**

C 语言标准（ISO/IEC 9899，C17）6.5.3.2 节「Address and indirection operators」规定：

> The unary & operator yields the address of its operand. ... The unary * operator denotes indirection. ... if the operand has type ''pointer to type'', the result has type ''type''.

大意：`&` 交出一个对象的**地址**，`*` 是**间接寻址**——对一个类型是「指向 T 的指针」的操作数用 `*`，结果的类型就是 `T`。标准刻意不提「地址就是一个整数」，因为标准里的地址是抽象的东西；把指针当整数算，是**实现**允许、但标准不保证的事。

出处：ISO/IEC 9899:2018（C17）§6.5.3.2 第 3、4 段。

⚠ 陷阱 · **「指针就是地址」——这句话会让你在后面的每一节里踩坑**

你会怎么被坑到：一旦心里认定「指针 = 地址」，下面这些问题你都会答错——

- 为什么 `int *` 与 `char *` 指同一个地址，读出来的东西完全不同？（9.3 节：**类型**决定一次解引用读写几个字节）
- 为什么 `p + 1` 不是「地址加 1」？（9.6 节：指针运算按**元素**走，前进 `sizeof(*p)` 个字节）
- 为什么 `free(p)` 之后 `p` 里的地址还照样打印得出来，程序却已经错了？（9.8 节：地址还在，**对象没了**）

真相：指针变量的值确实是一个地址，但指针的**类型**带着两样地址之外的信息——**一次读几个字节**、以及**按几个字节一步往前走**。所以本书的行文会是「这个指针指向类型 `int` 的对象」，而不是「这个指针是一个地址」。

9.3 指针不是地址：类型决定一次读几个字节

上一节留下的第一个问题：为什么类型这么重要？拿一段真程序看最清楚。下面这段把同一个 `int` 变量的地址，分别交给 `int *` 与 `char *`：

c examples/ch04/ptr_types/src/main.c 第 5–21 行

```c
int main(void) {
    int v = 0x41424344;                 /* 四个 ASCII 字节: 'A' 'B' 'C' 'D' */ 四个字节的整数
    const int *ip = &v; int * 看它
    const char *cp = (const char *)&v; char * 也看它
    unsigned char bytes[4];

    printf("同一个地址，两种类型的指针都指得过来: \n");
    printf("  (int *) 这个地址            = %p\n", (const void *)ip);
    printf("  (char *) 同一个地址         = %p\n", (const void *)cp);
    printf("  它们是同一个地址吗          = %s\n", ((const void *)ip == (const void *)cp) ? "是" : "不是"); 两个指针的值一模一样
    printf("  一次解引用读几个字节: int* = %zu\n", int_read_size());
    printf("  一次解引用读几个字节: char*= %zu\n", char_read_size());

    printf("按 int 读: *ip = 0x%08X\n", read_head_as_int(&v));
    read_bytes(&v, bytes);
    printf("按 char 逐字节读: 0x%02X 0x%02X 0x%02X 0x%02X\n", bytes[0], bytes[1], bytes[2], bytes[3]);
    printf("其中的 cp[0] 当成字符是 '%c', cp[3] 是 '%c'\n", cp[0], cp[3]);
```

6 四个字节的整数
7 int * 看它
8 char * 也看它
14 两个指针的值一模一样

两个「读法」的实现住在模块里，故意写成两个小函数，免得编译器把答案提前算好：

c examples/ch04/ptr_types/src/ptr_types.c 第 5–23 行

```c
int read_head_as_int(const void *p) {
    const int *ip = (const int *)p;
    return *ip;
}

void read_bytes(const void *p, unsigned char out[4]) {
    const char *cp = (const char *)p;
    for (int i = 0; i < 4; i++) {
        out[i] = (unsigned char)cp[i];
    }
}

size_t int_read_size(void) {
    return sizeof(int);
}

size_t char_read_size(void) {
    return sizeof(char);
}
```

5 按 int 读，一次四个字节
6 先把地址看成 int 的地址
7 这一次解引用读四个字节
10 按 char 读
11 这一次解引用读一个字节

真跑出来：

终端输出 终端输出 ptr-types-out

```
同一个地址，两种类型的指针都指得过来：
  (int *) 这个地址           = 00000000005FFE8C
  (char *) 同一个地址        = 00000000005FFE8C
  它们是同一个地址吗         = 是
  一次解引用读几个字节：int* = 4
  一次解引用读几个字节：char*= 1
按 int 读：*ip = 0x41424344
按 char 逐字节读：0x44 0x43 0x42 0x41
其中的 cp[0] 当成字符是 'D'，cp[3] 是 'A'
挪一步：ip + 1 比 ip 大 4 个字节
挪一步：cp + 1 比 cp 大 1 个字节
对齐要求：int 是 4，char 是 1
从 raw + 1（没按 int 对齐）硬按 int 读：0x11414243
同样那几个字节改用 memcpy 搬：0x11414243
```

这份输出里有三组数字要连起来读：

1. `(int *)` 与 `(char *)` **打出来的地址完全相同**，`它们是同一个地址吗 = 是`。地址就是一个数，谁指过来都一样。
2. **「一次解引用读几个字节」：** `int *` **是 4，** `char *` **是 1。**这才是类型真正管的事。`*ip` 一次读 4 个字节，`cp[i]` 一次读 1 个字节——同一块内存，两种读法。
3. `0x41424344` **与** `0x44 0x43 0x42 0x41`：`v` 的十六进制值是 `41424344`（ASCII 里 `'A'`=0x41、`'B'`=0x42…），而按字节逐个读出来是 `44 43 42 41`——**反的。**这不是 bug：本机是**小端**（little-endian），一个多字节整数里，**低位字节住在低地址。**所以 `cp[0]` 打出的是 `'D'`（0x44），`cp[3]` 才是 `'A'`。

内存形态 同一块内存的两种读法

第 1 步 v 这块内存里有四个字节

v 的地址 : 0x…5FFE8C，一块四个字节的内存 | 字节 0 : 0x44 ('D') | 字节 1 : 0x43 ('C') | 字节 2 : 0x42 ('B') | 字节 3 : 0x41 ('A')

第 1 帧是这块内存的真实内容（四个字节，从低地址到高地址是 `44 43 42 41`）。第 2 帧是 `int *` 的读法：一次吃四个字节，按小端规则拼成一个 32 位整数 `0x41424344`。第 3 帧是 `char *` 的读法：一次只吃一个字节，所以它**看不到**一个整数，只能看到一个字符。第 4 帧把两个指针摆在一起：**值相同、类型不同**——它们指向同一块内存，但「一次看多少」不一样。

> {} **语法规则** · **`void *`：一个不带类型的地址**
>
> 本节的模块里好几个函数的参数写的是 `const void *p`（比如 `read_head_as_int`、`read_via_memcpy`）。`void *` 是**通用指针**：任何「指向对象的指针」都能赋给它、也能从它转回来，但标准同时给了它两条限制——**不能解引用它**（`*p` 编不过，因为编译器不知道一次该读几个字节）、**不能对它做指针算术**（`p + 1` 编不过，因为它不知道一步该走多远）。
>
> 所以在函数体里必须先转成具体类型：`const int *ip = (const int *)p;` 之后才谈得上解引用。**这一步转换恰恰是类型的责任**：`int *` 与 `char *` 的差别、对齐的约束、一次读几个字节，全都在这句转换里定下来了。

> 大白话 · **同样一块地，两种量法**
>
> **这**段内存像一块四个格子的巧克力。`int *` 是拿一把四格量尺：一次量满四格，说「这一板是 0x41424344」。`char *` 是拿一把一格量尺：一次量一格，说「第一格是 'D'」。
>
> 尺子不一样，量出来的答案自然不一样，但**地还是那块地**。真正决定「量几格」的，不是地址，是**尺子的类型**。

> 标准原文 · **标准原文：`(int *)` 这次转换意味着什么**
>
> 同一个标准 6.3.2.3 节「Pointers」说：
>
> > A pointer to an object type may be converted to a pointer to a different object type. If the resulting pointer is not correctly aligned for the referenced type, the behavior is undefined.
>
> 大意：**「指向 T 的指针」可以转成「指向 U 的指针」**，这是标准允许的；但转换出来的指针如果**没有按 U 的要求对齐**，那行为就是未定义的（9.4 节会拿真程序试这件事）。
>
> **请注意标准没说的一句话**：它从来没说「你可以用 `int *` 去读一个 `char` 数组里的四个字节」——那样做会同时踩到对齐规则与**严格别名**规则（第 19 单元讲）。标准给出的正统做法是 `memcpy`，9.4 节会演示。
>
> 出处：ISO/IEC 9899:2018（C17）§6.3.2.3 第 7 段。

9.4 对齐与 `memcpy`：同一串字节的三种取法

上面那段程序留下的最后一个问题：把一个**没对齐**的地址硬按 `int` 读，会发生什么？本机实测给你答案。同一个例子里的后半段：

c examples/ch04/ptr_types/src/main.c 第 25–31 行

```c
25
26      char raw[8] = {(char)0x44, (char)0x43, (char)0x42, (char)0x41,
```

```
27                      (char)0x11, (char)0x22, (char)0x33, (char)0x44};
28      printf("对齐要求: int 是 %zu, char 是 %zu\n", int_align(), char_align());
29      printf("从 raw + 1 (没按 int 对齐) 硬按 int 读: 0x%08X\n", read_unaligned_int(raw)); 故意从 raw + 1 开始读
30      printf("同样那几个字节改用 memcpy 搬: 0x%08X\n", read_via_memcpy(raw + 1)); 同样的字节，换 memcpy 搬
31      return 0;
```

29 故意从 raw + 1 开始读
30 同样的字节，换 memcpy 搬

这两个函数的差别，只有一行：

c examples/ch04/ptr_types/src/ptr_types.c 第 25–41 行

```
25  size_t int_align(void) {
26      return _Alignof(int);
27  }
28
29  size_t char_align(void) {
30      return _Alignof(char);
31  }
32
33  int read_unaligned_int(const char *p) {
34      return *(const int *)(p + 1); 硬按 int 读一个没对齐的地址
35  }
36
37  int read_via_memcpy(const void *p) {
38      int v;
39      memcpy(&v, p, sizeof v); memcpy 按字节搬，不管对齐
40      return v;
41  }
```

34 硬按 int 读一个没对齐的地址
39 memcpy 按字节搬，不管对齐

上面那份输出里的最后两行是：

- `从 raw + 1 (没按 int 对齐) 硬按 int 读: 0x11414243`
- `同样那几个字节改用 memcpy 搬: 0x11414243`

两个结果一模一样，但只有第二种是标准保证的。这就是本节要你带走的那条区别：

大白话 · **「能跑」与「保证能跑」是两件事**

`0x11414243` 这个值，两个函数都算对了。区别不在结果，而在**承诺**：

- 第一种写法（`*(const int *)(p + 1)`）对标准来说已经踩到**未定义行为**了——标准只允许你把指针转成别的类型，但如果新地址没按 `int` 对齐，行为就是未定义的（这是对齐规则，不是「越界」：`raw + 1` 那个地址本身还在数组里，越界说的是 9.6 节那种算到数组外面去的指针）。它今天在这台 x86-64 上「能跑」，只是因为 x86-64 的硬件允许未对齐访问（慢一点而已）；
- 第二种写法（`memcpy`）标准明确保证：它按**字节**搬，**不要求对齐**，也不关心你搬的是什么东西。

所以规矩很简单：**要跨类型看字节，用 `memcpy`；要用别的类型解引用，先确认地址对齐。**换成 ARM 上某些配置、或者换成一个会把未对齐访问变成陷阱的编译器，第一种写法会当场给你一个非法指令。

工程惯例 · **工程惯例：为什么很多库宁可 `memcpy` 一下**

你在标准库里会反复看到这种形状：先 `memcpy` 到一个对齐的局部变量里，用完再 `memcpy` 回去。看起来啰嗦，但这是**唯一**既能跨平台、又不会踩别名规则的做法。

代价是两件事：多一次内存拷贝（现代编译器通常能把它优化成一条 `mov`，这是它自己决定的事，不是你保证的事）；以及你得自己在注释里说清「这里搬的是什么类型」。第 11 单元讲结构体 padding 时，还会再遇到同一套规矩。

{} 语法规则 · `printf` 的 `%p` 配什么类型

`%p` 要的是 `void *`。把一个 `int *` 直接交给 `%p`，**本机就会警告**（我实测：`warning: format '%p' expects argument of type 'void *', but argument 2 has type 'int *' [-Wformat=]`，`-Wall -Wextra` 给的），所以本书所有打印指针的地方都写成 `(void *)p`：**这是 `%p` 的格式约定，不是类型转换的魔法**——`(void *)` 只是把「指向某类型」这层信息丢掉，好让 `printf` 用一个统一的方式打印它（本机打出来是 16 位十六进制）。

顺带一句：`%p` 不是标准里为了好看定的，它的输出格式由实现决定——标准只要求「实现定义」，所以同一段代码在不同运行库上打出来的样子可能不同（本机有前导零，有些平台打印 `0x` 前缀）。想比较两个地址就**用 `==` 比较指针**，别去比字符串。

9.5 空指针：判空是接口的一部分

一个指针可以指向任何地方，**也可以谁都不指。**C 里表示「谁都不指」的那个值是**空指针**，写出来是 `NULL`（它从 `stddef.h` 来；`NULL` 在标准里就是一个「值为 0 的整数常量表达式」，见出处）。

空指针最要命的地方不是它自己，而是**解引用它。**真程序：

c examples/ch04/null_deref/src/main.c 第 5–17 行

```c
int main(void) {
    int x = 1;

    if (write_checked(&x, 5) == 0) {
        printf("判空版：写成功，x = %d\n", x);
    }
    printf("判空版：拿一个空指针来试，返回 %d（程序没崩）\n", write_checked(NULL, 5));

    /* 下面这一行是本例子要演示的错：没有任何判空 */
    write_unchecked(NULL, 5);
    printf("这行永远不会被打印\n");
    return 0;
}
```

8 先判空再写的版本
11 拿空指针试它，它拒绝
14 不判空的版本，这一行会崩

两个函数都在模块里，差别就是那个 `if`：

c examples/ch04/null_deref/src/null_deref.c 第 5–15 行

```c
int write_checked(int *p, int value) {
    if (p == NULL) {
        return -1;                      /* 判空是这个接口的一部分 */
    }
    *p = value;
    return 0;
}

void write_unchecked(int *p, int value) {
    *p = value;
}
```

6 先看它是不是空的
7 空的就拒绝，不动内存
9 这里才是解引用
13 这个版本没有任何检查

跑它。这次没有常规的输出了——**程序在写下去的那一刻就死了**：

终端输出 终端输出 null-deref-out

```
（无输出）
```

终端会话 同一条命令的真实现象：stdout 一个字节都没有 ch04-null-deref-term

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/null_deref.c -o null_deref.exe
$ ./null_deref.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
退出码 139
$ ./null_deref.exe | wc -c
0
$ rm -f null_deref.exe
```

这段会话里有两个数字：`退出码 139` 是 **bash 的口径**（bash 把「被信号终止」写成 `128 + 信号号`，11 号信号就是段错误），`wc -c` 数出来的 `0` 是 **stdout 的字节数**。Windows 口径的那个数是 3221225477，写在清单里、由流水线逐次核对——同一处终止，两种写法，别以为看到了两个不同的崩溃。

实验记录 · **本机实测：这个例子的四个数字**

- 工具链：gcc 16.1.0（本书统一 `-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 编译命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/null_deref.c -o null_deref.exe`，编译退出码 **0**（一条警告都没有）；
- 运行结果：**stdout 一个字节都没有**，退出码 **3221225477**（也就是 `0xC0000005`，Windows 的「访问冲突」；在 MSYS 的 bash 里，你会看到它显示成 `139`，那是「128 + 信号 11」的写法，同一个终止的另一种口径）；
- 清单 `examples/ch04/null_deref/null_deref.json` 里 `"mode": "runtime_error"`、`"expect": { "exit": 3221225477 }`——这个数字不是我写上去的，是流水线真跑出来、再由我写进清单的。

为什么 stdout 是空的？因为 `printf` 的输出先攒在**缓冲区**里，程序被操作系统终止的那一刻，缓冲区里的东西根本没机会写出去。**「崩溃前明明打印过」而输出却没有**——这件事你在排错时会遇到无数次，第 18 单元（调试）专门讲怎么用 `fflush` 与 stderr 绕过它。

陷阱 · **崩溃现场「一行输出都没有」，你会把怀疑方向搞反**

你会看到什么：程序里明明有好几条 `printf`，崩了之后终端上什么都没有，于是你怀疑「是不是 `printf` 之前的代码就崩了」。

真相：输出没出来，很可能只是**缓冲区没来得及刷**。判断方法只有一条——看**退出码**：3221225477 是「访问冲突」，说明程序死在某一次非法访存上，与它之前打印过什么无关。

修法：想在崩溃前留下痕迹，要么显式 `fflush(stdout)`（本书 `double_free.c` 那个探针就是这么做的），要么写 `stderr`（标准 7.21.3 节只说标准错误流「初始**不是全缓冲**」，没说它必须无缓冲；本机实测它能把崩溃前写的内容留住）。别靠猜。

标准原文 · **标准原文：空指针与解引用**

两段原文，出自同一个标准的两个地方。

6.3.2.3 节「Pointers」第 3 段：

> An integer constant expression with the value 0, or such an expression cast to type void *, is called a null pointer constant.

大意：**值为 0 的整数常量表达式**（比如直接写 `0`），或者它转成 `void *` 之后的写法，叫**空指针常量**。`NULL` 就是标准库里替这个值起的一个名字（它住在 `stddef.h` 里）。

6.5.3.2 节「Address and indirection operators」第 4 段：

> If an invalid value has been assigned to the pointer, the behavior of the unary * operator is undefined.

大意：**如果一个指针里装的是无效的值，对它用 `*` 就是未定义行为**。空指针就是最典型的「无效值」。

出处：ISO/IEC 9899:2018（C17）§6.3.2.3、§6.5.3.2。

工程惯例 · **工程惯例：判空到底该谁做**

标准只规定「解引用空指针是 UB」，它**从不说谁该判空**。这件事是接口设计问题，主流惯例是：**判空写在「知道该怎么处理」的那一层。**

- 库函数的一般做法：**接受空指针，并把它定义成一个有意义的结果。**本例子里的 `write_checked(NULL, 5)` 返回 `-1`，意思就是「我拒绝写」——这个 `-1` 是接口契约的一部分，写在 `include/null_deref.h` 的注释里（第 4 行）。
- 调用者的一般做法：**如果这个指针是你自己从 `malloc` 拿的，你要判**（`malloc` 失败会返回空指针）；如果它是别人交给你、且契约上说「不会是空」的，那就不必每次都判——但那时**文档与命名要写清楚。**

代价要说清：每多一次判空，就多一次分支（性能上几乎为零，可读性上是实打实的）；而**不判**的代价是「一次空指针解引用，进程直接没了」。这就是为什么很多团队的规矩是「对外接口一律判，内部控制流内部的指针不判」。

⚠ **陷阱** · **「我判过空了」——判过一次并不等于安全**

你会怎么被坑到：你在函数开头老老实实写了 `if (p == NULL) return -1;`，之后整段代码就再也没看它。程序平时好好的，有一天崩了，位置却在那个判空**后面**很远的地方。

下面三种情况，判空都拦不住：

- **指针在别处被改掉了**：`p` 是上一层的局部变量，中间调用的某个函数里有 `free(p); p = NULL;`（9.8 节那种写法）——可那只是**它自己形参的副本**，你的 `p` 还指着已经还回去的内存。9.8 节的 gdb 会话就是这件事的现场：`free` 之后 `p` 里的地址一个字没变。
- **判的是空指针，坏的是悬垂指针**：`NULL` 只表示「谁都不指」，而**悬垂指针不是空指针**——它有一个看起来完全正常的地址，`p != NULL` 成立，解引用照样是 UB。判空筛掉的是「从来没指过」的那一类，筛不掉「指过、但对象已经没了」的那一类。
- **同一块内存有多条路**：两个指针指向同一块 `malloc` 出来的内存（别名），其中一个还回去了，另一个**没有任何办法知道。**C 里没有「这个指针还有几个主人」的记账——所以惯例是把所有权写进接口（谁分配谁释放），而不是指望判空兜底。

真相：判空只回答「这个指针里是不是那个约定的空值」，它回答不了「它指的东西还在不在」。想覆盖后者，只能靠**所有权纪律**（9.8 节的 visibility 图）与**工具**（本机的 `-fanalyzer` 会报 `-Wuse-after-free` 那一族警告，9.9 节见过）。

9.6 指针运算按元素走：`p + 1` 是几步

指针能做的算术只有三种：加一个整数、减一个整数、两个指针相减。三种都**按元素**走，不按字节走。这句话用一个真程序就能钉住：

c examples/ch04/ptr_arith/src/main.c 第 5–18 行

```c
int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;                         /* 数组名退化成指向首元素的指针 */ 数组名退化成指针
    int *last = a + 4; 最后一个元素的地址；它的「下一格」是 a + 5

    printf("sizeof(a) 整个数组 = %zu, sizeof(a[0]) 单个元素 = %zu\n", sizeof a, sizeof a[0]);
    printf("在函数里 sizeof(参数) = %zu (那是指针的大小) \n", sizeof_in_function(a));
    printf("p = a 之后，p 与 &a[0] 是同一个地址吗：%s\n", (p == &a[0]) ? "是" : "不是");

    printf("p + 1 比 p 前进 %td 个字节\n", (char *)(p + 1) - (char *)p); p + 1 前进几个字节
    printf("a + 4 比 a 前进 %td 个字节\n", (char *)(a + 4) - (char *)a);
    printf("last - p = %td (隔了几个元素，不是几个字节) \n", last - p); 两个指针相减，数的是元素
    printf("a[2] 与 *(a + 2) 的差：%d\n", diff_of_two_forms(a, 2));
    print_ints(a, 5);
```

7 数组名退化成指针
8 最后一个元素的地址；它的「下一格」是 a + 5
14 p + 1 前进几个字节
16 两个指针相减，数的是元素

三个函数住在模块里——其中 `sizeof_in_function` 就是第 8 单元留的那个坑（数组名当参数会「退化」成指针）：

c examples/ch04/ptr_arith/src/ptr_arith.c 第 5–17 行

```c
size_t sizeof_in_function(const int *a) {
    return sizeof a;
}

void print_ints(const int *a, size_t n) {
    for (size_t i = 0; i < n; i++) {
        printf("  a[%zu] = %d\n", i, a[i]);
    }
}

int diff_of_two_forms(const int *a, size_t i) {
    return a[i] - *(a + i);
}
```

5 参数里只有一个指针，没有元素个数
6 函数里 sizeof 得到 8，那是指针
9 所以要另给一个 n
16 a[i] 与 *(a + i) 是同一件事

真跑出来：

终端输出 终端输出 ptr-arith-out

```
sizeof(a) 整个数组 = 20, sizeof(a[0]) 单个元素 = 4
在函数里 sizeof(参数) = 8 (那是指针的大小)
p = a 之后，p 与 &a[0] 是同一个地址吗：是
p + 1 比 p 前进 4 个字节
a + 4 比 a 前进 16 个字节
last - p = 4 (隔了几个元素，不是几个字节)
a[2] 与 *(a + 2) 的差：0
  a[0] = 10
  a[1] = 20
  a[2] = 30
  a[3] = 40
  a[4] = 50
```

这份输出里有五件事，每一件都能单独回答一个新手问题：

1. **`sizeof(a) = 20`、`sizeof(a[0]) = 4`**：数组是**一整块**，`sizeof` 数的是整块（5 个 `int`，20 字节）。第 8 单元讲过的 `sizeof(a)/sizeof(a[0])` 就是这么数出 5 个元素的。
2. **`在函数里 sizeof(参数) = 8`**：数组名一进参数就**退化**成指针，`sizeof` 量到的只有那张地址纸条。所以「函数里必须同时传长度」不是风格问题，是**唯一的办法**（看模块第 9 行：参数里写着 `const int *a, size_t n`）。
3. **`p + 1 比 p 前进 4 个字节`、`a + 4 比 a 前进 16 个字节`**：一次前进 `sizeof(*p)` 个字节。`p` 是 `int *`，所以一步 4 字节；四步就是 16 字节。
4. **`last - p = 4`**：两个**同一数组里**的指针相减，得到的是「隔了几个元素」，不是字节数。想要字节数，得先把两个指针都转成 `char *` 再减（上面那两行就是这么干的）。
5. **`a[2] 与 *(a + 2) 的差：0`**：`a[i]` 与 `*(a + i)` **完全是一回事**——不是「差不多」，是标准规定的同一个表达式。

补一句写法上的实事：两个指针相减的结果类型是 `ptrdiff_t`（本机是有符号 64 位），所以本书一律用 `%td` 打印它；而「前进几个字节」这种数我们是用 `(char *)` 转换之后再减得到的——**指针相减只在「同一个数组里」定义良好**（标准 6.5.6 节第 9 段），隔了两块内存的指针相减，得到的数没有意义（也常常是负数）。

🕮 **标准原文** · **标准原文：`a[i]` 到底是什么**

C 语言标准 6.5.2.1 节「Array subscripting」第 2 段：

> The definition of the subscript operator `[]` is that `E1[E2]` is identical to `(*((E1)+(E2)))`.

大意：**`E1[E2]` 与 `*((E1)+(E2))` 是同一个东西**（`identical`，一字不差）。所以 `a[2]` 就是 `*(a + 2)`，而 `2[a]` 也是合法的——它会先被当成 `*(2 + a)`，加法的两个操作数可以换位置，于是结果一样（能跑，但没人这么写）。

同一份标准的 6.5.6 节「Additive operators」第 8 段，规定了指针加减的**合法范围**：

> If both the pointer operand and the result point to elements of the same array object, or one past the last element of the array object, the evaluation shall not produce an overflow; otherwise, the behavior is undefined.

大意：**只有结果仍落在同一个数组里（或者正好落在最后一个元素的「下一格」）时，指针加减才是定义良好的**；出到外面就是未定义行为。

出处：ISO/IEC 9899:2018（C17）§6.5.2.1、§6.5.6。

「下一格」（one past the end）是标准给的一个**特例**：`a + 5` 合法，但 `a[5]`（解引用它）不合法。这个「可以算，不许读」的边界，正是 `for (p = a; p != a + 5; p++)` 这类循环能成立的原因。

⚠ **陷阱** · **「越界指针」比越界下标更隐蔽**

你会怎么被坑到：把地址算出去，本身就已经是未定义行为——**不需要解引用**。看 `examples/ch04/ptr_offset_trapv/src/main.c` 第 11 行：

那个例子里，`offset` 是累加出来的一个 `int`（溢出了，是负数），第 11 行把它加到一个 `char` 数组的首地址上。这一行**只是算了一下指针**，没有读也没有写，程序照样跑到最后一行才结束——但这已经是 UB 了。

真相：指针运算的合法性只由标准那一句话管：结果必须还落在同一个数组里（含「下一格」）。算出去之后再回到数组里，中间那一段就是 UB，编译器**有权**假设永远不会发生（第 19 单元会看到它怎么利用这个假设）。

我会怎么误判：我以前写过「先用一个越界指针算个地址、检查它是否落在范围内、不行就退回去」的代码，心里想的是「我又没解引用，能有什么问题」。真相是：检查那一步就已经在 UB 的地界上了；正确的写法是**用整数算**（必要时用 `uintptr_t` 装地址），最后才转换回指针。

9.7 生命周期：栈上的对象与静态区的对象

上一节那个坑的另一半是「谁活着」。指针最容易出错的用法，就是**交出一个已经死掉的对象的地址**。真程序：

c examples/ch04/dangling_stack/src/main.c 第 5–14 行

```c
int main(void) {
    char *bad = greeting_bad(); 拿坏版本的指针
    const char *ok = greeting_ok(); 拿好版本的指针

    printf("坏版本返回的指针是空的吗：%s\n", (bad == NULL) ? "是" : "不是");
    printf("坏版本读出来：[%s]\n", bad);
    use_stack_again(); 把栈再用一遍
    printf("换一个函数把栈用了一遍之后，再读坏版本：[%s]\n", bad); 再读一次坏版本
    printf("好版本读出来：[%s]\n", ok);
    return 0;
```

6 拿坏版本的指针
7 拿好版本的指针
11 把栈再用一遍
12 再读一次坏版本

两个版本的区别只有一个词——`static`：

c examples/ch04/dangling_stack/src/dangling_stack.c 第 5–22 行

```c
char *greeting_bad(void) {
    char buf[16] = "hello"; 一块局部数组，住在栈上
    return buf;                         /* 函数一返回，buf 的生命就到头了 */ 把它的地址交出去
}

const char *greeting_ok(void) {
    static char buf[16] = "hello";      /* static：住在静态区，一直在 */ 同一个函数，加一个 static
    return buf;
}

void use_stack_again(void) {
    volatile char other[16];
    for (int i = 0; i < 15; i++) {
        other[i] = 'X';
    }
    other[15] = '\0';
    printf("use_stack_again 用了自己的一块栈内存（第一个字节是 %c）\n", (char)other[0]);
}
```

6 一块局部数组，住在栈上
7 把它的地址交出去
11 同一个函数，加一个 static

跑出来是这样：

终端输出 终端输出 dangling-stack-out

```
坏版本返回的指针是空的吗：是
坏版本读出来：[(null)]
use_stack_again 用了自己的一块栈内存（第一个字节是 X）
换一个函数把栈用了一遍之后，再读坏版本：[(null)]
好版本读出来：[hello]
```

上面这份输出里有两个反直觉的地方：坏版本**没有崩**，而且它读出来的既不是 `hello`、也不是什么乱码，而是 `(null)`。要看清为什么，先把编译时的反应与跑起来的样子摆在一起——编译器在编译的那一刻就已经在说话了：

终端会话 编译警告 + 真跑：坏版本读出来是 (null) ch04-dangling

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/dangling_stack.c -o dangling.exe
src/dangling_stack.c: In function 'greeting_bad':
src/dangling_stack.c:7:12: warning: function returns address of local variable [-Wreturn-local-addr]
    7 |     return buf;                         /* 函数一返回，buf 的生命就到头了 */
      |            ^~~
$ ./dangling.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
坏版本返回的指针是空的吗：是
坏版本读出来：[(null)]
use_stack_again 用了自己的一块栈内存（第一个字节是 X）
换一个函数把栈用了一遍之后，再读坏版本：[(null)]
好版本读出来：[hello]
退出码 0
$ rm -f dangling.exe
```

这段会话里有本单元最反直觉的一幕：**你猜坏版本会读到垃圾，实测读到的是 `(null)`**。为什么？因为这次编译器不只警告，它还**看穿了** `return buf;` 这件事，直接把返回值换成了空指针。证据在汇编里：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -S src/dangling_stack.c -o dangling.s` 之后，`greeting_bad` 的第 7 行对应的是 `movl $0, %eax`——**返回 0**。所以 `bad == NULL` 为真，`%s` 打出的 `(null)` 是本机运行库对空指针的照顾（标准没要求它这么做）。

调用树 函数返回时，栈帧连同里面的东西一起消失

第 1 步 main 调 greeting_bad，栈上多了一帧

```
main() ：栈上的一帧，里面住着 bad 与 ok
greeting_bad() ：栈上又多一帧，里面住着 buf[16]
greeting_bad() 的 buf[16] ："hello" 就写在这里
main() → greeting_bad() 调用
```

第 2 步 greeting_bad 把 buf 的地址交出去，然后返回

```
greeting_bad() ：准备返回，它这一帧马上被收回
main() ：收到一个地址（指向已经不属于你的那一帧）
main() → greeting_bad() 返回之后，这条调用链只剩 main
```

第 3 步 main 再调用 use_stack_again，栈上那块地方被重新使用

```
main() ：还在原地
use_stack_again() ：它的局部数组很可能就落在刚才 buf 的位置上
main() → use_stack_again() 同一块栈空间，换了主人
```

第 1 帧是调用发生时的样子：栈上多出一帧，`buf` 住在那里面。第 2 帧是返回的那一刻：**帧被收回**，`buf` 这块内存不再属于你——而那个地址（一张纸条）还在 `main` 手里。第 3 帧说明为什么「栈上的悬垂指针」特别危险：下一个函数调用会**复用同一块栈空间**，你手里的地址指向的内容会变（本机的实测是「编译器直接不给地址、给空指针」，但那是这一次的实现选择，不是规律）。

标准原文 · **标准原文：什么是「生命期」**

> C 语言标准 6.2.4 节「Storage durations of objects」第 2 段：
>
> > The lifetime of an object is the portion of program execution during which storage is guaranteed to be reserved for it. An object exists, has a constant address, and retains its last-stored value throughout its lifetime. … The value of a pointer becomes indeterminate when the object it points to (or just past) reaches the end of its lifetime.
>
> 大意：**对象的「生命期」= 保证给它留着内存的那段时间**；在这段时间里它一直存在、地址不变、值不变。最后那句更要紧：**当它指向的对象（或对象之后的「下一格」）生命期结束时，这个指针的值就变成不确定的（indeterminate）**——也就是说，这时候连「把那个地址打印出来」或「拿它比较」都不是标准保证的事。
>
> 出处：ISO/IEC 9899:2018（C17）§6.2.4 第 2 段。

大白话 · **局部变量、static 变量、malloc 出来的，各活多久**

- **局部变量**（`char buf[16]`，没加 `static`）：生命期是**它所在的那次函数调用**。函数一返回，内存就被收回——它住在**栈**上，栈是「函数调用专用的临时区」。所以把它的地址交出去，等于交给别人一张写着「我马上要搬走」的地址。
- `static` **局部变量**（`static char buf[16]`）：名字还只在这个函数里看得见，但内存**从程序开始到结束一直留着**——它住在**静态区**。所以交它的地址是安全的。本机实测（见下面那张图与会话）：静态区里的对象地址在 `0x00007FF6…` 这一带，栈上的局部变量在 `0x00000000005FFE…` 这一带，**差了十万八千里**——因为它们压根不在一个区里。
- `malloc` **出来的**：生命期**由你决定**——从 `malloc` 成功那一刻开始，到你 `free` 为止。它住在**堆**上。9.8 节讲它。

这三种生命期在标准里的名字分别是 **automatic**（自动存储期，栈上那些）、**static**（静态存储期）和 **allocated**（分配存储期，`malloc` 那些）。名字记不住没关系，记住「**谁负责收回**」就够了：前两种是**自动**收回的（编译器与程序启动/结束负责），只有第三种要**你亲手** `free`。

三种存储期摆成一张表，这一节的两个坑就都能落位：

存储期	哪些东西	谁负责收回	把地址交出去安全吗
automatic（自动）	没加 `static` 的局部变量、函数参数	函数返回时自动收回	**不安全**：`greeting_bad` 就是它
static（静态）	全局变量、加了 `static` 的局部变量	程序结束时	安全，但两次调用共用同一块内存
allocated（分配）	`malloc` 出来的	**你**：`free`	安全，直到你 `free` 为止（9.8 节）

注意第一列里「局部变量」出现了两次——**同一个词，两种生命期，区别就是那个 `static`**。这也是标准为什么要用「存储期」这个词：它讲的是「这块内存归谁管、管到什么时候」，与「这个名字在哪个作用域里看得见」（那叫**作用域**）是两件事。

内存形态 **内存的三个区：静态区 / 栈 / 堆**

第 1 步 静态区 —— 程序一开始就在，程序结束才走

全局变量 int g_data ：住在静态区

static 局部变量 ：也住在静态区

静态区的地址（本机实测）：0x00007FF762133000 这一带

第 2 步 栈 —— 函数调用期间才存在的区域

main() 的局部变量 local ：地址 0x00000000005FFE94

greeting_bad() 的 buf ：函数一返回就不存在了

栈往哪边长（本机实测）：一次次往里调用，局部变量 0x…5FFE5C → 0x…5FFE1C → 0x…5FFDDC，递减

第 3 步 堆 —— malloc 要来的区域，地址由分配器给

malloc(4) 返回的地址 ：本机实测每次运行都不一样

连着 malloc 四次（本机实测）：0x…7E1B60 → 0x…7E1BE0 → 0x…7E1B80 → 0x…7E1C00（不单调）

free 之后 ：这块内存回到分配器手里，指针本身不变

第 4 步 三个区摆在一起看

说明：「谁在低地址、谁在高地址」是本机实现的选择，标准一个字都没规定

静态区 ：0x00007FF6… 这一带（本机）

堆 ：0x00000000007xxxxx 这一带

栈 ：0x00000000005FFE… 这一带

上面第 4 帧里的地址是本机真跑出来的（见下面会话），但**「谁在低地址、谁在高地址」不是标准规定**，是实现选择：本机把静态区放在高地址、栈放在低地址附近。

有一句流传很广的话：「栈向下长、堆向上长」。这两半我都量过：**栈向下在本机量得出来**（下面会话第 6 到 9 行：一次次往里调用，局部变量地址 `0x…5FFE5C` → `0x…5FFE1C` → `0x…5FFDDC`，一路递减）；**而「堆向上长」我用 `malloc` 量不出来**——连着四次 `malloc` 的地址是 `0x…7E1B60` → `0x…7E1BE0` → `0x…7E1B80` → `0x…7E1C00`，**不单调**（第二次运行又是另一串不单调的值）。所以在这一节里，我只把它当「常见实现的说法」，**不当结论**；真要依赖地址之间的相对位置，就是**不可移植**的写法。

终端会话 三个区的地址真跑（第一次）：栈逐层递减，堆不单调 ch04-regions

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g mem_regions.c -o mem_regions.exe
$ ./mem_regions.exe
静态区：&g_data = 00007FF762133000
栈    ：&local  = 00000000005FFE94
堆    ：heap    = 00000000007DEAA0
嵌套调用里的局部变量（看栈往哪边长）：
  第 1 层调用的局部变量：00000000005FFE5C
  第 2 层调用的局部变量：00000000005FFE1C
  第 3 层调用的局部变量：00000000005FFDDC
连着 malloc 四次（看堆的地址是不是单调的）：
  第 1 次：00000000007E1B60
  第 2 次：00000000007E1BE0
  第 3 次：00000000007E1B80
  第 4 次：00000000007E1C00
```

终端会话 （续）再跑一次：静态区与栈一模一样，堆又换了一批地址 ch04-regions-2

```
$ ./mem_regions.exe
静态区：&g_data = 00007FF762133000
栈    : &local  = 00000000005FFE94
堆    : heap    = 0000000000074EAA0
嵌套调用里的局部变量（看栈往哪边长）：
  第 1 层调用的局部变量：00000000005FFE5C
  第 2 层调用的局部变量：00000000005FFE1C
  第 3 层调用的局部变量：00000000005FFDDC
连着 malloc 四次（看堆的地址是不是单调的）：
  第 1 次：0000000000751B20
  第 2 次：0000000000751D20
  第 3 次：0000000000751B60
  第 4 次：0000000000751D40
$ rm -f mem_regions.exe
```

这两段会话有四组数字，一组一组对：

- **静态区与栈**：两次运行**一模一样**（`0x00007FF762133000`、`0x00000000005FFE94`）；同一个进程里一层层往里调用时，栈上的局部变量一路**递减**（`0x…5FFE5C` → `0x…5FFE1C` → `0x…5FFDDC`）——这是「栈向下长」在本机的实证；
- **堆**：不只两次运行不一样（`0x…7DEAA0` 对 `0x…74EAA0`），连**同一次运行里**四次 `malloc` 出来的地址都不单调（`0x…7E1B60` → `0x…7E1BE0` → `0x…7E1B80` → `0x…7E1C00`）。

再补一条我自己踩出来的事实：**静态区的地址也不保证每次相同。**上面会话里那个 `0x00007FF762133000` 是**那一次**的值；本书写这一节时，同一个 `dangling_stack` 程序在两条不同的命令下量到过 `0x00007FF74E463000` 与 `0x00007FF6600C3000` 两个不同的静态区地址——连「程序加载到哪儿」都是运行期定的。这两件事的推论是同一条：**地址本身不是可以当证据的东西。**所以那个探针程序 `mem_regions.c` 故意没有配清单，它的证据形式是这段会话，而不是「逐字对账」的输出块（输出块要求每次运行都逐字相同）。

工程惯例 · **工程惯例：`static` 是权宜之计，不是标准解法**

把局部数组改成 `static`（或者退回一个全局数组）确实能让地址活下来，很多标准库函数也这么干（比如有些老接口返回一个「内部缓冲区」的地址）。但代价写在这里：

- **两次调用会共用同一块内存**：`greeting_ok()` 返回的地址永远相同，调用两次拿到的字符串会被后一次覆盖；
- **不能重入**：多线程或多层嵌套调用时，两个调用者会互相踩（第 15 单元讲字符串库函数时会再遇到这个坑）；
- **生命周期失控**：那块内存一直占着，程序跑多久它就留多久。

更常见的做法是**让调用者出内存**（`void greet(char *out, size_t n)`），或者**动态分配并约定谁释放**（9.8 节）：分配的责任与释放的责任写进接口，而不是藏在函数里面。

9.8 堆：`malloc` / `free`、悬垂状态与 double free

堆上的内存有一个和栈完全不同的特点：**它不会自己消失。**你 `malloc` 多少，它就占着多少，直到你 `free`。所以「谁分配、谁释放」必须在接口上写清楚：

谁能看见谁 **谁分配谁释放：malloc 与 free 的配对**

第 1 步 main 向堆要一块内存

main() ：需要一块放 int 的地方 | malloc(4) ：分配器从堆里切一块给它

malloc(4) → main() 返回这块内存的地址

第 2 步 用完还回去，接口上写清「谁负责还」

main() ：用完之后 free(p) | 分配器 ：可以再把它分给别人

free(p) → 分配器 这块内存回到分配器手里

第 3 步 还回去之后，指针本身一点没变

main() 手里的 p ：还是那个地址（free 不会把它置空） | 那块内存 ：可能已经住着别人了

第 1 帧是配对的开始：`malloc` 给出地址。第 2 帧是配对的结束：**由拿到地址那一层负责还。**第 3 帧是新手最常搞错的一件事：`free(p)` 只是把内存还回去，**它不会把 `p` 变成空指针。**真程序把这件事一分为二：

c examples/ch04/heap_after_free/src/main.c 第 6–19 行

```c
int main(void) {
    int *p = (int *)malloc(sizeof(int));   /* 这块内存住在堆上 */ 向堆要一块内存
    if (p == NULL) {
        return 1;
    }

    *p = 42;
    printf("释放前 *p = %d\n", *p);

    free(p);                               /* 还回去了 */ 还回去
    *p = 7;                                /* 错的：又往还回去的内存里写 */ 错的：又往还回去的内存里写
    say_done();                            /* 到这一步，程序看起来一切正常 */ 到这一步程序看起来一切正常

    return 0;
}
```

7 向堆要一块内存
15 还回去
16 错的：又往还回去的内存里写
17 到这一步程序看起来一切正常

这个例子的模块只有一行函数（`say_done`），头文件里却写了长长一段注释——因为**「为什么只有一行」本身就是这一节的知识点**：

c examples/ch04/heap_after_free/include/heap_after_free.h 第 4–13 行

```c
/*
 * 这个例子的模块故意做得极薄：要演示的两句（free 与它之后那次写）必须待在 这两句必须在同一个编译单元里
 * 同一个编译单元里，编译器才看得见它们，也才可能在 -O2 下把那句写删掉。 否则优化器看不见它们
 *
 * 惯例（本例故意违反）：谁分配谁释放 —— malloc 出来的内存，由知道「它能用多久」 惯例：谁分配谁释放
 * 的那个人 free，并且在 free 之后不再碰那个指针。 而且 free 之后不再碰它
 */

/* 打印一行收尾说明 */
void say_done(void);
```

5 这两句必须在同一个编译单元里
6 否则优化器看不见它们
8 惯例：谁分配谁释放
9 而且 free 之后不再碰它

跑它。这条会话的看点不是「打印了什么」，而是**退出码**：

终端会话 基础参数（-O0）：要打印的都打印了，退出码却是 127 ch04-uaf-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/heap_after_free.c -o uaf_o0.exe
src/main.c: In function 'main':
src/main.c:16:8: warning: pointer 'p' used after 'free' [-Wuse-after-free]
   16 |     *p = 7;                                /* 错的：又往还回去的内存里写 */
      |     ~~~^~~
src/main.c:15:5: note: call to 'free' here
   15 |     free(p);                               /* 还回去了 */
      |     ^~~~~~
$ ./uaf_o0.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
释放前 *p = 42
写完了，程序先若无其事地走完
退出码 127
```

终端会话 （续）同一份源码换成 -O2：一样的两行输出，退出码却是 0 ch04-uaf-2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/heap_after_free.c -o uaf_o2.exe
src/main.c: In function 'main':
src/main.c:16:8: warning: pointer 'p' used after 'free' [-Wuse-after-free]
   16 |     *p = 7;                                /* 错的：又往还回去的内存里写 */
      |     ~~~^~~
src/main.c:15:5: note: call to 'free' here
   15 |     free(p);                               /* 还回去了 */
      |     ^~~~~~~
$ ./uaf_o2.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
释放前 *p = 42
写完了，程序先若无其事地走完
退出码 0
$ rm -f uaf_o0.exe uaf_o2.exe
```

两段会话摆在一起，问题就来了：**同一份源码、同样的两行输出，一个退出码是 127、一个是 0**。第一个 127 是 bash 的口径，Windows 口径是 **3221226356**（`0xC0000374`）——写坏了分配器的账本，分配器随后发现了。而 `-O2` 下它变成了 0——因为那句写**根本没被执行**，9.9 节会用汇编证明这件事。

但「退出码具体是多少」这件事本身也不牢靠。我把 `-O0` 那份二进制逐字节复制成几个不同名字，在同一台机器上量了一轮（同一个程序、同一次会话，各跑 3 次，读数都稳定）：

启动方式	`A0.exe`	`B.exe`	名字 30 个字符的那个	闸门实际用的名字
相对路径（`./名字`）	3221225477	3221226356	3221226356	—
绝对路径	3221226356	3221226356	3221225477	3221226356（每次都是）

同一份机器码，只因为**启动时那串路径不同**（相对还是绝对、名字有多长），就会在两个终止码之间跳：**3221226356**（`0xC0000374`，分配器在进程退出时发现账本被写坏）与 **3221225477**（`0xC0000005`，当场非法访问）。这不是规律，只是「那串路径影响了进程启动时的内存排布」。而 `-O2` 那一栏无论怎么启动都是 **0**——因为那句写根本没被执行，不是运气。

所以：**同一个错，连「怎么死的」都取决于怎么启动它。**这正是 UB 的含义——不是「一定会崩」，而是「谁都不欠你一个确定的结果」。清单里的 opt-diff 只声明「两档退出码不同」；闸门固定把可执行文件编译到 `dist/work/<id>/` 下、用绝对路径启动，所以「3221226356 对 0」这个对照在闸门里是可复现的——**它证明的是「两档有可观察的差异」，不是「这个错一定给出 3221226356」。**

调试实录 · **调试实录：`free` 之后，指针里的地址一点没变**

「`free` 会不会顺手把 `p` 置空」这个问题不需要猜，gdb 看一眼就知道。下面这段是 `examples/ch04/session-gdb-pointers.txt` 的后半段，调试的正是上一小节那个例子（编译命令与上面 `-O0` 那一行一模一样）。

会话里做了四件事：在第 15 行（`free(p);`）上打断点；停住时 `p p` 记下指针的值；`next` 让 `free` 真的执行完；再 `p p`、`p *p`、`x/4xb p` 看指针与那块内存现在是什么。下面这块是会话原文，命令与输出都在里面。

终端会话 gdb：free 之前与之后，p 里的地址是同一个 ch04-gdb-2

```
$ gdb --batch -ex "break src/main.c:15" -ex run -ex "p p" -ex next -ex "p p" -ex "p *p" -ex "x/4xb p"
./uaf.exe
Breakpoint 1 at 0x1400014f7: file src/main.c, line 15.
[New Thread 41048.0x137a0]
Thread 1 hit Breakpoint 1, main () at src/main.c:15
15          free(p);                               /* 还回去了 */
$1 = (int *) 0x71cdc0
16          *p = 7;                                /* 错的：又往还回去的内存里写 */
$2 = (int *) 0x71cdc0
$3 = 7446016
0x71cdc0:       0x00    0x9e    0x71    0x00
$ rm -f uaf.exe
```

`$1` 与 `$2` 都是 `(int *) 0x71cdc0`——`free` **没动过** `p`，`p` 手里还是那张写着 `0x71cdc0` 的纸条。而那块内存已经不是你的了：`$3 = 7446016`（也就是 `0x719e00`）与那四个字节 `00 9e 71 00` 一起告诉你，分配器把它的账本写在了这里。**地址还在、对象没了**——这就是**悬垂指针**（dangling pointer），对它读或写都叫 **use-after-free**。

第 2 帧是本节的中心图：**指针与对象是两件事**，`free` 只处理了对象，没处理指针。第 3 帧是它的后果。第 4 帧是惯例上的补救：还回去之后**立刻把指针置空**（`free(p); p = NULL;`），这样下一次误用会当场崩掉、而不是悄悄写坏别人的数据——代价是**它只能管住这一个指针**：如果同一块内存还有别的指针（别名），那些指针照旧悬着。

> ⚠ **陷阱** · **double free：同一块内存还两次，程序当场终止**
>
> **你会怎么被坑到**：两条 `free(p)` 摆在不同的分支里（比如「提前返回」的那条路上也 `free` 了一次），你以为只会走一条路。程序平时跑得好好的，某个输入下突然整个进程没了。
>
> 本机的真实现象（探针程序 `examples/ch04/double_free.c`，同样没有清单，证据是会话 `session-double-free-pointers.txt`）：它先把两行输出打了出来（因为那里显式 `fflush(stdout)`），然后**被终止**——退出码 **3221226356**（`0xC0000374`，同一个「堆损坏」代码），stderr 一个字节都没有。`fflush` 那一行不是装饰：不加它，你会连那两行输出都看不到，还会误以为「程序在第一次 `free` 之前就崩了」。
>
> **真相**：分配器在自己的账本里记着「这块内存是空闲的」，第二次 `free` 等于把同一块内存还两次，账本直接乱掉，本机的运行库选择**立刻终止进程**（这是运行库的照顾，不是标准的要求；标准只说「UB」，见下面）。**注意这里没有报错文本**：终止时 stderr 是空的，唯一的线索就是那个退出码。

上面这段「本机实测」不是我说了算的，探针程序把两次 `free` 的现场留了下来（会话文件 `examples/ch04/session-double-free-pointers.txt`）：

终端会话 double free：两行输出都留下了，进程随后被终止 — ch04-double-free

```
$ ./double_free.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
第一次 free 之前 *p = 42
第一次 free 走完了
退出码 127
$ rm -f double_free.exe
```

要紧的是这两行输出**都打印出来了**：程序没有死在「第一次 `free` 之前」——它把那两行都写进了管道，然后死在**第二次 `free` 里面**。这不是猜的：探针里我在第二次 `free` 的下一行加了一句写 stderr、再写一个文件，两样都**没有出现**（`stderr` 为空、文件没被创建），说明第二次 `free` 根本没有返回。对照一下 9.8 节那个 `heap_after_free`：它的 `-O0` 版本在 `main` 最后一行写 stderr 是**能打出来**的，那个才是「死在进程退出时」。**这两处别混**：同一族错误，一个死在调用里、一个死在退出时。

这次会话里 bash 给的是 `127`，Windows 口径是 **3221226356**（`0xC0000374`）；如果你在 cmd 里跑它，同一个终止会显示成 `-1073740940`（同一个 32 位数的有符号写法）。**三个数字是同一件事**——本书清单里一律写无符号的那个（3221226356），因为闸门抓回来的就是它。

标准原文 · 标准原文：`free` 的契约

C 语言标准 7.22.3 节「Memory management functions」对 `free` 的规定：

> The `free` function causes the space pointed to by `ptr` to be deallocated, that is, made available for further allocation. If `ptr` is a null pointer, no action occurs. Otherwise, if the argument does not match a pointer earlier returned by a memory management function, or if the space has been deallocated by a call to `free` or `realloc`, the behavior is undefined.

大意：`free` 把 `ptr` 指向的空间**还回去**；`ptr` 是空指针时**什么都不做**（这就是为什么「先判空再 `free`」在某些老代码里很常见）；除此之外，如果这个指针不是分配函数返回的、或者这块空间**已经被释放过**，行为就是未定义的——标准只说 UB，后果由实现决定（本机的实现选择是：**终止进程**）。

出处：ISO/IEC 9899:2018（C17）§7.22.3 第 2 段。

9.9 UB 之下编译器有多大自由度

到现在为止，这一节里的错误程序有一个共同点：**它们都「看起来正常」**。越界写可能什么都不发生，`free` 之后再写可能照样打印，悬垂指针可能给你一个空指针——那编译器到底把这些代码变成了什么？看汇编。

两份文件都在例子目录里：`main-O0.s` 是基础参数（`-O0`）编出来的，`main-O2.s` 是 `-O2` 编出来的，源码都是 `heap_after_free/src/main.c`。先看 `-O0`：`free(p);`（源码第 15 行）与它之后那句写（第 16 行）**都在**。

asm examples/ch04/heap_after_free/main-O0.s 第 40–58 行

```
40  .L2:
41      .loc 1 12 8
42      movq    -8(%rbp), %rax
43      movl    $42, (%rax)
44      .loc 1 13 5
45      movq    -8(%rbp), %rax
46      movl    (%rax), %eax
47      leaq    .LC0(%rip), %rcx
48      movl    %eax, %edx
49      call    printf
50      .loc 1 15 5
51      movq    -8(%rbp), %rax
52      movq    %rax, %rcx
53      call    free    这一句就是 free(p)
54      .loc 1 16 8
55      movq    -8(%rbp), %rax
56      movl    $7, (%rax)    这一句是 *p = 7，写的是已经还回去的内存
57      .loc 1 17 5
58      call    say_done
```

53 这一句就是 free(p)
56 这一句是 *p = 7，写的是已经还回去的内存

再看 `-O2`：同一份源码，`malloc`、`free`、那句写**全都不见了**——被优化器整段删掉，只剩一句 `printf` 与收尾的 `say_done`。

asm examples/ch04/heap_after_free/main-O2.s 第 24–37 行

```
24      call    __main
25  .LVL0:
26      .loc 1 7 5 is_stmt 1 view .LVU2
27      .loc 1 8 5 view .LVU3
28      .loc 1 12 5 view .LVU4
29      .loc 1 13 5 view .LVU5
30      movl    $42, %edx
31      leaq    .LC0(%rip), %rcx
32      call    printf    打印 42，但 42 是优化器直接算出来的常数
33  .LVL1:
34      .loc 1 15 5 view .LVU6
```

```
35      .loc 1 16 5 view .LVU7
36      .loc 1 17 5 view .LVU8
37      call    say_done 只有收尾那句还在
```

32 打印 42，但 42 是优化器直接算出来的常数
37 只有收尾那句还在

编译器视角 · 编译器视角：它凭什么删掉那两句

优化器的推理链是这样的（每一步都合法）：

1. `*p = 42;` 之后紧跟 `printf(..., *p)`，中间没有任何东西能改 `*p`，所以 `*p` 就是 42——**直接把常数 42 传给 `printf`**，连带那句写也不再需要；
2. `free(p); *p = 7;`——按标准，`free(p)` 之后 `p` 就是**悬垂指针**，对它解引用是 UB。**UB 的意思就是：编译器可以假设它从不发生。**既然不会发生，`*p = 7` 这句写就**没有任何可观察的效果**，可以删；
3. 写被删掉之后，那次 `malloc` 也没有可观察的效果了（没有人读它），于是 `malloc` 与 `free` 一起被删；
4. 删完之后，`__main`、`printf`、`say_done` 三次调用就是整个程序——所以 `-O2` 的退出码是 0，而 `-O0` 是 3221226356。

这条链每一步都对，这就是为什么「换个优化等级行为就变了」不能怪编译器——它只是按标准给的许可在办事。这也正是本节要你记住的那句话：**UB 不是「一定会崩」，而是「谁都不欠你一个确定的结果」。**

清单 `examples/ch04/heap_after_free/heap_after_free.json` 就是这个对照的机器核对版：它写的是 `"mode": "detect"`、`"detect": "opt-diff"`、`"expect": { "differs": "exit" }`——闸门按工具链配置里的两档（`-O0` 与 `-O2`）各编一次、各跑一次，发现退出码不同才算这份证据成立（闸门这次量到的是 3221226356 对 0）。**注意它证明的是什么**：opt-diff 只能说明「同一份源码在两档下有可观察的差异」，它不能证明「差异只可能出现在这个例子上」——我也拿 `dangling_stack` 试过两档，行为一模一样（都是 `(null)`、退出码 0），所以它当不了这个样本；而 `heap_after_free` 在两档之间的差别有汇编撑腰（下一节），这是实测加证据的结果，不是挑好看的写。

那么，有没有办法**在出错之前**就抓到这类问题？本机有三样能用的手段。先看最强的那样：编译器自己的**静态分析器**。

c examples/ch04/heap_oob_fanalyzer/src/main.c 第 6–21 行

```c
int main(void) {
    int *a = (int *)malloc(4 * sizeof(int));   /* 4 个 int，一共 16 字节 */ 分配 16 个字节
    if (a == NULL) {
        return 1;
    }

    for (size_t i = 0; i < 4; i++) {           /* 合法范围是 a[0] … a[3] */ 合法范围是 a[0] 到 a[3]
        a[i] = (int)i;
    }
    a[5] = 99;                                 /* 常量下标 5：越界了 */ 常量下标 5，越界

    printf("越界写完了，程序还活着：\n");
    print_forward(a, 4);
    free(a);
    return 0;
}
```

7 分配 16 个字节
12 合法范围是 a[0] 到 a[3]
15 常量下标 5，越界

用 `-fanalyzer` 编它，编译器在**编译期**就把这件事指出来了：

终端会话 `-fanalyzer`：编译期就指出这处越界写 ch04-analyzer-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -fanalyzer -c src/main.c -o main.o
src/main.c: In function 'main':
src/main.c:15:10: warning: heap-based buffer overflow [CWE-122] [-Wanalyzer-out-of-bounds]
   15 |     a[5] = 99;                                  /* 常量下标 5：越界了 */
      |     ~~~~~^~~~
  'main': events 1-6
    7 |     int *a = (int *)malloc(4 * sizeof(int));   /* 4 个 int，一共 16 字节 */
      |                     ^~~~~~~~~~~~~~~~~~~~~~~~
      |                     |
      |                     (1) capacity: 16 bytes
    8 |     if (a == NULL) {
      |        ~
      |        |
      |        (2) following 'false' branch (when 'a' is non-NULL)... ->┐
```

终端会话 （续）越界了几个字节，它也算给你 ch04-analyzer-2

```
   15 |     a[5] = 99;                                  /* 常量下标 5：越界了 */
      |     ~~~~~~~~~
      |          |
      |          (6) ⚠️  out-of-bounds write from byte 20 till byte 23 but region ends at byte 16
src/main.c:15:10: note: write of 4 bytes to beyond the end of the region
   15 |     a[5] = 99;                                  /* 常量下标 5：越界了 */
```

这份诊断值得逐样读：`heap-based buffer overflow` 是问题的名字；`[CWE-122]` 是业界给这类缺陷统一编的号（**CWE** 是一份公开的缺陷分类表，122 号就是「堆上的缓冲区溢出」）；方括号里 `[-Wanalyzer-out-of-bounds]` 是控制这条诊断的**开关名**——与第 2 单元讲的警告开关是同一套机制，只是这一族开关归 `-fanalyzer` 打开的分析器管。中间那段 ASCII 图是它一步步推出来的过程：`(1) capacity: 16 bytes` 是「这块内存有 16 字节」，`(6) out-of-bounds write from byte 20 till byte 23` 是「写了第 20 到第 23 个字节，可这块内存到第 16 个字节就结束了」。

实验记录 · 本机实测：分析器能做与不能做的

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -fanalyzer -c src/main.c -o main.o`；
- **编译退出码是 0**。这一点非常反直觉：分析器只负责**报告**，它不阻断编译。所以「编译过了」绝不等于「代码没问题」；
- **它必须看得见形状**：这个例子里的两件事——`malloc(4 * sizeof(int))` 的**常量大小**、`a[5]` 的**常量下标**——都写在同一个函数里，分析器才推得出来。我试过把填 0 到 3 的那个循环挪进模块里的另一个函数，**它一个字都不报**；也试过换成「变量下标的循环形状」，那时它只会报一条 `possible-null-dereference`（CWE-690），与越界无关。**能报出来的是「死板但确定」的形状**；
- **阴性对照**：去掉 `-fanalyzer` 再编同一份源码，一条分析器诊断都没有——所以那句诊断确实归它管，不是基础参数给的；
- 这个例子**跑起来什么事都没有**（前四个元素照样打印、退出码 0）——这就是本节开头那句话的证据：**越界写可能什么现象都没有**。

清单 `examples/ch04/heap_oob_fanalyzer/heap_oob_fanalyzer.json` 里写的是 `"mode": "detect"`、`"detect": "fanalyzer"`，证据声明是编译期的 `stderrContains`——**没有** `expect.exit`，因为它是编译期检测，闸门不会去运行这个程序（一个已经写坏堆的程序，跑它没有意义）。

再补一个我自己量过的数：分析器**不一定很慢**——在这个 21 行的小文件上，带与不带 `-fanalyzer` 的编译耗时都是 **0.10 秒**（各跑三次，这是文件太小的缘故；工程大了它才贵）。真正要付的代价是**它的判断有误报**，所以它的输出要当「线索」看，不要当判决书：报出来的地方值得回头看一遍代码，没报的地方也不能当成安全。

分析器看的是**所有可能执行的路径**，所以它靠推理，因此既会漏（形状不够死板）也会误报。想看**真的跑起来时**发生了什么，就得靠运行期的两道防线。第一道在栈上：

c examples/ch04/stack_smash/src/main.c 第 5–18 行

```c
#ifndef N
#define N 9                                 /* 默认越界 1 个字节；用 -DN=64 试更大的 */ 默认越界 1 个字节
#endif

int main(void) {
    char buf[8]; 一块 8 字节的数组
    int scratch[8] = {0, 0, 0, 0, 0, 0, 0, 0};

    fill_pattern(buf, N); 往它里面写 N 个字节

    printf("往 buf[8] 里写了 %d 个字节，函数走到了这里\n", N);
    printf("scratch[0] = %d, scratch[7] = %d\n", scratch[0], scratch[7]);
    return 0;
}
```

6 默认越界 1 个字节
10 一块 8 字节的数组
13 往它里面写 N 个字节

写字节的那个函数在模块里，循环上界由调用者给——编译器**看不到**它有多大：

c examples/ch04/stack_smash/src/stack_smash.c 第 3–7 行

```c
void fill_pattern(char *buf, int n) {
    for (int i = 0; i < n; i++) { 上界由调用者决定
        buf[i] = 'A'; 一个字节一个字节地写
    }
}
```

4 上界由调用者决定
5 一个字节一个字节地写

终端会话 越界 1 个字节：基础构建什么都没发生，加了保护就终止 ch04-smash-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/stack_smash.c -o smash.exe
$ ./smash.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
往 buf[8] 里写了 9 个字节，函数走到了这里
scratch[0] = 0, scratch[7] = 0
退出码 0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fstack-protector-all src/main.c src/stack_smash.c -
o smash_guard.exe
$ ./smash_guard.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
*** stack smashing detected ***: terminated
退出码 127
```

终端会话 （续）越界 64 个字节：基础构建自己也崩了 ch04-smash-2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DN=64 src/main.c src/stack_smash.c -o smash64.exe
$ ./smash64.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
退出码 139
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DN=64 -fstack-protector-all src/main.c
src/stack_smash.c -o smash64_guard.exe
$ ./smash64_guard.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
*** stack smashing detected ***: terminated
退出码 127
$ rm -f smash.exe smash_guard.exe smash64.exe smash64_guard.exe
```

两段会话里加的这道保护，把「什么都没发生」变成了可观测的终止。被终止那一次，程序在 `printf` 之前就已经死了，所以 stdout 一个字节都没有——清单里那个 `output` 块里只有一行哨兵，就是这件事的机器核对版。

终端输出 终端输出 stack-smash-out

```
（无输出）
```

两段会话放在一起，是本节最想让你看见的一组对照：

越界量	基础构建（`-O0`）	加 `-fstack-protector-all`
1 个字节（`N=9`）	**什么都没发生**：输出照打，退出码 0	`*** stack smashing detected ***`，退出码 3221226505
64 个字节（`-DN=64`）	输出一行都没有，退出码 3221225477	`*** stack smashing detected ***`，退出码 3221226505

同一个程序、同一处错误，只因为**越界多少**不同，基础构建下就有了「完全正常」与「访问冲突」两种命运。

实验记录 · 本机实测：`-fstack-protector-all` 抓住了什么

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fstack-protector-all src/main.c src/stack_smash.c -o smash_guard.exe`，再 `./smash_guard.exe`；
- 加保护编译运行：stderr 是 `*** stack smashing detected ***: terminated`，退出码 **3221226505**（`0xC0000409`）；
- **阴性对照**（同一份源码去掉 `-fstack-protector-all`）：`N=9` 时退出码 **0**、stderr 为空；`-DN=64` 时退出码 **3221225477**、stderr 也为空。「stack smashing」那行字**只在加了保护时出现**，所以这份证据确实归这个开关管；
- 它保护的是**有局部数组的函数**：编译器在那块数组旁边放一个「哨兵值」，函数返回前检查它有没有被改写；被改了就立刻终止。**堆上的越界它抓不到**；
- 本机**没有** AddressSanitizer（链接会失败：`collect2.exe: error: ld returned 5`），所以「运行期精确的越界位置」这本书给不了——能给的只有崩溃退出码、这行 smashing 文本、以及 gdb 的栈（第 18 单元）。

两个清单 `stack_smash.json`（默认 `N=9`）与 `stack_smash_big.json`（`-DN=64`）都写了 `"detect": "stack-protector"` 与 `"expect": { "exit": 3221226505 }`——同一个例子、两种越界量，各自被真跑核对过。

第二道运行期防线，专治「用 `int` 算指针偏移」这种错：

c examples/ch04/ptr_offset_trapv/src/main.c 第 5–14 行

```c
int main(void) {
    char block[16];
    int offset = accumulate_offset(500000000, 5);
    printf("累加出来的 offset = %d\n", offset);
    char *p = block + offset;          /* 用这个偏移算指针（没有解引用它） */
    printf("用它算出来的指针指向 %p（这一步本身就已经越界了）\n", (void *)p);
    return 0;
}
```

7 累加 5 次，每次 5 亿
9 打印这个偏移
11 用它算指针，不解引用

c examples/ch04/ptr_offset_trapv/src/ptr_offset_trapv.c 第 3–9 行

```c
int accumulate_offset(int step, int times) {
    int offset = 0;
    for (int i = 0; i < times; i++) {
        offset += step;                /* 第 5 次就不在 int 的范围里了 */
    }
    return offset;
}
```

6 第 5 次就不在 int 的范围里了

终端会话 -ftrapv：溢出那一刻进程直接终止，连 printf 的缓冲都带走了 ch04-trapv

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/ptr_offset_trapv.c -o offset.exe
$ ./offset.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
累加出来的 offset = -1794967296
用它算出来的指针指向 FFFFFFFF9562F790（这一步本身就已经越界了）
退出码 0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -ftrapv src/main.c src/ptr_offset_trapv.c -o
offset_trap.exe
$ ./offset_trap.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
退出码 127
$ rm -f offset.exe offset_trap.exe
```

加了 `-ftrapv` 的那一次，stdout 一样是空的：

终端输出 终端输出 trapv-out

```
（无输出）
```

对照读这两次运行：**基础构建**下累加的结果是 `-1794967296`（五亿乘以五早就出了 `int` 的范围，回绕成了负数），程序若无其事地跑完、退出码 0；**加了 `-ftrapv` 之后**，进程在溢出那一刻就被终止，退出码 **3221226505**，**stdout 一个字节都没有**——因为 `printf` 的结果还在缓冲区里，进程被终止时直接丢了。清单 `ptr_offset_trapv.json` 里那个 `output` 块里只有一行哨兵「（无输出）」，就是这件事的机器核对版。

实验记录 · 本机实测：三种检测手段各自抓什么

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 三条可复现命令（都在各自例子的目录里跑）：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -fanalyzer -c src/main.c -o main.o`（分析器）、`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fstack-protector-all src/main.c src/stack_smash.c -o smash_guard.exe`（栈保护）、`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -ftrapv src/main.c src/ptr_offset_trapv.c -o offset_trap.exe`（溢出陷阱）；| 手段 | 什么时候说话 | 本机的证据 | 抓不到的 | | --- | --- | --- | --- | | `-fanalyzer` | **编译期**（退出码仍是 0） | `heap-based buffer overflow [CWE-122] [-Wanalyzer-out-of-bounds]` | 形状不够死板的越界；有误报 | | `-fstack-protector-all` | **运行期**（终止） | `*** stack smashing detected ***` + 3221226505 | 堆上的越界；没有局部数组的函数 | | `-ftrapv` | **运行期**（终止） | 退出码 3221226505，stdout 丢空 | 无符号回绕、浮点，以及任何「不是有符号整数溢出」的错 |

三行证据都通过了「**阴性对照**」：去掉那个参数再编译运行一遍，声明的证据一条都不出现——所以它们确实归各自的检测器管，不是基础参数碰巧报出来的。本书所有 `detect` 清单都会做这一步核对（闸门去掉检测器参数复算一遍），编造不出一份「假抓到的」证据。

本机没有 ASan/UBSan，`-Wstringop-overflow` 也不能当检测开关用（基础参数里的 `-Wall` 已经把它开着，加不加档位诊断一字不差，证据归不到它头上）——所以这本书里「运行期越界」的证据永远是三样：**退出码、stderr 上的那行字、gdb 的栈**。

9.10 `const` 与指针：三种写法，从右往左读

指针与 `const` 放在一起有**三种**写法，区别只有一个词的位置：

c examples/ch04/const_ptr/src/const_ptr.c 第 7–20 行

```c
int const_demo(void) {
    int x = 1;
    int y = 2;
    const int *p1 = &x;         /* 指向「常量」的指针：不能通过 p1 改 x */ 指向「常量」的指针
    int *const p2 = &x;         /* 常量指针：p2 自己不能改指向 */ 常量指针
    const int *const p3 = &x;   /* 两个都不能改 */ 两个都锁住

    *p1 = 10;                   /* 错：不能通过 p1 改它指的东西 */ 四种非法赋值里的第一种
    p2 = &y;                    /* 错：p2 自己不能改 */
    *p3 = 30;                   /* 错：不能通过 p3 改 */
    p3 = &y;                    /* 错：p3 自己也不能改 */

```

```
    return *p1 + *p2 + *p3;
}
```

```
10 指向「常量」的指针
11 常量指针
12 两个都锁住
14 四种非法赋值里的第一种
```

{} 语法规则 · 从右往左读，一次读一个词

声明的读法只有一个规则：**从变量名出发，先往右，再往左。**

- `const int *p1`：从 `p1` 往右没有东西，往左看 `*`——「`p1` 是一个指针」；再往左 `int` 带着 `const`——「指向 `const int`」。所以**不能通过 `p1` 改它指的东西**，但 `p1` 可以改指向（`p1 = &y;` 合法）。
- `int * const p2`：`const` 紧挨着 `p2`，锁住的是** `p2` 自己**——它必须一直指着 `x`，但 `*p2 = 5;` 可以改 `x`。
- `const int * const p3`：两边都锁：既不能改 `*p3`，也不能改 `p3`。

一句话记法：**`const` 在 `*` 右边（`* const p`）锁的是指针本身；在 `*` 左边（`const int *`）锁的是它指的东西。**

编译它。下面这块是 gcc 真实吐出来的 stderr（逐字，连对齐的空格都没动）：

编译诊断 编译诊断 `ch04-const-ptr-diag`

```
src/const_ptr.c: In function 'const_demo':
src/const_ptr.c:14:9: error: assignment of read-only location '*p1'
   14 |     *p1 = 10;                  /* 错：不能通过 p1 改它指的东西 */
      |         ^
src/const_ptr.c:15:8: error: assignment of read-only variable 'p2'
   15 |     p2 = &y;                   /* 错：p2 自己不能改 */
      |        ^
src/const_ptr.c:16:9: error: assignment of read-only location '*(const int *)p3'
   16 |     *p3 = 30;                  /* 错：不能通过 p3 改 */
      |         ^
src/const_ptr.c:17:8: error: assignment of read-only variable 'p3'
   17 |     p3 = &y;                   /* 错：p3 自己也不能改 */
      |        ^
```

四个错误，两两一组，规律很清楚：`assignment of read-only location '*p1'` 是「`*p1` 这个**位置**是只读的」（锁的是对象），`assignment of read-only variable 'p2'` 是「`p2` 这个**变量**是只读的」（锁的是指针本身）。第 3 条里那串 `*(const int *)p3` 是编译器告诉你它把 `p3` 的类型理解成了什么——正好就是 `const int *const` 从右往左读出来的结果。

实验记录 · 本机实测：`-Werror` 在这里其实没起作用

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/const_ptr.c -o app.exe`，然后把同一行加上 `-Werror` 再编一次；
- 清单 `examples/ch04/const_ptr/const_ptr.json` 的 `flags` 里带着 `-Werror`（本书对「讲防线」的例子一律这么配：这些例子是**编译期就该拦住**的），但我实测发现：**去掉 `-Werror`，诊断一字不差**——两次编译的 stderr 都是同样的 13 行（`diff` 出来是空的），编译器退出码也都是 1。

原因是这四处都是**硬错误**（`error:`）：C 语言里直接给一个 `const` 对象赋值，本身就是非法，不需要靠「把警告升级成错误」来拦。

真正需要 `-Werror` 的是**另一个方向**——把 `const int *` 交给一个 `int *` 参数（**丢弃** `const`）。本机实测：那种代码**只报一条警告**（`warning: passing argument 1 of 'set_first' discards 'const' qualifier from pointer target type [-Wdiscarded-qualifiers]`），编译退出码是 **0**；加上 `-Werror` 才变成编译失败。这条差别属于「接口设计」，是下一个单元（`chapters/ch04/03-pointer-idioms.md`）的主角。

工程惯例 · **工程惯例：接口上的 `const` 是谁的承诺**

函数参数上写不写 `const`，**标准不管你**，但它是这个接口对调用者的承诺，惯例是：

- **只读的输入**写 `const T *`：告诉调用者「我不会改你的东西」，也告诉编译器「这里可以放心优化」。本例子 `include/const_ptr.h` 第 5 行的 `int read_through(const int *p);` 就是这个形状；
- **要写的输出**才写 `T *`，并且在注释或命名上说明「它会写你的对象」；
- **锁指针自己的** `T * const` 参数**很少见**：形参本来就是副本，锁它没有意义。「锁指针自己」主要用在**局部变量**上（防止自己在后面的代码里改错指向）。

代价：接口一旦承诺了 `const T *`，实现里就不能改那个对象了（改了编不过）——这正是你要的；反过来说，**事后想改成 `const` 就难了**，因为要去掉所有「丢弃 const」的调用点，所以这件事要在**写第一版接口时**定下来。

9.11 三条要写成习惯的规矩

指针的行为到这里就讲完了。把最容易反复出错的三件事收成三条规矩，每条都能在上面找到证据：

1. **看到 `p[i]`，就在心里读成 `*(p + i)`**：它们完全等价（9.6 节的标准原文里写的是 `identical`）。这条规矩让你在「数组」与「指针」之间来回翻译时不会犯迷糊——`p[i]` 里的 `p` 是不是数组名，都不影响它。`ptr_arith` 里那个 `diff_of_two_forms` 就是拿这两种写法互相减，实测结果永远是 `0`。
2. **接口上「从哪开始」与「有几个」永远成对出现**：函数参数写成 `const T *a, size_t n`（`ptr_arith` 与 `ptr_types` 两个模块都是这个形状）。数组名一进参数就**退化**成指针，`sizeof` 再也问不出长度——**没有任何办法在函数里把长度找回来**（`sizeof_in_function` 实测给出 8，那是指针的大小）。
3. **判空写在「知道该怎么处理」的那一层**：能拒绝就拒绝（返回错误码 `-1`），拒绝不了就别解引用（那是 3221225477）。判空的责任与后果都要写在头文件注释里（`null_deref.h` 第 4 行）。

最后回到工程上：这一节新增了哪些文件、各自是干什么的。

目录树 **这一节新增的文件，逐个点名**

第 1 步 **本节建起的十个例子（前五个，各带 src/ + include/ + 清单）**

```
examples/
├── ch01/ : 第 1 章的项目（本节没动它）
└── ch04/
    ├── ptr_basics/ : & 与 *、指针参数 ← 本课新增
    ├── ptr_types/ : 一次读几个字节、对齐 ← 本课新增
    ├── null_deref/ : 空指针解引用、判空 ← 本课新增
    ├── ptr_arith/ : p + 1 与 p - q ← 本课新增
    └── dangling_stack/ : 返回局部数组地址 ← 本课新增
```

第 2 步 **另外五个例子，十个到齐**

```
examples/
└── ch04/
    ├── 前五个例子目录 : 同上
    ├── heap_after_free/ : free 之后再写 ← 本课新增
    ├── heap_oob_fanalyzer/ : 堆越界（分析器） ← 本课新增
    ├── stack_smash/ : 栈越界（两种量） ← 本课新增
    ├── ptr_offset_trapv/ : 用 int 算偏移 ← 本课新增
    └── const_ptr/ : 三种 const 写法 ← 本课新增
```

第 3 步 **会话与探针（十个例子目录都在，不再重复）**

```
examples/
└── ch04/
    ├── 十个例子目录 : 各带清单 + src/ + include/
    ├── session-gdb-pointers.txt : gdb 看指针的会话 ← 本课新增
    ├── session-regions-pointers.txt : 三个区的地址 ← 本课新增
    ├── session-double-free-pointers.txt : double free ← 本课新增
    ├── mem_regions.c : 三个区的探针 ← 本课新增
    └── double_free.c : double free 的探针 ← 本课新增
```

第 4 步 **两份汇编，9.9 节的对照材料**

```
examples/
└── ch04/
    └── heap_after_free/
        ├── main-O0.s : -O0 汇编，那句写还在 ← 本课新增
        └── main-O2.s : -O2 汇编，那句写被删 ← 本课新增
```

第 1 帧里的十个目录，每个都配了一份 `.json` 清单——清单里写着「这份程序怎么编译、跑完该输出什么、退出码是多少」，所以本章里每一块输出、每一段诊断、每一个退出码，都不是我抄上去的，而是流水线真跑出来、再与我写下的核对过的。第 2 帧那三样东西是**证据的另外三种形式**：

- **会话**（`session-*.txt`）：抓下来的终端原文。它适合**地址这类每次都不一样的输出**（9.7 节的三个区），或者像 gdb 那样带交互的输出；
- **汇编**（`main-O0.s` / `main-O2.s`）：证明「优化器真的把哪几句删了」（9.9 节）；
- **探针**（`mem_regions.c` / `double_free.c`）：单文件小程序，**故意没有清单**（一个的地址每次运行都变、一个是会被终止的程序），证据形式同样是会话。

下一单元（`chapters/ch04/03-pointer-idioms.md`，指针的用法与约定）从这里接着往工程方向走：`const` 在接口两边怎么配、`T **` 怎么用来改指针本身、`char *a[3]` 与 `char (*a)[3]` 怎么读、函数指针怎么当回调。这一节里那些「惯例」会在那里变成一整套接口形状。

小结 · **这一节的要点**

- **指针是一个变量**，里面装的是地址；`&` 与 `*` 是一对逆运算；真正决定行为的是**类型**——一次解引用读几个字节（`int *` 是 4、`char *` 是 1）、一步走多远，都由它定，本机是小端（低位字节住低地址）；
- **跨类型看字节用** `memcpy`：未对齐的硬解引用在本机能跑、在别的架子上可能直接陷阱，标准说它是 UB；`a[i]` 与 `*(a + i)` 完全等价，指针运算按元素走，越界指针的**计算本身**就是 UB；
- **生命期分三种**：栈上的自动存储期（函数返回就没了）、静态区（程序全程在）、堆（你 `free` 才没）；空指针解引用退出码 **3221225477**、stdout 可能一个字节都没有，`free` **不会**把指针置空，double free 在本机当场终止；
- **UB 不是「一定会崩」**：`-fanalyzer` 在编译期指认越界（编译仍退出 0），`-fstack-protector-all` 把栈越界变成 `*** stack smashing detected ***`（3221226505），`-ftrapv` 把有符号溢出变成终止（stdout 会丢）；`const` 的位置决定锁谁——从变量名出发「先右后左」地读。

自测题 · **自测题（先自己答，再对答案）**

1. `int x = 42; int *p = &x;` 之后，`&x`、`p`、`*p`、`&p` 四个表达式里，哪几个是**地址**、哪几个是**值**？`sizeof p` 是多少，为什么？
2. `int v = 0x41424344;` 之后用 `char *cp = (char *)&v;` 读，`cp[0]` 是多少？为什么不是 `0x41`？
3. `int a[5]; int *p = a;` 时，`p + 1` 比 `p` 大几个字节？`a + 4 - a` 是多少？把 `p` 换成 `char *`，这两个答案会变吗？
4. 一个函数 `char *f(void) { char buf[16] = "hi"; return buf; }`：它为什么错？本机实测调用者拿到的是什么？
5. `free(p);` 之后 `p` 里的值变了吗？为什么 `free(p); p = NULL;` 仍然救不了「同一块内存还有别的指针」的情况？

注 · **自测题答案**

1. `&x` 与 `p` 是**地址**（同一个地址）；`&p` 也是地址，但那是**指针变量自己**的地址（与 `&x` 不同）；`*p` 是**值**（42）。`sizeof p` 是 **8**——指针的大小由机器决定，与它指向什么类型无关（本机 8 字节）。
2. `cp[0]` 是 **0x44**。本机是**小端**：多字节整数里**低位字节住在低地址**，`0x41424344` 的最低字节是 0x44，排在第一个；`cp[3]` 才是 0x41（也就是 `'A'`）。
3. `p + 1` 比 `p` 大 **4** 个字节（`sizeof(int)`）；`a + 4 - a` 是 **4**（指针相减数的是**元素个数**）。把 `p` 换成 `char *`，第一个答案变成 **1**，第二个不变——相减得到元素个数，与元素大小无关（要字节数得先转成 `char *` 再减）。

4. 因为 `buf` 是**自动存储期**的对象，生命期只到函数返回；把它的地址交出去等于交出一张「马上作废」的地址条，标准说这是 UB（编译器会警告 `-Wreturn-local-addr`）。本机实测：编译器直接把这个返回值变成了空指针，调用者拿到 `NULL`，打印出来是 `(null)`——**这是本机实现的选择，不是规律**（换一种写法编译器就没看穿，真能读到栈上残留的字节）。
5. **没变。**gdb 实测：`free` 之前 `p` 是 `0x71cdc0`，`free` 之后还是 `0x71cdc0`——`free` 只把内存还给分配器，不碰你的变量。置空只**管住这一个**指针；同一块内存如果还有别的指针（别名），那些指针照样悬垂，用它们仍是 use-after-free。这也是为什么本书反复强调「谁分配谁释放」，而不是「记得置空」。

指针的用法与约定：接口、多级指针与函数指针

◎ 学习目标 · 看完这一单元，你应该能

- 说出接口上 `const T *` 与 `T *` 各代表谁的承诺，并把「丢掉 const」的那个调用点改对；
- 用「输出参数 + 错误码」写一个能交出两个结果、且失败时不碰输出参数的函数；
- 从变量名出发把 `char *a[3]`、`char (*a)[3]`、`int (*f)(int, int)` 读成人话，并说出它们的 `sizeof` 各是多少；
- 写出一张函数指针数组组成的分发表，并给 `qsort` 写一个能排 `char *` 数组的 comparator；
- 说出 `memcpy` 与 `memmove` 在本机的实测差别（没有差别），并解释为什么那不能当作「用 memcpy 搬重叠内存」的理由。

10.1 先看工程长什么样

上一单元（`chapters/ch04/02-pointers-and-memory.md`）讲的是指针**是什么**：一个装着地址的变量、类型决定一次读几个字节、栈与堆上的对象各活多久。这一单元换个角度问：**当你把指针写进函数签名，它代表什么约定？** 输入还是输出、谁分配谁释放、回调怎么传——这些都不是语法，是**接口的形状**，而形状全写在声明里。

这一节是第 4 章的第三单元、全书的第 10 单元。第 1 帧是这一节故事的起点：`examples/ch04/` 里已经躺着前两个单元留下的例子（数组与字符串那一批、指针与内存那十个），本节一个都不动它们。

目录树 **这一节给 examples/ch04/ 添了九个例子目录和四个会话文件**

第 1 步 **本节开始时，ch04 里已经有前两个单元留下的东西**

```
examples/
├── ch01/ … ch08/ ：各章的项目，本节不动
└── ch04/
    ├── 第 8 单元的十二个例子 ：数组与字符串
    ├── 第 9 单元的十个例子 ：指针与内存
    └── 探针与会话 ：warn_probe.c 等
```

第 2 步 **本节新写的九个例子，前三个**

```
examples/
└── ch04/
    ├── const_params/ ：三种 const ← 本课新增
    ├── const_loss/ ：丢了 const ← 本课新增
    └── out_params/ ：输出参数 + 错误码 ← 本课新增
```

第 3 步 **中间三个**

```
examples/
└── ch04/
    ├── ptr_to_ptr/ ：改指针本身 ← 本课新增
    ├── array_of_ptr/ ：指针数组与二维数组 ← 本课新增
    └── funcptr_dispatch/ ：分发表 ← 本课新增
```

第 4 步 最后三个，九个到齐

```
examples/
└── ch04/
    ├── qsort_callback/ : qsort 回调 ← 本课新增
    ├── memcpy_overlap/ : 重叠搬迁 ← 本课新增
    └── ptr_int/ : 指针当整数 ← 本课新增
```

第 5 步 两个会话，落在各自的例子目录里

```
examples/
└── ch04/
    ├── const_loss/
    │   └── session-discard-qualifiers.txt ← 本课新增
    └── memcpy_overlap/
        └── session-restrict.txt ← 本课新增
```

第 6 步 另外两个会话

```
examples/
└── ch04/
    ├── array_of_ptr/
    │   └── session-param-decay.txt ← 本课新增
    └── qsort_callback/
        └── session-gdb-funcptr.txt ← 本课新增
```

第 7 步 每个例子目录内部都是这个形状（以 const_params 为例）

```
examples/ch04/const_params/
├── src/ : main.c 与 const_params.c ← 本课新增
├── include/ : const_params.h ← 本课新增
└── const_params.json : 清单 ← 本课新增
```

第 2 到 4 帧是本节搭起来的九个目录（分三帧列完）。第 5、6 帧是四个**会话文件**，各自落在它讲的那个例子目录里（不是 `ch04/` 根下）——终端原文，装的是「每次运行都不一样」或带交互的证据（gdb 的地址就是典型）。第 7 帧是目录内部的形状：**定义放 `src/`、声明放 `include/`、编译与对账的参数放清单**——第 1 单元 1.7 节讲过分文件的理由，从第二个项目起本书不允许单文件工程。

九个目录各负责一件事，后面每一节都会点名：

- `const_params`、`const_loss` —— 接口上的 `const` 承诺，以及**丢掉**它会看到什么（10.2 节）；
- `out_params` —— 输出参数 + 错误码这套惯例（10.3 节）；
- `ptr_to_ptr` —— `T **` 到底在改谁（10.4 节）；
- `array_of_ptr` —— `char *names[]` 与 `char rows[3][8]` 的对照（10.5 节）；
- `funcptr_dispatch`、`qsort_callback` —— 函数指针、分发表与回调（10.6、10.7 节）；
- `memcpy_overlap`、`ptr_int` —— 重叠搬迁、把指针当整数看（10.9、10.10 节）。

工程惯例 · 工程惯例：为什么接口这件事要单独学一节

标准对函数签名只有一个硬要求：**每个实参的值必须能赋给一个「形参类型去掉限定符」的对象**（6.5.2.2 节第 2 段，10.2 节会引原文）。**怎么写签名、参数代表输入还是输出、谁负责释放内存，标准一个字都没规定**——全是惯例。

惯例值得单独学，是因为它**有代价、也能被验证**：`const char *` 写进签名，实现里就改不动它指向的东西，代价是你得在**写第一版接口时**想清楚；反过来，写成 `char *` 却打算只读，调用者看不出，早晚有人把只读字符串交给它（10.2 节的 `const_loss` 就是真实报错）。

这一节做的每一件事，都是把一条**口头惯例**变成签名上一眼能看见的东西。

10.2 接口的两端：只读输入与可写输出

先说结论：函数参数里的指针，只有两种意图。**要么是「你给我的数据，我只读」**，写成 `const T *`；**要么是「我给你的数据会写进去」**，写成 `T *`。第三种写法（锁住指针自己）在形参上几乎没用，但它必须会读，不然你在真实代码里会被它绊住。

三种写法摆在一起长这样（`examples/ch04/const_params/include/const_params.h`，注释就是这份接口的说明书）：

c examples/ch04/const_params/include/const_params.h 第 6–14 行

```c
/*
 * 接口上三种 const 写法，区别只在 const 挨着谁：
 *   1. const char *text —— 只读输入：函数承诺不改调用者的数据
 *   2. int *out        —— 可写输出：函数一定会写调用者的对象
 *   3. int * const base —— 锁住指针自己（形参上少见，实现里常见）
 */
size_t count_matches(const char *text, char target); 只读输入，我不会改它
int set_first(int *out, size_t n, int value); 可写输出，我一定会写它
void dump(int * const base, size_t n); 锁住指针自己，形参上少见
```

12 只读输入，我不会改它
13 可写输出，我一定会写它
14 锁住指针自己，形参上少见

读法在上一单元给过，这里要补的是**它落在调用点上的样子**。下面是同一个例子的 `main`，前两行是合法调用，中间那两行是这份例子故意写错的：

c examples/ch04/const_params/src/main.c 第 4–19 行

```c
int main(void) {
    int values[3] = {10, 20, 30};
    const int *read_only = values;      /* 只读视角：不许通过它改 */ 只读视角
    int *writable = values;             /* 可写视角 */ 可写视角
    const char *text = "pointer";

    printf("只读输入 count_matches(\"%s\", 'o') = %zu\n", text, count_matches(text, 'o'));
    int rc = set_first(writable, 3, 1);
    printf("可写输出 set_first 返回 %d, values[0] 变成 %d\n", rc, values[0]);

    set_first(read_only, 3, 1);         /* 错 1：把只读视角交给要写的参数 */ 错 1：交给了要写的参数
    writable = read_only;               /* 错 2：赋值也会丢掉 const */ 错 2：赋值也会丢

    dump(values, 3);
    return 0;
}
```

6 只读视角
7 可写视角
14 错 1：交给了要写的参数
15 错 2：赋值也会丢

这份程序**编不过**——但注意它编不过的原因：那两行本身是**警告**，是清单里那个 `-Werror` 把它们升级成了错误。gcc 原样吐出来是这样（逐字，连对齐的空格都没动）：

编译诊断 编译诊断 ch04-const-params-diag

```
src/main.c: In function 'main':
src/main.c:14:15: error: passing argument 1 of 'set_first' discards 'const' qualifier from pointer target
type [-Werror=discarded-qualifiers]
   14 |     set_first(read_only, 3, 1);        /* 错 1：把只读视角交给要写的参数 */
      |               ^~~~~~~~~
In file included from src/main.c:1:
include/const_params.h:13:20: note: expected 'int *' but argument is of type 'const int *'
   13 | int set_first(int *out, size_t n, int value);
      |               ~~~~~^~~
src/main.c:15:14: error: assignment discards 'const' qualifier from pointer target type [-Werror=discarded-
qualifiers]
   15 |     writable = read_only;              /* 错 2：赋值也会丢掉 const */
      |              ^
cc1.exe: all warnings being treated as errors
```

两处错误，规律很清楚：`passing argument 1 … discards 'const' qualifier from pointer target type` 说的是**实参比形参少了 const**，编译器还贴心地把两边的类型并排摆给你看（`expected 'int *' but argument is of type`

`'const int *'`）；第二条 `assignment discards 'const' qualifier` 是同一件事发生在赋值上。**「丢 const」这个动作本身是单向的**，标准 6.3.2.3 节第 2 段只允许反过来加：

> **标准原文 · 标准原文：为什么丢 const 要报警告**
>
> 同一个标准的 6.5.16.1 节「Simple assignment」列出了简单赋值必须满足的条件之一：
>
> > the left operand has atomic, qualified, or unqualified pointer type, and (considering the type the left operand would have after lvalue conversion) both operands are pointers to qualified or unqualified versions of compatible types, and the type pointed to by the left has all the qualifiers of the type pointed to by the right;
>
> 大意：**左边指针指向的类型必须带上右边指向的类型的所有限定符。**也就是说「`int *` ← `const int *`」不满足这个条件，而「`const int *` ← `int *`」满足（左边多了个 `const`，多出来的不算违规）。这条约束管的是**赋值**；**传参**则由 6.5.2.2 节第 2 段管住——「每个实参的类型，必须让它的值能赋给一个『形参类型去掉限定符』的对象」，于是同一条约束也落在调用点上。至于「必须给诊断」，那是 5.1.1.3 节的要求（违反任何约束或语法规则，实现至少要给一条诊断）；**诊断是 `warning` 还是 `error`，标准不管**——本机选了前者，也就是你上面看到的这条。
>
> 出处：ISO/IEC 9899:2018（C17）§6.5.16.1 第 1 段、§6.3.2.3 第 2 段。

「只报警告」这件事值得单独钉一下，因为它决定了**这份例子为什么要带 `-Werror`**。本节的第二个例子 `const_loss` 把这件事做到最小：接口要写调用者的缓冲区，所以第一个参数是 `char *`；而 `main` 手里只有一个 `const char *`：

c examples/ch04/const_loss/src/main.c 第 4–14 行

```c
int main(void) {
    const char *src = "hello";
    char buf[16] = {0};

    size_t n = uppercase_into(buf, sizeof buf, src);
    printf("正常调用：写了 %zu 个字符，buf = \"%s\"\n", n, buf);

    uppercase_into(src, sizeof src, "world");   /* 错：把只读的起点交给了要写的接口 */ 错：只读的起点交给要写的接口

    return 0;
}
```

11 错：只读的起点交给要写的接口

接口本身在头文件里，两行就够（第 7 行那个 `char *out` 就是那个「会写」的参数）：

c examples/ch04/const_loss/include/const_loss.h 第 6–9 行

```c
/* 这个接口要写调用者的缓冲区，所以第一个参数是 char *（没有 const） */
size_t uppercase_into(char *out, size_t cap, const char *src); 要写的参数没有 const

#endif
```

7 要写的参数没有 const

现在做那个实验：同一份源码，一次**不带** `-Werror`，一次带。下面三块是 `examples/ch04/const_loss/session-discard-qualifiers.txt` 里的会话原文，按「编过 → 跑崩 → 被拦住」三步拆开。

那份被流水线拦住的诊断（清单里 `"mode": "compile_fail"`、`flags` 带 `-Werror`）就是下面这一块——它是这份例子在 `-Werror` 下编译器吐出来的全部 stderr：

编译诊断 编译诊断 ch04-const-loss-diag

```
src/main.c: In function 'main':
src/main.c:11:20: error: passing argument 1 of 'uppercase_into' discards 'const' qualifier from pointer
target type [-Werror=discarded-qualifiers]
   11 |     uppercase_into(src, sizeof src, "world");   /* 错：把只读的起点交给了要写的接口 */
      |                    ^~~
In file included from src/main.c:1:
include/const_loss.h:7:29: note: expected 'char *' but argument is of type 'const char *'
    7 | size_t uppercase_into(char *out, size_t cap, const char *src);
      |                       ~~~~~~^~~
cc1.exe: all warnings being treated as errors
```

终端会话 第 1 次：基础参数，只有 warning，退出码 0 ch04-discard-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c
src/const_loss.c -o app.exe; echo "编译退出码 $?"
src/main.c: In function 'main':
src/main.c:11:20: warning: passing argument 1 of 'uppercase_into' discards 'const' qualifier from pointer
target type [-Wdiscarded-qualifiers]
   11 |     uppercase_into(src, sizeof src, "world");   /* 错：把只读的起点交给了要写的接口 */
      |                    ^~~
In file included from src/main.c:1:
include/const_loss.h:7:29: note: expected 'char *' but argument is of type 'const char *'
    7 | size_t uppercase_into(char *out, size_t cap, const char *src);
      |                       ~~~~~~^~~
编译退出码 0
```

终端会话 （续）第 2 步：带着这处错误跑起来，进程当场死 ch04-discard-2

```
$ ./app.exe; echo "退出码 $?"
退出码 139
$ gdb --batch -ex run -ex "bt" ./app.exe
[New Thread 110960.0x17efc]
Thread 1 received signal SIGSEGV, Segmentation fault.
0x00007ff642601595 in uppercase_into (out=0x7ff642604000 "hello", cap=8, src=0x7ff64260403a "world") at
src/const_loss.c:10
10              out[i] = (char)toupper((unsigned char)src[i]);
#0  0x00007ff642601595 in uppercase_into (out=0x7ff642604000 "hello", cap=8, src=0x7ff64260403a "world") at
src/const_loss.c:10
#1  0x00007ff642601524 in main () at src/main.c:11
```

终端会话 （续）第 3 步：同一个错误，加了 -Werror 就编不过 ch04-discard-3

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -Werror src/main.c
src/const_loss.c -o app.exe; echo "编译退出码 $?"
src/main.c: In function 'main':
src/main.c:11:20: error: passing argument 1 of 'uppercase_into' discards 'const' qualifier from pointer
target type [-Werror=discarded-qualifiers]
   11 |     uppercase_into(src, sizeof src, "world");   /* 错：把只读的起点交给了要写的接口 */
      |                    ^~~
In file included from src/main.c:1:
include/const_loss.h:7:29: note: expected 'char *' but argument is of type 'const char *'
    7 | size_t uppercase_into(char *out, size_t cap, const char *src);
      |                       ~~~~~~^~~
cc1.exe: all warnings being treated as errors
编译退出码 1
$ rm -f app.exe
```

三步摆在一起，结论有四条：

- **同一个错误，两次编译的措辞几乎一样**（`discards 'const' qualifier` 一个字都没变），变的只有 `warning:` 与 `error:` 那个词、方括号里的开关名（从 `-Wdiscarded-qualifiers` 变成 `-Werror=discarded-qualifiers`），以及最后多出来的 `cc1.exe: all warnings being treated as errors`；
- **退出码从 0 变成 1。**不带 `-Werror` 时编译器认为这份程序**可以编**，`app.exe` 照样产出来——也就是说：接口上丢了 `const`，**默认的构建流水线不会拦你**；
- 那两行 `note: expected 'char *' but argument is of type 'const char *'` 是编译器替你做的类型对照，读诊断时先看它，比看那一串英文容易得多；

- **第 2 步才是这件事的分量**：编出来了，`./app.exe` 却**退出码 139**（SIGSEGV），gdb 停在 `src/const_loss.c:10` ——`out` 拿到的正是只读的 `"hello"`。**丢 const 不是「只是难看」，是「跑起来就死」。**

实验记录 · 本机实测：丢 const 的三步退出码

- 工具链：gcc 16.1.0（本书统一 `-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 第一次命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/const_loss.c -o app.exe`，诊断是**一条 warning**，**编译退出码 0**；
- 第 2 步：直接运行第 1 步编出来的 `app.exe`，**退出码 139**（SIGSEGV），gdb 停在 `src/const_loss.c:10`；
- 第 3 步：同一行加上 `-Werror`，同一处变成 **error**，**编译退出码 1**；
- 这份例子的清单 `examples/ch04/const_loss/const_loss.json` 里 `"mode": "compile_fail"`、`"flags": ["-Iinclude", "-Werror"]`：教学点就是「编译器默认不拦、工程上要靠 `-Werror` 拦」，而闸门要求 `compile_fail` 的清单真的编不过，所以 flags 里必须带它。
- 顺带说清界限：**编译诊断本身不是 UB**——真正踩 UB 的是运行期那一次写。程序带着这处错误照样能编出来（退出码 0），但一跑就死（第 2 步：退出码 139，gdb 停在写只读字符串的那一行）。**这就是 `const` 值得守的原因：它把「跑起来才死」的问题，提前成了一条编译期警告。**

工程惯例 · 工程惯例：`const T *` 与 `T *` 是谁的承诺

标准不管你写不写 `const`（6.5.16.1 只要求类型能配上），但接口上的 `const` 是这个函数对调用者的**承诺**，惯例有三条：

- **只读的输入一律写 `const T *`**：调用者一看就知道「我的数据不会被改」，而且实现里也改不动（改了编不过）——**这是把制度写进类型里**；
- **要写出去的对象写 `T *`**，并在头文件注释里说清「它会写你哪几个字节」。本节的 `out_params`（10.3 节）与 `array_of_ptr`（10.5 节）都是这个形状；
- **形参上的 `T * const` 基本没用**：形参是一份副本（上一单元讲过，指针也不例外），锁住副本管不到调用者，也不改变接口语义。它偶尔被当成「我保证不改这个指针变量」的自我约束写在函数体里（形参在函数体内就是局部对象），但作为**接口**它什么也没承诺。

代价也要说清：接口一旦承诺了 `const T *`，**日后想把它改成 `T *` 只需删一个词**——编译器只给警告、照样放行（上面那个实验），这个承诺很容易被悄悄破坏；反过来，从一个没写 `const` 的接口**补上** `const`，会立刻在实现里撞出一串「不能改它」的编译错误。所以「这块是不是只读」要在**写第一版接口**时定下来。

陷阱 · 「const 只是给人看的注释，加不加都一样」

你会怎么被坑到：你在头文件里顺手写下 `size_t uppercase_into(char *out, …)`，心里想的是「我只往里写，不会读它」。三个月后有人在另一个文件里写 `uppercase_into(src, …)`，`src` 是个 `const char *`——**编译通过**（只有一条警告，很可能被淹没在几百行输出里），程序跑起来却往只读内存里写；本节实测的后果是**当场 SIGSEGV**（退出码 139）。

我会怎么误判：我以前觉得「警告又不是错误，先跑起来再说」。真相是：这条警告是编译器**唯一**会告诉你的地方——类型系统能管的边界就到这里，它不能替你判断「你心里想的那件事」，只能判断「两个类型配不配」。

怎么查：把警告当错误跑一遍（加 `-Werror`，或者把编译输出存下来 grep 一遍 `discards`），上面那份实验里两次编译的区别就是全部证据。

10.3 输出参数 + 错误码：C 怎么「返回两个东西」

一个函数想交出两个结果，C 没有这个语法（不像别的语言有元组）。惯例是把**状态放在返回值、结果放在输出参数**：返回值 `0` 表示成功、负数当错误码，真正要交出去的东西通过 `T *` 参数写回调用者手里。

这个惯例里最容易被忽略的一条约定是：**失败时一个字节都不许写。**头文件把三条约定写在了注释里：

c examples/ch04/out_params/include/out_params.h 第 6–14 行

```c
/*
 * C 没有「一个函数返回两个值」的语法，惯例是：
 *   返回值    = 状态（0 成功，负数是错误码）  返回值是状态
 *   额外的结果 = 通过 T * 参数写出去（输出参数）  结果从参数走出去
 * 约定：失败时不碰输出参数 —— 调用者手里的旧值不会被污染。  失败时不碰输出参数
 */
int divide_checked(int a, int b, int *quotient, int *remainder);
int parse_int_checked(const char *text, int *out_value);
const char *error_text(int code);
```

8 返回值是状态
9 结果从参数走出去
10 失败时不碰输出参数

`divide_checked` 的实现把这条约定摆得最直白——两个 `return -N` 都发生在**任何写动作之前**：

c examples/ch04/out_params/src/out_params.c 第 6–16 行

```c
int divide_checked(int a, int b, int *quotient, int *remainder) {
    if (quotient == NULL || remainder == NULL) {  接口第一件事：判空
        return -1;                      /* 接口的第一件事：判空 */
    }
    if (b == 0 || (a == INT_MIN && b == -1)) {  除数为零或商装不下，直接返回
        return -2;                      /* 除数为零，或商装不下（INT_MIN / -1）*/
    }
    *quotient = a / b;  走到这里才写第一个结果
    *remainder = a % b;  第二个结果
    return 0;                           /* 0 表示「两个输出参数都写好了」 */
}
```

7 接口第一件事：判空
10 除数为零或商装不下，直接返回
13 走到这里才写第一个结果
14 第二个结果

第二个函数要把字符串转成 `int`，它多了一层麻烦：溢出时 `strtol` 会把 `errno` 置成 `ERANGE`，同时**返回一个被夹住的值**（`LONG_MAX` / `LONG_MIN`）——这个值与一个真实结果没法区分（这也是 C 里另一种常见的报错方式，第 12 单元会再遇到）：

c examples/ch04/out_params/src/out_params.c 第 18–33 行

```c
int parse_int_checked(const char *text, int *out_value) {
    if (text == NULL || out_value == NULL) {
        return -1;
    }
    char *end = NULL;
    errno = 0;                          /* strtol 用 errno 报告越界 */  先清掉上一次留下的错误
    long long v = strtoll(text, &end, 10);  strtoll 比 long 宽，安全
    if (end == text || *end != '\0') {
        return -2;                      /* 整串不都是数字 */
    }
    if (errno == ERANGE || v < INT_MIN || v > INT_MAX) {  越界也算参数不合法
        return -2;                      /* 超出 int 能表示的范围 */
    }
    *out_value = (int)v;
    return 0;
}
```

23 先清掉上一次留下的错误
24 strtoll 比 long 宽，安全
28 越界也算参数不合法

调用点长这样——注意每一次都先把 `code` 接住、再决定要不要读那些输出参数：

c examples/ch04/out_params/src/main.c 第 5–16 行

```c
int main(void) {
    int q = 0;
    int r = 0;
    int code = divide_checked(17, 5, &q, &r); 两个输出参数都是 & 取的地址
    printf("divide_checked(17, 5) 返回 %d: 商 = %d, 余数 = %d\n", code, q, r);

    code = divide_checked(17, 0, &q, &r); 同一个调用，除数为零
    printf("divide_checked(17, 0) 返回 %d (%s) , q 与 r 没被碰过: %d %d\n",
           code, error_text(code), q, r);

    code = divide_checked(INT_MIN, -1, &q, &r); 商装不下的那一次
    printf("divide_checked(INT_MIN, -1) 返回 %d (%s) \n", code, error_text(code));
```

8 两个输出参数都是 & 取的地址
11 同一个调用，除数为零
15 商装不下的那一次

跑出来的结果（这块输出由流水线逐字对账，不是我抄的）：

终端输出 终端输出 out-params-out

```
divide_checked(17, 5) 返回 0: 商 = 3, 余数 = 2
divide_checked(17, 0) 返回 -2 (参数不合法) , q 与 r 没被碰过: 3 2
divide_checked(INT_MIN, -1) 返回 -2 (参数不合法)
parse_int_checked("  -42x") 返回 -2 (参数不合法) , value 还是 0
parse_int_checked("-42") 返回 0, value 变成 -42
parse_int_checked("99999999999") 返回 -2 (参数不合法) , value 还是 -42
```

六行输出，逐行读：

- **`divide_checked(17, 5) 返回 0: 商 = 3, 余数 = 2`**：一次调用交出两个数，17 ÷ 5 的商与余数都对；
- **`divide_checked(17, 0) 返回 -2, q 与 r 没被碰过: 3 2`**：除数为零时函数在写之前就返回了，`q`、`r` 里还是**上一次**的 3 与 2——「失败时不碰输出参数」的实测样子。若实现里先写后判，你会在这里看到垃圾值，而且很难发现；
- **`divide_checked(INT_MIN, -1) 返回 -2 (参数不合法)`**：`INT_MIN / -1` 的商在 `int` 里装不下，**标准规定这时 `/` 与 `%` 都是未定义行为**（6.5.5 节第 6 段）——所以这个接口把「除数为零」和「商装不下」一起挡在写之前；
- **`parse_int_checked("  -42x") 返回 -2`**：`strtoll` 会跳过前导空格、也允许后面跟非数字字符，「整串都是数字」这条约束得由**我们**加（`*end != '\0'`）；
- **`parse_int_checked("-42") 返回 0, value 变成 -42`**：成功的那一次；
- **`parse_int_checked("99999999999") 返回 -2`**：这个数比 `int` 能表示的最大值大得多。**这一条是我写第一版时踩的坑**：我最初用 `long` 接 `strtol` 的结果，再判 `v > INT_MAX`——在 Windows 上 `long` 也是 32 位，`strtol` 遇到装不下的数会**返回一个被夹住的值**（`2147483647`）而只在 `errno` 里留下 `ERANGE`，于是我那个判断永远不成立、函数高高兴兴返回 0，把 `2147483647` 当成了「解析成功」。改成先用 `errno` 判、再用更宽的 `long long` 接值，才是对的。

{} 语法规则 · 输出参数的写法与它欠的三句话

形状只有一条：**要交出去的结果，用 `T *` 收，调用者拿 `&` 取地址交进来。**

- 参数写成 `int *out`（输出一个 int）、`const char **out`（输出一个字符串指针）；
- 返回值写 `int`，`0` 是成功、负数是错误码（正数留给「有效值的个数」这种用法）；
- 接口必须写清三件事：**失败时输出参数写不写**（本书一律「不写」）、**谁负责分配**（见下面的惯例块）、**错误码各是什么意思**（`error_text` 那种函数就是干这个的）。

长度与指针永远成对出现：输出参数是 `(char *out, size_t cap)`，`cap` 是你那块缓冲区**总共多大**（含结尾那个 `'\0'` 的位置）——第 8 单元的 `fgets` 与第 15 单元的 `snprintf` 都是这个形状。

工程惯例 · 工程惯例：谁分配谁释放，写进接口而不是记在脑子里

「谁分配谁释放」是这一行最有用的规矩，它的完整形式是：**分配那块内存的人，负责把它还回去**，并且这句话要能从头文件里读出来。三种常见写法：

- **调用者出内存**（最主流）：`void greet(char *out, size_t cap, const char *who);` —— `out` 是调用者的数组，函数只负责写。好处是谁也不用记着 `free`，代价是调用者得先知道「要多大」；
- **函数分配、调用者释放**：`char *make_greeting(const char *who);` 返回 `malloc` 来的指针，**头文件注释里必须写「用完 free」**。代价很实在：调用者漏一次 `free` 就是一处泄漏（第 12 单元讲）；
- **函数分配、函数释放**：返回一个「内部缓冲区」的地址，下一次调用会把它覆盖掉（上一单元 `static` 那段讲过它的代价：不能重入）。

代价的读法是一致的：**只要内存的生命期跨过了函数边界，就必须有人把它写成文字**——写在头文件注释里、或者干脆让它在类型上消失（像第一种那样，压根没有要还的东西）。

10.4 多级指针：改指针本身要用 `T **`

上一单元的 `ptr_basics` 里演示过一个「失败的交换」：函数里写 `a = b; b = tmp;`，调用者的两个指针一点没动。原因不是指针特殊，而是**C 的参数永远是值传递**。要改调用者手里的那个指针，函数就得拿到**那个指针的地址**——于是参数类型变成 `T **`。

这份接口把三件事并排摆开，头文件里一句话一件：

c examples/ch04/ptr_to_ptr/include/ptr_to_ptr.h 第 6–16 行

```c
/* 改「指针指向的值」：一个 T * 就够 */
void set_pointed(int *p, int value);

/* 改「指针本身」：必须收到那个指针的地址，所以参数是 T ** */ 参数是 T **，收的是指针的地址
void point_to(int **pp, int *target);

/* 两级解引用：先顺一层找到指针，再顺一层找到对象 */ 两级解引用
void set_via_double(int **pp, int value);

/* char *argv[] 与 char **argv 是同一个类型 —— 这就是 main 的第二个参数 */ argv 与 char ** 同一个类型
void show_args(char **argv);
```

9 参数是 T **，收的是指针的地址
12 两级解引用
15 argv 与 char ** 同一个类型

实现三行，分别是三种意图：

c examples/ch04/ptr_to_ptr/src/ptr_to_ptr.c 第 4–14 行

```c
void set_pointed(int *p, int value) {
    *p = value; 一个 *，改的是指向的对象
}

void point_to(int **pp, int *target) {
    *pp = target;                   /* 改的是调用者手里那个指针 */ 一个 *，改的是调用者那个指针
}

void set_via_double(int **pp, int value) {
    **pp = value;                   /* 先顺一层到指针，再顺一层到对象 */ 两个 *，先到指针再到对象
}
```

5 一个 *，改的是指向的对象
9 一个 *，改的是调用者那个指针
13 两个 *，先到指针再到对象

`char *argv[]` 那一句注释不是装饰：`show_args` 的参数写的是 `char **argv`，而 `main` 的第二个参数在标准里是 `char *argv[]`——**这两种写法是同一个类型**（6.7.6.3 节讲声明符的那条规则：形参里的 `T a[]` 与 `T *a` 等价）。所以下面 `main` 里自己造一份 `fake_argv` 交给它，和把真正的命令行参数交给它是一回事：

c examples/ch04/ptr_to_ptr/src/main.c 第 9–25 行

```c
9       printf("一开始: a = %d, b = %d, *p = %d\n", a, b, *p);
10
11      set_pointed(p, 10); 只给了 T *
12      printf("set_pointed(p, 10) 之后: a = %d, p 还指着 a = %s\n",
13             a, (p == &a) ? "是" : "不是");
14
15      point_to(&p, &b); 交出去的是 &p
16      printf("point_to(&p, &b) 之后: a = %d, b = %d, *p = %d, p 指着 b = %s\n",
17             a, b, *p, (p == &b) ? "是" : "不是");
18
19      set_via_double(&p, 20); 交出去的是 &p，想改的还是 p
20      printf("set_via_double(&p, 20) 之后: b = %d, p 还指着 b = %s\n",
21             b, (p == &b) ? "是" : "不是");
22
23      printf("main 的第二个参数就是这个类型（这里自己造一份）：\n");
24      char *fake_argv[] = {"prog", "-v", "file.txt", NULL}; 一个以 NULL 收尾的指针数组
25      show_args(fake_argv);
```

11 只给了 T *
15 交出去的是 &p
19 交出去的是 &p，想改的还是 p
24 一个以 NULL 收尾的指针数组

真跑出来：

终端输出 终端输出 ptr-to-ptr-out

```
一开始: a = 1, b = 2, *p = 1
set_pointed(p, 10) 之后: a = 10, p 还指着 a = 是
point_to(&p, &b) 之后: a = 10, b = 2, *p = 2, p 指着 b = 是
set_via_double(&p, 20) 之后: b = 20, p 还指着 b = 是
main 的第二个参数就是这个类型（这里自己造一份）：
  argv[0] = "prog"
  argv[1] = "-v"
  argv[2] = "file.txt"
```

前四行一组读，答案全在「谁被改了」上：

- **`set_pointed(p, 10)` 之后 `a = 10`，而 `p` 还指着 `a`**：参数只是一个 `T *`，函数顺着它改了**指向的对象**；
- **`point_to(&p, &b)` 之后 `p` 指着 `b` 了**：这一次交出去的是 `&p`（指针自己的地址），函数里的 `*pp = target` 改的正是 `main` 手里那个 `p`；
- **`set_via_double(&p, 20)` 之后 `b = 20`，而 `p` 还指着 `b`**：`**pp = value` 顺了两层——第一层 `*pp` 拿到 `p`，第二层 `*pp` 拿到 `b`。所以它要的是 `T **`，改的却是 `T`。

最后三行是 `show_args` 打的（它上面那一行「main 的第二个参数就是这个类型」是 `main` 自己打的）：`fake_argv` 是一个以 `NULL` 收尾的指针数组，这也正是真实 `argv` 的形状——**`p[argc]` 是空指针**，所以 `for (i = 0; argv[i] != NULL; i++)` 这种循环不用另外传长度。

大白话 · 二级指针没那么玄：它是「指针的地址」

把内存想成一排信箱。`int **pp` 的意思是：这个信箱里写的，是另一个信箱的编号；而那个信箱里写的，才是数据的编号。

- `set_pointed(int *p, …)`：你交出一张写着 `a` 的纸条，函数按纸条找到 `a`，改里面的值——**纸条是副本，但它指着的地方是真的**；
- `point_to(int **pp, …)`：你把**放纸条的那个抽屉**指给函数看，函数把抽屉里的纸条换成另一张（换成 `b` 的编号）；
- `set_via_double(int **pp, …)`：函数打开抽屉拿到纸条，再按纸条找到数据去改。

所以「改指针本身」和「改指针指向的东西」的区别，就是**你交出去的是纸条，还是放纸条的那个抽屉。**

> ⚠ **陷阱 · `T **` 用错的两种典型样子**
>
> **你会怎么被坑到**：这两种写法都能编过，但都不做你想做的事。
>
> - **想改指针，却传了指针**：`void point_to(int *pp, int *target) { pp = target; }` 再调 `point_to(p, &b);` ——形参 `pp` 是一份副本，改它等于什么都没做。编译不报错（类型完全合法），运行时调用者的 `p` 一动不动；
> - **写成了 `T **`，却顺手解引用了一次**：`void set_via_double(int **pp, int value) { *pp = value; }` ——这一行的类型就错了（`int **` 赋 `int` 值），编译器会拦；但换个更宽的类型（比如把 `value` 写成指针）它就悄悄编过，然后你改掉的是**指针**，不是它指的东西。
>
> **真相**：二级指针的**每一个 `*` 都对应「顺着走一层」**，写完先数一遍：`pp` 是指针的地址、`*pp` 是那个指针、`**pp` 才是数据。想改什么，就写到第几层。

10.5 声明读法：`char *a[3]` 与 `char (*a)[3]`

这两个声明长得很像，意思却相反——一个是**指针数组**（三个格子，每格一个地址），一个是**数组指针**（一个格子，里面是「指向三个元素的数组」的地址）。读法只有一条，而且它就是 C 声明符的规则（6.7.6 节）：**从变量名出发，先往右看，再往左看，遇到括号就掉头。**

同一份程序里把两种写法都跑了一遍：

c examples/ch04/array_of_ptr/src/main.c 第 5–24 行

```c
5       char *names[3] = {"ada", "bob", "christopher"};   /* 指针数组：三格地址条 */ 往右看是 [3]，在读一个数组
6       char rows[3][8] = {"ada", "bob", "erin"};         /* 二维数组：三行八字节 */ 往右是 [3][8]，是二维数组
7       char tight[3][5] = {"ada", "bob", "erin"};        /* 行更窄的二维数组 */ 每行只有 5 个字符
8
9       printf("sizeof(names) = %zu, sizeof(names[0]) = %zu (一格是一个指针) \n",
10              sizeof names, sizeof names[0]);
11      printf("sizeof(rows)  = %zu, sizeof(rows[0])  = %zu (一格是八个字符) \n",
12              sizeof rows, sizeof rows[0]);
13      printf("sizeof(tight) = %zu, sizeof(tight[0]) = %zu\n",
14              sizeof tight, sizeof tight[0]);
15
16      printf("names 的数组地址与它存的地址相同吗：%s\n", 数组地址 vs 它存的地址
17              ((void *)&names[0] == (void *)names[0]) ? "是" : "不是");
18      printf("rows 的数组地址与第一行的地址相同吗：%s\n", 两个地址是同一处
19              ((void *)&rows[0] == (void *)rows[0]) ? "是" : "不是");
20
21      printf("往前走一格：names + 1 走 %td 字节，rows + 1 走 %td 字节，tight + 1 走 %td 字节\n", 三种「走一格」各走多少
22              (char *)(names + 1) - (char *)names,
23              (char *)(rows + 1) - (char *)rows,
24              (char *)(tight + 1) - (char *)tight);
```

5 往右看是 [3]，在读一个数组
6 往右是 [3][8]，是二维数组
7 每行只有 5 个字符
16 数组地址 vs 它存的地址
18 两个地址是同一处
21 三种「走一格」各走多少

真跑出来：

终端输出 终端输出 array-of-ptr-out

```
sizeof(names) = 24, sizeof(names[0]) = 8 (一格是一个指针)
sizeof(rows)  = 24, sizeof(rows[0])  = 8 (一格是八个字符)
sizeof(tight) = 15, sizeof(tight[0]) = 5
names 的数组地址与它存的地址相同吗：不是
rows 的数组地址与第一行的地址相同吗：是
往前走一格：names + 1 走 8 字节，rows + 1 走 8 字节，tight + 1 走 5 字节
  函数里 sizeof(names) = 8, sizeof(rows) = 8 (都只剩一个指针)
三个名字一共占 20 个字符（连 '\0'），它们在 names 数组之外
  names[0] = "ada"
  names[1] = "bob"
  names[2] = "christopher"
  rows[0] = "ada"
  rows[1] = "bob"
  rows[2] = "erin"
sizeof(char (*)[3]) = 8 (数组指针本身就是一个指针)
```

输出里有两处**巧合**，先点破，免得你把它们当成规律：

- `sizeof(names)` 是 **24**，`sizeof(rows)` 也是 **24**——因为本机的指针正好 8 字节，3 × 8 = 24，而 `rows` 是三行八字节，3 × 8 = 24。**两个 24 是完全不同的两件事**：前者是三个地址条，后者就是那 24 个字符本身；
- `names + 1` 走 **8** 字节，`rows + 1` 也走 **8** 字节——第一个 8 是「一格指针」的大小，第二个 8 是「一行」的大小（一行正好八个 `char`）。

输出的最后一行还给了数组指针的读数：`sizeof(char (*)[3]) = 8`——`char (*a)[3]` **这个类型本身就是一个指针**，所以它和 `char *` 一样大；那个 `3` 只影响 `a + 1` 走多远。

`tight[3][5]` 就是为了把第二个巧合戳破而存在的：它的 `sizeof` 是 **15**（3 × 5），`tight + 1` 走 **5** 字节。**指针数组的步长永远是指针的大小，二维数组的步长才是行长**——数字一换就看得出来了。

真正区分这两种写法的，是输出里那两行判断：`names` 的两者**不是**同一个（字符在别处），`rows` 的两者**是**同一个（字符就在数组里）。

第 1、2 帧是这两种写法的全部区别：`names` **里装的是地址，字符在别处；** `rows` **里装的就是字符。**第 3、4 帧是代价与好处：`rows[i]` 一行只有 8 字节，名字长了装不下（`"christopher"` 需要 12 字节）；`names[i]` 一格只有 8 字节，但**它指过去的地方可以有多长就有多长。**

这两种写法一旦进了函数参数，就都只剩一个指针——编译器会亲口告诉你这件事：

c examples/ch04/array_of_ptr/src/array_of_ptr.c 第 17–20 行

```c
void show_param_sizes(char **names, char rows[][8]) {
    printf("  函数里 sizeof(names) = %zu, sizeof(rows) = %zu (都只剩一个指针) \n", 这一行在参数里问 sizeof，编译器要来提醒你
           sizeof names, sizeof rows);
}
```

18 这一行在参数里问 sizeof，编译器要来提醒你

终端会话 编译器替你确认：参数里的 rows 是 char (*)[8] ch04-decay

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c
src/array_of_ptr.c -o app.exe
src/array_of_ptr.c: In function 'show_param_sizes':
src/array_of_ptr.c:19:33: warning: 'sizeof' on array function parameter 'rows' will return size of 'char (*)
[8]' [-Wsizeof-array-argument]
   19 |            sizeof names, sizeof rows);
      |                                 ^~~~
src/array_of_ptr.c:17:42: note: declared here
   17 | void show_param_sizes(char **names, char rows[][8]) {
      |                                     ~~~~~^~~~~~~~~
$ rm -f app.exe
```

那句 `will return size of 'char (*)[8]'` 最值钱：`char (*)[8]` 读作「一个指针，它指向八个 char 的数组」，也就是**数组指针**。所以「参数里的 `char rows[][8]`」与「调用者那边的 `char rows[3][8]`」**不是同一个类型**：前者是一个指针（8 字节），后者是整块内存（24 字节）。函数里那个 `sizeof(rows) = 8` 就是这件事的直接读数。

顺带一个边界：`rows + 1` 合法，是因为 `rows` 退化成指向**第一行**的指针，而它的「下一格」正好是第二行——都在同一个数组里（6.5.6 节第 8 段）。**而 `names[0]` 与 `names[1]` 指着的字符是两块不相干的内存**，它们之间做减法没有任何意义。

10.6 函数指针：变量里装的是函数地址

先把结论说掉：**函数也住在内存里，所以函数也有地址。**函数指针就是一个装着这种地址的变量——上一单元那句「指针是一个变量，里头装的是地址」一个字都不用改。它在工程里有两类用处，本质相同（都是「把函数当数据存起来」）：**回调**（交给别人，在合适时机调用你，10.7 节的 `qsort`）与**分发表**（按名字排成表，运行时查表，本节）。

这份分发表例子把「声明、赋值、调用」摆在一起。头文件前一半是类型定义，后一半是表的一行：

c examples/ch04/funcptr_dispatch/include/funcptr_dispatch.h 第 6–17 行

```c
/* binop_fn 是一个类型名：指向「两个 int 进、一个 int 出」的函数 */
typedef int (*binop_fn)(int, int); 读法：f 是一个指针

int op_add(int a, int b);
int op_sub(int a, int b);
int op_mul(int a, int b);

/* 分发表的一行：一个名字 + 一个函数地址 */
typedef struct { 一行是一格
    const char *name; 名字
    binop_fn fn; 函数地址
} op_entry;
```

7 读法：f 是一个指针
14 一行是一格
15 名字
16 函数地址

第 7 行那个 `typedef` 是这一节唯一需要背下来的形状。**读法**：从 `binop_fn` 出发，先往右——撞上 `)`，按「遇到括号就掉头」的规则转回左，看到 `*`——「它是一个指针」；再往右看 `(int, int)`——「它指向一个接受两个 int 的函数」；最后往左看 `int`——「那个函数返回 int」。合起来：**`binop_fn` 是指向「两个 int 进、一个 int 出」的函数的指针。**

实现里三件事各一行：函数定义（第 4 到 6 行）、按名字查表（返回**指向表里那一行的指针**）、通过函数指针调用：

c examples/ch04/funcptr_dispatch/src/funcptr_dispatch.c 第 4–19 行

```c
4  int op_add(int a, int b) { return a + b; } 三个普通函数，函数名就是地址
5  int op_sub(int a, int b) { return a - b; }
6  int op_mul(int a, int b) { return a * b; }
7
8  const op_entry *find_op(const op_entry *table, size_t n, const char *name) {
9      for (size_t i = 0; i < n; i++) {
10         if (strcmp(table[i].name, name) == 0) {
11             return &table[i]; 返回表里那一行的地址
12         }
13     }
14     return NULL;
15 }
16
17 int apply(const op_entry *entry, int a, int b) {
18     return entry->fn(a, b);            /* 通过函数指针调用 */ 通过函数指针调用
19 }
```

4 三个普通函数，函数名就是地址
11 返回表里那一行的地址
18 通过函数指针调用

`main` 里的表就是三行花括号，每一行把一个名字和一个函数名配起来：

c examples/ch04/funcptr_dispatch/src/main.c 第 4–16 行

```c
4  static const op_entry table[] = {
5      {"add", op_add}, 函数名直接当函数地址用
6      {"sub", op_sub},
7      {"mul", op_mul},
8  };
9
10 int main(void) {
11     printf("sizeof(binop_fn) = %zu, sizeof(&op_add) = %zu (函数地址也是 8 字节) \n",
12            sizeof(binop_fn), sizeof(&op_add));
13
14     binop_fn f = op_add;               /* 函数名就是函数地址 */ 声明读法：f 是一个指针
15     printf("f = op_add: f(7, 5) = %d, (*f)(7, 5) = %d, f == op_add = %s\n", 两种调用写法，结果一样
16            f(7, 5), (*f)(7, 5), (f == op_add) ? "是" : "不是");
```

5 函数名直接当函数地址用
14 声明读法：f 是一个指针
15 两种调用写法，结果一样

真跑出来：

终端输出 终端输出 funcptr-dispatch-out

```
sizeof(binop_fn) = 8, sizeof(&op_add) = 8 (函数地址也是 8 字节)
f = op_add: f(7, 5) = 12, (*f)(7, 5) = 12, f == op_add = 是
mul(7, 5) = 35
sub(7, 5) = 2
找不到 "div"：分发表里没有这一行
```

前两行把「函数指针也是指针」钉住了：`sizeof(binop_fn)` 与 `sizeof(&op_add)` 都是 **8**，和 `int *`、`char *` 一样大。而 `f(7, 5)` 与 `(*f)(7, 5)` **都是 12**——C 特地把 `*` 与 `&` 都做成可省的：`&` 见 6.3.2.1 节第 4 段（函数指示符会自动转成指向函数的指针），`*` 见 6.5.3.2 节第 4 段（解引用函数指针得到函数指示符）。

后半段是分发表真正的好处：**名字是运行期的数据，不是编译期的语法。**想加一个操作，只要在表里多写一行，一个 `if` 都不用改：

c examples/ch04/funcptr_dispatch/src/main.c 第 18–26 行

```c
    const char *want[3] = {"mul", "sub", "div"};
    for (size_t i = 0; i < 3; i++) {
        const op_entry *e = find_op(table, sizeof table / sizeof table[0], want[i]); 表的大小用 sizeof 算出来，不写死
        if (e == NULL) {
            printf("找不到 \"%s\"：分发表里没有这一行\n", want[i]);
        } else {
            printf("%s(7, 5) = %d\n", e->name, apply(e, 7, 5)); 查到了就调那一行
        }
    }
```

20 表的大小用 sizeof 算出来，不写死
24 查到了就调那一行

输出的后三行就是这次查表的结果：`mul(7, 5) = 35`、`sub(7, 5) = 2`、`找不到 "div"：分发表里没有这一行`——**没查到的路也要走一遍**，因为分发表的接口（`find_op` 找不到返回 `NULL`）和 10.2 节的「判空」是同一条规矩。

函数指针变量里到底装了什么，一张图说清：

内存形态 **函数指针变量里的那个地址**

第 1 步 函数也有地址，而且地址也是 8 个字节

`op_add` ：一段代码，住在只读的那块内存里

`&op_add` ：它的地址，本机 8 个字节

`sizeof(binop_fn) = 8` ：与 `int *` 一样大

第 2 步 `binop_fn f = op_add;` —— 变量里装的是那个地址

`f` ：一格八字节的变量

`f == op_add` ：真，因为里面就是同一个地址

`f` → `op_add` 箭头指着那段代码

第 3 步 f(7, 5) 与 (*f)(7, 5) 是同一件事

`f(7, 5)` ：顺着 `f` 里的地址调用

`(*f)(7, 5)` ：先解引用再调用，结果一样

两次都返回 `12` ：加法是 `op_add` 干的

第 4 步 分发表就是「一格一格的函数地址」

`table[0]` ：`{"add", op_add}`

`table[1]` ：`{"sub", op_sub}`

`table[2]` ：`{"mul", op_mul}`

第 1、2 帧是「函数指针也是指针」这句话的全部内容：每个函数名的值就是它那段代码的地址（本机 8 字节）。第 3 帧解释为什么要认两种调用写法：它们等价，你在别人的代码里两种都会看到。第 4 帧是分发表：**一张表把「名字」和「要调的那段代码」配起来。**

{} 语法规则 · 函数指针的声明：括号不能省

形状是 `返回类型 (*变量名)(参数类型列表)`。三个位置各管一件事：

- **`(*变量名)` 的括号不能省。** `int *f(int, int)` 读作「`f` 是一个函数，接受两个 int，返回 `int *`」；`int (*f)(int, int)` 读作「`f` 是一个指针，指向接受两个 int、返回 int 的函数」。**区别只在那一对括号**，而它们是两个完全不同的东西；
- 参数列表**只写类型就够了**（`int (*f)(int, int)`），带参数名也行（`int (*f)(int a, int b)`）；
- 觉得难读就用 `typedef` 起个名字（本节的 `binop_fn`），之后 `binop_fn f = op_add;` 就一眼能读了——**工程代码里 typedef 是常态**，因为回调的签名往往很长。

赋值与调用各有一对等价写法：`f = op_add;` 与 `f = &op_add;`；`f(7, 5)` 与 `(*f)(7, 5)`。

> ⚠ **陷阱** · **`int *f(int, int)` 与 `int (*f)(int, int)`：少一对括号，意思全反**
>
> **你会怎么被坑到**：想写一个「返回 int 的函数指针」，结果写成了 `int *f(void)` ——编译器认为你在**声明一个函数**，它返回 `int *`。于是你在函数体里写 `return op_add;`，本机报的是 `error: returning 'int (*)(int, int)' from a function with incompatible return type 'int *' [-Wincompatible-pointer-types]`（gcc，`-std=c17`）。报错指向的是 `return` 那一行、不是声明本身——你很容易跑去改 `op_add`，改半天才发现错在签名。
>
> **真相**：C 的声明符规则是「从名字出发、先右后左」（6.7.6 节），`*` 与 `()` 的**先后由括号决定**：没有括号时 `f` 先与 `(int, int)` 结合，`*` 只好去管返回值；有括号时 `f` 先与 `*` 结合，成了指针。
>
> **怎么查**：往这行声明里塞一个故意错的类型赋值（比如 `binop_fn g = 1;`），让编译器把它「理解成了什么」打给你看——报错里会带回推出来的类型，比你自己读快得多。

10.7 回调：`qsort` 为什么能排任何东西

标准库的 `qsort` 是个通用排序：它**不认识你的类型**，却能把你的数组排好。它做到这一点的办法，是把「两个元素怎么比大小」这件事**交还给你**——你交一个**比较函数的地址**给它，它在需要比较的时候回调你。

标准对它的签名与要求只有这么几行：

> 🕮 **标准原文** · **标准原文：`qsort` 的签名**
>
> 7.22.5.2 节「The qsort function」的 Synopsis：
>
> > void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));
>
> 大意：四个参数依次是「数组首地址」「元素个数」「一个元素占几个字节」「比较函数的地址」。**前三个里没有一个带类型**，类型信息是靠后两个数字补出来的。
>
> 出处：ISO/IEC 9899:2018（C17）§7.22.5.2 第 1 段。

> 🕮 **标准原文** · **标准原文：比较函数的约定**
>
> 同一节第 2、3 段：
>
> > The qsort function sorts an array of nmemb objects, the initial element of which is pointed to by base. The size of each object is specified by size.
> >
> > The contents of the array are sorted into ascending order according to a comparison function pointed to by compar, which is called with two arguments that point to the objects being compared. The function shall return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.
>
> 大意：`qsort` 排的是 `base` 指向的 `nmemb` 个对象，每个对象占 `size` 个字节；排序顺序**由你交的那个比较函数说了算**，它每次被调用时拿到**两个指向待比较对象的指针**，返回值小于零 / 等于零 / 大于零分别表示「前一个小 / 相等 / 前一个大」。第 4 段则把「相等的元素怎么办」写明了：
>
> > If two elements compare as equal, their order in the resulting sorted array is unspecified.
>
> 大意：相等元素的先后**未指定**——所以别指望它是稳定排序。
>
> 出处：ISO/IEC 9899:2018（C17）§7.22.5.2 第 2、3、4 段。

你的活儿就是写出那个比较函数，**签名一个字都不能差**：

c examples/ch04/qsort_callback/include/qsort_callback.h 第 6–11 行

```c
/* 签名必须与 qsort 要的一模一样：int (*)(const void *, const void *) */
int cmp_int_asc(const void *a, const void *b); 排 int 的比较函数
int cmp_str(const void *a, const void *b); 排字符串数组的比较函数

void print_ints(const int *a, size_t n);
```

```
11  void print_strs(char **a, size_t n);
```

7 排 int 的比较函数
8 排字符串数组的比较函数

第一个 `return` 的写法值得抄：`(x > y) - (x < y)` 一口气给出 `-1 / 0 / 1` 三态，比写三个 `if` 短，也不会漏掉相等的情况：

c examples/ch04/qsort_callback/src/qsort_callback.c 第 5–15 行

```c
int cmp_int_asc(const void *a, const void *b) {
    int x = *(const int *)a;          /* void * 不带类型，先转回 int * */ void * 先转回 int *
    int y = *(const int *)b;
    return (x > y) - (x < y);         /* 负 / 0 / 正，正是 qsort 要的三态 */ 三态一次算出来
}

int cmp_str(const void *a, const void *b) {
    const char *const *pa = (const char *const *)a;   /* 元素本身是 char * */ 元素本身是 char *，所以还要再一层
    const char *const *pb = (const char *const *)b;
    return strcmp(*pa, *pb);
}
```

6 void * 先转回 int *
8 三态一次算出来
12 元素本身是 char *，所以还要再一层

`main` 里把比较函数的地址交给 `qsort`——**注意第 9 行那个变量**，它就是「函数指针」在本节的落点：

c examples/ch04/qsort_callback/src/main.c 第 5–15 行

```c
int main(void) {
    int nums[7] = {42, 7, 19, 3, 88, 1, 19};
    char *words[5] = {"pear", "apple", "fig", "banana", "date"};

    int (*cmp)(const void *, const void *) = cmp_int_asc;   /* 括号里的 * 先结合 */ 一个函数指针变量
    int (*str_cmp)(const void *, const void *) = cmp_str;

    printf("元素个数 = %zu，一个元素 %zu 字节\n", sizeof nums / sizeof nums[0], sizeof nums[0]); 元素个数与元素大小
    printf("排序前：");
    print_ints(nums, 7);
    qsort(nums, 7, sizeof nums[0], cmp); 把函数指针交出去
```

9 一个函数指针变量
12 元素个数与元素大小
15 把函数指针交出去

真跑出来：

终端输出 终端输出 qsort-callback-out

```
元素个数 = 7，一个元素 4 字节
排序前：  42 7 19 3 88 1 19
排序后：  1 3 7 19 19 42 88
字符串数组：一个元素 8 字节（那是指针）
排序前：  pear apple fig banana date
排序后：  apple banana date fig pear
```

六行输出读三组：

- **`元素个数 = 7，一个元素 4 字节`**：`sizeof nums / sizeof nums[0]` 是数元素个数的标准写法（第 8 单元讲过）；`qsort` 要的正是这两个数——**它只认字节，不认类型**；
- **`42 7 19 3 88 1 19` → `1 3 7 19 19 42 88`**：排序真的发生了。原数组里有两个 `19`，输出里也是两个；
- **字符串那一组**：`pear apple fig banana date` → `apple banana date fig pear`，**但元素大小那一行是 8 字节**——因为 `words` 是一个 `char *` 数组，一个元素就是**一个指针**。所以 `cmp_str` 拿到的 `const void *` 指向的是

一个 `char *`，必须转成「指向 `char *` 的指针」（代码里写的是 `const char *const *`）再解引用去 `strcmp`：**这就是「元素类型」这件事最容易被绊倒的地方。**

回调发生时，两边的世界长什么样，用一张卡片图看清楚：

第 1 帧是「交接」：`main` 交给 `qsort` 的是数组首地址、元素个数、元素大小与**函数地址**。第 2 帧是关键——**`qsort` 不知道你在排 `int` 还是 `char *`**，它只有 `void *base` 和一个字节数，要比较时就把两个元素的地址回调给你的 `cmp_int_asc`。第 3 帧是协作成立的原因：**签名必须完全一致**（6.3.2.3 节第 8 段：转成不兼容类型再调用是 UB）。

回调**真的发生**了这件事，得让调试器来作证。下面这段会话停在 `cmp_int_asc` 的第 7 行，然后做三件事：打出这个函数的地址、打调用栈、回到 `main` 那一帧看看 `cmp` 变量里装的是什么。

终端会话 gdb：p cmp、bt，以及回调发生时谁调用了谁 ch04-gdb-funcptr

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c
src/qsort_callback.c -o app.exe
$ gdb --batch -ex "break src/qsort_callback.c:7" -ex run -ex "p cmp_int_asc" -ex "bt" -ex "frame 2" -ex "p
cmp" ./app.exe
Breakpoint 1 at 0x140001649: file src/qsort_callback.c, line 7.
[New Thread 92476.0x15db0]
Thread 1 hit Breakpoint 1, cmp_int_asc (a=0x5ffe84, b=0x5ffe80) at src/qsort_callback.c:7
7           int y = *(const int *)b;
$1 = {int (const void *, const void *)} 0x7ff6303a1630 <cmp_int_asc>
#0  cmp_int_asc (a=0x5ffe84, b=0x5ffe80) at src/qsort_callback.c:7
#1  0x00007ffb96b5aac4 in ucrtbase!qsort () from C:\WINDOWS\System32\ucrtbase.dll
#2  0x00007ff6303a1593 in main () at src/main.c:15
#2  0x00007ff6303a1593 in main () at src/main.c:15
15          qsort(nums, 7, sizeof nums[0], cmp);
$2 = (int (*)(const void *, const void *)) 0x7ff6303a1630 <cmp_int_asc>
$ rm -f app.exe
```

这份会话里有四样东西，一样一样对：

- **`$1 = {int (const void *, const void *)} 0x7ff6303a1630 <cmp_int_asc>`**：这是 `cmp_int_asc` 这个**函数**的地址，gdb 连它的类型和名字一起打给你了；
- **`bt` 的三帧**：`#0 cmp_int_asc`（你停住的地方）、`#1 ucrtbase!qsort`（调用者**住在运行库的 DLL 里**）、`#2 main`（源头）。**回调不是一个说法，是一条真的调用链**——你在自己的源码里永远搜不到「谁调用了 `cmp_int_asc`」，因为那是运行库在调用它；
- **`p cmp` 打出来 `0x7ff6303a1630 <cmp_int_asc>`：与 `$1` 是同一个地址。**这就把「函数指针变量里装的是函数地址」从一句话变成了一个读数；
- 那行 `#2 … in main () at src/main.c:15` 里的 **15**，正是 `qsort(nums, 7, sizeof nums[0], cmp);` 那一行——`main` 在第 15 行把控制权交出去，之后就一直在等 `qsort` 返回。

同一件事画成调用树，就是这一节最后一张图：

调用树 **回调发生时的调用树（对着 bt 的三帧画）**

第 1 步 **main 在第 15 行调用 qsort**

```
main() ：src/main.c:15，手里握着 nums 与 cmp
```

第 2 步 **qsort 住在本机的运行库里**

```
main() ：还在第 15 行等结果
qsort() ：ucrtbase.dll 里的实现
main() → qsort() 调用
```

第 3 步 **排序算法内部回调你的比较函数**

```
qsort() ：每需要比较两个元素就调一次
cmp_int_asc() ：src/qsort_callback.c:7，真的进来了
qsort() → cmp_int_asc() 回调
```

第 4 步 **gdb 的 bt 把这条链原样打出来**

```
cmp_int_asc() ：#0
qsort() ：#1，ucrtbase!qsort
main() ：#2，src/main.c:15
```

第 1、2 帧是上下关系（`main` 等、`qsort` 干活），第 3 帧是这一节的中心：**`qsort` 与 `cmp_int_asc` 之间没有一行直接的调用代码**——`cmp_int_asc` 从来没有出现在 `qsort` 的源码里，它是通过参数里那个地址被调用的。第 4 帧是 `bt` 的三行输出，前三帧是这条链在故事里的样子。

工程惯例 · **工程惯例：比较函数怎么写才不容易错**

`qsort` 的契约只有「三个返回值方向」，所以比较函数的写法有自己的惯例：

- **别写 `return a - b;`**：两个 `int` 相减可能溢出（上一单元讲过的有符号溢出 UB），排一个「大正数在前」的数组时就会翻车。用 `(x > y) - (x < y)` 或者三个 `if`；
- **元素类型是「指针」时，参数多一层**：排 `int` 数组时 `const void *` 指向 `int`，转 `const int *` 就够；排 `char *` 数组（也就是字符串数组）时它指向**一个指针**，要转成 `const char *const *` 再解引用。本节 `cmp_str` 那两行就是这件事；
- **不要在比较函数里改数据、也不要 `free`**：`qsort` 会在它认为合适的时候调用你，你可能被调用的次数**没有保证**（同一对元素比较两次是允许的）。

代价说清：这几条都不是标准强制的（标准只说返回值的方向），但各自对应一个真实的踩坑现场。

10.8 `void *` 与接口上的数组：为什么签名长成那样

回头看两个标准库函数的签名，现在应该能读出设计意图了：

- `void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));`
- `void *memcpy(void *restrict s1, const void *restrict s2, size_t n);`

它们都用 `void *` 收指针、都用**额外的两个数字**（元素大小 + 个数，或者字节数）补上类型信息丢掉的那部分。原因只有一个：**这两个函数要对「任何类型」都管用。** `void *` 是 C 里唯一的通用指针——标准 6.3.2.3 节第 1 段给它的承诺很小，就一句话：

标准原文 · **标准原文：`void *` 能干什么**

6.3.2.3 节「Pointers」第 1 段：

> A pointer to void may be converted to or from a pointer to any object type. A pointer to any object type may be converted to a pointer to void and back again; the result shall compare equal to the original pointer.

大意：**任何「指向对象」的指针都能转成 `void *`、也能转回来，转回来之后与原指针相等。**标准就承诺了这么多——它没说 `void *` 能解引用，也没说能对它做算术。

出处：ISO/IEC 9899:2018（C17）§6.3.2.3 第 1 段。

`void *` 的这两条限制不是设计缺陷，而是**信息不足的必然结果**：编译器不知道它指向什么类型，就不知道一次该读写几个字节、一步该走多远。本机把这两件事的实际后果摆在这里：

编译器视角 · 编译器视角：`void *` 的两条限制在本机长什么样

拿一个 `void *p` 分别写 `int x = *p;` 与 `void *q = p + 1;`，用本书的固定参数（`-std=c17 -Wall -Wextra -Wpedantic -O0 -g`）编译：

- 第一行：`warning: dereferencing 'void *' pointer` 加 `error: void value not ignored as it ought to be`，**编译退出码 1**——`*p` 的结果是 void，不能当值用；
- 第二行：`warning: pointer of type 'void *' used in arithmetic [-Wpointer-arith]`，**编译退出码 0**——本机只给警告，照样编过。

大意：解引用那条编不过；**算术那条能编过，但那不等于合法。**标准 6.5.6 节第 2 段要求参与加法的是「指向**完整对象类型**的指针」，而 `void` 是「不能补全的不完整类型」（6.2.5 节第 19 段）——这是约束违反，实现本该给诊断；GCC 把它当扩展放行（按字节走），你不该依赖。

这也解释了 `qsort` / `memcpy` 为什么非得带数字参数：类型信息被 `void *` 抹掉之后，**只剩字节数能告诉函数「搬多少」**。所以在 `cmp_int_asc` 里必须写 `*(const int *)a`——**这一步转换就是「把类型信息交回来」的地方。**上一单元那句话在这里再成立一次：一次读几个字节、对齐的约束，全定在这句转换里。

工程惯例 · 工程惯例：接口上写 `T *` + 长度，还是写 `T a[]`

标准对函数参数有一条明确的规则：**形参里的 `T a[]` 与 `T a[N]` 都会被改写成 `T *a`**（6.7.6.3 节），`N` 那个数字**完全不参与类型。**本节的接口分成两类，区别只在「长度从哪来」：

- **以 `'\0'` 收尾的字符串**不需要长度：`count_matches(const char *text, char target)`（const_params）靠 `'\0'` 就知道读到哪；
- **其余数组与缓冲区一律 `T *` + 长度**：`chars_in(char **names, size_t n)`（array_of_ptr）、`uppercase_into(char *out, size_t cap, const char *src)`（const_loss）

具体做法就两条：

- **一律写 `T *`，长度单独一个参数**（`size_t n` 或 `size_t cap`）。理由：`T a[]` 与 `T *` 是同一个类型，写成前者只会让人误以为「长度传进来了」；
- **长度参数放指针后面**：`(const T *a, size_t n)` 这个顺序在标准库里随处可见，一眼就能看出它们是配对的。

工程惯例 · （续）`cap` 与 `n` 别混，方括号里的数字不是长度

同一个位置上的 `size_t`，在输入型与输出型接口里含义不同：

- **`cap`（容量）**：输出型接口收的是「你最多能写多少」（含结尾 `'\0'` 的位置），本节的 `uppercase_into(char *out, size_t cap, …)` 就是它；
- **`n`（实际长度）**：输入型接口收的是「这里面有几个元素」，`chars_in(char **names, size_t n)` 就是它；
- **方括号里的数字是类型的一部分**，与「有几个元素」无关：`char rows[][8]` 里那个 `8` 是**行宽**，元素个数永远靠参数（`size_t n`）。

代价：多一个参数、多一处可能写错的数（`sizeof` 与真实长度对不上是经典 bug，10.11 节会列）；好处是**函数里唯一能知道长度的办法就是这个参数**——上一单元的 `sizeof_in_function` 实测过：函数里的 `sizeof(a)` 只剩 8。

10.9 `restrict`、`memcpy` 与重叠：结果一样，承诺不一样

这一节最反直觉：**同一件事，两个函数的实测结果一模一样，其中一个却是未定义行为。**三个函数，差别只有一个词：

c examples/ch04/memcpy_overlap/src/memcpy_overlap.c 第 5–18 行

```c
void copy_two_buffers(void) {
    char src[8] = "abcdefg";
    char dst[8] = {0};
    memcpy(dst, src, sizeof dst);      /* 两块内存不重叠：这是 memcpy 该干的活 */ 两块不重叠，随便搬
    printf("两块不重叠的内存：dst = \"%s\"\n", dst);
}
```

```
11
12  void shift_with_memcpy(char *buf) {
13      memcpy(buf + 2, buf, 8);          /* 源与目标重叠：标准没承诺这一段 */ 源与目标重叠了
14  }
15
16  void shift_with_memmove(char *buf) {
17      memmove(buf + 2, buf, 8);         /* 同一件事，memmove 保证对 */ 同一件事，memmove 保证对
18  }
```

8 两块不重叠，随便搬
13 源与目标重叠了
17 同一件事，memmove 保证对

这份程序真跑出来的 stdout 是下面这一块（由流水线逐字对账），它一共五行、退出码 0：

终端输出 终端输出 memcpy-overlap-out

```
两块不重叠的内存：dst = "abcdefg"
搬之前         : "0123456789"
memcpy 搬完后  : "0101234567"
memmove 搬完后: "0101234567"
两次结果一样吗：一样
```

编译它，编译器在**基础参数下**就给了一条警告——这句话值得逐字读（会话原文）：

终端会话 -Wrestrict：编译器在编译期就看出这两块内存重叠了 ch04-restrict

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c
src/memcpy_overlap.c -o app.exe
src/memcpy_overlap.c: In function 'shift_with_memcpy':
src/memcpy_overlap.c:13:5: warning: 'memcpy' accessing 8 bytes at offsets 2 and 0 overlaps 6 bytes at offset
2 [-Wrestrict]
   13 |     memcpy(buf + 2, buf, 8);          /* 源与目标重叠：标准没承诺这一段 */
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
$ ./app.exe; echo "退出码 $?"
两块不重叠的内存：dst = "abcdefg"
搬之前         : "0123456789"
memcpy 搬完后  : "0101234567"
memmove 搬完后: "0101234567"
两次结果一样吗：一样
退出码 0
$ rm -f app.exe
```

会话里有三件事，一件比一件要紧：

1. `-Wrestrict` **那句话把「重叠了几个字节」都算出来了**：`accessing 8 bytes at offsets 2 and 0 overlaps 6 bytes at offset 2` ——目标从偏移 2 开始、搬 8 个字节，源从偏移 0 开始，两者重叠了 6 个字节。编译器没跑你的程序，它是从这两行代码的地址关系**推**出来的；
2. `memcpy 搬完后` **与** `memmove 搬完后` **打出来都是** `0101234567`，连最后那行 `两次结果一样吗：一样` 都是程序自己判的。**在这台机器上，这两种写法看不出任何差别**；
3. **退出码是 0**，程序干干净净跑完。

那为什么还要用 `memmove`？因为决定这件事的是**标准**，不是这一次的运行结果：

标准原文 · **标准原文：`memcpy` 不谈重叠**

> 7.24.2.1 节「The memcpy function」第 2 段：
>
> > The memcpy function copies n characters from the object pointed to by s2 into the object pointed to by s1. If copying takes place between objects that overlap, the behavior is undefined.
>
> 大意：`memcpy` 从 `s2` 指向的对象复制 `n` 个字符到 `s1` 指向的对象；**如果复制发生在重叠的对象之间，行为未定义。**
>
> 出处：ISO/IEC 9899:2018（C17）§7.24.2.1 第 2 段（原文抄自 WG14 公开的委员会草案 N1570；本机没有正式版正文的可读副本）。

标准原文 · **标准原文：`memmove` 明确谈重叠**

7.24.2.2 节「The memmove function」第 2 段：

> The memmove function copies n characters from the object pointed to by s2 into the object pointed to by s1. Copying takes place as if the n characters from the object pointed to by s2 are first copied into a temporary array of n characters that does not overlap the objects pointed to by s1 and s2, and then the n characters from the temporary array are copied into the object pointed to by s1.

大意：`memmove` 做同一件事，但**它被要求「好像先把那 n 个字符搬进一个不与两边重叠的临时数组，再搬过去」**——这句话就是为了让重叠的搬迁有确定的语义而写的。

出处：ISO/IEC 9899:2018（C17）§7.24.2.2 第 2 段。

还有第三个词要交代：那两个签名里的 `restrict`。它不是装饰——它是一条**你向编译器作出的承诺**：

标准原文 · **标准原文：`restrict` 是一次承诺**

6.7.3 节「Type qualifiers」第 8 段：

> An object that is accessed through a restrict-qualified pointer has a special association with that pointer. This association, defined in 6.7.3.1 below, requires that all accesses to that object use, directly or indirectly, the value of that particular pointer. The intended use of the restrict qualifier (like the register storage class) is to promote optimization, and deleting all instances of the qualifier from all preprocessing translation units composing a conforming program does not change its meaning (i.e., observable behavior).

大意：`restrict` 意味着「这个对象的所有访问，都通过这一个指针走」——**它是给优化器的许可**；标准还明说：在一个合规程序里把所有 `restrict` 删掉，**可见行为不会变。**

出处：ISO/IEC 9899:2018（C17）§6.7.3 第 8 段。

标准原文 · **标准原文：违反那条承诺的代价**

6.7.3.1 节「Formal definition of restrict」第 4 段最后一句：

> If these requirements are not met, then the behavior is undefined.

同一节第 9 段把好处与代价写透了：

> The benefit of the restrict qualifiers is that they enable a translator to make an effective dependence analysis of function f without examining any of the calls of f in the program. The cost is that the programmer has to examine all of those calls to ensure that none give undefined behavior.

大意：不满足那些要求，**行为未定义**；好处是编译器不用看调用点就能做依赖分析，**代价是每处调用都得由你保证不踩 UB。**

出处：ISO/IEC 9899:2018（C17）§6.7.3.1 第 4、9 段。

把这几段话摆在一起：`memcpy` 的签名里写着 `restrict`，而**重叠正是对那条承诺最直接的违反**——那一行同时踩了「违反 `restrict`」与「重叠时 UB」两件事。本机这次两个函数结果一样，只是**这一次的实现选择**：`memcpy` 内部按什么顺序搬，标准没规定（我也没反汇编核对），能确定的只是「这一次恰好对」。换个长度、档位或运行库，它完全可能给出别的内容。

> ⚠ **陷阱 · 「我试过了，结果是对的」——这句话不能当证据**
>
> **你会怎么被坑到**：你把重叠的搬迁写成 `memcpy`，跑了一遍、结果正确，于是这件事就过去了。半年后换了平台或编译器版本，或者只是把字节数从 8 改成 12，同一行开始给出**别的字符串**——不崩、不报错、不打日志，你只能对着「内容不对」发呆。
>
> **你还会怎么误判**：看到 `-Wrestrict` 那条警告，以为是「这条警告太严了，加个 `-Wno-restrict` 就干净了」。真相是：**警告在描述一个真实的不确定点**，压掉它只是把唯一的提示也删了（顺带一句：本书的清单不许写 `-Wno-*`，闸门会报 `flag_not_allowed`）。
>
> **你会怎么被坑到**：你把重叠的搬迁写成 `memcpy`，跑了一遍、结果正确，于是这件事就过去了。半年后换了平台、换了编译器版本，或者只是把字节数从 8 改成 12，同一行代码开始给出**别的字符串**——它不崩、不报错、不打日志，你只能对着一个「内容不对」的结果发呆。
>
> **怎么查**：看到 `memcpy` 的两个指针之间有加减、有同一个数组里的偏移，就停下来问一句「它们会不会重叠」——会重叠就换 `memmove`，没有任何理由赌一把。

真相：UB 的含义不是「一定会崩」，而是「**谁都不欠你一个确定的结果**」。这次的 `0101234567` 只证明「本机这一次恰好这样」，不证明这段代码是对的。

10.10 指针当整数用：`sizeof`、`%p` 与 `uintptr_t`

指针的大小是**实现事实**，不是标准数字。这一节把本机的事实全打出来了：

c examples/ch04/ptr_int/src/main.c 第 5–22 行

```c
int main(void) {
    int a[4] = {10, 20, 30, 40};
    int *p = &a[1];
    int *q = &a[3];
    uintptr_t up = (uintptr_t)p;       /* 地址装进一个整数里 */ 把地址装进一个整数里
    uintptr_t uq = (uintptr_t)q;

    printf("sizeof(int *) = %zu, sizeof(char *) = %zu, sizeof(void *) = %zu\n", 三种对象指针一样大
           sizeof(int *), sizeof(char *), sizeof(void *));
    printf("sizeof(函数指针) = %zu, sizeof(uintptr_t) = %zu\n",
           sizeof(&dummy), sizeof(uintptr_t)); 函数指针也是 8

    printf("p = %p, q = %p (%%p 打出来是十六进制) \n", (void *)p, (void *)q);
    printf("(uintptr_t)p = %" PRIuPTR ", (uintptr_t)q = %" PRIuPTR "\n", up, uq); 同一个地址的两种打印
    printf("整数相减 = %" PRIdPTR " 字节，两个指针相减 = %td 个元素\n",
           (intptr_t)(uq - up), q - p);
    printf("index_of(a, q) = %td, sizeof(q - p) = %zu (那是 ptrdiff_t) \n",
           index_of(a, q), sizeof(q - p));
}
```

9 把地址装进一个整数里
12 三种对象指针一样大
15 函数指针也是 8
18 同一个地址的两种打印

真跑出来：

终端输出 终端输出 ptr-int-out

```
sizeof(int *) = 8, sizeof(char *) = 8, sizeof(void *) = 8
sizeof(函数指针) = 8, sizeof(uintptr_t) = 8
p = 00000000005FFE84, q = 00000000005FFE8C (%p 打出来是十六进制)
(uintptr_t)p = 6291076, (uintptr_t)q = 6291084
整数相减 = 8 字节，两个指针相减 = 2 个元素
index_of(a, q) = 3, sizeof(q - p) = 8 (那是 ptrdiff_t)
```

六行输出里有四组事实：

- **`sizeof(int *) = 8, sizeof(char *) = 8, sizeof(void *) = 8`**：本机所有**对象指针**都是 8 字节（64 位）——指针的大小与它指的东西无关，这个数字是这台机器与这个编译目标的实现事实；

- **`sizeof(函数指针) = 8`**：本机的函数指针也正好 8 字节。**但要注意边界**：6.3.2.3 节第 1 段那句「任何指向对象的指针都能与 `void *` 互转」**只覆盖对象指针**——函数指针转 `void *` 并回传，标准没有承诺（本机做得到，只是因为函数与数据在同一套地址空间）。这条是**本机事实**，不是定律；
- **`p = 00000000005FFE84` 与 `(uintptr_t)p = 6291076`**：**同一个地址的两种打印。** `%p` 打的是十六进制（本机带前导零），而 `uintptr_t` 是一个**能装下指针的整数类型**（`<stdint.h>` 提供，本机 8 字节无符号），转过去之后就能用 `PRIuPTR` 这种可移植的格式打出来。这行和上一行要是把值拿去比对，你会发现 `0x5FFE84` 正是 6291076；
- **`整数相减 = 8 字节，两个指针相减 = 2 个元素`**：同一对地址，两种减法。`q - p` 是**指针相减**，得到的是**元素个数**（`&a[3]` 与 `&a[1]` 隔了 2 个 `int`）；要字节数，得先把两个指针都转成整数（或者都转成 `char *`）再减。

「指针相减只在同一个数组里才定义良好」出自标准 6.5.6 节第 9 段：

标准原文 · 标准原文：指针相减的合法范围

6.5.6 节「Additive operators」第 9 段：

> When two pointers are subtracted, both shall point to elements of the same array object, or one past the last element of the array object; the result is the difference of the subscripts of the two array elements. The size of the result is implementation-defined, and its type (a signed integer type) is ptrdiff_t defined in the <stddef.h> header. If the result is not representable in an object of that type, the behavior is undefined.

大意：两个指针相减时，**它们必须指向同一个数组对象的元素**（或者最后一个元素的下一格），结果才是「两个下标之差」；结果类型是 `ptrdiff_t`（本机 8 字节的有符号整数），这个类型多宽由实现决定；**连这个差都装不进 `ptrdiff_t` 时，同样是未定义行为。**

出处：ISO/IEC 9899:2018（C17）§6.5.6 第 9 段。

指针的**大小比较**（`<` / `>` / `<=` / `>=`）还有一条更窄的规矩，它和相减是两件事：

标准原文 · 标准原文：指针的大小比较只在同一数组里有定义

6.5.8 节「Relational and equality operators」第 5 段（节选）：

> If two pointers to object types both point to the same object, or both point one past the last element of the same array object, they compare equal. … All pointers to members of the same union object compare equal. … In all other cases, the behavior is undefined.

大意：只有「同一个对象 / 同一个数组里的元素（含最后一格的下一格）」才谈得上谁大谁小；**其余情况行为未定义。**注意这条**不管 `==` / `!=`**：相等比较对任意合法指针都有定义（6.5.9 节第 6 段），比的是「同一个对象、同一个函数，或者都为空」。

出处：ISO/IEC 9899:2018（C17）§6.5.8 第 5 段、§6.5.9 第 6 段。

这条规矩**编译器不会替你守**：我写了一段 `p < q`（`p`、`q` 分别指向两个不相干的数组），用本书的固定参数编，**一条警告都没有**、退出码 0，跑出来是 `0`——那个 `0` 没有任何意义。同样一段代码换成 `<` 比较两个指针再打印，读者只会看到「一个数」，看不出它踩了 UB。

`index_of` 是**相减**那条规则的落地：它把两个指针相减这件事**收进了一个函数**，于是调用者必须保证传进来的是同一个数组里的两个指针——这个保证没法写进类型里，只能写进注释：

c examples/ch04/ptr_int/src/ptr_int.c 第 7–9 行

```
7  ptrdiff_t index_of(const int *base, const int *p) {
8      return p - base;                /* 只在同一个数组里才有意义 */  只在同一个数组里才有意义
9  }
```

8 只在同一个数组里才有意义

工程惯例 · **工程惯例：什么时候才该把指针转成整数**

`uintptr_t`（以及有符号的 `intptr_t`）是「能装下 `void *`、转回来还等于原指针」的整数类型，7.20.1.4 节还写明**这两个类型是可选的**（实现可以不提供）；能用它的场合比新手想的少得多：

- **可以转**：打印、日志、哈希（把地址当不稳定但可比较的键）、对齐检查。本节的 `(uintptr_t)p` 就是第一种；
- **不要拿它做算术再转回指针**：`(int *)((uintptr_t)p + 4)` 在本机能跑，但**标准不保证**这个整数能变回一个合法指针——**指针运算的合法性由 6.5.6 节管（必须落在同一个数组里）**，绕道整数并不会让你逃出那条规则；
- **不要拿它比较大小**：`<` / `>` / `<=` / `>=` 只在「同一个数组对象的元素（或最后一格的下一格）」之间有定义（6.5.8 节第 5 段，其余情况是 UB）；`==` / `!=` 则对任意合法指针都有定义（6.5.9 节第 6 段），比的是「是不是同一个对象 / 同一个函数 / 都为空」。

代价：把地址装进整数，你就**丢掉了类型**，编译器再也帮不了你（这也是 `%p` 要配 `(void *)` 的原因：打印地址本来就要丢掉类型信息）。

⚠ 陷阱 · **拿 `%p` 的输出当数字用**

你会怎么被坑到：你在排错时打了一行 `printf("%p\n", (void *)p);`，看到 `00000000005FFE84`，于是想把它的低几位取出来当标识符用，或者拿两次打印的字符串做比较判断「这两个指针一样不一样」。

真相：`%p` 的输出格式**由实现决定**（标准只说它用来打印指针，没说长什么样）。本机是带前导零的十六进制、没有 `0x` 前缀，别的运行库可能打印 `0x5ffe84` 甚至 `(nil)`。字符串不同不代表指针不同。

怎么查 / 怎么写：要比较两个指针，**直接用** `==`；要当数字用，先转成 `uintptr_t` 再按整数打印（配 `PRIuPTR`）。本书所有打印地址的地方都写 `(void *)p`，理由在上一单元讲过：`%p` 要的就是 `void *`，不写这一句 `-Wformat` 会警告你。

10.11 缺陷清单：指针参数上的四个常见错误，以及一条救不了你的惯例

前面九节的例子合起来，就是这份清单。第三条在本节有实测（`-Werror` 前后两种结果），第一、第四条的证据在上一单元（路径都指到那里），最后那条惯例（`free(p); p = NULL;`）则要特别说清它的**界限**。

缺陷	长什么样	后果（本机实测）	本节的例子
没判空	函数直接用 `*p = …`，接口上不写「会不会收到空指针」	解引用空指针：退出码 3221225477（上一单元实测）	`null_deref`（上一单元）、`out_params` 的 `return -1`
长度与指针不同步	传了指针忘了长度，或者 `sizeof` 在函数里问长度	函数里 `sizeof(参数)` 只剩指针的大小（本机 8），算出来的「元素个数」是错的	`array_of_ptr` 的 `sizeof(rows) = 8`
`const` 丢失	只读的指针交给要写的接口	**默认只是 warning、退出码 0**；加 `-Werror` 才拦得住	`const_loss`
返回局部地址	把函数里局部数组的地址交出去	本机实测：编译器直接把这个返回值改成空指针，调用者拿到 `(null)`	`dangling_stack`（上一单元）

前两条一句话就能记住：**判空写在「知道该怎么处理」的那一层**；**长度与指针永远成对出现**（本书的输出型接口写 `(T *out, size_t cap)`）。第三条是那个不太舒服的事实——**类型系统能管的边界就到警告为止**，工程上靠 `-Werror` 收紧。第四条是这个清单里唯一「编译器会主动救你」的一条：

c examples/ch04/dangling_stack/src/dangling_stack.c 第 5–8 行

```
5  char *greeting_bad(void) {
6      char buf[16] = "hello";
7      return buf;                    /* 函数一返回，buf 的生命就到头了 */ 函数一返回，这块栈内存就不属于你了
8  }
```

7 函数一返回，这块栈内存就不属于你了

上一单元实测过它的两种命运：编译器报 `warning: function returns address of local variable [-Wreturn-local-addr]`，本机还顺手把这个返回值**换成空指针**，调用者拿到 `(null)`（本机实现的选择，不是规律）。**这里补的**

是接口那一半：想交出字符串，正确形状是让**调用者出内存**（`size_t greeting(char *out, size_t cap)`），或者「函数分配、注释写清谁 `free`」。**这件事没有「修一下」的写法，只有换形状。**

工程惯例 · 惯例的界限：`free(p); p = NULL;` 救得了什么

上一单元的 gdb 会话实测过：`free(p)` **不会**动 `p` 里的地址（`free` 前后都是 `0x71cdc0`）。所以「还回去之后立刻置空」成了一条常见惯例。

它救得了什么：`p = NULL` 之后，你再拿 `p` 去解引用会**当场崩**（退出码 3221225477），而不是悄悄写坏分配器的账本、等到程序退出时才报一个莫名其妙的 3221226356。**把「悄悄出错」变成「当场出错」，是这条惯例的全部价值。**

工程惯例 · （续）它救不了什么，而正解是什么

- **别名指针。**同一块内存如果还有另一个指针 `q`，`free(p); p = NULL;` 对 `q` **毫无影响**——`q` 照样悬着，用它读写照样是 use-after-free。C 里没有「这块内存还有几个主人」的记账，`p = NULL` 只撕掉了一张纸条；
- **把 `p` 交出去的那些函数。**`p` 的副本可能住在别的栈帧里，或者已经存进了别的结构体。

所以正解是所有权纪律，不是置空：谁分配谁释放要写进接口（10.3 节的惯例块），谁拥有这块内存是**设计问题**，而 `p = NULL` 只是事后的一张创可贴。写法上仍然推荐置空——它的代价是零，收益是「下次误用会响」。

10.12 这一节新增的文件，与要带走的东西

回到文件看一眼：

目录树 **本节新增：九个例子目录与四个会话文件**

第 1 步 本节新增的九个例子，前三个

```
examples/
└── ch04/
    ├── const_params/ : 三种 const 写法 ← 本课新增
    ├── const_loss/ : 丢 const 的那处调用 ← 本课新增
    └── out_params/ : 输出参数 + 错误码 ← 本课新增
```

第 2 步 中间三个

```
examples/
└── ch04/
    ├── ptr_to_ptr/ : T ** 改指针本身 ← 本课新增
    ├── array_of_ptr/ : 指针数组与二维数组 ← 本课新增
    └── funcptr_dispatch/ : 分发表 ← 本课新增
```

第 3 步 最后三个

```
examples/
└── ch04/
    ├── qsort_callback/ : qsort 的比较函数 ← 本课新增
    ├── memcpy_overlap/ : 重叠搬迁 ← 本课新增
    └── ptr_int/ : 把指针当整数看 ← 本课新增
```

第 4 步 两个会话，落在各自的例子目录里

```
examples/
└── ch04/
    ├── const_loss/ ← 本课新增
    │   └── session-discard-qualifiers.txt ← 本课新增
    └── memcpy_overlap/ ← 本课新增
        └── session-restrict.txt ← 本课新增
```

第 5 步 另外两个会话

```
examples/
└── ch04/
    ├── array_of_ptr/ ← 本课新增
    │   └── session-param-decay.txt ← 本课新增
    └── qsort_callback/ ← 本课新增
        └── session-gdb-funcptr.txt ← 本课新增
```

九个例子里有**两个编不过**：`const_params` 与 `const_loss`（都是 `compile_fail`：前者演示两处「丢 const」的调用点，后者缩到最小、连运行后果一起演示）。其余七个真编译、真运行，输出由流水线逐字对账。第 4、5 帧那四个会话文件是另一类证据：装的是**每次运行都不一样**（gdb 的地址）或**要把命令与输出一起看**的东西。

小结 · 这一节的要点

- **接口上的指针只有两种意图**：只读输入写 `const T *`、可写输出写 `T *`；形参上的 `T * const` 对调用者什么也没承诺。「丢 const」在默认参数下**只是一条警告、退出码 0**，要拦住它得靠 `-Werror`（实测：同一处从 warning 变 error、退出码 0 变 1，而真跑起来是 139）；
- **输出参数 + 错误码**是 C 返回多个结果的惯例：返回值 `0` 成功、负数是错误码，额外的结果用 `T *` 写出去，**失败时一个字节都不写**（`divide_checked(17, 0)` 实测把上一次的 `q`、`r` 原样留着）；谁分配谁释放必须写进接口；
- **多级指针不是玄学**：`T **` 收的是「指针的地址」，`*pp` 改指针、`**pp` 改数据；`char *argv[]` 与 `char **argv` 同一个类型。**声明从名字出发、遇括号掉头**：`char *a[3]` 是三格地址条（`sizeof` 24、一步 8 字节），`char (*a)[3]` 是数组指针（`sizeof` 8、一步 3 字节）；`int *f(int,int)` 与 `int (*f)(int,int)` 差一对括号；

小结 · （续）这一节的要点

- **函数指针就是装着函数地址的变量**（本机 8 字节）：`f(7,5)` 与 `(*f)(7,5)` 等价，函数名直接当地址用；分发表是「名字 + 函数地址」的数组；回调是库把比较方式交还给你的机制——gdb 的 `bt` 实测是 `#0 cmp_int_asc / #1 ucrtbase!qsort / #2 main`；
- `void *` **不带类型**，所以 `qsort` 要 `size`、`memcpy` 要字节数，回调里必须自己转回来；`*p` 与 `p + 1` 在标准里都不合法（本机前者编不过、后者只给警告）；接口上写 `T *` + 长度是主流，因为**函数里 `sizeof(参数)` 只剩指针的大小**（本机 8）；
- **结果一样不等于标准承诺**：重叠的 `memcpy` 与 `memmove` 在本机给出一模一样的字符串（都退出 0），但标准写着重叠时 UB，`restrict` 那句承诺的违反也是 UB——**证据是 `-Wrestrict` 的诊断与标准原文，不是运行结果**；指针大小与 `%p` 的样子都是实现事实，`uintptr_t` 只在少数场合该用。

自测题 · 自测题（先自己答，再对答案）

1. 一个函数想「读一段字符串、并把它转成大写写进另一块缓冲区」，它的签名该怎么写？说明每个参数代表输入还是输出。如果调用者手里只有一个 `const char *`，编译器会说什么、退出码是多少？
2. `int divide_checked(int a, int b, int *q, int *r)` 在 `b == 0` 时返回 `-2`。调用者 `int q = 1, r = 1; int c = divide_checked(7, 0, &q, &r);` 之后，`q` 与 `r` 是多少？为什么这个答案很重要？
3. `int a = 1, b = 2; int *p = &a;` 之后调用 `f(&p, &b)`，`f` 的参数该是什么类型才能让 `p` 改指向 `b`？如果参数写成 `int *pp`，`f` 里那句 `pp = &b;` 会有什么效果？
4. 说出下面三个声明的读法与 `sizeof`（本机）：`char *a[3]`、`char (*a)[3]`、`int (*f)(int, int)`。
5. `char buf[16] = "0123456789";` 之后，`memcpy(buf + 2, buf, 8)` 与 `memmove(buf + 2, buf, 8)` 本机跑出来一样吗？该用哪一个，理由是什么？

注 · 自测题答案

1. 写成 `size_t uppercase_into(char *out, size_t cap, const char *src)`：`out` 是**输出**（会写它，所以不带 `const`）、`cap` 是它的**容量**（连结尾 `'\0'` 一起算）、`src` 是**输入**（只读）。把 `const char *` 交到 `out` 那一刻，编译器给 `warning: passing argument 1 … discards 'const' qualifier from pointer target type [-`

> `Wdiscarded-qualifiers]`、**退出码 0**；加 `-Werror` 才是 error、退出码 1；真跑则 SIGSEGV（退出码 139）。正解是让调用者提供可写缓冲区。
> 2. 都是 **1**。`divide_checked` 在写任何输出参数**之前**就 `return -2` 了——「失败时不碰输出参数」的效果。它重要是因为调用者可以放心复用上一次的结果；若实现里先写后判，调用者读到的就是**脏值**，而且很难发现。
> 3. 参数该写成 `int **pp`（收「指针的地址」，调用时写 `f(&p, &b)`），函数体里写 `*pp = &b;`。若写成 `int *pp`，`pp = &b;` 改的是**形参这份副本**——调用者的 `p` 一动不动（编译还能通过，因为类型合法）。想改谁，就得拿到谁的地址。
> 4. `char *a[3]`：`a` 先与 `[3]` 结合（数组），每格是 `*`（指针）——**指针数组**，`sizeof` **24**，`a + 1` 走 8 字节。`char (*a)[3]`：括号让 `a` 先与 `*` 结合（指针），它指向三个 `char` 的数组——**数组指针**，`sizeof` **8**，`a + 1` 走 3 字节。`int (*f)(int, int)`：指向「两个 int 进、返回 int」的函数，`sizeof` **8**。
> 5. **一样**：两者都得到 `"0101234567"`、退出码 0。但**必须用 `memmove`**——7.24.2.1 第 2 段写着 `memcpy` 在对象重叠时行为未定义，7.24.2.2 第 2 段则要求 `memmove` 好像先搬进不重叠的临时数组；`memcpy` 签名里那两个 `restrict` 也是同一件事（6.7.3.1 第 4 段）。理由不是「结果碰巧对」，而是**标准对 `memcpy` 没有任何承诺**。

第 5 章

结构体与动态内存

本章 3 个单元 · 页眉从这里之后的页面开始出现

1. 结构体：把数据打包，以及看不见的 padding 零基础 · 第 11 单元 · 建议 90 分钟
2. 动态内存：malloc 与生命周期 零基础 · 第 12 单元 · 建议 80 分钟
3. 自引用结构与链表：用指针串起数据 零基础 · 第 13 单元 · 建议 90 分钟

结构体：把数据打包，以及看不见的 padding

◎ 学习目标 · 看完这一单元，你应该能

- 定义一个结构体类型，用 `= {…}` 与 `= {.成员 = 值}` 两种写法初始化它，并说清整体赋值到底拷走了什么；
- 用 `offsetof` 与 `sizeof` 把一块结构体内存画出来，指出哪几个字节是 padding，并解释 `sizeof(struct X)` 为什么比成员之和大；
- 判断一次函数调用里的结构体是「拷了一份」还是「递了个地址」，说出两种传参各自的代价；
- 认出浅拷贝：结构体里装着指针时，整体赋值只复制那个地址，两个副本共用同一块堆内存，跟着来的就是两次 `free`；
- 说出 `-Wpadded`、`packed`、`fwrite 结构体` 三件事各自的代价，并分清哪些该写进头文件、哪些只能留在实现里。

11.1 先看工程长什么样

结构体解决的是一个很朴素的问题：**我想把几个值绑在一起当一个东西用。**一个点有 `x` 和 `y`，一个人有姓名、年龄、分数——分开写成一堆变量，你就没法「把一个人整体传给函数」「整体存进数组」。结构体就是把它们打包成一个新的类型。

这一节是第 5 章的第一单元，也是全书的第 11 单元。照老规矩，先看**文件**：这一节要建起第五个项目 `examples/ch05/`，并且第一次让「**类型定义**住进头文件」。第 1 帧画的是本节故事的起点——`examples/` 下已经躺着前面四章的项目，而第 5 章的项目还不存在。

目录树 **这一节的例子工程：第五个项目 examples/ch05/ 从无到有**

第 1 步 本节开始时，examples/ 下已经有前四个项目

```
examples/
├── ch01/ ：第 1 章
├── ch02/ ：第 2 章
├── ch03/ ：第 3 章
└── ch04/ ：第 4 章
```

第 2 步 本节建起第五个项目，先摆上三个例子

```
examples/
├── 前四个项目 ： 本节不动
└── ch05/ ← 本课新增
    ├── struct_basics/ ： 定义与初始化 ← 本课新增
    ├── struct_compare_fail/ ： 写错的 == ← 本课新增
    └── struct_layout/ ： 偏移与填充 ← 本课新增
```

第 3 步 结构体本身看完了，接着看它怎么被拷贝

```
examples/
└── ch05/
    ├── 前三个例子 ： 都带清单
    ├── struct_reorder/ ： 换序省字节 ← 本课新增
    ├── struct_pass/ ： 两种传参 ← 本课新增
    └── shallow_copy/ ： 只拷地址 ← 本课新增
```

第 4 步 拷得深一点，再换一种内存用法

```
examples/
└── ch05/
    ├── 前六个例子 ： 都带清单
    ├── deep_copy/ ： 自己分配 ← 本课新增
    ├── struct_union/ ： 共用一块内存 ← 本课新增
    └── struct_packed/ ： packed 的代价 ← 本课新增
```

第 5 步 结构体里还能装结构体与数组

```
examples/
└── ch05/
    ├── 前九个例子 ：都带清单
    ├── flexible_array/ ：头尾一次分配 ← 本课新增
    ├── with_padded/ ：报出填充 ← 本课新增
    └── struct_oob_fanalyzer/ ：越界写 ← 本课新增
```

第 6 步 进接口的两件事，各有例子

```
examples/
└── ch05/
    ├── 前十二个 ：都带清单
    ├── struct_nested_array/ ：嵌套与数组 ← 本课新增
    └── opaque_counter/ ：不透明结构体 ← 本课新增
```

第 7 步 位域与 fwrite，各有一次真跑

```
examples/
└── ch05/
    ├── 前十四例 ：都带清单
    ├── bitfield_probe/ ：位域的真字节 ← 本课新增
    └── fwrite_probe/ ：填充进了文件 ← 本课新增
```

第 8 步 四段抓下来的会话，都住在各自的例子里

```
examples/
└── ch05/
    ├── 十六个例子 ：都带清单
    └── with_padded/
        ├── session-padded.txt ：诊断原文 ← 本课新增
        └── session-padded-probe.txt ：探针原文 ← 本课新增
```

第 9 步 packed 的诊断

```
examples/
└── ch05/
    ├── 十六个例子 ：都带清单
    └── struct_packed/
        └── session-packed.txt ：诊断原文 ← 本课新增
```

第 10 步 分析器的视角

```
examples/
└── ch05/
    ├── 十六个例子 ：都带清单
    └── struct_oob_fanalyzer/
        └── session-analyzer.txt ：分析器视角 ← 本课新增
```

第 1 帧是这一节故事的起点。第 2 到 7 帧是本节搭起来的 `examples/ch05/` ——**十六个**例子目录，分六帧列完：每个都是「源文件放 `src/`、声明与**类型定义**放 `include/`、编译与对账的参数写在 `.json` 清单里」的最小工程（第 1 单元 1.7 节讲过分文件的理由，从第二个项目起本书不允许单文件工程）。第 8 到 10 帧标出四段**会话**：它们是编译器真吐出来的原文，抓下来存在各自的例子目录里、再逐字注入正文（后面 11.9、11.10、11.12 三节要用）。

先记住这十六个目录各自负责的**一件事**，后面每一节都会点名其中一个：

- `struct_basics`、`struct_layout`、`struct_reorder` —— 结构体是什么、它在内存里怎么摆、摆法怎么影响大小（11.2 到 11.4 节）；
- `struct_pass`、`shallow_copy`、`deep_copy` —— 传参的代价、整体赋值的浅与深（11.5 到 11.7 节）；
- `struct_union`、`with_padded`、`struct_packed`、`flexible_array` —— 同一块内存的多种读法、让编译器报账、别乱省字节（11.8 到 11.11 节）；
- `struct_nested_array`、`opaque_counter`、`bitfield_probe`、`fwrite_probe` —— 嵌套与数组、不透明结构体、位域、`fwrite` 这四件「进接口前要想清楚」的事（11.12 节）；
- `struct_compare_fail`、`struct_oob_fanalyzer` —— 两个「故意出问题」的例子：一个编不过，一个编译器在编译期就指出越界（11.2、11.12 节）。

工程惯例 · 工程惯例：为什么这次要把类型定义放进头文件

标准只规定了一件事：**一个成员表在同一个作用域里只能定义一次**（6.7.2.3 节第 1 段：A specific type shall have its content defined at most once）。它对你把这段定义写在哪个文件里毫无规定。那为什么本书从这一节起，把 `struct Point { … };` 写进 `include/struct_basics.h`？

- **类型定义是接口的一部分**：`main.c` 里要写 `struct Point a;`，编译器必须知道 `Point` 里有哪些成员、各占几个字节——否则它连 `a.x` 该往哪偏移都算不出来。把定义放进头文件，谁 `#include` 谁就看得见；
- **一份定义，多处使用**：`src/main.c` 与 `src/struct_basics.c` 都 `#include "struct_basics.h"`，两边看到的是**同一个类型**，布局与 `sizeof` 自然一致。要是各写一份、而且写得不一样，编译器与链接器**都不会替你发现**——本机实测：两个源文件里各写一个 `struct Point`（一个 `int x, y`，一个 `long long x, y`）再链接，退出码是 **0**，运行打出来的却是 `sum = 6291088` 而不是 15；
- **代价要记住**：头文件里放**定义**意味着「改一个成员，所有包含它的源文件都要重编」；而且这份定义对所有包含者都是**公开**的——想藏起来，就得用 11.12 节的「不透明结构体」。

方向性的一句话：**函数声明、类型定义、宏可以进头文件；函数体、变量定义不该进**（写重了会撞符号，第 17 单元讲透）。

11.2 结构体就是一个盒子：定义、访问、两种初始化

先看类型定义本身。下面这几行来自 `examples/ch05/struct_basics/include/struct_basics.h`：

c examples/ch05/struct_basics/include/struct_basics.h 第 4–9 行

```
4  /* 一个「点」：结构体把两个 int 打包成一个整体。
5     类型定义放在头文件里 —— 谁 #include 它，谁就看得见这个类型。 */
6  struct Point {  类型定义进头文件，谁 include 谁看得见
7      int x;
8      int y;
9  };
```

6 类型定义进头文件，谁 include 谁看得见

第 6 行的 `struct Point` 里，`Point` 叫**标签**（tag），不是类型名——完整的类型名要写 `struct`、空格、`Point` 三样（这个区别在 11.12 节会变成一条惯例）。访问成员用点号：`p.x`、`p.y`。函数体里也一样，只多了一件事：参数是结构体时按值传：

c examples/ch05/struct_basics/src/struct_basics.c 第 5–8 行

```
5  /* p.x、p.y：点号后面写成员名，是「访问成员」的写法 */
6  void print_point(const char *label, struct Point p) {
7      printf("%s = (%d, %d)\n", label, p.x, p.y);  点号后面写成员名
8  }
```

7 点号后面写成员名

两种初始化写法，一眼就能分清：

c examples/ch05/struct_basics/src/main.c 第 6–9 行

```
6      struct Point a = {1, 2};              /* 按定义顺序：x = 1, y = 2 */
7      struct Point b = {.y = 5, .x = 4};    /* 指定初始化：顺序随便，名字说了算 */  指定初始化
8      struct Point zero = {0};              /* 只给一个 0：所有成员都变成 0 */  没点名的成员补 0
9      struct Point half = {.x = 7};         /* 没点名的成员自动是 0 */
```

7 指定初始化
8 没点名的成员补 0

- `= {1, 2}` 是**按顺序**给：第一个值给 `x`，第二个给 `y`。少给几个，剩下的**按「静态对象」的规矩补 0**——这是标准 6.7.9 节第 21 段的规定（余下的部分按静态存储期对象的方式隐式初始化，也就是全 0）；
- `= {.y = 5, .x = 4}` 是**指定初始化**（designated initializer）：`.成员名 = 值`，顺序随便写，甚至可以只写一部分。标准 6.7.9 节第 7 段给这种写法的约束是：`. identifier` 里的名字必须是那个类型的一个成员名；

- `= {0}` 是「全部清零」的老写法；`= {.x = 7}` 里那个没点名的 `y` 是 0——两种写法用的是同一条补 0 的规矩。

跑出来是这样（下面每一行都是这个程序真跑出来的 stdout，由流水线逐字对账）：

终端输出 终端输出 — struct-basics-out

```
sizeof(struct Point) = 8
a      = (1, 2)
b      = (4, 5)
zero   = (0, 0)
half   = (7, 0)
struct Point d = a; 之后:
  a = (1, 2)
  d = (1, 2)
d.x = 100 之后:
  a = (1, 2)
  d = (100, 2)
按成员逐个比: a 与 d 的 x 相等? 不是
```

四行 `a` / `b` / `zero` / `half` 说的就是上面那四条规矩。真正要紧的是后面四行：`struct Point d = a;` 之后改 `d.x`，**`a` 一点没变。**

c examples/ch05/struct_basics/src/main.c 第 17–27 行

```c
17      struct Point d = a;                     /* 整体赋值: a 的内存整份拷给 d */ 整体赋值: a 的内存整份拷给 d
18      printf("struct Point d = a; 之后: \n");
19      print_point("  a", a);
20      print_point("  d", d);
21
22      d.x = 100;                              /* 只改 d 的成员 */ 改 d 不动 a
23      printf("d.x = 100 之后: \n");
24      print_point("  a", a);
25      print_point("  d", d);
26
27      printf("按成员逐个比: a 与 d 的 x 相等? %s\n", (a.x == d.x) ? "是" : "不是");
```

17 整体赋值: a 的内存整份拷给 d
22 改 d 不动 a

这就是「**整体赋值 = 整体拷贝**」：第 17 行不是让 `d` 指向 `a`，而是把 `a` 这块内存里的字节**拷一份**给 `d`。所以第 22 行改 `d.x` 影响不到 `a`。这一点与数组完全不同——数组名不能被赋值，`int b[2] = a;` 根本编不过；结构体是少数能「整体赋值」的聚合类型。

那能不能直接 `a == d`？不能。下面这个例子（`examples/ch05/struct_compare_fail/`）是**故意写错**的：

c examples/ch05/struct_compare_fail/src/main.c 第 3–9 行

```c
3   int main(void) {
4       struct Point a = {1, 2};
5       struct Point b = {1, 2};
6
7       /* 下一行是故意写错的: C 的 == 只认算术类型与指针, 不认结构体 */
8       return a == b; 故意写错的那一行
9   }
```

8 故意写错的那一行

编译器会这样拒绝它：

编译诊断 gcc 的真实诊断（原文照抄） — ch05-struct-compare-diag

```
src/main.c: In function 'main':
src/main.c:8:14: error: invalid operands to binary == (have 'struct Point' and 'struct Point')
    8 |     return a == b;
      |              ^~
```

标准原文 · 标准原文：`==` 允许比什么

C 语言标准（ISO/IEC 9899，C17）6.5.9 节「Equality operators」第 2 段规定：

> One of the following shall hold: — both operands have arithmetic type; — both operands are pointers to qualified or unqualified versions of compatible types; — one operand is a pointer to an object type and the other is a pointer to a qualified or unqualified version of void; or — one operand is a pointer and the other is a null pointer constant.

大意：`==` 的两边只允许是**算术类型**、**指针**（加一个空指针常量的特例），四条里必须占一条。结构体不是算术类型也不是指针，所以 `a == b` 直接违反这条约束——这叫**约束违反**（constraint violation），编译器必须报一个诊断。

顺带记住它为什么不该被允许：结构体里可能有**填充字节**，那些字节的内容标准不承诺（11.3 节），所以「两个结构体相等」根本没法用「字节都一样」来定义。

出处：ISO/IEC 9899:2018（C17）§6.5.9 第 2 段。本单元所有标准原文都按公开的 C11 草案 N1570 逐字核对——C17 是技术勘误，没有重编这些条款的段号。

⚠ 陷阱 · 结构体不能用 `==`，你会这样被坑到

你会看到什么：写成 `if (a == b)` 之后，编译器报一句 `invalid operands to binary == (have 'struct Point' and 'struct Point')`，位置就在这一行。

你会怎么误判：三个念头会依次冒出来——「是不是少了括号」「是不是要写 `&a == &b`」「是不是要 `#include` 点什么」。于是你加上取地址写成 `&a == &b`，编译器不报错了——**但它比的是两个对象的地址**，两个不同的对象永远不相等，哪怕是同样的值。

真相：结构体没有「相等」这个概念，标准只给了「怎么拷」，没给「怎么比」。想比就必须自己写一个函数，**逐个成员比**（`a.x == b.x && a.y == b.y`），这也是为什么头文件里通常住着一个 `point_equal` 而不是等着语言给你一个 `==`。要小心的是：既然「相等」是你定义的，那**哪些成员参与比较**就是接口的一部分——比如「两条消息」比不比 `len`，得写清楚。

11.3 `sizeof` 为什么比成员之和大：对齐与 padding

现在把结构体摊开在内存里看。这个例子（`examples/ch05/struct_layout/`）里的结构体故意摆得不友好：一个 `char` 后面跟着 `int`。

c examples/ch05/struct_layout/include/struct_layout.h 第 4–16 行

```c
/* 布局演示：成员类型大小不一，中间必然要塞填充字节 */
struct Person {
    char   grade;      /* 1 字节 */
    int    age;        /* 4 字节 */
    char   name[4];    /* 4 字节 */
    double score;      /* 8 字节 */
};

/* memcmp 演示：两个成员值完全一样，只有填充字节不同的结构体 */
struct Tiny {
    char c;
    int  i;
};
```

5 成员类型大小不一

9 一个 8 字节的 double

`offsetof(类型，成员名)` 会告诉你「这个成员离结构体开头几个字节」。它住在 `<stddef.h>` 里，展开后是一个常量：

c examples/ch05/struct_layout/src/struct_layout.c 第 7–16 行

```c
/* offsetof(类型，成员名) 展开后就是「这个成员离结构体开头几个字节」的常量 */
```

```c
void print_person_layout(void) {
    printf("struct Person 的成员偏移：\n");
    printf("  grade 偏移 %2zu    (大小 %zu) \n", offsetof(struct Person, grade), sizeof(char)); 偏移量是编译期常量
    printf("  age   偏移 %2zu    (大小 %zu) \n", offsetof(struct Person, age), sizeof(int));
    printf("  name  偏移 %2zu    (大小 %zu) \n", offsetof(struct Person, name), sizeof(char[4]));
    printf("  score 偏移 %2zu    (大小 %zu) \n", offsetof(struct Person, score), sizeof(double));
    printf("sizeof(struct Person) = %zu，四个成员加起来只有 %zu\n",
           sizeof(struct Person), sizeof(char) + sizeof(int) + sizeof(char[4]) + sizeof(double));
}
```

10 偏移量是编译期常量

四个数字要连起来看（这个程序的完整输出在下面那个 `output` 块里）：`grade` 在偏移 0（占 1 字节），但 `age` 不在偏移 1，而在**偏移 4**。中间那 3 个字节谁也没写，它们的名字叫 **padding**（填充字节）。`score` 是 `double`（8 字节），它落在偏移 16——所以偏移 12 到 15 之间又有 4 个字节是 padding。四个成员加起来 `1 + 4 + 4 + 8 = 17`，而 `sizeof(struct Person)` 是 **24**，多出来的 7 字节全是填充。

内存形态 **struct Person 的 24 个字节：谁是成员，谁是 padding**

第 1 步 成员按声明顺序往里放，位置由对齐要求决定

`grade ： 偏移 0，1 字节` `padding ： 偏移 1…3` `age ： 偏移 4，4 字节`

第 2 步 接着放 name，为了 double 又从 12 空到 15

`grade ： 偏移 0，1 字节` `padding ： 偏移 1…3` `age ： 偏移 4…7` `name ： 偏移 8…11` `padding ： 偏移 12…15` `score ： 偏移 16，8 字节`

第 3 步 数一遍总数：24 个字节里 7 个是填充

`grade ： 偏移 0` `age ： 偏移 4（前面空 3）` `name ： 偏移 8` `score ： 偏移 16（前面空 4）` `合计 ： 17 字节成员 + 7 字节填充 = 24`

第 1 帧里那 3 个字节是本节最该记住的东西：**它存在，但它不装任何东西**。第 2 帧接着放 `name` 和 `score`，中间又空出 4 个字节（`name` 只占 4 字节，而 `score` 要求从 8 的倍数开始）。第 3 帧把账算平：24 字节 = 17 字节成员 + 7 字节填充，**尾部一个字节都没多补**（`score` 的 8 个字节正好落在 16 到 23）。图里每个偏移数字，都和 `offsetof` 打出来的一致——这不是画出来的示意图，是量出来的。

标准原文 · 标准原文：填充字节可以出现在哪

同一个标准，6.7.2.1 节「Structure and union specifiers」里有三句话把这件事说完了：

> Each non-bit-field member of a structure or union object is aligned in an implementation-defined manner appropriate to its type.
>
> There may be unnamed padding within a structure object, but not at its beginning.
>
> There may be unnamed padding at the end of a structure or union.

大意：① 每个成员按它的类型对齐，**怎么对齐由实现决定**（标准只要求「合适」，没规定具体数字）；② 结构体**内部**可以有未命名的填充，但**开头不会有**（第一个成员永远在偏移 0）；③ 结构体**结尾**也可以有填充。

出处：ISO/IEC 9899:2018（C17）§6.7.2.1 第 14、15、17 段。

「对齐」这个词本身也有定义，在 6.2.8 节第 1 段：

标准原文 · 标准原文：什么叫「对齐」

> Complete object types have alignment requirements which place restrictions on the addresses at which objects of that type may be allocated. An alignment is an implementation-defined integer value representing the number of bytes between successive addresses at which a given object can be allocated.

大意：一个类型对「它能放在哪些地址上」有要求，这个要求叫对齐；对齐值是**实现定义的**整数。所以 24 这个数字是**本机 gcc 的选择**，不是语言定律——标准只要求「按类型合适地对齐」，别的实现完全可以算出别的大小。

出处：ISO/IEC 9899:2018（C17）§6.2.8 第 1 段。

那填充字节里到底是什么？标准给了一个很直白的说法：

标准原文 · 标准原文：填充字节里装的是「未指定值」

6.2.6.1 节「General」第 6 段讲的是「把值存进对象时会发生什么」：

> When a value is stored in an object of structure or union type, including in a member object, the bytes of the object representation that correspond to any padding bytes take unspecified values.

大意：把一个结构体值存进一个结构体对象时，那些**对应填充字节的位置取未指定的值**。注意「未指定」不是「随机」也不是「保持原样」——标准的意思是：**你别去看它，看了也不作数。**

出处：ISO/IEC 9899:2018（C17）§6.2.6.1 第 6 段。

「未指定」的后果，可以用 `memcmp` 亲手摸一次。下面这个例子把两个结构体的成员写成完全一样的值，只把填充字节写成不同：

c examples/ch05/struct_layout/src/struct_layout.c 第 18–22 行

```c
18  void fill_tiny(struct Tiny *t, char c, int i, unsigned char fill) {
19      memset(t, fill, sizeof *t);      /* 连填充字节一起写成 fill */ 连填充字节一起写成已知值
20      t->c = c;                        /* 再写真正的成员 */
21      t->i = i;
22  }
```

19 连填充字节一起写成已知值

c examples/ch05/struct_layout/src/main.c 第 9–19 行

```c
9       struct Tiny p;
10      struct Tiny q;
11      fill_tiny(&p, 'x', 7, 0xAA);          /* 填充字节 = 0xAA */ 填充字节填成 0xAA
12      fill_tiny(&q, 'x', 7, 0x00);          /* 填充字节 = 0x00 */ 填充字节填成 0x00
13
14      printf("\n两个 struct Tiny: p.c = %c, p.i = %d; q.c = %c, q.i = %d\n",
15             p.c, p.i, q.c, q.i);
16      printf("成员逐个比: %s\n", (p.c == q.c && p.i == q.i) ? "相等" : "不相等");
17      printf("sizeof(struct Tiny) = %zu, 成员只占 %zu\n",
18             sizeof(struct Tiny), sizeof(char) + sizeof(int));
19      printf("memcmp(&p, &q, sizeof p) = %d\n", memcmp(&p, &q, sizeof p)); 两个成员一模一样，它却说不同
```

11 填充字节填成 0xAA
12 填充字节填成 0x00
19 两个成员一模一样，它却说不同

终端输出 终端输出 struct-layout-out

```
struct Person 的成员偏移:
  grade 偏移  0    (大小 1)
  age   偏移  4    (大小 4)
  name  偏移  8    (大小 4)
  score 偏移 16    (大小 8)
sizeof(struct Person) = 24, 四个成员加起来只有 17
两个 struct Tiny: p.c = x, p.i = 7; q.c = x, q.i = 7
成员逐个比: 相等
sizeof(struct Tiny) = 8, 成员只占 5
memcmp(&p, &q, sizeof p) = 1
```

输出里 `成员逐个比: 相等` 与 `memcmp(&p, &q, sizeof p) = 1` 这两行放在一起，就是这一节的包袱：**成员逐个比是「相等」，`memcmp` 却给出一个非 0 的值。**因为 `memcmp` 比的是**字节**，而第 2 个字节起 p 是 `0xAA`、q 是 `0x00`——两个结构体的「洞」里装的东西不一样。

标准原文 · 标准原文：`memcmp` 比不了结构体

7.24.4.1 节「The `memcmp` function」第 2 段的函数说明后面跟着一条脚注（编号 310），说的正是这件事：

> The contents of "holes" used as padding for purposes of alignment within structure objects are indeterminate. Strings shorter than their allocated space and unions may also cause problems in comparison.

大意：结构体里那些为对齐留下的「洞」，内容是不确定的（indeterminate）；比它时会有麻烦。同一节第 3 段还规定返回值只要**符号**对：大于零、等于零、小于零分别表示「前者大、一样大、后者小」，具体数值不作承诺——本机返回的是 `1`，别的实现可能返回 `170` 或别的正数，所以判等只能写 `memcmp(...) == 0`，不能写 `== 1`。

出处：ISO/IEC 9899:2018（C17）§7.24.4.1 第 3 段与脚注 310，§7.24.4 第 1 段。

⚠ 陷阱 · 「sizeof 比成员之和大」不是 bug，`memcmp` 比结构体才是 bug

你会看到什么：算出 `sizeof(struct Person) = 24`、成员只占 17，第一反应是「编译器算错了」或者「我是不是漏写了什么」。

你会怎么误判：于是你开始「优化」——把成员顺序调一调、或者给结构体加个 `#pragma pack`，看到 `sizeof` 变小了就以为赚了（11.9、11.10 两节会告诉你代价是什么）；再遇到 `memcmp` 判等失败，你会怀疑「是不是有一个成员没赋上值」，然后一行一行加打印——而两个结构体的成员其实一模一样。

真相：填充字节是**为了对齐**才出现的，它是实现的选择（§6.7.2.1 第 14、15、17 段），不是你的 bug；而 `memcmp` 比的是字节，填充字节的内容标准不承诺（§6.2.6.1 第 6 段、§7.24.4.1 脚注 310），所以**拿 `memcmp` 判断两个结构体是否「内容相等」不可靠。**要判等就写一个逐成员比的函数。

有人会想到「先 `memset(&x, 0, sizeof x)` 清零、再逐个赋值，填充字节不就一直是 0 了吗」——不行：**向成员存值这个动作本身**就会让填充字节取未指定的值（同一条 §6.2.6.1 第 6 段，「including in a member object」）。真要用 `memcmp`，顺序得反过来：**所有成员都写完之后**，再 `memset` 或逐字节清零填充区，此后不再碰任何成员；而且这个前提必须写在注释里。更稳的做法是别比内存字节，改比「你要比的那几个成员」。

11.4 换个成员顺序，省下的是真金白银

同一个 `struct Person` 的 24 字节里，7 个字节是纯浪费。这个例子（`examples/ch05/struct_reorder/`）拿同一批成员（一个 `char`、一个 `int`、一个 `char`、一个 `long long`）摆两种顺序：

c examples/ch05/struct_reorder/include/struct_reorder.h 第 4–17 行

```c
/* 同一批成员 (char、int、char、long long) , 两种排法 */
struct Messy {  先小后大
    char      flag;
    int       count;
    char      tag;
    long long total;
```

```
10  };
11
12  struct Tidy {  先大后小
13      long long total;
14      int       count;
15      char      flag;
16      char      tag;
17  };
```

5 先小后大
12 先大后小

终端输出 终端输出 struct-reorder-out

```
struct Messy (char, int, char, long long)
  flag  偏移  0
  count 偏移  4
  tag   偏移  8
  total 偏移 16
  sizeof = 24
struct Tidy (long long, int, char, char)
  total 偏移  0
  count 偏移  8
  flag  偏移 12
  tag   偏移 13
  sizeof = 16
同一批成员：24 字节 → 16 字节，每个实例省 8 字节
一个 1000 个元素的数组：24000 字节 → 16000 字节
```

对照读输出里的两组偏移：

- `struct Messy`：`flag`@0，`count`@4（前面空 3），`tag`@8，`total`@16（前面空 7）→ 成员 14 字节 + 填充 10 = **24 字节**；
- `struct Tidy`：`total`@0，`count`@8，`flag`@12，`tag`@13 → 成员 14 字节 + **内部 0 + 尾部 2 = 16 字节。**

注意 `struct Tidy` 里那 2 个字节：`tag` 在偏移 13，只占 1 字节，而整个结构体要按 `long long` 的 8 字节对齐（`sizeof` 必须是 8 的倍数），所以 14 补到 16。**没有任何填充的结构体是很少见的**，别把「没有内部空洞」当成「没有 padding」。

同一批成员、同样的信息量，一种排法是 24 字节，另一种是 16 字节——**每个实例省 8 个字节。**输出最后还替你算了一笔账：1000 个元素的数组从 24000 字节降到 16000 字节。

工程惯例 · 工程惯例：按对齐从大到小排成员

标准对成员顺序的规定只有一句「地址按声明顺序递增」（§6.7.2.1 第 15 段），对怎么排**没有要求。**把大对齐的成员排前面、小的排后面，是主流做法，理由就是上面那 8 个字节；代价有三条要说清：

- **别为了省字节牺牲可读性**：一个「姓名、年龄、分数」按逻辑顺序排就很好读，硬按对齐排会让人看不懂。**先按语义排，量到浪费大再调**——`-Wpadded`（11.9 节）和 `offsetof` 就是量它的工具；
- **顺序是接口的一部分**：成员顺序一改，`sizeof` 就变，谁按旧的大小分配了内存（比如写进文件、发给别的进程）谁就对不上；
- **别把「省了 8 字节」当成硬指标**：一个进程里十万个结构体才是一兆字节的事；在你自己的小工具里，可读性通常更值钱。

11.5 传参：按值拷一份，按指针递地址

结构体的每个字节都能被拷，所以「把结构体传给函数」就有一个绕不开的问题：拷几次？这个例子（`examples/ch05/struct_pass/`）用一个 68 字节的结构体来量这件事：

c examples/ch05/struct_pass/include/struct_pass.h 第 4–17 行

```
4   /* 68 个字节的结构体：一次按值传递就要拷 68 字节 */
5   struct Payload {  68 字节的大结构体
6       char data[64];
7       int  len;
```

```
};

/* 按值传：形参是调用者那一份的拷贝。caller 只是为了打印「原件住在哪」 */ 按值传
void by_value(struct Payload p, const struct Payload *caller);

/* 按指针传：只递一个地址（本机 8 字节），改的是原件 */
void by_pointer(struct Payload *p); 按指针传

/* 按值返回：把结果放进调用者给的位置，代价见 11.5 节 */ 按值返回
struct Payload make_payload(char first);
```

5 68 字节的大结构体
10 按值传
14 按指针传
16 按值返回

三个函数长得差不多，区别只有参数与返回方式：

c examples/ch05/struct_pass/src/struct_pass.c 第 6–19 行

```
void by_value(struct Payload p, const struct Payload *caller) {
    printf("  形参 p 住在 %p\n", (void *)&p); 形参自己的地址
    printf("  形参的地址 == 调用者的地址? %s\n", (&p == caller) ? "是" : "不是（另开了一份）"); 和调用者的地址比
    p.data[0] = 'V';                        /* 改的是这份拷贝 */
    p.len = 999;
    printf("  在函数里改拷贝：p.data[0] = %c, p.len = %d\n", p.data[0], p.len);
}

void by_pointer(struct Payload *p) {
    printf("  形参 p 里装的地址 = %p\n", (void *)p);
    p->data[0] = 'P';                       /* p->x 就是「顺着地址去改它指向的成员」 */ p 里装着地址
    p->len = 888; 顺着地址改原件
    printf("  顺着地址改原件：p->data[0] = %c, p->len = %d\n", p->data[0], p->len);
}
```

7 形参自己的地址
8 和调用者的地址比
16 p 里装着地址
17 顺着地址改原件

返回值那一路，这个例子只做一件能看见的事：在函数里把自己那个局部对象的地址打出来，回到 `main` 再打一次接住的对象的地址——两个地址一样不一样，就是「返回有没有拷一份」的直接证据：

c examples/ch05/struct_pass/src/struct_pass.c 第 21–29 行

```
struct Payload make_payload(char first) {
    struct Payload p;

    memset(p.data, 0, sizeof p.data);
    p.data[0] = first;
    p.len = 1;
    printf("  按值返回：函数里的局部对象 p 住在 %p\n", (void *)&p); 返回之前先报一下自己的地址
    return p;
}
```

27 返回之前先报一下自己的地址

c examples/ch05/struct_pass/src/main.c 第 6–26 行

```
int main(void) {
    struct Payload x;
    strcpy(x.data, "hello");
```

```c
    x.len = 5;

    printf("sizeof(struct Payload) = %zu\n", sizeof(struct Payload));
    printf("x 住在 %p (main 里那个对象) \n\n", (void *)&x);

    printf("调用 by_value(x): \n");
    by_value(x, &x);
    printf("  回到 main: x.data[0] = %c, x.len = %d\n\n", x.data[0], x.len);

    printf("调用 by_pointer(&x): \n");
    by_pointer(&x);
    printf("  回到 main: x.data[0] = %c, x.len = %d\n", x.data[0], x.len);

    printf("\n调用 make_payload('Z') 并按值接住: \n");
    struct Payload y = make_payload('Z');
    printf("  main 里接下返回值的 y 住在 %p (与上面那个地址比一比) \n", (void *)&y);
    return 0;
}
```

11 68 字节
12 原件住在哪
15 按值传
23 接住返回值

终端输出 终端输出 struct-pass-out

```
sizeof(struct Payload) = 68
x 住在 00000000005FFE60 (main 里那个对象)
调用 by_value(x):
  形参 p 住在 00000000005FFDC0
  形参的地址 == 调用者的地址? 不是 (另开了一份)
  在函数里改拷贝: p.data[0] = V, p.len = 999
  回到 main: x.data[0] = h, x.len = 5
调用 by_pointer(&x):
  形参 p 里装的地址 = 00000000005FFE60
  顺着地址改原件: p->data[0] = P, p->len = 888
  回到 main: x.data[0] = P, x.len = 888
调用 make_payload('Z') 并按值接住:
  按值返回: 函数里的局部对象 p 住在 00000000005FFD30
  main 里接下返回值的 y 住在 00000000005FFE10 (与上面那个地址比一比)
```

按四件事读这份输出：

1. **地址**：`by_value` 里的 `&p` 与 `main` 里的 `&x` **不是同一个地址**——值传递真的另开了一份；`by_pointer` 里 `p` 存的地址与 `&x` **是同一个**——它只是把地址递了过去（本机指针 8 字节，与 68 字节相比差别很大）；
2. **改动是否可见**：`by_value` 里改了拷贝，回到 `main` 后 `x.data[0]` 还是 `h`；`by_pointer` 里改的是原件，回到 `main` 后 `x.data[0]` 变成了 `P`；
3. **返回也拷了一份**：`make_payload` 里的 `p` 与 `main` 里接住返回值的 `y` 是**两个不同的地址**——`-O0` 下它先在自己的栈帧里造一份，再放进调用者的位置；
4. **语法对应关系**：结构体变量用 `.` 访问成员（`p.data[0]`）；结构体**指针**用 `->` 访问成员（`p->data[0]`）——`p->data[0]` 就是 `(*p).data[0]` 的简写。

地址那几串数字是本机给的，都是**栈地址**：同一台机器、同一个可执行文件路径下每次都一样，换台机器就会变。要紧的是「两个地址是不是同一个数」，不是这串数字本身。

工程惯例 · 工程惯例：多大才该改成指针

标准对参数怎么传**一个字都没规定**（它只说「参数按值传」），所以「多大改指针」纯粹是惯例。流传最广的阈值是「**结构体超过 16 字节就传 `const` 指针**」，理由是这样：

- 一两个机器字大小的结构体（本机一个 `struct Point` 是 8 字节），按值传与传指针的差别很小，读起来按值更省事；
- 再大就变成**逐字节的内存拷贝**（本机 68 字节要拷 68 个字节），而传指针只要递 8 个字节；调用次数多的时候，这个差别是真的；
- 传指针时**必须加 `const`**（`const struct Payload *p`）：一是表达「我只读」，二是调用者敢放心传进来。

返回结构体是另一件事：小的结构体（本例的 8 字节 `struct Point`）由 ABI 用寄存器带回，几乎免费；大的就得靠「调用者提供位置 + 被调函数拷进去」，**`-O0` 下这笔拷贝是实的**（上面那个例子的第 3 条就是证据）。优化器可能把它省掉，但标准不承诺。**代价**还有一条：返回指针就必须交代「这块内存谁释放」（11.6、11.7 节）。阈值是**惯例不是标准**：小结构体按值、大结构体传 `const` 指针，是让代码好读的约定，不是性能定律。

11.6 整体赋值只拷「表面」：浅拷贝

到这里「整体赋值 = 整体拷贝」一直很乖。但结构体里可以装**指针**，而指针只是一个地址——这时候整体赋值拷的是这个**地址**，不是地址那头的东西。先看类型：

c examples/ch05/shallow_copy/include/shallow_copy.h 第 4–12 行

```c
/* 一条消息：文字住在堆上，结构体里只有一个地址。
   这个结构体自己不管释放 —— 谁分配谁释放，见 11.7 节 */
struct Message {
    char *text; 结构体里只有一个地址
    int   len;
};

/* 造一条消息：为 text 分配一块堆内存，把内容拷进去 */
struct Message make_message(const char *s);
```

7 结构体里只有一个地址

`text` 是一个 `char *`，它指向的文字住在**堆**上（`make_message` 里 `malloc` 出来的）。现在看程序：

c examples/ch05/shallow_copy/src/main.c 第 7–26 行

```c
int main(void) {
    struct Message a = make_message("hello");
    printf("a: text 里装着一个地址，那块内存里是 \"%s\" (len = %d) \n", a.text, a.len);

    struct Message b = a;                   /* 整体赋值：只把 text 里那个地址拷了一份 */ 整体赋值，只拷了地址
    printf("b = a 之后：b.text 指向的那块内存里是 \"%s\" (len = %d) \n", b.text, b.len);
    printf("a.text == b.text ? %s\n", (a.text == b.text) ? "是，两个成员装的是同一个地址" : "不是"); 两个成员装的是同一个地址

    b.text[0] = 'H';                        /* 顺着 b.text 改那块内存 */ 顺着地址改内存
    printf("b.text[0] = 'H' 之后：a.text = %s，b.text = %s\n", a.text, b.text);
    printf("a.len = %d，b.len = %d —— 长度是各自的一份，文字不是\n", a.len, b.len);

    free(a.text); 还第一次
    printf("free(a.text) 走完了，接下来 free(b.text)\n");
    fflush(stdout);                         /* 先把上面这些行送出去 */

    free(b.text);                           /* 同一块内存还第二次 */ 还第二次
    printf("这行不会打印\n");
    return 0;
}
```

```
26  }

11 整体赋值，只拷了地址
13 两个成员装的是同一个地址
15 顺着地址改内存
19 还第一次
23 还第二次
```

终端输出 终端输出 shallow-copy-out

```
a: text 里装着一个地址，那块内存里是 "hello" (len = 5)
b = a 之后：b.text 指向的那块内存里是 "hello" (len = 5)
a.text == b.text ？是，两个成员装的是同一个地址
b.text[0] = 'H' 之后：a.text = Hello, b.text = Hello
a.len = 5, b.len = 5 —— 长度是各自的一份，文字不是
free(a.text) 走完了，接下来 free(b.text)
```

程序在 `free(b.text)` 那一行**没有走完**：清单 `examples/ch05/shallow_copy/shallow_copy.json` 写的是 `"mode": "runtime_error"`、`"expect": { "exit": 3221226356 }`——这个退出码是流水线真跑出来的（**3221226356** 就是 `0xC0000374`，Windows 的「堆损坏」）。上面那段输出之所以能留下来，全靠第 21 行那句 `fflush(stdout)`（缓冲区没刷的后果见第 9 单元 9.8 节）。

内存形态 **浅拷贝之后：两个结构体指向同一块堆内存**

第 1 步 **a 与那块堆内存**

`a：text 里是一个地址` `堆块：“hello” 加结尾的 0`

a → 堆块 a.text 指向它

第 2 步 **b = a 之后，地址被拷了一份，堆块没多**

`a：text、len 各一份` `b：text 里是同一个地址`

a → 堆块 a.text 指向它
b → 堆块 b.text 指向同一块

第 3 步 **改 b.text[0]，改的是那块共用的内存**

`堆块：“Hello” 加结尾的 0`

b → 堆块 把首字母改成 'H'
a → 堆块 a 也看见 'H'

第 1 帧是初始状态：`a` 里只有一个地址，文字本身不在结构体里。第 2 帧是 `struct Message b = a;` 之后的样子——**结构体对象多了一个，堆内存没有多**，两个 `text` 成员装的是同一个地址。第 3 帧是后果：改 `b.text[0]`，`a` 看到的也变了（输出里那句 `a.text = Hello, b.text = Hello` 就是它）。而 `len` 是各自的一份：`a` 改 `len` 不会影响 `b`——**值成员各一份，指针成员指着同一块。**

这叫**浅拷贝**：拷的是结构体这块内存的字节，指针成员里那个地址照抄一份，地址那头的东西**一个字节都没复制。**

⚠ 陷阱 · **浅拷贝的下半场：两次 free 把分配器的账本弄坏**

你会看到什么：程序打印完你写的几行字，然后**当场终止**。你在 cmd 里跑会看到退出码 `-1073740940`；在 MSYS 的 bash 里看到 `127`；闸门用无符号口径抓回来是 **3221226356**——三个数字是同一件事（0xC0000374，堆损坏）。stderr 一个字节都没有，没有报错行可以给你看。

你会怎么误判：最早的反应是「是不是我 `printf` 写错了」「是不是 `hello` 字符串有问题」；接着会怀疑 `make_message` 里的 `malloc` 失败（可它没返回 `NULL`）；再往后会发现一件更吓人的事——**把 `free(b.text)` 删掉，程序就正常了**。于是你得出一个错误结论：「少 free 一次就好了」——不，你只是把双重释放换成了**内存泄漏**，而且还留着一个随时可能改坏别人数据的悬垂指针。

真相：`a.text` 与 `b.text` 是**同一个地址**，`free` 一次就把那块内存还给了分配器，第二次 `free` 同一个地址是**未定义行为**。标准 7.22.3.3 节第 2 段写得很直白：如果参数不是分配函数先前返回的指针、或者那块空间已经被 `free` / `realloc` 释放过，**the behavior is undefined**（不是「约束违反」——它没有用 shall 的措辞）。本机的现象是进程被终止、退出码 3221226356（`0xC0000374`，Windows 的「堆损坏」代码）：堆的内部结构被破坏了，至于具体是哪一步发现的，这个数字不告诉你。第 12 单元会把 `free` 的语义讲透。**正确的做法只有一条：谁分配谁释放，而且只有一份所有权**——要么别整体赋值（传 `const` 指针），要么做 11.7 节的深拷贝。

实验记录 · **本机实测：这一节的数字都是跑出来的**

- 工具链：gcc 16.1.0（`-std=c17`，即 C17），目标三元组 x86_64-w64-mingw32；固定参数 `-Wall -Wextra -Wpedantic -O0 -g`；
- `struct_basics`：`sizeof(struct Point) = 8`，整体赋值后改 `d.x = 100`，`a.x` 仍是 1；
- `struct_layout`：`offsetof` 给出 0 / 4 / 8 / 16，`sizeof(struct Person) = 24`，成员之和 17；两个成员相同的 `struct Tiny` 用 `memcmp` 比出 **1**（非 0），逐成员比是「相等」；
- `struct_reorder`：24 字节 → 16 字节，每个实例省 8 字节；
- `struct_pass`：`sizeof(struct Payload) = 68`，按值传的形参地址与调用者不同、按指针传的相同；
- `shallow_copy`：`runtime_error` 退出码 **3221226356**（0xC0000374），stdout 是崩溃前 `fflush` 出去的那几行。

每条输出都由流水线逐字对账：正文里的 `output` 块不是抄的，是 `run.js` 跑完程序回写的。

11.7 深拷贝：多一次分配，换一份自由

浅拷贝的病根是「两个结构体共用一个堆块」。治法不是删掉某一次 `free`（那只是把双重释放换成泄漏），而是让副本**自己有一块内存**。这就是深拷贝：多分配一次，把地址那头的东西也复制一份。

接口上先把话说清（`examples/ch05/deep_copy/include/deep_copy.h`）：

c examples/ch05/deep_copy/include/deep_copy.h 第 4–17 行

```c
/* 和 shallow_copy 里同一个形状的消息结构体 —— 区别只在「怎么拷贝」 */
struct Message {
    char *text;
    int   len;
};

/* 造一条消息：自己分配 text */
struct Message make_message(const char *s);

/* 深拷贝：给副本单独分配一块内存，再逐字段拷过去 */
struct Message clone_message(const struct Message *src); 传进来一个只读的源

/* 释放一条消息，并把指针置空（谁分配谁释放） */
void free_message(struct Message *m); 释放这件事也写在接口里
```

14 传进来一个只读的源
17 释放这件事也写在接口里

实现里只多了一步——那一步就是「深」的全部含义：

c examples/ch05/deep_copy/src/deep_copy.c 第 20–32 行

```c
/* 深拷贝多出来的那一步就在第 25 行：副本自己再要一块内存 */
struct Message clone_message(const struct Message *src) {
    struct Message m;

    m.len = src->len;                                   /* 先拷不值钱的字段 */ 先拷不值钱的字段
    m.text = (char *)malloc((size_t)src->len + 1);      /* 再给文字要一块新的 */ 副本自己再要一块内存，这就是「深」
    if (m.text == NULL) {
        m.len = 0;
        return m;
    }
    memcpy(m.text, src->text, (size_t)src->len + 1);    /* 把文字搬过去 */ 把文字搬过去
    return m;
}
```

24 先拷不值钱的字段
25 副本自己再要一块内存，这就是「深」
30 把文字搬过去

c examples/ch05/deep_copy/src/main.c 第 5–22 行

```c
int main(void) {
    struct Message a = make_message("hello");
    struct Message b = clone_message(&a);        /* 深拷贝：一次分配 + 逐字段拷 */ 一次分配 + 逐字段拷

    printf("a: text 里是 \"%s\" (len = %d) \n", a.text, a.len);
    printf("b = clone_message(&a): b 自己分配了一块内存来放文字\n");
    printf("a.text == b.text ? %s\n", (a.text == b.text) ? "是" : "不是 —— 两块不同的内存"); 两个地址不同

    b.text[0] = 'H';
    printf("b.text[0] = 'H' 之后：a.text = %s，b.text = %s\n", a.text, b.text);

    free_message(&a); 各还各的
    free_message(&b);                            /* 各还各的 */ 第二次释放也合法
    printf("两次 free_message 都走完了，a.text 与 b.text 都被置空\n");
    printf("sizeof(struct Message) = %zu —— 结构体本身没变大，多的是一块堆内存\n",
           sizeof(struct Message));
    return 0;
}
```

7 一次分配 + 逐字段拷
11 两个地址不同
16 各还各的
17 第二次释放也合法

终端输出 终端输出 deep-copy-out

```
a: text 里是 "hello" (len = 5)
b = clone_message(&a): b 自己分配了一块内存来放文字
a.text == b.text ? 不是 —— 两块不同的内存
b.text[0] = 'H' 之后：a.text = hello，b.text = Hello
两次 free_message 都走完了，a.text 与 b.text 都被置空
sizeof(struct Message) = 16 —— 结构体本身没变大，多的是一块堆内存
```

输出里三件事连起来读，就是浅与深的分界：`a.text == b.text` 是**「不是」（两块内存）、改 `b` 的字节不影响** `a`、两次 `free_message` 都合法。而 `sizeof(struct Message)` 还是 16——**深拷贝没有让结构体变大，它多出来的是一块堆内存**：多一次 `malloc`、多一次 `free`、多一次失败检查（第 26 行的 `if (m.text == NULL)` 不是摆样子）。

工程惯例 · **工程惯例：深拷贝的代价，与「值语义」这个词**

标准对「怎么算深拷贝」没有任何规定——它只规定整体赋值是「值的拷贝」（这份拷贝里指针成员的值照抄）。深拷贝是**你写出来的一份契约**，代价有三条：

- **一条所有权规则变成了两条**：每次拷贝都要配一次释放。接口上必须写清「`clone_message` 的返回值由调用者 `free_message` 」——不写，接你代码的人就会漏掉一半；
- **每次拷贝都可能失败**：多一次 `malloc` 就多一次判 `NULL`（本机分配失败返回 `NULL`，见第 12 单元）；
- **多一次分配与一次释放的开销**：小字符串看不出来，循环里做几百万次就看出来了。

好处也是实的：副本之间**完全独立**——改一个不影响另一个，各释放各的，没有共享状态。这套「谁分配谁释放、谁也别省」的规矩，专业上叫结构体的**值语义**（value semantics）：结构体被赋值时表现得像一个值，而不是一张通往别处的纸条。C 里这个「像值」是要你自己保证的——所以**结构体里有没有指针，决定了这个结构体能不能随便整体赋值**，这一点必须写进头文件的注释里。

11.8 `union`：同一块内存的几种读法

`union` 与 `struct` 的语法几乎一模一样，只有一条区别：**成员共用同一块内存**，而不是一人一块。这个例子（`examples/ch05/struct_union/`）用 4 个字节演示三种读法：

c examples/ch05/struct_union/include/struct_union.h 第 4–13 行

```c
/* union：所有成员共用同一块内存，大小按最大的那个成员算。
   一次只用一个成员 —— 写进去之后，是你在决定「怎么读它」 */
union Word {
    unsigned int  u;      /* 4 字节 */ 整数读法
    unsigned char b[4];   /* 4 字节 */ 字节读法
    float         f;      /* 4 字节 */ 浮点读法
};

/* 打印一个 float 的位模式（同一块内存，三种读法） */
void print_float_bits(float value);
```

7 整数读法
8 字节读法
9 浮点读法

c examples/ch05/struct_union/src/struct_union.c 第 5–13 行

```c
void print_float_bits(float value) {
    union Word w;

    w.f = value;                        /* 按 float 写进去 */ 按 float 写进去
    printf("  w.f = %g\n", (double)w.f);
    printf("  w.u = %10u (按 unsigned int 读同一块内存) \n", w.u); 换一种读法读同一块内存
    printf("  w.b = %02X %02X %02X %02X (按 4 个 char 读，从低地址到高地址) \n",
           w.b[0], w.b[1], w.b[2], w.b[3]);
}
```

8 按 float 写进去
10 换一种读法读同一块内存

终端输出 终端输出 struct-union-out

```
sizeof(union Word) = 4：三个成员共用同一块内存
先写 w.u = 0x01020304，再换读法：
  w.b = 04 03 02 01
  w.f = 2.38794e-38（按 float 解释这 4 个字节就是另一个数）
1.0f 的位模式：
  w.f = 1
  w.u = 1065353216（按 unsigned int 读同一块内存）
  w.b = 00 00 80 3F（按 4 个 char 读，从低地址到高地址）
-2.5f 的位模式：
  w.f = -2.5
  w.u = 3223322624（按 unsigned int 读同一块内存）
  w.b = 00 00 20 C0（按 4 个 char 读，从低地址到高地址）
```

三个数字要连着看：

- **`sizeof(union Word) = 4`**：三个成员都占 4 字节，union 的大小就是最大成员的大小（结构体是「成员 + 填充」，union 是「取最大」）；
- **`1.0f` 的位模式是 `00 00 80 3F`**，按 `unsigned int` 读出来是 **1065353216**。这不是魔法：`1.0f` 的 IEEE-754 单精度表示是 `0x3F800000`，只是本机把它**从低地址到高地址**存成了 `00 00 80 3F`——这叫**小端字节序**；
- **先写 `w.u = 0x01020304`，再按字节读是 `04 03 02 01`**：同一块内存，你换一种成员去读，得到的是另一套解释。按 `float` 读出来是 `2.38794e-38`——注意这是一个**合法**的有限 float（本机的浮点就是 IEEE-754 单精度），它只是极小、与你想表达的数毫无关系。**「能解释成一个数」不等于「这么解释有意义」**：位模式按 float 读出来是什么，取决于浮点的表示（是不是 IEEE-754、多少位指数），而那由实现决定。

标准原文 · **标准原文：`union` 的大小**

6.7.2.1 节第 16 段：

> The size of a union is sufficient to contain the largest of its members. The value of at most one of the members can be stored in a union object at any time.

大意：union 的大小足以容纳**最大的那个成员**；同一时刻里面**至多只有一个成员的值**。这就是为什么要「记住最后一个写的是哪个成员」——这是**你的责任**，编译器不替你记。

出处：ISO/IEC 9899:2018（C17）§6.7.2.1 第 16 段。

标准原文 · **标准原文：换一个成员读同一块内存**

这件事 C 是明确允许的，脚注 95（在 6.5.2.3 节「Structure and union members」下）：

> If the member used to read the contents of a union object is not the same as the member last used to store a value in the object, the appropriate part of the object representation of the value is reinterpreted as an object representation in the new type as described in 6.2.6 (a process sometimes called ''type punning''). This might be a trap representation.

大意：如果你用来读的成员不是最后写入的那个成员，那部分对象表示会**按新类型重新解释**（这种做法俗称 type punning）；重新解释出来的结果**可能是陷阱表示**（trap representation，即那个位模式在新类型里不表示任何合法值）。顺带记住：字节顺序本身是实现定义的——6.2.6.1 节第 2 段说，对象是一片连续的字节，它们的**数量、顺序与编码**要么被标准明确规定、要么由实现决定。

出处：ISO/IEC 9899:2018（C17）§6.5.2.3 脚注 95，§6.2.6.1 第 2 段。

注 · **预告：严格别名（第 19 单元）**

「换成员读」在 C 里是被脚注 95 明确许可的；但**换指针读**完全不是一回事——`(int *)&some_double` 这种写法碰的是 6.5 节第 7 段（俗称**严格别名规则**，strict aliasing），优化器会据此认为「这两个指针不可能指向同一块内存」。这件事的完整规矩、以及它在 `-O0` 与 `-O2` 下行为不同的实测，留到第 19 单元（`c19`）讲。现在只要记住一句话：**union 换成员读是语言给的许可；指针换类型读不是。**

> ⚠ **陷阱 · 拿 union「省内存」：你会同时丢掉可读性与可移植性**
>
> 你会看到什么：写好一个 `union` 省下了 4 个字节，程序在本机跑得好好的；同事把代码挪到另一台机器（或者换一个编译器版本），结果全乱了——数字对不上、浮点变成荒唐值。
>
> 你会怎么误判：第一反应是「浮点精度问题」或者「编译器有 bug」，然后开始加打印、改运算顺序，越改越远。
>
> 真相：同一个 union 里「写 A 读 B」的结果依赖**三样实现定义的东西**：哪个成员最后被写（你忘了就是你的锅）、**字节顺序**（本机是小端，标准没钉死）、以及**类型自己的表示**（浮点是不是 IEEE-754、`int` 几位、有没有陷阱表示——§6.2.6.1 第 2 段把数量、顺序、编码都交给实现）。所以：① 每次换读法之前，注释里必须写清「现在里面装的是哪个成员」；② union 不该被拿来做「省内存的容器」——它省下的那几个字节远比不上它带来的沟通成本；③ 用 `union` 看浮点位模式是**教学与调试**的常见用法，写进产品代码前先想清楚：真要跟文件或网络打交道，就该显式规定「哪种字节序、每个字段几字节」（见 11.12 节关于 `fwrite` 的那条）。

11.9 让编译器把 layout 的账报出来：`-Wpadded`

`offsetof` 要你自己一个个写；gcc 还有一个开关会主动告诉你「这里塞了填充」：`-Wpadded`。它**不在** `-Wall -Wextra -Wpedantic` 里——这一点下面用一条命令验证。这个例子（`examples/ch05/with_padded/`）把 `-Wpadded` 写进了清单的 `flags`（`"flags": ["-Iinclude", "-Wpadded"]`）：

c examples/ch05/with_padded/include/with_padded.h 第 4–15 行

```
4   /* 两个成员：char 后面是 int，中间必然要塞 3 个字节 */
5   struct S1 {  char 后面跟 int，中间必然塞 3 个字节
6       char c;
7       int  i;
8   };
9
10  /* 三个成员：成员之间塞一次，结构体尾部再塞一次 */
11  struct S2 {  尾部还会再补一次
12      char  a;
13      int   b;
14      short s;
15  };
```

5 char 后面跟 int，中间必然塞 3 个字节
11 尾部还会再补一次

程序自己打印出来的两个 `sizeof` 是这样：

终端输出 终端输出 with-padded-out

```
sizeof(struct S1) = 8，成员只占 5
sizeof(struct S2) = 12，成员只占 7
S2 的 b 偏移 4，s 偏移 8
两个结构体都用上了：one.i = 1，two.s = 3
```

对着头文件读这两行：`struct S1` 的 `sizeof` 是 8、成员只占 5——差的 3 个字节在 `char c` 与 `int i` 之间；`struct S2` 是 12、成员只占 7——成员之间空 3 字节（`b` 在偏移 4、`s` 在偏移 8），总共用到 10 字节，尾部再补 2 字节凑成 12。

`-Wpadded` 把这个账主动报了出来：

终端会话 `-Wpadded` 的真实诊断（原文照抄） ch05-padded-session

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -Wpadded src/main.c
src/with_padded.c -o with_padded.exe
In file included from src/main.c:3:
include/with_padded.h:15:1: warning: padding struct size to alignment boundary with 2 bytes [-Wpadded]
   15 | };
      | ^
In file included from src/with_padded.c:4:
include/with_padded.h:15:1: warning: padding struct size to alignment boundary with 2 bytes [-Wpadded]
   15 | };
      | ^
$ echo $?
0
$ rm -f with_padded.exe
```

三件事要读懂：

1. **诊断位置是头文件，不是源文件**：`include/with_padded.h:15:1` 指向 `struct S2` 的收尾大括号；`In file included from src/main.c:3` 告诉你「我是从 `main.c` 第 3 行的 `#include` 进到这里的」。一个头文件被两个源文件包含，同一个问题就报**两遍**——这不是编译器啰嗦，是两个编译单元各自发现了它（第 17 单元讲编译单元时会再遇见这个现象）；
2. **它说的是「结构体尾部为了对齐补了 2 个字节」**：`struct S2` 是 `char + int + short = 7` 字节，但为了让 `sizeof` 是 `int` 的整数倍（4 的倍数），尾部补到 12。这就是 §6.7.2.1 第 17 段说的「结尾也可以有填充」；
3. **退出码是 0**：警告不是错误，程序照编照跑（上面那块例子输出就是它跑出来的）。想让它变成错误得自己加 `-Werror`——但本书的规矩是**基础参数里不许加** `-Werror`，因为我们要看的是警告本身。

实验记录 · **本机实测：`-Wpadded` 不在 `-Wall` 里，而且它只报了一部分**

- 工具链：gcc 16.1.0（`-std=c17`，即 C17），目标三元组 x86_64-w64-mingw32；
- 「不在 `-Wall` 里」的验证命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -Q --help=warnings`，输出里 `-Wpadded` 一行是 `[disabled]`；把参数换成只写 `-O0` 也一样是 `[disabled]`。所以它会一直沉默，直到你显式写上 `-Wpadded`；
- 同一个例子里，`struct S1`（`char c; int i;`）内部明明有 **3 个字节**填充，`-Wpadded` **一个字都没说**；`struct S2` 尾部的 2 个字节它报了。

为了不靠记忆，我把九种形状的探针也留在了例子目录里（`examples/ch05/with_padded/padded_probe.c`，它没有配清单，证据是下面这段会话）：

终端会话 九种形状的探针：它只报了三种 ch05-padded-probe

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -fdiagnostics-color=never -Wpadded padded_probe.c -o
padded_probe.exe
padded_probe.c:8:8: warning: padding struct size to alignment boundary with 3 bytes [-Wpadded]
    8 | struct C { int i; char c; };                /* 尾部填充 3 字节 */
      |        ^
padded_probe.c:9:8: warning: padding struct size to alignment boundary with 2 bytes [-Wpadded]
    9 | struct D { char a; int b; short s; };       /* 内部 3 + 尾部 2 */
      |        ^
padded_probe.c:11:8: warning: padding struct size to alignment boundary with 6 bytes [-Wpadded]
   11 | struct F { double d; char c; char e; };     /* 尾部 6 字节 */
      |        ^
$ echo $?
0
$ rm -f padded_probe.exe
```

九种形状里只有 **C、D、F** 被报（都是「尾部为了凑对齐补字节」）：`A`（内部 3 字节）、`B`（内部 7 字节）、`E`（内部 2 字节）、`H`（`char` + `long double`，内部 15 字节）这些**成员之间的填充一句都没提**；同样有 3 字节尾部填充的 `G`（`typedef` 出来的匿名结构体）也没报。`union U` 本来就没有填充，不在对照之列。

结论：别把 `-Wpadded` 当成「判断有没有 padding」的万能开关——**它是「带标签的结构体尾部浪费」的提醒器。**想知道成员之间的填充，还是得靠 `offsetof` 与 `sizeof` 自己算（11.3、11.4 节）。这条是本机 gcc 的行为，不是标准的规定：标准里根本没有 `-Wpadded` 这个选项。

11.10 `packed`：省下的字节，用可移植性换

`-Wpadded` 报出来的那 2 个字节，有一种「关掉」的办法：告诉编译器「这个类型一个字都不要塞」。这就是 `packed`。这个例子（`examples/ch05/struct_packed/`）把默认排法与两种 `packed` 写法摆在一起。先看默认的那一个与属性写法：

c examples/ch05/struct_packed/include/struct_packed.h 第 4–18 行

```c
/* 默认排法：编译器为了「每个成员都落在自己的对齐边界上」塞进填充字节 */
struct Unpacked {                       // 默认排法
    char c;
    int  i;
    char d[3];
};

/* 一个字都不许塞的两种写法。它们不是 C 标准的东西，是编译器扩展：
   ① __attribute__((packed)) —— 挂在类型上（gcc 认得这一种）
   ② #pragma pack(push, 1)   —— 一段区间里生效（MSVC 风格的代码常用） */
struct PackedAttr {                     // 同一个成员表，声明成 packed
    char c;
    int  i;
    char d[3];
} __attribute__((packed));              // 属性挂在收尾大括号后面
```

5 默认排法
14 同一个成员表，声明成 packed
18 属性挂在收尾大括号后面

另一种写法不改类型本身，而是把「不许塞」写成一个**区间**：进去开始生效，出来恢复原样：

c examples/ch05/struct_packed/include/struct_packed.h 第 20–26 行

```c
#pragma pack(push, 1)                   // 从这里开始「不塞」
struct PackedPragma {
    char c;
    int  i;
    char d[3];
};
#pragma pack(pop)                       // 到这里恢复
```

20 从这里开始「不塞」
26 到这里恢复

程序做的事就三件：打印三种排法的偏移与大小、取一个没对齐成员的地址去读、再用 `memcpy` 安全地读一次。先看前半段：

c examples/ch05/struct_packed/src/main.c 第 13–31 行

```c
int main(void) {
    print_packed_layout();

    struct PackedAttr pa;
    pa.c = 'A';
    pa.i = 7;
    memcpy(pa.d, "xyz", 3);

    int *ip = &pa.i;                    /* 取一个没对齐成员的地址 */ // 取没对齐成员的地址
    printf("\n&pa.c = %p, &pa.i = %p (差 %zu 个字节) \n",
           (void *)&pa.c, (void *)&pa.i, (size_t)((char *)&pa.i - (char *)&pa.c));
    printf("ip 的值 %p 能被 4 整除吗? %s\n",
           (void *)ip, ((unsigned long long)(size_t)ip % 4 == 0) ? "能" : "不能");
    printf("顺着 ip 读: *ip = %d\n", *ip); // 顺着它读
    show_member(&pa.i);                 // 把地址交给函数

    int safe = 0;
    memcpy(&safe, &pa.i, sizeof safe);   /* 对标准友好的写法：按字节搬出来 */
```

```
31      printf("用 memcpy 按字节搬出来：safe = %d\n", safe);
```

21 取没对齐成员的地址
26 顺着它读
27 把地址交给函数

（续）后半段换用 `#pragma pack` 声明的那个结构体，做同样的取地址动作：

c **（续）同一个动作，换一种声明方式** examples/ch05/struct_packed/src/main.c 第 33–41 行

```
33      struct PackedPragma pp;
34      pp.i = 99;
35      printf("\n同一个布局、用 #pragma pack 写：sizeof = %zu，i 的偏移 = %zu，直接读 pp.i = %d\n",
36             sizeof pp, offsetof(struct PackedPragma, i), pp.i);
37
38      int *jp = &pp.i;                    /* 同样没对齐，只是类型是另一种写法声明出来的 */ 同样没对齐，换一种声明方式
39      printf("顺着 jp 读 pp.i = %d（这一行编译器不报警告 —— 对比见下面的诊断原文）\n", *jp);
40      return 0;
41  }
```

38 同样没对齐，换一种声明方式

终端输出 终端输出 struct-packed-out

```
struct Unpacked    : c 偏移 0，i 偏移 4，d 偏移 8，sizeof 12
struct PackedAttr  : c 偏移 0，i 偏移 1，d 偏移 5，sizeof 8
struct PackedPragma: c 偏移 0，i 偏移 1，d 偏移 5，sizeof 8
&pa.c = 00000000005FFE98，&pa.i = 00000000005FFE99（差 1 个字节）
ip 的值 00000000005FFE99 能被 4 整除吗？不能
顺着 ip 读：*ip = 7
顺着传进来的指针读：7
用 memcpy 按字节搬出来：safe = 7
同一个布局、用 #pragma pack 写：sizeof = 8，i 的偏移 = 1，直接读 pp.i = 99
顺着 jp 读 pp.i = 99（这一行编译器不报警告 —— 对比见下面的诊断原文）
```

输出里的三个 `sizeof` 是这一节最直白的一句话：默认排法 **12** 字节，两种 `packed` 写法都是 **8** 字节（1 + 4 + 3，一个填充字节都没有）。但输出后半段就是账单了：`&pa.i` 与 `&pa.c` **只差 1 个字节**，那个地址**不能被 4 整除**。x86-64 的标量访存**允许**不对齐，所以 `*ip = 7` 照常读了出来——这不是「没问题」，而是**这台 CPU 宽容**：换成要求对齐的架构（不少 ARM）就是一次异常，而标准从头到尾没承诺过「不对齐也能读」（见下面的标准原文）。编译器对这件事的态度，看它自己说的话：

终端会话 取 packed 成员的地址：编译器的真实警告 ch05-packed-session

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/main.c -o main.o
src/main.c: In function 'main':
src/main.c:21:15: warning: taking address of packed member of 'struct PackedAttr' may result in an unaligned
pointer value [-Waddress-of-packed-member]
   21 |     int *ip = &pa.i;                    /* 取一个没对齐成员的地址 */
      |               ^~~~~
src/main.c:27:17: warning: taking address of packed member of 'struct PackedAttr' may result in an unaligned
pointer value [-Waddress-of-packed-member]
   27 |     show_member(&pa.i);
      |                 ^~~~~
$ echo $?
0
$ rm -f main.o
```

这份诊断里有两条值得单独说：

- 它**只对 `__attribute__((packed))` 声明的类型报**（两条警告都点名 `struct PackedAttr`）；同一个程序里那一句 `int *jp = &pp.i;`（`#pragma pack` 声明的类型）**一个警告都没有**。这是本机 gcc 16.1.0 的实际行为——同一个布局，两种写法，一条报、一条不报。所以别以为「没警告就是没问题」：`#pragma pack` 那一路的地址一样没对齐（输出里 `i 的偏移 = 1` 就是证据）；
- 它不拦你，只说「may result in an unaligned pointer value」。**编译器给了允许，语言没给**——这就是下一段标准原文要讲的事。

标准原文 · 标准原文：对齐是每个类型的硬要求

6.2.8 节「Alignment of objects」第 1 段：

> Complete object types have alignment requirements which place restrictions on the addresses at which objects of that type may be allocated.

大意：每个完整对象类型都有**对齐要求**，它限制这个类型的对象**能放在哪些地址上。** `packed` 让一个 `int` 成员落在偏移 1，等于让一个 `int` 对象落在违反它对齐要求的地址上——这是编译器扩展造出来的状态，**标准没有为「沿着这样的地址访问」定义行为。**

如果你再把这种地址**转成另一种对象类型的指针**，标准有一句更直接的话（6.3.2.3 节第 7 段）：

> A pointer to an object type may be converted to a pointer to a different object type. If the resulting pointer is not correctly aligned for the referenced type, the behavior is undefined.

大意：对象指针转成别的对象类型的指针后，**结果若对目标类型没有正确对齐，行为未定义。** `(long *)&pa.i` 这种写法就是直接撞上这一条。

出处：ISO/IEC 9899:2018（C17）§6.2.8 第 1 段，§6.3.2.3 第 7 段。

⚠ 陷阱 · `packed` 在本机「看起来没事」，这正是它最坑的地方

你会看到什么：程序打印出 `*ip = 7`，与直接读 `pa.i` 一模一样。你在 x86-64 的 Windows 上跑一百次都对。

你会怎么误判：于是你认为「不对齐访问没什么大不了」，把 `packed` 用在了「要跟文件/网络协议对齐」的结构体上，甚至把 `&pa.i` 这样的指针传给了别的函数（例子里的 `show_member` 就是）；等你把代码拿到 ARM 或者别的架构上，或者编译器把这段代码向量化（一次搬 16 字节的指令对对齐更敏感），就会出现「值读错」或者当场异常——而你在本机是复现不出来的。

真相：对齐要求是**每个类型自己的事**（§6.2.8 第 1 段），`packed` 造出的对象违反了它；**标准没有为沿着不对齐地址的访问定义行为**，而把这种地址转成别的指针类型更是明确的未定义行为（§6.3.2.3 第 7 段）。本机「跑对了」只说明这台 x86-64 的标量访存宽容——同一份代码在要求对齐的 CPU 上会异常，在向量化之后也可能异常。要安全地读一个没对齐的成员，标准做法是**按字节搬**：例子里的 `memcpy(&safe, &pa.i, sizeof safe)`（输出里 `safe = 7` 就是它），编译器会把这几行翻成对得起任何架构的机器码。还有一条更省事的规矩：**别把 `packed` 写进你给别人用的头文件**——它省下的字节是你们两个人的协议的一部分，一改全乱。

11.11 柔性数组成员：头部与尾巴一次分配

最后一个「结构体里带数组」的形态：数组放在**结尾**，而且**不写长度。**这叫**柔性数组成员**（flexible array member），它让「头 + 尾巴」变成一次分配（`examples/ch05/flexible_array/`）：

c examples/ch05/flexible_array/include/flexible_array.h 第 6–18 行

```c
/* 头部 + 尾巴：尾巴这个成员「不占 sizeof」，长度由头部记着。
   在栈上定义一个也行，但那样它没有尾巴的空间 —— data 只有在整块分配出来的内存里才有意义；
   这种结构体也不能作为数组元素或另一个结构体的成员 */
struct IntVec {
    size_t len;      /* 头部：几个元素 */ 头部：记着几个元素
    int    data[];   /* 柔性数组成员：sizeof 里不含它 */ 尾巴：不写长度
};

/* 一次分配「头部 + n 个 int」，返回值由调用者 free */
struct IntVec *intvec_new(size_t n);

/* 打印这个向量的元素 */
void intvec_print(const struct IntVec *v);
```

10 头部：记着几个元素
11 尾巴：不写长度

分配的时候把两段的字节数加起来，一次要够：

c examples/ch05/flexible_array/src/flexible_array.c 第 6–17 行

```c
struct IntVec *intvec_new(size_t n) {
    /* 一次要够：头部的字节数 + 尾巴要的字节数 */
    struct IntVec *v = (struct IntVec *)malloc(sizeof *v + n * sizeof *v->data); sizeof 只有头部，尾巴要自己加
    if (v == NULL) {
        return NULL;
    }
    v->len = n;
    for (size_t i = 0; i < n; i++) {
        v->data[i] = (int)(i * i);
    }
    return v;
}
```

8 sizeof 只有头部，尾巴要自己加

终端输出 终端输出 flexible-array-out

```
sizeof(struct IntVec) = 8 (柔性数组成员不计在内)
sizeof(size_t) = 8, sizeof(int) = 4
n = 3: 一次分配了 20 字节 —— len = 3, data = 0 1 4
n = 4: 一次分配了 24 字节 —— len = 4, data = 0 1 4 9
n = 5: 一次分配了 28 字节 —— len = 5, data = 0 1 4 9 16
三次分配、三次释放，程序走到这里
```

三个数字连起来读：`sizeof(struct IntVec) = 8`（本机 `size_t` 8 字节，`data` 一个字节都不算），而 `n = 3` 那次分配的字节数是 `8 + 3 * 4 = 20`——**头与尾巴在同一块内存里连续放着**，所以 `free(v)` 一次就全还了。这就是柔性数组成员的全部价值：**一次分配、一次释放、数据连续**（连续意味着遍历快、缓存友好）。

标准原文 · **标准原文：柔性数组成员**

6.7.2.1 节第 18 段：

> As a special case, the last element of a structure with more than one named member may have an incomplete array type; this is called a flexible array member. In most situations, the flexible array member is ignored. In particular, the size of the structure is as if the flexible array member were omitted except that it may have more trailing padding than the omission would imply.

大意：作为特例，**含多个具名成员**的结构体的**最后一个**成员可以是不完整数组类型，这叫柔性数组成员；多数场合它被忽略——**结构体的大小等于「把它拿掉」之后的大小**（只是可能多几个尾部填充字节）。

注意两个「必须」：它必须是**最后一个**成员；所在结构体必须**至少有另一个具名成员**（标准原文用词是 more than one named member，也就是说不能只有一个柔性数组成员）。另外它的规矩还包括：这种结构体**不能**作为数组元素、也不能作为另一个结构体的成员。

出处：ISO/IEC 9899:2018（C17）§6.7.2.1 第 18 段。

工程惯例 · **工程惯例：「一次分配」比「两次分配」好在哪**

另一种常见做法是分成两块：`struct IntVec { size_t len; int *data; };`，先 `malloc` 结构体、再 `malloc` 数组。两种都能用，惯例上的取舍是：

- **一次分配**（柔性数组）：少一次分配、少一次释放、少一次失败分支；数据连续，遍历时的缓存命中更好。代价是**大小定死**：想扩容就得再 `malloc` 一块更大的、搬过去、`free` 旧的（`realloc` 帮得上忙，见第 12 单元）；
- **两次分配**（指针成员）：扩容只需要给 `data` 做 `realloc`，结构体本身不动；代价是**多一层间接**（指针 → 数组）、多一次释放、多一处可能失败的地方，而且「谁释放」要写得更清楚（11.6 节的浅拷贝坑就在这儿等着）。

柔性数组成员是 **C99 加进来的**（在那之前大家写 `int data[1];` 再「多要一点内存」，那种写法至今还能在旧代码里见到）。要不要用它，取决于你的数据会不会长大：**定长就用它，会长大就用指针 + 扩容。**

11.12 结构体进接口：五条纪律

结构体的定义一旦写进头文件，它就变成了**接口**的一部分。下面五条都会给证据与代价；其中 ②③④⑤ 各配了一个能跑的例子（① 的证据是标准原文与本书全部例子的写法）。

① `typedef` 是惯例，不是必须。 你可以写 `typedef struct Point Point;`，之后就能写 `Point a;` 而不必写 `struct Point a;`。它为什么合法？因为 `struct` 的标签与普通标识符分属**两个命名空间**：

> **标准原文 · 标准原文：标签与普通标识符是两个命名空间**
>
> 6.2.3 节「Name spaces of identifiers」第 1 段列出了四类命名空间：
>
> > — label names (disambiguated by the syntax of the label declaration and use); — the tags of structures, unions, and enumerations (disambiguated by following any of the keywords struct, union, or enum); — the members of structures or unions; each structure or union has a separate name space for its members (disambiguated by the type of the expression used to access the member via the . or -> operator); — all other identifiers, called ordinary identifiers (declared in ordinary declarators or as enumeration constants).
>
> 大意：结构体的**标签**（`struct Point` 里的 `Point`）占一个命名空间，普通标识符（函数名、变量名、`typedef` 名字）占另一个。所以 `typedef struct Point Point;` 里那句「名字一样」根本不冲突：左边那个在标签命名空间，右边那个在普通标识符命名空间。同一节的脚注 32 补了一句：**标签只有一个命名空间**（`struct`、`union`、`enum` 的标签共用一处）——所以 `struct Foo` 与 `union Foo` 不能同时存在。
>
> 出处：ISO/IEC 9899:2018（C17）§6.2.3 第 1 段与脚注 32。

惯例上，`typedef struct { … } Point;`（匿名标签 + typedef）在库代码里很常见；而 Linux 内核风格明确要求**不要**用 typedef 掩饰结构体，理由是「读者看到 `Point a;` 不知道它是结构体还是整数」。所以这不是对错问题：**写给自己看的小工具随便，写给别人用的接口要把「这是个结构体」留在名字里**（本书的例子一律用 `struct Point`）。

② 嵌套结构体与结构体数组：都是「摊平」，都能用 `sizeof` 与 `offsetof` 量。 这个例子（`examples/ch05/struct_nested_array/`）把三种形态摆在一起：

`c` examples/ch05/struct_nested_array/include/struct_nested_array.h 第 6–21 行

```c
/* 嵌套：一个线段由两个点组成，而点本身是结构体 */
struct Point {
    int x;
    int y;
};

struct Segment {
    struct Point a;      /* 结构体成员：整个 Point 摊平在这里 */ 嵌套的结构体成员
    struct Point b;
};

/* 结构体里的数组：定长 4 个点，n 记着用了几个 */
struct Path {
    size_t n;
    struct Point pt[4]; 结构体里的数组
};
```

13 嵌套的结构体成员
20 结构体里的数组

`c` examples/ch05/struct_nested_array/src/main.c 第 5–17 行

```c
int main(void) {
    print_layout();

    /* 结构体数组：三个元素，第一种用嵌套的初始化列表 */
    struct Point line[3] = {{0, 0}, {1, 2}, {.y = 5}}; 嵌套初始化列表
    printf("\nstruct Point line[3]: 数组本身占 %zu 字节，一个元素 %zu 字节\n",
           sizeof line, sizeof line[0]);
    printf("元素个数 = %zu\n", sizeof line / sizeof line[0]); 元素个数
    for (size_t i = 0; i < 3; i++) {
```

```
        printf("  line[%zu] = (%d, %d)\n", i, line[i].x, line[i].y);
    }
    printf("line[1] 与 line[0] 隔着 %zu 个字节 —— 正好是一个 struct Point\n",
           (size_t)((char *)&line[1] - (char *)&line[0]));
```

9 嵌套初始化列表
12 元素个数

（续）后半段做两件事：在结构体里装数组并量步长；把整个结构体赋值一份，看数组有没有被一起拷走。

c **（续）结构体里的数组与整体赋值** examples/ch05/struct_nested_array/src/main.c 第 19–39 行

```
    /* 结构体里装数组：pt 的偏移与步长都能量出来 */
    struct Path path;
    path.n = 3;
    for (size_t i = 0; i < path.n; i++) {
        path.pt[i].x = (int)i;
        path.pt[i].y = (int)(i * i);
    }
    printf("\npath.pt[2] = (%d, %d)；相邻元素相差 %zu 字节\n", 相邻元素差多少
           path.pt[2].x, path.pt[2].y,
           (size_t)((char *)&path.pt[1] - (char *)&path.pt[0]));

    /* 装着数组的结构体可以整体赋值：数组也被拷了一份 */
    struct Path copy = path; 整体赋值
    copy.pt[0].x = 99;
    printf("整体赋值后改 copy.pt[0].x = 99：path.pt[0].x = %d，copy.pt[0].x = %d\n",
           path.pt[0].x, copy.pt[0].x);

    struct Segment seg = {{1, 2}, {3, 4}};
    printf("\n嵌套结构体 seg = ");
    print_segment(&seg);
    printf("，sizeof = %zu\n", sizeof seg);
```

26 相邻元素差多少
31 整体赋值

终端输出 终端输出 struct-nested-array-out

```
sizeof(struct Point) = 8
sizeof(struct Segment) = 16，成员之和 = 16
  a 偏移 0，b 偏移 8
sizeof(struct Path) = 40
  n 偏移 0，pt 偏移 8
struct Point line[3]：数组本身占 24 字节，一个元素 8 字节
元素个数 = 3
  line[0] = (0, 0)
  line[1] = (1, 2)
  line[2] = (0, 5)
line[1] 与 line[0] 隔着 8 个字节 —— 正好是一个 struct Point
path.pt[2] = (2, 4)；相邻元素相差 8 字节
整体赋值后改 copy.pt[0].x = 99：path.pt[0].x = 0，copy.pt[0].x = 99
嵌套结构体 seg = (1, 2)-(3, 4)，sizeof = 16
```

输出里四组数字各说一件事：

- `sizeof(struct Segment) = 16`，`a` 偏移 0、`b` 偏移 8——**嵌套的结构体成员是摊平在父结构体里的**，不是「一个指针指向另一个结构体」；访问要一层层写：`seg.a.x`；
- `sizeof(struct Path) = 40` = `size_t n` 8 字节 + `struct Point pt[4]` 32 字节；`pt[1]` 与 `pt[0]` 相差 **8**，正好是 `sizeof(struct Point)`——这就是「步长」，`pt[i]` 在地址上等于「起始 + i × 8」；
- `line[1]` 与 `line[0]` 也差 8；`sizeof line / sizeof line[0] = 3` 是元素个数——**数组本身不存长度**，这个数是编译期算出来的；
- **数组不能整体赋值，但「装着数组的结构体」可以**：`struct Path copy = path;` 把 4 个点一起拷走，改 `copy.pt[0].x` 不影响 `path`（最后一行 `0` 与 `99`）。判据还是 11.6 节那条：看 `sizeof` 给成员多少字节——数组与嵌套结构体是真占，指针是假占。

③ 位域：布局由实现决定，所以别写进接口。 你可以写 `struct Flags { unsigned a : 3; unsigned b : 5; };` 让两个字段挤进同一个存储单元，但标准对布局几乎什么都没承诺：

标准原文 · **标准原文：位域的布局**

6.7.2.1 节第 11 段：

> An implementation may allocate any addressable storage unit large enough to hold a bit-field. … If insufficient space remains, whether a bit-field that does not fit is put into the next unit or overlaps adjacent units is implementation-defined. The order of allocation of bit-fields within a unit (high-order to low-order or low-order to high-order) is implementation-defined. The alignment of the addressable storage unit is unspecified.

大意：① 存放位域的**存储单元**多大由实现决定；② 一个位域放不下时，是换一个单元还是**与相邻单元重叠**，由实现决定；③ 同一个单元内位域的分配顺序（从高位到低位，还是反过来）由实现决定；④ 这个单元的**对齐方式没有规定。**

出处：ISO/IEC 9899:2018（C17）§6.7.2.1 第 11 段。

三个「实现定义」加一个「未规定」，所以「两个字段挤进一个字节」这句话要打折扣：**那 8 个位确实挤在同一个字节里，但整个结构体仍要占一个完整的存储单元。**本机实测（`examples/ch05/bitfield_probe/`）：

c examples/ch05/bitfield_probe/include/bitfield_probe.h 第 6–10 行

```c
/* 两个字段挤在同一个存储单元里：a 占 3 位、b 占 5 位 */
struct Flags {
    unsigned a : 3;
    unsigned b : 5;
};
```

8 3 位给 a
9 5 位给 b

c examples/ch05/bitfield_probe/src/main.c 第 5–21 行

```c
int main(void) {
    printf("sizeof(struct Flags) = %zu (8 个位仍然要占一整个存储单元) \n",
           sizeof(struct Flags));

    struct Flags f = {0};
    f.a = 5;                                /* 101 */
    f.b = 17;                               /* 10001 */
    printf("a = 5, b = 17: ");
    dump_bytes(&f, sizeof f);

    f.b = 1;
    printf("把 b 改成 1:   ");
    dump_bytes(&f, sizeof f);

    printf("上面这串字节是这台机器的排法 —— 标准说它由实现决定\n");
    return 0;
}
```

6 sizeof 是多少
12 先看 a 为 5、b 为 17
16 只改 b

终端输出 终端输出 bitfield-probe-out

```
sizeof(struct Flags) = 4 (8 个位仍然要占一整个存储单元)
a = 5, b = 17: 8D 00 00 00
把 b 改成 1:   0D 00 00 00
上面这串字节是这台机器的排法 —— 标准说它由实现决定
```

输出里三个数字：`sizeof(struct Flags) = 4`（不是 1）、`a = 5、b = 17` 时第一个字节是 `8D`（`8D` = 二进制 `1000 1101`：低 3 位 `101` 是 `a`，接着 5 位 `10001` 是 `b`）、把 `b` 改成 1 之后变成 `0D`。这是**本机**的排法（低位在

前、存储单元 4 字节）；换一台机器可能就是另一个字节、另一个 `sizeof`。所以：**位域只该留在实现内部**，别写进给别人用的头文件。要跨实现地用「位」，就用 `unsigned` 加移位与掩码——但别以为它们没有坑：左移负数、移位量大于等于宽度都是 UB，负数右移是实现定义（第 4 单元 `c04` 讲过）。

④ 别直接把结构体 `fwrite` 出去。 三条理由叠在一起：

- **填充字节**：padding 的内容标准不承诺（§6.2.6.1 第 6 段），写进文件就是写进一串「不知道是什么」的字节；
- **字节序**：多字节成员的存储顺序是实现定义的（§6.2.6.1 第 2 段）；本机是小端，换大端机器读同一个文件就是另一个数；
- **类型宽度**：`int` 在本机是 4 字节，别的实现可以是 2；`long` 的宽度更随平台变（本机 `sizeof(long)` 是 4，64 位 Linux 上通常是 8）。

标准对 `fwrite` 的规定反而很实在：

标准原文 · 标准原文：`fwrite` 写的就是对象表示

> 7.21.8.2 节第 2 段：
>
> > For each object, size calls are made to the fputc function, taking the values (in order) from an array of unsigned char exactly overlaying the object.
>
> 大意：`fwrite` 写的就是**对象在内存里的那串字节**，一个不多一个不少——包括填充字节。本机实测（`examples/ch05/fwrite_probe/`）：
>
> 出处：ISO/IEC 9899:2018（C17）§7.21.8.2 第 2 段。

c examples/ch05/fwrite_probe/src/main.c 第 6–25 行

```c
int main(void) {
    struct Record r;
    memset(&r, 0xAA, sizeof r);          /* 先把填充字节写成 0xAA，好看见它们 */ 先把填充写成 0xAA
    r.tag = 'A';
    r.id = 0x01020304;
    r.kind = 7;

    printf("sizeof(struct Record) = %zu，三个成员只占 %zu\n",
           sizeof r, sizeof(char) + sizeof(int) + sizeof(short));
    printf("内存里的字节：");
    dump_bytes(&r, sizeof r); 看内存里的字节

    printf("fwrite 整个结构体，再读回来：\n"); fwrite 再读回来
    if (roundtrip(&r) != 0) {
        printf("  临时文件打不开或者读写失败\n");
        return 1;
    }
    printf("0xAA 是填充字节，它们也一起进了文件；id 的四个字节是 04 03 02 01（小端）\n");
    return 0;
}
```

8 先把填充写成 0xAA
16 看内存里的字节
18 fwrite 再读回来

终端输出 终端输出 fwrite-probe-out

```
sizeof(struct Record) = 12，三个成员只占 7
内存里的字节：41 AA AA AA 04 03 02 01 07 00 AA AA
fwrite 整个结构体，再读回来：
  写进去又读回来的 12 个字节：41 AA AA AA 04 03 02 01 07 00 AA AA
0xAA 是填充字节，它们也一起进了文件；id 的四个字节是 04 03 02 01（小端）
```

读输出：内存里 12 个字节，三个成员只占 7 个，其余 5 个是 `AA`（我们故意写进去的填充值）；`fwrite` 再读回来，`AA` **一个不少地跟着进了文件**——换成真正的程序，那 5 个字节可能是上一次运算留下的垃圾。所以正确做法是**自己定协议**：用固定宽度的类型（`uint32_t`）、一个字段一个字段地写、显式规定字节序。这就是「序列化」，第 14 单元（`c14`，文件与 I/O）会把这件事做完整。

⑤ **不透明结构体：把定义藏进 `.c`，头文件里只留名字。** 头文件里可以只写一句 `struct Counter;`——这是**不完整类型**（6.7.2.3 节第 4 段：类型在定义成员的收尾大括号之前是 incomplete，之后才 complete），只要你不去访问成员、不算 `sizeof`，编译器就允许你**用它的指针**。这个例子（`examples/ch05/opaque_counter/`）是真能跑的三文件工程：

c examples/ch05/opaque_counter/include/counter.h 第 4–16 行

```c
/* 不透明结构体：头文件里只有名字，没有成员表。
   调用者能拿着 struct Counter *，但看不见里面有什么，也不能在栈上开一个 */
struct Counter;

/* 造一个计数器；失败返回 NULL。返回值必须交给 counter_free */
struct Counter *counter_new(int start);

int counter_get(const struct Counter *c);

void counter_add(struct Counter *c, int delta);

/* 释放；之后这个指针就不能再用了 */
void counter_free(struct Counter *c);
```

6 只有名字，没有成员
9 造一个，失败返回 NULL
16 释放也写在接口里

c examples/ch05/opaque_counter/src/main.c 第 5–19 行

```c
int main(void) {
    struct Counter *c = counter_new(10);
    if (c == NULL) {
        printf("分配失败\n");
        return 1;
    }
    printf("初始值       = %d\n", counter_get(c));
    counter_add(c, 5);
    printf("加 5 之后    = %d\n", counter_get(c));
    counter_add(c, -3);
    printf("再减 3 之后  = %d\n", counter_get(c));
    counter_free(c);
    printf("释放之后，指针变量还在，但它指向的东西没了 —— 别再用它\n");
    return 0;
}
```

6 只拿指针
12 只能走函数
16 释放

终端输出 终端输出 opaque-counter-out

```
初始值       = 10
加 5 之后    = 15
再减 3 之后  = 12
释放之后，指针变量还在，但它指向的东西没了 —— 别再用它
```

调用者能拿到 `Counter *`、把它传给下一个函数、再交给 `counter_free`，但**看不见里面有什么**——`c->value` 在 `main.c` 里编不过。把「谁能看见定义、谁只见名字」画出来就是下面三帧：

谁能看见谁 **不透明结构体：谁能看见定义，谁只见名字**

第 1 步 定义住在 .c 里，头文件只有名字

counter.h（头文件） ：只有名字与声明　　counter.c（实现） ：完整定义在这里

counter.c → counter.h #include "counter.h"

第 2 步 调用者能用指针，看不到里面

main.c（调用者）：只拿到一个指针

main.c → counter.h #include "counter.h"
counter.c ⇒ main.c counter_new 交回指针

第 3 步 想越过接口摸成员，编译器不放行

counter.c → 成员 value 只有这里可见
main.c → 成员 value c->value 编不过

第 1 帧：`counter.c` 要 `#include "counter.h"` 才看得见声明（箭头从 `.c` 指向 `.h`），而完整的成员表只住在 `.c` 里。第 2 帧：调用者只拿到指针，`counter.c` 通过 `counter_new` 交回它（用 `=>` 表示数据流）。第 3 帧：`main.c` 里写 `c->value` 会当场编不过——本机 gcc 报的是 `error: invalid use of undefined type 'struct Counter'`（我单独写了一份两行的探针验证过），更没法在栈上写 `struct Counter c;`（不知道要开几个字节）。取舍是：

- **换来的**：定义可以随便改（加字段、换类型），只要函数签名不变，调用者**不需要重编**；实现细节不被误用；
- **付出的**：所有操作都得走函数，调用者不能直接读成员、不能按值传、不能按值存；每次访问多一层间接；**调用者也没法在栈上开一个实例**（不知道大小），所以对象怎么分配、由谁释放，都成了接口的一部分（我们的例子放在堆上，由 `counter_free` 收尾；库自己用 `static` 对象也是一种做法，但那样就不能有多个实例）。

什么时候用它？**实现细节会变、或者你想强制调用者别乱摸内部时**——库的公开接口是典型场合。什么时候不用？**成员就是稳定数据、大家一起读更方便时**（比如一个 `struct Point` 传坐标），摊开写更好读。这是惯例，不是标准：标准只规定「名字可见不等于定义可见」这件事本身。

结构体的成员里可以有数组（`int slot[4]`），而数组就有越界。这个例子（`examples/ch05/struct_oob_fanalyzer/`）是 `mode: detect` + `detect: fanalyzer` 的清单，看的是**编译器**怎么说。下面两块是会话原文，按「先看它指认哪一行、再看越界了几个字节」拆开：

终端会话 -fanalyzer：编译期指出这处越界写 ch05-struct-analyzer-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -fanalyzer -c src/main.c -
o main.o
src/main.c: In function 'main':
src/main.c:12:15: warning: stack-based buffer overflow [CWE-121] [-Wanalyzer-out-of-bounds]
   12 |     r.slot[5] = 99;                   /* 常量下标 5：跳过 4，写到数组外面去了 */
      |     ~~~~~~~~~~^~~~
  'main': events 1-4
    6 |     struct Ring r;                    /* 这个结构体本身是合法的：8 + 16 = 24 字节 */
      |                 ^
      |                 |
      |                 (1) capacity: 24 bytes
    7 |     r.len = 4;
    8 |     for (size_t i = 0; i < 4; i++) {   /* 合法下标是 slot[0] … slot[3] */
      |                        ~~~~~
      |                          |
      |                          (2) following 'true' branch (when 'i <= 3')... ->┐
```

终端会话 （续）越界了几个字节，它也算给你 ch05-struct-analyzer-2

```
   12 |     r.slot[5] = 99;                   /* 常量下标 5：跳过 4，写到数组外面去了 */
      |     ~~~~~~~~~~~~~~
      |               |
      |               (4) ⚠️  out-of-bounds write from byte 28 till byte 31 but 'r' ends at byte 24
src/main.c:12:15: note: write of 4 bytes to beyond the end of 'r'
```

编译器视角 · **编译器视角：静态分析器给的是一份「可能性」报告**

两块合起来读：`struct Ring` 是 24 字节（`8 + 4 * 4`），`r.slot[5]` 要写的是第 28 到 31 字节——**越过了 `r` 的结尾**。注意三件事：① 这段诊断来自**静态分析器**（`-fanalyzer`），编译退出码仍然是 **0**，程序能不能编过与它无关；② 它报的是 `CWE-121`（栈上的缓冲区溢出），因为 `r` 是栈上的局部对象；③ 这个形状是**常量下标**（`slot[5]`）——分析器只能跟踪它能证明的下标；把下标换成循环变量，它通常就报不出来了（第 9 单元 `c09` 里那次「循环形状只会得到 null-dereference 一类」的实测就是这个道理）。所以 `-fanalyzer` 是**一份额外的检查**，不是安全证明。

小结 · 这一节的要点

- **结构体把成员打包成一块连续内存**：声明顺序就是地址顺序，访问用 `.`（对象）与 `->`（指针）；嵌套的结构体与数组都是「摊平」进父结构体的；整体赋值是整份拷贝，但只拷到「表面」——值成员各一份，指针成员只抄地址；
- **`sizeof` 比成员之和大**，多出来的是对齐用的 padding（本机 `struct Person` 24 字节、成员 17）；`==` 根本不能比结构体，`memcmp` 也不可靠（填充字节的内容不承诺）；
- **传参按值就拷整份**（本机 `struct Payload` 68 字节），大结构体的惯例是传 `const` 指针；按值返回大结构体在 `-O0` 下也是一次真拷贝；浅拷贝的下半场是两次 `free`（本机实测 3221226356），要独立就深拷贝；
- **进接口的纪律**：类型定义可以进头文件；`packed` 用可移植性换字节（不对齐的访问没有标准保证）、位域别写进接口（本机 `sizeof` 是 4、字节排法由实现定）、结构体别直接 `fwrite`（填充字节与字节序都会进文件）、不想暴露内部就用「头文件只留名字」。

自测题 · 自测题（先自己答，再对答案）

1. `struct Person { char grade; int age; char name[4]; double score; };` 在本机 `sizeof` 是 24、成员只占 17。那 7 个字节分别在哪里？为什么 `age` 的偏移是 4 而不是 1？
2. 两个结构体的成员逐个比全都相等，`memcmp(&a, &b, sizeof a)` 却返回一个非 0 的值。最可能的原因是什么？标准里哪一处支持你的判断？
3. `struct Message b = a;`（`Message` 里有一个 `char *text`）之后，改 `b.text[0]`，为什么 `a` 看到的内容也变了？这个程序接下来两次 `free` 会发生什么（说出本机实测的现象与退出码）？
4. 什么时候该把 `#pragma pack` 或 `__attribute__((packed))` 写进一个结构体？说出它的收益与至少两条代价（含标准依据）。
5. `struct IntVec { size_t len; int data[]; };` 在本机 `sizeof` 是 8。要装 5 个 `int`，`malloc` 该要多少字节？为什么这样比「`malloc` 结构体 + 再 `malloc` 一个数组」省事？

注 · 自测题答案

1. 那 7 个字节是 padding：`grade` 占 1 字节（偏移 0），为了让 `int` 落在 4 的倍数上，偏移 1–3 空 3 个字节，`age` 在偏移 4；`name` 在偏移 8（4 字节），为了让 8 字节的 `double` 落在 8 的倍数上，偏移 12–15 又空 4 个字节，`score` 在偏移 16。合计 3 + 4 = 7 字节填充。标准依据：§6.7.2.1 第 14 段（成员按类型以「实现定义的方式」对齐）、第 15 段（内部可以有未命名填充，但开头不会有）。这些数字是**本机 gcc 16.1.0** 算出来的（`offsetof` 输出为 0/4/8/16）。
2. 填充字节的内容不一样：标准说把值存进结构体对象（包括它的成员）时，对应填充字节的位置取**未指定的值**（§6.2.6.1 第 6 段），而且 `memcmp` 的脚注专门点名了结构体里的这些「洞」内容不确定（§7.24.4.1 脚注 310）。所以「成员都一样」不等于「字节都一样」——判等要自己写逐成员比较。若真要用 `memcmp`，必须**先把所有成员写完**，再清零填充区、此后不再碰成员（先 `memset` 再赋值不行：赋值这个动作本身就会把填充字节变成未指定值）。
3. 因为整体赋值只拷了 `text` 里那个**地址**：`a.text` 与 `b.text` 指向同一块堆内存，所以改 `b.text[0]` 等于改 `a` 看到的那块内存。接下来 `free(a.text)` 之后 `free(b.text)` 是**同一块内存还两次**——本机实测：进程被终止，退出码 **3221226356**（`0xC0000374`，Windows 把这个代码叫「堆损坏」），stderr 空，stdout 只留下崩溃前 `fflush` 出去的那几行（见 `shallow_copy.json` 的 `"mode": "runtime_error"`、`"expect": { "exit": 3221226356 }`）。这个代码说明堆的内部结构被破坏，但「是哪一步发现的」它不告诉你；标准对此只有一句：行为未定义。
4. 只在两种情况下值得用：① 与某份**外部协议/文件格式**约定了逐字节布局，而且两端都按同一份声明来写；② 只在**同一个程序内部**用、只跟自己的代码打交道，并且你愿意为那几个字节承担「换编译器就重测」的代价。收益是 `sizeof` 从 12 变 8（本机实测），省下填充字节。代价至少两条：① 成员会落在不对齐的地址上（本例 `i` 的偏移是 1），沿着这种地址访问**标准没有为它定义行为**（§6.2.8 第 1 段的对齐要求被违反；再把它转成别的指针类型就是明确的 UB，§6.3.2.3 第 7 段），本机只是「CPU 宽容」；② 它不是 C 标准的东西（编译器扩展），换编译器/换架构就可能不认，而且本机实测 gcc 16.1.0 的 `-Waddress-of-packed-member` 只对 `__attribute__((packed))` 那种写法报警告，`#pragma pack` 那条路一个警告都不给——「不报警告」不等于「没问题」。要读没对齐的成员，用 `memcpy` 按字节搬。
5. `sizeof(struct IntVec) + 5 * sizeof(int) = 8 + 20 = 28` 字节（本机 `size_t` 8 字节、`int` 4 字节）。省事之处在于**头与尾巴在同一块内存里**：一次 `malloc`、一次 `free`、数据连续，没有第二处可能失败的分支，也不用写「先释

> 放哪一块」；代价是容量定死，要扩容得整块搬家（第 12 单元的 `realloc` 就是干这个的）。柔性数组成员必须是结构体的**最后一个**成员，而且这个结构体不能作为数组元素或另一个结构体的成员（§6.7.2.1 第 18 段）。

动态内存：malloc 与生命周期

◎ 学习目标 · 看完这一单元，你应该能

- 说出 `malloc` / `calloc` / `realloc` / `free` 各自做了什么，并解释为什么 `malloc` 拿到的那块内存「内容不确定」而 `calloc` 的是全 0；
- 写出一次分配的四件套：问对大小、判空、写进去、还回去，并说出「谁分配谁释放」落在哪一行；
- 用一次注定失败的扩容，把 `p = realloc(p, n)` 的错误写得现出原形，并说清失败时原来那块内存会去哪；
- 解释 `free` 之后指针为什么还在、双重释放与 use-after-free 在本机各自长什么样（给出退出码）；
- 用结构体封装一个会长的数组，接口上把所有权与边界检查写清楚，并说出本机没有 ASan/valgrind 时能靠什么补。

12.1 先看工程长什么样

学动态内存最容易走偏的地方，是把 `malloc` 当成「申请一个变量」。它不是。它做的事只有一件：**向堆要一块内存，交回一个地址**。所以这一节照老规矩来——先看**文件**：这一节的每个例子都回答一个小问题，问题的答案全部落在「那块内存在哪、谁负责还、还完以后指针怎么了」这三件事上。

目录树 **这一节的例子工程：从「上一单元刚收尾」到本节的九个例子**

第 1 步 本节开始时，前四章各有项目，第 5 章已有上一单元的十二个例子

```
examples/
├── ch01/ : 第 1 章的项目
├── ch02/ : 第 2 章的项目
├── ch03/ : 第 3 章的项目
├── ch04/ : 第 4 章的项目
└── ch05/ : 上一单元的十二个例子
```

第 2 步 本节先加五个例子，每个回答一个小问题

```
ch05/
├── malloc_basics/ : 要、写、读、还 ← 本课新增
├── calloc_zero/ : 内容对照 ← 本课新增
├── alloc_fail/ : 要得太多会怎样 ← 本课新增
├── heap_vs_stack/ : 住址与寿命 ← 本课新增
└── realloc_grow/ : 两种扩容 ← 本课新增
```

第 3 步 再加四个（帧 2 那五个仍在），本节九个到齐

```
ch05/
├── double_free/ : 还两次 ← 本课新增
├── use_after_free/ : 还完接着用 ← 本课新增
├── heap_oob_fanalyzer/ : 堆越界 ← 本课新增
└── dynamic_array/ : 动态数组 ← 本课新增
```

第 4 步 九个目录长得一样，拿一个拆开看

```
ch05/
└── malloc_basics/
    ├── malloc_basics.json : 清单：怎么编译、对哪个输出块
    ├── include/
    │   └── malloc_basics.h : 声明
    └── src/
        ├── main.c : 组装与打印
        └── malloc_basics.c : 函数的定义
```

这是第 5 章的第二单元，也是全书的第 12 单元。前面第 4 章（`examples/ch04/`）已经量过栈、堆、静态区三个区，也见过 `free` 之后指针不变；这一节第一次**完整地把堆用起来**：从 `malloc` 到 `free` 的每一步，书上的规则都要落到能

跑的程序上。上面那幅目录树的第 1 帧画的是**这一节故事的起点**：`examples/` 下已经有第 1 到第 5 章五个项目，第 5 章里躺着上一单元（结构体与对齐）留下的**十二个**例子目录——本节一个都不动它们。

第 1 帧是这一节故事的起点：五个项目都在，`ch05/` 里那十二个目录是上一单元留下的（`struct_basics`、`struct_layout`、`shallow_copy` … `struct_oob_fanalyzer`），本节只**加**不改。第 2、3 帧是本节新增的**九个例子目录**（分两帧列完，帧 3 里 `+` 标的就是后四个）：每个都是「源文件放 `src/`、声明放 `include/`、编译与对账的参数写在 `.json` 清单里」的最小工程。第 4 帧把其中一个拆开，让你看见这个形状——九个目录长得一模一样，只有文件内容不同。

与上一单元相比，目录树上只多了**这九个例子目录**、四段会话与两个探针，**没有出现新的目录类型**——`src/` + `include/` + 清单这个形状从第 4 章起就定下来了。四段会话分别放在 `double_free/`、`use_after_free/`、`heap_oob_fanalyzer/`、`alloc_fail/` 里；两个**探针**（`heap_dirs.c`、`leak_probe.c`）直接放在 `examples/ch05/` 下，故意**没有清单**：它们打印的是地址、或者要真的占住几个 GiB 内存，不适合每次构建都跑，所以证据形式是抓下来的会话（`session-heap-dirs.txt`、`session-leak.txt`，第 4 章的两个探针也是这个规矩）。

先说清这九个目录各自管哪一件事，后面每一节都会点名其中一个：

- `malloc_basics`、`calloc_zero`、`alloc_fail` —— 一块内存怎么到手、里面有什么、要不到的时候怎么办（12.2 到 12.4 节）；
- `heap_vs_stack`、`realloc_grow`、`dynamic_array` —— 堆与栈的住址与寿命、扩容、把「会长的数组」封装起来（12.7 到 12.9 节）；
- `double_free`、`use_after_free`、`heap_oob_fanalyzer` —— 还回去之后接着用、还两次、写到外面去，以及编译器与运行库各自能看出什么（12.6、12.10 节）。

> **工程惯例** · **工程惯例：每个例子一个目录，声明与定义分家**
>
> 标准对「一个 C 程序分几个文件」只有一个要求：有且只有一个 `main`。**怎么分，全是惯例。**本书从第二个项目起要求每个例子都是 `src/` + `include/` 的最小工程，理由有三条：
>
> - **每条结论都能被单独验证**：`calloc_zero` 只演示「两个分配函数的内容不一样」这一件事，出问题时你立刻知道是哪一行；
> - **接缝写在明面上**：`include/xxx.h` 里只放**声明**（函数名、参数、返回值），`src/xxx.c` 里放**定义**（函数体），`src/main.c` 只负责组装与打印；
> - **能被别人接手**：九个小目录比一个三千行的 `main.c` 好读得多。
>
> 代价是要多学一件事：头文件里写什么、不写什么。写重了会撞符号，写漏了会 `implicit declaration`——这件事在第 17 单元讲透，现在只要记住方向：**声明进头文件，定义进源文件。**

12.2 向堆要一块内存：要、写、读、还

`malloc` 的用法只有一句话：**给它一个字节数，它给你一个地址。**地址有两种：一块能用的内存，或者空指针。先看真程序——这是 `examples/ch05/malloc_basics/src/main.c` 的整个 `main`：

c examples/ch05/malloc_basics/src/main.c 第 10–20 行

```c
int main(void) {
    int *p = (int *)malloc(sizeof *p);          /* 向堆要 sizeof(int) 个字节 */
    if (p == NULL) {                            /* 分配可能失败，必须判 */ 先判空；败了就别往下走
        perror("malloc");
        return 1;
    }

    printf("malloc(sizeof(int)) 给的指针是空的吗：%s\n", (p == NULL) ? "是" : "否");
    printf("谁都没写过它，里面已经有非零字节吗：%s\n", 谁都没写过它，里面有东西
           any_nonzero_byte((const unsigned char *)p, sizeof *p) ? "有" : "没有");
    printf("自己写进去、再读回来：%d\n", write_then_read(p, 42));
```

12 先判空；败了就别往下走
18 谁都没写过它，里面有东西

后半段是「还回去」与「0 字节请求」——注意第 22 行 `free` 之后 `p` 还是原来的值。

c （续）后半段 examples/ch05/malloc_basics/src/main.c 第 21–31 行

```c
21
22      free(p);                                    /* 还回去 */ 还回去，指针一个字没变
23      printf("free 之后 p == NULL 吗: %s\n", (p == NULL) ? "是" : "否");
24      p = NULL;                                   /* 惯例：还完立刻置空 */
25      printf("自己置空之后 p == NULL 吗: %s\n", (p == NULL) ? "是" : "否");
26
27      void *z = malloc(0);                        /* 0 字节：实现定义 */ 0 字节：实现定义
28      printf("malloc(0) 是空指针吗: %s\n", (z == NULL) ? "是" : "否");
29      free(z);                                    /* free(NULL) 什么都不做 */
30      return 0;
31  }
```

22 还回去，指针一个字没变
27 0 字节：实现定义

第 11 行那个写法 `malloc(sizeof *p)` 值得盯一眼：**它量的不是类型，是「`p` 指的那个东西有多大」**。以后元素是结构体、是 `struct Node`，这个写法照样对—— `sizeof(int)` 一旦改成结构体，你忘了改的另一处就会错，而 `sizeof *p` 不会。这就是本书所有分配都写成 `malloc(n * sizeof *p)` 的原因：

{} 语法规则 · 分配一次的固定形状

`T *p = (T *)malloc(n * sizeof *p);` —— 三件事各就各位：

- `n * sizeof *p` 是**字节数**，而且这个乘法是「以后改动最小」的写法；
- 返回值是 `void *`，赋给具体类型的指针要写 `(T *)`（C 里可以省，本书一律写出来，为的是让读者看见类型）；
- `p` 的两种坏值要分清：**空指针**（要不到）与**悬垂指针**（曾经要到了、后来还了）。判空只筛得掉第一种，这句话 12.6 节会用真程序钉住。

⚠ 陷阱 · `n * sizeof *p` 这个乘法，自己也会害你

你会怎么被坑到：你记住了「`malloc(n * sizeof *p)` 是标准写法」，以为抄下来就安全了。这个乘法有两个真实的坑：

- **用 `int` 装字节数**：`int bytes = n * 4;` ——n 大一点就出了 `int` 的范围、回绕成负数。12.11 节那个例子正是这么写的：6 亿个元素，算出来是 `-1894967296`，再转成 `size_t` 去 `malloc`，等于要一个天文数字；
- **用 `size_t` 装，但乘出范围**：`size_t` 是无符号的，回绕不会报错，只会**悄悄变成一个小数**——于是你只要到了一小块内存，却按「n 个元素」去写，越界就发生了。

检查写法只有一行（`sizeof *p` 是编译期常数，这次除法是安全的，`SIZE_MAX` 来自 `<stdint.h>`）：

```c
if (n > SIZE_MAX / sizeof *p) { /* 太大了，直接拒绝 */ }
```

代价是多一次除法；收益是你不用猜「这次会不会刚好没溢出」。本例子里的 `malloc(sizeof *p)` 没有这个问题（n 就是 1）；12.9 节那个动态数组就有，见那一节末尾。

跑出来是这样：

终端输出 终端输出 malloc-basics-out

```
malloc(sizeof(int)) 给的指针是空的吗: 否
谁都没写过它，里面已经有非零字节吗: 有
自己写进去、再读回来: 42
free 之后 p == NULL 吗: 否
自己置空之后 p == NULL 吗: 是
malloc(0) 是空指针吗: 否
```

六行输出连起来读：

- **`malloc(sizeof(int)) 给的指针是空的吗: 否`** —— 本机这次要到了。第二行更要紧：**「谁都没写过它，里面已经有非零字节吗：有」**。这一句话是本节第一个反直觉点：`malloc` 给你的不是一张白纸，它给的是一块**没擦过的地**。你写进去之前读它，本机读到的是上一次用它的人留下的字节（标准只说「值不确定」，而且读一块从没写过的内存本来就不是标准保证的事——本书用它来说明「不能依赖」，不是推荐你去读）；

- **`自己写进去、再读回来：42`** —— 写进去之后才是你的；
- **`free 之后 p == NULL 吗：否`** —— 还回去这件事**不会**碰你的指针（第 4 章用 gdb 量过：`free` 前后 `p` 里是同一个地址）。所以「自己置空」是你写的，不是 `free` 送的；
- **`malloc(0) 是空指针吗：否`** —— 0 字节的请求是**实现定义**的：本机给了一个非空的、可以 `free` 的指针；标准允许它给空指针。所以「`malloc(0)` 返回什么」不能当常识背，也不能当判空逻辑的依据。

那块内存和那张指针条的关系，画出来是这样——这四帧是本单元的中心图：

第 1 帧是「没擦过的地」，第 2 帧是「写进去才归你」，第 3 帧是**悬垂指针**（dangling pointer）——地址还在、对象没了（本机实测：`free` 前后打印出来的地址一模一样；标准 6.2.4 节说对象生命期结束后指针的值变成 indeterminate，所以「地址还在」是**本机这一次**的现象，不是标准保证的）；第 4 帧是惯例上的补救：`free(p); p = NULL;`，那一帧里已经**没有箭头**了。之所以只是「补救」，是因为**它只管得住这一个指针**：同一块内存如果还有别的指针指着（别名），那些指针照样悬着。这件事的根源在标准里写得很清楚。

标准原文 · 标准原文：malloc 与 calloc 各自承诺什么

C 语言标准（ISO/IEC 9899，C17）7.22.3.3 节「The `malloc` function」：

> The `malloc` function allocates space for an object whose size is specified by `size` and whose value is indeterminate.

大意：`malloc` **只保证「这块地方给你」，不保证里面的值**——标准用的词是 `indeterminate`（不确定）。所以第 1 帧里那些旧字节不是本机的毛病，是标准明确不承诺的东西。

标准原文 · 标准原文：malloc 与 calloc 各自承诺什么（续）

同一份标准 7.22.3.4 节「The `calloc` function」：

> The `calloc` function allocates space for an array of `nmemb` objects, each of whose size is `size`. The space is initialized to all bits zero.

大意：`calloc` **多承诺一件事：把整块内存按位清 0**。两节放在一起读，「`malloc` 不清零、`calloc` 清零」这句话就有了出处——它不是经验，是两个函数各自的契约。

出处：ISO/IEC 9899:2018（C17）§7.22.3.3、§7.22.3.4。

> ⚠ **陷阱** · **「我试了，malloc 给我的都是 0」——这句话会骗你一整年**
>
> **你会怎么看到它**：写个小程序 `int *p = malloc(16); printf("%d\n", p[0]);`，跑出来是 `0`；再跑几次还是 `0`。于是你得出结论「`malloc` 会清零」，从此分配完直接开始读。
>
> **真相**：你看到的是**这台机器这一次恰好给了你一块干净的地。**本机实测：同一个程序里，我分别要 4 个字节和 16 个字节的两块内存，**4 字节那块里有非零字节（分配器留下的记账残留），16 字节那块恰好全是 0**——同样的 `malloc`，两次结果都不一样。标准说的 `indeterminate` 就是这个意思：**没有承诺。**

> ⚠ **陷阱** · **「我试了，malloc 给我的都是 0」——这句话会骗你一整年（续）**
>
> **你怎么会被坑到**：在你自己机器上永远看不到问题，换一台机器（或换一次分配顺序、换一个运行库）就开始打出荒唐的数字，而且这种 bug 极难查——它不出现在报错里，只出现在结果里。
>
> **修法只有一条**：分配完要么自己写满，要么改用 `calloc`，永远不要读没写过的那块内存。

12.3 要不到的时候：NULL、errno 与 perror

`malloc` 失败不会抛异常、不会打印任何东西——它只交回一个空指针。所以在写「要一块内存」的时候，你必须先回答一个问题：**要不到怎么办？** 12.1 节那个骨架里，这件事落在第 12 到 15 行：判空、报一句、返回。真程序用三种大小的请求把这件事量出来（`examples/ch05/alloc_fail/src/main.c`）：

c examples/ch05/alloc_fail/src/main.c 第 12–29 行

```c
static void report(size_t n) {
    errno = 0;                          /* 先清掉，免得看到上一次留下的原因 */ 先清掉上次留下的原因
    void *p = try_alloc(n);

    if (p == NULL) { 失败这条路必须处理
        printf("要 %zu 字节：失败了 (NULL) ，errno = %d (%s) \n", n, errno, strerror(errno)); 原因在 errno 里，strerror 给出文字
    } else {
        printf("要 %zu 字节：成功 (非 NULL) \n", n); 成功了也一样要还
        free(p);                        /* 谁分配谁释放 */
    }
}

int main(void) {
    report(1024 * 1024);                        /* 1 MiB */
    report(((size_t)1) << 32);                  /* 4 GiB */
    report(((size_t)1) << 40);                  /* 1 TiB */
    return 0;
}
```

13 先清掉上次留下的原因
16 失败这条路必须处理
17 原因在 errno 里，strerror 给出文字
19 成功了也一样要还

`main` 只负责挑三个大小来试：

c **（续）后半段** examples/ch05/alloc_fail/src/main.c 第 24–29 行

```c
int main(void) {
    report(1024 * 1024);                        /* 1 MiB */
    report(((size_t)1) << 32);                  /* 4 GiB */
    report(((size_t)1) << 40);                  /* 1 TiB */
    return 0;
}
```

终端输出 终端输出 alloc-fail-out

```
要 1048576 字节：成功 (非 NULL)
要 4294967296 字节：成功 (非 NULL)
要 1099511627776 字节：失败了 (NULL) ，errno = 12 (Not enough space)
```

三个数字里有一个会让人意外：

- **1 MiB 要得到**，这不意外；
- **4 GiB 也要得到**——这是本机的实测结果。分配器不是立刻把 4 GiB 的内存条铺满，它先把这块地方**记账**记下来，等你真去写的时候才逐页落实。所以「要得到 4 GiB」不等于「你有 4 GiB」；
- **1 TiB 要不到**：返回空指针，并且 `errno` 是 **12**（`ENOMEM`），旁边那行字就是 `strerror` 把 12 翻译成人话的结果。

`errno` 与 `perror` 的用法要分两件事记：**`errno` 是一个「可以被库函数改写的左值」**（标准 7.5 节：它展开成一个类型为 `int`、具有线程存储期的可修改左值；库函数失败时可能往里写一个编号——不是「任何库函数都会写」，也不是「它会一直保持上一次的值」），**`perror` / `strerror` 负责把编号翻译成文字。** `perror("malloc")` 会把 `malloc: Not enough space` 这样的**整行写到 stderr**（标准错误流），所以它不会出现在上面这块 stdout 里；`strerror(errno)` 只返回那段文字，你可以自己决定打到哪。12.1 节那段骨架里用的就是 `perror`。第 14 单元讲文件与 I/O 时，`perror` 会天天出现——打开文件失败、读失败，报错的形状都是这一套。

⚠ 陷阱 · 只判 NULL 不判 errno，你会把「原因」丢掉

你会怎么被坑到：`errno` 不是「上一次调用留下的原因」，它是**全局的、会被任何库函数改写的**一个变量。所以下面两种写法都是错的：

- `if (errno) { /* 出错了 */ }` —— `errno` 可能还留着上一次别处失败的编号，与这次分配毫无关系；
- 分配失败了却只看返回值、不看清 `errno`，日志里就只剩下一句「失败了」——是**要得太多**，还是**内存碎片太多**，你分不出来。

正确顺序：调用**之前** `errno = 0`（把上一次的状态清掉），调用**之后**立刻看返回值；要原因就在紧挨着的下一行去读 `errno`——中间不要插入别的库函数调用（它们可能把 `errno` 改掉）。上面程序第 13 行那个 `errno = 0` 就是这个意思。

🕮 标准原文 · 标准原文：NULL 是标准的，errno 不是

两件事要分开看，因为它们的来源不同。

「失败给空指针」是标准规定的。7.22.3.3 节「The `malloc` function」的 Returns 一句：

> The `malloc` function returns either a null pointer or a pointer to the allocated space.

大意：**返回的要么是空指针，要么是可用的地址**——标准把「失败」这件事就定义成空指针，没有别的出口。同一族的 `calloc`、`realloc` 也一样（7.22.3.4、7.22.3.5）。

「失败时把原因写进 `errno`」不是 ISO C 的要求。7.22.3 节整节讲分配函数，里面**一个 `errno` 都没有**；标准里 `errno` 的正式用途是给数学函数与转换函数报告 `EDOM` / `ERANGE` 这类范围问题（7.5 节、7.22.1 节）。把 `ENOMEM` 写进 `errno` 是 **POSIX** 给 `malloc` 加的要求，也是本机运行库（UCRT）的实际行为——所以你读到的 `12` 与 `Not enough space` 是**本机的事实**，不是语言的承诺。

出处：ISO/IEC 9899:2018（C17）§7.22.3.3、§7.5。

12.4 calloc：擦干净的地，以及乘法替你算

`calloc` 收两个参数：**要几个、每个几字节。**它比 `malloc` 多两件事：把整块内存清 0，以及**替你做那个乘法并检查它有没有溢出。**真程序把两块内存摆在一起（`examples/ch05/calloc_zero/src/main.c`）：

c examples/ch05/calloc_zero/src/main.c 第 13–23 行

```c
13      int *m = (int *)malloc(sizeof *m);          /* 1 个 int，不清零 */ 1 个 int，不清零
14      if (m == NULL) {
15          perror("malloc");
16          return 1;
17      }
18      int *c = (int *)calloc(COUNT, sizeof *c);   /* COUNT 个 int，清零 */ 4 个 int，清零
19      if (c == NULL) {
20          perror("calloc");
```

```
        free(m);
        return 1;
    }
```

13 1 个 int，不清零
18 4 个 int，清零

下面这几行是「看结果」的部分：逐个字节问一遍，再把 `calloc` 的四个元素打印出来。

c **（续）后半段** examples/ch05/calloc_zero/src/main.c 第 25–33 行

```c
    const unsigned char *mb = (const unsigned char *)m;
    const unsigned char *cb = (const unsigned char *)c;

    printf("malloc 那 1 个 int 里有非零字节吗：%s\n",
           any_nonzero_byte(mb, sizeof *m) ? "有" : "没有");
    printf("calloc 那 %d 个 int 里有非零字节吗：%s\n",
           COUNT, any_nonzero_byte(cb, COUNT * sizeof *c) ? "有" : "没有");
    printf("calloc 它们打印出来：%d %d %d %d\n", c[0], c[1], c[2], c[3]);
    printf("calloc 它们加起来：%ld\n", sum_of_ints(c, COUNT));
```

28 同一件事，两种结果
32 四个元素都是 0

终端输出 终端输出 calloc-zero-out

```
malloc 那 1 个 int 里有非零字节吗：有
calloc 那 4 个 int 里有非零字节吗：没有
calloc 它们打印出来：0 0 0 0
calloc 它们加起来：0
```

同一台机器、同一次运行，两个结果正好相反：`malloc` 那块**有**非零字节，`calloc` 那块**没有**；把 `calloc` 那四个 `int` 打印出来是四个 `0`，加起来也是 `0`。这就是 12.2 节那条标准原文的实物版：`malloc` 的值不确定、`calloc` 的按位清 0。

实验记录 · **本机实测：这一节的工具链与三个读数**

- 工具链：gcc 16.1.0（本书统一 `-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32，运行库是 UCRT64；
- 复现命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/calloc_zero.c -o app.exe` 然后 `./app.exe`，编译**一条警告都没有**；
- 读数：`malloc` 那 1 个 `int` 里有非零字节（**有**）、`calloc` 那 4 个 `int` 里没有（**没有**）、`calloc` 的四个元素全是 `0`；
- 稳定性：这个程序我一连跑了 **10 次**，输出逐字相同——因为「`calloc` 给了 0」和「`malloc` 那块有残留」在本机这一次是稳定的；但**换成 16 字节的请求，同一个程序再跑 12 次，`malloc` 那块恰好全是 0**。所以上面这些读数只说明「这一次是这样」，说明不了「`malloc` 会给什么」。

这条纪律对本书的所有例子都成立：能被写进输出块的数字，都是流水线真跑出来、逐字对账过的；而「这次恰好是 0」这种结论，只能当现场看，不能当规律背。

工程惯例 · **工程惯例：什么时候用 calloc，什么时候用 malloc**

两条惯例，都有代价：

- **要一块「结构上必须是空」的内存，用 `calloc`**：数组、计数表、要拿去当 hash 表用的桶——这些地方「0」是有意义的值，省掉一次 `memset` 也省掉一次忘写的机会。代价是**「清 0」这件事本身不免费**：标准要求那些字节是 0，实现可以真的写一遍，也可以让你拿到一页「写时才复制」的零页——本机实测 `calloc(1, 4 GiB)` 与 `malloc(4 GiB)` 都是 **0 毫秒**返回，说明它没有真去写那 4 GiB。所以**既别指望它一定花掉那次写，也别指望它一定省下**：这是实现的事；
- **别的场合用 `malloc`**：马上要整体写一遍的地方（比如 `memcpy` 一个文件进来），先清零是白干。代价是你得自己记住「这块内存现在还没初始化」。

工程惯例 · **工程惯例：什么时候用 calloc，什么时候用 malloc（续）**

还有一条常被说成「`calloc` 更安全」的事，要拆开看：**标准没有要求 `calloc` 检查那个乘法**（7.22.3.4 节一个字都没提溢出），检查是实现的自由。本机实测 `calloc(SIZE_MAX, 4)` 给空指针、`errno` 是 12——但同一台机器上 `malloc(SIZE_MAX * 4)` 也给空指针，所以这一次**两者分不出来**。真正稳的结论只有一条：用 `malloc` 时那个乘法是**你自己**算的，算错了（回绕、或者用 `int` 算）就是另一个故事，12.11 节末尾会看到它真被抓一次。

12.5 一块内存的一生：分配 → 使用 → 释放

前面三节讲的是「怎么拿到手」。这一节把整条线一次画完：一块堆内存从出生到死亡只有三步，而**出错的地方只有两条岔路**——忘了还，和还两次。先把这两条岔路画出来，后面每一节都是在这张图上补细节。

图 / 搜索过程 **一块内存的一生，以及两条岔路**

第 1 步 分配 —— 从堆里切一块给你

`malloc` ：按字节数从堆里切一块　　`p` ：拿到地址，里面的值不确定

`malloc` → `p` 地址交到你手里

第 2 步 使用 —— 写它、读它，全顺着这张纸条

那块内存 ：写进去之后才是你的东西

`p` → 那块内存 写与读都要经过它

第 3 步 释放 —— 还回去

`free(p)` ：那块内存回到分配器手里　　`p` ：纸条还在，地址一个字没变

第 4 步 岔路一：忘了还（泄漏）

那块内存 ：一直占着，进程结束前谁也拿不到　　分配器 ：账上永远欠着这一块

第 5 步 岔路二：还两次（double free）

`free(p)` 第一次 ：正常还回去　　`free(p)` 第二次 ：同一块还两次，账本乱掉

第 1 到 3 帧是正路：**分配要判空，使用要先写，释放要有人负责。**第 4 帧是**泄漏**（leak）：那块内存没有被还回去，程序还活着，但它永远拿不回来了——它不可观测、不报错，所以你只能靠纪律发现它，本机也没有工具替你数（12.11 节）。第 5 帧是**双重释放**：12.6 节会用真程序量出它的后果。

泄漏有多难发现，值得单独量一次。`examples/ch05/leak_probe.c` 是一个**故意不还**的探针：一块一块要 1 MiB、每块都碰一下第一个字节、**一块都不还**，一直要到 4 GiB 为止。它跑出来是这样：

终端会话 泄漏 4 GiB：一句抱怨都没有，退出码 0　　ch05-leak-term

```
$ ./leak_probe.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
泄漏了 4096 块 × 1 MiB，第 4097 次也还是要到了（没报错）
退出码 0
```

这四行里没有一个字说「你泄漏了」：程序把 4 GiB 全攥在手里、正常退出（代码走的是 `n == CAP` 那条，所以打的是「也还是要到了」）。**这就是泄漏的可观测现象：它在进程内部没有现象。**你能看见它的场合只有两个——进程的内存占用一路涨（任务管理器里那个数），或者某一天分配开始返回空指针（把 `leak_probe.c` 的上限再放大，就会走到「要不到内存了」那一条）。两者都不是「报错」，所以这件事只能靠纪律：谁分配谁释放。

工程惯例 · **工程惯例：谁分配谁释放，写进接口而不是记在脑子里**

标准只管到「`free` 不能还两次」，它**一个字都没说谁该还。**这件事是接口设计问题，主流惯例是三条：

- **分配的责任与释放的责任写在同一份接口上**：`include/dynamic_array.h` 第 9 到 12 行那三句话就是这个形状——「谁创建的、谁负责还、还完什么样」写在函数声明旁边。12.9 节会看到它落在代码上；
- **`malloc` 的函数把所有权交给调用者**：函数返回一个「新要来的指针」，它就不该在里面偷偷释放（12.9 节的 `xstrdup` 就是这样）；
- **函数内部的临时内存，函数自己释放**：如果一个 `malloc` 出来的块从没离开过这个函数，那就在**每一条返回路径上**都还掉。

代价要说清：所有权写进注释不会让编译器替你检查，写漏了照样泄漏；收益是**接手的人不用猜**——这正是 12.7 节那次「失败了以后该 free 哪一个」能被一眼看出来的原因。

大白话 · **用大白话说清「生命周期」与「所有权」**

把堆想成**自助仓库**，`malloc` 是「开一格」，`free` 是「退租」。

- **生命期**：从你开格那一刻起，到你退租那一刻止。这期间格子里的东西一直在、地址也不变；退租之后，仓库可以把这一格租给别人；
- **所有权**：本子上写着「这一格是谁开的」。谁开的谁负责退，不然仓库就一直被这一格占着——这就是泄漏；
- **`free` 不会撕掉你手里的取物单**：单据上那个格号还在，你照着号码再去开那一格的门，就是往**别人的**格子里伸手——这就是 use-after-free 与双重释放共同的根源。

严谨的说法只有一句：**指针是「格号」，对象是「格子」**，这两样要分开记账。

12.6 还回去之后：双重释放与现实中的 UB

先把「正确做法」和「错误做法」摆在一起。这个例子里的正确做法是一个小函数：还回去、并且立刻把指针置空（`examples/ch05/double_free/src/double_free.c`）：

c examples/ch05/double_free/src/double_free.c 第 6–12 行

```c
void free_once(int **p) {
    if (p == NULL || *p == NULL) { 空指针：什么都不做
        return;                    /* 空指针：什么都不做 */
    }
    free(*p); 还回去
    *p = NULL;                     /* 置空，第二次调用就走上面那条路了 */ 置空，第二次就走上面那条路
}
```

7 空指针：什么都不做
10 还回去
11 置空，第二次就走上面那条路

`main` 里的两条路是分开的：先用 `free_once` 还两次（安全），再**直接** `free` 同一个指针两次（不安全）：

c examples/ch05/double_free/src/main.c 第 25–35 行

```c
    *p = 42;
    printf("第一次 free 之前 *p = %d\n", *p); 第一次还之前先看一眼
    fflush(stdout);                    /* 让这两行先离开缓冲区 */ 让这两行先离开缓冲区
    free(p); 第一次 free
    printf("第一次 free 走完了\n");
    fflush(stdout);

    free(p);                           /* 第二次：同一块内存还两次 */ 第二次：同一块内存还两次
    printf("这一行永远看不到\n");
    return 0;
```

```
35  }
```

```
26 第一次还之前先看一眼
27 让这两行先离开缓冲区
28 第一次 free
32 第二次：同一块内存还两次
```

第 27 行的 `fflush(stdout)` 不是装饰：**程序被终止时，缓冲区里还没写出去的东西会一起丢掉**，不加它你连前两行都看不到，还会以为「它在第一次 `free` 之前就崩了」。跑出来是：

终端输出 终端输出 `double-free-out`

```
用 free_once 还两次：程序还活着，safe 是空指针吗：是
第一次 free 之前 *p = 42
第一次 free 走完了
```

三行都打印出来了，程序却没走完——它死在**第二次 `free` 里面**。这一句在本机会话里量得出来：

终端会话 编译期先说一遍：`-Wuse-after-free`，但它不拦你 `ch05-double-free-term`

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/double_free.c -o double_free.exe
src/main.c: In function 'main':
src/main.c:32:5: warning: pointer 'p' used after 'free' [-Wuse-after-free]
   32 |     free(p);                                /* 第二次：同一块内存还两次 */
      |     ^~~~~~
src/main.c:28:5: note: call to 'free' here
   28 |     free(p);
      |     ^~~~~~
```

真正要命的是**运行期**：三行输出都留下了，进程却没走完。

终端会话 （续）运行现场：三行输出都留下了，进程随后被终止 `ch05-double-free-run`

```
$ ./double_free.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
用 free_once 还两次：程序还活着，safe 是空指针吗：是
第一次 free 之前 *p = 42
第一次 free 走完了
退出码 127
$ rm -f double_free.exe
```

这里出现了一个让人发懵的 `127`。它**不是** `128 + 信号号`——那是 POSIX 上「被信号杀死」的写法（第 4 章那个 139 才是 `128 + 11`）。MSYS 的 bash 在 Windows 上把这种「进程被系统终止」折叠成了 `127`；直接问 Windows 要退出码（`Start-Process ... | % ExitCode`）拿到的是 **-1073740940**，它的无符号写法正是 **3221226356**（`0xC0000374`）。本书清单里一律写后者，因为流水线抓回来的就是它——`examples/ch05/double_free/double_free.json` 里写的是 `"mode": "runtime_error"`、`"expect": { "exit": 3221226356 }`，那个数字是从真跑里抄下来的，不是猜的。**三个数字是同一件事的三种写法**：bash 的 127、PowerShell 的 -1073740940、清单里的 3221226356。

> ⚠ **陷阱** · **双重释放的三种常见形状，和它最坑的地方**
>
> **你会怎么被坑到**：`free` 两次通常不会写在相邻两行（那样太蠢了），它会藏在三种形状里：
>
> - **两个分支各还一次**：正常路径还了，出错路径也还了，某个条件让你两条都走过；
> - **两个人共用一块内存**（别名）：你把指针交给别人、自己也留着，各自以为「我才是负责人」——这与「谁分配谁释放」那条惯例是同一枚硬币的两面；
> - **循环里还错元素**：清空列表或数组时把同一个元素释放两次（第 13 单元讲链表时会正面遇到）。

⚠ 陷阱 · 双重释放的三种常见形状，和它最坑的地方（续）

最坑的地方是：**运行期**没有任何文字提示——本机终止时 stderr 是空的，唯一的线索是退出码 3221226356。编译期倒是有一句警告（上面会话第 3 行 `warning: pointer 'p' used after 'free' [-Wuse-after-free]`），但它**只警告、不拦你**：程序照样编出来、照样跑。而退出码这件事本身也不牢靠——第 4 章量过同一个错误程序，只因为启动路径不同，就在 3221226356 与 3221225477 之间跳。

真相：标准说的是「UB」，运行库给什么就是什么。你能做的不是「预测它怎么死」，而是让这一行**不可能被执行第二次**：`free_once` 那种「还完立刻置空」的写法，把第二次调用变成了一次无害的空指针检查。

标准原文 · 标准原文：free 的契约，以及「还两次」为什么是 UB

C 语言标准（ISO/IEC 9899，C17）7.22.3.2 节「The `free` function」：

> The `free` function causes the space pointed to by `ptr` to be deallocated, that is, made available for further allocation. If `ptr` is a null pointer, no action occurs. Otherwise, if the argument does not match a pointer earlier returned by a memory management function, or if the space has been deallocated by a call to `free` or `realloc`, the behavior is undefined.

大意三句：`free` 把空间**还回去、让别人能再用**；`ptr` 是空指针时**什么都不做**（所以 `free_once` 第二次调用是安全的，这不是巧合，是标准给的）；除此之外，指针不是分配函数给的、或者**这块空间已经被释放过**，行为就是未定义的——最后这一句就是「双重释放是 UB」的出处。

标准原文 · 标准原文：free 的契约，以及「还两次」为什么是 UB（续）

顺带一句：`free` 没有「把指针置空」这个动作，标准里也从来没有承诺过它。指针是你变量里的一个值，`free` 拿到的只是这个值的一份拷贝——**它连你是谁都不知道。**

出处：ISO/IEC 9899:2018（C17）§7.22.3.2。

还回去之后接着用：这才是「优化档位」的正样本

双重释放已经是后果最明显的那个错。更常见的是**还回去之后接着用**（use-after-free）：读它、写它，编译照过、程序照跑。真程序（`examples/ch05/use_after_free/src/main.c`）：

c examples/ch05/use_after_free/src/main.c 第 16–26 行

```c
    *p = 42;
    printf("释放前 *p = %d\n", *p);

    free(p);                                /* 还回去了 */ 还回去了
    printf("释放后立刻读一次 *p = %d\n", *p);   /* 读已经还回去的内存 */ 读已经还回去的内存
    *p = 7;                                 /* 再往里面写一次 */ 再往里面写一次
    printf("写进去再读一次 *p = %d\n", *p);
    note_after();

    return 0;
}
```

```
19 还回去了
20 读已经还回去的内存
21 再往里面写一次
```

编译它，gcc 会在**编译期**就给三处警告（`-Wuse-after-free` 那一族）；但程序照样编得出来、也照样跑。先看其中一处：

终端会话 编译期先说一遍：-Wuse-after-free，但它不拦你 ch05-uaf-warn

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/use_after_free.c -o uaf_o0.exe
src/main.c: In function 'main':
src/main.c:22:5: warning: pointer 'p' used after 'free' [-Wuse-after-free]
   22 |     printf("写进去再读一次 *p = %d\n", *p);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
src/main.c:19:5: note: call to 'free' here
   19 |     free(p);                            /* 还回去了 */
      |     ^~~~~~
```

同一份源码换一个优化档位，行为就不一样了——下面两块是两次**运行**的原文：

终端会话 -O0：读到的是分配器留下的残留，退出码 127（Windows 口径 3221226356） ch05-uaf-o0

```
$ ./uaf_o0.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
释放前 *p = 42
释放后立刻读一次 *p = 7651200
写进去再读一次 *p = 7
到这行为止，程序看起来一切正常
退出码 127
```

终端会话 （续）同一份源码 -O2：那一行也是个荒唐的数，退出码却是 0 ch05-uaf-o2

```
$ ./uaf_o2.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
释放前 *p = 42
释放后立刻读一次 *p = 1163136
写进去再读一次 *p = 7
到这行为止，程序看起来一切正常
退出码 0
```

两段会话要连起来读，注意**同一份源码**这四个字：

档位	释放后立刻读一次 *p	退出码（Windows 口径）
`-O0`（本书基础参数）	一个荒唐的大数（这次是 7651200）	3221226356
`-O2`	另一个荒唐的数（这次是 1163136）	0

实验记录 · **本机实测：这一处 UB 在两档下的读数**

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 复现命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/use_after_free.c -o uaf_o0.exe`，以及把 `-O0` 换成 `-O2` 的同一条命令（会话文件 `session-after-free.txt` 里有完整原文）；
- **`-O0` 那一行「读出来的数」通常每次都不同**（不保证：我也量到过两次相同的读数）：我一共量到过 7496080、6906256、1008016、7430544、7651200 五个值。它读的是分配器写在那块内存里的**记账残留**，所以「读到了什么」本身不是可以依赖的东西；

实验记录 · **本机实测：这一处 UB 在两档下的读数（续）**

- **`-O2` 那一侧退出码是 0，`-O0` 那一侧是 3221226356。**用 `gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude -S src/main.c -o main-O2.s` 看一眼汇编就明白：`-O2` 把那两次**写**都删掉了，`*p = 42` 与 `*p = 7` 后面那两次读各自被折成了常数 42 与 7（所以那两行照样打印 42 和 7）；**但 `free` 之后那次读没有被折掉**，汇编里它是一条实打实的 `movl (%rbx), %edx`，读的正是分配器的残留——这就是「释放后立刻读一次」那一行两档都荒唐、而且每次可能不同的原因。被删掉的是两次写，于是**没人写坏分配器的账本**，进程退出时自然不出事。这属于「UB 之下编译器有多大自由度」，第 19 单元收口；
- 清单 `examples/ch05/use_after_free/use_after_free.json` 写的是 `"mode": "detect"`、`"detect": "opt-diff"`、`"expect": { "differs": "stdout" }`：闸门按工具链配置里的两档（`-O0` 与 `-O2`）各编一次、各跑一次，两档的 stdout 不同才算这份证据成立。

它证明的是什么，要说清楚：opt-diff 只能证明「这里存在优化敏感的未定义行为」，它不能断定 UB 的种类，也不能说明「换了档位一定不会出错」——`-O2` 这次退出 0，只是因为它的选择恰好没写坏账本。

12.7 realloc：正确与错误两种扩容

`realloc` 是唯一一个「要到手之后还能变大」的分配函数。它有两个特点，一个让人舒服，一个让人栽跟头：

- 舒服的：**老内容会跟着过来**（标准原文见下），所以它是「数组长大」的标准做法；
- 栽跟头的：它的返回值**可能是新地址，也可能是空指针**。写成 `p = realloc(p, n)` 的那一瞬间，老地址就被盖掉了——失败时你既没有新块，也找不回老块。

把两种写法放进同一个模块里对比（`examples/ch05/realloc_grow/src/realloc_grow.c`）：

c examples/ch05/realloc_grow/src/realloc_grow.c 第 9–17 行

```c
size_t grow_checked(int **p, size_t old_n, size_t new_n) {
    int *tmp = (int *)realloc(*p, new_n * sizeof **p);   /* 先用临时指针接住 */ 先拿临时指针接住
    if (tmp == NULL) {
        return old_n;                  /* 失败：原来的那一块还在，*p 没动 */ 失败：老块没动，把老的长度还回去
    }
    *p = tmp;                          /* 成功了才改 *p */ 成功了才改
    return new_n;
}
```

10 先拿临时指针接住
13 失败：老块没动，把老的长度还回去
15 成功了才改

错误写法只差一行，问题就出在那句赋值上：

c （续）后半段 examples/ch05/realloc_grow/src/realloc_grow.c 第 19–26 行

```c
int grow_bad(int **p, size_t new_n) {
    *p = (int *)realloc(*p, new_n * sizeof **p);   /* 错：返回值直接盖掉老指针 */ 错：返回值直接盖掉老指针
    if (*p == NULL) {
        return -1;                     /* 这时原来那块内存已经没人记得了 */
    }
    return 0;
}
```

20 错：返回值直接盖掉老指针

`main` 用**一次注定失败的扩容**把两者分开——数字看着不大，乘上 4 个字节就装不下了：

c examples/ch05/realloc_grow/src/main.c 第 24–34 行

```c
    size_t n = grow_checked(&v, 2, 4);          /* 正确写法：2 个扩到 4 个 */ 正确写法：2 个扩到 4 个
    v[2] = 30;
    v[3] = 40;
    printf("正确写法扩到 %zu 个：老数据还在 %d %d，新数据 %d %d\n",
           n, v[0], v[1], v[2], v[3]);

    int *before = v;
    size_t still = grow_checked(&v, 4, TOO_BIG);  /* 同一条路，只是这次会失败 */ 同一条路，只是这次会失败
    printf("要一个装不下的量：grow_checked 返回 %zu（原来的个数没变）\n", still);
    printf("v 还是那一块内存吗：%s；v[0] 还是 %d\n", (v == before) ? "是" : "否", v[0]); 失败之后老指针还在
    free(v);
```

24 正确写法：2 个扩到 4 个
31 同一条路，只是这次会失败
33 失败之后老指针还在

错误写法那一半也值得看原文——它就在同一个 `main` 的后半段：

c examples/ch05/realloc_grow/src/main.c 第 36–48 行

```c
    int *bad = ints_new(2);
    if (bad == NULL) {
```

```c
        perror("ints_new");
        return 1;
    }
    bad[0] = 10;
    bad[1] = 20;
    errno = 0;
    int rc = grow_bad(&bad, TOO_BIG);            /* 错误写法：返回值盖掉了老指针 */ 返回值盖掉老指针
    int saved = errno;                           /* 原因必须在紧挨着的地方取走 */ 原因先存下来
    printf("错误写法：返回 %d，bad 现在是空指针吗：%s\n", rc, (bad == NULL) ? "是" : "否"); 再打印
    printf("这时 errno 说：%s\n", strerror(saved));
    /* bad 已经是 NULL，老那块内存的地址丢了 —— 这就是泄漏 */
```

44 返回值盖掉老指针
45 原因先存下来
46 再打印

终端输出 终端输出 realloc-grow-out

```
正确写法扩到 4 个：老数据还在 10 20，新数据 30 40
要一个装不下的量：grow_checked 返回 4（原来的个数没变）
v 还是那一块内存吗：是；v[0] 还是 10
错误写法：返回 -1，bad 现在是空指针吗：是
这时 errno 说：Not enough space
```

五行输出要分三段读：

- **第 1 行是成功的那一组**：从 2 个扩到 4 个之后，`v[0]`、`v[1]` 还是 10 和 20——老内容过来了。这是 `realloc` 的承诺，不是运气；
- **第 2、3 行是 `grow_checked` 失败的那一次**：同一份代码、只是这次要得太多，`realloc` 给回空指针。**关键在「老指针有没有被改」**：返回值仍是原来的个数 `4`，`v` 还是那一块、`v[0]` 还是 `10`——所以调用者还能照常 `free(v)`；
- **第 4、5 行换成了另一个函数 `grow_bad`**：同一个「要得太多」，但返回值直接盖掉了老指针，于是 `bad` 变成空指针，**原来那块内存的地址已经没人记得了**——这就是泄漏，而且是「分配器还在替你记着、你自己永远拿不回来」的那一种。第 5 行那段文字来自本机的 `strerror`：`Not enough space`。代码里第 45 行的 `int saved = errno;` 是 12.3 节那条规矩的落实：**原因要在紧挨着的地方取走**，别等两行 `printf` 之后再去看它。

`realloc` 的另一半是**搬家的代价**：

内存形态 **realloc：内容会被搬走，地址可能变**

第 1 步 原来那一块装 2 个

v ：纸条上写着地址 A

v → 老块（2 个） 容量只有 2

第 2 步 要 4 个，分配器先试原地加长

v ：纸条上还是 A　　老块（2 个）：后面正好有空位，就地把容量加到 4

第 3 步 后面没空位时换一块地，内容先搬过去

新块（4 个）：realloc 返回的就是它的地址　　老块（2 个）：已经被还回去了，别再碰

v ：只有「成功之后」才被赋成新地址

第 4 步 失败那一次，两块都要看

realloc 返回 NULL：什么都没分配，新地址不存在　　老块（2 个）：还在原地，内容一个字没动

调用者手里的 v：仍然指着老块 —— 所以还要 free 它

第 1、2 帧是「原地加长」，第 3 帧是「换地方」：**有没有搬家由分配器决定，你的代码两种都要能活。**这正是「必须用临时指针接住」的原因——`p = realloc(p, n)` 在第 3 帧那种情况下的写法没错，可到了第 4 帧就会丢掉老地址。第 4 帧是本节的中心：**失败时返回的不是「新地址」而是「空指针」，老块一个字节都没动**，所以那份责任还在你身上。

标准原文 · **标准原文：realloc 承诺了什么、失败时不许做什么**

C 语言标准 7.22.3.5 节「The `realloc` function」：

> The `realloc` function deallocates the old object pointed to by `ptr` and returns a pointer to a new object that has the size specified by `size`. The contents of the new object shall be the same as that of the old object prior to deallocation, up to the lesser of the new and old sizes. … If `size` is nonzero and memory for the new object is not allocated, the old object is not deallocated.

标准原文 · **标准原文：realloc 承诺了什么、失败时不许做什么（续）**

大意两句：`realloc` **把老对象还掉、给一个指定大小的新对象**，新对象里「两边都有的那部分字节」必须与老的一模一样（这就是「老数据还在」的出处）；最后一句更关键——**要一个非零大小而要不到时，老对象不会被释放**。所以「`realloc` 失败会导致内存泄漏」这句话本身就错了：**泄漏的是你那句** `p = realloc(p, n)` ——标准把老块留给了你，是你的赋值把它弄丢了。

出处：ISO/IEC 9899:2018（C17）§7.22.3.5。

工程惯例 · **工程惯例：扩容为什么按倍长，而不是每次加一个**

`dynamic_array` 那个例子里，容量是 2 → 4 → 8 这样**翻倍**长的（12.9 节的输出能看到这三个数）。为什么不是「满了就加 1」？算一笔账就清楚了：

- **每次加 1**：从 1 个装到 n 个，要扩容 n 次，而每次扩容都要把已有内容搬一遍——总共搬 `1 + 2 + … + n`，是 **n^2 量级**；1 万个元素要搬 50,005,000 次（约 5 千万），而且随元素个数平方增长；
- **按倍长**：第 k 次扩容搬 `2^k` 个元素，而后面的容量涨得更快，总搬运量是 **2n 量级**——均摊到每一次 push 上是常数（这就是「摊销代价」的意思）。代价是**最多浪费一半空间**：容量 8、装了 5 个，那 3 个位置是空着的。

所以主流做法是「翻倍」或「乘 1.5」，选哪个是空间与时间的取舍；**不能选的是「每次加 1」**。本例子用 `realloc` 实现扩容还有一个附带的好处：分配器在能原地加长时会真的原地加长，那一半的搬运就被省掉了——**但不能指望它**，代码必须在「换了地方」时也正确。

12.8 堆与栈：住址、寿命、大小

前面几节一直在「堆」上做事。这一节把堆和栈摆在一起，量三件事：**东西住在哪、活多久、什么时候非用堆不可**。地址是最容易量错的东西，所以这个例子的程序里**一个绝对地址都不打印**——它只打印**关系**（谁比谁低、差多少、还回去的那块会不会被立刻重用）。原因是：本机的堆地址**每次运行都不一样**，把它贴进输出块会立刻变成一条对不上账的假证据。

「寿命」那一半用两个函数对照，它们的差别只有一句话（`examples/ch05/heap_vs_stack/src/heap_vs_stack.c`）：

c examples/ch05/heap_vs_stack/src/heap_vs_stack.c 第 15–19 行

```c
int *stack_block(void) {
    int local = 7;                  /* 住在栈上，函数一返回就没了 */
    return &local;                  /* 故意写的错：交出一张马上作废的地址条 */
}
```

16 住在栈上，函数一返回就没了
18 故意写的错：交出一张马上作废的地址条

`main` 先用它和「堆版本」对照，再把地址方向量出来：

c examples/ch05/heap_vs_stack/src/main.c 第 18–27 行

```c
    int *h = heap_block();
    printf("函数返回之后，堆上的 *h 还是 %d\n", *h); 堆上的那块活过了函数返回

    int *s = stack_block();
    printf("函数返回之后，那份栈上的地址是空指针吗：%s\n", (s == NULL) ? "是" : "否"); 栈上那份地址已经是废纸

    size_t n = walk_stack(stack_addr, DEPTH);
    printf("栈上连着走了 %zu 层，最后那层比第一层低吗：%s（低 %td 字节）\n", 栈的方向：一层层往里走，地址一路变低
           n, (stack_addr[n - 1] < stack_addr[0]) ? "是" : "否",
           (char *)stack_addr[0] - (char *)stack_addr[n - 1]);
```

19 堆上的那块活过了函数返回
22 栈上那份地址已经是废纸
25 栈的方向：一层层往里走，地址一路变低

第二块量的是堆：

c examples/ch05/heap_vs_stack/src/main.c 第 29–34 行

```c
    size_t got = alloc_many(heap_addr, BLOCKS);

    void *kept = heap_addr[1];
    free(heap_addr[1]);                 /* 把中间那块还回去 */ 把中间那块还回去
    void *again = malloc(sizeof(int));  /* 再要一小块 */
    printf("刚才还回去的那块，是不是被立刻分了出来：%s\n", (again == kept) ? "是" : "否"); 刚还回去的那块被立刻再分了出来
```

32 把中间那块还回去
34 刚还回去的那块被立刻再分了出来

终端输出 终端输出 heap-vs-stack-out

```
函数返回之后，堆上的 *h 还是 42
函数返回之后，那份栈上的地址是空指针吗：是
栈上连着走了 4 层，最后那层比第一层低吗：是（低 192 字节）
刚才还回去的那块，是不是被立刻分了出来：是
```

四行输出里有四件事：

- **`函数返回之后，堆上的 *h 还是 42`** —— 堆上的对象**活过了函数返回**。这不是运气，是标准说的：`malloc` 出来的对象生命期从分配开始，到 `free` 为止，中间跨过多少次函数返回都不算数；
- **`那份栈上的地址是空指针吗：是`** —— 返回局部变量的地址是错的，编译器给了 `warning: function returns address of local variable [-Wreturn-local-addr]`。**本机这次拿回的是空指针**，所以要在这里说清楚：这是**这台机器的编译器这一次的选择**（它看穿了这件事，直接把返回值换成了空指针），不是规律；换一种写法它就看不穿，你能读到栈上残留的字节。别把「没崩」当「没写错」；
- **`栈上连着走了 4 层，最后那层比第一层低吗：是（低 192 字节）`** —— 栈**往下长**：一层层往里调用，局部变量的地址一路变低，四层差了 192 字节（每层 64 字节，这个数随函数里的局部变量多少而变）；
- **`刚才还回去的那块，是不是被立刻分了出来：是`** —— **你刚还回去的内存，下一次分配就可能立刻落到你手里。**所以还完还用那张旧指针，十有八九是在碰别人的东西（12.6 节那个 use-after-free 就是这么来的）。

「堆向上长」这句话，我没能把它量成规律。 想要真读数，用 `examples/ch05/heap_dirs.c` 那个探针（它故意没有清单）：它打印三个区的地址，再问两个问题：

终端会话 探针的真读数：三个区的位置 + 两个判断 ch05-heap-dirs

```
$ ./heap_dirs.exe
静态区 &g_static = 00007FF6D1657030
栈     &local    = 00000000005FFE8C
堆     第 1 块    = 0000000000107670
连着要四块，地址递增吗：否
还回去的那块被立刻重用吗：否
```

三行地址读得出三件事：**静态区在** `0x00007FF6…` **一带、栈在** `0x00000000005FFE…` **一带、堆在** `0x000000000010…` **一带**（这是本机这一次的排布，换一台机器、甚至换一次启动方式都可能不同；堆的地址每次运行都会变，所以它只能进会话、不能进输出块）。两个判断里，前一个说的是**方向**：这一次连着四次 `malloc(16)` 的地址**不是**递增的；后一个说的是**重用**：这次还回去的那块没有被立刻再分出来（而 `heap_vs_stack` 那个程序里量到的却是「是」——12.8 节的输出块）。同一台机器上两个程序给出两个答案，**这就是「方向不是规律」的实证**：分配器按自己的账本切，切法随分配顺序、请求大小、启动方式变。会话文件里还有一段连跑五次的记录（第 8 到 18 行），那五次也都是「否」——但那只是这五次。

大白话 · **用大白话说清「栈」与「堆」的差别**

把程序想成一栋楼：

- **栈是电梯里那把小推车**：谁进电梯（谁被调用）谁用，出去（函数返回）就交还。它**快**、**自动**，但**地方小**（栈很小——第 13 单元会用一条深递归的链表把它量出来），而且**不能带走**；
- **堆是地下仓库**：地方大、谁都能租、租多久你说了算。代价是**要自己退租**（`free`）、**要自己记账**（谁开的谁退）、**还会碎**——一块块租出去又还回来之后，仓库里剩下的是一堆不连续的空隙，所以「还有 4 GiB 空着却要不到一块连续的 4 GiB」是会发生的（这就是碎片）；
- **什么时候非用堆不可**？三个信号：① 大小要**跑起来才知道**（数文件的字节数、用户输入几个数）；② 对象要**活过这个函数**（返回一块数据给调用者）；③ 对象**太大**，栈上放不下。

严谨的收尾只有一句：栈与堆是**实现**给的两个区域，它们的方向、默认大小、碎片怎么处理，标准一个字都没规定——所以本节所有的地址读数都是「本机这一次」，而规则（谁分配谁释放、多大、活多久）才是可以依赖的那部分。

12.9 用结构体封装动态数组：接口、边界与所有权

前面八节用的都是「一根指针 + 一个长度」。真写程序时，这两样东西总是成对出现——把它们打包进一个结构体，接口上再写清所有权，就是 C 里最常见的「动态数组」。先看接口（`examples/ch05/dynamic_array/include/dynamic_array.h`）：

c examples/ch05/dynamic_array/include/dynamic_array.h 第 15–19 行

```c
typedef struct {
    int *data;          /* 指向堆上的那块内存；没数据时是 NULL */ 指向堆上那块内存；没数据时是 NULL
    size_t size;        /* 已经放了几个 */
    size_t cap;         /* 一共能放几个 */ 一共能放几个
} IntVec;
```

16 指向堆上那块内存；没数据时是 NULL
18 一共能放几个

接口就是这几个函数，每个都带着「成功返回什么、失败返回什么」：

c （续）后半段 examples/ch05/dynamic_array/include/dynamic_array.h 第 21–31 行

```c
/* 要 cap 个位置。返回 0 = 成功，-1 = 分配失败（这时三个成员都是 0/NULL）。 */
int vec_init(IntVec *v, size_t cap); 失败给 -1，三个成员都是 0/NULL

/* 在末尾追加一个值；满了就扩容。返回 0 = 成功，-1 = 失败（失败时内容不变）。 */
int vec_push(IntVec *v, int value);

/* 取第 i 个。返回 0 = 成功并把值写进 *out；-1 = 越界（*out 不动）。 */
int vec_get(const IntVec *v, size_t i, int *out);

/* 还内存并复位成空。对同一个 IntVec 调两次也安全。 */
void vec_free(IntVec *v); 调两次也安全
```

22 失败给 -1，三个成员都是 0/NULL
31 调两次也安全

三个数字成员各有分工：`data` 是**那根指针**，`size` 是「放了几个」（用到的部分），`cap` 是「能放几个」（要来的部分）。扩容只在 `size == cap` 时发生，而扩容那几行正是 12.7 节教过的写法：

c examples/ch05/dynamic_array/src/dynamic_array.c 第 21–35 行

```c
int vec_push(IntVec *v, int value) {
    if (v->size == v->cap) {            /* 满了，先扩容 */ 满了才扩容
        size_t new_cap = (v->cap == 0) ? 1 : v->cap * 2;
        int *tmp = (int *)realloc(v->data, new_cap * sizeof *tmp); 临时指针接住

        if (tmp == NULL) {
            return -1;                  /* 失败：原来的数据一个字都没动 */ 失败：老数据一个字没动
        }
        v->data = tmp;                  /* 成功了才改 */ 成功了才改
        v->cap = new_cap;
    }
    v->data[v->size] = value;
    v->size++;
    return 0;
}
```

22 满了才扩容
24 临时指针接住
27 失败：老数据一个字没动
29 成功了才改

写进新位置之后，两个计数器各自加一：

c **（续）后半段** examples/ch05/dynamic_array/src/dynamic_array.c 第 31–35 行

```c
    }
    v->data[v->size] = value;
    v->size++;
    return 0;
}
```

这段 `vec_push` 有一个**已知的缺口**，真写库时要补上：`v->cap * 2` 与 `new_cap * sizeof *tmp` 都没有做溢出检查。`cap` 走到 `SIZE_MAX / 2` 时翻倍会回绕成一个小数，`new_cap * sizeof` 也可能回绕；补法就是 12.2 节那条 `if (v->cap > SIZE_MAX / 2 / sizeof *v->data) return -1;`。本例子故意不补，是为了让「扩容」那五行好读；代价是它在极端大小下会违反接口上那句「失败时内容不变」——**承诺与实现之间差的那一步，恰恰是这类封装最常见的漏洞。**同理，`xstrdup` 的 `n + 1` 在 `n == SIZE_MAX` 时也会回绕；这一条在实际中碰不到（你不可能有一段那么长、还读得完的字符串），但写库的人会在注释里点一句。

`main` 用它压进 5 个数，中间会撞两次「满了」，所以能直接看到容量怎么长：

c examples/ch05/dynamic_array/src/main.c 第 17–28 行

```c
    for (int i = 1; i <= 5; i++) {
        if (vec_push(&v, i * 11) != 0) {
            vec_free(&v);
            return 1;
        }
        printf("压入 %2d 之后：size = %zu, cap = %zu\n", i * 11, v.size, v.cap); 每压一个就打印一次现状
    }

    int out = -1;
    int rc = vec_get(&v, 4, &out);      /* 先取，再打印：参数求值顺序不由你定 */
    printf("取下标 4：%s，值是 %d\n", (rc == 0) ? "成功" : "越界", out); 先取，再打印
    printf("取下标 9：%s\n", (vec_get(&v, 9, &out) == 0) ? "成功" : "越界（被拒绝了）"); 越界被接口拒绝
```

22 每压一个就打印一次现状
27 先取，再打印
28 越界被接口拒绝

终端输出 终端输出 dynamic-array-out

```
开好之后：size = 0，cap = 2
压入 11 之后：size = 1，cap = 2
压入 22 之后：size = 2，cap = 2
压入 33 之后：size = 3，cap = 4
压入 44 之后：size = 4，cap = 4
压入 55 之后：size = 5，cap = 8
取下标 4：成功，值是 55
取下标 9：越界（被拒绝了）
xstrdup 拿到的副本：谁分配谁释放
副本与原文是同一块内存吗：否
还完之后：size = 0，cap = 0，data 是空指针吗：是
```

容量变过两次，是 2 → 4 → 8：`2 → 2 → 4 → 4 → 8` 这一列就是 12.7 节那条「按倍长」的实物（`cap` 只在 `size` 撞上它时才变）。另外两行是**边界检查**的效果：取下标 4 成功、值 55；取下标 9 被接口**拒绝**，程序没有任何异常——`vec_get` 那一行 `if (i >= v->size) return -1;` 就是全部的防线。写完还要还：`vec_free` 负责 `free` 加复位，所以它能被安全地调两次。

同一个例子里还有一个自写的小函数，专门用来说明**所有权**怎么写在脸上：

c examples/ch05/dynamic_array/src/xstrdup.c 第 5–12 行

```c
char *xstrdup(const char *s) {
    size_t n = 0;

    while (s[n] != '\0') {            /* 先数长度（不含结尾那个 0） */ 先数长度
        n++;
    }
    char *copy = (char *)malloc((n + 1) * sizeof *copy); 要 n+1 个字节，多出来的那个放 0
    if (copy == NULL) {
```

8 先数长度
11 要 n+1 个字节，多出来的那个放 0

拿到内存之后，搬的动作只有三行：

c **（续）后半段** examples/ch05/dynamic_array/src/xstrdup.c 第 13–19 行

```c
        return NULL;
    }
    for (size_t i = 0; i <= n; i++) {  /* 连结尾的 '\0' 一起搬 */ 连结尾的 0 一起搬
        copy[i] = s[i];
    }
    return copy;                       /* 这块内存归调用者 */ 这块内存归调用者
}
```

15 连结尾的 0 一起搬
18 这块内存归调用者

工程惯例 · **工程惯例：strdup 是 POSIX，不是 ISO C**

很多书、很多代码里都写着 `char *s = strdup(msg);`——**它不在 ISO C 里。** `strdup` 是 POSIX 定义的：它在 `<string.h>` 里，做的是「要一块 n+1 字节的内存，把字符串连结尾的 `\0` 一起搬过去」，返回的指针**归调用者**（要自己 `free`）。标准 C 没有它，所以本书的例子一律自己写一个 `xstrdup`，把两件事说清楚：

- **它分配内存**：调用者要负责还——这一点必须写在声明旁边，不能靠「看名字猜」（`include/dynamic_array.h` 第 33、34 行就是写给调用者看的那两句话）；
- **它在失败时给空指针**，所以调用点要判。

代价是名字不统一（各家项目里可能叫 `xstrdup`、`str_dup`、`copy_string`），收益是**这份代码在任何一个只认 ISO C 的编译器上都能编过。**这也是一条更一般的纪律：**看到标准库里没有的函数名，先问它是谁定义的**——POSIX 的、Windows API 的、某个库的，混进来就不可移植了。

12.10 编译器能看见什么：-fanalyzer 与堆越界

运行期没有工具在我们这边（下一节细说），所以第一道防线是**编译器自己的静态分析器**。先看它要抓的那个程序：这次要 3 个点（每个点 8 字节），却写了第 5 个点的横坐标（`examples/ch05/heap_oob_fanalyzer/src/main.c`）：

c examples/ch05/heap_oob_fanalyzer/src/main.c 第 12–22 行

```c
int main(void) {
    Point *pts = (Point *)malloc(COUNT * sizeof *pts);   /* 3 个点，一共 24 字节 */ 要 3 个点，一共 24 字节
    if (pts == NULL) {
        return 1;
    }

    for (size_t i = 0; i < COUNT; i++) {        /* 合法下标是 0 … 2 */ 合法下标是 0 … 2
        pts[i].x = (int)i;
        pts[i].y = (int)i * 10;
    }
    pts[4].x = 99;                              /* 常量下标 4：越界写了 */ 常量下标 4：越界写了
```

13 要 3 个点，一共 24 字节
18 合法下标是 0 … 2
22 常量下标 4：越界写了

越界写完，程序照常往下走：

c （续）后半段 examples/ch05/heap_oob_fanalyzer/src/main.c 第 23–30 行

```c

    printf("一个点占 %zu 字节，这次要了 %d 个\n", point_size(), COUNT); 程序连这行都走到了
    printf("越界写完了，程序还活着，前三个点还是好的：\n");
    print_points(pts, COUNT);

    free(pts);
    return 0;
}
```

24 程序连这行都走到了

终端输出 终端输出 heap-oob-out

```
一个点占 8 字节，这次要了 3 个
越界写完了，程序还活着，前三个点还是好的：
  pts[0] = (0, 0)
  pts[1] = (1, 10)
  pts[2] = (2, 20)
```

它真的跑完了：三个点打印正常，退出码 0。**这就是越界写最危险的样子**——运行期什么都没发生。换 `-fanalyzer` 编一次，编译器在**编译期**就把这笔账算给你：

终端会话 -fanalyzer：编译期就指出这处越界写（编译退出码仍是 0） ch05-analyzer-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -fanalyzer -c src/main.c -o main.o
src/main.c: In function 'main':
src/main.c:22:14: warning: heap-based buffer overflow [CWE-122] [-Wanalyzer-out-of-bounds]
   22 |     pts[4].x = 99;                          /* 常量下标 4：越界写了 */
      |     ~~~~~~~~~^~~~
```

终端会话 （续）它先记下这块内存有多大 ch05-analyzer-2

```
   13 |     Point *pts = (Point *)malloc(COUNT * sizeof *pts);   /* 3 个点，一共 24 字节 */
      |                          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~
      |                          |
      |                          (1) capacity: 24 bytes
```

终端会话 （续）再把「写到第几个字节」算出来 ch05-analyzer-3

```
   22 |     pts[4].x = 99;                          /* 常量下标 4：越界写了 */
      |     ~~~~~~~~~~~~~
      |              |
      |              (6) ⚠️  out-of-bounds write from byte 32 till byte 35 but region ends at byte 24
src/main.c:22:14: note: write of 4 bytes to beyond the end of the region
   22 |     pts[4].x = 99;                          /* 常量下标 4：越界写了 */
```

编译器视角 · **编译器视角：这份诊断里的三样东西**

- `heap-based buffer overflow` **是问题的名字**，`[CWE-122]` 是业界给这类缺陷统一编的号（CWE 是一份公开的缺陷分类表，122 号就是「堆上的缓冲区溢出」）；
- **方括号里** `[-Wanalyzer-out-of-bounds]` **是控制这条诊断的开关名**——与第 2 单元讲过的警告开关是同一套机制，只是这一族开关归 `-fanalyzer` 打开的分析器管；
- **中间那些** `(1)`、`(6)` **是它一步步推出来的过程**：先记住 `capacity: 24 bytes`（第 1 步），再把常量下标换算成字节偏移，最后给出 `out-of-bounds write from byte 32 till byte 35 but region ends at byte 24` ——想写第 32 到 35 个字节，可这块内存到第 24 个字节就结束了。

这一套推导要求**形状够死板**：`malloc` 的大小和越界的下标都必须是**常量**、而且都写在这一个函数里。第 9 单元（`chapters/ch04/02-pointers-and-memory.md` 9.9 节）量过这件事：把填 0 到 3 的循环挪进模块里的另一个函数，分析器就一个字都不报；换成「变量下标的循环形状」，它只会报一条 `possible-null-dereference`（CWE-690），与越界无关。所以它能报出来的是「死板但确定」的形状。

实验记录 · **本机实测：-fanalyzer 抓到了什么、没抓到什么**

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -fanalyzer -c src/main.c -o main.o`；
- **编译退出码是 0**。这一点最反直觉：分析器只负责**报告**，它不阻断编译。所以「编译过了」绝不等于「代码没问题」；也正因为退出码是 0，本书这个例子的清单写成 `detect` 而不是 `compile_fail`；
- **阴性对照**：去掉 `-fanalyzer` 再编同一份源码，一条分析器诊断都没有——所以那句 `heap-based buffer overflow` 确实归它管，不是基础参数 `-Wall -Wextra` 碰巧报出来的。本书所有 `detect` 清单都要过这一步核对（闸门会去掉检测器参数复算一遍），编造不出一份「假装抓到了」的证据；
- **它不一定很慢**：这个 30 行的文件上，我量了几次，同一条命令的读数在 100 到 400 毫秒之间跳（文件太小，噪声比差别大）；能说的只有一句「这个尺寸上它不明显拖慢编译」，工程大了它才贵。真正要付的代价是**判断有误报**——所以它的输出当**线索**看，不要当判决书：报出来的地方值得回头看一遍代码，没报的地方**不能**当成安全。

实验记录 · **本机实测：-fanalyzer 抓到了什么、没抓到什么（续）**

清单 `examples/ch05/heap_oob_fanalyzer/heap_oob_fanalyzer.json` 里写的是 `"mode": "detect"`、`"detect": "fanalyzer"`，证据是编译期的 `stderrContains`（`-Wanalyzer-out-of-bounds` 与 `CWE-122` 两片），没有 `expect.exit` ——分析器是编译期检测，闸门不会去运行这个程序。

12.11 本机没有 ASan：剩下的四样手段

讲到这里必须交代一句实话：**本机没有 AddressSanitizer，也没有 valgrind**。这不是「懒得装」：`-fsanitize=address,undefined` 在这台机器上能编译、**链接失败**（`collect2.exe: error: ld returned 5`，安装里没有 libasan/libubsan），`clang` 与 MSVC 也都没有装。所以本书**永远不写**「已用 ASan 验证」这种句子——凡涉及内存错误，证据只能来自下面这四种能跑的手段：

手段	什么时候说话	本单元里的证据	抓不到的
`-Wall -Wextra`（本书固定参数）	编译期**提醒**	`warning: pointer 'p' used after 'free' [-Wuse-after-free]`；`warning: function returns address of local variable`	它只是提醒，**编译照过、程序照跑**
`-fanalyzer`	编译期（退出码仍 0）	`heap-based buffer overflow [CWE-122]`	形状不够死板的越界；有误报
`-ftrapv`	运行期（终止）	有符号整数溢出时退出码 **3221226505**，stdout 丢空	无符号回绕、浮点、以及任何不是「有符号溢出」的错
退出码 + stdout	运行期	double free **3221226356**、use-after-free 两档行为不同	它只给「结果」，不告诉你**哪里**错了
gdb（手动跑）	你设了断点之后	第 4 章那次 `bt` 打出 `#0 … at src/main.c:15`	它得**你先怀疑到某处**才用得上；崩在分配器里时，栈顶往往不是你的代码

第三行那个 `-ftrapv` 有一个很现实的用武之地：**算大小的时候。**看这个程序——6 亿个元素，按 `int` 算字节数：

c examples/ch05/alloc_fail/src/size_overflow.c 第 10–23 行

```c
int main(void) {
    int count = element_count();            /* 6 亿 */ 元素个数由模块给出
    int bytes = count * 4;                  /* 用 int 算字节数：4 倍积少成多 */ 用 int 算字节数：这一步就溢出了

    printf("元素个数 = %d，按 int 算出来的字节数 = %d\n", count, bytes);
    if (bytes < 0) {
        printf("算出来是负数，说明这一步已经出了 int 的范围\n");
    }

    int *p = (int *)malloc((size_t)bytes); /* 再把它转成 size_t 去要内存 */ 再把它转成 size_t 去要内存
    printf("拿这个数去 malloc，返回的是空指针吗：%s\n", (p == NULL) ? "是" : "否");
    free(p);
    return 0;
}
```

11 元素个数由模块给出
12 用 int 算字节数：这一步就溢出了
19 再把它转成 size_t 去要内存

终端输出 终端输出 trapv-size-out

（无输出）

终端会话 同一份源码：基础构建打印一个负数，加 -ftrapv 当场终止 ch05-size-overflow

```
$ ./overflow.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
元素个数 = 600000000，按 int 算出来的字节数 = -1894967296
算出来是负数，说明这一步已经出了 int 的范围
拿这个数去 malloc，返回的是空指针吗：是
退出码 0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -ftrapv src/size_overflow.c src/alloc_fail.c -o overflow_trap.exe
$ ./overflow_trap.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
退出码 127
```

这段会话里的对照很清楚：**基础构建**下 `count * 4` 回绕成了一个负数 `-1894967296`，程序若无其事跑完、退出码 0；**加了** `-ftrapv` 之后，进程在乘出范围的那一刻被终止，退出码 **3221226505**（清单 `alloc_fail/size_overflow_trapv.json` 里写的就是它，而且 `output` 块里只有一行哨兵）。注意 stdout 为空的原因

是**没来得及打印**：第 12 行的乘法在第一条 `printf` 之前就已经触发陷阱，缓冲区里根本还没有东西——不要把它当成「缓冲被一起带走」的例子。

工程惯例 · 工程惯例：没有 ASan，就把「不变量」写进代码

工具抓不到的东西，只能靠纪律补。C 里最值钱的两条纪律是**断言**与**不变量**：

- **断言**（`assert(v->size <= v->cap)`）是「这里必须成立，不成立就该当场停下来」的检查。它便宜、能立刻把错误钉在**出错的地方**而不是几百行之后；代价是**发布版通常要关掉它**（定义 `NDEBUG`），所以**别把有副作用的表达式放进 `assert`**（比如 `assert(free_and_check())`）；
- **不变量**是「这个结构在任何时刻都必须满足的性质」——`IntVec` 的三条就很典型：`size <= cap`、`data == NULL` 当且仅当 `cap == 0`、`vec_free` 之后三者必须同时复位。本例子没有把这三条写进头文件（`include/dynamic_array.h` 第 9 到 12 行写的是**所有权**，不是不变量）；真写库时应当把不变量排在所有权旁边——下一个人改代码时才有判断依据。

还有一条更一般的：**让证据出现在离错误最近的地方**。把缓冲区里的输出先 `fflush` 出去、把「读到的值」打印出来，都是在给将来的自己留现场——本单元那两个崩溃例子（`double_free`、`use_after_free`）之所以能量出数字，全靠这一点。

12.12 每次 malloc，先问三个问题

动态内存这一节可以压缩成三条要变成习惯的规矩。它们的顺序就是**出错频率的顺序**：

1. **每次 `malloc`，先问三个问题：谁释放？什么时候释放？要不到怎么办？** 三个问题都答得出来才允许写这一行。答不出来的时候，通常意味着这块内存应该由调用者分配（`void fill(char *out, size_t n)` 那种形状）——第 4 章 9.11 节那条「接口上『从哪开始』与『有几个』永远成对出现」在这里变成「**接口上『谁分配』与『谁释放』也永远成对出现**」；
2. **分配完先写，再用；读一块内存之前，先想清楚它是谁写的。** `malloc` 的值不确定（7.22.3.3 节）、`realloc` 多出来的那部分也不确定——这两处是本章最容易被忽略的「不确定」；
3. **还回去之后别再碰，并且立刻把指针置空。** 置空只管得住这一个指针（别名管不住），但它把「下一次误用」从「悄悄写坏别人」（本机上往往要到进程退出才报 3221226356）变成「当场一次空指针解引用」（本机是 3221225477 那一族访问冲突）——这两种都是 UB、都不保证发生，只是后者离出错的地方近得多。在没有 ASan 的机器上，这是你能给自己争取到的最好条件。

这一节新增的九个例子目录，各自管一件事，回头查的时候按这张表找：

目录	它回答的那个问题
`malloc_basics`	一块内存怎么到手、里面是什么、`free` 之后指针怎么了
`calloc_zero`	`calloc` 与 `malloc` 的内容差别（清 0 与不清 0）
`alloc_fail`	要不到内存时：NULL、`errno`、`strerror`，以及大小算溢出
`heap_vs_stack`	住址方向、寿命、什么时候非用堆不可
`realloc_grow`	`realloc` 的正确与错误写法，失败时老内存去哪
`double_free`	同一块内存还两次的现场与退出码
`use_after_free`	还回去之后接着用：两档优化下的不同表现
`heap_oob_fanalyzer`	堆越界写：`-fanalyzer` 的编译期诊断
`dynamic_array`	结构体封装的动态数组：接口、边界、所有权

下一单元（`chapters/ch05/03-linked-structures.md`，自引用结构与链表）从这里接着走：把「一块内存」换成「一堆用指针串起来的内存」，那时本节这三条规矩会一条不落地全部用到——尤其是「先取 `next` 再 `free`」。

小结 · 这一节的要点

- `malloc` **只承诺「地方给你」，不承诺里面的值**（7.22.3.3 节原文用的词是 `indeterminate`）；`calloc` 多一条「按位清 0」（7.22.3.4 节）。**分配失败只给空指针**；把原因写进 `errno` 是 POSIX 与本机运行库的行为，不是 ISO C 的要求——所以判空是必须的，看 `errno` 是加分的；
- `free` **只处理「格子」，不处理「格号」**：还完之后指针一个字节都没变（第 4 章 gdb 量过，本节 `malloc_basics` 又量了一次）。双重释放的后果是 UB，**本机实测**退出码 **3221226356**、stderr 一个字都没有；use-after-free 更能藏，同一份源码在 `-O0` 与 `-O2` 下行为不同（stdout 不同、退出码 3221226356 对 0）；
- `realloc` **必须用临时指针接返回值**：它成功时内容会跟过来（7.22.3.5 节），失败时**老块一个字节都没动**——标准把老块留给了你，是 `p = realloc(p, n)` 那句赋值把它弄丢了。扩容按倍长（2 → 4 → 8），均摊到每次 push 是常数，代价是最多浪费一半空间；

小结 · 这一节的要点（续）

- **本机没有 ASan/valgrind**：能用的四样是固定警告参数、`-fanalyzer`（编译期，退出码仍 0，形状要死板）、`-ftrapv`（有符号溢出，3221226505）、以及退出码 + stdout + gdb。剩下的靠纪律补：**每个** `malloc` **问三个问题**、分配完先写再用、还完立刻置空。

自测题 · 自测题（先自己答，再对答案）

1. `malloc(16)` 拿到的那 16 个字节，与 `calloc(4, 4)` 拿到的相比有什么不同？这个区别是「本机的经验」还是「标准的承诺」？标准里对应的两句话分别在哪一节？
2. `p = realloc(p, new_n * sizeof *p);` 这行代码在什么情况下会泄漏？按标准的说法，`realloc` 失败时老那块内存是什么状态？正确写法长什么样？
3. `free(p);` 之后 `p` 里的值变了吗？为什么 `free(p); p = NULL;` 仍然救不了「同一块内存还有别的指针」的情况？
4. 本机实测：同一块内存 `free` 两次，退出码是多少、stderr 里有什么？为什么本书不把它写成「一定会崩」？
5. 本机没有 ASan/valgrind 的情况下，想把「堆越界写」变成看得见的证据，用哪个开关、它要求代码是什么形状、编译退出码是多少？想把「用 `int` 算字节数溢出」变成证据，又该用哪个开关？

注 · 自测题答案

1. `calloc` **那 16 个字节保证是全 0，** `malloc` **那 16 个字节没有承诺**（本机这次恰好也是全 0，但 4 字节的请求在本机就被量到过非零字节）。这是**标准的承诺**，不是经验：`malloc` 那节写的是 `whose value is indeterminate`（§7.22.3.3），`calloc` 那节写的是 `The space is initialized to all bits zero`（§7.22.3.4）。
2. **只要** `p` **指向一块老内存、** `size` **非零，而这次** `realloc` **失败，就会泄漏**（`p` 是空指针时 `realloc` 等价于 `malloc`，失败没有老块可丢；`size` 为 0 时另有规定，见 §7.22.3.5）。按 §7.22.3.5「If `size` is nonzero and memory for the new object is not allocated, the old object is not deallocated」——老块**不会被释放**，是那句赋值把老地址盖成了空指针，于是没人记得它了。正确写法是用临时指针接：`int *tmp = realloc(p, n); if (tmp == NULL) { /* 老 p 还在，该 free 就 free */ } else { p = tmp; }`。
3. **没变。** `free` 拿到的只是指针值的一份拷贝，它不会去改你的变量（本机 gdb 实测：`free` 前后是同一个地址）。置空只**管住这一个**指针：同一块内存如果还有别的指针（别名），那些指针照样悬着，用它们读或写仍然是 use-after-free——这也是「谁分配谁释放」比「记得置空」更根本的原因。
4. 退出码 **3221226356**（`0xC0000374`），**stderr 是空的**——没有任何文字提示，唯一的线索就是这个码。不写成「一定会崩」是因为标准只说这是 UB；第 4 章还量过：同一个错误程序只因为启动路径不同，就会在 3221226356 与 3221225477 之间跳。所以正确说法是「本机这一次的读数 + 它是 UB」。
5. 堆越界用 `-fanalyzer`：要求 `malloc` 的大小与越界下标都是**常量**、并且都写在这一个函数里（常量形状），编译**退出码仍是 0**（它只报告，不阻断编译），并且只在这一份源码里报——去掉参数就不报。字节数溢出用 `-ftrapv`：本机实测退出码 **3221226505**、stdout 一个字节都没有（缓冲区被一起带走了）。

自引用结构与链表：用指针串起数据

◎ 学习目标 · 看完这一单元，你应该能

- 写出一个自引用结构体（`struct Node { int value; struct Node *next; }`），并说清 `next` 为什么必须是指针、写成 `struct Node next;` 时编译器会怎么驳回你；
- 用头插和尾插各建一条链，说清「带尾指针」与「每次从头找尾」在 2000 个节点上差多少步（本机实测的数字）；
- 写对删除节点与释放整条链的顺序，说出「先 free 再取 next」在这台机器上真的会看到什么，以及 `list_free` 为什么收的是 `Node **`；
- 用快慢指针判断一条链有没有环，并数出环长、相遇用了几步；
- 在 gdb 里顺着 `next` 一段段看一条链，并解释「一百万个节点的递归遍历」为什么以退出码 3221225725 收场。

13.1 先看工程长什么样

这一节是第 5 章的第三个单元，也是全书的第 13 单元。按老规矩，先看**文件和目录**——链表这个题目最容易被讲成一堆「指针怎么指」的动画，但它的每一个坑最后都落在具体文件上：源文件里那个 `next` 是怎么声明的、清单里这段程序怎么编译、正文里那段输出是谁跑出来的。

第 5 章的例子都放在同一个项目目录 `examples/ch05/` 里：本节开始之前，前两个单元（结构体与动态内存）的例子已经躺在那里了。这一节往这个目录里加**八个**例子目录，外加五段抓下来的终端会话。

目录树 **这一节的例子工程：往 ch05 里再加八个目录**

第 1 步 本节开始时，第 5 章的项目已经在场

```
examples/
├── ch01/ ：第 1 章的项目
├── ch04/ ：第 4 章的项目
└── ch05/ ：第 5 章的项目，前两个单元的例子已经在里面
```

第 2 步 先摆上「怎么串起来」的三个例子

```
examples/
└── ch05/
    ├── 前两个单元的例子 ：结构体、动态内存
    ├── list_head_insert/ ：头插、迭代与递归遍历 ← 本课新增
    ├── list_append/ ：尾插与尾指针 ← 本课新增
    └── stack_list/ ：链表栈、链表队列 ← 本课新增
```

第 3 步 再加上「怎么拆下来」的三个例子

```
examples/
└── ch05/
    ├── 前面三个例子 ：list_head_insert / append / stack_list
    ├── list_delete/ ：删中间、删头、删尾，另含成本对照 ← 本课新增
    ├── list_free/ ：释放整条链，并置空 head ← 本课新增
    └── list_bad_order/ ：先 free 再取 next ← 本课新增
```

第 4 步 最后两个例子，以及本节抓下来的五段会话（前六个目录同上）

```
examples/
└── ch05/
    ├── circular_detect/ ：快慢指针判环 ← 本课新增
    ├── tree_preorder/ ：二叉树前序遍历 ← 本课新增
    ├── session-gdb-list.txt ：gdb 顺着 next 看 ← 本课新增
    ├── session-analyzer-list.txt ：分析器怎么看空指针 ← 本课新增
    ├── session-warning-list.txt ：编译器的 use-after-free 警告 ← 本课新增
    ├── session-cost-list.txt ：数组与链表的耗时对照 ← 本课新增
    └── session-cycle-free.txt ：有环的表释放会重复 free ← 本课新增
```

第 5 步 **每个例子目录里都是这三样（以 list_delete 为例）**

```
examples/
└── ch05/
    └── list_delete/
        ├── src/ : main.c、list_delete.c、sentinel_list.c 等 ← 本课新增
        ├── include/ : list_delete.h、sentinel_list.h 等 ← 本课新增
        └── list_delete.json : 清单（这个目录一共四份） ← 本课新增
```

第 1 帧是故事的起点：`examples/` 下已有第 1、4 章的项目，第 5 章也已开张（前两个单元的例子在里面）。第 2 到第 4 帧是本节要加进去的八个目录，三个一组、三个一组、两个收尾。**一个目录一个主题，但一个主题可以有好几份程序**——下面这张清单里，括号里的节号就是它在哪一节登场：

- `list_head_insert` —— 头插、两种遍历，外加第 13.2 节那个「内嵌自身」的反面教材（13.2、13.3、13.12 节用它）；
- `list_append` —— 尾插与尾指针，附带两种写法的**步数对照**（13.4 节）；
- `stack_list` —— 用链表做栈和队列，和数组栈放在一起比接口（13.11 节）；
- `list_delete` —— 删除的六个边界与哨兵版、数组与链表的成本对照、以及第 13.7 节那个空指针分析器（13.5、13.7、13.8 节）；
- `list_free` —— 释放整条链，并让同一次 `list_free` 能被安全地调两次（13.6 节）；
- `list_bad_order` —— 反面教材：先 `free` 再取 `next`（13.7 节）；
- `circular_detect` —— 快慢指针判环（13.10 节）；
- `tree_preorder` —— 二叉树的前序 / 中序 / 后序遍历（13.13 节）。

第 5 帧以 `list_delete` 为例说明目录内部的摆法：它里面**有四份清单**（`list_delete.json`、`list_delete_cost.json`、`list_delete_sentinel.json`、`list_delete_fanalyzer.json`），`src/` 里除了 `list_delete.c` 还有哨兵版、成本对照、分析器那几组源文件。一个目录放几份清单是允许的——它们演示的是同一个主题的不同侧面，各有各的输出块要对账。摆法本身和第 1 单元 1.7 节讲的分文件规矩是同一条：**声明放头文件、定义放源文件。**

工程惯例 · **工程惯例：一个例子一个目录、声明与定义分家**

标准对「一个程序分几个文件」只有一个约束：有且只有一个 `main`。**分几个文件、目录怎么起名，全是惯例。**本书从第二个项目（`examples/ch02/`）起要求每个例子都是 `src/` + `include/` 的最小工程，理由是三条：

- **每条结论都要能单独验证**：`list_delete` 里的每一份清单只回答一个问题（删除的边界、代价、哨兵、空指针），出问题时你能立刻知道是哪一份；
- **接缝写在明面上**：`include/list_delete.h` 里只有声明（名字、参数、返回值），`src/list_delete.c` 里是函数体，`src/main.c` 只负责组装与打印；
- **能被别人接手**：八个核心模块从 41 行到 149 行（`wc -l` 数出来的），比一个八百行的 `main.c` 好读得多。

代价是多学一件事：同一个 `struct Node` 会在好几个例子里各写一遍（有意的，改一个不牵动别的例子）；第 7 章讲多文件工程时再把公共声明收进头文件。

13.2 自引用结构体：`next` 为什么必须是指针

先把节点定义看清楚。它住在 `examples/ch05/list_head_insert/include/list_head_insert.h` 里：

c examples/ch05/list_head_insert/include/list_head_insert.h 第 6–19 行

```c
/* 一个节点：一个整数 + 一个「指向同一种节点」的指针。
   next 的类型是 struct Node *，不是 struct Node ——
   这句话是自引用结构体的全部秘密，13.2 节用它解释。 */
typedef struct Node {
    int value;
    struct Node *next;  类型是 struct Node *，不是 struct Node
} Node;  分号结束这个结构体声明，别漏

Node *node_new(int value);                      /* 开一个新节点 */
Node *list_push_front(Node *head, int value);   /* 头插，返回新的头 */
```

```
16  void list_print(const Node *head);              /* 迭代遍历并打印 */
17  size_t list_count_iter(const Node *head);       /* 迭代数节点 */
18  size_t list_count_rec(const Node *head);        /* 递归数节点 */
19  void list_free_all(Node *head);                 /* 从头到尾释放 */
```

11 类型是 struct Node *，不是 struct Node
12 分号结束这个结构体声明，别漏

第 6 到 12 行是注释加结构体；结构体本身是第 9 到 12 行那四行。**第 11 行那句 `struct Node *next;` 是这一节的支柱**：节点里装的不是「另一个节点」，而是「另一个节点在哪」。

先回答一个新手一定会问的问题：为什么不能写 `struct Node next;`？把它真的写出来，编译器当场驳回。`examples/ch05/list_head_insert/src/selfref.c` 就是这份反面教材：

c **故意把 next 写成「节点自己」** examples/ch05/list_head_insert/src/selfref.c 第 1–11 行

```c
/* 反面教材：把 next 的类型写成「节点自己」，而不是「节点的指针」。
   它自己的清单是 list_head_insert_selfref.json (mode=compile_fail) 。 */
#include "list_head_insert.h"

typedef struct BadNode {
    int value;
    struct BadNode next;          /* ← 错在这一行：节点里塞了一整个节点 */
} BadNode;

int main(void) {
    return 0;
```

编它，gcc 的真实 stderr 只有三行（错误行、源码回显、插入符）：

编译诊断 编译诊断 selfref-diag

```
src/selfref.c:7:20: error: field 'next' has incomplete type
    7 |     struct BadNode next;          /* ← 错在这一行：节点里塞了一整个节点 */
      |                    ^~~~
```

第一行那句 `field 'next' has incomplete type` 就是标准说的「成员的类型不完整」。**为什么不完整**：编译器读到第 7 行时，`struct BadNode` 还没读完（右花括号还没出现），它对这个类型只知道名字、不知道大小；而「放一个成员」必须先知道成员有多大。指针就没这个问题——本机的指针一律 8 个字节，跟它指向什么无关（第 9 单元讲指针时实测过）。所以规则是：**能放指针，不能放自己。**

标准原文 · 标准原文：结构体里为什么不能放它自己

C 语言标准（ISO/IEC 9899，C17）6.7.2.1 节「Structure and union specifiers」第 3 段：

> A structure or union shall not contain a member with incomplete or function type (hence, a structure shall not contain an instance of itself, but may contain a pointer to an instance of itself), ...

大意：结构体里不许放「类型还不完整」或者函数类型的成员——**所以结构体里不能放一个自己的实例，但可以放一个指向自己实例的指针。**括号里那半句把规则一次说全了：前半个分句是「不许」，后半个分句的 `may` 是「可以」。

出处：ISO/IEC 9899:2018（C17）§6.7.2.1 第 3 段。

{} 语法规则 · 自引用结构体的写法：四行，一个字都不能少

骨架四行：`typedef struct Node {` 起名字、`int value;` 放数据、`struct Node *next;` 放自引用指针、`} Node;` 收尾并定下别名。第三行必须写 `struct Node *`：别名 `Node` 要到最后一行才生效，在结构体**内部**编译器还不认识它——代价是每次多打 `struct` 这 7 个字符。

本机的节点长这样：一个节点 16 个字节。

内存形态 **一个 16 字节的节点：4 的数据 + 4 的对齐空隙 + 8 的指针**

第 1 步 按地址从低到高看这 16 个字节

`value ：偏移 0，4 个字节，装着整数` `空隙 ：偏移 4，4 个字节，为了让 next 对齐，什么都不放`

`next ：偏移 8，8 个字节，装着「下一个节点在哪」`

第 2 步 两个节点串起来：next 是一条箭头，不是「里面还有一个节点」

`A 节点 ：value = 1，next = NULL` `B 节点 ：value = 2，next 指着 A` `head ：一个 8 字节的指针，指着 B`

B 节点 → A 节点 8 个字节里装的就是这个箭头的起点
head → B 节点

第 3 步 走到链尾：NULL 是「到此结束」的记号

`A 节点 ：value = 1，next = NULL` `B 节点 ：value = 2，next 指着 A`

head → B 节点
B 节点 → A 节点
A 节点 → NULL 不是节点，是一个保证「不再往下走」的指针值

第 1 帧里的「空隙」不是浪费：`next` 是 8 字节的指针，编译器要让它落在 8 的倍数地址上，所以 `int value` 后面补 4 个字节。第 2 帧是链表最要紧的一张图——**每个节点自己都住在堆上的一块独立内存里，节点之间只有「地址」这一条线。**第 3 帧那个 `NULL` 是链表的终点站：它不是节点，是一个值，用来告诉所有遍历代码「到此为止」。

实验记录 · 本机实测：16 和 8 是跑出来的

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令：在 `examples/ch05/list_head_insert/` 里跑 `gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/list_head_insert.c -o app.exe`，再跑 `./app.exe`；
- 结果：程序第一行打出 `sizeof(Node) = 16 字节，sizeof(head) = 8 字节` ——**节点 16 字节**（4 + 4 空隙 + 8），**指针 8 字节**。这两行就长在第 13.3 节的输出块里，由流水线逐字对账。

陷阱 · 自引用结构体的三个坑

你会怎么被坑到：这三个坑都能编过或者报错报得很远，你得知道自己在看什么——

- 写成 `struct Node next;`：诊断是 `field 'next' has incomplete type`。它的措辞里一个「may not be self」都没有，所以新手会去怀疑别的地方；真相是「你要求编译器把一个还不完整的东西塞进去」。
- 结构体末尾漏了分号（`} Node` 后面没 `;`）：报错会跑到**下一个**变量声明或函数上，说 `expected ';'`。这是 C 里最经典的错位报错。
- 在结构体内部写 `Node *next;`：诊断是 `unknown type name 'Node'`。那时 `Node` 这个别名还没出生——**别名是在 `} Node;` 那一刻才有的。**

真相只有一句：**在结构体内部，只能用它自己已经完整的部分。**指针是完整的（大小固定 8 字节），所以指针行得通。

13.3 头插：一次调用串一个节点

头插是建链表最省事的办法：新节点永远接在**最前面。**`list_push_front` 只有三句，住在 `examples/ch05/list_head_insert/src/list_head_insert.c`：

c examples/ch05/list_head_insert/src/list_head_insert.c 第 17–36 行

```c
Node *list_push_front(Node *head, int value) {
    Node *p = node_new(value);
    p->next = head;                 /* 新节点的尾巴接上原来的头 */ 新节点的 next 接上原来的头
    return p;                       /* 新节点成为新的头 */ 返回新头，调用者必须接住它
}
```

```
22
23  void list_print(const Node *head) {
24      for (const Node *p = head; p != NULL; p = p->next) {   遍历的条件永远是 p != NULL
25          printf("%d -> ", p->value);
26      }
27      printf("NULL\n");
28  }
29
30  size_t list_count_iter(const Node *head) {
31      size_t n = 0;
32      for (const Node *p = head; p != NULL; p = p->next) {
33          n++;
34      }
35      return n;
36  }
```

19 新节点的 next 接上原来的头
20 返回新头，调用者必须接住它
24 遍历的条件永远是 p != NULL

这段代码里有三处值得停下来看：

1. **第 19 行**：`p->next = head;`。它装的是**改 `head` 之前**那个头指针。顺序不能反——先改 `head` 再写 `p->next`，新节点就指向自己，链当场成环。
2. **第 20 行**：函数返回新的头。头插会改「谁是头」，所以调用者必须这么写：`head = list_push_front(head, 1);`。忘了接返回值，新节点就成了没人知道的孤儿——**它还在堆上，谁也找不到它，也永远释放不了。**
3. **第 24 行**：遍历条件是 `p != NULL`，不是 `p->next != NULL`——后者会漏掉**最后一个节点**。打印、计数、释放都用前者。

跑一下这个例子里的 `main`：

c examples/ch05/list_head_insert/src/main.c 第 30–39 行

```
30      Node *head = NULL;
31      printf("sizeof(Node) = %zu 字节, sizeof(head) = %zu 字节\n", sizeof(Node), sizeof head);   16 与 8 这两个数字就是它打出来的
32      head = list_push_front(head, 1);   第一次头插，接住返回的新头
33      head = list_push_front(head, 2);
34      head = list_push_front(head, 3);
35      printf("头插三次之后: ");
36      list_print(head);
37      printf("迭代数出来 %zu 个\n", list_count_iter(head));   迭代数，走一遍循环
38      printf("递归数出来 %zu 个\n", list_count_rec(head));
39      list_free_all(head);
```

31 16 与 8 这两个数字就是它打出来的
32 第一次头插，接住返回的新头
37 迭代数，走一遍循环

终端输出 终端输出 list-head-insert-out

```
sizeof(Node) = 16 字节, sizeof(head) = 8 字节
头插三次之后: 3 -> 2 -> 1 -> NULL
迭代数出来 3 个
递归数出来 3 个
```

三行头插，插进去的顺序是 1、2、3，打印出来却是 `3 -> 2 -> 1`。**这不是 bug，这就是头插的定义**：每个新节点都成为新的头，所以最后一个插进去的排在最前面。想要「插进去什么顺序、打印就是什么顺序」，得用下一节的尾插。

内存形态 **头插三次：head 是唯一那个「把手」**

第 1 步 空表

`head` ：值是 `NULL`，表是空的

第 2 帧与第 3 帧合起来就是头插的两步：**先让新节点的 `next` 接上老链，再让 `head` 指过来**。顺序反了（第 3 帧先做），新链的头就丢了。第 4 帧是第二次头插：`head` 从 A 挪到 B，`A` 本身一个字都没改——**头插改的永远是两个东西：新节点的 `next`、`head` 自己。**

大白话 · 用大白话说链表

把内存想成一排寄存柜，链表就是「每个柜子里除了东西，还塞着一张写着**下一个柜子编号**的纸条」。

- `head` 是你手里那张纸条（编号 3 号柜）；
- 每个节点的 `next` 是柜子里的纸条，指向下一个柜子；
- 最后一个柜子里的纸条写的内容是「没有」——程序里就是 `NULL`；
- **头插**是「新开一个柜子，把手里那张纸条塞进去，再把手里那张换成新柜子的编号」；
- **删除**是「把前一个柜子里那张写着「3 号柜」的纸条，改成「4 号柜」」——柜子 3 就再也没人指得到它了，这时你才能把它清空（`free`）。

于是「遍历」就是照着纸条一路走下去，条件自然是「手里这张纸条不是「没有」」（`p != NULL`）。

13.4 尾插与尾指针：一条链的「把手」

头插只用改两处，所以它便宜。尾插不行：新节点要接在**最后**，你得先找到最后那个节点——找它要一个一个走。把这件事和「链表的把手」一起看清楚，就是这一节。

`examples/ch05/list_append/` 这个例子把一条链包成一个小小的结构体，头、尾、长度都记着：

c **一条链的「把手」：头指针 + 尾指针 + 长度** examples/ch05/list_append/include/list_append.h 第 11–24 行

```c
/* 一条链的「把手」：头指针 + 尾指针 + 长度。
   有尾指针，尾插才是 O(1)。 */
typedef struct List {
    Node *head;
    Node *tail;
    size_t len;
} List;

void list_init(List *l);

/* 两种尾插，都返回「这次调用走了多少步」。
   步数 = 从一个节点顺着 next 跳到下一个节点的次数。 */
size_t list_append_fast(List *l, int value);   /* 用 tail，0 步 */
size_t list_append_slow(List *l, int value);   /* 每次从头走到尾 */
```

两种尾插的实现只差一处：有没有用 `tail`。

c examples/ch05/list_append/src/list_append.c 第 23–36 行

```c
/* 尾插的正确写法：新节点接在 tail 后面，然后 tail 挪到新节点上；
```

```
    空表是特例 —— 那时 head 也必须一起改。 */
size_t list_append_fast(List *l, int value) {
    Node *p = node_new(value);
    if (l->tail == NULL) {                  空表是特例，head 也要改
        l->head = p;                        /* 空表：头尾都是它 */  头尾都指向这唯一一个节点
        l->tail = p;
    } else {
        l->tail->next = p;                  /* 接在原来的尾后面 */  接在原来的尾后面
        l->tail = p;                        /* 尾指针挪过去 */  尾指针挪到新节点
    }
    l->len++;
    return 0;                               /* 一步都不用走 */
}
```

27 空表是特例，head 也要改
28 头尾都指向这唯一一个节点
31 接在原来的尾后面
32 尾指针挪到新节点

第 27、28 行是那个必须单独处理的**特例**：空表没有尾节点，`tail->next` 会解引用空指针，所以「表是空的」要单独走一条路，把 `head` 和 `tail` 一起指到新节点上。这不是风格问题，是正确性问题——漏了它，第一次尾插就崩。

不用尾指针、每次都从头找尾的写法，唯一的额外工作就是那个 `while` 循环（注意它仍然在维护 `tail`，只是找尾不用它）：

c examples/ch05/list_append/src/list_append.c 第 38–56 行

```
/* 不用尾指针去找尾的写法：每插一个都要从头走到尾（tail 它照样维护）。 */
size_t list_append_slow(List *l, int value) {
    Node *p = node_new(value);
    size_t steps = 0;
    if (l->head == NULL) {
        l->head = p;
        l->tail = p;
    } else {
        Node *cur = l->head;
        while (cur->next != NULL) {         /* 找最后一个节点 */  一路找下去，每一步都记一次
            cur = cur->next;
            steps++;
        }
        cur->next = p;
        l->tail = p;
    }
    l->len++;
    return steps;
}
```

47 一路找下去，每一步都记一次

把两种写法放在一起数步数，差别就出来了：

c examples/ch05/list_append/src/main.c 第 7–22 行

```
int main(void) {
    List fast;
    List slow;
    list_init(&fast);  空表要先初始化，head 与 tail 都置 NULL
    list_init(&slow);  两条链各初始化一次

    /* 前五个先插进去，看看顺序对不对 */
    size_t fast_steps = 0;
    size_t slow_steps = 0;
    for (int i = 0; i < 5; i++) {
        fast_steps += list_append_fast(&fast, i);  用尾指针的尾插
        slow_steps += list_append_slow(&slow, i);  每次从头找尾的尾插
    }
    printf("尾插五个数: ");
```

```
21      list_print(&fast);
```

10 空表要先初始化，head 与 tail 都置 NULL
11 两条链各初始化一次
17 用尾指针的尾插
18 每次从头找尾的尾插

接着把它插到 2000 个，并把两种写法的步数分别累加起来：

c examples/ch05/list_append/src/main.c 第 23–38 行

```c
    /* 再插到 N 个，两种写法的步数分别累计 */
    for (int i = 5; i < N; i++) {
        fast_steps += list_append_fast(&fast, i);
        slow_steps += list_append_slow(&slow, i);
    }
    printf("插到 %d 个节点：\n", N);
    printf("  用尾指针走的步数      = %zu\n", fast_steps);   用尾指针走的步数
    printf("  每次从头找尾走的步数 = %zu\n", slow_steps);   每次从头找尾走的步数
    printf("  尾节点的值（读 tail，不走一步）= %d\n", fast.tail->value);   尾节点的值直接读 tail
    printf("  两条链内容一致 = %s\n", list_equal(&fast, &slow) ? "是" : "否");

    list_free(&fast);
    list_free(&slow);
    printf("list_free 之后：head == NULL 是 %s，len = %zu\n",
           fast.head == NULL ? "真" : "假", fast.len);
    return 0;
```

29 用尾指针走的步数
30 每次从头找尾走的步数
31 尾节点的值直接读 tail

终端输出 终端输出 list-append-out

```
尾插五个数：0 -> 1 -> 2 -> 3 -> 4 -> NULL（5 个）
插到 2000 个节点：
  用尾指针走的步数      = 0
  每次从头找尾走的步数 = 1997001
  尾节点的值（读 tail，不走一步）= 1999
  两条链内容一致 = 是
list_free 之后：head == NULL 是 真，len = 0
```

这两个步数要连起来读：**用尾指针的走 0 步**（它直接拿 `tail->next` 下手），**每次从头找尾的走了 1997001 步**。这个数不是随便冒出来的：插第 1 个走 0 步、插第 2 个也走 0 步；从第 3 个起，插第 k 个要走 k-2 步（它前面有 k-1 个节点，走到倒数第二个就停）。所以总数是 0+0+1+…+1998 = **1997001**——**节点数翻一倍，这个数差不多翻四倍**。这就是「O(1) 与 O(n)」的道理，先说结论，术语留到算法分册。

工程惯例 · 工程惯例：尾指针谁维护、踩到空表怎么办

「带尾指针」不是标准规定的东西，是主流做法，代价与注意点都很具体：

- **谁改链，谁负责把 `tail` 一起改。**改 `head` 不改 `tail`（或者反过来）是这类代码最常见的 bug，而且往往在小数据上跑得通——因为那时的 `tail` 碰巧还对。本书的做法是把 `head` / `tail` / `len` 打包进一个 `List`，让「改链」只发生在少数几个函数里；
- **空表是一个必须单独想清楚的形态。** `list_append_fast` 里的 `if (l->tail == NULL)` 就是为空表准备的；删元素删到表空时，`head` 与 `tail` 又得一起回到 `NULL`（`stack_list` 里的队列出队就是这个形状：弹出最后一个元素后 `rear` 也要清掉，13.11 节会看到）；
- **别忘了判 `malloc` 的返回值。**本例里的 `node_new` 判了：返回 `NULL` 就写一行消息到 stderr 再退出。链表每插一个节点都要向分配器要一块内存，这是链表与数组最本质的差别——**数组的内存一次要够，链表的内存一次要一点。**

13.5 删除节点：断链重连，先取 next 再 free

删除一个节点是三件事，顺序错了就出事：**找到它、把它从链上摘下来、还内存。**第一部分是「找」：

c examples/ch05/list_delete/src/list_delete.c 第 39–52 行

```c
/* 删掉第一个值为 value 的节点。
   两件事必须分清楚：
   - 摘链（改指针）在 free 之前，改完就没有谁指着 cur 了；
   - cur 的 next 要在 free **之前**读走（这里靠「摘链」顺手保住了它）。 */
Node *list_remove(Node *head, int value) {
    Node *prev = NULL;
    Node *cur = head;
    while (cur != NULL && cur->value != value) {
        prev = cur;
        cur = cur->next;
    }
    if (cur == NULL) {
        return head;                    /* 没找到：表原样返回 */
    }
```

46 两个条件，一个都不能少
47 prev 一直跟在 cur 后面
48 两个人一起往前走
51 没找到，表原样返回

`prev` 与 `cur` 一起往前走，是链表删除的标准走法：`cur` 是「现在看这个」，`prev` 是「上一个是谁」——**摘链需要的是前一个节点，不是当前节点。**这也是链表与数组最不一样的地方：数组删一个元素靠下标，链表删一个节点必须知道它的前驱。

c examples/ch05/list_delete/src/list_delete.c 第 53–61 行

```c
    Node *next = cur->next;             /* 先记下后继，才谈得上释放 cur */
    if (prev == NULL) {
        head = next;                    /* 删的是头节点：头指针要往前挪 */
    } else {
        prev->next = next;              /* 断链重连：跳过 cur */
    }
    free(cur);
    return head;
}
```

53 先把后继记下来，才谈得上释放 cur
55 删的是头，改的是 head
57 断链重连，跳过 cur
59 现在没人指着 cur 了，可以还回去

第 53 行那句 `Node *next = cur->next;` 是整个删除里最要紧的一句，理由和第 13.7 节那个反面教材直接相关：**`next` 要在 `free` 之前读走。**第 54 到 58 行是两种「摘链」：删的是头节点时改 `head` 自己，删的是中间或尾巴时改 `prev->next`。第 59 行的 `free(cur)` 在改完指针之后——这时整条链已经不包含 `cur` 了，释放它是安全的。

跑一下这六种情形（驱动代码在 `examples/ch05/list_delete/src/main.c`，六段几乎一样）：

c examples/ch05/list_delete/src/main.c 第 7–22 行

```c
int main(void) {
    printf("一、删中间: ");
    Node *h = list_from_array(kFive, 5);
    list_print(h);
    printf("          删 3 之后: ");
    h = list_remove(h, 3);
    list_print(h);
    list_free(h);

    printf("二、删头: ");
    h = list_from_array(kFive, 5);
```

```
18      list_print(h);
19      printf("          删 1 之后: ");
20      h = list_remove(h, 1); 删头同样要接住返回值
21      list_print(h);
22      list_free(h);
```

9 先按数组建一条链
12 删 3，接住新的头
13 头变了，打印的就是新的头
20 删头同样要接住返回值

终端输出 终端输出 list-delete-out

```
一、删中间: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
          删 3 之后: 1 -> 2 -> 4 -> 5 -> NULL
二、删头: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
          删 1 之后: 2 -> 3 -> 4 -> 5 -> NULL
三、删尾: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
          删 5 之后: 1 -> 2 -> 3 -> 4 -> NULL
四、删一个没有的值: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
          删 99 之后: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
五、空表: NULL
          删 1 之后: NULL
六、单元素: 7 -> NULL
          删 7 之后: NULL
```

这十二行输出要按「边界」读，每一对都是同一个函数在两种形态下的表现：

- **删中间**（第一、二行）：`prev->next` 跳过 `3`，链从 `1 2 3 4 5` 变成 `1 2 4 5`；
- **删头**（第三、四行）：`prev` 是 `NULL`，走的是「改 `head`」那条路，删完之后**新的头是 2**——调用者必须用返回值更新自己的头指针，忘接就是老问题：丢了这一整条链的把手；
- **删尾**（第五、六行）：`cur` 的 `next` 本来就是 `NULL`，`prev->next = next` 把尾巴摘掉，链尾自然落到 `4` 上；
- **删一个没有的值**（第七、八行）：`cur` 一路走到 `NULL`，函数原样返回——**不报错、不崩**，这是有意设计成这样的；
- **空表**（第九、十行）：`cur = head` 是 `NULL`，第 46 行那两个条件里的第一个立刻失败，`while` 一次都不进，返回 `NULL`。**空表删东西是合法操作**；
- **单元素**（第十一、十二行）：删的是头也是尾，删完 `head` 变成 `NULL`，表回到空表状态。

内存形态 **删中间那个节点：先断链，再 free**

第 1 步 **要找的是 3：prev 停在 2，cur 停在 3，next 先记下来**

节点1 : value = 1　节点2 : value = 2, prev 停在这里　节点3 : value = 3, cur 停在这里　节点4 : value = 4　节点5 : value = 5

节点1 → 节点2
节点2 → 节点3 这一条马上要被改掉
节点3 → 节点4
节点4 → 节点5
节点5 → NULL

第 2 步 **断链重连：把节点2 的 next 直接指向节点4**

节点1 : value = 1　节点2 : value = 2　节点3 : value = 3, 已经没人指着它了　节点4 : value = 4　节点5 : value = 5

节点1 → 节点2
节点2 → 节点4 跳过节点3
节点4 → 节点5
节点5 → NULL

第 3 步 free 之后，那块内存还给了分配器

节点1 : value = 1 | 节点2 : value = 2 | 节点4 : value = 4 | 节点5 : value = 5

节点1 → 节点2
节点2 → 节点4
节点4 → 节点5
节点5 → NULL

第 1 帧是「找」的结果：`prev`、`cur`、`next` 各就各位。第 2 帧是「摘链」：**改的是 `prev` 的 `next`，`cur` 自己一个字都没动**——它只是没人指着了。第 3 帧是「还内存」：那块内存回到分配器手里，链的形状没变。三帧就是那三句的顺序：**记 `next` → 改指针 → `free`**。

带哨兵的表：把「删头」这个特例消掉

上面那个 `list_remove` 有一个绕不开的分支：删的要是第一个节点，`prev` 是 `NULL`，得去改 `head`。**这个特例可以用一个哨兵节点消掉**——表头永远放一个不存数据的节点，真正的第一个元素变成 `head->next`：

c examples/ch05/list_delete/src/sentinel_list.c 第 27–41 行

```c
/* 有了哨兵，删除只剩一条路：prev 从哨兵出发，永远存在。
   「删掉第一个元素」不再是特例 —— 它的前驱就是哨兵。 */
int sentinel_remove(SNode *head, int value) {
    SNode *prev = head;                 /* prev 从哨兵出发，永远不是 NULL */
    while (prev->next != NULL && prev->next->value != value) {   /* 看的是 prev->next，不是 prev */
        prev = prev->next;
    }
    if (prev->next == NULL) {
        return 0;                       /* 没找到 */
    }
    SNode *dead = prev->next;
    prev->next = dead->next;            /* 断链重连，只有这一种写法 */   /* 只有这一种断链写法了 */
    free(dead);
    return 1;
}
```

30 prev 从哨兵出发，永远不是 NULL
31 看的是 prev->next，不是 prev
38 只有这一种断链写法了

`prev` 从哨兵出发，于是「删第一个元素」和「删中间」走的是**同一段代码**——它的前驱是哨兵。跑一遍四个位置：

c examples/ch05/list_delete/src/sentinel_main.c 第 7–20 行

```c
int main(void) {
    SNode *head = sentinel_new();       /* 空表 = 一个哨兵 */   /* 空表就是「只有一个哨兵」 */
    printf("刚建好的空表: ");
    sentinel_print(head);

    for (int i = 1; i <= 5; i++) {      /* 头插还是插在哨兵后面 */
        sentinel_push(head, i);         /* 头插 1..5 */
    }
    printf("头插 1 到 5 之后: ");
    sentinel_print(head);

    printf("删第一个元素 5 (无哨兵时要走「改 head」那条分支) : %s\n",   /* 删第一个元素：走的是同一段代码 */
           sentinel_remove(head, 5) ? "删掉了" : "没找到");
    sentinel_print(head);
```

8 空表就是「只有一个哨兵」
12 头插还是插在哨兵后面
18 删第一个元素：走的是同一段代码

接着换三个位置再删，包括一个不存在的值：

c examples/ch05/list_delete/src/sentinel_main.c 第 22–30 行

```c
22      printf("删中间的 3: %s\n", sentinel_remove(head, 3) ? "删掉了" : "没找到");
23      sentinel_print(head);
24
25      printf("删尾巴 1: %s\n", sentinel_remove(head, 1) ? "删掉了" : "没找到");
26      sentinel_print(head);
27
28      printf("删一个没有的值 99: %s\n", sentinel_remove(head, 99) ? "删掉了" : "没找到");
29      sentinel_print(head);
30      printf("表里还剩 %zu 个元素（哨兵不算）\n", sentinel_count(head));
```

22 删中间
28 删一个没有的值：函数返回 0

终端输出 终端输出 list-sentinel-out

```
刚建好的空表：哨兵 -> NULL
头插 1 到 5 之后：哨兵 -> 5 -> 4 -> 3 -> 2 -> 1 -> NULL
删第一个元素 5（无哨兵时要走「改 head」那条分支）：删掉了
哨兵 -> 4 -> 3 -> 2 -> 1 -> NULL
删中间的 3：删掉了
哨兵 -> 4 -> 2 -> 1 -> NULL
删尾巴 1：删掉了
哨兵 -> 4 -> 2 -> NULL
删一个没有的值 99：没找到
哨兵 -> 4 -> 2 -> NULL
表里还剩 2 个元素（哨兵不算）
```

还有一件事：**哨兵自己也要释放。**它不存数据，但同样是一次 `malloc` 来的，所以 `sentinel_free` 必须从哨兵开始走（`head` 本身也要 `free`），不能从 `head->next` 开始——本例 `sentinel_main` 最后一行调的就是它。

代价也写清楚：**多一个节点**（每次建表都 `malloc` 一个）、**遍历要从 `head->next` 开始**（忘了就会把哨兵的占位值也数进去）、**空表的判断变成 `head->next == NULL`**。换来的是删除与头插都不再分「头 / 非头」两路——**用一个节点换掉一个分支**，这是链表里最常见的一笔交易。工程上两种写法都有人用：前者不额外要内存，后者调用处分支更少。

{} 语法规则 · 头插、尾插、删除、查找，各自改哪几个指针

四种操作改的指针数量差别很大，这张表值得记下来（它也是 13.8 节「代价对照」的一半）：

- **头插**：改 2 处——新节点的 `next`、`head`；
- **尾插（有尾指针）**：改 2 处——老尾节点的 `next`、`tail`（空表时改 `head` 与 `tail`）；
- **删除（已经有前驱 `prev`）**：改 2 处——`prev->next`（或 `head`）、外加被删节点的 `next` 要先读走；
- **按值查找**：改 0 处，但要从头上**一个一个走。**

最下面那条是链表的命门：**想删谁，得先找到谁；找到谁之前，得先走到谁。**后面两节算的就是这笔账。

13.6 释放整条链：`list_free` 收的是 `head` 的地址

释放整条链有一个必须记住的顺序：**先把下一个节点的地址存进局部变量，再释放当前节点。**理由很直接——`free` 之后那块内存就不属于你了，再去读它的 `next` 就是第 13.7 节那个反面教材。`examples/ch05/list_free/` 这个例子把这件事和「所有权」一起演示：

c examples/ch05/list_free/src/list_free.c 第 40–51 行

```c
40  /* 释放整条链。两处关键：
41     1. 每个节点在 free 之前先把 next 存进局部变量 —— 不然下一个节点就找不到了；
42     2. 参数是 Node **，所以函数结束前能把调用者的 head 置空 —— 这样再调一次是安全的。 */
43  void list_free(Node **head) {
```

```
    Node *p = *head;
    while (p != NULL) {
        Node *next = p->next;              /* 先存下一站 */ 先把下一站存进局部变量
        free(p); 再还这一站
        p = next;
    }
    *head = NULL;                          /* 还回去一个「干净的 NULL」 */ 顺手把调用者的 head 置空
}
```

46 先把下一站存进局部变量
47 再还这一站
50 顺手把调用者的 head 置空

参数是 `Node **head`，与 13.5 节「返回新头」的 `list_remove` 形成对照——两种写法都在解决同一件事：**想改一个 `Node *` 变量的值，必须拿到它的地址**（第 9 单元的「值传递」）。收地址的好处是能顺手置空：

c examples/ch05/list_free/src/main.c 第 5-16 行

```
int main(void) {
    Node *head = list_build(4);
    printf("释放前: head 是%s, 链上有 %zu 个节点\n", head == NULL ? "空" : "非空", list_count(head));

    list_free(&head); 第一次释放
    printf("第一次 list_free 之后: head == NULL 是 %s\n", head == NULL ? "真" : "假"); 释放完之后 head 真的变成 NULL 了吗

    list_free(&head);                      /* 同一个地址再传一次: 这是合法调用 */ 同一个地址再传一次，这是合法调用
    printf("第二次 list_free 之后: head == NULL 是 %s, 链上还有 %zu 个节点\n", 第二次调用什么都没发生
           head == NULL ? "真" : "假", list_count(head));

    return 0;
```

9 第一次释放
10 释放完之后 head 真的变成 NULL 了吗
12 同一个地址再传一次，这是合法调用
13 第二次调用什么都没发生

终端输出 终端输出 list-free-out

```
释放前: head 是非空, 链上有 4 个节点
第一次 list_free 之后: head == NULL 是 真
第二次 list_free 之后: head == NULL 是 真, 链上还有 0 个节点
```

这份输出的三行分别是：释放前链上 **4 个节点**、第一次 `list_free` 之后 `head == NULL` 是**真**、第二次调用之后**还是真、而且链上 0 个节点。「同一个 `list_free` 能安全地调两次」不是巧合，是标准明确允许的**：

标准原文 · 标准原文：free 与「什么都没发生」的那一次

C 语言标准（ISO/IEC 9899，C17）7.22.3.3 节「The `free` function」第 2 段：

> The `free` function causes the space pointed to by `ptr` to be deallocated, that is, made available for further allocation. If `ptr` is a null pointer, no action occurs. Otherwise, if the argument does not match a pointer earlier returned by a memory management function, or if the space has been deallocated by a call to `free` or `realloc`, the behavior is undefined.

大意：`free` 把 `ptr` 指的那块空间还回去；**如果 `ptr` 是空指针，什么都不做**；否则，如果这个指针不是内存管理函数返回过的、或者这块空间**已经释放过**了，那就是**未定义行为**。

出处：ISO/IEC 9899:2018（C17）§7.22.3.3 第 2 段。

这条标准把「调两次安全」的条件说死了：**第二次传进去的必须是空指针**。本例第一次调用结束时执行了 `*head = NULL;`，所以第二次传的是 `NULL`。反过来，两个指针指着同一个节点、只 `free` 了一个、另一个没置空，第二次 `free` 就是那句 `behavior is undefined`——与第 12 单元讲 `double free` 同一条。

大白话 · **用大白话说「谁负责置空」**

置空只能由持有那个指针的人来做：节点在链表模块里 `malloc`，释放也就住在同一个模块，`main` 只把「把手」交出去；`Node **` 让函数能改调用者手里那个变量，把它变成 `NULL`，所以再调一次不出事。而**别名**（好几个指针指着同一个节点）谁都救不了——`list_free` 只能把它知道的那一个置空。

13.7 两种把链表写坏的方式

这一节是反面教材专场。两种错法都能在编译期拿到证据，但来源不同：第一种是**一条普通警告 + 一次「看起来完全正常」的运行**；第二种只有**静态分析**（我们不运行它——跑起来必然撞上本节末尾那个 3221225477）。

先 free 再取 next

第一种错法就是第 13.6 节说的那个顺序反了。`examples/ch05/list_bad_order/src/main.c` 里这段循环是故意写坏的：

c examples/ch05/list_bad_order/src/main.c 第 14–24 行

```c
    size_t freed = 0;
    Node *p = head;
    while (p != NULL) {
        Node *before = p->next;             /* 只为了对照：记下 free 之前的 next */ 只为了对照，记下 free 之前的 next
        free(p); 先把节点还回去
        printf("  释放第 %zu 个节点：从同一格读到的 next 与 free 之前%s\n",
               freed + 1, (p->next == before) ? "一样" : "不一样"); 从已经还回去的那块内存里再读 next
        p = p->next;                        /* ← 悬垂读：靠它才走到下一个节点 */ 靠这个读出来的值继续往下走
        freed++;
    }
    printf("循环走完了：一共释放了 %zu 个节点，程序没有崩\n", freed);
```

17 只为了对照，记下 free 之前的 next
18 先把节点还回去
20 从已经还回去的那块内存里再读 next
21 靠这个读出来的值继续往下走

终端输出 终端输出 list-bad-order-out

```
链上有 3 个节点
按错的顺序释放：free 之后再取 next
  释放第 1 个节点：从同一格读到的 next 与 free 之前一样
  释放第 2 个节点：从同一格读到的 next 与 free 之前一样
  释放第 3 个节点：从同一格读到的 next 与 free 之前一样
循环走完了：一共释放了 3 个节点，程序没有崩
```

这份输出是这一节最要紧的证据，因为它长得一点都不像出错：三次释放，每次都「读到的 next 与 free 之前一样」，循环走完，退出码 0。也就是说「先 `free` 再取 `next`」在这台机器上**既没崩、也没读到垃圾值**——还回去的那块内存**没被立刻擦掉。这就是它最坏的地方**：写了未定义行为，测试却全绿。

编译器其实早就看出来了。同一份代码在本书的固定参数下编译，会收到两条 `-Wuse-after-free` 警告（会话原文如下）：

终端会话 编译器的原话：pointer 'p' used after 'free' ch05-warning-list

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c
src/list_bad_order.c -o app.exe
src/main.c: In function 'main':
src/main.c:21:11: warning: pointer 'p' used after 'free' [-Wuse-after-free]
   21 |         p = p->next;                          /* ← 悬垂读：靠它才走到下一个节点 */
      |         ~~^~~~~~~~~
src/main.c:18:9: note: call to 'free' here
   18 |         free(p);
      |         ^~~~~~~
src/main.c:20:29: warning: pointer 'p' used after 'free' [-Wuse-after-free]
   20 |                 freed + 1, (p->next == before) ? "一样" : "不一样");
      |                            ~^~~~~~
src/main.c:18:9: note: call to 'free' here
   18 |         free(p);
      |         ^~~~~~~
$ echo $?
0
```

两条警告指着代码里两处 `p->next`，后面都跟着一句 `note: call to 'free' here`——**编译器把「你在这里 free 了它」也标出来了。**这份证据归 `-Wall` 管（它就在固定参数里），不归任何检测模式，所以它每次编译都会出现，不管你跑不跑这个程序。

编译器视角 · **编译器视角：「看起来正常」与「真的是对的」是两件事**

`-Wuse-after-free` 报的是「这个指针在 `free` 之后还被用到了」，不是「这次运行会出错」。两者的差别在写代码时比在运行时更值得关心：

- **它有来源**：警告把 `free` 的位置（第 18 行）和「再用它的位置」（第 20、21 行）一起给出来，读的时候要一对一对地看；
- **它只在能看出来的时候报。**把指针绕两圈（存进数组、经过函数、类型转换）之后，编译器就不一定还认得出来——所以警告缺席不等于代码没问题；
- **UB 不等于崩溃**：本机实测是「退出码 0 + 三次「一样」」。**未定义行为的意思是「这台机器这次恰好没出事」**——换分配器、换机器，或在这两次读之间再要一块内存，结果都可能变。

另一半话也得说完整：**「读到旧值」只是 UB 可能的结局之一。**这块内存一旦被分配器再借出去，读到的就是新主人写下的东西；具体读到什么、会不会当场崩，取决于分配器和这一刻堆的样子。我**没有**替这一句给出固定读数——它在同一台机器上也会变，给一个具体数字反而是骗人。能给你固定读数的是上面那条 `-Wuse-after-free`：它在编译期就成立，每次编译都在。

没判空就顺着指针取成员

第二种错法是「找」的时候没判空。找节点的函数天生有两种结果：找到（返回那个节点）和没找到（返回 `NULL`）。第二种结果一旦被当成第一种用，就是空指针解引用。`examples/ch05/list_delete/src/null_main.c` 就是这么写的：

c examples/ch05/list_delete/src/null_main.c 第 7–24 行

```c
/* 契约：找到就返回那个节点，找不到返回 NULL。 */
static Item *find_item(Item *head, int value) {
    for (Item *p = head; p != NULL; p = p->next) {
        if (p->value == value) {
            return p;
        }
    }
    return NULL; 契约写在注释里：找不到返回 NULL
}

int main(void) {
    Item *head = NULL;
    head = chain_push(head, 1);
    head = chain_push(head, 2);
```

```
21
22      printf("链上只有 1 和 2，下面去找 99\n");
23      int v = find_item(head, 99)->value;        /* ← 没判空就顺着它取 value */  没判空就顺着它取 value
24      printf("v = %d\n", v);
```

14 契约写在注释里：找不到返回 NULL
23 没判空就顺着它取 value

这份代码**编得过**（`-Wall -Wextra -Wpedantic` 一声不吭），但换成 `-fanalyzer` 再编一次，编译器在**编译期**就把这条路径走了一遍：

终端会话 `-fanalyzer`：编译期就报出这处空指针解引用 ch05-analyzer-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -fanalyzer -Iinclude src/null_main.c
src/null_check.c -o app.exe
src/null_main.c: In function 'main':
src/null_main.c:23:9: warning: dereference of NULL '0' [CWE-476] [-Wanalyzer-null-dereference]
   23 |     int v = find_item(head, 99)->value;        /* ← 没判空就顺着它取 value */
      |         ^
```

终端会话 （续）它把那条路走到底，最后一步就是解引用 NULL；编译退出码仍是 0 ch05-analyzer-2

```
  'main': events 6-7
    |
    |   23 |     int v = find_item(head, 99)->value;        /* ← 没判空就顺着它取 value */
    |      |         ~   ^~~~~~~~~~~~~~~~~~~
    |      |             |   |
    |      |             |   (6) returning to 'main' from 'find_item'
    |      |             (7) ⚠  dereference of NULL 'find_item(head, 99)'
    |
$ echo $?
0
```

第一块的第三行是诊断本体：`dereference of NULL '0' [CWE-476] [-Wanalyzer-null-dereference]`。读它要分三段：`dereference of NULL` 是问题（解引用空指针）、`[CWE-476]` 是业界给这类缺陷编的号（CWE 是一份公开的缺陷分类表，476 号就是「NULL 指针解引用」）、`[-Wanalyzer-null-dereference]` 是控制这条诊断的开关名。第二块是这条路径的结尾：`(7) ⚠ dereference of NULL 'find_item(head, 99)'`，紧接着 `$ echo $?` 打出 **0**——**分析器报了问题，编译仍然成功。**

实验记录 · 本机实测：这条警告确实归 -fanalyzer 管

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令（带检测器）：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -fanalyzer -Iinclude src/null_main.c src/null_check.c -o app.exe`；
- 结果：stderr 里出现 `-Wanalyzer-null-dereference` 与 `CWE-476`，**编译退出码 0**（上面第二块最后两行就是）；
- 阴性对照：把 `-fanalyzer` 去掉、其他参数一字不改再编一次，**stderr 一个字都没有**、退出码仍是 0——所以这句诊断确实归这个分析器，不是基础警告碰巧报出来的；
- 声明这次检测的清单是 `examples/ch05/list_delete/list_delete_fanalyzer.json`（`mode` 是 `detect`、`detect` 是 `fanalyzer`、`expect.stderrContains` 里写着那两个关键词），流水线会去掉检测器参数再复算一遍，证据对不上就不算数。

`-fanalyzer` **是静态分析，它给的是「编译器看到的」**：它可能误报，也可能因为路径太绕而漏报；它不运行程序，所以「这里一定会崩」这种话它不说，也不要替它说。

> ⚠ **陷阱 · 链表里两个最容易漏的边界**
>
> **你会怎么被坑到**：链表代码在小数据上跑得通，在边界上崩掉——而边界恰恰是链表最常见的形态：
>
> - **空表**：`head` 是 `NULL` 时，`head->value` 会解引用空指针。本机实测的退出码是 **3221225477**（0xC0000005 访问冲突）；如果崩溃前没有 `fflush(stdout)`，经管道抓取时 stdout 会是空的（缓冲区没来得及刷），我另写了一个先 `fflush` 再解引用的探针，看到的是退出码 3221225477、stdout 只有那一行 `before`。所以**每个收 `Node *` 的函数开头都要想一句：传进来是 `NULL` 怎么办**——`list_remove` 的答案是「原样返回」，`list_free` 的答案是「什么都不做」；
> - **先 `free` 再取 `next`**：本节前半的证据是「看起来完全正常」（退出码 0）。所以**不要用「跑一遍没问题」来判断这类代码对不对**——要按顺序检查每一处 `free` 之后有没有再用那个指针。
>
> 两条共同的确定性做法：**判空**，以及**该置空的时候立刻置空。**

13.8 链表还是数组：代价怎么比

链表不是「更好的数组」。这一节把两边的账算清楚，用的都是**数出来的**数字。
`examples/ch05/list_delete/src/cost_array.c` 里是数组那一半：

c examples/ch05/list_delete/src/cost_array.c 第 3–9 行

```c
size_t array_remove_at(int *a, size_t n, size_t i) {
    size_t moved = 0;
    for (size_t k = i; k + 1 < n; k++) {   从要删的位置开始
        a[k] = a[k + 1];                   /* 后面每一个都得往前挪一格 */   后面每一个都往前挪一格
        moved++;   挪一格就记一次
    }
    return moved;
```

5 从要删的位置开始
6 后面每一个都往前挪一格
7 挪一格就记一次

而链表删第一个节点，只需要把 `head` 往前挪一格。把两件事放在同一个程序里跑（`examples/ch05/list_delete/src/cost_main.c`）：

c examples/ch05/list_delete/src/cost_main.c 第 22–41 行

```c
    printf("同一件事：删掉第一个元素（一共 %d 个）\n", N);
    size_t n = N;                          /* 数组的「逻辑长度」：删一个就少一个 */
    size_t moved = array_remove_at(a, n, 0);   删一个，搬移多少个是数出来的
    n--;
    printf("  数组：搬移了 %zu 个元素（约 %zu 字节），逻辑长度变成 %zu\n",   逻辑长度跟着减一
           moved, moved * sizeof(int), n);
    head = list_remove(head, 0);
    printf("  链表：头指针往前挪一格，搬移 0 个元素\n");
    printf("  删完之后：数组第一个是 %d，链表第一个节点是 %d\n", array_get(a, 0), head->value);

    printf("同一件事：取现在最后一个元素（第 %zu 个）\n", n);
    int last = array_get(a, n - 1);        /* 下标直接算出地址，读一次 */   真的把最后一个元素读出来
    printf("  数组：a[%zu] 读一次就是 %d（下标直接算地址，没有循环）\n", n - 1, last);
    size_t hops = 0;
    const Node *p = head;
    while (p->next != NULL) {
        p = p->next;                       /* 链表只能一个一个走 */   链表只能一路走，步数也是数出来的
        hops++;
    }
    printf("  链表：顺着 next 走了 %zu 步才到 %d\n", hops, p->value);
```

24 删一个，搬移多少个是数出来的
26 逻辑长度跟着减一
33 真的把最后一个元素读出来
38 链表只能一路走，步数也是数出来的

终端输出 终端输出 list-delete-cost-out

```
同一件事：删掉第一个元素（一共 100000 个）
  数组：搬移了 99999 个元素（约 399996 字节），逻辑长度变成 99999
  链表：头指针往前挪一格，搬移 0 个元素
  删完之后：数组第一个是 1，链表第一个节点是 1
同一件事：取现在最后一个元素（第 99999 个）
  数组：a[99998] 读一次就是 99999（下标直接算地址，没有循环）
  链表：顺着 next 走了 99998 步才到 99999
```

这几行里的数字，要按两组对照来读：

- **删第一个元素**：数组搬移了 **99999 个元素**（约 399996 字节），链表搬移 **0 个元素**。这就是「链表插入删除便宜」的准确含义——便宜在**不需要搬东西**。多数情况下，搬的是「内存里的一大段字节」，改的是「两个指针变量」；
- **取现在的最后一个元素**：删完第一个之后只剩 99999 个，最后一个的下标是 **99998**。数组那边一句 `array_get(a, 99998)` 就读出来了——**没有循环**，下标直接算出地址；链表那边**走了 99998 步**。这就是「链表随机访问贵」的准确含义——贵在**只能一个一个走**。

所以取舍不是「谁更好」，而是**你更常做哪件事**：

你要做的事	数组	链表
按下标取第 i 个	直接算出地址，走 0 步	从头上走 i 步
在已知位置插入 / 删除	后面每个都要挪	改两个指针
内存布局	一整块连续	一个个散在堆上
每个元素的开销	就是元素本身	元素 + 一个 8 字节指针 + 分配器的管理开销

最后两行那条差别，在数据量大到装不进缓存时最明显。下面是拿同一批 1000000 个整数、同一件事（全部求和）做的一次真跑。探针的源码是 `examples/ch05/list_delete/bench_hops.c`——它**故意没有清单**：耗时每次都不一样，进不了逐字对账的 `output` 块，所以它的证据形式是抓下来的会话。

终端会话 探针：1000000 个元素扫 30 遍，数组 vs 链表 ch05-cost-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g bench_hops.c -o bench.exe
$ ./bench.exe
1000000 个元素扫 30 遍（每遍求和，结果互相验证）：
  数组：求和 = 14999985000000，一共 4 ms，平均 0.13 ms/遍
  链表：求和 = 14999985000000，一共 324 ms，平均 10.80 ms/遍
```

终端会话 （续）同一个探针的另外两次运行 ch05-cost-2

```
$ ./bench.exe
1000000 个元素扫 30 遍（每遍求和，结果互相验证）：
  数组：求和 = 14999985000000，一共 4 ms，平均 0.13 ms/遍
  链表：求和 = 14999985000000，一共 320 ms，平均 10.67 ms/遍
$ ./bench.exe
1000000 个元素扫 30 遍（每遍求和，结果互相验证）：
  数组：求和 = 14999985000000，一共 4 ms，平均 0.13 ms/遍
  链表：求和 = 14999985000000，一共 356 ms，平均 11.87 ms/遍
```

数组大约 **0.13 毫秒/遍**，链表大约 **10.7 到 11.9 毫秒/遍**——**差了八十倍上下**。这两边的代码做的事情完全一样（每个元素加一次），差别只在「内存里怎么走」：

- 数组是**一整块**：CPU 读第一个元素时会把附近一整段内存搬进缓存，后面几十个元素的求和几乎是白送的；
- 链表是**一堆小块**：每个节点在堆上的哪个角落、离下一个节点多远，都不确定（本例是头插建的，节点在内存里基本是反着排的）。每跟着 `next` 跳一次，缓存基本都要重新装一次。

工程惯例 · 工程惯例：什么时候别用链表

「链表插入删除是 O(1)」这句话没错，但要加三个前提，否则你会选错数据结构：

- **前提一：你已经站在那个位置上。**链表删一个节点的 O(1) 是「已经知道 `prev` 和 `cur` 」之后的事；**找到它**那一步是 O(n)（第 13.5 节的 `while` 就是它）。所以「先按值查找、再删除」这个组合，两边一样慢；
- **前提二：数据量不大，或者你几乎不遍历。**那个八十倍是在 100 万个元素上测出来的；数据量小的时候，缓存轮不到你做主；
- **前提三：你不按下标访问。**链表的第 i 个元素没有「地址直接算出来」这回事，一旦访问模式变成「随机取第几个」，链表的代价立刻暴露（本例是 99998 步）。

反过来，真正该用链表的场合很具体：**元素个数变化很大、而且增删总是发生在两头或已知位置**——栈、队列（13.11 节），以及算法分册里的链表题。**一句话：链表赢在「不用搬」，输在「不能跳」。**

13.9 调试链表：gdb 顺着 next 走

链表出错时，最容易犯的错是「打印一堆东西猜哪里断了」。gdb 里有一组命令专门看这个：`p *head` 看头节点、`p *head->next` 看第二个节点——**把它们串起来就是把一条链一段段摊开。**下面这段会话是真的（文件在 `examples/ch05/session-gdb-list.txt` ）：

终端会话 gdb：p *head、p *head->next，一直走到 NULL　　ch05-gdb-list

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/list_head_insert.c -o app.exe
$ gdb --batch -ex "break src/main.c:35" -ex run -ex "p *head" -ex "p *head->next" -ex "p *head->next->next" -ex "p head->next->next->next" -ex "p head->next->next->next->value" -ex "p head->value" ./app.exe
Breakpoint 1 at 0x14000151e: file src/main.c, line 35.
[New Thread 150620.0x20ad8]
Thread 1 hit Breakpoint 1, main () at src/main.c:35
35          printf("头插三次之后: ");
$1 = {value = 3, next = 0x6da000}
$2 = {value = 2, next = 0x6da630}
$3 = {value = 1, next = 0x0}
$4 = (struct Node *) 0x0
Cannot access memory at address 0x0
$5 = 3
$ rm -f app.exe
```

调试实录 · 调试实录：六条 p 命令，把一条三个节点的链看穿

上面那块会话原文的命令行里是 8 个 `-ex` 参数：`break src/main.c:35`（在头插完三次、准备打印那一行停住）、`run`（跑起来），加上 **6 条** `p`（打印）。命令与输出就是上面的会话原文，结论写在下面：

- `p *head` 打出 `$1 = {value = 3, next = 0x6da000}`：头节点的**结构**，`value = 3`，`next` 是第二个节点的地址；
- `p *head->next` 打出 `$2 = {value = 2, next = 0x6da630}`：第二个节点；再往下一层 `p *head->next->next` 是第三个（`$3`：`value = 1`，`next = 0x0`）；
- `p head->next->next->next` 打出 `$4 = (struct Node *) 0x0`：**走到 NULL 了。**再往前一步 `p head->next->next->next->value`，gdb 直接说 `Cannot access memory at address 0x0`——**这就是「链到此结束」在调试器里的样子**；
- 最后 `p head->value` 打出 `$5 = 3`，回到头节点，说明这几条表达式之间没有副作用（`p` 只是读，不改链）。

会话里那串 `0x6da000` 每次运行都不一样（第 9 单元就说过），**所以看链表时不要盯住地址值，要看「谁指着谁」**：稳定的只有 `{value = ..., next = ...}` 这个形状。另外两条经验：

- 表达式太长时可以逐段走：`set $p = head->next->next` 再 `p *$p`——gdb 的变量（`$p`）是调试器自己的，跟程序里的变量没关系；
- 有环的链在 gdb 里**不会自己停**：`while (p != NULL) p = p->next;` 会在环上转一辈子——先怀疑「有没有环」（下一节），再看别处。

13.10 环检测：快慢指针

链表的尾巴如果没指到 `NULL`，而是指回了链上的某个节点，那它就不再是一条链，而是一个环。**有环的链会让第 13.3 节那个标准的遍历条件失效**：`p != NULL` 永远为真，循环永远不结束。这不是理论问题——两个链表接错了、删除时把

`next` 接回自己，都会当场造出一个环。

本机实测：给一条 5 节点的链（`1 2 3 4 5`）把尾巴指回 3 号，再用 `p != NULL` 走它，**1000 步撞上上限还没停**；同样一条无环链，**5 步就走到 `NULL`**（两个数都在下面的输出里）。

判环的办法叫**快慢指针**（也叫 Floyd 判圈算法）。它的实现只有几行，住在 `examples/ch05/circular_detect/src/circular_detect.c`：

c examples/ch05/circular_detect/src/circular_detect.c 第 48–66 行

```c
/* 快慢指针：慢的每次一步、快的每次两步。
   无环时快的先走到 NULL；有环时快的必追上慢的。 */
CycleReport cycle_check(const Node *head) {
    CycleReport r;
    r.has_cycle = 0;
    r.rounds = 0;
    r.cycle_len = 0;
    r.meet_value = -1;

    const Node *slow = head;
    const Node *fast = head;
    while (fast != NULL && fast->next != NULL) {   /* 两个结束条件缺一不可 */
        slow = slow->next;                         /* 慢的一步 */   /* 慢的走一格 */
        fast = fast->next->next;                   /* 快的两步 */   /* 快的走两格 */
        r.rounds++;                                /* 走一轮，两个指针各动一次 */
        if (slow == fast) {                        /* 追上就是有环 */   /* 追上就是有环 */
            r.has_cycle = 1;
            r.meet_value = slow->value;
            r.cycle_len = 1;
```

59 两个结束条件缺一不可
60 慢的走一格
61 快的走两格
63 追上就是有环

c examples/ch05/circular_detect/src/circular_detect.c 第 67–74 行

```c
            for (const Node *p = slow->next; p != slow; p = p->next) {
                r.cycle_len++;                     /* 从相遇点绕一圈回到它自己 */   /* 从相遇点绕一圈，数回它自己 */
            }
            return r;
        }
    }
    return r;                                      /* 走到 NULL：没有环 */   /* 快的走到 NULL，说明没有环 */
}
```

68 从相遇点绕一圈，数回它自己
73 快的走到 NULL，说明没有环

循环的结束条件写成 `fast != NULL && fast->next != NULL`，不是 `slow != NULL`：**跑得快的那个人先撞线**，它要走两步，所以得同时确认「它不是 NULL」和「它的下一个不是 NULL」（不然第 61 行的 `fast->next->next` 会解引用空指针）。

图 / 搜索过程 **快慢指针：慢的每次走一格，快的每次走两格，追上就是有环**

第 1 步 起跑：两个指针都站在 1 号节点上

节点1 ：slow 与 fast 都在这里　节点2 ：还没走到　节点3 ：还没走到　节点4 ：还没走到　节点5 ：还没走到

节点1 → 节点2
节点2 → 节点3
节点3 → 节点4
节点4 → 节点5
节点5 → 节点3 尾指回 3 号，这就是环

第 2 步 **第 1 轮：slow 到 2，fast 到 3**

节点2 ：slow 走到这里　节点3 ：fast 走到这里　节点4 ：还没走到　节点5 ：还没走到

节点2 → 节点3
节点3 → 节点4
节点4 → 节点5
节点5 → 节点3 环还在这里

第 3 步 **第 2 轮：slow 到 3，fast 到 5**

节点2 ：slow 的上一步　节点3 ：slow 走到这里　节点4 ：fast 路过　节点5 ：fast 走到这里

节点2 → 节点3
节点3 → 节点4
节点4 → 节点5
节点5 → 节点3

第 4 步 **第 3 轮：两个都落到 4 号上，相遇**

节点2 ：slow 走过　节点3 ：slow 走过　节点4 ：slow 与 fast 在这里相遇　节点5 ：fast 的上一步

节点2 → 节点3
节点3 → 节点4
节点4 → 节点5
节点5 → 节点3

第 1 帧是起跑线：两个指针都站在头节点上。第 2、3 帧是每走一步的样子——`fast` **每步比** `slow` **多走一格。**第 4 帧是相遇：两个指针在 4 号节点上重合，这时就能断定「有环」。为什么一定会相遇？因为一旦进了环，`fast` 每走一步就比 `slow` 近一格，而环的长度是有限的——**差的距离只减不增，迟早减到 0。**

跑五种情形看看：

c　examples/ch05/circular_detect/src/main.c　第 24–41 行

```c
    Node *a = chain_build(five, 5);
    show("二、1 -> 2 -> 3 -> 4 -> 5 -> NULL", a, 5);   无环的链

    Node *b = chain_build(five, 5);
    chain_close(b, 2);                          /* 尾指回 3 号节点 */   把尾节点的 next 指回 3 号
    show("三、1 -> 2 -> 3 -> 4 -> 5 -> 3 (尾指回 3 号) ", b, 5);   同一个 chain_close 造出来的环

    Node *c = chain_build(nine, 1);
    chain_close(c, 0);                          /* 自己指回自己 */
    show("四、9 -> 9 (单节点自环) ", c, 1);

    Node *d = chain_build(three, 3);
    chain_close(d, 1);                          /* 尾指回 2 号节点 */
    show("五、1 -> 2 -> 3 -> 2 (尾指回 2 号) ", d, 3);

    printf("用 p != NULL 的老办法遍历（上限 1000 步）：\n");
    printf("  无环的表：走了 %zu 步走到 NULL\n", walk_bounded(a, 1000));   没有环：几步就走到 NULL
    printf("  有环的表：走了 %zu 步撞上上限也没停\n", walk_bounded(b, 1000));   有环：撞上限也没停
```

25 无环的链
28 把尾节点的 next 指回 3 号
29 同一个 chain_close 造出来的环
40 没有环：几步就走到 NULL
41 有环：撞上限也没停

终端输出 终端输出 circular-detect-out

```
一、空表
  有环 = 否
二、1 -> 2 -> 3 -> 4 -> 5 -> NULL
  有环 = 否，循环走了 2 轮，快指针先撞上 NULL
三、1 -> 2 -> 3 -> 4 -> 5 -> 3（尾指回 3 号）
  有环 = 是，循环走了 3 轮相遇于节点 4，环长 = 3
四、9 -> 9（单节点自环）
  有环 = 是，循环走了 1 轮相遇于节点 9，环长 = 1
五、1 -> 2 -> 3 -> 2（尾指回 2 号）
  有环 = 是，循环走了 2 轮相遇于节点 3，环长 = 2
用 p != NULL 的老办法遍历（上限 1000 步）：
  无环的表：走了 5 步走到 NULL
  有环的表：走了 1000 步撞上上限也没停
```

这份输出里的数字都能对上账：

- **有环的三种形态都抓到了**：整条链成环（循环走 3 轮相遇于 4 号、环长 3）、单节点自环（1 轮、环长 1）、尾指回中间（2 轮、环长 2）；
- **环长**是相遇之后从相遇点绕一圈数出来的（第 67 到 69 行），**它与 `rounds` 不是一回事**：前者是环上有几个节点，后者是循环走了几轮；
- **两个对照**：无环的表只走了 **2 轮**就结束——**不是慢指针走到 `NULL`，而是快指针先撞线**（那时 `fast` 停在最后一个节点上、它的 `next` 是 `NULL`，慢指针还停在 3 号节点）；用老办法 `p != NULL` 走无环的表是 **5 步**、走有环的表是 **1000 步**（撞上限）——**这就是「普通遍历在有环的表上会挂住」的实测样子。**

有环的表还有第二个麻烦：释放它。第 13.6 节那个 `list_free` 顺着 `next` 走，在环上会**第二次、第三次 `free` 同一块内存。**这不是推演——把那个循环原样搬出来喂一条 3 节点环（探针在 `examples/ch05/list_free/cycle_free.c`），本机实测是这样：

终端会话 有环的表交给 list_free：堆检查当场终止进程 ch05-cycle-free

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g cycle_free.c -o cycle.exe
$ powershell -NoProfile -Command "./cycle.exe; 'exit=' + $LASTEXITCODE"
3 个节点接成环，交给 list_free 那个循环：
exit=-1073740940
$ powershell -NoProfile -Command "./cycle.exe; 'exit=' + $LASTEXITCODE"
3 个节点接成环，交给 list_free 那个循环：
exit=-1073740940
$ rm -f cycle.exe
```

`exit=-1073740940` 是 **0xC0000374（堆损坏）** 的有符号写法：程序打印完 `fflush` 过的那一行就被终止了——**它有退出码，不是挂起。**它比「卡死」好查一点，但也就到此为止：堆检查只告诉你「有人重复释放了」，不会告诉你环在哪。

⚠ 陷阱 · 有环的表让「释放整条链」变成重复 free

你会怎么被坑到：上面那段会话说明得很清楚——`while (head != NULL) { next = head->next; free(head); head = next; }` 这个写法在**无环**的表上是对的（第 13.6 节整节都在用它），一旦表里有环，它就会绕回来把已经还回去的节点再 `free` 一次。你会以为「释放循环是标准写法，不会有事」，而真相是**标准写法的前提是「链一定有界」。**

两条纪律：

- **造链的时候只允许一头是「开放」的**：新节点只往头或尾接，绝不把已有的 `next` 指回链上的老节点（本例里那个 `chain_close` 是**故意**造环用的，名字就要起得让人一眼看出它在干什么）；
- **释放之前先判环**（或者约定「这条链一定有界」）。真要在有环的表上释放，得像本例的 `chain_free_n` 那样**只释放链上的有限个节点**，绝不能写 `while (head != NULL)`。

13.11 用链表实现栈和队列

栈和队列是链表最经典的用途：增删**只发生在两头**，正好是链表最便宜的地方（13.8 节的「前提一」）。`examples/ch05/stack_list/` 里同时有数组栈、链表栈、链表队列和一份对照驱动。先看链表栈：

c examples/ch05/stack_list/src/stack_list.c 第 66–75 行

```c
int lstack_push(ListStack *s, int value) {
    StackNode *p = snode_new(value);
    if (p == NULL) {
        return -1;
    }
    p->next = s->top;                       /* 头插：新节点成为新的栈顶 */ 头插：新节点接在栈顶上
    s->top = p; 栈顶挪到新节点
    s->size++;
    return 0;
}
```

71 头插：新节点接在栈顶上
72 栈顶挪到新节点

c examples/ch05/stack_list/src/stack_list.c 第 77–87 行

```c
int lstack_pop(ListStack *s, int *out) {
    if (s->top == NULL) {
        return -1;                          /* 空了 */
    }
    StackNode *dead = s->top; 要弹出去的那个
    *out = dead->value;
    s->top = dead->next;                    /* 摘链在 free 之前 */ 先改栈顶，再释放
    free(dead); 现在没人指着它了
    s->size--;
    return 0;
}
```

81 要弹出去的那个
83 先改栈顶，再释放
84 现在没人指着它了

出栈的三句和 13.5 节的删除是同一个套路：**先摘链（改 `top`），再 `free`**。队列多一个 `rear`：

c examples/ch05/stack_list/src/stack_list.c 第 110–124 行

```c
int lqueue_push(ListQueue *q, int value) {
    StackNode *p = snode_new(value);
    if (p == NULL) {
        return -1;
    }
    if (q->rear == NULL) {                  /* 空队列是特例：头尾都要指过来 */ 空队列是特例：头尾都要指过来
        q->front = p; 出队端也指过来
        q->rear = p;
    } else {
        q->rear->next = p;                  /* 接在队尾后面 */ 接在队尾后面
        q->rear = p;                        /* 尾指针挪过去 */
    }
    q->size++;
    return 0;
}
```

115 空队列是特例：头尾都要指过来
116 出队端也指过来
119 接在队尾后面

c examples/ch05/stack_list/src/stack_list.c 第 126–139 行

```c
int lqueue_pop(ListQueue *q, int *out) {
    if (q->front == NULL) {
        return -1;
```

```
129         }
130         StackNode *dead = q->front;
131         *out = dead->value;
132         q->front = dead->next;                  /* 队头往后挪一格 */ 队头往后挪一格
133         if (q->front == NULL) {
134             q->rear = NULL;                     /* 队列空了，尾指针也要清掉 */ 队列空了，尾指针也要清掉
135         }
136         free(dead);
137         q->size--;
138         return 0;
139     }
```

132 队头往后挪一格
134 队列空了，尾指针也要清掉

这段里第 115、134 行是两处**必须单独处理的空队列**：入队时队列是空的（`rear == NULL`），要把 `front` 与 `rear` 一起指到新节点；出队把最后一个元素拿走后，`rear` 必须跟着回到 `NULL`——不然它悬在一个已经 `free` 掉的节点上，下一次入队就会写到那块内存里。

四种东西跑同一套操作：

c examples/ch05/stack_list/src/main.c 第 17–27 行

```
17      printf("依次压入 1 2 3 4 5: \n");
18      int refused = 0;
19      for (int i = 1; i <= 5; i++) {
20          if (astack_push(&as, i) != 0) { 数组栈第 5 个压不进去
21              refused = i;
22          }
23          (void)lstack_push(&ls, i); 链表栈一直压
24      }
25      printf("  数组栈（容量 4）：第 %d 个压不进去（push 返回 -1）\n", refused); 把这件事打到屏幕上
26      printf("  链表栈：5 个全部压进去了\n");
27      printf("栈里现在有几个：数组栈 %zu，链表栈 %zu\n", astack_size(&as), lstack_size(&ls));
```

20 数组栈第 5 个压不进去
23 链表栈一直压
25 把这件事打到屏幕上

接着把两个栈弹空，再走一遍队列（先进先出）：

c examples/ch05/stack_list/src/main.c 第 29–49 行

```
29      printf("出栈顺序（后进先出）：\n  数组栈：");
30      int v = 0;
31      while (astack_pop(&as, &v) == 0) { 数组栈出栈
32          printf("%d ", v);
33      }
34      printf("（再弹一次返回 %d，栈空）\n", astack_pop(&as, &v)); 再弹一次会失败，说明真的空了
35      printf("  链表栈：");
36      while (lstack_pop(&ls, &v) == 0) { 链表栈出栈
37          printf("%d ", v);
38      }
39      printf("（再弹一次返回 %d，栈空）\n", lstack_pop(&ls, &v));
40
41      printf("队列（先进先出）：");
42      for (int i = 1; i <= 5; i++) {
43          (void)lqueue_push(&lq, i);
44      }
45      printf("入队 1 2 3 4 5，出队 ");
46      while (lqueue_pop(&lq, &v) == 0) { 队列出队
47          printf("%d ", v);
48      }
```

```
49      printf("\n");
```

```
31 数组栈出栈
34 再弹一次会失败，说明真的空了
36 链表栈出栈
46 队列出队
```

终端输出 终端输出 stack-list-out

```
依次压入 1 2 3 4 5:
  数组栈（容量 4）：第 5 个压不进去（push 返回 -1）
  链表栈：5 个全部压进去了
栈里现在有几个：数组栈 4，链表栈 5
出栈顺序（后进先出）：
  数组栈：4 3 2 1  （再弹一次返回 -1，栈空）
  链表栈：5 4 3 2 1  （再弹一次返回 -1，栈空）
队列（先进先出）：入队 1 2 3 4 5，出队 1 2 3 4 5
```

输出里最值得看的是**两个栈的分叉点**：同样压 5 个数，数组栈在第 5 个上返回 `-1`（满了），链表栈全部成功。这不是谁更好，而是两种「满」的定义不同：

问题	数组栈（`ArrayStack`）	链表栈（`ListStack`）
容量	开栈时定死（本例 4）	只要 `malloc` 成功就能长
压栈	写一个下标，`top++`	一次 `malloc` + 改两个指针
满了	`push` 返回 `-1`，调用者处理	不会满（只会 `malloc` 失败）
内存	一整块连续内存	一个节点一块，散在堆上
出栈	`top--` 再读（内存不还给系统）	摘链 + `free`（真还回去）

接口长得一样：两者都有 `push` / `pop` / `size`，`pop` 都是「返回 0 成功、-1 失败，真值从出参 `int *out` 拿回来」。这就是数据结构的「抽象」：调用者只关心「后进先出」，不关心底下是数组还是链表。

工程惯例 · 工程惯例：什么时候用数组栈、什么时候用链表栈

这是本书里第一次出现「同一个接口、两种实现」的选择题，工程上的判据是两条：

- **元素个数的上限是否可知**：知道上限（比如编译器里固定 256 层的括号嵌套）就用数组栈——它一次要够内存，换来的是每次操作只改一个下标，而且内存连续（第 13.8 节那个八十倍在这里也成立）；
- **上限未知、或者差别极大**（比如解析用户输入），就用链表栈或「会自己长」的动态数组。链表把「要多少给多少」这件事做到了极致，代价是每个元素多一个指针、外加分配器的管理开销；
- **队列要用链表，别忘了 `rear`**。数组做队列要么搬元素，要么用环形缓冲（「空」与「满」得单独判断）；链表队列头出尾进都是常数时间，代价是本节那两处空队列特例。

代价写清楚：**链表版每个元素都是一次 `malloc`**（本例判了返回值），元素一多，分散的内存会让遍历变慢（第 13.8 节实测的 80 倍）。

13.12 递归遍历与栈深度

第 13.3 节的两个遍历函数里，有一个是用递归写的。把它单独拿出来看一遍：

c examples/ch05/list_head_insert/src/list_head_insert.c 第 30–43 行

```
30  size_t list_count_iter(const Node *head) {
31      size_t n = 0;
32      for (const Node *p = head; p != NULL; p = p->next) {   循环：栈上永远只有这一个局部变量
33          n++;
34      }
35      return n;
36  }
```

```
37
38  size_t list_count_rec(const Node *head) {
39      if (head == NULL) { 递归出口
40          return 0;                           /* 递归出口：走到头了 */ 走到头了，返回 0
41      }
42      return 1 + list_count_rec(head->next);      /* 我算一个，剩下的交给下一层 */ 我算一个，剩下的交给下一层
43  }
```

32 循环：栈上永远只有这一个局部变量
39 递归出口
40 走到头了，返回 0
42 我算一个，剩下的交给下一层

两个函数做的事完全一样，写法差了十万八千里。递归为什么这么短？它把「剩下的部分」整个交给下一次调用，而**每次调用都要在栈上占一份空间**（返回地址、参数 `head` 等——第 7 单元讲调用栈时画过，这里是它在链表上的第二次出现）。代价可以量：

调用树 **递归数节点：list_count_rec 一层层往下走**

第 1 步 **main 把整条链交给 list_count_rec**

```
main() ：准备数一条 3 个节点的链
main() → list_count_rec(A) A 是头节点
```

第 2 步 **A 层算自己那 1 个，把「剩下的」交给下一层**

```
main() ：还在等结果
list_count_rec(A) ：我算 1 个
list_count_rec(A) → list_count_rec(B) 剩下的交给你
```

第 3 步 **一直走到 NULL，碰上递归出口**

```
list_count_rec(A) ：1 + ?
list_count_rec(B) ：1 + ?
list_count_rec(C) ：1 + ?
list_count_rec(NULL) ：返回 0，递归到这里停 ← 本步新增
list_count_rec(NULL) ⇒ list_count_rec(C) 返回 0
```

第 4 步 **一层层把数加回来，最后交还给 main**

```
list_count_rec(C) ⇒ list_count_rec(B) 返回 1
list_count_rec(B) ⇒ list_count_rec(A) 返回 2
list_count_rec(A) ⇒ main() 返回 3
```

第 3 帧是转折点：走到 `NULL` 时**栈上同时压着四层**（A、B、C 加停在 `main` 里的一层）。层数 = **链长 + 1**（那次 `NULL` 调用也要一层）——下一段实测要证明的就是它。

把链加长到 **1000000 个节点**，两种写法各跑一次（同一个 `main`，靠编译参数切换；两个清单分别写在 `examples/ch05/list_head_insert/` 下）：

终端输出 终端输出 list-head-deep-iter-out

```
头插建起了 1000000 个节点
迭代数出来 1000000 个
```

终端输出 终端输出 list-head-deep-rec-out

```
头插建起了 1000000 个节点
```

两次运行的差别只有一样：**遍历的方式。**

实验记录 · 本机实测：一百万个节点的递归遍历会耗尽栈

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 迭代版：`flags` 里加 `-DDEEP_N=1000000`，清单 `list_head_insert_deep_iter.json`，`mode` 是 `run`——**退出码 0**，打印出「头插建起了 1000000 个节点」与「迭代数出来 1000000 个」（上面第一个输出块）；
- 递归版：同一个 `-DDEEP_N=1000000` 再加 `-DUSE_RECURSION`，清单 `list_head_insert_deep_rec.json`，`mode` 是 `runtime_error`——**退出码 3221225725**（0xC00000FD，Windows 上「栈溢出」的那个码），程序在递归往下走的某一步撞到了栈底（我没有用 gdb 去数它停在第几层，只能说它死在递归链的某一层上）；
- 递归版的 stdout 只有一行「头插建起了 1000000 个节点」：程序里那句 `fflush(stdout)` 是刻意的——**不刷的话这一行也会丢**（stdout 进管道是全缓冲，进程被终止时缓冲区来不及送出去；第 14 单元会专门讲缓冲，这里只要记住：崩溃前想留下输出，就得显式 `fflush`）；
- 为什么是 3221225725 而不是 3221225477：**栈溢出是操作系统在进程碰到栈底那一刻直接终止它**；访问冲突是访问了不属于自己的地址。两类错，两个码。

这份证据里最要紧的一条是**对照**：同一台机器、同一份 1000000 个节点、同样的编译参数，把递归换成迭代就从「栈溢出」变成「退出码 0」。**递归的代价不是代码长短，是栈的深度。**

{} 语法规则 · 什么时候把递归换成迭代

判据只有一条：**递归深度会不会大到你控制不住。**

- 链表的遍历、求长度、释放——**深度就是链长**，一条 100 万个节点的链就能把默认栈用光（本机实测：3221225725）。这类「顺着链走」的递归，一律改成迭代（一个 `while` 就够）；
- 树的遍历——深度是**树高**。树高通常远小于节点数（一棵平衡的 100 万节点二叉树高度约 20），所以递归写起来更短更清楚（下一节就是这个例子）；
- 真要深递归又改不成迭代，就得显式要更大的栈（平台相关参数），属于第 7 章的工具话题。

一句话：**深度与数据规模同阶的递归要小心，深度与规模的对数同阶的递归放心用。**

13.13 二叉树前序遍历：递归的自然归宿

链表是「一个节点最多一个后继」，把它放宽到**两个**，就是二叉树。节点定义与链表节点是同一个形状，只是多一个指针。遍历它最自然的写法就是递归，`examples/ch05/tree_preorder/src/tree_preorder.c` 里三个函数长得几乎一样：

c examples/ch05/tree_preorder/src/tree_preorder.c 第 18–27 行

```c
/* 三个函数长得一模一样，只有「先做什么」的顺序不同。
   这一行 printf 的位置，就是「根」在被访问的顺序里的位置。 */
void preorder(const TreeNode *root) {
    if (root == NULL) { 递归出口：空节点什么都不做
        return;                         /* 递归出口：空节点什么都不做 */
    }
    printf("%d ", root->value);         /* ① 先管自己 */ 先管自己
    preorder(root->left);               /* ② 再管左子树 */ 再管左子树
    preorder(root->right);              /* ③ 最后管右子树 */ 最后管右子树
}
```

21 递归出口：空节点什么都不做
24 先管自己
25 再管左子树
26 最后管右子树

另外两个函数只把 `printf` 挪了个位置，其余一个字没变：

c examples/ch05/tree_preorder/src/tree_preorder.c 第 29–45 行

```c
void inorder(const TreeNode *root) {
    if (root == NULL) {
        return;
```

```
    }
    inorder(root->left);
    printf("%d ", root->value);  中序：printf 夹在两个递归调用中间
    inorder(root->right);
}

void postorder(const TreeNode *root) {
    if (root == NULL) {
        return;
    }
    postorder(root->left);
    postorder(root->right);  后序：printf 放到最后
    printf("%d ", root->value);
}
```

34 中序：printf 夹在两个递归调用中间
43 后序：printf 放到最后

三行 `printf` 的位置，就是三种遍历的全部区别：`preorder` 先打印自己（前序）、`inorder` 夹在两个递归调用中间（中序）、`postorder` 放到最后（后序）。递归写法的好处在这里体现得最清楚——你不用管「怎么走到下一个节点」，只要写清「先做什么」。

调用树 **前序遍历展开：先自己、再左、最后右**

第 1 步 从根开始，第一个动作是打印自己

```
preorder(1) ：打印 1
```

第 2 步 打印完自己，去看左孩子

```
preorder(1) ：已经打印过 1
preorder(1) → preorder(2) 左孩子是 2
```

第 3 步 2 也是先打印自己，再看它的左孩子

```
preorder(2) ：打印 2
preorder(4) ：打印 4
preorder(2) → preorder(4) 左孩子是 4
```

第 4 步 4 的两个孩子都是 NULL，回到 2 走右边

```
preorder(4) ：左右都是 NULL，返回
preorder(5) ：打印 5
preorder(2) → preorder(5) 右孩子是 5
```

第 5 步 左子树全部走完，回到根再走右边

```
preorder(3) ：打印 3
preorder(1) → preorder(3) 右孩子是 3
```

第 1 帧到第 5 帧连起来，顺序就是 `1 2 4 5 3`——**5 个节点要 5 帧**，因为前序遍历每到一处都要「先办事、再往下」。跑出来是这样：

c examples/ch05/tree_preorder/src/main.c 第 13–34 行

```
int main(void) {
    TreeNode *t4 = tnode(4, NULL, NULL);  先建叶子
    TreeNode *t5 = tnode(5, NULL, NULL);
    TreeNode *t2 = tnode(2, t4, t5);
    TreeNode *t3 = tnode(3, NULL, NULL);
    TreeNode *root = tnode(1, t2, t3);  最后建根，一棵树就出来了

    printf("这棵树：1 是根，2 和 3 是它的两个孩子，4 和 5 挂在 2 下面\n");
    printf("节点个数 = %zu，高度 = %d\n", tree_count(root), tree_height(root));  节点数与高度

    printf("前序（根 左 右）：");
    preorder(root);
    printf("\n");
    printf("中序（左 根 右）：");
    inorder(root);
```

```
28      printf("\n");
29      printf("后序（左 右 根）：");
30      postorder(root);
31      printf("\n");
32
33      root = tree_free(root); 释放之后返回 NULL，理由见下面
34      printf("释放之后 root == NULL 是 %s\n", root == NULL ? "真" : "假");
```

14 先建叶子
18 最后建根，一棵树就出来了
21 节点数与高度
33 释放之后返回 NULL，理由见下面

终端输出 终端输出 tree-preorder-out

```
这棵树：1 是根，2 和 3 是它的两个孩子，4 和 5 挂在 2 下面
节点个数 = 5，高度 = 3
前序（根 左 右）：1 2 4 5 3
中序（左 根 右）：4 2 5 1 3
后序（左 右 根）：4 5 2 3 1
释放之后 root == NULL 是 真
```

最后一行 `释放之后 root == NULL 是 真` 是刻意设计成这样的（`tree_free` 在第 64 到 72 行）：释放一棵树必须用**后序**——先把左右两个孩子释放掉，再释放自己。前序释放会怎样？`free(root)` 之后你还得去访问 `root->left`，而那已经是还回去的内存了（第 13.7 节那个坑）。所以 `tree_free` 的两个递归调用必须写在 `free` 前面，而且它返回 `NULL`，让调用者能 `root = tree_free(root);` 一次性把把手也清掉。

> ⓘ 注 · **和算法分册的关系**
> 这一节只讲**把树建起来、递归走完、干净拆掉**。二叉搜索树的插入查找、平衡、按层遍历（要用队列）都归《算法》分册；`tree_preorder` 是那边的第一块砖：节点定义、递归写法、后序释放都已定好。

13.14 这一节新增了什么，以及接下来

回到 13.1 那棵目录树，这一节加进去的东西可以逐个点名了：

目录树 🗀 **这一节新增的文件，逐个点名**

第 1 步 先摆上「怎么串起来」的三个例子目录

```
examples/
└── ch05/
    ├── list_head_insert/ ：头插、两种遍历、自引用反面教材 ← 本课新增
    ├── list_append/ ：尾插与尾指针、步数对照 ← 本课新增
    └── stack_list/ ：链表栈、链表队列、数组栈 ← 本课新增
```

第 2 步 再加上「怎么拆下来」的三个

```
examples/
└── ch05/
    ├── 前面三个例子 ：同上，不再重复列
    ├── list_delete/ ：六个边界、哨兵版、成本对照 ← 本课新增
    ├── list_free/ ：释放整条链并置空 head ← 本课新增
    └── list_bad_order/ ：先 free 再取 next ← 本课新增
```

第 3 步 最后两个例子，以及五段抓下来的会话

```
examples/
└── ch05/
    ├── 前面六个例子 ：同上
    ├── circular_detect/ ：快慢指针判环 ← 本课新增
    ├── tree_preorder/ ：二叉树三种遍历 ← 本课新增
    ├── session-gdb-list.txt ：gdb 顺着 next 走 ← 本课新增
    ├── session-analyzer-list.txt ：分析器报空指针 ← 本课新增
    ├── session-warning-list.txt ：编译器的 use-after-free 警告 ← 本课新增
    ├── session-cost-list.txt ：数组与链表的耗时对照 ← 本课新增
    └── session-cycle-free.txt ：有环的表释放会重复 free ← 本课新增
```

第 1 到 3 帧那八个目录里，**每个都配了清单**（`.json`）：怎么编译、该输出什么、退出码是多少都写在里面，所以这一章里每块输出、每个退出码都是流水线真跑出来再核对过的。第 3 帧那五段会话是**证据的另一种形式**：gdb 带交互、分析器诊断太长、耗时每次都不同、编译警告不属于 `compile_fail`（它编译成功了）、重复 free 那份的重点是退出码——它们进不了逐字对账的 `output` 块，所以先抓成文件再整段注入（对的是会话文件）。

小结 · 这一节的要点

- **自引用结构体只能放自己的指针**：`struct Node *next` 是 8 个字节的地址，`struct Node next` 是「不完整的类型」，编译器当场驳回（本机实测：`field 'next' has incomplete type`）；一个节点 16 字节、一个指针 8 字节，都是 `sizeof` 打出来的。**建链与拆链，改的都是指针**：头插改 2 处、尾插（有 `tail`）改 2 处、删除改 2 处。**无环链上遍历的条件是 `p != NULL`**——有环时这个条件永远为真，所以判环要用快慢指针（`fast != NULL && fast->next != NULL`）；
- **删除与释放的顺序都是「先取 `next`，再 `free`」**——顺序反了，本机实测的结果是「退出码 0、三次都读回原来的值」，也就是说**跑起来看不出错**（编译器的 `-Wuse-after-free` 才是可靠信号）。带哨兵的表用**一个节点换掉了「删头」那个分支**（代价：遍历得从 `head->next` 开始、空表判断变成 `head->next == NULL`）。**所有权用类型表达**：`list_free(Node **head)` 收地址，是为了释放完把调用者的 `head` 置空，于是它**能安全地调两次**（标准 7.22.3.3：`free(NULL)` 什么都不做）；而两个指向同一节点的指针各 `free` 一次，标准说那是未定义行为——**有环的表更糟**，同一个循环会把已释放的节点再 `free` 一遍（本机实测：退出码 -1073740940，堆损坏）；
- **取舍要看代价**：数组删头搬移 99999 个元素、链表搬移 0 个；数组 `a[99998]` 一句读出来（没有循环）、链表走 99998 步；100 万个整数求和，数组 0.13 毫秒/遍、链表 10.7 到 11.9 毫秒/遍（差八十倍上下）。**链表赢在「不用搬」，输在「不能跳」**；
- **证据分四种**：真跑的 stdout（逐字对账）、编译器的警告（`-Wuse-after-free`）、静态分析（`-fanalyzer` 的 `-Wanalyzer-null-dereference`，编译退出码仍是 0）、以及运行期的终止（深递归栈溢出 **3221225725**）。**这四样要分清楚，别把「编译器看到了」说成「程序崩了」**。

自测题 · 自测题（先自己答，再对答案）

1. `typedef struct BadNode { int value; struct BadNode next; } BadNode;` 为什么编不过？gcc 给出的诊断里，关键的那半句是哪个词？
2. 依次头插 1、2、3 之后打印，顺序是什么？为什么？如果调用时忘了写 `head = list_push_front(head, 1);`，会发生什么？
3. 删除一个节点时 `Node *next = cur->next;` 为什么必须在 `free(cur)` 之前执行？「先 `free` 再取 `next`」在本机实测的结果是什么（退出码、输出）？
4. 同一份 `list_free(&head)` 连续调两次是安全的，而「两个指针指向同一个节点、各 `free` 一次」是未定义行为。这两件事的差别在哪一句话上？
5. 一条 1000000 个节点的链，用递归数节点与用迭代数节点，本机实测的退出码各是多少？换成一棵 1000000 个节点的平衡二叉树，同样用递归，为什么通常又没问题了？

注 · 自测题答案

1. 因为 `struct BadNode` 在第 7 行还没读完，编译器只知道名字、不知道大小，而成员必须有完整的类型；诊断里的关键词是 `incomplete type`（原文：`error: field 'next' has incomplete type`）。改成 `struct BadNode *next;` 就可以——指针的大小是固定的 8 个字节（本机实测），跟它指向什么无关。
2. 打印出来是 `3 -> 2 -> 1`。头插的定义就是「每个新节点都成为新的头」，所以最后插的排最前。忘接返回值的话，`head` 还是老的头指针，新节点**还在堆上但没有任何人指着它**——链上少了它，而且它永远不会被释放（内存泄漏）。
3. 因为 `cur->next` 住在那块内存里，`free(cur)` 之后那块内存就不属于你了，再去读它是未定义行为。本机实测：退出码 **0**，三次循环每次都打印「从同一格读到的 next 与 free 之前一样」——**它没崩，这才是最坏的结果**。编译器会在编译期给出两条 `-Wuse-after-free` 警告（`pointer 'p' used after 'free'`），那才是可靠的信号。
4. 差别在标准 7.22.3.3 第 2 段那句 `If ptr is a null pointer, no action occurs`：第一次 `list_free` 结尾执行了 `*head = NULL;`，所以第二次传进去的是空指针，标准说「什么都不做」；而两个别名指针各 `free` 一次时，第二次传进去的是**非空指针**、且那块空间**已经释放过**，正好撞上同一段的 `behavior is undefined`。

5. 递归版退出码 **3221225725**（0xC00000FD，栈溢出），迭代版退出码 **0**。因为链表的递归深度**就是链长**（100 万层，每层都要占一份栈帧，本机默认栈装不下），而平衡二叉树的递归深度是**树高**——100 万个节点的平衡二叉树高度约 20 层，栈上多压 20 层调用完全不是事。

第 6 章

标准库与文件

本章 2 个单元 · 页眉从这里之后的页面开始出现

1. 文件与 I/O：把结果留下来 零基础 · 第 14 单元 · 建议 90 分钟
2. 标准库工具箱：字符串、解析、排序与数学 零基础 · 第 15 单元 · 建议 90 分钟

文件与 I/O：把结果留下来

◎ 学习目标 · 看完这一单元，你应该能

- 说清「流」是什么，指出 stdin / stdout / stderr 各归谁用、为什么错误信息要走 stderr；
- 用 `fopen` 的六种模式打开文件、判掉 `NULL`、用 `fclose` 收尾，并说出本机文本模式与二进制模式差在哪几个字节上；
- 用 `fgetc` / `fgets` / `fscanf` 读文件：解释 `fgets` 为什么留下换行、太长时怎么截断、怎么把一行完整读回来；
- 说出 `while (!feof(fp))` 为什么把最后一行读两遍，并解释缓冲区、`fflush`、`fclose` 与「崩溃前输出丢了」的关系；
- 用 `fseek` / `ftell` / `rewind` 量大小，用 `argc` / `argv` 写一个小工具，并说清退出码与重定向的约定。

6.1 先看工程长什么样

前面十三个单元里，程序说的话都只说给屏幕听：`printf` 打完，程序一退，那些字就没了，想再看一眼只能重跑。这一单元解决的是「**留下来**」：把结果写进文件，把输入从文件里读回来，把出错的说明送到该去的地方。

还是老规矩，先看**文件**。本单元是第 6 章的第一单元，也是全书的第 14 单元，要在 `examples/` 下建起第六个项目。下面第 1 帧是这一节故事的起点：`examples/` 下已经有前五个项目，还没有 `ch06`。

目录树 **第六个项目 examples/ch06/：从「没有 ch06」到十个例子目录加数据文件**

第 1 步 本节开始时，examples/ 下已经有五个项目

```
examples/
├── ch01/ ：第一个项目，单文件（本书唯一的例外）
├── ch02/ … ch05/ ：第二到第五个项目
└── ch06/ ：还不存在，本节要建
```

第 2 步 建起第六个项目，先摆上「写」与「读」

```
examples/
├── ch01/ … ch05/ ：前五个项目
└── ch06/ ← 本课新增
    ├── file_write_read/ ：写一个文件再读回来 ← 本课新增
    ├── newline_bytes/ ：文本与二进制差几个字节 ← 本课新增
    ├── fgets_long_line/ ：一行太长会被剁断 ← 本课新增
    └── feof_trap/ ：while 先问 feof 的坑 ← 本课新增
```

第 3 步 再加崩溃与命令行与两种存法

```
examples/
├── ch01/ … ch05/ ：前五个项目
└── ch06/
    ├── fflush_crash/ ：崩溃之前 stdout 里有什么 ← 本课新增
    ├── fflush_kept/ ：显式 fflush 之后再崩 ← 本课新增
    ├── mycat/ ：argc 与 argv 与退出码 ← 本课新增
    └── record_text_vs_bin/ ：一份记录的两种存法 ← 本课新增
```

第 4 步 最后两个例子，十个到齐

```
examples/
├── ch01/ … ch05/ ：前五个项目
└── ch06/
    ├── bad_path/ ：打不开文件怎么报告 ← 本课新增
    └── handle_not_string/ ：把句柄当字符串（编不过） ← 本课新增
```

第 5 步 本单元第一次出现数据文件

```
examples/
└── ch06/
    ├── fgets_long_line/data/longline.txt : 一行超长 ← 本课新增
    ├── feof_trap/data/three_lines.txt : 正好三行 ← 本课新增
    ├── mycat/data/two_lines.txt : 两行 ← 本课新增
    └── record_text_vs_bin/data/people.csv : 三条记录 ← 本课新增
```

第 6 步 外加四个抓下来的终端会话

```
examples/
└── ch06/
    ├── session-mycat.txt : 用法与退出码与重定向 ← 本课新增
    ├── session-newline-bytes.txt : od 与 wc 与 python ← 本课新增
    ├── session-bad-path.txt : perror 与 2> ← 本课新增
    └── session-record-sizes.txt : ftell 与 wc 对照 ← 本课新增
```

第 2 到第 4 帧是本节搭起来的 `examples/ch06/` ——**十个**例子目录，每个都是「源文件放 `src/` 、声明放 `include/` 、编译与对账的参数写在 `.json` 清单里」的最小工程（第 1 单元 1.7 节讲过分文件的理由：从第二个项目起，本书不允许单文件工程）。其中九个是能跑的程序，第十个 `handle_not_string` 是**故意编不过**的：它考的正是这一单元最容易犯的错——把文件句柄当成文件内容。

第 5、6 帧是本单元新出现的两类东西：

- **数据文件**：`fgets_long_line/data/longline.txt` 这类**输入**，程序读的就是它，它不属于任何源文件，只是一份文本；这是本书第一次出现 `data/` 目录；
- **抓下来的终端会话**：`session-*.txt` 四个文件，里面的每一行都来自真实终端，正文里用 `term` 块注入，与文件逐字对账（1.3 节的 `session-savetemps.txt` 就是这个形状）。

工程惯例 · **工程惯例：数据文件为什么不能叫 .json**

本书的流水线把 `books/*/examples/**/*.json` **每一个** `.json` 都当成清单去校验（清单要写 `id` 、 `sources` 、 `mode` 、 `unit` 这些字段）。所以 `examples/` 下**不放非清单的** `.json` ：要配数据文件就用 `.txt` / `.csv` / `.tsv` ，它们不会被误当成清单。

这不是标准的规定，是本项目的纪律，理由很实在：**机器分不清「这是数据」还是「这是配置」**。名字分开了，工具链就不用猜。代价是数据文件不能享受 JSON 的语法检查——你自己写错了格式（比如 CSV 少一列），没有任何东西会替你报错，只能靠程序里的返回值与断言。

大白话 · **把「文件」说成人话**

屏幕和键盘是**一次性的**：程序一退出，屏幕上打过的字就跟没打过一样。文件是**留得住的**：它是一串字节，躺在磁盘上，你不去改它它就不变，明天、换一台机器打开它，还是同一串字节。

所以「做 I/O」这件事，本质上是**在程序和磁盘之间搬字节**。这一单元里所有的名字——流、缓冲、模式、定位——都是在回答搬的**过程**里冒出来的问题：搬多少、什么时候搬、搬错了怎么知道。

6.2 流：程序与外界之间的那根管子

`printf` 往哪儿写？ `fopen` 打开的是「文件」还是别的什么东西？C 给出的答案只有一句话：**你操作的不是文件，是流（stream）**。

流是一条**字节进出的通道**。你可以往它里写字节、从它里读字节，可以问它「读到头了吗」「出错了吗」，可以关掉它。至于通道的另一头是磁盘上的一个文件、是键盘、是屏幕、还是另一个程序，**流的接口不管**——这就是为什么同一个 `fprintf` 既能往屏幕写，也能往文件写。

程序一启动，这三个流已经替你开好了，不用 `fopen` ：

流	谁在用它	通常连到哪
`stdin`	输入	键盘，或者重定向进来的文件

流	谁在用它	通常连到哪
`stdout`	正常输出	屏幕，或者重定向出去的文件
`stderr`	出错的说明	屏幕，**默认只连屏幕，不跟 `>` 走**

标准里对这三个流的地位说得很直接：

标准原文 · 标准原文：程序一启动就有三个流

> At program startup, three text streams are predefined and need not be opened explicitly — standard input (for reading conventional input), standard output (for writing conventional output), and standard error (for writing diagnostic output). As initially opened, the standard error stream is not fully buffered; the standard input and standard output streams are fully buffered if and only if the stream can be determined not to refer to an interactive device.

大意：程序启动时就有三个**文本流**预定义好了，不用显式打开——标准输入（读常规输入）、标准输出（写常规输出）、标准错误（写**诊断**输出）。而一开始，标准错误流**不是全缓冲的**；标准输入与标准输出**只有当它们可以确定不是交互设备时才全缓冲。**

这一句里藏着本单元两个大问题的答案：为什么错误信息要走 `stderr`、为什么崩溃前的 `printf` 会丢。出处：ISO/IEC 9899:2018（C17）§7.21.3。

「流是一个参数」这件事，看一眼真代码最清楚。下面这个函数来自 `examples/ch06/file_write_read/`，它的第三个参数就是一个**要写到哪儿的流**：

c examples/ch06/file_write_read/src/file_write_read.c 第 23–42 行

```c
int read_notes(const char *path, FILE *out) {
    FILE *fp = fopen(path, "r");              /* "r": 读，文件必须已经存在 */ r 只读，文件必须已存在
    if (fp == NULL) {
        perror(path);
        return -1;
    }
    char line[128];
    int count = 0;
    while (fgets(line, sizeof line, fp) != NULL) {  /* 拿返回值当循环条件 */ 拿返回值当循环条件
        count++;
        fprintf(out, "%d| %s", count, line);        /* fgets 连换行一起读进来了 */ 写给 out，不写死 stdout
    }
    if (ferror(fp)) {                         /* 循环结束要分清：读完了，还是出错了 */
        perror(path);
        fclose(fp);
        return -1;
    }
    fclose(fp);
    return count;
}
```

24 r 只读，文件必须已存在
31 拿返回值当循环条件
33 写给 out，不写死 stdout

第 29 行的 `fprintf(out, …)` 是这一节的钥匙：内容写给谁，由 `out` 决定。调用它的时候传 `stdout`，内容就上屏幕；传一个用 `fopen` 打开的 `FILE *`，内容就进文件；传 `stderr`，内容就走错误那条管子。同一个函数，三种去处，一行代码都不用改。

工程惯例 · 工程惯例：为什么错误信息一定要走 stderr

标准只说 `stderr` 是「写诊断输出」的流，**没有**规定你必须把错误写在那里。但主流做法一致：**正常结果走 `stdout`，出错的说明走 `stderr`**。理由是为了让重定向干净：

- 你敲 `mycat data/two_lines.txt > out.txt`，只想要「文件的内容」进 `out.txt`；如果错误信息也在 `stdout` 里，它会一起被写进结果文件，把数据弄脏；
- 而错误信息留在屏幕上（`stderr` 默认不跟 `>` 走），你能当场看见「哦，第三个文件打不开」。

代价是：**看输出顺序会骗人。**两个流各走各的，谁先到屏幕取决于缓冲（这一节 6.7 会用一个真实会话解释为什么）。所以调试时更稳的做法是把两个流一起抓下来：`./app > out.txt 2>&1`（`2>&1` 的意思是「让 2 号流也走到 1 号流去」）。

6.3 打开与关闭：模式表，以及那个必须判的 NULL

除了那三个预定义流，其他流都要自己开：`fopen` 打开，`fclose` 关掉。`fopen` 的第一个参数是文件名，第二个参数是**模式字符串**：

模式	含义（标准原文的措辞）	文件不存在时	已有的内容
`"r"`	打开文本文件**读**	失败，返回 `NULL`	保留
`"w"`	截断到 0 字节，或创建文本文件**写**	创建	**丢掉**
`"a"`	追加：打开或创建文本文件，在**文件末尾**写	创建	保留，写在后面
`"r+"`	打开文本文件**读写**	失败	保留
`"rb"`	打开**二进制**文件读	失败	保留
`"wb"`	截断到 0 字节，或创建**二进制**文件写	创建	**丢掉**

标准对返回值的说法只有一句，但它是整个单元最重要的一句：

标准原文 · 标准原文：fopen 失败返回什么

> The fopen function opens the file whose name is the string pointed to by filename, and associates a stream with it. … If the file cannot be opened, the fopen function returns a null pointer.

大意：`fopen` 把名字与一个流关联起来；**打不开就返回空指针。**——返回空指针这件事，是它唯一告诉你「失败」的方式，所以**每次 `fopen` 都要判。**

出处：ISO/IEC 9899:2018（C17）§7.21.5.3。

关于 `fclose`，标准里有一个短语值得单独记住：

标准原文 · 标准原文：fclose 会做什么

> The fclose function causes the stream pointed to by stream to be flushed and the associated file to be closed. Any unwritten buffered data for the stream are delivered to the host environment to be written to the file; any unread buffered data are discarded.

大意：`fclose` 做两件事——**先把流冲干净（flush），再关掉文件。**还没写出去的缓冲数据会被交给宿主环境去写进文件；还没读的缓冲数据直接丢掉。

「先冲再关」这半句是 6.7 节的伏笔：**如果你在程序正常结束之前就离开了（崩溃、被强杀、`abort`），那没 `fclose` 就等于没冲。**反过来要记住：程序从 `main` 返回或调用 `exit` 时，运行库会替你冲掉所有还开着的流——所以「忘了 `fclose`」本身不会让文件变空，**「忘了 `fclose` 又崩了」才会**（6.8 有一个专门拆这个误判的 trap）。

出处：ISO/IEC 9899:2018（C17）§7.21.5.2。

下面这个真实的失败路径来自 `examples/ch06/bad_path/`。它要打开的文件**故意不存在**：

c examples/ch06/bad_path/src/bad_path.c 第 7–16 行

```c
int open_or_report(const char *path) {
    FILE *fp = fopen(path, "r");
    if (fp == NULL) {                       /* 这就是失败路径：必须处理，不能接着用 fp */ 失败路径，必须处理
        perror(path);                       /* 打到 stderr：前缀 + 系统给的原因 */ perror 打到 stderr
        fprintf(stderr, "  出错时的 errno = %d (%s) \n", errno, strerror(errno)); errno 与它的文字说明
        return -1;
    }
    fclose(fp);
    return 0;
}
```

9 失败路径，必须处理
10 perror 打到 stderr
11 errno 与它的文字说明

`perror` 是标准函数，它的行为标准写得很细：先把参数当**前缀**打出来，跟一个冒号和空格，再跟一句系统给的错误说明，最后换行；**并且这一串是打到 stderr 去的**。然后是 `main`：

c examples/ch06/bad_path/src/main.c 第 5–13 行

```c
int main(void) {
    printf("试着打开一个不存在的文件……\n");
    if (open_or_report("data/there_is_no_such_file.txt") != 0) { 故意打开不存在的文件
        printf("失败了。上面那些错误信息在 stderr 上，printf 这里看不见它\n");
        return 1;                           /* 非 0 退出码 = 这次运行不成功 */ 非 0 退出码等于「这次不成功」
    }
    printf("居然打开了，这就怪了\n");
    return 0;
}
```

7 故意打开不存在的文件
9 非 0 退出码等于「这次不成功」

真跑一遍，用 `2>` 把错误那条管子单独接到文件里，就能看清「两条管子各行其道」：

终端会话 失败路径的终端实况：stdout 与 stderr 各走各的 ch06-bad-path-term

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/bad_path.c -o bad_path.exe
$ ./bad_path.exe 2> err.txt
试着打开一个不存在的文件……
失败了。上面那些错误信息在 stderr 上，printf 这里看不见它
$ echo $?
1
$ cat err.txt
data/there_is_no_such_file.txt: No such file or directory
  出错时的 errno = 2 (No such file or directory)
```

这一次运行的 **stdout**（逐字对账过的）只有两行，它自己就在说「真正的原因不在这里」：

终端输出 终端输出 bad-path-out

```
试着打开一个不存在的文件……
失败了。上面那些错误信息在 stderr 上，printf 这里看不见它
```

两处细节值得停一下：

- 屏幕上**只有两行**：错误那两行被 `2>` 拐进了 `err.txt`，`cat` 出来才看见；
- `echo $?` 是 **1**：`main` 里那句 `return 1` 就是它。

终端里 `$?` 看到的那个数叫**退出码**（exit status），也是 `main` 的返回值。C 标准规定它的含义只有「0 表示成功、`EXIT_FAILURE` 表示不成功」这两种是**定义了**的：

标准原文 · **标准原文：退出码只有 0 与 EXIT_FAILURE 是标准钉死的**

> If the value of status is zero or EXIT_SUCCESS, an implementation-defined form of the status successful termination is returned. If the value of status is EXIT_FAILURE, an implementation-defined form of the status unsuccessful termination is returned. Otherwise the status returned is implementation-defined.

大意：返回 0 或 `EXIT_SUCCESS` 表示成功，返回 `EXIT_FAILURE` 表示不成功，**其余任何数值的含义由实现自己定。**

所以 `return 1` / `return 2` 这些「非 0 数值」是**惯例**（大家都拿它当「失败 + 失败原因编号」用），不是标准保证的。这条惯例的完整玩法写在 6.11 的命令行一节。

出处：ISO/IEC 9899:2018（C17）§5.1.2.2.3。

6.4 文本模式与二进制模式：同一个 "\n"，两个长度

`"w"` 与 `"wb"` 只差一个字母 `b`，写出去的东西却**不一样长。**

读这个例子的时候，先注意它的**形状**，而不是结论：同一个函数、同一份内容、同一句 `fputs`，唯一的变量是传进来的 `mode`。这就是对照实验——两个文件之间任何一点差别，都只能归到那个 `b` 上。你自己以后怀疑「某个操作改了字节」时，也照这个形状写：**只改一个变量，其余全部相同。**

下面这个程序写进去的内容完全一样——`'a'`、换行、`'b'`、换行，四个字符——只是一个用 `"w"` 写，一个用 `"wb"` 写。写完之后它还会把两个文件都重新打开、一个字节一个字节地数一遍，再把每个字节按十六进制打出来。**先看它怎么开文件，再看数字**：

c examples/ch06/newline_bytes/src/newline_bytes.c 第 3–19 行

```c
int write_two_lines(const char *path, const char *mode) {
    FILE *fp = fopen(path, mode);          /* mode 由调用者给 */
    if (fp == NULL) {
        perror(path);
        return -1;
    }
    if (fputs("a\nb\n", fp) == EOF) {      /* 同一份内容，两个字节的 'a' 是同一个 */
        perror("fputs");
        fclose(fp);
        return -1;
    }
    if (fclose(fp) != 0) {
        perror("fclose");
        return -1;
    }
    return 0;
}
```

4 mode 由调用者给
9 同一份内容，写两遍

写完之后要「数一数磁盘上真有几个字节」，这一步的关键是**用 `"rb"` 重新打开**（同一个文件，换个模式看）。这里有个容易忽略的细节：数出来的**不是**「文件里有多少个字符」，而是「有多少个字节」。对纯英文文本来说这两个数碰巧一样（一个字符一个字节）；只要文件里有中文，两个数立刻分家——UTF-8 编码下一个汉字是 3 个字节，GBK 编码下通常是 2 个（本机源码与输出都是 UTF-8，所以你在本书里看到的汉字都按 3 字节算）。「字符数」与「字节数」从一开始就是两个概念。

c **（续）数字节的那个函数** examples/ch06/newline_bytes/src/newline_bytes.c 第 21–34 行

```c
long count_bytes(const char *path) {
    FILE *fp = fopen(path, "rb");          /* 必须以 rb 打开：文本模式会把 \r\n 折回 \n */
    if (fp == NULL) {
        perror(path);
        return -1;
    }
```

```
    long n = 0;
    int c;
    while ((c = fgetc(fp)) != EOF) {          /* fgetc 一次一个字节，读到 EOF 为止 */
        n++;
    }
    fclose(fp);
    return n;
}
```

数出来是这样。`main` 除了报数字，还把每个字节按十六进制打了一遍——因为「差 2 个字节」这句话本身没有信息量，**差的是哪两个字节**才有：

终端输出 终端输出 newline-bytes-out

```
写进去的内容都是：'a' 换行 'b' 换行（4 个字符）
text.txt（用 "w"  写）：6 字节
bin.txt  （用 "wb" 写）：4 字节
差值：2 字节
text.txt 的字节：61 0d 0a 62 0d 0a
bin.txt  的字节：61 0a 62 0a
61='a'  62='b'  0a=换行  0d=回车
--- 换成 fseek(SEEK_END) + ftell 来量 ---
text.txt 用 "r"  打开：6 字节
text.txt 用 "rb" 打开：6 字节
bin.txt  用 "rb" 打开：4 字节
本机两种模式给出同一个数；标准并没承诺文本模式下这个数有意义
```

`text.txt` 是 **6** 个字节，`bin.txt` 是 **4** 个字节，差出来的 `0d 0a` 里的 `0d` 就是**回车符 CR**。用 `od -c` 看更直观，`\r` 会原样显示出来：

终端会话 od -c、wc -c、python：三个独立工具数同一份文件 ch06-newline-od

```
$ od -c text.txt
0000000   a  \r  \n   b  \r  \n
0000006
$ od -c bin.txt
0000000   a  \n   b  \n
0000004
$ wc -c text.txt bin.txt
 6 text.txt
 4 bin.txt
10 total
$ python -c "print(open(\"text.txt\",\"rb\").read())"
b'a\r\nb\r\n'
$ python -c "print(open(\"bin.txt\",\"rb\").read())"
b'a\nb\n'
```

三个工具、同一份文件、同一组数字：程序自己数出来 6 / 4，`od -c` 看见 `\r`，`wc -c` 报 6 / 4，`python` 把字节原样印出来是 `b'a\r\nb\r\n'` 与 `b'a\nb\n'`。**当四个来源都对上时，你才可以说「这不是我的程序算错了」**——这也是本书每个数字都要真跑一遍的理由。

这 2 个字节的差别来自**文本模式**。先别把它当成「Windows 的怪毛病」——标准早就为这种事留好了说辞，而且说辞比你想的更宽松：标准压根不承诺「你写进去几个字符，磁盘上就有几个字节」。它把流分成两类，各自给了一句保证。标准在讲流的那一节里说得很明白（这是本单元最该背下来的一句标准原文）：

标准原文 · **标准原文：文本流**

> A text stream is an ordered sequence of characters composed into lines, each line consisting of zero or more characters plus a terminating new-line character. … Characters may have to be added, altered, or deleted on input and output to conform to differing conventions for representing text in the host environment. Thus, there need not be a one-to-one correspondence between the characters in a stream and those in the external representation.

大意：文本流是「按行组织」的，每一行以换行结尾（**最后一行需不需要换行由实现决定**）；**为了迁就宿主环境表示文本的惯例，字符可能在输入输出时被增加、改动或删除**——所以流里的字符与外部表示之间**不必**一一对应。

出处：ISO/IEC 9899:2018（C17）§7.21.3。

标准原文 · **标准原文（续）：二进制流**

> A binary stream is an ordered sequence of characters that can transparently record internal data.

大意：二进制流则是「能透明地记录内部数据」的字节序列。注意「透明」这个词：它不换算、不改写，你写进去什么字节，读回来就是什么字节。**一个例外要说在前面**：标准紧接着允许实现在二进制流的**末尾**追加「实现定义个数的空字符」，所以从二进制流里读到的字节数**可能**比你写进去的多——本机实测 `bin.txt` 是 4 字节、没有追加，但这不是你能依赖的保证。

本机的惯例是：文本模式下写 `\n`，落到磁盘上变成 `\r\n` 两个字节；读回来的时候再折回一个 `\n`。这就解释了两件事：文件**变大**了，而你的程序**感觉不到**（读回来的还是 `\n`）。

出处：ISO/IEC 9899:2018（C17）§7.21.3。

实验记录 · **本机实测：6 比 4，两处证据**

- 工具链：gcc 16.1.0（`-std=c17`，即 C17），目标三元组 x86_64-w64-mingw32，系统 Windows 10 x64 (MSYS2 UCRT64)；
- 程序自己数的：`text.txt` 6 字节、`bin.txt` 4 字节，差 2（`examples/ch06/newline_bytes/src/main.c` 真跑，输出见上面的 `output` 块）；
- `od -c` 看的：文本文件是 `a \r \n b \r \n`，二进制文件是 `a \n b \n`；
- `wc -c` 对的：`6 text.txt` / `4 bin.txt`——与程序数出来的完全一致；
- `python` 直接读字节：`b'a\r\nb\r\n'` 与 `b'a\nb\n'`（`od -c`、`wc -c`、两条 `python` 命令都在上面两段会话里，逐字抓下来的）。

别把 6 当定律：这是 Windows 上文本模式的换算结果。同一个程序拿到按 Unix 惯例的系统上跑，文本模式与二进制模式会给出同样的 4——那时你敲的命令、写的代码一个字都不用改，结果却变了。这正是「标准允许外部表示与流内字符不必一一对应」的意思：**换算规则由实现定。**

顺带把一个容易踩的坑说清楚：**数字节、按字节移动，都要用 `"rb"` 打开。**因为文本模式会在读的时候把 `\r\n` 折成 `\n`，你按「读到的字符数」去算，算出来的就不是磁盘上的字节数了。程序里 `size_by_seek` 那一段量出来的是：

终端会话 `fseek(SEEK_END) + ftell 量出来的大小` `ch06-newline-ftell`

```
--- 换成 fseek(SEEK_END) + ftell 来量 ---
text.txt 用 "r"  打开: 6 字节
text.txt 用 "rb" 打开: 6 字节
bin.txt  用 "rb" 打开: 4 字节
本机两种模式给出同一个数；标准并没承诺文本模式下这个数有意义
```

最后那句不是客气话。标准对文本模式的 `ftell` 说得很难听：**它返回的数不保证有意义**（6.9 节给原文）。本机这两种模式恰好给出同一个数，那是这台机器的实现细节，不是你能依赖的保证。

6.5 读：fgetc / fgets / fscanf 与「一行太长会怎样」

读文件有三个常用函数，分工不一样：

函数	读什么	关键行为
`fgetc(fp)`	**一个字节**	返回 `int`（要能装下 `EOF`），读到末尾返回 `EOF`
`fgets(buf, n, fp)`	**至多 n-1 个字符**	遇到换行或文件末尾就停，**换行会被留下**
`fscanf(fp, "…", …)`	**按格式解析**	返回值是「成功填了几个字段」；解析失败的收尾很麻烦

`fgets` 是这三个里最常用的，也是坑最多的。坑在于：**它读的不一定是「一行」**。标准原文把这句写得毫不含糊：

标准原文 · **标准原文：fgets 到底答应了你什么**

> The fgets function reads at most one less than the number of characters specified by n from the stream pointed to by stream into the array pointed to by s. No additional characters are read after a new-line character (which is retained) or after end-of-file. A null character is written immediately after the last character read into the array.

大意：`fgets` **最多读 n-1 个字符**；读到换行符（**换行本身会被保留**）或文件末尾就不再往下读；最后在读到的内容后面补一个空字符。

出处：ISO/IEC 9899:2018（C17）§7.21.7.2。

「最多 n-1 个」就是截断的根源。下面这个数据文件的第一行有 52 个字符——注意它只是一个普通文本文件，不是 C 代码：

log **数据文件 longline.txt：第一行 52 个字符** examples/ch06/fgets_long_line/data/longline.txt 第 1–2 行

```log
01234567890123456789012345678901234567890123456789012345678901
END
```

用 16 字节的缓冲区去读它，会发生这样的事：

终端输出 终端输出 fgets-long-line-out

```
--- 错误用法：只有 16 字节的缓冲区，一行被剁成几段 ---
第 1 段：长度 15，末尾没有换行 |012345678901234|
第 2 段：长度 15，末尾没有换行 |567890123456789|
第 3 段：长度 15，末尾没有换行 |012345678901234|
第 4 段：长度 8，末尾有换行 |5678901|
第 5 段：长度 4，末尾有换行 |END|
一共读出来 5 段，可这个文件只有 2 行
--- 正确做法：把段拼回去，数到换行才算一行 ---
第 1 行读完：共 53 个字节（含行尾换行）
第 2 行读完：共 4 个字节（含行尾换行）
一共读出来 2 行，对上了
```

5 段，2 行。第 1~3 段各 15 个字符（就是 n-1），第 4 段 8 个字符带着换行，第 5 段是第二行。也就是说：`fgets` **每被调用一次，只交出它读到的这一段**，不要把它当成「读一行」。

c examples/ch06/fgets_long_line/src/fgets_long_line.c 第 34–54 行

```c
int read_by_line(const char *path, FILE *out) {
    FILE *fp = fopen(path, "r");
    if (fp == NULL) {
        perror(path);
        return -1;
    }
    char buf[CHUNK];
    size_t total = 0;
    int lines = 0;
    while (fgets(buf, sizeof buf, fp) != NULL) {  小缓冲区，循环里拼
        total += strlen(buf);  每段长度累加
        if (strchr(buf, '\n') != NULL) {      /* 这一段里有换行 = 整行读完了 */  有换行才算一行读完
            lines++;
```

```
            fprintf(out, "第 %d 行读完：共 %zu 个字节（含行尾换行）\n", lines, total);
            total = 0;
        }
    }
    if (total > 0) {                        /* 最后一行没有换行也要收尾 */
        lines++;
        fprintf(out, "第 %d 行读完：共 %zu 个字节（文件末尾没有换行）\n", lines, total);
    }
```

43 小缓冲区，循环里拼
44 每段长度累加
45 有换行才算一行读完

这就是「把一行完整读回来」的通用写法：**循环里一直拼，拼到看见换行为止。**它不依赖缓冲区多大——缓冲区小只是多转几圈，结果一样。（本机实测那一行拼出来是 53 字节：52 个字符 + 1 个换行。）

⚠ 陷阱 · 用 fgets 时你会被这样坑到

你会看到：数出来的「行数」比你用记事本打开看到的行数多，内容还被切成了几段。

你会怎么误判：以为是文件坏了、编码不对，或者回去改 `fgets` 的第二个参数，想着「调大点就好了」。调到 1024 确实能过这一次——下一份数据里有一行 5000 个字符，你又得回来。

真相是：**`fgets` 的契约里从来没写「读一行」，它写的是「最多读 n-1 个字符」。**所以正确的用法只有两种：要么循环拼到换行为止（上面的写法），要么在读之前就知道「这个文件的每一行都不会超过多少字节」——后者只要有一次不成立，就会静默出错。另外记住 `fgets` **连换行一起给你**：想打印时去掉它，得自己把末尾那个 `'\n'` 换成 `'\0'`，而且不能想当然地认为「一定有」——最后一行没有换行时就没有。

6.6 读完与出错：`feof` 与 `ferror` 的正确用法

`while (!feof(fp))` 是新手最爱写的循环，也是这一单元里**唯一一个「写得像对的、结果就是错的」**的坑。下面这个程序用一个正好三行的数据文件把两种写法摆在一起：

log **数据文件 three_lines.txt: alpha / beta / gamma** examples/ch06/feof_trap/data/three_lines.txt 第 1–3 行

```
alpha
beta
gamma
```

先别看解释，只看数出来的行数：上半段是错误写法，下半段是正确写法，同一个文件、同一个缓冲区。

终端输出 终端输出 feof-trap-out

```
--- while (!feof(fp)): 先问再读 ---
  1: alpha
  2: beta
  3: gamma
  4: gamma
--- while (fgets(...) != NULL): 读了才问 ---
  1: alpha
  2: beta
  3: gamma
错误写法数出 4 行，正确写法数出 3 行
```

看第 4 行：`gamma` 被打印了**两遍**。这不是打错了，是 `while (!feof(fp))` 的必然结果。原因在标准那句原文里：

标准原文 · 标准原文：feof 说的是「上一次读」的结果

> The feof function returns nonzero if and only if the end-of-file indicator is set for stream.

大意：`feof` 只在**流的文件末尾指示器已经置位**时返回非 0。——而那个指示器是**某一次读失败之后**才被置上的。它回答的是「上一次读有没有撞到末尾」，不是「下一次读会不会撞到末尾」。

出处：ISO/IEC 9899:2018（C17）§7.21.10.2。

`fgets` 在读失败时做什么，标准也写清了：

标准原文 · 标准原文：fgets 读不到东西时，数组会怎样

> The fgets function returns s if successful. If end-of-file is encountered and no characters have been read into the array, the contents of the array remain unchanged and a null pointer is returned. If a read error occurs during the operation, the array contents are indeterminate and a null pointer is returned.

大意：读到东西就返回 `s`；**如果在没读到任何字符的情况下撞上文件末尾，数组的内容保持不变**，并返回空指针；**如果过程中发生了读错误，数组的内容是「不确定的」**（indeterminate，标准不保证它是什么）。

出处：ISO/IEC 9899:2018（C17）§7.21.7.2。

最后那半句正是本节的要害：`fgets` 失败时也可能返回空指针，两种失败的**后果不一样**——撞到末尾时数组原封不动（所以你会看见「上一行的副本」），读错误时数组里是什么只有天知道。**光看返回值分不出是哪一种**，这就是为什么循环退出后还要问一句 `ferror`。

两句话拼起来，「读两遍」的经过就完整了（本节这个文件是正常读到末尾，属于第一种）：

图 / 搜索过程 **while (!feof(fp)) 的四圈：第四圈是白转的**

第 1 步 第 1 圈，读了 alpha

`!feof(fp)` ：文件末尾指示器还没置位，先进来 ｜ `alpha` ：`fgets` 读到 `"alpha\n"`，打印

第 2 步 第 3 圈，读了 gamma

`gamma` ：`fgets` 读到 `"gamma\n"`，打印 ｜ 文件位置 ：已经贴到文件末尾

第 3 步 第 4 圈，什么都没读，但也没报错

`!feof(fp)` ：指示器还没置位 — 上一圈读成功了 ｜ `gamma` ：`fgets` 返回空指针，数组里还是上一行的内容

`gamma` → 屏幕 于是 gamma 又被打印一遍

第 4 步 第 5 次检查才退出

`feof(fp)` ：这一次终于非 `0` 了（第 4 圈那次失败把指示器置上了） ｜ 循环 ：退出，可屏幕上已经多了一行

第 2 圈读的是 `beta`，与第 1 圈一样平淡，图上略过了；从第 3 圈开始才是故事发生的地方。第 3 帧是全部问题的所在：**那一次失败的读没有告诉你**（你没看 `fgets` 的返回值），却留下了一个「上一行的副本」。注意第 4 帧说的是「第 5 次**检查**」——循环体只执行了 4 次，第 5 次走到 `while` 的条件就退出了。

正确的循环条件就一条：**拿读函数的返回值当条件**。同时，循环退出以后还要分清是「读完了」还是「出错了」——那是 `ferror` 的工作。

下面两块来自同一个文件，第一个是错的写法，第二个是对的：

c examples/ch06/feof_trap/src/feof_trap.c 第 11–15 行

```
11      while (!feof(fp)) {                     /* feof 说的是「上一次读」的结果 */ 先问再读，问题就在这
12          fgets(line, sizeof line, fp);       /* 返回值没看 — 读失败时 line 里还是上一行 */ 返回值没看，读失败也往下走
13          n++;
14          fprintf(out, "  %d: %s", n, line); 上一行的内容被重打一次
15      }
```

11 先问再读，问题就在这
12 返回值没看，读失败也往下走
14 上一行的内容被重打一次

c examples/ch06/feof_trap/src/feof_trap.c 第 28–36 行

```
28      while (fgets(line, sizeof line, fp) != NULL) {  /* 读到东西才进循环 */ 读到东西才进循环
29          n++;
```

```
30          fprintf(out, "  %d: %s", n, line);
31      }
32      if (ferror(fp)) {                       /* 循环退出后分清：读完了，还是出错了 */ 退出后分清读完与出错
33          perror(path);
34          fclose(fp);
35          return -1;
36      }
```

28 读到东西才进循环
32 退出后分清读完与出错

`ferror` 与 `feof` 是一对，问的是完全不同的两件事：`feof` 问「是不是读到文件末尾了」，`ferror` 问「是不是读错了」（磁盘坏道、网络流断开这类）。**循环退出后至少要问一句 `ferror`**——`feof` 不用问，因为「读到末尾」本来就不是错误，它只是告诉你「读完了」；真正会让你把「读到一半出错」当成「正常读完」的，是 `ferror` 那一句。漏了它，数据少了一半也没人吭声，那是最难查的一类 bug。

实验记录 · 本机实测：4 行与 3 行

- 工具链：gcc 16.1.0（`-std=c17`），`examples/ch06/feof_trap/src/feof_trap.c` 真跑；
- `while (!feof(fp))` 数出 **4** 行（`alpha` / `beta` / `gamma` / `gamma`），最后一行重复；
- `while (fgets(...) != NULL)` 数出 **3** 行，与文件里的行数一致；
- 两份输出都在同一个 `output` 块里，由流水线逐字对账（命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/feof_trap.c -o app`）。

顺便说明一件事：如果这个文件**最后一行没有换行**，`while (!feof(fp))` 反而不会重复那最后一行——因为在读最后一个字节时就已经撞到文件末尾、指示器当场置位了。**一个「有时候对、有时候错」的 bug 才最难查**，所以别去记「哪种文件会重复」，直接换正确的写法。

6.7 缓冲与 fflush：崩溃前那两行去哪了

现在回答 6.2 留下的问题：为什么崩溃之前 `printf` 过的东西会丢。先看两个只差一行代码的程序，它们做的事完全一样——打印两行，然后**故意解引用空指针**把自己弄死：

c examples/ch06/fflush_crash/src/fflush_crash.c 第 5–12 行

```
5   void print_then_crash(void) {
6       printf("第一行：崩溃之前我确实打印过\n");  打到 stdout
7       printf("第二行：这两行还躺在缓冲区里\n");  这一行还在缓冲里
8       /* 故意：空指针解引用，进程会被操作系统直接终止 */
9       int *p = NULL;  故意：空指针
10      *p = 1;  进程在这里被终止
11      printf("这一行永远到不了\n");
12  }
```

6 打到 stdout
7 这一行还在缓冲里
9 故意：空指针
10 进程在这里被终止

c examples/ch06/fflush_kept/src/fflush_kept.c 第 5–15 行

```
5   void print_flush_then_crash(void) {
6       printf("第一行：崩溃之前我确实打印过\n");
7       printf("第二行：这两行已经不在缓冲区里了\n");
8       if (fflush(stdout) != 0) {              /* 显式把缓冲区交出去 */  一行之差：显式把缓冲交出去
9           perror("fflush");
10      }
11      /* 故意：空指针解引用，进程会被操作系统直接终止 */
12      int *p = NULL;
13      *p = 1;
14      printf("这一行永远到不了\n");
```

```
15  }
```

8 一行之差：显式把缓冲交出去

两个程序真跑出来的 stdout，一个满、一个空：

终端输出 终端输出 fflush-crash-out

（无输出）

终端输出 终端输出 fflush-kept-out

```
第一行：崩溃之前我确实打印过
第二行：这两行已经不在缓冲区里了
```

第一个（没有 `fflush`）的 stdout 是**空的**；第二个把那两行留下来了。这不是「崩溃太突然」，而是**那两行字到死都还在进程自己的缓冲区里。**

「缓冲区在哪儿」这句话，光靠说很难讲清，画成一条路就清楚了。下面四帧跟着**一个字节**走：它从 `printf` 出发，经过两个站，最后才落到磁盘上——中间任何一站断了，它就没了：

图 / 搜索过程 **一个字节从 printf 到磁盘：三站路与两个岔口**

第 1 步 printf 只是把字节放进缓冲区

你的程序 ： printf("hi\n") | fp 的用户态缓冲 ： 字节先躺在这里 ← 本步新增

你的程序 → fp 的用户态缓冲 printf 与 fwrite 都只走到这里

第 2 步 缓冲区满了，或者你调了 fflush

你的程序 | fp 的用户态缓冲 ： 刚刚被交出去 | 内核的文件缓冲 ： 字节搬到了这一站 ← 本步新增

fp 的用户态缓冲 → 内核的文件缓冲 fflush(fp) 或缓冲区写满

第 3 步 fclose 把两件事一起做完

你的程序 | 内核的文件缓冲 ： 内核负责最终落盘 | 磁盘上的文件 ： 到这一站才算真的留下 ← 本步新增

fp 的用户态缓冲 → 内核的文件缓冲 fclose 先冲缓冲

内核的文件缓冲 → 磁盘上的文件 再关句柄

第 4 步 没有 fflush 也没有 fclose，两站的字节一起没

进程被终止 ： 用户态缓冲随进程消失 ← 本步新增 | fp 的用户态缓冲 ： 里面的字节没了 | 你的程序 ： 屏幕上什么都没有

第 1 帧是 `printf` 的真相：**它不写文件，它写缓冲区。**第 2 帧解释「什么时候真的往下走」——缓冲写满，或者你显式 `fflush`。第 3 帧是 `fclose`，它把「冲缓冲」和「关文件」一起做完（6.3 引的那句标准原文就是这件事）。第 4 帧是崩溃：进程被操作系统直接终止，**用户态缓冲里的字节跟着进程一起消失**，第 2 站根本没被碰到，所以磁盘上什么都没有。

为什么会有缓冲区这种设计？因为它把「很多次小写」变成「少数几次大写」：不缓冲的话，每个 `printf` 都要陷进内核一次，慢得没法用。代价就是你得知道它在那儿。

标准原文 · **标准原文：fflush 干了什么**

> If stream points to an output stream or an update stream in which the most recent operation was not input, the fflush function causes any unwritten data for that stream to be delivered to the host environment to be written to the file; otherwise, the behavior is undefined.

大意：对一个**输出流**（或者最后一次操作不是读的更新流），`fflush` 把还没写出去的数据交给宿主环境去写文件；**对其他情况，行为未定义。**

最后半句要记住：`fflush(stdin)` 不是「清空输入缓冲」的标准做法——标准明说这是未定义行为。（顺手说一句：`fflush(NULL)` 会冲**所有**输出流，这是合法用法。）

出处：ISO/IEC 9899:2018（C17）§7.21.5.6。

那什么时候是全缓冲、什么时候不是？回到 6.2 引的那句 §7.21.3：**标准错误流一开始不是全缓冲的**，标准输入与标准输出**只有当它们能确定不是交互设备时**才全缓冲。翻译成人话：

- 输出到**屏幕**（交互设备）：通常按行缓冲，每打一行就出去，所以你在终端里敲 `./app` 时看到的是实时的；
- 输出**被重定向**（`> out.txt`）或者**接进管道**（`./app | grep x`）：全缓冲，攒够一整块才出去；
- `stderr`：**不是全缓冲**（标准只保证这一点）；本机上它表现为写一句出去一句，也就是不缓冲。

所以「崩溃前有没有输出」这件事，**同一个程序在终端里跑和在管道里跑，答案可能不一样。**本书的所有输出都是经管道抓回来的（流水线要拿到 stdout 才能对账），所以你会看到「stdout 是空的」这种结果——那不是 bug，是缓冲。

工程惯例 · 工程惯例：要留下证据，就显式 fflush

想让「崩溃前打印过什么」这件事在**任何**情况下都可复现，主流做法是三条，按可靠程度排：

1. 把关键信息写 `stderr`——它不缓冲，写完就在；
2. 在容易崩的地方之前显式 `fflush(stdout)`（本节的 `fflush_kept` 就是这么做的）；
3. 记录时别只写 stdout，把两个流一起抓：`./app > out.txt 2>&1`。

代价是：程序里多出几行「不是为了功能、而是为了可诊断」的代码。这些行的价值不在平时，在出事那一天——没有它们，你手上只剩一个退出码。

实验记录 · 本机实测：同一个崩溃，两种 stdout

- 工具链：gcc 16.1.0（`-std=c17`），系统 Windows 10 x64 (MSYS2 UCRT64)；
- `examples/ch06/fflush_crash/`：退出码 **3221225477**（Windows 的访问冲突，0xC0000005），stdout **为空**——上面那个 output 块里只有哨兵「（无输出）」；
- `examples/ch06/fflush_kept/`：退出码同样 **3221225477**，stdout 是那两行（逐字见上面的 output 块）；
- 两个清单都是 `mode: runtime_error`，`expect.exit` 写的就是实测的 3221225477——流水线会核对这个数，写错就报 `exit_mismatch`。

口径说明：在 Git Bash 里手工跑同一个崩溃，你会看到 `Segmentation fault` 与退出码 **139**（128 + 信号 11）；本书正文里的「本机实测退出码」统一用 Windows 口径（3221225477）。两个数说的是同一次崩溃，别以为是两件事。

6.8 写：三个函数的返回值，以及「关掉才算写完」

写文件有三个函数，分工和读那边一一对应：

函数	写什么	返回值
`fprintf(fp, "…", …)`	按格式写	写出去的**字符数**；出错返回负值
`fputs("…", fp)`	一串字符，不解析格式	出错返回 `EOF`，否则返回非负值
`fwrite(ptr, size, n, fp)`	一块**原始字节**	成功写出的**元素个数**（可能小于 n）

返回值不是摆设：**它是你唯一能知道「写失败了」的途径。**标准对 `fprintf` 的措辞是：

标准原文 · 标准原文：写函数的返回值

> The fprintf function returns the number of characters transmitted, or a negative value if an output or encoding error occurred.
>
> The fwrite function returns the number of elements successfully written, which will be less than nmemb only if a write error is encountered.

大意：`fprintf` 返回**写出去的字符数**，遇到输出错误或编码错误返回负值；`fwrite` 返回**成功写出的元素个数**，只有写错误才会让它小于你要求的个数（元素就是「size 字节一块」的意思，6.10 节会用到）。

出处：ISO/IEC 9899:2018（C17）§7.21.6.1、§7.21.8.1。

下面这段真实的写函数做了三处检查：`fopen` 判 `NULL`、`fputs` 判 `EOF`、`fclose` 判不等于 0：

c examples/ch06/file_write_read/src/file_write_read.c 第 3–21 行

```c
int write_notes(const char *path) {
    FILE *fp = fopen(path, "w");              /* "w": 写，文件不存在就建 */
    if (fp == NULL) {                         /* fopen 失败返回 NULL，必须判 */ 返回 NULL 必须判
        perror(path);
        return -1;
    }
    fprintf(fp, "第一行：把结果留下来\n");      /* 按格式写 */
    if (fputs("第二行：这一行是 fputs 写的\n", fp) == EOF) {  /* 原样写一串字符 */ fputs 也要判 EOF
        perror("fputs");
        fclose(fp);
        return -1;
    }
    fprintf(fp, "第三行：数字 %d\n", 42);
    if (fclose(fp) != 0) {                    /* 关掉才算真的写完 */ 关掉才算真的写完
        perror("fclose");
        return -1;
    }
    return 0;
}
```

5 返回 NULL 必须判
10 fputs 也要判 EOF
16 关掉才算真的写完

第 16 行那句 `fclose(fp) != 0` 值得单独说一句：**它是「写成功了吗」的最后一问。**6.3 引的标准原文说 `fclose` 会「先把缓冲冲出去、再关掉文件」——也就是说，**前面几次 `fprintf` 真的落盘，可能发生在 `fclose` 这一下。**如果这一步失败了（磁盘满、U 盘被拔了），你前面根本没机会知道。

那两句 `fprintf` 的返回值呢？这段代码**没有逐个判**，因为它们的格式串与参数都是常量、当场就能看出对不对得上。但这是一个取舍，不是规矩：真要做到一丝不漏，就把每一个写函数的返回值都判一遍（`fprintf` 返回负值、`fputs` 返回 `EOF`、`fwrite` 返回的元素数少于你要的个数，都是「写失败了」）。**判不判是取舍，「知道它可能失败」不是可选项。**

跑起来是这样（`main` 先写、再读回来打印）：

终端输出 终端输出 file-write-read-out

```
=== 第一步：写 notes.txt ===
=== 第二步：再读回来 ===
1| 第一行：把结果留下来
2| 第二行：这一行是 fputs 写的
3| 第三行：数字 42
=== 一共读到 3 行 ===
sizeof(FILE)   = 8 字节（这个数由实现决定，标准没规定）
sizeof(FILE *) = 8 字节（你手上那个句柄本身）
```

两块值得注意：

- 第 3 行往后每一行前面那个 `1|` / `2|` 是 `read_notes` 加的行号（6.2 那个函数的 `fprintf(out, "%d| %s", …)`），**行号后面那一段是原样读回来的**，包括行尾的换行；
- 最后两行是 `report_file_type` 打的：本机上 `FILE` 这个类型占 **8** 字节。这个数先记住，6.12 要用它讲一件重要的事。

> ⚠ **陷阱 · 「文件写出来是空的」：先分清是哪一种空**
>
> 你会看到：程序退出码 0、屏幕上 `printf` 一行不差，可打开那个文件是 **0 字节**，或者少了最后几行。
>
> 你会怎么误判：先怀疑路径、权限、`fopen` 失败——可 `fopen` 真失败会走 `perror`，你没看到它就说明打开成功了；接着你会怀疑「忘了 `fclose`」。**这一条要小心**：从 `main` 正常返回或调用 `exit` 时，运行库会替你冲掉所有缓冲，所以「正常退出 + 忘了 `fclose`」通常**不会**得到空文件。
>
> 真相通常是三条之一：① **写对了，但不在你以为的地方**——相对路径按进程的当前目录算，不是按源文件的位置；② **程序在写完之前被终止了**（崩溃、`abort`、被强杀），用户态缓冲随进程消失；③ **写的时候出错了而你没看返回值**（磁盘满、U 盘被拔、或者文件是用 `"r"` 打开的）。所以：先 `ls` 确认文件在哪，再看进程怎么结束的，最后才怀疑返回值。

6.9 定位：fseek / ftell / rewind，与「怎么量一个文件多大」

流有一个**文件位置指示器**，你可以理解成「下一站读/写哪个字节」。三个函数围着它转：

- `fseek(fp, offset, whence)`：把位置挪到 `whence` 加 `offset` 的地方；`whence` 取 `SEEK_SET`（从头算）、`SEEK_CUR`（从当前位置算）、`SEEK_END`（从末尾算）；
- `ftell(fp)`：报告当前位置；
- `rewind(fp)`：回到开头（等于 `fseek(fp, 0, SEEK_SET)`，但**不返回错误**，这是它的缺点）。

`fseek(fp, 0, SEEK_END)` 加 `ftell(fp)`，就是「量一个文件多大」的标准写法。思路很朴素：**把读指针推到末尾，再问它现在站在第几个字节。**剩下的问题只有一个——推之前、推之后，流的状态是否允许你这么做：

c examples/ch06/newline_bytes/src/newline_bytes.c 第 52–72 行

```c
long size_by_seek(const char *path, const char *mode) {
    FILE *fp = fopen(path, mode);
    if (fp == NULL) {
        perror(path);
        return -1;
    }
    if (fseek(fp, 0, SEEK_END) != 0) {          /* 跳到文件末尾 */ 跳到文件末尾
        perror(path);
        fclose(fp);
        return -1;
    }
    long pos = ftell(fp);                       /* 当前位置就是字节数 */ 当前位置就是字节数
    rewind(fp);                                 /* 回到开头 */ 回到开头
    if (fseek(fp, 0, SEEK_SET) != 0) {          /* 再确认一次回得去 */
        perror(path);
        fclose(fp);
        return -1;
    }
    fclose(fp);
    return pos;
}
```

58 跳到文件末尾
63 当前位置就是字节数
64 回到开头

但这个数在**文本模式**下不能信。标准原文说得比谁都直白：

标准原文 · **标准原文：文本模式下 ftell 的数「不是有意义的度量」**

> The ftell function obtains the current value of the file position indicator for the stream pointed to by stream. For a binary stream, the value is the number of characters from the beginning of the file. For a text stream, its file position indicator contains unspecified information, usable by the fseek function for returning the file position indicator for the stream to its position at the time of the ftell call; the difference between two such return values is not necessarily a meaningful measure of the number of characters written or read.

大意：对**二进制流**，`ftell` 给出的是「从文件开头数几个字节」；对**文本流**，它的位置指示器里装的是**未指定的信息**，唯一保证是「能喂回给 `fseek` 回到原地」——**两个这样的数相减，不一定是读写字符数的有意义度量。**

出处：ISO/IEC 9899:2018（C17）§7.21.9.4。

这对我们意味着：**量大小、按字节操作，一律用 `"rb"` 打开。**本机这一条实测有点意思——UCRT 的实现给文本模式也返回物理字节数，两种模式恰好一样（6.4 那个会话的 `text.txt 用 "r" 打开：6 字节` 就是它）。但这不是标准给的保证，**换一个库就可能不一样**，所以代码里别依赖它。

接着把「程序自己量的」和「系统工具量的」对一遍。这是 `record_text_vs_bin` 两个输出文件的实测：

终端会话 `ftell 量出来的大小与 wc -c 对着看` — ch06-record-sizes

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/record_text_vs_bin.c -o
record_text_vs_bin.exe
$ ./record_text_vs_bin.exe | grep 字节
people.txt: 33 字节 (文本)
people.bin: 120 字节 (二进制)
sizeof(Record) = 40 字节 (id 4 + name 32 + score 4, 本机没有 padding)
$ wc -c people.txt people.bin
 33 people.txt
120 people.bin
153 total
```

33 与 33、120 与 120——程序用 `fseek` + `ftell` 量出来的，与 `wc -c` 数出来的完全一致。这就是那个实验的结论：**你写的 `size_by_seek` / `file_size` 没数错。**顺带看一眼 `153 total`：`wc` 连总数都替你算了。

6.10 一份记录，两种存法：文本还是二进制

前面都是在讲「怎么搬字节」，这一节回答「**同一份数据，该搬成什么样子**」。用一个只有三个字段的小结构体来对比：

c examples/ch06/record_text_vs_bin/include/record_text_vs_bin.h 第 4–15 行

```c
#include <stdio.h>

#define NAME_MAX 32 名字最长 32 字节

typedef struct { 三个字段的结构体
    int id;
    char name[NAME_MAX]; 定长数组，不是指针
    int score;
} Record;

/* 读 "1,Ada,95" 这种文本行；返回条数，失败（打不开）返回 -1。 */
int load_text(const char *path, Record *recs, int max);
```

6 名字最长 32 字节
8 三个字段的结构体
10 定长数组，不是指针

输入是一份 CSV（逗号分隔值，就是「一行一条记录、字段用逗号隔开」的纯文本），它本身不是 C 代码：

csv **数据文件 people.csv：三条记录** examples/ch06/record_text_vs_bin/data/people.csv 第 1–3 行

```csv
1,Ada,95
2,Alan,88
```

```
3,Grace,91
```

文本存法用 `fscanf` 读、`fprintf` 写，格式串自己定。这里有个与 6.5 呼应的点：`fscanf` 是「按格式解析」，它比 `fgets` 聪明（能一次取出三个字段），但也更脆——格式串里少一个逗号、数据里多一个空格，都可能让它在半路停下来而不报错。所以用它的时候，**返回值是唯一的哨兵**：

c examples/ch06/record_text_vs_bin/src/record_text_vs_bin.c 第 3–16 行

```c
int load_text(const char *path, Record *recs, int max) {
    FILE *fp = fopen(path, "r");
    if (fp == NULL) {
        perror(path);
        return -1;
    }
    int n = 0;
    while (n < max &&
           fscanf(fp, "%d,%31[^,],%d", &recs[n].id, recs[n].name, &recs[n].score) == 3) {
        n++;                                    /* fscanf 的返回值 = 成功填了几个字段 */
    }
    fclose(fp);
    return n;
}
```

11 返回值等于成功填进去了几个字段

第 11 行那个 `== 3` 是 `fscanf` 的用法要点：**它返回的是「成功赋值的字段个数」**，所以「读到完整一条」的判据是等于 3，不是等于 1。格式串里的 `%31[^,]` 是「最多 31 个非逗号字符」——31 这个数字与 `NAME_MAX 32` 配对（留一个字节给结尾的空字符），写宽了就越界。写出去那边是 `save_text`，格式串与读回来用的是同一套（`"%d,%s,%d\n"`），所以**同一个 `fscanf` 能读回自己写的文件。**

二进制存法用 `fwrite` / `fread`，直接搬内存里的字节。注意它搬的是**字节**，不是「值」：`int` 在内存里是 4 个字节，`fwrite` 就老老实实搬这 4 个字节，它不知道也不关心这 4 个字节代表什么数。所以二进制文件「快」的真正含义是**不需要解析**——读的时候按同样的顺序搬回来就行，不用像 `fscanf` 那样一个字符一个字符地认：

c examples/ch06/record_text_vs_bin/src/record_text_vs_bin.c 第 62–78 行

```c
int save_bin(const char *path, const Record *recs, int n) {
    FILE *fp = fopen(path, "wb");               /* 二进制输出一律用 "wb" */
    if (fp == NULL) {
        perror(path);
        return -1;
    }
    for (int i = 0; i < n; i++) {
        fwrite(&recs[i].id, sizeof recs[i].id, 1, fp);
        fwrite(recs[i].name, 1, sizeof recs[i].name, fp);
        fwrite(&recs[i].score, sizeof recs[i].score, 1, fp);
    }
    if (fclose(fp) != 0) {
        perror(path);
        return -1;
    }
    return 0;
}
```

63 二进制输出一律 wb
69 整数按内存里的样子写
70 32 个字节的定长名字

读回来时 `fread` 的返回值必须看——它是「读到了几个完整元素」。**三个字段各要问一句**：只查第一个字段是不够的，文件完全可能在一条记录的**中间**被截断，那时 id 读到了、名字或分数没读到，剩下的半条会带着旧值混进数组，而且 `n` 照样加一：

c examples/ch06/record_text_vs_bin/src/record_text_vs_bin.c 第 80–91 行

```c
int load_bin(const char *path, Record *recs, int max) {
    FILE *fp = fopen(path, "rb");               /* 二进制输入一律用 "rb" */
```

```c
    if (fp == NULL) {
        perror(path);
        return -1;
    }
    int n = 0;
    while (n < max) {
        /* 三个字段各问一句「真的读到了吗」：任何一块不全，这条记录就不能算数 */
        if (fread(&recs[n].id, sizeof recs[n].id, 1, fp) != 1) {
            break;
        }
```

89 要 1 个元素（4 字节的 id）

另外两个字段的检查长一个样，注意第二个用的是字节数（名字是按 1 字节为一块读的，所以返回值要等于 `sizeof name` 才是 32 个字节都读到了）：

c **（续）另外两个字段的检查** examples/ch06/record_text_vs_bin/src/record_text_vs_bin.c 第 92–102 行

```c
        if (fread(recs[n].name, 1, sizeof recs[n].name, fp) != sizeof recs[n].name) {
            break;                          /* 名字只读到一半：这条作废 */
        }
        if (fread(&recs[n].score, sizeof recs[n].score, 1, fp) != 1) {
            break;                          /* 分数没读到：这条作废 */
        }
        n++;
    }
    fclose(fp);
    return n;
}
```

整个程序跑出来的结果是这一节最该看的东西。它先读 CSV、再分别写成文本与二进制、再把两份都读回来——**读回来的值与原始值逐字段相同**，这不是巧合，是你自己选的格式串与字段顺序在两头对上了：

终端输出 终端输出 record-text-vs-bin-out

```
从 data/people.csv 读进来 3 条记录
原始记录（3 条）：
  id=1 name=Ada      score=95
  id=2 name=Alan     score=88
  id=3 name=Grace    score=91
people.txt: 33 字节（文本）
people.bin: 120 字节（二进制）
sizeof(Record) = 40 字节（id 4 + name 32 + score 4，本机没有 padding）
从文本读回来（3 条）：
  id=1 name=Ada      score=95
  id=2 name=Alan     score=88
  id=3 name=Grace    score=91
从二进制读回来（3 条）：
  id=1 name=Ada      score=95
  id=2 name=Alan     score=88
  id=3 name=Grace    score=91
```

三个数字并排看：文本 **33** 字节、二进制 **120** 字节、`sizeof(Record)` **40** 字节（40 × 3 = 120）。注意**二进制在这里更大**——因为 `name[32]` 是**定长**的：`Ada` 只占了 `A`、`d`、`a` 加一个结尾的 `\0` 共 4 个字节，**剩下的 28 个字节照样被写出去了。**

那 28 个字节里是什么？`recs` 是 `main` 里一个**没有初始化**的局部数组，`load_text` 只写了 `name` 的前 4 个字节，后面的 28 个**从来没被赋过值。**所以写出去的是**内存里原来摊着的东西**——可能是 0，也可能是上一次调用留下的残渣。这正是 6.10 末尾那条「别直接 `fwrite` 结构体」的实践建议的现场证据：**定长字段的尾巴是不确定的，它照样会进文件。**所以「二进制一定更省空间」是句想当然的话。

	文本（`fprintf` / `fscanf`）	**二进制（`fwrite` / `fread`）**
人能不能直接读	能，记事本能打开	不能，是字节堆
换语言 / 换工具读	能，格式自己定	得有同一套布局约定

	文本（`fprintf` / `fscanf`）	二进制（`fwrite` / `fread`）
要解析吗	每次都要（慢）	不用，直接搬（快）
大小	按内容变（`Ada` 就 3 字节）	按类型定长（`name[32]` 恒占 32）
逗号、换行出现在数据里	会被当成分隔符（要转义）	无所谓，任何字节都能存
跨机器读	通常行（文本惯例差异除外）	**危险**：字节序、`int` 宽度、padding

- **文本格式的软肋：一行坏了会怎样**

下面这个小例子是另一组实验，考的是 6.5 留下的那句「`fscanf` 解析失败的收尾很麻烦」。数据文件第 2 行**少了分数**：

csv **坏数据 people_bad.csv：第 2 行只有两个字段** examples/ch06/record_text_vs_bin/data/people_bad.csv 第 1–3 行

```csv
1,Ada,95
2,Alan
3,Grace,91
```

这个小例子把两种读法放进同一个程序，用同一份坏数据各跑一遍：

c **坏数据实验：同一个文件，两种读法** examples/ch06/record_text_vs_bin/src/bad_main.c 第 1–16 行

```c
#include <stdio.h>

#include "record_text_vs_bin.h"

int main(void) {
    Record recs[8];

    printf("--- 坏数据：直接拿 fscanf 读 ---\n");
    int bad = load_text("data/people_bad.csv", recs, 8);
    print_records(recs, bad, "fscanf 读到的", stdout);

    printf("--- 坏数据：fgets 分行 + sscanf 解析 ---\n");
    int fixed = load_text_lines("data/people_bad.csv", recs, 8, stdout);
    print_records(recs, fixed, "逐行解析读到的", stdout);
    return 0;
}
```

用 `load_text`（也就是 `fscanf`）去读它，结果只有 **1 条**：第 2 行读到 `2,Alan` 之后，格式串下一个字符要的是逗号、而输入里是换行，于是 `fscanf` 返回 **2**（成功填了 id 与 name 两个字段）——循环条件 `== 3` 不成立，**整个循环就此结束。** `Alan` 和 `Grace` 两条一声不响地没了，退出码还是 0。它不会告诉你「我在第几行出的错」，只给一个字段计数。

正确做法是**先分行、再解析**：用 `fgets` 管边界，用 `sscanf` 管字段，坏行整行丢掉并报告出来：

c examples/ch06/record_text_vs_bin/src/record_text_vs_bin.c 第 18–33 行

```c
int load_text_lines(const char *path, Record *recs, int max, FILE *out) {
    FILE *fp = fopen(path, "r");
    if (fp == NULL) {
        perror(path);
        return -1;
    }
    char line[128];
```

```c
    int n = 0;
    int lineno = 0;
    while (fgets(line, sizeof line, fp) != NULL) {
        lineno++;
        Record tmp;
        if (sscanf(line, "%d,%31[^,],%d", &tmp.id, tmp.name, &tmp.score) != 3) {
            fprintf(out, "  第 %d 行解析失败，整行丢掉：%s", lineno, line);
            continue;                       /* 坏行跳过，好行照收 */
        }
```

27 fgets 一次一整行
30 sscanf 与 fscanf 格式串相同
32 坏行跳过，好行照收

坏行已经被 `continue` 挡掉了，循环体只剩最后一步：把好行收进数组。循环结束后关文件、返回真正收下的条数：

c **（续）收下好行，关文件，返回条数** examples/ch06/record_text_vs_bin/src/record_text_vs_bin.c 第 34-40 行

```c
        if (n < max) {
            recs[n++] = tmp;
        }
    }
    fclose(fp);
    return n;
}
```

同一个文件、换这种读法，输出变成「第 2 行解析失败，整行丢掉：2,Alan」，然后收下另外两条。**坏行可见、好行不丢、程序还能继续**——这三件事合起来才叫「收尾」。注意 `sscanf` 与 `fscanf` 的格式串**一模一样**，只是输入源从「流」换成了「一个字符串」，所以这种改法几乎不用重写解析逻辑。

两段输出放在一起，差别就是全部：

终端输出 终端输出 record-bad-input-out

```
--- 坏数据：直接拿 fscanf 读 ---
fscanf 读到的（1 条）：
  id=1 name=Ada      score=95
--- 坏数据：fgets 分行 + sscanf 解析 ---
  第 2 行解析失败，整行丢掉：2,Alan
逐行解析读到的（2 条）：
  id=1 name=Ada      score=95
  id=3 name=Grace    score=91
```

工程惯例 · **工程惯例：别直接 fwrite 整个结构体**

你会很想写 `fwrite(&rec, sizeof rec, 1, fp)` ——一行就写完，多省事。**别这么写进文件格式**，理由有三条，每一条都会在换环境时咬你：

- **padding 是垃圾**：结构体里的对齐填充字节内容不确定（第 11 单元讲过），它会被原样写进文件，让「同一个文件」的内容每次不一样；
- **字节序**：本机是小端，换到大端机器读出来就是乱的；
- **类型宽度**：`int` 在这台机器是 4 字节，标准只保证它**至少** 16 位。

所以本节的 `save_bin` 是**逐字段**写的。代价是代码长一点、顺序必须与读那边严格一致（写错了不会报错，只会读出垃圾）。

实验记录 · 本机实测：两种存法都能读回来

- 工具链：gcc 16.1.0（`-std=c17`），`examples/ch06/record_text_vs_bin/` 真跑；
- 文本：`people.txt` **33** 字节；读回来三条记录，与原始数据逐字段一致；
- 二进制：`people.bin` **120** 字节（= `sizeof(Record)` 40 × 3）；`fread` 读回来同样三条，值一致；
- `wc -c` 对账：`33 people.txt` / `120 people.bin` ——与程序自己量的一致（见 6.9 的会话）。

这个例子**没有用任何检测器**：它演示的是「两种写法都合法、都读得回来」，不是「哪里写错了」。本书的检测器示例集中在指针与内存那几章（c04 / c08 / c09 / c12），这里不去凑。

6.11 命令行参数、重定向与退出码

到目前为止，程序要用的文件名都**写死在代码里**。真实工具不是这样：`mycat data/two_lines.txt` 里的文件名是**敲命令的时候给的**。C 用 `main` 的两个参数把命令行的东西交给程序：

c examples/ch06/mycat/src/main.c 第 5–7 行

```c
int main(int argc, char **argv) {
    return run(argc, argv, stdout);
}
```

5 两个参数的写法只在这里出现

标准对这两个参数的规定：

标准原文 · 标准原文：argc 与 argv

> The function called at program startup is named main. … It shall be defined with a return type of int and with no parameters … or with two parameters (referred to here as argc and argv, though any names may be used, as they are local to the function in which they are declared) …

大意：`main` 可以没有参数，也可以有两个。

出处：ISO/IEC 9899:2018（C17）§5.1.2.2.1。

标准原文 · 标准原文（续）：argv[0] 是程序名

> If the value of argc is greater than zero, the string pointed to by argv[0] represents the program name; argv[0][0] shall be the null character if the program name is not available from the host environment. If the value of argc is greater than one, the strings pointed to by argv[1] through argv[argc-1] represent the program parameters.

大意：`argc` 是参数个数，`argv` 是一个字符串数组。**`argv[0]` 是程序自己的名字**（拿不到程序名时它是个空串），`argv[1]` 到 `argv[argc-1]` 才是你敲的那些参数。

出处：ISO/IEC 9899:2018（C17）§5.1.2.2.1。

「`argv[0]` 是程序名」这件事有个实际后果：**在主流宿主上 `argc` 至少是 1**（`argv[0]` 占着那个位置），所以「用户没给参数」的判据是 `argc < 2`，不是 `argc == 0`。这里留一句余量：标准只说 `argc` **非负**（也就是允许 0，那时 `argv[0]` 不存在），所以别把「`argc >= 1`」当成语言定律——`argc < 2` 这个写法在两种情况下都是对的。`mycat` 就是照这个写的：

c examples/ch06/mycat/src/mycat.c 第 19–29 行

```c
int run(int argc, char **argv, FILE *out) {
    fprintf(out, "argc = %d\n", argc);        /* argv[0] 是程序自己的名字，这里不打印它 */
    for (int i = 1; i < argc; i++) {
        fprintf(out, "argv[%d] = %s\n", i, argv[i]);
```

```
    }

    if (argc < 2) {                                 /* 一个文件都没给：说清怎么用，然后返回 2 */ 一个文件都没给
        fprintf(stderr, "用法：mycat <文件>...\n");
        fprintf(stderr, "      用 - 表示标准输入，例如 mycat - 或 mycat data/two_lines.txt\n");
        return 2; 用法写到 stderr
    }
```

20 先报参数个数
25 一个文件都没给
28 用法写到 stderr

用法说完了，下面是真正的活：把 `argv[1]` 到 `argv[argc-1]` 逐个过一遍。

c **（续）逐个文件处理** examples/ch06/mycat/src/mycat.c 第 31–40 行

```
    int failed = 0;
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-") == 0) {        /* 约定：减号 = 标准输入 */
            int n = cat_stream(stdin, "(stdin)", out);
            if (n < 0) {
                failed = 1;
            }
            continue;
        }
        FILE *fp = fopen(argv[i], "r");
```

后面那半段是这个工具最有价值的部分：**一个文件打不开，不终止整个程序**——报告、继续、最后用一个非 0 的退出码说「这次不算全成功」：

c **（续）打不开就跳过，最后用退出码收口** examples/ch06/mycat/src/mycat.c 第 41–54 行

```
        if (fp == NULL) {                       /* 打不开不算致命：报告、继续、最后返回 1 */
            fprintf(stderr, "mycat: 打不开 %s\n", argv[i]);
            perror(argv[i]);
            failed = 1;
            continue;
        }
        int n = cat_stream(fp, argv[i], out);
        fclose(fp);
        if (n < 0) {
            failed = 1;
        }
    }
    return failed ? 1 : 0;
}
```

三个真实场景，用同一个可执行文件跑出来：

终端会话 不给参数：用法打到 stderr，退出码 2 ch06-mycat-usage

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/mycat.c -o mycat.exe
$ ./mycat.exe
用法：mycat <文件>...
      用 - 表示标准输入，例如 mycat - 或 mycat data/two_lines.txt
argc = 1
$ echo $?
2
```

这一段里有个**顺序陷阱**：`argc = 1` 那行跑到了用法后面。它不是打错了——`stderr` 不缓冲，当场就出去；`stdout` 这时候被接进了管道，是**全缓冲**，直到进程退出才一块儿冲出来。终端里直接跑（不接管道）时，你会看到正常的先后顺序。这正是 6.7 讲的东西的现场版。

终端会话 给文件：打不开的那个报告完继续，退出码 1 ch06-mycat-file

```
$ ./mycat.exe data/two_lines.txt
argc = 2
argv[1] = data/two_lines.txt
data/two_lines.txt 第 1 行: first line
data/two_lines.txt 第 2 行: second line
$ echo $?
0
$ ./mycat.exe data/nope.txt
mycat: 打不开 data/nope.txt
data/nope.txt: No such file or directory
argc = 2
argv[1] = data/nope.txt
$ echo $?
1
```

`data/nope.txt` 这个文件根本不存在，程序没有崩：它把原因打成两行（自己写的一句 + `perror` 给的系统说明），然后返回 **1**。

终端会话 重定向：< 喂输入，> 收输出 ch06-mycat-redirect

```
$ ./mycat.exe - < data/two_lines.txt > from-stdin.txt
$ echo $?
0
$ cat from-stdin.txt
argc = 2
argv[1] = -
(stdin) 第 1 行: first line
(stdin) 第 2 行: second line
```

这一行 `./mycat.exe - < data/two_lines.txt > from-stdin.txt` 里藏着两个方向的连接：

- `<` 把 `data/two_lines.txt` 接到**标准输入**上——于是程序里的 `stdin` 读到的就是文件内容，**程序一行都没改**；
- `>` 把**标准输出**接到 `from-stdin.txt`——于是 `fprintf(out, …)` 写的东西全进了文件，屏幕上只剩 `$` 后面的东西。

本书的清单里，喂标准输入用的是 `stdin` 字段（`mycat.json` 里写的是 `"stdin": "alpha\nbeta\n"`），作用与 `<` 完全一样，只是由流水线替你做：这就是「没有 `args` 之外的输入通道时，怎么给程序喂数据」的答案。`mycat` 这个清单的 `output` 块，就是它读到那段 `stdin` 之后打出来的：

终端输出 终端输出 mycat-out

```
argc = 2
argv[1] = -
(stdin) 第 1 行: alpha
(stdin) 第 2 行: beta
```

工程惯例 · 工程惯例：退出码的三种约定

标准只钉死了「0 成功、`EXIT_FAILURE` 不成功」（6.3 的原文）。那 `return 2` 算什么？**惯例**。主流约定是：

- `0` —— 成功；
- `1` —— 运行失败（文件打不开、数据不对、网络断了），**这是最通用的一档**；
- `2` —— **用法错**（参数没给、参数不认识），这一档在 GNU 工具里几乎人手一个（敲 `grep` 不带参数就能看到）。

为什么要把「用法错」和「运行失败」分开？因为它们要修的地方不一样：看到 2 去看帮助、检查自己敲的参数；看到 1 去查数据与文件。代价是：这套约定**没有标准保证**，换一个工具可能用别的数（`grep` 用 2 表示「有文件找不到」）。所以**别在脚本里依赖具体数值**，只判「是不是 0」——那一条才是标准钉死的。

6.12 FILE 是什么：你手上只有一个句柄

`fopen` 返回的那个 `FILE *`，到底是什么？答案是：**一个句柄（handle）**——它只是「哪个流」的编号，不是文件本身，也不是文件内容。这件事画出来是这样：

第 3 帧那个数字来自 6.8 的输出块（`sizeof(FILE) = 8 字节`），是**本机实测**。这里要老实交代一件事，因为它与你在网上常见的说法不一样：

标准原文 · 标准原文：标准只说 FILE 是「能记录流状态的对象类型」

> The header <stdio.h> declares three types, several macros, and many functions for performing input and output.
>
> The types declared are size_t ... and FILE which is an object type capable of recording all the information needed to control a stream, including its file position indicator, a pointer to its associated buffer (if any), an error indicator that records whether a read/write error has occurred, and an end-of-file indicator that records whether the end of the file has been reached ...

大意：`FILE` 是**一个对象类型**，它「能记录控制一个流所需的全部信息」——位置指示器、缓冲区指针、错误指示器、文件末尾指示器。

注意标准**没有**说它是「不完整类型」。它只要求这个类型装得下这些信息；至于要在头文件里把内容摊开（完整类型），还是只留一个名字（不完整类型），**由实现自己定**。出处：ISO/IEC 9899:2018（C17）§7.21.1。

大白话 · **把「完整」与「不完整」说成人话**

「**不**完整类型」的意思是：**编译器知道这个名字，但不知道它有几个字节。**所以你可以写 `FILE *p`（指针多大是知道的），但不能写 `FILE f`（要分配内存，就得知道多大）。

标准**没有规定** `FILE` 是不是完整类型，两种做法都合法：只给名字不给定义（那就是不完整类型，`sizeof(FILE)` 直接编不过），或者把结构体的定义摆在头文件里（本机就是这一种，`sizeof(FILE)` 能算出 8）。所以「`FILE` 是不完整类型」这句话**不能当成通用事实背**——它能成立，也能不成立，取决于你手上这套头文件。

不管是哪一种，对你的写法要求完全一样：**只能拿指针，只能通过函数去用它。**差别只在于——如果它是完整的，你「能」写出 `sizeof(FILE)` 甚至去碰它的成员，编译器不拦你；但那是别人的实现细节，换一套头文件就编不过、或者数值全变。

所以「别去 `sizeof(FILE)`」的准确说法是：**可以出于好奇 print 一下（6.8 就打了），但别把它当成接口的一部分**——别算它、别按它写文件、别去猜它的成员。那些都是库的私事，你依赖它，换一套头文件就翻车。

把句柄当成内容是这一单元最典型的初学者错误，编译器会当场把你拦下来。这个例子是故意写错的：

c examples/ch06/handle_not_string/src/handle_not_string.c 第 3–7 行

```c
int show_as_string(FILE *fp) {
    /* 想「把文件里的内容打出来」，于是把句柄交给了 %s —— 这一条编译器会拦 */
    printf("文件的内容是：%s\n", fp); 把 FILE * 交给了 %s
    return 0;
}
```

5 把 FILE * 交给了 %s

清单里给它加了 `-Werror`（因为这一条默认是**警告**而不是错误），编译真吐出来的是：

编译诊断 gcc 的真实诊断（原文照抄，flags 里加了 -Werror） handle-not-string-diag

```
src/handle_not_string.c: In function 'show_as_string':
src/handle_not_string.c:5:28: error: format '%s' expects argument of type 'char *', but argument 2 has type
'FILE *' {aka 'struct _iobuf *'} [-Werror=format=]
    5 |     printf("文件的内容是：%s\n", fp);
      |                            ~^     ~~
      |                             |     |
      |                             |     FILE * {aka struct _iobuf *}
      |                             char *
cc1.exe: all warnings being treated as errors
```

这份诊断里有三样东西正好收尾本单元：

- `expects argument of type 'char *', but argument 2 has type 'FILE *'` —— 一句话说清 `%s` 要的是字符串，你给的是句柄；
- `{aka 'struct _iobuf *'}` —— 编译器顺手把 `FILE` 的**真名**告诉你了，这就是第 2 帧里那个 `typedef`；
- `[-Werror=format=]` 与最后一行 —— 这条诊断本来由 `-Wformat` 这个开关控制（`-Wall` 里已经包含），加 `-Werror` 把它从警告升级成错误。

要打印文件内容，只有一条路：打开它、读出来、再打印读到的字节（6.5 的 `read_chunks` 就是这么做的）。

6.13 回到目录树：这一节新增了什么

现在可以对着 6.1 那棵树的**第 2 到第 6 帧**逐个点名了。本单元新增的东西一共三类：

十个例子目录（每个都是 `src/` + `include/` + 清单的最小工程）：

- `file_write_read` —— 写三行、读回三行，看 `fopen` / `fprintf` / `fputs` / `fgets` / `fclose` 的完整形状（6.8）；
- `newline_bytes` —— 同一份 `"a\nb\n"`，`"w"` 写出 6 字节、`"wb"` 写出 4 字节（6.4）；
- `fgets_long_line` —— 16 字节缓冲区把那条 52 字符的长行剁成 4 段（加上第二行一共 5 段），以及「拼回一整行」的写法（6.5）；

- `feof_trap` —— `while (!feof(fp))` 数出 4 行，正确的写法数出 3 行（6.6）；
- `fflush_crash` / `fflush_kept` —— 同一个崩溃、同一个退出码，一个 stdout 为空、一个留下了两行（6.7）；
- `mycat` —— `argc` / `argv`、用法、三种退出码、`<` 与 `>`（6.11）；
- `record_text_vs_bin` —— 同一份记录，文本 33 字节与二进制 120 字节两种存法（6.10）；
- `bad_path` —— 打开不存在的文件：`perror` 走 stderr，`return 1` 走退出码（6.3）；
- `handle_not_string` —— **故意编不过**的那一个，考「把句柄当内容」（6.12）。

四个数据文件（本书第一次出现 `.txt` / `.csv`）：`fgets_long_line/data/longline.txt`、`feof_trap/data/three_lines.txt`、`mycat/data/two_lines.txt`、`record_text_vs_bin/data/people.csv`。它们是**输入**，不是程序产物。

四个会话文件：`session-mycat.txt`、`session-newline-bytes.txt`、`session-bad-path.txt`、`session-record-sizes.txt`。正文里每一个 `term` 块都是从它们注入的，改一个字都会报 `term_does_not_match_source`。

顺带说清两件「不存在的东西」：例子跑起来会**生成** `notes.txt`、`text.txt`、`bin.txt`、`people.txt`、`people.bin` 与 `mycat.exe` 这类文件，它们出现在**运行时的当前目录**（也就是各自的例子目录），本书不把它们算作工程的组成部分；另外，本单元**没有新增任何检测器示例**——`detect` 那种「用工具抓一个错误」的演示归指针与内存那几章，这里不为凑数写假的。

工程惯例 · 工程惯例：失败路径也是要测的代码

本节十个例子里，有三个专门测「不顺利」的情况：`bad_path`（文件打不开）、`mycat` 跑不存在文件的那一段（打不开两个文件里的一个）、`fflush_crash`（进程被终止）。这不是为了好看。惯例是：

- 每个会失败的操作后面，**紧跟着就写它的处理**（本单元所有 `fopen` 后面都跟着 `if (fp == NULL)`）；
- 处理里要**说人话**：文件名叫出来、原因说出来（`perror` 就能带出系统给的说明）；
- **失败也要有一条能跑的命令去验证它**——`bad_path.json` 里写的是 `"expect": { "exit": 1, "stderrContains": [...] }`，也就是说「这条失败路径」在流水线里是**被断言过的**，不是靠我说它失败。

代价是代码长、清单多写几个字段。收益在出事那天：你手上不是「应该没问题」，而是一条能重跑的证据。

到这里，这一单元的主线就走完了：**打开 → 判断成败 → 读写 → 分清读完还是出错 → 关掉 → 用退出码交代结果。**下面把整条主线压成四句，然后五道自测题，答完再往下看下一单元。

小结 · 这一节的要点

- 你操作的不是文件，是**流**：`stdin` / `stdout` / `stderr` 一启动就有（§7.21.3），错误信息走 `stderr`；`fopen` 打不开返回 `NULL`（§7.21.5.3），`fclose` 是「先冲缓冲再关文件」（§7.21.5.2）。
- **文本模式会换算**：本机实测同一份 `"a\nb\n"`，`"w"` 写出 **6** 字节（多的 `0d` 是 CR），`"wb"` 写出 **4** 字节；量大小、按字节操作一律用 `"rb"`。
- `fgets` 的契约是「**最多读 n-1 个字符**」且**保留换行**（§7.21.7.2）；**不要** `while (!feof(fp))`（§7.21.10.2），拿返回值当条件，退出后用 `ferror` 分读完与出错。
- `printf` 写的是**缓冲区**不是磁盘（崩溃丢输出、退出码 3221225477；`fflush` 后留住了）；`fseek(SEEK_END)` + `ftell` 能量大小，但文本模式下这个数标准说「不是有意义的度量」（§7.21.9.4）；`argv[0]` 是程序名（§5.1.2.2.1），`1` / `2` 这类退出码是惯例。

五道题，覆盖事实、操作、判断三类。先自己答一遍——尤其第 3、5 题，它们考的是「标准怎么规定」与「主流怎么做」的分界，答得含糊就回上面找对应的原文。

自测题 · **自测题（先自己答，再对答案）**

1. 同一份内容 `"a\nb\n"`，用 `"w"` 与 `"wb"` 写出来各是几字节？差的那几个字节是什么？为什么你的程序读回来时感觉不到差别？
2. `while (!feof(fp)) { fgets(line, sizeof line, fp); puts(line); }` 为什么最后一行会打印两遍？正确的循环条件怎么写，循环退出后还要查哪个函数？
3. 一个程序 `printf` 了两行之后崩溃（进程被操作系统终止）。为什么有时候能在屏幕上看到那两行、有时候一行都看不到？要让它**总能**留下来，有两个办法，分别是什么？
4. 用 `fgets(buf, 16, fp)` 去读一行 52 个字符，一次调用会给你多少字节？要拿到完整的一行，写法是什么？
5. `argc` / `argv` 里，`argv[0]` 是什么？程序想表达「你参数给错了」，惯例上返回几？为什么这个数不能当成标准来依赖？

注 · **自测题答案**

1. `"w"` 是 **6** 字节、`"wb"` 是 **4** 字节（本机实测）；差的是两个 `\r`（字节 `0d`），文本模式把每个 `\n` 写成了 `\r\n`。读回来感觉不到差别，是因为输入时又折回成一个 `\n`（§7.21.3）。
2. `feof` 回答的是「**上一次**读有没有撞到末尾」，不是「下一次会不会」。读第 3 行成功后指示器还没置位，于是进了第 4 圈；这一圈 `fgets` 失败返回空指针，但循环没看返回值，`line` 里**还是上一行**，于是又打一遍。正确写法 `while (fgets(line, sizeof line, fp) != NULL)`，退出后用 `ferror(fp)` 查是不是读错了。
3. 因为 `stdout` 有**用户态缓冲**：接终端时按行缓冲，重定向或接管道时**全缓冲**。进程被操作系统终止时不会冲缓冲，字节随进程消失。要总能留下：① 写 `stderr`（标准保证它不是全缓冲，本机表现为不缓冲）；② 崩之前显式 `fflush(stdout)`。
4. 一次最多 **15** 字节（n-1，且这段里没有换行）。要拿到完整一行，就**在循环里把每段拼起来**，拼到看见 `'\n'` 为止（52 字符那一行拼出来是 53 字节，含行尾换行）。
5. `argv[0]` 是**程序自己的名字**（§5.1.2.2.1），`argv[1]` 起才是用户参数。「参数给错了」惯例返回 **2**（`1` 留给运行失败）。不能依赖它，是因为标准只钉死「0 或 `EXIT_SUCCESS` 成功、`EXIT_FAILURE` 不成功」，其余数值由实现定（§5.1.2.2.3）；脚本里只判「是不是 0」。

标准库工具箱：字符串、解析、排序与数学

学习目标 · **看完这一单元，你应该能**

- 说清 `string.h` 里哪些函数安全、哪些危险、哪些会改掉你传进去的原串，并解释 `strlen` / `strcpy` 为什么没有长度参数；
- 说出 `strncpy` 什么时候不补 `\0`，用 `offsetof` 把那次越界读**量成一个具体数字**，并说清 `strtok` 为什么不能嵌套调用；
- 解释为什么 `isalpha()` 传一个负的 `char` 是未定义行为，而本机跑起来却「看起来没事」——并说出这种「看起来没事」为什么更危险；
- 在 `scanf` / `fscanf` / `sscanf` 之间选对工具，读得懂它们的返回值、`%n` 与字段宽度，并用 `strtol` 的 `endptr` + `errno` 替掉不会报错的 `atoi`；
- 用 `snprintf` 的返回值判断截断，用 `qsort` / `bsearch` 排结构体数组，再把一个 CSV 读成结构体数组、排序、写回文件。

15.1 先看工程长什么样

学标准库最容易走偏的方式，是把函数当**条目**背（`strlen` 数长度、`strcpy` 拷贝……）。这一节我们照老规矩来：先看**文件**。原因很实际——这一章里所有的坑，都出在「函数只看得到自己那一个参数」这件事上，而这件事只有把**内存里的字节**和**真实的运行结果**摆在一起才讲得清。

本节往第 6 章的例子工程 `examples/ch06/` 里加**十四个例子目录**（上一单元已经建过一批文件 I/O 的例子，本节不碰它们）。每个例子都是「源文件放 `src/`、声明放 `include/`、怎么编译与对账写在 `<名字>.json` 清单里」的最小工程（第 1 单元 1.7 节讲过分文件的理由；从第二个项目起本书不允许单文件工程）。另外还有两组东西：**测试数据**（一份 CSV、一份空格分隔的 txt）和六段**抓下来的终端会话**。

目录树 **这一节的例子工程：在上一单元的项目里再加十四个目录**

第 1 步 **本节开始时，ch06/ 里是上一单元留下的文件 I/O 例子**

```
examples/
├── ch01/ … ch05/ ：前面五章的项目（本节不动）
└── ch06/ ：本章上一单元的项目（本节不动）
    └── 上一单元的十个例子目录 ： 文件读写、EOF、命令行参数
```

第 2 步 **先加字符串这一组；每个例子的四件套长这样**

```
examples/
├── ch01/ … ch05/ ：前面五章的项目
└── ch06/
    ├── string_ops/ ： 常用字符串函数 ← 本课新增
    │   ├── src/main.c ← 本课新增
    │   ├── src/string_ops.c ← 本课新增
    │   ├── include/string_ops.h ← 本课新增
    │   └── string_ops.json ← 本课新增
    ├── strncpy_trap/ ： 不补 \0 的那一格 ← 本课新增
    ├── strtok_reentry/ ： strtok 嵌套就坏 ← 本课新增
    ├── ctype_negative/ ： 负字符传给 ctype ← 本课新增
    └── strtol_vs_atoi/ ： 两个数字出口 ← 本课新增
```

第 3 步 **解析与格式化这一组，scanf_family 带一份数据**

```
examples/
├── ch01/ … ch05/ ：前面五章的项目
└── ch06/
    ├── sscanf_parse/ ： %n 与字段宽度 ← 本课新增
    ├── scanf_family/ ： 三兄弟各来一遍 ← 本课新增
    │   ├── src/main.c ← 本课新增
    │   ├── src/scanf_family.c ← 本课新增
    │   ├── include/scanf_family.h ← 本课新增
    │   ├── scanf_family.json ← 本课新增
    │   └── scores.txt ← 本课新增
    ├── strtol_vs_atoi/ ： 两个数字出口 ← 本课新增
    └── snprintf_bound/ ： 截断与返回值 ← 本课新增
```

第 4 步 **通用工具这一组**

```
examples/
├── ch01/ … ch05/ ：前面五章的项目
└── ch06/
    ├── math_demo/ ： sqrt / pow / fabs / round ← 本课新增
    ├── qsort_struct/ ： 排序 + bsearch ← 本课新增
    ├── rand_bias/ ： % n 的偏斜 ← 本课新增
    └── clock_timing/ ： clock() 测耗时 ← 本课新增
```

第 5 步 **综合练习与越界检测，各带自己的文件**

```
examples/
├── ch01/ … ch05/ ：前面五章的项目
└── ch06/
    ├── csv_tool/ ： 读 → 解析 → 排 → 写 ← 本课新增
    │   ├── src/main.c ← 本课新增
    │   ├── src/csv_tool.c ← 本课新增
    │   ├── include/csv_tool.h ← 本课新增
    │   ├── csv_tool.json ← 本课新增
    │   └── students.csv ← 本课新增
    └── strcpy_oob_fanalyzer/ ： 分析器抓的越界 ← 本课新增
```

第 6 步 另外留下的六段会话与两个探针

```
examples/
└── ch06/
    ├── strncpy_trap/session-warning.txt ← 本课新增
    ├── strncpy_trap/session-layout.txt ← 本课新增
    ├── strncpy_trap/probe_layout.c ← 本课新增
    ├── snprintf_bound/session-warning.txt ← 本课新增
    ├── snprintf_bound/probe_truncation.c ← 本课新增
    ├── clock_timing/session-timing.txt ← 本课新增
    ├── csv_tool/session-write.txt ← 本课新增
    └── strcpy_oob_fanalyzer/session-fanalyzer.txt ← 本课新增
```

第 1 帧是这一节故事的起点：`examples/` 下已经有前面五章的项目，`ch06/` 里躺着本章**上一单元**（文件与 I/O）留下的那一批例子——本节不碰它们。第 2 到 5 帧是本节往 `ch06/` 里加的**十四个**新例子目录；帧 2 与帧 3、5 各展开了一个例子的四件套（`src/main.c` 组装、`src/<名字>.c` 定义、`include/<名字>.h` 声明、`<名字>.json` 清单），其余目录是同样的结构。`students.csv` 与 `scores.txt` 是本单元的**输入数据**，分别只有 `csv_tool` 与 `scanf_family` 读它们。第 6 帧是另外留下的东西：六段抓下来的**会话**（每一段都对应正文里一个「本机实测」的数字）和两个**探针**。

两个探针为什么没有清单？`probe_layout.c` 的**输出**依赖栈布局与优化等级（`-O0` 与 `-O2` 结果不同），只能当「抓下来的现象」；`probe_truncation.c` 的输出虽稳定，但它要抓的是**编译期那三条警告**，而清单的 `output` 只对账 stdout。

先记住这十四个目录各自负责**一件事**，后面每一节都会点名其中一个：

- 字符串：`string_ops`（常用函数 + 边界）、`strncpy_trap`（不补 `\0`）、`strtok_reentry`（嵌套就坏）、`strcpy_oob_fanalyzer`（容量不够的三种证据）；
- 判断与解析：`ctype_negative`（负字符）、`sscanf_parse`（`%n` 与宽度）、`scanf_family`（三兄弟分工）、`strtol_vs_atoi`（谁在报错）、`snprintf_bound`（截断与返回值）；
- 通用工具：`qsort_struct`（排序与查找）、`math_demo`（数学函数）、`rand_bias`（随机数）、`clock_timing`（时间）；
- 综合练习：`csv_tool`（读 → 解析 → 排 → 写）。

工程惯例 · **工程惯例：一个结论一个例子，结论才有证据**

标准对「一个程序分几个文件」没有任何要求。本书从第二个项目起给每个例子配 `src/` + `include/` 的最小工程，理由是**每条结论都要能被单独验证**：`strncpy_trap` 只演示「不补 `\0`」这一件事，它出问题时你立刻知道是哪里错了；如果十四个结论挤在一个 `main.c` 里，你只会看到一个混乱的输出。

代价是要多写几行样板：一个 `.h` 放声明、一个 `.c` 放定义、`main.c` 只负责组装与打印。另一个代价是**两处声明要一致**：`.h` 里的原型与 `.c` 里的定义如果**类型**对不上，编译期就报 `conflicting types`（不是链接期）；参数**名字**可以不同，名字不是类型的一部分。

15.2 `string.h` 全景：谁安全、谁危险、谁改原串

这一节的表请当成工具用，不要当课文背。读的时候盯两件事：**它会不会改掉你传进去的那串字节**（第三列），**你要替它盯住什么**（第四列）——「看不看得见目标有多大」「找不到东西时怎么告诉你」都写在第四列里。

函数	它做什么	改原串吗	你要盯住什么
`strlen(s)`	数到 `\0` 为止有几个字符	不改	前提是 s 里**真的有** `\0`
`strcpy(dst, src)`	连同结尾的 `\0` 一起拷	改 dst	完全不知道 dst 多大
`strncpy(dst, src, n)`	最多拷 n 个字节	改 dst	源串不短于 n 时**不补** `\0`
`strcat(dst, src)`	把 src 接到 dst 后面	改 dst	dst 得有地方放下两串之和
`strncat(dst, src, n)`	最多接 n 个字符，并补一个 `\0`	改 dst	n 是「最多接几个」，不是「接完共几个」

函数	它做什么	改原串吗	你要盯住什么
`strcmp(a, b)`	逐字节比两个字符串	不改	返回值只有**符号**有意义
`strncmp(a, b, n)`	只比前 n 个字节	不改	常用来比前缀
`strchr` / `strrchr`	找字符，返回「从这个字符起」的尾巴	不改	找不到返回 `NULL`
`strstr(s, sub)`	找子串	不改	找不到返回 `NULL`
`strtok(s, sep)`	按分隔符切成若干段	**改 s**（把分隔符改写成 `\0`）	内部有静态状态，**不可重入**
`memcpy` / `memset`	按字节搬 / 按字节填	改目标	不看 `\0`，也不检查长度
`memcmp`	按字节比 n 个	不改	三值约定和 `strcmp` 一样

这张表里最容易被忽略的是**最后一列。** `strlen` 与 `strcpy` 的签名里没有长度参数，不是标准忘了写，而是它们的契约就是「以 `\0` 结尾的一串字节」——**长度不是参数，是数据自己带的信息。**我们把这两个函数自己写一遍，就明白这条约定了：

c examples/ch06/string_ops/src/string_ops.c 第 3–19 行

```c
size_t my_strlen(const char *s) {
    size_t n = 0;
    while (s[n] != '\0') {          /* 走到 '\0' 就停：整条规则就这一句 */ 走到 \0 就停，整条规则就这一句
        n++;
    }
    return n;
}

char *my_strcpy(char *dst, const char *src) {
    size_t i = 0;
    while (src[i] != '\0') { 一直拷到 \0 为止
        dst[i] = src[i];
        i++;
    }
    dst[i] = '\0';                  /* 结尾这个 '\0' 必须自己补上 */ 结尾这个 \0 必须自己补
    return dst;
}
```

5 走到 \0 就停，整条规则就这一句
13 一直拷到 \0 为止
17 结尾这个 \0 必须自己补

写出这两个函数，你会发现它们**只做一件事**：顺着源串一个字节一个字节地走。它们没有地方可以知道 dst 有多大——所以 `my_strcpy` 和 `strcpy` 一样危险。下面是本节这个例子的真实输出，字符串函数的边界行为都在里面：

终端输出 终端输出 string-ops-out

```
my_strlen("hello") = 5, strlen 也给 5
my_strlen("")    = 0 (空串：第一个字节就是结尾)
my_strcpy 之后 buf = "abc", buf[3] = 0 (它补了结尾)
strcmp("abc", "abd") = -1 -> 负数：左边排在前面
strcmp("abc", "abc") = 0 -> 0：两边相等
strcmp("abd", "abc") = 1 -> 正数：左边排在后面
strncmp("abc", "abd", 2) = 0 (只比前 2 个)
strchr 找 'b'     -> 尾巴 = "bcabc"
strrchr 找 'b'    -> 尾巴 = "bc"
strstr 找 "cab"   -> 尾巴 = "cabc"
strstr 找 "zzz"   -> 返回 NULL
strcat 之后 joined = "abcd"
strncat(..., 3) 之后 joined = "abcdefg", 长度 7
memset 之后 mem = "xxxxxxx" (sizeof mem = 8)
memcpy 之后 copy = "1234567"
```

几行要连起来读：

- 输出里 `my_strlen("") = 0` 那一行：空串不是「没有东西」，而是「第一个字节就是 `\0`」——所以长度是 0，函数**不会崩**；
- `strcmp` 那三行的 `-1 / 0 / 1`：`strcmp` 给出的是**三值**——负数、零、正数。本机恰好给 `-1` 和 `1`，但标准只保证**符号**（下一段的 `standard` 块有原文）；
- `strncat(..., 3) 之后 joined = "abcdefg"` 那一行：`joined` 从 `"abcd"` 变成 `"abcdefg"`——只接了 **3** 个字符。`n` 是「最多接几个」，不是「接完总共几个」，这两个数差一个（`strncat` 还会自己补一个 `\0`，而那一位不算在 `n` 里）；
- 最后一行 `memcpy 之后 copy = "1234567"`：`memcpy` 搬的是 `"1234567"` 这 **8** 个字节（含结尾的 `0`），所以 `copy` 后面有 `\0`，能当字符串打印。`memcpy` 自己不看 `\0`——那个 `\0` 只是碰巧被搬了过来。

标准原文 · **标准原文：`strcpy` 与 `strlen` 的契约**

C 语言标准（ISO/IEC 9899，C17）7.24.2.3 节「The strcpy function」：

> The strcpy function copies the string pointed to by s2 (including the terminating null character) into the array pointed to by s1.

7.24.6.3 节「The strlen function」：

> The strlen function returns the number of characters that precede the terminating null character.

大意：`strcpy` **把整条字符串连同结尾那个 `\0` 一起搬过去；`strlen` 数的是 `\0` 之前有多少个字符**（那个 `\0` 自己不算）。两句话里都出现了 `the terminating null character`（结尾的空字符）——这就是它们的全部依据：没人问「目标有几个字节」。

出处：ISO/IEC 9899:2018（C17）§7.24.2.3 第 2 段（Description）、§7.24.6.3 第 3 段（Returns）。

大白话 · **用大白话说这两句话**

把内存想成一排信箱，每个信箱装一个字节。C 的字符串就是「从某个信箱开始，一直念到某个信箱里写着 0 为止」。

- `strlen` 干的事是**数信箱**：从你指的起点开始数，数到那个写着 0 的信箱就停，**那个信箱本身不算**；
- `strcpy` 干的事是**抄信箱**：从起点开始，一个信箱一个信箱地抄到目标那边去，**连那个写着 0 的信箱也抄过去。**

所以这两个函数都**不知道自己走过了几个信箱**——它们只认那个 0。目标那边有 8 个信箱还是 8 万个，它们根本看不见。

15.3 `strncpy` 不带 `\0`：那一格谁读谁越界

知道上面那条约定，`strncpy` 的坑就很好懂了：它拷的是**字节数**，不是「字符串」。标准说它「最多拷 n 个」，装不满就用 `\0` 补足到 n 个——可**如果源串本来就比 n 长呢？** 它拷满 n 个字节就走了，**一个 `\0` 都不留。**

这个坑可以量成一个具体数字。做法是：把「没有 `\0` 的数组」和「另一个数组」放进同一个结构体，两个数组成员紧挨着。**注意「紧挨着」不能靠猜**——标准允许结构体内有未命名的填充（6.7.2.1 节），要靠 `offsetof` 量出来：

c examples/ch06/strncpy_trap/include/strncpy_trap.h 第 6–12 行

```c
/* 两个数组成员紧挨着：name 的最后一个字节之后就是 tail[0]。
   标准允许结构体内有填充，所以「挨不挨着」不能靠猜 —— 本机实测
   offsetof(struct record_buf, tail) 是 8，main 里会把这两个数打出来。 */
struct record_buf {
    char name[8];
    char tail[8];
};
```

9 结构体开头
10 8 格的数组
11 紧接着的 8 格

两个拷贝函数差在**两处**：`n` 从 `sizeof` 改成 `sizeof - 1`，再自己补一句结尾：

c examples/ch06/strncpy_trap/src/strncpy_trap.c 第 10–17 行

```c
10  void copy_unsafe(struct record_buf *r, const char *src) {
11      strncpy(r->name, src, sizeof r->name);        /* 拷满 8 格，不留结尾 */ 拷满 8 格，不留结尾
12  }
13
14  void copy_safe(struct record_buf *r, const char *src) {
15      strncpy(r->name, src, sizeof r->name - 1); 改成 n-1，留一格
16      r->name[sizeof r->name - 1] = '\0';           /* 自己补上，别指望 strncpy */ 那一格自己补 0
17  }
```

11 拷满 8 格，不留结尾
15 改成 n-1，留一格
16 那一格自己补 0

`main` 做了三件事：先量出这两个成员的位置，再把 `tail` 那 8 格预置成确定的字节，然后**照直问** `strlen` **要长度**：

c examples/ch06/strncpy_trap/src/main.c 第 10–21 行

```c
10      memset(&r, 0, sizeof r);
11      printf("sizeof r = %zu, offsetof(tail) = %zu (两格紧挨着，这是量出来的) \n", offsetof 量出两格挨着
12             sizeof r, offsetof(struct record_buf, tail));
13      copy_tail(&r, "TAILXXX");                    /* 8 个字节，最后一个是 0 */
14      printf("tail 里预置的 8 个字节: \"%s\"\n", r.tail);
15
16      copy_unsafe(&r, "abcdefghij");                /* 源串 10 个字符，n 是 8 */
17      printf("copy_unsafe 之后: \n");
18      dump_bytes("  name 里那 8 个字节", (const unsigned char *)r.name, sizeof r.name);
19      printf("  sizeof r.name = %zu, 8 个格子全都写满了\n", sizeof r.name);
20      printf("  strlen(r.name) = %zu  <- 越界: 它数到后面那格去了\n", strlen(r.name)); 它数到后面那格去了
21      printf("  printf(\"%%s\") 打出来是 \"%s\"\n", r.name); 打印也一路读过去
```

11 offsetof 量出两格挨着
20 它数到后面那格去了
21 打印也一路读过去

跑出来是这样：

终端输出 终端输出 strncpy-trap-out

```
sizeof r = 16, offsetof(tail) = 8 (两格紧挨着，这是量出来的)
tail 里预置的 8 个字节: "TAILXXX"
copy_unsafe 之后:
  name 里那 8 个字节: 61 62 63 64 65 66 67 68
  sizeof r.name = 8, 8 个格子全都写满了
  strlen(r.name) = 15  <- 越界: 它数到后面那格去了
  printf("%s") 打出来是 "abcdefghTAILXXX"
copy_safe 之后: strlen(r.name) = 7, 内容是 "abcdefg"
strncpy(pad, "hi", 8) 之后的 8 个字节: 68 69 00 00 00 00 00 00
  源串短的时候它倒是会补 0 —— 但只补到 n 个字节为止
```

四行要连起来读：

1. 第一行 `sizeof r = 16, offsetof(tail) = 8`：这是**量出来的布局**——`name` 占 0..7，`tail` 从第 8 个字节开始，中间没有空隙。（标准允许填充，所以这句话必须有这个读数撑腰，不能只靠推理。）
2. `name` 那 8 个字节是 `61 62 63 64 65 66 67 68`（`a` 到 `h`），**`strlen` 却报了 15。**多出来的 7 从哪来？就是紧挨着的 `tail` 里那 `TAILXXX` —— `strlen` 一路读过了 `name` 的边界，直到撞上 `tail[7]` 里那个 0 才停。
3. `printf("%s")` 打出的 `abcdefghTAILXXX` 是同一件事的另一种长相：**越界读的东西真的被送进了输出。**如果 `tail` 里装的是别的东西（比如你另一个变量、或者返回地址），打出来的就是别的样子——甚至直接崩。
4. 最后两行是 `strncpy` 的另一半性质：**源串比 n 短的时候它会用 0 补满**（`68 69 00 00 …`）。所以「`strncpy` 会补 `\0`」这句话只对了一半——**源串短才补，源串长就不补。**

这份输出能逐字对上，但它不是「正确答案」：`strlen(r.name) = 15` 是一次越界读的结果，它之所以每次都一样，是因为这个结构体的布局与 `tail` 的内容都被钉死了。换个结构体、少一个成员，数字就变（下面 `session-layout.txt`

那段会话就是证据）。**清单对账保证的是「这一次长这样」，正文要讲的是「你不该依赖它」。**

标准原文 · 标准原文：`strncpy` 与 `strncat` 各自补不补 `\0`

C 语言标准（ISO/IEC 9899，C17）7.24.2.4 节「The strncpy function」：

> The strncpy function copies not more than n characters (characters that follow a null character are not copied) from the array pointed to by s2 to the array pointed to by s1. If the array pointed to by s2 is a string that is shorter than n characters, null characters are appended to the copy in the array pointed to by s1, until n characters in all have been written.

大意：`strncpy` **最多拷 n 个字符**（`\0` 之后的字符不拷）；**只有当源串比 n 短时，才补** `\0` **直到凑满 n 个。**反过来说：源串不短于 n 时，它**一个** `\0` **都不会补**——这正是上面那个 `tail` 被读到的原因。

同一标准 7.24.3.2 节「The strncat function」：

> The strncat function appends not more than n characters (a null character and characters that follow it are not appended) from the array pointed to by s2 to the end of the string pointed to by s1. … A terminating null character is always appended to the result.

大意：`strncat` **最多接 n 个字符，并且总会补一个结尾。**两个函数的 `n` 含义不同，`\0` 的待遇也不同——这就是为什么 `strncat` 的 `n` 是「最多接几个」，而结果的长度最多是「原来的长度 + n」。

出处：ISO/IEC 9899:2018（C17）§7.24.2.4 第 2 段、§7.24.3.2 第 2 段。

第 1 帧是结构体的布局：`name` 与 `tail` 之间**没有空隙**（本机实测 `offsetof` 是 8）。第 2 帧是 `copy_unsafe` 干的事：8 格全写成 `a`..`h`，**一个 0 都没留。**第 3 帧是 `strlen` 的视角：它不知道 `name` 只有 8 格，它只认 0，于是数出 **15。**

⚠ 陷阱 · 「`strncpy` 更安全」是最常见的一句误传

你会看到什么：你老老实实用 `strncpy(dst, src, sizeof dst)` 代替 `strcpy`，用本书的基础参数（`-O0`）编译，一个字都不报，程序多数时候也正常。

你会怎么误判：你以为「带了 `n` 就是安全版本」，于是把 `strcpy` 全换掉，心里松了一口气。

真相：`n` 只是**写多少字节的上限**，它跟「结果是不是一条合法字符串」是两件事。源串长度 ≥ n 时结果没有结尾，此时**任何一个读字符串的函数都会越界读**——`strlen`、`printf("%s")`、`strcmp`、`strcat` 全会。

编译器帮不上多少忙：本机的 `-Wall` 只在「源串写在眼前、长度定死」这种形状上给 `-Wstringop-truncation`；源串是个参数时它看不见长度。**但它也不是完全不说话**：同一份代码用 `-O2` 编译，即使源串是参数，`-Wall` 也会报 `'strncpy' specified bound 8 equals destination size`——**警告的有无取决于优化等级，别把「`-O0` 下没警告」当成「没问题」。**

正解：要「最多拷 n-1 个字节，然后自己补 0」（`copy_safe` 就是），或者干脆用 `snprintf(dst, sizeof dst, "%s", src)`——它保证结尾，还顺便给你「本来要写多少」这个数（15.6 节）。

源码里那三条警告长这样（探针把源串写死了，所以三条都报了）：

终端会话 编译器唯一能帮你的场合：源串写在眼前 ch06-strncpy-warn

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never probe_layout.c -o layout_o0.exe 2>&1
probe_layout.c: In function 'lone_array':
probe_layout.c:9:5: warning: 'strncpy' output truncated copying 8 bytes from a string of length 10 [-
Wstringop-truncation]
    9 |     strncpy(name, "abcdefghij", sizeof name);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
probe_layout.c: In function 'neighbor_after':
probe_layout.c:17:5: warning: 'strncpy' output truncated copying 8 bytes from a string of length 10 [-
Wstringop-truncation]
   17 |     strncpy(name, "abcdefghij", sizeof name);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
probe_layout.c: In function 'big_neighbor':
probe_layout.c:25:5: warning: 'strncpy' output truncated copying 8 bytes from a string of length 10 [-
Wstringop-truncation]
   25 |     strncpy(name, "abcdefghij", sizeof name);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

读法只有一句：`output truncated copying 8 bytes from a string of length 10`——**源串 10 个字符，你只让它拷 8 个**，所以结尾肯定没地方放。编译器能算出这个前提（两个字面量都写在眼前）才报得出来；换成变量，它就只能沉默。

同一个坑的另一面：`strcpy` 写越界的三种证据

`strncpy` 是「读越界」，`strcpy` 是「写越界」——比你手里的目标大，就往后面写。这类问题在不同工具下会呈现三种完全不同的证据，正好可以练一遍「谁在说话」：

c examples/ch06/strcpy_oob_fanalyzer/src/main.c 第 6–14 行

```c
int main(void) {
    char id[8]; 目标只有 8 字节

    /* 这一句就是分析器要抓的形状：目标的大小在眼前看得见（8），
       源串的长度也是定死的（16 个字符 + 结尾）。 */
    strcpy(id, "0123456789ABCDEF"); 源串 17 个字节（含结尾）

    printf("strcpy 之后，id 那 8 个字节：\n");
    dump_hex("  id", (const unsigned char *)id, sizeof id);
```

7 目标只有 8 字节

11 源串 17 个字节（含结尾）

这个例子在清单里写的是 `"mode": "detect"`、`"detect": "fanalyzer"`——也就是让编译器用 `-fanalyzer` 做一遍**静态分析**。下面这段是抓下来的会话原文：

终端会话 第 1 种证据：-fanalyzer 静态分析（编译器退出码仍是 0） ch06-fanalyzer-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -fanalyzer -Iinclude src/main.c
src/copy_ops.c -o app.exe 2>&1 | head -n 15
src/main.c: In function 'main':
src/main.c:11:5: warning: stack-based buffer overflow [CWE-121] [-Wanalyzer-out-of-bounds]
   11 |     strcpy(id, "0123456789ABCDEF");
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  'main': events 1-2
    7 |     char id[8];
      |          ^~
      |          |
      |          (1) capacity: 8 bytes
......
   11 |     strcpy(id, "0123456789ABCDEF");
      |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
      |     |
      |     (2) ⚠  out-of-bounds write from byte 8 till byte 16 but 'id' ends at byte 8
src/main.c:11:5: note: write of 9 bytes to beyond the end of 'id'
```

它把整条推理链摆出来了：`(1) capacity: 8 bytes`（第 7 行的 `id` 有 8 个字节）→ `(2) out-of-bounds write from byte 8 till byte 16 but 'id' ends at byte 8`（第 11 行那句从第 8 个字节开始写、一路写到第 16 个）。注意最后那个 `note`：**`write of 9 bytes to beyond the end of 'id'`**——17 个字节里有 9 个写在了边界之外。

终端会话 （续）第 2、3 种证据：没有分析器时的警告，与真跑一次的退出码 ch06-fanalyzer-2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/copy_ops.c
-o app.exe; echo "编译器退出码 $?"
src/main.c: In function 'main':
src/main.c:11:5: warning: '__builtin_memcpy' writing 17 bytes into a region of size 8 overflows the
destination [-Wstringop-overflow=]
   11 |     strcpy(id, "0123456789ABCDEF");
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~
src/main.c:7:10: note: destination object 'id' of size 8
    7 |     char id[8];
      |          ^~
编译器退出码 0
$ { ./app.exe; } 2>/dev/null; echo "退出码 $?"
退出码 139
$ rm -f app.exe
```

三件事要分开记：

- **去掉 `-fanalyzer` 之后还有一条警告**（`-Wstringop-overflow=`，`-Wall` 带来的）——但它的措辞与角度都不同：它说的是「`__builtin_memcpy` 要写 17 个字节」，而分析器说的是「写进了一个 8 字节的对象、越界 9 个字节」。**两条都不是错误**：`编译器退出码 0`，可执行文件照样产出了。
- **真跑一次的结果是退出码 139。** `139` 是 bash 报的数（128 + 11，信号编号 11 就是 SIGSEGV）；同一个程序不经 bash 包装时，本机量到的退出码是 **3221225477**（`0xC0000005`，Windows 的「访问冲突」）。**同一个程序、两种退出码，只是报数的人不同。**
- 这一次**程序一行输出都没有。**因为 stdout 进管道是带缓冲的，进程被打死时缓冲区还没来得及刷。**想留下「崩溃之前打印过什么」，得显式 `fflush(stdout)`。**

实验记录 · **本机实测：越界的三种证据，各自由谁给出**

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- **证据一（编译器视角）**：`-fanalyzer` 报 `stack-based buffer overflow [CWE-121] [-Wanalyzer-out-of-bounds]`，**编译退出码 0**；清单里用 `"mode": "detect"` + `"detect": "fanalyzer"` 声明要抓的片段，闸门会去掉 `-fanalyzer` 再编译一遍做对照——本机实测不加参数时 `stack-based buffer overflow`、`CWE-121`、`-Wanalyzer-out-of-bounds` 三个片段一个都不出现，所以这条证据确实归 `-fanalyzer`；
- **证据二（基础警告）**：不加 `-fanalyzer` 时 `-Wall` 给出 `-Wstringop-overflow=` 那条，编译退出码仍是 0；
- **证据三（运行期）**：`./app.exe` 退出码 **139**（bash），不经 bash 时是 **3221225477**，stdout 与 stderr 都是空的（缓冲区没刷）。
- 这三条各自都不是「崩溃的原因说明」：前两条只是编译器提前算出来的，第三条才是真的动手。

工程惯例 · **工程惯例：什么时候该信哪条证据**

- **写着常量、长度在眼前**：`-Wall` 就会报（`-Wstringop-overflow` 这类）。顺手把警告清掉，成本最低；
- **要提前发现「变量绕了几圈」的越界**：`-fanalyzer` 能跟一段流程（它能把 `main` 调用的那个函数一路跟进去），代价是慢，而且有误报——**它是编译器的看法，不是运行结果**；
- **运行期真的崩了**：只有退出码与 stderr 是真的。本机没有 ASan / UBSan（`TOOLCHAIN.md` §4.1），所以「越界读到了哪个值」这类结论**只能靠程序自己打印出来**——就像 15.3 节那张结构体布局的做法。

为什么 15.3 节要绕一圈用结构体：因为单纯的局部数组 `char name[8]` 后面那格是什么，由编译器决定。本机实测（会话 `session-layout.txt`）：同一份危险写法，`-O0` 下三个变体分别读到 11、15、27（连同一份代码里换一个局部变量都会变），`-O2` 下三个全变成 14。**这种数字不能当结论，只能当现象。**

终端会话 同一个坑：换个编译等级，strlen 读到的地方就变了 ch06-strncpy-layout

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g probe_layout.c -o layout_o0.exe 2>/dev/null
$ ./layout_o0.exe; ./layout_o0.exe
变体 A (name 是唯一的局部数组) : strlen(name) = 11
变体 B (name 前面摆了另一块) : strlen(name) = 15
变体 C (name 前面摆了 16 字节) : strlen(name) = 27
变体 A (name 是唯一的局部数组) : strlen(name) = 11
变体 B (name 前面摆了另一块) : strlen(name) = 15
变体 C (name 前面摆了 16 字节) : strlen(name) = 27
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g probe_layout.c -o layout_o2.exe 2>/dev/null
$ ./layout_o2.exe; ./layout_o2.exe
变体 A (name 是唯一的局部数组) : strlen(name) = 14
变体 B (name 前面摆了另一块) : strlen(name) = 14
变体 C (name 前面摆了 16 字节) : strlen(name) = 14
变体 A (name 是唯一的局部数组) : strlen(name) = 14
变体 B (name 前面摆了另一块) : strlen(name) = 14
变体 C (name 前面摆了 16 字节) : strlen(name) = 14
$ rm -f layout_o0.exe layout_o2.exe
```

这段会话是本节最要紧的一段证据：**同一份源码、同一个编译器，只是换个优化等级，越界读到的地方就从 11/15/27 变成 14/14/14。**它说明的不是「`-O0` 更危险」，而是——**这件事根本不由你定。**这就是未定义行为的样子：不是「一定崩」，也不是「一定没事」，而是**结果不在你的控制范围里。**所以你在 `strncpy` 之后补上那一句 `dst[n-1] = '\0'`，不是「保险起见」，是**把不确定变成确定。**

`strtok`：它改你的原串，还把「读到哪」藏在自己肚子里

`strtok` 是 `string.h` 里唯一的「分段」函数：给一串文本和一个分隔符集合，它每次还你一段。它有两个性质，凑在一起就成了经典事故：

1. **它改你的原串**：找到分隔符就地改写成 `\0`。所以传进去的必须是**可写的**缓冲区，不能是字符串字面量；
2. **「下一次从哪开始」不存在你的变量里**，而是存在函数内部的**静态状态**里。你写 `strtok(NULL, ",")`，它就去翻自己肚子里那个位置——**下一次调用会覆盖它。**

第 2 条就是「不可重入」的意思：**同一个函数正在被外层用着，内层又调了一次，外层的进度就丢了。**这不是理论问题，写两层解析时必然撞上：

c examples/ch06/strtok_reentry/src/strtok_reentry.c 第 16-31 行

```c
void split_nested_strtok(const char *text) {
    char buf[64];

    copy_in(buf, sizeof buf, text);

    char *line = strtok(buf, "\n");   外层先切一行
    while (line != NULL) {
        printf("  行: %s\n", line);
        char *field = strtok(line, ",");   内层把同一个静态位置挪走了
        while (field != NULL) {
            printf("    字段: %s\n", field);
            field = strtok(NULL, ",");
        }
        line = strtok(NULL, "\n");    /* 外层的进度已经被上面那个循环改掉了 */   这里已经没有「上一行之后」了
    }
}
```

21 外层先切一行
24 内层把同一个静态位置挪走了
29 这里已经没有「上一行之后」了

终端输出 终端输出 strtok-reentry-out

```
数据是 "a,1\nb,2"，两行、每行两个字段
一、两层都用 strtok（错写法）：
  行：a,1
    字段：a
    字段：1
  只处理了第一行就结束了 —— 第二行凭空消失
二、两层各存一个指针（对写法）：
  行：a,1
    字段：a
    字段：1
  行：b,2
    字段：b
    字段：2
  两行、四个字段，一个不少
```

第 6 行那句「只处理了第一行」是全篇最值得记住的现象：**程序没有报错、没有崩溃、退出码 0**，它只是悄悄少做了一半的活。原因在第 29 行：内层循环结束时，`strtok` 肚子里的位置已经指到 `buf` 的末尾（内层一直切到字符串结束），外层再来一句 `strtok(NULL, "\n")`，拿到的就是「没有下一段」——`NULL`，循环结束。

正解：每一层自己拿一个指针记住「下一段在哪」——名字叫 `next_line` / `next_field`，各存各的。例子的第二半（`split_nested_manual`）就是这么写的，用 `strchr` 找分隔符、自己把它改成 `\0`，**不依赖任何函数的内部状态。**这也正是 POSIX 的 `strtok_r`（多一个 `char **saveptr` 参数）在做的事——它是扩展，不是 ISO C（15.9 节会讲这条边界）。

15.4 `ctype.h`：负字符是未定义行为，而本机看不出来

`<ctype.h>` 这一组函数（`isalpha` / `isdigit` / `isspace` / `toupper` …）看起来最无害：传一个字符，问一句「你是不是字母」。但它们的参数**有一个必须遵守的约定**：

c examples/ch06/ctype_negative/src/main.c 第 7–14 行

```c
    char c = (char)0xE9;          /* 一个字节，值是 233；装进 char 里就成了负数 */ 值是 233，装进 char 成了负数

    printf("(int)c = %d —— char 在本机是有符号的，0xE9 变成了 %d\n", (int)c, (int)c);
    printf("isalpha(c)                  = %d\n", isalpha(c));  错：传的是负数
    printf("isalpha((unsigned char)c)   = %d\n", isalpha((unsigned char)c));  对：先转 unsigned char
    printf("isdigit(c)                  = %d\n", isdigit(c));  这一句也是错的
    printf("toupper(c)                  = %d（这一次原样还回来，但这次调用本身就是 UB）\n",  这一句同样是错的
           toupper(c));
```

7 值是 233，装进 char 成了负数
10 错：传的是负数
11 对：先转 unsigned char
12 这一句也是错的
13 这一句同样是错的

注意第 10、12、13 行三句都是错写法：`isalpha` / `isdigit` / `toupper` 受的是同一条约束，把负的 `char` 交给它们中的任何一个都是未定义行为，不只是 `isalpha`。第 11 行才是对的。

终端输出 终端输出 ctype-negative-out

```
(int)c = -23 —— char 在本机是有符号的，0xE9 变成了 -23
isalpha(c)                  = 0
isalpha((unsigned char)c)   = 0
isdigit(c)                  = 0
toupper(c)                  = -23（这一次原样还回来，但这次调用本身就是 UB）
isalpha('A') = 1, isalpha('a') = 2 —— 非 0 就是真，具体数值由实现决定
isalpha(EOF) = 0 (EOF 是唯一允许传进去的负值)
0x80..0xFF 这 128 个字节里，被认成字母的个数：
  char 直接传        : 0 个
  先转 unsigned char : 0 个
"a1 中"（6 个字节）里：字母 1，数字 1，其它 4
```

这份输出要说三件事：

1. **本机什么都没发生。** `0xE9` 装进 `char` 里是 **-23**，把它直接交给 `isalpha` 返回 0，程序正常退出，连一条警告都没有。例子里还把这 128 个高位字节（`0x80`..`0xFF`）全试了一遍：**两种写法都是 0 个。这就是最危险的地方**——你写错了，机器不告诉你。
2. **返回非 0 不等于返回 1。**本机 `isalpha('A')` 是 **1**、`isalpha('a')` 是 **2**。所以判断只能写 `if (isalpha(u))`，**不能写** `if (isalpha(u) == 1)`。
3. **本机的 `char` 是有符号的**（`(int)c` 是 -23 而不是 233）。这不是标准规定的——标准把「`char` 有没有符号」交给实现决定；本机选择了有符号。

标准原文 · **标准原文：`ctype` 函数的参数**

C 语言标准（ISO/IEC 9899，C17）7.4.1 节「Character classification functions」：

> In all cases the argument is an int, the value of which shall be representable as an unsigned char or shall equal the value of the macro EOF. If the argument has any other value, the behavior is undefined.

大意：**这个参数的类型是 `int`，但它的值必须「能用 `unsigned char` 表示」或者「等于 `EOF`」；别的值一律未定义。** `unsigned char` 能表示的范围是 0..255（本机 `UCHAR_MAX` = 255，实测），所以负的 `char`（比如 -23）**不在允许的范围里。**标准给的合法写法只有两种：`isalpha((unsigned char)c)` 与 `isalpha(EOF)`。

同节第 2 段还有一句要紧的：**「The behavior of these functions is affected by the current locale.」**——这些函数的行为受当前 locale 影响。本节的程序没有调用 `setlocale`，所以处在默认的 `"C"` locale；换一个 locale，`0x80..0xFF` 里哪些字节算字母就可能变。**「`ctype.h` 只认 ASCII」这句话只在 `"C"` locale 下成立**，这一点后面还会说。

出处：ISO/IEC 9899:2018（C17）§7.4.1 第 1、2 段。

陷阱 · **「本机没崩」不是「这么写可以」**

你会看到什么：`isalpha(c)` 传负 `char`，本机返回 0、程序正常跑完、编译零警告。你会觉得「那就是没问题呗」。

你会怎么误判：你把这段代码当成正确写法复制到下一个项目里，甚至认为标准里那句「未定义」是危言耸听。

真相：`isalpha` 这类函数的实现通常是一张**查表**。合法的做法是先把参数转成 0..255 再查表；而传负数进去，就是在查表的**负数下标**上读——本机这一次恰好读到 0，别的实现（或同一个实现换一个版本）读到的是别的字节，结果是随机的真/假，也可能当场崩。**未定义行为的定义恰恰是「不保证任何一种结果」**，「这一次恰好正常」也是它的一种表现。

正解：处理**字节**时永远先转 `unsigned char`；只有处理**字符流**（`fgetc` / `getchar` 的返回值）时才写 `EOF` 判断，因为那时变量本身就是 `int`——循环写成「读到的值先存进一个 `int`，跟 `EOF` 比，比得过再 `(unsigned char)` 转一次交给 `isalpha`」。本节例子里 `classify` 是它的字符串版（下面就有），15.10 节的 CSV 读取是同一套习惯的按行版。

这个「先转 `unsigned char`」的习惯在例子的另一处直接用上了：`classify` 把 `"a1 中"` 拆成字母 / 数字 / 其它，`中` 的 3 个字节全部落进「其它」——

c examples/ch06/ctype_negative/src/ctype_negative.c 第 5–19 行

```c
void classify(const char *text, size_t *letters, size_t *digits, size_t *other) {
    *letters = 0;
    *digits = 0;
    *other = 0;
    for (size_t i = 0; text[i] != '\0'; i++) {
        unsigned char u = (unsigned char)text[i];   /* 先转，再问 */ 先转成 0..255，再问
        if (isalpha(u)) { 返回值只能当真假用
            (*letters)++;
        } else if (isdigit(u)) {
            (*digits)++;
        } else {
            (*other)++;
        }
    }
}
```

10 先转成 0..255，再问
11 返回值只能当真假用

输出最后一行 `"a1 中"（6 个字节）里：字母 1，数字 1，其它 4` 就是它的结果：**1 个字母、1 个数字、4 个「其它」**（一个空格 + `中` 的 3 个字节）。顺带说一句：`isalpha` 问的是**字节**，不是「字符」——UTF-8 里的汉字由三个字节组成，在默认的 `"C"` locale 下任何一个都不会被认成字母。**「这是个字母吗」这个问题，`ctype.h` 在 `"C"` locale 下只按 ASCII 回答；换 locale 结论就可能变（见上面 `standard` 块）。**

15.5 解析三兄弟与两个数字出口

解析就是「把一段文本里的数字、名字、分隔符拆出来」。标准库里干这件事的有两组工具：`scanf` 一家，和 `strtol` 一家。

`scanf` / `fscanf` / `sscanf` 的分工

函数	从哪读	什么时候用
`scanf(fmt, ...)`	标准输入	交互式小程序
`fscanf(fp, fmt, ...)`	你给的 `FILE *`	按格式读文件
`sscanf(s, fmt, ...)`	一个**字符串**	解析「已经在你手里的一行文本」——**首选**

三者的格式串与返回值规则完全一样，区别只在**输入源**。返回值是**成功赋值的字段个数**，这句话要逐字读：它数的是**字段**，不是字符。

- 全部顺利：返回字段数（下面例子里是 2 或 3）；
- 中途失败：返回**已经成功的那几个**（比如只读到 key，就返回 1）；
- 一上来就失败（或碰到输入末尾）：返回 `EOF`（`stdio.h` 里的一个负整数常量）。

所以「读成功了吗」必须写成 `if (sscanf(...) == 期望的字段数)`——**拿 `!= EOF` 当成功判据是错的**：返回 1 也不是 EOF，但那次解析只拿到了一半。

三兄弟各跑一遍，差别就具体了。这个例子的标准输入是 `42` 与 `abc` 两行，`scores.txt` 里有四行（第三行故意写坏）：

c examples/ch06/scanf_family/src/main.c 第 5–13 行

```c
int main(void) {
    printf("一、scanf：从标准输入读，返回值是「成功赋值的字段数」\n");
    int a = 0;
    int b = 0;
    int r1 = scan_one_int(&a);          /* 先读进来，再打印：别把调用塞进 printf 的参数里 */ 先调用、把结果存下来，再打印
    printf("  第一次 scanf(\"%d\") = %d，读到 %d\n", r1, a);
    int r2 = scan_one_int(&b); 这一次返回 0
    printf("  第二次 scanf(\"%d\") = %d（下一个东西不是数字，一个字段都没赋值）\n", r2);
```

```
13      printf("  那个不是数字的东西还留在输入流里，下一次读会再撞上它\n");
```

9 先调用、把结果存下来，再打印
11 这一次返回 0

c examples/ch06/scanf_family/src/main.c 第 15-30 行

```c
15      printf("二、fscanf：从文件读，格式对不上就停在那一行\n");
16      FILE *fp = fopen("scores.txt", "r");
17      if (fp == NULL) {
18          printf("  打不开 scores.txt\n");
19          return 1;
20      }
21      char name[16];
22      int value = 0;
23      int got = 0;
24      int rows = 0;
25      while ((got = read_pair(fp, name, sizeof name, &value)) == 2) {  循环条件就是「两个字段都读到了」
26          printf("  读到 %s = %d\n", name, value);
27          rows++;
28      }
29      printf("  读到 %d 条之后，fscanf 返回 %d（只读到了名字，分数没读到）\n", rows, got);  退出时 got 是 1，不是 EOF
30      fclose(fp);
```

25 循环条件就是「两个字段都读到了」
29 退出时 got 是 1，不是 EOF

终端输出 终端输出 scanf-family-out

```
一、scanf：从标准输入读，返回值是「成功赋值的字段数」
  第一次 scanf("%d") = 1，读到 42
  第二次 scanf("%d") = 0（下一个东西不是数字，一个字段都没赋值）
  那个不是数字的东西还留在输入流里，下一次读会再撞上它
二、fscanf：从文件读，格式对不上就停在那一行
  读到 alpha = 11
  读到 beta = 22
  读到 2 条之后，fscanf 返回 1（只读到了名字，分数没读到）
三、sscanf：从字符串读，%n 告诉你读掉了几个字符
  sscanf("  77 rest", "%d%n") = 1，v = 77，读掉 4 个字符
```

四行输出值得逐条对照：

- `第一次 scanf("%d") = 1，读到 42`：一个字段**成功赋值**，所以返回 **1**；
- `第二次 scanf("%d") = 0`：输入流下一个东西是 `abc`，转不成整数，**一个字段都没赋值**，返回 **0**。注意 `b` 保持原来的 0——**0 这个返回值不是「读到了 0」**；
- `读到 2 条之后，fscanf 返回 1`：`scores.txt` 第三行是 `bad`，`%15s` 成功读到 `bad`（一个字段），`%d` 失败，所以返回 **1**——`1` **正是「半个成功」的信号**；循环条件写 `== 2` 才停在这里；
- `sscanf(" 77 rest", "%d%n") = 1，v = 77，读掉 4 个字符`：**前导空格不算**，`77` 两个字符加前面两个空格一共 4 个（`%n` 报告的是「一共耗掉几个字符」，空格也算）。

三个函数的返回值语义完全一致，所以那段 `while (... == 2)` 换成 `fscanf` / `sscanf` 都成立——这正是把「解析」与「从哪读」分开的好处。

`%n` 与字段宽度：解析不靠猜

`sscanf` 有两个特别好用、又常被忘掉的工具：**字段宽度**（`%4d`、`%15[^=]`）说「这里最多读几个字符」；`%n` 说「到目前一共读掉了几个字符」——它本身**不计进返回值**。有了 `%n`，「这行还剩什么没读」就是一个可以算出来的数，不用猜：

c examples/ch06/sscanf_parse/src/sscanf_parse.c 第 5-16 行

```c
5   int parse_key_value(const char *line, char *key, size_t keysize, long *value, int *consumed) {
6       /* 现场拼格式串：%[^=] 读一串「不是 = 的字符」，宽度用 keysize-1 卡住，免得写爆；
```

```c
        %n 记下「读到这儿一共用掉了几个字符」，它不计进 sscanf 的返回值。 */
    char fmt[32];
    snprintf(fmt, sizeof fmt, "%%%zu[^=]=%%ld%%n", keysize - 1); 现场拼格式串

    int got = sscanf(line, fmt, key, value, consumed); %n 不计进返回值
    if (got != 2) { 凑不齐两个字段就是失败
        return -1;              /* 只读到 key (1) 或什么都没读到 (0)  */
    }
    return 0;
}
```

9 现场拼格式串
11 %n 不计进返回值
12 凑不齐两个字段就是失败

这里拼出来的格式串对 `keysize = 16` 就是 `%15[^=]=%ld%n`：`%15[^=]` 的意思是「最多读 15 个**不是** `=` 的字符」——那个 `15` 是**宽度**，不是「预留 15 字节」，它必须比目标缓冲区少 1，好给结尾 `\0` 留位置（和第 15.3 节的 `strncpy` 是同一个道理）。`main` 用它试了四种输入，另外单独演示了宽度的两种含义：

c examples/ch06/sscanf_parse/src/main.c 第 21–30 行

```c
    printf("二、字段宽度说的是「最多读几个字符」\n");
    int y = 0, m = 0, d = 0, used = 0;
    int got = sscanf("2026-10-18", "%4d-%2d-%2d%n", &y, &m, &d, &used); 宽度 4 说的是「最多读 4 个字符」
    printf("  sscanf(\"2026-10-18\", \"%%4d-%%2d-%%2d%%n\") = %d, 得到 %d-%d-%d, 读掉 %d 个字符\n",
           got, y, m, d, used);

    int x = 0, used2 = 0;
    int got2 = sscanf("12345", "%2d%n", &x, &used2); %2d 只吃两个字符
    printf("  sscanf(\"12345\", \"%%2d%%n\") = %d, x = %d, 读掉 %d 个字符（宽度封顶，后面不管）\n",
           got2, x, used2);
```

23 宽度 4 说的是「最多读 4 个字符」
28 %2d 只吃两个字符

终端输出 终端输出 sscanf-parse-out

```
一、%[^=] + %ld + %n 拆 key=value
  "key=42" -> key="key" value=42; 读掉 6 个字符，剩下 ""
  "key=42   # 后面还有话" -> key="key" value=42; 读掉 6 个字符，剩下 "   # 后面还有话"
  "key=abc" -> 失败：sscanf 连两个字段都没凑齐
  "=42" -> 失败：sscanf 连两个字段都没凑齐
二、字段宽度说的是「最多读几个字符」
  sscanf("2026-10-18", "%4d-%2d-%2d%n") = 3, 得到 2026-10-18, 读掉 10 个字符
  sscanf("12345", "%2d%n") = 1, x = 12, 读掉 2 个字符（宽度封顶，后面不管）
三、返回值只数「成功赋值的字段」，%n 不算进去
  sscanf("key=abc", "%15[^=]=%ld%n") = 1（只给了 key）
  %n 那个变量还是 -1 —— 中间的 %ld 没成功，%n 根本没轮到执行
```

四行 `key=value` 的输入给出的结论：

- `"key=42"` 那一行 `读掉 6 个字符，剩下 ""`：`key=42` 一共 6 个字符，全读完了；
- `"key=42 # 后面还有话"` 那一行 `剩下 " # 后面还有话"`：**这就是 `%n` 的价值**——它告诉你有 6 个字符被消费，剩下的那一截原样还在。于是「这行后面有没有多余的东西」可以判断，不用靠数手指；
- 两条 `失败：sscanf 连两个字段都没凑齐` 的原因其实不同：`key=abc` 是 `%ld` 撞上了字母，`=42` 是 `%15[^=]` 一个字符都匹配不到。**两次的返回值分别是 1 和 0**，但按「等于 2 才算成功」的判据，两次都被拦住了；
- 最后一节演示 `%n` 的另一个性质：**`%n` 是「执行到才算」**。`%ld` 失败后 `%n` 根本没轮到，所以那个变量保持你初始化的 `-1`。**给它一个「不可能是真结果」的初值，你才能分辨「没执行」和「读到第 0 个字符」。**

`atoi` 与 `strtol`：一个出口对三个出口

`atoi` 的签名是 `int atoi(const char *s)`——**它只能还你一个数。**这就意味着它没有地方告诉你「这次转换其实失败了」。`strtol` 有三个出口：返回值、`endptr`（读到哪停的）、`errno`（有没有超出范围）。

c examples/ch06/strtol_vs_atoi/src/strtol_vs_atoi.c 第 6–25 行

```c
int parse_with_atoi(const char *text) {
    return atoi(text);                  /* 只有这一个出口，出错也看不出来 */
}

int parse_with_strtol(const char *text, long *out, long *used) {
    errno = 0;                          /* strtol 只会往 errno 里加，不会替你清零 */ errno 要自己清零
    char *end = NULL;
    long v = strtol(text, &end, 10);

    *used = (long)(end - text);         /* 读掉了几个字符：end 是「停在哪」 */ end 与起点之差就是读掉的字符数
    *out = v;

    if (end == text) { end 没动过 = 一个数字都没读到
        return PARSER_NODIGIT;          /* end 没动过：连一个数字都没读到 */
    }
    if (errno == ERANGE) { 超出范围时 errno 是 ERANGE
        return PARSER_RANGE;            /* 超出 long 的范围，值被夹到了端点 */
    }
    return PARSER_OK;
}
```

11 errno 要自己清零
15 end 与起点之差就是读掉的字符数
18 end 没动过 = 一个数字都没读到
21 超出范围时 errno 是 ERANGE

四种输入跑出来的差别是这样的：

终端输出 终端输出 strtol-atoi-out

```
先看 atoi —— 它只有一个返回值：
  atoi("42") = 42
  atoi("12abc") = 12
  atoi("999999999999") = 2147483647
  atoi("abc") = 0
再看 strtol —— 返回值、end 指针、errno 三个出口：
  "42" -> v = 42, 读掉 2 个字符, 剩下 "", 结论：解析成功
  "12abc" -> v = 12, 读掉 2 个字符, 剩下 "abc", 结论：解析成功
  "999999999999" -> v = 2147483647, 读掉 12 个字符, 剩下 "", 结论：超出了 long 的范围
  "abc" -> v = 0, 读掉 0 个字符, 剩下 "abc", 结论：一个数字都没有
LONG_MAX = 2147483647, ERANGE = 34
注意 "999999999999" 那一行：atoi 和 strtol 给出的都是 2147483647，
光看数字看不出它已经出错了 —— 只有 errno 这一个出口能告诉你。
```

三个坑全在这份输出里：

1. `"12abc"`：**两个函数都返回 12**，谁也没报错。可 `strtol` 多给了你一句「读掉 2 个字符，剩下 `abc`」——**要不要接受「数字后面跟着垃圾」，是你的策略，不是库替你决定的**；
2. `"999999999999"`：**两个函数都给 `2147483647`**（本机 `long` 是 32 位，`LONG_MAX` 就是这个数）。这个值看起来**特别像**正确答案，其实它是个**端点**。区别只在一个地方：`strtol` 把 `ERANGE`（=34）写进了 `errno`，`atoi` 什么都没留下。**这一行是「本机现象」**：`atoi` 在出错时的行为标准没有规定（7.22.1.2 节只说「除了出错时的行为，等价于 `(int)strtol(nptr, NULL, 10)`」），换个库给别的数甚至崩掉都不奇怪——**正因为它连「出错」都不可依赖，才不该用它**；
3. `"abc"`：`atoi` **给 0**，和 `atoi("0")` 一模一样。**「0 这个值」和「解析失败」在 `atoi` 这里无法区分**——这就是「别用 `atoi`」的全部理由。

标准原文 · **标准原文：`strtol` 怎么告诉你出错了**

> C 语言标准（ISO/IEC 9899，C17）7.22.1.4 节「The strtol, strtoll, strtoul, and strtoull functions」：
>
> > If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of nptr is stored in the object pointed to by endptr, provided that endptr is not a null pointer. If the correct value is outside the range of representable values, LONG_MIN, LONG_MAX, LLONG_MIN, LLONG_MAX, ULONG_MAX, or ULLONG_MAX is returned …, and the value of the macro ERANGE is stored in errno.
>
> 大意：**「能认的数字」如果是空的或者形状不对，它就什么也不转，并把 `endptr` 指回起点**——这就是「一个数字都没读到」的判据（`end == text`）；**如果结果超出表示范围，它返回 `LONG_MAX` / `LONG_MIN` 这样的端点值，并把 `ERANGE` 存进 `errno`**。注意标准把「值」和「状态」分开放在两个出口里，这就是为什么你必须同时看 `endptr` 和 `errno`。
>
> 出处：ISO/IEC 9899:2018（C17）§7.22.1.4。

⚠ 陷阱 · **`atoi` 不会报错——而你以为它会**

你会看到什么：`atoi("999999999999")` 给你 `2147483647`，`atoi("abc")` 给你 `0`，编译零警告、程序正常跑完。

你会怎么误判：你看见一个正常的整数，就认为「解析成功了」，于是把 `atoi` 用在一个解析「配置文件里的数字」的小工具里。等到有人往文件里写了一个脏值，你的程序**不报错、不退出、继续拿着一串错的数字往下算**——这是最难查的一类故障。

真相：标准里 `atoi` 的定义只有一句话——「除了出错时的行为，等价于 `(int)strtol(nptr, NULL, 10)`」（7.22.1.2 节）。`NULL` **就是关键**：它连 `endptr` 都不要，所以「读到哪停的」这件事在 `atoi` 里被丢掉了；而 `int` 这个返回类型也装不下「错误」这个额外状态。

正解：要解析数字就写一个像 `parse_with_strtol` 这样的小函数，把 `endptr` 与 `errno` 都检查一遍，**返回值用「返回码」而不是「数字」**。代价是调用点变长几行；收益是脏数据会在**第一次遇到时**就现形。

15.6 `snprintf`：`n` 是缓冲区总长，返回值是「本来要写多少」

`snprintf` 是 C 库里最容易被误解的一个函数，因为**两个数字的含义都不是直觉上的那个**：

- 参数 `n` = **缓冲区一共有多少个字节**（含结尾的 `\0`），不是「可以写的字符数」；
- 返回值 = **假如地方够大，本来会写多少个字符**（不含结尾的 `\0`），**不是实际写了几个。**

c **被调用的那两个函数：一个写、一个只量** examples/ch06/snprintf_bound/src/snprintf_bound.c 第 5–11 行

```c
int write_line(char *buf, size_t bufcap, const char *name, int id) {
    return snprintf(buf, bufcap, "%s-%d-very-long-tail", name, id);
}

int measure_line(const char *name, int id) {
    return snprintf(NULL, 0, "%s-%d-very-long-tail", name, id);
}
```

`measure_line` 用的是 `snprintf` 的一个标准技巧：**缓冲区传 `NULL`、长度传 0，它什么都不写，只把长度算给你。**有了这个数，你就能一次分配出大小刚好的内存，不用先猜一个「应该够大」的数。

c examples/ch06/snprintf_bound/src/main.c 第 14–24 行

```c
int main(void) {
```

```
15      char small[8];
16      memset(small, '#', sizeof small);          /* 先全部弄脏，再看它改了哪几格 */
17      int n = write_line(small, sizeof small, "item", 42); 8 字节的缓冲区，内容远超它
18
19      printf("snprintf 的返回值 n = %d (不是 7) \n", n); 返回值 22 不是 7
20      printf("缓冲区里现在是 \"%s\", strlen = %zu, sizeof = %zu\n",
21             small, strlen(small), sizeof small);
22      dump_hex("那 8 个字节", (const unsigned char *)small, sizeof small); 十六进制看清它写了几格
23      printf("要装得下，缓冲区至少要有 %d 个字节\n", n + 1); 要装下得 n+1 个字节目
24      printf("先量后写：measure_line 也是 %d，一次就能算出该留多少\n", measure_line("item", 42));
```

17 8 字节的缓冲区，内容远超它
19 返回值 22 不是 7
22 十六进制看清它写了几格
23 要装下得 n+1 个字节目

终端输出 终端输出 snprintf-bound-out

```
snprintf 的返回值 n = 22 (不是 7)
缓冲区里现在是 "item-42", strlen = 7, sizeof = 8
那 8 个字节：69 74 65 6D 2D 34 32 00
要装得下，缓冲区至少要有 23 个字节
先量后写：measure_line 也是 22，一次就能算出该留多少
换成 32 字节的缓冲区：m = 22，内容是 "item-42-very-long-tail", strlen = 22
两次返回值一样吗？一样：返回值描述的是内容多长，与缓冲区多大无关
```

- `n = 22`：`item-42-very-long-tail` 是 22 个字符。**这个数和缓冲区多大完全无关**——换成 32 字节的缓冲区，返回值还是 22（倒数第三行）；
- 缓冲区里是 `item-42`（7 个字符），第 8 格是 `00`：`snprintf` 把 7 个字符写完，**在第 8 格（下标 7）放了结尾 `\0`**，然后停手。`sizeof` 是 8、`strlen` 是 7，这两个数在 15.2 节已经分过了；
- `要装得下，缓冲区至少要有 23 个字节`：`n + 1`。判据照着标准那句话写（返回非负**且**小于 `n`），所以代码里是 `if (n < 0 || n >= (int)sizeof buf) { /* 被截断了，或者出错了 */ }`——少了 `n < 0` 那一半，编码错误时会被误判成「完整」。

标准原文 · **标准原文：`snprintf` 的返回值是什么**

C 语言标准（ISO/IEC 9899，C17）7.21.6.5 节「The snprintf function」：

> ... the snprintf function returns the number of characters that would have been written had n been sufficiently large, not counting the terminating null character, or a negative value if an encoding error occurred. Thus, the null-terminated output has been completely written if and only if the returned value is nonnegative and less than n.

大意：**返回值是「假设 `n` 足够大，本来会写多少个字符」**（不数结尾的 `\0`）；**只有当返回非负、并且小于 `n` 时，输出才被完整写下**——这句话就是「截断判据」的出处。

出处：ISO/IEC 9899:2018（C17）§7.21.6.5。

这个坑同样会在编译器那里露头——但只在**源串与目标都写在眼前**的时候：

终端会话 编译器算得比你快：23 个字节要塞进 8 个格子 ch06-snprintf-warn

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never probe_truncation.c -o app.exe
probe_truncation.c: In function 'main':
probe_truncation.c:7:46: warning: '-very-long-tail' directive output truncated writing 15 bytes into a region
of size 1 [-Wformat-truncation=]
    7 |     int n = snprintf(buf, sizeof buf, "%s-%d-very-long-tail", "item", 42);
      |                                              ^~~~~~~~~~~~~~~
probe_truncation.c:7:13: note: 'snprintf' output 23 bytes into a destination of size 8
    7 |     int n = snprintf(buf, sizeof buf, "%s-%d-very-long-tail", "item", 42);
      |             ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
$ ./app.exe
n = 22, buf = "item-42"
$ rm -f app.exe
```

注意那条 `note` 里的数字：`23 bytes into a destination of size 8`——正是「返回值 22 + 结尾那个 `\0`」。编译器（`-Wformat-truncation`，属于 `-Wall`）把这件事在编译期就算清楚了。**但它只在格式串与缓冲区都写在同一个函数里时才算得出来**：本节的 `snprintf_bound` 把 `snprintf` 放进 `write_line` 里、把源串当参数传，编译器看不到长度，就一个字也不说。**别把「没警告」当成「没问题」。**

15.7 `qsort` / `bsearch`：比较器的三值约定

`qsort` 的签名里没有「怎么比」这个信息——它的第四个参数是一个**函数指针**，你把「怎么比」交给它。于是就有了全书最重要的一条约定：**比较器返回负数 / 零 / 正数，分别表示「第一个参数排在前面 / 两者等价 / 第一个参数排在后面」**。它和 `strcmp` 是同一套三值语义（所以按名字排序时，直接把 `strcmp` 的结果交出去就行）：

c examples/ch06/qsort_struct/src/qsort_struct.c 第 6–16 行

```c
int cmp_by_score(const void *a, const void *b) {
    const struct student *x = a;
    const struct student *y = b;
    return (x->score > y->score) - (x->score < y->score);  // 相减不行，比大小再相减才行
}

int cmp_by_name(const void *a, const void *b) {
    const struct student *x = a;
    const struct student *y = b;
    return strcmp(x->name, y->name);  // 直接借用 strcmp 的三值
}
```

9 相减不行，比大小再相减才行
15 直接借用 strcmp 的三值

第 9 行那个写法值得多看一眼：`(x > y) - (x < y)`。两个比较各给出 0 或 1，相减的结果只能是 `-1`、`0`、`1`——**它保证不溢出**（下面那个 `trap` 讲为什么这件事要紧）。两个参数都是 `const void *`，所以函数体里第一件事是把它们转回 `struct student *`（`bsearch` 传进来的 key 也是同一个类型，所以同一个比较器两边都能用）。

c examples/ch06/qsort_struct/src/main.c 第 16–25 行

```c
    qsort(arr, n, sizeof arr[0], cmp_by_score);  // 排序：数组、元素个数、元素大小、比较器
    printf("按分数升序：\n");
    print_all(arr, n);

    qsort(arr, n, sizeof arr[0], cmp_by_name);  // 换一个比较器就是换一种排法
    printf("按名字升序：\n");
    print_all(arr, n);

    struct student key = {"sun", 0};
    struct student *hit = bsearch(&key, arr, n, sizeof arr[0], cmp_by_name);  // bsearch 要求数组已经按同一个 key 排好
```

16 排序：数组、元素个数、元素大小、比较器
20 换一个比较器就是换一种排法
25 bsearch 要求数组已经按同一个 key 排好

终端输出 终端输出 qsort-struct-out

```
排之前：
lin:88 zhao:95 qian:72 sun:95 li:60 zhou:81
按分数升序：
li:60 qian:72 zhou:81 lin:88 sun:95 zhao:95
按名字升序：
li:60 lin:88 qian:72 sun:95 zhao:95 zhou:81
在这个按名字排好的数组里找 "sun"：找到，分数是 95
找 "wang"：返回 NULL
换成「按分数」的比较器再找一次 "sun"：返回 NULL（数组不是按分数排的，二分查找不认）
```

最后三行是 `bsearch` 的三条规矩，一条比一条重要：

- 在**按名字排好的**数组里找 `"sun"`：找到，`bsearch` 返回的是**指向数组里那个元素的指针**（所以 `hit->score` 是 95）；
- 找不存在的 `"wang"`：返回 `NULL`——`bsearch` **不告诉你「它应该在哪个位置」**，它只回答「在不在」；
- 同一个数组、把比较器换成「按分数」：返回 `NULL`。`bsearch` **要求数组已经按你给的那个比较器排好序**（它就是二分查找，靠「一半一定在左边」这个前提一步步砍区间）。数组没按这个 key 排，结果就是没意义的——这一次它恰好返回 `NULL`，但**这是运气，不是保证。**

⚠ **陷阱** · **比较器里写 `return a - b;` 会出事**

你会看到什么：`return x->score - y->score;` 在小数据上完全正常，排序结果跟 `(a > b) - (a < b)` 一模一样。

你会怎么误判：你看到它跑得对，就默认两种写法等价，从此一直用减法。

真相：两个 `int` 相减的结果可能**超出 `int` 能表示的范围**。本机 `INT_MAX` 是 2147483647、`INT_MIN` 是 -2147483648（实测：`sizeof(int)` 与 `sizeof(long)` 都是 4），两者的**数学差**是 **4294967295**——这个值在本机连 `long` 都装不下（`long` 也只有 4 字节），而在 `int` 里直接就溢出了，有符号整数溢出是**未定义行为**。于是排序算法拿着一个随机的符号继续砍区间，可能排出一个**没排好的数组**，也可能崩。

正解：永远用「比两次再相减」的写法（`(a > b) - (a < b)`），或者用显式的 `if` 分支。代价是多写几个字符；收益是这个比较器对**任何**输入都成立。`strcmp` 自己就是这么实现的——它也只保证符号。

📖 **标准原文** · **标准原文：比较器的约定与「平局的顺序」**

C 语言标准（ISO/IEC 9899，C17）7.22.5 节「Searching and sorting utilities」：

> The function shall return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second. If two elements compare as equal, their order in the resulting sorted array is unspecified.

大意：**比较器只看符号**（小于 / 等于 / 大于）；而且**两个「相等」的元素，排完之后谁在前面是不指定的**——`qsort` 不保证稳定。所以平局要自己定规则：本节 `csv_tool` 里的比较器在分数相同时会再按名字比一次（15.10 节会看到 `sun` 与 `zhao` 同为 95 分时的处理）。

出处：ISO/IEC 9899:2018（C17）§7.22.5.2 第 4、5 段（`qsort`；`bsearch` 在 §7.22.5.1）。

15.8 数学、随机与时间：三个常用头

- **`<math.h>`：参数与返回值都是 `double`**

c **三个真实的用法：斜边、近似相等、归到最近的刻度** examples/ch06/math_demo/src/math_demo.c 第 5–15 行

```c
double hypotenuse(double a, double b) {
    return sqrt(a * a + b * b);
}

int is_close(double a, double b, double eps) {
    return fabs(a - b) < eps;
}

double snap_to_step(double x, double step) {
    return round(x / step) * step;
}
```

c examples/ch06/math_demo/src/main.c 第 7–21 行

```c
    printf("sqrt(2.0)            = %.10f\n", sqrt(2.0));
    printf("hypotenuse(3.0, 4.0) = %.1f (3-4-5 直角三角形) \n", hypotenuse(3.0, 4.0));
    printf("pow(2.0, 10.0)       = %.1f\n", pow(2.0, 10.0));  pow 的两个参数都是 double
    printf("fabs(-3.5)           = %.1f\n", fabs(-3.5));
    printf("floor(-1.5)          = %.1f, ceil(-1.5) = %.1f\n", floor(-1.5), ceil(-1.5));
    printf("round(2.5)           = %.1f, round(-2.5) = %.1f (往远离 0 的那边) \n",
```

```c
            round(2.5), round(-2.5));
    printf("snap_to_step(7.3, 0.5) = %.1f\n", snap_to_step(7.3, 0.5));
    printf("0.1 + 0.2 == 0.3 ?     %d\n", 0.1 + 0.2 == 0.3); 浮点数不能直接比相等
    printf("is_close(0.1+0.2, 0.3, 1e-9) = %d (浮点数要这样比) \n",
            is_close(0.1 + 0.2, 0.3, 1e-9));
    printf("sqrt(-1.0)           = %f (不是一个数) \n", sqrt(-1.0)); 负数开方得到 NaN
    printf("sqrt(-1.0) == 自己 ?   %d (NaN 连自己都不等) \n", sqrt(-1.0) == sqrt(-1.0)); NaN 连自己都不等于自己
    printf("sizeof(sqrt(2.0))    = %zu (返回的是 double, 别往 int 里塞) \n",
            sizeof(sqrt(2.0)));
```

9 pow 的两个参数都是 double
15 浮点数不能直接比相等
18 负数开方得到 NaN
19 NaN 连自己都不等于自己

终端输出 终端输出 math-demo-out

```
sqrt(2.0)            = 1.4142135624
hypotenuse(3.0, 4.0) = 5.0 (3-4-5 直角三角形)
pow(2.0, 10.0)       = 1024.0
fabs(-3.5)           = 3.5
floor(-1.5)          = -2.0, ceil(-1.5) = -1.0
round(2.5)           = 3.0, round(-2.5) = -3.0 (往远离 0 的那边)
snap_to_step(7.3, 0.5) = 7.5
0.1 + 0.2 == 0.3 ?     0
is_close(0.1+0.2, 0.3, 1e-9) = 1 (浮点数要这样比)
sqrt(-1.0)           = -nan(ind) (不是一个数)
sqrt(-1.0) == 自己 ?   0 (NaN 连自己都不等)
sizeof(sqrt(2.0))    = 8 (返回的是 double, 别往 int 里塞)
```

几处要留意的：

- **`floor` / `ceil` / `round` 作用在负数上**：`floor(-1.5) = -2.0`（往下）、`ceil(-1.5) = -1.0`（往上）、`round(-2.5) = -3.0`（往**远离 0** 的那边，不是「四舍五入到 -2」）；
- **`0.1 + 0.2 == 0.3` 是 0**：浮点数不能用 `==` 比相等，要用「差值小于一个容忍值」，也就是 `fabs(a - b) < eps`（第 10 行给出 1）；
- **开负数得到 NaN**：本机打印出来是 `-nan(ind)`（这个写法是这台机器的库的写法，别的地方可能印 `nan`）。更要记住的是 `sqrt(-1.0) == sqrt(-1.0)` 是 **0**——`NaN` 连它自己都不等于，所以**判 NaN 要写 `x != x`**；
- `sizeof(sqrt(2.0))` 是 **8**：数学函数的参数与返回值都是 `double`。写成 `sqrt(2)` 也行（`2` 会被转成 `2.0`），但把结果往 `int` 里塞会**截断**（`sqrt(2)` 塞进 `int` 就是 1）。

实验记录 · 本机实测：链接数学函数不需要 `-lm`

- 工具链：gcc 16.1.0（`-std=c17`，即 C17），目标三元组 x86_64-w64-mingw32；
- 本节所有含 `<math.h>` 的例子都**只用**清单里那串基础参数编译：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude …`，**没有加** `-lm`，编译退出码 0；
- 原因：在这台机器上（MinGW / UCRT），数学函数就在默认链接的 C 运行库里，不在一个单独的 `libm` 里；
- **但脚本里写上 `-lm` 不亏**：Linux 上的 glibc / musl 把 `libm` 单独放在一个库里，少这一项你会看到 `undefined reference to 'sqrt'`（macOS 的数学函数在系统库里，不需要）。**这是一个「换机器就翻车」的差异，属于工程惯例，不是标准的事。**

`<stdlib.h>` 的 `rand` / `srand`：伪随机、可复现、有偏斜

`rand()` 产生的是**伪**随机数：它内部有一个状态，每次调用按固定公式推进一步。所以**同一个种子（`srand` 的实参）在同一个实现上必然给出同一串数**——这是特性，不是缺陷：调试的时候你正需要它。（标准只规定 `rand` 返回 0.. `RAND_MAX` 之间的伪随机数、`srand` 设种子，**没有规定用哪个公式**；本机这串 `41 18467 …` 是 UCRT 的序列，换一个库就换一串。）

c examples/ch06/rand_bias/src/main.c 第 12–20 行

```c
    printf("RAND_MAX = %d\n", RAND_MAX);

    srand(20261018); 固定种子：这一串数就定下来了
    sample_mod(6, counts, DRAW); 抽一万次，落进 6 个桶
    printf("rand() %% 6 抽 %d 次的分布：", DRAW);
    for (int i = 0; i < 6; i++) {
        printf(" %d", counts[i]);
    }
    printf("\n");
```

14 固定种子：这一串数就定下来了
15 抽一万次，落进 6 个桶

终端输出 终端输出 rand-bias-out

```
RAND_MAX = 32767
rand() % 6 抽 10000 次的分布：  1661 1658 1710 1640 1679 1652
rand() % 100 抽 10000 次：最少的桶 80 次，最多的桶 136 次
rand() % 30000 的结果里，小于 2768 的那些占了 1710 / 10000
如果每个结果一样容易出现，这个数应该在 922 附近
真相：0..2767 每个结果能被两条不同的 rand() 值撞出来，天生更重
srand(1) 之后前 5 个：  41 18467 6334 26500 19169
再 srand(1) 一次   ：  41 18467 6334 26500 19169
srand(2) 之后前 5 个：  45 29216 24198 17795 29484
```

- 第一行 `RAND_MAX = 32767`：本机 `rand()` 只会给出 0..32767 之间的数（标准只要求 `RAND_MAX` **至少** 32767）；
- `rand() % 6` 抽一万次：六个桶分别是 1661 / 1658 / 1710 / 1640 / 1679 / 1652，**都在 1666 附近晃**——这就是 `% 6` 的日常样子，看不出问题；
- `rand() % 100` 抽一万次：最少的桶 80 次、最多的 136 次。**这是个陷阱式的读数**：波动看着大，但它主要是**抽样波动**（一万次、每桶期望 100，标准差约 10）；
- 真正的病在 `rand() % 30000` 那一行：**结果里小于 2768 的占了 1710 次，而「公平」应该只有 922 次。**这不是波动，是**结构性偏斜**：`2768 = 32768 - 30000`，于是 `0..2767` 这些结果可以被**两条**不同的 `rand()` 值撞出来（比如 `0` 与 `30000` 都是 `% 30000 == 0`），别的结果只能被一条撞出来。`% n` **的偏斜大小取决于** `n` **与** `RAND_MAX + 1` **的差**，`n` 越接近 `RAND_MAX + 1`，偏斜越狠；
- 最后三行是**可复现性**：`srand(1)` 两次给出的前 5 个数一模一样（`41 18467 6334 26500 19169`），换成 `srand(2)` 就完全不同（`45 29216 24198 17795 29484`）。

怎么避开偏斜：不要用 `% n`，用「超出范围的采样丢掉重来」**（拒绝采样）——先算出「**`RAND_MAX + 1`** 里能整除 **`n`** 的最大区间」，落在区间外的丢掉再抽一次。代价是循环次数不定；收益是每个结果真的等概率。另外两条工程常识：**`rand()` 的质量不适合做密码学用途**（要随机数和安全有关，用平台提供的专门接口），**`rand()` **也不保证线程安全。**

`<time.h>`：`time` 是墙上时间，`clock` 是处理器时间

两者差一个字，用途完全不同：

- `time(NULL)` 给你 `time_t`——**本机**这个数是从 1970-01-01 起过的秒数（Unix epoch）。这是**实现事实**，不是标准保证：标准只说「编码未指定」（7.27.2.4 节）。**它每次运行都不一样**，所以它只能用来记录「什么时候发生的」；
- `clock()` 给你 `clock_t`：进程用掉的处理器时间，单位是 `CLOCKS_PER_SEC` 分之一个「时钟」。**它适合测一段代码花了多久。**

c examples/ch06/clock_timing/src/main.c 第 18–22 行

```c
18      time_t t0 = time(NULL);          /* 墙上时间：精度是一秒 */ 墙上时间：精度一秒
19      clock_t start = clock();         /* 处理器时间：这张表从程序开始跑算起 */ 处理器时间：开始
20      qsort(data, N, sizeof data[0], cmp_int);
21      clock_t end = clock();
22      time_t t1 = time(NULL); 同一段代码的两种读数
```

18 墙上时间：精度一秒
19 处理器时间：开始
22 同一段代码的两种读数

接着把两次读数相减；差值只在带参数时才打印（不带参数时输出必须是可复现的）：

c examples/ch06/clock_timing/src/main.c 第 27–36 行

```c
27      double secs = (double)(end - start) / (double)CLOCKS_PER_SEC; 秒数 = 时钟差 ÷ CLOCKS_PER_SEC
28      printf("CLOCKS_PER_SEC = %ld\n", (long)CLOCKS_PER_SEC);
29      printf("这一次 clock() 差是 %s\n", secs > 0.0 ? "一个正的数" : "0（小于一个时钟单位）");
30      printf("具体秒数每次运行都不同，所以它不能进输出块（要与真实 stdout 逐字对账）；\n");
31      printf("真实的几次读数在正文的会话记录里。\n");
32      if (argc > 1 && strcmp(argv[1], "--timing") == 0) { 只有带 --timing 才打印真读数
33          /* 命令行带 --timing 时才打印真读数：正文里那段会话就是这样抓的 */
34          printf("[带参数] clock 差 = %ld 个时钟单位（约 %.3f 秒）；time() 差 = %lld 秒\n", 两种计时并排
35                 (long)(end - start), secs, (long long)(t1 - t0));
36      }
```

27 秒数 = 时钟差 ÷ CLOCKS_PER_SEC
32 只有带 --timing 才打印真读数
34 两种计时并排

终端输出 终端输出 clock-timing-out

```
N = 200000，排序前前三个：5406 25372 4005
排序前校验和 = 3278681043
排序后第一个（最小）= 0，最后一个（最大）= 32767
排序后校验和 = 3278681043（与排序前一样，说明只是换了位置）
CLOCKS_PER_SEC = 1000
这一次 clock() 差是 一个正的数
具体秒数每次运行都不同，所以它不能进输出块（要与真实 stdout 逐字对账）；
真实的几次读数在正文的会话记录里。
time(NULL) 给出的数大于 0 吗？是（它每次运行都不一样）
strftime 写出来：2026-10-18 09:30:00（只由那个 struct tm 决定，可复现）
```

`N = 200000`、校验和 `3278681043` 两次一模一样——**这是「排序只是换了位置」的证据**（输出块的第 2 行与第 4 行）。而**耗时那一行没有出现在输出块里，这是故意的**：它每次都不一样，而本书的输出块要与真实 stdout **逐字对账。**要留下真实读数，只能像下面这样抓一段会话：

终端会话 真读数：同一段排序，clock() 是 20 个时钟单位，time() 是 0 秒 ch06-clock-timing

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c
src/clock_timing.c -o app.exe
$ ./app.exe --timing | grep 带参数
[带参数] clock 差 = 20 个时钟单位（约 0.020 秒）；time() 差 = 0 秒
$ ./app.exe --timing | grep 带参数
[带参数] clock 差 = 20 个时钟单位（约 0.020 秒）；time() 差 = 0 秒
$ ./app.exe --timing | grep 带参数
[带参数] clock 差 = 20 个时钟单位（约 0.020 秒）；time() 差 = 0 秒
$ rm -f app.exe
```

这一段就是 `clock()` 与 `time()` 的对照实验：同一段排序，`clock()` 量出 20 个时钟单位（本机 `CLOCKS_PER_SEC = 1000`，所以约 **0.020 秒**），`time()` 两次读数**一模一样**——差 0 秒。不是排序不花时间，而是 `time()` 的精度是**一秒**，20 毫秒在它眼里就是「没发生」。**要测毫秒级的东西只能用 `clock()`；要记录「这件事发生在哪一天几点」，只能用 `time()`。**

两个数的关系不是巧合：`CLOCKS_PER_SEC` 是「一秒分成多少个时钟」，所以「秒数 = 时钟差 ÷ `CLOCKS_PER_SEC`」。`clock_t` 的精度就是**一个时钟单位**（本机 1 毫秒），拿它测微秒级的事情同样测不准。

`strftime` **是这一族里唯一可复现的**：它只是把一个 `struct tm` 按格式写成字符串，跟「现在是几点」无关。两个坑写在例子里：`tm_year` 要从 **1900** 起算（写 `2026 - 1900`），`tm_mon` 从 **0** 起算（10 月写 `9`）。所以那一行 `2026-10-18 09:30:00` 是可以进输出块的——**它只由结构体决定，不由时钟决定。**

15.9 常用头一览与 ISO C / POSIX 的边界

标准库不止「字符串 + 解析 + 排序」。剩下这四个头你会在每个真实项目里碰到：

头文件	提供什么	本节的例子里何处用到
`<stddef.h>`	`size_t`、`ptrdiff_t`、`NULL`、`offsetof`	`string_ops.h`、`strncpy_trap.h`、`csv_tool.h` 等用它拿 `size_t`；`strncpy_trap` 用 `offsetof` 量出了成员偏移（15.3 节的读数）
`<stdbool.h>`	`bool`、`true`、`false`	本节没有用到——C 里判断条件本来就是「非 0 即真」，`bool` 只是让人读起来更清楚
`<stdint.h>`	`int32_t`、`uint64_t`、`intptr_t`…**宽度写死在名字里**的整数类型	本节没有用到——但只要你写二进制格式、协议或哈希，就该用它而不是 `int`
`<assert.h>`	`assert(条件)`：条件不成立就打印并终止	本节没有用到——它适合「这里绝对不该发生」的内部检查

三条使用纪律：

- `size_t` **是无符号的**：`for (size_t i = n - 1; i >= 0; i--)` 这种写法会**死循环**（`i` 减到 0 之后再减一次变成极大值）。要倒着走就写成 `for (size_t i = n; i-- > 0; )`；
- `assert` **会在** `NDEBUG` **下整句消失**：里面的**副作用也不会执行**（`assert(++i > 0)` 是错的写法），所以它只放「不该发生的检查」；
- `offsetof` **是宏，不是函数**：`offsetof(struct record_buf, tail)` 给出 `tail` 距离结构体开头的字节数。15.3 节能用「两个数组成员紧挨着」推算出越界读到哪，靠的就是这个布局事实。

工程惯例 · 工程惯例：ISO C 与 POSIX 的分界

标准库分两层，混起来用就会遇到「在我机器上能编，在别人机器上不行」：

- **ISO C（标准规定，本书用的就是它）**：`<string.h>`、`<stdlib.h>`、`<stdio.h>`、`<ctype.h>`、`<math.h>`、`<time.h>`…换编译器、换操作系统都在；
- **POSIX（操作系统给的扩展，不是 C 标准的一部分）**：`strdup`、`strcasecmp`、`getline`、`strtok_r`、`fileno`…**在 Linux/macOS 上默认就能用，在 Windows 的 MSVC 上直接找不到。**

三组最典型的例子：

- `strdup(s)`：帮你 `malloc` 一块内存并把 `s` 拷进去。ISO C 里没有它，C 的正统写法是「`malloc(strlen(s) + 1)` + `memcpy`」，或者就用 `snprintf`（15.6 节）避开这层手工活；
- `strcasecmp(a, b)`：忽略大小写的 `strcmp`。ISO C 没有，跨平台要自己写（转小写再比，用 `<ctype.h>` 的 `tolower`）；
- `getline(&line, &cap, fp)`：读一整行、自动扩容，比 `fgets` 好用得多。它是 POSIX 2008 的，ISO C 里没有——**本节的 CSV 读取用的就是纯 ISO C 的** `fgets` **+ 手工切**，多写十几行，换来的是任何 C 工具链都能编。

判断方法：随手一搜发现这个函数「在 Linux 上人人都用」，就再确认一句它在 `man` 手册里的章节号——POSIX 的手册在 **3p** 章，C 标准库在 **3** 章。**跨平台代码只用「C 标准 + 你明确选定的目标平台扩展」。**

15.10 综合练习：CSV 进来，CSV 出去

前面九节的零件在这里装成一个真正的小工具：**读一个 CSV → 解析成结构体数组 → 按分数排序 → 写回另一个 CSV。**数据流的每一步对应前面某一节：

图 / 搜索过程 **CSV → 记录数组 → 排序 → 写回：每一步用到哪一节的东西**

第 1 步 起点是一份文本文件

students.csv ：数据 + 注释 + 空行　fgets ：一次读一行

第 2 步 每行切一刀，数字那一半交给 strtol

strchr(line, ',') ：切第一刀 ← 本步新增　strtol(num, &end, 10) ：值、停在哪、errno ← 本步新增

line ⇒ 判断 通过才收下

第 3 步 收下的行落进结构体数组

rows[0..6] ：7 条记录 ← 本步新增　struct student ：名字 + 分数

strtol ⇒ rows[0..6] 通过才留下

第 4 步 qsort 按分数重排，平局按名字

rows[0..6] ：同分按名字定序 ← 本步新增　cmp_by_score_desc ：三值比较器

第 5 步 写回文件

fprintf ：写回 "名字,分数"

*rows[0..6] ⇒ sorted_out.csv 表头 + 7 行

第 1 帧是一切的起点：文件在盘上，程序要一行一行把它读进来。第 2 帧是**最要紧的一帧**——每一行都过五道检查（空行或 `#` 注释、有没有逗号、`strtol` 读到了数字没有、数字后面还有没有粘着别的东西、名字装不装得下），任何一道不过就整行跳过。第 3 帧是数据落进结构体数组：`rows` 是**静态数组**（`static struct student rows[MAX_ROWS]`）——它不占栈，也不需要 `malloc`，对 32 条记录这种规模是最省事的选择。第 4 帧是排序，注意平局的处理。第 5 帧是写回。

数据结构就三样，全部放在头文件里：

c **记录、上限、和一本「读盘的账」** examples/ch06/csv_tool/include/csv_tool.h 第 6–18 行

```c
#define MAX_ROWS 32

struct student {
    char name[16];
    int score;
};

/* 读盘的账：读了几行、收下几条、跳过几行 */
struct csv_stats {
    size_t lines;
    size_t kept;
    size_t skipped;
};
```

解析那一段是本节前面所有工具的交汇处，值得逐行读：

c examples/ch06/csv_tool/src/csv_tool.c 第 36–57 行

```c
        char *comma = strchr(line, ',');
        if (comma == NULL) {
            st->skipped++;                  /* 没有逗号：这行不是我们要的形状 */
            continue;
        }
        *comma = '\0';                      /* 一刀两断：左边是名字，右边是分数 */ 一刀两断：左边名字右边分数
        const char *num = comma + 1;
        while (*num == ' ') { 跳过数字前面的空格
            num++;                          /* 跳过数字前面的空格 */
        }

        errno = 0; errno 要自己清零
        char *end = NULL;
        long v = strtol(num, &end, 10); 数字与「读到哪」
        if (end == num || errno == ERANGE) {
```

```
51              st->skipped++;                  /* 一个数字都没读到，或者超出了范围 */
52              continue;
53          }
54          if (*end != '\0' && *end != ' ') {  数字后面粘着别的东西，整行不要
55              st->skipped++;                  /* 数字后面还粘着别的东西 */
56              continue;
57          }
```

41 一刀两断：左边名字右边分数
43 跳过数字前面的空格
47 errno 要自己清零
49 数字与「读到哪」
54 数字后面粘着别的东西，整行不要

三处细节对应前面三节：

- `*comma = '\0'` 是**就地改字符串**——和 `strtok` 的做法一样（把分隔符改写成 `\0`），区别是这里只切**第一刀**，而且改的是自己 `fgets` 读来的缓冲区，不是别人的字面量；
- `errno = 0` 那一句是 15.5 节讲过的规矩：`strtol` 只会往 `errno` 里写，不会替你清零；
- `if (*end != '\0' && *end != ' ')` 是**你的策略，不是库的策略**：它说「数字后面要么到头、要么是空格；别的一律不接受」。数据文件里那条 `wu,88abc` 就是被这一句拦下的。

终端输出 终端输出 csv-tool-out

```
读 students.csv：一共读了 11 行，收下 7 条，跳过 4 行
文件里的顺序：
zhao:95 qian:72 sun:95 li:60 zhou:81 lin:88 chen:91
按分数从高到低（同分按名字）：
sun:95 zhao:95 chen:91 lin:88 zhou:81 qian:72 li:60
已经把结果写进 sorted_out.csv（1 行表头 + 7 行数据）
```

11 行里收下 7 条——另外 4 行是 2 行注释、1 个空行、以及那条数字不合法（`88abc`）的 `wu`。第 3 行是文件里的原始顺序，第 5 行是排完之后；`sun` 与 `zhao` 都是 95 分，按比较器的第二条规则（名字升序）排成 `sun` 在前。

写完的文件长什么样，抓下来看一眼：

终端会话 跑一遍，再把写回的文件 cat 出来 ch06-csv-write

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/csv_tool.c
-o app.exe
$ ./app.exe | tail -n 4
zhao:95 qian:72 sun:95 li:60 zhou:81 lin:88 chen:91
按分数从高到低（同分按名字）：
sun:95 zhao:95 chen:91 lin:88 zhou:81 qian:72 li:60
已经把结果写进 sorted_out.csv（1 行表头 + 7 行数据）
$ cat sorted_out.csv
name,score
sun,95
zhao,95
chen,91
lin,88
zhou,81
qian,72
li,60
$ rm -f app.exe
```

`cat` 出来的 8 行是**真正的磁盘内容**——这就是「写回了」的证据。`sorted_out.csv` 是程序生成的产物（跑一次覆盖一次），所以它不属于本节的例子文件清单，只看这段会话。

工程惯例 · **工程惯例：CSV 工具里刻意没做的三件事**

真实的数据里 CSV 有三件事很难，本节**故意不做**，你要知道它们的代价：

1. **引号**：`"Smith, John",88` 里的逗号不是分隔符。要处理它得写一个状态机（在不在引号里是两态），不能再用 `strchr` 切第一刀；
2. **表头**：本节把第一行表头**当普通数据行**处理（`score` 不是数字，会被跳过），所以账里那 4 行「跳过」含了表头这一类东西。真实工程里通常先读一行、比对表头、再决定列的次序；
3. **转义与编码**：「逗号用 `\,` 转义」「字段里有换行」这些变体各有各的规矩（RFC 4180 是最常见的一份）。**本节只认最简那一档**：一行一条记录、一个逗号、名字不含逗号。

还是要说那句：**先写一个只认最简格式的版本，让它跑通再说。**上面三条每一条都是独立的一小步，别在第一版里就全塞进去。

15.11 这一节新增了什么，以及接下来

回到 15.1 那棵树：现在你可以对着**第 6 帧**逐个点名了。

- `examples/ch06/` 下的**十四个例子目录**，逐个点名：`string_ops`（常用函数与边界）、`strncpy_trap`（不补 `\0` 与 `offsetof` 读数）、`strtok_reentry`（嵌套调用坏在哪）、`ctype_negative`（负字符）、`sscanf_parse`（`%n` 与宽度）、`scanf_family`（三兄弟分工）、`strtol_vs_atoi`（两个数字出口）、`snprintf_bound`（截断与返回值）、`math_demo`（数学函数）、`qsort_struct`（排序与查找）、`rand_bias`（随机数偏斜）、`clock_timing`（两种时间）、`csv_tool`（综合练习）、`strcpy_oob_fanalyzer`（三种越界证据）——每个都带 `.json` 清单、`src/` 与 `include/`；
- `csv_tool/students.csv` —— 本单元唯一的**输入数据**（8 行数据 + 2 行注释 + 1 个空行，其中 1 行故意写脏）；
- 六段**会话**：`strncpy_trap/session-warning.txt`（`-Wstringop-truncation` 原文）、`strncpy_trap/session-layout.txt`（`-O0` 与 `-O2` 的读数）、`snprintf_bound/session-warning.txt`（`-Wformat-truncation` 原文）、`clock_timing/session-timing.txt`（三次 `clock()` 读数）、`csv_tool/session-write.txt`（跑一遍 + `cat` 写回的文件）、`strcpy_oob_fanalyzer/session-fanalyzer.txt`（`-fanalyzer` 原文与两种退出码）；
- 两个**探针**：`strncpy_trap/probe_layout.c` 与 `snprintf_bound/probe_truncation.c`。**它们故意不配清单**——它们的输出依赖编译器的布局与优化等级，只适合作为「抓下来的会话」出现，不适合当「每次都要对上的输出」。

下一章（第 7 章）把镜头从「库函数怎么用」转到「工程怎么搭」：预处理与宏、多文件与链接性、以及用 gdb 把程序停下来看。这一节里反复出现的「声明放头文件、定义放源文件」，到那一章会给出完整的规矩。

工程惯例 · **工程惯例：为什么这两个探针不配清单**

本书的清单是一份**契约**：它声明「这段程序怎么编译、跑完输出必须是什么」。探针做不到这一点——`probe_layout.c` 在 `-O0` 下给出 11/15/27，在 `-O2` 下给出 14/14/14（`session-layout.txt` 是原文）。**把这种输出写进清单，等于把「未定义行为的结果」当成预期值**，那比不写还坏：它会让人以为某个数字是「正确答案」。

所以本书的分工是：**清单管「这一次编译、这一次运行逐字长这样」的读数，会话管「同一份代码换个配置就变」的现象。**两者的区别不是「有没有 UB」，而是**这个读数会不会随编译配置变**：`probe_layout.c` 的 11/15/27 会变，所以它只能进会话；`strncpy_trap` 里那个 15 在本节的固定布局下每次运行都相同，可以进清单——**但正文必须同时说清「它是这一次的读数，不是你能依赖的保证」。**

小结 · 这一节的要点

- 字符串函数只看两件事：**改不改你的原串、看不看得见目标有多大**；`strlen` / `strcpy` 没有长度参数，是因为契约就是「以 `\0` 结尾的字节序列」；`strncpy` 只在源串比 n 短时补 `\0`，`strtok` 还会改坏原串并把「读到哪」藏起来。
- `ctype.h` 的参数必须是 `unsigned char` 转换后的值或 `EOF`——负 `char` 是 UB，本机实测「看起来没事」（返回 0），而这正是它最危险的地方。
- 解析优先 `sscanf`（配 `%n` 与字段宽度）或 `strtol`（看 `endptr` 与 `errno`）；`atoi` 不报错，别用。`snprintf` 的 `n` 是缓冲区总长、返回值是「本来要写多少」：本机 8 字节缓冲区返回 **22**、装的是 `item-42`。
- `qsort` / `bsearch` 靠比较器的三值约定（别写减法，会溢出）；`rand()` 同种子同实现可复现、`% n` 有结构性偏斜（本机 1710/10000 对公平的 922）；`clock()` 测处理器时间（本机 20 个时钟单位），`time()` 量不出这 20 毫秒。

自测题 · 自测题（先自己答，再对答案）

1. 15.3 节那个结构体里，`copy_unsafe(&r, "abcdefghij")` 之后 `strlen(r.name)` 报 **15**，而 `r.name` 只有 8 格。多出来的 7 个字符是从哪来的？为什么这个数字**不能依赖**？
2. 为什么 `isalpha(c)` 里传一个负的 `char` 是未定义行为？写出标准允许的两种写法，并说明「本机返回 0」为什么不能当成「这么写可以」的理由。
3. `snprintf(buf, 8, "%s-%d-very-long-tail", "item", 42)` 的返回值是多少？`buf` 里是什么？要完整装下至少需要多少字节？判据怎么写？
4. `atoi("999999999999")` 在本机返回什么？`strtol` 有没有办法让你知道这里出过问题？用哪两个出口判断？
5. `qsort` 的比较器为什么不能写成 `return a->score - b->score;`？另外，`bsearch` 对一个「没按这个比较器排过序」的数组会给出什么？

注 · 自测题答案

1. 本机实测给出 **15**（8 个字母一路数进紧挨着的另一块内存，直到碰上那个 0）。**不能依赖**的理由是：`name` 后面那几格装的是什么，不由你决定——本机 `session-layout.txt` 里同一份代码在 `-O0` 下给出 11/15/27、在 `-O2` 下全变 14。结论：`strncpy` 之后必须自己补 `dst[sizeof dst - 1] = '\0'`（或改用 `snprintf`）。
2. 因为 `isalpha` 这类函数的参数只允许两种值：能用 `unsigned char` 表示的（0..255），或者 `EOF`；负的 `char` 两种都不是。标准原文在 7.4.1 节：「In all cases the argument is an int, the value of which shall be representable as an unsigned char or shall equal the value of the macro EOF. If the argument has any other value, the behavior is undefined.」允许的两种写法是 `isalpha((unsigned char)c)` 与 `isalpha(EOF)`。本机返回 0 只是**这一次的巧合**（本机把 0x80..0xFF 全试了一遍，两种写法都是 0 个），它既不是标准承诺的结果，也不代表别的实现会给出同样的东西。
3. 返回 **22**（`item-42-very-long-tail` 的长度，不含结尾），`buf` 里是 `"item-42"`（7 个字符 + 一个结尾 `0`，正好 8 个字节），完整装下至少要 **23** 字节（22 + 结尾）。判据照着标准那句话写：返回值非负且**小于** `n` 才算完整，代码里就是 `if (n < 0 || n >= (int)sizeof buf) { /* 截断或出错 */ }`。
4. 返回 **2147483647**（本机 `LONG_MAX`，因为 `long` 是 32 位）——一个**看起来很正常**的数。有：`strtol` 会把 `ERANGE`（本机 = 34）写进 `errno`，同时用返回值给出端点值。两个出口是 `errno`（有没有超出范围）和 `endptr`（有没有读到数字、读到哪停的）。
5. 因为两个 `int` 相减可能超出 `int` 的范围（本机 `INT_MAX` 与 `INT_MIN` 的数学差是 4294967295，而 `int` 只有 4 字节，装不下），而有符号溢出是未定义行为，排序算法会拿着随机的符号继续排。正确写法是 `(a > b) - (a < b)`。`bsearch` 对没按该比较器排好的数组**不保证任何结果**：可能返回 `NULL`，也可能返回一个看起来命中的指针——本机 `qsort_struct` 的最后一行为这两种写法都留了措辞。

第 7 章

工程与工具：预处理、多文件与调试

本章 3 个单元 · 页眉从这里之后的页面开始出现

1. 预处理与宏：符号在编译之前怎么变形 零基础 · 第 16 单元 · 建议 90 分钟
2. 多文件工程：头文件、编译单元与链接性 零基础 · 第 17 单元 · 建议 90 分钟
3. 调试与排错：gdb、退出码与排错套路 零基础 · 第 18 单元 · 建议 120 分钟

预处理与宏：符号在编译之前怎么变形

◎ 学习目标 · 看完这一单元，你应该能

- 说出预处理在编译之前做的四件事（文件包含、宏替换、条件编译、行控制），并解释它为什么对类型一无所知；
- 写出对象宏与函数式宏，说出「全大写」是惯例而不是标准，并用括号纪律守住 `SQ(1+2)` 这类展开；
- 说清宏的两个经典咬人处：参数被求值两次、多语句宏在 `if…else` 里吃掉 `else`，并知道正解各长什么样；
- 用 `#`、`##`、`__FILE__`、`__LINE__`、`__func__` 写出一个能在发布版里自动消失的 LOG 宏；
- 用 `#if / #elif / #else / #endif` 把同一份源码编成两个程序，用 `#error` 把「这个平台不支持」变成编译期的一句话，并说出哪些场合不该用宏。

16.1 先看工程长什么样

宏最容易骗人的地方是它**长得像函数**。你会用函数的一切经验去用它——传参、拿返回值、指望类型检查——然后被咬。这一节我们要做的事，就是把「宏到底是什么」钉死在一件具体的事上：**它是一段文字，在编译之前被原地换掉。**换掉之后还剩什么、丢掉了什么，全都能用文件和命令看见。

大白话 · 把这一节的目的说成人话

你写的 `.c` 文件不是编译器直接读的。在它之前还有一道工序，叫**预处理**：它像一次「查找替换」，把 `#` 开头的那些行按规矩办掉，产出一份**新的文本**给编译器。宏就是这套替换规则里最常用的一件工具。

所以要理解宏，得先接受一件反常识的事：**编译器从来没有见过 `SQ` 这个名字。**它见到的只是替换之后的 `((1 + 2) * (1 + 2))`。你要问「宏有没有类型」「宏有没有作用域」，答案都藏在这句话里。

先看这一节要建的文件。本书从第 2 个项目起就不允许单文件工程，所以每个例子都是「一个目录 + `src/` + `include/` + 一份清单」的最小工程；这一节会建起第 7 章的项目目录 `examples/ch07/`，往里放十一个例子目录、一个**共用的宏头文件**，再加几段会话与一个探针。

目录树 **这一节的例子工程：从零建起 ch07/，再往里加宏实验目录与共用的宏头文件**

第 1 步 本节开始时，examples/ 下是前面六章的项目

```
examples/
└── ch01/ … ch06/ ：前面几章的项目（本节不动）
```

第 2 步 第 7 章的项目目录从这一节建起

```
examples/
├── 前面六章的项目 ：本节不动
└── ch07/ ← 本课新增
    ├── include/ ← 本课新增
    │   └── log.h ：本章共用的宏头文件 ← 本课新增
    └── macro_basics/ ：对象宏与函数式宏 ← 本课新增
```

第 3 步 把一个例子拆开看：最小工程就是这四样东西

```
examples/ch07/
├── include/log.h ：同上
└── macro_basics/
    ├── include/macro_basics.h ：宏与声明 ← 本课新增
    ├── src/main.c ：组装与打印 ← 本课新增
    ├── src/macro_basics.c ：函数定义 ← 本课新增
    └── macro_basics.json ：怎么编、对哪段输出 ← 本课新增
```

第 4 步 再加五个「咬人处」的例子

```
examples/ch07/
├── include/log.h : 同上
├── macro_basics/ : 同上 (含 src/ 与 include/)
├── macro_parens/ : SQ(1+2) 的两份写法 ← 本课新增
├── macro_side_effect/ : SQ(i++) ← 本课新增
├── macro_limits/ : 宏不能当变量用 ← 本课新增
├── do_while_macro/ : do … while (0) 的正解 ← 本课新增
└── do_while_bad/ : 裸花括号的反例 ← 本课新增
```

第 5 步 日志、拼接、条件编译、编译期报错，另加两样辅助

```
examples/ch07/
├── 前面六个例子目录 : 同上
├── log_macro/ : LOG 宏，两套编译参数 ← 本课新增
├── stringize_paste/ : # 与 ## 各一例 ← 本课新增
├── cond_compile/ : 同一份源码两条路 ← 本课新增
├── error_warning/ : #error 与 #warning ← 本课新增
├── generic_dispatch/ : _Generic 的类型分派 ← 本课新增
├── 6 个 session-*.txt : 本节自己抓的会话 ← 本课新增
└── pragma_probe.c : 未知 #pragma 的探针 ← 本课新增
```

第 1 帧是这一节故事的起点：`examples/` 下躺着前面六章的项目，而第 7 章的项目目录还不存在——下一单元（第 17 单元）会明说「第 7 章的目录由本章上一单元建了起来」，说的就是这一节。第 2 帧建起 `examples/ch07/`，先放两样东西：**本章共用的宏头文件** `include/log.h`，和第一个例子 `macro_basics/`。第 3 帧把一个例子拆开，让你看清最小工程的骨架：**宏与声明在 `include/`，定义与 `main` 在 `src/`，编译与对账的参数在 `.json` 里。**

第 4 帧是本节的四个「咬人处」各自配一个例子：括号纪律、求值两次、宏的边界、多语句宏（含一个故意编不过的反例）。第 5 帧补齐剩下的五件事：LOG 宏、`#` 与 `##`、条件编译、编译期报错、`_Generic`，外加六段抓下来的终端会话和一个单文件探针。注意帧 4 与帧 5 只写了目录名——它们的内部结构和帧 3 一模一样，不再重复画。

后面每一节都会回到它们中的一个：

- `macro_basics`、`macro_parens`、`macro_side_effect`、`macro_limits` —— 宏是文字、括号纪律、求值两次、宏做不到的事（16.3 到 16.5、16.13 节）；
- `do_while_macro`、`do_while_bad` —— 多语句宏的正解与反例（16.6 节）；
- `stringize_paste`、`log_macro` —— `#` 与 `##`、一个带开关的 LOG 宏（16.7、16.8 节）；
- `cond_compile`、`error_warning`、`generic_dispatch` —— 条件编译、编译期报错、类型分派（16.11 到 16.13 节）。

工程惯例 · **工程惯例：为什么宏头文件放在本章的 include/ 下**

标准对「宏写在哪个文件里」没有任何规定：宏是**预处理**的事，它只认「从这一行往后、在当前这个编译单元里有效」。把共用宏集中到一个头文件，是**惯例**（不是标准要求），理由是：

- **一处定义、多处一致**：日志格式、边界值、平台开关，各写一份迟早会不一致；
- **改一处就全改**：头文件里改一行，所有包含它的源文件下次编译都跟着变；
- `#undef` **有明确的落点**：宏集中在头文件里，「从哪一行起换一份定义」这件事只有一处需要想清楚。

代价是**污染**：头文件里的宏会跟着 `#include` 走进**每一个**包含它的编译单元，名字撞了就是你的事（16.10 节会把这件事画出来）。

16.2 预处理是编译之前的一遍文字加工

上一段说的「编译之前的工序」叫预处理。它一共做四件事，全部是**对文本做事**：

1. **文件包含**：`#include` 把那个文件的全部内容就地搬进来（第 1 单元 1.3 节亲眼看过：10 行源文件变成 1525 行）；
2. **宏替换**：`#define` 定下的名字，从这一行往后一律换成它对应的那串文字；
3. **条件编译**：`#if` / `#ifdef` 这些按条件**留着或删掉**整段文本；
4. **行控制**：`#line` 与编译器自己写下的行号标记，管的是「这段文字在原来源文件的第几行」。

四件事有一个共同点：**它们都不知道类型是什么。**预处理跑在编译之前，那时候还没有 `int`、没有 `struct`、没有函数签名——只有一串一串的字符（准确说是 preprocessing token）。所以「宏有没有类型」这个问题本身就问错了：宏不是值，它是**替换规则。**

这一点可以直接看。下面这段会话是真的：把 `macro_parens` 这份源码交给 `-E`（只做预处理就停），看它把 `SQ_OK` 与 `SQ_BAD` 变成了什么。

终端会话 预处理之后的真实文本：宏的名字全不见了 ch07-pre-parens

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -E src/main.c | tail -n 14
# 4 "src/main.c" 2
int main(void) {
    printf("SQ_OK(1+2)        = %d (期望 9) \n", ((1 + 2) * (1 + 2)));
    printf("sq(1+2)           = %d (期望 9) \n", sq(1 + 2));
    printf("SQ_BAD(1+2)       = %d (期望 9) \n", 1 + 2*1 + 2);
    printf("SUM_OK(1, 2) * 3  = %d (期望 9) \n", ((1) + (2)) * 3);
    printf("SUM_BAD(1, 2) * 3 = %d (期望 9) \n", (1) + (2) * 3);
    printf("SQ_OK(-3)         = %d (期望 9) \n", ((-3) * (-3)));
    printf("SQ_BAD(-3)        = %d (期望 9) \n", -3*-3);
    return 0;
}
```

三件事在这 14 行里一目了然：

- **宏的名字消失了**：`SQ_OK(1 + 2)` 变成了 `((1 + 2) * (1 + 2))`，`SQ_BAD(1 + 2)` 变成了 `1 + 2*1 + 2`。编译器看到的就是后面这些（第二行的 `sq(1 + 2)` 那个**函数调用**没变，因为函数不是宏）；
- **括号是你自己写进去的**：`SQ_OK` 的展开里那两层括号，是宏定义里本来就写着的——它不会自动出现；
- **第一行 `# 4 "src/main.c" 2` 是行控制**：它告诉编译器「下面这段文字回到 `src/main.c` 的第 4 行」。所以你以后看到的报错，行号永远指向**你的源文件**，而不是预处理结果。

还有一件顺带的事：宏**在运行期一个字节都不占**——替换完就退场，程序跑起来时内存里没有叫 `SQ_OK` 的东西。这是它与函数最本质的区别，也是它「免费」的代价：免费意味着没有类型、没有作用域、没有调试信息。

{} 语法规则 · 这一节要用的两种宏与一条关闭指令

- `#define 名字 一串文字` —— 对象宏；
- `#define 名字(参数) 一串带参数的文字` —— 函数式宏；
- `#undef 名字` —— 从这个点起，这个名字不再被替换。

三条规矩：`#define` 那一行**不写分号**（多写一个，那个分号就成了替换内容的一部分）；名字与左括号之间**不能有空格**（`#define DOUBLE (n)` 会变成对象宏，不是函数式宏）；替换发生在**从这一行往后**的文本里，包括后面 `#include` 进来的头文件。

工程惯例 · 工程惯例：全大写是惯例，不是标准

`ARRAY_MAX`、`DOUBLE`、`LOG` 一律全大写——这是**惯例**，标准里没有一个字要求你这么做。它买到的是「一眼能认出这是宏」：读到 `ARRAY_MAX` 你就知道它没有地址、不能取 `&`，而且在**运行期根本不存在**（16.2 节那份预处理结果就是证据：程序跑起来之后，内存里没有 `ARRAY_MAX` 这个名字）。代价是**宏与枚举/常量必须用两套命名**，团队里要有人守这个规矩。

16.3 宏是文字：对象宏与函数式宏

先说最常见的两种写法。下面这个头文件是这一节的第一个例子，`ARRAY_MAX` 是对象宏（一个名字换一串文字），`DOUBLE` 是函数式宏（长得像函数）：

c examples/ch07/macro_basics/include/macro_basics.h 第 4-14 行

```c
/* 对象宏：一个名字，替换成一串文字。全大写是惯例，不是标准的要求 */
#define ARRAY_MAX 10 一个名字换一串文字

/* 函数式宏：写法像函数，本质还是文字替换 */
#define DOUBLE(n) ((n) * 2) 看着像函数，其实是文本

/* 真函数：拿它对照——有类型、参数只求值一次、运行期占地方 */
int times_two(int n); 真函数：有类型，只求值一次

/* 它要用的宏定义写在另一个函数体里面：宏没有作用域，出了花括号还活着 */
int times_two_plus_one(int n);
```

5 一个名字换一串文字
8 看着像函数，其实是文本
11 真函数：有类型，只求值一次

用起来是这样。每一行打印出来的都是**宏展开之后**再去算的值：源码里你写的是 `ARRAY_MAX` 与 `DOUBLE(21)`，编译器看到的是 `10` 与 `(21) * 2`。

c examples/ch07/macro_basics/src/main.c 第 5-14 行

```c
int main(void) {
    printf("ARRAY_MAX        = %d\n", ARRAY_MAX); 宏在这里被换成 10
    printf("ARRAY_MAX + 5    = %d\n", ARRAY_MAX + 5);

    printf("DOUBLE(21)       = %d\n", DOUBLE(21)); 同一件事，两种做法
    printf("times_two(21)    = %d\n", times_two(21));
    printf("times_two_plus_one(21) = %d\n", times_two_plus_one(21));

    printf("DOUBLE(1.5)      = %.2f\n", DOUBLE(1.5)); 换成 double 也能用
    printf("DOUBLE('A')      = %d\n", DOUBLE('A')); 'A' 是 65
```

6 宏在这里被换成 10
9 同一件事，两种做法
13 换成 double 也能用
14 'A' 是 65

终端输出 终端输出 macro-basics-out

```
ARRAY_MAX        = 10
ARRAY_MAX + 5    = 15
DOUBLE(21)       = 42
times_two(21)    = 42
times_two_plus_one(21) = 43
DOUBLE(1.5)      = 3.00
DOUBLE('A')      = 130
&max_const       = 00000000005FFEAC (const 是个真变量，有地址)
sizeof(max_const)= 4, sizeof(MAX_ENUM) = 4
sizeof(ARRAY_MAX)= 4, sizeof(DOUBLE(1)) = 4
```

这段输出里藏着三个结论，一个一个说：

- `DOUBLE(21)` 与 `times_two(21)` 都是 **42**——宏与函数在这一件事上没区别，都算得出 21 的两倍；
- `DOUBLE(1.5)` 是 **3.00**、`DOUBLE('A')` 是 **130**——**同一个宏换着类型用，照样能编。**这是函数的反面：`times_two` 只收 `int`，你传 `1.5` 进去会先被转成 `1`。宏的「什么都能用」不是本事，是它**根本没有类型**：`((1.5) * 2)` 里的 `1.5` 就是字面量，编译器看到的是乘法，跟宏无关；

「没有类型」的另一面是「也不做参数检查」，而这件事的后果，编译器会当着你的面说。

16.4 括号纪律：`SQ(1+2)` 为什么得到 5

先看输出，再解释为什么。下面这个小程序把三种写法摆在一起：正解 `SQ_OK`、真函数 `sq`、反例 `SQ_BAD`。

c examples/ch07/macro_parens/src/main.c 第 5–16 行

```c
int main(void) {
    printf("SQ_OK(1+2)        = %d (期望 9) \n", SQ_OK(1 + 2));
    printf("sq(1+2)           = %d (期望 9) \n", sq(1 + 2));
    printf("SQ_BAD(1+2)       = %d (期望 9) \n", SQ_BAD(1 + 2));

    printf("SUM_OK(1, 2) * 3  = %d (期望 9) \n", SUM_OK(1, 2) * 3);
    printf("SUM_BAD(1, 2) * 3 = %d (期望 9) \n", SUM_BAD(1, 2) * 3);

    printf("SQ_OK(-3)         = %d (期望 9) \n", SQ_OK(-3));
    printf("SQ_BAD(-3)        = %d (期望 9) \n", SQ_BAD(-3));
    return 0;
}
```

6 每个参数一层、整串一层
7 函数不需要括号保命
8 同一个表达式，少了两层括号
11 参数包了，整串忘了包

终端输出 终端输出 macro-parens-out

```
SQ_OK(1+2)        = 9 (期望 9)
sq(1+2)           = 9 (期望 9)
SQ_BAD(1+2)       = 5 (期望 9)
SUM_OK(1, 2) * 3  = 9 (期望 9)
SUM_BAD(1, 2) * 3 = 7 (期望 9)
SQ_OK(-3)         = 9 (期望 9)
SQ_BAD(-3)        = 9 (期望 9)
```

这两份定义的差别只有括号，而括号写在头文件里：

c examples/ch07/macro_parens/include/macro_parens.h 第 4–12 行

```c
/* 反面教材：参数一个括号都没加 */
#define SQ_BAD(x) x*x
/* 正解：每个参数各加一层括号，整串再加一层 */
#define SQ_OK(x) ((x) * (x))
/* 另一种常见错法：参数包了，整串忘了包 */
#define SUM_BAD(a, b) (a) + (b)
#define SUM_OK(a, b) ((a) + (b))
```

5 反面教材：一个括号也没有
8 每个参数一层，整串一层
11 参数包了
12 整串忘了包

拿上面那段预处理结果对账，两个数字自己会说话：

- `SQ_BAD(1 + 2)` 展开成 `1 + 2*1 + 2`。乘号先算，等于 1 + 2 + 2 = **5**——而你要的是 9。**注意这不是编译错误**：`1 + 2*1 + 2` 完全合法，编译器不会拦你，程序照样跑，只是答案错了；
- `SUM_BAD(1, 2) * 3` 展开成 `(1) + (2) * 3`，等于 1 + 6 = **7**——而你要的是 9。这一次参数包了括号，**整串忘了包**，外面的 `* 3` 就只乘到了后半个参数。

两条纪律就藏在这两个数字里：**每个参数外面加一层括号**，**整个展开式外面再加一层括号**。第二层是很多人漏掉的那一层——它保的不是参数，是**调用点周围**的运算符优先级。

> ⚠ **陷阱 · 你会怎么被这两个数字坑到**
>
> 你会这样误判：写完 `SQ_BAD(1 + 2)` 看到程序跑出了 5，第一反应是「是不是 `printf` 的格式串写错了」，或者回 `main` 里把 `1 + 2` 改来改去。
>
> **真相是**：问题不在调用点，在**宏定义那一行**。宏展开是完全机械的文本替换，你写了什么括号它就带什么括号，一个都不多、一个都不少。所以：
>
> - 出错时的第一个动作应该是「把它展开念一遍」——`gcc -E` 会把展开后的文本原样给你（上一节那段会话就是干这个的）；
> - 第二个动作是去看**头文件**里那行 `#define`，而不是去改 `main.c` 里的调用。
>
> 还有一件更烦的事：`SQ_BAD(-3)` 打印出来的却是 9，**是对的**。因为 `-3*-3` 里一元负号的优先级本来就比乘号高。这类「有时候对、有时候错」的宏最坑人：你测了两个用例都过，第三个才炸。

括号加对了也不代表安全——下一节就是那个「加对了括号照样错」的例子。

16.5 求值两次：`SQ(i++)` 的账

下面这份 `SQ` 的括号已经写到了教科书级别——每个参数一层、整串一层，挑不出毛病：

c examples/ch07/macro_side_effect/include/macro_side_effect.h 第 4–8 行

```c
/* 括号都加对了，坑还在：x 在展开式里出现两次，就会被求值两次 */
#define SQ(x) ((x) * (x))  括号全对，坑还在

/* 函数版：实参先求值一次，然后才进函数体 */
int sq(int x);  函数版：实参只求值一次
```

5 括号全对，坑还在
8 函数版：实参只求值一次

坑在于 `x` 在这串文字里出现了**两次**。于是调用点写下的那个表达式，也会被**求值两次**。先看前两组对照：

c examples/ch07/macro_side_effect/src/main.c 第 5–12 行

```c
int main(void) {
    int i = 5;
    int a = SQ(i++);          /* 展开成 ((i++) * (i++))：自增发生两次 */  展开成 ((i++) * (i++))
    printf("SQ(i++) 得到 a = %d，此时 i = %d\n", a, i);

    int j = 5;
    int b = sq(j++);          /* 函数调用：j++ 只发生一次 */  j++ 只发生一次
    printf("sq(j++) 得到 b = %d，此时 j = %d\n", b, j);
```

7 展开成 ((i++) * (i++))
11 j++ 只发生一次

同一个坑换个写法（前置自增）再踩一次，外加一组「没有副作用」的对照，用来确认宏在干净参数上确实没问题：

c （续）后两组：前置自增与没有副作用的参数 examples/ch07/macro_side_effect/src/main.c 第 14–21 行

```c
    int k = 5;
    int c = SQ(++k);          /* 同样两次，只是这次是前置自增 */
    printf("SQ(++k) 得到 c = %d，此时 k = %d\n", c, k);

    int n = 5;
    printf("SQ(n) 得到 %d，此时 n = %d（没有副作用，两种写法一样）\n", SQ(n), n);
    return 0;
}
```

终端输出 终端输出 macro-side-effect-out

```
SQ(i++) 得到 a = 30，此时 i = 7
sq(j++) 得到 b = 25，此时 j = 6
SQ(++k) 得到 c = 49，此时 k = 7
SQ(n) 得到 25，此时 n = 5（没有副作用，两种写法一样）
```

数字自己会说话：`SQ(i++)` 之后 `i` 从 5 变成 **7**（自增发生了两次），得到 30；而函数版 `sq(j++)` 之后 `j` 只走到 **6**，得到 25——同一件事，两种做法，两个答案。不过这一次编译器抢在你前面开口了，只是它用的是**警告**，编译照样成功：

终端会话 编译器先说了一句：这里的行为是未定义的 `ch07-seqp`

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/macro_side_effect.c -o se-O0.exe
In file included from src/main.c:3:
src/main.c: In function 'main':
src/main.c:7:17: warning: operation on 'i' may be undefined [-Wsequence-point]
    7 |     int a = SQ(i++);          /* 展开成 ((i++) * (i++))：自增发生两次 */
include/macro_side_effect.h:5:23: note: in definition of macro 'SQ'
    5 | #define SQ(x) ((x) * (x))
      |                       ^
src/main.c:15:16: warning: operation on 'k' may be undefined [-Wsequence-point]
   15 |     int c = SQ(++k);          /* 同样两次，只是这次是前置自增 */
include/macro_side_effect.h:5:23: note: in definition of macro 'SQ'
    5 | #define SQ(x) ((x) * (x))
      |                       ^
```

读它有三步：`operation on 'i' may be undefined [-Wsequence-point]` 是「对 `i` 的运算可能是未定义的」，方括号里是管这条诊断的开关名；`src/main.c:7:17` 指向**调用点**；紧跟着那几行 `note: in definition of macro 'SQ'` 指向**宏定义那一行**——编译器把两处都指给你，是因为这件事的责任在两头：宏展开出了两次 `i++`，而 `i++` 是你写上去的。

跑起来的输出是：

终端会话 （续）真跑一次：两个数字对照 `ch07-seqp-run`

```
$ ./se-O0.exe
SQ(i++) 得到 a = 30，此时 i = 7
sq(j++) 得到 b = 25，此时 j = 6
SQ(++k) 得到 c = 49，此时 k = 7
SQ(n) 得到 25，此时 n = 5（没有副作用，两种写法一样）
```

标准原文 · **标准原文：同一份内存上两次没排队的修改**

> 标准 6.5 节「Expressions」第 2 段：
>
> > If a side effect on a scalar object is unsequenced relative to either another side effect on the same scalar object or a value computation using the value of the same scalar object, the behavior is undefined.
>
> 大意：**同一个标量对象上的两次副作用，如果谁先谁后没有规定，行为就是未定义的。**`((i++) * (i++))` 正好踩在这一句上：两个自增之间没有先后顺序，所以问题不是「先算左边还是先算右边」，而是「这段程序的行为标准根本没规定」。
>
> 出处：ISO/IEC 9899:2018（C17）§6.5 第 2 段。

实验记录 · **本机实测：这四个数字都是跑出来的**

- 工具链：gcc 16.1.0（`-std=c17`，即 C17），目标三元组 x86_64-w64-mingw32；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/macro_side_effect.c -o se-O0.exe`（工作目录 `examples/ch07/macro_side_effect/`）；
- 结果：`SQ(i++)` 得 **30**、此时 `i` = **7**；`sq(j++)` 得 **25**、此时 `j` = **6**；`SQ(++k)` 得 **49**、此时 `k` = **7**；程序退出码 **0**；
- 我又把同一份源码用 `-O2` 编了一份（`gcc -std=c17 -O2 -Iinclude src/main.c src/macro_side_effect.c -o se-O2.exe`），四个数字**与 `-O0` 逐字相同**。

最后一条要小心它的意思：两个优化档给出同样的答案，**不代表这里没问题**——未定义行为的含义是「标准不规定」，换编译器、换平台都可以给另一个数。要判断的从来不是「跑出来是什么」，而是「这条规则有没有被踩到」。

> ⚠ **陷阱 · 宏与函数的取舍，你会怎么被坑到**
>
> 你会被这样坑到：本地跑 `SQ(i++)` 得到 30，看起来完全正常，于是你把它留在代码里；换一台机器、换一个优化档、换一个编译器版本，它可能变成 25 或 36——而你连「这里可能出问题」的念头都没有。
>
> 你会怎么误判：把「本机跑出来的 30」当成「这段代码是对的」，于是连编译器那条 `-Wsequence-point` 也一并当成噪音。
>
> 真相是：这里要问的不是「结果是多少」，而是「行为有没有被规定」——30 只是这台机器这一次碰巧给的。
>
> 取舍的判断顺序是这样的：
>
> - **实参可能带副作用时，别用宏**：函数调用里实参只求值一次，这是语言规定；宏没有这条保证；
> - **想省一次函数调用时，先别动手写宏**：小函数编译器自己会内联（第 7 单元的函数调用栈、第 19 单元的优化边界都会碰到这件事）；
> - **宏留给函数做不到的事**：拼名字（`##`）、拿到源文件名与行号（`__FILE__` / `__LINE__`）、按 `-D` 开关整段代码（条件编译）——这几件才是宏的主场。

16.6 多语句宏：`do { … } while (0)` 挡的是什么

宏里放**一条表达式**的时候，加括号就够了。宏里放**好几条语句**的时候，麻烦换了一种形状：宏展开之后是一段**代码块**，而调用点上你还要写一个分号。`do { … } while (0)` 就是为这件事准备的写法：

c examples/ch07/do_while_macro/include/do_while_macro.h 第 4–13 行

```c
/* 多语句宏的正解：整块包进 do { ... } while (0)。
   调用时后面那个分号正好收在 while (0) 后面，不会漏到外面去。 */
#define SWAP_OK(a, b)           \ 宏名后面紧跟左括号
    do {                        \ 整块放进 do...while(0)
        int tmp_ = (a);         \
        (a) = (b);              \
        (b) = tmp_;             \
    } while (0) 只跑一次的循环

int bigger(int a, int b);
```

6 宏名后面紧跟左括号
7 整块放进 do...while(0)
11 只跑一次的循环

用它的人可以放心地把分号写上去，也可以把它塞进不带花括号的 `if…else`。下面先看第一条路（条件不成立，走 `else`）：

c examples/ch07/do_while_macro/src/main.c 第 9–14 行

```c
    if (x > y)
        SWAP_OK(x, y);          /* 分号收在 while (0) 后面，正好 */ 分号收在 while (0) 后面
    else
        printf("x=%d 不大于 y=%d，不用交换\n", x, y);

    printf("走完第一条 if…else：x = %d，y = %d\n", x, y);
```

10 分号收在 while (0) 后面

再看第二条路（条件成立，宏真的执行了），末尾再加一次「当普通语句」的调用：

c **（续）条件成立那一支，与一次普通调用** examples/ch07/do_while_macro/src/main.c 第 16–24 行

```c
    if (x < y)
        SWAP_OK(x, y);
    else
        printf("这一支不该被选中\n");

    printf("交换之后：x = %d，y = %d\n", x, y);

    SWAP_OK(x, y);              /* 当普通语句用也一样 */
```

```
24      printf("再换回来：x = %d，y = %d，大的那个是 %d\n", x, y, bigger(x, y));
```

终端输出 终端输出 do-while-macro-out

```
x=3 不大于 y=8，不用交换
走完第一条 if…else：x = 3，y = 8
交换之后：x = 8，y = 3
再换回来：x = 3，y = 8，大的那个是 8
```

输出按顺序读：第一次 `if (x > y)` 不成立（3 不大于 8），走 `else` 打了一行；第二次 `if (x < y)` 成立，`SWAP_OK` 真的把 `x`、`y` 换成了 8 和 3；最后再换回来，大的那个是 8。**三条路都走通了**，每次调用后面那个分号都没惹事。

现在把定义换成「只包一对裸花括号」，代码别处一个字不改：

c examples/ch07/do_while_bad/include/do_while_bad.h 第 4–6 行

```
4   /* 反面教材：多语句宏只包了一对裸花括号。
5      展开之后，调用点那个分号会落在花括号外面，把 if 提前关掉。 */
6   #define SWAP_BAD(a, b) { int tmp_ = (a); (a) = (b); (b) = tmp_; }   展开后分号落在花括号外面
```

6 展开后分号落在花括号外面

换掉定义之后，调用点一个字都不用改——这正是宏的好处，也正是它的危险：你改的是**另一个文件**里的一个字符，出问题的是**这里**。下面只看调用它的那几行：

c examples/ch07/do_while_bad/src/main.c 第 5–12 行

```
5   int main(void) {
6       int x = 3;
7       int y = 8;
8
9       if (x > y)   if 的条件
10          SWAP_BAD(x, y);          /* 展开成 { ... }; —— 这个分号把 if 关掉了 */   展开后多出一个分号
11      else   else 找不到它的 if 了
12          printf("x=%d 不大于 y=%d\n", x, y);
```

9 if 的条件
10 展开后多出一个分号
11 else 找不到它的 if 了

编译它。下面是 gcc 真的吐出来的 stderr 原文：

编译诊断 gcc 的真实诊断（原文照抄） do-while-bad-diag

```
src/main.c: In function 'main':
src/main.c:11:5: error: 'else' without a previous 'if'
   11 |     else
      |     ^~~~
```

这句报错只讲了一半的事实，得翻译成人话。展开之后，编译器看到的文本是 `if (x > y) { int tmp_ = (x); (x) = (y); (y) = tmp_; };` 紧跟着一个 `else`。按语法，`if` 只认领**一条语句**：这里它认领的是那对花括号，到 `}` 就结束了；后面那个**分号**是一条独立的空语句，跟这个 `if` 没有关系。于是 `else` 往前找「配得上它的 `if`」，可那个 `if` 已经结束——报错说的就是这件事。注意：它**不会**说「你在第 10 行多写了什么」，它只会说 `'else' without a previous 'if'`；你会盯着报错点（第 11 行的 `else`）看半天，而真正的问题在头文件里那行 `#define`（`do_while_bad.h` 第 6 行）。

工程惯例 · **工程惯例：`do { ... } while (0)` 是惯例，不是标准**

标准里没有哪一条写着「多语句宏要包 `do…while(0)` 」——它是 C 社区三十年攒出来的**惯例**，几乎所有 C 代码规范里都有这一条。理由就是上面那次编译：它让「宏展开后的整块」在语法上等价于**一条语句**，于是调用点那个分号有了归属，`if…else` 、`while` 、`for` 都不必额外加花括号。

代价是两点：读的人得先知道这个惯例（否则会想「这里为什么有个只跑一次的循环」）；`do…while(0)` 里的 `break` 会跳到 `while (0)` 后面，所以它既是「失败就跳出宏」的出口，也会让宏里的 `break` 跳出宏而不是外面的循环。

陷阱 · **宏后面那个分号，还有别的写法会踩**

先说你会看到什么。第一条，`#define ARRAY_MAX 10;` 多写一个分号：主错误落在**宏定义那一行**（`error: expected ')' before ';' token`，插入符指着 `10;`），下面紧跟 `note: in expansion of macro 'ARRAY_MAX'` 指到使用处——诊断里**把宏的名字写出来了**，不会「跑到下一行去、看起来跟宏无关」。第二条，另一种老写法 `#define SWAP_BAD(a, b) if (1) { … } else`：本机实测它在 `if … else` 里**能编、也能跑对**（`if (1)` 、`if (0)` 、`else if` 链三种摆法都试过，退出码 0、行为都对），代价是每个调用点都留一个空的 `else` ，`-Wall` 给你一条 `-Wempty-body` 。

你会怎么误判：看到 `expected ')' before ';'` ，第一反应是去改 `printf` 那一行的括号——因为主错误在宏定义、插入符却指着括号，你会以为括号写错了地方；看到 `-Wempty-body` ，又会以为自己的 `if…else` 漏了花括号。

真相是：这两条症状都来自**替换列表**。`#define` 那一行的内容是一串纯文本，多一个分号它就跟着走；`if (1) { … } else` 把一个悬空的 `else` 塞进了替换列表，于是每个调用点都多一个空分支。规矩只有一条：`#define` 那一行**永远不写分号**，要写分号的地方是**调用点**；多语句就用 `do { … } while (0)` ，别用 `if (1) … else` 。

16.7 `#` 与 `##` ：字符串化与拼接

宏有两个专门的运算符，只在宏定义里能用：`#` 把实参变成**字符串字面量**，`##` 把两边的记号**粘成一个**。

c examples/ch07/stringize_paste/include/stringize_paste.h 第 4–12 行

```c
/* # 把实参「原样」变成字符串字面量：写什么就打出什么，不求值 */
#define SHOW(expr) printf("%s = %d\n", #expr, (expr))   把实参变成字符串

/* ## 把两边的记号粘成一个新记号 */
#define MAKE_VAR(n) var_##n   粘成 var_1
#define MAKE_FN(n) fn_##n   粘成 fn_1

int fn_1(int x);
int fn_2(int x);
```

5 把实参变成字符串
8 粘成 var_1
9 粘成 fn_1

这三条定义里的 `#` 与 `##` 只在**宏定义里**才合法，普通函数里写 `#expr` 编译器会当场拒绝。用起来是这样：

c examples/ch07/stringize_paste/src/main.c 第 5–20 行

```c
int main(void) {
    int a = 3;
    int b = 4;

    SHOW(a + b);              /* 展开成 printf("%s = %d\n", "a + b", (a + b)) */   实参原样进了引号
    SHOW(a * b);
    SHOW(a + b * 2);          /* 里面的优先级不会丢：它只是被引号包起来 */   优先级不会丢

    int MAKE_VAR(1) = 100;    /* 粘成 int var_1 = 100; */   声明一个叫 var_1 的变量
    int MAKE_VAR(2) = 200;   再来一个 var_2
    printf("var_1 = %d, var_2 = %d\n", var_1, var_2);

    printf("MAKE_FN(1)(20) = %d\n", MAKE_FN(1)(20));   /* fn_1(20) */
```

```
18      printf("MAKE_FN(2)(20) = %d\n", MAKE_FN(2)(20));   /* fn_2(20) */
19      return 0;
20  }
```

```
 9 实参原样进了引号
11 优先级不会丢
13 声明一个叫 var_1 的变量
14 再来一个 var_2
```

终端输出 终端输出 stringize-paste-out

```
a + b = 7
a * b = 12
a + b * 2 = 11
var_1 = 100, var_2 = 200
MAKE_FN(1)(20) = 21
MAKE_FN(2)(20) = 200
```

要点全在**打印出来的字符串**里：

- `a + b = 7`：`#expr` 把 `a + b` 这段文字原样变成了 `"a + b"`。注意它是**文字**，不参与计算——`SHOW(a * b)` 打出 `a * b = 12`，`SHOW(a + b * 2)` 打出 `a + b * 2 = 11`，引号里就是你手写的那几个字符；
- `var_1 = 100, var_2 = 200`：`MAKE_VAR(1)` 粘成了 `var_1`，那是两个真实的变量，`printf` 直接用；
- `MAKE_FN(1)(20) = 21`、`MAKE_FN(2)(20) = 200`：粘出函数名再调用它，`fn_1` 返回 `x + 1`、`fn_2` 返回 `x * 10`。

`#` 与 `##` 最有用的地方就是**日志和调试打印**：`#expr` 让你不必把表达式抄第二遍（抄第二遍就会抄错，改代码时更会忘记同步），`##` 让你按同一个模板批量生成名字。它们的边界也很清楚：`#` 只能用在函数式宏的**参数**前面，`##` 只能粘**记号**——粘出来的必须是一个合法记号，`var_` 跟 `2` 粘成 `var_2` 可以，粘出一串乱七八糟的字符就编不过了。

标准原文 · 标准原文：`#` 与 `##` 各管什么

`#` 在 6.10.3.2 节「The # operator」里规定：

> Each # preprocessing token in the replacement list for a function-like macro shall be followed by a parameter as the next preprocessing token in the replacement list.

大意：`#` 后面**必须**紧跟一个宏参数——所以它只能出现在函数式宏里，`#123` 这种写法不合法。

`##` 在 6.10.3.3 节「The ## operator」里规定：

> A ## preprocessing token shall not occur at the beginning or at the end of a replacement list for either form of macro definition.

大意：`##` **不能**出现在替换列表的开头或结尾（`##x`、`x##` 都不合法），它两边必须各有一个记号。

出处：ISO/IEC 9899:2018（C17）§6.10.3.2、§6.10.3.3。

16.8 `__FILE__` / `__LINE__` / `__func__`：造一个 LOG 宏

调试打印要的三样信息是：**哪个文件、哪一行、哪个函数。**前两个是预定义的**宏**，第三个是一个**标识符**——这个区别很要紧。

c examples/ch07/include/log.h 第 9–16 行

```c
/* 交付宏：定义了 NDEBUG 就是「发布版」，整条日志连它的参数一起消失。
   __FILE__ 与 __LINE__ 由预处理填，__func__ 由编译器填。 */
#ifndef NDEBUG 没定义 NDEBUG 就是调试版
#define LOG(fmt, ...)                                                  \ LOG 收一个格式串
    printf("[%s:%d %s] " fmt "\n", __FILE__, __LINE__, __func__, __VA_ARGS__) 文件与行号由预处理填
#else
#define LOG(fmt, ...) ((void)0) 发布版里整条语句变空
#endif
```

11 没定义 NDEBUG 就是调试版
12 LOG 收一个格式串
13 文件与行号由预处理填
15 发布版里整条语句变空

用起来是这样。请留意每条日志的**前缀**：它记的是调用 `LOG` 的那一行，而不是 `log.h` 里写 `#define` 的那一行——这个差别马上就要用到。

c examples/ch07/log_macro/src/main.c 第 6–17 行

```c
int main(void) {
    int n = 6;

    LOG("程序开始，n = %d", n); 调用点的行号会被记下来
    int h = half(n);
    LOG("half(%d) 的结果是 %d", n, h); 同一个 LOG，这次在 main 里

    printf("正式输出：%d 的一半是 %d\n", n, h);

    LOG("程序结束，共写了 %d 条日志", 3); 三条日志逐条对得上
    return 0;
}
```

9 调用点的行号会被记下来
11 同一个 LOG，这次在 main 里
15 三条日志逐条对得上

终端输出 终端输出 log-macro-out

```
[src/main.c:9 main] 程序开始，n = 6
(half 自己那份 LOG) half(6) 得到 3
[src/main.c:11 main] half(6) 的结果是 3
正式输出：6 的一半是 3
[src/main.c:15 main] 程序结束，共写了 3 条日志
```

每行日志前面的 `[src/main.c:9 main]` 就是三样信息：文件名、调用点行号、当前函数名。它们对得上源码：第 9 行确实是 `LOG("程序开始，n = %d", n);`，而它就在 `main` 里。

这三样东西的来路**不一样**，`-E` 一眼就能看出来：

终端会话 预处理之后：两个换成字面量，一个原样留下 ch07-pre-log

```
$ gcc -std=c17 -Iinclude -I../include -E src/main.c | grep "程序开始"
    printf("[%s:%d %s] " "程序开始，n = %d" "\n", "src/main.c", 9, __func__, n);
$ gcc -std=c17 -Iinclude -I../include -E src/main.c | grep -c "__func__"
3
$ gcc -std=c17 -Iinclude -I../include -E src/main.c | grep -c "__LINE__"
0
```

一行就说清了：

- `__FILE__` 变成了 `"src/main.c"`，`__LINE__` 变成了 `9`——它们**是预处理宏**，预处理阶段就被换掉了；
- `__func__` **原样留在里面**（`grep` 数得到 3 处，正好是 3 次 `LOG` 调用），因为在预处理眼里它就是一个普通标识符。它是**编译器**的事：编译器在每个函数体开头替你放了一个 `static const char __func__[] = "函数名";`。

边界也不同：文件名与行号记的是**调用点**（宏写在头文件里，`__LINE__` 仍给调用行），`__func__` 是**当前函数**的名字，只在函数体里有意义。

变参宏：`...` 与 `__VA_ARGS__`

上面那个 `LOG` 的定义里还有一处要交代：`#define LOG(fmt, ...)` 里的 `...`。它叫**变参宏**：`...` 收下调用点剩下的全部实参，替换列表里用 `__VA_ARGS__` 把它们原样放回去——`LOG("half(%d) 的结果是 %d", n, h)` 里的 `n, h` 就是这么进到 `printf` 里的。`fmt` 单独留一个名字，是为了让格式串与后面的实参**拼在一起**（上一段展开结果里的 `"[" fmt "]"` 就是两个字符串字面量挨着写，编译器会把它们接成一个）。

变参宏有一条硬限制：本书用的这个标准版本要求展开时**至少给一个实参**。所以 `LOG("程序开始")` 这种「只给格式串、不带附加参数」的写法编不过——不是宏写错了，是语言规则：

编译诊断 gcc 的真实诊断（原文照抄） log-zero-arg-diag

```
src/zero_arg.c: In function 'main':
src/zero_arg.c:8:19: warning: ISO C99 requires at least one argument for the '...' in a variadic macro [-
Wpedantic]
    8 |     LOG("程序开始");
      |                   ^
In file included from src/zero_arg.c:5:
../include/log.h:13:77: error: expected expression before ')' token
   13 |     printf("[%s:%d %s] " fmt "\n", __FILE__, __LINE__, __func__, __VA_ARGS__)
      |                                                                             ^
src/zero_arg.c:8:5: note: in expansion of macro 'LOG'
    8 |     LOG("程序开始");
      |     ^~~
```

这条诊断把两件事分得很清楚：`warning: ISO C99 requires at least one argument for the '...' in a variadic macro [-Wpedantic]` 说的是规则本身（它是**警告**），而 `error: expected expression before ')' token` 是后果——`fmt` 后面那个逗号留了下来，`__VA_ARGS__` 展开成空，于是 `printf` 的参数表以「逗号 + 右括号」收尾，少了一个表达式。要让零参数也能用，得靠 GNU 的 `, ##__VA_ARGS__`（把前面那个逗号一起吃掉）或下一版标准里的 `__VA_OPT__`；本节的 `LOG` 故意不依赖这两样，只保证「至少给一个参数」——**可移植比多一种写法重要。**

⚠ 陷阱 · 你会以为「变参宏就是 printf 的语法糖」

你会被这样坑到：在别的项目里见过 `LOG("hello")` 能用，换到本书的 `-std=c17 -Wall -Wextra -Wpedantic` 下，同一个调用突然编不过；或者反过来，在 GCC 下用 `, ##__VA_ARGS__` 写得好好的，换 clang/MSVC 报一堆错。

你会怎么误判：去调用点反复改括号、改格式串——因为报错位置在 `printf` 的参数表上，看起来像是自己少写了一个参数。

真相是：**零实参的变参宏在 C17 里不合法**，能用的那些写法都是扩展（GNU 的 `, ##`）或更新的标准（`__VA_OPT__`）。看到这种「同一行代码换个编译器就编不过」，先问的不是「我的括号错了吗」，而是「我用了哪个工具链的扩展」。诊断里的 `[-Wpedantic]` 就是在提醒你这件事。

标准原文 · 标准原文：`__FILE__` 与 `__LINE__` 是宏

> 预定义宏在 6.10.8.1 节「Mandatory macros」里规定：
>
> > `__FILE__` The presumed name of the current source file (a character string literal). `__LINE__` The presumed line number (within the current source file) of the current source line (an integer constant).
>
> 大意：`__FILE__` 是「当前源文件名」的字符串字面量，`__LINE__` 是「当前行号」的整数常量——它们是**宏**，用的时候被换成字面量。
>
> 出处：ISO/IEC 9899:2018（C17）§6.10.8.1。

`__func__` 不在预定义宏那一节，它是另一类东西：

标准原文 · **标准原文：`__func__` 是标识符**

6.4.2.2 节「Predefined identifiers」：

> The identifier `__func__` is implicitly declared by the translator as if, immediately following the opening brace of each function definition, the declaration `static const char __func__[] = "function-name";` appeared, where function-name is the name of the lexically-enclosing function.

大意：**编译器**在「每个函数定义的左花括号后面」隐式声明 `static const char __func__[]`，内容是函数名。它是**标识符**不是宏——预处理阶段不认识这个名字，所以它不会被替换掉。

出处：ISO/IEC 9899:2018（C17）§6.4.2.2。

工程惯例 · **工程惯例：日志该往哪写**

上面这个 `LOG` 用的是 `printf`，日志与程序输出混在同一个流里。**这是为了这一节的流水线能逐字对账才这么写的**；真实项目里日志几乎都写 `stderr`（把宏里的 `printf` 换成 `fprintf(stderr, …)` 即可）。写 `stderr` 是**惯例**，标准没有规定日志该进哪个流，理由有三条：

- **两个流可以分开走**：`./app > result.txt` 只收正式输出，日志仍然打在屏幕上，不会污染结果文件；
- **崩溃时日志更可靠**：标准输出常常是**全缓冲**的，进程被终止时缓冲区里的内容会丢；`stderr` 默认不缓冲；
- **下游的程序不会被日志噎到**：`./app | grep xxx` 时，日志混进标准输出会把下游弄乱。

代价是你要记住「日志在 stderr」，排查时别只盯着 `result.txt` 发呆。

16.9 交付宏：`-DNDEBUG` 让日志在发布版里消失

同一个 LOG 宏，配上两套编译参数，就是两个程序。上面那一份是**调试版**；再编一份**发布版**，只需要在清单的 `flags` 里多写一个 `-DNDEBUG`（`-D` 就是「在命令行上写一个 `#define`」）：

json examples/ch07/log_macro/log_macro_release.json 第 4–6 行

```
4     "sources": ["src/main.c", "src/log_macro.c"],
5     "flags": ["-Iinclude", "-I../include", "-DNDEBUG"],
6     "mode": "run",
```

同一份 `main.c`、同一个 `log.h`，编出来跑的结果是：

终端输出 终端输出 log-macro-release-out

```
(half 自己那份 LOG) half(6) 得到 3
正式输出：6 的一半是 3
```

把这份输出与 16.8 节那份**对着看**，差别只有一处：三条 `[src/main.c:…]` 日志**全部消失**，只剩 `printf` 那一行正式输出——以及 `(half 自己那份 LOG) …`。最后这条很值得说一句：**它还在，是因为它不归 `log.h` 管**（`log_macro.c` 里那份 LOG 是那个文件自己写的，没有 `NDEBUG` 开关）。这正好把 16.10 节的主题提前摆出来了：交付宏必须统一在一处，各写各的就没法统一开关。

c examples/ch07/include/log.h 第 11–16 行

```
11  #ifndef NDEBUG 调试版：真打日志
12  #define LOG(fmt, ...)                                          \
13      printf("[%s:%d %s] " fmt "\n", __FILE__, __LINE__, __func__, __VA_ARGS__)
14  #else
15  #define LOG(fmt, ...) ((void)0) 发布版：整条语句变成空的
16  #endif
```

11 调试版：真打日志
15 发布版：整条语句变成空的

发布版走的是 `#else` 那一支：`LOG(...)` 整个被换成 `((void)0)`。于是宏的**参数一次都不求值**——`LOG("程序开始，n = %d", n)` 里的 `n` 在发布版里根本没被读。代价是：**日志参数里不许放有副作用的东西**（`LOG("%d", i++)` 在调试版

会让 `i` 加一、在发布版不会，同一个程序两个版本行为不同）。

🕮 标准原文 · 标准原文：`NDEBUG` 这个名字是从哪来的

> `NDEBUG` 不是本书发明的，它来自标准库的 `<assert.h>`。7.2 节「Diagnostics `<assert.h>`」里规定：
>
> > If NDEBUG is defined as a macro name at the point in the source file where `<assert.h>` is included, the assert macro is defined simply as `#define assert(ignore) ((void)0)`
>
> 大意：**如果 `NDEBUG` 被定义成宏名**（在哪一行定义的、是不是在命令行上定义的都行），那么 `assert` 这个宏就变成「什么都不做」。标准自己就是这样示范「交付版把调试代码整段拿掉」的——`#ifndef NDEBUG` 是照它的做法推广出来的**惯例**。
>
> 出处：ISO/IEC 9899:2018（C17）§7.2。

⚠ 陷阱 · 发布版里「什么都没有」，你会这样被坑到

你会被这样坑到：调试版里日志打得清清楚楚，发布版里什么都不打。你会这样误判：以为「发布版也走了这些代码，只是不打印」。**真相是这些代码根本不存在**：`((void)0)` 之后没有任何东西，`n` 没有被读、字符串没有进二进制、`printf` 也没有被调用。

三个由此而来的坑：

- **参数里的副作用会消失**：`LOG("%d", i++)` 两个版本行为不同；
- **变量可能变成「未使用」**：如果一个变量**只**出现在日志里，调试版没问题，发布版会给你一条 `-Wunused-variable`；
- **别用它做业务逻辑**：日志宏只负责日志。要在发布版也执行的检查，用函数、用返回值，不要藏在 `#ifndef NDEBUG` 里。

16.10 宏没有作用域：它属于包含它的那个编译单元

函数有作用域：写在一个函数体里的变量，出了花括号就没了。**宏没有这回事**——它管的是**文本范围**：从 `#define` 那一行起，一直到文件末尾（或者被 `#undef` 掉），中间隔着多少个花括号都不影响。下面这个例子把 `#define` 写在了**函数体里面**：

c examples/ch07/macro_basics/src/macro_basics.c 第 3–11 行

```c
int times_two(int n) {
#define SCALE 2                 /* 写在函数体里也一样：它不是局部变量 */ 写在函数体里
    return n * SCALE;
}

/* 出了 times_two 的花括号，SCALE 一样还在——所以这一行才能用它 */
int times_two_plus_one(int n) {
    return n * SCALE + 1; 出了花括号照样能用
}
```

4 写在函数体里
10 出了花括号照样能用

`times_two` 的花括号在类型系统里是实实在在的边界，对宏却不存在：第 10 行的 `n * SCALE` 照样能编过，跑出来的结果也证明它被换成了 `n * 2 + 1`（`times_two_plus_one(21) = 43`，见 16.3 节的输出块）。**写在函数体里，是作者的排版选择，不是作用域。**

比「花括号管不住它」更要紧的是**文件边界**：宏属于**包含它的那个编译单元**，与别的编译单元完全隔绝。

第 1 帧是 `log.h` 的正常用法：`#include` 把替换规则搬进 `src/main.c` 这个编译单元。第 2 帧是关键：`src/log_macro.c` 里**没有** `#include "log.h"`，所以 `log.h` 里那份 LOG 进不去——第 3 帧就是这件事的结果：`log_macro.c` 自己写了一份同名的 LOG，两份定义各自活在自己的编译单元里，**既不冲突，也不共享**。证据就在 16.8 节的输出里：同一份程序打印出两种完全不同格式的日志。

那 `src/log_macro.c` 那份自己写的 LOG 长什么样：

c examples/ch07/log_macro/src/log_macro.c 第 1–13 行

```c
/* 这个文件故意不 #include "log.h"：宏不会自己走过来。 */
   所以下面这一份 LOG 与 log.h 里那份同名，却是完全不同的两回事。 */
#include <stdio.h>

#include "log_macro.h"

#define LOG(fmt, ...) printf("(half 自己那份 LOG) " fmt "\n", __VA_ARGS__)

int half(int n) {
    int h = n / 2;
    LOG("half(%d) 得到 %d", n, h);
    return h;
}
```

1 故意不包含 log.h

7 另一份同名定义，格式完全不同

标准原文 · 标准原文：同一个名字不许定义两遍

「宏属于一个编译单元」的另一面是：**在同一个编译单元里**，同一个名字不允许被定义成两份不同的东西。6.10.3 节「Macro replacement」第 2 段：

> An identifier currently defined as an object-like macro shall not be redefined by another `#define` preprocessing directive unless the second definition is an object-like macro definition and the two replacement lists are identical.

大意：一个**已经被定义过**的名字，不许用另一条 `#define` 重新定义，除非两份的替换列表**完全相同**。原文这一条只说对象宏；紧接着的第二句把函数式宏也管上了——还要求**参数的个数与拼写完全相同**（`#define F(a)` 之后再 `#define F(b)`，一样算重定义）。标准对「相同」的解释是：预处理记号的个数、顺序、拼写相同，**空白分隔也相同，但所有空白分隔被视为同一种**——所以 `1 + 2` 与 `1 + 2` 是同一个替换列表（不报），`1 + 2` 与 `1+2` 不是（报 `redefined`）。想要换一份定义，得先用 `#undef` 把旧的关掉。

出处：ISO/IEC 9899:2018（C17）§6.10.3 第 2 段。

工程惯例 · **工程惯例：`#undef` 用来「就地换一份定义」**

`#undef` 是唯一的关闭手段，它买到的是「**这一段用这份定义，那一段用那份**」。怎么用它是**惯例**，不是标准要求（标准只规定 `#undef` 会让这个名字从此不再是宏）。最常见的两个用法：

- **换掉一个已有的宏值**：想给 `ARRAY_MAX` 换个大小，先 `#undef ARRAY_MAX` 再 `#define ARRAY_MAX 20`（不 `#undef` 直接重定义，编译器会给你 `warning: 'ARRAY_MAX' redefined`）；
- **不污染别人**：自己写了个临时宏，用完立刻 `#undef` 掉，免得它被下面的 `#include` 带进别的文件。

代价是「读代码时要知道宏现在还活着没有」——`#undef` 一多，读者就得一路往上找最后一处 `#define`。下一单元（第 17 单元）会把这件事推到头：一个头文件被两个源文件包含时，`#ifndef` 守门与 `#undef` 的分工。

16.11 条件编译：同一份源码，两个程序

条件编译做的是「**留着或删掉**」：`#if` 为假的那一段，连编译都不会编。这一节的项目 `cond_compile` 源码完全不动（两个源文件 `src/main.c` 与 `src/cond_compile.c` 一个字符都不改），只换命令行上的一个 `-DLEVEL`，同一个项目就编出两个程序：

c examples/ch07/cond_compile/include/cond_compile.h 第 4–8 行

```c
/* 没写 -DLEVEL 时在这里给自己一个默认值：
   默认值只写这一处，两个源文件都会看到同一个 LEVEL。 */
#ifndef LEVEL 没写 -DLEVEL 时
#define LEVEL 1 默认值只在头文件里写一次
#endif
```

6 没写 -DLEVEL 时
7 默认值只在头文件里写一次

默认值有了，先看那个「只在 LEVEL 为 2 时才存在」的函数——它的整段都住在一个 `#if` 里：

c examples/ch07/cond_compile/src/main.c 第 5–10 行

```c
/* 只有 LEVEL >= 2 才存在的函数：LEVEL=1 时连符号都不会有 */
#if LEVEL >= 2 条件为假时，整个函数消失
static int extra_add(int a, int b) {
    return a + b;
}
#endif
```

6 条件为假时，整个函数消失

再看 `main`。它体内还有一处 `#if`，决定打印哪一行；末尾那个 `#error` 是兜底：如果哪天有人给了一个既不是 1 也不是 2 的值，编译期就会停下：

c **（续）main 里的那一处条件与兜底** examples/ch07/cond_compile/src/main.c 第 12–25 行

```c
int main(void) {
    printf("这份程序是用 LEVEL = %d 编出来的\n", level_of());

#if LEVEL >= 2
    printf("额外功能：extra_add(2, 3) = %d\n", extra_add(2, 3));
#elif LEVEL == 1
    printf("基础功能：这一份里没有那个额外函数\n");
#else
#error "LEVEL 只能是 1 或 2"
#endif

    printf("main 里看到的 LEVEL 也是 %d\n", LEVEL);
    return 0;
}
```

`extra_add` 这个函数只住在 `#if LEVEL >= 2` 里面，所以不写 `-DLEVEL` 的时候它**根本不存在**。两次编译的输出：

终端输出 终端输出 cond-compile-default-out

```
这份程序是用 LEVEL = 1 编出来的
基础功能：这一份里没有那个额外函数
main 里看到的 LEVEL 也是 1
```

终端输出 终端输出 cond-compile-out

```
这份程序是用 LEVEL = 2 编出来的
额外功能：extra_add(2, 3) = 5
main 里看到的 LEVEL 也是 2
```

第一份是不写 `-DLEVEL` 的（头文件给了默认值 1），第二份是清单里写着 `-DLEVEL=2` 的。两条路径的差别是**整段代码的增减**：

图 / 搜索过程 **同一份源码的两条编译路径：条件编译把别的代码删掉了**

第 1 步 源码里同时写着两条路

`src/main.c`：一份源码　`main()`：两条路共用的入口　`extra_add()`：只在 LEVEL 为 2 时才编进来

第 2 步 不写 -DLEVEL，默认 LEVEL = 1

LEVEL = 1：来自 cond_compile.h 里的默认值 ← 本步新增　`extra_add()`：被 #if 删掉，连符号都没有

main() → LEVEL = 1 编出来的程序只有基础那一行

第 3 步 命令行写 -DLEVEL=2

LEVEL = 2：来自清单 flags 里的 -DLEVEL=2 ← 本步新增　`extra_add()`：进了这份程序，符号表里有它

main() → LEVEL = 2 多出一条「额外功能」

两张图对着看：第 2 帧里 `extra_add` 不是「没被调用」，是**被删掉了**；第 3 帧它才回到程序里。这不是我嘴上说的，`nm` 与 `gcc -E` 都能数出来：

终端会话 两次编译 + 两次 nm：没编进去的东西不存在 ch07-nm-levels

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/cond_compile.c -o lvl1.exe
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DLEVEL=2 src/main.c src/cond_compile.c -o lvl2.exe
$ nm lvl1.exe | grep -c extra_add
0
$ nm lvl2.exe | grep -c extra_add
1
$ gcc -std=c17 -Iinclude -E src/main.c | grep -c extra_add
0
$ gcc -std=c17 -Iinclude -DLEVEL=2 -E src/main.c | grep -c extra_add
2
```

怎么读这四行字：`nm` 列出可执行文件里的**符号**（函数名、变量名），`grep -c` 数的是「有几行命中」。LEVEL=1 那份里 `extra_add` 命中 **0** 行——符号表里没有它；LEVEL=2 那份命中 **1** 行——它就是被编进去了。后两条更彻底：直接在**预处理结果**里数，0 与 2 的差别说明在第一步文本处理时，那几行就已经被删掉了，后面的编译、汇编、链接压根看不到它。

实验记录 · **本机实测：两次编译与四条读数**

- 工具链：gcc 16.1.0（`-std=c17`，即 C17），目标三元组 x86_64-w64-mingw32，工作目录 `examples/ch07/cond_compile/`；
- 两条编译命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/cond_compile.c -o lvl1.exe`，以及同一条命令加 `-DLEVEL=2`（输出到 `lvl2.exe`）；两条都退出 **0**；
- 四条读数：`nm lvl1.exe | grep -c extra_add` = **0**、`nm lvl2.exe | grep -c extra_add` = **1**、`gcc -std=c17 -Iinclude -E src/main.c | grep -c extra_add` = **0**、加上 `-DLEVEL=2` 后 = **2**；
- 这四条读数来自抓下来的会话 `examples/ch07/session-nm-levels.txt`。

实验记录 · （续）本机实测：两次真实运行

- `lvl1.exe` 与 `lvl2.exe` 都真的跑过：前者打印「LEVEL = 1」那一份的三行，后者打印「额外功能：extra_add(2, 3) = 5」，退出码都是 **0**；
- 这两次运行的 stdout 分别对上 16.11 节的两段 `output` 块；
- 这份例子是**同一份源码配两个清单**（`cond_compile.json` 带 `-DLEVEL=2`，`cond_compile_default.json` 不带），所以「两条路径」不是一个程序里的两个分支，而是**两个可执行文件**。

标准原文 · 标准原文：`#if` 里的名字不是变量

> 条件编译的表达式怎么算，6.10.1 节「Conditional inclusion」有规定：
>
> > After all replacements due to macro expansion and the `defined` unary operator have been performed, all remaining identifiers (including those lexically identical to keywords) are replaced with the pp-number 0 …
>
> 大意：先把宏展开、把 `defined` 算完，然后**剩下还没被替换掉的名字一律当成 0**。所以 `#if LEVEL >= 2` 这句话里，`LEVEL` 不是变量，它就是「一个宏」；它没被定义过就等于 0，`#if` 因此为假。反过来，`#ifdef LEVEL` 问的是「这个名字**到这一点为止**被定义成宏了吗」——这正是理解 `#undef` 的关键：同一次编译里，同一个名字在文件的前半段与后半段可以是「有」或「没有」。
>
> 出处：ISO/IEC 9899:2018（C17）§6.10.1。

条件编译在工程里有三个真实用途，这一节**三个都演了**：前两个在下面，第三个（开关式调试代码）在 16.9 节用 `-DNDEBUG` 演过。顺便说清写法上的四种口吻，它们两两等价：`#ifdef X` 就是 `#if defined X`，`#ifndef X` 就是 `#if !defined X`。`defined` 是条件编译里唯一一个预先认识的运算符：它问的不是「这个名字的值是多少」，而是「**这个名字现在是不是一个宏**」。

1. **include guard（头文件的守门人）**：`#ifndef X_H / #define X_H / … / #endif` 保证同一个头文件在一次编译里只被念一遍。这一节每个头文件都用它（`cond_compile.h` 第 1、2、12 行就是），而它的完整故事属于第 17 单元——那时你会看到**不加**守门会撞成什么样；
2. **按平台 / 工具链分支**：同一个项目要给 Windows 和 Linux 编，`#if` 那些平台宏（下一节的 `TARGET_OS` 就是这类开关）选不同的实现；`gcc -dM -E - < /dev/null` 还能把编译器预定义的全部平台宏列出来——它们是**工具链**给的，不是语言给的；
3. **开关式调试代码**：整段整段的调试打印、性能计数，用 `#if` 包起来，编发布版时**一个字都不进二进制**（这与 16.9 节那个 `#ifndef NDEBUG` 是同一件事，只是粒度更大）。

陷阱 · 条件编译最坑人的地方：你读到的代码，不一定是它编出来的代码

你会被这样坑到：翻着源码排查问题，看见 `extra_add(2, 3)` 明明写在那儿。你会这样误判：源码里写着，就认定它被执行过。**真相是那几行可能根本没进二进制**——源码里写着的东西，要经过「条件编译→宏展开」两道剪切才轮到编译器看。

三个立即可用的排查动作：

- 想知道「这段到底编进去没有」，用 `gcc -E` 数一数，或者 `nm` 看符号；
- 想知道「这个开关现在是什么值」，在可疑的地方 `#if LEVEL != 2 / #error 不是 2 / #endif`——让编译器把你想要的事实**打在诊断里**（下一节就是这么干的）；
- 别用条件编译藏**业务逻辑**：同一个函数体在不同 `-D` 下做不同的事，是 bug 的温床；条件编译留给平台差异与调试开关。

16.12 `#error`、`#warning`、`#pragma`：把「不支持」写在编译期

条件编译最值钱的用法之一，是**把不该编的情况挡在门口**。下面这份程序要一个平台开关 `TARGET_OS`，值不对就由编译期开口。先看开关本身——三层条件，各管一种情况：

c examples/ch07/error_warning/src/main.c 第 5–12 行

```c
/* 平台值不对就别编了：让编译期说出这句话，比运行期崩在用户机器上便宜 */
#if !defined(TARGET_OS)
#error "请用 -DTARGET_OS=1 或 -DTARGET_OS=2 指明目标平台"
#elif TARGET_OS == 2
#warning "TARGET_OS=2 这一支没有在本机测过"
#elif TARGET_OS != 1
#error "TARGET_OS 只能是 1 (Windows) 或 2 (Linux) "
#endif
```

7 没给值就直接报错
9 选到这一支只警告
11 值不合法在这里停下

开关后面才是程序本体。注意 `main` 也被包在一个条件里——这样不合法的那些情况下，编译器只会报上面那一条 `#error`，不会顺手再抱怨「`main` 不存在」：

c （续）本体：只有合法的平台值才编 main examples/ch07/error_warning/src/main.c 第 14–19 行

```c
#if TARGET_OS == 1 || TARGET_OS == 2
int main(void) {
    printf("这份程序的目标平台：%s\n", platform_name());
    return 0;
}
#endif
```

这份例子的清单里写着 `-DTARGET_OS=7`（一个不合法的值），于是编译当场停住。下面是 gcc 真的吐出来的原文：

编译诊断 gcc 的真实诊断（原文照抄） error-warning-diag

```
src/main.c:11:2: error: #error "TARGET_OS 只能是 1 (Windows) 或 2 (Linux) "
   11 | #error "TARGET_OS 只能是 1 (Windows) 或 2 (Linux) "
      |  ^~~~~
```

这条诊断很短，但信息密度很高：`src/main.c:11:2` 是**你自己写的那一行**（`#error` 在第 11 行），引号里那串字就是你在 `#error` 后面写的话，一个字都没被加工。**这就是「把「这个平台不支持」变成编译期的一句话」**：不用等程序跑起来，不用看日志，编译的人第一眼就知道该加哪个参数。

三条结构设计值得抄走：

- **`#if !defined(X)` 先管「没给」、`#elif X != 1` 再管「给了但不合法」**：两条诊断分别把人往正确用法和取值范围上引；
- **`main` 本身也在条件里**（第 14 行）：不合法的那些情况下，编译器不会顺手再报「`main` 不存在」——**一条诊断说清一件事。**

`#warning` 适合「能用，但我得提醒你一句」的场合。把开关改成 `-DTARGET_OS=2`，编译照样成功，只是会多出**两条** warning——一条是 `-Wpedantic` 说 `#warning` 本身是扩展，另一条才是你写的那句话：

终端会话 #warning：编得过，但要说一句 ch07-warning

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DTARGET_OS=2 src/main.c src/error_warning.c -o app-linux.exe
src/main.c:9:2: warning: '#warning' before C23 is a GCC extension [-Wpedantic]
    9 | #warning "TARGET_OS=2 这一支没有在本机测过"
      |  ^~~~~~~
src/main.c:9:2: warning: #warning "TARGET_OS=2 这一支没有在本机测过" [-Wcpp]
$ ./app-linux.exe
这份程序的目标平台：Linux
```

这里有个**很值得记住**的细节：`-Wpedantic` 先给你一条**额外的**警告，说 `#warning` 在本书用的这个标准版本里还不是标准语法，它到 2024 年定稿的下一版标准（发展期叫 C2x）才被收进语法。程序编出来了，也跑起来了（输出 `这份程序的目标平台：Linux`）。

标准原文 · **标准原文：`#error` 做什么**

6.10.5 节「Error directive」：

> A preprocessing directive of the form `# error pp-tokens opt new-line` causes the implementation to produce a diagnostic message that includes the specified sequence of preprocessing tokens.

大意：`#error` 让实现**产生一条诊断**，诊断里**包含你写的那串字符**。标准只说「要有诊断」，没说它必须是 error 还是 warning，也没说编译是否必须停——各家实现自己定。另外，`#warning` 在本书这个标准版本里**不是预定义指令**，所以严格说它是工具链扩展，而不是标准语法。

出处：ISO/IEC 9899:2018（C17）§6.10.5。

`#pragma` 是同一类东西的另一个门类：它的意思是「对实现说一句话」。它的行为**由实现自己定义**，谁都不认识的那些则必须被忽略。先用一个单文件探针看看这两种情况：

c examples/ch07/pragma_probe.c 第 3–8 行

```c
#pragma once gcc 认识这个名字
#pragma this_pragma_does_not_exist 它从没听说过这个名字

int main(void) {
    return 0;
}
```

3 gcc 认识这个名字
4 它从没听说过这个名字

编译这个探针的会话如下。注意它一条错都没报：两个 pragma、两条提醒、程序照跑：

终端会话 认识的 pragma 有一句提醒，不认识的被忽略 ch07-pragma

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g pragma_probe.c -o pragma_probe.exe
pragma_probe.c:3:9: warning: '#pragma once' in main file [-Wpragma-once-outside-header]
    3 | #pragma once
      |         ^~~~
pragma_probe.c:4: warning: ignoring '#pragma this_pragma_does_not_exist ' [-Wunknown-pragmas]
    4 | #pragma this_pragma_does_not_exist
$ ./pragma_probe.exe
$ echo $?
0
```

两个 pragma、两种命运：`#pragma once` 是 gcc **认识**的（它做的事是「这个头文件只念一遍」），所以它没报「不认识」，只是提醒你「你把它写在 `.c` 文件里了，它通常出现在头文件里」；`#pragma this_pragma_does_not_exist` 是谁都不认识的名字，gcc 说 `ignoring ...`（**忽略**它）然后继续——程序编出来了，退出码 **0**。

标准原文 · **标准原文：`#pragma` 的分工**

6.10.6 节「Pragma directive」：

> A preprocessing directive of the form `# pragma pp-tokens opt new-line` causes the implementation to behave in an implementation-defined manner. The behavior might cause translation to fail or cause the translator or the resulting program to behave in a non-conforming manner. Any pragma that is not recognized by the implementation is ignored.

大意：`#pragma` 让实现「按自己的定义行事」——标准不管它具体干什么，只管两件事：**没被认出来的 pragma 必须被忽略**，以及**认出来的那些可能让程序变得不标准**（所以别指望换一个编译器还一样）。`#pragma once` 就属于后者：gcc、clang、MSVC 都支持它，但它**不是标准的那一种写法**；标准支持的写法是 `#ifndef` 守门（第 17 单元讲透）。

出处：ISO/IEC 9899:2018（C17）§6.10.6。

16.13 宏不能做的事，与什么时候不该用宏

上面每一节都在说宏能做什么，现在说它**做不到**的两件事——都是这一节的例子真的编不过换来的。

examples/ch07/macro_limits/src/main.c 第 5–13 行

```c
/* 这一份故意编不过：两处都在试「把宏当成变量或函数用」。 */
int main(void) {
    /* 试错一：对象宏没有地址，它不是变量 */
    printf("&ARRAY_MAX      = %p\n", (void *)&ARRAY_MAX); 给宏取地址

    /* 试错二：宏不做参数检查，展开成 ("hi") * 2 之后才撞上类型系统 */
    printf("DOUBLE(\"hi\")    = %d\n", DOUBLE("hi")); 给宏一个它算不了的参数
    return 0;
}
```

8 给宏取地址
11 给宏一个它算不了的参数

编译诊断 gcc 的真实诊断（原文照抄） macro-limits-diag

```
src/main.c: In function 'main':
src/main.c:8:46: error: lvalue required as unary '&' operand
    8 |     printf("&ARRAY_MAX      = %p\n", (void *)&ARRAY_MAX);
      |                                              ^
In file included from src/main.c:3:
include/macro_limits.h:8:24: error: invalid operands to binary * (have 'char *' and 'int')
    8 | #define DOUBLE(n) ((n) * 2)
      |                    ~   ^
      |                    |
      |                    char *
src/main.c:11:40: note: in expansion of macro 'DOUBLE'
   11 |     printf("DOUBLE(\"hi\")    = %d\n", DOUBLE("hi"));
      |                                        ^~~~~~
```

两条诊断，正好对上两条结论：

- `lvalue required as unary '&' operand` —— 第 8 行取 `&ARRAY_MAX` 编不过。宏展开完是 `&10`，而 `10` 是一个**字面量**，不是变量，没有地址。你没法指着宏里那串文字说「它住哪」——因为它**没有住处**，它就是文本；
- `invalid operands to binary * (have 'char *' and 'int')` —— 第 11 行 `DOUBLE("hi")` 展开成 `("hi") * 2`，编译器在这里才把类型问题指出来。**宏自己不检查参数类型**（它连类型是什么都不知道），它把检查的活儿推给「展开之后的编译器」。读这条诊断要分清主次：**主错误在宏定义那一行**（`include/macro_limits.h:8:24`，插入符指着 `((n) * 2)` 里的乘号），紧跟的 `note: in expansion of macro 'DOUBLE'` 才指到你的调用点（`src/main.c:11:40`）。换句话说，编译器骂的是**宏展开出来的那段文字**，调用点只是它被用在哪里。

那要表达「一个常量」该怎么办？同一个例子里的 `macro_limits.c` 给了另外两种写法，而 `macro_basics` 那份输出（16.3 节）打印的是**同名的**那两个名字，可以直接对着看：

examples/ch07/macro_limits/src/macro_limits.c 第 3–6 行

```c
/* 想表达「一个常量」，C 里有三样东西：宏、const、enum，它们不是同类。
   const 是只读变量（有地址）；enum 是整数常量（能当数组大小，但没有地址）。 */
const int max_const = 10; 只读变量：有地址
enum { MAX_ENUM = 10 }; 整数常量：能当数组大小
```

5 只读变量：有地址
6 整数常量：能当数组大小

对照那份输出：`sizeof(max_const)` 与 `sizeof(MAX_ENUM)` 都是 4，`&max_const` 还打出了一个**真实地址**（`00000000005FFEAC`）。但这里必须分清两件事，它们经常被混成一句「都是常量」：

- **`const int max_const = 10;` 是一个只读变量**：有类型（`int`）、有地址（`&max_const` 合法）、调试器里看得见。但在 C 里它**不是编译期常量**——`const int n = 10; int a[n];` 在块作用域会变成变长数组，放在文件作用域直接编不过（本机实测：`error: variably modified 'a' at file scope`）；

- `enum { MAX_ENUM = 10 };` **是一个整数常量**：有类型（`int`）、能当数组大小、能进 `case` 标号，但它**没有地址**——`&MAX_ENUM` 报的是 `lvalue required as unary '&' operand`，跟 `&ARRAY_MAX` 是同一条错（本机实测，两条都在本节的诊断里出现过）。

所以「替代宏」这件事要分开答：**要一个能当数组大小的编译期常量，用 `enum`**（或继续用对象宏）；**要一个只读的、有地址的对象，用 `const`**。宏那条路只有一个名字和一串文字，上面两条诊断就是走不通的证明。第 3 单元的 `literal_zoo` 例子用这三种写法量过同一个数组长度，第 4 单元讲过枚举的取值与类型——它们的取舍在这里收口：**宏是最后手段。**

宏能不能定义「类型安全」的接口

这是宏的天花板。宏在替换的那一刻**看不到类型**（16.2 节说过，预处理只知道一串一串的记号），所以它不可能在「替换之前」检查你传了什么。上一段那条 `DOUBLE("hi")` 的诊断就是最好的说明：报错的主位置在**宏定义那一行**（`include/macro_limits.h:8:24`），`note: in expansion of macro 'DOUBLE'` 才指到调用点——**宏自己一声没吭，是展开之后的编译器看不过去。**这就是「宏不能定义类型安全接口」的全部含义。

C11 起有一个专门为此准备的工具：`_Generic`。它让**编译器**按实参的类型，在几条候选里挑一条，于是「一个名字、多种类型」有了类型检查：

c examples/ch07/generic_dispatch/include/generic_dispatch.h 第 4–9 行

```c
/* 一个名字、两种类型：_Generic 让编译器按实参的类型挑一个函数。
   被挑中的是「真函数」，所以类型检查发生在函数调用上，而不是文本上。 */
#define ABS(x) _Generic((x), int: abs_int, double: abs_double)(x) // _Generic 按实参类型挑一支

int abs_int(int x);
double abs_double(double x);
```

6 _Generic 按实参类型挑一支

c examples/ch07/generic_dispatch/src/main.c 第 5–14 行

```c
int main(void) {
    printf("ABS(-3)   = %d (int 那一支) \n", ABS(-3));   // int 那一支
    printf("ABS(-3.5) = %.1f (double 那一支) \n", ABS(-3.5));   // double 那一支
    printf("ABS(2.5)  = %.1f (正的也一样) \n", ABS(2.5));

    int i = 5;
    printf("选支时求值吗：%s，i 还是 %d\n",
           _Generic((i++), int: "选了 int 那一支", default: "选了别的"), i);   // 控制表达式不求值
    return 0;
}
```

6 int 那一支
7 double 那一支
12 控制表达式不求值

终端输出 终端输出 generic-dispatch-out

```
ABS(-3)   = 3 (int 那一支)
ABS(-3.5) = 3.5 (double 那一支)
ABS(2.5)  = 2.5 (正的也一样)
选支时求值吗：选了 int 那一支，i 还是 5
```

三行输出说明它按类型选了不同的函数（`int` 走 `abs_int`、`double` 走 `abs_double`），最后一行说明另一件事：**挑分支的时候，被检查的那个表达式不求值**——`i` 仍然是 5。它的边界也要记住：

- 它**只在编译期选一支**，其余候选里的表达式仍然必须**类型正确**；所以它能让「一个名字支持几种类型」，不能替代函数重载；
- 它仍然是**宏**：括号纪律、求值次数、没有作用域，一条都不少（`ABS` 展开后 `x` 也出现两次）；
- 它是 C11 才有的东西，写法比函数长；只有当「一个名字要服务多种类型」这件事真的重要时才值得用，否则**写两个函数**最省事。

工程惯例 · 工程惯例：什么时候不该用宏——一张优先级名单

「宏是最后手段」不是口号，它可以排成一张顺序表。**这是一张工程惯例清单，不是标准要求**——标准只说宏能做什么，不告诉你该不该用。想做一件事时，从上往下问：

1. **能写成函数吗？** 能就用函数：有类型、参数只求值一次、能单步、能取地址（第 7 单元）；
2. **函数调用太贵吗？** 先加 `static inline`（放在头文件里，编译器可以就地展开），而不是回到宏——它两者兼得：有类型检查，又没有调用开销；
3. **要一个常量吗？** 要能当数组大小，用 `enum`；要一个只读对象，用 `const`（两者的差别见上一段）。

工程惯例 · （续）工程惯例：什么时候不该用宏

接着上一条名单往下问（这仍是工程惯例，不是标准要求）：

4. **要「一个名字、多种类型」？** C11 起有 `_Generic`，它按类型挑一支，能保住类型检查（见上面那个例子）；
5. **只剩这几件事才用宏**：`#` 字符串化、`##` 拼名字、`__FILE__` / `__LINE__`、条件编译（`-D` 开关）、以及那种「必须知道调用点上下文」的短小展开（比如 `LOG`、`assert`）。

代价与理由都写在这一节的例子里：宏每一次「省事」，都在类型检查、求值次数、调试信息这三件事上收利息。真要用，就守住本节的四条纪律——全大写、参数与整串都加括号、多语句包 `do…while(0)`、用完 `#undef`。

16.14 这一节新增了什么，以及接下来

回到 16.1 节那棵树，现在可以对着**第 5 帧**逐个点名了：

- `examples/ch07/include/log.h` —— 一个**专门放宏**的头文件：没有函数定义，只有 include guard、一个 `#include`，以及两份 `LOG` 定义（调试版与发布版各一份）。书里更早的头文件也夹带过宏（第 3 单元 `bitops.h` 里的几个标志位、每个头文件的 include guard），但那些是顺带的；这里宏是唯一的主角；
- `examples/ch07/macro_basics/`、`macro_parens/`、`macro_side_effect/`、`macro_limits/` —— 宏是文字、括号纪律、求值两次、宏做不到的事，四个例子各配清单；
- `examples/ch07/do_while_macro/`、`do_while_bad/` —— 多语句宏的正解与反例（后者是 `compile_fail`，诊断原文进了正文）；
- `examples/ch07/log_macro/` —— 一个 LOG 宏，三份清单（普通、带 `-DNDEBUG`、零参数的 `compile_fail`），三段真实输出/诊断；
- `examples/ch07/stringize_paste/`、`cond_compile/`、`error_warning/`、`generic_dispatch/` —— `#` 与 `##`、同一份源码的两条路、编译期报错、`_Generic` 的类型分派；
- `examples/ch07/session-*.txt`（本节自己抓的六段：两段预处理、一段 `nm`、一段求值警告、一段 `#warning`、一段 `#pragma`）与 `pragma_probe.c`（一个单文件探针）—— 正文里的 `gcc -E`、`nm`、警告与退出码都从它们来。

下一个单元（第 17 单元）接着讲**多文件**：头文件里放什么、一个头文件被两个源文件包含时会发生什么（include guard 的正解就在这里）、`static` 与 `extern` 决定「谁能看见谁」；再往后第 18 单元讲 gdb 与排错套路。这一节留下的宏纪律——**全大写、括号、`do…while(0)`、`#undef`**——后面每一章都继续生效。

工程惯例 · 工程惯例：一条命令编完，是本机的现实

本书的编译参数里固定有 `-Wall -Wextra -Wpedantic -O0 -g`，源文件全部列在一条 `gcc` 命令里，例如：

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/macro_basics.c -o app
```

这不是标准要求的，是这台机器的现实：本机**没有 make、没有 cmake**（`build/toolchain.json` 的 `unsupported` 里写着）。工程大了以后，你迟早要写构建脚本或 Makefile 来拼这条命令；在那之前，**把 gcc 的那行命令行记熟是划算的**——每个 `-I`、每个 `-D`、每个源文件的位置，你都要能自己说清。

小结 · 这一节的要点

- 预处理跑在编译**之前**，它做四件事：文件包含、宏替换、条件编译、行控制；它对**类型一无所知**，所以宏没有类型、没有作用域、也不做参数检查；
- 宏是**文字替换**：`SQ(1+2)` 展开成 `1 + 2*1 + 2` 得 5（要写 `((x) * (x))`）；`SQ(i++)` 展开出两次自增，**标准说这是未定义行为**（本机 gcc 实测 `i` 变 7，但那只是这台机器这一次的结果）；多语句宏要包 `do { … } while (0)`，否则分号会吃掉 `else`；
- 宏只认**文本范围**：它属于包含它的那个编译单元，跨不过 `.c` 的边界，`#undef` 是唯一的关闭手段；条件编译是「留着或删掉」（`-DLEVEL=2` 与不写它编出两个程序，`nm` 与 `gcc -E` 都数得出来），`#error` 把「不支持」变成编译期的一句话；
- `-DNDEBUG` 让 `LOG` 整条消失（连参数都不求值），这是从标准的 `<assert.h>` 推广出来的惯例；常量用 `enum`（能当数组大小）或 `const`（有地址，但不是编译期常量），行为用函数 / `static inline` / `_Generic` —— **宏是最后手段。**

自测题 · 自测题（先自己答，再对答案）

1. `#define SQ(x) x*x`。`SQ(1 + 2)` 算出来是多少？为什么不是 9？把它写成正确的版本。
2. `int i = 5; int a = SQ_OK(i++);`（`SQ_OK` 是加对括号的 `((x) * (x))`）执行完之后 `i` 是多少？如果换成函数 `int sq(int x)`，`i` 又是多少？这两个差别的根源在哪一条标准规定里？
3. 多语句宏为什么通常写成 `do { … } while (0)` 而不是直接包一对花括号？后一种写法放进 `if … else` 会报什么错？
4. `__FILE__`、`__LINE__`、`__func__` 三个里面，哪一个**不是**预处理宏？它由谁、在哪个阶段提供，用在「当前函数名」这件事上凭什么可靠？
5. 同一份 `cond_compile` 源码，一次不写 `-DLEVEL`、一次写 `-DLEVEL=2`。`extra_add` 这个函数在哪一份可执行文件里有符号？写出你用来证明它的两条命令（提示：一条看符号、一条看预处理结果）。

注 · 自测题答案

1. 是 **5**。宏展开是机械的文字替换：`SQ(1 + 2)` → `1 + 2*1 + 2`，乘法先算，等于 1 + 2 + 2 = 5。正确写法是 **`#define SQ(x) ((x) * (x))`**：每个参数加一层括号，整个展开式再加一层（参数那层管数值，外面那层管调用点周围的运算符）。
2. **标准不保证任何具体数值**：`i++` 在展开式里出现两次，两次之间有**没有顺序的副作用**，这就是未定义行为（标准 6.5 节第 2 段）。本机 gcc 16.1.0 的实测结果是宏版 `i` 变 **7**、函数版 `i` 是 **6**——但这只是这台机器这一次的结果，换编译器、换优化档都可能变。所以正确的答法是：**宏版没有确定答案（UB），函数版有确定答案（6）**；看到「本机是 7」不该记住 7，该记住的是「这里踩到了未定义行为」。
3. 因为 `do { … } while (0)` 让整块在语法上等价于**一条语句**，调用点那个分号正好收在 `while (0)` 后面。裸花括号的写法展开后是 `{ … };`，多出来的那个分号成了「空语句」而被当成 `if` 的主体，于是 `if` 提前结束，紧跟的 `else` 报 `'else' without a previous 'if'`（本机 gcc 原文见 16.6 节的诊断块）。
4. **`__func__` 不是宏。** `__FILE__` 与 `__LINE__` 是预定义宏（6.10.8.1 节），预处理阶段就被换成文件名字符串与行号整数；`__func__` 是**标识符**，由编译器在每个函数体开头隐式声明成一个 `static const char __func__[] = "函数名";`（6.4.2.2 节），所以它能在运行期被打印，也所以在预处理结果里原样留着（`grep -c "__func__"` 数得到，`grep -c "__LINE__"` 数不到）。
5. 只有 **`-DLEVEL=2` 那一份**有 `extra_add` 的符号。两条命令：`nm lvl2.exe | grep -c extra_add` 得到 1（`lvl1.exe` 得到 0），以及 `gcc -std=c17 -Iinclude -DLEVEL=2 -E src/main.c | grep -c extra_add` 得到 2（不写 `-DLEVEL` 时得到 0）——后者说明那几行在预处理阶段就被删掉了，不是「编进去但没被调用」。

多文件工程：头文件、编译单元与链接性

◎ 学习目标 · 看完这一单元，你应该能

- 说出一个 `.c` 被单独编成 `.o` 之后还差什么，以及两份 `.o` 之间的「名字」是在哪一步对上的；
- 把一份单文件草稿拆成 `main.c` + `greet.c` + `greet.h`，并说清哪一类东西必须进头文件、哪一类必须留在源文件；
- 看见本机那行 `collect2.exe: error: ld returned 5 exit status` 时，用 `nm` 把「少定义」与「重复定义」分开；
- 用 `static` / `extern` 把符号的可见范围写清楚，并说出「头文件里的 `const` 要加 `static`」是为什么；
- 给一个模块图指出依赖方向、用前向声明破掉真的循环依赖，并把它分成「一条命令」与「分步编译」两种做法。

17.1 先看工程长什么样

上一单元（c16 预处理与宏）讲的是**符号在编译之前怎么变形**；这一单元讲的是**符号在编译之后怎么汇合**。它要回答的问题只有一个：我把程序拆成好几个文件之后，编译器凭什么还能把它们拼成一个可执行文件。

照老规矩，先看文件。这一节的起点很朴素：`examples/` 下已经躺着前面几章的项目；第 7 章的目录由本章上一单元（c16 预处理与宏）建了起来，里面是宏的实验。本节要做的是往同一个目录里，从一个单文件草稿开始，长出一个三层工程。

目录树 **examples/ch07/：从一个单文件草稿长成三层工程**

第 1 步 本节开始时，前面六章的项目在位，第 7 章的项目已由上一单元建起

```
examples/
├── ch01/ … ch06/ ：前面几章的项目（本节不动）
└── ch07/ ：第 7 章的项目，里面是上一单元的宏实验
```

第 2 步 先写一个单文件草稿，它跑得起来，但只能自己用

```
examples/
├── 前面六章的项目 ： 本节不动
└── ch07/
    ├── 宏实验目录 ： 上一单元留下的，本节不动
    └── draft.c ： 一份草稿，16 行 ← 本课新增
```

第 3 步 拆成最小工程：源文件进 src/，声明进 include/

```
examples/ch07/
├── draft.c ： 草稿留着，当对照
└── two_files/ ← 本课新增
    ├── src/ ← 本课新增
    │   ├── main.c ← 本课新增
    │   └── greet.c ← 本课新增
    ├── include/ ← 本课新增
    │   └── greet.h ← 本课新增
    └── two_files.json ← 本课新增
```

第 4 步 把两类链接错误做成能复现的例子

```
examples/ch07/
├── two_files/ ： 上一帧的最小工程
├── undefined_helper/ ： 声明了没定义 ← 本课新增
└── double_definition/ ： 两份 .c 都定义同一个函数 ← 本课新增
```

第 5 步 重复包含与链接性，五组对照

```
examples/ch07/
├── no_guard/ ： 没有 guard，两次包含就重定义 ← 本课新增
├── guarded_include/ ： guard 与 #pragma once 各一份 ← 本课新增
├── header_function/ ： 头文件里放函数定义 ← 本课新增
├── static_scope/ ： 两个 .c 各有同名 static ← 本课新增
└── extern_shared/ ： 头文件里 extern，一处定义 ← 本课新增
```

第 6 步 头文件里的 const、不完全类型与循环依赖

```
examples/ch07/
├── const_in_header/ : const double PI 进了头文件 ← 本课新增
├── forward_decl/ : struct Node; 加指针 ← 本课新增
├── cycle_before/ : 两个头文件互相 #include ← 本课新增
└── cycle_after/ : 前向声明破坏 ← 本课新增
```

第 7 步 三层工程、分步编译与抓下来的会话

```
examples/ch07/
├── 前面几帧的例子目录 : two_files / undefined_helper / … / cycle_after
├── layered_app/ : io.c / logic.c / main.c ← 本课新增
├── split_build/ : 分步编译的对照例 ← 本课新增
├── build.sh : 分步编译的脚本 ← 本课新增
├── session-link-errors.txt ← 本课新增
├── session-ld.txt ← 本课新增
├── session-nm.txt ← 本课新增
├── session-build.txt ← 本课新增
├── session-build-sh.txt ← 本课新增
├── session-include-path.txt ← 本课新增
├── session-plain-run.txt ← 本课新增
└── session-gdb-multifile.txt ← 本课新增
```

第 1 帧是本节故事的起点：`examples/` 下是前面六章的项目，第 7 章的目录已经由本章上一单元（c16 预处理与宏）建好，里面躺着宏实验。第 2 帧先落下一个 `draft.c`——它确实是本节的第一版程序，但它是个反面教材：所有东西挤在一个文件里，别人想用 `greet_banner` 就得把整个文件复制走。第 3 帧把它拆成 `two_files/`：**源文件进 `src/`、头文件进 `include/`、编译参数进 `.json` 清单**，这就是本书从第 2 个项目起一直用的最小工程形状。第 4 到第 6 帧是七组对照例子，每一组只演示一件事（少定义、重复定义、没有 guard、guard 与 `#pragma once`、头文件里放函数定义、链接性、头文件里的 `const`、不完全类型、循环依赖）。第 7 帧是本节的两件沉淀物：一个三层工程（带一个 `build.sh`）和八个抓下来的终端会话。

先记住每一组例子对应哪一件事，后面会逐个点名：

- `two_files`、`undefined_helper`、`double_definition` —— 一个工程的**正面**与两类**链接期**的失败（17.2 到 17.5 节）；
- `no_guard`、`guarded_include`、`header_function` —— 重复包含的症状与两种修法（17.6 节）；
- `static_scope`、`extern_shared`、`const_in_header` —— 链接性：谁看得见谁（17.7 节）；
- `forward_decl`、`cycle_before`、`cycle_after` —— 不完全类型，以及真的循环依赖怎么破（17.8 节）；
- `layered_app`、`split_build`、`build.sh` —— 分层、一条命令与分步编译（17.8、17.9 节）。

工程惯例 · **工程惯例：一个模块 = 一对文件**

标准对这件事只有一个要求：一个程序有且只有一个 `main`。**分几个文件、文件叫什么名字，全是惯例。**本书的惯例是：

- **一个模块 = 一对文件**：`greet.h` 里放声明，`greet.c` 里放定义，名字必须一样；
- **头文件在 `include/`，源文件在 `src/`**，编译时加 `-Iinclude`；
- **依赖方向单一**：`main.c` 依赖 `logic.h`，`logic.c` 不依赖 `main.c`——箭头永远只朝一个方向走；
- **谁用谁包含**：一个 `.c` 里用到什么，就 `#include` 什么，不靠「别人替我包含过了」。

好处有三条：改一个 `.c` 只需要重编它自己；文件之间的接缝写在 `#include` 这一行上；别人接手时，看头文件就知道这个模块对外承诺了什么。

代价也说清楚：你要多学三件事——include guard、链接性、以及怎么在报错时分辨「编译期」和「链接期」。这一单元就是这三件事。

17.2 编译单元：每个 `.c` 自己编，符号在链接期汇合

先把反例摆上来。这是 `examples/ch07/draft.c`，本节的第一版程序，一个文件，16 行，跑得通：

c examples/ch07/draft.c 第 1–16 行

```c
/* 本节的第一版：一个文件的草稿。它跑得起来，但只能自己用。
   下面的 two_files/ 会把它拆成 main.c + greet.c + greet.h 三个文件。 */
#include <stdio.h>

static const char *greeting = "hello from draft.c";
int greet_banner(const char *who) {
    return printf("[%s] %s\n", who, greeting);
}

int main(void) {
    printf("greet() 返回 %s\n", greeting);
    int written = greet_banner("draft.c");
    printf("greet_banner 写出的字节数 = %d\n", written);
    return 0;
}
```

5 全大写的名字只在这个文件里；7:一个函数定义，别人拿不到；12:main 直接用了同一个文件里的名字

它没有任何错，但它有个说不出口的问题：**别人没法在只知道「有这么个函数」的前提下用它。** `nm draft.o` 里 `greet_banner` 确实是 `T`（外部链接），链接器那一关它过得去；真正缺的是**声明**和**文件边界**——同事要么把 16 行整段复制过去，要么把整个 `draft.o` 链进来（而那份 `.o` 里还带着一个 `main`，一链就撞）。从复制那一刻起，同一段代码有了两份，改了一处忘了另一处只是时间问题。

拆成工程就是把这件事变成一句 `#include`。先看头文件：

c examples/ch07/two_files/include/greet.h 第 1–10 行

```c
#ifndef GREET_H
#define GREET_H

/* 声明：说出「有这么个函数」，不给它分配任何内存、也不写函数体。 */
const char *greet(void);

/* 声明：打印一行问候，返回 printf 写出的字节数。 */
int greet_banner(const char *who);

#endif /* GREET_H */
```

1 guard 的第一句：问「定义过没有」
2 没有就定义，把门关上
5 函数声明：只有名字与参数表
8 再来一个声明

第 1、2 行的 `#ifndef` / `#define` 就是 include guard，17.6 节专门讲它；现在先看第 5、8 行——那是两句**声明**：它们只说「有这么个函数，它收什么、还什么」，**不产生任何目标码。**注意范围：说的是「这两句声明不产生目标码」，不是「头文件永远不产生目标码」——17.7 节那个 `static const double PI` 和 `static inline` 函数都会在每个 `.o` 里留下东西。

函数体留在源文件里：

c examples/ch07/two_files/src/greet.c 第 1–14 行

```c
#include <stdio.h>

#include "greet.h"

/* static：这个名字只在本编译单元里看得见（17.7 节讲链接性）。 */
static const char *kGreeting = "hello from greet.c";

const char *greet(void) {
    return kGreeting;
```

```
10  }
11
12  int greet_banner(const char *who) {   第二个定义
13      return printf("[%s] %s\n", who, greet());
14  }
```

3 它也包含同一个头文件
6 static：这个名字只在这个 .o 里可见
8 定义：函数体写在这里
12 第二个定义

调用者这一边：

c examples/ch07/two_files/src/main.c 第 1–12 行

```
1   #include <stdio.h>
2
3   #include "greet.h"   只要这一行，就用上了别人写的函数
4
5   int main(void) {
6       const char *msg = greet();   调用 greet()，这里只有一个声明
7       printf("greet() 返回 %s\n", msg);
8
9       int written = greet_banner("main.c");   调用 greet_banner()
10      printf("greet_banner 写出的字节数 = %d\n", written);
11      return 0;
12  }
```

3 只要这一行，就用上了别人写的函数
6 调用 greet()，这里只有一个声明
9 调用 greet_banner()

一条命令把它编出来、跑起来（本机没有 make/cmake，源文件全列在命令里）：

终端会话 加上 -Iinclude 之后：编得过，也跑得起来 ch07-build-two-files

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/greet.c -o
app.exe
$ ./app.exe
greet() 返回 hello from greet.c
[main.c] hello from greet.c
greet_banner 写出的字节数 = 28
$ echo $?
0
```

终端输出 终端输出 two-files-out

```
greet() 返回 hello from greet.c
[main.c] hello from greet.c
greet_banner 写出的字节数 = 28
```

`28` 这个数是真跑出来的：`"[main.c] "` 是 9 个字符，`"hello from greet.c"` 是 18 个，再加换行 1 个，正好 28。它来自 `greet_banner` 第 13 行那句 `return printf(...)` —— `printf` 的返回值就是它写出去的**字节数**。

现在把「谁看得见谁」画出来。这张图是本单元的骨架，后面每一节都在它上面加减边：

谁能看见谁 **谁能看见谁：两个编译单元、一个头文件与链接器**

第 1 步 **只有 main.c 的时候**

main.c（编译单元） ：只有 main 一份代码　　greet ：你在调用的名字

main.c → greet 没有声明，编译器当场报错

第 2 步 **#include "greet.h" 之后，声明进来了**

main.c（编译单元）　　**include/greet.h（头文件） ：里面只有声明** ← 本步新增

main.c → include/greet.h #include "greet.h"
include/greet.h → greet 声明：名字、参数、返回值

第 1 帧是新手最容易踩空的地方：编译器**只认它看见过的名字**，它不会替你去别的文件里找。第 2 帧是 `#include` 干的事——把 `greet.h` 的文字搬进 `main.c`，于是编译器看到了 `const char *greet(void);` 这行声明。第 3 帧是全节的中心：**`greet.c` 是另一次完全独立的编译**，它有自己的一份源文本、自己的名字表；它眼里没有 `main.c` 这回事。第 4 帧是链接期：两个 `.o` 各自带着一张「我欠谁一个名字 / 我提供哪些名字」的清单，链接器就按名字把它们对上。

标准原文 · 标准原文：程序不是一次翻完的

C 语言标准（ISO/IEC 9899，C17）5.1.1.1 节「Program structure」第 1 段：

> A C program need not all be translated at the same time. The text of the program is kept in units called source files... A source file together with all the headers and source files included via the preprocessing directive `#include` is known as a preprocessing translation unit. After preprocessing, a preprocessing translation unit is called a translation unit. ... The separate translation units of a program communicate by (for example) calls to functions whose identifiers have external linkage, manipulation of objects whose identifiers have external linkage, or manipulation of data files. Translation units may be separately translated and then later linked to produce an executable program.

大意：**一个 C 程序不必一次翻译完。**标准把「一个源文件 + 它 `#include` 进来的所有头文件」叫**翻译单元**（translation unit）——这就是我们平常说的「编译单元」在标准里的正式名字。而**不同的翻译单元之间的交流靠的是「有外部链接的名字」**：调用外部函数、读写外部对象。最后一句把整条流水线说完了：**翻译单元可以各自翻译，之后再链接。**

出处：ISO/IEC 9899:2018（C17）§5.1.1.1 第 1 段（含 Note）。

大白话 · 用大白话说这三帧

把每个 `.c` 想成**一间独立的办公室**，每个办公室有一张贴在墙上的名单（符号表）。

- `gcc -c greet.c` 就是让 `greet.c` 那间办公室自己干活：它把 `greet` 记到墙上，标注「**我提供 `greet`**」；
- `gcc -c main.c` 让另一间办公室干活：它把 `greet` 也记到墙上，标注「**我欠一个 `greet`，谁给我？**」；
- `gcc main.o greet.o` 就是**把两张名单摆在桌上对一遍**：欠的有人给 → 拼成程序；欠的没人给、或者两个人各给一份 → 两边都进行不下去。

所以「头文件」不是代码的一部分，它是**传话的纸条**：它让每间办公室知道外面有哪些名字可用，但它一丁点儿机器码都不产生。

⚠ 陷阱 · 「一个 `.c` 编不过，就是那个 `.c` 里写错了」——这个直觉会在链接期骗你

你会看到什么：命令行里 `src/main.c src/greet.c` 一起给，报错却只写着链接器那句 `collect2.exe: error: ld returned 5 exit status`（17.4 节），一个文件名、一个行号都没有。

你会怎么误判：你会回头去 `main.c` 里逐行找——拼写？参数？分号？然后一无所获，因为那两份源文件**都编译成功了**，每一份单独看都完美无缺。

真相：错的是**它们之间。**有了本节这张图，检错顺序就固定了：① 先确认报错发生在**编译**（有 `文件:行:列`）还是**链接**（没有行列，只有 `collect2` / `ld`）；② 是链接期，就去看**符号表**（17.5 节），那里写着每个 `.o` 欠谁、给谁；③ 才回去改代码。**在链接期错误上逐行读源码，是最费时间的一条路。**

17.3 头文件放什么、源文件放什么

上一节的头文件里只放了两行声明，于是问题来了：**头文件是不是只能放声明？** 不是。头文件里能放的东西一共三类，而**判断标准只有一条**：

放在头文件里的	例子	为什么可以
声明（函数的、变量的）	`const char *greet(void);`、`extern int g_hits;`	声明不产生目标码，念几遍都一样
宏	`#define SHARED_LIMIT 3`、include guard	宏是文本替换，编译之前就消失了
类型定义	`struct Point { int x; int y; };`、`typedef …`	只要内容每次都一样，"一个类型"还是同一个类型

反过来，**函数体**和**有初值的变量**属于「定义」，它们会产生目标码、要占内存，所以必须住在**一个** `.c` 里。唯一允许出现在头文件里的定义是「每个编译单元各要一份」的那种，写法是加 `static`——17.7 节会拿 `const double PI` 这件真事把这条规则钉死。

三类东西可以用同一个问题来分：**「如果 5 个 `.c` 各念一遍，会有 5 份吗？」**

- 声明：念 5 遍还是那一个名字、一份实现——安全；
- 宏：念 5 遍就被替换 5 次，但每次替换的结果一样，而且它在预处理结束时已经消失——安全（除非两次定义不同，那会得到 `warning: "N" redefined`）；
- 类型定义：5 个翻译单元各念一遍，只要文本一样，它们描述的就是**同一个类型**（标准 6.2.7 节要求这些声明相容），并不冲突；真正会出事的是**同一个翻译单元里念两遍**——那就是重定义（17.6 节）。
- 函数定义与有初值的变量：每念一遍就**产生一个符号**，5 份就撞车。函数落在 `T`，只读数据落在 `R`，可写数据落在 `D`（有初值）或 `B`（初值为 0）——17.7 节会逐个见到这四个字母。

`extern_shared/include/shared.h` 第 8 行那句 `#define SHARED_LIMIT 3` 就是「宏进头文件」的正面例子：它既是一个可以被所有编译单元共享的常量，又没有产生任何符号。代价是它**没有类型、没有作用域、调试器看不见它**——这三个缺点在 c16 里已经讲过。

`#include "x.h"` 与 `#include <x.h>` 到底差在哪

本节所有命令里都有 `-Iinclude` 这个参数。它到底改变了什么？先把参数去掉，看真实的后果：

终端会话 去掉 -Iinclude：fatal error，一个 .o 都没产出 ch07-no-include-path

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never src/main.c src/greet.c -o app.exe
src/main.c:3:10: fatal error: greet.h: No such file or directory
    3 | #include "greet.h"
      |          ^~~~~~~~~
compilation terminated.
src/greet.c:3:10: fatal error: greet.h: No such file or directory
    3 | #include "greet.h"
      |          ^~~~~~~~~
compilation terminated.
```

`fatal error` 是最硬的一种诊断：**编译在这里停住，一个 `.o` 都不会产出。**注意它列了两条——`main.c` 和 `greet.c` 都被头文件卡住了，这正是「每个 `.c` 独立面对同一份头文件」的另一种表现。

加上 `-Iinclude` 之后（也就是上一节那段会话第 12 到 18 行），同样的命令编过了。两句话解释清楚：

- `#include "greet.h"` 用**双引号**：先在**当前源文件所在目录**找，找不到再按 `<...>` 的规矩找；
- `#include <stdio.h>` 用**尖括号**：直接在**系统头文件目录**里找；
- `-Iinclude` 往「头文件目录」这张表的最前面插了一条：`include/` **也是头文件目录。**于是 `"greet.h"`（以及任何 `<...>` 形式）都能在这里被找到。

标准原文 · **标准原文：引号形式找不到时会退回去**

C 语言标准 6.10.2 节「Source file inclusion」第 3 段：

> A preprocessing directive of the form `# include "q-char-sequence" new-line` causes the replacement of that directive by the entire contents of the source file identified by the specified sequence between the `"` delimiters. The named source file is searched for in an implementation-defined manner. If this search is not supported, or if the search fails, the directive is reprocessed as if it read `# include <h-char-sequence> new-line` with the identical sequence...

大意：**引号形式先在实现定义的若干地方找；找不到，就退回按尖括号形式再找一遍。**而尖括号形式（同一节第 2 段）找的是 `a sequence of implementation-defined places`——标准刻意只说「由实现决定的一些地方」，**它压根不规定 `stdio.h` 住在哪。**

所以「`-Iinclude` 让 `"greet.h"` 找得到」这件事，是**我们这台机器上的 gcc 这么规定的**：`-I` 目录既进引号形式的搜索表，也进尖括号形式的搜索表（本机实测：加 `-Iinclude` 之后 `#include <greet.h>` 同样能找到）。换一个工具链，细节可能不同——标准把这块全部留给了实现。

出处：ISO/IEC 9899:2018（C17）§6.10.2 第 2、3 段。

工程惯例 · **工程惯例：为什么本项目统一用引号加 `-Iinclude`**

三种写法都合法：`#include "greet.h"`（头文件在 `include/`，靠 `-Iinclude`）、`#include "../include/greet.h"`（相对路径，能编过但丑陋）、`#include <greet.h>`（尖括号找私有头文件，能编过但语义混乱）。

本书统一用第一种：**引号表示「这是我们自己工程的头文件」，尖括号表示「这是系统提供的头文件」。**区分开的好处是：你在几千行代码里扫一眼就知道这一行是内部依赖还是外部依赖；而且换目录结构时，只需要改 `-I` 参数，不用改每一行 `#include`。

代价是每条编译命令都要带 `-Iinclude`。本机没有 make/cmake（见 `build/TOOLCHAIN.md` 的不可用表），所以这条参数只能写在命令里、或者写进 17.9 节那个 `build.sh` 里。

17.4 本机的链接错误只有一行

现在开始讲本单元**最反直觉**的一件事。下面这个例子的头文件里有两个声明，其中 `helper_double` 的实现在 `src/helper.c`；但编译命令里**只列了 `src/main.c` 和 `src/base.c`**——也就是「少列了一份源文件」：

c examples/ch07/undefined_helper/include/helper.h 第 1–10 行

```c
#ifndef HELPER_H
#define HELPER_H

/* 实现体在 src/base.c 里。 */
int helper_base(void); 实现在 base.c 里

/* 实现体在 src/helper.c 里 — 但那一份源文件没有被列进编译命令。 */
int helper_double(int n); 实现在 helper.c 里，而它没被列进命令

#endif /* HELPER_H */
```

5 实现在 base.c 里
8 实现在 helper.c 里，而它没被列进命令

c examples/ch07/undefined_helper/src/main.c 第 1–8 行

```c
#include <stdio.h>

#include "helper.h"

int main(void) {
    printf("helper_double(helper_base()) = %d\n", helper_double(helper_base())); 调用了两个外部名字
    return 0;
```

```
8   }
```

6 调用了两个外部名字

编它。下面是这台机器上**逐字**吐出来的 stderr——只有一个诊断块，里面**只有一行**：

编译诊断 本机真实 stderr：一行，没有文件名，没有行号 ch07-undefined-helper-diag

```
collect2.exe: error: ld returned 5 exit status
```

第二类链接错误长得**一模一样**。下面这个例子里，`a.c` 与 `b.c` 各自定义了一个同名的 `dup_value`：

c examples/ch07/double_definition/src/a.c 第 1–6 行

```c
#include "dup.h"

/* A 组认为这个函数该由自己提供。 */
int dup_value(void) { 第一个定义
    return 1;
}
```

4 第一个定义

c examples/ch07/double_definition/src/b.c 第 1–6 行

```c
#include "dup.h"

/* B 组也提供了一份，名字一模一样。 */
int dup_value(void) { 第二个定义，名字一样
    return 2;
}
```

4 第二个定义，名字一样

编它（两个源文件加上 `main.c` 一起给）：

编译诊断 本机真实 stderr：与「少定义」那一行逐字相同 ch07-double-definition-diag

```
collect2.exe: error: ld returned 5 exit status
```

两段会话放在一起看，你就明白本节开头那句话的意思了：

终端会话 两次真跑：两个完全不同的错，同一行诊断 ch07-link-errors

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/base.c -o
app.exe
collect2.exe: error: ld returned 5 exit status
$ echo $?
1
$ cd ../double_definition
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/a.c src/b.c
-o app.exe
collect2.exe: error: ld returned 5 exit status
```

「少了一个定义」和「多了一个定义」，本机给你的反馈是同一个字节序列，退出码都是 `1`。教科书上那句 `undefined reference to 'helper_double'` 你在本机**看不到**——它不是没发生，而是本机的默认环境没有把 `ld` 自己的诊断送到 stderr，只剩 `collect2` 的转述句。把 PATH 换成干净的 MSYS2 路径（第 1 单元 `undef_ref` 的例子就是这么钉住的），完整三行就会回来；这又是「同一份代码、换环境诊断就变样」的一个例子。

那一行是怎么来的

`collect2.exe` 是 gcc 在链接阶段临时叫起来的一个小工具——想知道它住在哪，直接问 gcc 就行：`gcc -print-prog-name=collect2`（本机实测它会打印出 `C:/msys64/ucrt64/bin/../lib/gcc/<目标三元组>/<gcc 版本>/collect2.exe`）。能确定的事实只有三层：

1. gcc 把 `-o app.exe` 交给 `collect2`，`collect2` 再调 `ld`；
2. 本机默认环境下，`ld` 自己那几行 `undefined reference …` / `multiple definition …` 没有出现在 stderr 上，只剩 `collect2` 的一句转述；

3. `ld` 以非 0 状态退出，`collect2` 把退出码写成 `collect2.exe: error: ld returned 5 exit status`。

至于第 2 步为什么、`ld` 的诊断在这一层去了哪里，本书没有查到可复述的机制，也不把它写成结论。你能确定的是：这一行是 `ld` 的**结论**，不是它的**理由**；想看理由，换干净 PATH（第 1 单元的做法）或直接调 `ld`（下面的反例）。

想看到完整过程，有人会加 `-v`。本机实测：`gcc -v -std=c17 … -o app.exe` 会多打几百行（预处理器的头文件搜索表、`as.exe` 的调用、`collect2.exe` 的完整命令行——那行命令里能看见 `crt2.o`、`crtbegin.o`、`-lmsvcrt` 这些平时被 gcc 自动带上的东西），但**最后一行还是那一句** `collect2.exe: error: ld returned 5 exit status`。`-v` 让你看见「gcc 替你做了什么」，看不见「ld 为什么不干」。

想看到课本上那句话，有一条笨办法：**跳过 gcc，自己直接调 `ld`**。它是真的，但代价立刻显形：

终端会话 反例：直接调 ld 会打印 undefined reference，同时也会多骂两句 ch07-direct-ld

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -Iinclude -c src/main.c -o m_nog.o
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -Iinclude -c src/base.c -o b_nog.o
$ ld -o uh_nog.exe m_nog.o b_nog.o
C:\msys64\ucrt64\bin\ld.exe: m_nog.o:main.c:(.text+0x9): undefined reference to `__main'
C:\msys64\ucrt64\bin\ld.exe: m_nog.o:main.c:(.text+0x15): undefined reference to `helper_double'
C:\msys64\ucrt64\bin\ld.exe: m_nog.o:main.c:(.text+0x26): undefined reference to `printf'
$ echo $?
1
$ rm -f m_nog.o b_nog.o uh_nog.exe
```

三行里只有**中间那行**是你真正想看的。另外两行是「想看到这句话」要付的代价：

- `undefined reference to '__main'`：`__main` 是 Windows 上 C 运行库的一个启动辅助函数，`gcc` 平时会自动带上 crt 目标文件把它填上；直接调 `ld` 没带，于是它成了缺的；
- `undefined reference to 'printf'`：`printf` 住在运行库（UCRT）里，`gcc` 平时会自动加 `-lc` 之类的链接参数把它接上；直接调 `ld` 也没有。

所以这个「笨办法」的正确用法是当反例看：它让你明白 gcc 平时替你做了多少事（补 crt、接运行库、把诊断压缩），也让你明白为什么本书不推荐你自己去调 `ld`。本机的正规出路不是绕开 gcc，而是**去看符号表**——下一节。

> ⚠ **陷阱 · `collect2` 那一行会不会骗你？会，而且骗法很隐蔽**
>
> **你会看到什么**：命令里明明有 `-Wall -Wextra -Wpedantic`，链接失败时这三个开关一个字的输出都没有，只有 `collect2` 那一行。
>
> **你会怎么误判**：以为「链接错误和警告无关，所以没输出是正常的」——对；但接着你会以为「一行诊断 = 一个很简单的问题」，于是从命令行的第一个文件名开始逐字检查。**真相是这一行信息量为零**：它既不说缺什么、也不说在哪，两类错误（少定义、多定义）共用它。
>
> **本机的另一个坑**：那个数字 `5` 是 `collect2` 转述的 `ld` 退出码，**不要记住它、更不要按它判断错误类型。**我在本机量过：上面两个完全不同的错误都给出 `5`。换一个链接器、换一个 gcc 版本，这个数字都可能变——它属于实现细节，不是诊断信息。
>
> **正确的第一步**：把「编译期」和「链接期」分开。有 `文件:行:列:` 的，是编译器在说话；只有 `collect2` / `ld` 字样的，是链接器。分清了这一步，再往下走 17.5 节。

17.5 用 nm 把两类错误分开

`collect2` 那一行不告诉你缺什么，但**目标文件自己记得。**每个 `.o` 里有一张**符号表**：它列出「这个 `.o` 提供了哪些名字」「这个 `.o` 还欠哪些名字」。读这张表的工具叫 `nm`（就是「列出名字」的那两个字母，名字的来历不用记）。本机的 `nm` 在 `/c/msys64/ucrt64/bin/nm`，`which nm` 能确认它已经在 `PATH` 上。

先看「少定义」那一半。把 `main.c` 和 `base.c` 各自编成 `.o`，然后问 `main.o`：你还欠谁？

终端会话 nm -u 列出「欠的名字」，再看另一个 .o 能不能还上 ch07-nm-undefined

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/main.c -o main.o &&
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/base.c -o base.o
$ nm -u main.o
                 U __main
                 U helper_base
                 U helper_double
                 U printf
$ nm base.o | grep helper
0000000000000000 T helper_base
```

这 4 行 `U` 要一个一个对：

- `U helper_double` —— **没有任何 `.o` 提供它。** `helper.c` 里明明写着函数体，但它没被列进编译命令，所以那个 `.o` 根本不存在。这就是「少列一个源文件」的符号表长相。
- `U helper_base` —— `base.o` 里有一行 `T helper_base`（上面最后一行就是），**有人还。**所以它不构成问题。
- `U printf` —— 它住在 C 运行库（本机是 UCRT）里，由 `gcc` 链接时自动接上。**这是运行库的，永远不用你管。**
- `U __main` —— 它也是运行库那边的：MinGW 的 CRT 在每个程序启动时要用到它，`gcc` 链接时由 crt 目标文件提供。**同样是运行库的。**我在本机核对过：最终链接出来的 `app.exe` 里，`nm` 看到的是 `T __main`——那一刻它已经被还上了，而且地址在 CRT 那一段。

看见了吗？**单看 `nm -u main.o`，「欠的名字」里既有我们自己缺的、也有运行库正常欠着的。**所以判据不是「有没有 `U`」，而是「**这个 `U` 有没有别的 `.o` 用 `T` 还上**」。少列一个源文件时，那个名字在整条命令行里**没有任何 `T` 对应**——这就是证据。

再看「重复定义」那一半。这次问两个 `.o`：你们各自**提供**了什么？

终端会话 两个 .o 各有一个 T dup_value：给重了 ch07-nm-dup

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/a.c -o a.o && gcc -
std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/b.c -o b.o
$ nm a.o | grep dup_value
0000000000000000 T dup_value
$ nm b.o | grep dup_value
0000000000000000 T dup_value
```

同一个名字，两个 `T`。这就是「重复定义」在符号表里的长相，干净利落：不需要看源码，也不需要理解 `collect2` 的退出码，两张表摆在一起就够了。

符号表里的字母怎么读

`nm` 输出里第二列的那个大写字母是全套的钥匙。本单元真正用到的只有六个，全部来自上面的实测输出：

字母	含义	本单元在哪里见到过
`U`	**undefined**：本 `.o` 引用了这个名字，但没定义	`U helper_double`、`U printf`
`T`	**text**，全局：本 `.o` 定义了它，且对外可见	`T dup_value`、`T helper_base`
`t`	text，**局部**：定义了，但只有本 `.o` 看得见（`static`）	`t bump`（17.7 节）
`B`	**bss**，全局：未初始化的全局数据	`B g_hits`（17.7 节）
`R`	**rodata**，全局：只读数据	`R PI`（坏版本）
`r`	rodata，**局部**：只读数据，但只有本 `.o` 看得见	`r PI`（修好之后）

一条规律贯穿全表：**大写 = 全局（global），小写 = 局部（local）。**而 `U` 没有大小写之分——它压根不是定义，只是一句「我欠着」。

（`nm` 还会打出一些以 `.` 开头的行，比如 `N .debug_info`、`p .pdata`、`r .xdata`。那些是**段符号**，不是函数或变量，看的时候直接跳过。）

同一个信息还有另一种看法：`objdump -t` 打的是同一张符号表，但它多给你一列**链接属性。**对 `static_scope/src/a.c` 编出来的 `a.o`：

终端会话 `objdump -t: scl 3 是 local, scl 2 是 global` ch07-objdump

```
$ cd ../static_scope
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/a.c -o a.o
$ objdump -t a.o | grep -E "bump|count_in_a"
[  2](sec  1)(fl 0x00)(ty   20)(scl   3) (nx 1) 0x0000000000000000 bump
[  4](sec  1)(fl 0x00)(ty   20)(scl   2) (nx 1) 0x000000000000000f count_in_a
```

`scl 3` 就是 `STB_LOCAL`（本地），`scl 2` 是 `STB_GLOBAL`（全局）——**和 `nm` 里的小写 `t` / 大写 `T` 是同一件事的两种写法。**看哪个都行：`nm` 短，适合每天用；`objdump` 细，适合你以后要查「这个符号在哪个段里」。本机这两个工具都在 `/c/msys64/ucrt64/bin/` 下。

实验记录 · **本机实测：nm 的读数**

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；`nm` 在 `/c/msys64/ucrt64/bin/nm`，实测可用；
- 「少定义」的真跑读数：`nm -u main.o` 打出 4 行 `U`——`__main`、`helper_base`、`helper_double`、`printf`；其中 `helper_base` 在 `base.o` 里是 `T`，`printf` 与 `__main` 属于运行库，只有 `helper_double` 谁都没有；
- 「重复定义」的真跑读数：`nm a.o | grep dup_value` 与 `nm b.o | grep dup_value` 各打出一行 `T dup_value`；
- 上面每一行都是 `session-nm.txt` 里的原文，由 `nm` 真跑抓下来，不是我抄的。

顺带一提：`nm` 默认按名字排序，加 `-u` 只列 `U`，加 `-g` 只列外部符号，加 `-l` 能把符号对到源码行号（前提是编译时带 `-g`）。本单元只用前两个。

陷阱 · **「有 `U` 就是错」——这条判据会让你把所有程序都当成坏的**

你会怎么被坑到：第一次用 `nm -u`，看到 `U printf`、`U __main`，心里一紧，以为程序有毛病，于是开始在 `main.c` 里找「哪个 `printf` 没定义」。

真相：每个用 `printf` 的 `.o` 都会有 `U printf`——因为 `printf` 的实现体从来不在你的工程里，它在运行库里，链接的时候才被接上。同理 `U __main` 每个 `.o` 都有。**`U` 只说明「当前这个 `.o` 没定义它」，不说明「整个程序缺它」。**

正确的判据（本单元最重要的一条操作）：把一个名字在**全部** `.o` 里搜一遍。有一个 `T` 就够；零个 `T` 才是缺，两个以上 `T` 才是重。

工程惯例 · **工程惯例：`nm` 该在什么时候用**

`nm` 不是排错时才想起来的工具，它有三个日常用途：

- **链接失败时**：对着命令行里的每一个 `.o` 跑一遍，找 `U` 与 `T` 的对不上（本节）；
- **确认 `static` 生效时**：你加了 `static` 却不放心，`nm | grep 名字` 看一眼是大写还是小写；
- **确认某个符号到底是「没有」还是「有但没被用上」时**：`.o` 里还留着符号，链接器却报错，十有八九是少列了那个 `.o`。

代价很低（一条命令），唯一的门槛是记住那几个字母。真记不住就去查 `man nm`（本机 MSYS2 里 `man` 不一定装；`nm --help` 一定有）。

17.6 include guard 与 `#pragma once`

回到头文件。17.3 节的 `greet.h` 开头有两行 `#ifndef GREET_H` / `#define GREET_H`，收尾有一行 `#endif`。那是**条件编译**（c16 的主角）最经典的一个用途——**让同一个头文件被包含多次时，只有第一次真正生效。**

先看不加它会发生什么。下面这个 `point.h` 故意没有 guard：

c examples/ch07/no_guard/include/point.h 第 1–8 行

```c
/* 注意：这个头文件故意没有 include guard —— 包含两次就会被念两遍。 */

struct Point {
    int x;
    int y;
};

int point_sum(struct Point p);
```

3 一个类型定义
8 一句函数声明

`main.c` 包含它两次：一次直接包含，一次通过 `geom.h` 间接包含：

c examples/ch07/no_guard/src/main.c 第 3–11 行

```c
#include "point.h"      /* 第一次念 point.h */
#include "geom.h"       /* 它里面的 #include "point.h" 会再念一遍 */

int main(void) {
    struct Point p = {3, 4};
    printf("point_sum = %d\n", point_sum(p));
    printf("geom_scale = %d\n", geom_scale(p, 10));
    return 0;
}
```

3 第一次
4 geom.h 里还会再包含一次

编译它，本机的真实诊断如下。注意这一块**有**文件名与行号——因为它死在**编译期**，不是链接期：

编译诊断 没有 guard 时的真实诊断：两个报错，都在 point.h 里 ch07-no-guard-diag

```
In file included from include/geom.h:4,
                 from src/main.c:4:
include/point.h:3:8: error: redefinition of 'struct Point'
    3 | struct Point {
      |        ^~~~~
In file included from src/main.c:3:
include/point.h:3:8: note: originally defined here
    3 | struct Point {
      |        ^~~~~
include/point.h:8:5: error: conflicting types for 'point_sum'; have 'int(struct Point)'
    8 | int point_sum(struct Point p);
      |     ^~~~~~~~~
include/point.h:8:5: note: previous declaration of 'point_sum' with type 'int(struct Point)'
    8 | int point_sum(struct Point p);
      |     ^~~~~~~~~
```

重复包含的三种症状

这块诊断里其实同时出现了**两种**症状；第三种要等两种修法讲完才看得到（本节末尾的 `header_function`），17.7 节还会再遇到它一次：

1. **类型重定义**：`error: redefinition of 'struct Point'`。`struct Point { … };` 是一句**类型定义**，同一个翻译单元里说两遍，编译器认为你自相矛盾——哪怕两遍一字不差。
2. **类型跟着声明一起烂掉**：`error: conflicting types for 'point_sum'`。第一次见到的 `struct Point` 和第二次见到的是「两个不同的类型」，于是 `int point_sum(struct Point)` 这句声明前后对不上。**这是第一种症状的连锁反应**，改掉第一个，第二个自动消失。
3. **头文件里放定义 → 链接期 multiple definition**：这一种在**编译期一个字都不报**，因为它只在「两个不同的翻译单元各包含一次」时暴露。

所以别把「忘了 guard」理解成一件小事：它要么在编译期把类型搅烂，要么在链接期把符号给重。

两种修法

修法一，也是标准做法——**include guard**：

c examples/ch07/guarded_include/include/point.h 第 1–16 行

```c
#ifndef POINT_H 先问「POINT_H 定义过没有」
#define POINT_H 没有才定义，然后往下念

/* include guard 的三个动作：先问「定义过没有」，没有就定义，念完再关上门。
   第二次包含时 POINT_H 已经定义，整个文件被跳过去。 */

struct Point {
    int x;
    int y;
};

typedef struct Point Point;

int point_sum(Point p);

#endif /* POINT_H */ 念完关门
```

1 先问「POINT_H 定义过没有」
2 没有才定义，然后往下念
16 念完关门

它的原理只有一句话：**第一次包含时 `POINT_H` 还没定义，于是 `#ifndef` 为真，整个文件被念一遍、`POINT_H` 被定义；第二次包含时 `#ifndef` 为假，整段被跳过。**从头到尾，被包含的文件内容没变，变的是「念不念」。

大白话 · 用大白话说 include guard

把 `point.h` 想成一份只该签一次的表格。

- `#ifndef POINT_H` 是门口那个人问一句：「这份表签过没有？」
- `#define POINT_H` 是「没签过，那就在本子上记一笔：签了」，然后放你进去把表念一遍；
- `#endif` 是门。

第二次有人拿着同一份表来，门口的记录上已经有名字了，`#ifndef` 判为假，**整份表被跳过**——不是「念了但没记住」，是**根本没念**。所以 guard 的名字（`POINT_H`）必须每个头文件都不一样；两个头文件用了同一个名字，第二个就会被误挡在门外，你会得到一堆「某某类型不认识」的报错，而且报错位置离真相很远。

回到严谨说法：guard 是**条件编译**的一个用法，靠一个宏的存在与否决定「这段文本编不编」，与 C 的类型系统、运行期都没有关系。

修法二，`#pragma once`：

c examples/ch07/guarded_include/include/geom.h 第 1–6 行

```c
#pragma once 一行顶三行
/* 与 include guard 干同一件事，少写三行。它是编译器的扩展，标准里没有（见 17.6 节）。 */

#include "point.h"      /* 自己也用 Point；这一次包含会被 point.h 的 guard 挡掉 */ 自己也要用 Point

int geom_scale(Point p, int k);
```

1 一行顶三行
4 自己也要用 Point

把两种写法混着用也没问题——本机的 gcc 两种都认。`main.c` 这次故意把 `point.h` 写了三遍：

终端会话 三种包含写法混在一起，照样编得过、跑得起来 ch07-guard-run

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/point.c
src/geom.c -o app.exe
$ ./app.exe
point_sum = 7
[geom] 缩放前 (3, 4)
geom_scale = 70
```

终端输出 终端输出 guarded-include-out

```
point_sum = 7
[geom] 缩放前 (3, 4)
geom_scale = 70
```

标准原文 · **标准原文：guard 用的是条件包含，不是新语法**

> C 语言标准 6.10.1 节「Conditional inclusion」规定 `#if` / `#ifdef` / `#ifndef` / `#elif` / `#else` / `#endif` 的语义：被跳过的组里只有预处理记号会被扫过，其余一概不处理。
>
> 大意：**guard 一点新东西都没有**——它就是 c16 讲过的条件编译。`#ifndef NAME` 问的是「`NAME` 现在有没有被 `#define`」，`#define NAME` 把它定义成一个空宏，`#endif` 收尾。**宏在预处理阶段就消失，所以 guard 不产生任何运行时代价。**
>
> 要强调的一点：**标准里没有 `#pragma once`**。它出现在标准的 6.10.6 节「Pragma directive」的框架下——那一节说，实现可以定义自己的 pragma，不认识的 pragma 必须被忽略。所以 `#pragma once` 是**编译器的扩展**，不是语言的一部分；本机的 gcc 认识它，换个编译器就得查文档。
>
> 出处：ISO/IEC 9899:2018（C17）§6.10.1、§6.10.6。

工程惯例 · **工程惯例：guard 的名字怎么起**

惯例是**用文件名**：`greet.h` → `GREET_H`，`point.h` → `POINT_H`。这样两个不同目录下同名的头文件会撞名——于是更稳的写法是加上项目或目录前缀，比如 `MYLIB_GREET_H`。

代价与取舍：guard 多三行，`#pragma once` 只一行；guard 是标准保证的、`#pragma once` 依赖编译器（本机 gcc 支持，但不是所有编译器都支持）；`#pragma once` 在「同一个文件通过不同路径被包含」（比如 `include/x.h` 与 `../include/x.h`）时，某些实现会认不出来，guard 则永远正确。

本书统一写 guard，只有在你已经确认目标编译器支持时才建议 `#pragma once`。至于「哪个更快」——两者都在预处理阶段完成，本机实测编译时间没有可观测的差别，别把这个当理由。

第三种症状：guard 挡不住「头文件里的定义」

guard 有一个很容易被误解的边界。下面这个 `util.h` 规规矩矩带了 guard，但函数的**定义**（带函数体）就写在头文件里：

c examples/ch07/header_function/include/util.h 第 1–11 行

```c
#ifndef UTIL_H
#define UTIL_H

/* 注意：这里放的是**定义**，不是声明 — 它带着函数体。
   include guard 拦不住它：guard 只保证「一个翻译单元里念一遍」，
   而这一份定义会跟着每个 #include 它的 .c 各生成一个 T util_twice。 */
int util_twice(int n) { 这是定义，带着函数体
    return n * 2; 函数体也一起进了每个包含它的 .c
}

#endif /* UTIL_H */
```

7 这是定义，带着函数体
8 函数体也一起进了每个包含它的 .c

两个 `.c` 都包含它、都调用它：

c examples/ch07/header_function/src/main.c 第 1–8 行

```c
#include <stdio.h>

#include "util.h" 包含头文件

int main(void) {
    printf("main: util_twice(21) = %d\n", util_twice(21)); 调用 util_twice
    return 0;
}
```

3 包含头文件
6 调用 util_twice

c examples/ch07/header_function/src/other.c 第 1–5 行

```c
#include "util.h" 另一个 .c 也包含同一份头文件

int other(void) {
    return util_twice(3); 也调用同一个函数
}
```

1 另一个 .c 也包含同一份头文件
4 也调用同一个函数

把两个源文件一起交给 gcc，本机的真实反馈还是那一行：

编译诊断 guard 齐全，照样死在链接期：本机 stderr 也只有这一行 ch07-header-function-diag

```
collect2.exe: error: ld returned 5 exit status
```

guard 为什么没救？ guard 的宏是**每个翻译单元各有一份**：`main.c` 的预处理把 `util.h` 念一遍，`other.c` 的预处理**也是第一次**念同一个头文件——两次都是「第一次包含」，`#ifndef UTIL_H` 两次都为真。于是两个 `.o` 里各出现一个外部链接的 `util_twice` 定义，链接器看到两份同名定义，判 multiple definition。

一句话：**guard 只保证「一个翻译单元里念一遍」，保证不了「整个程序里只有一份定义」。** 想只要一份，就把头文件里的那一行改成声明 `int util_twice(int n);`，把函数体搬进一个 `.c`——也就是 17.3 节的「一个模块 = 一对文件」。如果你确实想让每个 `.c` 自带一份小函数，那就用 17.7 节的 `static inline`：`static` 把名字关在本编译单元里，两份同名定义就不再冲突。

17.7 链接性：`static`、`extern` 与头文件里的 `const`

从 17.2 那张图开始，每个名字其实都带着一个属性：**它对这个程序里的其他翻译单元可不可见。**这个属性叫**链接**（linkage）。标准只定义了三种：**外部链接**（external）、**内部链接**（internal）、**无链接**（none，函数里的局部变量）。

- **`static`：把名字关在本编译单元里**

两个 `.c` 各写一个同名的 `bump`，都加 `static`：

c examples/ch07/static_scope/src/a.c 第 1–12 行

```c
#include "counter.h"

/* static：文件作用域的名字，出了这个 .c 就没人看得见。 */
static int bump(int n) { static：名字只在这个 .o 里
    return n + 1;
}

int count_in_a(void) {
    static int n = 0;               /* 静态存储期：跨调用保留 */ 静态存储期：跨调用保留
    n = bump(n);
    return n;
```

```
12  }
```

4 static：名字只在这个 .o 里
9 静态存储期：跨调用保留

c examples/ch07/static_scope/src/b.c 第 1–12 行

```c
#include "counter.h"

/* 同名，也是 static：两份 bump 在链接期互不打扰。 */
static int bump(int n) {
    return n + 10;
}

int count_in_b(void) {
    static int n = 0;
    n = bump(n);
    return n;
}
```

4 同名，也是 static
5 加的是 10，不是 1

如果去掉 `static`，这两个 `bump` 就是 17.4 节那个「重复定义」——链接期直接失败。加上 `static` 之后，它们连符号表里的**大小写**都变了：

终端会话 static 是 t，全局函数是 T ch07-nm-static

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/a.c -o a.o
$ nm a.o | grep -E "bump|count_in_a"
0000000000000000 t bump
000000000000000f T count_in_a
$ cd ../extern_shared
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/shared.c -o
shared.o
$ nm shared.o | grep g_hits
0000000000000000 B g_hits
```

`t bump` 是小写，`T count_in_a` 是大写——**小写就是「链接器不认这个名字」**。它还在 `.o` 里（调试和反汇编要用），但它**不进链接器的候选名单**。两份 `bump` 因此永远撞不上。

跑起来：

终端输出 终端输出 static-scope-out

```
a 组的计数:  1 2 3
b 组的计数:  10 20 30
```

两个计数器各自从 0 数到 3，增量一个 1、一个 10——**同名不等于同物。**

- **`extern`：把名字连到另一处定义**

`static` 是「关上门」，`extern` 是「留个门牌」。头文件里写：

c examples/ch07/extern_shared/include/shared.h 第 1–13 行

```c
#ifndef SHARED_H
#define SHARED_H

/* 声明：告诉每个编译单元「有这么个变量」，但不给它分配内存。
   内存由 src/shared.c 里那唯一一处定义提供。 */
extern int g_hits;

#define SHARED_LIMIT 3

void shared_reset(void);
int shared_next(void);
```

```
13  #endif /* SHARED_H */
```

6 extern：声明一个变量，但不分配内存
8 宏也能放头文件

定义只出现一次，在 `shared.c` 里，而且**不能再写 `extern`**：

c examples/ch07/extern_shared/src/shared.c 第 3–15 行

```c
3   /* 定义：整个程序里只有这一处给 g_hits 分配了内存。
4      这里不写 extern 只是惯例（写了 `extern int g_hits = 0;` 照样是定义，
5      编译器只会警告 `'g_hits' initialized and declared 'extern'`）。 */
6   int g_hits = 0; 没有 extern：这是定义，分配内存
7
8   void shared_reset(void) {
9       g_hits = 0;
10  }
11
12  int shared_next(void) { 改的是同一个 g_hits
13      g_hits++;
14      return g_hits;
15  }
```

6 没有 extern：这是定义，分配内存
12 改的是同一个 g_hits

`nm shared.o` 打出来的是 `B g_hits`——`B` 是「未初始化的全局数据段」。`g_hits = 0;` 里那个 `0` 让它落进了 `.bss`（本机实测；如果把初值改成 `42`，同一个符号会显示成 `D`，也就是「已初始化的数据段」）。大小写仍然是关键：**`B` 大写 = 全局，别的 `.o` 能连上它。**

终端输出 终端输出 extern-shared-out

```
起点: g_hits = 0
第 1 次之后: g_hits = 1 (函数返回 1)
第 2 次之后: g_hits = 2 (函数返回 2)
第 3 次之后: g_hits = 3 (函数返回 3)
```

`main.c` 里那个 `g_hits` 和 `shared.c` 里那个 `g_hits` 是**同一个对象**——因为头文件里的 `extern` 告诉编译器「别在这儿分配，去别处找」，而唯一一处分配在 `shared.c`。

标准原文 · **标准原文：链接的三条规矩**

C 语言标准 6.2.2 节「Linkages of identifiers」，三段话拼起来就是本节的规则：

（第 3 段）If the declaration of a file scope identifier for an object or a function contains the storage-class specifier `static`, the identifier has internal linkage.

（第 4 段）For an identifier declared with the storage-class specifier `extern` in a scope in which a prior declaration of that identifier is visible, if the prior declaration specifies internal or external linkage, the linkage of the identifier at the later declaration is the same as the linkage specified at the prior declaration. If no prior declaration is visible, or if the prior declaration specifies no linkage, then the identifier has external linkage.

（第 5 段）If the declaration of an identifier for a function has no storage-class specifier, its linkage is determined exactly as if it were declared with the storage-class specifier `extern`. If the declaration of an identifier for an object has file scope and no storage-class specifier, its linkage is external.

大意：`static` **在文件作用域 = 内部链接**；`extern` **= 外部链接**（除非前面已经有一个内部链接的声明）；**函数和文件作用域的变量，什么都不写也默认外部链接。**

最后那句是本单元最容易记反的一条：`int greet_banner(const char *who)` 前面**什么都没写**，它照样是外部链接、照样能跨翻译单元调用。`extern` 不是「允许外部用」，它是「明确声明这是外部的」——对一个本来就有外部链接的函数，写不写 `extern` 都一样。

出处：ISO/IEC 9899:2018（C17）§6.2.2 第 3、4、5 段。

头文件里的 `const`：C 与 C++ 分道扬镳的地方

现在讲本单元**最有欺骗性**的一个坑。`const double PI` 写在头文件里，看起来像一句「常量声明」，C 里它就是一句**定义**——而且默认带**外部链接**：

examples/ch07/const_in_header/include/pi.h 第 1–4 行

```c
/* 坏版本：这一行是一个「定义」，不是一个声明。
   头文件会被多个编译单元包含，于是每个 .o 里都有一个全局的 PI。 */

const double PI = 3.141592653589793;  这是一个定义：有初值、有存储
```

4 这是一个定义：有初值、有存储

两个 `.o` 各包含它一次，于是各有一个全局的 `PI`：

examples/ch07/const_in_header/src/bad_ring.c 第 1–6 行

```c
#include "pi.h"

/* 另一个编译单元也把 pi.h 包含进来，于是它也定义了一个 PI。 */  第二个编译单元也包含了 pi.h
double bad_ring(double r) {  于是它也定义了一个 PI
    return 2.0 * PI * r;
}
```

3 第二个编译单元也包含了 pi.h
4 于是它也定义了一个 PI

链接结果？**还是那一行** `collect2`：

编译诊断 常量放错地方的代价：又是一行 collect2 ch07-const-in-header-diag

```
collect2.exe: error: ld returned 5 exit status
```

`nm` 能提前把这件事看出来。坏版本的两个 `.o` 各有一个**大写** `R PI`：

终端会话 大写 R 是全局只读数据，小写 r 是局部只读数据 ch07-nm-const

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/bad_main.c -o
bad_main.o && gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c
src/bad_ring.c -o bad_ring.o
$ nm bad_main.o | grep PI
0000000000000000 R PI
$ nm bad_ring.o | grep PI
0000000000000000 R PI
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/good_main.c -o
good_main.o && gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c
src/good_ring.c -o good_ring.o
$ nm good_main.o | grep PI
0000000000000000 r PI
$ nm good_ring.o | grep PI
0000000000000000 r PI
```

修法就是一个词：

c examples/ch07/const_in_header/include/pi_fixed.h 第 1–4 行

```c
/* 修法：加上 static，每个编译单元各自拥有一份自己的 PI。
   它只在这一个 .o 里可见，链接期不再与别人撞名。 */

static const double PI = 3.141592653589793; 加上 static：每个编译单元各有一份自己的
```

4 加上 static：每个编译单元各有一份自己的

`R` 变成 `r`，链接器不再看它，两个 `.o` 各活各的。跑起来：

终端输出 终端输出 const-in-header-out

```
good_area(1.0)  = 3.141593
good_ring(1.0)  = 6.283185
```

⚠ **陷阱** · **「`const` 就是常量，放头文件天经地义」——这个直觉来自 C++**

你会怎么被坑到：你从别的语言（尤其是 C++）带来一个习惯——把常量写进头文件，`const double PI = 3.14;`，然后理直气壮地 `#include` 到所有地方。C++ 里这么写没问题，C 里链接期直接失败，而且**只有一行 `collect2`**，你会完全找不到方向。

真相：**C 里文件作用域的 `const` 对象，默认是外部链接。**标准 6.2.2 节第 5 段那句「没有存储类说明符的对象声明，链接是外部的」不区分 `const`——C 的 `const` 只表示「不许通过这个声明改它」，它**不改变链接**。而 C++ 的标准另有规定：命名空间作用域的 `const` 默认**内部链接**。同一个词、两种默认值，这就是全部的原因。

这条差别的代价：在 C 里，头文件里的 `const` 要么加 `static`（每个编译单元一份，本机实测可行），要么只在头文件里写 `extern const double PI;`、在一个 `.c` 里写定义。**绝不能什么都不加就往头文件里放一个有初值的 `const`。**

顺带记住一个更普遍的判断法：头文件里的每一行，你都要问一句「它被 5 个 `.o` 各念一遍，会有 5 份吗？」有 5 份会冲突 → 加 `static` 或改成声明。

`inline` 的取舍：本书只用 `static inline`

有人会想：既然头文件里放函数定义会撞名，那我加 `inline` 是不是就好了？C99 的 `inline` 确实有一套绕口的规则（`inline` 定义与外部定义两套模型，还要一个 `.c` 来「兜底」），但它解决的是另一件事（让编译器在多个翻译单元里各生成一份内联体，同时保证只有一份外部定义），**不是**给初学者用的。

本书的统一做法只有一句：

{} 语法规则 · 头文件里的函数：写 `static inline`

形式就是三个词连写：`static inline int clamp(int x, int lo, int hi) { … }`。

- `static` 把它关在本编译单元里，所以不会撞名（17.7 节）；
- `inline` 提示编译器「能在调用点展开就展开」——注意是**提示**，不是命令，编译器有权不听；
- 返回类型与参数写全，和普通函数一模一样。

代价是：**每个包含它的 `.c` 各有一份自己的副本**——代码体积会变大，而且每个 `.c` 里那份的地址不同（所以不能拿它的地址做跨文件比较）。对小工具函数，这个代价可以忽略；对大函数，就别往头文件里放。

`static inline` 是 C99 起就有的写法，也是 C 标准库里大量函数的实现方式。C99 `inline` 的两个模型本书不展开——你只要记住：**在头文件里写函数，加 `static`；想让它的调用没有额外开销，再多加一个 `inline`。**

把 `static` 与 `extern` 画在「谁看得见谁」上

第 1 帧是 17.4 节的重复定义，第 2 帧是 `static_scope`，第 3 帧是 `extern_shared`。图的读法只有一句：**一个名字能不能出现在链接器的候选名单里，取决于它有没有 `static`**——`static` 的名字缩在 `.c` 的方框里，`extern`（以及默认的外部链接）的名字才伸得出去。

17.8 分层与前向声明：把循环依赖破掉

17.7 节把「一个名字能不能出去」讲完了；这一节讲「**应该让谁依赖谁**」。这是设计问题，标准一个字都没规定，但它是多文件工程里唯一真正需要动脑子的部分。

先看一个摆对了的三层工程。它只有三个模块，依赖是单向的：

c examples/ch07/layered_app/include/logic.h 第 1–8 行

```c
#ifndef LOGIC_H
#define LOGIC_H

/* 业务层：只算数，一个字节都不打印 —— 打印是 io 层的事。 */ 只算数，不打印
int fib(int n); 声明
int fib_sum(int n);

#endif /* LOGIC_H */
```

4 只算数，不打印
5 声明

c examples/ch07/layered_app/src/main.c 第 1–14 行

```c
#include "io.h"
#include "logic.h"

/* main 连 <stdio.h> 都没包含：打印这件事被 io 层接走了。 */
int main(void) {
    io_write_note("layered_app: main 只用两个头文件");

    for (int i = 0; i < 7; i++) {
        io_write_row(i, fib(i));
    }

    io_write_total(7, fib_sum(7));
    return 0;
}
```

1 用 io 层的打印
2 用 logic 层的算法
4 没有 stdio.h，也不需要
9 打印一行结果

注意 `main.c` **没有** `#include <stdio.h>`——它一次 `printf` 都不写，打印全部交给 `io.c`。这不是洁癖，它让依赖方向变成一条直线：

- `main` → `io`、`main` → `logic`（`main` 在顶层，依赖下面两层）；
- `io` 谁都不依赖（它只依赖 `<stdio.h>`，那是系统的）；
- `logic` 谁都不依赖（它连打印都不做）。

箭头全部指向下层，没有一条往回指。 这样的结构里，你想单独测 `logic`，直接编 `logic.c` 加一个测试用的 `main` 就行，不用拖上任何别的东西。

这个工程一共六个文件，一眼就能数出层次：三个 `.c`（`io.c`、`logic.c`、`main.c`）加两个 `.h`（`io.h`、`logic.h`），再加一份 `.json` 清单。`io.c` 只包含 `io.h`，`logic.c` 只包含 `logic.h`，**它们互相不知道对方存在**——这一点在 17.9 节会变成一句话：改 `logic.h` 不需要重编 `io.o`。

终端输出 终端输出 layered-app-out

```
# layered_app: main 只用两个头文件
fib(0) = 0
fib(1) = 1
fib(2) = 1
fib(3) = 2
fib(4) = 3
fib(5) = 5
fib(6) = 8
前 7 项之和 = 20
```

循环依赖：两个模块互相需要

真实工程里最常见的一种坏结构是**循环依赖**：`list` 需要知道 `node` 是什么，`node` 又需要知道 `list` 是什么。先看一个真的编不过的版本：`a.h` 包含 `b.h`，`b.h` 又包含 `a.h`，而 `b.h` 里把 `struct A` **按值**嵌了进来：

c examples/ch07/cycle_before/include/a.h 第 4–10 行

```c
#include "b.h"

/* A 里放一个**指向** B 的指针 —— 本来不需要 B 的完整定义。 */
struct A {
    int value;
    struct B *partner;
};
```

4 两头互相包含
9 指针，本来不需要 B 的完整定义

c examples/ch07/cycle_before/include/b.h 第 4–10 行

```c
#include "a.h" 又反过来包含 a.h

/* B 里却把 A **整个**放了进来 —— 这一行需要 A 的完整定义。 */
struct B {
    int value;
    struct A partner; 按值放整个 A，这里需要 A 的完整定义
};
```

4 又反过来包含 a.h
9 按值放整个 A，这里需要 A 的完整定义

`a.h` 展开到 `b.h` 时，`struct A` 还只有一个名字、没有成员表；include guard 又让 `a.h` 不会第二次展开。于是 `struct A` 在 `b.h` 里是**不完全类型**，按值放它编不过：

编译诊断 循环包含 + 按值嵌入：真实诊断落在 b.h 的成员上 ch07-cycle-before-diag

```
In file included from include/a.h:4,
                 from src/main.c:3:
include/b.h:9:14: error: field 'partner' has incomplete type
    9 |     struct A partner;
      |              ^~~~~~~
In file included from include/a.h:4,
                 from src/a.c:1:
include/b.h:9:14: error: field 'partner' has incomplete type
    9 |     struct A partner;
      |              ^~~~~~~
```

破法只有一条：**至少让一头只拿指针。**`b.h` 不再包含 `a.h`，改成一句前向声明：

c examples/ch07/cycle_after/include/b.h 第 4–11 行

```c
/* 破坏：这里**不** #include "a.h"，只写一句前向声明。
   B 里只放指向 A 的指针，所以不需要 A 的完整定义。 */
struct A; 只声明名字，不给成员表

struct B {
    int value;
    struct A *partner; 只放指针，不需要 A 的大小
};
```

6 只声明名字，不给成员表
10 只放指针，不需要 A 的大小

`a.h` 仍然包含 `b.h`——这条**单向**包含不再形成环。两边都不再按值需要对方，完整的定义各自留在 `.c` 里，程序就能编过、跑起来：

c **两头都只拿指针：链接、运行** examples/ch07/cycle_after/src/main.c 第 5–17 行

```c
int main(void) {
    struct A a;
    struct B b;

    a_init(&a, 10);
    b_init(&b, 20);
    a_link(&a, &b);
    b_link(&b, &a);

    printf("a.value = %d, a 的伙伴 = %d\n", a_value(&a), a_partner_value(&a));
    printf("b.value = %d, b 的伙伴 = %d\n", b_value(&b), b_partner_value(&b));
    return 0;
}
```

终端输出 终端输出 cycle-after-out

```
a.value = 10, a 的伙伴 = 20
b.value = 20, b 的伙伴 = 10
```

`forward_decl` 是同一个破法的另一种长相：

c examples/ch07/forward_decl/include/node.h 第 1–14 行

```c
#ifndef NODE_H
#define NODE_H

/* 前向声明（不完全类型）：只说「有 struct Node 这么个东西」，
   不说它有几个成员、占几个字节 —— 所以这里还写不出 sizeof(struct Node)。 */
struct Node;                                  /* 前向声明：只有名字，没有成员 */

struct Node *node_create(int value);          /* 返回指针，编译器不需要知道它有多大 */
int node_value(const struct Node *n);         /* 参数也是指针 */
struct Node *node_next(const struct Node *n);
void node_set_next(struct Node *n, struct Node *next);
void node_free_all(struct Node *head);

#endif /* NODE_H */
```

6 前向声明：只有名字，没有成员
8 返回指针，编译器不需要知道它有多大
9 参数也是指针

第 6 行的 `struct Node;` 就是前向声明。它引入一个**不完全类型**（incomplete type）：编译器知道「有这么个类型」，但**不知道它多大、有几个成员。**而第 8 到 12 行全都只用 `struct Node *`——**指针的大小是固定的（本机 8 字节），与它指的东西多大无关**，所以这些声明在「不知道 `Node` 长什么样」的情况下照样成立。

完整定义只在一个地方：

c examples/ch07/forward_decl/src/node.c 第 1–9 行

```c
#include <stdlib.h>

#include "node.h"

/* 定义只出现在这一个编译单元里：别的 .c 看不到这三行。 */
struct Node {                                 /* 这里是真定义，只有这一个 .c 看得到 */
    int value;
    struct Node *next;
};
```

6 这里是真定义，只有这一个 .c 看得到

于是 `list.c` 可以只搬指针、完全不认识 `Node` 的内部；而 `main.c` 更极端——它只包含 `list.h`：

c examples/ch07/forward_decl/src/main.c 第 1–18 行

```c
#include <stdio.h>

#include "list.h"                             /* 只包含 list.h */

/* main.c 只包含 list.h，连 node.h 都没提 —— 它拿到的 struct Node
   永远是一个「不完全类型」，只用它的指针就够了。 */
int main(void) {
    struct Node *head = NULL;                 /* 手里只有一个不完全类型的指针 */
    for (int i = 1; i <= 4; i++) {
        head = list_push_front(head, i * i);
    }

    printf("链表长度 = %d\n", list_length(head));
    printf("节点之和 = %d\n", list_sum(head));

    node_free_all(head);
    return 0;
}
```

3 只包含 list.h
8 手里只有一个不完全类型的指针

终端输出 终端输出 forward-decl-out

```
链表长度 = 4
节点之和 = 30
```

不完全类型到底「不完全」到什么程度？ 用一个真实的失败来钉：

c examples/ch07/forward_decl/src/sizeof_probe.c 第 1-6 行

```c
#include "list.h"

/* 这一份故意越界：struct Node 在这里是不完全类型，没有大小可问。 */
int probe_size(void) {
    return (int)sizeof(struct Node); 不完全类型没有大小可问
}
```

5 不完全类型没有大小可问

编译诊断 对不完全类型用 sizeof：编译器一句话就把你挡住了 ch07-forward-decl-sizeof-diag

```
src/sizeof_probe.c: In function 'probe_size':
src/sizeof_probe.c:5:24: error: invalid application of 'sizeof' to incomplete type 'struct Node'
    5 |     return (int)sizeof(struct Node);
      |                        ^~~~~~
```

这条诊断是本节最干净的一条：**前向声明给你的是「名字」，不是「布局」**。名字够用来声明指针、声明函数参数，不够用来算大小、不够用来访问成员（`n->value` 在 `list.c` 里同样编不过）、也不够用来定义它的数组。

标准原文 · **标准原文：`sizeof` 与不完全类型**

> C 语言标准 6.5.3.4 节「The `sizeof` and `_Alignof` operators」第 1 段：
>
> > The `sizeof` operator shall not be applied to an expression that has function type or an incomplete type...
>
> 大意：`sizeof` **不许用在不完全类型上**——因为「大小」正是它缺的那条信息。
>
> 前向声明本身写在 6.7.2.3 节「Tags」里：一个只出现 tag、没有成员表的 `struct Node;` 声明，引入的就是一个该 tag 的不完全类型；等到同一作用域里出现带成员表的定义，这个类型才补全。**补全只能发生一次**——这也解释了为什么 `struct Node { … };` 写两遍会报重定义（17.6 节）。
>
> 出处：ISO/IEC 9899:2018（C17）§6.5.3.4、§6.7.2.3。

工程惯例 · **工程惯例：模块划分的三条硬规矩**

这三条都不是标准，是三十年攒下来的经验，但每一条都有可检查的后果：

1. **一个模块 = 一对文件**，名字一样（`node.c` / `node.h`）。后果：你在报错里看到文件名就知道该开哪个头文件。
2. **依赖方向单一**：`a.c` 包含 `b.h` 之后，`b.c` 里就不许出现 `#include "a.h"`。后果：一旦出现，你会在链接期得到一个「两边都欠对方」的结构——虽然 C 允许它编过（只要都是声明），但任何一处改动都会牵动两个模块，编译顺序、测试顺序全乱。
3. **循环依赖用前向声明破**：让「被依赖更多」的那一头只暴露指针。后果：`node.h` 不再需要 `list.h`，箭头上少了一条边。

代价要说清：前向声明换来的是**你不能再直接访问成员**，只能通过 `node_value()`、`node_next()` 这类函数。多写几个小函数是真的麻烦，但换来的是「`Node` 的内部布局只在一个文件里变」——这是本节所有例子里最值钱的一条。

17.9 一条命令 vs 分步编译：头文件是合同

回到构建。前面所有例子用的都是「一条命令编完」：源文件全列在命令行里，`gcc` 从 `.c` 一路做到 `.exe`。它简单、不容易漏掉某个文件，但有一个代价：**改一个文件，所有文件都重编一遍。**

先用一个小例子把两种做法摆在一起。`split_build` 只有两个 `.c`：

c examples/ch07/split_build/src/stats.c 第 1–19 行

```c
#include "stats.h"

int stats_sum(const int *a, int n) { 第一个函数
    int total = 0;
    for (int i = 0; i < n; i++) {
        total += a[i];
    }
    return total;
}

int stats_max(const int *a, int n) { 第二个
    int best = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] > best) {
            best = a[i];
        }
    }
    return best;
}
```

3 第一个函数
11 第二个

第三个函数 `stats_span` 借用了第二个——它在**同一个编译单元**里，所以直接调就行，不需要任何声明之外的仪式：

c **(续) stats_span: 同文件里直接调用 stats_max** examples/ch07/split_build/src/stats.c 第 21–23 行

```c
int stats_span(const int *a, int n) {
    return stats_max(a, n) - a[0];
}
```

一条命令：

终端输出 终端输出 split-build-out

```
sum  = 29
max  = 9
span = 2
```

分步编译把这一条命令拆成「N 次 `-c`」加「一次链接」。`-c` 的意思是「只编译、不链接」——它产出一个 `.o`，里面有机器码和符号表，但还缺别人：

终端会话 分步编译：两个 -c，一次链接，再运行 ch07-split-build

```
$ gcc -MM -Iinclude src/main.c
main.o: src/main.c include/stats.h
$ gcc -MM -Iinclude src/stats.c
stats.o: src/stats.c include/stats.h
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/stats.c -o stats.o
&& gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/main.c -o main.o
$ gcc main.o stats.o -o app.exe && ./app.exe
sum  = 29
max  = 9
span = 2
```

前四行是本节的关键工具：**`gcc -MM` 会打印「这个 `.o` 依赖哪些文件」**。`-M` 会把系统头文件全列出来（几百行），`-MM` 只列用户自己的头文件——所以输出干净得刚好能读：两个 `.o` 都依赖 `include/stats.h`。

编译器视角 · **编译器视角：`-MM` 是谁算出来的**

`-MM` 不是链接器也不是编译器本体给的，是**预处理器**给的。原因很简单：只有跑过 `#include` 展开，才知道「这一份 `.c` 到底读进了哪些文件」。所以 `gcc -MM` 实际上做了「预处理一遍、只输出依赖关系、不产出任何代码」——它和 `-E` 是同一阶段的两种输出。

推论有两条：① 依赖表里的路径就是预处理器**真正找到**的那些文件（所以你写的 `-Iinclude` 会体现在里面）；② 想让这张表跟着 `#include` 自动变，只有「每次编之前跑一遍 `-MM`」这一条路——这正是 make 干的事，也是 `build.sh` 里那张手写表要替代的东西。

改一个头文件，哪些 `.o` 过期

现在做那个「改头文件」的实验。改内容和 `touch` 它，对构建系统来说是**同一件事**——它只看时间戳：

终端会话 动一下头文件，两个 .o 立刻变成过期的 ch07-touch-header

```
$ touch include/stats.h
$ ls -l --time-style=full-iso include/stats.h main.o stats.o
-rw-r--r-- 1 Administrator 197121  258 2026-10-21 14:09:34.036712100 +0800 include/stats.h
-rw-r--r-- 1 Administrator 197121 3134 2026-10-21 14:09:33.476070800 +0800 main.o
-rw-r--r-- 1 Administrator 197121 3200 2026-10-21 14:09:33.133411900 +0800 stats.o
```

`include/stats.h` 的时间是 `:34.036…`，`main.o` 是 `:33.476…`，`stats.o` 是 `:33.133…`。**两份 `.o` 都比头文件旧**——也就是说，如果头文件里改的是函数声明，这两份 `.o` 里存着的还是旧版本的信息。

为什么是两份而不是一份？看上面 `-MM` 的输出就明白了：`main.o` 与 `stats.o` **都** `#include "stats.h"`。这就是本节那句「**头文件是合同**」的全部含义：

> 你改的不是一个文件，你改的是**所有包含它的编译单元**共同遵守的那份约定。

所以改完必须重编两份、再链接：

终端会话 重编两份 .o，再链接一次 ch07-rebuild

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/stats.c -o stats.o
&& gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/main.c -o main.o
$ gcc main.o stats.o -o app.exe && ./app.exe
sum  = 29
max  = 9
span = 2
```

把这一串动作画成图：

图 / 搜索过程 **构建依赖图：哪一步重编、哪一步重链接**

第 1 步 **三份源文件各自包含一个头文件**

src/io.c：只包含 io.h　src/logic.c：只包含 logic.h　src/main.c：包含 io.h 与 logic.h　include/io.h　include/logic.h

src/io.c → include/io.h #include
src/logic.c → include/logic.h #include
src/main.c → include/io.h #include
src/main.c → include/logic.h #include

第 2 步 **一次 -c 产出一个 .o，这一步看得到头文件**

io.o ← 本步新增　logic.o ← 本步新增　main.o ← 本步新增

src/io.c → io.o gcc -c
src/logic.c → logic.o gcc -c
src/main.c → main.o gcc -c

第 1、2 帧说的是「**编译期看得见头文件，链接期看不见**」：`-c` 那一步把头文件的内容吃进 `.o`，之后 `.o` 里只剩符号名；所以头文件改了，**必须回到 `-c` 那一步重来**，光重新链接是没用的。第 3 帧说的是链接器只认 `.o`——它对齐的是名字，不是源码。第 4 帧是 `-MM` 那张表的用处：**它告诉你哪几个 `.o` 过期。**

这里要说清一件我实测过的事：`layered_app` 里 `io.c` 只包含 `io.h`，所以改 `logic.h` 时它**不该**重编。`gcc -MM` 的原文证明了这一点：

终端会话 三份源文件各自的真实依赖 ch07-nm-layered-deps

```
$ cd ../layered_app
$ gcc -MM -Iinclude src/io.c
io.o: src/io.c include/io.h
$ gcc -MM -Iinclude src/main.c
main.o: src/main.c include/io.h include/logic.h
$ gcc -MM -Iinclude src/logic.c
logic.o: src/logic.c include/logic.h
```

`io.o` 的依赖里没有 `logic.h`。所以「改一个头文件会让所有 `.o` 过期」这句话**只在「所有 `.c` 都包含那个头文件」时成立**——`split_build` 就是这样，`layered_app` 就不是。判断依据永远是 `-MM` 的输出，不是感觉。

`build.sh`：没有 make 的时候自己判断

本机没有 make/cmake（见 `build/TOOLCHAIN.md` 的不可用表），所以「哪一步该重编」只能自己写。本节例子里那个 `build.sh` 做了最保守的判断：**只要任何一份头文件比 `.o` 新，就重编。**

bash examples/ch07/build.sh 第 1–13 行

```
#!/usr/bin/env bash
# examples/ch07/build.sh —— 分步编译：先各自编成 .o，再链接。
# 本机没有 make/cmake，所以「哪一步该重编」只能由脚本自己判断；
# 判断依据只有一条：源文件或它依赖的头文件，有没有比 .o 新。
set -e

cd "$(dirname "$0")/layered_app"   脚本可以被任何人从任何目录跑，所以先定位

CC=gcc
CFLAGS="-std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude"
OBJS="main.o io.o logic.o"
PAIRS="main.o:src/main.c io.o:src/io.c logic.o:src/logic.c"   手写的「.o 对 .c」表
HEADERS="include/io.h include/logic.h"   手写的头文件表
```

7 脚本可以被任何人从任何目录跑，所以先定位
12 手写的「.o 对 .c」表
13 手写的头文件表

第 12 行那张 `PAIRS` 表把「哪个 `.o` 由哪个 `.c` 来」写死了，第 13 行则把所有头文件当成**每一份 `.o` 的共同依赖**——这是保守做法。判断本身只有 9 行：

bash examples/ch07/build.sh 第 15–23 行

```bash
needs_build() {
    local obj=$1 src=$2 h
    if [ ! -f "$obj" ]; then return 0; fi .o 根本不存在，当然要编
    if [ "$src" -nt "$obj" ]; then return 0; fi 源文件比 .o 新
    for h in $HEADERS; do
        if [ "$h" -nt "$obj" ]; then return 0; fi 任何头文件比 .o 新
    done
    return 1 三条都没命中，跳过
}
```

17 .o 根本不存在，当然要编
18 源文件比 .o 新
20 任何头文件比 .o 新
22 三条都没命中，跳过

它跑两次，差别一眼就看出来：

终端会话 第一次：三份都还没编过，全部 compile ch07-buildsh-1

```
$ bash build.sh
compile  src/main.c -> main.o
compile  src/io.c -> io.o
compile  src/logic.c -> logic.o
link     main.o io.o logic.o -> app.exe
```

终端会话 第二次：什么头文件都没动，三条全变成 skip ch07-buildsh-2

```
$ bash build.sh
skip     src/main.c (main.o 比它新)
skip     src/io.c (io.o 比它新)
skip     src/logic.c (logic.o 比它新)
link     main.o io.o logic.o -> app.exe
```

第二次只做了链接。这看起来省不了多少——三个小文件而已；但把 `logic.c` 想成三千行，你就明白为什么「只重编该重编的」是工程上的硬需求。代价是：**这张依赖表是手写的**（第 12、13 行），加一个模块就得改脚本，而且第 13 行的 `HEADERS` 是保守的（任何头文件变了就全重编）。真正的 make 会用 `gcc -MM` 自动生成这张表——本机没有 make，所以我们停在「手写一张表」这一步。

分步编译还有一个顺带的好处，这一点新手常常忽略：**`-c` 让错误更早暴露，也更容易定位。**一条命令编三个文件时，三个文件的诊断会混在一起，你分不清哪个是「前一个文件的错误吓出来的」；先 `-c` 一遍，每个 `.o` 的诊断只属于它自己。本书所有例子的命令之所以总把源文件全列出来，是因为闸门要一次编完、一次对账；**你自己做工程时，分步走更好查。**

工程惯例 · **工程惯例：「头文件是合同」的三条推论**

这句话不是一个比喻，它有三条可以直接落地的推论：

- **改头文件 = 改所有包含它的编译单元**：所以头文件要**尽量稳定**。把实现细节（私有的结构体、内部函数）留在 `.c` 里，头文件里只放别人真正要用的东西。
- **头文件要自足**：一个头文件被单独 `#include` 时应当能编过。`node.h` 如果用到 `size_t` 就得自己 `#include <stddef.h>`，不能指望「反正调用者会包含」。`forward_decl/src/list.c` 里那句 `#include <stddef.h>` 就是这件事的现场——它用 `NULL`，就自己把它带进来。
- **头文件不能有副作用**：不能在里面写 `printf`、不能分配内存、不能依赖「包含它的顺序」。它只描述形状。

代价：拆得越细，头文件越多，改一处牵连的面越大。所以「什么该进头文件」不是越多越好——**只放合同，不放实现。**

多文件调试：`-g` 让断点跨文件

最后一件小事，但它是 `-g` 在多文件工程里最直接的用处：断点可以打在**另一个文件**里。

终端会话 break logic.c:12：断点落在另一个源文件的第 12 行 ch07-gdb-multifile

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude src/main.c src/io.c
src/logic.c -o app.exe
$ gdb --batch -ex "break logic.c:12" -ex run -ex bt ./app.exe
Breakpoint 1 at 0x1400015e8: file src/logic.c, line 12.
[New Thread 90180.0x1de34]
Thread 1 hit Breakpoint 1, fib (n=0) at src/logic.c:12
12      }
#0  fib (n=0) at src/logic.c:12
#1  0x00007ff7c62214df in main () at src/main.c:9
$ rm -f app.exe
```

三行关键信息：

- `Breakpoint 1 at …: file src/logic.c, line 12.` —— 你写的是 `logic.c:12`，gdb 按**文件名**找到了 `src/logic.c`，说明它读了 `.o` 里那张「文件与行号」的对照表（那是 `-g` 放进去的）；
- `Thread 1 hit Breakpoint 1, fib (n=0) at src/logic.c:12` —— 停下的是 `logic.c` 里的 `fib`；
- `#1 … in main () at src/main.c:9` —— **调用者在另一个文件里**，第 9 行。`bt` 的每一层都带着自己的文件与行号，这就是多文件调试的全部意义。

（`0x1400015e8` 与 `0x00007ff7308214df` 这两串地址每次运行都会变，别拿它们当证据；稳定的是 `src/logic.c:12` 与 `src/main.c:9`。）

17.10 这一节新增了什么，以及接下来

回到 17.1 的那棵树，现在你可以对着**第 7 帧**逐个点名了：

- `examples/ch07/draft.c` —— 单文件草稿，本节的反面教材，16 行；
- `examples/ch07/two_files/` —— 最小工程：`src/main.c` + `src/greet.c` + `include/greet.h`；
- `examples/ch07/undefined_helper/`、`double_definition/` —— 两类链接错误，各有一份 `compile_fail` 清单；
- `examples/ch07/no_guard/`、`guarded_include/` —— 重复包含的坏例子与两种修法；
- `examples/ch07/static_scope/`、`extern_shared/`、`const_in_header/` —— 链接性三件套；
- `examples/ch07/forward_decl/` —— 前向声明与不完全类型，外加一个 `sizeof` 探针；
- `examples/ch07/layered_app/` —— 三层工程，`build.sh` 编的就是它；
- `examples/ch07/split_build/` —— 分步编译的对照例；
- `examples/ch07/build.sh` —— 没有 make 时的替代品；
- 八个 `session-*.txt` —— 本节所有终端证据的原文。

下一个单元（c18 调试与排错）会接着这里往下走：程序真的崩了时，怎么用退出码、`gdb` 与 `fflush` 把现场找回来。

小结 · 这一节的要点

- **每个 `.c` 独立编译**成一个 `.o`，符号在**链接期**按名字汇合；`#include` 只是把声明搬进来，不产生机器码（17.2）；
- 头文件里只放三类东西：**声明、宏、类型定义**；函数体与有初值的变量留在 `.c` 里，非要在头文件里给定义就加 `static`（17.3、17.7）；
- 本机的链接失败**只有一行** `collect2.exe: error: ld returned 5 exit status`，「少定义」与「重复定义」长得一模一样——分辨它们靠 `nm`：`U` 找不到 `T` 是缺，两个 `T` 是重（17.4、17.5）；
- `static` 是内部链接（`nm` 里小写 `t` / `r`），`extern` 与「什么都不写」是外部链接（大写的 `T` / `B` / `R`）；**C 的文件作用域 `const` 默认外部链接**，所以头文件里的 `const` 必须加 `static`（17.7）；
- 前向声明 `struct Node;` 给出一个**不完全类型**：够声明指针与参数，不够取大小、不能访问成员；它也正是破循环依赖的手段（17.8）；
- `-c` 加链接等于把「一条命令」拆开；改头文件会让**所有包含它的 `.o` 过期**，`gcc -MM` 是判断范围的工具（17.9）。

自测题 · 自测题（先自己答，再对答案）

1. 你把 `greet.c` 从编译命令里删掉，`main.c` 一字没改。报错会出现在编译期还是链接期？屏幕上大概会看到几行？你怎么在不动源码的情况下确认「缺的是 `greet`」？
2. 头文件里能放哪三类东西？为什么 `int greet_banner(const char *who) { return 0; }` 这句放进头文件会出事，而 `int greet_banner(const char *who);` 不会？
3. `main.c` 与 `greet.c` 都写了 `#include "greet.h"`。如果 `greet.h` 里写着 `const double PI = 3.14;`，会发生什么？给出两种修法，并说出加 `static` 之后 `nm` 里的字母为什么会变。
4. `struct Node;` 这一行让编译器知道什么、不知道什么？为什么 `struct Node *node_create(void);` 能编过，而 `sizeof(struct Node)` 不能？
5. 你改了 `include/logic.h`，但只重新链接（`gcc main.o io.o logic.o -o app.exe`），为什么程序可能还是旧行为？哪些 `.o` 该重编，你怎么知道？

注 · 自测题答案

1. 出现在**链接期**：`greet.c` 不参与，就没有哪个 `.o` 提供 `greet`，而 `main.o` 里有一个 `U greet`。屏幕上是**一行**：`collect2.exe: error: ld returned 5 exit status`（本机实测），退出码 1。确认办法是 `nm -u main.o` 看到 `U greet`，再对整条命令行的每个 `.o` 跑 `nm` 找 `T greet`——一个都没有，就是缺它。
2. 三类是**声明、宏、类型定义**。函数**定义**会产生机器码、要占一个符号；头文件被 5 个 `.c` 各包含一次，就有 5 份 `T greet_banner`，链接期报 multiple definition（本机同样只有一行 `collect2`）。而**声明**不产生目标码，念几遍都一样。
3. 两个 `.o` 各有一个全局的 `PI`（`nm` 里是**大写** `R PI`），链接期报重复定义——本机实测仍然只有一行 `collect2.exe: error: ld returned 5 exit status`。两种修法：① 头文件里写 `static const double PI = 3.14;`，每个编译单元各一份；② 头文件里写 `extern const double PI;`，在**一个** `.c` 里写定义。加 `static` 之后符号从 `R`（global）变成 `r`（local），链接器不再看它，所以不撞了。
4. 它让编译器知道「有一个叫 `Node` 的结构体类型」，**不知道它有几个成员、占几个字节**。指针的大小与所指对象的大小无关，所以声明 `struct Node *` 和以它为参数的函数都没问题；`sizeof` 要的正是「多大」这条缺失的信息，所以编不过（本机诊断：`invalid application of 'sizeof' to incomplete type 'struct Node'`）。同理 `p->value` 也编不过。
5. 因为**头文件的内容是在 `-c` 那一步被吃进 `.o` 的**，链接器只认 `.o` 里的符号名，看不见头文件。改了头文件，任何包含它的 `.o` 都过期。判断范围用 `gcc -MM -Iinclude src/xxx.c`：输出里列出 `include/logic.h` 的那些 `.c`，对应的 `.o` 都要重编；`io.c` 的依赖里没有 `logic.h`（本机实测 `io.o: src/io.c include/io.h`），所以它不用动。

调试与排错：gdb、退出码与排错套路

学习目标 · 看完这一单元，你应该能

- 程序崩了之后第一眼看**退出码**与 **stderr**，并说出 `3221225477`、`139`、`-1073741819` 三个数为什么是同一件事；
- 用 `gdb --batch -ex …` 把一次调试写成**一条可以重跑的命令**，并分清行、函数、条件、`watch` 四类断点各自适合抓什么；
- 在崩溃现场用 `bt` 找到**自己代码里的那一行**（`#0` 与 `#1` 各是谁），并说出 `p`、`x/8xb`、`info locals`、`info frame` 各看到什么；
- 解释 `-O2 -g` 下变量为什么变成 `<optimized out>`、行号为什么会漂，从而知道调试期该用哪一档编译；
- 手边没有调试器时，用 `printf` 二分法、`-D` 开关式打印、`assert` 与 `fflush(stdout)` 把范围夹死；拿到「结果不对」的程序，先按排错清单与静态检查白拿证据，再动手调试。

18.1 先看工程长什么样

前面两个单元把「一个 C 工程长什么样」讲完了：宏住在头文件里、声明与定义分家、每个编译单元各管各的。这一节要讲的是**出问题时你手里有什么**——所以工程里除了程序，还要多出一样东西：**证据**。

这一节是全书的第 18 单元，也是第 7 章的第 3 单元。它前面两个单元（`chapters/ch07/01-preprocessing-and-macros.md` 与 `chapters/ch07/02-multiple-files-and-linkage.md`）已经把「宏」与「多文件」讲完了，`examples/ch07/` 里因此已经躺着它们留下的一堆东西。本节在同一个项目里再加七个例子目录与一批抓下来的会话。

目录树 **这一节的例子工程：从上一节留下的东西，到七个例子加一批会话**

第 1 步 本节开始时，ch07 里已经躺着前两个单元留下的东西

```
examples/
└── ch07/
    ├── macro_basics/、macro_parens/ 等 ：宏实验，本节不动
    ├── two_files/、layered_app/ 等 ：多文件与三层模块，本节不动
    └── include/log.h、build.sh、session-nm.txt ：同上
```

第 2 步 本节往 ch07 里加前四个例子

```
examples/
└── ch07/
    ├── crash_null/ ：空指针崩溃的现场 ← 本课新增
    ├── crash_oob/ ：越界写踩到了邻居 ← 本课新增
    ├── wrong_sum/ ：结果算错了，但程序不崩 ← 本课新增
    └── uninit_value/ ：没写初值的累加器 ← 本课新增
```

第 3 步 再加上另外三个，七个到齐

```
examples/
└── ch07/
    ├── 前四个例子目录 ：同上
    ├── opt_misleading/ ：-O2 下的行号漂移 ← 本课新增
    ├── fanalyzer_first/ ：编译期就抓住的空指针 ← 本课新增
    └── double_free_debug/ ：第二次 free 的现场 ← 本课新增
```

第 4 步 每个例子目录里长这样（以 crash_null 为例）

```
examples/
└── ch07/
    └── crash_null/
        ├── crash_null.json ：清单：怎么编译、退出码是多少 ← 本课新增
        ├── include/crash_null.h ：两个函数的声明 ← 本课新增
        └── src/main.c、src/crash_null.c ：定义 ← 本课新增
```

第 5 步 本节另外留下七份抓下来的会话

```
examples/
└── ch07/
    ├── crash_null/
    │   └── session-gdb-crash.txt ：崩溃现场的整段 gdb 会话 ← 本课新增
    ├── crash_oob/
    │   └── session-gdb-watch.txt ：watch 抓住越界的那一次写 ← 本课新增
    ├── uninit_value/
    │   └── session-o0-o2.txt ：同一份源码两档的真跑读数 ← 本课新增
    ├── opt_misleading/
    │   └── session-gdb-o0-o2.txt ：两档的调试会话对照 ← 本课新增
    ├── double_free_debug/
    │   └── session-double-free.txt ：第二次 free 的崩溃现场 ← 本课新增
    ├── fanalyzer_first/session-analyzer.txt ：分析器的原文与阴性对照 ← 本课新增
    └── session-gdb-debug.txt ：本节的完整排错实录 ← 本课新增
```

第 1 帧是这一节故事的起点：前两个单元在 `examples/ch07/` 里留下了宏实验与多文件工程（`build.sh` 就是那个单元把编译命令固定下来的产物）。第 2、3 帧是本节新增的**七个例子目录**，每个都是「源文件放 `src/`、声明放 `include/`、编译参数与期望输出写在 `.json` 清单里」的最小工程。第 4 帧把一个摊开看：`crash_null` 里只有两个函数加一个 `main`，两边的代码加起来不到 25 行——因为**一个例子只讲一个坑**，出问题时你才能立刻知道是谁错了。第 5 帧是本节另外留下的**七份会话**：它们是从真实终端抓下来的原文，也是这一节大部分证据的载体——调试器的输出每次都不一样（地址、线程号都在变），适合抓成会话，而不是逐字对账的 `output` 块。

先把七个例子各自要讲的那一件事记住，后面每一节都会点名其中一个：

- `crash_null`、`double_free_debug` —— 两个**真的会崩**的程序：空指针解引用、同一块内存还两次；

- `crash_oob` —— 越界写了，程序**不崩**，只是把邻居的数字改了；
- `wrong_sum`、`uninit_value` —— 两个**不崩但结果不对**的程序：一个算错，一个没写初值；
- `opt_misleading`、`fanalyzer_first` —— 两个用来讲「证据从哪来」的程序：一个证明优化会骗调试器，一个证明编译器能在编译期就抓住空指针。

工程惯例 · **工程惯例：为什么排错要先摆好「证据的架子」**

标准对「怎么排错」一个字都没有规定——这是工程惯例。它有一个很实在的好处，也有一个要付的代价。

好处是**每条结论都能被单独验证**：`crash_oob` 只演示「越界写能改到邻居」，它不崩、不做别的事，所以一旦看到 `neighbor` 变了，你不用再排除第二种可能。命令与期望值写进 `.json` 清单，任何人拿到这个目录都能复现你说的现象。

顺带把「`src/` 放定义、`include/` 放声明」这件事在排错上的两面说清（这套摆法第 17 单元讲过）：好处是**出问题时一眼知道该去哪个文件找**——`.c` 里是行为，`.h` 里是接口；代价是**改一次头文件会波及所有 `#include` 它的编译单元**，得整体重编。本节七个例子的头文件都只有几行声明，正是为了让这个代价可以忽略。

另一个代价是多花几分钟：写一个例子要配一份清单，抓一次会话要存一个文件。换来的是「**你说的现象我看得见**」——排错能力的一半就长在这里。

18.2 崩溃第一步：看退出码，不是看屏幕

程序崩了，第一反应往往是去看屏幕，而**屏幕什么都没有**。看这个例子——它真跑起来是这样，先看调用者：

c examples/ch07/crash_null/src/main.c 第 5–15 行

```c
int main(void) {
    int *p = find_record(7);                    /* key 合法：拿到 records[1] */
    printf("合法的那次读出来：%d\n", read_value(p));

    p = find_record(-1);                        /* key 不合法：拿到空指针 */ key 不合法，拿回来的是空指针
    printf("拿到的是空指针吗：%s\n", (p == NULL) ? "是" : "不是");

    printf("下面这一句里，空指针会被解引用\n");
    printf("读出来：%d\n", read_value(p));      /* 崩在这一句 */ 这一句里空指针被解引用
    return 0;
}
```

9 key 不合法，拿回来的是空指针
13 这一句里空指针被解引用

被调用的 `read_value` 住在另一个编译单元里，它没有判空：

c examples/ch07/crash_null/src/crash_null.c 第 5–13 行

```c
int *find_record(int key) {
    static int records[3] = {10, 20, 30};
    return (key >= 0) ? &records[key % 3] : NULL; key 合法就给地址，不合法就给空指针
}

int read_value(const int *record) {
    /* 没有判空：契约写着「调用者负责」，而这一次调用者忘了 */
    return *record; 没有判空，直接解引用
}
```

7 key 合法就给地址，不合法就给空指针
12 没有判空，直接解引用

这个程序的标准输出是**一个字节都没有**：

终端输出 终端输出 crash-null-out

```
（无输出）
```

它不是没打印过——`main` 的第 7、10、12 行都在打印。是**输出还攒在缓冲区里没写出去**，进程就被操作系统终止了。同一条命令的真实现象是这样：

终端会话 同一个崩溃：stdout 与 stderr 都是 0 字节，退出码 139 `ch07-crash-exit`

```
$ ./app.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
退出码 139
$ ./app.exe | wc -c
0
$ ./app.exe 2>&1 | wc -c
0
```

`wc -c` 数的是**字节数**：stdout 是 **0**，连 stderr 一起算也是 **0**。`| cat` 是为了让这次运行经过管道（理由见下面那个 `trap`）。这段会话是抓下来的原文，一句都没有改。

但**也有崩溃会在 stderr 上留一行字**——那不是操作系统写的，是程序自己写的。下面这次崩溃的 stderr 上就有一行（这个 `-DCHECK_AVERAGE` 构建是什么，18.6 会讲；这里只看现象）：

终端会话 有字的崩溃：stderr 上一行断言，两个终端两种退出码 `ch07-crash-stderr`

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DCHECK_AVERAGE src/main.c src/wrong_sum.c -o
app_check.exe
$ ./app_check.exe 2>&1 | cat; echo "退出码 ${PIPESTATUS[0]}"
Assertion failed: a10 == 5, file src/main.c, line 13
退出码 127
$ cmd //v:on //c "app_check.exe & echo errorlevel=!errorlevel!"
Assertion failed: a10 == 5, file src/main.c, line 13
errorlevel=-1073740791
$ rm -f app_check.exe
```

这一行 `Assertion failed: a10 == 5, file src/main.c, line 13` 把**表达式、文件、行号**都写出来了——它比「3221225477」有用得多。可退出码仍然要单独看：bash 给它 **127**、cmd 给它 **-1073740791**，流水线抓到的原始值是 **3221226505**（`0xC0000409`）。**同一个终止，三个口径。**而 bash 这次的 127 **不是** `128 + 信号号`（那是上面 139 的算法），它只是 bash 对这个异常码的另一种译法，与「命令找不到」那个 127 撞了数而已。

于是排错的第一步有两条并行的线索：**退出码告诉你「怎么死的」，stderr 告诉你「死之前它自己说了什么」**。两个都要看，两个都可能空着。

标准原文 · 标准原文：退出码从哪来

> 标准里没有「崩溃」这个词。它只规定一件事：程序**正常结束**时，怎么把状态交回给运行它的环境。
>
> 5.1.2.2.3 节「Program termination」第 1 段：
>
> > If the return type of the main function is a type compatible with int, a return from the initial call to the main function is equivalent to calling the exit function with the value returned by the main function as its argument; reaching the } that terminates the main function returns a value of 0.
>
> 大意：`main` 里 `return 0;` 的那个 0，等价于调用 `exit(0)`；就算你不写 `return`，执行到 `main` 收尾的 `}` 也**返回 0**。
>
> 7.22.4.4 节「The `exit` function」第 5 段规定这个值最后去哪：
>
> > Finally, control is returned to the host environment. If the value of status is zero or EXIT_SUCCESS, an implementation-defined form of the status successful termination is returned. …
>
> 大意：最后控制权交回**宿主环境**；而「这个状态值以什么形式返回」是**实现定义**的——`0` 在 bash 里是 `0`、在别的地方长什么样，标准不管。
>
> 出处：ISO/IEC 9899:2018（C17）§5.1.2.2.3、§7.22.4.4。
>
> 要紧的那半句在标准之外：程序**非正常终止**（被信号或异常干掉）时，标准一个字都没承诺——那时的「退出码」是操作系统替你记下的**终止原因**，跟 `return` 的数字没有关系。

实验记录 · 本机实测：同一个崩溃的三个写法

- 工具链：gcc 16.1.0（本书统一 `-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32，调试器 gdb 17.2；
- 编译命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/crash_null.c -o app.exe`，编译退出码 **0**、一条警告都没有；
- 运行结果：stdout **0 字节**、stderr **0 字节**；退出码在三个口径下分别是——
  - 本书清单里写 **3221225477**（流水线抓到的原始值，也就是 `0xC0000005`「访问冲突」）；
  - MSYS 的 bash 里 `echo $?` 显示 **139**（bash 把「被信号终止」写成 `128 + 信号号`，11 号信号就是段错误 SIGSEGV）；
  - cmd 里 `%errorlevel%` 显示 **-1073741819**（同一个 32 位数按有符号读的样子）。
- 三个数是**同一件事**：`3221225477` 的十六进制是 `0xC0000005`，按 32 位有符号读是 `3221225477 - 4294967296 = -1073741819`；而 `139 = 128 + 11` 是 bash 换的另一套账本。

这也是本书在清单里写**原始值**的原因（`crash_null` 是 3221225477、`double_free_debug` 是 3221226356）：**那是闸门真抓到的那个数**，另外两个是终端替你翻译过的。

陷阱 · 「屏幕上没输出，所以崩在 printf 之前」

你会看到什么：程序里有四条 `printf`，崩完终端上一片空白，你顺着代码往前找，觉得「连第一条都没执行到」。

你会怎么误判：你会把断点设在第一条 `printf` 之前，结果发现它**被执行了**，于是开始怀疑人生。

真相：标准输出流在**不是**交互式终端时是**全缓冲**的——输出先攒在内存的缓冲区里，攒满一块（多大由实现决定）或者程序**正常结束**（或者你显式 `fflush`）才真正写出去。进程被终止的那一刻，缓冲区直接消失。所以「没输出」与「没打印过」是**两件完全不同的事**，判断依据只有一条：**退出码**。3221225477 说明它死在一次非法访存上，与它之前打印过什么无关。

想亲眼看这个差别：把程序**直接**放在终端里跑（不接管道），终端是行缓冲的，那时你会看到崩溃前的那几行。本书的会话统一走管道抓取，所以你在这里只看到 0。

18.3 gdb 上手：把一次调试写成一条命令

gdb 是个交互式程序，但**排错时最有用的形态是「一次性脚本」**——因为一条命令才能被复制、被重跑、被抓进会话记录。

{} 语法规则 · 一条能重跑的命令

形式就一行：`gdb --batch -ex "命令" -ex "命令" … 可执行文件`。

- `--batch`：执行完就退出，不进交互界面（所以能写进脚本、能贴进会话）；
- `-ex`：一条 gdb 命令，**按写下的顺序**执行（`"break …"`、`"run"`、`"bt"` 都是命令），可以连着写很多次；
- 引号别省：`-ex "break src/crash_null.c:12"` 里的空格与冒号是 gdb 语法的一部分；
- 调试期可执行文件必须带调试信息，就是本书固定参数里那个 `-g`。**没有 `-g`，gdb 只知道函数名，不知道行号、不知道变量名。**

断点有四类，各自管一件事：

断点	写法	什么时候用
行断点	`break src/crash_null.c:12`	你已经怀疑到某一行
函数断点	`break read_value`	只知道是哪个函数出错，不知道哪一行
条件断点	`break src/crash_null.c:12 if record == 0`	这一行会被执行很多次，你只想停在满足条件的那一次
观察点	`watch p.neighbor`	不知道是**谁**改了这个变量，想让它一被改就停

停住之后，单步下去用的是另外五个命令。前两个的差别是**跨不跨函数**，后三个是「怎么往前走」：

命令	一句话
`next`	执行下一行，**不进入**它调用的函数（把整个调用当成一步）
`step`	执行下一行，**进入**它调用的函数
`finish`	把当前函数跑完，停在调用者那里
`until`	一直跑，直到走出当前这一层循环（或者走到当前函数末尾）
`continue`	接着跑，等下一个断点 / 观察点

这五个不是「背下来」的事，而是回答两类问题：`step` 回答「问题在不在我调用的这个函数里」，`next` 回答「这一行本身有没有做错事」。**顺序上也有讲究**：先用 `next` 把本层走完（成本低），发现「本层每一步都对、结果却不对」时，再用 `step` 钻进被调用的那一层。

跨文件断点是这一节要专门讲清的一件事：**你的 `main` 和出问题的函数往往不在同一个文件里。**看这张图，`crash_null` 的两个编译单元是怎么被 `-g` 与 `bt` 连起来的：

第 1 帧是编译期的样子：`main.c` 里只有 `#include "crash_null.h"` 搬进来的两行声明，`read_value` 的**定义**在另一个文件里。第 2 帧是链接之后：两边合成一个 `app.exe`，`-g` 把「哪个函数的哪条机器码对应哪个文件的哪一行」写了进去。第 3 帧是断点：你写 `break src/crash_null.c:12` 时，gdb 是在**被调用者那一侧**下断点——所以第 4 帧的 `bt` 才可能跨文件把两个编译单元接起来。

工程惯例 · 工程惯例：断点写在「谁错」那一侧

这是惯例，标准不管调试器。三种写法各有用处，代价也不同：

- `break read_value`（函数断点）：好处是不用记行号、改几行代码也不会失效；代价是这个函数被调一百次它就停一百次；
- `break src/crash_null.c:12`（行断点）：最精确；代价是**改代码就得改断点**，所以本书的会话里那条命令是连同编译命令一起抓下来的；
- `break src/crash_null.c:12 if record == 0`（条件断点）：把「哪一次」也写进命令里，一次命中就是你要的那一次；代价是条件里的变量必须**当时可见**（优化过的代码里它可能已经不存在了，见 18.5）。

本书的规矩是：**会话里抓下来的那条命令，复制出来就能重跑**——所以命令里一律带文件名，不裸写行号。

下面两次都是真跑出来的。第一次是**条件断点**：同一个第 12 行，第一次调用时 `record` 是有效地址、第二次是空指针——加上 `if record == 0` 之后，它只停你要的那一次。

终端会话 条件断点：只在 record 是空指针的那一次停 ch07-gdb-cond

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/crash_null.c -o app.exe
$ gdb --batch -ex "break src/crash_null.c:12 if record == 0" -ex run -ex bt ./app.exe
Breakpoint 1 at 0x1400015b8: file src/crash_null.c, line 12.
[New Thread 99892.0x18960]
Thread 1 hit Breakpoint 1, read_value (record=0x0) at src/crash_null.c:12
12          return *record;
#0  read_value (record=0x0) at src/crash_null.c:12
#1  0x00007ff7fe671537 in main () at src/main.c:13
```

对照一下不带条件的同一份代码（下面 18.4 的前两段就停过两次）。它停在 `src/crash_null.c:12`，而 `#1` 说的是 `src/main.c:13`——那就是「不合法的那次调用」。

第二次是**观察点**。`crash_oob` 里有个结构体，两个字段挨着放：

c examples/ch07/crash_oob/include/crash_oob.h 第 4–11 行

```c
/* 两个字段挨着放：buf 占前 8 个字节，neighbor 紧跟在它后面 */
struct pair {
    char buf[8];
    int neighbor;
};

/* 把 value 写到 buf[index]：index 由调用者给，这里不做边界检查 */
void store(struct pair *p, int index, char value);
```

5 两个字段在同一个结构体里
6 前 8 个字节
7 紧跟在 buf 后面

往 `buf[index]` 里写的那个函数没有做边界检查：

c examples/ch07/crash_oob/src/crash_oob.c 第 3–5 行

```c
void store(struct pair *p, int index, char value) {
    p->buf[index] = value;              /* index 越界时，写的是 neighbor 那块地方 */
}
```

4 index 越界时，写的是邻居那块地方

跑起来是这样——**程序一点事都没有**：

终端输出 终端输出 crash-oob-out

```
越界之前：neighbor = 111111
越界之后：neighbor = 5943815
```

`neighbor` 从 `111111` 变成了 `5943815`，可退出码是 0，一条错误信息都没有。这种「写坏了别人的数据但自己活得好好的」错误，靠打印很难定位；`watch` 是专门抓它的：

终端会话 watch p.neighbor：一被改就停，还告诉你改前改后的值 ch07-gdb-watch-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/crash_oob.c -o app.exe
$ gdb --batch -ex "break src/main.c:12" -ex run -ex "watch p.neighbor" -ex continue -ex "p index" -ex "p
value" -ex "x/8xb p->buf" -ex "x/s p->buf" -ex bt ./app.exe
Breakpoint 1 at 0x1400014e8: file src/main.c, line 12.
[New Thread 94360.0x14504]
Thread 1 hit Breakpoint 1, main () at src/main.c:12
12          printf("越界之前：neighbor = %d\n", p.neighbor);
Hardware watchpoint 2: p.neighbor
Thread 1 hit Hardware watchpoint 2: p.neighbor
Old value = 111111
New value = 5943815
store (p=0x5ffea0, index=10, value=90 'Z') at src/crash_oob.c:5
```

`Hardware watchpoint 2: p.neighbor` 是 gdb 说「我给这个变量的内存挂了一个硬件观察点」（x86-64 的调试寄存器只有四个，所以硬件观察点最多四个）。停下时那两行 `Old value` / `New value` 就是这次改动的**前后值**；而 `store`

`(... ) at src/crash_oob.c:5` 说的是「这次写已经完成、程序停在第 5 行」——真正的写在第 4 行，第 5 行是函数收尾的 `}`。**观察点是写完之后才触发的**，所以 gdb 报的是下一行；看源码时要往前看一行。继续看它手里是什么：

终端会话 （续）index = 10、value = 'Z'，bt 指出是 main 第 14 行让它写的 ch07-gdb-watch-2

```
5       }
$1 = 10
$2 = 90 'Z'
0x5ffea0:       0x61    0x62    0x63    0x64    0x65    0x66    0x67    0x68
0x5ffea0:       "abcdefgh\a?Z"
#0  store (p=0x5ffea0, index=10, value=90 'Z') at src/crash_oob.c:5
#1  0x00007ff7faf31510 in main () at src/main.c:14
```

三件事一一对上：`$1 = 10` 是越界的那个下标；`$2 = 90 'Z'` 是被写进去的字节；`bt` 的 `#1` 说这次调用来自 `src/main.c:14`——正是 `store(&p, 10, 'Z');` 那一行。`0x61 0x62 …` 那两行是 `x/8xb` 与 `x/s` 按**字节**读同一块内存（下面 18.4 会讲这两个命令）。

⚠ 陷阱 · 越界写「什么都没发生」，比崩溃更难查

你会看到什么：程序跑完了，退出码 0，输出也都对——只是某几个数字莫名其妙不对；改一改代码顺序，现象又没了。

你会怎么误判：你会去怀疑打印格式、怀疑算法、怀疑编译器，因为「它根本没崩啊」。

真相：越界写是**未定义行为**（UB），而 UB 最典型的形态就是**看不出异常**。标准 6.5.6 节「Additive operators」第 8 段规定：

> If both the pointer operand and the result point to elements of the same array object, or one past the last element of the array object, the evaluation shall not produce an overflow; otherwise, the behavior is undefined.

大意：指针加减的结果**必须还落在同一个数组里**（或者正好是最后一个元素的「下一格」）；出到外面就是未定义行为。上面那次 `buf[10]` 正是算到了数组外面——所以标准不欠你任何解释，它今天改了邻居、明天可能什么都不改、也可能当场崩（第 19 单元会讲它在优化器手里还能变成什么样）。

最直接的两样是 `watch`（知道变量被谁改的）与静态分析器（本机的 `-fanalyzer` 有时能在编译期算出来，见 18.9）；此外 `-fstack-protector-all` 能在**栈上的数组**被越界写时把它变成一次终止，`assert` 则适合把边界写成代码（见 18.8）。靠肉眼看代码，你看不出 `10` 比 `8` 大——那行代码长得太正常了。

18.4 崩溃现场：bt 的 #0 与 #1

现在回到 `crash_null`，看一次完整的崩溃现场。下面三段是 `examples/ch07/crash_null/session-gdb-crash.txt` 的原文，按三次停住拆开。

调试实录 · 调试实录：一条命令看穿一个空指针

只有一条命令，做的事按顺序是：在第 12 行下断点 → `run` → `bt` → 看一眼 `*record` 与它的四个字节 → `continue`（等下一次命中）→ `bt`、`info locals`、`info args`、`info frame` → `next` 让它真的崩 → 再 `bt`。

读的时候盯三处：`record` **的值**（第一次是合法地址、第二次是 `0x0`）、`bt` **的** `#0` **与** `#1`（谁在最里面、谁在调用）、**崩溃之后 gdb 把停在哪一行。**三段会话原文紧接着放在下面，命令与输出都在里面。

终端会话 第一次命中：record 是有效地址，*record 是 0x14 ch07-gdb-null-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/crash_null.c -o app.exe
$ gdb --batch -ex "break src/crash_null.c:12" -ex run -ex bt -ex "p/x *record" -ex "x/4xb record" -ex
continue -ex bt -ex "info locals" -ex "info args" -ex "info frame" -ex next -ex bt ./app.exe
Breakpoint 1 at 0x1400015b8: file src/crash_null.c, line 12.
[New Thread 47380.0x133bc]
Thread 1 hit Breakpoint 1, read_value (record=0x7ff640ad3004 <records+4>) at src/crash_null.c:12
12          return *record;
#0  read_value (record=0x7ff640ad3004 <records+4>) at src/crash_null.c:12
#1  0x00007ff640ad14d7 in main () at src/main.c:7
$1 = 0x14
0x7ff640ad3004 <records.0+4>:   0x14    0x00    0x00    0x00
```

终端会话 （续）第二次命中：record = 0x0 ch07-gdb-null-2

```
Thread 1 hit Breakpoint 1, read_value (record=0x0) at src/crash_null.c:12
12          return *record;
#0  read_value (record=0x0) at src/crash_null.c:12
#1  0x00007ff640ad1537 in main () at src/main.c:13
No locals.
record = 0x0
Stack level 0, frame at 0x5ffe80:
 rip = 0x7ff640ad15b8 in read_value (src/crash_null.c:12); saved rip = 0x7ff640ad1537
 called by frame at 0x5ffec0
 source language c.
 Arglist at 0x5ffe70, args: record=0x0
 Locals at 0x5ffe70, Previous frame's sp is 0x5ffe80
 Saved registers:
  rbp at 0x5ffe70, rip at 0x5ffe78, xmm15 at 0x5ffe78
```

终端会话 （续）next 之后：进程被终止，bt 说出是谁调的 ch07-gdb-null-3

```
Thread 1 received signal SIGSEGV, Segmentation fault.
0x00007ff640ad15bc in read_value (record=0x0) at src/crash_null.c:12
12          return *record;
#0  0x00007ff640ad15bc in read_value (record=0x0) at src/crash_null.c:12
#1  0x00007ff640ad1537 in main () at src/main.c:13
```

三段合起来读，信息量比看上去大：

① **`bt` 的第一行就是「最里面」。** `#0 read_value (record=0x0) at src/crash_null.c:12` 是当前停住的函数，`#1 … in main () at src/main.c:13` 是**调用它**的那一行。所以只要 `#1` 是 `main`，你就已经找到了自己代码里的位置——**不用管前面那串 `0x00007ff6…`**，那是返回地址，每次运行都不一样。

② **`record = 0x0` 就是病根。** 第一次命中时它是一串合法地址（会话里那个 `0x…3004`），第二次是 `0x0`（空指针）。中间那句 `record = 0x0` 是 `info args` 打出来的——注意 `info locals` 那行说的是 **`No locals.`**：这个函数除了参数以外没有局部变量，`info locals` 只管局部变量，**参数要用 `info args`，或者用 `info frame` 一次看全**（那边写着 `Arglist at 0x5ffe70, args: record=0x0`）。

③ **`next` 之后 gdb 说 `Thread 1 received signal SIGSEGV, Segmentation fault.`** 然后停在**同一个第 12 行**上——不是它没动，而是**这一行就是出事的那一行**。这跟你第 1 单元见过的编译错误刚好相反：编译器的报错常常往后面指一行，而 `-O0 -g` 下调试器停的**就是**出事的那一行（换成 `-O2 -g` 这条就不成立了，见 18.5 的行号漂移）。

把这次崩溃画成调用树，`bt` 就不再是一堆井号了：

调用树 **崩溃现场的 bt 与调用树**

第 1 步 **main 在第 13 行，手里是一个空指针**

```
main() ：src/main.c:13，p 里是 0x0
```

第 2 步 **把指针交给 read_value，调用链变成两层**

```
main() ：还在原地等
read_value(record=0x0) ：src/crash_null.c:12，bt 的 #0 ← 本步新增
main() → read_value()  bt 的 #1 → #0
```

第 3 步 **next 让解引用真的发生**

```
read_value(record=0x0) ：解引用空指针，进程被终止
main() ：那个 #1 永远等不到返回
main() → read_value()  这条边还在栈上（#1 → #0）
```

第 1 帧是调用前：`main` 手里握着空指针。第 2 帧是栈上的样子：多了一层 `read_value`，`bt` 里它就是 `#0`、`main` 是 `#1`——**编号越小越靠里**。第 3 帧是 `next` 之后的结局：最里面那一层把空指针解引用了，进程被终止，调用链停在那一刻。

「看数据」这件事一共就五六个命令，各看一样东西：

命令	看什么	本节的实例
`p record`	按变量自己的类型打印一个值	`$1 = (const int *) 0x0`
`p/x *record`	按十六进制打印解引用出来的值	`$1 = 0x14`（就是 20）
`p *p`	顺着指针取回它指向的对象	第 4 章那个例子里的 `$4 = 42`
`x/4xb record`	**不看类型**，直接把 4 个字节按十六进制打出来	`0x14 0x00 0x00 0x00`
`x/s p->buf`	按 C 字符串读，一路上读到值为 0 的字节才停	见 18.3 那段会话的 `"abcdefgh…"`
`info locals` / `info args`	这个函数的局部变量 / 参数	`No locals.` 与 `record = 0x0`
`info frame`	当前这一帧的所有信息（地址、参数、保存的寄存器）	`Arglist at 0x5ffe70, args: record=0x0`

注意 `x/4xb record` 打出来的 `0x14 0x00 0x00 0x00`：**低位字节住在低地址**（本机是小端），所以 20 反过来写才是 14 在最前面。`x/s` 那一行也值得回味：它**不管数组边界**，一路读到值为 0 的字节才停——`buf` 只有 8 个字节，可它接着把**邻居** `neighbor` **自己的字节**也读了进来（先是写进去的 `'Z'`=0x5A，再是它的高字节 0x00，于是停住），终端把其中不是合法 UTF-8 的那个字节显示成了占位符。顺便说清：`struct pair` 这两个字段之间**没有填充**——`neighbor` 就在偏移 8 处、整个结构体 12 字节（我用 `offsetof` 量过）。

> ⚠ **陷阱** · **地址每次都不一样，别把地址当证据**
>
> **你会看到什么**：上面那段会话里，同一条命令你在自己机器上跑，`record` 的地址、`[New Thread …]` 里的线程号、`#1` 后面那串 `0x00007ff6…` **全都不同。**于是你开始怀疑「是不是我的环境不对」。
>
> **真相**：这些数都是**运行期**才定下来的——程序加载到哪、栈在哪儿、线程号是多少，每次运行都可以不同（有的系统上每次运行都会变）。所以你**只需要读三样稳定的东西**：函数名、文件名:行号、变量的值。地址只在一件事上有用：判断「这两个东西**是不是同一个**」（比如第 4 章那个例子里，`p` 与 `&x` 打出来相等）。
>
> 本书的所有会话都是这么读的：抓一次原文，说话时只用那三样稳定的信息。

刚才那个空指针的 `bt` 只有两层，因为出事的函数是被 `main` 直接调的。换一个更接近现实的现场：`double_free_debug` 里第二次 `free` 的崩溃——它的 `bt` 会先给你**五层别人的代码**：

终端会话 双重释放的现场：前五层是运行库，倒数第二层才是你 `ch07-double-free-bt`

```
$ gdb --batch -ex run -ex bt ./app.exe
[New Thread 82128.0x1692c]
第一次 release 之前: *p = 42
第一次 release 走完了, p 里的地址变了吗: 没变
warning: Critical error detected c0000374
Thread 1 received signal SIGTRAP, Trace/breakpoint trap.
0x00007ffb99765a25 in ntdll!LdrResSearchResource () from C:\WINDOWS\SYSTEM32\ntdll.dll
#0  0x00007ffb99765a25 in ntdll!LdrResSearchResource () from C:\WINDOWS\SYSTEM32\ntdll.dll
#1  0x00007ffb996dae02 in ntdll!RtlSetProcessPreferredUILanguages () from C:\WINDOWS\SYSTEM32\ntdll.dll
#2  0x00007ffb997f333a in ntdll!RtlLengthCurrentClearRunForwardEx () from C:\WINDOWS\SYSTEM32\ntdll.dll
#3  0x00007ffb996f790a in ntdll!RtlFreeHeap () from C:\WINDOWS\SYSTEM32\ntdll.dll
#4  0x00007ffb96b22d7b in ucrtbase!_free_base () from C:\WINDOWS\System32\ucrtbase.dll
#5  0x00007ff6a1b115c2 in release (p=0x6bbff0) at src/double_free_debug.c:15
#6  0x00007ff6a1b11549 in main () at src/main.c:15
```

这段 `bt` 要**从下往上读**：

- `#6 … in main () at src/main.c:15` —— 你的代码：`main` 的第 15 行调用了 `release`；
- `#5 … in release (p=…) at src/double_free_debug.c:15` —— 你的代码：`release` 里的 `free(p)`；
- `#4` 到 `#0` 全是 `ucrtbase.dll` 与 `ntdll.dll` 里的名字——**那不是 bug 的位置，那是运行库在检查时发现问题的地方。**

最后那行 `warning: Critical error detected c0000374` 很有用：`c0000374` 就是这次终止码 `3221226356`（`0xC0000374`）的十六进制写法——**同一个数的两种写法**，与 18.2 讲的那三个数是一回事。

（这个例子的 stdout 里有内容：`main` 在第二次 `free` 之前调了 `fflush(stdout)`。不加那一句，那两行输出就会像 `crash_null` 一样消失——`fflush` 的用法见 18.6。）

18.5 `-O2` 与 `-g` 一起用会骗你

到 18.4 为止，本书所有调试都在 `-O0 -g` 下做的。为什么不用优化档？因为 **`-O2 -g` 会给你一份「行号对不上、变量看不见」的调试信息**——`-g` 让它记下调试信息，`-O2` 又把它记下的东西改掉了。

看这个例子，它有一个 `static` 小函数和一个调用它的函数：

c examples/ch07/opt_misleading/src/opt_misleading.c 第 3–13 行

```c
static int helper(int v) {              /* static：只给这个编译单元用，-O2 会把它内联掉 */ static 小函数，-O2 会内联
    int local = v * 3; 一个局部量
    return local + 1; 返回值
}

int combine(int a, int b) {
    int doubled = a * 2;
    int offset = b + 7;
    int out = helper(doubled) + helper(offset); 调用它两次
    return out;
}
```

3 static 小函数，-O2 会内联
4 一个局部量
5 返回值
11 调用它两次

同一份源码编译两次，gdb 命令**一个字没改**：

终端会话 `-O0 -g`：行号是对的，变量也看得见 ch07-opt-o0

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/opt_misleading.c -o app_o0.exe
$ gdb --batch -ex "break helper" -ex run -ex "info locals" -ex "p local" -ex "p v" -ex bt ./app_o0.exe
Breakpoint 1 at 0x14000157b: file src/opt_misleading.c, line 4.
[New Thread 102736.0x152a4]
Thread 1 hit Breakpoint 1, helper (v=6) at src/opt_misleading.c:4
4           int local = v * 3;
local = 32763
$1 = 32763
$2 = 6
#0  helper (v=6) at src/opt_misleading.c:4
#1  0x00007ff64e0d15bf in combine (a=3, b=4) at src/opt_misleading.c:11
#2  0x00007ff64e0d14f6 in main () at src/main.c:8
```

终端会话 （续）同一份源码换成 `-O2 -g`：三处都不一样了 ch07-opt-o2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/opt_misleading.c -o app_o2.exe
$ gdb --batch -ex "break helper" -ex run -ex "info locals" -ex "p local" -ex "p v" -ex bt ./app_o2.exe
Breakpoint 1 at 0x1400014c0: helper. (2 locations)
[New Thread 82568.0x18a24]
Thread 1 hit Breakpoint 1.1, helper (v=<optimized out>) at src/opt_misleading.c:12
12          return out;
local = <optimized out>
$1 = <optimized out>
$2 = <optimized out>
#0  helper (v=<optimized out>) at src/opt_misleading.c:12
#1  combine (a=a@entry=3, b=b@entry=4) at src/opt_misleading.c:11
#2  0x00007ff6d9d027e4 in main () at src/main.c:8
```

两段摆在一起，差别有三处，每一处都能单独要你的命：

1. **断点的位置不一样。** `-O0` 下 `Breakpoint 1 at 0x…: file src/opt_misleading.c, line 4.`——就是函数体的第一行；`-O2` 下变成 `Breakpoint 1 at 0x…: helper. (2 locations)`——**一个函数断点变成了两个位置**，因为

`helper` 被**内联**到了 `combine` 里（调用了两次，就有两份代码）。gdb 停的那一处报的是 `src/opt_misleading.c:12`，那是 `combine` 里的 `return out;`——**行号漂了。**

2. **变量不见了。** `-O0` 下 `v=6`、`local=32763` 都看得见（注意 `local` 那个 32763 是**还没赋值时的垃圾值**，因为断点停在赋值那一行之前）；`-O2` 下 `v=<optimized out>`、`local=<optimized out>`——**这两个变量在机器码里已经不存在了**，gdb 只能这么告诉你。
3. **调用链里的函数变成了「拼出来的」。** `-O0` 的 `bt` 是 `#0 helper → #1 combine → #2 main`，三层都是真的函数；`-O2` 的 `bt` 还是三行，但 `#0` 那个 `helper` 是**内联出来的**——它没有自己的栈帧，gdb 靠调试信息替你「拼」出来。

第三段的对照更直白：**同一个函数，两档给出的行号差了 4 行**（第 8 行 → 第 12 行）。

终端会话 行号漂移：combine 是从第 8 行开始的，-O2 说它从第 12 行开始 ch07-opt-line

```
$ gdb --batch -ex "info line combine" -ex "break combine" -ex run -ex bt ./app_o2.exe
Line 12 of "src/opt_misleading.c" starts at address 0x1400014c0 <combine> and ends at 0x1400014c4
<combine+4>.
Breakpoint 1 at 0x1400014c0: file src/opt_misleading.c, line 12.
[New Thread 99384.0x15088]
Thread 1 hit Breakpoint 1, combine (a=a@entry=3, b=b@entry=4) at src/opt_misleading.c:11
11          int out = helper(doubled) + helper(offset);
#0  combine (a=a@entry=3, b=b@entry=4) at src/opt_misleading.c:11
#1  0x00007ff6d9d027e4 in main () at src/main.c:8
```

`info line combine` 是「问 gdb：这个函数的第一条机器码在哪一行」，它回答 `Line 12`——而函数在源码里明明从第 **8** 行开始。紧接着 `break combine` 也落在第 12 行（再往下 `next` 一次就直接回 `main` 了，因为整个函数体已经被合成一句话）。

`opt_misleading` 这个程序本身在两档下的输出是一样的（下面这三行是 `-O0` 档的真跑结果，由清单逐字对账）——**优化会改变的是「调试时你看到什么」，不一定改变「程序算出什么」**：

终端输出 终端输出 opt-misleading-out

```
combine(3, 4) = 53
count_matches(data, 6, 1) = 2
count_matches(data, 6, 7) = 0
```

编译器视角 · **编译器视角：`-O2` 之后「变量」这个词就不成立了**

编译器在 `-O2` 下做的是**改写**，不是逐句翻译：

1. `helper` 是个 `static` 小函数，只在本编译单元用——**内联**它比调用便宜，于是它的函数体被搬进 `combine`，原来的函数可能整个消失（这就是「2 locations」的来历）；
2. `local = v * 3; return local + 1;` 两句话合起来就是 `v * 3 + 1`——**中间量 `local` 不再需要一个存放的地方**，它进了寄存器，然后就没了；
3. 值一旦只在寄存器里活着，**没有任何内存地址**可以给 gdb 指过去。gdb 的 `<optimized out>` 不是「读失败了」，而是「调试信息里写着：这个位置在这一点上不存在」。

所以调试期的规矩不是「优化不好」，而是**分工**：`-O0 -g` 是**给人看**的，`-O2` 是**给机器跑**的。你要看行为就用前者，要看优化做了什么（像第 4 章那样去读汇编）才用后者。

工程惯例 · **工程惯例：调试期一律 `-O0 -g`，交付期再谈优化**

标准对优化档位什么也没说（那是编译器的开关，不是语言的一部分）。工程上的惯例是**把它写进构建脚本的两个目标**：`debug` 用 `-O0 -g`，`release` 用 `-O2`（或 `-O3`），而不是让人临时敲。

代价说清楚：`-O0` 编出来的程序慢、体积大（本机这个 20 行的小例子感觉不出来；大工程的差距可以很大，具体倍数取决于代码，本书没有量过），而且**有些 bug 只在 `-O2` 下出现**（未定义行为被优化器利用，第 19 单元专讲）。所以理智的做法是：**先用 `-O0 -g` 把逻辑调对，再用 `-O2` 跑一遍回归**——两边都要过，才叫真的对。

反过来的写法（拿 `-O2 -g` 当日常调试档）会带来上面那三样麻烦，而且**它失败的时机很讨厌**：小函数里变量还看得见，稍微长一点就变成 `<optimized out>`，你会以为是 gdb 坏了。

18.6 没有调试器时：printf、二分与 assert

调试器不是随时都有：别人的机器上没有、生产环境的机器上装不了、有些问题一挂上调试器就跑不出来。这一节用一个**完整的排错实录**把「没有调试器时怎么办」讲完——案例是 `wrong_sum`：一个**不崩、结果不对**的程序。

先看它错在哪。源码是这样的（模块里的两个函数）：

c examples/ch07/wrong_sum/src/wrong_sum.c 第 12–25 行

```c
int sum_to(int n) {
    int total = 0;
    for (int i = 1; i <= n; i++) {
        total += i;
    }
    TRACE("  [trace] sum_to(%d) 累加到 %d\n", n, total); 中间结果打印（用 -D 开关打开）
    return total;
}

int average_to(int n) {
    int total = sum_to(n); 先把和算出来
    TRACE("  [trace] average_to(%d) 拿到 total = %d，除数用的是 %d\n", n, total, n - 1); 再打印一次中间结果
    return total / (n - 1);            /* 错的：应该除以 n */ 病根就在这一行
}
```

17 中间结果打印（用 -D 开关打开）
22 先把和算出来
23 再打印一次中间结果
24 病根就在这一行

跑出来的三行输出，第一行是对的，后两行不对：

终端会话 现象：1 到 10 的平均值应该是 5，它给了 6 ch07-debug-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/wrong_sum.c -o app.exe
$ ./app.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
sum_to(10)      = 55
average_to(10)  = 6
average_to(100) = 51
退出码 0
```

终端输出 终端输出 wrong-sum-out

```
sum_to(10)      = 55
average_to(10)  = 6
average_to(100) = 51
```

走错路一：怀疑打印格式。 6 与 51 看上去都「像是」打印环节出了问题（四舍五入？`%d` 配错类型？）。我做的第一件事是用本书固定参数重新编一遍，**一条格式警告都没有**；而且同一段输出里第一个数 **55** 完全正确（1 到 10 的和就是 55）——打印这条路是好的。**一分钟排除一个方向，这就是「先静态后动态」的意思**：`-Wall -Wextra -Wpedantic` 里有 `-Wformat`，格式串与参数不匹配它会直接喊。

走错路二：怀疑被调用的那个函数。 `average_to` 的每一分计算都建立在 `sum_to` 的返回值上，而 `5050 ÷ 99` 与 `5050 ÷ 100` 只差 1，看起来「差不多，可能是累加求和精度的问题」。于是我用**同一份源码**编了一份带调试打印的版本——`-DTRACE_WRONG_SUM`，`wrong_sum.c` 第 5 到 10 行的那个条件编译块会把 `TRACE` 从「什么都不做」换成「打印一行」：

终端会话 printf 二分法：把中间结果打出来，看它在哪一侧开始不对 ch07-debug-2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DTRACE_WRONG_SUM src/main.c src/wrong_sum.c -o
app_trace.exe
$ ./app_trace.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
  [trace] sum_to(10) 累加到 55
  [trace] sum_to(10) 累加到 55
  [trace] average_to(10) 拿到 total = 55，除数用的是 9
  [trace] sum_to(100) 累加到 5050
  [trace] average_to(100) 拿到 total = 5050，除数用的是 99
```

这五行里三件事立刻分明了：`sum_to(10)` 累加到 `55` **是对的**（排除走错路二）；`sum_to(100)` 累加到 `5050` 也是对的；而 `average_to(10)` 拿到 `total = 55`，除数用的是 `9` ——**除数写错了**，应该是 10。

为什么叫「二分法」：你手里有一个「输入 → 输出」的黑盒，中间有很多步。每一行中间结果的打印都把嫌疑区间一分为二——`total = 55` 这一行证明「问题不在这行之前」，那它就在这行之后、在输错的那行之前。**一次打印砍掉一半，几次就能夹到一行上。**

这个例子的调试打印是**用 `-D` 开关打开的**，而不是直接把 `printf` 写进源码。这么写有两个好处，也是本书推荐的做法：**同一份源码**能编出「带打印」与「不带打印」两个版本，谁都没有被改过；而且打开它的那个参数（`-DTRACE_WRONG_SUM`）会和你抓下来的输出**一起留在会话里**，别人照着敲就能复现。

第三步用调试器确认到行——只下了一个断点，然后让 gdb 把两种写法**都算一遍**：

终端会话 gdb 断点：把「正确的写法」与「写下的写法」并排算一次 ch07-debug-4

```
$ gdb --batch -ex "break src/wrong_sum.c:24" -ex run -ex "info locals" -ex "p total / n" -ex "p total / (n - 1)" -ex bt ./app.exe
Breakpoint 1 at 0x14000157e: file src/wrong_sum.c, line 24.
[New Thread 102080.0x138e4]
Thread 1 hit Breakpoint 1, average_to (n=10) at src/wrong_sum.c:24
24          return total / (n - 1);            /* 错的：应该除以 n */
total = 55
$1 = 5
$2 = 6
#0  average_to (n=10) at src/wrong_sum.c:24
#1  0x00007ff738e314d4 in main () at src/main.c:8
```

`$1 = 5` 是 `p total / n`（**应该是的**答案），`$2 = 6` 是 `p total / (n - 1)`（**实际**算出来的），两个数并排一摆，第 24 行就是病根。这一招在没有调试器的环境里做不到，但**在有调试器的时候非常值得用**：`p` 后面可以写**任何表达式**（包括你「本来想写」的那一行），拿它和代码里写的那一行比。

同一份源码还可以再往前一步：把「期望值」直接写成代码。`wrong_sum/src/main.c` 里有一段用 `-DCHECK_AVERAGE` 打开的断言：

c examples/ch07/wrong_sum/src/main.c 第 11–14 行

```
11  #ifdef CHECK_AVERAGE
12      assert(s == 55);                /* 这一条会过：sum_to 是对的 */ 这一条会过：sum_to 是对的
13      assert(a10 == 5);               /* 这一条会响：1 到 10 的平均值应该是 5 */ 这一条会响：平均值应该是 5
14  #endif
```

12 这一条会过：sum_to 是对的
13 这一条会响：平均值应该是 5

这段断言由 `-DCHECK_AVERAGE` 打开。它失败时的样子、以及**这次终止的三个口径**，18.2 已经看过：`Assertion failed: a10 == 5, file src/main.c, line 13`，bash 报 **127**、cmd 报 **-1073740791**、原始值是 **3221226505**（`0xC0000409`）。**断言写出来的东西比退出码有用得多**——表达式、文件、行号一次给全。注意本机这条消息里**没有函数名**，而标准脚注给的样例格式里有 `function abc` 一段：

标准原文 · 标准原文：`assert` 是什么

7.2 节「Diagnostics」第 1 段规定 `NDEBUG` 的作用：

> If NDEBUG is defined as a macro name at the point in the source file where <assert.h> is included, the assert macro is defined simply as `#define assert(ignore) ((void)0)`

大意：**在 `#include <assert.h>` 那一刻**，如果 `NDEBUG` 已经被定义，`assert` 就变成一句什么都不做的空话（这就是为什么发布版可以用 `-DNDEBUG` 把这些检查全部关掉）。

7.2.1.1 节「The `assert` macro」第 2 段规定它失败时做什么：

> The assert macro puts diagnostic tests into programs; it expands to a void expression. When it is executed, if expression (which shall have a scalar type) is false (that is, compares equal to 0), the assert macro writes information about the particular call that failed (including the text of the argument, the name of the source file, the source line number, and the name of the enclosing function …) on the standard error stream in an implementation-defined format. It then calls the abort function.

大意：`assert(表达式)` 在表达式为假时，把「这次调用」的信息（**参数的原文、文件名、行号、所在函数名**）按**实现定义的格式**写到标准错误流，然后调用 `abort`。

出处：ISO/IEC 9899:2018（C17）§7.2、§7.2.1.1。「实现定义的格式」这一句正好解释了上面那处差别：本机的实现选择不打函数名。

再加一句 `-DNDEBUG`，同一份源码的断言就整段消失：

终端会话 （续）加上 -DNDEBUG：断言被编译掉，程序照常跑完 ch07-debug-3b

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DCHECK_AVERAGE -DNDEBUG src/main.c src/wrong_sum.c
-o app_ndebug.exe
$ ./app_ndebug.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
sum_to(10)      = 55
average_to(10)  = 6
average_to(100) = 51
退出码 0
```

这几样手段各自付什么代价，值得写清楚：

工具	抓住什么	代价
`printf` / `fprintf(stderr, …)`	任意位置的中间值	要改（或用 `-D` 开）代码、要重编；打印本身会改变时序，并发问题可能因此消失
`-D` 开关式调试打印	同一份源码的两个版本，证据可复现	源文件里要预先埋好条件编译块（本例在 `wrong_sum.c` 第 5–10 行）；开关一多，同一份源码就有很多种形态
`assert`	把「我认为这里必然是……」写成代码，失败时给出文件与行号	有运行开销（每个断言一次判断）；发布版必须 `-DNDEBUG` 关掉；**不能拿它做用户输入的检查**（用户输入不合法不是程序的 bug）
`fflush(stdout)`	崩溃前已经打印过的内容（缓冲区强制刷出）	每处都要手动写；漏了就白打
二分注释法（`#if 0 … #endif`）	把一半代码从程序里摘掉，看现象在哪一半消失	摘掉的代码**不参与编译**，警告也一起消失；改源码，不适合留证据

二分注释法是最后一招：把可疑的一半代码用 `#if 0 … #endif` 关掉（就是本章第 1 单元讲的条件编译），重新编译，看现象在哪一半消失。我在 `crash_oob` 的一份副本上做过：把 `src/main.c` 第 14 行那句 `store(&p, 10, 'Z');` 用 `#if 0` / `#endif` 包起来，再跑 `gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/crash_oob.c -o bisect.exe && ./bisect.exe`，输出变成 `越界之前：neighbor = 111111` 与 `越界之后：`

`neighbor = 111111` ——**现象消失了**，说明问题就出在被关掉的那一句上。它的代价写在表的最后一行：关掉的代码不参与编译，警告也一起消失；所以它适合**夹范围**，不适合**留证据**。

顺带把 18.4 里那个问题补完：`double_free_debug` 的 stdout 为什么留得住？因为 `main` 在每次 `printf` 后面写了 `fflush(stdout)`（`src/main.c` 第 9、13 行）——那是**显式刷缓冲**，与调试打印是同一类手段里的另一种。

终端输出 终端输出 double-free-out

```
第一次 release 之前：*p = 42
第一次 release 走完了，p 里的地址变了吗：没变
```

18.7 未初始化读：两档读数不一样

第二个「不崩但结果不对」的例子更阴：**它的结果不保证稳定**。这个函数忘了给累加器赋初值：

c examples/ch07/uninit_value/src/uninit_value.c 第 3–9 行

```c
int sum_uninitialized(int n) {
    int total;                      /* 没有初值：这里读的是这块栈内存原来的字节 */ 没有初值，读的是这块栈内存原来的字节
    for (int i = 1; i <= n; i++) {
        total += i; 第一次累加就把这个「原来的字节」用上了
    }
    return total;
}
```

4 没有初值，读的是这块栈内存原来的字节
6 第一次累加就把这个「原来的字节」用上了

主程序把同一个函数**调用两次**，并把两次的结果与「写对的那份」都打出来：

终端会话 -O0 下连跑两次：读数一模一样，但两次调用之间不一样 ch07-uninit-o0

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/uninit_value.c -o app_o0.exe
$ ./app_o0.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
写对的那份 sum_initialized(100) = 5050
没写初值，第一次读到的 total   = 5050
没写初值，第二次读到的 total   = 10100
两次读到的一样吗：不一样
两次相差：5050
退出码 0
$ ./app_o0.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
写对的那份 sum_initialized(100) = 5050
没写初值，第一次读到的 total   = 5050
没写初值，第二次读到的 total   = 10100
两次读到的一样吗：不一样
两次相差：5050
退出码 0
```

终端输出 终端输出 uninit-out

```
写对的那份 sum_initialized(100) = 5050
没写初值，第一次读到的 total   = 5050
没写初值，第二次读到的 total   = 10100
两次读到的一样吗：不一样
两次相差：5050
```

读数要连起来看：**第一次 5050、第二次 10100，差正好 5050。**这说明第二次调用用的是**第一次调用留下的那一格栈内存**（同一个栈位置被复用），于是上一次的 5050 变成了这一次的初值。而第一次那一格在那一刻是 0（所以 0 + 5050 正好等于正确答案）——**「那一格是 0」是本机的观察结果，不是标准保证的事情**；换一个编译器、换一次运行，它可以是任何字节。

这不是从结果反推的：在 `return total;` 上停两次，`&total` 两次指向同一个地址。

终端会话 gdb：两次调用的 &total 是同一个地址，值分别是 5050 与 10100 ch07-uninit-addr

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/uninit_value.c -o app_o0.exe
$ gdb --batch -ex "break src/uninit_value.c:8" -ex run -ex "p &total" -ex "p total" -ex continue -ex "p
&total" -ex "p total" ./app_o0.exe
Breakpoint 1 at 0x140001586: file src/uninit_value.c, line 8.
[New Thread 152116.0x1ecfc]
Thread 1 hit Breakpoint 1, sum_uninitialized (n=100) at src/uninit_value.c:8
8           return total;
$1 = (int *) 0x5ffe6c
$2 = 5050
Thread 1 hit Breakpoint 1, sum_uninitialized (n=100) at src/uninit_value.c:8
8           return total;
$3 = (int *) 0x5ffe6c
$4 = 10100
$ rm -f app_o0.exe
```

`$1` 与 `$3` 都是 `(int *) 0x5ffe6c`，`$2` 是 5050、`$4` 是 10100——**同一格内存，第一次调用结束时留下了 5050，第二次就从 5050 起算**，于是 5050 + 5050 = 10100。第一次那一格当时是 0，所以第一次的结果「碰巧」等于正确答案。

接下来把同一份源码换成 `-O2` 再编一次。编译时会多出一条警告，而读数**变了**：

终端会话 （续）-O2：编译器先警告，再给出完全不同的两个读数 ch07-uninit-o2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/uninit_value.c -o app_o2.exe
src/uninit_value.c: In function 'sum_uninitialized':
src/uninit_value.c:6:15: warning: 'total' may be used uninitialized [-Wmaybe-uninitialized]
    6 |         total += i;
      |         ~~~~~~^~~~
src/uninit_value.c:4:9: note: 'total' was declared here
    4 |     int total;                          /* 没有初值：这里读的是这块栈内存原来的字节 */
      |         ^~~~~
$ ./app_o2.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
写对的那份 sum_initialized(100) = 5050
没写初值，第一次读到的 total   = 5050
没写初值，第二次读到的 total   = 5050
两次读到的一样吗：一样
两次相差：0
退出码 0
```

终端会话 （续）再跑一次 -O2：还是「一样」，两次都等于正确答案 ch07-uninit-o2b

```
$ ./app_o2.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
写对的那份 sum_initialized(100) = 5050
没写初值，第一次读到的 total   = 5050
没写初值，第二次读到的 total   = 5050
两次读到的一样吗：一样
两次相差：0
退出码 0
```

终端输出 终端输出 uninit-o2-out

```
写对的那份 sum_initialized(100) = 5050
没写初值，第一次读到的 total   = 5050
没写初值，第二次读到的 total   = 5050
两次读到的一样吗：一样
两次相差：0
```

两组读数摆在一起，有三件事必须分清：

1. **同一个程序、同一份输入，两档给出的是两套不同的事实**：`-O0` 说「两次不一样、差 5050」，`-O2` 说「两次一样、都是 5050」。这不是「哪一档算错了」——而是**这段代码本来就没有一个标准承认的答案**（下面 `standard` 块给条款）。
2. **`-O2` 还顺手给了一条警告**：`warning: 'total' may be used uninitialized [-Wmaybe-uninitialized]`。有意思的是 **`-O0` 下它一个字都不说**——因为这条警告要靠优化器分析数据流。这也是「未初始化」在本书排错清单里排第二的原因：**它在最常用的调试档下往往静默。**

3. **两档的「稳定」不是同一回事。** `-O0` 那一份我连续跑了五次，五次读数完全相同（5050 / 10100）——那是因为**同一个进程的栈初始状态每次恰好一样**（换一行代码、换一个构建都可能变），不是语言给的承诺。`-O2` 那一份也跑了五次、五次相同，但原因完全不同：**它根本没有去读那一格**（本节最后一段用汇编说明）。两种「稳定」都不受标准保护——第 19 单元会专门讲这种「看起来稳定、其实没有保障」的状态有多脆。

实验记录 · 本机实测：这个例子的全部读数

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- `-O0` 两份命令各跑两次（会话原文见上），四次读数**完全一致**：`5050 / 10100 / 不一样 / 相差 5050`，退出码都是 **0**；
- `-O2` 两次读数**完全一致**：`5050 / 5050 / 一样 / 相差 0`，退出码都是 **0**；编译期多出 `-Wmaybe-uninitialized` 一条警告（编译退出码仍是 0）；
- 两档的读数**不同**，这就是「不稳定」的实证；本书把两档各写进一份清单（`uninit_value.json` 与 `uninit_value_o2.json`），两份 stdout 都由流水线逐字对账——所以上面那两个输出块不是我挑的好看的，而是两档各自**真跑出来**的。

这一节的结论不是「-O2 修好了它」：`-O2` 下那个不确定的初值没有参与运算，只是因为优化器把整个循环改写成了别的形状——我拿 `gcc -std=c17 -O2 -Iinclude -S src/uninit_value.c -o uninit_o2.s` 看过那份汇编，`sum_uninitialized` 里是 `xorl %edx, %edx` 加一串闭式算术，**根本没有一次「读那一格」**。同样的未初始化读，换一份代码、换一个优化档，完全可能给出第三种结果。标准的态度是：**这种代码本来就不该存在**。

标准原文 · 标准原文：没写初值的对象

6.7.9 节「Initialization」第 10 段：

> If an object that has automatic storage duration is not initialized explicitly, its value is indeterminate. If an object that has static or thread storage duration is not initialized explicitly, then: ...

大意：**自动存储期**（就是普通的局部变量）的对象如果没显式初始化，它的值是**不确定的**（indeterminate）；而静态存储期的对象不初始化时，标准反而规定得清清楚楚（算术类型为 0、指针为**空指针**）。

「不确定的值」本身还不是 UB——**读它**才是。6.3.2.1 节「Lvalues, arrays, and function designators」第 2 段给的就是这一环：

> If the lvalue designates an object of automatic storage duration that could have been declared with the register storage class (never had its address taken), and that object is uninitialized (not declared with an initializer and no assignment to it has been performed prior to use), the behavior is undefined.

大意：一个自动存储期的对象如果**从没被取过地址**（本来可以声明成 `register`），而且**在用到它之前既没初始化、也没被赋过值**，那么读它就是未定义行为。`total` 正好符合：没有初值，第一次 `total += i` 就先读了它。

至于「未定义行为」是什么意思，3.4.3 节「undefined behavior」给的定义是：

> behavior, upon use of a nonportable or erroneous program construct or of erroneous data, for which this International Standard imposes no requirements

大意：**未定义行为**就是「这份标准对它不提任何要求」。3.4.3 节第 2 段的注还专门说明这个范围可以有多大：从**完全不动**（结果不可预测），到**按环境特有的方式办**（可能给诊断），一直到**终止翻译或执行**。

出处：ISO/IEC 9899:2018（C17）§6.7.9、§6.3.2.1、§3.4.3。

所以「我的机器上跑出来是 5050」这句话在法律上毫无意义：**它今天可以给你 5050，明天可以给你任何东西，而且不需要给你任何提示。**

18.8 排错清单：按概率往下查

调试没有捷径，但**有顺序**。下面这张表是本书推荐的次序，按「在这台机器上被工具抓到的概率」从高到低排；每条在本书里都有真实例子或实测撑着，出处写在最后一列：

先查什么	你会看到什么	本机怎么抓
输入边界	空输入、0、负数、超大值下行为不同	把边界当测试数据跑一遍；`assert` 把前提写下来
未初始化	读数不稳定、换优化档结果就变	`-O2` 下的 `-Wmaybe-uninitialized`（18.7）
越界	别人的数据被改了、或者干脆没反应	`watch`（18.3）、`-fanalyzer` 的 `-Wanalyzer-out-of-bounds`
生命周期	用了已经还回去 / 已经返回的对象	`-fanalyzer` 的 `-Wanalyzer-use-after-free`；`bt` 看第二处 `free` 从哪来（18.4）
判空	退出码 3221225477、`bt` 里那个指针是 `0x0`	断点 + `info args`（18.4）、`-fanalyzer` 的 `-Wanalyzer-null-dereference`（18.9）
格式串	打出来的是荒唐的数字 / `%s` 打空指针	`-Wall` 里的 `-Wformat` 默认就开着（第 1 单元讲过）
返回值被忽略	`malloc` 返回空、`fopen` 失败、`scanf` 没读进来	本机 `malloc` 的声明带 `warn_unused_result`，忽略它 `-Wall` 就报 `-Wunused-result`；`fopen` / `scanf` 没标这个属性、编译器不喊，只能自己查

三条纪律，比这张表的顺序更要紧：

1. **一次只改一件事。**改两处再跑，现象变了你也不知道是哪一处起作用；本书所有例子都是「一个例子一个坑」，就是为此。
2. **先看证据，再看代码。**退出码 → stderr → gdb 的 `bt` → 变量值，一路下来你手里该有三个**具体的事实**（比如「退出码 3221225477」「`record = 0x0`」「`#1` 在 `src/main.c:13`」），再去读代码。
3. **把「我以为」变成「我测过」。**`assert`、临时打印、`-D` 开关，都是把一个猜测变成一条可复现证据的手段。

清单里排第一的「输入边界」值得单独说一句：本书的例子几乎都不做输入检查（`crash_oob` 的 `store` 连下标都不看），因为**输入边界错**是现实中最常见的一类。而它也是最容易用 `assert` 提前抓到的：`assert(index >= 0 && index < 8);` 写在 `store` 开头，18.3 那次越界就会当场在那里停下，而不是悄悄改掉邻居。

18.9 先静态后动态：能白拿的先拿

前面所有工具都要**把程序跑起来。**但有一类证据**编译期就能拿到**，而且几乎不要钱：编译器的警告，以及它在 `-fanalyzer` 下打开的**静态分析器。**

这个例子的空指针解引用，`-fanalyzer` 在**编译期**就指出来了：

c examples/ch07/fanalyzer_first/src/fanalyzer_first.c 第 5–16 行

```c
int *find_slot(int key) {
    static int slots[4] = {1, 2, 3, 4};
    if (key < 0 || key >= 4) {
        return NULL;
    }
    return &slots[key];
}
```

```c
12
13  int run_once(void) {
14      int *slot = find_slot(9);          /* 常量 9：一定越出范围，拿到的必然是空指针 */ 常量 9，一定走上面那一支
15      return *slot;                      /* 没有判空：编译器在编译期就能看出来 */ 没有判空
16  }
```

7 key 越出范围的判断
8 这一支返回空指针
14 常量 9，一定走上面那一支
15 没有判空

终端会话 -fanalyzer：编译期报出「解引用空指针」，编译退出码仍是 0 ch07-analyzer-1

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -fanalyzer -c src/fanalyzer_first.c -o fa.o
src/fanalyzer_first.c: In function 'run_once':
src/fanalyzer_first.c:15:12: warning: dereference of NULL 'slot' [CWE-476] [-Wanalyzer-null-dereference]
   15 |     return *slot;                      /* 没有判空：编译器在编译期就能看出来 */
      |            ^~~~~
```

终端会话 （续）它把推理过程一行行摆出来：从 run_once 进到 find_slot ch07-analyzer-2

```
  'run_once': events 1-2
    |
    |   13 | int run_once(void) {
    |      |     ^~~~~~~~
    |      |     |
    |      |     (1) entry to 'run_once'
    |   14 |     int *slot = find_slot(9);          /* 常量 9：一定越出范围，拿到的必然是空指针 */
    |      |                 ~~~~~~~~~~~~
    |      |                 |
    |      |                 (2) calling 'find_slot' from 'run_once'
    |
    +--> 'find_slot': events 3-4
```

终端会话 （续）推理的收尾：NULL 从 find_slot 返回，在 run_once 里被解引用 ch07-analyzer-3

```
  'run_once': events 9-10
    |
    |   14 |     int *slot = find_slot(9);          /* 常量 9：一定越出范围，拿到的必然是空指针 */
    |      |                 ^~~~~~~~~~~~
    |      |                 |
    |      |                 (9) return of NULL to 'run_once' from 'find_slot'
    |   15 |     return *slot;                      /* 没有判空：编译器在编译期就能看出来 */
    |      |            ~~~~~
    |      |            |
    |      |            (10) ⚠️  dereference of NULL 'slot'
```

第一条诊断值得逐样读：`dereference of NULL 'slot'` 是问题的名字；`[CWE-476]` 是业界给这类缺陷统一编的号（**CWE** 是一份公开的缺陷分类表，476 号就是「空指针解引用」）；方括号里的 `[-Wanalyzer-null-dereference]` 是控制这条诊断的**开关名**——与第 2 单元讲的警告开关是同一套机制，只是这一族开关归 `-fanalyzer` 打开的分析器管。中间那些 `(1) … (10)` 是它**一步步推出来的过程**：`(4) following 'false' branch (when 'key >= 0')` 说明常量 9 让 `key < 0` 为假、`key >= 4` 为真，于是走 `return NULL;`，最后在 `run_once` 里被解引用。

但**编译退出码是 0**。这一点非常反直觉：分析器只负责**报告**，它不阻断编译。所以「编译过了」绝不等于「代码没问题」。同一条命令去掉 `-fanalyzer`，一个字都不报（下面那个块的第一条命令就是阴性对照）；而这个程序真跑起来确实是崩的：

终端会话 （续）阴性对照不报，程序真跑也真的崩 ch07-analyzer-4

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -c src/fanalyzer_first.c -o fa0.o
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/fanalyzer_first.c -o app.exe
$ ./app.exe | cat; echo "退出码 ${PIPESTATUS[0]}"
退出码 139
```

实验记录 · 本机实测：静态分析器能做与不能做

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 命令：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -fdiagnostics-color=never -fanalyzer -c src/fanalyzer_first.c -o fa.o`；
- 结果：stderr 上一条 `-Wanalyzer-null-dereference`，**编译退出码 0**；去掉 `-fanalyzer` 再编同一份源码，一个字都没有（阴性对照成立，所以这条诊断确实归它管）；
- 这个例子**自己就是清单里 `detect=fanalyzer` 的那个**：`fanalyzer_first.json` 里写的是 `"mode": "detect"`、`"detect": "fanalyzer"`，证据声明是编译期的 `stderrContains`——**没有** `expect.exit`，因为它是编译期检测，闸门根本不会去运行这个程序（一个注定要崩的程序，跑它没有意义）；
- 它抓不到的东西同样要记住：**分析器只在有检查器的缺陷族上有输出**。本书其它单元实测过它的边界——同一处越界，写成**常量下标**它报 `-Wanalyzer-out-of-bounds`（CWE-122），把这段代码挪进**另一个编译单元**（分析器一次只看一个 TU），它就**不报了**——同一个文件里换一个函数它照样报（我实测过）；换成变量下标的循环形状，它报的会是一条与越界无关的 `possible-null-dereference`（第 4 单元有完整对照）；`-O2` 报的 `-Wmaybe-uninitialized` 也**不是**分析器的功劳，那是优化器的数据流警告。

工程惯例 · 工程惯例：本机没有 ASan，也没有 valgrind

这条要说得直白：**这本书给不出「运行期精确的越界位置」。**

- **AddressSanitizer（`-fsanitize=address,undefined`）本机不可用**。我实测过：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fsanitize=address,undefined asan_probe.c -o asan_probe.exe` 的 stderr 是两行 `ld.exe: cannot find -lasan: No such file or directory` 与 `ld.exe: cannot find -lubsan: No such file or directory`，最后一行 `collect2.exe: error: ld returned 1 exit status`，编译退出码 **1**（`ld returned` 后面那个数是链接器的状态码，本书 `build/toolchain.json` 里记的是 5、我这次跑出来是 1；**要紧的是它链接失败**，不是那个数字）——这套 MSYS2 安装里没有 libasan/libubsan；
- **valgrind 也没装**：`which valgrind` 在本机 PATH 上找不到它（valgrind 是 Linux 工具，Windows 上本来就没有）；
- 所以本书「运行期越界」的证据只有三样：**退出码、stderr 上那行字**（如 `*** stack smashing detected ***`）、**gdb 的栈**。想抓得准一点就换编译开关：`-fstack-protector-all` 能护住栈上的数组，`-fanalyzer` 能在编译期报出一部分形状死板的越界。

代价与替代写在这里：**没有 ASan，本书就不可能给你「越界写到了第几号字节」这种精度**——那需要运行期插桩。能替代它的是**把边界检查写进代码**（`assert`、显式的长度参数与判空），这也是为什么本书一路强调接口上「从哪开始、有几个」成对出现。

18.10 可复现：把一次排错记成一份清单

这一节所有「实测」都靠同一样东西撑着：**每个例子配一份 `.json` 清单**。清单不是给编译器看的，是给「你下次还能不能复现」看的。这是 `crash_null` 的那一份：

`json` **清单：把「怎么编译、跑完该是什么」写死** `examples/ch07/crash_null/crash_null.json` 全文件

```json
{
  "id": "ch07-crash-null",
  "lang": "c",
  "sources": ["src/main.c", "src/crash_null.c"],
  "flags": ["-Iinclude"],
  "mode": "runtime_error",
  "unit": "chapters/ch07/03-debugging.md",
  "output": "crash-null-out",
  "expect": { "exit": 3221225477 }
}
```

字段各自管一件事：

字段	它记下来的是什么	为什么非有不可
`sources`	这个程序由哪几个 `.c` 组成	多文件工程里「少列一个文件」是最常见的复现失败原因
`flags`	额外编译参数（这里是 `-Iinclude`）	少了它，`#include "crash_null.h"` 就找不到
`mode`	期望它怎么结束：`run` / `runtime_error` / `compile_fail` / `detect`	`runtime_error` 明确说「这个程序**应该**崩」
`unit`	这段输出对账到哪一个写作单元	让正文里的输出块与真实 stdout 一一对上
`output`	正文里哪个 `output` 块负责贴它的 stdout	「没输出」也有专门的写法（哨兵），见下
`expect`	**证据声明**：退出码、stderr 里必须出现的片段、或者两档之间的差异	没有它，「跑过了」可以是什么都没验证

清单体系里有两条约定值得单独记住，它们都是本机实测过的：

- **崩溃的程序，stdout 通常一个字节都没有。**所以 `crash_null` 的输出块里写的是一行哨兵「（无输出）」——它表示「这个清单**期望** stdout 为空」；程序哪天真打印了东西，对账反而会失败（报 `stale_output`）。想留下崩溃前的输出，就得像 `double_free_debug` 那样显式 `fflush(stdout)`；
- **同一个数的三种写法必须写清口径。**清单里写 `3221225477`（流水线抓到的原始值），会话里贴 `139`（bash 的口径），正文里把 `0xC0000005` 与 `-1073741819` 都点一遍——它们**是同一件事**，但混着写就会变成三个「传说」。

再来一遍这一节的完整流程，也就是这本书希望你养成的习惯：**命令 → 输出 → 退出码 → 结论**，四样一起记；下次要跟别人（或者三个月后的自己）解释「这个例子为什么会崩」，把清单和会话丢过去就够了。

18.11 这一节新增了什么，以及接下来

回到 18.1 的那棵树：现在可以对着第 2 到第 5 帧逐个点名了。

目录树 🗀 **这一节结束时，examples/ch07/ 长这样**

第 1 步 **先给每个例子配齐「一套最小工程」**

```
🗀 examples/
└── 🗀 ch07/
    ├── 🗀 crash_null/
    │   ├── 🗀 crash_null.json ： 清单 ← 本课新增
    │   ├── 🗀 include/ ： 声明 ← 本课新增
    │   └── 🗀 src/ ： 定义 ← 本课新增
    └── 🗀 crash_oob/、wrong_sum/、uninit_value/ ： 同上
```

第 2 步 **另外三个例子，各有各的证据形式**

```
🗀 examples/
└── 🗀 ch07/
    ├── 🗀 opt_misleading/ ： 两档的调试会话 ← 本课新增
    ├── 🗀 fanalyzer_first/ ： 编译期检测的清单 ← 本课新增
    └── 🗀 double_free_debug/ ： 崩溃现场的会话 ← 本课新增
```

第 3 步 **未初始化那个例子有两份清单（两档各一份）**

```
🗀 examples/
└── 🗀 ch07/
    └── 🗀 uninit_value/
        ├── 🗀 uninit_value.json ： -O0 的读数
        └── 🗀 uninit_value_o2.json ： -O2 的读数 ← 本课新增
```

第 4 步 **七份抓下来的会话，与它们各自的例子放在一起**

```
🗀 examples/
└── 🗀 ch07/
    ├── 🗀 session-gdb-debug.txt ： 完整排错实录 ← 本课新增
    ├── 🗀 crash_null/、crash_oob/、uninit_value/ ： 各带自己的会话
    └── 🗀 opt_misleading/、double_free_debug/、fanalyzer_first/ ： 同上
```

- `crash_null/` —— 空指针崩溃与 `bt`（18.2、18.4）；清单 `mode: runtime_error`、退出码 3221225477，输出块是哨兵；
- `crash_oob/` —— 越界写改到邻居，`watch` 抓住（18.3）；清单 `mode: run`，退出码 0——**没崩也是错**；
- `wrong_sum/` —— 逻辑错，`printf` 二分法与 gdb 断点两种定位（18.6）；源码里埋了 `-DTRACE_WRONG_SUM` 与 `-DCHECK_AVERAGE` 两个开关；
- `uninit_value/` —— 未初始化读的两档读数（18.7）；**两份清单**，两份输出块；
- `opt_misleading/` —— `-O2 -g` 下的 `<optimized out>` 与行号漂移（18.5）；清单按 `-O0` 跑，两档的对照在会话里；
- `fanalyzer_first/` —— 编译期就抓住的空指针解引用（18.9）；清单 `mode: detect`、`detect: fanalyzer`；
- `double_free_debug/` —— 第二次 `free` 的崩溃现场（18.4）；清单 `mode: runtime_error`、退出码 3221226356，输出块里留着 `fflush` 保住的两行。

下一单元（`chapters/ch08/01-undefined-behavior-and-optimization.md`，未定义行为与优化边界）从这里接着往深走：这一节反复出现的「换个档结果就变」到底是谁在动手、`volatile` 能管住什么、编译器凭什么敢删掉你的代码——那些问题的答案都在第 19 单元。

小结 · 这一节的要点

- **崩溃第一步是退出码，第二步是 stderr**：同一个空指针崩溃，清单里是 **3221225477**（`0xC0000005`）、bash 里是 **139**（`128+11`）、cmd 里是 **-1073741819**；而带字的崩溃（`assert`）是 **3221226505**、bash **127**、cmd **-1073740791**。stdout 与 stderr 都可能是 **0 字节**——输出攒在缓冲区里，进程被终止时一起消失；
- **gdb 写成一条命令**（`--batch -ex`）才能被复制与重跑；四类断点（行 / 函数 / 条件 / `watch`）各管一件事；`bt` 的编号越小越靠里，先找**属于自己代码**的那一层，再用 `p`、`p/x`、`x/8xb`、`x/s`、`info locals`、`info args`、`info frame` 看数据（地址每次都不同是正常的）；
- **`-O2 -g` 会骗你**（`<optimized out>`、一个断点多个位置、行号漂移），调试期用 `-O0 -g`；没有调试器时靠 `printf` 二分法、`-D` 开关式打印、`assert`、`fflush(stdout)`；能编译期白拿的证据先拿（`-Wall -Wextra -Wpedantic`、`-fanalyzer`），但分析器只覆盖部分缺陷族，本机也没有 ASan 与 valgrind；
- **可复现 = 清单**：命令、源文件、参数、退出码一起记下来；崩溃程序的 stdout 用哨兵「（无输出）」声明。

自测题 · 自测题（先自己答，再对答案）

1. 同一个空指针崩溃，为什么会在三个地方看到 `3221225477`、`139`、`-1073741819` 三个数？它们之间是什么关系？`0xC0000005` 又是什么？
2. `bt` 打出来 `#0 read_value (record=0x0) at src/crash_null.c:12` 与 `#1 … in main () at src/main.c:13`，你要去哪一行找问题？`record=0x0` 说明什么？
3. 同一份源码，`-O0 -g` 下 `p local` 打印出值，`-O2 -g` 下打印 `<optimized out>`，并且 `break helper` 报 `helper. (2 locations)`、`combine` 的行号从 8 变成 12。这两件事各自的成因是什么？
4. `uninit_value` 在 `-O0` 下给出 5050 与 10100，在 `-O2` 下两次都是 5050。能不能说「`-O2` 把它修对了」？为什么？
5. `-fanalyzer` 报出了空指针解引用，可编译退出码是 0。这说明「编译通过」意味着什么、不意味着什么？本机拿不到 ASan，这个缺口该用什么补？

注 · 自测题答案

1. 它们是**同一个数**（或者同一件事）的三种读法：`3221225477` 写成十六进制是 `0xC0000005`（Windows 的「访问冲突」），按 32 位有符号读是 `3221225477 - 4294967296 = -1073741819`（cmd 的 `%errorlevel%` 就这么显示）；而 `139 = 128 + 11` 是 bash 的另一套账本（11 号信号 SIGSEGV）。本书清单里一律写流水线真抓到的 `3221225477`。
2. 去 **`src/crash_null.c` 的第 12 行**找（`#0` 是最内层、出事的那一行）；`#1 … at src/main.c:13` 是**谁调用进来的**，所以那一行也值得一起看。`record=0x0` 说明传进来的是**空指针**——`#1` 那一行把 `find_record(-1)` 的结果直接交给了 `read_value`，中间没有判空。
3. `<optimized out>` 是因为 `-O2` 把 `helper` 内联、把中间量放进寄存器之后**这些变量在机器码里已经不存在了**，调试信息里没有地址可指；`helper. (2 locations)` 是「这个函数被内联到两处」的直接证据，而行号从 8 漂到 12 是「整个函数体被合成一句话、调试信息把行号挂在了合并后的位置上」。结论：调试期用 `-O0 -g`。

4. **不能。** `-O2` 下两次读数一样、都等于正确的 5050，只是因为这个不确定的初值**没有参与运算**（优化器把循环算成了别的东西）。这段代码本身就是**未定义行为**（`total` 是自动存储期对象、没有初始化：§6.7.9 说它的值**不确定**，§6.3.2.1 第 2 段说这种对象在被读时是**未定义行为**），所以标准不保证任何一档的读数；换个编译器或换一份代码，完全可能给出第三种结果。正确的做法是给累加器写 `int total = 0;`。
5. 它说明**编译通过只意味着「语法与类型层面没有拦住你的地方」**，不意味着程序是对的：分析器只负责**报告**，不阻断编译（本机实测：带 `-fanalyzer` 时编译退出码仍是 0，而且这个程序真跑起来确实崩）。本机没有 ASan（链接时 `cannot find -lasan`）与 valgrind，所以运行期的内存错误只能靠**退出码、stderr 那行字、gdb 的栈**，再加上把边界检查与判空**写进代码**（`assert`、长度参数）。

第 8 章

未定义行为、优化与收尾

本章 2 个单元 · 页眉从这里之后的页面开始出现

1. 未定义行为与优化边界：编译器有多大自由度 零基础 · 第 19 单元 · 建议 90 分钟
2. 收尾：从会写 C 到能维护一个 C 程序 零基础 · 第 20 单元（全书最后一节） · 建议 120 分钟

未定义行为与优化边界：编译器有多大自由度

◎ 学习目标 · 看完这一单元，你应该能

- 把一段行为准确归进「定义良好 / 未指定 / 实现定义 / 未定义」四档，并各举出一个 C 里的例子；
- 说出 UB 之下编译器被允许做的三件事（假设不发生、删掉检查、给出任意结果），并用 `-O0` 与 `-O2` 的真实差异说明；
- 背出本书出现过的十二类 UB，指出每一类在哪一节留下过证据；
- 解释 `volatile` 唯一保证的那一件事，并用本机汇编说明它为什么不保证原子、顺序与可见性；
- 用 `-fanalyzer`、`-ftrapv` 与 `opt-diff` 把 UB 变成可复现的证据，并说出每一种（以及本机装不上的 ASan / UBSan）各自抓不到什么。

8.1 先看工程长什么样

这一节的对象不再是一个程序，而是一组**小实验**。上一单元结束时，`examples/` 下已经有了七个项目；这一节要建第八个：`examples/ch08/`。每个例子仍然是「`src/` + `include/` + 清单」的最小工程，因为它们要干的事只有一件——把一个含糊的问题（「这样写会怎样？」）变成一条能被命令回答的问题。

目录树 **第八个项目：一组专门用来问「会怎样」的小工程**

第 1 步 本节开始时，examples/ 下已经有七个项目

```
examples/
└── ch01/ … ch07/ ：前七个项目
```

第 2 步 新建 ch08，先放两个「读数」类例子

```
examples/
├── ch01/ … ch07/ ：前七个项目
└── ch08/ ← 本课新增
    ├── ub_oob_read/ ：越界读 ← 本课新增
    └── ub_uninit/ ：未初始化读 ← 本课新增
```

第 3 步 读数类的另外两个

说明：帧 3 到帧 9 只画这一批新增的目录，前面画过的不重复（单帧放不下十六个）

```
examples/
└── ch08/
    ├── ub_strict_alias/ ：类型双关与 memcpy ← 本课新增
    └── ub_memcpy_overlap/ ：重叠 memcpy ← 本课新增
```

第 4 步 行为与防线，先两个

```
examples/
└── ch08/
    ├── ub_div_zero/ ：除零 ← 本课新增
    └── overflow_guard/ ：溢出检查函数 ← 本课新增
```

第 5 步 行为与防线的另外两个

```
examples/
└── ch08/
    ├── overflow_trapv/ ：给 -ftrapv 的靶子 ← 本课新增
    └── signed_overflow_loop/ ：循环形状的溢出 ← 本课新增
```

第 6 步 内存与分析器视角，先两个

```
examples/
└── ch08/
    ├── heap_uaf/ ：free 之后读 ← 本课新增
    └── oob_fanalyzer_const/ ：分析器抓得到 ← 本课新增
```

第 7 步 分析器与 volatile，各一个

```
examples/
└── ch08/
    ├── oob_fanalyzer_loop/ : 分析器报偏了 ← 本课新增
    └── volatile_loop/ : 加不加 volatile ← 本课新增
```

第 8 步 volatile 汇编与补码，各一个

```
examples/
└── ch08/
    ├── volatile_asm/ : 五个 volatile 函数 ← 本课新增
    └── twos_complement_ub/ : INT_MIN 的两条 UB ← 本课新增
```

第 9 步 最后两个例子

```
examples/
└── ch08/
    ├── crash_stdout/ : 崩溃前那行输出 ← 本课新增
    └── unspecified_order/ : 未指定行为 ← 本课新增
```

第 1 帧是本节故事的起点：前七个项目各管一件事。第 2 到第 9 帧是这一节搭起来的 `examples/ch08/` ——十六个例子目录，**每帧只画这一批新增的两个**（第 3 帧的「说明」行写着这条约定：单帧放不下十六个，前面画过的不重复）。每个目录基本只演示**一个**现象：这个例子只回答「越界读会读到什么」，那个例子只回答「`volatile` 在汇编里长什么样」。唯一的例外是 `twos_complement_ub`：它把 `INT_MIN` 的两条 UB 放在一起，因为那是同一个数的两面。本节另外留下的八份**会话**、三个**探针**与五份**汇编**不在这里画，它们在 8.11 节那张图里。把例子拆得这么碎，是故意的：UB 的每一个现象都依赖「这一份二进制、这一次运行」，混在一起就说不清是谁造成的。

工程惯例 · **工程惯例：为什么一个现象一个目录，而且每条结论都要留命令**

标准对「怎么组织例子」没有任何规定，这是本书自己的规矩，理由是**证据要能被单独复核**：

- 一个目录只演示一个现象，读者才能把「改动」与「结果变化」对上号；把三种 UB 塞进一个 `main`，出了问题你也不知道是哪一句；
- 每个例子都要有清单（`examples/ch08/*/*.json`），清单里写明怎么编、怎么跑、期望什么。正文里的输出块由流水线逐字对账，不是手抄的；
- 有些现象**跑出来才算数**（越界读到什么、两档读数差多少），有些现象**跑不出来也得讲**（严格别名、重叠 `memcpy`）——后一类必须留下另一种证据：编译器的警告原文，或者标准条款。这两类在正文里会分开标。

代价是文件多：十六个目录、十七份清单。好处是这一章之后你回头查「当时那个结论是怎么来的」，只需要打开一个目录。

实验记录 · **这一节的工具链与复现方式**

- 编译器：gcc 16.1.0，统一用 `-std=c17`（C17），目标三元组 x86_64-w64-mingw32；系统 Windows 10 x64 (MSYS2 UCRT64)；
- 本节有一半结论要靠**另一次编译**才看得见（`-O2`、`-ftrapv`、`-fanalyzer`）：清单里那条用 `-O0`，正文会话里另给一条显式写 `-O2` 的；少数例子只有 output 块、没有会话；
- 清单由 `node dev-notes/build/run.js --unit … --fix` 驱动：真编译、真运行，再把 stdout 逐字回写进正文，对不上就报 `stale_output`；
- 会话文件（`session-*.txt`）里，命令本身也留在文件里；末尾的 `2>/dev/null`（或 `2>&1`）是命令的一部分，重跑才会得到与文件一模一样的结果；
- 汇编用 `gcc -O2 -S`（**不带** `-g`）抓取——这样 `.s` 里没有 `.loc` 行，读起来干净；带 `-g` 的版本会多出几百行调试指令。

8.2 四档行为：先给每件事起对名字

「这是 UB 吗？」这个问题，八成问错了。C 里跟「行为由谁决定」有关的情况有**四档**，它们的严重程度、排查方法、能不能依赖，全都不一样。

档位	谁说了算	C 里的例子	本书的实测
定义良好（well-defined）	标准说死了	`int x = 3 + 4;` 一定得到 7	到处都是
未指定（unspecified）	标准给了几种可能，实现挑一种，但**不必告诉你**	`printf("%d %d", f(), g())` 里两个实参谁先求值	本机先算右边（8.2 的会话）
实现定义（implementation-defined）	实现挑一种，但**必须写进文档**	`char` 带不带符号、负数右移是算术还是逻辑移位、`sizeof(long)`	`(char)200` = -56；`-8 >> 1` = -4；`long` 4 字节
未定义（undefined）	标准**什么都不说**	越界读、有符号溢出、除零、`free` 之后再读	8.3 起逐条给现象

「定义良好」是默认档，也是本书前七章几乎所有代码所处的档位：`3 + 4` 得 7、`sizeof("hi")` 是 3、`strlen("hi")` 是 2——这些结论在标准里被钉死了，换机器、换编译器都不会变。真正的麻烦来自另外三档，因为它们的共同点是「**你的代码没有唯一的答案**」，而你在读代码时几乎看不出这一点。四档的叫法值得背下来，因为它们对应四种完全不同的处理方式：定义良好不用管；未指定只能「不依赖顺序」；实现定义要去查手册、或者像下面的探针那样自己读一次；未定义必须改代码。

未指定行为的最小观察面是**实参求值顺序。**`examples/ch08/unspecified_order/` 里两个函数各自打印一行再返回，主程序把它们一起塞进 `printf`：

c examples/ch08/unspecified_order/src/unspecified_order.c 第 5–13 行

```c
int left_tag(void) {
    printf("  先算的是：左参数\n");
    return 1;
}

int right_tag(void) {
    printf("  先算的是：右参数\n");
    return 2;
}
```

6 左参数被求值时打印
11 右参数被求值时打印

c examples/ch08/unspecified_order/src/main.c 第 5–10 行

```c
int main(void) {
    printf("准备算 printf(\"%d %d\", left_tag(), right_tag())\n");
    printf("两个返回值：%d 和 %d\n", left_tag(), right_tag());
    printf("标准没规定哪个先算 —— 本机是先算右边那个\n");
    return 0;
}
```

7 两个函数谁先跑，标准不管
8 本机是右边先跑

终端输出 终端输出 unspecified-order-out

```
准备算 printf("%d %d", left_tag(), right_tag())
  先算的是：右参数
  先算的是：左参数
两个返回值：1 和 2
标准没规定哪个先算 —— 本机是先算右边那个
```

运行结果很干脆：打印的是**右参数**那一行在前。这**不是** UB：程序每次都会打印两行，结果集合是确定的（`1` 和 `2` 两个返回值不会变），变的是顺序。所以你可以依赖「两行都会打印」，但**不能**依赖「哪一行在前」。

分界线在这儿：未指定的行为**有一个确定的答案集合**，程序不会因此做任何「不可能」的事；未定义的行为**连答案集合都没有。**这条分界很实用——看到一段依赖求值顺序的代码，你该做的是把两个调用拆成两句（`int a = f(); int b = g();`），这样顺序由你决定；看到一段 UB，改顺序是没用的，必须改写法。顺带记一条容易混的：同一个表达式里**两**

次修改同一个对象（`i = i++ + 1;` 这种）从 C99 起就是**未定义**，不是未指定——6.5 节第 2 段规定，一个标量对象上的副作用如果与另一个未定序的副作用（或取值）撞在一起，行为未定义。

实现定义的证据是一次性读出来的。`examples/ch08/impl_defined.c` 是个没有清单的探针，跑一遍就知道这台机器替你做了哪些决定：

终端会话 一次性读出这台机器的「实现决定」 ch08-impl-defined

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g impl_defined.c -o impl_defined.exe
$ ./impl_defined.exe
CHAR_BIT          = 8
sizeof(int)       = 4
sizeof(long)      = 4
sizeof(void *)    = 8
LONG_MAX          = 2147483647
(char)200         = -56     (char 带不带符号由实现决定)
-8 >> 1           = -4      (负数右移是算术还是逻辑移位由实现决定)
0x01020304 的四个字节 = 04 03 02 01   (低地址在前 = 小端)
sizeof(struct pair)  = 8   (1 + 4 = 5? 看对齐补了几个字节)
offsetof(pair, i)    = 4
```

这里有两个数字值得停一下：`sizeof(long)` 是 **4**（不是 8，`long` 在本机和 `int` 一样宽），`(char)200` 是 **-56**（本机的 `char` 带符号）。这两条都不是 UB，也都不是「定律」——标准把选择权交给实现，只要求实现把它写在本手册里。你写出依赖它们行为的代码，换一台机器就可能变，所以本书叫它们「实现定义」。

标准原文 · **标准原文：什么叫未定义行为**

> C17 的 3.4.3 节「undefined behavior」给出的定义是：
>
> > behavior, upon use of a nonportable or erroneous program construct or of erroneous data, for which this International Standard imposes no requirements
>
> 大意：**标准对这种行为不施加任何要求。**标准还接了一句注释，把可能的结果列成三条：**完全忽略**这种情况（结果不可预测）；**以环境特有的、有文档的方式行为**（比如弹一个陷阱）；或者**终止翻译或执行**（编译报错、进程被杀）。三条并列，不是一条。
>
> 出处：ISO/IEC 9899:2018（C17）§3.4.3，以及同节第 2 段的注释。

⚠ 陷阱 · **「没报错」不等于「没 UB」**

你这么判定的：程序跑完了、退出码 0、输出看起来也对，于是认为这段代码没问题。

你怎么会被坑到：本单元后面每一个 opt-diff 例子，`-O0` 那一档都能「正常走完」——越界读打印出一个数、`free` 之后读打印出 7、有符号溢出循环打印「循环结束，一共 31 轮」，退出码全是 0。真相是：这些程序在**每一档**里都是 UB，只是 `-O0` 那一档恰好没有把 UB 变成你能看见的事故。编译器在 `-O2` 下给你换一种行为，不是因为 `-O2` 制造了 bug，而是因为 UB 的许可本来就允许它这么做。

8.3 UB 之下，编译器被允许做三件事

标准对 UB 的「不施加要求」落到编译器身上，有三条常用的推论。它们不是三条独立规则，而是同一句许可的三个侧面。

1. **假设它不发生**：优化器可以把「有符号加法不会溢出」当成前提。于是 `i += i` 配 `while (i > 0)` 在 `-O2` 下被认为是无限循环，回绕那件事被当成不可能。
2. **删掉由它守护的检查**：既然溢出不会发生，`if (i + 1 > i)` 就恒真，检查可以整个删掉；既然越界不会发生，边界检查也可以删。
3. **给出任意行为，包括看起来完全正常**：越界读可以读到邻居、读到一个常数、或者读到一个每次都不同的垃圾——三者都合法。

一个例子就能把第 1 条与第 3 条同时摆出来。`examples/ch08/signed_overflow_loop/src/signed_overflow_loop.c` 里那个循环，`i` 每轮翻倍，直到 `i += i` 越过 `INT_MAX`：

c examples/ch08/signed_overflow_loop/src/signed_overflow_loop.c 第 5–19 行

```c
int overflow_walk(void) {
    int a[4] = {1, 2, 3, 4};
    int i = 1;
    int steps = 0;

    while (i > 0) {                    // 循环条件是 i > 0
        if (i < 4) {
            printf("a[%d] = %d\n", i, a[i]);
        }
        i += i;                        /* 第 31 轮之后越界：UB */   // i 翻倍，第 31 轮越过 INT_MAX
        steps++;
    }
    printf("循环结束，一共 %d 轮\n", steps);
    return steps;
}
```

10 循环条件是 i > 0

14 i 翻倍，第 31 轮越过 INT_MAX

同一份源文件，两档编译，两次运行：

终端会话 有符号溢出的循环：一档走完，一档被终止 ch08-loop-2levels

```
$ cd signed_overflow_loop                              # 有符号溢出的循环：一档走完、一档被终止
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/signed_overflow_loop.c -o loop_O0.exe
2>/dev/null
$ ./loop_O0.exe ; echo "退出码 $?"
a[1] = 2
a[2] = 3
循环结束，一共 31 轮
退出码 0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/signed_overflow_loop.c -o loop_O2.exe
2>/dev/null
$ ./loop_O2.exe ; echo "退出码 $?"
退出码 139
```

`-O0` 走完 31 轮，`-O2` 连一行输出都没有就被终止了。这里的 `139` 是 bash 的记法（128 + 11，11 是 SIGSEGV）；闸门记录的 Windows 退出码是 **3221225477**（0xC0000005，访问冲突），两份数字说的是同一件事：进程没有走到结尾。为什么？一个合理的解释是：`-O2` 认定 `i` 永远是正的（否则 `i += i` 就溢出了，而溢出被假设为不发生），于是 `while (i > 0)` 被当成恒真；接下来它按「`i` 单调增长」去安排后面的取用，地址就算到了一个不该去的地方。

把它编成四种样子，行为分成四类，这张图值得记住：

图 / 搜索过程 **同一份源码的四种走法：许可与开关**

第 1 步 -O0，什么都不假设

-O0 ： 回绕真的发生了，31 轮后结束

程序 → 退出 0 a[1]、a[2]、循环结束

第 2 步 -O2，假设溢出不发生

-O2 ： 循环条件被当成恒真

程序 → 进程被终止 退出码 3221225477，stdout 为空

第 3 步 -O2 -fwrapv，把溢出定义成回绕

-O2 -fwrapv ： 标准许可被收回，回绕按补码算

程序 → 退出 0 与 -O0 相同的三行

第 4 步 -O2 -ftrapv，把溢出定义成终止

-O2 -ftrapv ： 溢出时插入陷阱

程序 → 进程被终止 退出码 3221226505

第 1 帧是 `-O0`。第 2 帧是 `-O2` 的默认立场：**假设 UB 不发生**，于是把「不可能」写进了代码。第 3 帧与第 4 帧表示：GCC 给了两个开关，把「有符号溢出」这件 UB 重新变成有定义的行为——`-fwrapv` 定义成补码回绕，`-ftrapv` 定义成陷阱。四种走法都是同一份源码：

终端会话 四种编法（前半）：-O0 与 -O2 — ch08-four-flags

```
$ cd signed_overflow_loop                              # 同一份源码，四种编法
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/signed_overflow_loop.c -o loop.exe
$ ./loop.exe ; echo "退出码 $?"
a[1] = 2
a[2] = 3
循环结束，一共 31 轮
退出码 0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/signed_overflow_loop.c -o loop.exe
$ ./loop.exe ; echo "退出码 $?"
退出码 139
```

终端会话 （续）四种编法（后半）：-fwrapv 与 -ftrapv — ch08-four-flags-2

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -fwrapv -g -Iinclude src/main.c src/signed_overflow_loop.c -o
loop.exe
$ ./loop.exe ; echo "退出码 $?"
a[1] = 2
a[2] = 3
循环结束，一共 31 轮
退出码 0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -ftrapv -g -Iinclude src/main.c src/signed_overflow_loop.c -o
loop.exe
$ ./loop.exe ; echo "退出码 $?"
退出码 127
```

`-fwrapv` 那一档与 `-O0` 的输出逐字相同——它把「回绕」这件事重新定义好了，优化器就不再能假设溢出永不发生。`-ftrapv` 那一档被终止（bash 报 127；闸门记录的 Windows 退出码是 **3221226505**，0xC0000409）。注意这两条命令行里的开关是**给你读代码时用的**：本书的清单不允许用 `-fno-*` 去关警告，而 `-fwrapv` / `-ftrapv` 这类**定义行为**的开关可以写进检测清单，`-ftrapv` 就是本单元 8.9 节的一份证据。

8.4 优化在做什么，以及它凭什么「改语义」

「优化」不是把代码变魔术，它主要是五类改写，每一类都以「程序的可见行为不变」为前提：

改写	在干什么	本单元的证据（都真跑过）
常量折叠	能在编译期算出来的就不留到运行期	`12345 * 6789` 在**两档**汇编里都没有乘法——前端就把它折成 `83810205` 了；越界读那一格也被折成常数 0
死代码消除	结果没人用的计算、走不到的分支直接删	`-O0` 还把 `83810205` 存进栈上一格（没人读），`-O2` 连这条 store 都删了
不变量外提	循环里不变的东西搬到循环外，只算一次	`sum_plain`：`g` 从「每轮读一次」变成「读一次再乘 n」
内联	小函数展开到调用点	`call_square`：`-O2` 汇编里没有 `call`，只剩 `imull %ecx, %ecx`；`-O0` 下是一条 `call square`
向量化	一次处理多个元素	本单元没有触发（`opt_levels.c` 的串行求和也没被向量化）；它同样以「没有越界」为前提

「内联」与「死代码消除」这两行，证据在 `examples/ch08/opt_effects.c` 这个探针里。它只有三个函数：一个只有一句乘法的 `square`、一个调用它的小函数、一个算完常量却不用的函数。

c examples/ch08/opt_effects.c — 第 12–24 行

```c
12  static int square(int x) {
13      return x * x;
14  }
```

```c
int call_square(int x) {
    return square(x);                   /* 小函数：-O2 会把它展开到调用点 */ 小函数，-O2 会把它展开
}

int folded_but_unused(void) {
    int unused = 12345 * 6789;          /* 常量折叠之后没人用，-O2 整句删掉 */ 算出来的常量没人用
    (void)unused;
    return 7;
}
```

17 小函数，-O2 会把它展开
21 算出来的常量没人用

asm examples/ch08/opt_effects-O2.s 第 7–21 行

```asm
call_square:
    .seh_endprologue
    imull   %ecx, %ecx 没有 call，只剩一句乘法
    movl    %ecx, %eax
    ret
    .seh_endproc
    .p2align 4
    .globl  folded_but_unused
    .def    folded_but_unused; .scl    2;  .type   32; .endef
    .seh_proc   folded_but_unused
folded_but_unused:
    .seh_endprologue
    movl    $7, %eax 乘法整句没了，直接 movl $7
    ret
    .seh_endproc
```

9 没有 call，只剩一句乘法
19 乘法整句没了，直接 movl $7

对比 `-O0` 会看到两件不同的事。第一件：`call_square` 里有一条 `call square`（`opt_effects-O0.s` 第 31 行），而 `-O2` 把它展开成了 `imull`——这是**内联**。第二件：`folded_but_unused` 在 `-O0` 下**也没有乘法**，前端早就折成了 `movl $83810205, -4(%rbp)`（第 47 行），只是它还把结果存进栈上一格；`-O2` 连这条 store 都删了（只剩 `movl $7, %eax`）。所以「常量折叠」在 `-O0` 就有，「死代码消除」才要等优化器——这两件事经常被混成一句「优化器把常量算了」，其实不是一回事。

还有第三件：`-O2` 把 `call_square(7)` 的结果 `49` 直接写进了 `main`（`opt_effects-O2.s` 第 37 行 `movl $49, %edx`），`main` 里根本没有调用 `call_square`。内联、常量折叠、把结果传给调用者是同一条流水线上的三步，不是三条独立规则。

这五类改写有一个共同前提：**程序的行为是定义良好的**。优化器做的每一步等价变换，都建立在「这个程序没有 UB」之上。反过来读这句话，就是本单元最要紧的一条：一旦你的程序里有 UB，优化器手里那份「等价」的证明就不再成立，它做的任何改写都还是合法的——因为它可以假设那条路径根本不会发生。所以「UB 让优化有理有据地改语义」不是编译器在报复你，而是你在合同里签了「这一条不会发生」，它按合同办事。`-fwrapv` 与 `-fno-strict-aliasing` 之所以有效，是因为它们把合同改写了：前者把溢出重新定义为回绕，后者撤回了「不同类型的指针不会指向同一块内存」这个假设。

回到表里的「不变量外提」，它有直接的汇编证据。下面两个函数只差一个 `volatile`：

c examples/ch08/volatile_loop/src/volatile_loop.c 第 11–25 行

```c
int sum_plain(int n) {
    int s = 0;
    for (int i = 0; i < n; i++) {
        s += g;                         /* -O2：只读一次，再乘 n */ 循环里读普通全局 g
    }
    return s;
}

int sum_volatile(int n) {
```

```
20      int s = 0;
21      for (int i = 0; i < n; i++) {
22          s += vg;                        /* -O2：每一轮都真的读一次 */ 循环里读 volatile 全局 vg
23      }
24      return s;
25  }
```

14 循环里读普通全局 g
22 循环里读 volatile 全局 vg

终端输出 终端输出 volatile-loop-out

```
普通全局：sum_plain(4)     = 12
volatile：sum_volatile(4) = 12
读回来 vg = 3
```

`g` 在循环里没有被谁写过，所以「每轮读一次」与「读一次再乘 n」对单线程程序是同一件事——`-O2` 选了后者。这不是 bug，这正是优化的定义。可如果那个全局变量会被**别的东西**改（中断、另一块硬件、另一个线程），前提就不成立了。`volatile` 就是把「它会被别的东西改」告诉编译器。

asm examples/ch08/volatile_loop/volatile_loop-O2.s 第 17–25 行

```
17  sum_plain:
18      .seh_endprologue
19      movl    g(%rip), %eax 只读一次
20      xorl    %edx, %edx
21      imull   %ecx, %eax 读到的值直接乘 n，循环没了
22      testl   %ecx, %ecx
23      cmovle  %edx, %eax
24      ret
25      .seh_endproc
```

19 只读一次
21 读到的值直接乘 n，循环没了

asm examples/ch08/volatile_loop/volatile_loop-O2.s 第 30–45 行

```
30  sum_volatile:
31      .seh_endprologue
32      testl   %ecx, %ecx
33      jle .L9
34      xorl    %eax, %eax
35      xorl    %edx, %edx
36      .p2align 4
37      .p2align 4
38      .p2align 3
39  .L8:
40      addl    $1, %eax
41      addl    vg(%rip), %edx 每一轮都真的读 vg
42      cmpl    %eax, %ecx
43      jne .L8 循环还在，跳回去
44      movl    %edx, %eax
45      ret
```

41 每一轮都真的读 vg
43 循环还在，跳回去

`sum_plain` 里根本没有循环：`movl g(%rip), %eax` 读一次，`imull %ecx, %eax` 乘 n。`sum_volatile` 里循环原样保留，每一轮都执行一次 `addl vg(%rip), %edx`——`volatile` 的全部魔力就在这：**访问必须真的发生，次数不能改，顺序不能挪。**这两份汇编的抓取命令在 `session-asm.txt` 里，你可以自己重跑。

优化档位怎么选，以及两个「收回许可」的开关

`-O0` 不优化（本书所有例子的基础档位）、`-O1` 开一批便宜又安全的改写、`-O2` 是发行版的常规选择、`-O3` 再加激进的向量化与展开、`-Os` 以体积优先。这是**工程惯例**，标准里没有 `-O` 这个东西。但「越高越快、`-Os` 越小」是别人的经验，不是你机器上的事实，所以本书让 `opt_levels.c` 自己量一遍：一段一亿次的一亿次求和，五个档位各编一次。

终端会话 五个档位：目标文件大小与一次运行的耗时 ch08-opt-levels

```
$ cd ub_oob_read && cd ..                          # 在 examples/ch08 里做这次对照
$ for O in -O0 -O1 -O2 -O3 -Os; do gcc -std=c17 $O -c opt_levels.c -o ol$O.o 2>/dev/null; done
$ stat -c "%n %s 字节" ol-O0.o ol-O1.o ol-O2.o ol-O3.o ol-Os.o
ol-O0.o 930 字节
ol-O1.o 894 字节
ol-O2.o 1042 字节
ol-O3.o 1042 字节
ol-Os.o 1042 字节
$ for O in -O0 -O1 -O2 -O3 -Os; do gcc -std=c17 $O opt_levels.c -o ol$O.exe 2>/dev/null; done
$ for O in -O0 -O1 -O2 -O3 -Os; do s=$(date +%s%N); ./ol$O.exe > /dev/null; e=$(date +%s%N); echo "$O $(( (e-s)/1000000 )) ms"; done
-O0 162 ms
-O1 109 ms
-O2 93 ms
-O3 76 ms
-Os 79 ms
```

读这张表要小心两件事。第一，**目标文件大小在这里几乎不区分档位**（894 到 1042 字节）：这个循环太小，`-Os` 省不出东西——「`-Os` 一定更小」是错觉，它的收益要看代码规模。第二，**耗时每一次都不同**（这几行数是某一次运行，换一次机器、换一次后台负载都会变），能看的是量级：`-O0` 大约是 `-O2` 的两倍，`-O1` 已经拿到大部分好处，`-O3` 与 `-Os` 在这段代码上没有进一步差别（串行求和没法向量化）。这就是取舍的实际含义：先量，再选。

本书自己的规矩是：讲解用 `-O0`（调试器里的行号对得上）、证明「这里对优化敏感」时只对照 `-O0` 与 `-O2`（也就是清单里 `detect: "opt-diff"` 的两档），`-O3` 不进教学基线。

`-fwrapv` 的代价要说清楚：它把有符号溢出定义成补码回绕，于是编译器**不能再假设「x + 1 > x」恒真**，也不能再利用「循环变量不会为负」——一些优化会因此放弃。你换来的是「溢出时行为可预测」，代价是「同一段代码可能慢一点」。`-fno-strict-aliasing` 的代价同理：它让编译器不敢再做基于「两个不同类型的指针不会指向同一块内存」的优化，换来的是那些类型双关的老代码不会突然算错。这两个开关都是**给旧代码续命**用的，不是新代码的首选；新代码该做的是把 UB 去掉，而不是把优化关掉。

8.5 本书出现过的 UB 清单

这一节把全书前面出现过、后面还会出现的 UB 集中成一张表。**每一行都要能指到一个现象**，否则它就只是一句「注意不要」——那种句子读者记不住。

#	UB	本单元的证据（在哪看）
1	数组越界读 / 写	`ub_oob_read`：`-O0` 读到 6291120 一类邻居值，`-O2` 折成常数；写的那一半是 `oob_fanalyzer_*`
2	空指针解引用	`crash_stdout`：退出码 3221225477
3	读未初始化的对象	`ub_uninit`：`-O0` 读到 32763，`-O2` 读到 0
4	有符号整数溢出	`signed_overflow_loop`：`-O0` 退出 0，`-O2` 3221225477；`overflow_trapv`：3221226505
5	除零（以及 `INT_MIN / -1`）	`ub_div_zero`：3221225620；`overflow_div.c`：3221225621
6	`free` 之后读或写	`heap_uaf`：`-O0` 读到 7，`-O2` 读到分配器里的字节（每次还不一样）
7	双重释放（double free）	第 4 章的 `double_free.c` 探针（崩溃会话）与第 7 章的 `double_free_debug` 例子
8	违反 `restrict` 的约定	`ub_memcpy_overlap`：重叠的两段交给 `memcpy`，`-Wrestrict` 警告
9	源与目标重叠的 `memcpy`	同上一行（`restrict` 是它的类型层依据）
10	格式串与参数类型不匹配	第 2 单元 2.4 的 `-Wformat=` 警告（`warn_zoo.c:40`：`%d` 配了 `double`）
11	非 `main` 函数走到结尾却没有 `return`	第 7 单元 7.5：`missing_return(7)` 走的那条路没写 `return`，读回来是 23

#	UB	本单元的证据（在哪看）
12	数据竞争（两个线程无同步地访问同一对象）	本书不讲线程，但条款在：§5.1.2.4 第 25 段把数据竞争定义为 UB

这十二项是本册**正文里出现过或明确提过**的全部，不是标准里 UB 的总清单——标准列出的种类远不止这些（还有变长数组长度非正、`longjmp` 跳到已经退出的函数里、修改字符串字面量等等），本书没有用到它们，所以不在这里凑数。前十一项各自能指到一段正文或一个例子；第 12 项本书没有线程的例子，只能指到条款，所以它的「证据」一栏写的是节号。表格的用法不是背，而是**对照**：你写下一行指针运算、一次 `free`、一个 `+` 的时候，先在表里找一下有没有对应项。第 1 到第 6 项在本单元都有真跑出来的读数，第 7、10、11 项在各自那一章有证据；第 12 项只有条款——这也是它被放在最后一行的原因：**讲不出证据的东西，至少要标明出处。**

标准原文 · 标准原文：有符号溢出为什么是未定义

6.5 节第 5 段：

> If an exceptional condition occurs during the evaluation of an expression (that is, if the result is not mathematically defined or not in the range of representable values for its type), the behavior is undefined.

大意：表达式求值如果出现「结果不在这个类型能表示的范围内」这种事，行为就是未定义的。注意它没有说「实现可以自己决定怎么办」——那是「实现定义」的待遇；这里说的是「标准不管」。**无符号**整数是例外：6.2.5 节第 9 段规定它按模 2^N 回绕，所以 `unsigned` 的溢出是定义良好的。

出处：ISO/IEC 9899:2018（C17）§6.5 第 5 段、§6.2.5 第 9 段。

8.6 越界读与未初始化读：把「读到什么」摆到桌面上

这两件事经常被打包成一句「会读到垃圾」。垃圾也分两种：一种只在越界时出现，一种在你根本没越界时就出现（没赋初值）。先看**越界读。**`ub_oob_read` 里有两个函数，一个读局部数组的第 4 格，一个读静态数组的第 4、5 格：

c examples/ch08/ub_oob_read/src/ub_oob_read.c 第 8–19 行

```c
void neighbor_read(void) {
    int a[3] = {10, 20, 30};
    int b[3] = {1, 2, 3};

    printf("a[3] (越界读)     = %d\n", a[3]); 下标 3 已经出界
    printf("b[0] (合法读)     = %d\n", b[0]);
}

void static_read(void) {
    printf("table[3] (越界读)  = %d\n", table[3]);
    printf("table[4] (越界读)  = %d\n", table[4]); table 只有 3 个元素
}
```

12 下标 3 已经出界

18 table 只有 3 个元素

终端会话 越界读：同一份源码，两档、两套读数 ch08-oob-read

```
$ cd ub_oob_read                                    # 越界读：两档读数不同
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/ub_oob_read.c -o oob_O0.exe
2>/dev/null
$ ./oob_O0.exe
a[3] (越界读)     = 6291120
b[0] (合法读)     = 1
----
table[3] (越界读)  = 0
table[4] (越界读)  = 942951440
程序正常走完，退出码 0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/ub_oob_read.c -o oob_O2.exe
2>/dev/null
$ ./oob_O2.exe
a[3] (越界读)     = -2042728384
b[0] (合法读)     = 1
----
table[3] (越界读)  = 0
table[4] (越界读)  = 0
程序正常走完，退出码 0
```

读这份会话要抓住三件事：

1. `b[0]` 是唯一的**合法读**，两档都是 `1`；`table[3]` 虽然两档也都是 `0`，但它自己就是越界读（下标 3 超出 3 个元素），只是这一次读到的字节恰好是 0。合法读稳定，说明程序本身没有别的随机源；
2. `a[3]` 与 `table[4]` 两档都不同，而且 `table[4]` 在 `-O2` 下变成了 `0`。`-O2` 并没有「读到 0」，它是把这次越界读**折成了一个常数**（编译器看见数组大小，就按自己的理解填了一个值）；
3. `6291120` 这个数我连着跑过很多次都没变，但 `table[4]` 每次重新编译都可能换一个——同一份代码、同一个档位，我先后抓到过 `942951440`、`961367056`，闸门的日志里还出现过 `1894730768`。**别背这些数**，它们是「这一次的布局」的产物。

这也就解释了为什么这个例子的清单只有一份 `detect: "opt-diff"`，而没有 `mode: "run"` 的输出块：程序确实打印得出东西，但打印的内容不是稳定的，拿它当「标准答案」会对不上账。闸门能对账的只是「两档不同」这件事本身。

未初始化读是另一种垃圾，而且它**不需要越界**：

c examples/ch08/ub_uninit/src/ub_uninit.c 第 9–20 行

```c
int uninit_local(void) {
    int x;                      /* 没有初值 */ 只有声明，没有初值
    return x;                   /* 读它：未定义行为 */ 读它
}

int uninit_member(void) {
    struct point p;
    p.x = 1;
    p.y = 2;
    /* p.z 从没赋值 */
    return p.z;                 /* 读它：未定义行为 */ 结构体里没写过的成员
}
```

10 只有声明，没有初值
11 读它
19 结构体里没写过的成员

终端会话 未初始化读：两档给出两个不同的数 ch08-uninit

```
$ cd ub_uninit                                          # 未初始化读：两档读数不同
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/ub_uninit.c -o uninit_O0.exe
2>/dev/null
$ ./uninit_O0.exe
未初始化的局部变量 = 32763
结构体没写到的成员 = 0
程序正常走完（读到什么不影响退出码）
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/ub_uninit.c -o uninit_O2.exe
2>/dev/null
$ ./uninit_O2.exe
未初始化的局部变量 = 0
结构体没写到的成员 = 0
程序正常走完（读到什么不影响退出码）
```

`32763` 与 `0` 的差别不是「编译器算错了」，而是两档对「这块内存里有什么」采取了不同的态度：`-O0` 真的去读栈上那一格（那里残留着上一次调用留下的字节），`-O2` 干脆把它当成 0（它有权这么做，因为读未初始化对象是 UB）。结构体那个成员两档都是 0，也只是因为那一块栈恰好是 0——换一次调用、换一个函数，它就可能变。

这两件事画出来是同一个问题的两张脸：**一块内存里有什么，不由你的声明决定，而由谁最后一次写过它决定。**第 1 帧与第 2 帧是未初始化与「用过又还回去」的垃圾；第 3、4 帧换成严格别名：同一块四个字节，换一种类型去读，得到的是完全不同的数。

内存形态 同一块字节的两种解释：垃圾与类型双关

第 1 步 没写过的局部变量

x ：栈上一格，从没被写过

x → 32763 -O0 真去读，读到的是残留字节

第 2 步 free 之后又被复用

p ：指向已经还回去的 4 个字节　　q ：同尺寸分配，极可能拿到同一块

q → 7 把 7 写进这一块

p → 7/-O2 读到垃圾 旧指针 p 读同一块内存

第 3 步 1.5f 的四个字节

f ：float，4 个字节

f → 00 00 c0 3f 小端，按字节看是 0x3FC00000

第 4 步 两种解释

bits_bad → 1069547520 按 int 读同一块字节

bits_good → 1069547520 memcpy 拷出来再按 int 读

第 1 帧的数字来自 `ub_uninit` 的 `-O0` 读数，第 2 帧来自 `heap_uaf`（`-O0` 读到 7，`-O2` 读到分配器里的字节），第 3、4 帧是 8.7 节的主角：`*(int *)&f` 与 `memcpy`。前两帧的结论是「别读你没写过、或者已经还回去的内存」；后两帧的结论更微妙：**即使内存是合法的、内容是确定的，换一种类型去读它也可能不合法**——那就是严格别名。

8.7 严格别名与重叠内存：两件「跑不出问题」的事

这一节的两个例子在本机跑起来**一点问题都没有**——它们存在的意义，恰恰是让你知道「跑起来没问题」这句话有多不值钱。

第一件：把一个 `float` 的字节当 `int` 读。两种写法，一种违规一种合规：

c examples/ch08/ub_strict_alias/src/ub_strict_alias.c 第 5–13 行

```
5  int bits_bad(float f) {
6      return *(int *)&f;              /* 把 float 的字节当成 int 读：UB */  类型双关：违反严格别名
7  }
8
```

```c
int bits_good(float f) {
    int bits;
    memcpy(&bits, &f, sizeof bits);     /* 合法：按字节复制，不涉及类型别名 */ memcpy 按字节搬，合法
    return bits;
}
```

6 类型双关：违反严格别名
11 memcpy 按字节搬，合法

c examples/ch08/ub_strict_alias/src/ub_strict_alias.c 第 15–22 行

```c
int bits_union(float f) {
    union {
        float f;
        int i;
    } u;
    u.f = f;
    return u.i;                         /* C 允许读另一个成员；结果依赖布局与字节序 */ C 允许读另一个成员，但仍依赖布局与字节序
}
```

21 C 允许读另一个成员，但仍依赖布局与字节序

终端输出 终端输出 strict-alias-out

```
f = 1.500000
坏写法 *(int *)&f    = 1069547520
好写法 memcpy 之后    = 1069547520
另一种 union 之后     = 1069547520
```

同一个数：`1069547520` 就是 `1.5f` 那四个字节按 `int` 读出来的值（十六进制 `0x3FC00000`，见 8.6 第 3 帧）。**坏写法在这个程序里没有任何症状**——两档、多次运行都是这个数。证据不在运行结果里，在编译器的警告里：

终端会话 -O2 才报：type-punned pointer 会破坏严格别名 ch08-strict-alias-warn

```
$ cd ub_strict_alias                                  # 严格别名：-O2 才报的那句警告
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -c -Iinclude src/ub_strict_alias.c -o strict_alias.o 2>&1
（-O0 下没有输出：这句警告只在优化时才报）
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -c -Iinclude src/ub_strict_alias.c -o strict_alias.o 2>&1
src/ub_strict_alias.c: In function 'bits_bad':
src/ub_strict_alias.c:6:13: warning: dereferencing type-punned pointer will break strict-aliasing rules [-
Wstrict-aliasing]
    6 |     return *(int *)&f;                /* 把 float 的字节当成 int 读：UB */
      |            ^~~~~~~~
```

注意这条警告**只在 `-O2` 出现**，`-O0` 下一次都没有。这不是警告不稳定，而是严格别名本身就是一条给优化器用的规则：`-O0` 不做基于类型的优化，所以它「不关心」；`-O2` 一旦打算把 `f` 缓存在寄存器里，就得按这条规则决定「那个 `int` 指针能不能改到 `f`」。

标准原文 · 标准原文：严格别名规则

6.5 节第 7 段（俗称「严格别名规则」）：

> An object shall have its stored value accessed only by an lvalue expression that has one of the following types: a type compatible with the effective type of the object, a qualified version of a type compatible with the effective type of the object, a type that is the signed or unsigned type corresponding to the effective type of the object, a type that is the signed or unsigned type corresponding to a qualified version of the effective type of the object, an aggregate or union type that includes one of the aforementioned types among its members …, or a character type.

大意：一个对象的**存储值**只能通过这几类左值去访问：与它的有效类型兼容的类型、加了限定符的版本、对应的有/无符号类型、包含上述类型的聚合或联合，或者**字符类型**（列举有删节）。`float` 的对象用 `int` 左值去访问，不在这张名单里——所以 `*(int *)&f` 是 UB；而 `memcpy` 是按**字符类型**（`unsigned char` / `char`）逐字节搬，落在最后一项的许可里。

出处：ISO/IEC 9899:2018（C17）§6.5 第 7 段。

上面第三段就是合规的两种替代，本机跑出来都是同一个数 `1069547520`：

1. **`memcpy`**（首选）：`memcpy(&bits, &f, sizeof bits)`。它明确「按字节复制」，编译器认得这个模式，常常直接优化成一条寄存器搬移，没有任何运行期开销；结果只取决于字节，不取决于对齐与别名。
2. **`union`**（C 里可用，C++ 里不行）：把 `float` 与 `int` 放进同一个 `union`，写一个成员、读另一个。C 标准允许这样读（6.5.2.3 节：用非当前成员读时，那段对象表示按新类型重新解释），但它仍然依赖**字节序**与布局，而且同一份代码拿给 C++ 编译器就是 UB。本书的推荐是：「取字节」用 `memcpy`，「同一块内存的多种解释」用 `union` 只在 C 里、而且清楚自己在做什么的时候。

`restrict` 与 `const`、`inline` 常被一起问起，这里一次说清它们的**语义层次**：

关键字	它到底是什么	误用长什么样
`const`	「通过这个名字不可写」的编译期约定	以为 `const` 对象真的不会被改。别处拿非 `const` 指针去改它是 **UB**（6.7.3 节第 6 段），不是「能改」；放在只读段的还会当场崩
`restrict`	你对编译器的**承诺**：这块内存不会被别的途径访问	两个 `restrict` 指针指向重叠区域 → UB（`memcpy` 的声明就是这样，所以下面那个重叠搬移的例子违规）
`inline`	展开的**建议**，标准明确说它不保证	以为加了 `inline` 一定更快；或者以为不加就一定慢

三者都不是「性能开关」，而是**合同**：`const` 是对读代码的人的承诺，`restrict` 是对优化器的承诺，`inline` 只是提个建议。写下去就要守，尤其是 `restrict`——违背它不会有任何警告，只会得到一段「在 `-O2` 下算错」的代码。

第二件更隐蔽：把重叠的两段内存交给 `memcpy`。

c examples/ch08/ub_memcpy_overlap/src/ub_memcpy_overlap.c 第 5–11 行

```c
void shift_with_memcpy(char *buf) {
    memcpy(buf + 2, buf, 8);          /* 源与目标重叠：UB */ 源与目标重叠，UB
}

void shift_with_memmove(char *buf) {
    memmove(buf + 2, buf, 8);         /* 重叠时该用它 */ memmove 就是为这个准备的
}
```

6 源与目标重叠，UB
10 memmove 就是为这个准备的

c examples/ch08/ub_memcpy_overlap/src/ub_memcpy_overlap.c 第 13–17 行

```c
void shift_forward(char *buf) {
    for (int i = 0; i < 8; i++) {
        buf[i + 2] = buf[i];          /* 前向逐字节：先写的字节会把后读的源踩掉 */ 前向逐字节：先写的字节把后读的源踩掉
    }
}
```

15 前向逐字节：先写的字节把后读的源踩掉

终端会话 三种搬法：memcpy 与 memmove 相同，前向手写就不同 ch08-overlap

```
$ cd ub_memcpy_overlap                              # 重叠 memcpy：-Wall 就带着 -Wrestrict
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/ub_memcpy_overlap.c -o overlap.exe
2>&1
src/ub_memcpy_overlap.c: In function 'shift_with_memcpy':
src/ub_memcpy_overlap.c:6:5: warning: 'memcpy' accessing 8 bytes at offsets 2 and 0 overlaps 6 bytes at
offset 2 [-Wrestrict]
    6 |     memcpy(buf + 2, buf, 8);           /* 源与目标重叠：UB */
      |     ^~~~~~~~~~~~~~~~~~~~~~
$ ./overlap.exe
memcpy     之后 a = [0101234567]
memmove    之后 b = [0101234567]
前向逐字节 之后 c = [0101010101]
memcpy 与 memmove 相同？ 相同
```

终端输出 终端输出 memcpy-overlap-out

```
memcpy     之后 a = [0101234567]
memmove    之后 b = [0101234567]
前向逐字节 之后 c = [0101010101]
memcpy 与 memmove 相同？ 相同
```

`memcpy` 与 `memmove` 的结果**逐字节相同**，`memcmp` 也确认了；而同一件事手写成「前向逐字节」就得到 **`0101010101`** ——完全不同的结果。这正是它危险的地方：**重叠时结果取决于复制方向**，前向会自我覆盖（`buf[2] = buf[0]` 之后，读 `buf[2]` 拿到的已经是新值），后向不会。本机的 `memcpy` 这一次表现得像后向（或整块）复制，所以碰巧对了；换一个 libc、换一个长度、换一个优化档，它完全可能变成前向，结果就变成 `0101010101`。标准允许它怎么选，因为标准根本不许你传重叠的区间。证据是编译器那句 `-Wrestrict`（`-Wall` 就带着它，`-O0` 也报）与标准：

标准原文 · **标准原文：memcpy 要求不重叠**

7.24.2.1 节「The memcpy function」：

> The memcpy function copies n characters from the object pointed to by s2 into the object pointed to by s1. If copying takes place between objects that overlap, the behavior is undefined.

大意：`memcpy` 从 `s2` 指向的对象往 `s1` 指向的对象搬 n 个字符；**如果两个对象重叠，行为未定义。**7.24.2.2 节的 `memmove` 则明确写成「先把 n 个字符从 `s2` 复制到一个不重叠的临时数组，再从临时数组复制到 `s1`」——它就是为了重叠而生的。`memcpy` 声明里那两个 `restrict` 就是这条规则的**类型层**表述。

出处：ISO/IEC 9899:2018（C17）§7.24.2.1、§7.24.2.2。

8.8 `volatile`：它保证一件小事，不保证三件大事

`volatile` 被误解的程度比 UB 本身还高。它只保证一件事：**对这个对象的每一次访问都真的发生，编译器不许合并、不许删除、不许挪到别处**；它不保证原子性、顺序与可见性。先看保证的那一件，证据是汇编。

`volatile_asm` 把五个最小的函数摆在一起：写一次、读一次、加一次、自增并返回新值、自增丢弃返回值。它们的 `-O2` 汇编是这样：

asm examples/ch08/volatile_asm/volatile_asm-O2.s 第 7-28 行

```asm
store_v:
    .seh_endprologue
    movq    %rcx, g(%rip)   一次写就是一条 movq
    ret
    .seh_endproc
    .p2align 4
    .globl  load_v
    .def    load_v; .scl    2;  .type   32; .endef
    .seh_proc   load_v
load_v:
    .seh_endprologue
    movq    g(%rip), %rax   一次读就是一条 movq
    ret
    .seh_endproc
    .p2align 4
    .globl  add_v
    .def    add_v;  .scl    2;  .type   32; .endef
    .seh_proc   add_v
add_v:
    .seh_endprologue
    addq    %rcx, g(%rip)   读改写合成一条 addq
    ret
```

9 一次写就是一条 movq
18 一次读就是一条 movq
27 读改写合成一条 addq

asm examples/ch08/volatile_asm/volatile_asm-O2.s 第 34-49 行

```asm
bump_v:
    .seh_endprologue
    movq    g(%rip), %rax   先读
    addq    $1, %rax    再加
    movq    %rax, g(%rip)   再写回，三步
    ret
    .seh_endproc
    .p2align 4
    .globl  self_inc
    .def    self_inc;   .scl    2;  .type   32; .endef
    .seh_proc   self_inc
self_inc:
    .seh_endprologue
    addq    $1, g(%rip)   g++ 合成一条 addq
    ret
    .seh_endproc
```

36 先读
37 再加
38 再写回，三步
47 g++ 合成一条 addq

终端输出 终端输出 volatile-asm-out

```
store 之后 load      = 5
add(2) 之后 load     = 7
bump 的返回值        = 8
self_inc 之后 load   = 9
```

`bump_v` 与 `self_inc` 的差别本身也是一课：同样一句「加一」，需要返回值时是三条指令，不需要时是一条。三条指令的版本在另一个核看来，就是「读了、加了、还没写」——中间那个窗口里任何人读到 `g` 都是旧值；一条指令的版本在硬件层面也仍然是读改写，只是窗口更短，标准并没有因此把它变成原子操作。

读这份汇编，先确认「保证」：`store_v` 是一条 `movq %rcx, g(%rip)`，`load_v` 是一条 `movq g(%rip), %rax`——每一次 C 里的赋值/取值都对应一条真的内存访问，没有被优化掉。`bump_v` 是三句（读、加、写回），因为 `++g` 要返

回新值；`self_inc` 是**一条** `addq $1, g(%rip)`，因为 `g++` 不需要返回值，编译器就把读改写合成一条指令。这五个函数里没有一条 `lock`。

再看「不保证」的三件：

- **不保证原子**：`addq $1, g(%rip)` 是一条指令，但它是「读内存、加一、写回」三件事合成的一条——CPU 执行它的时候不是不可分割的，另一个核完全可能在这中间插进来。要原子，需要的是 `lock` 前缀或者 `stdatomic.h` 的原子类型，而 `volatile` 一个都不给。本机的 `volatile long long` 读/写恰好是单条 `movq`，那也是因为 8 字节是这台机器的自然宽度；换成 `volatile struct` 或不对齐的访问，一条 C 语句就会变成多条指令，撕裂（torn read/write）立刻可见。
- **不保证顺序**：编译器不会把两个 `volatile` 访问彼此调换，但它完全可以把**普通**访问搬到 `volatile` 访问的前后；CPU 也会按自己的规则重排普通读写。标准只要求「在序列点上 `volatile` 对象的值与抽象机一致」，没有给你内存屏障。
- **不保证可见性**：它不刷缓存、不插 `mfence`。上面那份汇编里 `grep -n "lock\|mfence\|sfence\|lfence\|xchg\|cmpxchg"` 一行都没有——这是 `session-asm.txt` 里真跑出来的结果，不是我的断言。

标准原文 · 标准原文：volatile 到底说了什么

6.7.3 节的类型限定符一节里，关于 `volatile` 的那一段说：

> An object that has volatile-qualified type may be modified in ways unknown to the implementation or have other unknown side effects. Therefore any expression referring to such an object shall be evaluated strictly according to the rules of the abstract machine, as described in 5.1.2.3. … What constitutes an access to an object that has volatile-qualified type is implementation-defined.

大意：`volatile` 对象可能被实现不知道的方式改动，所以引用它的表达式必须**严格按抽象机的规则**求值（也就是每一次该发生的访问都发生）；至于「什么算一次访问」，由实现定义。5.1.2.3 节把「访问 `volatile` 对象」列为**副作用**（side effect），并规定在序列点上这些副作用必须已经完成、后续的还没开始。

注意它没说什么：这两节一个字都没提多个线程、原子性、内存序。C 标准的线程与原子语义在 `stdatomic.h`（7.17 节）里，`volatile` 不在那套模型里。

出处：ISO/IEC 9899:2018（C17）§6.7.3、§5.1.2.3。

⚠ 陷阱 · `volatile` 当锁：你会看到「好像能用」

你这么写的：`volatile int lock = 0;`，加锁时 `while (lock) ; lock = 1;`，解锁时 `lock = 0;`。单线程测试完美通过，于是认为锁写好了。

你会怎么误判：这段代码的每一次读、每一次写都「真的发生了」（`volatile` 保证了这一点），汇编里也看得见，所以你找不到反例。真相是：`while (lock) ;` 与 `lock = 1;` 之间有一条缝，两个线程可以同时看到 `lock == 0`，然后都把它设成 1——**检查与设置不是一个原子动作。** `volatile` 只保证「每次都去读」，不保证「读到的值与随后的写之间有排他性」。锁需要的是原子读改写（`lock cmpxchg` 一类）加内存序，那是 `stdatomic.h` 或者 pthread 的领域。

单线程程序**不可能**演示出这个失败——这也是本单元只讲条款、不演示「非原子」的原因：拿一个单线程程序去「证明」它原子，本身就是一次错误的实验设计。

工程惯例 · **工程惯例：什么时候真的需要 volatile**

标准没有「要求」你在任何地方写 `volatile`——它只在一处**给了你一条出路**（信号），工程上另一处离不开它（MMIO）。其余用法基本都是误用：

- **内存映射 I/O（MMIO）**：硬件寄存器被映射到某个地址，读一次清一次标志、写一次发一次命令。编译器若把「连续读两次」合并成一次，硬件行为就变了——这里 `volatile` 是正解。代价是每次访问都真的访存，而且它仍然不保证原子（64 位寄存器在 32 位总线上可能要读两次）。
- **信号处理函数里与主流程共享的 `sig_atomic_t`**：7.14.1.1 节第 5 段给了一条出路——读写 `volatile sig_atomic_t` 不算 UB。`sig_atomic_t` 保证一次读写不会被信号打断成两半，`volatile` 保证编译器每次都去读。
- **误用的典型**：拿它当锁、拿它当「线程间通信的开关」、拿它当「禁止编译器乱序」的屏障。三样它都不做。要线程安全就上 `stdatomic.h` 或线程库；要屏障就用真正的原子操作与内存序。

一句话：`volatile` 是给**硬件**和**信号**用的，不是给线程用的。

8.9 检测手段：三种抓得住的，与一个抓不住的缺口

这一节把检测清单摊开：每种手段抓什么、抓不到什么、证据是退出码还是警告原文。

第一档：`-fanalyzer`（编译期，静态分析）。 它不运行程序，只在「编译器视角」里推演路径。同一个「越界写」，换个形状，它的表现完全不同：

终端会话 常量形状：容量与长度都写在纸面上，它抓到了 ch08-fanalyzer

```
$ cd oob_fanalyzer_const                              # 常量形状：分析器抓到了
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fanalyzer -Iinclude src/main.c src/oob_const.c -o const.exe
2>&1 | head -n 5
src/oob_const.c: In function 'overwrite_heap':
src/oob_const.c:12:5: warning: heap-based buffer overflow [CWE-122] [-Wanalyzer-out-of-bounds]
   12 |     memcpy(buf, "0123456789ABCDEF", 17);          /* 写 17 个字节：越界 9 个 */
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  'overwrite_heap': events 1-4
$ echo "编译退出码 ${PIPESTATUS[0]}"
编译退出码 0
```

终端会话 （续）循环形状：报的是空指针，真正越界写的一个字没提 ch08-fanalyzer-loop

```
$ cd oob_fanalyzer_loop                               # 循环形状：报的是别的，越界没提
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fanalyzer -Iinclude src/main.c src/oob_loop.c -o loop.exe
2>&1 | head -n 4
src/oob_loop.c: In function 'fill_slots':
src/oob_loop.c:10:14: warning: dereference of possibly-NULL 'p' [CWE-690] [-Wanalyzer-possible-null-
dereference]
   10 |         p[i] = i;                                 /* i == 4 时写到第 5 个位置：越界 */
      |         ~~~~~^~~
$ ./loop.exe
累加和 = 10
```

两份清单都在 `detect: "fanalyzer"` 下通过了：一份声明它必须报 `[-Wanalyzer-out-of-bounds]` 与 `CWE-122`，另一份声明它必须报 `[-Wanalyzer-possible-null-dereference]` 与 `CWE-690`。这两份证据说明的是同一件事的两面：

- **它能报**：`malloc(8)` 与 `memcpy(..., 17)` 这种「数字都写在纸面上」的形状，它算得出来，还会画出一条从分配、判空到越界的路径。
- **它会漏、也会报偏**：循环里 `i <= 4` 写第 5 个 `int` 是真的越界写（程序真跑起来也确实写了，`累加和 = 10` 就是证据，标准下这已经是 UB），它一个字没提；报的是「`p` 可能是空指针」——这次运行里 `p` 不是空指针，所以这是一条**这一次不成立的告警**，而真正发生的那件事不在报告里。

所以 `-fanalyzer` 的正确定位是「多一道筛查」，不是「证明没有 UB」：它给出的永远是「从编译器看到的这条路径上，可能……」。还要记住：**它没有整数溢出检查器**——`gcc -Q --help=warnings` 里名字带 analyzer 且与溢出/除法有关的开关只有 `-Wanalyzer-shift-count-overflow` 与 `-Wanalyzer-tainted-divisor` 两项，所以别指望它抓 `INT_MAX`

`+ 1`。留一条证据在这儿：对 `overflow_trapv` 那份源码（裸加法 `INT_MAX + 1`）跑 `-fanalyzer`，stderr 一行都没有，编译退出码 0。

终端会话 同一个分析器，对裸加法一个字都不说 ch08-fanalyzer-overflow

```
$ cd ../overflow_trapv                                  # 有符号溢出：分析器一个字都不说
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fanalyzer -Iinclude src/main.c src/overflow_trapv.c -o
ovf.exe 2>&1 | wc -l
0
$ echo "编译退出码 ${PIPESTATUS[0]}"
编译退出码 0
```

第二档：`-ftrapv`（运行期，只抓有符号整数溢出）。 `overflow_trapv` 这个例子只有一个裸加法，它的清单写的是 `detect: "trapv"`：

c examples/ch08/overflow_trapv/src/main.c 第 6–12 行

```c
int main(void) {
    int r = add_unguarded(INT_MAX, 1); 裸加法：溢出就是 UB
    printf("INT_MAX + 1 = %d\n", r);
    fflush(stdout);                    /* 加了它，输出也只留在正常那一档 */
    printf("这行只有在没被终止时才会打印\n");
    return 0;
}
```

7 裸加法：溢出就是 UB

闸门对这份清单做了两次运行，日志里写着：带 `-ftrapv` 那次**退出码 3221226505、stdout 为空**；去掉 `-ftrapv` 的阴性对照那次**退出码 0、打印了 `INT_MAX + 1 = -2147483648` 和另一行。** `-ftrapv` 只覆盖 `+`、`-`、`*` 三种运算，不管无符号回绕，也不管浮点；它是运行期检测，只有真的执行到那一步才会终止——这一点和 `-fanalyzer` 正好互补。

第三档：`-fstack-protector-all`（运行期，只抓栈上的越界写）。 它往函数的栈帧里放一个哨兵，函数返回时核对；不对就打印 `*** stack smashing detected ***: terminated` 并以 **3221226505** 退出。第 4 章的 `stack_smash` 就是它的靶子，这里不重复。

名字里的 `-all` 是有含义的：它保护**每一个函数**，包括没有数组的函数。用 `stack_probe.c`（只有一个 `int y = x + 1;`）实测：不加参数时汇编里一处 `__stack_chk` 都没有（0 处），加了 `-fstack-protector-all` 是 11 处。

终端会话 没有数组的函数：-all 也会插哨兵 ch08-stack-probe

```
$ cd ..                                                 # 栈保护：-all 与默认的差别
$ gcc -std=c17 -O0 -S stack_probe.c -o stack_probe_plain.s
$ gcc -std=c17 -O0 -fstack-protector-all -S stack_probe.c -o stack_probe_all.s
$ grep -c stack_chk stack_probe_plain.s stack_probe_all.s
stack_probe_plain.s:0
stack_probe_all.s:11
```

不带 `-all` 的 `-fstack-protector` 才只挑「有局部数组或 `alloca`」的函数——那是另一档开关，别把两者的能力混在一起。它的边界还是硬：只看得见**栈帧返回时**的破坏，堆越界、越界读、以及不经过栈帧返回的破坏，它都看不见。

缺口：本机没有 ASan / UBSan。 把四种手段放在一张表里，边界就清楚了：

手段	阶段	抓什么	抓不到什么	证据形态
`-fanalyzer`	编译期	能算出来的越界、可疑的空指针路径	循环里的越界写（本机报偏）、整数溢出（没有检查器）、只有运行期才知道的输入	警告原文，编译退出码仍是 0
`-ftrapv`	运行期	`+` `-` `*` 的有符号溢出	无符号回绕、浮点、越界、泄漏	进程被终止，退出码 3221226505

手段	阶段	抓什么	抓不到什么	证据形态
`-fstack-protector-all`	运行期	每个函数栈帧里的越界写（`-all` 就是所有函数）	堆越界、越界读、不经过栈帧返回的破坏	stderr 的 `*** stack smashing detected ***` + 3221226505
`opt-diff`	对比	「这里对优化敏感」	**具体是哪一类 UB**	两档的 stdout 或退出码不同
ASan / UBSan	运行期	本机**没有**（链接失败）	全部	无

这张表最下面一行是实测而不是选择：`-fsanitize=address,undefined` 能编译但链接失败（`collect2.exe: error: ld returned 5`，安装里没有 `libasan` / `libubsan`），本机也没有 clang。这意味着：

- 你**拿不到**「越界发生在第几行、越界了几个字节」这种运行期精确报告；
- 你能拿到的是**退出码 + stderr + 汇编 + 分析器警告**这四类证据，本单元每一处「实测」都标明用的是哪一类；
- 没有检查器不代表没有 UB。8.5 那张表里，格式串错配、缺 `return`、数据竞争，本机**一个检查器都没有**——只能靠代码审查与测试。

这一节的读法也在这儿：**本单元的四份 opt-diff 清单**是第四类证据，它不指出 UB 的种类，只证明「这里的行为对优化档位敏感」（四份清单的 `expect.differs` 依次是 `stdout`、`stdout`、`stdout`、`exit`）。四个形状都真跑过：

- `ub_oob_read`：两档的 6 行输出在 `a[3]` 与 `table[4]` 两处不同（`table[4]` 在 `-O2` 下变成 0）；
- `ub_uninit`：`-O0` 读到 `32763`，`-O2` 读到 `0`；
- `heap_uaf`：`-O0` 读到 `7`，`-O2` 读到分配器里的字节（每次都不一样）；
- `signed_overflow_loop`：`-O0` 退出 0，`-O2` 退出 3221225477。

为什么这里不列具体读数：这四处里只有 `uninit` 与 `overflow_loop` 的读数是稳定的；`oob_read` 的越界值和 `heap_uaf` 的 `-O2` 值每次编译、每次运行都可能换一个（8.6 已经说过）。把会变的数写进正文，读者一跑就对不上——这正是 UB 的脾气，也提醒你：**opt-diff 的证据是「两档不同」，不是某个具体的数。**

8.10 把 UB 变成可观测现象

UB 最要命的地方是它可能「看起来正常」。所以真正工程化的做法不是「避免写出 UB」（那是目标），而是**给每一处可疑的地方装一个观测点**。本单元用到的四种做法，按成本从低到高：

1. **前置条件 + 断言**：函数入口就检查参数，`assert(p != NULL);`、`assert(i >= 0 && i < n);`。`assert` 在 `NDEBUG` 定义时会被整个去掉，所以它只适合「不该发生」的内部不变量；对外部输入要用返回错误码。
2. **把检查写进算式**：`overflow_guard` 用 `__builtin_add_overflow` 把「会不会溢出」变成一个可以问的布尔值，而不是一个「算完看结果对不对」的赌博。它真跑的输出在下面。
3. **边界测试**：对每个数组，测 `0`、`n-1`、`n` 三个下标；对每个整数，测 `INT_MAX`、`INT_MIN`、`0`。边界测试不是「多测几个数」，而是把「刚好在边界上」的那一格单独造出来：本单元两个越界形态都是**差一格**——`a[3]`（`a` 有 3 个元素）、`i <= 4` 配 `malloc(4 * sizeof *p)`。`off-by-one` 之所以是最高频的越界，就是因为它在 `for (i = 0; i < n; i++)` 这种最常见的循环里永远不会暴露。
4. **保住崩溃前的输出**：这一条最容易被忽略，也最容易救你一次调试。

先说第 2 条，因为它把「UB」变成了「错误码」：

c examples/ch08/overflow_guard/src/overflow_guard.c 第 5-16 行

```c
int add_checked(int a, int b, int *out) {
    int sum;
    if (__builtin_add_overflow(a, b, &sum)) {   /* 溢出：不写结果 */ 先算再问，溢出就不写结果
        return -1;
    }
    *out = sum;
    return 0;
```

```
12  }
13
14  int add_naive(int a, int b) {
15      return a + b;                   /* 溢出就是 UB，不报错 */ 裸加法，溢出就是 UB
16  }
```

7 先算再问，溢出就不写结果
15 裸加法，溢出就是 UB

终端输出 终端输出 overflow-guard-out

```
3 + 4        -> 没溢出，结果 7
INT_MAX + 1  -> 溢出，结果没有被写进去，还是 7
INT_MIN - 1  -> 溢出，结果还是 7
裸加法 INT_MAX + 1 = -2147483648（这次是 -2147483648，但这是运气）
```

前三行是检查函数给出的三个答案，第四行是裸加法给出的「答案」——`-2147483648` 这个数**没有意义**，它只是在 `-O0` 这一档、这一次运行里恰好是这样。同一个道理，`INT_MIN` 自己也有两条围绕它的 UB，这是从整数边界那一节接过来的：

终端会话 INT_MIN 的取负与除法：一个算出来是负数，一个被终止 ch08-tmin

```
$ cd twos_complement_ub                          # INT_MIN 取负，与 INT_MIN / -1
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/twos_complement_ub.c -o tmin.exe
2>/dev/null
$ ./tmin.exe
-1          =          -1  字节: ff ff ff ff
INT_MAX     =  2147483647  字节: ff ff ff 7f
INT_MIN     = -2147483648  字节: 00 00 00 80
----
-INT_MIN        = -2147483648
abs(INT_MIN)    = -2147483648  (比 0 还小)
INT_MIN + INT_MIN = 0
$ ./overflow_div.exe -1 ; echo "退出码 $?"
准备算 INT_MIN / -1
退出码 127
```

终端输出 终端输出 twos-complement-out

```
-1          =          -1  字节: ff ff ff ff
INT_MAX     =  2147483647  字节: ff ff ff 7f
INT_MIN     = -2147483648  字节: 00 00 00 80
----
-INT_MIN        = -2147483648
abs(INT_MIN)    = -2147483648  (比 0 还小)
INT_MIN + INT_MIN = 0
```

`-INT_MIN` 印出来还是 `-2147483648`，`abs(INT_MIN)` 也是——因为 `INT_MIN` 的相反数在这个类型里根本不存在。这一条**不会崩**，所以比崩溃更难发现：`abs()` 返回负数，下游的比较与索引就全乱了。而 `INT_MIN / -1` 会真的被终止（会话里 bash 报 127，闸门记录的 Windows 退出码是 **3221225621**，0xC0000095 整数溢出），和除零的 **3221225620**（0xC0000094）只差一个编号。

再说第 4 条，也就是 `fflush`。同一个空指针写，两次运行，唯一的区别是有没有在崩溃前刷缓冲：

c examples/ch08/crash_stdout/src/crash_stdout.c 第 5–15 行

```
5   void print_then_crash(int flush_first) {
6       printf("这一行是在崩溃之前打印的\n"); 这行只是进了缓冲区
7       if (flush_first) {
8           fflush(stdout);             /* 把缓冲区里的字节先推出去 */ 刷出去就真的是「写出去了」
9       }
10
11      int *p = NULL;
12      *p = 1;                         /* 空指针写：UB，本机退出码 3221225477 */ 空指针写：UB
13
14      printf("走不到这一行\n");
```

```
15  }
```

```
 6 这行只是进了缓冲区
 8 刷出去就真的是「写出去了」
12 空指针写：UB
```

终端输出 终端输出 crash-noflush-out

```
（无输出）
```

终端输出 终端输出 crash-flush-out

```
这一行是在崩溃之前打印的
```

终端会话 同一个崩溃，两次运行：没刷的 stdout 是空的 ch08-crash

```
$ cd crash_stdout                                  # 同样崩溃两次：一次没刷新，一次刷新了
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/crash_stdout.c -o crash_plain.exe
2>/dev/null
$ ./crash_plain.exe ; echo "退出码 $?"
退出码 139
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DCH08_FLUSH src/main.c src/crash_stdout.c -o
crash_flush.exe 2>/dev/null
$ ./crash_flush.exe ; echo "退出码 $?"
这一行是在崩溃之前打印的
退出码 139
```

两次的退出码一样（bash 报 139，闸门记录 3221225477），但第一个程序的 stdout **一行都没有**——`printf` 只是把字放进了缓冲区，进程被终止时缓冲区跟着没了。第二个程序在崩溃前 `fflush(stdout)`，那一行才真的写到管道里。两份清单分别把 stdout 记成「（无输出）」与那一行字，闸门逐字对账。同一个道理也解释了 `ub_div_zero`：它先打印 `准备算 100 / 0`，紧接着 `fflush`，所以除零（退出码 3221225620）之后你还能看到这一行：

终端输出 终端输出 div-zero-out

```
准备算 100 / 0
```

工程惯例 · **工程惯例：调试崩溃时的三条做法**

这不是标准，是常年踩坑攒下来的：

- **崩溃点附近若已经打印过东西，先 `fflush(stdout)` 或改打 `stderr`**（`stderr` 默认无缓冲）。否则你会看到「程序什么都没打印就崩了」，然后去错误的地方找原因。
- **断言写内部不变量，错误码写外部输入。** `assert` 会被 `NDEBUG` 整段去掉，拿它校验用户输入，发布版里就什么都不剩了。
- **把「这次没事」与「永远没事」分开记。** `add_naive(INT_MAX, 1)` 这次打印 `-2147483648`，记成「本次实测」；不要记成「溢出会回绕」。后者需要 `-fwrapv` 或 `unsigned` 才成立。

代价：多写几行检查、多刷几次缓冲、多记几行日志。换来的是崩溃时你手里有一条线索，而不是一个退出码。

8.11 可移植性守则，以及这一节留下了什么

这一节的所有结论都指向一句：**凡是标准没钉死的东西，都不要写进「这就是 C」的认知里。**下面五条是本书反复强调的，本机读数都在 `session-impl-defined.txt` 里：

别依赖	本机读数	为什么
结构体的布局与大小	`sizeof(struct pair)` = 8（1 字节的 `char` + 4 字节的 `int`，中间补了 3 个字节），`offsetof(pair, i)` = 4	对齐规则由实现定；成员的顺序虽被标准固定，中间填充不固定
字节序	`0x01020304` 的四个字节是 `04 03 02 01`（低地址放低位，小端）	大端机器上反过来
指针大小	`sizeof(void *)` = 8	32 位机器上是 4

别依赖	本机读数	为什么
位域的顺序与宽度	本书在整数那一章讲过它不可移植；这一条没有单独做本机探针	位的分配顺序、跨存储单元的规则都由实现定
`int` / `long` 的宽度	`sizeof(int)` = 4，`sizeof(long)` = 4，`LONG_MAX` = 2147483647	`long` 在本机和 `int` 一样宽；想要 64 位用 `long long` 或 `int64_t`

第五行最容易骗人：不少教材写「`long` 是 64 位」，本机不是。你把「用 `long` 存 64 位」写进代码，在 Windows 上就是 4 个字节。下一单元 `c20` 的 `portability_probe` 会把这一组事实系统打印一遍（`CHAR_BIT`、`char/short/int/long/long long/void */size_t` 的 `sizeof`、`INT_MAX` / `LONG_MAX` / `LONG_MIN`、字节序）；定宽类型的宽度、位域与结构体布局**不在**那个探针里，要查就照本节 `impl_defined.c` 的写法自己加两行。这里只给出方向：**要宽度就用定宽类型，要布局就用序列化，要字节序就显式判断。**

这一节新增了什么

回到 8.1 那张图的第 9 帧，十六个例子目录到齐了；本节另外留下的附件是下面这张图：

目录树 🗀 **这一节的附件：会话、探针与汇编**

第 1 步 三份会话，覆盖 opt-diff、四种编法与汇编抓取

```
examples/ch08/
├── 十六个例子目录 ：各带清单 + src/ + include/
├── session-opt-diff.txt ：四个形状的两档读数 ← 本课新增
├── session-flags.txt ：-O0 / -O2 / -fwrapv / -ftrapv ← 本课新增
└── session-asm.txt ：汇编抓取、栈保护与 lock 的 grep ← 本课新增
```

第 2 步 另外三份会话，是警告、分析器与崩溃现场

```
examples/ch08/
├── session-warnings.txt ：-Wstrict-aliasing 与 -Wrestrict ← 本课新增
├── session-fanalyzer.txt ：分析器的三条原文 ← 本课新增
└── session-crash.txt ：崩溃、fflush 与 INT_MIN / -1 ← 本课新增
```

第 3 步 实现事实与优化档位，各带一个探针

```
examples/ch08/
├── session-impl-defined.txt ：实现决定的读数 ← 本课新增
├── impl_defined.c ：那份会话的探针（没有清单） ← 本课新增
├── session-opt-levels.txt ：五个档位的大小与耗时 ← 本课新增
└── opt_levels.c ：那份会话的探针 ← 本课新增
```

第 4 步 改写效果：一个探针加两份汇编

```
examples/ch08/
├── opt_effects.c ：内联与死代码消除的探针 ← 本课新增
├── opt_effects-O0.s ：-O0 汇编，call 还在 ← 本课新增
└── opt_effects-O2.s ：-O2 汇编，call 与乘法都没了 ← 本课新增
```

第 5 步 volatile 的汇编，按目录分开

```
examples/ch08/
├── volatile_loop/ ← 本课新增
│   ├── volatile_loop-O0.s ：-O0 汇编 ← 本课新增
│   └── volatile_loop-O2.s ：-O2 汇编 ← 本课新增
└── volatile_asm/ ← 本课新增
    └── volatile_asm-O2.s ：五个函数的 -O2 汇编 ← 本课新增
```

第 1、2 帧是六份会话，第 3 帧是两份会话与它们的探针（第七、八份会话在这里），第 4 帧是优化效果的探针与两份汇编，第 5 帧是 volatile 的三份汇编。逐个点名：

- **十六个例子目录**（`ub_oob_read` … `unspecified_order`）：每个都是 `src/ + include/ + 清单` 的最小工程，每个只回答一个问题。十七份清单里，七份是 `mode: "run"`（各有一个真实 stdout 输出块），三份是 `runtime_error`（`ub_div_zero` 与 `crash_stdout` 的两条路），七份是 `detect`（四份 `opt-diff`、一份 `trapv`、两份 `fanalyzer`）；

- **八份会话**（`session-*.txt`）：opt-diff 的两档读数、四种编法、汇编抓取与栈保护、两条警告、分析器的三条原文、崩溃与 `fflush`、实现决定、五个档位的大小与耗时。正文里的读数大多能在这八份文件里找到原文；少数几处（8.5 表里的退出码、8.9 的 `3221226505`、8.7 的 `1069547520`）来自清单运行时的闸门日志（`dist/logs/run-<id>.log`）与 output 块，那里才是它们的原文；
- **三个探针**（`impl_defined.c`、`opt_levels.c`、`opt_effects.c`）：都没有清单，因为它们不输出「程序的结果」，只输出「实现的事实」与「优化器改了什么」；
- **五份汇编**（`opt_effects-O0.s` / `-O2.s`、`volatile_loop-O0.s` / `-O2.s`、`volatile_asm-O2.s`）：`-S` 抓下来的真产物，8.4 与 8.8 的结论都从它们来。

下一单元（`c20`）会用另一个角度收尾：一个能交给别人的 C 程序长什么样——接口、测试、构建脚本、审查清单，以及一份完整的可移植性探针。这一节留下的十六个例子不会白费：那里面每一个「会怎样」的答案，都是你以后审查别人代码时的对照表。

小结 · 这一节要带走的四件事

- 四档行为要分清：定义良好、未指定、实现定义、未定义；只有最后一档「标准什么都不说」，前三档都能查到依据。
- UB 之下编译器的三个自由度（假设不发生、删掉检查、给出任意结果）都有真跑的读数支撑：同一个循环 `-O0` 退出 0、`-O2` 退出 3221225477；同一段越界读两档读数不同。
- `volatile` 只保证「每次访问都真的发生」；汇编里 `movq` / `addq` 都在、`lock` 与屏障一个都没有，所以它不保证原子、顺序与可见性——要线程安全请用 `stdatomic.h`。
- 证据分四类：退出码、警告原文、汇编、opt-diff 差异。本机没有 ASan/UBSan，说不清的就标「没有检查器」，不要拿「跑起来没事」当结论。

自测题 · 自测题（先自己答，再对答案）

1. 下面四句各属于哪一档：`int x = 3 + 4;`、`printf("%d", f() + g())` 里 `f` 与 `g` 谁先调用、`(char)200` 的值、`a[3]`（`a` 只有 3 个元素）？
2. `signed_overflow_loop` 在 `-O0` 下退出 0，在 `-O2` 下退出 3221225477。是 `-O2` 引入了 bug 吗？如果不是，`-O2` 凭什么这么做？
3. 本机 `volatile long long` 的 `g++` 汇编是单条 `addq $1, g(%rip)`。为什么不能据此说它是原子的？你需要什么证据才能反驳「volatile 能当锁」？
4. 重叠的 `memcpy` 在本机跑出来的结果与 `memmove` 完全一样。那我们是靠什么判定它违规的？给出两样东西。
5. 一个程序在崩溃前 `printf` 过一行，但通过管道抓到 stdout 是空的。为什么？要怎么改才能保住那一行？

注 · 自测题答案

1. `int x = 3 + 4;` 是**定义良好**（标准说死了结果是 7）；`f` / `g` 的调用顺序是**未指定**（结果集合确定，顺序由实现挑，不必告诉你）；`(char)200` 是**实现定义**（本机 -56，因为 `char` 带符号，实现要写进文档）；`a[3]` 是**未定义**（标准什么都不说，本机 `-O0` 读到 6291120 一类值、`-O2` 折成常数）。
2. 不是。UB 的许可本来就允许编译器假设它不发生：`i += i` 溢出被当成不可能，于是 `while (i > 0)` 恒真，后面的取用被按「`i` 单调增长」安排，地址就算到了不该去的地方。`-O0` 那一档「正常」只是恰好没把 UB 变成事故。想收回许可就用 `-fwrapv`（回绕）或 `-ftrapv`（陷阱）。
3. 因为「一条指令」不等于「一个不可分割的操作」：`addq $1, g(%rip)` 仍是读内存、加一、写回三步合成，另一个核可以在中间插进来。标准（6.7.3、5.1.2.3）只保证「每次访问都发生」，一个字都没提原子性、顺序与可见性；汇编里 `lock` / `mfence` 一个都没有（本单元 `session-asm.txt` 的 grep 结果是空）。要原子得用 `stdatomic.h` 或 `lock` 前缀，单线程程序演示不出这个失败。
4. 靠标准的 7.24.2.1（「如果两个对象重叠，行为未定义」）与编译器那句 `-Wrestrict` 警告（`-Wall` 就带着，`-O0` 也报）。运行结果**不是**证据：本机的 `memcpy` 对 8 字节的小块恰好从头往后搬，重叠时碰巧也对，换个 libc 或换个长度就可能出错。
5. 因为 `printf` 写的是全缓冲的 stdout，崩溃时缓冲区没来得及刷。两个改法：崩溃前显式 `fflush(stdout)`，或者改写到默认无缓冲的 `stderr`。本单元 `crash_stdout` 的两份清单（其中一份多一个 `-DCH08_FLUSH`）就是这两种跑法的真实 stdout：一份是「（无输出）」，一份是那一行字。

收尾：从会写 C 到能维护一个 C 程序

◎ 学习目标 · 看完这一单元，你应该能

- 说出一份「能交给别人的 C 程序」从需求到交付要经过哪七步，并指出每一步的产出物是什么；
- 用一条命令或一个 `build.sh` 把多文件工程构建出来，并用**退出码**判断这次构建成没成；
- 把逻辑写成纯函数、用 `assert` 给边界用例写测试，并说清 `assert` 为什么不能拿来做参数校验；
- 拿一份二十条的人工审查清单去读别人的代码，在六类最常见的缺陷里认出至少一半；
- 打印出本机的实现事实（各类型的 `sizeof`、`LONG_MAX`、字节序），并说出其中哪几条不许当定律背。

20.1 先看工程长什么样

前面十九个单元教的是「怎么写」；这一节教的是「怎么交出去」。这两件事的差距不在语法上，而在**接缝**上：一个能在你机器上跑的程序，和一个别人敢改的程序，中间隔着接口、测试、构建脚本、审查清单这四样东西。

照老规矩，先从**文件和目录**看起。这一节是《C 语言》分册的最后一节，前面那一节（`chapters/ch08/01-undefined-behavior-and-optimization.md`）已经在 `examples/ch08/` 里摆了十六个例程；本节要在这个目录里再加**五个目录组**，它们合起来是一份完整的、能交给别人的小程序。

目录树 **第 8 章的这一节：从「一堆例程」到「一份能交出去的程序」**

第 1 步 本节开始时，examples/ 下已经有八个项目

```
examples/
├── ch01/ … ch07/ ：前七章的十八个项目（各带 src/、include/、清单）
└── ch08/
    └── 第八单元的十六个例程 ：ub_oob_read、volatile_loop 等，本节不动它们
```

第 2 步 本单元在 examples/ch08/ 里加上五个目录组

```
examples/ch08/
├── 第八单元的十六个例程 ：不动
├── final_app/ ：综合工程：读文本 → 解析 → 排序 → 统计 → 写结果 ← 本课新增
├── hidden_bugs/ ：一段藏了六个缺陷的代码，一次只打开一个 ← 本课新增
├── portability_probe/ ：打印这台机器的实现事实 ← 本课新增
├── testable_pure/ ：把逻辑写成纯函数，用 assert 测它 ← 本课新增
└── build_and_run/ ：一条命令 + 一个构建脚本 ← 本课新增
```

第 3 步 这五个目录组里装的是什么

```
examples/ch08/
├── final_app/
│   ├── include/ ：三份合同（类型、错误码、契约） ← 本课新增
│   ├── src/ ：main.c / app.c / parse.c / report.c ← 本课新增
│   ├── src/selfcheck.c ：错误路径与解析边界的自检程序 ← 本课新增
│   └── data/scores.txt ：两万行输入，其中十一行是脏行 ← 本课新增
├── build_and_run/build.sh ：本项目第一条构建脚本，里面只有一条 gcc ← 本课新增
├── hidden_bugs/src/ ：review_me.c（六个缺陷）+ main.c（-DBUG 开关） ← 本课新增
├── portability_probe/src/ ：impl_facts.c（问机器）+ main.c（打印） ← 本课新增
├── testable_pure/src/ ：stats_pure.c（纯函数）+ main.c（23 条断言） ← 本课新增
└── 各目录的清单 ：十三份 .json，写明编译参数、模式与期望输出 ← 本课新增
```

第 4 步 全书收齐时的样子（八章、二十个单元）

```
dev-notes/books/c/
├── chapters/ch01/ … ch08/ ：二十个单元（含本节）
├── examples/ch01/ … ch07/ ：前七章的十八个项目
└── examples/ch08/
    ├── 第八单元的十六个例程
    ├── final_app/ ：带输入数据与自检程序的完整工程 ← 本课新增
    ├── testable_pure/ ：assert 测试 ← 本课新增
    ├── build_and_run/ ：构建脚本 ← 本课新增
    └── hidden_bugs/ +portability_probe/ ：其余两组 ← 本课新增
```

第 1 帧是这一节的起点：`examples/` 下已经有八个项目，`examples/ch08/` 里是一堆「一次只讲一件事」的例程，它们各自都能编译、都能跑，但**合起来不是一个程序**。第 2 帧是本节新增的五个目录组，每一组只回答一个问题：一个真程序长什么样（`final_app`）、缺陷藏得住吗（`hidden_bugs`）、这机器的事实是什么（`portability_probe`）、逻辑怎么单独测（`testable_pure`）、怎么用一条命令构建（`build_and_run`）。第 3 帧把它们各自装了什么摆出来——注意**测试与构建脚本不在 `final_app` 里**：`final_app` 有的是输入数据和一个自检程序（`src/selfcheck.c`），`assert` 单测在 `testable_pure`，`build.sh` 在 `build_and_run`。第 4 帧是全书收齐时的样子：八章二十个单元，上面这三样东西（数据、测试、构建脚本）在 `examples/ch08/` 里第一次同时出现——**它们合起来才是这一节要交给你的东西**。

五个目录组各自的一句话：

- `final_app/` —— 综合工程：四个源文件 + 一个自检程序 + 三个头文件 + 两份输入数据。它把读文本、解析、排序、统计、写结果五步串成一条链；
- `hidden_bugs/` —— 一段藏着六个缺陷的代码。六个缺陷住在同一个文件里，清单一次只打开一个（第 1 号在开了栈保护时会终止进程，六个一起打开就只看得见它）；
- `portability_probe/` —— 实现事实探针：把 `sizeof`、`LONG_MAX`、字节序打印出来；
- `testable_pure/` —— 纯函数 + `assert` 测试：逻辑与输入输出分开，单独测；
- `build_and_run/` —— `build.sh`：本项目（`examples/ch08/`）的第一条构建脚本，里面只有一条 `gcc`。

> 工程惯例 · **工程惯例：为什么「一个例子一个目录」这一条在最后一节才见效**
>
> 从第二个项目（第 2 章）起，本书要求每个例子都是 `src/` + `include/` 的最小工程。当时给的理由是「编译速度、接缝可见、别人能接手」——那三条都是**预告**。到了这一节，它们第一次全都兑现了：
>
> - `final_app` 的 `parse.c` 与 `report.c` 互不相识，所以你能单独测 `parse.c`、单独换掉 `report.c` 的排版；
> - `hidden_bugs` 的六个缺陷分别在六个函数里，所以能一次打开一个、各留一条证据；
> - `portability_probe` 只把「问机器」这一件事放进 `impl_facts.c`，`main.c` 只负责打印。
>
> 代价还是那一条：你要多维护三份头文件。头文件写错（漏声明、重复定义）时，报错会出现在**别的**文件里——所以在行数不多的时候就把这个习惯练出来，比在三千行的 `main.c` 里练要便宜得多。

20.2 一个程序的全貌：从需求到交付

先把这个程序的需求说完，一个字都不含糊：

> **需求**：`data/scores.txt` 里每行是「名字 分数」，名字与分数之间用一个空格分开；有些行是空行、`#` 开头的注释行或写坏的行。程序要读这个文件，把能认出来的记录按分数从高到低排好（同分按名字），打印**前 8 名**和一组统计（条数、平均分、最高分、最低分、中位数）。出错时**干净地退出**，并且三种情况各有一个退出码：**打不开文件是 2、一条有效记录都没解析出来是 3、内存不够是 4**。

需求写清楚了，后面的每一步才有对错可判。这七步是：

1. **需求** —— 上面那段话。谁写、写多细，决定了后面每一步会不会返工；
2. **模块划分** —— 读文件一组、解析一组、排序与统计一组、写结果一组。划分的判据只有一条：**能不能单独验证**；
3. **接口** —— 三个头文件。里面只放「别人需要知道什么」：类型、函数声明、错误码、契约（参数能不能是 NULL、返回值什么时候有效）；
4. **实现** —— 四个源文件。`main.c` 只决定「读哪个文件、写到哪」，流程在 `app.c`，活由 `parse.c` 与 `report.c` 干；
5. **测试** —— 分两层落地：`final_app/src/selfcheck.c` 把「两个错误码 + 解析的边界」钉住（它有自己的 `main`，与主程序分开编），`testable_pure` 演示纯函数怎么用 `assert` 一条条测；
6. **构建** —— 一条 `gcc` 命令固定进 `build.sh`。同一条命令、同一组参数，别人跑出来的结果才和你一样；
7. **交付** —— 把「怎么构建、怎么跑、输出应该长什么样」一起交出去。本书的做法是把期望输出写进清单，让机器逐字对账——这就是**交付物的一部分**。

七步里最容易跳过的是第 3 步。它的产出不是一个文件，而是**一份合同**：别人只看 `include/` 就能知道每个模块能做什么、不能做什么。这句话有个直接后果：**头文件里不放实现**，而 `.c` 里的函数要在头文件里有一条声明。

接口先于实现，看真东西。这是 `examples/ch08/final_app/include/app.h` 里那份合同：

c examples/ch08/final_app/include/app.h 第 7–20 行

```c
/* 一条记录：名字 + 分数。
   名字用定长数组而不是指针 —— 这样整个 Score 可以被复制、被排序搬来搬去，
   不用再管「它指向的那串字符是谁的、还活着吗」。 */
typedef struct { 一条记录：名字 + 分数；16:上限只写一遍；20:读文件、跑完整条链、写到 out
    char name[16];
    int  score;
} Score;

/* 一次能处理的记录条数上限。只写一遍，别处都引用它。 */
#define APP_MAX_RECORDS 20000

/* 程序的入口函数：读 path、跑完整条链，把结果写到 out。
   返回 0 表示成功；非 0 是给 shell 看的错误码，main 会原样交给操作系统。 */
int app_run(const char *path, FILE *out);
```

10 一条记录：名字 + 分数；16:上限只写一遍；20:读文件、跑完整条链、写到 out

三行注释里藏着三个决定，都值得停一下：

- `char name[16]` 而不是 `char *name`：**整个 `Score` 可以被复制。**排序时 `qsort` 要把记录搬来搬去，定长数组搬的是内容，指针搬的只是一个地址——后者会引出「这块字符串是谁的、谁负责释放」的一串麻烦；
- `APP_MAX_RECORDS` 是一个**宏**，不是散落在各处的字面量 `20000`。上限只写一遍，将来要改就改这一行；
- `app_run` 的最后一个参数是 `FILE *out`：**「写到哪」由调用者决定。**所以这个程序自己不开输出文件；测试时把 `out` 指向一个临时文件，或者干脆还指向 `stdout`（ISO C 没有「写进内存」的 `FILE *`，那是 POSIX 的 `open_memstream`，本机没有）。

再看 `include/parse.h` 这一份。它把「失败」说全了——这是新手最容易含糊过去的地方：

c examples/ch08/final_app/include/parse.h 第 8–28 行

```c
/* 解析的结果码：0 是成功，负数是不同的失败原因。
   「返回 0 表示成功」是 C 里的主流写法（系统调用也是这个形状）。 */
enum {
    PARSE_OK        =  0, 0 是成功；12:负数是失败；21:两个计数一起带回来；28:契约写在函数旁边
    PARSE_ERR_NULL  = -1,   /* 传进来的是空指针：调用方违约 */
    PARSE_ERR_FULL  = -2,   /* 记录数超过 cap：这份输入太大 */
    PARSE_ERR_EMPTY = -3    /* 一条有效记录都没有 */
};

/* 「收到多少、跳过多少」两个数，用一个结构体一次带回来 */
typedef struct {
    size_t kept;      /* 收下的行数 */
    size_t skipped;   /* 跳过的行数：空行、注释、坏行、名字太长 */
} ParseStats;

/* 把 text 里的文本解析进 records[0..cap)。
   契约：text 与 records 不能为 NULL（违反返回 PARSE_ERR_NULL）；
         cap 是数组容量，超了返回 PARSE_ERR_FULL。
   成功返回 PARSE_OK，并把两个计数写进 *stats；
   **只有返回 PARSE_OK 时 *stats 才有效**。 */
int parse_records(const char *text, Score *records, size_t cap, ParseStats *stats);
```

11 0 是成功；12:负数是失败；21:两个计数一起带回来；28:契约写在函数旁边

最后那句加粗的话就是合同的意义：它预先回答了「出错时 `*stats` 能不能用」这种一定会有人问的问题。不写它，每个调用者都要自己猜一次，猜错的后果是读一块没被写过的内存。

有了合同，各模块之间谁认识谁就固定下来了。**箭头单向，不许回头**：

第 1 帧的 `main.c` 短到可以整段贴出来（11 行）——它只做两件事：

c examples/ch08/final_app/src/main.c 第 1–11 行

```c
#include <stdio.h>

#include "app.h"

/* 这个程序只做两件事：决定读哪个文件、把结果交给谁。
   真正的流程在 app_run 里，那一段见 src/app.c。 */
int main(int argc, char *argv[]) {
    const char *path = (argc > 1) ? argv[1] : "data/scores.txt"; 不给参数就读默认那份数据

    return app_run(path, stdout); 返回值原样交给操作系统
}
```

8 不给参数就读默认那份数据
10 返回值原样交给操作系统

第 8 行是命令行参数的处理：给了参数就用参数，没给就用默认数据文件。第 10 行把 `app_run` 的返回值直接当 `main` 的返回值——**退出码就是这么从最里层传到 shell 的**，中间不需要任何额外机制。

第 2 帧的 `app.c` 是唯一知道「先解析、再排序、最后写结果」这个顺序的地方。它的主体是 `app_run`：

c examples/ch08/final_app/src/app.c 第 61–81 行

```c
int app_run(const char *path, FILE *out) {
    const char *why = NULL;
    char *text = read_whole_file(path, &why);
    ParseStats stats = {0, 0};
    Score *records;
    Report summary;
    clock_t t0, t1;
    int rc;

    if (text == NULL) { 打不开文件：退出码 2
        fprintf(stderr, "读不了 %s: %s\n", path, why);
        return 2;                           /* 打不开文件 */
    }
    records = alloc_records(); 一次开好整块记录数组
    if (records == NULL) {
```

```
76          fprintf(stderr, "内存不够，连记录数组都开不出来\n");
77          free(text);
78          return 4;                               /* 内存不够：单独一个码 */ 内存不够：退出码 4
79      }
80      rc = parse_records(text, records, APP_MAX_RECORDS, &stats);
81      free(text);                                 /* 用完就地还，别攒到最后 */ 用完就地还，别攒到最后
```

70 打不开文件：退出码 2
74 一次开好整块记录数组
78 内存不够：退出码 4
81 用完就地还，别攒到最后

接着是后半段：三项工作、两处计时、一处 `fflush`：

c examples/ch08/final_app/src/app.c 第 82–97 行

```
82      if (rc != PARSE_OK) {                       解析失败：退出码 3
83          fprintf(stderr, "解析失败：%s\n", parse_error_text(rc));
84          free(records);
85          return 3;                               /* 解析失败（含一条有效记录都没有） */
86      }
87      t0 = clock();
88      sort_records(records, stats.kept);          排序
89      t1 = clock();
90      summary = report_stats(records, stats.kept);
91      report_write(out, records, stats.kept, &summary, stats.skipped);  写结果
92      fflush(out);                                /* 先把结果冲出去，再写计时 */ 先把结果冲出去，再写计时
93      fprintf(stderr, "排序 %zu 条记录，用时 %.6f 秒 (clock()) \n",
94              stats.kept, (double)(t1 - t0) / (double)CLOCKS_PER_SEC);
95      free(records);
96      return 0;
97  }
```

82 解析失败：退出码 3
88 排序
91 写结果
92 先把结果冲出去，再写计时

有几个细节是刻意的，也都值得学：

- **三种失败三种码**：打不开文件是 2、解析失败（含「一条有效记录都没有」）是 3、内存不够是 4。调用这个程序的人不用去猜，看退出码就知道折在哪一步。这三个码不是写在纸上就算数的——`src/selfcheck.c` 会真的把 2 与 3 走一遍，第 4 个用故障注入单独走（20.5 会给证据）；
- **每个失败出口都配了 `free`**：`records` 开不出来时先 `free(text)`，解析失败时先 `free(records)`。这类「配对了没有」的检查是审查清单里的一条（20.7 会逐条列）；
- **计时那一行写给 stderr，不写给 stdout**。原因是第 7 步那件事：stdout 是**交付物**（要让机器逐字对账），一次跑一个数的耗时进 stdout 会让输出每次都不一样。分开走，两边都干净。

第 3 帧说的是只管干活的那两个模块。`report.c` 里「写结果」长这样：

c examples/ch08/final_app/src/report.c 第 45–63 行

```
45  void report_write(FILE *out, const Score *records, size_t n, const Report *r,
46                    size_t skipped) {
47      size_t top;
48
49      if (out == NULL || records == NULL || r == NULL) {  参数不全就什么都别写
50          return;                         /* 没有一个能用的参数，那就什么都别写 */
51      }
52      top = (n < REPORT_TOP) ? n : REPORT_TOP;  最多写前 8 名
53      fprintf(out, "== 前 %zu 名 ==\n", top);
54      for (size_t i = 0; i < top; i++) {
55          fprintf(out, "%2zu  %-8s %d\n", i + 1, records[i].name, records[i].score);  名字左对齐 8 列，分数跟在后面
56      }
57      fprintf(out, "\n== 统计 ==\n");
58      fprintf(out, "记录数 = %zu (跳过 %zu 行) \n", n, skipped);
```

```
59        fprintf(out, "平均分 = %.2f\n", r->mean);
60        fprintf(out, "最高分 = %d\n", r->max);
61        fprintf(out, "最低分 = %d\n", r->min);
62        fprintf(out, "中位数 = %.2f\n", r->median);
63    }
```

49 参数不全就什么都别写
52 最多写前 8 名
55 名字左对齐 8 列，分数跟在后面

它跑出来是这样（两万行输入，其中十一行是空行、注释行或坏行）：

终端输出 终端输出 final-app-out

```
== 前 8 名 ==
 1  user00761 999
 2  user00803 999
 3  user00965 999
 4  user01359 999
 5  user03881 999
 6  user03973 999
 7  user04538 999
 8  user05045 999
== 统计 ==
记录数 = 19989（跳过 11 行）
平均分 = 496.77
最高分 = 999
最低分 = 1
中位数 = 496.00
```

这份输出一共十六行（十五行内容 + 中间一个空行），它**就是交付物**：谁拿到这个程序，跑一遍 `./app`，就应该看到这些字。本书流水线对它的要求是逐字一致——差一个空格都会报错。这份「期望输出」不是文档，它是能被机器核对的东西（20.5 会讲这个思路怎么用在测试上）。

20.3 代码风格与可读性：让人敢改你的代码

风格不是审美问题。**代码是写给下一个人读的，而那个人常常是三个月后的你自己**——所以风格的每一条都能翻译成一句「这样写会不会让人误判」。五条最值得练的：

1. **名字说清用途，不用缩写。** `parse_records`、`report_write`、`count_above`：动词开头、宾语在后。反例是 `pr`、`doIt`、`tmp2`——它们不是错，只是每个读的人都要回跳一次上下文；
2. **缩进一致，一个项目一套。**本节的例子统一四空格、不用 Tab（Tab 在不同编辑器里显示宽度不同，一混就乱）；
3. **注释写「为什么」，不写「是什么」。** `x++; /* x 加一 */` 这种注释等于没说；本节例子里真正有用的是这种：`copy_bytes(buf, src, 9); /* 缺陷：8 格的地方写了 9 个字节 */`——它写的是**这句代码与它想干的事之间的差距**；
4. **函数短。**判据不是行数，是「一次能读完、只干一件事」。本节三个例子的真实长度是：`report_write` **19 行**、`parse_line` **32 行**、`parse_records` **40 行**。后两个已经偏长——它们的「一件事」分别是「解析一行」和「按行切分」，还能不能再拆是一个判断问题；如果你接手时要往里加规则，第一件事就是想这个。
5. **头文件是合同**：里面放类型、函数声明、错误码、契约说明，**不放实现。**同样一份声明被多个 `.c` 包含时，靠 `#ifndef` 那三行（include guard）保证只展开一次，这是第 7 章讲过的。

有一条容易被忽略的**代价**：这些规矩都会让你**现在多打字。**写 `report_write` 比写 `pr` 多敲十一个字符，写契约注释比不写多花两分钟。换来的东西是三个月后有人（包括你）能在十分钟内看懂这份代码在干什么、边界在哪。这笔帐在超过 300 行的程序上永远是划算的。

> ⚠ **陷阱 · 「注释写的和代码干的事不一样」比没有注释更坑**
>
> 你会看到的现象：一个函数上面写着「把分数夹到 [0, 100]」，函数体里却是 `if (v > 100) return 100;`——**下界根本没管。**
>
> 你会怎么误判：读代码时眼睛先扫过大字注释，于是心里默认「下界已经处理过了」，于是当你看到 `-5` 被原样传下去时，会去别的地方找原因。
>
> 真相是：注释是**声明**，代码才是**事实**；两者不一致时，永远是代码说了算。这一条直接决定你读陌生代码的顺序——**先读头文件与函数签名，再读实现，最后才看注释**（20.11 会把顺序写成步骤）。本书的 `parse_line` 里那句「宁可不收，也不偷偷截断」对应的就是第 21–23 行那三行真实的检查：注释里的每个承诺，都能在代码里找到出处。

20.4 防御性编程：三种策略各有各的位置

「防御性编程」有一半的误区：很多人以为它的意思是**到处写 `if (p == NULL) return;`**。真相是它有三种策略，各自解决不同的问题，选错了会白干活甚至帮倒忙。

策略一：参数校验，返回错误码。 用在**跨越模块边界**的地方，也就是别人能传错的地方。看 `parse_records` 的头几行：

c examples/ch08/final_app/src/parse.c 第 41–49 行

```c
/* 把整块文本按行切开，一行一行交给 parse_line */
int parse_records(const char *text, Score *records, size_t cap, ParseStats *stats) {
    const char *p = text;
    size_t kept = 0;
    size_t skipped = 0;

    if (text == NULL || records == NULL || stats == NULL) {   三个指针一次全查
        return PARSE_ERR_NULL;          /* 参数校验：花几纳秒，省一场崩溃 */   花几纳秒，省一场崩溃
    }
```

47 三个指针一次全查
48 花几纳秒，省一场崩溃

一次查三个指针，成本是几次比较；省下的是「有人传了 NULL，程序在几千行之外崩掉、还查不出是谁传的」。注意它返回的是**错误码**而不是 `abort()`：调用者可能真的想用 `NULL` 表示「这次没有输入」，那就不该炸掉整个程序。

策略二：断言，抓「不可能发生」的事。 `assert` 管的是**程序内部的逻辑不变量**，不是别人的输入。第 20.5 节会大量用它。标准说得很清楚：

标准原文 · 标准原文：`assert` 失败时到底发生了什么

C 语言标准（ISO/IEC 9899，C17）7.2.1.1 节「The `assert` macro」的 Description 里写：

> The `assert` macro puts diagnostic tests into programs; it expands to a void expression. When it is executed, if expression (which shall have a scalar type) is false (that is, compares equal to 0), the `assert` macro writes information about the particular call that failed (including the text of the argument, the name of the source file, the source line number, and the name of the enclosing function — the latter are respectively the values of the preprocessing macros `__FILE__` and `__LINE__` and the identifier `__func__`) on the standard error stream in an implementation-defined format. It then calls the `abort` function.

大意：**断言失败时做两件事——往 stderr 写一条诊断，然后调用 `abort` 终止进程。**诊断必须包含四样东西（表达式原文、源文件名、行号、所在函数名）；**只有「格式」是 `implementation-defined`。**

同一章的 7.2 节第 1 段规定 `assert` 可以被整体关掉：

> If `NDEBUG` is defined as a macro name at the point in the source file where `<assert.h>` is included, then `assert` is defined simply as `#define assert(ignore) ((void)0)`.

大意：**只要在包含 `<assert.h>` 之前定义了 `NDEBUG`，`assert` 就变成一句什么都不做的空语句。**7.2 节第 2 段接着规定：`assert` 必须实现成**宏**，不是真正的函数。

出处：ISO/IEC 9899:2018（C17）§7.2.1.1 第 2 段（Description）；§7.2 第 1 段（`NDEBUG`）与第 2 段（必须实现成宏）。

这段话直接给出了「为什么不能拿 `assert` 做参数校验」：发布版常常带 `-DNDEBUG` 编译，那时候你写的一整排 `assert` 全部消失，本该拦住的东西一个都拦不住。参数校验必须能在发布版里活着，所以它得写成真正的 `if` 加返回值。

本机实测过一次「断言失败长什么样」。`testable_pure` 这个例子带一个开关 `-DASSERT_FAILS`，打开之后故意让一条断言不成立：

实验记录 · 本机实测：断言失败时的读数

- 工具链：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32；
- 编译：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DASSERT_FAILS src/main.c src/stats_pure.c -o app`；
- 运行结果：**stdout 一个字节都没有**，stderr 是 `Assertion failed: clamp_int(11, 0, 10) == 11, file src/main.c, line 62`，退出码 **3221226505**（`0xC0000409`）；
- 清单：`testable_pure/testable_pure_fail.json`（`mode: runtime_error`，`expect.exit: 3221226505`），由流水线真跑核对。

四个细节值得记：① 诊断里有**表达式原文和行号**（`file src/main.c, line 62`），所以断言失败不需要猜；② 它**没有**打函数名——而标准要求诊断「包括」表达式原文、文件名、行号、函数名四项，`implementation-defined` 只修饰**格式**：所以这是本机 UCRT 少打了一项，**不是标准给它的自由**；③ 退出码不是 `1` 也不是 `2`，而是 `0xC0000409`——那是 `abort()` 在这台机器上的收尾方式（第 9 单元的 `-ftrapv`、`-fstack-protector-all` 崩溃也是这个码，它们属于同一类：**进程被强行终止**）；④ 这个数是 Windows 的进程退出码，本书清单用 Node 的 `spawnSync` 读它；同一个程序在 MSYS bash 里 `echo $?` 会得到 **127**（bash 把「被异常终止」映射成 127）。两个数说的是同一件事，别以为看到了两次不同的崩溃。

策略三：干净退出。用在「程序已经救不回来了，但要让外面知道为什么」的地方。它有三条纪律，`app.c` 里三个失败出口都照着做：

1. **诊断走 stderr**，不要混进 stdout（stdout 是交付物，混进去就没人能对账了）；
2. **给一个可判定的退出码**：`0` 成功、`2` 打不开文件、`3` 解析失败、`4` 内存不够。调用者只看退出码就能分流；
3. **退出前把该还的还掉**：`free(text)`、`free(records)`。虽然进程结束时操作系统会回收，但一份「遵守自己定的规矩」的代码才经得起别人加功能（比如以后要在 `main` 里循环调用 `app_run`）。

> ⚠ **陷阱 · 「反正进程要退出了，不 free 也没事」——这句话会咬人**
>
> 你会看到的现象：本节的 `app.c` 在 `return 2` 之前老老实实 `free(text)`，看起来很啰嗦，因为进程马上就没了，操作系统会把内存全部收回。
>
> 你会怎么误判：把这个判断推到别的地方——「这个函数在错误路径上不 free 也没事嘛」。然后有一天，这个函数被放进一个循环里调用（比如批量处理一千个文件），而它正好是**另一种用法**：那时候泄漏的是一千份内存。
>
> 真相是：`free` 的意义不只是「还给操作系统」，更是「这段内存的使命结束了」这个**声明**。声明得越早、越靠近它的作用域末尾，越不容易在改需求时留下漏洞。第 12 单元讲过的那条规矩在这里再次生效：**谁分配，谁释放；在哪个作用域分配，就在哪个作用域结束前释放。**

20.5 测试：把「我觉得对」换成「跑一遍就知道」

测试这件事在新手眼里只有一个形象：写一堆测试代码，然后祈祷它有用。它真正在做的事只有一件：**把「我希望它是对的」变成一条能自动重复的判据。**

第一招是**把逻辑写成纯函数**。纯函数 = 给同样的输入永远给同样的输出、不打印、不读全局变量、不碰调用者的其他内存。这条性质不是洁癖，它直接换来一个能力：**这个函数可以脱离整个程序被单独测。**本节的 `testable_pure` 就是这么摆的：

c examples/ch08/testable_pure/src/stats_pure.c 第 16–35 行

```c
int parse_int(const char *s, int *out) {
    char *end = NULL;
    long long value;

    if (s == NULL || out == NULL) {
        return 0;                       /* 参数校验：宁可不干活，也不赌运气 */
    }
    if (*s == '\0') {
        return 0;
    }
    value = strtoll(s, &end, 10);
    if (end == s || *end != '\0') {
        return 0;                       /* 一个字符都没吃进去，或者尾巴上有垃圾 */
    }
    if (value < INT_MIN || value > INT_MAX) {
        return 0;                       /* 超出了 int 能表示的范围 */
    }
    *out = (int)value;
    return 1;
}
```

20 空指针先拦掉
27 一个字符都没吃进去，或有垃圾
30 超出 int 范围
33 到这一步才敢写

这个函数有四道关卡，每一道都对应一类会让人踩坑的输入：空指针、空串、不是数字、数字太大。测试要做的就是把每一道都撞一遍——**尤其是边界。**所谓边界用例，就是「刚好在界上、刚好在外面一格」的那些值：

c examples/ch08/testable_pure/src/main.c 第 1–18 行

```c
#include <assert.h>
#include <limits.h>
#include <stdio.h>

#include "stats_pure.h"

/* 每跑一条用例就数一个数，最后报出总数 —— 让「测了多少」也变成可见的数字 */
static int checks = 0;
#define CHECK(cond) do { checks++; assert(cond); } while (0)

/* 每个函数一组用例。挑的都是容易想漏的地方：边界上、边界外、空输入、极端值。 */
static void test_clamp(void) {
```

```
    CHECK(clamp_int(5, 0, 10) == 5);            /* 区间里面 */
    CHECK(clamp_int(-1, 0, 10) == 0);           /* 下边界外面 */
    CHECK(clamp_int(11, 0, 10) == 10);          /* 上边界外面 */
    CHECK(clamp_int(0, 0, 0) == 0);             /* 上下界相等 */ 上下界相等
    CHECK(clamp_int(INT_MAX, 0, 10) == 10);     /* 极端值也得夹住 */ 极端值也得夹住
}
```

8 数一共测了多少条
9 每测一条就记一笔
16 上下界相等
17 极端值也得夹住

第二招是**让测试自己数数并报出结论**。跑一遍就看到全部通过：

终端输出 终端输出 testable-pure-out

```
全部 23 条断言通过
```

第三招是**专门测错误路径**。正常路径你天天在跑，错误路径往往一次都没跑过——`fopen` 失败、`malloc` 失败、一条记录都没解析出来。它们不测，就等于没写。`final_app` 用两种办法把它们逼出来。第一种是给分配加一个测试开关（**故障注入**）：

c examples/ch08/final_app/src/app.c 第 50–59 行

```
/* 一次开好整块记录数组。
   测试时用 -DAPP_FAIL_ALLOC 让它直接返回 NULL，把「内存不够」那条路真走一遍 ——
   这叫故障注入：错误路径不制造出来，就永远没被跑过。 */
static Score *alloc_records(void) { 把分配包成一个小函数
#ifdef APP_FAIL_ALLOC 测试时把这条路打开
    return NULL; 直接返回 NULL，假装内存不够
#else
    return malloc(APP_MAX_RECORDS * sizeof(Score));
#endif
}
```

53 把分配包成一个小函数
54 测试时把这条路打开
55 直接返回 NULL，假装内存不够

第二种是写一个**有自己的** `main` 的自检程序，把「打不开文件」和「空文件」真的走一遍（它和主程序分开编，清单里不列 `src/main.c`）：

c examples/ch08/final_app/src/selfcheck.c 第 19–33 行

```
int main(void) {
    /* 文本里一共六行：2 行好记录，4 行该被跳过 */
    const char *text =
        "alice 90\n"
        "\n"
        "# 注释\n"
        "bob 80\n"
        "nodigits\n"
        "this_name_is_way_too_long 70\n";
    ParseStats st = {0, 0};
    Score rec[8];

    /* 1. 打不开文件 → 退出码 2；2. 空文件 → 退出码 3 */
    expect_int("打不开文件的退出码", 2, app_run("data/no_such_file.txt", stdout)); 不存在的文件 → 2
    expect_int("空文件的退出码", 3, app_run("data/empty.txt", stdout)); 空文件 → 3
```

32 不存在的文件 → 2
33 空文件 → 3

它顺手把解析的边界也钉住了——那六行里哪两行该收、哪四行该跳：

c examples/ch08/final_app/src/selfcheck.c 第 35–43 行

```c
    /* 3. 解析的边界：收下几行、跳过几行、内容对不对 */
    expect_int("解析结果码", PARSE_OK, parse_records(text, rec, 8, &st));
    expect_int("收下的行数", 2, (long)st.kept);  六行里收下两行
    expect_int("跳过的行数", 4, (long)st.skipped);  跳过四行
    expect_int("第一条的名字", 0, strcmp(rec[0].name, "alice"));
    expect_int("第二条的分数", 80, rec[1].score);

    if (failures == 0) {
        printf("自检通过：两个退出码与解析边界都对\n");
```

37 六行里收下两行
38 跳过四行

两个清单把这两半都变成了机器判据。自检程序跑出来是一行：

终端输出 终端输出 final-app-selfcheck-out

```
自检通过：两个退出码与解析边界都对
```

故障注入那一次，stdout **一行都没有**（所以块里只有哨兵），退出码是 **4**——清单 `ch08-final-app-failalloc` 把 `"expect": {"exit": 4}` 与这个空输出一起钉住了：

终端输出 终端输出 final-app-failalloc-out

```
（无输出）
```

这样一来，三种失败码都有了机器判据：**2**（打不开文件）与 **3**（一条有效记录都没有）出自自检程序，**4**（内存不够）出自故障注入那一次。三个都真跑过，不是写在纸上。

工程惯例 · 工程惯例：把「期望输出」当成交付物的一部分

写测试有三种常见做法，各自适合不同的阶段：

- `assert` **型单测**（本节的做法）：测试和被测代码住在同一个可执行文件里，跑一遍、看退出码。优点是零依赖、几秒钟就能加一组用例；缺点是每一个新用例都要改代码、重新编译；
- **对账文件型**（本书清单的做法）：把「期望输出」写成文件，跑一遍程序、逐字比对。优点是**连自己都没意识到的行为变化也能发现**（比如 `report_write` 的列宽被谁改了一个字符）；缺点是只覆盖整体输出，出错时定位没单测细；
- **框架型**（第三方测试框架）：适合几百个用例以上的项目，本册不引入——本机没有包管理器，多一个依赖就多一处环境问题。

本节的 `final_app` 用的是第二种：`final-app-out` 那个块里的十六行就是期望输出，流水线每次真跑都对一遍。`testable_pure` 用的是第一种。两条都保留，是因为它们抓的是不同种类的错误：**单测抓「这个函数算错了」，对账抓「整个程序的行为变了」。**

⚠ 陷阱 · 测试通过不等于没问题：断言只测你想到的那些情况

你会看到的现象：`testable_pure` 打出「全部 23 条断言通过」，于是心里认为 `stats_pure.c` 没有缺陷。

你会怎么误判：把「测试通过」当成「代码正确」的证明。它其实只证明了**这 23 条用例覆盖到的那部分**正确。

真相是：测试的覆盖面取决于**你想到了什么**。本节那 23 条里有好几条是专门为「边界」写的（`[0,0]`、`INT_MAX`、空串、`INT_MAX + 1` 的字符串），因为边界才是出错的地方；而 `parse_int` 里还有一类输入没人测——前后带空格的 `" 42"`。写这一节时我用真程序试了一遍：`parse_int(" 42", &v)` 在本机返回 **1**、`v` 是 **42**（`strtoll` 会跳过前导空白）；而 `parse_int("42 ", &v)` 返回 **0**（尾巴上的空格被 `*end != '\0'` 那一关拦住了）。也就是说，**前空格放过、后空格不放过**——这个行为是我在写这一节时才发现它和直觉不一样的，而且**至今没有任何用例钉住它。**

要不要允许前后空格，是一个需求问题，不是实现问题。把它补进用例，这个行为才算被钉住；不补，它随时会在下一次修改里悄悄变掉。这就是「把测试当判据」的全部意思。

20.6 构建与自动化：一条命令、一个脚本、一个退出码

第 6 步「构建」听起来像大工程，其实在这个规模上只有三样东西：**一条命令**、**一个脚本**、**一个退出码**。先看命令长什么样——它把两个源文件一次交给 `gcc`：

bash examples/ch08/build_and_run/build.sh 第 1–7 行

```bash
# 构建这个例子：一条命令，把所有源文件一次交给 gcc。
# -Iinclude 让 #include "counter.h" 能被找到；
# set -e 让「任何一步失败就立刻停下」，并把那个非 0 的退出码交给调用者。
set -e

gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/*.c -o app   # 一条命令，所有源文件一次交给 gcc
echo "构建成功：app.exe"   # 成功时打印一句，失败时这一行根本不会执行
```

6 一条命令，所有源文件一次交给 gcc

7 成功时打印一句，失败时这一行根本不会执行

第 6 行的 `src/*.c` 是 shell 的通配：`counter.c` 与 `main.c` 两个文件会被展开进去，顺序是字母序。这一行就是从第 4 章起反复出现的那条命令，只是把源文件列表换成了通配。

真跑一遍，三样东西一次看全——产物、退出码、程序自己的输出：

终端会话 真跑 build.sh：产物、退出码、以及程序自己的输出 ch08-build

```
$ bash build.sh
构建成功：app.exe
$ echo $?
0
$ ls -l app.exe
-rwxr-xr-x 1 Administrator 197121 138220 Oct 18 21:42 app.exe
$ ./app
行数   = 2
词数   = 10
字节数 = 59
第一个词有 5 个字节
$ rm -f app.exe
```

`-rwxr-xr-x … 138220 Oct 18 21:42 app.exe` 那一行是**产物**（本机 138220 字节）；`echo $?` 打出的 **0** 是**退出码**；最后那四行是这个例子自己打印的东西。注意 `$ ./app` 里的 `app` 后面没有 `.exe`——在 bash 里运行 `./app` 时它会自动找到 `app.exe`。

那四行输出还有一个身份：它是这个例子的**期望输出**，写在清单 `build_and_run.json` 里被逐字对账。会话块（`term`）与输出块（`output`）是两套机制，别混：会话块对的是**这一次抓下来的终端记录**，输出块对的是**每一次真跑的 stdout**。同一个例子的输出块长这样：

终端输出 终端输出 build-and-run-out

```
行数   = 2
词数   = 10
字节数 = 59
第一个词有 5 个字节
```

现在看失败的那一次。构建脚本存在的第二个理由就是这一半：**失败要立刻停，并且把失败告诉外面**。下面这段会话把一份少了分号的源文件拷进 `src/` 再跑一次脚本：

终端会话 真跑 build.sh（失败路径）：gcc 报错，脚本停在那里，退出码 1 ch08-build-fail

```
$ cp broken/typo.c src/typo.c
$ bash build.sh
src/typo.c: In function 'main':
src/typo.c:5:29: error: expected ';' before 'return'
    5 |     printf("少了一个分号\n")
      |                             ^
      |                             ;
    6 |     return 0;
      |     ~~~~~~
$ echo $?
1
$ rm -f src/typo.c
```

三行要连起来读：

- `src/typo.c:5:29: error: expected ';' before 'return'` —— 错误在**第 5 行第 29 列**，插入符 `^` 指着的位置就是缺分号的地方；这一行还顺手把 `return 0;` 抄给你看，意思是「我读到这儿才发现上面那条语句没结束」（第 1 单元讲过这种「报错行比错误行晚一行」的现象）；
- 第 2 行 `$ bash build.sh` 之后**没有**出现「构建成功」——因为 `set -e` 让脚本在 `gcc` 失败的那一刻就停了，后面那行 `echo` 根本没被执行；
- `$ echo $?` 是 **1**：`gcc` 的退出码被脚本原样传了出来。**这一个数字就是自动化系统的判据**：脚本的调用者（人、CI、本书的流水线）不需要读你的输出，看这个数就够了。

工程惯例 · **工程惯例：没有 make 也能活，但「一条命令」有它自己的代价**

主流 C 项目用 `make` / `cmake` 生成构建文件，理由是「只重编改动的文件、依赖关系写下来」。本节为什么不用？

- **本机没有**：`where make` 与 `where cmake` 在本机都报「用提供的模式无法找到文件」（`build/toolchain.json` 的 `unsupported` 里也记着这两条），本书不引入装不上的工具；
- **它现在不需要**：本节的工程一共四个源文件，一条 `gcc` 一次就编完。`make` 省下的是「大项目的重编译时间」，这里省不出东西来；
- **它的代价**：改一个头文件时，你得自己知道要重编哪些 `.c`。如果你把 `include/app.h` 里的 `Score` 改了一个字段，**必须把四个源文件全部重编**——通配写法（`src/*.c`）顺手就把这件事办了，这也是它在这个规模上比 `make` 更省心的原因。

还有一件事本书**不做**：把 `.o` 打包成静态库（`ar` + `.a`）。它是真实工程里的常规做法，但它属于「怎么把一个工程发给别人用」的问题，和本节要讲的东西不是一回事。那一步留给《项目实战》。

20.7 二十条审查清单，与六个真实缺陷

代码审查（code review）听起来很正式，做起来就是**拿着清单一行一行看**。清单的意义在于：它把「经验」变成「可以照着做的动作」，让没经验的人也能抓住大部分问题。下面这二十条按类别排：前四条抓越界，第 5–7 条抓生命周期，第 8–10 条抓空指针与返回值，第 11–12 条抓格式串，第 13–16 条抓整数，最后四条是边界、编译警告与风格：

#	检查什么	抓的是哪类错
1	这个下标有没有可能**等于**数组长度？	越界（差一格）
2	循环边界写的是 `<` 还是 `<=`？	越界（差一格）
3	缓冲区的容量与写进去的长度是同一个来源吗？	越界（硬编码的数字对不上）
4	`memcpy` / `strcpy` / `sprintf` 的目标够不够大？	越界（没有长度上限）
5	每一个 `malloc` 有没有配对的 `free`？**错误出口**上有没有漏？	资源配对
6	`free` 之后还有没有人用这个指针？	生命周期（悬垂）
7	同一个指针会不会被 `free` 两次？	生命周期（双重释放）
8	返回指针的函数（`fopen` / `malloc` / `strchr`），调用者查 NULL 了吗？	空指针

#	检查什么	抓的是哪类错
9	返回值有没有被忽略（`fgets` / `fclose` / `scanf`）？	错误被吃掉
10	出错时是**真的离开**了，还是只打印一句就继续往下跑？	错误被吃掉
11	格式串的 `%d` / `%zu` / `%s` 与实参类型对得上吗？	格式串错配
12	`%s` 打印的东西一定以 `'\0'` 结尾吗？	格式串错配
13	有符号与无符号混着比较、相减了吗？	整数类型
14	`a - b < 0` 里的 `a`、`b` 是无符号数吗？	整数类型（永远不成立）
15	加法、乘法会不会溢出？尤其是「长度 × 元素大小」	整数类型
16	类型转换里有没有丢信息（`long long` → `int`、`size_t` → `int`）？	整数类型
17	传进函数的每块内存，函数知道它有多长吗？	边界（数组退化后长度丢了）
18	空指针、空串、长度 0 这三种输入，函数能原样返回吗？	边界
19	编译时真的一条警告都没有吗（`-Wall -Wextra -Wpedantic`）？	上游信号
20	有没有「注释说的和代码做的不一样」？	可读性

清单要练到「不用看表也能过一遍」才算学会。现在就拿它来审一份真代码——`examples/ch08/hidden_bugs/src/review_me.c` 里有**六个函数，每个藏着一个缺陷。**先自己找，找到几个算几个，再看下面的对照。

第一个缺陷在拷贝那一段。注意 `copy_bytes` 自己没错，错在调用它的人：

c examples/ch08/hidden_bugs/src/review_me.c 第 8–21 行

```c
/* 干活的拷贝函数：它只信调用者那句「dst 装得下 n 个字节」 */
static void copy_bytes(char *dst, const char *src, size_t n) {
    for (size_t i = 0; i < n; i++) {
        dst[i] = src[i];
    }
}

/* 第 1 号：把 src 拷进一个 8 字节的缓冲区 */
void step_1_copy(const char *src) {
    char buf[8];

    copy_bytes(buf, src, 9);                /* 缺陷：8 格的地方写了 9 个字节 */ 8 格的地方写了 9 个字节
    printf("缓冲区里现在是 %.8s\n", buf);   /* 能走到这一行，看起来一切正常 */
}
```

19 8 格的地方写了 9 个字节

第二个缺陷把「错误」当成「空文件」吃了进去，第三个缺陷是一个永远不成立的判断：

c examples/ch08/hidden_bugs/src/review_me.c 第 23–43 行

```c
/* 第 2 号：数一数文件里有多少行 */
void step_2_read(const char *path) {
    char line[64];
    size_t lines = 0;

    FILE *fp = fopen(path, "r");            /* 返回值一次都没看过 */ 返回值一次都没看过
    while (fgets(line, sizeof line, fp) != NULL) {
        lines++;
    }
    fclose(fp);
    printf("文件 %s 一共有 %zu 行\n", path, lines);
}
```

```c
/* 第 3 号：检查空间够不够 */
void step_3_room(size_t used, size_t room) {
    if (used - room < 0) {                  /* 无符号相减，永远不会小于 0 */ 无符号相减，永远不会小于 0
        printf("空间不够：还差 %zu 格\n", room - used);
    } else {
        printf("空间够用：已用 %zu / 容量 %zu\n", used, room);
    }
}
```

28 返回值一次都没看过
38 无符号相减，永远不会小于 0

第四个缺陷在格式串上，第五个在生命周期上：

c examples/ch08/hidden_bugs/src/review_me.c 第 45–64 行

```c
/* 第 4 号：打印全部得分合计 */
void step_4_report(void) {
    long long total = 5000000000LL;         /* 五十亿，超出了一个 int 的范围 */

    printf("全部得分合计 = %d 分\n", total); long long 配了 %d
}

/* 第 5 号：取出最后一次记下的分数 */
int step_5_last(void) {
    int *scores = malloc(4 * sizeof *scores);
    int last;

    if (scores == NULL) {
        return -1;
    }
    scores[0] = 42;
    free(scores);                           /* 已经还回堆里了 */ 已经还回堆里了
    last = scores[0];                       /* 还回去之后又去读 */ 还回去之后又去读
    return last;
}
```

49 long long 配了 %d
61 已经还回堆里了
62 还回去之后又去读

第六个缺陷是**什么都不做**：分配了内存，用完之后没有还：

c examples/ch08/hidden_bugs/src/review_me.c 第 66–79 行

```c
/* 第 6 号：累计 0 到 7 的平方和 */
int step_6_tally(void) {
    int *tmp = malloc(8 * sizeof *tmp);
    int sum = 0;

    if (tmp == NULL) {
        return -1;
    }
    for (int i = 0; i < 8; i++) {
        tmp[i] = i * i;
        sum += tmp[i];
    }
    return sum;                             /* tmp 到这里就该还了，可是没有 */ tmp 到这里就该还了，可是没有
}
```

78 tmp 到这里就该还了，可是没有

六个缺陷住在同一个文件里，可是**不能让它们同时发生**——第 1 号一发生进程就没了，后面五个的现象你再也看不到。所以这份代码带一个开关：

c examples/ch08/hidden_bugs/src/main.c 第 5–23 行

```c
/* 六个缺陷里哪一个真的被触发，由这个开关决定（清单里用 -DBUG=n 传进来）。
```

```c
    为什么不让六个同时发生：第 1 号在开了栈保护时会直接终止进程，
    后面五条的现象你再也看不到。 */
#ifndef BUG 清单用 -DBUG=n 传进来
#define BUG 0 默认一个都不打开
#endif

int main(void) {
    switch (BUG) {
    case 1: step_1_copy("12345678"); break; 第 1 号：越界写
    case 2: step_2_read("no_such_file.txt"); break;
    case 3: step_3_room(120, 100); break;
    case 4: step_4_report(); break;
    case 5: printf("读回来的分数 = %d\n", step_5_last()); break;
    case 6: printf("累计 = %d\n", step_6_tally()); break;
    default: printf("六个缺陷全关：程序正常运行\n"); break;
    }
    return 0;
}
```

8 清单用 -DBUG=n 传进来
9 默认一个都不打开
14 第 1 号：越界写

六个缺陷、六份清单、六条真跑证据：

实验记录 · 本机实测：六个缺陷各自的证据

工具链与命令先说清：gcc 16.1.0（`-std=c17`，也就是 C17），目标三元组 x86_64-w64-mingw32。每一条都从 `examples/ch08/hidden_bugs/` 目录里跑，命令的形状统一是 `gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -DBUG=n src/main.c src/review_me.c`（n 换成对应编号；检测模式的清单还会在它上面追加检测参数）：

- **第 1 号（越界写，清单 `ch08-hb1`）** —— 检测模式 `stack-protector`：命令是 `gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -fstack-protector-all -Iinclude -DBUG=1 src/main.c src/review_me.c`。运行结果：stderr 是 `*** stack smashing detected ***: terminated`，退出码 **3221226505**（这个数是从清单里读到的 Windows 退出码；在 MSYS bash 里 `echo $?` 会显示 127，口径见 20.4 实验块的第四条）。**把 `-fstack-protector-all` 去掉再编一次，它就变回「退出码 0、stdout 照打 缓冲区里现在是 12345678」** —— 流水线自己会做这个阴性对照；证据归不到保护器头上就不算证据。所以「越界写会崩」这句话是错的，**只有加了保护才会崩**；
- **第 2 号（没看返回值，清单 `ch08-hb2`）** —— 真跑（`-DBUG=2`）：退出码 **0**，stdout 见下面第一个输出块。它不崩，只是**把错误吃掉了**。但要说得更准：`fopen` 返回 NULL 之后，代码把 NULL 交给了 `fgets` 与 `fclose` ——**把空指针传给库函数是未定义行为**（C17 的 7.1.4 节：参数值无效——包括空指针——行为未定义），本机只是碰巧「读到 NULL 就返回、`fclose(NULL)` 也没发作」。**不崩 ≠ 定义良好**；
- **第 3 号（无符号相减，清单 `ch08-hb3`）** —— 真跑：退出码 0，stdout 见第二个输出块。基础参数下编译器就报了：`src/review_me.c:38:21: warning: comparison of unsigned expression in '< 0' is always false [-Wtype-limits]`；
- **第 4 号（格式串错配，清单 `ch08-hb4`）** —— 真跑：退出码 0，stdout 见第三个输出块。基础参数下的警告是 `src/review_me.c:49:29: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long long int' [-Wformat=]`；
- **第 5 号（`free` 之后又读，清单 `ch08-hb5`）** —— 这个缺陷**两层都抓到了**：基础参数（`-Wall -Wextra`）报 `src/review_me.c:62:10: warning: pointer 'scores' used after 'free' [-Wuse-after-free]`；加 `-fanalyzer`（清单的检测模式）再报一条 `src/review_me.c:62:10: warning: use after 'free' of 'scores' [CWE-416] [-Wanalyzer-use-after-free]`。两次编译退出码都是 0；而**真跑看不出任何问题**，见下面的会话；
- **第 6 号（忘了 `free`，清单 `ch08-hb6`）** —— 只有 `fanalyzer` 抓得到：`src/review_me.c:78:12: warning: leak of 'tmp' [CWE-401] [-Wanalyzer-malloc-leak]`。基础参数对它一个字都不报，真跑也看不出问题。

六个缺陷里，**基础警告抓到 3、4、5 号，分析器再抓到 5、6 号，运行期保护只抓到 1 号**——而第 2 号要靠人读（它既不是编译器认得的「确定错误」，也不会崩）。这个分工，就是 20.9 那张「三层防线」表的由来。

三个「跑完不崩、退出码 0」的输出块，正好是这一节最想让你看的东西——**问题全在内容里**：

终端输出 终端输出 hb-2-out

```
文件 no_such_file.txt 一共有 0 行
```

终端输出 终端输出 hb-3-out

```
空间够用：已用 120 / 容量 100
```

终端输出 终端输出 hb-4-out

```
全部得分合计 = 705032704 分
```

第 5、6 号的会话更是如此。编译那一步的警告被丢进了 `/dev/null`，那是为了先看清运行结果——**那三条基础警告正对应第 3、4、5 号**，20.9 会把它们一条条摆出来：

终端会话 真跑第 5、6 号缺陷：看着完全正常，退出码 0 ch08-hb-runs

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DBUG=5 src/main.c src/review_me.c -o hb5.exe
2>/dev/null
$ ./hb5 ; echo $?
读回来的分数 = 42
0
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude -DBUG=6 src/main.c src/review_me.c -o hb6.exe
2>/dev/null
$ ./hb6 ; echo $?
累计 = 140
0
$ rm -f hb5.exe hb6.exe
```

读这几段证据要读出层次来：

- 第 1 号**只在加了保护时才被抓住**（`stack smashing detected` + 3221226505）；不加保护时它退出 0、输出照打。**UB 不是「一定会崩」**；
- 第 2、3、4 号**跑起来不崩**，只有对着输出读一遍才发现结论荒唐（「0 行」「空间够用：已用 120 / 容量 100」「705032704 分」）。其中 2 号是未定义行为（把 NULL 交给了库函数），3、4 号编译器都留了警告；
- 第 5 号**运行期完全看不出来**（打印 42，看着还挺对），但基础警告 `-Wuse-after-free` 与分析器都点了名；第 6 号只有分析器抓得到（基础参数一个字不报）。

> ⚠ **陷阱 · 「在我机器上能跑」是这六个缺陷的共同挡箭牌**
>
> 你会看到的现象：这六个程序**都编译通过**；除了第 1 号在开了栈保护时会被终止，其余情况**全都退出 0**。如果你只跑一遍、看到退出码是 0，就会把它当成合格的代码交出去。
>
> 你会怎么误判：把「没崩」当成「对」。第 2 号的输出 `文件 no_such_file.txt 一共有 0 行` 甚至会让你觉得「这个文件果然是空的」；第 5、6 号的输出（42、140）更是**完全正确**——它们错的地方不在结果，而在「读了一块已经还回去的内存」和「漏了一块内存」这两件事本身。还有一层更隐蔽：第 2 号「没崩」纯属本机的运气，它做的事（把 NULL 交给 `fgets`）在标准里就是未定义行为。
>
> 真相是：**退出码 0 只说明「进程正常结束了」，不说明「结果是对的」，更不说明「没有未定义行为」**。所以这一节要求三样东西同时存在：真跑的输出（抓第 3、4 号这种荒唐结论）、检测器（抓第 1、5、6 号）、人工清单（抓第 2 号这种连编译器都不报的）。**三样缺一样都不行**，这就是为什么 20.9 说「静态分析要纳入日常，但不能只靠它」。

20.8 可移植性：别拿本机的读数当定律

「在我机器上能跑」的另一半是「换个机器就跑不了」。C 里有一大类事实是**实现定义**的：标准给了下限，具体多少由实现决定。所以第一件事是把本机的事实**测出来**，而不是背下来。`portability_probe` 这个例子的 `impl_facts.c` 就是干这件事的：

c examples/ch08/portability_probe/src/impl_facts.c 第 6–21 行

```
6  ImplFacts probe_facts(void) {
```

```
    ImplFacts f;

    f.char_bit = CHAR_BIT; 一个字节几位
    f.size_char = sizeof(char);
    f.size_short = sizeof(short);
    f.size_int = sizeof(int);
    f.size_long = sizeof(long); long 在这台机器上几个字节
    f.size_long_long = sizeof(long long);
    f.size_void_ptr = sizeof(void *);
    f.size_size_t = sizeof(size_t);
    f.int_max = INT_MAX; 指针几个字节
    f.long_max = LONG_MAX;
    f.long_min = LONG_MIN;
    return f;
}
```

9 一个字节几位
13 long 在这台机器上几个字节
17 指针几个字节

第二段函数回答「这台机器按什么顺序存一个整数」，用的是 `memcpy` 而不是指针强转：

c examples/ch08/portability_probe/src/impl_facts.c 第 23–34 行

```
const char *byte_order_of(uint32_t value) {
    unsigned char bytes[sizeof value];

    /* 用 memcpy 看一个整数的字节，是标准允许的取法：
       指针转换加解引用（比如 *(unsigned char *)&value）虽然这里也能用，
       但「按另一种类型读同一块内存」是有前提的，第 8 章第 1 单元讲过。 */
    memcpy(bytes, &value, sizeof value); 用 memcpy 看整数的字节，这是标准允许的取法
    if (bytes[0] == (unsigned char)(value & 0xFFu)) {
        return "小端（低位字节在前）";
    }
    return "大端（高位字节在前）";
}
```

29 用 memcpy 看整数的字节，这是标准允许的取法

`main.c` 只负责把它打印出来：

c examples/ch08/portability_probe/src/main.c 第 6–24 行

```
/* 把「这台机器的事实」打印出来。这些数字一条都不许记成定律：
   换一台机器、换一个编译器，其中好几条就会变。 */
int main(void) {
    ImplFacts f = probe_facts();

    printf("CHAR_BIT          = %d 位/字节\n", f.char_bit);
    printf("sizeof(char)      = %zu\n", f.size_char);
    printf("sizeof(short)     = %zu\n", f.size_short);
    printf("sizeof(int)       = %zu\n", f.size_int);
    printf("sizeof(long)      = %zu\n", f.size_long); long 是几个字节 —— 别背，跑一遍
    printf("sizeof(long long) = %zu\n", f.size_long_long);
    printf("sizeof(void *)    = %zu\n", f.size_void_ptr);
    printf("sizeof(size_t)    = %zu\n", f.size_size_t); 指针大小跟着机器走
    printf("INT_MAX           = %d\n", f.int_max);
    printf("LONG_MAX          = %ld\n", f.long_max);
    printf("LONG_MIN          = %ld\n", f.long_min);
    printf("0x01020304 的字节序 = %s\n", byte_order_of(0x01020304u)); 字节序也是问出来的
    return 0;
}
```

15 long 是几个字节 —— 别背，跑一遍
18 指针大小跟着机器走
22 字节序也是问出来的

真跑出来的读数是这样（gcc 16.1.0、`-std=c17`、目标三元组 x86_64-w64-mingw32）：

终端输出 终端输出 portability-out

```
CHAR_BIT          = 8 位/字节
sizeof(char)      = 1
sizeof(short)     = 2
sizeof(int)       = 4
sizeof(long)      = 4
sizeof(long long) = 8
sizeof(void *)    = 8
sizeof(size_t)    = 8
INT_MAX           = 2147483647
LONG_MAX          = 2147483647
LONG_MIN          = -2147483648
0x01020304 的字节序 = 小端（低位字节在前）
```

这张表里最容易被误记成定律的是 `sizeof(long) = 4`。很多人以为「64 位机器上 `long` 就是 8 字节」——这台机器**是** 64 位（`sizeof(void *) = 8`），`long` 却仍然是 4 字节。所以它不是定律，是一条本机读数。标准对这件事的说法是：

标准原文 · **标准原文：`long` 至少有多宽**

C 语言标准（ISO/IEC 9899，C17）5.2.4.2.1 节「Sizes of integer types `<limits.h>`」给出的最小值表里有这两行：

> — minimum value for an object of type `long int` `LONG_MIN` −2147483647 // −(2^31 − 1) — maximum value for an object of type `long int` `LONG_MAX` +2147483647 // 2^31 − 1

紧接着的一句是：这些「实现定义的值」的**绝对值必须大于或等于**表里给出的数。

大意：**标准只保证 `long` 至少能表示 32 位有符号整数**，多出来的位数由实现自己决定。本机正好就是这条下限（`LONG_MAX = 2147483647`），所以「本机 `long` 是 4 字节」这件事合情合理，也**绝不是所有机器都这样。**

出处：ISO/IEC 9899:2018（C17）§5.2.4.2.1。

`CHAR_BIT`（一个字节几位）至少是 8 这条出自 5.2.4.2.1；而 `sizeof(char)` 恒为 1 出自另一节——**6.5.3.4（`sizeof` 运算符）规定：作用于 `char`、`unsigned char`、`signed char` 时结果是 1。本机读数是 8 和 1，与这两条都对得上。至于字节序**，标准没有规定一个多字节整数内部的字节怎么排（它只规定值怎么表示，不规定字节顺序），所以只能问机器、不能背规则：本机 `0x01020304` 在内存里第一个字节是 `04`（低位在前），这就是小端。

工程惯例 · **工程惯例：可移植性守则里最值钱的五条**

以下五条不是标准条文，是几十年踩出来的工程共识。每条后面括号里是「不遵守会怎样」：

- **别依赖结构体布局**（字段之间可能有填充字节，`sizeof(struct)` 不等于字段之和——第 11 单元讲过）：不遵守就会在换编译器后读写错位；
- **别依赖字节序**（跨机器传二进制数据时必须显式转换）：不遵守就会得到 0x04030201 这种反向的数；
- **别依赖 `int` / `long` 的宽度**（要固定宽度用 `<stdint.h>` 的 `int32_t` / `uint64_t`）：不遵守就会在 32 位机器上溢出；
- **别依赖 `char` 有没有符号**（要明确就用 `signed char` / `unsigned char`）：不遵守就会出现 `(char)200` 在这台机器上是 **−56**（本机 `char` 是带符号的）、在另一台机器上却是 **200** 的区别；
- **别依赖指针和 `int` 一样大**：不遵守就会把指针截断成 32 位，得到一个必然出错的地址。

这五条的共同点：**它们全都是「本机能跑、别处不一定」**。所以可移植性的第一步永远是把读数**打印出来**，而不是照着一句口诀改代码。

20.9 性能与静态分析：先正确，再快

性能这一节只讲一件事：**先正确，再快；而且「快了多少」必须测出来，不能靠感觉。**测量工具就在标准库里——`<time.h>` 的 `clock()`，它给出进程消耗的处理器时间。`app.c` 里用它在排序前后各取一次值：

c examples/ch08/final_app/src/app.c 第 87–95 行

```c
87      t0 = clock();
88      sort_records(records, stats.kept);
89      t1 = clock();
90      summary = report_stats(records, stats.kept);
91      report_write(out, records, stats.kept, &summary, stats.skipped);
92      fflush(out);                            /* 先把结果冲出去，再写计时 */
93      fprintf(stderr, "排序 %zu 条记录，用时 %.6f 秒 (clock()) \n",
94              stats.kept, (double)(t1 - t0) / (double)CLOCKS_PER_SEC);
95      free(records);
```

87 排序前
89 排序后
93 只报耗时，不写进 stdout

同一份清单真跑，`-O0` 与 `-O2` 各跑**五遍**（只跑一遍不足以看出这个数会跳）：

终端会话 真跑：同一份数据，-O0 与 -O2 各五遍 ch08-timing

```
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.c src/app.c src/parse.c src/report.c -o app
$ rm -f /tmp/t.txt; for i in 1 2 3 4 5; do ./app >/dev/null 2>>/tmp/t.txt; done; cat /tmp/t.txt
排序 19989 条记录，用时 0.003000 秒 (clock())
排序 19989 条记录，用时 0.003000 秒 (clock())
排序 19989 条记录，用时 0.003000 秒 (clock())
排序 19989 条记录，用时 0.003000 秒 (clock())
排序 19989 条记录，用时 0.003000 秒 (clock())
$ gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -Iinclude src/main.c src/app.c src/parse.c src/report.c -o app
$ rm -f /tmp/t.txt; for i in 1 2 3 4 5; do ./app >/dev/null 2>>/tmp/t.txt; done; cat /tmp/t.txt
排序 19989 条记录，用时 0.002000 秒 (clock())
排序 19989 条记录，用时 0.003000 秒 (clock())
排序 19989 条记录，用时 0.003000 秒 (clock())
排序 19989 条记录，用时 0.003000 秒 (clock())
排序 19989 条记录，用时 0.003000 秒 (clock())
$ rm -f app.exe
```

`-O0` 五遍都是 **0.003000**，`-O2` 五遍是 **一次 0.002000、四次 0.003000**。这个结果和直觉相反——**打开 `-O2` 并没有让排序变快**（那一格差别落在一个刻度的抖动里）。原因不难找：真正干活的 `qsort` 住在运行库里，你的编译器优化不了它；而这里量的是「排序」这一段，不是整个程序。

实验记录 · 本机实测：`clock()` 的三个脾气

- 工具链：gcc 16.1.0，`-std=c17`（C17），目标三元组 x86_64-w64-mingw32；数据是 `data/scores.txt` 的 **19989 条有效记录**；复现命令就是上面 term 块里那两条 `gcc` 与那个五遍循环；
- 读数：`-O0` 五次 **0.003 / 0.003 / 0.003 / 0.003 / 0.003**；`-O2` 五次 **0.002 / 0.003 / 0.003 / 0.003 / 0.003**——**它不是每次都一样**，多跑几遍才看得出来；
- 脾气一：**粒度是 1 毫秒**。这个程序用 `%.6f` 打印，所以 0.003000 里的后三位是补位，**不是微秒级精度**；换成 3000 条记录时读数会变成 0.000000——不是「零耗时」，是「不到一个刻度」；
- 脾气二：**它量的是处理器时间，不是墙上时钟**。程序在等磁盘 I/O 的时候不算在内；
- 脾气三：**它只覆盖你圈住的那几行**。这里圈住的是 `sort_records`，解析、读文件、写结果都不在里面。想量整程序就得用外部计时器，而那个数还会被进程启动时间淹掉（这个程序太小）。

所以这个数字的用法只有一种：**当数量级看、当对照看**（改一版代码，几遍读数放在一起比）。拿它去宣称「我的排序比别人的快 3 倍」是没有意义的。

第二件事是把**静态分析纳入日常**——不用记得跑，让它成为默认动作的一部分。本书的固定参数已经是第一层：`-Wall -Wextra -Wpedantic` 会在编译期报出很多「你自己没注意到」的东西。加一层 `-fanalyzer`，编译器会在编译期做路径分析，能在**没有运行程序**的情况下指出越界、空指针、内存泄漏这类问题：

编译器视角 · 编译器视角：同一份代码，两层警告各说了什么

工具链 gcc 16.1.0，在 `examples/ch08/hidden_bugs/` 下跑两条命令（**两条的编译退出码都是 0**）：

- 基础参数：`gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -fdiagnostics-color=never -Iinclude -c src/review_me.c -o review_me.o`；
- 再加分析器：同一条命令末尾加 `-fanalyzer`。

基础参数报了**三条**，一条对应一个缺陷：

- `src/review_me.c:38:21: warning: comparison of unsigned expression in '< 0' is always false [-Wtype-limits]` —— 第 3 号；
- `src/review_me.c:49:29: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long long int' [-Wformat=]` —— 第 4 号；
- `src/review_me.c:62:10: warning: pointer 'scores' used after 'free' [-Wuse-after-free]` —— 第 5 号。

加上 `-fanalyzer` 之后再**多两条**：

- `src/review_me.c:62:10: warning: use after 'free' of 'scores' [CWE-416] [-Wanalyzer-use-after-free]` —— 还是第 5 号，但这是分析器从路径上推出来的；
- `src/review_me.c:78:12: warning: leak of 'tmp' [CWE-401] [-Wanalyzer-malloc-leak]` —— 第 6 号。

分析器那两条带着 `CWE` 编号（416 是 use-after-free、401 是内存泄漏），那是国际通用的弱点分类编号，可以直接拿去查。要注意的是：**这是「编译器看到的」，不是「运行结果」**——它没有运行你的程序，只是沿着控制流走了一遍。所以它会有误报（报告里会出现「可能有」的语气），也会漏报：**第 2 号在这五条警告里一条都没有。**

工程惯例 · 工程惯例：三层防线，各自抓什么

本书能用到的检测手段就这三种，它们的能力边界必须记清楚，否则你会以为「跑过检测器就没问题了」：

- `-Wall -Wextra -Wpedantic`**（编译期，永远开着）**：本节这份代码上抓到三条——恒假的无符号比较（`-Wtype-limits`）、格式串与实参类型不符（`-Wformat=`）、`free` 之后使用（`-Wuse-after-free`，只给 `-Wall` 就会报，不需要 `-fanalyzer`）。它抓不到越界写、内存泄漏，也抓不到第 2 号那种「传了 NULL 但表面正常」；
- `-fanalyzer`**（编译期，静态分析）**：在基础警告之外再抓到泄漏与「`free` 之后使用」的另一条路径；代价是编译变慢，而且会有误报和漏报；
- `-fstack-protector-all` / `-ftrapv`**（运行期，由清单声明）**：只有**真的发生**才发作，发作的形式是**先往 stderr 打一行**（比如 `*** stack smashing detected ***`）**再终止进程**（退出码 3221226505），而不是变成一条「还能继续跑」的警告。它抓不到堆上的问题，也抓不到没越界的逻辑错误。

再往外一层是人工清单（20.7 那二十条）——它抓的是检测器**永远抓不到**的那一类：代码完全合法、只是结论错（第 3 号那个无符号相减，标准规定它回绕，不认为这是错），或者像第 2 号那样把错误咽了下去。**这三层没有一层能替代另一层。**

20.10 明确不讲的东西，以及为什么

一个负责任的收尾要告诉你**边界在哪**。下面这些内容本书故意不讲，每一条都有理由：

不讲的东西	理由
下一代 C 标准的新语法（`nullptr`、`constexpr`、`typeof`、泛型）	本书从头到尾统一用 C17；新标准的语法在各编译器上成熟度不一，学它不如先把手上的标准用熟
多线程与 `<threads.h>`、原子类型	本机的教学口径是单线程。本书只把「数据竞争属于未定义行为」列进第 8 章第 1 单元的清单，不写教程
`setjmp` / `longjmp` 与信号的完整教程	它们改变控制流的方式和 C 的常规写法差得太远，收益远小于学习成本

不讲的东西	理由
POSIX 细节（`fork` / `socket` / `getline` / `strdup`）	它们不是 ISO C：`fork` 与 `getline` 在本机这套配置下根本没有（`implicit declaration`）；`strdup` 能编过，那是实现额外给的，换个编译器就不保证还在；`socket` 要走的是 Winsock 那一套，不是 POSIX 的头文件。要用的时候单独查，别当成 C 的一部分背
嵌入式 / 寄存器操作 / 示波器	本册的边界是纯软件 C
变长数组（VLA）	C11 起它是「实现可选」的特性，边界行为和栈溢出的关系又不好讲；要动态长度直接用 `malloc`
把 `.o` 打包成静态库（`ar` + `.a`）	属于「怎么把工程发给别人用」，交给《项目实战》分册

有一样东西本书**讲了但没讲完**：`qsort` 的比较函数。它的完整用法（返回值约定、稳定性、与 `bsearch` 的配合）在第 15 单元；本节只用到「分数高的排前面」这一种形状。

20.11 接下来三条路，各自的第一站

到这里，C 语言这一册的正文就结束了。接下来有三条路，每条都从你刚学会的东西直接接上：

第 2 帧那条路的「第一站」是**类型安全与 RAII**：C++ 分册会从「构造与析构」开始，把你在这一节里手写的「每个出口都要配一次 `free` 」变成由语言保证的事——那正是本节 `app.c` 里那五处 `free`（`read_whole_file` 一处、`app_run` 四处）想解决的问题。第 3 帧那条路的第一站是**数组与双指针**，难度 1–2，前置就是你已有的数组、指针、结构体、动态内存这四件套。第 4 帧那条路的第一站是把这一节的多文件工程**继续长大**：加模块、加测试、加构建脚本——你已经有了全套零件。

最后一件工具是**读别人的代码**。它会用在你工作的每一天（接手别人的项目、查一个库怎么用、看懂一份报错背后的实现）。顺序只有一条原则：**接口先于实现。**

1. **先读 `main`**：它是唯一一个「全都要用到」的地方，于是它告诉你这个程序分了几块、顺序是什么（本节的 `main.c` 只有 11 行，整条流程全在 `app_run` 里）；
2. **再读 `include/` 里的每一个头文件**：类型、错误码、函数签名、契约。到这里你应该能说出「这个程序大概在干什么」，还不用看一行实现；
3. **然后读调用链上的函数体**：从 `main` 出发，遇到不认识的名字就去它的头文件里查签名，再进它的 `.c`。不要从第一个文件的第一行往下读——那是读小说的方式，不是读代码的方式；
4. **最后读注释与边界**：注释里承诺的东西，回代码里找它的出处（20.3 那条陷阱就讲这件事）；
5. **拿清单过一遍**：就是 20.7 那二十条。这套动作并不慢；一个几千行的模块，二十分钟能过完一遍。

20.12 这一节新增了什么

回到 20.1 那棵目录树，现在可以对着**第 3 帧**逐个点名了：

- `examples/ch08/final_app/` —— 本节的主角。三份合同 `include/app.h`（`Score`、`APP_MAX_RECORDS`、`app_run`）、`include/parse.h`（错误码与契约）、`include/report.h`（排序、统计、写结果）；四个源文件 `src/main.c`、`src/app.c`、`src/parse.c`、`src/report.c`；两个数据文件 `data/scores.txt`（两万行，其中十一行是脏行）与 `data/empty.txt`（零字节，专供错误码 3 的测试）；一个自检程序 `src/selfcheck.c`；三份清单 `final_app.json`（主程序）、`final_app_selfcheck.json`（自检）、`final_app_failalloc.json`（故障注入，`expect.exit: 4`）；会话 `session-timing.txt`；
- `examples/ch08/hidden_bugs/` —— `src/review_me.c`（六个缺陷）+ `src/main.c`（`-DBUG=n` 开关）+ `include/review_me.h` + 六份清单（`hidden_bugs_1.json` … `hidden_bugs_6.json`）+ `session-runs.txt`；
- `examples/ch08/portability_probe/` —— `src/impl_facts.c` 问机器要事实，`src/main.c` 打印，`portability_probe.json` 对账；
- `examples/ch08/testable_pure/` —— `src/stats_pure.c`（四个纯函数）+ `src/main.c`（23 条断言）+ 两份清单（`testable_pure.json` 通过、`testable_pure_fail.json` 故意失败）；
- `examples/ch08/build_and_run/` —— `build.sh`（本节新增的构建脚本）、`src/counter.c` 与 `src/main.c`、`include/counter.h`、`broken/typo.c`（故意写错的源文件）、`build_and_run.json`、两份会话。

本节新增的清单**一共十三份**（`final_app` 三份、`hidden_bugs` 六份、`portability_probe` 一份、`testable_pure` 两份、`build_and_run` 一份）。它们各自回答一个问题，没有一份是凑数的。

小结 · 这一节的要点

- 一份「能交出去」的 C 程序要过七步：需求 → 模块划分 → 接口 → 实现 → 测试 → 构建 → 交付；**接口先于实现**，头文件就是合同，合同里承诺的事必须在代码里找得到出处；
- 防御与测试各有分工：**参数校验**要能在发布版里活着（`assert` 不行，`NDEBUG` 一开它就没了），**断言**管内部不变量，**干净退出**给三种失败三个码（2 / 3 / 4）；测试要写成纯函数 + 边界用例，还要专门制造错误路径；
- 构建就是一条命令固定成一个脚本，**退出码是唯一的判据**；二十条清单抓的是越界、生命周期、NULL、返回值、格式串、整数类型这几类共性缺陷，**退出码 0 不等于结果对，更不等于没有未定义行为**；
- 可移植性与性能都靠**测量**：本机 `sizeof(long) = 4`、`LONG_MAX = 2147483647`、`sizeof(void *) = 8`、`CHAR_BIT = 8`、小端，标准只给下限；`clock()` 的粒度是 1 毫秒，读数会跳，`-O2` 也不是万灵药——三层防线（编译警告 / `-fanalyzer` / 运行期保护）缺一不可。

自测题 · 自测题（先自己答，再对答案）

1. `final_app` 的 `main.c` 只有 11 行。为什么要把流程放进 `app_run`（`app.c`）而不是直接写在 `main` 里？这么做的代价是什么？
2. `parse.h` 里写着「只有返回 `PARSE_OK` 时 `*stats` 才有效」。这句话为什么必须写在**头文件**里，而不是写在 `parse.c` 的注释里？
3. 为什么 `assert` 不能用来做参数校验？标准里的哪一句话决定了这件事？
4. 第 2 号缺陷（`fopen` 的返回值没查）跑出来退出码是 0，输出却是一句荒唐的话。它属于二十条清单里的哪几条？为什么连 `-fanalyzer` 都没报它？
5. 本机实测 `sizeof(long) = 4` 而 `sizeof(void *) = 8`。这两条读数分别说明什么？如果你要写一份能在 32 位与 64 位机器上都正确的代码，「用 `long` 存指针」这种做法错在哪？

注 · 自测题答案

1. 因为「决定读哪个文件、写到哪」与「怎么跑完整条链」是两件事，分开之后：`app_run` 可以在测试里被直接调用（把 `out` 换成一个临时文件，或者仍用 `stdout`），而 `main` 只是入口。代价是**多了一个文件与一份头文件**，`main` 与流程之间多了一层间接，读代码时要多跳一次。
2. 因为**契约是给调用者看的**。调用者在别的 `.c` 文件里，他只会 `#include "parse.h"`；写在 `parse.c` 的实现注释里，他根本不会打开那个文件。判据很简单：**只有调用者需要知道的事才进头文件，只有维护者需要知道的事留在 `.c` 里**。

3. 因为 `assert` 可以在编译时被整体关掉：标准 7.2 节第 1 段规定，只要在包含 `<assert.h>` 之前定义了 `NDEBUG`，`assert` 就变成 `#define assert(ignore) ((void)0)`——一整排校验全部消失。发布版常常带 `-DNDEBUG` 编译，所以参数校验必须写成真正的 `if` 加返回值。`assert` 管的是「程序内部的逻辑不变量」，那是开发期才需要炸的东西。
4. 属于第 8 条（返回指针的函数有没有查 NULL）与第 9、10 条（返回值被忽略、出错时没有真的离开）；它还顺带违反第 18 条（坏输入应当被处理）。另外它不只是「逻辑错」：`fopen` 返回 NULL 之后，代码把 NULL 交给了 `fgets` 与 `fclose`，**把空指针传给库函数是未定义行为**（7.1.4 节），本机不崩只是这一次的运气。`-fanalyzer` 没报它——真跑的五条警告里没有 `step_2` 这一条。原因我只能说到「它的检查器没把这条路径连成它会报的缺陷」，**分析器没报不等于这里没问题**，这正是人工清单存在的理由。
5. 第一条说明 64 位机器上 `long` 仍可能只有 32 位（本机就是）；第二条说明这台机器的指针是 64 位。「用 `long` 存指针」在 64 位机器上会把指针截断成 32 位，得到的地址必然不对——标准完全没有保证「指针和某种整数一样大」，要存指针就用指针类型，要固定宽度整数就用 `<stdint.h>` 里的 `intptr_t` / `uintptr_t`。